经典译丛

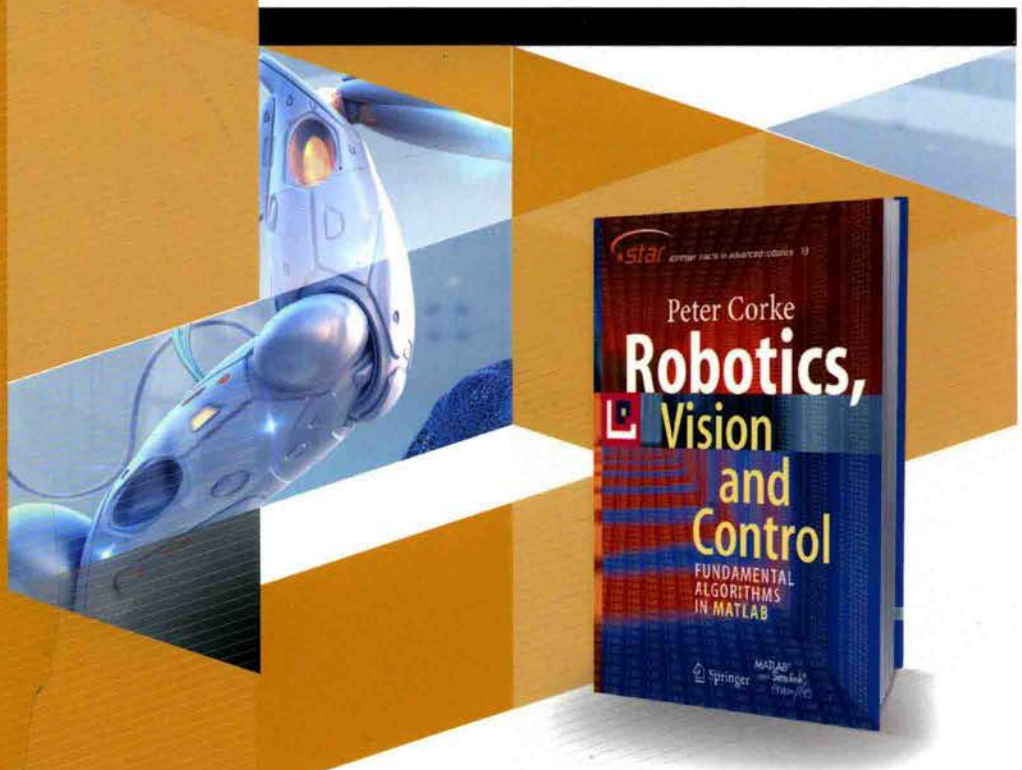
人工智能与智能系统

Springer

机器人学、机器视觉与控制
——MATLAB算法基础

Robotics, Vision and Control
Fundamental Algorithms in MATLAB

【澳】 Peter Corke 著
刘 荣 等译



Robotics, Vision and Control: Fundamental Algorithms in MATLAB

中国工信出版集团
電子工業出版社
PUBLISHING HOUSE OF ELECTRONICS INDUSTRY
http://www.phei.com.cn

经典译丛 · 人工智能与智能系统

机器人学、机器视觉与控制
——MATLAB 算法基础

Robotics, Vision and Control
Fundamental Algorithms in MATLAB

[澳] Peter Corke 著
刘 荣 等译

電子工業出版社
Publishing House of Electronics Industry
北京 · BEIJING

内 容 简 介

本书是关于机器人学和机器视觉的实用参考书，第一部分“基础知识”（第2章和第3章）介绍机器人及其操作对象的位置和姿态描述，以及机器人路径和运动的表示方法；第二部分“移动机器人”（第4章至第6章）介绍其基本运动控制模式及其导航和定位方法；第三部分“臂型机器人”（第7章至第9章）介绍其运动学、动力学和控制方面的知识；第四部分“计算机视觉”（第10章至第14章）包括光照与色彩，图像形成和处理技术，图像特征提取，以及基于多幅图像的立体视觉技术；第五部分“机器人学、视学与控制”（第15章和第16章）分别讨论基于位置和基于图像的视觉伺服及更先进的混合视觉伺服方法。本书已根据作者在网站公布的勘误表进行了细节更正。

本书可以作为学习机器人技术和视觉控制的高年级本科生和研究生的教辅书籍，也可作为课程仿真实验的指导书。另外也适合作为从事机器人技术研究和开发人员的技术参考书。

Translation from the English language edition:
Robotics, Vision and Control: Fundamental Algorithms in MATLAB by Peter Corke.
Copyright © 2011 Springer Berlin Heidelberg. Springer Berlin Heidelberg is a part of Springer Science + Business Media.
Authorized Simplified Chinese language edition by Publishing House of Electronics Industry. Copyright © 2015.
All Rights Reserved.

本书中文简体字版专有出版权由 Springer Science + Business Media, LLC 授予电子工业出版社。未经出版者预先书面许可，不得以任何方式复制或抄袭本书的任何部分。

版权贸易合同登记号　图字：01-2013-2463

图书在版编目(CIP)数据

机器人学、机器视觉与控制：MATLAB 算法基础/(澳)科克(Corke,P.)著；刘荣等译.
北京：电子工业出版社，2016.5
(经典译丛·人工智能与智能系统)
书名原文：Robotics, Vision and Control: Fundamental Algorithms in MATLAB
ISBN 978-7-121-25990-6

Ⅰ. ①机…　Ⅱ. ①科…②刘…　Ⅲ. ①机器人学②计算机视觉　Ⅳ. ①TP24②TP302.7

中国版本图书馆 CIP 数据核字(2015)第094010号

策划编辑：马　岚
责任编辑：周宏敏
印　　刷：北京京师印务有限公司
装　　订：北京京师印务有限公司
出版发行：电子工业出版社
　　北京市海淀区万寿路173信箱　邮编　100036
开　　本：787×1092　1/16　　印张：35　　字数：985千字　　彩插：6
版　　次：2016年5月第1版
印　　次：2016年5月第1次印刷
定　　价：99.00元

凡所购买电子工业出版社图书有缺损问题，请向购买书店调换。若书店售缺，请与本社发行部联系，联系及邮购电话：(010)88254888，88258888。

质量投诉请发邮件至 zlts@phei.com.cn，盗版侵权举报请发邮件至 dbqq@phei.com.cn。

本书咨询联系方式：classic-series-info@phei.com.cn。

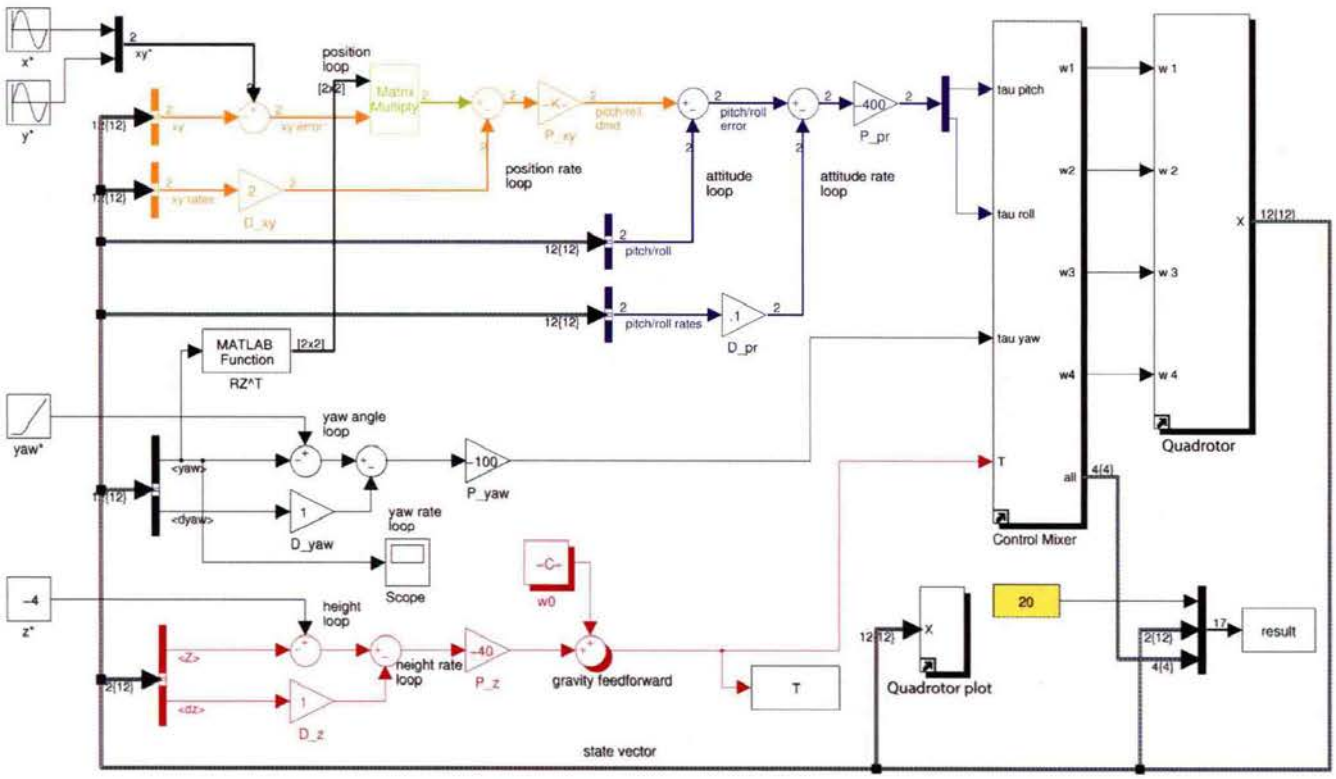


图 4.17　Simulink 模型 `sl_quadrotor`，它是四旋翼飞行器的一个闭环仿真模型。飞行器起飞后将以等高度飞行一圈。其中，动力学模块执行方程（4.9）的运算，混合模块则执行其逆运算，同时限制旋翼的转速。`Quadrotor` 模块在输出 12 个元素的状态向量 x 时用了一条 Simulink 总线

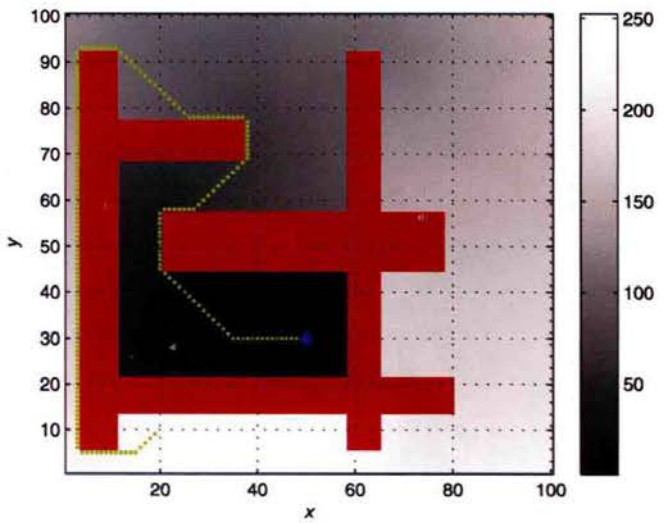


图 5.5　距离变换路径。红色网格代表障碍区。灰色背景的亮度代表单元格到目标的距离，距离单位是单元格大小，如右侧的标尺所示

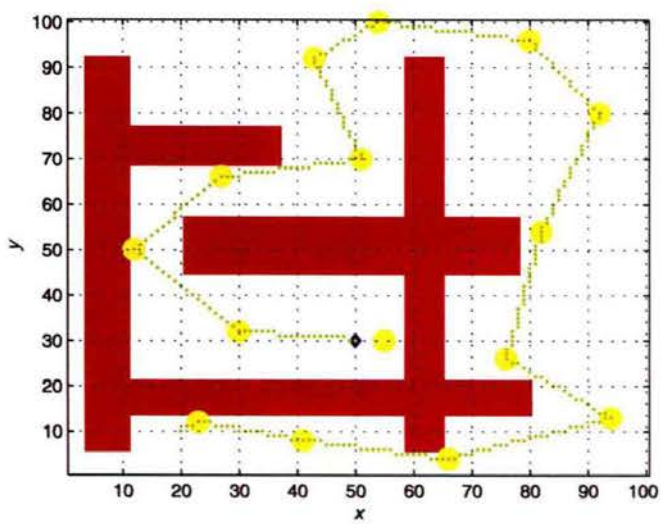


图 5.11　概率路线图（PRM）规划器得到的机器人路径，用绿点线表示。机器人走过的路线图节点用黄色加亮显示

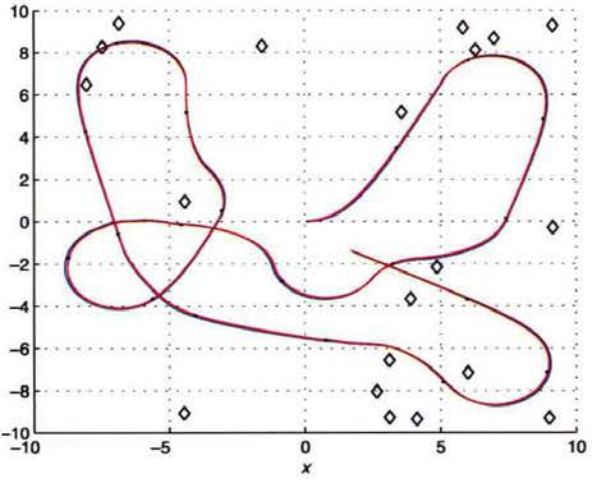


图 6.7　EKF 定位方法，显示了机器人的实际路径（蓝色），以及用里程计和地标估计出的路径（红色）。机器人从原点出发

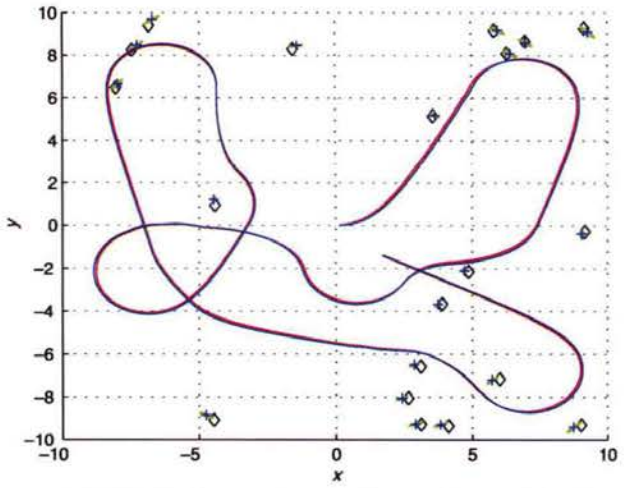


图 6.10　同时定位并制图。图中显示了机器人真实路径（蓝色）和估计路径（红色），它们叠加在真实地图上（黑符号◇）。估计的地图特征用符号+表示，附带一个 5σ置信椭圆

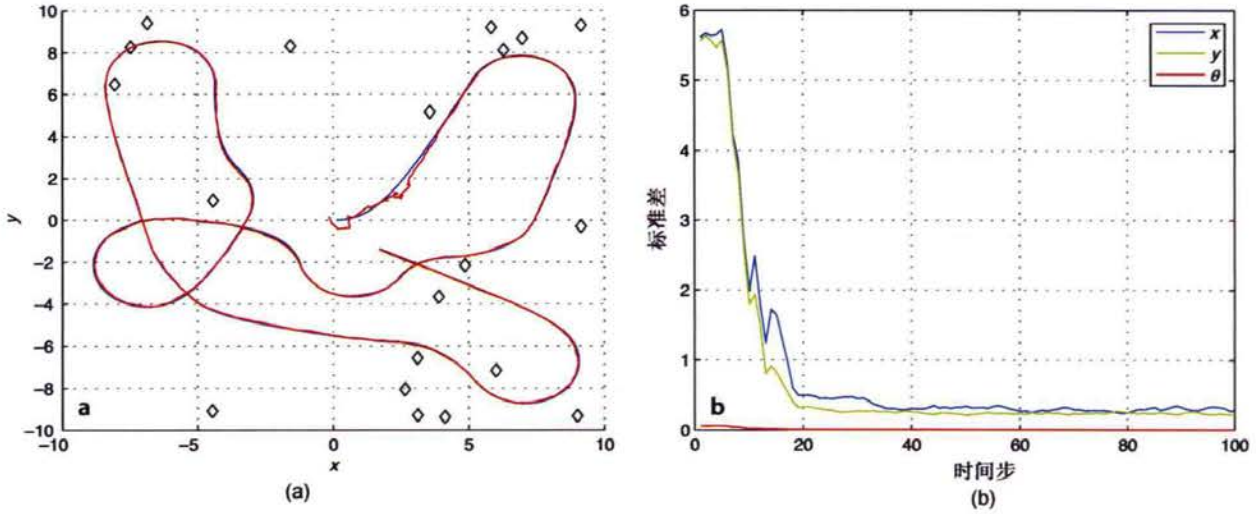


图 6.13　粒子滤波器的结果：(a)蓝色代表机器人的真实路径，红色代表估计路径；(b)粒子标准差随时间变化的曲线

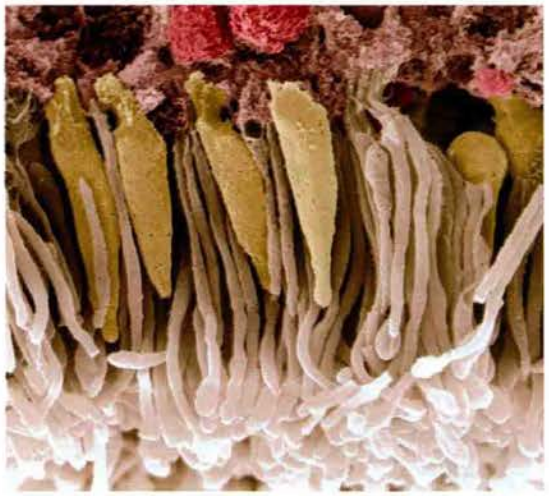


图 10.6　人眼中视杆细胞和锥细胞的彩色扫描电子显微照片。这些细胞的直径范围是 0.5～4 μm。它们包含不同类型的被称为光视蛋白的光敏蛋白质。感光细胞的细胞体（红色）位于杆细胞和锥细胞的上层

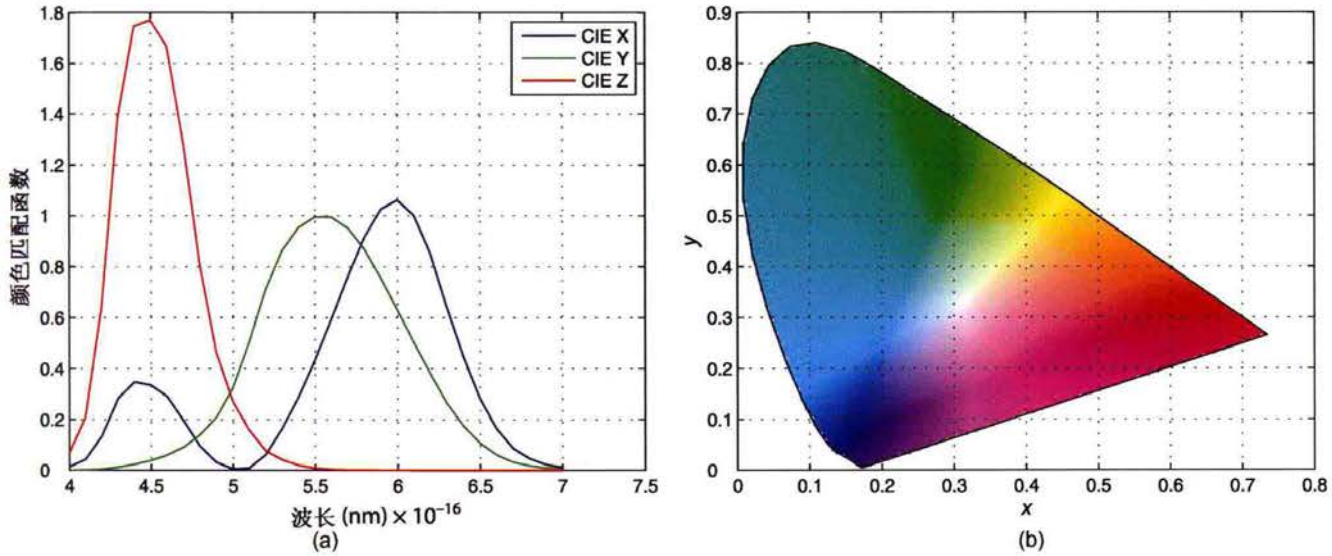


图 10.12　(a)基于虚构三原色的对标准观察者的颜色匹配函数，**X**、**Y**（光强）、**Z** 已被 CIE（1987）表格化；(b)*xy* 色度平面上的颜色

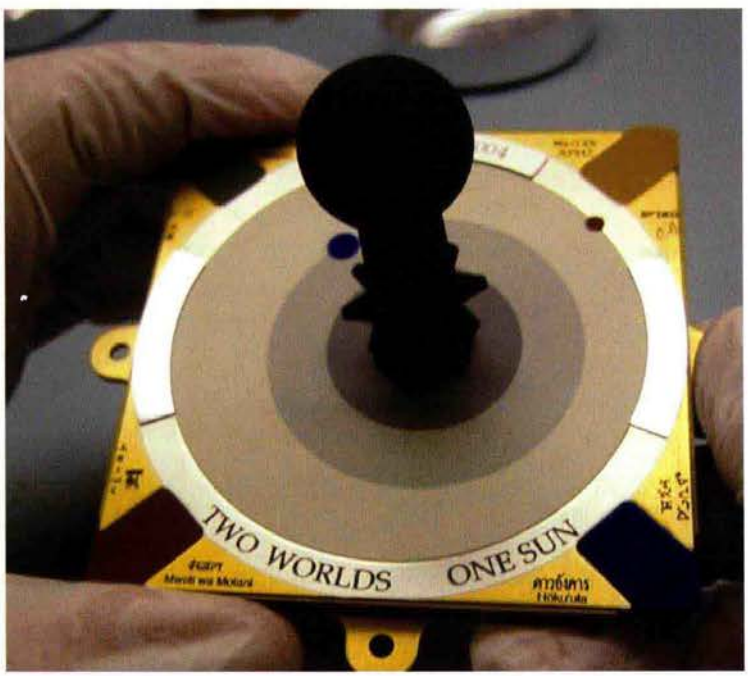
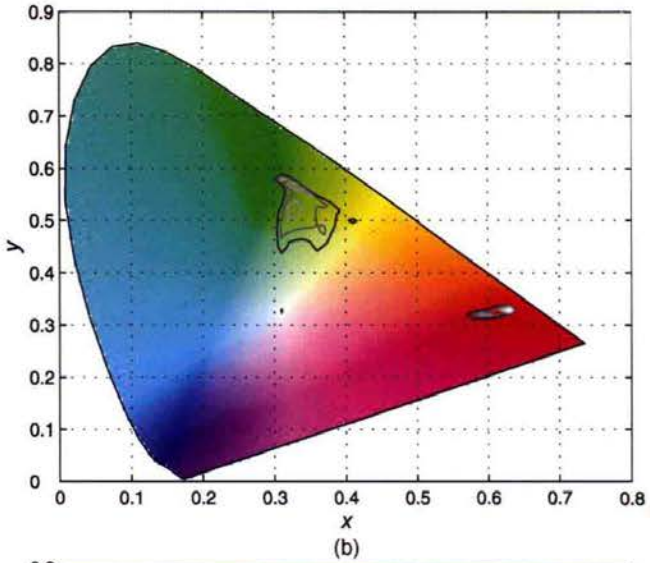
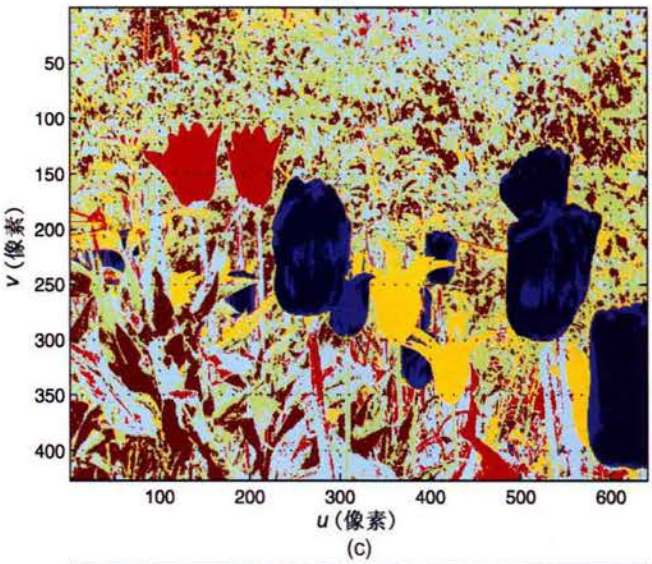


图 10.16　用于火星探测器上全景相机的标定靶。已知反射率和色度（红、黄、绿、蓝和灰色阴影）的区域和用于设定相机的白平衡。中心杆有非常低的反射率，也可充当日晷。它上面刻了一条格言，是关于日晷的最好传说。（照片来自 NASA/JPL/Cornell/Jim Bell）

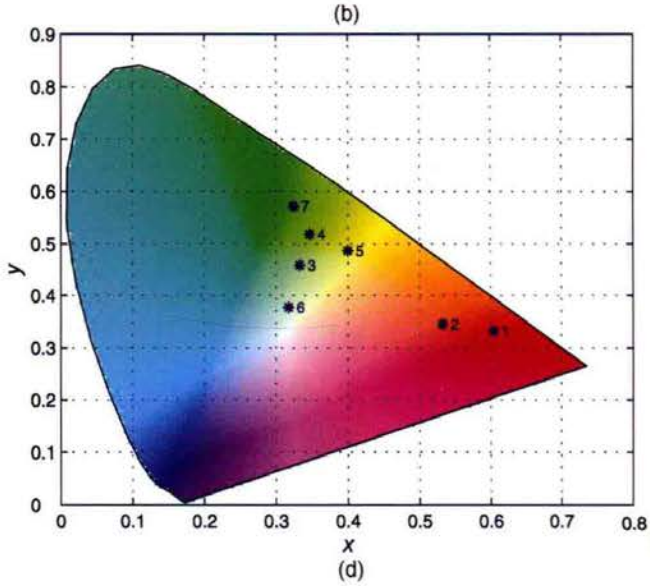
(a)



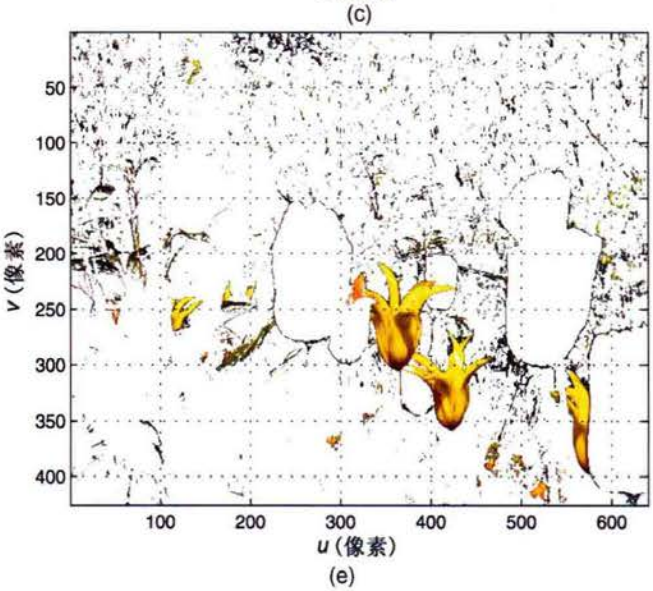
(b)



(c)



(d)



(e)

图 10.19　彩色图像分割。(a)原始彩色图像；(b)叠加了频率等高线的 xy 色度平面；(c)颜色错乱的标签图像；(d)色度图上的群族中心；(e)只有类型 5（黄色）像素

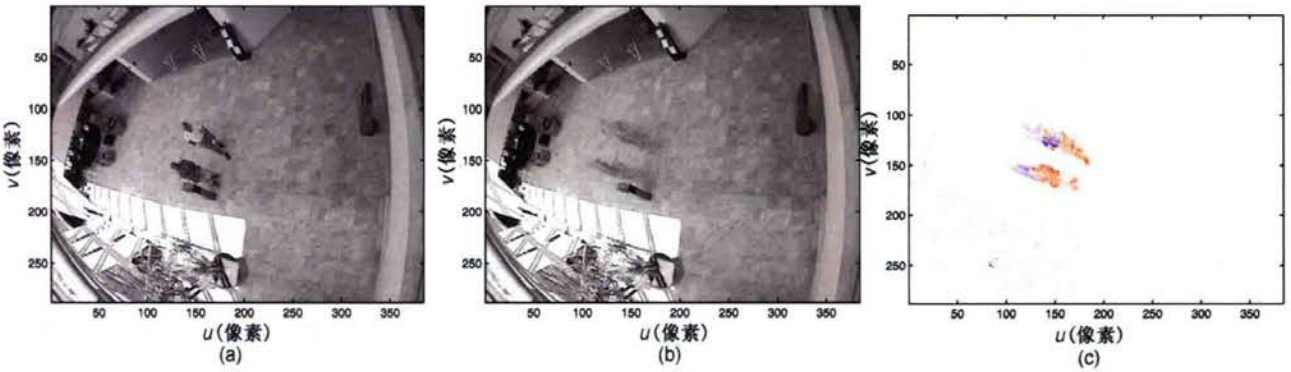


图 12.10　对于 INRIA 的 LeftBag 图像序列在第 250 帧处的图像序列分析示例。(a)当前图像；(b)被估计的背景图像；(c)当前图像与被估计背景图像之间的差值，其中，白色部分为 0，红色和蓝色部分分别是负值和正值，值的大小通过颜色强度来表示（视频来自 EC 投资的 CAVIAR 工程集/IST 2001 37540）

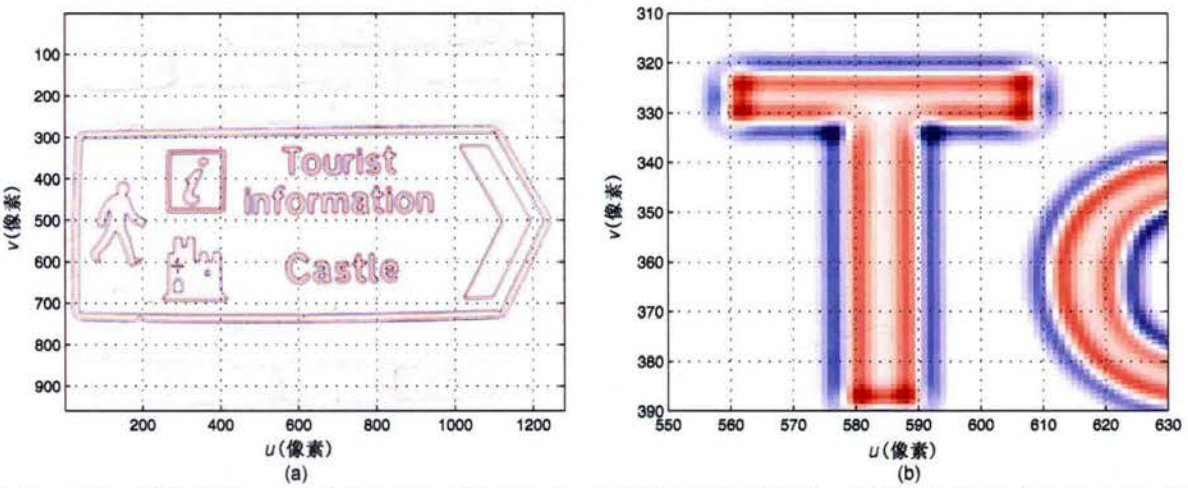


图 12.19　高斯拉普拉斯算子。(a)高斯拉普拉斯算子结果；(b)字母 T 周围的特写，其中蓝色和红色分别表示正值和负值

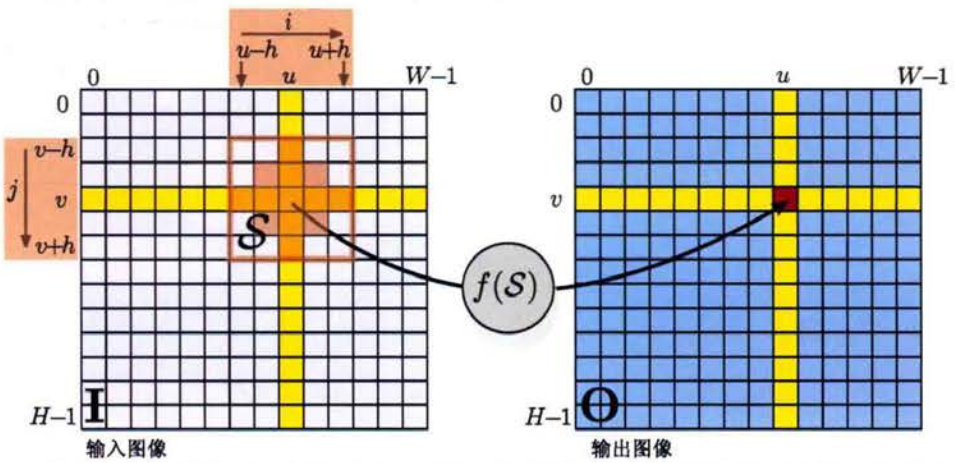


图 12.24　形态学图像处理操作。该操作只针对结构化元素（红色方形轮廓）中的选中像素（红色）

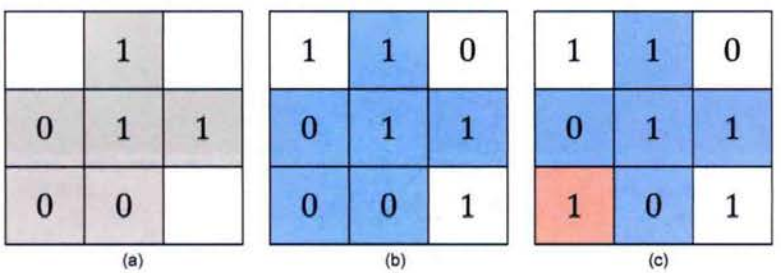


图 12.29　击中与否运算。(a)结构元素值可取 0,1 或者显示为白色；(b)击中样例；(c)未击中样例，红色像素与结构元素不一致

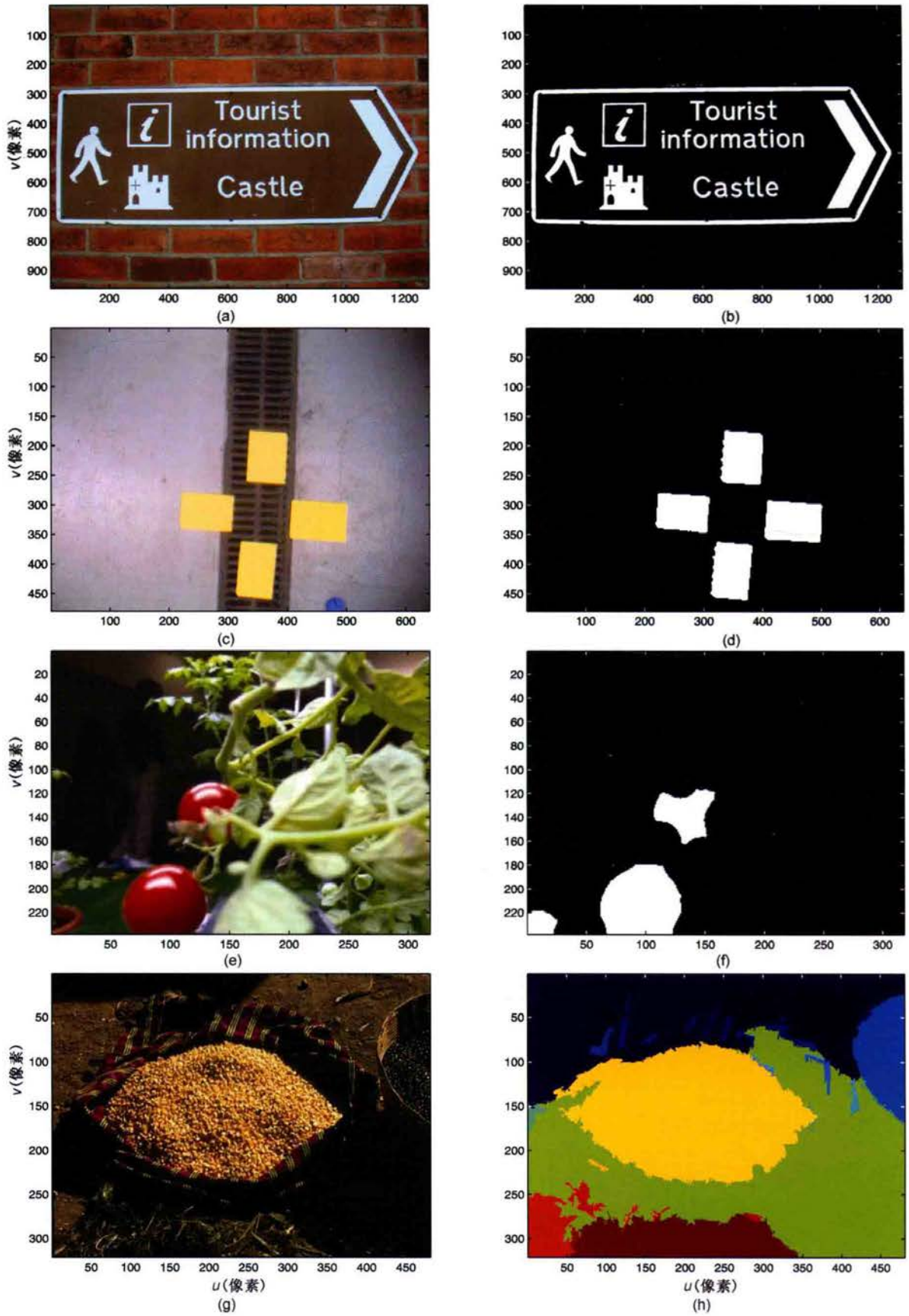


图 13.1　像素分类的实例。左侧是输入图像，右侧是分类后图像。分类是针对不同应用的，像素已经被分类为目标物体（白色）或非目标物体（黑色）。感兴趣的目标是：(a)标牌上的单独字母；(b)黄色目标；(c)红色番茄。(d)是一个多级分割，其中像素已经被分为 28 类，它们代表了在场景中局部共性的像素组

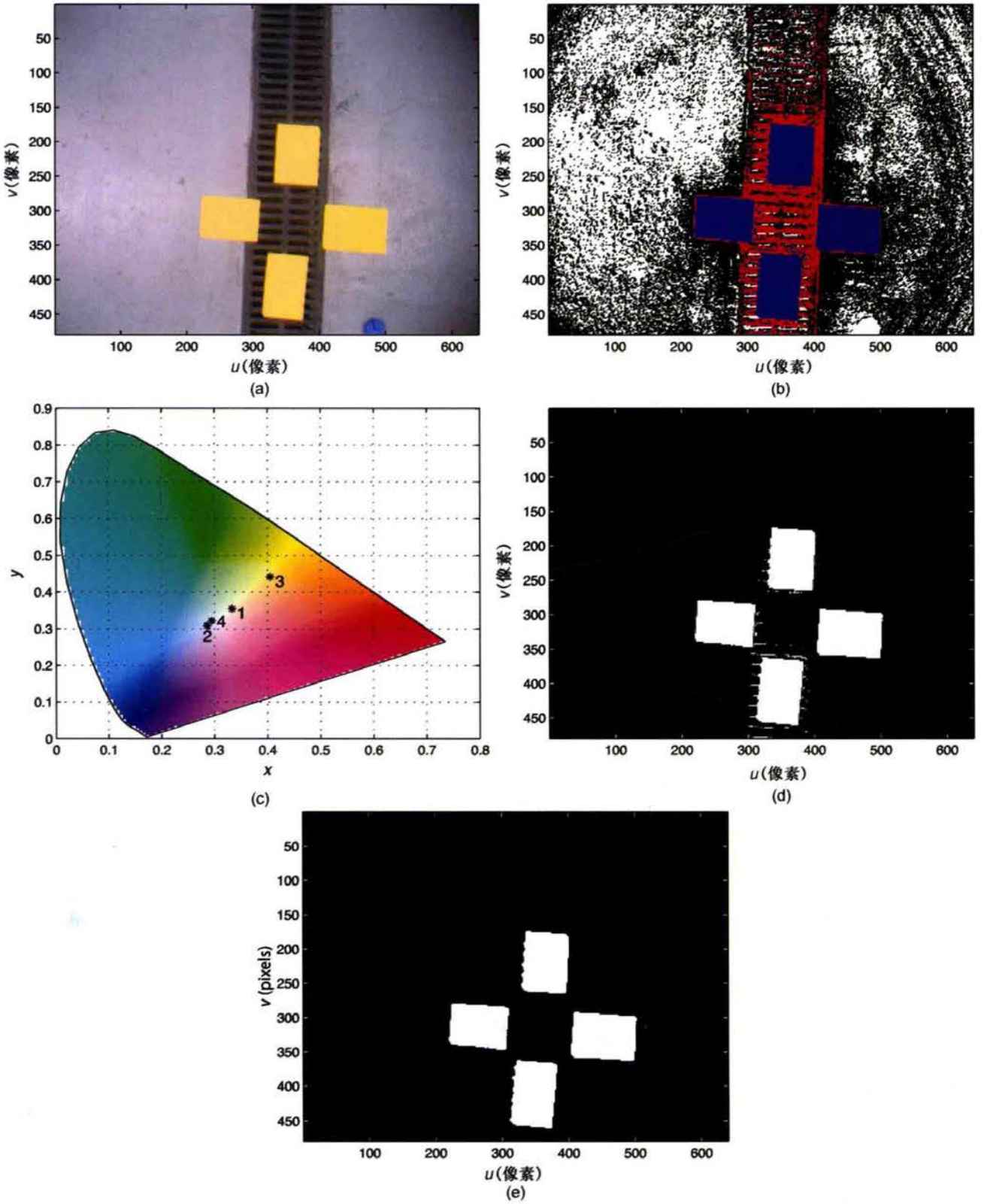


图 13.6　目标图像示例。(a)原始图像；(b)用伪色彩显示的像素分类（C=4）；(c)xy 色度空间的点集中心；(d)类型 c=3 的所有像素；(e)用一个圆形结构化元素进行形态学闭操作后的结果

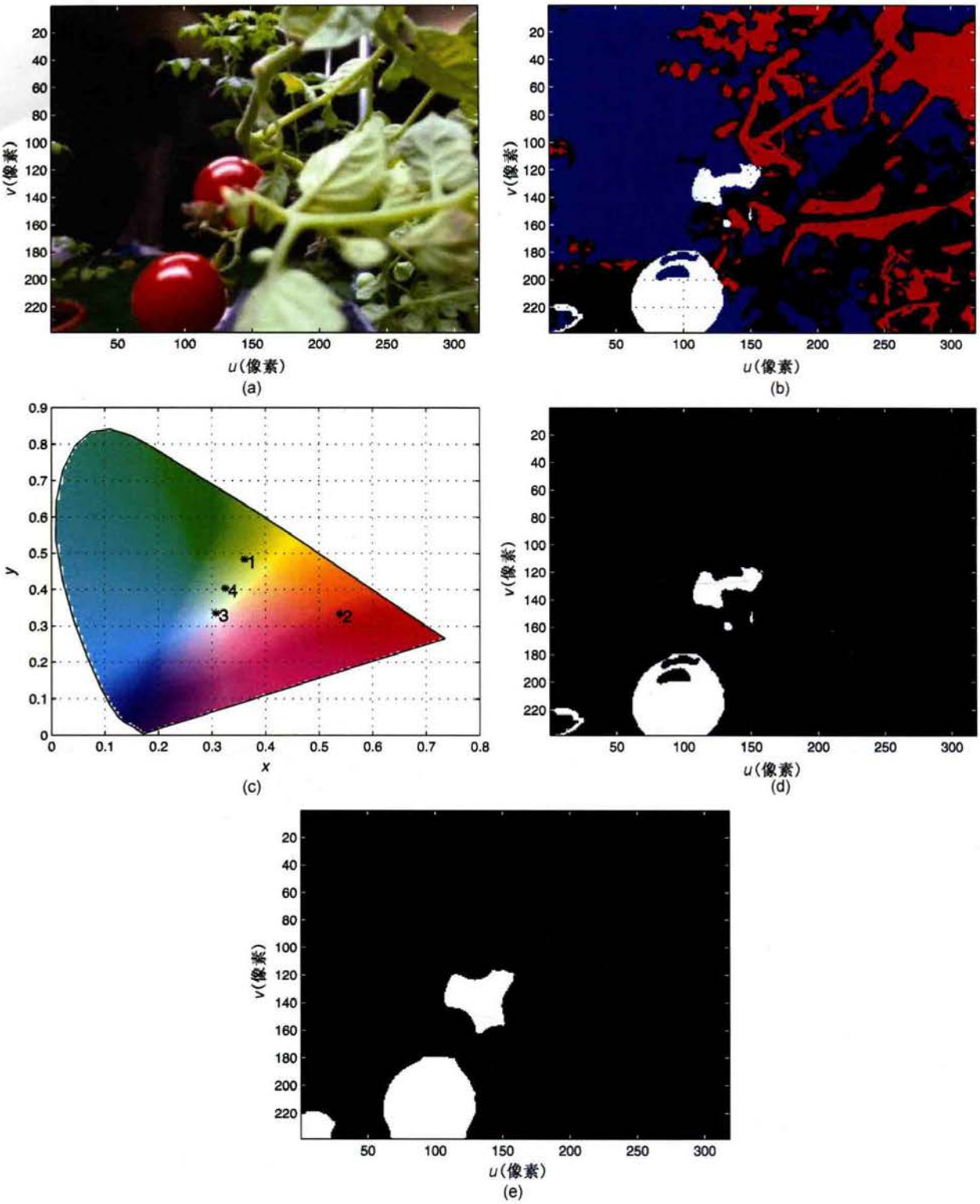


图 13.7　菜园图像示例。(a)原始图像（由 MIT 分布式机器人菜园项目提供）；(b)用伪色彩显示的像素分类（C=4）；(c)在 xy 色度二维空间中的点集中心；(d)类型 c=2 的所有像素；(e)用半径为 15 的圆形结构化元素进行形态学闭操作后的结果

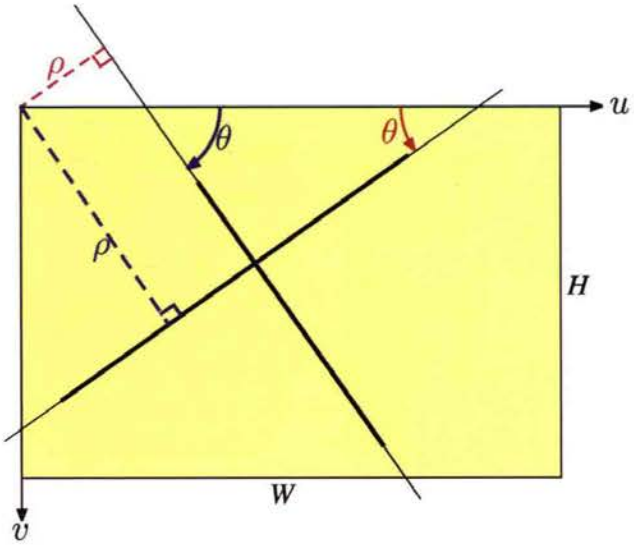


图 13.17　两条线段的(θ, ρ)参数化。正值显示为蓝色，负值显示为红色

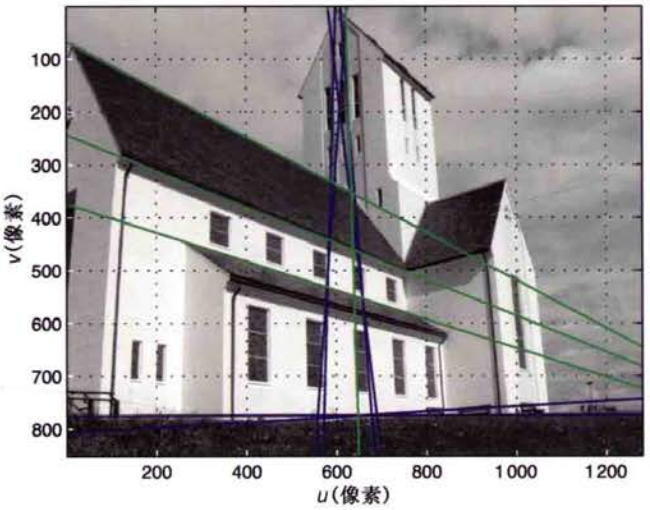


图 13.19　真实图像的霍夫变换。蓝色线对应于 10 条最强的选票峰值。叠加的绿色线是边缘段长度至少为 100 像素的那些直线

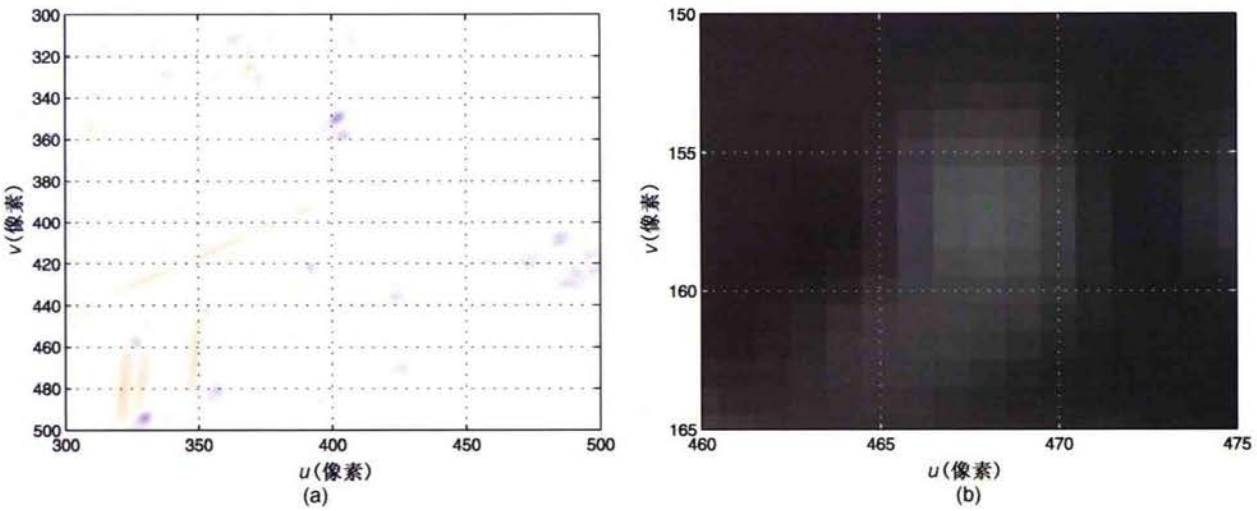


图 13.21　Harris 角点强度。(a)显示为一幅图像的角点强度放大视图（蓝色为正，红色为负）；(b)显示检出特征点的角点强度放大视图（亮度与强度成比例）

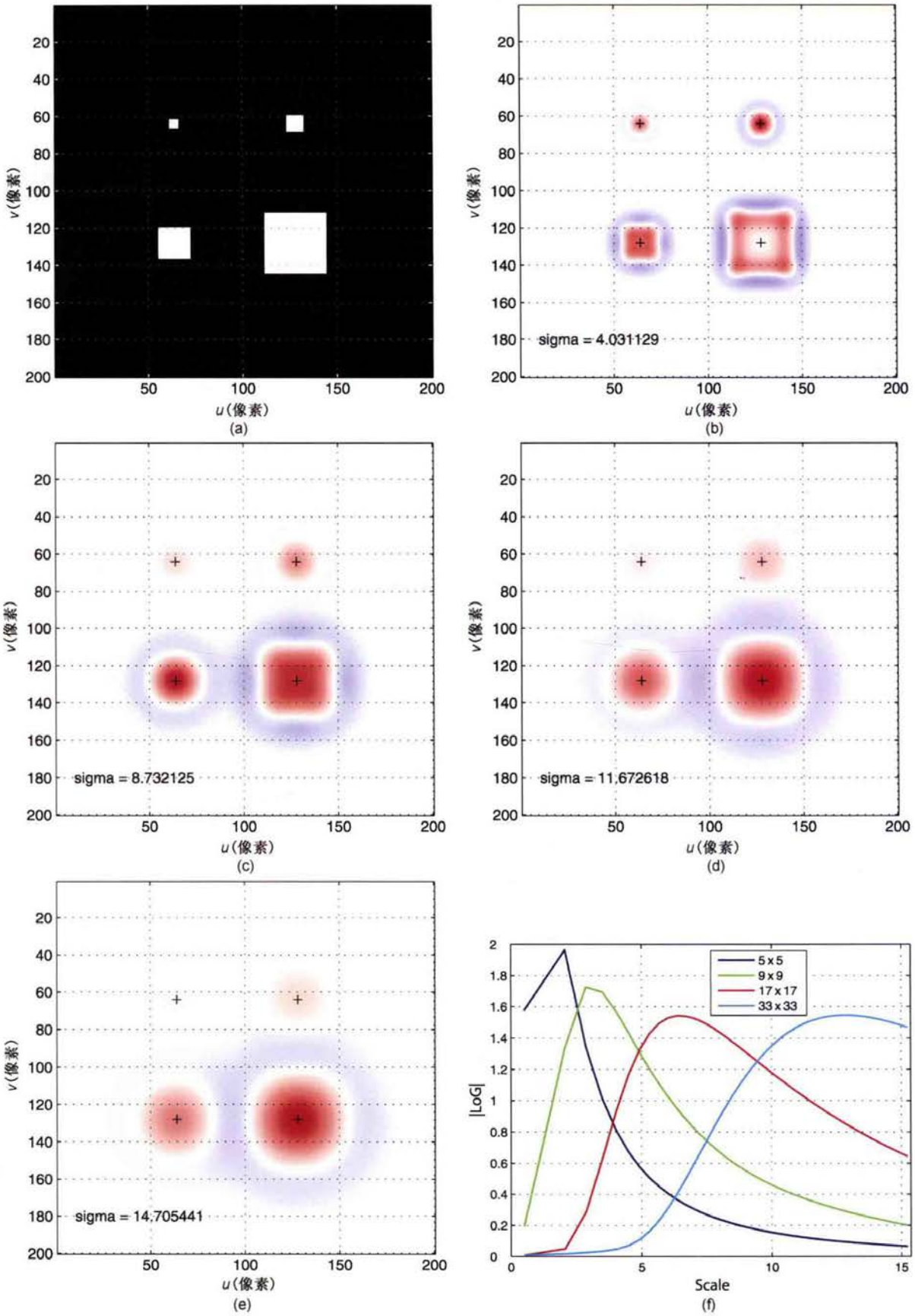


图 13.23　空间尺度示例。(a)具有方块大小分别为 5×5、9×9、17×17 和 33×33 的合成图像 $\boldsymbol{I}$；(b)~(e)对于渐增的尺度值，归一化的高斯拉普拉斯操作$\sigma^2 L \otimes G(\sigma) \otimes I$，$\sigma$值显示在左下角。使用了伪色彩显示：红色代表负，蓝色代表正；(f)每个正方形中心处（由“+”指示）随σ变化的高斯-拉普拉斯操作的幅值

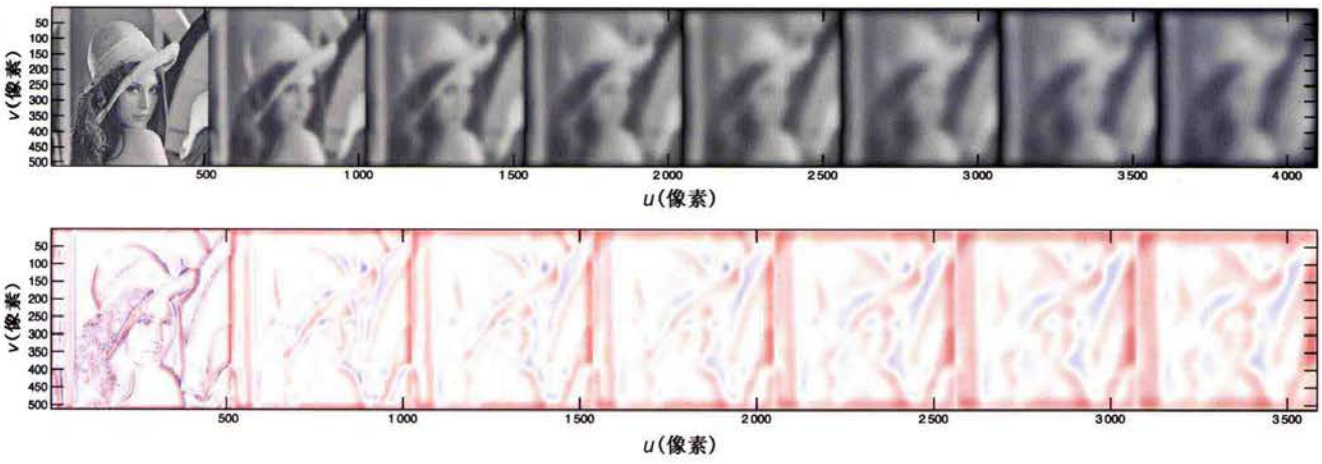


图 13.25　$\sigma=2$ 的尺度空间序列：（上图）高斯序列；（下图）高斯-拉普拉斯序列

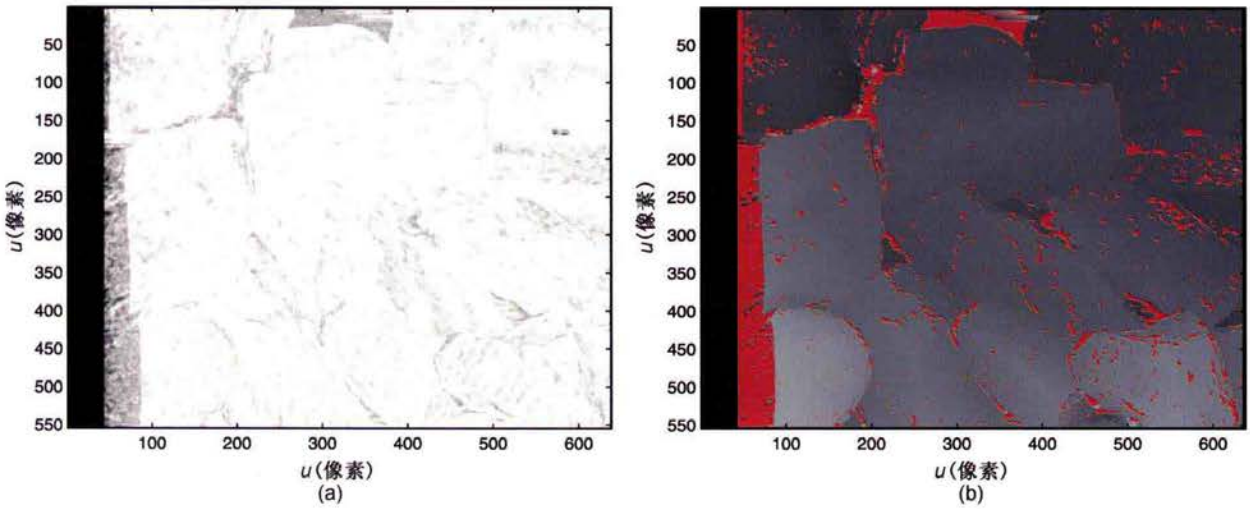


图 14.27　立体模板相似性。(a)相似性图像，其中越明亮的地方表示相似度越高；(b)差异图像，具有较低相似度得分的像素被标记为红色

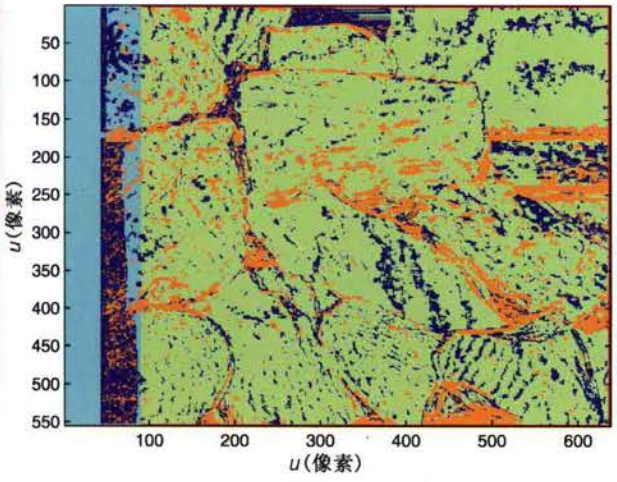


图 14.31　每个像素基础上的立体匹配状态。好的立体匹配（绿色）；差异搜索范围超过右侧图像左边缘（蓝绿色）；最大相似度较低（蓝色）；非尖锐峰值（橙色）；搜索模板超出图像边缘（红色）

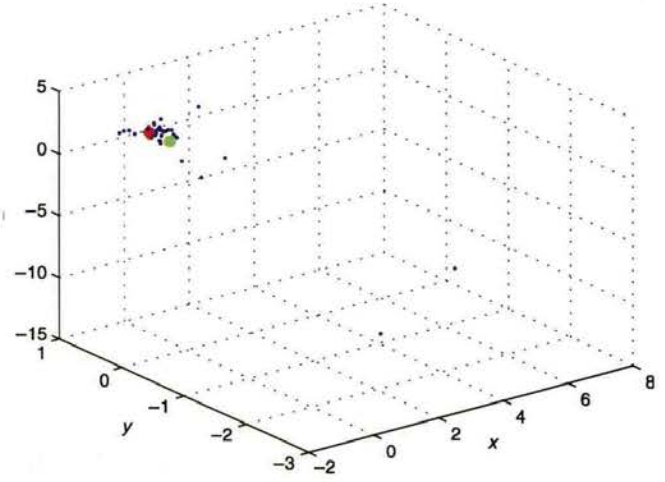


图 14.43　估计位置的扩散，像素噪声是σ=0.5。蓝点是成对估计的点位置，红色菱形标志是空间点（在相机坐标系中）的真实位置，绿色圆圈是估计位置的中值

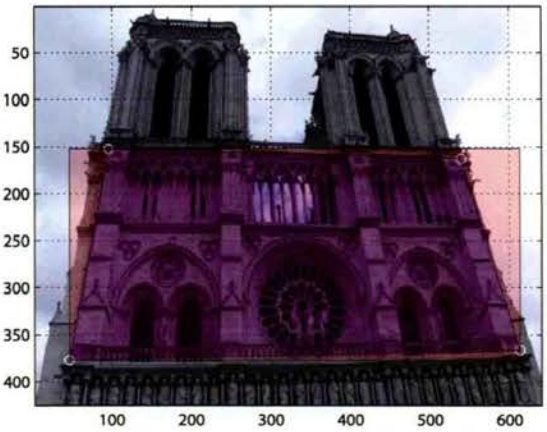


图 14.44　从地面上拍摄的照片显示了透视缩减的影响，它使建筑物呈现为梯形状（也称为梯形失真）。手动挑选了建筑物近似前平面上的4个点，用白色○标记点指示（巴黎圣母院）

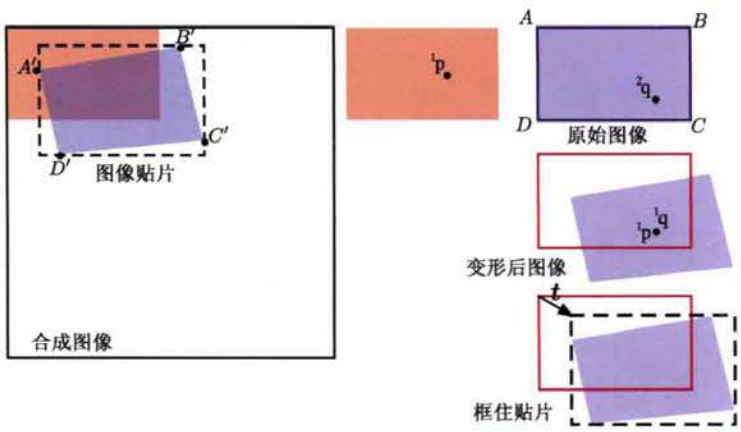


图 14.47　图像序列中的第一幅显示为红色，第二幅显示为蓝色。第二幅图像被变形成图像贴片，然后被拼接进合成图像

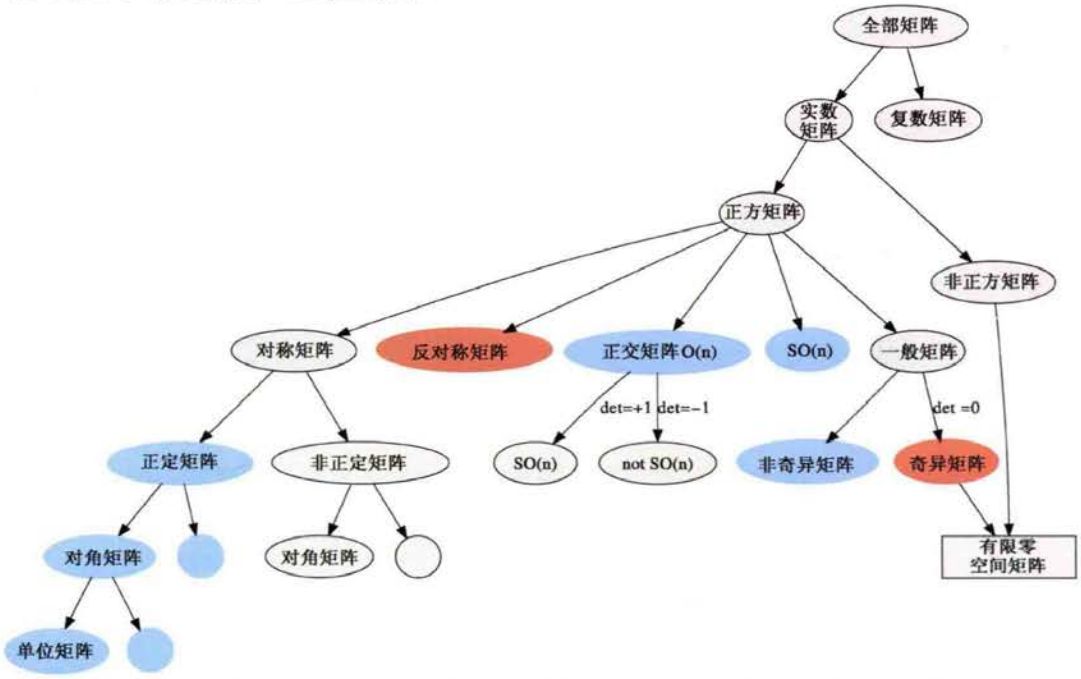


图 D.1　矩阵分类。总是奇异的矩阵用红色表示，总是非奇异的矩阵用蓝色表示

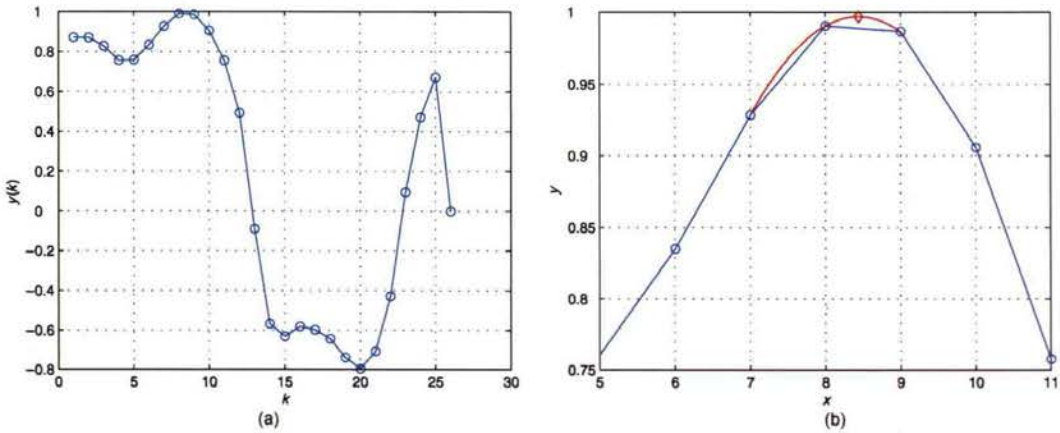


图 K.1　峰值拟合。(a)一个拥有多个局部最大值的离散信号；(b)在第一个最大值附近的局部放大图，含拟合曲线（红色）以及估计的峰值（红色的）

译 者 序

从世界上第一台工业机器人诞生至今，机器人技术已经有了长足的发展。智能机器人是现今机器人领域研究的热点，其特点之一是能够融合多种传感器的信息做出分析、判断和决策，以适应复杂的环境和任务。随着硬件成本的降低和计算机性能的提升，视觉传感器已逐渐成为机器人常用的感知器件。与其他种类传感器相比，视觉传感器能提供更加丰富和全面的信息，因而必将成为智能机器人的主要传感器。

目前，国内外有关机器人技术和机器视觉技术的专著很多，但大多是两种技术单独论述。即使有的机器人专著中包含了视觉技术，也是作为机器人传感器的一种简要介绍。本书是将机器人学与机器视觉知识有机结合的产物。虽然书中阐述了机器人学和机器视觉各自的基础知识，但更注重如何利用视觉信息控制机器人的运动，将两者融合运用。本书侧重知识的实用性，图片说明及应用实例贯穿全书，很少有理论方程的长篇大论。即使在讲述理论基础知识的同时，也穿插了很多实例，可以让读者对学到的知识点及时进行验证，使读者不断获得掌握知识的满足感，增加学习兴趣。另外，本书给出的实例算法和程序都是基于目前国内大专院校和研究所广泛使用的数学仿真软件包 MATLAB。该软件包功能强大，而且有很多开源的工具箱，其中本书主要使用的“机器人学工具箱”和“机器视觉工具箱”就是作者自己编写的。读者可以利用 MATLAB 中现成的函数、模块及工具箱完成机器人的运动和控制仿真，从而省去大量编程时间，便于快速验证和掌握所学的知识，也为下一步用其他语言编程打下了基础。

全书共分为 5 部分。第一部分是机器人学基础(第 2 章和第 3 章)，介绍了机器人及其操作对象的位置和姿态表达，以及机器人路径和运动的表示方法；第二部分和第三部分介绍了两类典型的机器人，其中第二部分(第 4 章至第 6 章)是关于移动机器人的，介绍了其基本运动控制模式及其导航和定位方法；第三部分(第 7 章至第 9 章)针对关节式机械臂，介绍了其运动学、动力学和控制方面的知识；第四部分(第 10 章至第 14 章)涉及了机器视觉技术的基础知识，包括光照与色彩、图像形成、图像处理、图像特征提取，以及立体视觉技术；第五部分(第 15 章和第 16 章)介绍了基于视觉的机器人控制方法，包括基于位置的视觉伺服和基于图像的视觉伺服，以及更先进的混合视觉伺服方法。

本书可以作为学习机器人技术和视觉控制的高年级本科生和研究生的教辅书籍。现有的专业教材一般理论性较强，知识比较抽象，难以理解。同时，要在实际机器人上开展大量实验成本既高，也耗时。本书中大量的仿真实验实例会帮助学生更容易和及时地验证并掌握相关知识。该书也可以作为从事机器人技术研究和开发人员的技术参考书，可以帮助他们在机器人投入实际制作前对机器人的运动和控制进行仿真，验证其设计的合理性和有效性。

参加本书翻译工作的有刘荣(翻译第 1 章、第 6 章、第 15 章和第 16 章及附录)、薛彤(翻译第 2 章和第 3 章)、陈锐(翻译第 4 章和第 5 章)、徐江平(翻译第 7 章至第 9 章)、李扬(翻译第 10 章至第 12 章)和翟士民(翻译第 13 章和第 14 章)。全书由刘荣负责统稿和审校。鉴于译者水平有限，难免有错误和不妥之处，敬请广大读者批评指正。

序　言

很久以前，我收到一本很厚的博士论文让我评审。论文的主题是关于机器人视觉控制的，作者是彼得·科克(Peter Corke)。以下是我对该论文评语的一段摘录：这是一篇杰作，无论在论文构思还是实际内容上都是出类拔萃的，其质量之高堪称博士生论文的表率。

二十多年来，彼得·科克一直致力于机器人与视觉二者的集成技术研究和探索，并且取得了很多突出的成果。现在，他的这些成果终于在本书中与大家见面了。在这本新书中，不仅有前沿理论，还有应用实例，它包含了作者多年来在井下机器人、无人飞行器、水下机器人、野外作业机器人等领域探索中所取得的富有特色的基础和应用研究成果。

虽然现在已有很多关于机器人以及视觉的专著，但很少有像这本书一样在两种技术的集成、剖析以及例证方面做得如此出色的。书中对有关技术的探讨和论述既深入透彻，又通俗易懂。对于机器人领域正在或将要从事这方面研究的人员而言，这是一本绝对有价值的参考书。本书涉及的技术比较全面，对于要实现机器人视觉任务的几乎每个方面都有涵盖与分析，这一点读者可以从书中对 MATLAB 工具箱(Toolbox)软件的有效使用中体会到。

本书内容按读者接受相关知识的先后逻辑进行安排，从移动机器人基础到机器人导航、定位，以及机械臂的运动学、动力学和关节控制，再到摄像机建模、图像处理、特征提取以及多图像几何学。最后，将以上技术知识通过集成到视觉伺服系统中进行更广泛的探讨。在上述过程中，作者展示了如何利用强有力的数字工具和有效软件对复杂问题进行分解并最后解决的方法。

Springer Tracts in Advanced Robotics(STAR，施普林格先进机器人技术集)系列丛书致力于将机器人学领域的最新研究进展引入学术界，内容选择是基于它们的重要性和研究质量。出版该系列丛书的目的是要通过对机器人学一些关键技术开发进展的广泛而及时的传播，推动学术界研究人员之间的交流与合作，并为该技术的进一步发展做出贡献。

彼得·科克的这本专著以其跨学科性、缜密的构思和精彩的内容为 STAR 系列丛书增色良多。

欧萨马·卡蒂布(Oussama Khatib)

加州，斯坦福

2011 年 7 月

前　言

告诉我，我会忘记。
给我看，我可能记住。
让我参与其中，我才会理解。
——中国谚语

机器人和机器视觉的研究都涉及到用算法来处理大量数据。这些数据来源于各种传感器，如测量车轮转速的、测量机械臂关节转角的、或者测量机器人环境观察图像中各像素亮度的。对于很多的机器人应用，需要实时处理的数据量非常大，就视觉处理而言，数据会达到每秒几十到几百兆字节的量级。

过去，机器人和机器视觉领域中的许多技术进步都是由于数据处理方法的改进而推动的，并且这种促进作用仍然继续着。这些数据处理方法的改进一方面体现在大量新的并且更有效的算法的涌现，另一方面则是摩尔定律下的机器本身计算能力的大幅提升。回想 20 世纪 80 年代中我踏入机器人和视觉研究领域时，IBM 公司的个人计算机才刚推出，它用的仅仅是主频 4.77 MHz 的 16 位处理器，以及 16 KB 的内存(扩展后才 256 KB)。经过 25 年后，计算机的计算能力已经翻番了 16 次，也就是增长了大约 65 000 倍。在 20 世纪 80 年代后期，能够实时进行处理图像的系统需要 19 英寸宽的机柜才能装下(见图 0.1)。而今天，一块小小的高性能处理芯片就能胜过那时的图像处理能力。

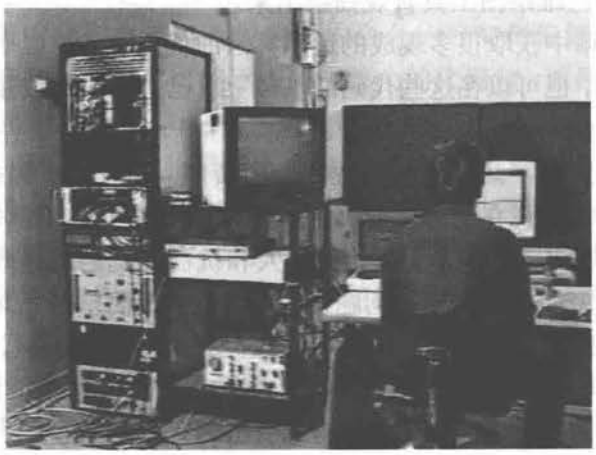


图 0.1　很久以前，要完成基于视觉的机器人控制需要很多设备。该图片是 1992 年作者在实验室里和一大堆图像处理与机器人控制的设备在一起

在机器人和机器视觉近一段时间的发展历程中，研究者们开发出了大量的各种算法，它们是学术界通过技术积累获得的重要成就。然而这些算法不仅数量多，而且都比较复杂，对于刚涉足该领域的人来说是不小的障碍。要想从这么多算法中进行选择，首当其冲的问题就是：

为解决这个特定问题，哪一个才是最佳算法？

一种策略是去尝试多种不同的算法，看哪种算法解决问题的效果最好。但这样做又带来新的问题：

我怎样才能不用花很长时间去编辑和调试程序，就能评判出原始论文中的算法是否适合我自己的数据？

有两种解决方案可以帮助我们。一种是利用通用的数学工具软件，它们能简化算法编程。这类工具中包括商用软件包，如 MATLAB、Mathematica 和 MathCad，也有开源软件包，如 SciLab，Octave 和 PyLab。所有这些工具软件包都能轻松自然地处理向量和矩阵，也能绘制出漂亮和复杂的图形，同时具有交互式的编程环境。另一种是开源软件模块。很多学者都把自己开发的算法以开源形式共享出来，有的是用上面提到的通用数学软件包编写的，有的则是用目前主流的 C、C++和 Java 语言编写的。

我本人 15 来年一直活跃在开源软件的圈子，曾经管理过两个开源的 MATLAB 工具箱软件：一个是机器人的，另一个是机器视觉的。这两个工具箱可以追述到我做博士论文时期，并且从那时起它们的功能一直在扩展，还随着 MATLAB 软件语言的改进不断提升（这一点在当时非常重要）。机器人工具箱还被翻译成多种其他编程语言，如 Python，SciLab 和 LabView。

这两个工具箱有一些很重要的特征。第一，它们已经很长时间被很多人使用以解决各种不同的问题，因此代码的可信度较高。在检验新的算法，亦或旧的相同算法要用其他语言编写或者在新环境中执行时，工具箱起到“黄金标准”的作用。

第二，这些工具箱能让使用者接触到真实的问题，而非简单无实际意义的例子。对于超过两个关节的真实机器人，或者有几百万像素的真实图像，一般人没有外力帮助是无法对它们进行计算的。第三，工具箱能让我们窥探问题的本质，而不是迷失在纷繁的细枝末节上。用这些工具箱我们能快速而简单地进行实验，开展“如果-就会”（what-if）的探究，并能把结果用 MATLAB 强大的二维和三维绘图工具直观描绘出来。

第四，可以从工具箱中获取很多现成的通用算法的代码。你可以研读这些代码，可以把它们直接用到你的问题上，也可以在这些代码的基础上自己进行扩充或重写。总之，这些现成的代码至少能让你知道如何起步。

这些工具箱还通常附带一些简单的教程和参考资料。多年来，很多读者都请求我把这些材料变成一本正式的书籍。现在它终于成真了！这本书是在我的工具箱提供的教程材料基础上扩展而来的，它增加了更多的例子，并让机器人和机器视觉的叙述既相互独立又紧密关联。我想要展示给读者的是怎样将一个复杂问题进行分解，并用简单的几句代码就能解决。

从性格特征上说，我是比较喜欢动手的那类人。我喜欢自己编程，喜欢对数据进行分析，因此对我而言，自己编写工具软件去解决机器人及视觉方面的问题就再自然不过了。书中探讨的内容既是基于自己的兴趣，也是从真实问题引出的，这些问题主要是多年来我亲自从事机器人和机器视觉研究中观察到的。我希望学完这本书后也能点燃你对书中内容的兴趣和热情。

我特别希望在本书中为机器人学者较全面地介绍机器视觉方面的知识。在以往的机器人教科书中，视觉部分的内容通常集中在简单的黑白图像处理。而在本书中，视觉部分涉及的内容将更广，包括彩色视觉、高级图像分割，如最大稳定极值区域及图割技术、图像卷绕、立体视觉、运动估计以及图像检索。另外，书中还涉及用鱼眼镜头和折反射光学系统的非透视成像技术。以上这些内容在机器人实际应用中越来越重要，但在一般书中很少探讨。视觉是一种功能强大的传感技术，机器人学者应当对现代视觉技术有全面深入的了解。本书最后一部分给读者展示了视觉设备是如何作为机器人控制的主传感器使用的。

读者会发现本书与其他教科书有很大不同，这是作者有意为之。第一，市面上已经有很多深入研究机器人或机器视觉的图书，但它们大多是机器人与视觉技术分离的，集成者很少。因此将机器人与视觉二者集成是本书的主要目标。

第二，程序在本书中占了很大篇幅。事实上，程序是我们研究的各种算法的具体表现。算法是比较抽象的，程序则可以一句句读懂，而且我们还可以任意从程序中拿出一段进行修改，然后再放回去。市面上已有很多科技书籍就是使用程序来演示一个问题是如何解决的，如 *LaTeX: A document preparation system*（Lamport 1994），*Numerical Recipes in C*（Press et al. 2007），*The Little Lisper*（Friedman et al. 1987），以及 *Structure and Interpretation of Classical Mechanics*（Sussman et al. 2001），本书在这方面可以说是受了它们的影响。本书中给出的很多例程展示了如何使用工具箱软件，并且仅用几行 MATLAB 语句就给出了一个及时的体验。

第三，本书内容是基于 MATLAB 软件包及其工具箱编写的。相对于其他书籍，这样的好处是能够容易地处理一些更实际和更复杂的问题。

本书对于程序和例子的强调并不意味着基础理论不重要。相反，它们是非常重要的，本书只是采用了一种二者相互补充的方式。学习本书的同时最好参考一下其他在基础理论上论述更严密的材料。本书每章在结束时都提供了一些延伸阅读的内容，列出了相关的书籍和论文。

写这本书其实给我提供了一个很好的机会去重新审视我的那些工具箱，对其中的程序代码进行修改和扩展。特别要说的是我这次更好地利用了 MATLAB 新提供的一些面向对象的特征简化了工具箱的用户界面，并减少了其中的文件数量。

修改这些工具箱程序也让我以更广的视角去看待各种开源代码。现在网上已经有很多优秀的程序代码，特别是有关机器视觉方面的，所以我完全没有必要从头开始自己编写算法程序，只需要去找到那些最好的子程序代码，然后把它们整合在一起就可以了。这样做的困难之处在于每个代码的作者都有他们自己对于数据结构命名的习惯和偏好，这些数据从简单的向量到复杂的数组都有。我的办法是尽可能不去修改单个的代码包，而是用一些轻量包装，特别是用作类去把它们封装起来。

我要感谢以下人员，他们提供的代码要么被嵌入，要么被包装进了我的工具箱。机器人工具箱的贡献者如下：牛津大学的保罗·纽曼（Paul Newman）提供了移动机器人定位及建图代码，耶鲁大学的保罗·庞兹（Paul Pounds）提供了四旋翼直升机仿真器代码。机器视觉工具箱的贡献者如下：皮特·考乌斯（Peter Kovesi）提供了随机抽样一致性算法（RANSAC）代码；瑞士洛桑理工大学计算机视觉实验室（CVLab-EPFL）的弗朗西斯科·莫雷罗·罗古尔（Francesco Moreno-Noguer）、文森特·来皮特（Vincent Lepetit），以及帕斯卡·福安（Pascal Fua）提供了姿态估算算法代码；帕斯卡·吉崔尔（Pascal Getreuer）提供了彩色空间变换代码；牛津大学的视觉几何工作组（Visual Geometry Group）的多人提供了几何视觉的数值算法代码（来自于哈特利（Hartley）及齐瑟曼（Zisserman）2003 出版图书的网站）；安德烈·维达尔迪（Andrea Vedaldi）和布莱恩·福克尔森（Brian Fulkerson）提供了 K 均值（k-means）聚类算法和最稳定性极值区域（MSER）算法；佩德罗·菲尔岑斯瓦布（Pedro Felzenszwalb）提供了基于画面的图像分割软件；以及荷兰特温特大学的迪尔克-让·克鲁恩（Dirk-Jan Kroon）提供了超鲁棒性特征（SURF）检测器算法。另外，由吉安-耶威斯·薄古尔特（Jean-Yves Bouguet）提供的摄像机标定工具箱（Camera Calibration Toolbox）未经任何修改就集成到我的工具箱中。

几百年来，世界各国涌现出了很多杰出的数学家、物理学家以及工程师，他们的成就对于

今天的机器人和视觉科学研究仍然是至关重要的。他们中很多人的名字成为一些术语前的形容词，如哥氏(Coriolis)、高斯(Gaussian)、拉普拉斯(Laplacian)、笛卡儿(Cartesian)等，有的直接就是一个名词术语，如雅可比(Jacobian)，或者计量单位名称，如牛顿(Newton)和库仑(Coulomb)。其实这些伟大科学家的经历都很奇特，因为他们在当时的正式职业只是医生、炼金术士、投机商、占星家、哲学家或雇佣兵，科学研究只能算他们的业余爱好。为了了解我们都站在了哪些人的肩膀上，我在书中还穿插了一些有关那些伟大科学家们的历史小故事，作为背景材料供读者赏读。

在我本人的职业生涯中，我非常有幸能同许多优秀的人共事，他们给我的事业很多启发和引导。很久以前在墨尔本大学，是约翰·安德森(John Anderson)点燃了我对控制技术的兴趣，而格雷汉姆·霍姆斯(Graham Holmes)则鼓励我"在编程前先要好好思考"，这个好建议我至今还努力去遵从。在1988～1989年期间，我曾在宾夕法尼亚大学的GRASP实验室与理查德·保罗(Richard Paul)共事过10个月，这段工作经历可以说改变了我以后的人生。

我开发MATLAB工具箱是源自我的博士论文，我的博士导师是墨尔本大学的马尔科姆·古德(Malcolm Good)，他和来自澳大利亚联邦科学与工业研究组织(CSIRO)的保罗·邓恩(Paul Dunn)给我提了很多好问题并指导我的研究工作。拉兹罗·尼米斯(Naszlo Nemes)则在生活和组织方面给我很多有益的建议，并鼓励我把我编写的程序代码公开和共享。到目前为止，我职业生涯的大部分时间是在澳大利亚联邦科学与工业研究组织度过的，在那里我有幸参与了很多不同的机器人研究项目，并与很多有天赋的同事和朋友共事。在撰写本书期间，我转到澳大利亚昆士兰理工大学工作，学校给了我很多空余时间完成本书。我以前的学生贾思明·班克斯(Jasmine Banks)、卡恩·亚瑟(Kane Usher)、保罗·庞兹(Paul Pounds)和彼得·汉森(Peter Hansen)分别教给了我很多有关立体视觉、非完整性、四旋翼直升机、广角视场方面的知识。

我还要感谢保罗·纽曼(Paul Newman)，他数次邀请我去牛津大学，我在那里完成了本书的很多重要章节。我还要感谢达尼拉·鲁斯(Daniela Rus)邀请我去麻省理工学院，我在那里完成了本书的第一稿。达尼拉、保罗和塞德里克·普拉达里尔(Cédric Pradalier)还在本书早期素材准备方面给予了我很多建设性的意见和建议。当然，我还要感谢MATLAB软件的发行商MathWorks公司，他们有专为作者提供的帮助计划。施普林格(Springer)出版社为本书出版提供了全程巨大的帮助，我与他们的合作非常愉快。我要特别感谢本书的编辑托马斯·迪驰林格(Thomas Ditzinger)以及阿明·施塔斯(Armin Stasch)，他们将我的手稿录入并完成排版。

虽然我已尽了最大努力去避免，但书中肯定还会有错误。如果读者发现有错误的地方，或者对本书内容有改进建议，请发Email给我，我的邮箱是rvc@petercorke.com。

最后我想说，与一名作家一起生活总是不容易的，很多书籍和网站还对此有专门的探讨。所以，我要把我最深情的感谢送给菲利帕(Phillipa)，她在我写作本书过程中给予我极大的支持和鼓励，并且还在世界各地"陪伴"此书很长时间。

彼得·科克

布里斯班，昆士兰

2011年6月

目　录

第 1 章　绪言 …… 1
1.1　关于本书 …… 7
1.1.1　MATLAB 软件 …… 7
1.1.2　读者对象及必备知识 …… 9
1.1.3　符号与约定 …… 10
1.1.4　怎样使用本书 …… 10
1.1.5　使用本书教学 …… 11
1.1.6　本书梗概 …… 11

第一部分　基 础 知 识

第 2 章　位置与姿态描述 …… 14
2.1　二维空间位姿描述 …… 17
2.2　三维空间位姿描述 …… 22
2.2.1　三维空间姿态描述 …… 22
2.2.2　平移与旋转组合 …… 34
2.3　本章总结 …… 36
延伸阅读 …… 38
习题 …… 38
第 3 章　时间与运动 …… 39
3.1　轨迹 …… 39
3.1.1　平滑一维轨迹 …… 39
3.1.2　多维的情况 …… 42
3.1.3　多段轨迹 …… 43
3.1.4　三维空间姿态插值 …… 44
3.1.5　笛卡儿运动 …… 45
3.2　时变坐标系 …… 48
3.2.1　旋转坐标系 …… 48
3.2.2　增量运动 …… 48
3.2.3　惯性导航系统 …… 50
3.3　本章总结 …… 52
延伸阅读 …… 52
习题 …… 53

第二部分　移动机器人

第 4 章　移动机器人载体 …… 59
4.1　机动性 …… 59
4.2　移动机器人小车 …… 62
4.2.1　移动到一个点 …… 65
4.2.2　跟踪一条直线 …… 67
4.2.3　跟踪一般路径 …… 68
4.2.4　运动到一个位姿 …… 70
4.3　飞行机器人 …… 72
4.4　本章总结 …… 78
扩展阅读 …… 79
习题 …… 80

第 5 章　机器人导航 …… 82
5.1　反应式导航 …… 82
5.1.1　Braitenberg 车 …… 83
5.1.2　简单自动机 …… 85
5.2　基于地图的路径规划 …… 87
5.2.1　距离变换 …… 88
5.2.2　D* …… 90
5.2.3　沃罗诺伊路线图法 …… 92
5.2.4　概率路线图方法 …… 95
5.2.5　RRT …… 98
5.3　本章总结 …… 100
扩展阅读 …… 100
习题 …… 101

第 6 章　机器人定位 …… 103
6.1　航迹推算 …… 107
6.1.1　机器人建模 …… 107
6.1.2　位姿估计 …… 109
6.2　使用地图 …… 113
6.3　创建地图 …… 117
6.4　定位并制图 …… 119
6.5　蒙特卡罗定位 …… 121
6.6　本章总结 …… 125
扩展阅读 …… 125
习题 …… 127

第三部分　臂型机器人

第 7 章　机械臂运动学 …… 132
7.1　描述一台机械臂 …… 132
7.2　正运动学 …… 135
7.2.1　两连杆机器人 …… 135
7.2.2　六轴机器人 …… 138
7.3　逆运动学 …… 141
7.3.1　封闭形式解 …… 142
7.3.2　数值解 …… 144
7.3.3　欠驱动机械臂 …… 145
7.3.4　冗余机械臂 …… 145
7.4　轨迹 …… 148
7.4.1　关节空间运动 …… 148
7.4.2　笛卡儿运动 …… 151
7.4.3　通过奇异位形的运动 …… 152
7.4.4　位形转换 …… 153
7.5　高级问题 …… 154
7.5.1　关节角偏移 …… 154
7.5.2　确定 D-H 参数 …… 154
7.5.3　改进 D-H 参数 …… 156
7.6　应用：绘图 …… 157
7.7　应用：一个简单的步行机器人 …… 158
7.7.1　运动学 …… 158
7.7.2　单腿运动 …… 160
7.7.3　四腿运动 …… 161
7.8　本章总结 …… 162
扩展阅读 …… 162
习题 …… 164

第 8 章　速度关系 …… 166
8.1　机械手的雅可比矩阵 …… 166
8.1.1　坐标系之间的速度转换 …… 170
8.1.2　末端执行器坐标系的雅可比矩阵 …… 171
8.1.3　解析雅可比矩阵 …… 171
8.1.4　雅可比条件及可操纵性 …… 172
8.2　分解速率运动控制 …… 175
8.2.1　雅可比矩阵的奇异性 …… 178
8.2.2　欠驱动机器人的雅可比矩阵 …… 178
8.2.3　过驱动机器人的雅可比矩阵 …… 180

8.3 力的关系 …… 181
8.3.1 坐标系间的力旋量转换 …… 181
8.3.2 力旋量转换至关节空间 …… 182
8.4 逆运动学：一个通用数值方法 …… 182
8.5 本章总结 …… 184
扩展阅读 …… 184
习题 …… 184
第 9 章 动力学与控制 …… 186
9.1 运动方程 …… 186
9.1.1 重力 …… 188
9.1.2 惯量矩阵 …… 189
9.1.3 科氏矩阵 …… 191
9.1.4 有效载荷的影响 …… 192
9.1.5 基座力 …… 193
9.1.6 动态可操作性 …… 193
9.2 传动系统 …… 194
9.2.1 摩擦 …… 195
9.3 正向动力学 …… 197
9.4 机械臂关节控制 …… 198
9.4.1 驱动器 …… 198
9.4.2 独立关节控制 …… 199
9.4.3 刚体动力学补偿 …… 205
9.4.4 柔性传动 …… 208
9.5 本章总结 …… 209
扩展阅读 …… 210
习题 …… 211

第四部分 计算机视觉

第 10 章 光与色彩 …… 217
10.1 光的谱表示 …… 217
10.1.1 吸收 …… 219
10.1.2 反射 …… 220
10.2 色彩 …… 221
10.2.1 颜色再造 …… 225
10.2.2 色度空间 …… 227
10.2.3 色彩名称 …… 231
10.2.4 其他颜色空间 …… 231
10.2.5 两种原色之间的转换 …… 233
10.2.6 什么是白色 …… 235

10.3 高级议题 …… 235
10.3.1 颜色的不变性 …… 236
10.3.2 白平衡 …… 237
10.3.3 由于吸收的颜色变化 …… 238
10.3.4 伽玛 …… 238
10.3.5 应用：彩色图像 …… 240
10.4 本章总结 …… 242
扩展阅读 …… 243
习题 …… 244
第 11 章 图像形成 …… 246
11.1 透视变换 …… 246
11.1.1 透镜畸变 …… 255
11.2 相机标定 …… 256
11.2.1 齐次变换法 …… 257
11.2.2 分解相机标定矩阵 …… 258
11.2.3 位姿估计 …… 259
11.2.4 相机标定工具箱 …… 260
11.3 非透视成像模型 …… 261
11.3.1 鱼眼镜头相机 …… 262
11.3.2 反射折射相机 …… 264
11.3.3 球形相机 …… 266
11.4 统一化成像 …… 268
11.4.1 映射广角图像到球面上 …… 269
11.4.2 合成透视图像 …… 270
11.5 本章总结 …… 272
扩展阅读 …… 272
习题 …… 275
第 12 章 图像处理 …… 276
12.1 获取图像 …… 276
12.1.1 来自文件的图像 …… 276
12.1.2 来自附带相机的图像 …… 280
12.1.3 来自电影文件的图像 …… 280
12.1.4 来自网络的图像 …… 281
12.1.5 来自代码的图像 …… 282
12.2 一元操作 …… 284
12.3 二元操作 …… 287
12.4 空间操作 …… 291
12.4.1 互相关 …… 291
12.4.2 模板匹配 …… 302

12.4.3 非线性操作 …… 307
12.5 数学形态学 …… 309
12.5.1 去除噪声 …… 312
12.5.2 边界检测 …… 313
12.5.3 形态交离变换 …… 314
12.6 形状变化 …… 315
12.6.1 裁剪 …… 315
12.6.2 图像缩放 …… 316
12.6.3 图像金字塔 …… 317
12.6.4 图像变形 …… 318
12.7 本章总结 …… 321
扩展阅读 …… 323
习题 …… 324
第 13 章 图像特征提取 …… 326
13.1 区域特征 …… 326
13.1.1 分类 …… 328
13.1.2 图像表示 …… 337
13.1.3 图像描述 …… 341
13.1.4 简要回顾 …… 351
13.2 直线特征 …… 351
13.3 点特征 …… 356
13.3.1 经典角点检测器 …… 356
13.3.2 尺度空间角点检测器 …… 361
13.4 本章总结 …… 366
扩展阅读 …… 367
习题 …… 368
第 14 章 使用多幅图像 …… 370
14.1 特征匹配 …… 371
14.2 多视图几何学 …… 375
14.2.1 基本矩阵 …… 377
14.2.2 本质矩阵 …… 379
14.2.3 估计基本矩阵 …… 381
14.2.4 平面单应性 …… 385
14.3 立体视觉 …… 390
14.3.1 稀疏立体匹配 …… 390
14.3.2 密集立体匹配 …… 394
14.3.3 峰值细化 …… 401
14.3.4 清理和重构 …… 403
14.3.5 三维纹理映射显示 …… 405

14.3.6 补色立体图 …… 406
14.3.7 图像矫正 …… 407
14.3.8 平面拟合 …… 409
14.3.9 3D 点集匹配 …… 410
14.4 结构和运动 …… 412
14.5 应用：透视矫正 …… 418
14.6 应用：拼接 …… 421
14.7 应用：图像匹配和检索 …… 423
14.8 应用：图像序列处理 …… 430
14.9 本章总结 …… 432
扩展阅读 …… 432
习题 …… 436

第五部分 机器人学、视觉与控制

第 15 章 基于视觉的控制 …… 444
15.1 基于位置的视觉伺服 …… 445
15.2 基于图像的视觉伺服 …… 448
15.2.1 相机和图像运动 …… 448
15.2.2 控制特征运动 …… 453
15.2.3 深度 …… 457
15.2.4 控制性能分析 …… 459
15.3 使用其他图像特征 …… 461
15.3.1 直线特征 …… 461
15.3.2 圆特征 …… 463
15.4 本章总结 …… 465
扩展阅读 …… 465
习题 …… 467

第 16 章 高级视觉伺服 …… 469
16.1 *XY/Z* 分割的 IBVS …… 469
16.2 使用极坐标的 IBVS …… 471
16.3 对一个球面相机的 IBVS …… 473
16.4 应用：机械臂机器人 …… 476
16.5 应用：移动机器人 …… 477
16.5.1 完整约束移动机器人 …… 477
16.5.2 非完整约束移动机器人 …… 478
16.6 应用：飞行机器人 …… 480
16.7 本章总结 …… 481
扩展阅读 …… 481
习题 …… 482

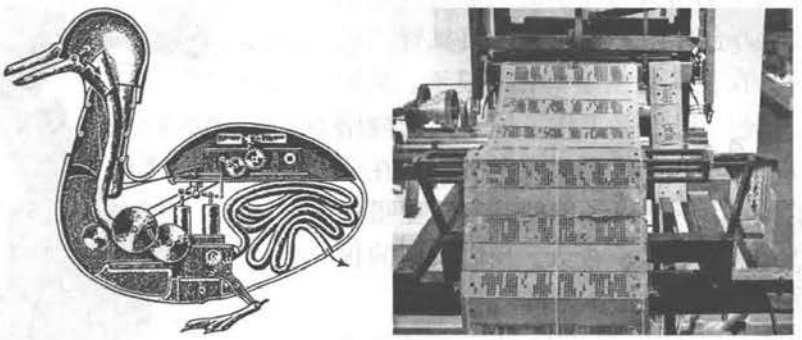
附　录

附录 A　安装工具箱 …… 486
附录 B　Simulink 软件 …… 488
附录 C　MATLAB 对象 …… 492
附录 D　线性代数复习 …… 498
附录 E　椭圆 …… 504
附录 F　高斯随机变量 …… 510
附录 G　雅克比矩阵 …… 513
附录 H　卡尔曼滤波 …… 515
附录 I　齐次坐标系 …… 519
附录 J　图 …… 521
附录 K　峰值搜索 …… 524
术语 …… 527
参考文献 …… 531

第1章 绪 言

机器人这个名字的含义可谓仁者见仁，智者见智。科幻图书和电影对于人们认识机器人以及机器人能做什么都有着深刻的影响。遗憾的是，现实中的机器人与大众的期望相距甚远。但有一件事是确信无疑的，即机器人学将是本世纪一项重要的技术。像真空吸尘机器人这类产品正开始引领智能化机器的来临，它们将逐渐走进我们的家庭和工厂。

在18世纪的时候，欧洲就对自动机非常痴迷了，如图1.1(a)所示就是沃康松(Vaucanson)的机械鸭子。这些机械装置按当时的标准来说已算相当复杂了，它们能展现一些类似仿生的行为。沃康松在他的机械鸭中就采用了一套凸轮机构来完成一系列动作，他还继续探索过将人织丝的动作机械化。贾卡德(Jacquard)受沃康松的思想启发，开发出了一台织布机，如图1.1(b)所示。这台机器可以说是真正可编程的织布机，要编织的图案是靠打孔卡片上的一系列孔洞来编码的。这台织布机有着现代机器人的很多烙印：它能执行某个实际任务，而且是可编程的。



(a) (b)

图1.1 早期的可编程机械。

(a)沃康松的鸭子(1739)是一款自动机器，它能做扇动翅膀、吃谷物和排便的动作，靠一个钟表发条驱动，但只有一个动作程序；(b)贾卡德的织布机(1801)是一款可再编程的机器，它的运动程序存储在打孔卡片上(照片由乔治·兰豆(George P. Landow)拍摄，来自www. victorianweb. org)

机器人的英文名称“robot”来自于捷克作家卡尔·查别克(Karel Capek)的科幻剧目《罗萨姆的万能机器人》(*Rossum's Universal Robots*)。剧中的机器人是一些人工制造的人形或仿人形的机械。在捷克语中，“robot”的意思是劳动者。故事的结局是机器人开始反抗人类并最终战胜了人类，这种结局还被后来很多的机器人故事借鉴。伊萨克·阿西莫夫(Isaac Asimov)则写作了一系列的机器人作品，包括从1950年至1985年出版的多篇小说和短故事。阿西莫夫的作品更多地探讨了人与机器人之间的冲突以及道德问题，在他的故事里机器人都被安装了一个“正子脑”，并在其中输入了“机器人三原则”命令。阿西莫夫的这些故事对后来的机器人科

幻小说和电影影响很大，而这些小说和电影又进一步确立了机器人在大众心目中的形象。到20世纪中期，世界上开始兴起了“控制论”这一新的学科，虽然控制论在今天仍然不为大多数人所知，但在当时却是一门相当热的关于生命理解和自动机研制的前沿科学。

第一项现代意义上的机器人专利是在1954年由乔治·C. 德维尔(George C. Devol)申请，并于1961年正式发布的。该机器人包括一个机械臂及其末端的夹持手，整个装置安装在导轨上。机器人的动作序列是靠保存在一个转动鼓轮上的磁性图进行编码的。而第一家生产机器人的公司，名为“Unimation”，就是由德维尔以及约瑟夫·恩格尔伯格(Joseph Engelberger)在1956年创建的，他们生产的第一台机器人(图1.2)于1961年正式投入使用。

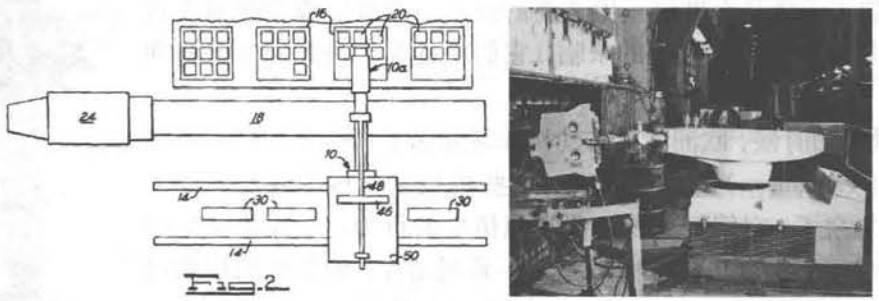


图1.2 通用自动化。(a)德维尔的专利所描述的机器平面图；(b)在通用汽车厂工作的第一台Unimation公司的机器人(照片由乔治·C. 德维尔提供)

德维尔和恩格尔伯格最初设想的机器人自动化终于实现了，数百万台如图1.3所示的机械臂式机器人被生产出来，在工厂里从事焊接、喷漆、装卸料、电子装配、打包、码垛等工作。机器人的应用直接导致生产效率的增加和产品质量的提高。在劳动力成本较高的国家，机器人还能使其制造业得以生存，而不是抢工人的饭碗。今天，我们买到的很多产品都是由机器人加工生产出来的。

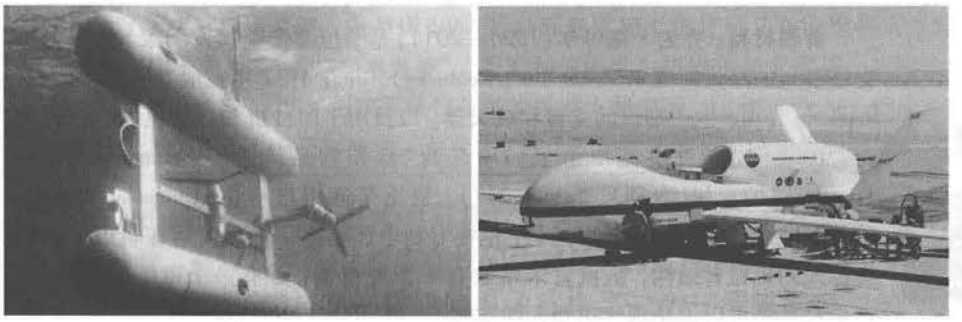
图1.3 一台现代的由ABB公司出产的六自由度机器人，它可以用于工厂自动化。这种机器人的技术实际上也是源自图1.2所示的Unimation机器人

按现在对机器人的分类，以上第一代机器人应该属于工业机器人。其他机器人分类还包括服务机器人——能提供诸如清洁、个人护理或医疗康复等服务，野外机器人——能在如图1.4所示的户外作业，以及人形机器人——具有人类的外形特征(图1.6(b))。

工业机器人最典型的就是一台安装于固定基座上的机械臂，它能在其工位上重复执行某个动作任务。被加工零件要按一定顺序呈现在机器人面前，这样就能最大限度地发挥机器人高速和高精度的特点。高速运动的机器人是非常危险的，为了安全工作人员必须要被阻隔在机器人的工作范围之外。

野外和服务机器人的开发面临着一些特殊而意义重大的挑战。第一项挑战就是机器人必须要在复杂、混乱且变化的环境中作业和移动。医院中送货的机器人必须要面对拥挤的人群以及不断改变停放位置的各种推车。火星探测机器人则要在事先没有探测环境精确地图的情

况下，在各种岩石和陨石坑之间自主导航。机器人车，如那些参加美国国防部无人驾驶车挑战赛(DARPA Grand Challenges)[Buehler et al. 2007]的汽车，必须要准确沿道路行驶，还要遵守道路上的交通信号灯及交通法规。



(a) (b)

图1.4 非陆基型移动机器人。(a)海底自主水下机器人(AUV)，来自澳大利亚野外机器人中心(罗杰·翰龙(Roger Hanlon)拍摄)；(b)全球鹰无人飞行器(UAV)(照片由NASA提供)

对于上述机器人的第二项挑战是它们不能危及在场人员的安全。如医院送货机器人需要在人群中移动和作业，机器人汽车中则搭载了乘客，而外科手术机器人则要进入人体内操作。

背景材料：《罗萨姆的万能机器人》(RUR)片段。开场时，何雷娜·葛罗丽(Helena Glory)去采访"罗萨姆万能机器人公司"的总裁哈里·多民(Harry Domin)和他的机器人秘书苏拉(Sulla)。

多民：苏拉，让葛罗丽小姐看看你。

何雷娜(站起身并伸出手)：很高兴认识你。你待在这里一定非常难，因为这儿与世隔绝。[机器人公司建在一座孤岛上]

苏拉：葛罗丽小姐，可我根本不知道外面的世界是怎么样的。请坐。

何雷娜(坐下)：你是从哪里来的？

苏拉：就来自这里，这家公司。

何雷娜：噢，你出生在这里？

苏拉：是的，我产自这里。

何雷娜(大吃一惊的样子)：什么？

多民(大笑)：苏拉不是人类，葛罗丽小姐，她是一个机器人。

何雷娜：噢，请原谅我……

该剧目的全剧可以从以下网站找到：http://ebooks.adelaide.edu.au/c/capek/karel/rur.(右侧图片来自国会图书馆，图片编号96524672)

那么到底机器人是什么呢？至今有很多关于机器人的定义，但各有侧重。对本书而言，下述定义更为恰当：

机器人是一种目标导向的具有感知、规划和运动能力的机器。

机器人感知其环境，再利用这些信息，并结合它的工作目标，去规划它的运动。这里的运动可以是移动机器人手臂上的工具去抓一个物体，或者是驱动一个移动机器人到达某个地方。

感知能力对机器人是至关重要的。本体感觉传感器可以测量机器人自身的状态，如机器人手臂上的关节转角，移动机器人上车轮转过的圈数，或一个电机转动形成的电流。体外感觉

传感器则测量机器人外部世界的状态。这种传感器可以是真空吸尘机器人上简单的触碰开关，用于检测机器人是否碰到墙面；它也可以是一个用于检测机器人到在轨卫星之间距离的 GPS 接收器，或是一个用于检测机器人朝向与地磁场方向夹角的磁罗盘。

背景材料：乔治·德维尔(1921—2011)是一位多产美国发明家，他出生在肯塔基州的路易斯维尔(Louisville，Kentucky)，并在 1932 年就成立了 United Cinephone 公司，生产唱机的支臂和放大器，以及用于印钞机和打捆机的登记管理器。1954 年，他在美国申请了一项名为“程控物品传送(Programmed Article Transfer)”的专利(专利号 2,988,237)，提出了通用自动化(Universal Automation)的概念。需要特别指出的是，他在专利中还描述了一种安装在轨道上的极坐标式的机械臂结构，机械臂末端装有一个夹持器，并有一个可编程控制器。这个机械臂可以说是现代机器人的先驱。

2011 年德维尔入选了美国“国家发明家名人堂”。(左侧照片由乔治·德维尔提供)

约瑟夫·恩格尔伯格(1925—)是一名美国工程师和企业家，常被人称作“机器人之父”。他于 1946 年和 1949 年在哥伦比亚大学分别获得学士和硕士学位。恩格尔伯格一直坚持不懈地推动机器人的发展。1966 年，他和一个 Unimation 公司的机器人一起出现在强尼·卡森(Johnny Carson)主持的“今夜秀(*The Tonight Show*)”电视节目中，机器人表演了倒啤酒、打高尔夫球、指挥乐队的动作。他还极力推动机器人在日本的发展，使得日本在机器人技术的投资和开发上都突飞猛进。另外，他还就自动化技术在太空开发中的价值在美国国会作证。恩格尔伯格出版了两本机器人专著，分别是 *Robotics in Practice*(1980)和 *Robotics in Service*(1989)，其中第一本还被翻译成了 6 种语言。

恩格尔伯格创建并担任 Unimation 机器人公司的首席执行官直到 1982 年。在 1984 年他又建立了 Transitions Research 公司，后来更名为 HelpMate 机器人公司，最后将该公司出售。恩格尔伯格现在仍然活跃在老人护理机器人的开发和推广工作中。他是美国国家工程院士，他还获得过“贝克曼奖”(Beckman Award)和日本国际奖(Japan Prize)。每年，国际机器人工业联合会(Robotics Industries Association)还会以恩格尔伯格的名义给那些“对机器人科学及应用发展做出突出贡献的人”颁奖。

这种传感器还可以是能发射声波、光波或无线电波的主动型传感器，它们通过检测信号发出与返回之间的时间差计算出前方障碍物的距离。

背景材料：控制论、人工智能和机器人学。作为一个研究领域，控制论的繁荣期从 20 世纪 30 年代持续到 60 年代，其推动力是神经病学、反馈、控制以及信息论这些学科产生的新思想和新结果的相互碰撞与交叉。神经病学的研究表明，人的大脑是一个由神经元组成的电子网络。贝尔实验室的哈罗德·布莱克(Harold Black)、亨里克·伯德(Henrik Bode)和哈里·奈奎斯特(Harry Nyquist)则在负反馈和电子网络的稳定性方面开展了富有成果的研究。克劳德·香农(Claude Shannon)的信息论描述了数值信号，而阿兰·图灵(Alan Turing)则探索了计算的基础理论。沃尔特·皮茨(Walter Pitts)和华伦·麦卡洛克(Warren Mcculloch)在 1943 年提出了人工神经元概念，并展示了这种神经元如何执行简单的逻辑运算功能。1951 年，马文·明斯基(Marvin Minsky)在他的毕业项目中用一个 B24 自驾仪和 3000 个真空管建造了随机神经模拟增强计算机(SNARC)，它可能是世界上第一台基于神经网络的学习机器。威廉·格雷·沃尔特(William Grey Walter)的机器乌龟则展示出了生物的行为。照此趋势，或许电子大脑也能被造出来。

一本非常重要的早期专著是诺伯特·维纳(Norbert Wiener)的《控制论：或在动物和机器中的控制与通信》(*Cybernetics or Control and Communication in the Animal and the Machine*)[Wiener 1965]。他提出一个控制系统的特征之一是要用到在工程和生物系统中常见的反馈。该思想后来还被应用于进化生物学、心理学以及经济学。

1956年在美国达特茅斯学院(Dartmouth College)召开了一个具有分水岭意义的会议，会议由约翰·麦卡锡(John McCarthy)主持，参会者包括明斯基、香农、赫尔伯特·西蒙(Herbert Simon)、艾伦·纽维尔(Allen Newell)及其他一些人。该会议提出并定义了我们今天熟知的人工智能(AI)的概念，强调了数值计算和符号操作，并开创了在机器人学、视觉、自然语言、语义学以及推理方面新的研究方向。随后，麦卡锡和明斯基在MIT成立了人工智能研究组。麦卡锡1962年离开麻省理工学院后又在斯坦福大学建立了人工智能实验室。明斯基则专注于人为简化的"积木世界"。西蒙和他的学生纽维尔在卡耐基·梅隆大学对于人工智能的研究卓有成效，并由此在1979年创建了该校著名的机器人研究所。上述这些人工智能研究组在机器人和计算机视觉研究开发方面都非常有影响力。至今，专注于控制论的学会团体和出版物依然十分活跃。

摄像机是一种被动捕获外界环境反射光线的传感器。根据我们自己的感受，眼睛在用于识别、导航、避障及操作时是最有效的传感器，因此视觉对于机器人研究人员来说一直备受关注。图1.5展示了怎样从一幅原始图像重构出一个三维线框模型的早期研究工作，也体现了其中涉及的难点问题。仅用一个相机的突出问题是三维的结构只能从二维图像推测出来。三维重构的另一个方法是采用立体视觉，即用两个或多个相机的图像去计算外界的三维结构。图1.6(a)所示的火星探测机器人就在它的桅杆上安装了一台立体相机。

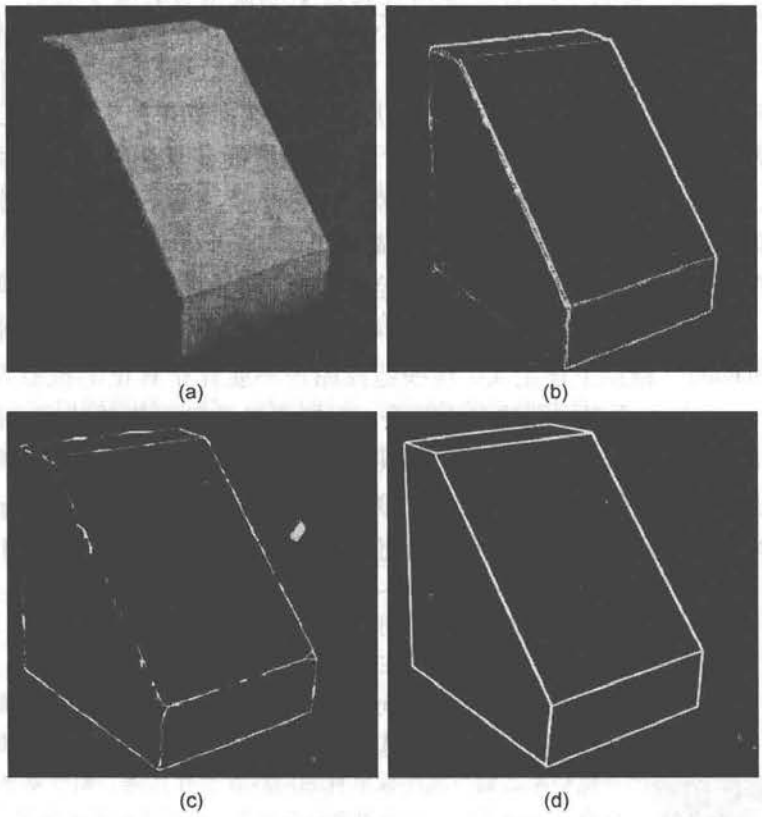
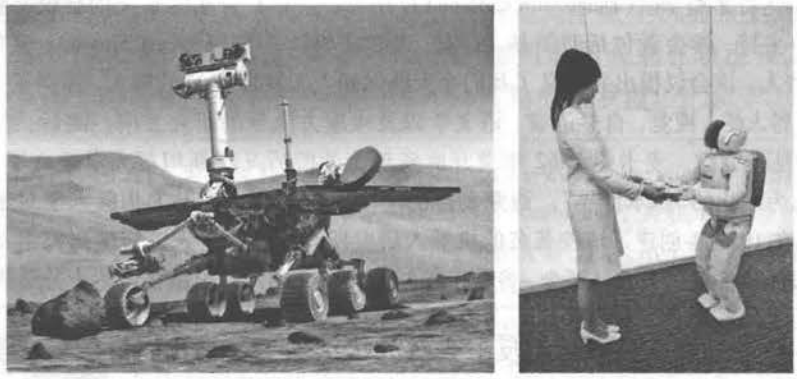


图1.5 早期用计算机图像估算物体形状和姿态的结果，来自1963年罗伯茨(L. G. Roberts)在MIT林肯实验室(Lincoln Lab)的博士论文研究[Roberts 1963]。(a)原始照片；(b)梯度图像；(c)连贯的特征点；(d)重构线图

本书将专注于用相机作为机器人的传感器。在本书第五部分的机器视觉中，将讲述用计算机去处理来自一台或多台相机的图像，并提取其数值特征。例如，如何确定场景中一个红色圆形物体的坐标，或者根据外界相对于机器人的移动表象来确定真正机器人移动的距离。

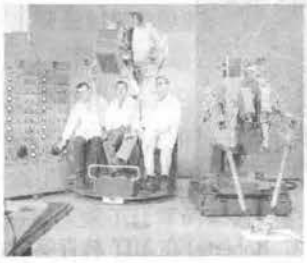


(a) (b)

图 1.6 两种不同类型的移动机器人。(a)火星探测器。注意其桅杆上的两个摄像头，它们能提供立体视觉，使机器人能计算出其环境的三维结构(图片由NASA/喷气推进实验室(JPL)/康奈尔大学提供)；(b)本田公司的阿西莫(Asimo)人形机器人(照片由日本本田公司提供)

如果机器人所处的环境根本没有变化，那机器人完全可以依靠一张精确的环境地图去完成作业，它仅需确定自己在地图中的位置，而不需要去感知外界的状态。可以想象一下你驾驶一辆前挡风玻璃完全被遮挡住的车，而仅依靠 GPS 系统进行导航。如果路上仅有你一辆车，你或许能非常成功地从 A 点开到 B 点，即使速度很慢。然而如果路上还存在其他车辆、行人、信号灯或施工等复杂状况时，你可能就寸步难行了。为了能在复杂的道路环境中驾驶，你必须观察车外的情况，即感知外界并采取相应的行动。对于人类来说，上述行为非常简单，甚至不假思索就能做，但想要通过编程让一个机器去做就很难了。

遥操作机器人是依靠人类操作员远距离遥控的像机器人的机器。或许最早的遥操作机器人是 1898 年尼古拉·特斯拉(Nikola Tesla)展示的一款无线电遥控船，特斯拉也将它称作“远程自动装置”(teleautomaton)。根据上述定义，这些遥控船还不能算是真正的机器人，但它们是机器人的重要祖先之一，而且至今对于那些人不能去，而机器自身又不能单独完成的很多复杂任务，它们仍然很重要[Goldberg and Siegwart 2001; Goldberg 2001]。例如，在海底探测泰坦尼克号残骸的水下机器人就是远程遥操作航行器(ROV)。美国的火星探测器“勇气号”(Spirit)和“机遇号”(Opportunity)虽然能在火星表面自主导航，但还是需要人类操作员向它们发出高级目标指令。



背景材料：二战期间的“曼哈顿项目”(The Manhattan Project)制造出了世界上第一个核武器，在研制过程中需要处理放射性材料。美国阿尔贡国家实验室的雷·戈尔茨(Ray Goertz)开发了一个远程遥控机械臂，它既能重复操作人员灵活的动作，又让他们远离要处理的危险材料。使用时，操作员通过一幅很厚的铅玻璃窗或电视画面观看操作任务。对于那些人不能去，而机器自身又不能单独完成的很多复杂任务，遥操作机器人仍然十分重要。例如，在海底探测泰坦尼克号残骸的水下机器人。(左侧照片由阿尔贡国家实验室提供)

也就是说，操作员告诉机器人去到哪里，然后机器人自己决定具体的行进路线。在火星上机器人自主决策是必要的，因为火星到地球的通信延时长达几分钟。有些机器人的控制是混合式的，即控制任务要与人类操作员分享或交换。在交换控制中，控制权要在人类操作员和计算机之间来回切换。例如，飞行员可以将飞机的控制权交给自动驾驶仪，也可以拿回来。在分

享控制中，控制任务是由人类操作员和计算机共同执行的。例如，自主行驶客车可以由计算机来使车保持在道路中间并躲避碰撞，同时由操作员来控制车速。

背景材料： Unimation 公司(1956—1982)。德维尔在寻找资金开发他的通用自动化技术，并最终遇到了当时还在曼宁、麦斯威尔和穆尔公司(Manning, Maxwell and Moore)当工程师的约瑟夫·恩格尔伯格。1956 年他们在康涅狄克州的丹伯里(Danbury)共同创立了 Unimation——世界上第一家机器人公司。该公司后来被美国统一柴油机公司(Consolidated Diesel Corp.)收购，成为其子公司，更名为 Unimate 公司。公司的第一台机器人于 1961 年进入了通用汽车公司在新泽西的压铸车间。1968 年，他们将专利转让给了日本的川崎重工(Kawasaki Heavy Industries)，该公司生产出了日本第一台工业机器人。恩格尔伯格一直担任公司的首席执行官，直到 1982 年公司被西屋公司收购。从这家公司出来的人和技术对世界机器人技术的发展都有非常大的影响。

1.1 关于本书

本书是关于机器人和计算机视觉两种技术的，既有它们单独的论述，也有组合起来作为机器人视觉的阐述。机器人和计算机视觉各自都是内容丰富的大话题，组合之后涵盖的知识必然很广。本书的目的不是要蜻蜓点水地介绍一下，而是要让读者深刻体会到机器人加上视觉后是什么？以及它还能做什么？尽管内容会很多。

本书目的如下：

- 通过理论讲解和举例为读者打下宽广而坚实的知识理解基础；
- 通过强有力的数值工具及其基础软件使读者能够解决更复杂的问题，而不仅仅是作为一般教科书；
- 通过用相对少的代码去解决复杂问题，给读者提供及时的体验和满足；
- 对已有的很多关于机器人和计算机视觉的教科书，可以作为它们的补充；
- 通过亲自动手开展数值实验，可以提升读者的直觉思维能力；
- 尽量减少方程式的数量，把它们放在需要增加价值和清晰度的地方(我的个人观点)。

本书的写作是将背景知识、理论及例子以一种组合方式呈现出来。程序代码和例子在书中是首要的，因此它们直接放在正文中，而不是附在每章的末尾或链接到某个网站。举例是和问题讨论交织在一起的，如：

```
>> Ts = ctraj(T1, T2, t);
>> p = transl(Ts);
>> plot(t, p);
```

其中 MATLAB 的代码阐明了要讨论的话题，而且通常会得到一条曲线或一个较生硬的数值结果，然后再对这些结果进行讨论。所举的例子是为了展示如何使用相关的 MATLAB 工具箱，这些知识随后便可应用于其他问题。而且本书中大多数曲线也都是通过例子的代码直接画出的。因此，从某种意义上说本书就是一个大的 MATLAB 脚本。

1.1.1 MATLAB 软件

工欲善其事，必先利其器。

——中国谚语

本书的计算基础是 MATLAB 软件——一款 Mathworks 公司的产品。MATLAB 是一种交互

式的数学软件编程环境，能较容易地完成线性代数计算、数据分析以及高质量图形绘制。MATLAB 也是一款流行的软件包，多数学生和研究人员对它都很熟悉。该软件还支持一种特有的编程语言，能够编写出复杂的算法。

MATLAB 的另一特长是支持工具箱，工具箱中集合了针对某一特定题目的函数。工具箱可以从 MathWorks 公司、第三方公司以及私人处获得。有的工具箱是需要付钱的商品，有的则是开源免费的。本书内容是基于作者开发的两个开源的工具箱，即机器人工具箱和机器视觉工具箱。应用这两个 MATLAB 工具箱就能将一台现代个人计算机变成为一个功能强大和使用方便的环境，用以探索机器人、机器视觉以及视觉控制中的复杂问题。这两个工具箱都是免费的，通过 GNU 宽通用公共许可证(GNU LGPL)发布。

机器人工具箱(RTB)中包含了用于模拟移动机器人和机械臂的各种不同的函数。最初的机器人工具箱可以追溯到 20 世纪 90 年代早期，那时只针对机械臂式的机器人，而且只支持一种非常普通的用矩阵以及后来的 MATLAB 对象来表示串联机械臂结构的方法。在这个工具箱中，可以创建任意的串联式机械臂，并提供了计算机械臂正向和逆向运动学及动力学的函数。工具箱中还包含了对诸如向量的数据类型进行操作及转换的函数、齐次变换函数、三角度表示函数，以及单位四元数函数，它们对于描述刚体三维位置和姿态非常有用。

本书中采用的机器人工具箱版本增加了模拟移动机器人的新功能。目前的机器人工具箱包含了四轮移动机器人和四旋翼直升机的模型，以及上述机器人的控制器。它还提供了用于机器人路径规划、定位和地图绘制的标准算法。

背景材料： 今天我们使用的 MATLAB 软件已有很长的历史了。它最早始于 20 世纪 70 年代在阿尔贡实验室开展的 LINPACK 和 EISPACK 两个项目，其目的是开发出一款高质量、可靠及便携的数学软件。LINPACK 中包含了有关线性代数方程求解的程序集，而 EISPACK 则是用于计算矩阵特征值和特征向量的数值算法库。以上两个程序包都是用 FORTRAN 语言编写的，这种语言在当时，甚至在今天都是编写大规模数值计算程序的首选。

当时还在新墨西哥大学的克里夫·莫勒尔(Cleve Moler)参与了以上两个项目的工作，并于 20 世纪 70 年代后期编写出了第一个版本的 MATLAB。用这个软件能方便地调用 LINPACK 和 EISPACK 程序库去求解问题，而不用再亲自去编写和编译 FORTRAN 代码。很快，MATLAB 被传播到其他大学，并在应用数学以及工程界获得了很多忠实的用户。1984 年，克里夫·莫勒尔和杰克·里特(Jack Little)一起成立了 MathWorks 公司，公司主要使用新发布的 IBM PC——世界上第一台广泛使用的台式计算机。

克里夫·莫勒尔于 1961 年在加州理工大学获得学士学位，并在斯坦福大学获得博士学位。他在多所大学担任过数学和计算机科学的教授，包括密歇根大学、斯坦福大学和新墨西哥大学。他还长期担任美国工业及应用数学学会(Society for Industrial and Applied Mathematics，SIAM)的主席，并在 1997 年当选为美国国家工程院院士。更多资料请参见 http://www.mathworks.com/company/aboutus/founders/clevemoler.html，其中有克里夫·莫勒尔的一个视频，还有 http://history.siam.org/pdfs2/Moler_final.pdf。

机器视觉工具箱(MVTB)则提供了非常丰富的函数，用于相机建模、图像处理、图像特征提取、多视像几何以及基于视觉的控制。虽然机器视觉工具比机器人工具箱出来得晚，但它绝不是 MATLAB 自带的视觉处理工具箱(Image Processing Toolbox，IPT)的翻版。虽然有一些功能是相同的，但机器视觉工具箱的问世要比视觉处理工具箱早好几年[①]。机器视觉工具箱中包

---

① 这些函数被称为 MEX 文件，它们是用 C 语言编写的，但编写方式特殊，这样 MATLAB 就能像调用 M 语言函数一样地调用它们。

含了很多函数，包括图像获取与显示，图像信号过滤，图像中团、点、线的特征提取，数学形态学，图像卷绕，立体视觉，单应矩阵与基本矩阵估计，鲁棒性估计，视觉雅克比矩阵，几何相机建模，相机标定，以及彩色空间运算。对于现代计算机而言，处理一幅大小适中的图像完全能达到“实时”水平，从而允许闭环控制。

本书介绍的两个工具箱都以源代码形式提供。程序的主体是用 MATLAB 的 M 语言编写的，只有一些函数为了增加计算效率使用了 C 语言。总体而言，工具箱程序是以一种平铺直叙的方式编写的，这样更便于读者理解，即便可能会损失一点儿计算效率。如果你是才开始研究机器人或机器视觉，那这两个工具箱将是你最好的编程启蒙，在此基础上你就可以创建自己的项目。

本书虽然提供了很多例程来展示如何使用工具箱的函数去解决某些特定的问题，但它并非一本函数参考手册。想要深入了解所有工具箱函数，可以通过 MATLAB 的内建帮助机制获得详细的描述文档。这种方式可以让程序代码不断进化并获得新的特点，同时还能保持向前的兼容性，也不致使本书太快就过时。本书附录 A 提供了如何获得工具箱的详细方法，以及一些网上群聊资源。

1.1.2 读者对象及必备知识

本书主要面向大学本科三、四年级的学生，以及一年级的研究生。对于本科生，本书可以作为机器人或机器视觉课程的辅助教材，或者是在学习这两门课程中帮助学生完成一个大课题。学生们首先应该学习本书第一部分及附录，以便掌握基础的概念，然后再去学习其他想学的部分：移动机器人，机器人机械臂，计算机视觉，以及基于视觉的控制。工具箱为解决相关问题提供了一套实用的工具，而每章最后的练习则在书中给出的例子之外为读者提供了更多的问题。

对那些专业是机器人技术的一年级研究生，如果本科已学过了工程和计算机科学的课程，本书则可以帮助他们在本科学过的知识与以后开展机器人和机器视觉深入研究需要的知识之间架起一座桥梁。本书的工作代码库可以帮助开展他们的研究，能使他们在探索自己的问题时更快进入角色，工作更有成效。由于能得到工具箱程序的源代码，他们就可以对它进行修改以满足自己的需要，并且当到了需要用其他语言编写他们自己的算法时(这个阶段迟早会来临)，书中的工具箱还能用于对他们的程序进行检查核对。

对那些已不是学生的研究人员或公司技术员，本书可以作为研究机器人和机器视觉中某项技术的有用参考书，也可以作为他们使用相关工具箱的指导手册。

学习及使用本书需要以下本科水平的前期知识：线性代数(矩阵、向量、特征值)，基础集合论，基础图论，概率论，动力学(力、力矩、惯量)，拉普拉斯变换和传递函数，线性控制(比例控制、比例-微分控制、比例-积分控制)及控制框图绘制。专业是计算机科学的学生很可能没有学过拉普拉斯变换以及经典控制方法，不过这些知识仅会出现在本书的 9.4 节中，而且如果需要还可以参考赫勒斯坦(Hellerstein)等人写的书[Hellerstein et al. 2004]。本书还要求读者比较熟练地掌握 MATLAB 编程，以及面向对象的编程技术(如C++，Java 或 Python)。另外，虽然不是必须，但如果对 MATLAB 的图形化框图建模工具 Simulink 很熟悉，也将对本书的学习有所帮助。附录中还提供了上述很多知识的复习材料。

1.1.3 符号与约定

由于本书涉及的内容太广，常用符号显然不够用，因此难免有些符号在本书中的不同地方会有不同的含义。

本书中还有很多 MATLAB 代码，它们是用等宽字体呈现的，如：

```
>> a = 2 + 2
a =
     4
```

MATLAB 的命令提示符是 ≫，紧跟的就是用户发给 MATLAB 的命令。下面几行没有提示符的就是 MATLAB 的执行结果。

书中文字及代码里的所有函数、类型和方法都被交叉引用，而且在书的最后还有各自的索引。书中所有的 MATLAB 代码段还可以从附录 A 介绍的本书网页中获得。虽然本书展示了工具箱中很多函数的用法，但它并不是专门的编程手册，所有工具箱函数最权威的信息可以从帮助文档获得。

本书使用彩色框[①]表示不同类型的材料。橙色信息框突显出其中的材料特别重要，而橙红色警示框则提醒那些新手可能出现的陷阱。蓝色框中提供了有关技术、历史和个人背景的信息，这些信息只是正文内容的补充，但对正文理解不起关键作用。

对于作者来说，很难把握书籍在完整性、清晰性和简洁性之间的恰当分寸。因此，本书的很多细节都被放到了注释及蓝色框中，读者在第一次阅读时可以忽略它们。然而，如果想要深入理解一个特殊的算法并应用到自己的程序中，那么了解那些细节和微小差异就非常重要了，这时就需要阅读注释中的内容[②]。

本书每章结束都有名为“总结”的一节，它首先总结了本章重要内容，然后对扩展阅读资料进行了讨论，最后提供了一些练习题。每章正文中会尽量少地对参考资料进行引用，而是把前沿工作和参考资料的讨论放在了“扩展阅读”部分，对相关算法提供了更严谨或更完整的描述。“练习题”部分实际上是对每章所讨论问题的延展，一般来说与这一章中探讨的某一特殊程序有关。练习题会由简到难，既有示例程序的直接延伸，也有一些挑战性的问题。

1.1.4 怎样使用本书

最好的学习方法是实践。虽然本书中给出了一些 MATLAB 的命令及其响应，但自己去实践和体会还是非常有必要的。当你自己去运行这些命令时，可以采用你喜欢的格式观看结果，用不同的方式绘制结果曲线，或是用不同的数据或参数去试验某个算法。你也可以去查看工具箱函数的帮助文档，发掘它们其他的特点和选项，然后自己去试验；或者是先读这些函数代码，明白它们如何工作，然后尝试去修改它们。

大多数 MATLAB 命令都很简短，在键盘上输入它们并不费劲。而在本书网站中(见附录 A)还列出了本书所有的 MATLAB 命令(多达 1600 行)，你也可以从中剪切并粘贴到 MATLAB 界面，或者下载这些命令然后用来创建你自己的程序。

---

① 本译本表示形式：橙色信息框中文了用楷体表示；橙红色警示框用楷体字并加两细分割线表示；蓝色框为“背景材料”，用小号字及细线分割线表示。——编者注

② 它们是以旁注的形式放在相应标记的一侧。本译本改用脚注的形式。

1.1.5 使用本书教学

本书可以用于机器人技术、机电一体化以及计算机视觉几门课程的辅助教学。这几门课都应包括对坐标系及组成方式的介绍，这部分内容在本书第2章介绍。如果只是移动机器人或图像处理技术的课程，只教授二维的知识就够了。但如果是通用机器人或多视图几何课程，则二维和三维的知识都需要讲授。本书中大多数图片(MATLAB生成的图以及线条图)都可以从本书网站中获得它们的PDF格式文件，而且可以免费使用。本书所有的程序代码也可以从本书网站下载，可以在讲课或辅导中用作演示内容的基础。

每章后的练习题既可以用作课后作业，也可以当作讲课或辅导中的问题举例。练习中的多数问题都是开放性的，要鼓励学生去探索和发现算法中参数的影响以及性能的局限性。探索时要对性能指标和什么算法最优展开讨论或辩论。另外，真正要理解一个算法还必须对其中参数的影响有一个正确的认识，以及什么情况下算法会失效。

本书内容的讲授顺序可以调整，甚至颠倒。例如，可以首先提出一个特定问题，然后再讲授解决这个问题的必要知识。特定问题可以从7.6节、14.5节~14.8节、16.6节挑选，也可以从练习题中选择。那些特别具有挑战性的练习题已被标示出来。

对于研究生，本书“扩展阅读”中提到的论文和专著可以作为布置给他们的阅读作业的基本材料，这些材料也可以作为阅读小组或期刊俱乐部的研讨对象。

1.1.6 本书梗概

虽然我承诺的是一本能即刻体验的书，但在开始体验之前，有些关于机器人的基本概念是我们必须要先掌握的，而且要掌握好。本书的第一部分介绍了有关位姿与坐标系的概念，即如何描述一个机器人及其操作对象的位置和姿态。我们讨论了如何将两个位姿之间的运动分解为基本的平移与旋转运动，以及如何将基本运动组合成更复杂的运动。第2章讨论了物体位姿如何在计算机中进行描述。第3章则讨论了如何生成一系列机器人位姿，使它在空间和时间上能平滑地沿某一路径运动，该章还讨论了速度与位姿微分之间的关系。

有了上述这些基础知识，就可以去研究我们的第一个议题——机器人。机器人可以分为两大类：移动机器人和机器人机械臂。这两类机器人将分别在本书第二部分和第三部分进行研讨。

第二部分从第4章开始，首先讨论了移动机器人的机动性，介绍了欠驱动、非完整性等概念，然后探讨了机器人小车和四旋翼飞行器两种移动机器人载体的运动模型。该章还讨论了针对机器人小车的各种控制方法，如移动到一个点、沿某条路径移动以及运动到一个特定位姿。第5章的内容主要是关于机器人的导航，即机器人如何在A到B的两点间找出一条移动路径。对导航问题的研究涉及了两种情况，即有地图和没有地图。大多数导航技术都需要知道机器人的位置信息，因此第6章讨论了获取机器人位置信息的各种方法，包括轨迹推算、路标观察以及使用地图。我们还展示了机器人如何绘制环境地图，甚至在未知环境中绘制地图的同时还能确定自己的位置。

第三部分涉及的是机器人机械臂，更准确地说是串联式机械臂。机械臂主要应用于装配、焊接、材料处理、甚至外科手术等任务中。第7章讨论了机械臂的运动学问题，主要涉及机械臂的关节角度与末端工具三维位姿之间的关系。该章还讨论了为末端工具生成光滑路径的技术，并给出了两个实例，一个是一台机械臂在平面上写一个字母，另一个是多个机械臂被用作

一台简单步行机器人的腿。第 8 章讨论了关节角度变化率与末端工具位姿变化率之间的关系，其中介绍了雅克比矩阵，以及奇异位姿、可操作性、零空间运动、旋转速率运动控制等概念。该章还讨论了机器人的欠驱动和过驱动，以及机器人逆运动学的一般数值解法。第 9 章介绍了串联式机械臂的动力学方程，以及机械臂关节力与关节运动之间的关系。该章还讨论了关节控制系统的设计，涵盖了诸如惯性与负载变化、柔性传动、独立关节控制以及非线性控制等重要议题。

计算机视觉是一个较大的研究领域，主要议题是怎样通过图像处理来使得图像效果得以增强，解读出图像场景中的内容，以及重构出场景的 3D 模型。本书第四部分研讨的就是机器视觉，它属于计算机视觉研究的一个分支。本书对机器视觉的定义是“从图像中提取出数值特征，并用作机器人控制的输入信息”。第四部分内容从第 10 章开始，讨论了机器视觉的一些基础知识，如光源、照明及色彩。第 11 章介绍了使用镜头创建透视图像的几何模型，并讨论了诸如相机标定、位姿估算等议题。该章还介绍了使用广角镜头及反射镜系统产生非透视图像，以及与透视图像的关系。第 12 章讨论了图像处理技术，但仅限二维信息处理，即将一幅图像变换成另一幅图像。讨论的内容从真实世界图像获取开始，然后介绍了用于图像处理的各种算术和逻辑运算。随后该章介绍了一些空间算子，如互相关、分割、形态滤波，以及改变图像的形状和大小。上述运算或算子都是下一章内容讨论的基础。第 13 章介绍了如何从图像中提取数值特征，这些特征包括一个场景中均质的区域、线条或者特殊点，它们是基于视觉的机器人控制的基础。第 14 章讨论了如何从一个场景不同视角的图像特征中得到隐藏其中的三维几何信息，以及多相机视角间的空间关系。

本书第五部分讨论了如何使用从相机图像中提取的特征信息区控制机械臂以及移动机器人，即所谓的基于视觉的控制或视觉伺服。这部分综合了前面各章介绍的概念和知识。第 15 章介绍了经典的视觉伺服方法，即基于位置和基于图像的视觉伺服，并讨论了它们各自的局限性。针对经典方法的局限，第 16 章介绍了一些更现代的方法，并涉及非透视相机、欠驱动机械臂和移动机器人的应用。

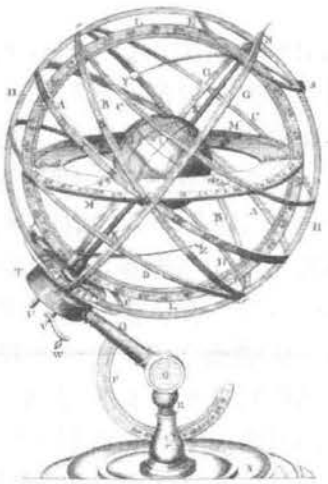
本书虽然涉及内容很广，但其中任何一部分都可以单独挑出来学习，只需偶尔参看一下前面章节中必要的知识。第 2 章是全书唯一的必学内容。而第二、三、四部分则分别讲述了移动机器人、机器人机械臂和机器视觉的知识，它们之间的关联性不是很强，可以单独学习。当然，为了凸显及时体验的宗旨，也可以直接跳到某一章开始学习体验，必要时再翻看前面的内容。

第一部分

基础知识

第2章　位置与姿态描述
第3章　时间与运动

第2章　位置与姿态描述



机器人和计算机视觉中的一个基本要求是能够表示物体在环境中的位置和方向。这些物体包括机器人、摄像机、工件、障碍物和路径。

空间中的点是数学中一个熟悉的概念，它可以被描述为一个坐标向量，也被称为一个约束向量，如图2.1(a)所示。向量表示点相对于某个参考坐标系的位移。一个坐标系或笛卡儿坐标系统，是由一组正交轴构成的，这些轴相交于一个被称为原点的点。

更多时候我们需要考虑组成物体的一组点。我们认为物体是刚性的，构成它的点相对于物体坐标系保持固定的相对位置，如图2.1(b)所示。然而我们表示物体位置和方向时并不是描述其上单独的点，而是用该物体坐标系的位置和方向来描述。坐标系有自己的标记，比如这里的$\{B\}$，其坐标轴x_B和y_B采用坐标系的标记作为其下标。

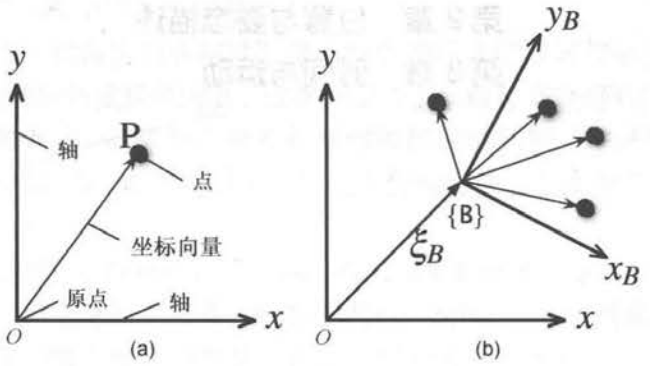


图2.1　(a)点**P**由一个相对于绝对坐标系的坐标向量表示。(b)这些点是用相对于物体坐标系$\{B\}$的向量表示，该物体坐标系又由一个相对位姿ξ_B表示。坐标轴由带有开放式箭头的粗线表示，向量由一条带有斜箭头的细线表示，而一条末端为实心箭头的粗线表示一个位姿

坐标系的位置和方向总称为位姿，图形上表示为一组坐标轴。相对于一个参考坐标系的某个坐标系的相对位姿用符号ξ表示。图2.2显示了两个坐标系$\{A\}$、$\{B\}$，以及$\{B\}$相对于$\{A\}$的相对位姿${}^A\xi_B$。前面的上标表示参考坐标系，下标表示被描述的坐标系。我们也可以认为${}^A\xi_B$描述了一组动作——对$\{A\}$施加平移和旋转使它转化为$\{B\}$。如果没有初始上标，我们默认位姿的变化是相对于用O表示的世界坐标系的。

在图2.2中的**P**点可用任何一个坐标系表示。用式子表示为

$$ {}^A\boldsymbol{p} = {}^A\xi_B \cdot {}^B\boldsymbol{p} \tag{2.1} $$

等式右侧表示从$\{A\}$到$\{B\}$然后到**P**的动作。运算符“·”将一个向量转换为一个新的向量，它们是用一个不同的坐标系来描述的相同点。

相对位姿一个重要的特点是它们可以被合成或组合。以图 2.3 所示情况为例，如果一个坐标系可以被其他坐标系用相对位姿描述，那么它们的关系可以记为

$$^{A}\xi_{C} = {}^{A}\xi_{B} \oplus {}^{B}\xi_{C}$$

用语言可以表述为，$\{C\}$ 相对于 $\{A\}$ 的位姿可由 $\{B\}$ 相对于 $\{A\}$ 的位姿和 $\{C\}$ 相对于 $\{B\}$ 的位姿合成得到。我们利用运算符“⊕”表示相对位姿的合成。

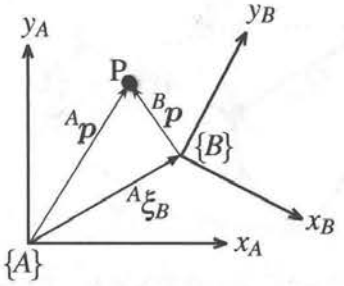


图 2.2　点 **P** 既可以用相对于坐标系 $\{A\}$ 的坐标向量表示，又可以用相对于坐标系 $\{B\}$ 的坐标向量表示。坐标系 $\{B\}$ 相对于坐标系 $\{A\}$ 的位姿记作 $^{A}\xi_{B}$

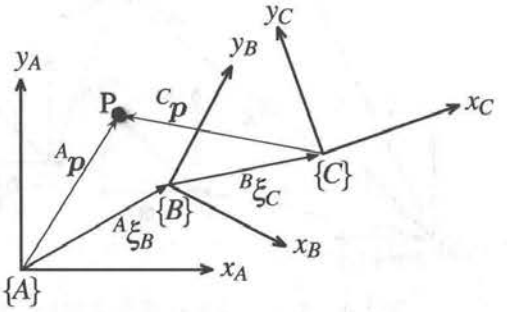


图 2.3　点 **P** 可以用相对于坐标系 $\{A\}$，$\{B\}$ 或 $\{C\}$ 的坐标向量表示。这些坐标系用相对位姿描述

在这种情况下，点 **P** 可以被表示为

$$^{A}\boldsymbol{p} = (^{A}\xi_{B} \oplus {}^{B}\xi_{C}) \cdot {}^{C}\boldsymbol{p}$$

在本章后面，我们会将这几个抽象的符号“ξ”、“·”和“⊕”，转换成可以在 MATLAB 中实现的标准的数学对象和运算符。

在前面的例子中主要讨论了二维坐标系的情况，能适用于一大类机器人的问题，特别是在平面世界里作业的移动机器人。对于其他问题，我们需要用三维坐标系来表示三维世界中的物体，如飞行机器人、水下机器人的位姿或者机器人手臂夹持的工具末端。

背景材料：在相对位姿的合成中，我们可以通过确认运算符⊕两侧的上标和下标字母匹配来检验相对坐标系是否正确。我们可以去掉中间的上、下标字母：

$$^{X}\xi_{Z} = {}^{\textcircled{X}}\xi_{\textcircled{X}} \oplus {}^{\textcircled{Y}}\xi_{\textcircled{Z}}$$

只留下左右两头加圈显示的上、下标字母。

图 2.4 中给出了一个更复杂的三维变换的例子，其中每个实体上都固接了一个三维坐标系，并表示出了它们之间的相对位姿。固定摄像头从固定的视角观察物体，并估计物体相对于摄像头本身的位姿。另一个摄像头是不固定的，它固接到机器人上，与机器人有固定的相对位姿，并估计出物体相对于该摄像头的位姿。

另一个表示空间关系的方式是有向图（见附录 J），如图 2.5 所示。图中的每个节点代表一个位姿，每条边代表一个相对位姿。从 X 到 Y 的箭头记作 $^{X}\xi_{Y}$，表示 Y 相对于 X 的位姿。前面我们提到过可以用运算符⊕来复合成相对位姿，这里可以用它写出一些空间关系：

$$\xi_{F} \oplus {}^{F}\xi_{B} = \xi_{R} \oplus {}^{R}\xi_{C} \oplus {}^{C}\xi_{B}$$
$$\xi_{F} \oplus {}^{F}\xi_{R} = {}^{0}\xi_{R}$$

每个方程表示了图中的一个闭环①。方程等号两侧的每一边各表示一条网络的通路，即一组按照从头到尾顺序连接的边(箭头线)。等式两边的起始节点和结束节点必须相同。

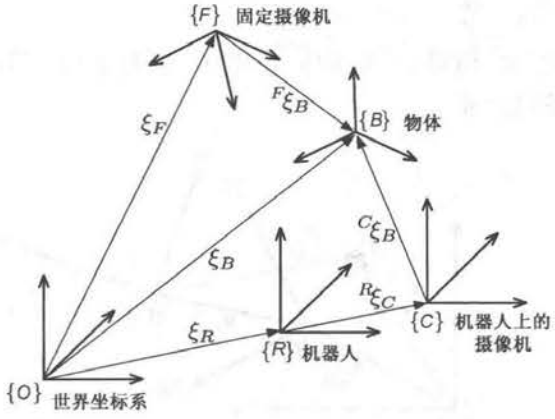


图 2.4　多个三维坐标系及其相对位姿

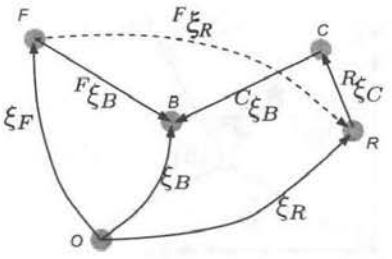


图 2.5　图 2.4 的空间示例被表示为一个有向图

位姿表示的一个非常有用的属性是其代数运算能力。以上面第二个闭环方程为例，它表示机器人的位姿等同于两个相对位姿的合成：从世界坐标系到固定摄像机的相对位姿，以及从固定摄像机到机器人的相对位姿。

背景材料：勒内·笛卡儿(René Descartes，1596—1650)是一个法国哲学家、数学家及兼职雇佣兵。

他因哲学论断“我思，故我在”，或“我思考，所以我存在”，或“我想，因此我在”而著名。他是一个多病的孩子，因而养成了一个伴随终身的习惯：躺在床上思考直到上午。有一个可能是杜撰的故事，讲的是在这样一个早晨，他看见一只苍蝇爬过天花板，这时他意识到可以用它与天花板两条边的距离来描述它的位置。这个坐标系，即笛卡儿坐标系，构成了现代(分析)几何的基础，并影响了现代微积分的发展。在瑞典时，由于女王克里斯汀的邀请，他不得不早上5点起床，这打破了他的终身习惯——因此得了肺炎而死。他的遗体后来被运回巴黎，然后又搬动了好几次，现在还有一些关于他的遗体存放位置的争论。他死后，罗马天主教会曾把他的作品列入违禁书籍。

我们可以通过在等式两边同时加上一个逆 ξ_F 来消除 ξ_F，记作

$$\ominus\xi_F \oplus \xi_F \oplus {}^F\xi_R = \ominus\xi_F \oplus \xi_R$$
$$ {}^F\xi_R = \ominus\xi_F \oplus \xi_R$$

其结果就是机器人相对于固定摄像机的位姿。

下面是几个代数运算规则：

$$\xi \oplus 0 = \xi,\ \xi \ominus 0 = \xi$$
$$\xi \ominus \xi = 0,\ \ominus\xi \oplus \xi = 0$$

① 在数学对象术语中位姿组构成了一个群，即一系列对象的集合，它支持一种运算结果仍属于该群的关联二元运算符(复合算子)，一种求逆运算和一种单位元素。在这里，该群是二维或三维的特殊欧几里得群，分别记作 $SE(2)$ 或 $SE(3)$。

其中 0 表示一个零相对位姿。一个位姿可以有逆位姿：

$$\ominus {}^{X}\xi_{Y} = {}^{Y}\xi_{X}$$

该逆位姿在图中可表示为从 Y 到 X 的一条箭头线。相对位姿可以被复合或合成：

$${}^{X}\xi_{Y} \oplus {}^{Y}\xi_{Z} = {}^{X}\xi_{Z}$$

需要注意的是位姿的代数运算规则不同于一般的代数运算规则，位姿复合运算中各个项的位置不能交换：

$$\xi_1 \oplus \xi_2 \neq \xi_2 \oplus \xi_1$$

$\xi_1 \ominus \xi_2 = 0$ 时才可以互换位置。可以用相对位姿将一个点从一个坐标系中的一个向量转换为另一个坐标系中的另一个向量：

$${}^{X}\boldsymbol{p} = {}^{X}\xi_{Y} \cdot {}^{Y}\boldsymbol{p}$$

那么，什么是 ξ？它可以是支持上述代数运算的任何数学对象，并适用于我们身边常见的问题。它还取决于我们讨论的是二维还是三维的问题。在本章接下来的内容中我们将讨论一些对象，其中包括向量以及一些更奇特的数学对象，如齐次变换矩阵、正交旋转矩阵和四元数等。幸运的是，所有这些数学对象都是非常适合 MATLAB 的数学编程环境。

简要回顾一下前面的内容：

1. 一个点用坐标向量来表示，它代表该点在参考坐标系中的位移；
2. 一个刚体可以由其上的一组点代表，该刚体可以用单独一个坐标系描述，并且组成它的点可以用它们在该坐标系中的位移来表示；
3. 一个物体在坐标系中的位置和方向称为它的位姿；
4. 一个相对位姿表示一个坐标系相对于另一个坐标系的位姿，记作代数变量 ξ；
5. 一个点可以用不同坐标系中的不同坐标向量来描述，向量之间通过坐标系相对位姿来转换，其运算符为“ · ”；
6. 用相对位姿写成的代数表达式是可以进行代数运算的。

本章的剩余部分将讨论在机器人和计算机视觉中常见的各种 ξ 的具体表现。

2.1　二维空间位姿描述

二维世界或平面，是我们在高中学习欧几里得几何时就熟悉的。笛卡儿坐标系，或以 x 轴和 y 轴为正交轴的坐标系，通常绘制成 x 轴水平、y 轴竖直，两轴的交点称为原点。平行于坐标轴的单位向量用 $\hat{\boldsymbol{x}}$ 和 $\hat{\boldsymbol{y}}$ 表示。一个点用其在 x 轴和 y 轴上的坐标 (x, y) 表示，或者写为有界向量：

$$\boldsymbol{p} = x\hat{\boldsymbol{x}} + y\hat{\boldsymbol{y}} \tag{2.2}$$

在图 2.6 中的一个坐标系 $\{B\}$，我们希望用参照系 $\{A\}$ 来描述它。可以清楚地看到，$\{B\}$ 的原点已被向量 $\boldsymbol{t} = (x, y)$ 所取代，然后逆时针旋转一个角度 θ。因此，位姿的一个具体表示就是三维向量 ${}^{A}\xi_{B} \sim (x, y, \theta)$，我们使用符号 ~ 表示这两种表

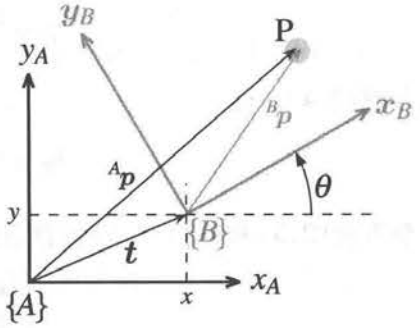


图 2.6　两个二维坐标系 $\{A\}$、$\{B\}$，以及一个点 $\mathbf{P}$。$\{B\}$ 相对于 $\{A\}$ 做旋转和平移

示是等价的。遗憾的是，这种表示方法不方便复合，因为

$$(x_1, y_1, \theta_1) \oplus (x_2, y_2, \theta_2)$$

两边的位姿都是复杂的三角函数。所以，我们将使用一种不同的方法来表示旋转。

该方法是考虑一个任意点 **P** 相对于每个坐标系的向量，并确定${}^{A}\boldsymbol{p}$ 和${}^{B}\boldsymbol{p}$ 之间的关系。再次回到图 2.6，我们将问题分成两部分：旋转，然后平移。

背景材料：欧几里得(Euclid of Alexandria，**约公元前 325—公元前 265**)是被誉为“几何之父”的埃及数学家。在其专著《元素》(*Elements*)中，他从一小组公理中推断出了几何对象和整数的属性。

《元素》可能是在数学史上最成功的一本书，它描述了平面几何原理，是大多数人首次接触几何和公式化证明的基础，也是现代欧几里得几何的基础。欧几里得距离是简单的在一个平面上两个点之间的距离。欧几里得还写了《光学》一书，主要介绍几何视觉和透视。

先只考虑旋转的情况，我们创建一个新坐标系 $\{V\}$，其坐标轴平行于坐标系 $\{A\}$ 的轴，但其原点与坐标系 $\{B\}$ 的原点重合，如图 2.7 所示。根据方程(2.2)，我们可以将点 P 用 $\{V\}$ 中定义坐标轴的单位向量表示为

$$\begin{aligned} {}^{V}p &= {}^{V}x\hat{\boldsymbol{x}}_V + {}^{V}y\hat{\boldsymbol{y}}_V \\ &= (\hat{\boldsymbol{x}}_V \quad \hat{\boldsymbol{y}}_V)\begin{pmatrix} {}^{V}x \\ {}^{V}y \end{pmatrix} \end{aligned} \tag{2.3}$$

上式被写作一个行向量和一个列向量的点积。

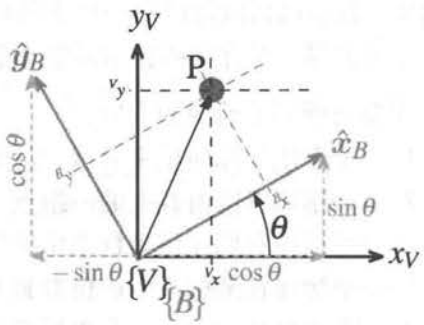


图 2.7 二维平面中旋转后的坐标系。点**P**可以分别看作在 $\{V\}$ 或 $\{B\}$ 坐标系中的点

坐标系 $\{B\}$ 可以用它的两个正交轴表示，这里用两个单位向量代表：

$$\begin{aligned} \hat{\boldsymbol{x}}_B &= \cos\theta\hat{\boldsymbol{x}}_V + \sin\theta\hat{\boldsymbol{y}}_V \\ \hat{\boldsymbol{y}}_B &= -\sin\theta\hat{\boldsymbol{x}}_V + \cos\theta\hat{\boldsymbol{y}}_V \end{aligned}$$

上式用矩阵形式可以分解成：

$$(\hat{\boldsymbol{x}}_B \quad \hat{\boldsymbol{y}}_B) = (\hat{\boldsymbol{x}}_V \quad \hat{\boldsymbol{y}}_V)\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \tag{2.4}$$

用方程(2.2)可以在坐标系 $\{B\}$ 中将 **P** 点表示为

$${}^{B}p = {}^{B}x\hat{\boldsymbol{x}}_B + {}^{B}y\hat{\boldsymbol{y}}_B = (\hat{\boldsymbol{x}}_B \quad \hat{\boldsymbol{y}}_B)\begin{pmatrix} {}^{B}x \\ {}^{B}y \end{pmatrix}$$

代入方程(2.4)，得

$${}^{B}p = (\hat{\boldsymbol{x}}_V \quad \hat{\boldsymbol{y}}_V)\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}\begin{pmatrix} {}^{B}x \\ {}^{B}y \end{pmatrix} \tag{2.5}$$

现在令方程(2.3)和方程(2.5)各自右侧的系数部分相等，可得

$$\begin{pmatrix} {}^{V}x \\ {}^{V}y \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}\begin{pmatrix} {}^{B}x \\ {}^{B}y \end{pmatrix}$$

上式描述了点如何通过坐标系旋转从坐标系 $\{B\}$ 变换到坐标系 $\{V\}$。这种类型的矩阵被称为旋转矩阵，记作${}^{V}\boldsymbol{R}_B$。

$$\begin{pmatrix} {}^{V}x \\ {}^{V}y \end{pmatrix} = {}^{V}\boldsymbol{R}_{B}\begin{pmatrix} {}^{B}x \\ {}^{B}y \end{pmatrix} \tag{2.6}$$

旋转矩阵${}^{V}\boldsymbol{R}_{B}$具有一些特殊的属性。首先，它是正规化的(也称为标准正交)，因为它的每列都是单位向量且相互正交的①。实际上矩阵的每列都是简单地将$\{B\}$定义在$\{V\}$中的单位向量，因此根据定义它们都是单位长度且正交的。

其次，它的行列式是+1，这意味着$\boldsymbol{R}$属于特殊的二维正交群，或$\boldsymbol{R}\in SO(2)\subset\mathbb{R}^{2\times 2}$。而且单位行列式还意味着一个向量在变换后的长度是不变的，即$|{}^{B}\boldsymbol{p}| = |{}^{V}\boldsymbol{p}|$，$\forall\theta$。

正交矩阵有一种非常方便的属性：$\boldsymbol{R}^{-1} = \boldsymbol{R}^{\mathrm{T}}$，即它的逆矩阵和转置矩阵相同。因此，我们可以重新将方程(2.6)整理为

$$\begin{pmatrix} {}^{B}x \\ {}^{B}y \end{pmatrix} = ({}^{V}\boldsymbol{R}_{B})^{-1}\begin{pmatrix} {}^{V}x \\ {}^{V}y \end{pmatrix} = ({}^{V}\boldsymbol{R}_{B})^{T}\begin{pmatrix} {}^{V}x \\ {}^{V}y \end{pmatrix} = {}^{B}\boldsymbol{R}_{V}\begin{pmatrix} {}^{V}x \\ {}^{V}y \end{pmatrix}$$

我们注意到，该矩阵的求逆就是将矩阵的上、下标交换位置，并可以得出恒等式$\boldsymbol{R}(-\theta) = \boldsymbol{R}(\theta)^{\mathrm{T}}$。

这里我们观察到一个有趣的事实：我们描述一个旋转的时候，不是用代表旋转角度的一个标量，而是用了一个有4个元素的2×2矩阵。但这4个元素并不是独立的，矩阵的每一列都是一个单位的大小，这提供了两个约束，列与列之间还都是正交的，这提供了另一种约束。4个元素加上3个约束，这样还是只剩下1个真正独立的值。旋转矩阵是一个非最小化表示的典型例子，虽然这种表示有一些缺点，诸如需要增加内存等，但它具备的优势更加突出，如可复合性。

描述位姿的第二部分就涉及图2.6所示的各坐标系原点的平移。由于坐标系$\{V\}$和$\{A\}$的轴是平行的，所以可以简单地进行向量相加：

$$\begin{pmatrix} {}^{A}x \\ {}^{A}y \end{pmatrix} = \begin{pmatrix} {}^{V}x \\ {}^{V}y \end{pmatrix} + \begin{pmatrix} x \\ y \end{pmatrix} \tag{2.7}$$

$$= \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}\begin{pmatrix} {}^{B}x \\ {}^{B}y \end{pmatrix} + \begin{pmatrix} x \\ y \end{pmatrix} \tag{2.8}$$

$$= \begin{pmatrix} \cos\theta & -\sin\theta & x \\ \sin\theta & \cos\theta & y \end{pmatrix}\begin{pmatrix} {}^{B}x \\ {}^{B}y \\ 1 \end{pmatrix} \tag{2.9}$$

或简写为

$$\begin{pmatrix} {}^{A}x \\ {}^{A}y \\ 1 \end{pmatrix} = \begin{pmatrix} {}^{A}\boldsymbol{R}_{B} & \boldsymbol{t} \\ \boldsymbol{0}_{1\times 2} & 1 \end{pmatrix}\begin{pmatrix} {}^{B}x \\ {}^{B}y \\ 1 \end{pmatrix} \tag{2.10}$$

其中，$\boldsymbol{t} = (x, y)$代表坐标系的平移变换，而坐标系旋转变换用${}^{A}\boldsymbol{R}_{B}$表示。因为$\{A\}$和$\{V\}$的轴是平行的，所以${}^{A}\boldsymbol{R}_{B} = {}^{V}\boldsymbol{R}_{B}$。将P点的坐标向量用齐次形式表达为

$${}^{A}\tilde{\boldsymbol{p}} = \begin{pmatrix} {}^{V}\boldsymbol{R}_{B} & \boldsymbol{t} \\ \boldsymbol{0}_{1\times 2} & 1 \end{pmatrix}{}^{B}\tilde{\boldsymbol{p}}$$

$$= {}^{A}\boldsymbol{T}_{B}\,{}^{B}\tilde{\boldsymbol{p}}$$

${}^{A}\boldsymbol{T}_{B}$称为齐次转换矩阵。这个矩阵有一个非常特殊的结构，并且属于特殊的二维欧几里得群，即$\boldsymbol{T}\in SE(2)\subset\mathbb{R}^{3\times 3}$。

① 参见附录D中有关向量、矩阵与线性代数的知识。

背景材料：一个向量(x, y)可以写成齐次形式$\tilde{\boldsymbol{p}} \in \mathbb{P}$，$\tilde{\boldsymbol{p}} = (x_1, x_2, x_3)$，其中$x = x_1/x_3$，$y = x_2/x_3$，$x_3 \neq 0$。这里齐次向量的维度增加了一个，在平面上的点就用三维向量表示。为了将点转换为齐次形式，我们通常会附加一个1，变为$\tilde{\boldsymbol{p}} = (x, y, 1)$。字母上的波浪线表示该向量是齐次形式的。

齐次向量具有一个重要的属性，即$\tilde{\boldsymbol{p}}$等价于$\lambda\tilde{\boldsymbol{p}}$，$\forall \lambda \neq 0$，记作$\tilde{\boldsymbol{p}} \simeq \lambda\tilde{\boldsymbol{p}}$。这意味着$\tilde{\boldsymbol{p}}$在平面中代表相同的点，而与比例系数无关。在本书第四部分讨论计算机视觉时，齐次表示法非常重要。更多细节见附录I。

通过与方程(2.1)的比较，很显然${}^A\boldsymbol{T}_B$代表了相对位姿：

$$\xi(x, y, \theta) \sim \begin{pmatrix} \cos\theta & -\sin\theta & x \\ \sin\theta & \cos\theta & y \\ 0 & 0 & 1 \end{pmatrix}$$

相对位姿ξ的一种具体表示是$\xi \sim \boldsymbol{T} \in SE(2)$，以及$\boldsymbol{T}_1 \oplus \boldsymbol{T}_2 \mapsto \boldsymbol{T}_1\boldsymbol{T}_2$，这是标准的矩阵乘法。

$$\boldsymbol{T}_1\boldsymbol{T}_2 = \begin{pmatrix} \boldsymbol{R}_1 & \boldsymbol{t}_1 \\ \boldsymbol{0}_{1\times 2} & 1 \end{pmatrix}\begin{pmatrix} \boldsymbol{R}_2 & \boldsymbol{t}_2 \\ \boldsymbol{0}_{1\times 2} & 1 \end{pmatrix} = \begin{pmatrix} \boldsymbol{R}_1\boldsymbol{R}_2 & \boldsymbol{t}_1 + \boldsymbol{R}_1\boldsymbol{t}_2 \\ \boldsymbol{0}_{1\times 2} & 1 \end{pmatrix}$$

上文给出的代数运算规则之一是$\xi \oplus 0 = \xi$。我们知道在矩阵中$\boldsymbol{TI} = \boldsymbol{T}$，其中$\boldsymbol{I}$是单位矩阵，所以对于位姿0对应的是$\boldsymbol{I}$单位矩阵。另一条规则是$\xi \ominus \xi = 0$。我们知道矩阵$\boldsymbol{T}\boldsymbol{T}^{-1} = \boldsymbol{I}$，这意味着$\ominus \boldsymbol{T} \mapsto \boldsymbol{T}^{-1}$

$$\boldsymbol{T}^{-1} = \begin{pmatrix} \boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}_{1\times 2} & 1 \end{pmatrix}^{-1} = \begin{pmatrix} \boldsymbol{R}^T & -\boldsymbol{R}^T\boldsymbol{t} \\ \boldsymbol{0}_{1\times 2} & 1 \end{pmatrix}$$

对于点$\tilde{\boldsymbol{p}} \in \mathbb{P}^2$，有$\boldsymbol{T} \cdot \tilde{\boldsymbol{p}} \mapsto \boldsymbol{T}\tilde{\boldsymbol{p}}$，这是一个标准的矩阵-向量积。

为了使上述推导更形象具体，下面我们将使用MATLAB工具箱展示一些具体数值化的例子。首先用函数se2创建一个齐次变换：

```
>> T1 = se2(1, 2, 30*pi/180)
T1 =
    0.8660   -0.5000    1.0000
    0.5000    0.8660    2.0000
         0         0    1.0000
```

它代表(1, 2)的平移和30°的旋转。相对于世界坐标系，我们可以将该变换用绘图函数绘制如下：

```
>> axis([0 5 0 5]);
>> trplot2(T1, 'frame', '1', 'color', 'b')
```

绘图函数中的参数规定该坐标系的标签是{1}，且绘制成蓝色(图中$\mathrm{Y}_1\{1\}\mathrm{X}_1$坐标系)，如图2.8所示。我们再创建另一个平移(2, 1)，零旋转的相对位姿：

```
>> T2 = se2(2, 1, 0)
T2 =
     1     0     2
     0     1     1
     0     0     1
```

在图中用红色(图中$\mathrm{Y}_2\{2\}\mathrm{X}_2$坐标系)绘制出来：

```
>> hold on
>> trplot2(T2, 'frame', '2', 'color', 'r');
```

现在，我们可以将两个相对位姿复合：

```
>> T3 = T1*T2
T3 =
    0.8660   -0.5000    2.2321
    0.5000    0.8660    3.8660
         0         0    1.0000
```

在图中用绿色(图中 $Y_3\{3\}X_3$ 坐标系)绘制出来：

```
>> trplot2(T3, 'frame', '3', 'color', 'g');
```

我们看到在复合中，坐标系{2}的平移(2，1)是相对于坐标系{1}的。要注意的是，最终坐标系{3}相对于世界坐标系的平移量并不是(3，3)，因为它是相对于旋转后的坐标系{1}再平移的。复合运算的不可交换性可用以下例子证明：

```
>> T4 = T2*T1;
>> trplot2(T4, 'frame', '4', 'color', 'c');
```

可以看出，坐标系{4}与坐标系{3}不同。

现在我们相对于世界坐标系定义一个点(3，2)：

```
>> P = [3 ; 2 ];
```

这是一个列向量，把它加入图中：

```
>> plot_point(P, '*');
```

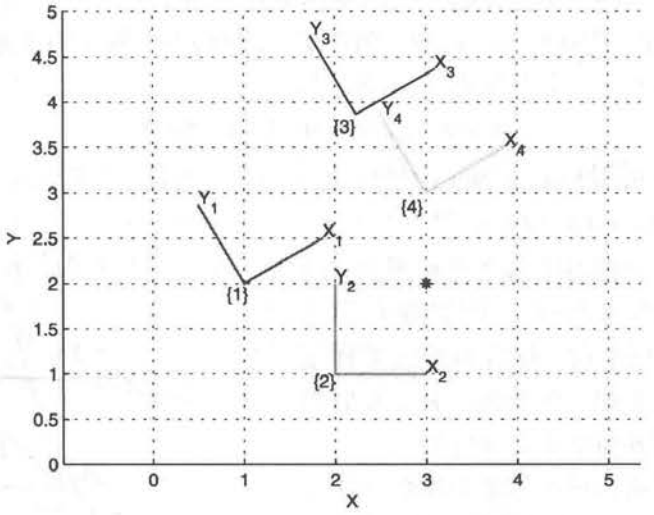


图2.8　使用工具箱函数 trplot2 绘制出的坐标系

用式(2.1)确定该点相对于坐标系{1}的坐标如下：

$$^0\boldsymbol{p} = {}^0\xi_1 \cdot {}^1\boldsymbol{p}$$

将上式重新整理，得

$$^1\boldsymbol{p} = {}^1\xi_0 \cdot {}^0\boldsymbol{p} = ({}^0\xi_1)^{-1} \cdot {}^0\boldsymbol{p}$$

下面代入具体数值：

```
>> P1 = inv(T1) * [P; 1]
P1 =
    1.7321
   -1.0000
    1.0000
```

首先，通过附加一个1将欧几里得点转换为齐次形式。该齐次形式的结果在坐标系{1}中有一个负的 y 坐标。使用工具箱函数还可以进行反变换，操作如下：

```
>> h2e( inv(T1) * e2h(P) )
ans =
    1.7321
   -1.0000
```

这里又变回了欧几里得坐标点结果。辅助函数 e2h 将欧几里得坐标点转换为齐次形式，而 h2e 进行逆转换。更简洁的表达方式可写成

```
>> homtrans( inv(T1), P)
ans =
    1.7321
   -1.0000
```

该相同点相对于坐标系{2}有

```
>> P2 = homtrans( inv(T2), P)
P2 =
     1
     1
```

2.2 三维空间位姿描述

三维情况实际上是前一节讨论的二维情况的延伸。我们在二维坐标系上增加一个额外的坐标轴，通常用 z 表示，它同时与 x 轴和 y 轴正交。z 轴的方向服从右手规则，并构成右手坐标系。与各坐标轴平行的单位向量记作 $\hat{\boldsymbol{x}}$、$\hat{\boldsymbol{y}}$ 和 $\hat{\boldsymbol{z}}$[①]：

$$\hat{\boldsymbol{z}}=\hat{\boldsymbol{x}}\times\hat{\boldsymbol{y}},\ \hat{\boldsymbol{x}}=\hat{\boldsymbol{y}}\times\hat{\boldsymbol{z}};\ \hat{\boldsymbol{y}}=\hat{\boldsymbol{z}}\times\hat{\boldsymbol{x}} \tag{2.11}$$

坐标系中的一个点 $\mathbf{P}$ 可用其 x，y 和 z 的坐标值(x, y, z)或者一个约束向量表示：

$$\boldsymbol{P}=x\hat{\boldsymbol{x}}+y\hat{\boldsymbol{y}}+z\hat{\boldsymbol{z}}$$

图 2.9 展示了一个相对于参考坐标系{A}的坐标系{B}。可以清楚地看到，{B}的原点通过向量 $\boldsymbol{t}=(x, y, z)$进行平移，然后再通过某种复杂方式进行了旋转。正如二维情况一样，我们如何表示坐标系之间的方向是非常重要的。

我们的做法还是从相对于每个坐标系的任意一点 $\mathbf{P}$ 出发，然后再确定${}^{A}\boldsymbol{P}$ 和${}^{B}\boldsymbol{P}$ 之间的关系。我们仍然从两方面考虑：旋转和平移。三维情况下的旋转非常复杂，我们接下来在整个小节中都会对它进行讲解。

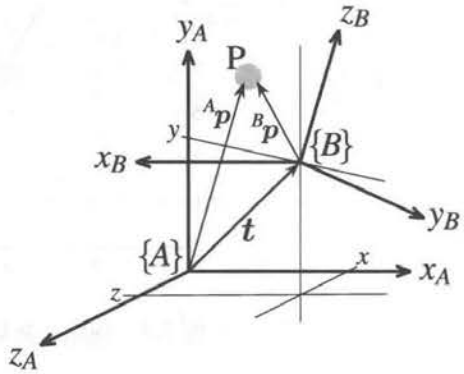


图 2.9 两个三维坐标系{A}和{B}。{B}相对于{A}旋转和平移

2.2.1 三维空间姿态描述

任何两个独立的正交坐标系都可以通过一系列(不超过 3 次)相对于坐标轴的旋转联系起来，但其中连续的两次旋转不能绕同一轴线。

——欧拉旋转定理(奎伯斯，1999)

图 2.9 中显示了两个右手坐标系，其方向差异很大，我们如何相对于一个坐标系去表示另一个坐标系呢？可以想象一下，将坐标系{A}拿在我们手里旋转，直到与坐标系{B}方向一致。欧拉旋转定理指出，任何旋转都可以看作是由一系列相对不同坐标轴的旋转组合而成的。

首先考虑绕单个坐标轴的旋转。图 2.10 显示了一个右手坐标系，以及它绕不同坐标轴旋转不同角度的情形。

① 在所有这些单位向量运算中，从左到右的符号排列(忽略等号)是以一种 xyz 的顺序进行循环置换的。

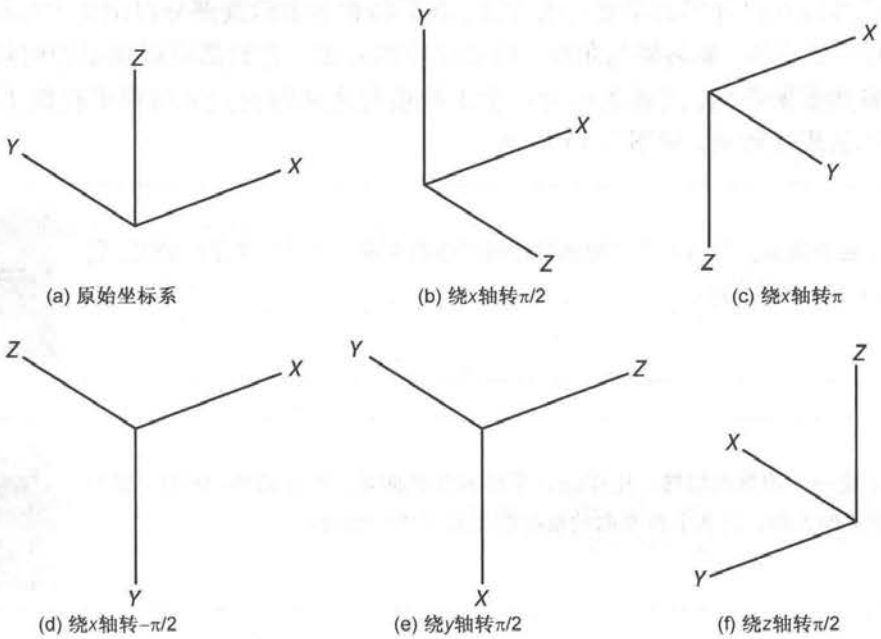


图 2.10　一个三维坐标系的旋转。(a)原始坐标系；(b) ~ (f)原始坐标系经各种旋转后的情况

但绕坐标轴连续旋转就没那么简单了，其中存在一些微妙的变化。如图 2.11 所示，它演示了一个坐标系按不同的顺序旋转的情形。可以看到，虽然旋转角度相同，但最终得到的坐标系方向完全不同，它取决于旋转的顺序。这是三维世界里一个深奥且让人迷惑的特征，而且它曾困扰了数学家们很长一段时间。我们在本章中使用位姿代数的意义就在于运算符⊕是不可交换的，它体现了上述旋转顺序的重要性。

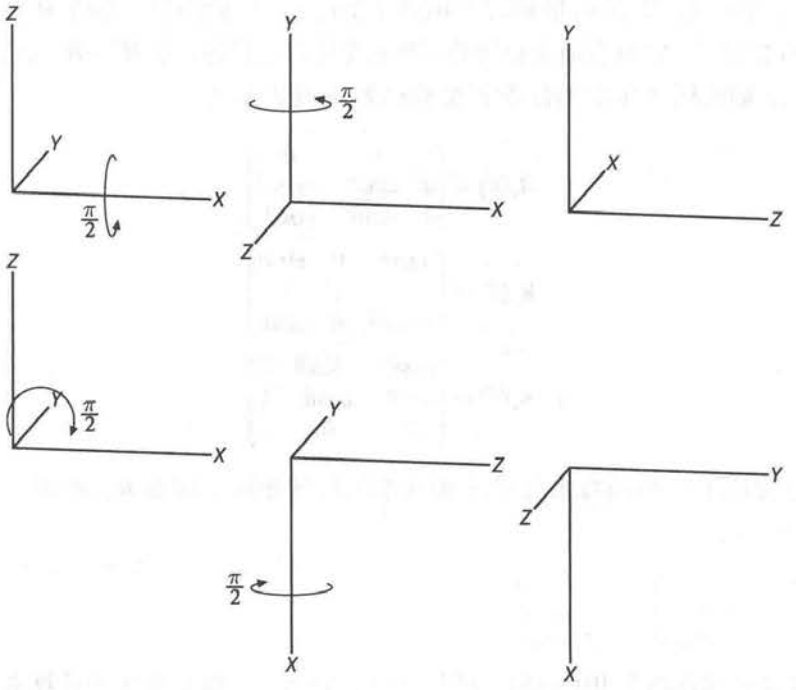
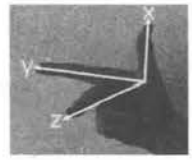


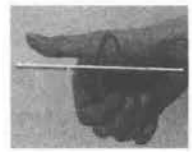
图 2.11　旋转顺序不可交换性的例子。在上面一排中坐标系先绕 x 轴旋转 π/2，然后绕 y 轴旋转 π/2。在下面一排旋转顺序正好颠倒了。两种旋转的结果明显不同

数学家已经提炼出许多表示旋转的方法，我们将在本节后续部分讨论其中几种：正交旋转矩阵，欧拉和卡尔丹角，旋转轴与角度，以及单位四元数。它们都可以表示为向量或矩阵，即 MATLAB 的自然数据类型，或者表示为一个工具箱自定义的类。工具箱中提供了许多函数可以将这些表示法相互转换，如图 2.15 所示。

背景材料：右手定则。所谓右手坐标系是指可以由右手前三个手指的指向确定，它们分别指向 x，y 和 z 轴的方向。



背景材料：绕一个向量的旋转。用你的右手抓握住该向量，并让拇指（你的 x 轴手指）指向向量箭头的方向，其余手指卷曲的指向就是角度增加的方向。



2.2.1.1 正交旋转矩阵

正如在二维情况下一样，我们可以用相对于参考坐标系的坐标轴单位向量表示它们所在坐标系的方向。每一个单位向量有 3 个元素，它们组成了 3×3 阶正交矩阵 ${}^{A}\boldsymbol{R}_{B}$：

$$\begin{pmatrix} {}^{A}x \\ {}^{A}y \\ {}^{A}z \end{pmatrix} = {}^{A}\boldsymbol{R}_{B} \begin{pmatrix} {}^{B}x \\ {}^{B}y \\ {}^{B}z \end{pmatrix} \tag{2.12}$$

上式将一个相对于坐标系 $\{B\}$ 的向量旋转为相对于坐标系 $\{A\}$ 的向量。矩阵 $\boldsymbol{R}$ 属于特殊三维正交群，或 $\boldsymbol{R} \in SO(3) \subset \mathbb{R}^{3\times3}$。它具有前文提到的标准正交矩阵的特性，如 $\boldsymbol{R}^{\mathrm{T}} = \boldsymbol{R}^{-1}$ 以及 $\det(\boldsymbol{R}) = 1$。

分别绕 x,y,z 轴旋转 θ 角后的标准正交旋转矩阵可表示为

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{pmatrix}$$

$$R_y(\theta) = \begin{pmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{pmatrix}$$

$$R_z(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

工具箱中也提供了一些函数来计算这些基本的旋转矩阵，例如 $\boldsymbol{R}_x(\theta)$ 是

```
>> R = rotx(pi/2)
R =
    1.0000         0         0
         0    0.0000   -1.0000
         0    1.0000    0.0000
```

这个旋转的效果也显示在图 2.10(b)中。函数 `roty` 和 `rotz` 则分别用于计算 $\boldsymbol{R}_y(\theta)$ 和 $\boldsymbol{R}_z(\theta)$。

要绘制出相应的坐标系，可以用

```
>> trplot(R)
```

如图 2.12(a)所示。如果要想使一个旋转更加生动有力，可以使用工具箱函数 tranimate 制作一个旋转动画：

```
>> tranimate(R)
```

它将展示世界坐标系旋转到指定坐标系的过程。

背景材料：在正交旋转矩阵中，从左至右的列向量可以告诉我们旋转后新坐标系各个轴相对于当前坐标系的方向。例如，如果

```
R =
    1.0000         0         0
         0    0.0000   -1.0000
         0    1.0000    0.0000
```

则新坐标系的 x 轴仍在以前 x 轴方向(1, 0, 0)，但其 y 轴却在以前的 z 轴方向(0, 0, 1)，而新的 z 轴在以前 y 轴的反方向(0, −1, 0)。这里，之所以 x 轴没变，是因为这个旋转是围绕 x 轴发生的。行向量则相反，它们表示了当前坐标系的各个轴在新坐标系中的方向。

为了说明旋转的复合，我们继续旋转图 2.12(a)所示的坐标系，这一次绕它的 y 轴旋转：

```
>> R = rotx(pi/2) * roty(pi/2)
R =
    0.0000         0    1.0000
    1.0000    0.0000   -0.0000
   -0.0000    1.0000    0.0000
>> trplot(R)
```

绘制出如图 2.12(b)所示的坐标系。在这个坐标系中，x 轴现在指向世界坐标系 y 轴的方向。

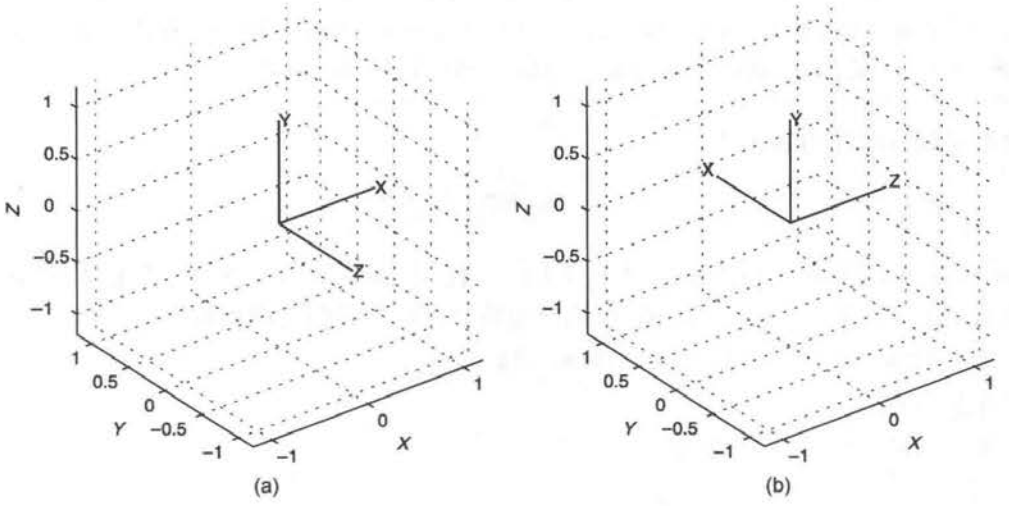


图 2.12　用函数 trplot 显示坐标系旋转。(a)参考坐标系绕 x 轴旋转 $\pi/2$；(b)坐标系 a 绕 y 轴旋转 $\pi/2$

如果颠倒以上旋转的顺序，可以看出旋转的不可交换性：

```
>> roty(pi/2)*rotx(pi/2)
ans =
    0.0000    1.0000    0.0000
         0    0.0000   -1.0000
   -1.0000    0.0000    0.0000
```

得到的结果是完全不同的。

回忆一下欧拉旋转定理，它指出任何旋转都可以用不超过 3 次绕坐标轴的旋转表示。这意味着，一般情况下两个坐标系之间的任意旋转均可分解为一组绕三个旋转轴转动的角度——这个问题将在下一节讨论。

正交矩阵有 9 个元素，但它们不是独立的。每一列都是单位长度，这提供了 3 个约束。列与列之间相互正交，又提供了另外 3 个约束[①]。9 个元素加上 6 个约束，实际上只有 3 个独立的值。

2.2.1.2 三角度表示法

欧拉旋转定理要求绕 3 个轴依次旋转，但不能绕同一轴线连续旋转两次。旋转顺序分为两种：欧拉式和卡尔丹式，分别以欧拉和卡尔丹(Cardano)的名字命名。

背景材料：莱昂哈德·欧拉(Leonhard Euler，1707—1783)，瑞士著名数学家和物理学家，主导 18 世纪的数学研究。他是约翰·伯努利(Johan Bernoulli)的学生。他应用新的数学技术，如微积分，解决了力学和光学的许多问题。他还创造了我们现在使用的函数符号，$y=F(x)$。在机器人学方面，我们使用了他的旋转定理以及关于转动动力学的运动方程。

他非常多产，所写专著有 75 部，而且几乎一半的作品都是他在完全失明后生命的最后 17 年所著。

欧拉式是绕一个特定的轴重复旋转，但不是连续的：XYX、XZX、YXY、YZY、ZXZ 或 ZYZ。卡尔丹式的特点是绕 3 个不同轴旋转：XYZ、XZY、YZX、YXZ、ZXY 或 ZYX。一般来说，所有这些序列均被统称为欧拉角，共有 12 种形式可供选择。

现在通常是将所有三角度表示法都称作欧拉角，但这种称呼不够确切，因为一共有 12 种不同形式的欧拉角可供选择。在某一特定技术领域中，所说欧拉角往往指一种特定的旋转角顺序。

ZYZ 序列的欧拉角表示为

$$\boldsymbol{R}=\boldsymbol{R}_z(\phi)\boldsymbol{R}_y(\theta)\boldsymbol{R}_z(\psi) \tag{2.13}$$

它常用在航空和机械动力学中，并在本书工具箱中使用。欧拉角是一个三维向量 $\boldsymbol{\Gamma}=(\phi,\theta,\psi)$。

例如，要计算 $\boldsymbol{\Gamma}=(0.1,0.2,0.3)$ 的等价旋转矩阵，我们可以这样写：

```
>> R = rotz(0.1) * roty(0.2) * rotz(0.3);
```

或采用更简便的方式：

```
>> R = eul2r(0.1, 0.2, 0.3)
R =
    0.9021   -0.3836    0.1977
    0.3875    0.9216    0.0198
   -0.1898    0.0587    0.9801
```

上述问题的逆命题是要找到给定旋转矩阵的欧拉角，可采用以下函数：

```
>> gamma = tr2eul(R)
gamma =
    0.1000    0.2000    0.3000
```

① 如果矩阵列向量是 $\boldsymbol{c}_i$，$i\in 1\cdots 3$，那么 $\boldsymbol{c}_1\cdot\boldsymbol{c}_2=\boldsymbol{c}_2\cdot\boldsymbol{c}_3=\boldsymbol{c}_3\cdot\boldsymbol{c}_1=0$。

但是，如果 θ 为负时，如

```
>> R = eul2r(0.1 , -0.2, 0.3)
R =
    0.9021   -0.3836   -0.1977
    0.3875    0.9216   -0.0198
    0.1898   -0.0587    0.9801
```

其反函数结果为

```
>> tr2eul(R)
ans =
   -3.0416    0.2000   -2.8416
```

它返回了一个正的 θ 值和两个不同的 ϕ、ψ 值。但这组欧拉角对应的旋转矩阵仍与前一组是相同的：

```
>> eul2r(ans)
ans =
    0.9021   -0.3836   -0.1977
    0.3875    0.9216   -0.0198
    0.1898   -0.0587    0.9801
```

两组不同的欧拉角对应同一个旋转矩阵，说明从旋转矩阵到欧拉角的映射是不唯一的，而工具箱函数返回的角度 θ 始终为正。

对于 $\theta=0$ 的情况：

```
>> R = eul2r(0.1 , 0, 0.3)
R =
    0.9211   -0.3894         0
    0.3894    0.9211         0
         0         0    1.0000
```

反函数返回的角度值为

```
>> tr2eul(R)
ans =
         0         0    0.4000
```

它与原值完全不同。其实在这种情况下，从式(2.13)得出的旋转矩阵是

$$\boldsymbol{R}=\boldsymbol{R}_z(\phi)\boldsymbol{R}_z(\psi)=\boldsymbol{R}_z(\phi+\psi)$$

由于 $\boldsymbol{R}_y=\boldsymbol{I}$，所以可化简为只是 $\phi+\psi$ 的一个函数。对于逆运算而言，只能确定这个和的值。要想得到其中每个值，只能按惯例取 $\phi=0$。$\theta=0$ 的情况实际上是一个奇异点，我们将在下一节详细讨论。

另一种广泛使用的旋转角顺序是横滚-俯仰-偏航角，即

$$\boldsymbol{R}=\boldsymbol{R}_x(\theta_r)\boldsymbol{R}_y(\theta_p)\boldsymbol{R}_z(\theta_y) \tag{2.14}$$

它用于描述船舶、飞机和车辆的姿态时非常直观。横滚、俯仰和偏航(也称为侧倾、姿态和航向)是指分别绕 x、y、z 轴的旋转。这个 xyz 角序列，即专业上的卡尔丹角，也被称为泰特-布莱恩角(Tait-Bryan)或导航角[①]。对于航空及地面车辆而言，通常定义 x 轴为前进的方向、z 轴垂直向下、y 轴指向右手方向[②]。例如：

① 根据苏格兰物理学家、四元数支持者彼得·泰特(Peter Tait)和早期的威尔斯空气动力学家乔治·布莱恩(George Bryan)的名字命名。

② 注意，大多数机器人学教科书[Paul 1981；Sicilianoet al. 2008；Spong et al. 2006]都交换了其中的 x 轴和 z 轴，把向前的方向定义为 z 轴方向而不是 x 轴方向，也就是说是绕 z 轴而不是 x 轴横滚。在第8版之前的机器人工具箱也采用这个顺序。但现在的工具箱中默认的是 xyz 顺序，不过也可以用'zyx'选项来指定 zyx 的顺序。

```
>> R = rpy2r(0.1, 0.2, 0.3)
R =
    0.9363   -0.2896    0.1987
    0.3130    0.9447   -0.0978
   -0.1593    0.1538    0.9752
```

其逆运算为

```
>> gamma = tr2rpy(R)
gamma =
    0.1000    0.2000    0.3000
```

横滚-俯仰-偏航序列允许每个角度值有任意正负号，不会产生多解的情况。但它也有一个奇异点，即当 $\theta_p = \pm\pi/2$ 时，不过这个点刚好在大多数车辆可能的姿态范围以外。

背景材料：吉罗拉莫·卡尔丹(Girolamo Cardano，1501—1576)是意大利文艺复兴时期的数学家、医生、占星师，外加赌徒。他出生在意大利的帕维亚，是一个极有数学天赋的律师的私生子。他曾在帕多瓦大学学习医学，不久之后便成为诊断伤寒病的第一人。赌博也是他养活自己的一个方式，他写了一本关于机会博弈的书 *Liber de ludo aleae*(《论赌博游戏》)，是有关概率问题的第一个系统论述，其中还包含了有效的作弊方法。他的家庭存在很多问题：他的长子因毒害其妻子受到刑罚，他的女儿是一名妓女，死于梅毒(他因此曾写过一篇关于梅毒治疗的论文)。他还计算并发布了耶稣的星座，但被人指责为异端并因此入狱，直到他发誓并放弃了他的教授职位才获释。

他在 1545 年出版的书 *ARS MAGNA* 中公布了三次和四次方程的解法，他还发明了密码锁，以及由三个同心圆组成的万向支架，可以让罗盘或陀螺仪自由转动(见图 2.13)，他的发明还包括具有万向节的卡尔丹轴，这种轴至今仍在汽车上使用。

2.2.1.3 奇异点及万向节锁

上面叙述的三旋转角度表示方式中，一个根本的问题是奇异点。当中间的旋转轴平行于第一个或第三个旋转轴时这种情况就会发生。对于万向节锁(因电影《阿波罗 13 号》而出名的术语)，也存在同样的问题。

用于导航的机械陀螺仪如图 2.13 所示。在其最核心的装配结构中有 3 个相互正交的框架，它们能使安装于其中的稳定体相对于宇宙静止。陀螺仪通过这个万向节机构连接到飞船机体上，这样无论飞船做任何机动飞行，都不会给陀螺仪内部的稳定平台施加外力矩。通过测量这些万向框架的轴相对于稳定平台的转动角度，就可以确定飞船的航行姿态——直接显示出飞船的横滚-俯仰-偏航角，图示设计中是卡尔丹角的 YZX 序列[①]。

现在考虑陀螺仪中间万向架旋转角(相对于飞船的 z 轴旋转)为 90°时的情况。这时陀螺仪的内万向架与外万向架的轴对齐，它们的旋转轴线重合。因为这两个旋转轴平行，这时陀螺仪只有两个有效的旋转轴，而不是原来的三个——我们称之为丢失了一个自由度[②]。

---

① "登月舱坐标系是右手坐标系，$+X$ 轴朝上，$+Z$ 轴朝前，$+Y$ 轴指向右边。旋转变换矩阵由一个基于 2-3-1 顺序的欧拉角构成，即：首先关于 Y 轴倾斜，然后绕 Z 轴滚动，最后关于 X 轴俯仰。正旋转分别是上倾，右滚，左偏"[Hoag 1963]。

② 该问题对于这个特殊陀螺仪的应用是一个明显的制约因素[Hoag 1963]。这个问题可以通过增加第四个万向支架得以缓解，该方法在别的航天器上曾使用，但在阿波罗登月舱上因为重量和空间的原因并未采用。

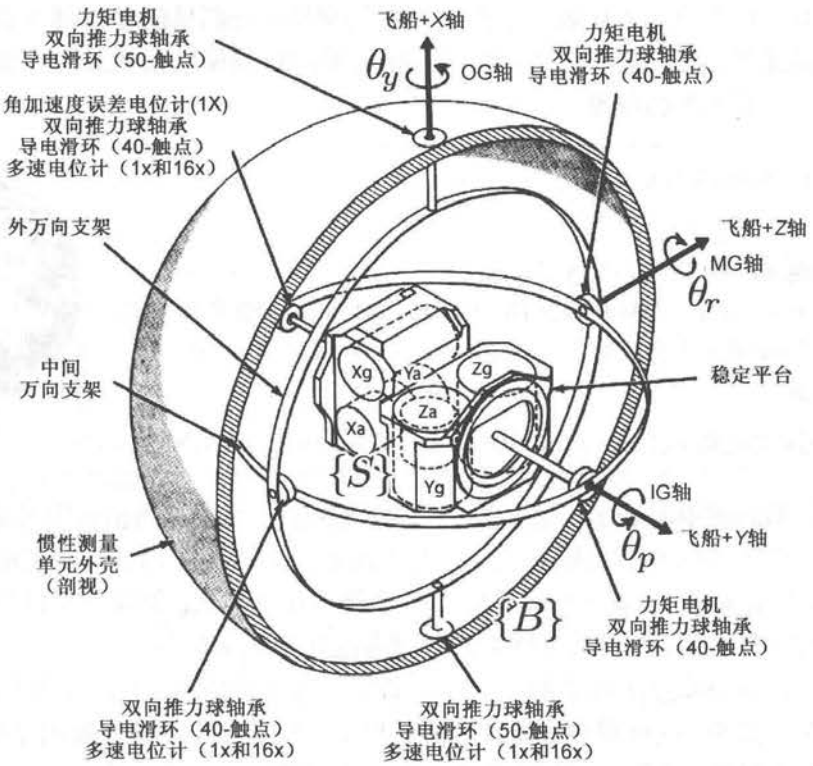


图 2.13　阿波罗月球舱(LM)使用的惯性测量单元(IMU)原理图。该飞船的坐标系定义为 x 轴向上，z 轴向前，y轴向右。从稳定平台坐标系$\{S\}$开始，逐级向外到飞船体坐标系$\{B\}$，其旋转角的顺序是 YZX 。被标记为X_g、Y_g和Z_g的元件分别是绕x、y和z轴的角加速度传感器，而被标记为X_a、Y_a和Z_a的元件分别是沿x、y和z轴的线性加速度传感器(阿波罗操作手册，LMA790-3-LM)

从数学(而非机械)上看，这个问题可以通过建立一种登月舱坐标系来解释，其中固联在飞船机体上的坐标系$\{B\}$相对于固联在稳定平台上的坐标系$\{S\}$做旋转，并且可以表示为

$$^{S}\boldsymbol{R}_{B} = \boldsymbol{R}_{y}(\theta_{p})\boldsymbol{R}_{z}(\theta_{r})\boldsymbol{R}_{x}(\theta_{y})$$

当 $\theta_r = \pi/2$ 时，可以应用循环旋转规则得到下面的恒等式：①

$$\boldsymbol{R}_{y}(\theta)\boldsymbol{R}_{z}(\tfrac{\pi}{2}) \equiv \boldsymbol{R}_{z}(\tfrac{\pi}{2})\boldsymbol{R}_{x}(\theta)$$

进而得到

$$^{S}\boldsymbol{R}_{B} = \boldsymbol{R}_{z}(\tfrac{\pi}{2})\boldsymbol{R}_{x}(\theta_{p})\boldsymbol{R}_{x}(\theta_{y}) = \boldsymbol{R}_{z}(\tfrac{\pi}{2})\boldsymbol{R}_{x}(\theta_{p}+\theta_{y})$$

上式中没有表示出飞船绕 y 轴的旋转。这就带来了问题，因为航天器绕 y 轴的旋转将使稳定平台也旋转，从而破坏其相对于恒星的精确对准。

---

① 旋转须服从循环旋转规则：

$R_X(\pi/2)\ R_Y(\theta)\ R_X(\pi/2)^{\mathrm{T}} \equiv R_Z(\theta)$

$R_Y(\pi/2)\ R_Z(\theta)\ R_Y(\pi/2)^{\mathrm{T}} \equiv R_X(\theta)$

$R_Z(\pi/2)\ R_X(\theta)\ R_Z(\pi/2)^{\mathrm{T}} \equiv R_Y(\theta)$

以及反循环旋转规则：

$R_Y(\pi/2)^{\mathrm{T}}\ R_X(\theta)\ R_Y(\pi/2) \equiv R_Z(\theta)$

$R_Z(\pi/2)^{\mathrm{T}}\ R_Y(\theta)\ R_Z(\pi/2) \equiv R_X(\theta)$

一个自由度的缺失意味着在数学上我们不能反变换，我们只能建立两个角度之间的线性关系。在这种情况下，我们能做得最好的就是确定俯仰角和偏航角的总和。前面我们用欧拉角的奇异点也看到了类似的现象。

背景材料： 任务时钟：02 08 12 47

- **飞行员：**“去，盖德。”
- **盖德：**“他越来越接近万向节锁了。”
- **飞行员：**“收到。船长，建议他带上 C3，C4，B3，B4，C1 和 C2 的推进器，并通知他越来越接近万向节锁。”
- **船长：**“收到。”

《阿波罗 13 号控制通信环任务》(1970)[Lovell and Kluger 1994，p131；NASA 1970]。

所有三角度形式的姿态表示，无论欧拉式或卡尔丹式，当两连续轴共线时都会遇到万向节锁同样的问题。对于 ZYZ 形式的欧拉角，它发生在 $\theta = k\pi$，$k \in \mathbb{Z}$ 时，对于用横滚-俯仰-偏航角的情况，会发生在 $\theta_p = \pm(2k+1)\pi/2$ 时。虽然都存在奇异点，但我们可以想办法让奇异点不在航行体正常运行时出现，这需要明智地选择角度序列和坐标系。

奇异点是采用最简化方法带来的一个不幸后果。为了消除这个问题，我们必须采取不同的姿态描述方法。其中，阿波罗登月舱团队的人提出一种用四个万向支架的系统，其成功的关键是引进了第四个参数，我们将很快在 2.2.1.6 节中讨论。

2.2.1.4 双向量表示法

对于关节臂式机器人，一般会在它的末端执行器上固联一个坐标系 $\{E\}$，如图 2.14 所示。通常情况下，工具的轴线为坐标系的 z 轴，并被称为接近向量，记为 $\hat{\boldsymbol{a}} = (a_x, a_y, a_z)$。对于某些应用来说，定义接近向量比定义欧拉角或横滚-俯仰-偏航角更为方便。

然而定义出 z 轴的方向还不足以表示完整坐标系——我们还需要确定 x 轴和 y 轴的方向。为了确定末端执行器的姿态，我们定义一个与接近向量正交的向量，它位于机器人手爪的两个手指之间，被称为姿态向量，记为 $\hat{\boldsymbol{o}} = (o_x, o_y, o_z)$。有了接近和姿态这两个单位向量，就足以完全确定旋转矩阵了：

$$\boldsymbol{R} = \begin{pmatrix} n_x & o_x & a_x \\ n_y & o_y & a_y \\ n_z & o_z & a_z \end{pmatrix} \tag{2.15}$$

因为剩下的一列可以用式(2.11)计算，即 $\hat{\boldsymbol{n}} = \hat{\boldsymbol{o}} \times \hat{\boldsymbol{a}}$。

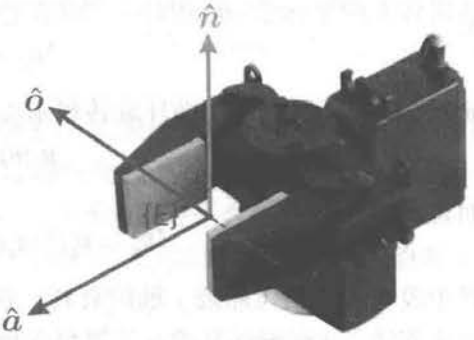


图 2.14 机器人末端执行器的坐标系用一个接近向量$\hat{\boldsymbol{a}}$和一个姿态向量$\hat{\boldsymbol{o}}$定义了它的位姿，从而可以计算出向量$\hat{\boldsymbol{n}}$。$\hat{\boldsymbol{n}}$、$\hat{\boldsymbol{o}}$和$\hat{\boldsymbol{a}}$分别对应于末端执行器坐标系的x轴、y轴和z轴(照片由lynxmotion. com提供)

即使两个向量 $\hat{\boldsymbol{a}}$ 和 $\hat{\boldsymbol{o}}$ 不是正交的，它们仍然能定义一个平面，而且计算出来的 $\hat{\boldsymbol{n}}$ 仍垂直于这个平面。在这种情况下，我们需要用 $\hat{\boldsymbol{o}}' = \hat{\boldsymbol{a}} \times \hat{\boldsymbol{n}}$ 来计算一个新的向量值，它仍然在 $\hat{\boldsymbol{a}}$ 和 $\hat{\boldsymbol{o}}$ 定义的平面中，只不过与 $\hat{\boldsymbol{a}}$ 和 $\hat{\boldsymbol{n}}$ 都正交[①]。

① 只要它们是不平行的。

使用工具箱函数，可以执行以下计算：

```
>> a = [1 0 0]';
>> o = [0 1 0]';
>> oa2r(o, a)
ans =
     0     0     1
     0     1     0
    -1     0     0
```

任何两个非平行的向量都足以定义一个坐标系。对于一个摄像头，通常我们取光轴为 z 轴，摄像头的左侧为 x 轴。对于移动机器人，通常我们规定重力加速度方向(用加速度计测量)为 z 轴方向，前进方向(用电子罗盘测量)为 x 轴方向。

2.2.1.5 绕任意向量旋转

对于空间中两个任意姿态的坐标系，总可以在空间里找到某个轴，使其中一个坐标系绕该轴旋转一个角度就能与另一个坐标系姿态重合。以先前使用过的一个旋转

```
>> R = rpy2r(0.1 , 0.2, 0.3);
```

我们可以确定如下的一个角度和一个向量：

```
>> [theta, v] = tr2angvec(R)
theta =
    0.3816
v =
    0.3379    0.4807    0.8092
```

其中，theta 是旋转的角度大小，v 是绕其旋转的向量[①]。

这些信息实际上包含在 R 的特征值和特征向量中。使用 MATLAB 的内部函数 eig 可以求矩阵的特征值和特征向量：

```
>> [v,lambda] = eig(R)
v =
   0.6655                0.6655                0.3379
  -0.1220 - 0.6079i     -0.1220 + 0.6079i      0.4807
  -0.2054 + 0.3612i     -0.2054 - 0.3612i      0.8092
lambda =
   0.9281 + 0.3724i           0                      0
        0                0.9281 - 0.3724i            0
        0                     0                 1.0000
```

矩阵 lambda 中的对角线元素即返回的特征值，其相应的特征向量位于矩阵 v 中对应的列向量。

一个正交旋转矩阵总有一个实特征值 $\lambda=1$，以及一对共轭复特征值 $\lambda=\cos\theta\pm i\sin\theta$，其中 θ 代表旋转的角度。根据特征值和特征向量的定义有

$$\boldsymbol{R}\boldsymbol{v}=\lambda\boldsymbol{v}$$

其中，$\boldsymbol{v}$ 是对应于 λ 的特征向量。当 $\lambda=1$ 时：

$$\boldsymbol{R}\boldsymbol{v}=\boldsymbol{v}$$

这意味着对应的这个特征向量 $\boldsymbol{v}$ 是不随旋转发生改变的。这样的向量只有一个，而旋转就是以这个向量为轴发生的。上面的例子中第三个特征值等于 1，所以旋转轴应是矩阵 v 中的第三列。

① 这不是唯一的。绕 $\boldsymbol{v}$ 向量旋转 θ 角后，两个坐标系的姿态就相同了。

反过来，使用罗德里格斯(Rodrigues)旋转方程，可以从角度和向量计算出相应的旋转矩阵：[①]

$$\boldsymbol{R} = \boldsymbol{I}_{3\times3} + \sin\theta S(\boldsymbol{v}) + (1-\cos\theta)(\boldsymbol{v}\boldsymbol{v}^{\mathrm{T}} - \boldsymbol{I}_{3\times3})$$

前面我们熟悉的一个绕 x 轴旋转 $\pi/2$ 的例子，其旋转矩阵为

```
>> R = angvec2r(pi/2, [1 0 0])
R =
    1.0000         0         0
         0    0.0000   -1.0000
         0    1.0000    0.0000
```

我们注意到，绕任意轴旋转这种表示法都可以参数化为 4 个数字：3 个代表旋转轴，1 个代表旋转角度。然而，空间中的方向可以用一个只有两个独立参数的单位向量表示，因为第三个元素可以由下式计算：

$$v_3 = \sqrt{1 - v_1^2 - v_2^2}$$

这样真正独立的参数只有 3 个。或者，我们也可以把单位向量和角度相乘，得到另外一种 3 个参数的表示方法 $\boldsymbol{v}\theta$。虽然这些表示方法对于数据存储来说是最小和最高效的，但在分析研究时是存在问题的。其他学者还提出过多种变形，包括 $\boldsymbol{v}\sin(\theta/2)$ 和 $\boldsymbol{v}\tan(\theta)$，但它们在 $\theta=0$ 时都呈现病态。

奥凌迪·罗德里格斯(Olinde Rodrigues，1795 年—1850)是一个犹太裔法国银行家和数学家，其专著涉及的范围很广，包括政治、社会改革和银行。他于 1816 年在巴黎大学获得数学博士学位，他也因研究与勒让德多项式有关的方程而首次出名。1840 年，他发表了著名的旋转方程，并以他的名字命名，这也许是世界上第一次用一个标量和一个向量来表示旋转。不过后人有时候会误将他的这个方程称为欧拉-罗德里格斯方程。他死后被葬在巴黎的佩雷拉雪兹公墓。

2.2.1.6 单位四元数

四元数法出自于汉密尔顿(Hamilton)出色的研究工作；然而它虽然精妙绝伦，但对那些接触过它的人来说却是纯粹的恶魔，包括克拉克·麦克斯韦(Clark Maxwell)。

开尔文(Kelvin)勋爵，1892

四元数自从 150 年前被汉密尔顿(W. R. Hamilton)发现以来一直存在争议，但它们对机器人学者却有很大的实用价值。四元数是复数的一种扩展，或叫超复数，记作一个标量加上一个向量：

$$\begin{aligned}\mathring{q} &= s + \boldsymbol{v} \\ &= s + v_1\mathrm{i} + v_2\mathrm{j} + v_3\mathrm{k}\end{aligned} \tag{2.16}$$

其中 $s\in\mathbb{R}$，$\boldsymbol{v}\in\mathbb{R}^3$，正交复数 i，j 和 k 定义如下：[②]

$$\mathrm{i}^2 = \mathrm{j}^2 = \mathrm{k}^2 = \mathrm{ijk} = -1 \tag{2.17}$$

我们将一个四元数表示为

$$\mathring{q} = s < v_1, v_2, v_3 >$$

① $S(\cdot)$代表在方程(3.5)和附录 D 中定义的斜对称矩阵。

② 乘法规则与正交向量 $\hat{\imath}$，$\hat{\jmath}$ 和 $\hat{k}$ 的外积相同。

早期反对四元数的一个理由是其乘法不可交换，但正如我们在上面看到的，这种不可交换性正好符合坐标系旋转的情况。除去最初的争论不说，四元数以其格式优雅、功能强大、计算简单已被广泛应用于机器人、计算机视觉、计算机图形学以及航空航天惯性导航领域。

在本书的工具箱中，四元数是由名为 Quaternion 的类来实现的。构造函数将传递的参数转换为四元数，例如：

```
>> q = Quaternion( rpy2tr(0.1, 0.2, 0.3) )
q =
0.98186 < 0.064071, 0.091158, 0.15344 >
```

威廉·罗文·汉密尔顿爵士(Sir William Rowan Hamilton，1805—1865)是爱尔兰数学家、物理学家和天文学家。他是一个语言神童，13 岁时就掌握了古典和现代欧洲语言，以及波斯语、阿拉伯语、印度斯坦语、梵文和马来语。汉密尔顿在 17 岁时自学数学，并发现拉普拉斯天体力学的一处错误。他在都柏林的三一学院(Trinity College，Dublin)度过了他的一生，当他还是该校一个本科生时就被任命为天文学教授和爱尔兰的皇家天文学家。除了四元数，他还为光学、动力学和代数的发展做出贡献。他还写诗并与沃兹沃斯(Wordsworth)通信，沃兹沃斯劝他把更多的精力贡献到数学上。

据传说，最关键的四元数方程，即方程(2.17)，是在 1843 年当汉密尔顿与他妻子在都柏林皇家运河边散步时突然想到的，下面是在布鲁姆桥上一块牌匾上的纪念文字：

威廉·罗文·汉密尔顿爵士于 1843 年 10 月 16 日走过这里时灵感突现，发现了四元数乘法的基本公式 $i^2 = j^2 = k^2 = ijk = -1$，并刻在了这座桥的石头上。

虽然他原来的刻字已不复存在，但这座桥却成为数学家和物理学家的圣地。

为了描述坐标系的旋转，我们使用单位四元数。这些四元数为一个单位大小，即 $|\mathring{q}| = 1$ 或 $s^2 + v_1^2 + v_2^2 + v_3^2 = 1$。例如：

```
>> q.norm
ans =
    1.0000
```

单位四元数具有一个特殊属性，它可以被看作是绕单位向量 $\hat{\boldsymbol{n}}$ 旋转了 θ 角，该旋转与四元数组的关系为

$$s = \cos\frac{\theta}{2},\quad \boldsymbol{v} = \left(\sin\frac{\theta}{2}\right)\hat{\boldsymbol{n}} \tag{2.18}$$

上式与 2.2.1.5 节中的角度-旋转轴表达式相似。

Quaternion 类可以重载一些标准方法和函数。四元数的乘法通过重载乘法运算符调用：[①]

```
>> q = q * q;
```

求一个四元数的共轭为

① 如果将四元数表示为 4 元向量(s, v_1, v_2, v_2)，则四元数之间的乘积可表示为一个矩阵-向量积：

$$\mathring{q} \oplus \mathring{q}' = \begin{pmatrix} s & -v_1 & -v_2 & -v_3 \\ v_1 & s & -v_3 & v_2 \\ v_2 & v_3 & s & -v_1 \\ v_3 & -v_2 & v_1 & s \end{pmatrix} \begin{pmatrix} s' \\ v_1' \\ v_2' \\ v_3' \end{pmatrix}$$

两个正交旋转矩阵的复合需要进行 27 次乘法和 18 次加法，而通过四元数的形式只需要 16 次乘法和 12 次加法，这对嵌入式系统尤其重要。

```
>> q.inv()
ans =
0.98186 < -0.064071, -0.091158, -0.15344 >
```

一个四元数乘以它的逆四元数为

```
>> q*q.inv()
ans =
1 < 0, 0, 0 >
```

或者

```
>> q/q
ans =
1 < 0, 0, 0 >
```

得出一个单位四元数，它代表一个无效旋转。

一个四元数可以用以下方式转化为一个正交旋转矩阵：

```
>> q.R
ans =
    0.9363   -0.2896    0.1987
    0.3130    0.9447   -0.0978
   -0.1593    0.1538    0.9752
```

我们也可以用以下函数绘制一个四元数所指的方向：

```
>> q.plot()
```

绘图结果从形式上看与图 2.12 类似。

在四元数的情况下，广义位姿是 $\xi \sim \mathring{q} \in \mathbb{Q}$，且

$$\mathring{q}_1 \oplus \mathring{q}_2 \mapsto s_1 s_2 - \boldsymbol{v}_1 \cdot \boldsymbol{v}_2, < s_1 \boldsymbol{v}_2 + s_2 \boldsymbol{v}_1 + \boldsymbol{v}_1 \times \boldsymbol{v}_2 >$$

上式被称为四元数积或汉密尔顿积，并有

$$\ominus \mathring{q} \mapsto \mathring{q}^{-1} = s, < -\boldsymbol{v} >$$

这是四元数的共轭。零位姿 $0 \mapsto 1\langle 0,0,0 \rangle$，为单位四元数。一个向量 $\boldsymbol{v} \in \mathbb{R}^3$ 被旋转，表示为 $\mathring{q} \cdot \boldsymbol{v} \mapsto \mathring{q}\mathring{q}(\boldsymbol{v})\mathring{q} - 1$，其中 $\mathring{q}(\boldsymbol{v}) = 0$，$\langle \boldsymbol{v} \rangle$ 被称为纯四元数。

将一个三元向量传递给构造函数，将产生一个纯四元数：

```
>> Quaternion([1 2 3])
ans =
0 < 1, 2, 3 >
```

其中的标量为 0。使用重载的乘法运算符，一个向量可以被一个四元数旋转：

```
>> q*[1 0 0]'
ans =
    0.9363
    0.3130
   -0.1593
```

工具箱中的运算操作相当完整，在线文档中详细描述了 Quaternion 的许多方法和特性。

2.2.2 平移与旋转组合

现在我们回到三维空间中相对位姿的表示，即两个坐标系之间位置和姿态的变化，如图 2.9 所示。前面曾经讨论了几种不同的姿态表示法，我们需要将它们与平移变换相结合，创造出一个完整的相对位姿表示方法。两种最实用的表示方法是：四元数向量对和 4×4 齐次变换矩阵。

对于向量-四元数的情况，有 $\xi \sim (t, \mathring{q})$，其中 $t \in \mathbb{R}^3$ 是坐标系原点相对于参考坐标系的笛卡儿位置，$\mathring{q} \in \mathbb{Q}$ 是坐标系相对于参考坐标系的姿态。

加法定义如下：

$$\xi_1 \oplus \xi_2 = (t_1 + \mathring{q}_1 \cdot t_2, \mathring{q}_1 \oplus \mathring{q}_2)$$

取负数为

$$\ominus \xi = (-\mathring{q}^{-1} \cdot t, \mathring{q}^{-1})$$

一个点坐标向量通过下式在坐标系之间变换：

$${}^X p = {}^X \xi_Y \cdot {}^Y p = \mathring{q} \cdot {}^Y p + t$$

另一种办法是：可以使用一个齐次变换矩阵来表示旋转和转换。推导方法与方程(2.10)描述的二维情况类似，但因为增加了 z 轴，所以进行了扩展：

$$\begin{pmatrix} {}^A x \\ {}^A y \\ {}^A z \\ 1 \end{pmatrix} = \begin{pmatrix} {}^A R_B & t \\ 0_{1\times 3} & 1 \end{pmatrix} \begin{pmatrix} {}^B x \\ {}^B y \\ {}^B z \\ 1 \end{pmatrix}$$

坐标系原点之间的笛卡儿平移向量是 t，姿态的变化由一个 3×3 正交子矩阵 R 表示，其余向量都表示成齐次形式，这样可以写成

$$\begin{aligned} {}^A \tilde{p} &= \begin{pmatrix} {}^A R_B & t \\ 0_{1\times 3} & 1 \end{pmatrix} {}^B \tilde{p} \\ &= {}^A T_B \, {}^B \tilde{p} \end{aligned} \tag{2.19}$$

${}^A T_B$ 是一个 4×4 阶齐次变换。这个矩阵有一个非常特殊的结构，属于特殊的三维欧几里得群，记作 $T \in SE(3) \subset \mathbb{R}^{4\times 4}$。

相对位姿 ξ 的一个具体表现形式是 $\xi \sim T \in SE(3)$，而 $T_1 \oplus T_2 \mapsto T_1 T_2$，属于标准的矩阵乘法。

$$\begin{pmatrix} {}^V x \\ {}^V y \end{pmatrix} = {}^V R_B \begin{pmatrix} {}^B x \\ {}^B y \end{pmatrix} \tag{2.20}$$

前文给出的位姿代数中，其中一条运算规则是 $\xi \oplus 0 = \xi$。在矩阵中，我们知道有 $TI = T$，其中 I 是单位矩阵，因此位姿 $0 \mapsto I$ 就代表单位矩阵。另一个位姿代数的规则是 $\xi \ominus \xi = 0$。我们知道在矩阵中有 $TT^{-1} = I$，这意味着 $\ominus T \to T^{-1}$

$$T^{-1} = \begin{pmatrix} R & t \\ 0_{1\times 3} & 1 \end{pmatrix}^{-1} = \begin{pmatrix} R^{\mathrm{T}} & -R^{\mathrm{T}} t \\ 0_{1\times 3} & 1 \end{pmatrix} \tag{2.21}$$

4×4 的齐次变换在机器人学和计算机视觉中都是非常常用的。本书工具箱也支持对它的操作，并将作为三维位姿描述的一个具体形式在全书中使用。

工具箱中有许多函数可以创建齐次转换。例如可以通过以下函数实现变换的合成：

```
>> T = transl(1, 0, 0) * trotx(pi/2) * transl(0, 1, 0)
T =
    1.0000         0         0    1.0000
         0    0.0000   -1.0000    0.0000
         0    1.0000    0.0000    1.0000
         0         0         0    1.0000
```

函数 transl 创建了一个有平移但无旋转的相对位姿，而函数 trotx 则返回一个绕 x 轴旋转 $\pi/2$ 的 4×4 齐次变换矩阵：旋转部分与 rotx(pi/2)相同，平移部分为零[①]。我们可以将以上函数组合对应的坐标系变化描述如下：首先沿着 x 轴方向前进一个单位长度，然后绕 x 轴旋转 90°，接着再沿新的 y 轴，也就是原来的 z 轴前进一个单位长度。所得矩阵的最后一列，表示了沿原坐标系的 x 轴和 z 轴各平移一个单位长度的结果。从最终位姿矩阵的姿态部分，则可以看出是绕 x 轴旋转的效果。我们可以用如下函数绘制出相应的坐标系：

```
>> trplot(T)
```

要提取矩阵 T 中的旋转矩阵部分，可用

```
>> t2r(T)
ans =
    1.0000         0         0
         0    0.0000   -1.0000
         0    1.0000    0.0000
```

平移部分是一个向量，可用以下函数获得：

```
>> transl(T)'
ans =
    1.0000    0.0000    1.0000
```

2.3 本章总结

在这一章中，我们学会了如何在二维和三维世界中表示点和位姿。点由坐标系的坐标向量表示。一个刚体上的一组点可以用一个坐标系描述，而且其组成点可由相对于刚体坐标系的位移表示。任何坐标系相对于另一个坐标系的位置和方向，可用相对位姿 ξ 表示。相对位姿可以按顺序组合(合成或复合)，并且我们还演示了相对位姿如何进行代数运算的操作方法。一个重要的代数运算法则是运算变量不可交换——相对位姿组合的顺序不可交换。

我们已经在二维和三维的情况下探讨过用正交旋转矩阵表示姿态，并用它的延伸——齐次变换矩阵表示姿态和平移。三维下的旋转微妙且复杂，我们也看过其他表示方法，如欧拉角、横滚-俯仰-偏航角和四元数。这些数学对象中，一些本身就被 MATLAB 支持，而另外一些需要通过工具箱的函数或类才被支持。

本章总结了两个重要的经验。第一个是有许多数学对象都可以用来描述位姿，并在表 2.1 中做出了总结。无所谓好坏——每种方法都有它的长处和短处，我们只需根据手头上要解决的问题选择适合的方法。有时候我们想要一种向量式的描述法，这种情况下(x, y, θ)或$(x, y, z, \boldsymbol{\Gamma})$就比较适用，但这种描述不能简单地进行复合。有时我们只需要描述三维旋转，这种情况下用 $\boldsymbol{\Gamma}$ 或 q 就更合适。本书工具箱支持许多不同表示法之间的转换，如图 2.15 所示。然而在一般情况下，本书后续章节中将主要使用齐次变换的方法。

第二个经验是：坐标系是你的朋友。在许多视觉和机器人问题中，必不可少的第一步是给所有的对象分配坐标系，然后以有向图的形式标明相对位姿，并写出位姿的环路方程。图 2.16 教你如何在纸面外建立一个坐标系，它可以拿在手上移动和旋转——使这些思路更直观。

---

① 许多工具箱函数既可返回正交旋转矩阵，也能返回齐次变换矩阵，只需稍加改变函数名，例如 rotx 和 trotx，以及 rpy2r 和 rpy2tr 等。有些工具箱函数可以接受正交旋转矩阵或齐次变换矩阵，但不接受平移部分，例如 tr2rpy 或 Quaternion。

表 2.1　本章介绍的位姿 ξ 的各种具体描述法总结

描述法	⊕	⊖	平移	旋转	维数	MATLAB
$(x,y,\theta)\in\mathbb{R}\times$			√	√	2D	
$\boldsymbol{T}\in SE(2)$	$\boldsymbol{T}_1\boldsymbol{T}_2$	$\boldsymbol{T}^{-1}$	√	×	2D	`se2(x,y)`
$\boldsymbol{R}\in SO(2)$	$\boldsymbol{R}_1\boldsymbol{R}_2$	$\boldsymbol{R}^{\mathrm{T}}$	×	√	2D	`se2(0,0,th)`
$\boldsymbol{T}\in SE(2)$	$\boldsymbol{T}_1\boldsymbol{T}_2$	$\boldsymbol{T}^{-1}$	√	√	2D	`se2(x,y,th)`
$(x,y,z,\Gamma)\in\mathbb{R}^3\times\mathbb{S}^3$			√	√	3D	
$\boldsymbol{R}\in SO(3)$	$\boldsymbol{R}_1\boldsymbol{R}_2$	$\boldsymbol{R}^{\mathrm{T}}$	×	√	3D	`rotx,roty,...`
$\Gamma\in\mathbb{S}^3$	×			√	3D	`tr2eul,eul2tr`
$\Gamma\in\mathbb{S}^3$	×			√	3D	`tr2rpyl,rpy2tr`
$\boldsymbol{T}\in SE(3)$	$\boldsymbol{T}_1\boldsymbol{T}_2$	$\boldsymbol{T}^{-1}$	√	√	3D	`transl(x,y,z)+trotx(th)`
$\dot{q}\in\mathbb{Q}$	$\dot{q}_1\dot{q}_2$	$\dot{q}^{-1}$	×	√	3D	`quaternion`

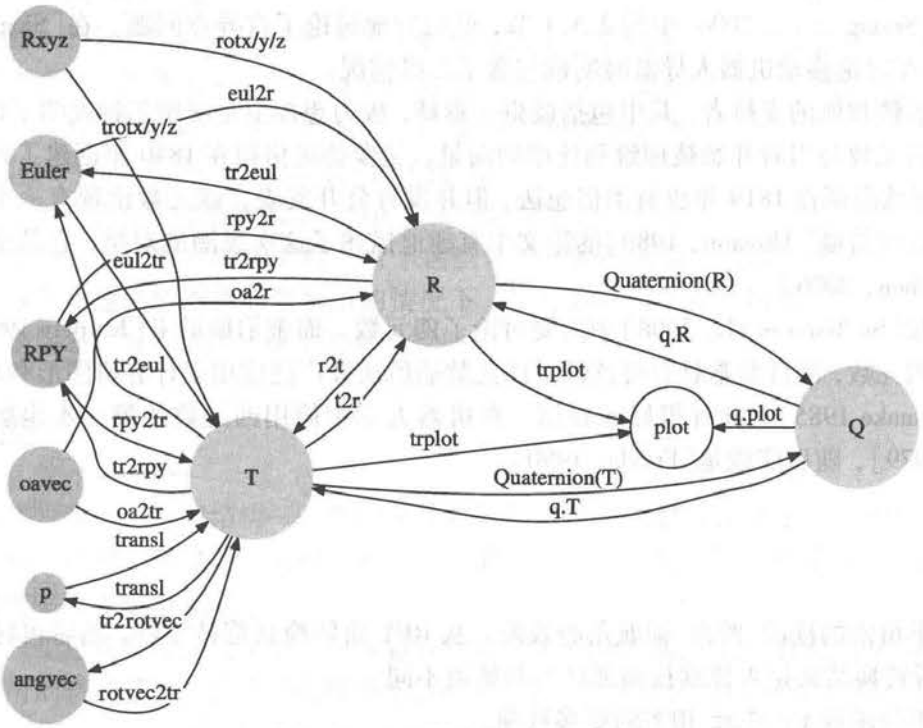


图 2.15　各旋转描述法之间的转换

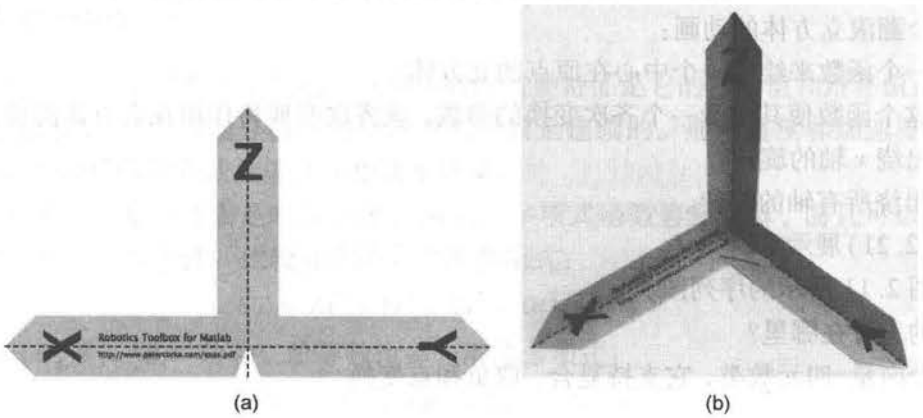


图 2.16　建立自己的坐标系。(a)从 http://www.petercorke.com/axes.pdf 中获取 PDF文件；(b)将它剪下来，沿虚线折叠，然后钉起来

现在，我们已经有了进一步学习的坚实基础。有关的符号已被定义和说明，我们也尝试使用了 MATLAB 来开始工作。下一章将讨论随时间改变的坐标系，然后我们就准备转向讨论具体的机器人问题。

延伸阅读

本章的论述策略是数学加图形的混合方式，内容涵盖了二维和三维的情况，先是抽象的数学描述和运算符，随后是直观的图形说明。机器人学的一些标准教科书，如参考文献[Spong et al., 2006]、[Craig, 2004]、[Siciliano et al., 2008]和[Paul, 1981]，都介绍了三维情况下的齐次变换矩阵，但方法各有所不同。这些书也较好地讨论了其他一些表示法，如角向量法和三角度法。在[Spong et al., 2006]中的 2.5.1 节，就很好地讨论了奇异点问题。在[Siegwart et al., 2011]中，在讨论移动机器人导航时明确包含了二维情况。

汉密尔顿和他的支持者，其中包括彼得·泰特，极力想捍卫是汉密尔顿发明了四元数，他们混淆了四元数与当时开始被理解和使用的向量。是罗德里格斯在 1840 年创建了四元数的关键方程，虽然高斯在 1819 年也有类似想法，但并没有公开发表。四元数出现有一个暴风雨般的开端。在阿特曼[Altmann, 1989]的论文中有趣地描述了这次思潮的对抗，它甚至已被写成小说[Pynchon, 2006]。

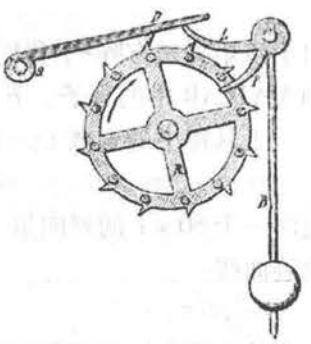
在文献[Siciliano et al., 2008]中简要讨论了四元数。而奎伯斯的书[Kuipers 1999]则全面地介绍了四元数，而且非常具有可读性。四元数插值法被广泛应用于计算机图形和动画，经典论文[Shoemake 1985]对此有很好的介绍。在机器人学中使用四元数的第一本出版物可能是[Tailor, 1979]，随后应该是[Funda, 1990]。

习题

1. 探讨一下负值的横滚-俯仰-偏航角的效果。从 RPY 角转换到旋转矩阵，然后再转换回 RPY 角，看看转换结果是否像欧拉角那样与初始值不同。
2. 探索一下与函数 `trplot` 相关的更多选项。
3. 使用函数 `tranimate` 演示一下相对于各种轴的坐标系平移及旋转动作。
4. 制作一个翻滚立方体的动画：
   a) 编写一个函数来绘制一个中心在原点的立方体。
   b) 修改这个函数使其接受一个齐次变换的参数，该齐次变换将作用在立方体的顶点上。
   c) 动画出绕 x 轴的旋转。
   d) 动画出绕所有轴的旋转。
5. 用方程(2.21)展示 $\boldsymbol{T}\boldsymbol{T}^{-1}=\boldsymbol{I}$。
6. 绘制如图 2.11 所示的序列图。
7. 笛卡儿的头骨在哪里？
8. 创建一个向量-四元数类，它支持复合、取负和点变换。

第3章　时间与运动

完成任何事情都需要时间，这就是时间存在的唯一理由。
阿尔伯特·爱因斯坦



在前面的章节中，我们学习了如何在二维和三维空间中来描述物体的位姿。本章将在前一章的基础上对某些概念进行延展，研究位姿为时间函数的物体。

对于机器人而言，我们希望创建一个让机器人能跟从的随时间变化的位姿，例如机器人末端执行器能沿着既定的位姿轨迹去抓住一个物体。3.1 节讨论了如何生成一个位姿的时间序列，即一条轨迹，以便从初始位姿平滑地转变到最终位姿。

3.2 节讨论了位姿变化率的概念，即位姿的时间导数，以及它与力学中速度和角速度概念的相关性。有了这些知识，我们就能够在已知速度和角速度传感器的测量值的情况下，及时算出一个移动物体的预计位姿。这是所谓的位姿逆问题，也是惯性导航中的基本原理。

3.1　轨迹

一条路径只是一个空间结构——空间中从初始位姿过渡到最终位姿的一个图形。轨迹是具有特定时间属性的一条路径。例如，从 A 到 B 是路径，但如果规定了 10 秒的时间或 2 ms^{-1} 的速度，则变成从 A 到 B 的轨迹。

轨迹的一个重要特征是要平滑——位置和姿态随时间流畅地变化。我们将从如何在一维空间产生平滑的轨迹开始，然后扩展到多维的情况，最后讨论如何生成分段线性轨迹，它能使机器人不间断地经过一系列中间点。

3.1.1　平滑一维轨迹

我们从时间的标量函数开始讨论。这种函数的重要特征是它的初始值和最终值是确定的，而且函数是光滑的。所谓光滑是指它的低阶时间导数是连续的。通常速度和加速度都必须是连续的，有时加速度的导数或加速度率也需要连续。

这种函数的一个常见代表是时间多项式函数。多项式函数容易计算，而且可以方便地提供所需的连续性和边界条件。比较常用的是五次多项式：

$$s(t) = At^5 + Bt^4 + Ct^3 + Dt^2 + Et + F \tag{3.1}$$

其中时间 $t \in [0, T]$。其一阶和二阶导数也是光滑的多项式：

$$\dot{s}(t) = 5At^4 + 4Bt^3 + 3Ct^2 + 2Dt + E \tag{3.2}$$

$$\ddot{s}(t) = 20At^3 + 12Bt^2 + 6Ct + 2D \tag{3.3}$$

轨迹上定义了位置、速度和加速度的边界条件，一般情况下速度和加速度的边界条件均为零①。

在 $t=0$ 和 $t=T$ 时分别将边界条件代入方程(3.1)～方程(3.3)，得到6个方程，我们可以写成如下的矩阵形式：

$$\begin{pmatrix} s_0 \\ s_T \\ \dot{s}_0 \\ \dot{s}_T \\ \ddot{s}_0 \\ \ddot{s}_T \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ T^5 & T^4 & T^3 & T^2 & T & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 5T^4 & 4T^3 & 3T^2 & 2T & 1 & 0 \\ 0 & 0 & 0 & 2 & 0 & 0 \\ 20T^3 & 12T^2 & 6T & 2 & 0 & 0 \end{pmatrix} \begin{pmatrix} A \\ B \\ C \\ D \\ E \\ F \end{pmatrix}$$

由于该矩阵是方阵②，我们可以使用标准的线性代数方法来求解系数向量 (A,B,C,D,E,F)，如 MATLAB 中的算子。五次多项式的加速度是一条光滑的三次曲线，加速度率是一个抛物线。

工具箱中的函数 tpoly 可以生成一个五次多项式轨迹，如式(3.1)所述。例如：

```
>> s = tpoly(0, 1, 50);
```

返回一个 50×1 的列向量，其值在0到1范围内分50个时间步平滑变化。我们可以绘制出该轨迹曲线：

```
>> plot(s)
```

相应的速度和加速度可以通过增加输出选项参数返回：

```
>> [s,sd,sdd] = tpoly(0, 1, 50);
```

速度和加速度分别为 sd 和 sdd。如图3.1(a)所示，可以看出初始和终点的速度和加速度都是零——函数默认值。初始速度和终点速度也可以被设置为非零值：

```
>> s = tpoly(0, 1, 50, 0.5, 0);
```

在这种情况下，初始速度为0.5和最终速度为0。图3.1(b)所示的结果也体现出多项式轨迹的一个问题。非零的初始速度导致多项式轨迹的中间值超过终点值——轨迹中的峰值达到5，超出0到1的范围。

多项式轨迹另一个很实际的问题是它的速度，可以从图3.1(a)的中间一幅看出。其速度在 $t=25$ 时达到最大值，这意味着大多数时间的速度都远小于这个最大值。计算其平均速度为

```
>> mean(sd) / max(sd)
ans =
    0.5231
```

只有最大值的52%。一个真正的机器人关节都有一个额定的最大速度，而且为了使关节运动时间最短，应使其运行在最大速度上的时间尽可能长。因此，我们希望速度曲线的顶部是一条平直线。

一种公认的较理想的选择是采用混合曲线轨迹，即由中间的恒速段平直线加上两侧的加速段和减速段多项式曲线构成的轨迹。回到第一个例子，其混合轨迹是

```
>> s = lspb(0, 1, 50);
```

其中的参数与 tpoly 中的含义相同，轨迹曲线如图3.2(a)所示。轨迹由一条线段(匀速)与

---

①

时间	s	$\dot{s}$	$\ddot{s}$
$t=0$	s_0	$\dot{s}_0$	$\ddot{s}_0$
$t=T$	s_T	$\dot{s}_T$	$\ddot{s}_T$

② 这是选择五次多项式的原因。它有6个系数，能够满足其位置、速度、加速度在起点和终点的总共6个边界条件。

两条抛物线混合而成，因此得名 lspb。术语“混合”(blend)常用来指轨迹加入线性部分。像函数 tpoly 一样，lspb 也可以返回速度和加速度：

```
>> [s,sd,sdd] = lspb(0, 1, 50);
```

这种类型的轨迹由于其速度-时间曲线的形状也被称作梯形轨迹，并普遍应用于工业马达驱动中[①]。

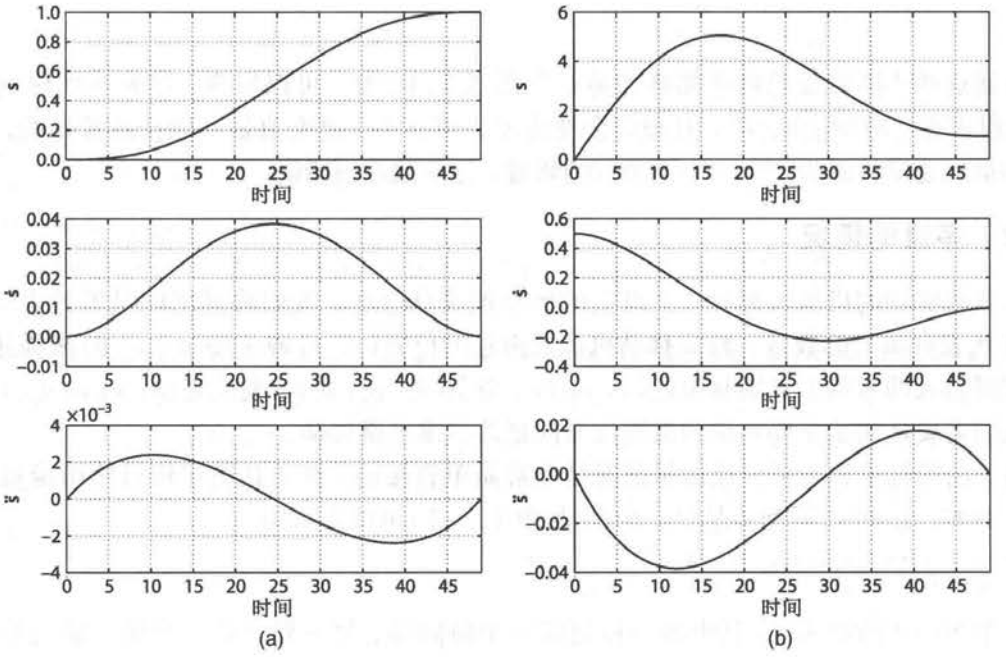


图 3.1　五次多项式轨迹。从上到下依次是位置、速度和加速度随时间变化的曲线。(a)起点和终点速度均为零；(b)起点速度为0.5，终点速度为0

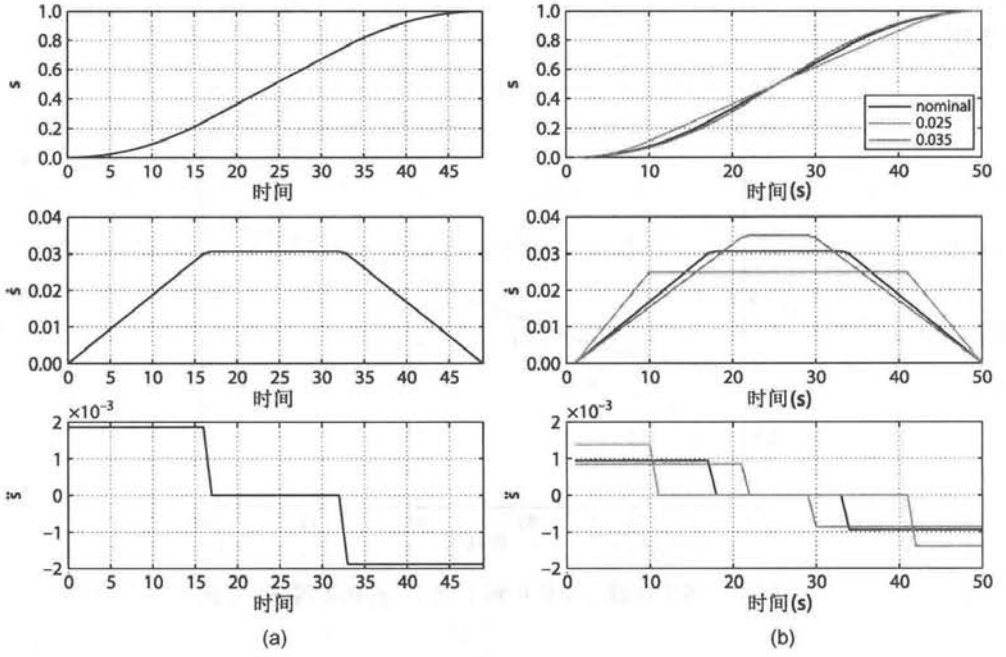


图 3.2　直线与抛物线混合(LSPB)轨迹。(a)直线段为默认速度；(b)直线段为指定速度

① 梯形轨迹的速度是光滑的，但加速度不光滑。

函数 lspb 自己选择的直线段速度为

```
>> max(sd)
ans =
    0.0306
```

但也可以通过第四个输入参数为直线段指定一个速度：

```
>> s = lspb(0, 1, 50, 0.025);
>> s = lspb(0, 1, 50, 0.035);
```

上述这些不同情形的轨迹都叠加显示在图 3.2(b)中。可以发现，随着直线段速度的增大，该段占有的时间会减少[①]，且最终会变为零。事实上，速度值是不能任意选择的，过高或过低的最大速度值都将产生一个不可行的轨迹，且函数返回错误。

3.1.2 多维的情况

大多数实用的机器人都有一个以上的运动轴或自由度。我们将其用向量形式表示为 $\boldsymbol{x} \in \mathbb{R}^M$，$M$ 代表自由度的数目。轮式移动机器人由它的位置(x, y)或位姿(x, y, θ)来描述，而关节臂式机器人的末端工具则有位置(x, y, z)，姿态$(\theta_r, \theta_p, \theta_y)$，或位姿$(x, y, z, \theta_r, \theta_p, \theta_y)$。因此我们需要从初始位姿向量到最终位姿向量的多维平滑运动。

将平滑的标量轨迹扩展成向量情况是非常简单直接的，在工具箱中可以使用函数 mtraj 完成。例如，分 50 个时间步从(0, 2)移动到(1, −1)可以表示为

```
>> x = mtraj(@tpoly, [0 2], [1 -1], 50);
>> x = mtraj(@lspb, [0 2], [1 -1], 50);
```

可得一个 50×2 的矩阵 x，其中每一行对应一个时间步，每一列对应一个轴。输入给 mtraj 的第一个参数是一个函数，即 tpoly 或 lspb，它生成一个标量轨迹。以 lspb 为例，绘制其轨迹为

```
>> plot(x)
```

如图 3.3 所示。

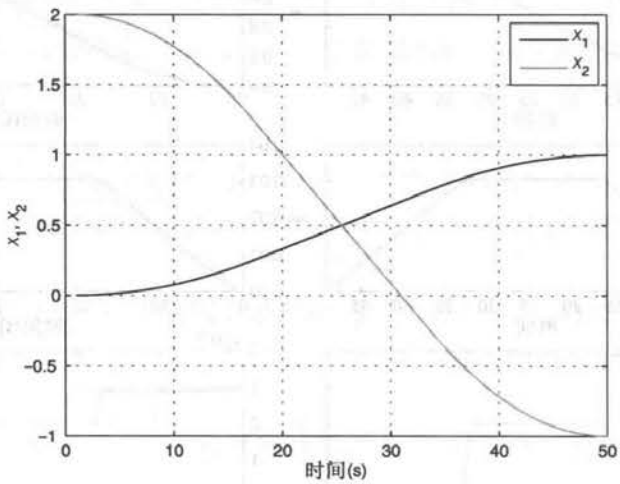


图 3.3 多维运动。x_1从 0 到 1 变化，x_2从 2 到 −1 变化

① 该系统是过约束的，有 5 个约束条件(总时间，起点和终点的位置及速度)，却有 6 个自由度(混合时间，3 个抛物线系数和两个线性系数)。

对于三维空间中的位姿问题，可以考虑先用以下方法将位姿齐次矩阵 $\boldsymbol{T}$ 转换为一个六维向量：

```
>> x = [transl(T); tr2rpy(T)']
```

然后再产生相应的轨迹。不过后面会看到，对于描述姿态的 3 个角度的插值会存在一定的限制。

3.1.3　多段轨迹

机器人应用中经常要求机器人平滑地沿一条路径运动，并不停顿地经过一个或多个中间节点。这样做或许是要在工作空间中避开障碍物，也可能是为了执行一项需要高精度连续轨迹的任务，如在制造业中要涂抹一圈密封胶。

为了使问题一般化，考虑该轨迹含有 N 个中间点 $\boldsymbol{x}_k$，$k \in [1, N]$，因此有 $N-1$ 个运动段。由上节可知，$\boldsymbol{x}_k \in \mathbb{R}^M$ 是位姿的一种向量表示。

机器人从静止状态 $\boldsymbol{x}_1$ 开始运动，到 $\boldsymbol{x}_N$ 结束停止，但在通过（或接近）中间点时不能停顿。生成这个运动轨迹的最大问题是过约束。为了实现速度连续，我们只能舍弃让轨迹能达到每个中间点。在一维情况下这个问题更容易理解，如图 3.4 所示。该运动轨迹包括直线运动段与多项式曲线段的混合，如同 `lspb`，但在这里我们选择了五次多项式，因为它们能够匹配位置、速度和加速度在起点和终点的边界条件。

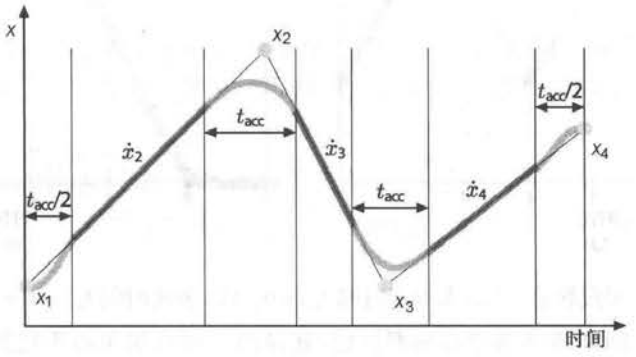


图 3.4　具有 4 个点和 3 个运动段的多段轨迹生成。深色表示恒速运动段，浅色代表加速运动区间

轨迹的第一段从速度为零的初始位姿 $\boldsymbol{x}_1$ 开始加速，然后汇接入指向第二个位姿 $\boldsymbol{x}_2$ 的直线段。汇接时间被设为常数 t_{acc}，在到达 $\boldsymbol{x}_2$ 之前的 $t_{\text{acc}}/2$ 时间，直线轨迹又汇接上一段多项式曲线，持续 t_{acc} 时间后再次汇接到从 $\boldsymbol{x}_2$ 到 $\boldsymbol{x}_3$ 的直线段，然后就一直重复这一过程。可以给每个直线段设定一个恒速值 $\dot{\boldsymbol{x}}_k$。因此，汇接混合中的平均加速度是

$$\ddot{\boldsymbol{x}}_{\text{av}} = \frac{\dot{\boldsymbol{x}}_{k+1} - \dot{\boldsymbol{x}}_k}{t_{\text{acc}}}$$

如果该轴的最大加速能力已知，那么就可以计算出最小混合时间①。

对于多轴的情况，很可能在某个特定运动段其中一些轴要比其他轴需要移动更多距离，而如果各个关节有不同的速度限制时，将会使轨迹生成变得复杂。为解决上述问题，第一步需要确定哪个轴将是完成最慢的，依据是每个轴对应这个运动段的运动距离以及该轴最大可达速

① 该轴会达到其峰值加速度，而不是平均加速度。在五次多项式系数已知时，可以用方程(3.3)计算出混合的峰值加速度。

度。这样可以算出该段的运动持续时间和各轴实际所需的速度。这确保了所有轴在同一时间达到下一个目标 $\boldsymbol{x}_k$。

工具箱函数 mstraj 可以基于中间点矩阵生成一个多段多轴轨迹。例如，有 4 个中间点的两轴运动可以用以下方法生成：

```
>> via = [ 4,1; 4,4; 5,2; 2,5 ];
>> q = mstraj(via, [2,1], [], [4,1], 0.05, 0);
```

函数 mstraj 的第一个参数是中间点矩阵，每点对应矩阵的一行。剩余的参数分别是：每轴的最大速度向量，每段的运动时间向量[①]，起点各轴坐标，采样时间间隔，以及加速时间[②]。函数 mstraj 返回一个矩阵，行对应的是时间每步，列对应的是各个轴。在没有提供输出参数时 mstraj 将绘制出轨迹曲线，如图 3.5(a)所示。从这个例子所给的参数表明，第一个轴的最大速度比第二个轴更大。然而在最后一个运动段，两轴是以相同的速度运动的，因为这段的运动时间是由最慢的轴决定的。

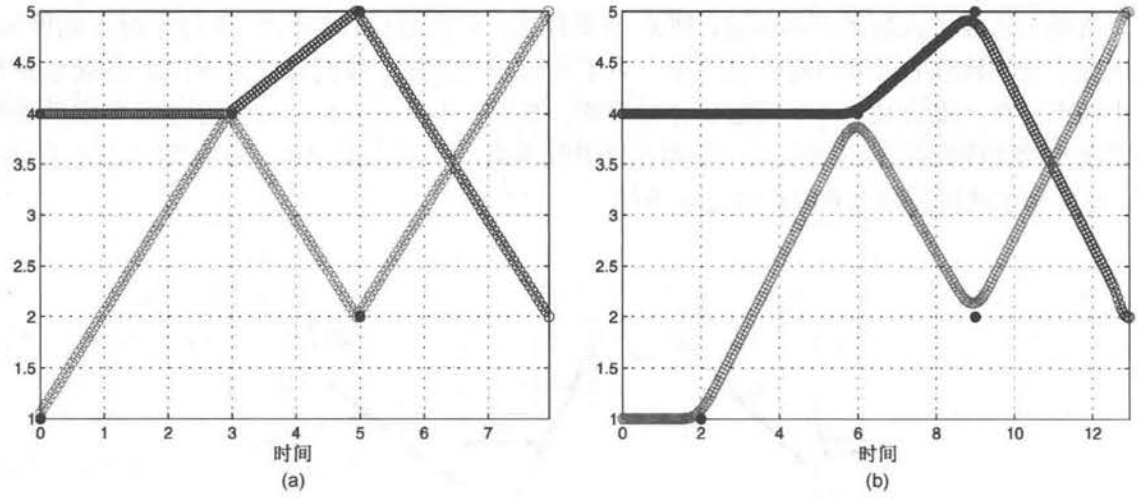


图 3.5 多段轨迹。(a)无加速时间 $t_{acc}=0$；(b)加速时间 $t_{acc}=1$ s。离散时间轨迹点用空心圆圈标记，途经的中间点用实心黑色圆点标记

如果增加加速时间：

```
>> q = mstraj(via, [2 1], [], [4 1], 0.05, 1);
```

轨迹会变得更圆滑，如图 3.5(b)所示。这是因为增加加速时间意味着轨迹上可以有更多的时间来完成多项式曲线段的混合。该函数也接受可选的起点和终点的速度参数，而且 t_{acc} 可以是一个向量，给出每次混合的加速度时间。

记住，这个函数只是简单地对向量代表的位姿进行插值。在这个例子中，向量被假定为笛卡儿坐标向量，但这个函数也适用于欧拉角或横滚-俯仰-偏航角，然而这个函数不是对坐标系旋转进行插值的理想方式。有关对姿态的插值的讨论见下一节。

3.1.4 三维空间姿态插值

在机器人学中，我们经常需要对姿态进行插值。例如，我们需要机器人的末端执行器平滑

① 除非每段的最大轴速度或运动时间可以确认，否则该参数均被设置为 MATLAB 的空矩阵[ ]。

② 给定加速时间时，要圆整为时间增量的整数倍。

地从姿态 ξ_0 改变到 ξ_1。利用第 2 章中介绍的符号，我们需要某个函数 $\xi(s)=\sigma(\xi_0,\xi_1,s)$，其中 $s\in[0,1]$，函数的边界条件为 $\sigma(\xi_0,\xi_1,0)=\xi_0$，$\sigma(\xi_0,\xi_1,1)=\xi_1$，而且 $\sigma(\xi_0,\xi_1,s)$ 要能平滑地经过 s 内定义的一系列中间位姿。如何实现它很大程度上取决于 ξ 的具体表示法。

如果位姿由正交旋转矩阵表示，$\xi\sim\boldsymbol{R}\in SO(3)$，可以考虑用简单的线性插值 $\sigma(\boldsymbol{R}_0,\boldsymbol{R}_1,s)=(1-s)\boldsymbol{R}_0+s\boldsymbol{R}_1$。但这是不可行的，这样插值后得到的通常不再是有效的正交矩阵，因为正交矩阵必须满足的列向量范数为 1 以及列向量之间正交的条件。

一个可行且常用的选择是尝试三角度表示法，如欧拉角或横滚-俯仰-偏航角，$\xi\sim\Gamma\in\mathbb{S}^3$，这样就可以使用线性插值法：

$$\sigma(\boldsymbol{\Gamma}_0,\boldsymbol{\Gamma}_1,s)=(1-s)\boldsymbol{\Gamma}_0+s\boldsymbol{\Gamma}_1$$

例如，我们定义了两个姿态：

```
>> R0 = rotz(-1) * roty(-1);
>> R1 = rotz(1) * roty(1);
```

得到等价横滚-俯仰-偏航角：

```
>> rpy0 = tr2rpy(R0);  rpy1 = tr2rpy(R1);
```

然后分 50 个时间步在它们之间生成一条轨迹：

```
>> rpy = mtraj(@tpoly, rpy0, rpy1, 50);
```

通过动画演示，很容易观察轨迹变化过程[①]：

```
>> tranimate( rpy2tr(rpy) );
```

由于姿态变化较大，可以看出坐标系的旋转轴线沿轨迹的变化。这个动作虽然平滑，但有时看起来也不太协调。另外，如果在所采用的三角度系统中有 ξ_0 或 ξ_1 之一接近奇异点，也会出现问题。

采用单位四元数的插值法只比三角度向量法复杂一点点，它在空间中产生一个绕固定轴旋转的姿态变化。首先，我们利用工具箱函数找到与这两个位姿等价的四元数：

```
>> q0 = Quaternion(R0);
>> q1 = Quaternion(R1);
```

然后对它们进行插值：

```
>> q = interp(q0, q1, [0:49]'/49);
>> about(q)
q [Quaternion] : 1x50 (1656 字节)
```

[②]

得到一个有 50 个四元数对象的向量。我们也可以用动画演示轨迹变化：

```
>> tranimate(q)
```

四元数插值法是通过使用球形线性插值（slerp）来实现的，其中单位四元数要在一个四维超球面上沿一个大圆路径运动。而在三维空间中的结果就是绕定轴的转动。

3.1.5　笛卡儿运动

另一个常见的需求是在 $SE(3)$ 中生成两位姿之间的光滑路径，它同时涉及位置及姿态的变化。在机器人学中这通常被称作笛卡儿运动。

我们将初始和最终位姿都表示成齐次变换矩阵：

---

① rpy 是一个 50×3 矩阵，而 rpy2tr 的结果是 3.1.5 节中提过的 4×4×50 矩阵。

② 括号中字节数的大小在不同的 MATLAB 版本和计算机型号下会有差异。

```
>> T0 = transl(0.4, 0.2, 0) * trotx(pi);
>> T1 = transl(-0.4, -0.2, 0.3) * troty(pi/2)*trotz(-pi/2);
```

工具箱函数 trinterp 提供了沿路径单位化距离 $s \in [0, 1]$ 中的位姿插值。例如，其中间的位姿是

```
>> trinterp(T0, T1, 0.5)
ans =
    0.6667   -0.3333    0.6667         0
   -0.6667   -0.6667    0.3333         0
    0.3333   -0.6667   -0.6667    0.1500
         0         0         0    1.0000
```

其中平移部分是用线性插值，旋转部分是用四元数插值法 interp 进行球形插值。

两个位姿之间 50 个分步的轨迹可用以下方法生成：

```
>> Ts = trinterp(T0, T1, [0:49]/49);
```

参数分别对应的是起点和终点的位姿，以及从 0 到 1 线性变化的路径长度。由此产生的轨迹 Ts 是一个三维矩阵：

```
>> about(Ts)
Ts [double] : 4x4x50 (6400 字节)
```

代表的是每个时间分步(第三个指数 50)对应的齐次变换矩阵(前两个指数 4×4)。其中，路径上第一点的齐次变换是：

```
>> Ts(:,:,1)
ans =
    1.0000         0         0    0.4000
         0   -1.0000         0    0.2000
         0         0   -1.0000         0
         0         0         0    1.0000
```

最简单直观的表现方法仍然是通过动画：

```
>> tranimate(Ts)
```

它将展现出坐标系从位姿 T1 平移、旋转到位姿 T2 的变化过程。

该轨迹的平移部分由以下方式获得：

```
>> P = transl(Ts);
```

它将以矩阵形式返回轨迹的笛卡儿位置坐标：

```
>> about(P)
P [double] : 50x3 (1200 字节)
```

可以看出，P 有 50 行 3 列，每一行对应每个时间分步上的位置向量。

位置向量的变化可以用 plot 函数画出：

```
>> plot(P);
```

如图 3.6(a)所示。图 3.6(b)还展示了以横滚-俯仰-偏摆角格式画的姿态变化曲线：

```
>> rpy = tr2rpy(Ts);
>> plot(rpy);
```

从图中看到，位置坐标随时间的变化既平滑又呈线性，而姿态随时间的变化也平滑，但不是线性的[①]。

---

① 横滚-俯仰-偏航角之所以不呈现时间线性，因为它们是对线性变化的四元数进行非线性变换的结果。
第一点后横滚角出现的不连续，是由于角度在此刻刚好从 $-\pi$ 到 $+\pi$ 绕圆一周。
最后两个点之间的不连续性，是因为对于横滚-俯仰-偏航角，其最终姿态是一个奇异点。

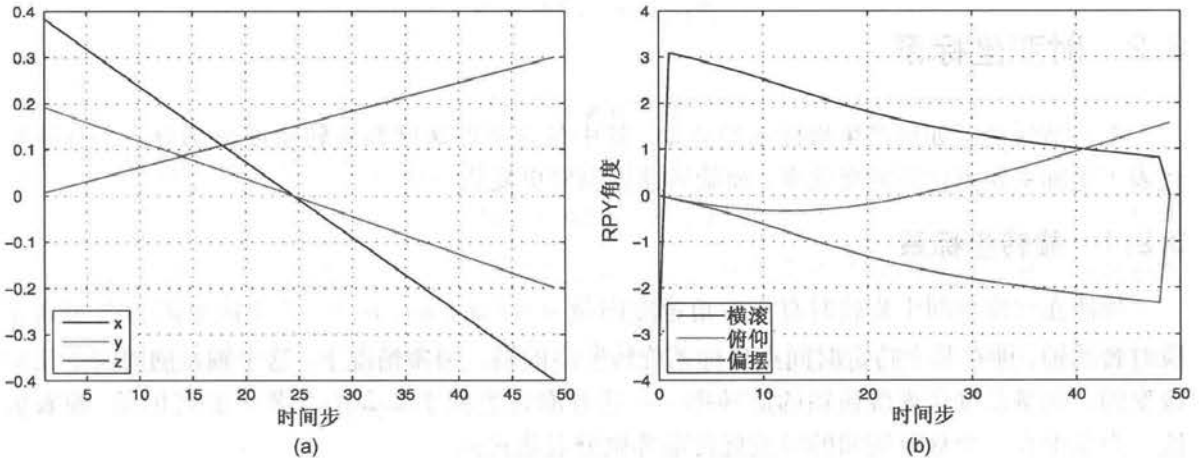


图 3.6　笛卡儿运动。(a)笛卡儿位置时间曲线；(b)横滚-俯仰-偏摆角时间曲线

然而，平移运动在第一个和最后一个点上的速度和加速度是不连续的。问题在于虽然轨迹在空间中是平滑的，但沿轨迹的步距 s 在时间上并不平滑。路径起点上的速度值从零跳跃到有限值，然后在终点又突降至零——没有相应的起始加速和结束减速。我们可以用前面讨论过的标量函数 `tpoly` 和 `lspb` 来创建一个时间上平滑的 s，这样沿路径的运动也平滑了。我们只需将传递给 `trinterp` 的第三个参数更改为一个沿路径的单位化距离向量即可：

```
>> Ts = trinterp(T0, T1, lspb(0,1, 50) );
```

这样做对于轨迹是不变的，但这时坐标系会沿路径逐渐加速到一个恒定速度，然后结束时再减速，这一点可以从图 3.7(b)中轨迹姿态部分较光滑的曲线反映出来。工具箱中还提供了一个方便的速记函数 `ctraj` 来进行轨迹插值：

```
>> Ts = ctraj(T0, T1, 50);
```

其中的参数是初始和最终的位姿，以及分步数量。

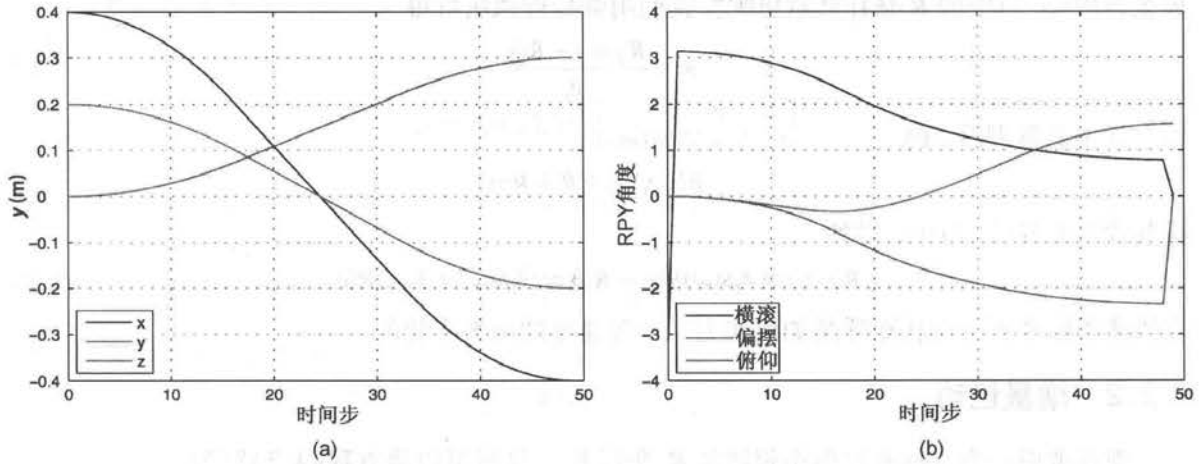


图 3.7　具有 LSPB 路径距离分布的笛卡儿运动。(a)笛卡儿位置的时间曲线；(b)横滚-俯仰-偏摆角的时间曲线

3.2 时变坐标系

上一节讨论了如何产生坐标系的运动，其中包含平移速度和旋转速度两部分。平移速度代表了坐标系原点位置的变化率，而旋转速度则要更复杂一些。

3.2.1 旋转坐标系

物体在三维空间中旋转时有一个角速度向量 $\omega = (\omega_x, \omega_y, \omega_z)$。这个向量的方向定义了瞬时转动轴，即在某个特定时间点坐标系旋转所绕的轴。通常情况下，这个轴是随时间变化而改变的。向量长度代表绕该轴的旋转率——这方面它类似于 2.2.1.5 节中旋转的角-轴表示法。力学中有一个众所周知的时变旋转矩阵微分表达式：

$$\dot{\boldsymbol{R}}\langle t\rangle = \boldsymbol{S}(\boldsymbol{\omega})\boldsymbol{R}\langle t\rangle \tag{3.4}$$

其中，$\dot{\boldsymbol{R}}\langle t\rangle \in SO(2)$或$SO(3)$，$\boldsymbol{S}(\cdot)$是一个斜对称矩阵[①]，其三维情况下的具体形式如下：

$$\boldsymbol{S}(\boldsymbol{\omega}) = \begin{pmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{pmatrix} \tag{3.5}$$

它的性质可参见附录 D。利用工具箱函数，可以得到

```
>> S = skew([1 2 3])
S =
     0    -3     2
     3     0    -1
    -2     1     0
```

工具箱函数 vex 具有逆解的功能：将斜对称矩阵转换成一个向量：[②]

```
>> vex(S)'
ans =
     1     2     3
```

那么方程(3.4)中的 $\dot{\boldsymbol{R}}$ 是什么意思呢？我们用微分近似法可得

$$\dot{\boldsymbol{R}} \approx \frac{\boldsymbol{R}\langle t+\delta_t\rangle - \boldsymbol{R}\langle \delta t\rangle}{\delta_t} \tag{3.6}$$

将上式重新整理后，得

$$\boldsymbol{R}\langle t+\delta_t\rangle \approx \delta_t\dot{\boldsymbol{R}} + \boldsymbol{R}\langle t\rangle$$

将其代入方程(3.4)中，得到

$$\boldsymbol{R}\langle t+\delta_t\rangle \approx \delta_t\boldsymbol{S}(\boldsymbol{\omega})\boldsymbol{R}\langle t\rangle + \boldsymbol{R}\langle t\rangle \approx \left(\delta_t\boldsymbol{S}(\boldsymbol{\omega}) + \boldsymbol{I}_{3\times 3}\right)\boldsymbol{R}\langle t\rangle \tag{3.7}$$

它描述了标准正交旋转矩阵是如何作为一个角速度的函数变化的。

3.2.2 增量运动

现在考虑一个坐标系经微小旋转从 $\boldsymbol{R}_0$ 变到 $\boldsymbol{R}_1$。这时可以将方程(3.7)写为

① 也称作角速度矩阵。

② 在斜对称矩阵中，每个元素都出现两次，但符号相反。在一些算法中，我们计算得到的是斜对称矩阵的近似值，这时符号相反的两个元素其绝对值可能有些许不同，所以 vex 取两个元素的平均值。

$$\boldsymbol{R}_1 = (\delta_t \boldsymbol{S}(\boldsymbol{\omega}) + \boldsymbol{I}_{3\times3})\boldsymbol{R}_0$$

将上式重新整理后，得

$$\delta_t \boldsymbol{S}(\boldsymbol{\omega}) = \boldsymbol{R}_1 \boldsymbol{R}_0^{\mathrm{T}} - \boldsymbol{I}_{3\times3}$$

再对其两侧均使用运算符 vex，即求 $\boldsymbol{S}(\cdot)$的逆，得到

$$\delta_\Theta = \mathrm{vex}(\boldsymbol{R}_1 \boldsymbol{R}_0^{\mathrm{T}} - \boldsymbol{I}_{3\times3}) \tag{3.8}$$

其中，$\delta_\Theta = \delta_t \boldsymbol{\omega}$ 是一个三维向量，单位是角度，它表示一个绕世界坐标系的 x、y 和 z 三轴的无穷小转动[①]。

我们曾经一再强调过旋转变换的不可交换性，但对于无穷小角度变化的乘法来说却是可交换的。可以用具体数字的例子证明如下：

```
>> Rdelta = rotx(0.001)*roty(0.002)*rotz(0.003)
ans =
    1.0000   -0.0030    0.0020
    0.0030    1.0000   -0.0010
   -0.0020    0.0010    1.0000
```

以上结果是与下面的计算相同的(取 4 位有效数字)：

```
>> roty(0.002) * rotx(0.001)*rotz(0.003)
ans =
    1.0000   -0.0030    0.0020
    0.0030    1.0000   -0.0010
   -0.0020    0.0010    1.0000
```

使用方程(3.8)，可以重新获得上面矩阵对应的微小旋转角 δ_Θ：

```
>> vex( Rdelta - eye(3,3) )'
ans =
    0.0010    0.0020    0.0030
```

现在给出两个差异极小的位姿 ξ_0 和 ξ_1，可以用一个六维向量来表示它们间的差异：

$$\boldsymbol{\delta} = \Delta(\xi_0, \xi_1) = (\boldsymbol{\delta}_d, \boldsymbol{\delta}_\Theta) \tag{3.9}$$

它由位移增量和旋转增量两部分组成。$\boldsymbol{\delta} \in \mathbb{R}^6$ 的值实际上是用空间速度乘以 δ_t 得到的，其中空间速度将在第 8 章中进一步讨论。如果位姿都用齐次变换矩阵的形式表示，那么位姿差异是

$$\boldsymbol{\delta} = \Delta(\boldsymbol{T}_0, \boldsymbol{T}_1) = \begin{pmatrix} \boldsymbol{t}_1 - \boldsymbol{t}_0 \\ \mathrm{vex}(\boldsymbol{R}_1 \boldsymbol{R}_0^{\mathrm{T}} - \boldsymbol{I}_{3\times3}) \end{pmatrix} \tag{3.10}$$

其中，$\boldsymbol{T}_0 = (\boldsymbol{R}_0, \boldsymbol{t}_0)$，$\boldsymbol{T}_1 = (\boldsymbol{R}_1, \boldsymbol{t}_1)$。在工具箱中它可以用函数 `tr2delta` 求得。

方程(3.9)的逆运算为

$$\xi = \Delta^{-1}(\boldsymbol{\delta}) \tag{3.11}$$

而齐次变换表示法的逆运算是

$$\boldsymbol{T} = \begin{pmatrix} \boldsymbol{S}\boldsymbol{\delta}_\Theta & \boldsymbol{\delta}_d \\ \boldsymbol{0}_{3\times1} & 0 \end{pmatrix} + \boldsymbol{I}_{4\times4} \tag{3.12}$$

在工具箱里它用函数 `delta2tr` 求解。

举例说明：

---

① 这是在世界坐标系中，因为是$(\delta_t \boldsymbol{S}(\boldsymbol{\omega}) + \boldsymbol{I}_{3\times3})$左乘 $\boldsymbol{R}_0$。

```
>> T0 = transl(1,2,3)*trotx(1)*troty(1)*trotz(1);
>> T1 = T0*transl(0.01,0.02,0.03)*trotx(0.001)*troty(0.002)*trotz(0.003)
T1 =
    0.2889   -0.4547    0.8425    1.0191
    0.8372   -0.3069   -0.4527    1.9887
    0.4644    0.8361    0.2920    3.0301
         0         0         0    1.0000
```

函数 Δ(·)的计算是由工具箱函数 tr2delta 完成的：

```
>> d = tr2delta(T0, T1);
>> d'
ans =
    0.0191   -0.0113    0.0301    0.0019   -0.0011    0.0030
```

该位姿变化(位移)由相对于世界坐标系的平移增量和旋转增量组成。给定这个位移量以及初始位姿，可求得最终位姿为

```
>> delta2tr(d) * T0
    0.2889   -0.4547    0.8425    1.0096
    0.8372   -0.3069   -0.4527    1.9859
    0.4644    0.8361    0.2920    3.0351
         0         0         0    1.0000
```

它非常接近于上面 T1 给出的真实值，误差是由于位移量不是无穷小这一事实所致。更多关于六维向量的位移将会在下一节和第 8 章中多次用到。

3.2.3 惯性导航系统

惯性导航系统是一个“黑匣子”，用来测算它相对于惯性参考系(宇宙)的速度、方向和位置。它无需外部信号输入，如来自卫星的无线电信号，这使得它非常适合用于潜艇、航天器和弹道导弹。惯性导航系统主要测量其加速度和角速度，并整合出它们的时间关系。

早期的惯性导航系统，如图 2.13 所示，使用机械万向支架来使安装在稳定平台上的加速度传感器保持相对于恒星的一个固定状态。这个稳定平台上测量出的加速度经积分就可获得平台的速度，再积分就获得其位置。为了在数小时或数天时间内实现精确的位置估计，万向支架和陀螺仪必须具有非常好的品质，才能使得稳定平台不产生漂移，且加速度传感器也要非常精确。

背景材料：惯性导航系统(INS)由一台计算机和运动传感器组成，它通过航迹推算法不断计算运动物体的位置、方向和速度。它不需要外部参考信号，这对于不能使用无线导航辅助设备或者需要免受无线电干扰的交通工具(如潜水艇或航天器)是非常重要的。

陀螺仪是测量绕某特定轴角速度的传感器。早期的旋转传感器实际上采用了一种旋转盘——像儿童的玩具，但今天陀螺仪一般泛指所有的角速度传感器。为了测量角速度，一般使用三轴装配的陀螺仪——3 个陀螺仪的测量轴相互正交。现代陀螺仪的设计基于了各种各样的物理原理，如在环形激光和光纤陀螺仪中的微小振动梁或相对论效应。加速度计是一个测量沿某特定轴线性加速度的传感器。加速度计的工作原理是测量传感器内一个小质量块受到的力。运动物体的加速度向量使用三轴装配的加速度计测量。集成了陀螺仪和加速度计这些传感器的系统统称为一个惯性测量单元(IMU)。

许多重要的研发工作都是在查尔斯·斯塔克·德雷珀(Charles Stark Draper)领导下的麻省理工学院仪器实验室(MIT Instrumentation Laboratory)中开展的。1953 年，他们开发了空间惯性参考设备(SPIRE)系统，重 1200 公斤，它引导一架 B-29 轰炸机在 12 小时内从马萨诸塞州飞到洛杉矶，飞行全程没有飞行员的帮助，德雷珀也在飞机上。1954 年，第一个独立的潜艇导航系统(SINS)开始服役。仪器实验室还开发了阿波罗导航计算机，在 1969 年，这个 1 立方英尺大小的计算机引导阿波罗登月舱成功登上了月球表面。

今天，基于光纤陀螺仪的高性能惯性导航系统应用广泛，重量只有约 1 公斤，而采用低成本 MEMS 技术的陀螺仪重量可小于 100 克，价格也只需几十美元。

而在现代的捷联式惯性测量系统中，加速度传感器和角速度传感器是与飞行器刚性连接的。3 个相互正交安装的陀螺仪可以测量出角速度 ω 的 3 个分量，再使用方程(3.7)就可以不断计算出${}^0\boldsymbol{R}_B$，即固联于飞行器的坐标系$\{B\}$相对于恒星坐标系$\{0\}$的姿态。

方程(3.7)的一种离散时间表达式为

$$\boldsymbol{R}\langle k+1\rangle = \delta_t \boldsymbol{S}(\boldsymbol{\omega})\boldsymbol{R}\langle k\rangle + \boldsymbol{R}\langle k\rangle \tag{3.13}$$

它可以用于对位姿的变化进行数值积分，从而估计飞行器的姿态。测出的飞行器坐标系加速度${}^B\boldsymbol{a}$可以用下式旋转到惯性参考坐标系：

$${}^0\boldsymbol{a} = {}^0\boldsymbol{R}_B\,{}^B\boldsymbol{a}$$

然后对它积分两次就可以得到在惯性坐标系中飞行器的位置。实际惯性测量系统的工作采样频率很高，通常为几百赫兹，而且采用高阶数值积分法，而不是简单的如方程(3.13)的矩形积分法。

在方程(3.13)中，我们将矩阵$\delta_t \boldsymbol{S}(\boldsymbol{\omega})\boldsymbol{R}(t)$与一个正交旋转矩阵相加，但这存在问题，因为其结果不是一个正交矩阵。然而如果被加项很小，那么结果会接近于正交矩阵，并且可以对它进行矫正。这一过程称为正交化，其强化了对正交矩阵中元素的限制。它包括以下步骤，其中$\boldsymbol{c}_i$是$\boldsymbol{R}$的第i列。首先假定第三列是正确的：①

$$\boldsymbol{c}_3' = \boldsymbol{c}_3$$

然后，使第一列正交于后两列：

$$\boldsymbol{c}_1' = \boldsymbol{c}_2 \times \boldsymbol{c}_3'$$

但后两列不一定正交，所以构造一个新的第二列向量：

$$\boldsymbol{c}_2' = \boldsymbol{c}_1' \times \boldsymbol{c}_3'$$

最后每一列都规范化为单位大小：

$$\boldsymbol{c}_i'' = \frac{\boldsymbol{c}_i'}{|\boldsymbol{c}_i'|},\quad i = 1\cdots 3$$

工具箱中的规范化由以下函数实施：

```
>> T = trnorm(T);
```

它和 2.2.1.4 节中讨论的用两个单位向量表示位姿的问题相当类似。在一个使用方程(3.13)的姿态估算系统中，$\boldsymbol{R}$应在每一步积分后规范化②。

另一选择是用单位四元数方法，而且通常情况下它会使运算得以简化。等价于方程(3.4)的四元数，其微分被定义为

$$\dot{\mathring{q}} = \frac{1}{2}\mathring{q}(\omega)\mathring{q} \tag{3.14}$$

它可以通过工具箱函数 dot 完成：

```
>> qd = q.dot(omega);
```

四元数变化率的积分通过下式获得：

$$\mathring{q}\langle k+1\rangle = \delta_t \dot{\mathring{q}} + \mathring{q}\langle k\rangle$$

① 这就是为什么惯性导航系统的工作采样频率要高，而δ_t很小的原因。

② 因为有限精度计算产生的误差累积，旋转矩阵的不断复合运算会使矩阵变得非规范化。

正如方程(3.7)，上式中的加法也存在问题，其结果可能不再是一个单位四元数。要规范化一个四元数，先要计算该四元数的范数，再用其每个元素直接除以它的范数：

$$\mathring{q}\langle k+1\rangle' = \frac{\mathring{q}\langle k+1\rangle}{\left|\mathring{q}\langle k+1\rangle\right|}$$

背景材料：查尔斯·斯塔克·德雷珀博士(1901—1987)是一个美国的科学家和工程师，被誉为“惯性导航之父”。他出生在密苏里的温莎，就读于密苏里大学，在1922年于斯坦福大学获得了他的心理学学士学位，然后，分别在1928年和1938年于麻省理工学院获得电化学工程学士、物理学硕士和博士学位。他在麻省理工学院时已经开始教学，1939年正式成为航空工程专业的终生教授。他是麻省理工学院仪器实验室的创始人和主管，该实验室对惯性导航的理论和实践做出了重要贡献，满足了冷战和空间计划的需要。

德雷珀在1961年被评为时代杂志年度人物之一，并于1981年入选国家发明家名人堂。为了纪念他，仪器实验室改名为查尔斯·德雷珀实验室(CSDL)。(左边的照片由查尔斯·斯塔克·德雷珀实验室股份有限公司提供。)

在工具箱中使用以下函数实现四元数规范化：

```
>> q = q.unit();
```

相比于正交旋转矩阵法，四元数法更常用于捷联式惯性导航系统的旋转更新方程。当然，两种方法的结果是相同的，只是四元数法的计算成本明显更少。

3.3 本章总结

本章从两个视角分别探讨了随时间变化的位姿。第一个是创建一系列机器人要跟从的位姿，即轨迹。轨迹的一个重要特点是要平滑——位置和姿态随时间连续变化。我们先从在一维情况下生成平滑轨迹的方法开始，然后扩展到多维情况，最后是要求经过一系列中间点的分段线性轨迹。

第二个视角是研究正交旋转矩阵的时间导数，以及它们与力学中诸如速度和角速度这些概念的联系。这使我们可以求解位姿的逆问题，即先取得传感器测量值，再估算出一个移动物体的位姿——惯性导航原理。我们还介绍了无穷小运动 δ，它和空间速度有关，我们将在第8章再次涉及。

旋转插值和角速度积分都用到正交旋转矩阵法和四元数法。两种方法的结果是等价的，但四元数法的公式更简洁、计算更有效。

延伸阅读

最早从事机械臂笛卡儿轨迹生成问题研究的是保罗[Paul1972, 1979]和泰勒[Taylor 1979]。保罗还研究了多段轨迹的问题[Paul 1979, 1981]，而劳埃德和海华则讨论了分段间的过渡或混合的概念[Lloyd and Hayward 1991]。这些早期的文献以及其他文献，都收录在布雷迪等人编辑的《机器人运动》(*Robot Motion*)一书中[Brady et al. 1982]。文献[Spong et al., 2006]中则详细描述了多项式和LSPB轨迹，而在文献[Siciliano et al. 2008]和[Graig 2004]中则详细介绍了多段轨迹方法。

有关正交旋转矩阵、反对称矩阵与角速度之间的关系，可参考文献[Spong et al., 2006]，其中有详细的论述。

在格劳弗斯的书中介绍了惯性导航原理[Groves, 2008]，该书还涉及了全球定位系统(GPS)和其他无线电定位系统，这些内容将在本书第二部分中介绍。《数字阿波罗》(Digital Apollo)[Mindell, 2008]这本书非常通俗易懂，它讲述了惯性导航系统在阿波罗登月计划中发展的故事。在论文[Croke et al., 2007]中介绍了惯性传感器的原理，它在功能上等效于哺乳动物内耳中对保持平衡起关键作用的传感器。

习题

1. 对于一个从 0 到 1 分 50 步的 `tpoly` 轨迹，探讨不同的起点和终点速度对它造成的影响，包括好的和坏的。在什么情况下五次多项式会超调？为什么？
2. 对于一个从 0 到 1 分 50 步的 `lspb` 轨迹，探讨指定速度值对恒定速度段产生的影响。可能的最低和最高速度界限是多少？
3. 对于一个从 0 到 1 的轨迹，并给定其最大可能速度为 0.025，试比较每一个 `tpoly` 和 `lspb` 轨迹所需的分步数。
4. 使用 `tranimate` 函数来比较分别用四元数、欧拉角和横滚-俯仰-偏航角表示的旋转插值。提示：使用四元数的 `interp` 函数，以及结合 `tr2eul` 和 `eul2tr` 函数的 `mtraj` 函数。对于终点姿态是一个奇异点的情况重复进行插值。发生了什么？
5. 上题中，对于中间点经过一个奇异点的情况重复插值。发生了什么？
6. 研究出一种方法来定量比较不同姿态插值方法的表现。提示：在单位球上绘制单位向量 $\hat{z}$ 的运动轨迹。
7. 对于书中 `mstraj` 函数的例子(第 44 页)：
   a)以不同的起点和终点速度重复该例子。
   b)研究加速时间增长的影响。绘制出总时间随加速时间变化的函数。
8. 修改 `mstraj` 函数，使得在确定分段时间时必须考虑加速度限制。
9. 设计出一个惯性测量系统。首先创建一个随时间变化的角速度信号，例如：

```
>> t = [0:0.01:10]';
>> w = [0.1*sin(t) 0.2*sin(0.6*t) 0.3*sin(0.4*t)];
```

   a)估算出每个时间步上的旋转矩阵，然后用动画演示出来。
   b)使用四元数法重复 a)。
   c)加入少量的高斯噪声 $w = w + \text{randn}(\text{size}(w)) \times 0.001$，然后重复 a)和 b)。会产生什么样的变化？
   d)研究增大噪声后的效果。

第二部分

移动机器人

第4章　移动机器人载体
第5章　机器人导航
第6章　机器人定位

在这部分中，我们将探讨移动机器人，属于机器人中具有在较大范围环境里运动能力的一类。下面的图片展示了各种移动机器人，有的能在地面上移动，有的能在水面上运动，也有能在空中翱翔的飞行机器人，还有能在水中游动的水下机器人。它们体现了机器人平台的多样性，所谓的机器人平台是指机器人的物理结构及其驱动方式。

然而这些移动机器人在其功能和作业方式上都比较相似。对于一台移动机器人，其最重要的功能之一就是要能运动到某个地方。这个地方可能是环境中具有某些特定特征的场所，例如接近一个发光源，也有可能就是地图上某个几何坐标位置或参考点。但无论哪种情况，机器人都必须要经过一条路径才能到达它的目的地，而在这个过程中机器人会遭遇到各种挑战，如行进途中的障碍物、地图信息不完整或根本就没有地图。

应对上述挑战的策略之一是让机器人用非常简单的方法感知周围环境，并对感知的情况做出相应行动。例如，诞生于20世纪40年代的机器乌龟艾尔西(Elsie)就具有对环境的反应能力，它能够在预先没有明确的路径规划或光源位置的情况下自己找出目标光源。与简单的反应式方法不同，20世纪60年代研制的机器人沙克(Shakey)则采用了另一种策略，它依靠3D感知传感器可以生成一张周围环境的地图，然后根据地图合理地规划出一条能到达其终点的路径。

以上两种方法实际上揭示了实现移动机器人导航的两种相反的途径。反应式系统更简单，响应也更快，因为其感知信息是直接与动作对应的，即无需花费资源来保存和维护周围环境的信息，更不用具备对环境信息的推理能力，在自然界中这种反应式策略更多地被昆虫这类简单生物使用。而采取获得地图并对它解析推理的系统则需要更多的资源，但能够完成更复杂的任务。在自然界里，这种策略通常被哺乳动物这类高级生物采用。

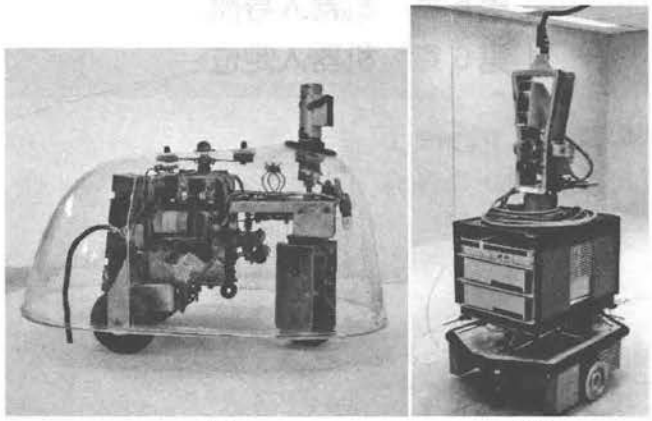


图 II.1 (a)机器乌龟艾尔西。1948年由英国布里斯托(Bristol)大学的布尔登研究所(Burden Institute)研制。现在收藏于史密森学会(Smithsonian Institution)，但未公开展示(照片由Reuben Hoggett藏馆提供)；(b)沙克机器人。1968年由SRI国际研究所研制，现收藏于美国山景城(Mountain View)的计算机博物馆(Computer Museum)(照片由SRI提供)

世界上第一款商业化应用的移动机器人诞生于20世纪80年代，即自动引导小车(AVG)，它们被开发出来在工厂里运输各种物料。现在，自动引导小车的技术已经非常成熟了。这种能大范围移动的轮式车辆通常依靠固定参照物来实现移动方向的引导，如地板上喷画的一条彩色引导线，或埋在地板下能发射射频信号的电缆，或者是墙上安装的条形码路标。近十年来，移动机器人技术取得了长足进步，已不需要专门的引导设施就能实现自主移动。典型例子是用于火星探测的自主移动机器人，DARPA大挑战赛中的自动汽车[Buehler et al. 2007,

2010)]，以及低成本的小型机器人真空吸尘器。一些户外作业机器人系统，如矿井运输车、港口集装箱搬运车、农用自动驾驶拖拉机等，现在也都进入商业应用。移动机器人并不局限在地面上，近年来在其他环境中作业的移动机器人也大量涌现，如无人飞行器(UAV)、自主水下航行器(AUV)，以及被称为自主水面航行器(ASV)的机器人船只。

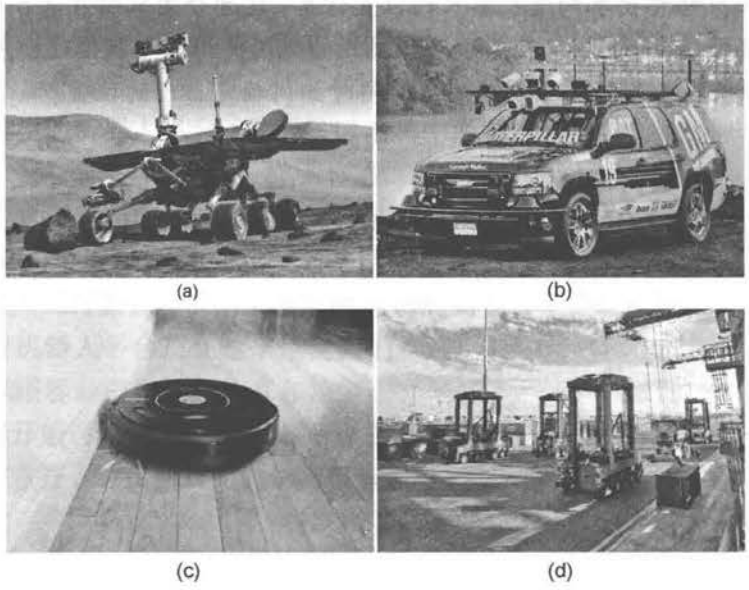


图 II.2　几款地面移动机器人。(a)火星探测机器人，2004(图片由 NASA/JPL/康奈尔大学提供)；(b)波什(Boss)机器人，塔坦(Tartan)赛车队的自动小车，2007年获Darpa城市大挑战赛(Urban Grand Challenge)的冠军(卡耐基·梅隆大学)；(c)屋霸(Roomba)机器人真空吸尘器，2008(照片由iRobot公司提供)；(d)布里斯班港使用的一台集装箱搬运自动跨运车，2006(照片由布里斯班港公司提供)

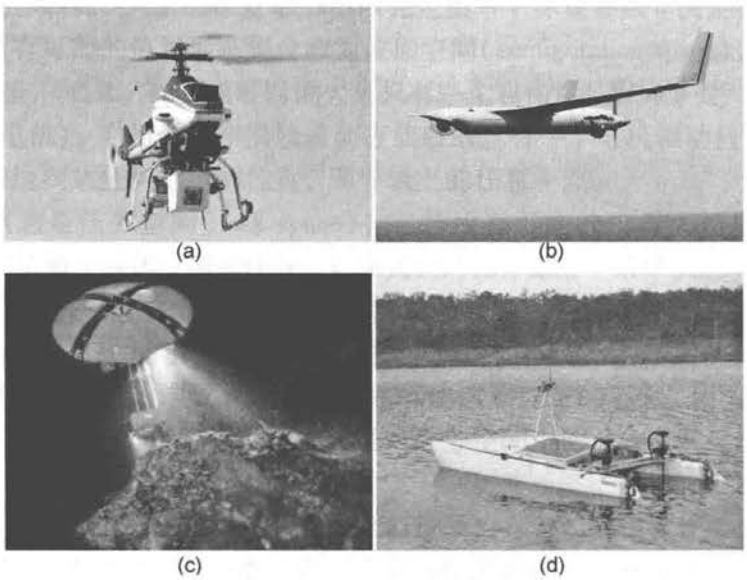
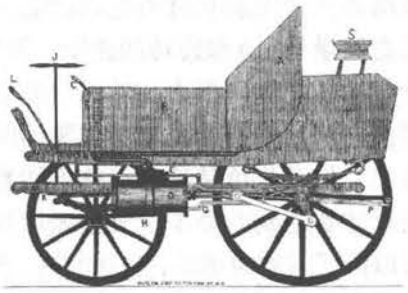


图 II.3　几款空中和水中移动机器人。(a)雅马哈 RMAX 直升机，具有 3 米的桨径；(b)固定翼机器人飞行器(照片为"扫描鹰(Scaneagle)"无人机，由英西图公司(Insitu)提供)；(c)DEPTHX："深潜热量探测器"(Deep PhreaticThermal Explorer)，一款有6个推进桨的水下机器人，2007年由Stone Aerospace公司与CMU联合开发(照片由David Wettergreen拍摄，版权属卡耐基·梅隆大学)；(d)自动航行船(照片由Matthew Dunbabin提供)

在本书这一部分的章节中，将主要介绍移动机器人的基本知识。第 4 章将讨论两种典型机器人平台的运动与控制：在地面上移动的轮式小车，和在三维空间中运动的飞行机器人——特别是四旋翼飞行机器人。第 5 章将涉及移动机器人的导航。前面我们介绍过基于反应式和基于规划式的机器人引导方法，本章将详细探讨在有障碍的环境中这两种方法的具体技术。大多数机器人导航策略都要求知道机器人的位置信息，这部分内容将在第 6 章中讨论，其中包括航迹推算(Dead Reckoning)，以及基于路标观测的地图使用等。我们还会展示机器人如何创建出一幅地图，以及在未知环境中创建地图的同时如何确定自己的位置。

第4章　移动机器人载体



本章将讨论机器人平台如何移动，即它的位姿如何作为输入控制信号的函数随时间变化。在本书第二部分开始时展示了很多类型的机器人平台，本章只讨论其中两种比较典型的平台。第一个是像汽车一样工作在二维空间中的轮式机器人。它可以向前或向后移动，行进方向则可以通过改变转向轮的角度来调整。另一个是四旋翼飞行机器人，是一种无人飞行器，它是在三维空间运动的典型例子。四旋翼飞行机器人平台现在越来越受人们欢迎，主要是因为它们可以很容易进行模拟仿真和控制。

然而，在我们开始讨论这两个机器人平台之前，需要先了解一些关于机动性的概念，它具有一般性，但非常重要，对我们后面的学习很有益处。

4.1　机动性

之前我们已经介绍过移动机器人及其运动模式的多样性，本节将讨论机动性，即机器人如何实现在空间中的运动。

我们首先考虑一个简单的例子——火车。火车沿着轨道移动，其位置可以用它沿着轨道的距离数据来描述。因此，火车的位置形态可以完全由一个标量参数 q 决定，该参数也被称为它的广义坐标。所有可能的位置形态集合称为位形空间(configuration space)，或C-空间，记为 $\mathcal{C}$，且 $q \in \mathcal{C}$。在这里，$\mathcal{C} \subset \mathbb{R}$。我们也可以说火车只有一个自由度，因为 q 是一个标量。火车上还有一个驱动器(电机)，它推动火车沿轨道前进或后退。有一个电机驱动且仅有一个自由度，列车就能完全受控地到达任何指定的位置，即导轨上的任意一点。

另一个重要概念是任务空间(task space)，它是所有可能的机器人位姿 ξ 构成的集合，且 $\xi \in \mathcal{T}$。任务空间依赖于具体应用或任务。如果我们的任务是沿一条导轨运动，则 $\mathcal{T} \subset \mathbb{R}$。如果我们只关心火车在一个平面上的位置，则 $\mathcal{T} \subset \mathbb{R}^2$。如果考虑一个三维的世界，则 $\mathcal{T} \subset SE(3)$，火车高度随上下坡而变化，其方向因沿着曲线行进而变化。很明显，对于后两种情况，火车任务空间的维数超过其位形空间的维数。然而列车并不能达到任意的姿态，因为它被约束在一条固定导轨上移动。在这种情况下，我们说火车是沿任务空间中的一条流线运动，且存在一个映射 $q \mapsto \xi$。

有趣的是，其他许多运输工具与火车有着相似特征——它们都很擅长前进方向的运动，但侧向移动的性能很差。如汽车、气垫船、船舶和飞行器都表现出这一特征，它们需要复杂的操纵才能实现侧向移动。然而，这种特征的设计却是非常合理的，因为它迎合了我们对运输工具最常用的运动功能要求。其他不常用的运动，如汽车停泊、船舶停靠或飞机降落等，对操作的要求都更为复杂，人类需要更多的学习才能掌握这些技能。而且这种设计的好处是简化了系统结构，特别是减少了所需的驱动器数目。

接下来要讨论一种气垫船，其两个螺旋桨的轴线平行但不共线。两个螺旋桨推力的合力形成一个向前的推动力，如果推力不同也形成了一个转矩。气垫船在平面上移动时，它的位姿可以完全用广义坐标 $\boldsymbol{q}=(x, y, \theta) \in \mathcal{C}$ 来描述，这里 $\mathcal{C} \subset \mathbb{R}^2 \times \mathbb{S}$。气垫船的位形空间有 3 个维度，因此它具有 3 个自由度。

由于气垫船只有两个驱动器，比它具有的自由度少一个，因此它是一个欠驱动系统。这种驱动方式会限制其自由运动的能力。在任何时间点上，我们可以控制气垫船的前向加速度(与推力矢量平行)和旋转加速度，但气垫船的侧向(即横向)加速度始终为零，因为它的驱动器不会产生任何侧向推力。然而，通过一些像开车一样的细致操纵，也可以经过一定的路径把气垫船侧向停靠回初始位置。欠驱动的优点是驱动器的数量减少，如气垫船上只用了两个，而不是三个。但其代价是移动载体不能直接运动到其位形空间中的任何一点，而是必须经过一定的路径才能达到。如果给气垫船增加一个螺旋桨，且其轴线垂直于已有的两个螺旋桨，那么它就可以沿任意方向运动，包括前进、侧移和旋转。在这种情况下，气垫船的任务空间是 $\mathcal{T} \subset SE(2)$，与位形空间是等效的。

一架直升机有 4 个驱动器。主旋翼产生推力矢量，矢量大小由旋翼总距控制，矢量方向由横向和纵向的周期变距控制。直升机的第四个驱动器，即尾旋翼提供了一个偏航力矩。直升机的位姿可以由 6 个广义坐标 $\boldsymbol{q}=(x, y, z, \theta_r, \theta_p, \theta_y) \in \mathcal{C}$ 来描述，其中包括了它在三维空间中的位置和姿态，姿态方向由滚转-俯仰-偏转角表示。直升机的位形空间 $\mathcal{C} \subset \mathbb{R}^3 \times \mathbb{S}^3$ 具有 6 个维度，因此该移动载体具有 6 个自由度。由于自由度多于驱动器数目，这种直升机也是欠驱动的，它没有办法提供俯仰和滚转方向的旋转加速，然而巧妙的是这些不能直接驱动的自由度对于直升机操作来说是不需要的——直升机能自然保持滚转和俯仰角的稳定平衡。直升机飞行中重力还起到了非常大的作用，重力就像一个额外的驱动器，提供了一个恒定的向下的力。这使得直升机可以利用推力矢量的水平分量实现侧向加速，而推力矢量的垂直分量则由重力抵消了。因此如果没有重力的作用，直升机就无法侧向飞行。直升机的任务空间是 $\mathcal{T} \subset SE(3)$。

固定翼飞机的特点是向前移动非常高效，它也有 4 个驱动器(向前推进器、副翼、升降舵和方向舵)。飞机的推进器提供向前方向的加速度，其他控制面则对飞机施加多种力矩：方向舵提供偏转力矩，副翼提供滚转力矩，升降舵提供俯仰力矩。固定翼飞机的位形空间与直升机相同，也有 6 个维度。因此，固定翼飞机也是欠驱动的，它没有办法提供侧向加速。固定翼飞机的任务空间是 $\mathcal{T} \subset SE(3)$。

第二部分开始时显示的 DEPTHX 水下机器人也有一个 6 个维度的位形空间 $\mathcal{C} \subset \mathbb{R}^3 \times \mathbb{S}^3$，但与飞行器不同的是它是完全驱动的。它的 6 个驱动器可以产生任意方向的力和力矩，允许它沿任何方向或绕任何轴加速。它的任务空间是 $\mathcal{T} \subset SE(3)$[①]。

最后，我们来讨论一下车轮——人类最伟大的创造之一。约公元前 3000 年人类发明了车轮，公元前 2000 年前出现了两轮马车。今天，四轮的汽车已无处不在，全球的汽车总数接近 10 亿。汽车的高效性以及我们对它的熟悉，使它们成为实现平面运动的首选载体[②]。

在平面上移动时，一辆车的位姿由其广义坐标 $\boldsymbol{q}=(x, y, \theta) \in \mathcal{C}$ 来描述，且 $\mathcal{C} \subset \mathbb{R}^2 \times \mathbb{S}$，有

① 一些低成本的航模飞机没有方向舵而是仅仅依靠副翼来实现飞机的侧倾和转向。一些更便宜的航模飞机甚至没有升降舵，而仅仅依靠引擎速度来控制飞机的高度。

② 网址 http://hypertextbook.com/facts/2001/MarinaStasenko.shtml.

3个维度。但一辆车仅有两个驱动器，一个是实现前进或后退移动的，另一个是改变行驶方向的[①]。因此车也是欠驱动的。正如我们已经指出过的，欠驱动的车辆不能直接移动到其位形空间中的任何一点，它必须要经过一些通常是非线性的路径。从日常开车经验我们知道，要侧向移动车身是不可能的，我们需要学习一些经验才能把车停到马路边——即平行停车。一辆汽车也不能原地旋转，但我们可以经过一些路径组合，使车辆停回原来的位置，但却旋转了180°——即一个三点反向。车辆的这种欠驱动特定给控制和路径规划带来的挑战将在本书的其他部分讨论。尽管有诸多限制，汽车仍是迄今为止人类发明的最简单和最有效的地面交通工具。

标准车轮对移动方向的要求较高，通常要沿着垂直于轮轴的方向滚动。虽然标准车轮很难满足我们时常会需要的车体侧向移动的要求，但它在大角度拐弯时还是体现了明显的好处——车轮与路面之间的横向摩擦力无需任何代价就抵消了车体转弯需要的离心加速度，否则还需要额外的驱动器来提供这种力。更先进的可实现侧滚的轮子也已经研发出来了，图4.1显示了这种全方位车轮，也叫瑞典轮，其外形与普通车轮相似，只是在它的轮周上安装了很多被动小滚子，这些小滚子的转轴与车轮的轴线垂直。这种全向轮的驱动方式与普通轮一样，但在横向方向上具有非常低的摩擦。还有一种类似于老式鼠标中滚球的球形车轮，它由两个驱动器控制，可以实现任意方向上的运动[②]。

在机器人领域，一辆移动车体经常被描述为一个非完整系统。非完整这个术语来自数学概念，意思是车体运动受到一个或以上的非完整约束。一个完整约束可以用一个由位形变量 x，y 和 θ 组成的方程来描述。而一个非完整约束只能用位形变量的微分方程描述，而且无法积分成一个位形变量的约束方程。这样的系统也称为非可积系统。正如我们已经讨论过的，这些系统的一个重要特征是它们不能直接从一个位形转移到另一个位形——它们必须经过一系列操作或动作顺序。

图4.1 全向轮，也称瑞典轮。请注意其轮周上的小滚子，它们可以使车轮在沿轮轴方向运动时几乎没有摩擦力（图片由Vex机器人公司提供）

通过滑移转向的车辆，如坦克，可以在任意点实现原地转弯，但是要实现侧向平移，还是必须经过停止、转弯、前进、再停止、再转弯这个过程。这种操纵方式，或称时变控制策略，是非完整系统的重要标志。坦克有两个驱动器，每条履带一个，就像轮式汽车一样，也是欠驱动的。

我们将上述已经讨论过的移动载体的机动性参数总结列于表4.1中。其中第二列是其自由度，即位形空间的维数，第三列是驱动器的数目，第四列表示该车辆是否完全驱动。

① 与飞机和水下机器人不同，对于汽车我们通常考虑的是它的速度而不是力和力矩。

② 我们也可以从控制理论来思考这个问题。布罗克特定理[Brockett 1983]表明，这些系统虽然是可控的，但是不能通过使用可微分的，甚至连续的纯状态反馈控制器使其达到期望的稳定状态，而需要用到时变的，或者非线性的控制策略来实现。

表 4.1 3 种不同类型的交通工具的控制参数汇总。符号 +g 表示可以将重力视为一个额外驱动

种 类	自 由 度	控 制 数	是否完全驱动
火车	1	1	√
气垫船	3	2	×
直升机	6	4 + g	×
固定翼飞机	6	4 + g	×
6 推进水下机器人	6	6	√
汽车	3	2	×

4.2 移动机器人小车

轮式车辆是一类非常有效的运输工具，而且是第二部分开始时给出的大部分地面移动机器人的原型。本节将为这类移动小车建立一个模型，并开发一些控制器，分别实现小车移动到一点、沿一条路径移动、跟随任意路径以及最后运动到特定位姿。

四轮车常用的模型是图 4.2 所示的双轮自行车模型。其后轮固定在车体上，前轮可以绕水平轴转动以实现车辆转向①。

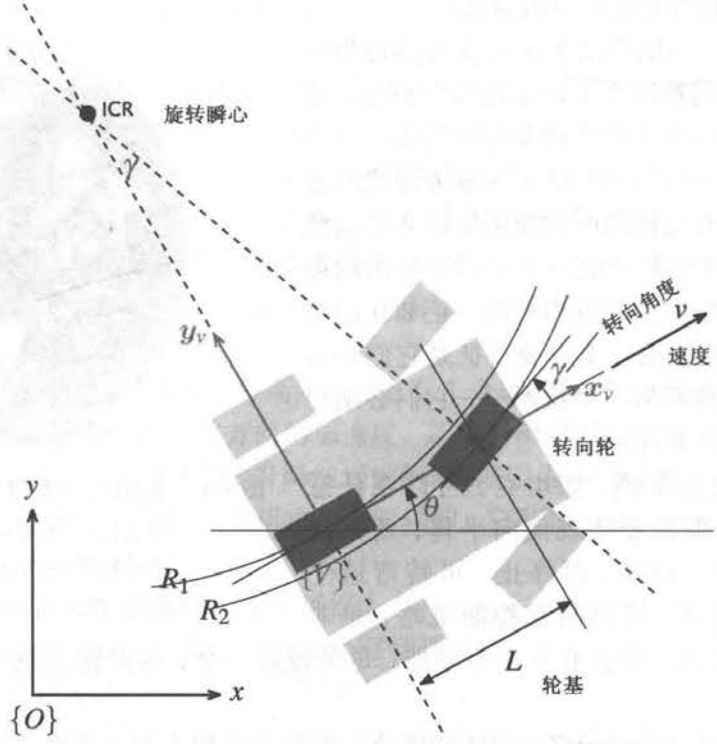


图 4.2 车辆的自行车模型。四轮车体外形以淡灰色显示，近似的双轮以深灰色表示。车体坐标系为浅色，世界坐标系为深色。转向轮的角度为γ，后轮在x方向上的速度为v。前后轮的轴线用虚线表示，其交点为旋转瞬心（ICR），而前后轮到瞬心的距离分别为R_1和R_2

该车的位姿可以用图 4.2 所示的坐标系$\{V\}$来表示，其中 x 轴为车辆的前进方向，而坐标原点设在后轮的中心。该车的位形由广义坐标$\boldsymbol{q}=(x, y, \theta) \in \mathcal{C}$表示，其中$\mathcal{C} \subset SE(2)$。该车的

① 常被误称为阿克曼（Ackerman）模型。

速度依据定义为其在 x 方向上的速度 v，而在 y 方向上的速度是零，因为车轮不能侧移。在小车坐标系 $\{V\}$ 中表示为[①]

$$^{V}\dot{x} = v,\ ^{V}\dot{y} = 0$$

图中虚线表示车轮无法在此方向上移动，虚线的交点即旋转瞬心(ICR)。相对于此瞬心，车辆上一个参考点将沿一个圆弧形轨迹运动，其角速度为

$$\dot{\theta} = \frac{v}{R_1} \tag{4.1}$$

通过简单的几何知识可知转弯半径 $R_1 = L/\tan\gamma$，其中 L 是该车辆或轮基的长度。正如我们所期望的那样，转弯半径随着车辆长度的增加而增加。转向角 γ 的大小是由机械结构限定的，其最大值对应于 R_1 的最小值。

背景材料：车体坐标系。我们将要使用的坐标系统，对各种车辆来说也是一种常见的坐标系统：x 轴沿正前向(纵向运动)，y 轴指向左侧(横向运动)，因而 z 轴垂直向上。但对于航空航天和水下应用来说，z 轴通常是向下的，x 轴还是正前方。

如果固定转向角度不变，小车将沿着一条圆弧移动。正因为如此，一般道路的弯曲处都设计成圆弧线或者回旋线[②]，这样驾驶员可以通过固定方向盘角度或者连续平滑地改变方向盘角度，较容易地跟随道路走向行驶。需要注意的是 $R_2 > R_1$，这意味着前轮必须走过一条长的路径，因此它旋转的速度比后轮要快。当四轮车辆行驶在一个转弯处时，前面两个转向轮所走的圆弧路径的半径也不同，因此它们的转向角 γ_L 和 γ_R 应该有一个极小的差别。这种差别可以由常用的阿克曼(Ackerman)转向机构实现，它可以减小对轮胎的磨损和撕扯。两个驱动车轮在转弯时也必须以不同速度转动，这就是为什么需要在发动机和驱动轮之间要安装一套差动齿轮箱的原因。

机器人小车在世界坐标系中的速度是 $(v\cos\theta,\ v\sin\theta)$，结合方程(4.1)，可以将它的运动方程写为

$$\begin{aligned}\dot{x} &= v\cos\theta\\ \dot{y} &= v\sin\theta\\ \dot{\theta} &= \frac{v}{L}\tan\gamma\end{aligned} \tag{4.2}$$

这个模型被称为小车的运动学模型，因为它只描述了小车的运动速度，并未涉及导致速度变化的力和力矩。小车朝向的变化率 $\dot{\theta}$ 也被称为转动速率、朝向转速或偏摆速率，该参数可以由陀螺仪直接测量，也可以由左右轮的角速度值推算出来。因为左右轮的转动半径不一样，所以其转速也不一样。

在世界坐标系中，可以写出一个沿小车 y 方向的速度表达式：

$$\dot{y}\cos\theta - \dot{x}\sin\theta \equiv 0$$

它属于非完整约束。这个方程不能通过积分变成 x，y 和 θ 之间的关系。

① 其他知名的模型包括 Reeds-Shepp 模型，该模型只有 3 种速度：前进、后退和停止，而 Dubbins 模型的车只有两个速度：前进和停止。

② 曲率半径平滑变化的曲线。

方程(4.2)还阐明了有关轮式车辆的其他一些重要特征。若 $v=0$ 则 $\dot{\theta}=0$，即在车辆不动的时候不可能实现转向。我们从开车的经验中也可以知道，车辆转弯必须是在行进中进行的。当转角变为 $\pi/2$，即前轮与后轮垂直时，车辆就无法前进了。

背景材料：鲁道夫·阿克曼(Rudolph Ackerman，1764—1834)是一位德国发明家，出生在萨克森州的施内贝格(Schneeberg)。因为经济原因，他没有上过大学，而是像他爸爸一样成了一名马鞍匠。曾有一段时间，他既做马鞍，又造马车。1795 年他还在伦敦成立了一个打字社和绘画学校。他曾发行过一本非常流行的杂志，名为《艺术、文学、商业、制造业、时尚与政治集萃》(*The Repository of Arts*, *Literature*, *Commerce*, *Manufactures*, *Fashion and Politics*)，其中收集了各式各样的科技文章，包括水泵、照明、平版印刷、时尚图样和家具设计。他曾在切尔西建厂来生产一种专用纸，供风景园林和微缩景观画家使用，他还为此申请了一个可用于布和纸防水的专利。他去世后被埋葬在伦敦的肯萨尔绿野公墓(Kensal Green Cemetery)。

1818 年阿克曼代理德国发明家乔治·兰肯斯培尔格(George Lankensperger)在英国申请并获得了 4212 号专利，专利内容是一种转向机构，它可以确保转向轮以同一个中心做圆周运动。其实在 18 世纪 60 年代，伊拉斯谟·达尔文(Erasmus Darwin，查尔斯的祖父)就提出了同样的设计方案并进行了测试。随后，法国人查尔斯·让托德(Charles Jeantaud)对此机构进行了改进并用到车辆上，而且沿用至今。因此，该机构也被称为阿克曼转向。

图 4.3 展示了一个用 Simulink 完成的自行车模型，它实现了方程(4.2)描述的运动关系，其中还包括一个最大速度的限制器，一个模拟有限加速度的速度变化率限制器，以及一个模拟转向轮有限转动范围的转向角限制器。而以下 Simulink 模型

```
>> sl_lanechange
```

如图 4.4 所示，就在一个有恒速要求的系统中使用了 `Bicycle` 模块。它给转向轮施加了先正后负的两个脉冲信号，其随时间的变化曲线如图 4.5(a)所示。而系统在 xy 平面内的输出运动轨迹如图 4.5(b)所示，可以看出是一个简单的变道轨迹。

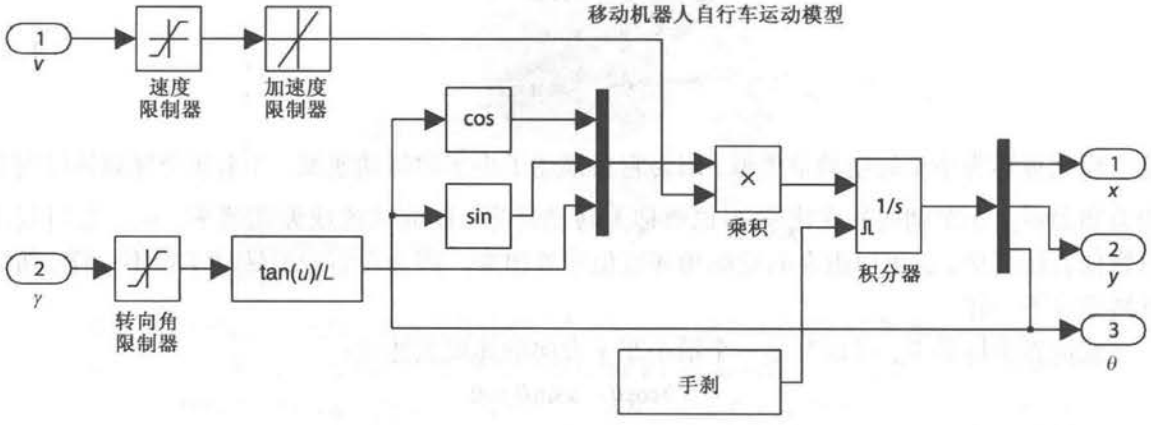


图 4.3 用工具箱的 `Bicycle` 模块实现自行车运动模型。速度输入信号部分包含了一个模拟有限加速度的速度变化率限制器，以及速度大小限制器和转向轮转角限制器。图中名为“手刹”的模块输出一个常量信号(可从参数对话框设定)，它用于复位积分器并强制回到初始条件

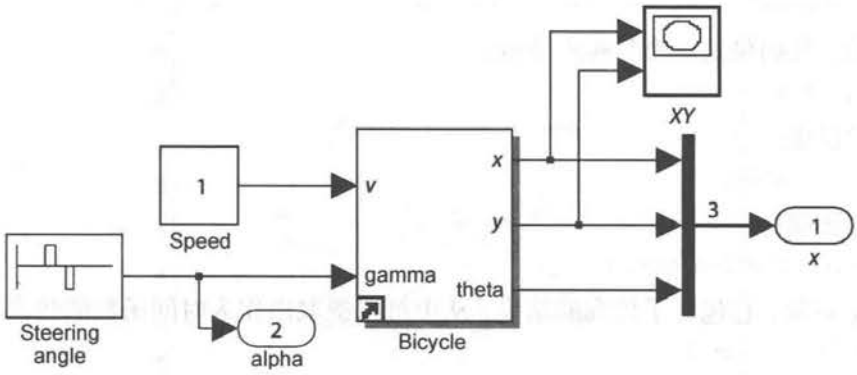


图 4.4　Simulink 模型 sl_lanechange，输出一个变换车道的结果。脉冲发生器驱动转向角先左后右转

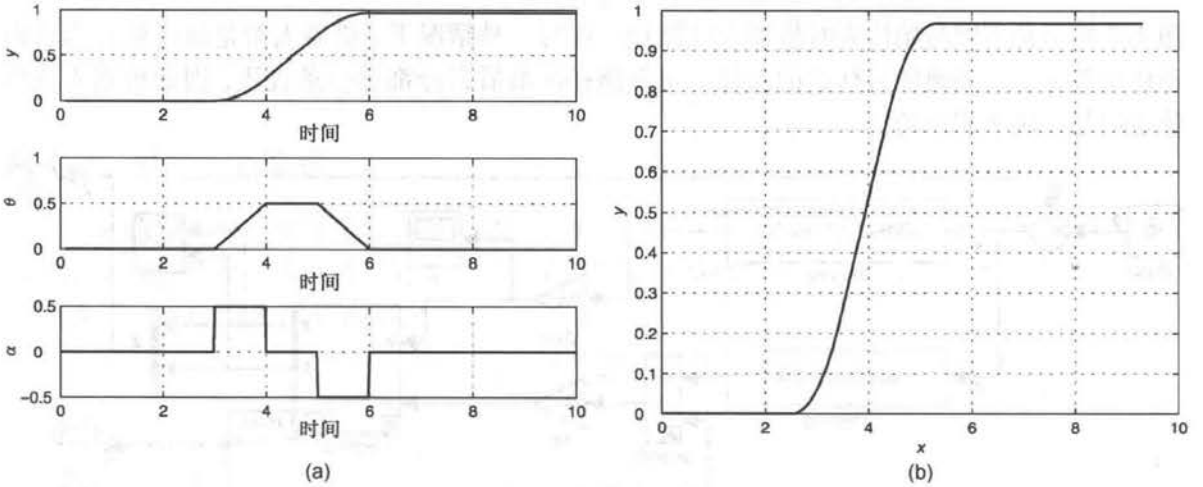


图 4.5　简单变换车道动作。(a)车辆随时间变化的响应曲线；(b)在 xy 平面上的运动，车辆向着 x 轴正前方移动

4.2.1　移动到一个点

下面讨论机器人向平面上一个目标点(x^*, y^*)移动的问题。移动过程中我们将控制机器人的速度，使其大小与机器人距离目标的远近成正比：

$$v^* = K_v\sqrt{(x^* - x)^2 + (y^* - y)^2}$$

机器人的转向目标角使用世界坐标系中的相对角度：

$$\theta^* = \arctan\frac{y^* - y}{x^* - x}$$

并使用比例控制器来调整车辆的转向角：

$$\gamma = K_h(\theta^* \ominus \theta),\ K_h > 0$$

注意其中运算符$\ominus$的使用，因为 θ^*，$\theta \in \mathbb{S}$，因此我们要求角度差也要在 $\mathbb{S}$ 中。上述控制的 Simulink 模型为①

① 工具箱中的函数 angdiff 用于计算两个角度的差，并返回属于 $[-\pi, +\pi)$ 中的一个差值。该函数在工具箱 Simulink Roblocks 模块组中也可以找到。

```
>> sl_drivepoint
```

如图 4.6 所示。我们设定一个目标点坐标：

```
>> xg = [5 5];
```

以及一个初始位姿：

```
>> x0 = [8 5 pi/2];
```

然后开始仿真运动：

```
>> r = sim('sl_drivepoint');
```

变量 r 是一个对象，它包含了仿真的结果，从中可以提取出作为时间函数的仿真数据：

```
>> q = r.find('yout');
```

可以画出在平面上该机器人的路径：

```
>> plot(q(:,1), q(:,2));
```

图 4.7 所示是不同起始位姿时机器人的路径。在每一种情况下，机器人都是通过前行和转向动作交替走上一条通往目标点的路径。每条路径的最后部分都是一条直线，因此机器人最终的走向是取决于起点的。

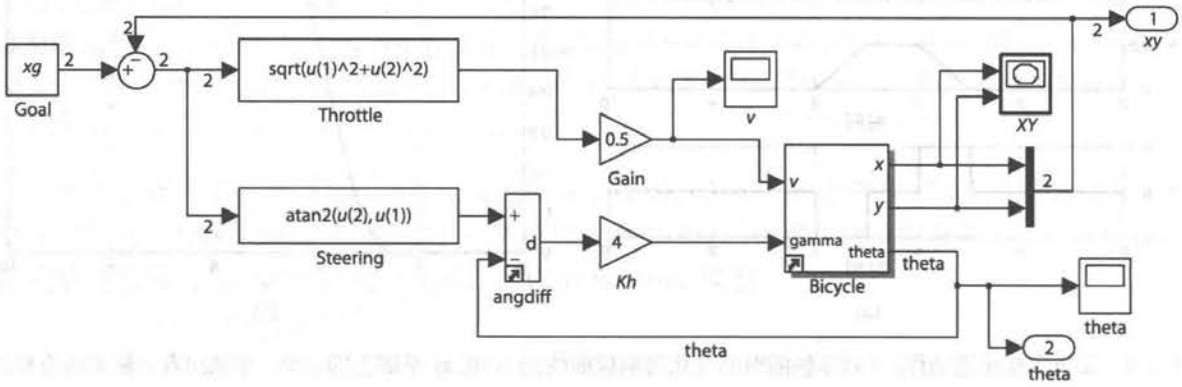


图 4.6 sl_drivepoint，驱动机器人到一个点的 Simulink 模型

背景材料：要运行名为 model 的 Simulink 模型，我们先加载它：

```
>> model
```

这时会弹出一个新窗口，以方框图的形式显示该模型。启动该模型的仿真可以在键盘上键入 Control + T，或者从模型窗口的工具栏选项中选择 Simulation + Start。模型也可以从 MATLAB 的命令行直接运行：

```
>> sim('model')
```

许多工具箱模型还可以通过创建另外的画面来显示机器人动画或图形。

有的模型可以向 MATLAB 的工作区写入数据供后续分析。有的模型仅简单地具有一些未连接的输出端口。在本章介绍的所有模型中，其仿真数据的输出选项都设置为 Format = Array，故信号都按照端口号的顺序排列在一起，形成一个行向量。这些行向量再堆叠形成一个矩阵 yout，其中每一行代表一个采样时间点的输出，而对应的时间值构成了另一个名为 tout 的向量。所有这些变量都可以在仿真执行后返回：

```
>> r = sim('model')
```

其包含在对象 r 中。通过显示 r，就能列出它包含的变量，而这些变量的值可由 find 方法获得：

```
>> t = r.find('tout');
```

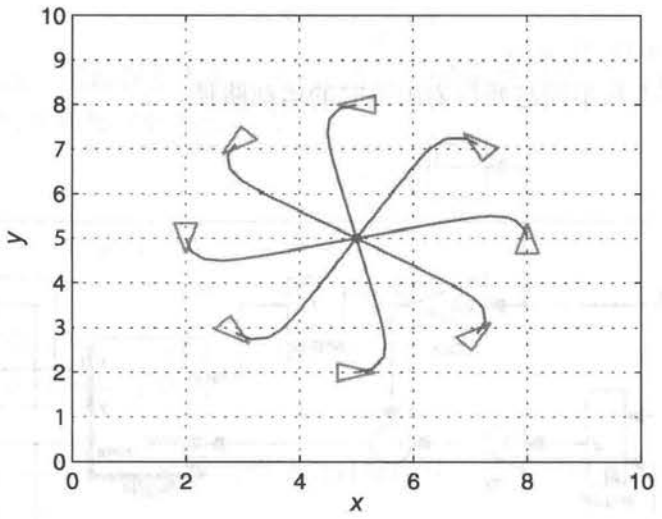


图 4.7　不同的初始位姿时 sl_drivepoint 模型的仿真结果。目标点是(5, 5)

4.2.2　跟踪一条直线

移动机器人另一种有用的功能是跟踪平面上由 $ax + by + c = 0$ 定义的一条线。这需要两个控制器来调整机器人的运动方向。一个控制器用于操纵机器人，使其与直线的垂直距离最短。根据方程(I.1)，该距离可表示为

$$d = \frac{(a, b, c) \cdot (x, y, 1)}{\sqrt{a^2 + b^2}}$$

采用以下比例控制器：

$$\alpha_d = -K_d d,\ K_d > 0$$

它将会控制机器人始终转向目标直线。第二个控制器用于调整机器人的朝向角或方向，使其与直线保持平行：

$$\theta^* = \arctan\frac{-a}{b}$$

同样，也可以使用比例控制器：

$$\alpha_h = K_h(\theta^* \ominus \theta),\ K_h > 0$$

两个控制器组合后的控制律为

$$\gamma = -K_d d + K_h(\theta^* \ominus \theta)$$

其结果是通过控制机器人的转向轮，使机器人不断接近目标直线，并沿着它运动。

上述控制方法的 Simulink 模型为

```
>> sl_driveline
```

如图 4.8 所示。我们用一个三元向量(a, b, c)来设定目标直线：[①]

```
>> L = [1 -2 4];
```

以及一个初始位姿：

```
>> x0 = [8 5 pi/2];
```

① 见附录 I。

然后开始仿真运动：

```
>> r = sim('sl_driveline');
```

如图 4.9 所示为机器人从不同起始位姿出发时的运动路径。

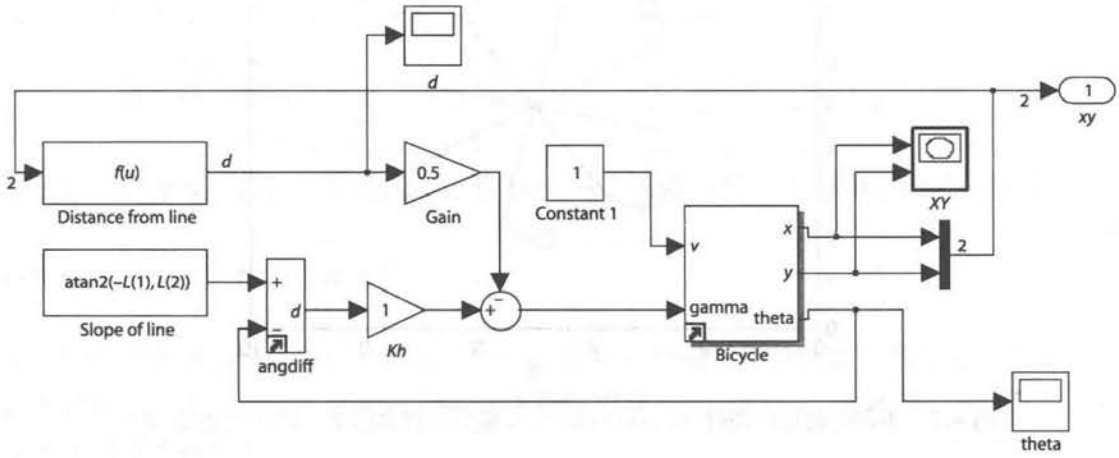


图 4.8 Simulink 模型 sl_driveline 控制机器人沿着一条直线运动。直线参数(a,b,c)被设置为在工作空间中的变量L

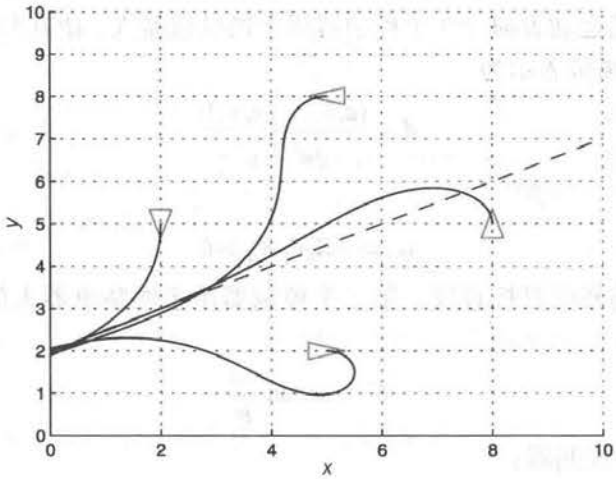


图 4.9 从不同的初始位姿向直线(1, -2, 4)运动的仿真结果

4.2.3 跟踪一般路径

除了直线轨迹之外，我们也希望机器人能跟踪由 xy 平面上的一般曲线所定义的一条路径。这种路径可能来自于某种运动规划器(如本书 5.2 节中讨论的)产生的一系列坐标位置，也可以是基于机器人传感器实时产生的。

路径跟踪问题的一个简单有效的算法是纯追踪。最简单的情况是目标点($x^*(t)$, $y^*(t)$)沿着路径以恒定速度运动，而机器人总是朝向目标点运动——有点类似于驴追胡萝卜。

这个问题非常类似于 4.2.1 节中讨论的控制问题，即移动到一点，不同的是这次的目标点是移动的。设机器人与被追踪点之间的距离目标值是 d^*，则跟踪误差可定义为

$$e = \sqrt{(x^* - x)^2 + (y^* - y)^2} - d^*$$

我们采用比例积分(PI)控制器来控制机器人的速度使跟踪误差趋近于零：

$$v^* = K_v e + K_i \int e dt$$

积分项是用来确保当跟踪误差为零时仍能提供一个有限的速度需求量 v^*。第二控制器则使机器人转向目标，相对角形式的目标值为

$$\theta^* = \arctan \frac{y^* - y}{x^* - x}$$

可以只用一个简单的比例控制器：

$$\alpha = K_h(\theta^* \ominus \theta), \quad K_h > 0$$

它将控制转向轮使机器人朝向目标点。

上述控制方法的 Simulink 模型为

```
>> sl_pursuit
```

如图 4.10 所示，其中包括一个沿单位圆移动的目标路径。然后可以对它进行仿真：

```
>> r = sim('sl_pursuit')
```

仿真结果如图 4.11 所示。在图 4.11(a)中，机器人从原点出发，然后不断追赶目标点的运动，直至跟踪上。图 4.11(b)展示了速度控制量在开始阶段的平滑迁跃，然后在接近目标跟踪距离时收敛到一个稳定值。

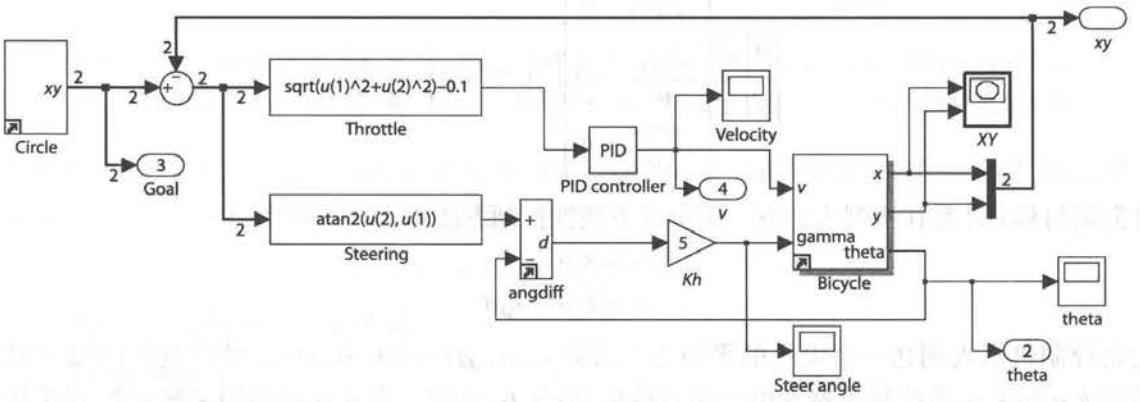


图 4.10　Simulink 模型 sl_pursuit 控制机器人用纯追踪法跟踪一个任意移动目标。在这个例子中，机器人是跟踪一个以0.1 Hz频率沿单位圆运动的点

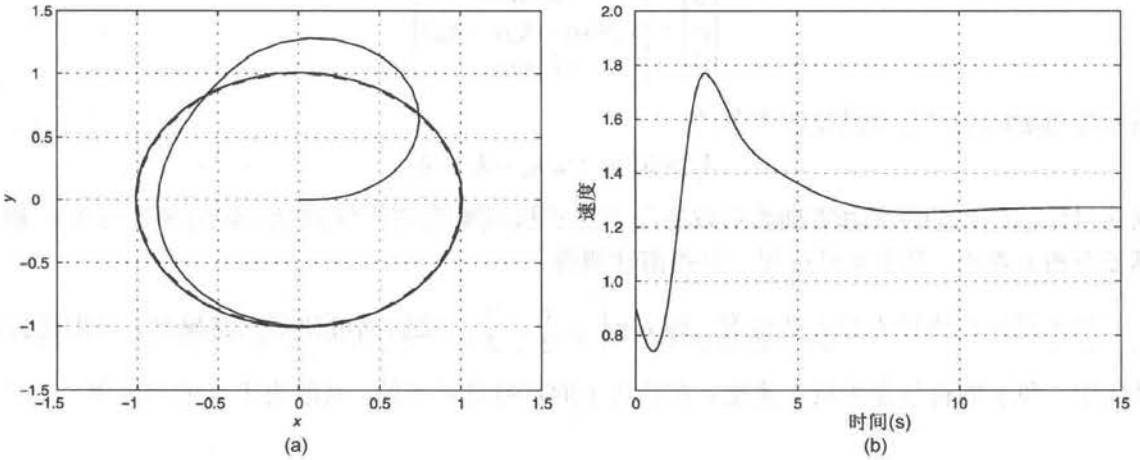


图 4.11　纯追踪法的仿真结果。(a)机器人在 xy 平面上的路径，虚线是被追踪的圆形轨迹，实线是机器人跟踪的实际运动轨迹；(b)机器人运动的速度-时间曲线

4.2.4 运动到一个位姿

我们要讨论的最后一个控制问题是将机器人运动到一个特定的位姿(x^*, y^*, θ^*)。图4.6所示的控制器可以控制机器人移动到目标位置，但机器人最终的姿态却依赖于它的起始位置。

为了控制机器人最终的姿态，首先将方程(4.2)重写成矩阵形式：

$$\begin{pmatrix}\dot{x}\\ \dot{y}\\ \dot{\theta}\end{pmatrix}=\begin{pmatrix}\cos\theta & 0\\ \sin\theta & 0\\ 0 & 1\end{pmatrix}\begin{pmatrix}v\\ \gamma\end{pmatrix}$$

然后使用图4.12中的符号将其变换成极坐标形式。我们对变量做如下转换：

$$\rho=\sqrt{\Delta_x^2+\Delta_y^2}$$

$$\alpha=\arctan\frac{\Delta_y}{\Delta_x}-\theta$$

$$\beta=-\theta-\alpha$$

从而可以得到

$$\begin{pmatrix}\dot{\rho}\\ \dot{\alpha}\\ \dot{\beta}\end{pmatrix}=\begin{pmatrix}-\cos\alpha & 0\\ \dfrac{\sin\alpha}{\rho} & -1\\ -\dfrac{\sin\alpha}{\rho} & 0\end{pmatrix}\begin{pmatrix}v\\ \gamma\end{pmatrix},\ 若\alpha\in\left(-\frac{\pi}{2},\frac{\pi}{2}\right]$$

并假定目标{G}是在机器人前方。采用以下线性控制方法：

$$v=k_\rho\rho$$

$$\gamma=k_\alpha\alpha+k_\beta\beta$$

它将控制机器人到达一个唯一的平衡点[①]，即$(\rho, \alpha, \beta)=(0, 0, 0)$。该控制器的设计思路是用$k_\rho\rho$和$k_\alpha\alpha$项控制机器人沿一条直线向目标{$G$}运动，而$k_\beta\beta$项则用来转动该直线从而使$\beta\to 0$。闭环控制系统为

$$\begin{pmatrix}\dot{\rho}\\ \dot{\alpha}\\ \dot{\beta}\end{pmatrix}=\begin{pmatrix}-k_\rho\cos\alpha\\ k_\rho\sin\alpha-k_\alpha\alpha-k_\beta\beta\\ -k_\rho\sin\alpha\end{pmatrix}$$

该系统稳定的前提是要满足以下条件：

$$k_\rho>0,\ k_\beta<0,\ k_\alpha-k_\rho>0$$

到达目标(ρ, α)的距离和方向角可以由一个照相机或激光测距仪测量，夹角β则可由α和车辆姿态角θ算得，其中θ可以用一个指南针测得。

对于目标在机器人后方的情况，即$\alpha\notin\left(-\frac{\pi}{2},\frac{\pi}{2}\right]$，我们仍可使用上述模型，只需把控制算法中$v$和$\gamma$的符号变为负。速度$v$在算法中的符号是不变的，其取决于$\alpha$的初始值。

① 该控制算法在$\rho=0$处不连续，从而满足了布罗克特(Brockett)定理。

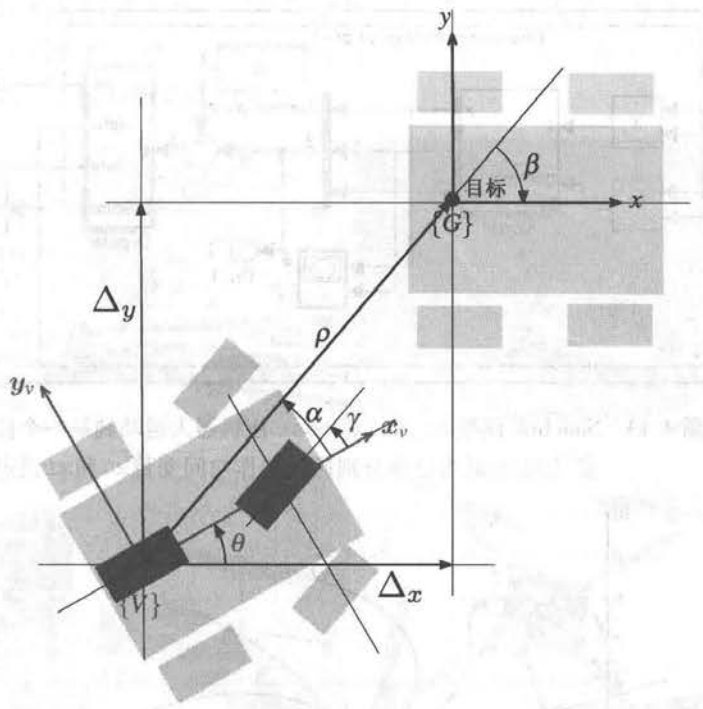


图 4.12　用极坐标符号表示的自行车模型。机器人向一个目标位姿运动：ρ 是到目标的距离，β 是相对于世界坐标系的目标向量角度，而 α 是相对于机器人坐标系的目标向量角度

至此，我们讨论了一种能驱动机器人达到姿态(0，0，0)的调节器。为使机器人运动到任意姿势(x^*，y^*，θ^*)，需要进行坐标变换：

$$x' = x - x^*,\ y' = y - y^*,\ \theta' = \theta,\ \beta = \beta' + \theta^*$$

位姿控制器仍然采用 Simulink 模型：

```
>> sl_drivepose
```

如图 4.13 所示。设定目标位姿：

```
>> xg = [5 5 pi/2];
```

初始位姿为

```
>> x0 = [8 5 pi/2];
```

然后开始仿真：

```
>> r = sim('sl_drivepose');
```

同样，仿真结果被存储在对象 r 中，并可以将机器人在平面上的路径绘制出来：

```
>> y = r.find('yout');
>> plot(y(:,1), y(:,2));
```

图 4.14 所示为不同初始位姿的机器人路径。机器人采用前进或后退的方式回到目标位姿，路径都很平滑①。

① 该控制器是基于线性的自行车模型，但 Simulink 的模型 Bicycle 有较强的非线性特性，包括转向角限制和速度变化率限制。

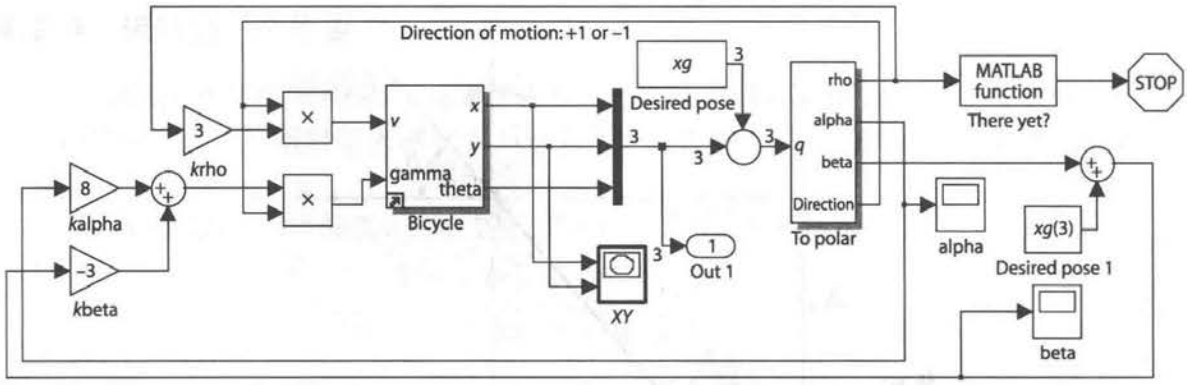


图 4.13 Simulink 模型 sl_drivepose 使机器人运动到某一个位姿。初始和最终位姿分别通过工作空间变量x0和xf设定

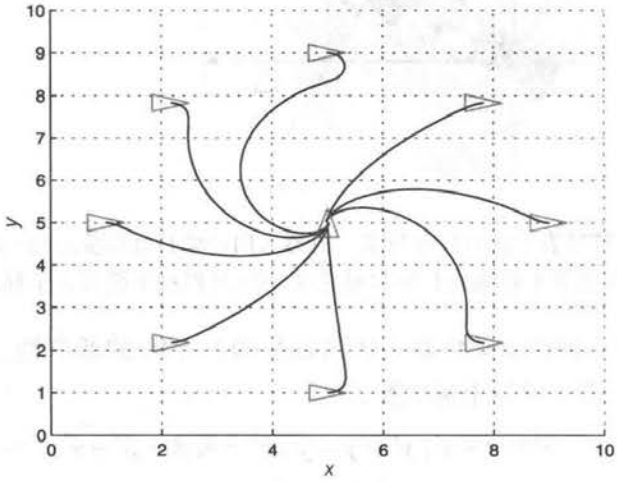


图 4.14 从不同的初始位姿运动到最终位姿(5, 5, π/2)的仿真结果。注意到在某些情况下机器人是以倒车状态回到最终位姿的

4.3 飞行机器人

想要飞翔，要做的仅仅是离开地面
——道格拉斯亚当斯(Douglas Adams)

飞行机器人或无人飞行器(UAV)现在正变得越来越常见，图 4.15 展示了不同大小和形状的飞行机器人，其应用领域覆盖军事行动、侦查监控、气象探测以及机器人技术研究等。固定翼无人飞行器的原理与客机相似，依靠机翼来提供升力，靠螺旋桨或喷气发动机提供推力，用操纵舵面实现机动。旋翼无人飞行器则有很多种配置，其中包括采用一个主旋翼加一个尾旋翼的传统直升机，采用同轴双旋翼的共轴反桨直升机，以及四旋翼飞行器。旋翼式无人飞行器主要用于探测和研究，其优势在于能垂直起降。

飞行机器人与地面机器人在一些重要技术参数上有所不同。首先，飞行机器人有 6 个自由度，且位形空间 $\boldsymbol{q} \in SE(3)$。其次，飞行机器人是由动力驱动的，因此它们的运动模型中必须包含力和力矩，而不像自行车模型只涉及速度，即飞行机器人要采用动力学模型而非运动学

模型。水下机器人与飞行机器人有很多相似之处，它们可以看成是在水下飞行的机器人，可以将固定翼和旋翼飞行器的相关技术移植使用。主要区别在于水下有一个向上的浮力，还有比在空气中大很多的阻力，以及附加质量。

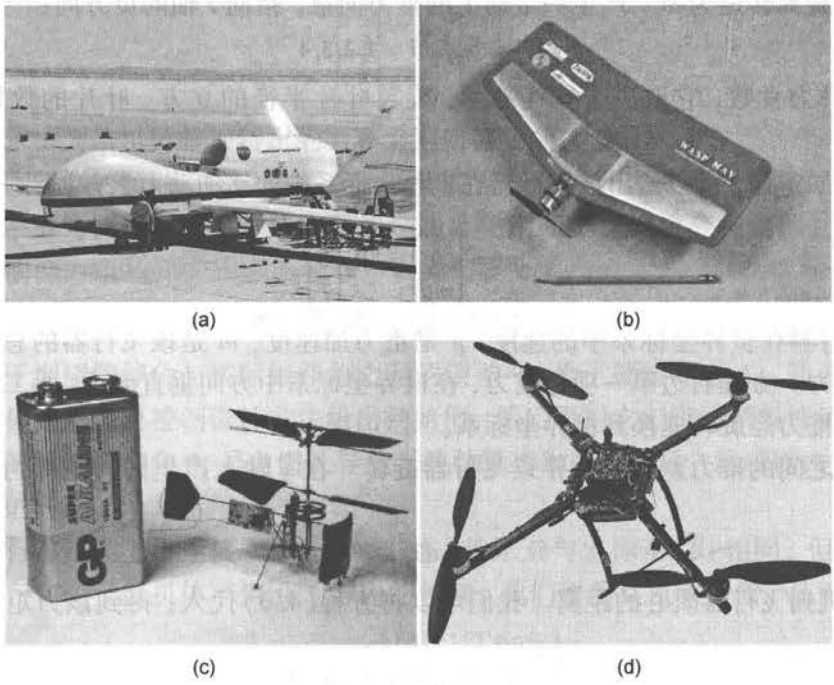


图 4.15　飞行机器人。(a)全球鹰(Global Hawk)无人飞行器(照片由 NASA 提供)；(b)一款微型空中飞行器(MAV)(照片由Aero Vironment公司提供)；(c)一架重1克，旋翼直径为70毫米的共轴反桨直升机(照片由Petter Murenand Proxflyer AS提供)；(d)一款四旋翼飞行器，也被称为X4，它有4个旋翼以及位于中心位置的传感和电控模块

本节将为类似图 4.16(d)所示的四旋翼飞行器建立运动模型。四旋翼飞行器现在很容易获得，既可以向商家购买，也可以从一些开源项目获取。与固定翼飞机相比，四旋翼飞行器的可操控性更好，而且可以在室内安全飞行，因而非常适合于实验室或业余爱好者使用。而与带有大尺寸主桨和尾桨的传统直升机相比，四旋翼飞行器更容易飞，而且没有复杂的斜盘传动机构，建模和控制也相对容易。

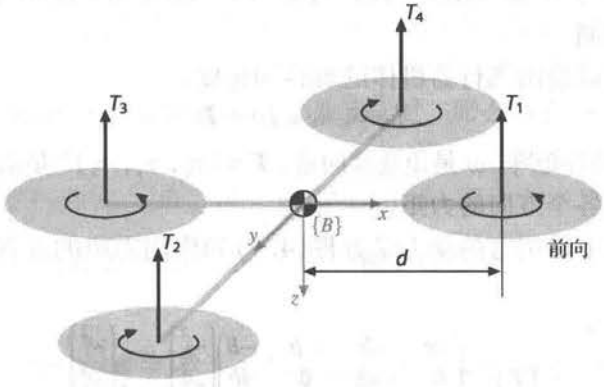


图 4.16　四旋翼飞行器示意图，包括 4 个旋翼及其推力矢量和旋转方向。机体坐标系$\{B\}$固联在飞行器上，其原点与飞行器质心重合。旋翼1和旋翼3逆时针旋转(从上往下看)，旋翼2和旋翼4顺时针旋转

图 4.16 展示了四旋翼飞行器的模型。机体坐标系 $\{B\}$ 的 z 轴向下，遵循航空技术惯例。该四旋翼飞行器有 4 个转动桨叶(旋翼)，标记为 1 至 4，安装在每个交叉臂的两端。旋翼由电子调速器控制的电机驱动。一些低成本四旋翼飞行器使用小型电机和减速齿轮系，以达到足够的扭矩。设旋翼转速为 ω_i，产生一个向上的推力向量，指向 z 轴的负方向：

$$T_i = b\omega_i^2,\ i = 1,2,3,4 \tag{4.3}$$

其中 $b>0$ 为升力常数，它取决于空气密度、旋翼叶片半径的立方、叶片的数量以及叶片的弦长。

由牛顿第二定律，可以给出该飞行器在世界坐标系中的移动动力学方程：

$$m\dot{v} = \begin{pmatrix} 0 \\ 0 \\ mg \end{pmatrix} - {}^0R_B \begin{pmatrix} 0 \\ 0 \\ T \end{pmatrix} \tag{4.4}$$

其中，v 是飞行器在世界坐标系中的速度，g 是重力加速度，m 是该飞行器的总质量，$T=\Sigma T_i$ 是总的向上推力。方程右边第一项是重力，在世界坐标系中方向垂直向下，第二项代表将机体坐标系中的总推力经旋转变换到世界坐标系。

每对旋翼之间的推力差异就会导致飞行器旋转。在 x 轴上产生的转矩即为横滚力矩，其大小为

$$\tau_x = dT_4 - dT_2$$

其中，d 是电机到飞行器质心的距离。我们可以将方程(4.3)代入，得到该力矩关于旋翼转速的表达式：

$$\tau_x = db\left(\omega_4^2 - \omega_2^2\right) \tag{4.5}$$

类似地，在 y 轴上产生的转矩即俯仰力矩，其大小是

$$\tau_y = db\left(\omega_1^2 - \omega_3^2\right) \tag{4.6}$$

电机施加到每个旋翼桨上的转矩与空气阻力力矩方向相反：

$$Q_i = k\omega_i^2$$

其中，k 与升力常数 b 的影响因素相同。该转矩会对飞行器机体施加一个反转力矩，其作用是使整个机体产生一个绕旋翼轴的转动，但方向与旋翼转向相反。因此在 z 轴上总的转矩是

$$\tau_z = Q_1 - Q_2 + Q_3 - Q_4 = k\left(\omega_1^2 + \omega_3^2 - \omega_2^2 - \omega_4^2\right) \tag{4.7}$$

其中不同的正负号对应于旋翼转动方向的不同。由上式可知，偏摆力矩可以简单地通过调整 4 个旋翼的转速来控制。

由欧拉运动方程可以给出飞行器机体的旋转加速度：

$$J\dot{\omega} = -\omega \times J\omega + \Gamma \tag{4.8}$$

其中，J 是 3×3 的机体惯性矩阵，ω 是角速度向量，$\Gamma=(\tau_x, \tau_y, \tau_z)^{\mathrm{T}}$ 是由方程(4.5)～方程(4.7)计算出的作用在机体上各个方向的力矩。

通过整合计算机体上作用力的动力学方程(4.4)和作用力矩的方程(4.8)，可以得到四旋翼飞行器的运动模型：

$$\begin{pmatrix} T \\ \Gamma \end{pmatrix} = \begin{pmatrix} -b & -b & -b & -b \\ 0 & -db & 0 & db \\ db & 0 & -db & 0 \\ k & -k & k & -k \end{pmatrix} \begin{pmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{pmatrix} = A \begin{pmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{pmatrix} \tag{4.9}$$

可以看出，所有力和力矩都是旋翼转速的函数。如果 $d>0$，则矩阵 $\boldsymbol{A}$ 是满秩的，可以通过求逆得到

$$\begin{pmatrix}\omega_1^2\\ \omega_2^2\\ \omega_3^2\\ \omega_4^2\end{pmatrix}=\boldsymbol{A}^{-1}\begin{pmatrix}T\\ \tau_x\\ \tau_y\\ \tau_z\end{pmatrix} \tag{4.10}$$

利用上式可以求出为给机体施加特定的力和力矩所需要的各旋翼转速。

为了控制飞行器的运动，我们引入一种嵌套式控制结构。这里以控制飞行器实现绕 y 轴俯仰及沿 x 轴平移为例进行说明。最内层的控制环使用一个比例微分控制器，它计算需要施加在机体上的俯仰力矩[①]：

$$\tau_y=K_p(\theta_p^*-\theta_p)+K_d(\dot{\theta}_p^*-\dot{\theta}_p)$$

计算主要是基于期望俯仰角与实际俯仰角之间的误差[②]。为了确定增益 $K_{\tau,p}$ 和 $K_{\tau,d}$，可以先根据近似动力学模型用经典控制设计方法给出初始值，然后再对它们进行调整修正，以达到最佳控制性能。实际机体俯仰角 θ_p 是由惯性导航系统估算得出的。之后，就可以由方程(4.10)确定各旋翼所需的转速。

假设在飞行器机体上再连接一个坐标系 $\{V\}$，其原点与坐标系 $\{B\}$ 相同，但其 x 轴和 y 轴始终与地面平行。为了使机体产生沿 ${}^Vx$ 方向的位移，我们要使其机头下俯，从而产生一个力：

$$\boldsymbol{f}=\boldsymbol{R}_y(\theta_p)\begin{pmatrix}0\\0\\T\end{pmatrix}=\begin{pmatrix}T\sin\theta_p\\0\\T\cos\theta_p\end{pmatrix}$$

它有一个分量：

$$\boldsymbol{f}_x=T\sin\theta_p\approx T\theta_p$$

使得飞行器可以沿 ${}^Vx$ 方向加速。我们可以用比例控制法控制这个方向的速度：

$$\boldsymbol{f}_x^*=mK_f\left({}^Vv_x^*-{}^Vv_x\right)$$

将上述两个方程联立，可以得到俯仰角：

$$\theta_p^*=\frac{m}{T}K_f\left({}^Vv_x^*-{}^Vv_x\right) \tag{4.11}$$

它就是为达到期望的前进速度所需要的俯仰角。实际的飞行器速度 ${}^Vv_x$ 可以由惯性导航系统或GPS信号接收器估算得到。对于在竖直方向上保持平衡的飞行器，所受推力总和应与重力相等，即有 $m/T\approx 1/g$。

如果在世界坐标系的 xy 平面上机器人位置是 $\boldsymbol{p}\in\mathbb{R}^2$，那么可以由比例控制算法给出期望的速度：

$$\boldsymbol{v}^*=K_p(\boldsymbol{p}^*-\boldsymbol{p}) \tag{4.12}$$

它是基于期望位置和实际位置之间的误差算出的。在坐标系 $\{V\}$ 中，期望的速度为

---

① 旋转动力学模型具有二阶传递函数 $\dfrac{\Theta_y(s)}{\tau_y(s)}=\dfrac{1}{Js^2+Bs}$，其中 B 是气动阻尼，通常很小。为了调节二阶系统，需要一个比例微分控制器。

② $\dot{\theta}_p^*$ 项通常可以被忽略。

$$^{V}\boldsymbol{v} = {}^{V}\boldsymbol{R}_0(\theta_y)\boldsymbol{v} = {}^{0}\boldsymbol{R}_V^{T}(\theta_y)\boldsymbol{v}$$

它是一个偏航角 θ_y 的函数。展开后得

$$\begin{pmatrix} {}^{V}v_x \\ {}^{V}v_y \end{pmatrix} = {}^{0}\boldsymbol{R}_V^{T}(\theta_y)\begin{pmatrix} v_x \\ v_y \end{pmatrix}$$

因此，为了达到期望的位置，我们可以先算出适当的转速，在此基础上再得到适当的俯仰角，从而产生推力使飞行器移动。造成这种间接关系的原因在于机器人是欠驱动的——我们只有 4 个旋翼的速度可以调整，但飞行器的位形空间却是六维的。为使四旋翼飞行器向前运动，就必须先让它头部下俯，使旋翼推力有一个水平分量来产生前进加速度①。而当飞行器接近目标位置时，又必须反方向上仰，使水平分力向后以实现减速。最后，机体旋转到水平姿态，使推力向量垂直向上。这里再次印证了前面的论述，即欠驱动的代价是增加操作的复杂性。同时也说明四旋翼飞行器的俯仰角是不能随意设定的，它是控制飞行器前进和后退的手段。

背景材料： 在 $SE(3)$ 中运动的物体，其转动惯量可用 3 × 3 的对称矩阵表示：

$$\boldsymbol{J} = \begin{pmatrix} J_{xx} & J_{xy} & J_{xz} \\ J_{xy} & J_{yy} & J_{yz} \\ J_{xz} & J_{yz} & J_{zz} \end{pmatrix}$$

主对角线元素是惯性矩，非对角线元素是惯性积。9 个元素中只有 6 个是独立的：3 个惯性矩和 3 个惯性积。如果物体的质量分布关于坐标系是对称的，则惯性积为零。

图 4.17（见彩图）展示了四旋翼飞行器的一个 Simulink 模型，它具有闭环控制结构。其中，Quadrotor 模块的 4 个输入量是 4 个旋翼的转速，然后它用方程(4.9)计算作用在四旋翼飞行器上的力和力矩，最后联立方程(4.4)、方程(4.8)和方程(4.9)算出位置、速度、姿态和姿态变化率。该模块的输出是状态向量 $\boldsymbol{x} = (x, y, z, \theta_r, \theta_p, \theta_y, \dot{x}, \dot{y}, \dot{z}, \dot{\theta}_r, \dot{\theta}_p, \dot{\theta}_y)$。按照航空技术惯例，我们用横滚-俯仰-偏航角参数来代表飞行器的姿态及姿态变化率。方框图中的控制部分包括多个嵌套的控制回路，用于计算所需的推力和扭矩，从而使飞行器移动到指定点。

在上述控制回路中，飞行器的位置控制回路位于框图的左上方。其中，蓝色的最内层回路控制飞行器的高度，其输入信号包括实际和期望的滚转及俯仰角，以及用于提供阻尼的横滚和俯仰角变化率。图中橙色的外层回路控制飞行器在 xy 平面上的位置，它通过获取滚转及俯仰角的变化，进而在 xy 平面期望的运动方向上提供相应的推力分量。在图中，方程(4.11)和方程(4.12)被合并成下式：

$$\theta_p^{*} = K_1\left({}^{V}p_x^{*} - {}^{V}p_x - K_2\,{}^{V}v_x\right)$$

在 xy 面内的位置误差先在世界坐标系中算出，然后通过 ${}^{0}\boldsymbol{R}_V^{T}(\theta_y)$ 旋转变换到坐标系 $\{V\}$ 中。请注意，根据图 4.16 所示的坐标系定义，沿 ${}^{V}x$ 方向的运动需要一个绕 y 轴的负转动（俯仰角），而沿 ${}^{V}y$ 方向的运动需要一个绕 x 轴的正转动（横滚角），因此在横滚和俯仰的回路中增益参数的符号是不同的。

① 总的推力也必须提高，以使推力的垂直分量仍然与重力平衡。

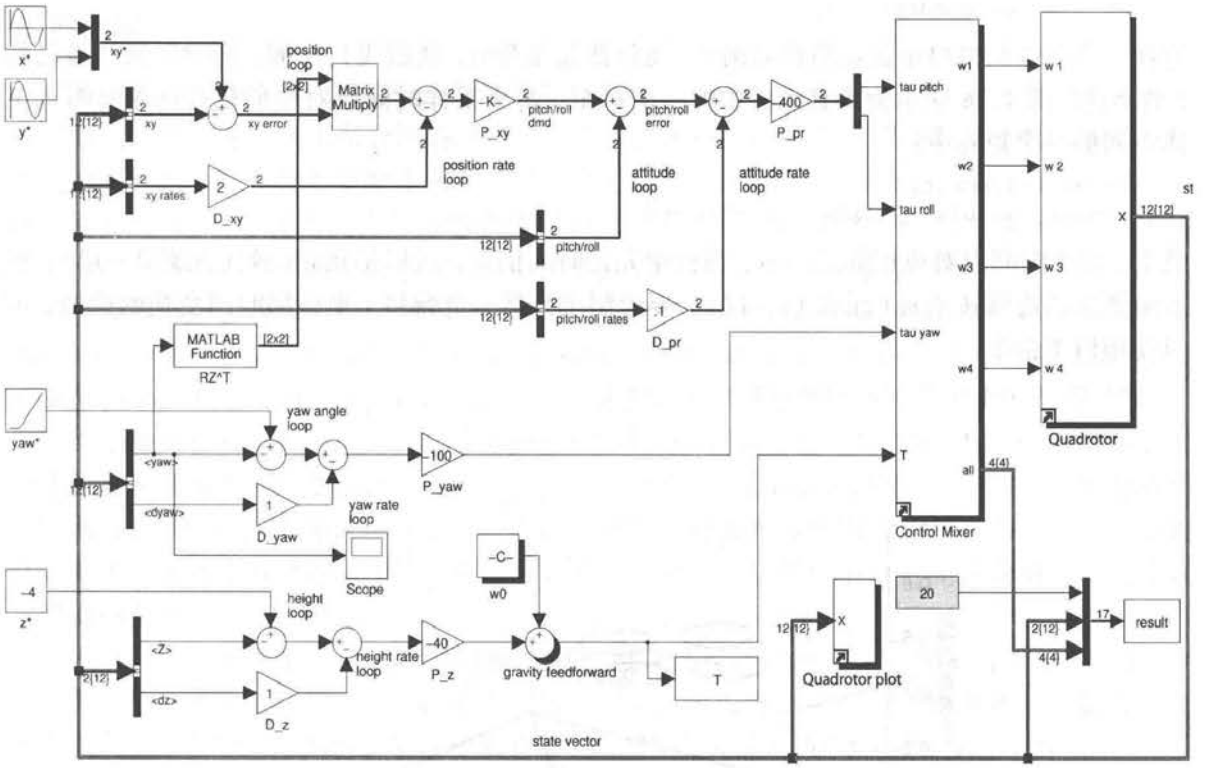


图 4.17　Simulink 模型 sl_quadrotor，它是四旋翼飞行器的一个闭环仿真模型。飞行器起飞后将以等高度飞行一圈。其中，动力学模块执行方程(4.9)的运算，混合模块则执行其逆运算，同时限制旋翼的转速。Quadrotor模块在输出12个元素的状态向量X时用了一条Simulink总线

飞行器的偏航角由一个比例-微分控制器控制：

$$\tau_z = K_p\left(\theta_y^* - \theta_y\right) + K_d\left(\dot{\theta}_y^* - \dot{\theta}_y\right)$$

如框图中的黑色控制回路所示。由于 $\dot{\theta}_y^*$ 很小，因此可以忽略不计。

飞行器的高度由一个比例-微分控制器控制：

$$T = K_p(z^* - z) + K_d(\dot{z}^* - \dot{z}) + \omega_0$$

如框图中红色的控制回路所示，它决定了旋翼的平均转速。控制器最后有一个附加项：

$$\omega_0 = \sqrt{\frac{mg}{4b}} \tag{4.13}$$

它是使旋翼推力与飞行器重量保持平衡的旋翼转速。这是一个典型前馈控制的例子，这里用以抵消重力的影响，否则重力将在高度控制回路中成为一个定常扰动。如果不用前馈控制，一种方法是在控制回路中用一个很大的增益值，但容易导致驱动器饱和或者不稳定。另一种方法是用一个比例-积分控制器(PI)，但这可能需要较长的时间来使积分项增加到一个有用的量值，而且会导致超调。我们在第 9 章研究机器人机械臂时还会再次遇到重力补偿问题。

一个特定四旋翼飞行器的参数可以用以下方式加载：

```
>> mdl_quadrotor
```

它将在工作空间里创建一个名为 quad 的结构，其元素就是四旋翼飞行器不同的动态性能参数。对该飞行器的仿真可以使用 Simulink 菜单或从 MATLAB 命令行执行：

```
>> sim('sl_quadrotor');
```

它在一个单独的窗口中显示仿真动画[①]。飞行器起飞升空，然后飞行一圈，同时缓慢绕自己的 z 轴旋转。图 4.18 所示为仿真飞行中的一个画面。每个采样时间点对应的仿真结果将写入工作空间的一个矩阵中：

```
>> about(result)
result [double] : 419x17 (56984 bytes)
```

其中，每个时间点对应矩阵的一行，每行中先是时间信息，然后是状态向量(元素 2 ~ 13)，最后是要求的旋翼转速 ω_i(元素 14 ~ 17)。要绘制飞行器 x 坐标和 y 坐标随时间变化的曲线，可以使用以下指令：

```
>> plot(result(:,1), result(:,2:3));
```

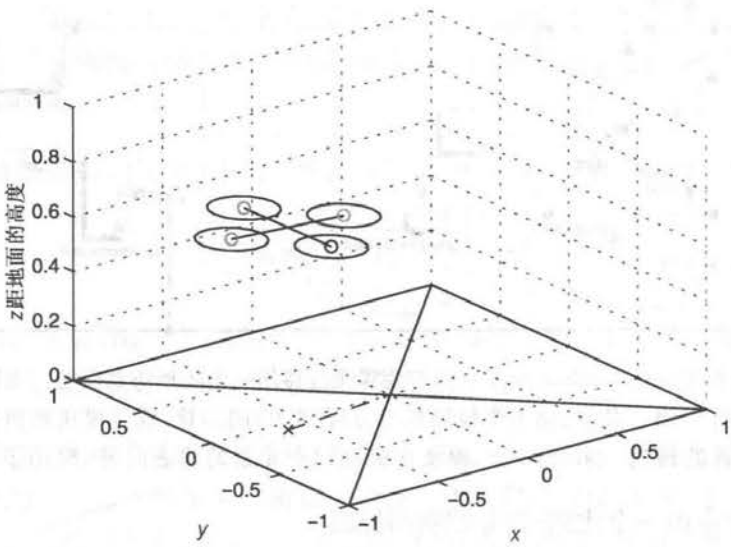


图 4.18 四旋翼飞行器仿真动画中的一帧画面。在地面上的标记点是飞行器质心的投影

4.4 本章总结

本章中讨论了机动性、位形空间和任务空间这些基本概念。我们还为两种不同的机器人平台建立了详细的模型。我们首先研究了类似汽车的机器人平台，它是众多地面机器人的原型。我们为该平台建立了一个运动学模型，并用它来开发了许多不同的控制器，使机器人平台能完成一些有用的任务，如移动到一个点、跟踪一条路径或者运动到一个位姿。然后讨论了一种简单的飞行器——四旋翼飞行器，并为它建立了一个动力学模型。在此基础上，我们开发了很多嵌套控制回路，实现了四旋翼飞行器飞行一圈的任务。这种嵌套控制方法将会在本书 9.4.2 节中有更详细的论述。

下一章将讨论如何在包含障碍的复杂环境中为机器人路径规划，以及怎样确定机器人在环境中的位置。

① 仿真开始之前，它要通过回调函数 PreLoadFcn 加载一个默认的四旋翼飞行器模型。PreLoadFcn 可在模型特性中设置为 File-Model Properties-Calbacks-PreLoadFcn。

扩展阅读

有关地面移动机器人的建模问题在西格沃特等人所著的书中有更全面的论述[Siegwart et al, 2011]。书中除了本书所介绍的自行车模型外，还对机器人上的驱动轮、转向轮和从动脚轮之间不同组合的多种车轮配置进行了深入讨论。这些研究题目在《机器人技术手册》(*Handbook of Robotics*)一书中有更为简洁的概括[Siciliano and Khatib, 2008, §17]。西格沃特等人的书还对我们后续章节要讨论的感知、定位和导航等问题也有详细的论述。

阿克曼的杂志现在可以上网浏览，网址为 http://smithandgosling. wordpress. com/2009/12/02/ackermanns-repository-of-arts，有关其运输转向机构的文章刊登在 1818 年 3 月和 4 月刊上。在文献[King-Hele, 2002]中，全面讨论了关于转向几何的前期工作以及达尔文的早期发明。

对于地面移动机器人而言，它用于制造工厂中的材料运输已相当成熟和普及，现在的研究前沿主要是在室外环境中能自主移动的车辆[Siciliano and Khatib, 2008, part F]。自动化乘用车的研究自 20 世纪 80 年代以来就已经开始，但至今尚未有任何商业化的产品——也许是因为我们的社会还没有准备好接受这项技术，或者是由于一旦发生事故可能引发的复杂法律问题还难以解决。

卡内基·梅隆大学的 Navlab 项目始于 1984 年，已经研制出了一系列名为 Navlabs 的自主移动车，并且还建立起了一个比较庞大的研究团队。他们研制的所有车辆都利用计算机视觉来进行导航。1995 年，Navlab 5 号自主车在有人监控下完成了 3000 英里的远征，开创了“无手横跨美国(No Hands Across America)”[Pomerleau 和 Jochem 1995, 1996]之举。该车 98% 的时间实现了自主转向控制，而且主要是通过视觉感知路边的白线完成的。

在欧洲，德国慕尼黑联邦国防军大学(Bundeswehr Universität München)的恩斯特·狄克曼斯(Ernst Dickmanns)和他的团队也研制了他们的自主控制车辆。1988 年，他们研制的 VaMoRs 系统，一辆 5 吨重梅赛德斯-奔驰面包车，实现了速度超过每小时 90 公里的自主行驶[(Dickmanns and Graefe, 1988b; Dickmanns and Zapp, 1987; Dickmanns, 2007]。欧洲在 1987 年至 1995 年之间还执行了一项普罗米修斯计划(Prometheus Project)。1994 年，该计划研制的机器人车辆 VaMP 和 VITA-2 在正常交通情况下以 130 多公里的时速沿巴黎多车道高速公路行驶了超过 1000 公里。这些车辆在公路上展示了自主驾驶的多种技能，包括两车编队行驶、自动跟踪其他车辆、超车时变更车道等。在 1995 年，一辆自主驾驶的梅赛德斯-奔驰 S 级轿车完成了从慕尼黑到哥本哈根 1600 公里的往返旅程，该车在德国高速公路上的时速超过了 175 公里，并且完成了诸如超车等的多项道路行驶科目。行驶过程中，人对该车实施干预的平均间隔为 9 公里，但在一段 158 公里的道路上它实现了无人干预的连续行驶。普罗米修斯计划在英国的子项目则研制了一辆自主驾驶的捷豹 XJ6，它采用视觉[Matthewset al, 1995]和雷达传感技术实现了沿车道行驶和防止碰撞。而近年来，在美国举行了一系列的自主车大挑战赛(Grand Challenges)，其中 2005 年的沙漠和 2007 年的城市挑战赛在各队论文汇编文献[Buehler et al, 2007, 2010]中有全面的描述。

飞行机器人和水下机器人尚未在标准的机器人学教材中讨论过。在《机器人技术手册》[Siciliano and Khatib, 2008]的第 41 章和第 43 章分别提供了有关空中和水下机器人的最新研究现状总结。米特勒的专著介绍了面向机器人的直升机理论[Mettler, 2003]，而有关直升机动力学最权威的参考则是普罗提编写的一部大书[Prouty, 2002]。安通尼利最近的一本书(现在是第二版)较全面地介绍了水下机器人的建模与控制[Antonelli, 2006]。

一些最早的关于四旋翼飞行器建模和控制的文章是由庞兹、马奥尼及其同事完成的[Hamel et al. 2002; Pounds et al. 2004, 2006]。庞兹2007年完成的论文[Pounds 2007]针对四旋翼飞行器提出了较全面的空气动力学建模方法，其中特别关注旋翼桨叶的拍动，这一现象虽在传统直升机上已众所周知，但大多数四旋翼飞行器的研究却忽略了它。目前在很多实验室都已经开发出了自己的四旋翼飞行器，而且有些飞行器已经上市面向业余爱好者或研究人员出售。一个名为 Mikrokopter 的开源项目还公开了四旋翼飞行器机身、推进系统以及控制软件的设计，并且可以在 http://www.mikrokopter.de/ucwiki/en 找到。基于四旋翼飞行器的基本原理，很容易通过增加更多旋翼进行扩展。现在具有6个、8个和16个旋翼的飞行器已经被开发出来，它们具有更大的负载能力以及更好的旋翼失效容错性。

习题

1. 一辆四轮车，长 $L=2$ m，两轮中心距为 1.5 m。
   a) 分别计算曲率半径为 10 m、50 m、100 m 的阿克曼转向对应的车轮转向角。
   b) 假设车辆以 80 公里的时速运动，分别计算当曲率半径是 10 m、50 m 和 100 m 时，车后轮转动速率之间的差异。
2. 写出一个车辆转弯速率相对于两个后轮角速度的表达式。分析车轮半径误差和车辆宽度误差对其造成的影响。
3. 编程实现 4.2.1 节中算子$\ominus$的运算，并与函数 `angdiff` 中的代码进行核对。
4. 移动到一个点(第65页)，画出 x、y、θ 随时间变化的曲线。
5. 纯追踪例子(第69页)：
   a) 探讨积分增益为零时会发生什么？现在减少圆周运动的频率至每秒 0.01 转，看看会发生什么？
   b) 积分增益设置为零时，要在控制器的输出中加一个常数，这个常数值应是多少？
   c) 修正纯追踪的例子，使机器人追踪一条类似障碍滑雪赛道的路径。
   d) 修正纯追踪的例子，使机器人追踪一个在一条直线上来回移动的目标。
6. 运动到一个姿态(第69页)：
   a) 尝试用一个不同的初始姿态重复这个例子。
   b) 尝试完成一个平行停车的动作，结果显示的路径是否实用？
7. 使用 MATLAB 的 GUI 界面做一个简单的方向盘和速度控制器，然后用它来创建一个非常简单的驾驶模拟器。也可以把某个现成的游戏方向盘和油门踏板嫁接到 MATLAB 中。
8. 四旋翼飞行器(73页)：
   a) 尝试用不同的控制增益重复实验。如果把阻尼增益减小到零会发生什么？
   b) 不要控制器中的重力正反馈项，而是用一个大的高度增益值来进行实验，或者就用一个比例积分(PI)控制器。
   c) 推导出方程(4.13)。
   d) 当飞行器具有非零的横滚和俯仰角时，推力的垂直分量值就会减小，飞行器将缓慢下降。尝试通过补偿垂直推力使飞行器保持高度。
   e) 仿真实现四旋翼飞行器的倒转飞行，即飞行中其 z 轴保持向上。

f)编写一个类似导弹发射运动的程序，先使四旋翼飞行器与水平方向成 45°起飞，然后取消掉所有推力。

g)编写一个使飞行器平稳降落的程序。

h)编写一个桶滚式机动的程序，先让四旋翼飞行器沿 x 方向水平飞行，然后从 0 到 2π 逐渐增加其横滚角。

i)编写一个翻筋斗机动的程序，先让四旋翼飞行器沿 x 方向水平飞行，然后从 0 到 2π 逐渐增加其俯仰角。

j)尝试给飞行器另外增加 4 个旋翼。

k)使用函数 `mstraj` 创建一条轨迹，轨迹上包含 10 个途经点(X_i, Y_i, Z_i, θ_y)。然后修正图 4.17 中的控制器，使飞行器能平滑追踪以上轨迹。

l)使用 MATLAB 的 GUI 界面制作一个简单的操纵杆，然后用它来创建一个非常简单的飞行模拟器。也可以把某个现成的游戏操纵杆嫁接到 MATLAB 中。

第5章　机器人导航

导航是引导一个运载工具到达预定位置的方法

——IEEE 标准 172-1983

机器人导航就是如何引导机器人朝着某一目标前进的问题。我们普通人的导航可以依赖地图和路标，大家很容易联想到，机器人导航是否也可以采用相同的方式？然而，现状是许多机器人在完成任务时并不依赖任何地图，这种方法称为反应式导航。例如，机器人寻着光源前行、沿地上白线行走、沿墙壁穿越迷宫以及沿随机路径完成房间吸尘等，它们在导航时直接依据环境光的强度，或相对于白线的位置，或是否与墙接触而做出反应。第二部分开始时给出的格雷·沃尔特(Grey Walter)开发的机器乌龟艾尔西(Elsie)就展示出了这种仿生行为能力——它可以对周围环境做出直接反应并寻找到一个光源。今天全球有超过500万个屋霸(Roomba)机器人在对房间地板进行自动吸尘，并且完全没有工作房间的地图信息，这些机器人沿着随机生成的路径工作，只有在探测到障碍物的时候才会改变路径。

更先进的机器人则使用基于地图的类人导航方法。这种方法可以支持更复杂的机器人任务，但方法本身也更为复杂。要实现这种导航方法还有许多前提条件，其中最基本的是要有一张工作环境的地图。另外，它还要求机器人的位置也必须是已知的。在下一章中，我们将专门讨论机器人如何确定它们的位置，以及如何创建环境地图。而在本章后续章节里，我们将主要讨论基于反应式导航和基于地图导航这两种方法，并以在平面环境中运动的轮式机器人为主要对象。

5.1　反应式导航

机器人可以在没有任何地图信息且不清楚自身所在位置的情况下完成复杂的工作任务，这种能力确实令人吃惊。如图5.1所示，自动吸尘机器人仅采用随机的运动以及接触传感器的信息就可以完成对整间房屋的清扫工作。在自然界，诸如蜜蜂、蚂蚁之类的昆虫也仅仅依靠感知到的信息去采集食物并返回巢穴，由于它们的神经元数量太少，远远不能在脑海中形成一个地图并在其中规划路径。其实，即使是单细胞生物，如鞭毛虫这类原生动物，它们也有找寻目标的行为。基于上述特征，我们需要修改前面关于机器人的定义：

一种能感知、规划并行动的目标导向机械。

简单生物所表现出的复杂行为也引起了早期控制论学者的兴趣。格雷·沃尔特的机器乌

龟表明它可以朝向一个光源移动，这种行为被称为趋光性(Phototaxis)[①]。这在当时还属于新兴学科的控制论中算是一个重要的成果。

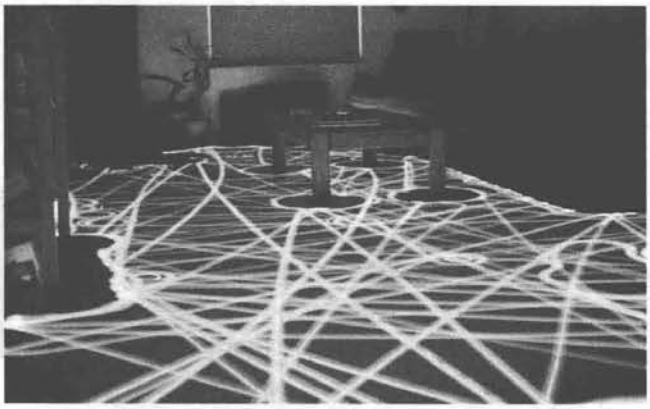
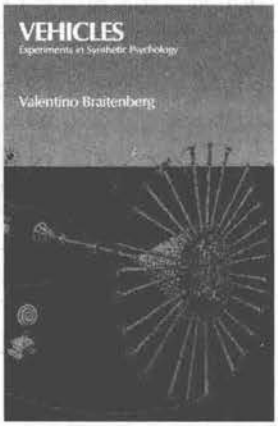


图 5.1　“屋霸”机器人清扫一间房屋的路径照片(Chris Bartlett 拍摄)

背景材料：瓦伦蒂诺·布赖滕贝格(Valentino Braitenberg，1926—)是具有意大利和奥地利双重国籍的神经科学家和控制论专家，也是位于德国图宾根的马普生物控制论研究所(Max Planck Institute for Biological Cybernetics)的前主任。他在 1986 年出版了专著《车辆：综合心理学实验》(*Vehicles*：*Experiments in Synthetic Psychology*，右侧是该书的封面图片，由 MIT 出版社出版)，该书描述了反应式目标达成型车辆，此类系统现在的专业名称为“Braitenberg 车”。



一辆 Braitenberg 车就是一个自动机或机器人，它集成了传感器、执行器以及它们之间用于产生目标导向行为的直接互连。格雷·沃尔特的机器乌龟虽然比该专业名称更早出现，但仍然算是这类车的典型例子。

这些车最早在概念上被描述为模拟电路。但是，最近基于相同原理的数控小型机器人也已经研发出来。

5.1.1　Braitenberg 车

Braitenberg 车是一种非常简单的目标达成机器人，其特点是传感器与电机之间直接连接。小车自身既没有其作业环境的信息，也没有任何路径规划可言[②]。

考虑机器人小车在二维平面上移动，寻求一个标量场的最大值(该标量场可以是光的强度或某种化学物质的浓度)[③]。其 Simulink 模型为

```
>> sl_braitenberg
```

如图 5.2 所示，它通过直接从传感器导出的转向信号来达到目标[④]。

为了增加梯度，我们需要估计在当前位置的梯度方向，这至少需要对势场进行两次测量的值。本例中我们在机器人两侧各安装一个传感器[⑤]，传感器建模由图 5.2 中的灰色模块所示，

① 更通常来说，趋向性(taxis)是指一个生物对某种刺激梯度的反应。

② 这是一个很好的哲学观点，该规划可以被认为隐含于电机和传感器之间的连接细节中。

③ 这类似于 4.2.1 节中讨论的移动到一个点的问题。

④ 这类似于 Braitenberg 车 4a。

⑤ 我们可以使用两个空间上分离的传感器同时进行测量，或者当机器人运动时测量一个传感器上不同时间点的数据。

并通过传感器相对于机器人的位置以及感知函数进行参数化。在这个例子中，传感器是安装在小车的横向或 y 方向 ±2 单位的位置。

背景材料：威廉·格雷·沃尔特(William Grey Walter, 1910—1977)是一位神经生理学家及控制论的创始人。他出生在美国密苏里州的堪萨斯城，曾就读于剑桥大学国王学院。因为没能获得剑桥大学的研究奖学金，他便到伦敦的医院从事神经生理学的研究，从 1939 年开始在布里斯托尔的博尔登神经学研究所(Burden Neurological Institute)工作。他开发了基于脑电扫描的脑地形图，它由在头皮上插入多个电极并通过三角剖分算法来确定大脑活动的幅度和位置。

沃尔特在当时的控制论这一新兴领域极负盛名。他建造机器人来研究怎样从神经的相互连接中引发复杂的反射行为。本书第二部分一开始展示了他的机器龟艾尔西(属 Machina Speculatrix 的一种)，图中没有外壳。制作于 1948 年的艾尔西是一个三轮机器人，具有趋光性，可以自己寻找路径到达充电站。他从 1951 年又推出第二代机器龟，其中一台现收藏于斯密斯学会(Smithsonian Institution)。沃尔特在《科学美国人》(*Scientific American*)上发表了许多受欢迎的文章(1950, 1951)，并在 1953 年出版了专著 *The Living Brain*。他在 1970 的一场车祸中受了很严重的伤，此后一直没有完全康复。(图片由 Reuben Hoggett 提供)

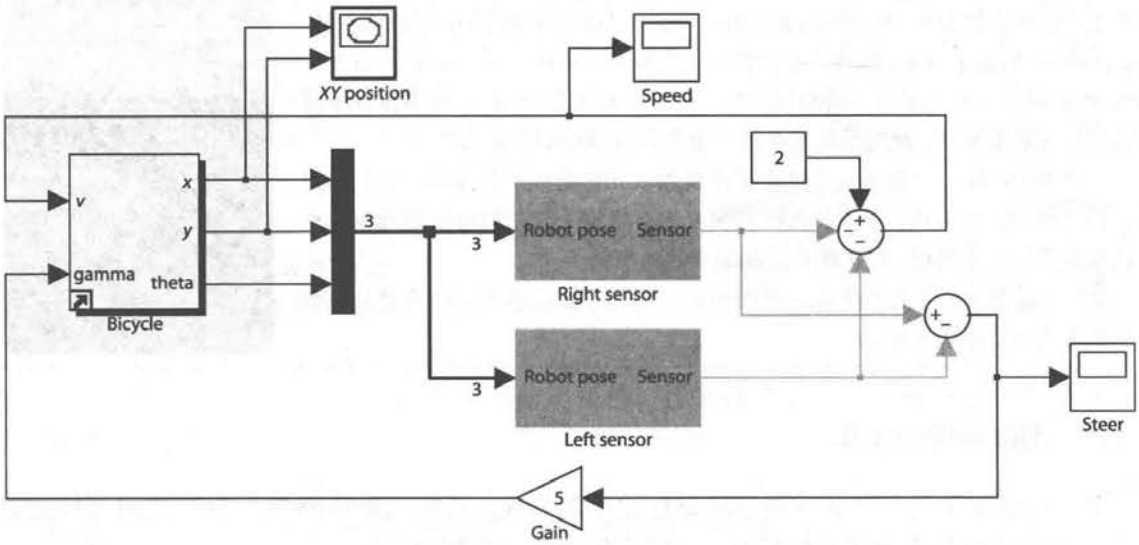


图 5.2 Simulink 模型 sl_braitenberg 驱动小车向一个标量函数的最大值运动。加装了控制器的小车是Braitenberg车的一个例子

被感知的势场由一个简单的平方反比函数定义：

```
1   function sensor = sensorfield(x, y)
2       xc = 60; yc = 90;
3       sensor = 200./((x-xc).^2 + (y-yc).^2 + 200);
```

该函数返回传感器值 $s(x,y) \in [0,1]$，它是一个在平面上传感器位置的函数。这个特殊函数在点(60, 90)有峰值。

小车的速度是：

$$v = 2 - s_R - s_L$$

其中，s_R 和 s_L 分别是在右侧和左侧的传感器值的读数。在目标点有 $s_R = s_L = 1$，此时的速度变为零。

转向角由两个传感器读数的差值决定：

$$\gamma = k(s_R - s_L)$$

因此，当左右侧传感器的值相等时机器人直行移动①。

我们可以从 Simulink® 菜单或命令行开始仿真：

```
>> sim('sl_braitenberg');
```

机器人的移动路径如图 5.3 所示。它的初始位姿可以通过改变 Bycicle 模块的参数来定义。我们看到机器人的速度随着与目标距离的减少而变慢，渐近地接近目标位置。

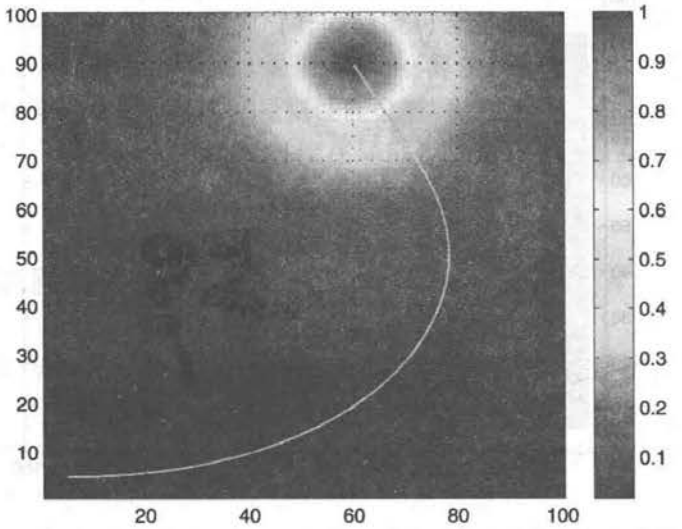


图 5.3　正向一个 2D 标量场最大值移动(并通过)的 Braitenberg 车的路径，该标量场的大小通过颜色编码

这种特定的传感-动作控制规律产生了某种特殊的机器人行为。我们还可以给机器人添加其他的控制逻辑，让它检测到它已经抵近目标，然后切换到停止行为。由于这个机器人唯一的功能就是向目标方向行进，所以一个简单的障碍物即可将其挡住。但是我们可以通过添加一个行为来处理这种情况，使机器人能绕过障碍物继续前进，也可以添加另一个行为使其在没有传感器信号的时候进行随机搜索。格雷·沃尔特的机器龟共有 4 种行为，可以基于光强及触觉传感器完成切换。

对多行为及其相互转换能力的研究产生了一种称为行为机器人学的方法。提出这种包容结构是为了规范不同行为之间的相互作用。复杂的行为(有人也称作似智能行为)都可以用这种系统进行说明。然而随着更多行为的加入，系统复杂性急剧增加，使得行为之间的相互作用复杂得难以描述和测试。最终不使用地图的代价变得难以承受。

5.1.2　简单自动机

另一种经典的反应式机器人被称为虫子，这种简单的自动机能避开不可达区域或障碍物搜索到达目标地点。针对虫子机器人的控制算法有很多，它们都具备感知接近障碍物的能力。在这方面它们与 Braitenberg 车很相似，但是虫子机器人在感知和驱动之间加入了一个状态机

① 飞蛾也采用这种相似的策略。它们的两根触须是十分敏感的气味探测器，可以用来引导雄蛾飞到一只正在释放费洛蒙的雌蛾处。

和其他逻辑。这种自动机还具有 Braitenberg 车缺乏的记忆能力①。本节中我们将重点讨论一种称为 bug2 的特殊算法。

首先我们给机器人加载一个障碍场：

```
>> load map1
```

它在工作区定义了一个 100 × 100 的矩阵变量 map。矩阵中的元素为 0 或 1，分别表示自由空间和障碍，如图 5.4 所示。生成这个地图的工具详见本书第 88 页。这个矩阵是一个网格占用的例子，将在下一节详细讨论。

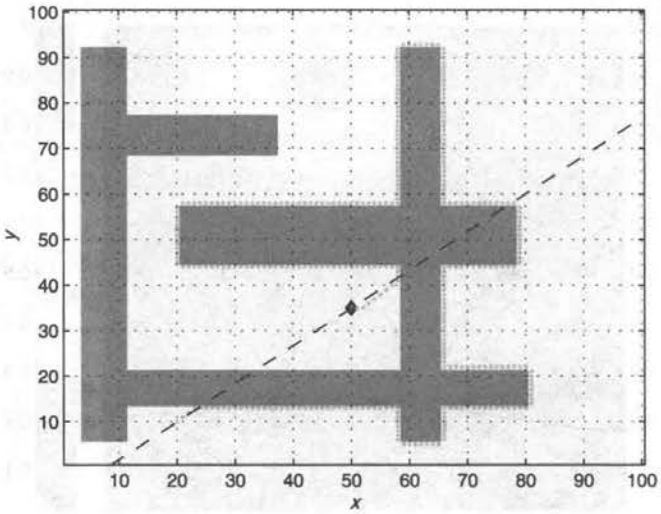


图 5.4 bug2 算法产生的路径由浅灰色的点线表示，目标点为菱形，深灰色区域为障碍区

在此我们先给定一些假设：首先，机器人在网格中活动，且只占一格；第二，机器人不具有任何非完整约束，可以移向任意相邻网格；第三，机器人可以确定自己在平面上的位置（这不是一个微不足道的问题，将在第 6 章详细讨论）；最后，机器人仅可以感知它的下一个位置及目标位置。机器人本身并不使用地图——地图是给仿真器使用以给机器人提供传感器输入。

我们创建一个 bug2 类的实例：

```
>> bug = Bug2(map);
```

目标是

```
>> bug.goal = [50; 35];
```

使用 path 方法来运行仿真：

```
>> bug.path([20; 10]);
```

其中的参数是机器人的初始位置。图 5.4 中的一系列浅灰色点线即表示这种算法产生的运动路径。

bug2 算法所用的策略非常简单，它先让机器人沿着一条直线向目标点运动。如果遇到障碍机器人就逆时针绕行，直到回到最初那条直线上的一点，这个点比之前遇到障碍时的点离目标更近②。

---

① 布赖滕贝格的书中描述了一系列复杂性不断增加的小车，其中一些还具有记忆。然而，Braitenberg 车这个词汇始终代表他描述的最简单的小车。

② 也有人提出 M 线代表着一个显性的规划。因此在导航方法的划分序列中，bug 算法应是介于 Braitenberg 车和基于地图规划的系统之间。

如果指定一个输出参数：

```
>> p = bug.path([20; 10]);
```

它将以矩阵 P 返回路径。查看该矩阵：

```
>> about(p)
p [double] : 332x2 (5312 bytes)
```

其中每一行表示路径上的一点，本例中一共是 332 个点。也可以不指定出发点调用该函数：

```
>> p = bug.path();
```

出发点将通过点击图上的任意点来选择。

bug2 算法的路径显然不是最佳，因为沿着障碍绕行至最初路线上耗费时间很多。而且本例中机器人如果顺时针绕行障碍物可能要快很多。bug 算法还衍生出很多变种，但它们都是针对某种环境提升了其性能，而对于其他环境则性能会下降。归根结底，不依赖地图使得这种算法的功能受到很大局限。没有全局意识就只能在局部生成最佳路径，但局部的最佳往往不是整体的最佳。

5.2　基于地图的路径规划

从日常生活中我们知道[①]，找到从 A 点到 B 点最佳路径的关键是要使用地图。一般来讲最佳意味着距离最短，但也应该考虑道路的可通过性带来的一些惩罚或代价。可通过性表示驶过某个区域的容易程度，有时走一条距离更远但路况更好的路反而会更快。一个更深思熟虑的规划者往往还会考虑车辆的运动学和动力学，会避开那些存在车辆无法完成的急转弯的路径。回顾我们前面给机器人下的定义：

> 一种能感知、规划并行动的目标导向机械。

本节将主要讲述机器人的规划。

表示一张地图以及机器人在其中位置的方法有很多。一种方法是把机器人位置表示成坐标$(x, y) \in \mathbb{R}^2$，把可通行区域或障碍区表示为多边形，每个多边形都包含一系列的顶点和边线。这种方式可能使表达格式非常紧凑，但要确定机器人和障碍物之间是否会发生碰撞就会涉及到对每条边线进行测试。

一个更为简单也更适于计算机操作的方式是网格占用法。它是把整个区域划分成若干网格，把每个网格分为占用或非占用。用 0 表示非占用网格，即机器人的可通行区域，1 表示占用网格，即不可通行区域或者说是障碍。单元格的大小取决于具体的应用。随着表示区域的扩大或者单元格的减小，计算机所需存储空间都会增加。然而对于现代的计算机而言，这种表示方式是非常可行的。例如，单元格大小为 1 米 ×1 米时，每平方千米面积只需要 125 字节的存储空间[②]。

在本节的其余部分，我们将使用示例程序来说明几个不同的规划器，它们都基于网格占用法。为了使表示一致，这些规划器所用的类都是从后面介绍的超级类 Navigation（见第 89 页）中派生出来。之前介绍过的 bug2 就是属于此类，因此后面要讲的例子就可以遵循相同的模式。

---

① 除了最简单的直线情况。

② 这里只考虑用单个二进制位表示一个单元格。当然占用网格也可以被压缩，或者是保存在仅有局部区域的磁盘上。

我们再给出一些假设：首先，机器人在网格中活动，且只占一格单元格；第二，机器人不含有任何非完整约束，且可以移动到相邻单元格；第三，机器人可以确定它在平面上的位置；第四，机器人可以使用地图计算它要走的路径。

背景材料：创建一个地图。一个占用网格就是一个矩阵，它对应二维空间的一片区域。所有 0 值元素代表机器人可以移动的自由空间，所有 1 值元素代表机器人不能移动的障碍物。我们可以用很多方法来创建地图。例如，我们可以创建一个全部为零的矩阵(代表所有都是自由空间)：

```
>> map = zeros(100, 100);
```

然后使用 MATLAB® 的运算操作，如

```
>> map(40:50,20:80) = 1;
```

来产生一个障碍，但这样做非常麻烦。为简化步骤，可以使用“工具箱”中的地图编辑器 makemap 来创建更复杂的地图，它使用了一个简单的交互式编辑器：

```
>> map = makemap(100)
makeworld:
  left button, click and drag to create a rectangle
  p - draw polygon
  c - draw circle
  e - erase map
  u - undo last action
  q - leave editing mode
```

这个交互式编辑器可以让你在占用网格中添加矩形、圆形和多边形，在这个例子中网格是 100×100。

需要注意的是占用网格对应的矩阵，其维度通常被表示为(行，列)，并且行数是这个矩阵的垂直维数。在笛卡儿坐标中我们通常先写水平 x 坐标，然后是 y 坐标。因此在程序代码中，占用网格矩阵总是以 y，x 表示。

所有的例子中都用到以下参数：

```
>> goal = [50; 30];
>> start = [20; 10];
>> load map1
```

它们分别对应目标点、起始点和世界地图。世界地图被加载到工作空间变量 map 中。使用上面描述的工具可以改变这些参数，这样占用网格也会改变。

5.2.1 距离变换

考虑只有一个代表目标的非零元素的矩阵。此矩阵通过距离变换得到另一个大小相同的矩阵，但其中每个元素的值是它到最初非零目标元素的距离。在机器人路径规划中，我们使用默认的欧氏距离。上述距离转换实际上是一种图像处理技术，在第 12 章将重点讨论①。

为了把距离转换用于机器人导航，我们创建一个对象 DXform，它是从类 Navigation 衍生出来的：

```
>> dx = DXform(map);
```

然后创建一个能达到指定目标的规划：

```
>> dx.plan(goal)
```

可以将其在屏幕上显示出来：

```
>> dx.plot(p)
```

① 两个点 (x_1, y_1) 和 (x_2, y_2) 之间的距离可以用欧氏距离 $\sqrt{\Delta x^2 + \Delta y^2}$，也可以用街区距离 $|\Delta x| + |\Delta y|$（又称为曼哈顿距离），其中 $\Delta x = x_2 - x_1$，$\Delta y = y_2 - y_1$。

如图 5.5(见彩图)所示。我们可以看见红色的障碍物区域正被叠加到距离地图上，其中任何一点的灰度等级表示了它到目标点的距离，从而为障碍物绕行提供了依据。

背景材料：超级类 Navigation。本章中所给例子使用的类都是从超级类 Navigation 衍生而出的，这个类是专为基于 2D 网格的导航设计的。每个例子基本上都包括以下模式。首先，通过调用类构造函数来创建一个基于类 Navigation 的对象实例。

```
>> nav = MyNavClass(world)
```

它被传递给占用网格。然后，计算出到达目标的一种规划：

```
>> nav.plan(goal)
```

该规划可以通过一下命令看到：

```
>> nav.plot()
```

然后计算从起始位置到目标的一条路径：

```
>> p = nav.path(start)
>> p = nav.path()
```

其中，p 是路径，即从起点到目标的一系列点，一行代表一个点，每一行包括点的 x 和 y 坐标。如果未指定 start(如第二个例子)，用户会被提示以交互方式点击选择起点。如果没有输出参数，则会显示机器人运动的动画。

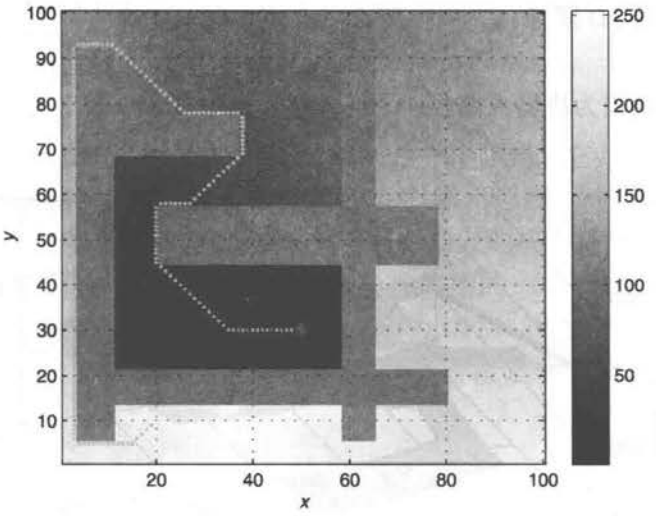


图 5.5　距离变换路径。红色网格代表障碍区。灰色背景的亮度代表单元格到目标的距离，距离单位是单元格大小，如右侧的标尺所示

上述最艰难的工作完成后，从任意点找到一条到达目标的路径就变得十分简单了。无论机器人从什么地方开始，都会朝着距离目标最近的相邻网格移动，这种算法一直持续，直到抵达目的地为止。例如，寻找一条从点 start 开始到目标的路径，只需输入：

```
>> dx.path(start);
```

之后屏幕上会显示机器人走向目标的过程动画。在图 5.5 中，该路径是一系列点线。

如果调用 path 方法时给定了一个输出参数，则会跳过动画，如

```
>> p = dx.path(start);
```

路径被返回到一个矩阵，每一行代表一个点。我们可以将这些点画出：

```
>> dx.plot(p)
```

要查看路径包括的点数，输入：

```
>> numrows(p)
ans =
   205
```

可以看出，其点数要比 bug2 所得小很多。与 bug2 不同，这种规划算法找到的是距离更短的顺时针绕行路径。

上述导航算法充分利用了其全局视野，并通过大量计算，发现了最短的路径。相反，bug2 算法没有这种全局观，只能舍近求远。当然，这种优化算法的代价就是计算的复杂性增加。距离变换的计算是迭代的，每步包含的计算量是 $O(N^2)$，而步数一共是 $O(N)$，其中 N 是地图维度。

我们可以通过以下指令显示距离变换的迭代过程：

```
>> dx.plan(goal, 0.1);
```

它展示出了距离值如同一个波阵面从目标点向外传播。波阵面先是向上移动，然后分离为左和右两部分，再向下移动，最后两个波阵面在沿直线 $x=32$ 的地图底部发生碰撞。函数中最后一个参数指定了每帧画面之间有 0.1 秒的暂停。尽管该规划的创建成本高昂，然而一旦创建完成，它就可用于从任何初始点到目标的路径计划。

现在我们已经把一个相当复杂的规划问题转化成为类似 Braitenberg 机器人处理的简单问题，即仅根据与目标的距离做出局部决策。机器人非常有效地沿着距离函数值减小的方向运动，我们可以绘制出其 3D 图像：

```
>> dx.plot3d(p)
```

如图 5.6 所示，黑色点线为机器人路径。

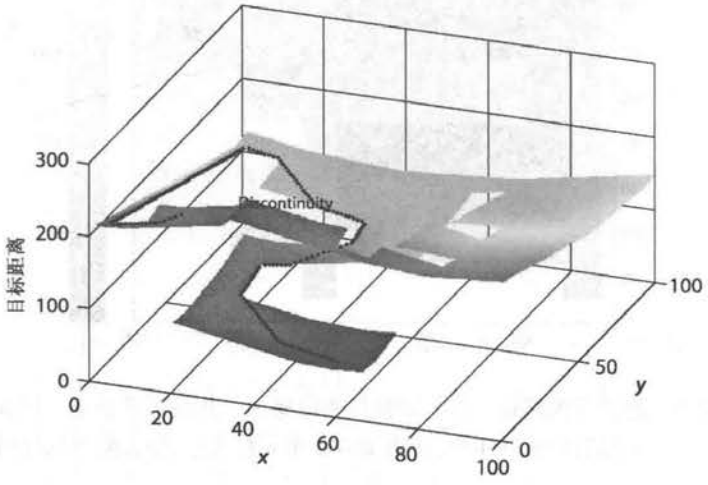


图 5.6 距离变换的三维表示，其中高度代表到目标的距离。机器人导航变成了简单的下坡运动。注意在波阵面相交的地方距离变换是不连续的

对于大规模的占用网格来说，这种规划方法会变得不切实际。本章后面将介绍一种路线图方法，它对于在大地图上寻找路径非常有效，可以大大降低计算成本。

5.2.2 D*

D* 是一种流行的机器人路径规划算法，有很多十分实用的特性[①]。D* 将占用网格推广为

① D* 算法是 A* 算法的一个扩展，是为了在一张图中寻找最小的成本路径，请参阅附录 J。

一张成本地图，其中 $c \in \mathbb{R}$，$c>0$ 代表在水平或者垂直方向上穿越每个单元的成本，斜对角穿越单元的成本则为 $c\sqrt{2}$，而对应于障碍物规定 $c=\infty$（MATLAB 中用 `Inf` 表示）。

D* 是通过寻找一条路径使得运行的总成本最低。如果我们感兴趣的是最短时间到达目的地，那么成本就是穿过单元所需的时间，它和可遍历性成反比。如果我们感兴趣的是减小对车子的损伤或者最大化乘客的舒适度，那么成本可能就是单元地形的粗糙程度。单元分配的成本取决于车子的特性，一辆大型的四轮驱动车在穿越不平整的道路时可能仅需要有限的成本，但是对于一辆小汽车，成本可能是无限大的。

D* 的关键特性是它支持增量式的重复规划，这是非常重要的。如果我们在移动过程中发现实际情况与地图不符合，或者是发现路线成本高于预期值或者完全被堵死了，我们就可以通过增量式重复规划找到一个更好的路线。增量式重复规划的计算成本比完全重复规划要低，前面讨论的距离转换法就属于完全重复规划。

为了使用工具箱来实现 D* 规划，我们采用与前面类似的步骤。首先建立一个 D* 导航对象：

```
>> ds = Dstar(map);
```

D* 规划将过去的占用网格地图转换为成本地图，从而可以得到

```
>> c = ds.costmap();
```

这里元素 c 取 1 或者无穷大，分别代表空闲和被占用的单元。

然后生成一个移动到目的地的规划：

```
>> ds.plan(goal);
```

它将产生一个密集的有向图（见附录 J）。每个单元是一个顶点，它具有成本，到达目标的距离，以及与最靠近目标的相邻单元的一个链接。每个单元还有一个状态 $t \in \{$NEW, OPEN, CLOSED$\}$。起初每个单元都处于 NEW 状态，只有目标单元的成本是 0，状态是 OPEN[①]。我们可以将所有的处于 OPEN 状态的单元看作从目标向外传播的一个波前，计算与 OPEN 态相邻的即将到达的单元的成本，然后这些单元依次被设定为 OPEN 状态，最初的单元则从 OPEN 态移除改为 CLOSED 态。在 MATLAB 中这个最初的规划阶段是非常慢的，需要花费几十秒的时间[②]：

```
>> ds.niter
ans =
       10558
```

以上指令显示了规划循环迭代的次数。

图 5.7 显示了从任何一个初始点到达目标的路径：

```
>> ds.path(start);
```

可以看到，机器人又一次选择了障碍左边的较短的路径，这和采用距离转换方法得到的结果一样。

---

① 距离变换实际上也是相对目标向外传播的一个波前。然而，D* 是用一组单元来有效地代表波的前沿，而距离变换每一次迭代时是基于每个像素来计算前沿的——其前沿是隐含在与有限成本单元相邻的无限成本单元（所有单元的初始值）处。

② D* 算法相比距离变换而言是更有效的，但它执行更慢，因为它完全是在 MATLAB® 代码中实现的，而距离变换是一个用 C 语言编写的 MEX 文件。

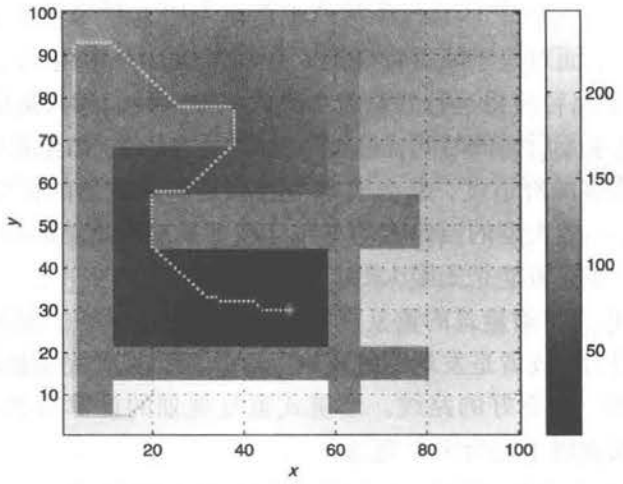


图 5.7 D* 规划路径。红色的单元表示障碍，所有可通过单元的成本是 1。背景的灰度代表了从任何一点到目标点的距离，右侧的标尺代表了不同的距离值

D* 的真正威力在于它能在任务中有效地更改成本地图。这种能力在机器人领域非常需要，这是因为实际的机器人传感器量程有限，而且机器人在行进过程中会发现更多不可预知的情况。我们可以采用 modify_cost 方法告诉 D* 所要做的修改，例如：

```
>> for y=78:85
>>     for x=12:45
>>         ds.modify_cost([x,y], 2);
>>     end
>> end
```

这里是将一个小的矩形区域的成本提高到 2 来模拟一片可通过性低的地形，这个区域在图 5.8 中用白色的虚线矩形表示出来了，其他的可到达单元的成本是 1。然后再一次应用规划算法来更新规划：

```
>> ds.plan();
```

但这次迭代的次数仅是

```
>> ds.niter
ans =
        3178
```

它只有原来规划所需次数的 30%。机器人的新路径为

```
>> ds.path(start);
```

如图 5.8 所示。成本改变虽然比较小，但是我们注意到在这个区域中增加的通行成本通过单元的细微变亮体现了出来，从成本角度来说这些单元距离目标更远了。与图 5.7 相比，机器人的路径移动到右边了，这样可以减小它穿过高成本区域的距离。D* 允许机器人在移动过程中随时更新地图，当重新规划后，机器人仅仅移动到附近的最低成本的单元，这样可以保证在规划改变后仍然可以保证运动的连续性。

5.2.3 沃罗诺伊路线图法

在路径规划的术语中，把生成一个计划称为“规划阶段”，而利用规划阶段的结果找到一条 A 到 B 的路径则称为“查询阶段”。前面介绍的两种算法，距离变换和 D* 在规划阶段都需要大量的计算，而在查询阶段都比较简单。但规划过程受目标位置的影响很大，一旦目标改

变，之前耗时的规划计算都必须全部重来。虽然 D* 允许成本地图改变后重新计算路径，但它也不支持目标的改变。

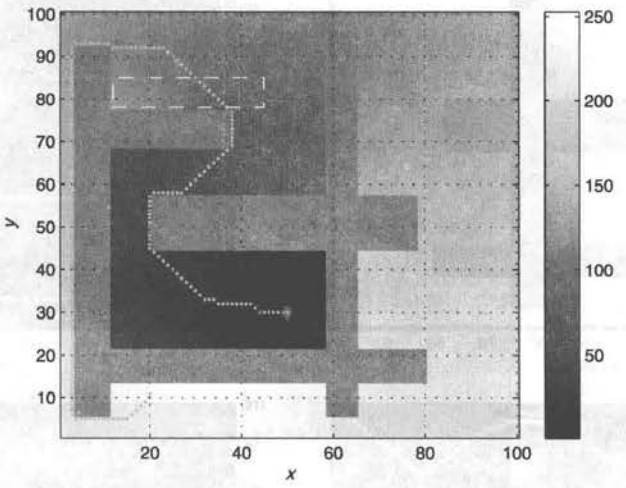
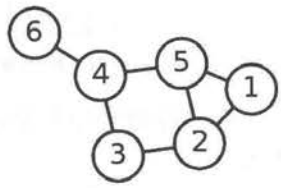


图 5.8　更改地图后 D* 规划得到的路径。白色虚线矩形代表高成本区域，与图 5.7 相比路径发生了变化

规划和查询的计算成本差异导致了路线图方法的产生，这种方法查询的既有起始位置也有目标位置。规划阶段提供的分析也支持不断变化的出发点和目标点。用坐火车出行做一个类比。我们先在当地找一条去最近火车站的路线，再乘火车到离目的地最近的车站，之后在当地再找到一条去目的地的路线。列车网络是不变的，所以在列车网络中规划路径非常简单明了。而在上下车当地规划去往和离开火车站的路线也很简单，因为它们的距离一般都很短。这样，机器人导航问题就变成要在环境中建立一个绕开障碍物的路网，就像铁路网一样。在文献中这样一个网络被称为路线图。路线图只需要计算一次，然后就像使用铁路网一样用它引导我们从任何初始位置去到任何目标位置。

背景材料：一幅图是对一组通过链接联系的对象的抽象表示，通常记为 $G(V、E)$，其图形化如右图所示。对象 V 也被称为顶点或节点，而把顶点两两相连的链接 E 也被称为边或弧。在本例中，边可以有方向(箭头)，也可以无方向。边可以有一个与之相关联的权重，或者称作从它的一个顶点移动到另一个顶点的成本。图中从一个顶点到另一个顶点的一系列边构成一个路径。图可以用于代表交通或通信网络，甚至社会关系。在数学中，有一个分支就称为图论。在图中，计算两个节点之间的最小成本路径可以用众所周知的 Dijstrka 算法或 A* 算法。



本书介绍的导航类中，都使用一个称为 PGraph 的简单 MATLAB 图类，请参阅附录 J。

下面我们将展示上述原理：使用一些图像处理技术在占用网格的自由空间中来创建一个路线图。创建这个路线图最关键的几个步骤如图 5.9 所示。第一步是要在地图中找到自由空间，即占用空间的补集：

```
>> free = 1 - map;
```

这是一个含有非零元素的矩阵。机器人在自由空间中可以自由移动，但其边界也是一个障碍物，因此我们将最外围的单元标记为非自由区域：

```
>> free(1,:) = 0; free(100,:) = 0;
>> free(:,1) = 0; free(:,100) = 0;
```

这个地图如图 5.9(a)所示，白色区域为自由空间。

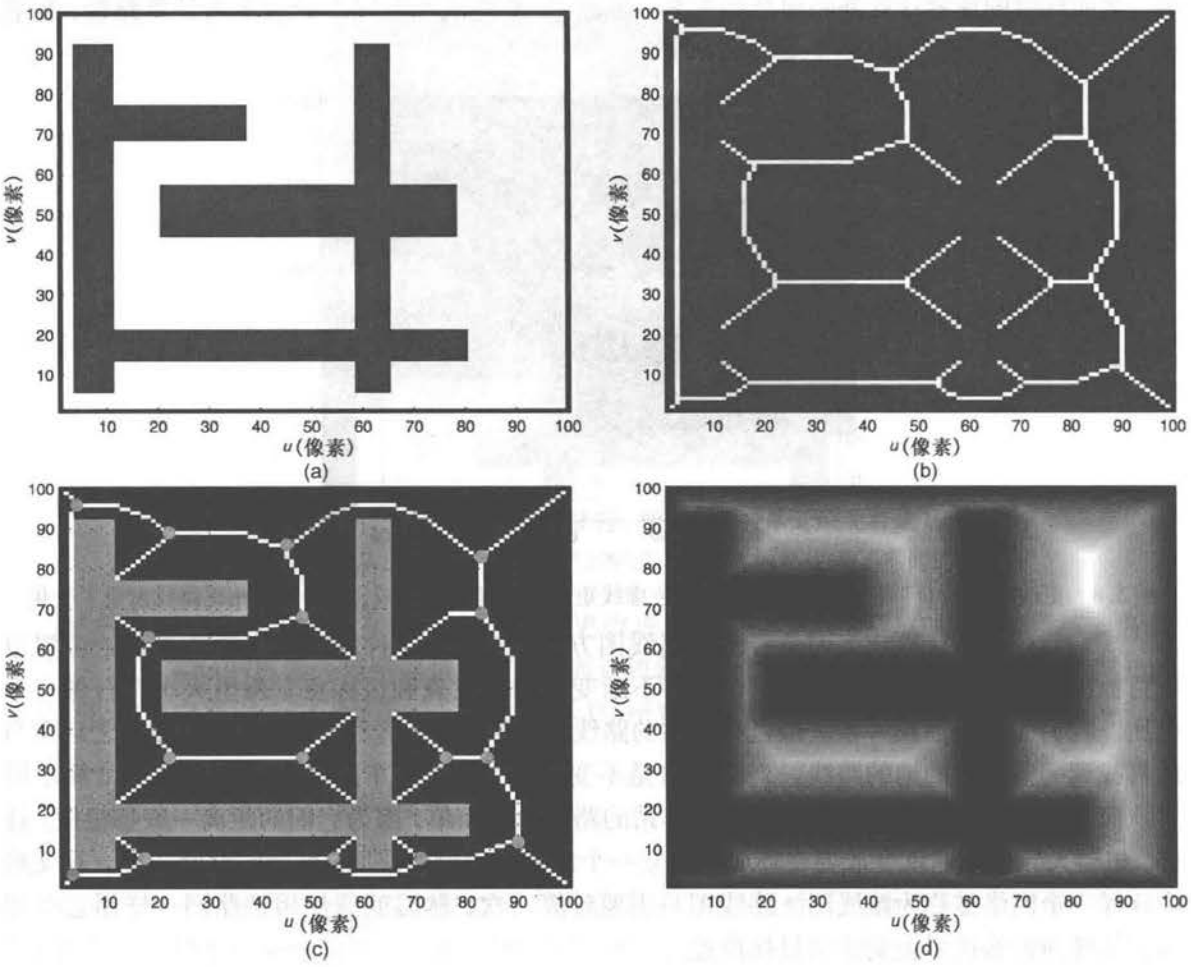


图 5.9 生成一个沃罗诺伊路线图的步骤。(a)白色单元表示自由空间；(b)自由空间框架是用不超过一个单元厚度的相邻单元构成的网络；(c)灰色区域表示障碍覆盖的框架，灰色点表示地图的连接点；(d)障碍的距离转换，像素值和离得最近的障碍的距离对应

将一种称为细化的形态学图像处理算法作用到图 5.9(a)上，可以算出自由空间的拓扑骨架：

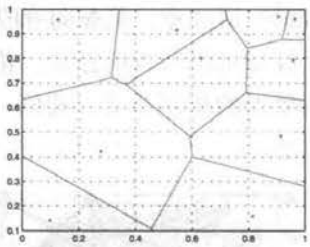
```
>> skeleton = ithin(free);
```

计算结果如图 5.9(b)所示。我们看到障碍物区变大，而白色单元代表的自由空间变成了细线相连的网络，它们与最初障碍物的边界是等距的。其中有关的图像处理函数，特别是形态学算子，将在第 12 章深入讨论。

图 5.9(c)显示的是自由空间网络叠加到原图上的效果[①]。我们已经创建好了一个覆盖全区域的路径网络，可以用它来实现避开障碍物的移动。这些路径就是一般化沃罗诺伊图的边。我们也可以通过计算图 5.9(a)中障碍物的距离变换得到类似结果，如图 5.9(d)所示。每个像素的值代表它到最近障碍物的距离，其脊线与图 5.9(b)所示的骨架是对应的。虽然细化或骨架化算法也与距离变换一样是计算量很大的迭代算法，但它为我们很好地展示了在自由空间中寻找路径的方法和原理。下一节将讨论一种相对简单的算法。

① 路线图上的接合处用灰色圆点表示。接合处或三叉点可以用形态学图像处理函数 triplepoint 来识别。

背景材料：一组平面点构成的沃罗诺伊网格或多边形，就是如右图所示的一些沃罗诺伊单元。每个单元对应一个多边形，并且包含了所有离该多边形较近的点。单元的边到最近的多边形是等距的。在广义的沃罗诺伊图中，所包含的单元是通过测量其到对象而不是到点的距离来定义的。在 MATLAB 中，可以通过以下指令产生一个沃罗诺伊图：



```
>> sites = rand(10,2)
>> voronoi(sites(:,1), sites(:,2))
```

乔治·沃罗诺伊(Georgy Voronoi，1868—1908)是一名俄国数学家，出生于现在的乌克兰。他曾在圣彼得堡大学学习，是安德烈·马科夫(Andrey Markov)的学生。作为沃罗诺伊的学生，鲍里斯·德劳内(Boris Delaunay)提出了同样著名的三角剖分法，它具有沃罗诺伊图的双重特性。

5.2.4　概率路线图方法

前面介绍的距离变换和骨架化方法的计算量都很大，对于较大的地图不适用。为改善上述问题，于是产生了基于概率的方法。这些方法在全局地图上稀疏采样，其中最著名的方法就是概率路线图法，或称 PRM 法。

用工具箱实现这个方法的第一步是创建一个 PRM 对象：

```
>> prm = PRM(map)
```

然后生成规划：

```
>> prm.plan()①
```

注意，这种方法并不把 goal 作为一个参数传给规划器，因为规划生成与目标位置是独立的。产生一条路径分为两个步骤：规划和查询。规划阶段首先在自由空间中找 N 个随机点，默认值是 100。然后用直线将每个点与其最近的邻点连接，不穿过障碍，这样就形成了一个节点数最小且开环的网络图。PRM 算法的优点就在于只要检验有限个点就可以确保所有点及其形成的路径是不穿过障碍的网。计算结果就存储在 PRM 对象中，该对象的概要可显示如下：

```
>> prm
prm =
PRM: 100x100
  graph size: 100
  dist thresh: 30.000000
  100 vertices
  712 edges
  3 components
```

其中给出了图中边和连接元素的数量。结果可以用下面的指令画出：

```
>> prm.visualize()
```

如图 5.10(a)所示。图上的灰点就是随机选出的点，点之间的连线都不穿过障碍物。只有小于 30 单元长的路径被选出来，这也是 PRM 类的距离阈值。图上的每条边都有一个相关联的成本值，代表了其两端节点之间的距离。节点的颜色表示它属于哪一个组件，每个组件被分配一个特定的颜色。我们看到图的左边缘处有两个点与路线图其他部分没有连接。

查询阶段即找到一个从起点到终点的路线。这时只需要简单地从起点移动到离它最近的节点，然后就沿路线图走，最后再从离终点最近的节点离开。

```
>> prm.path(start, goal)
```

① 为了能复制出下面的结果，一定要先用 randinit 来初始化随机数产生器。详见第 97 页。

上述指令会产生一个机器人在图上从起点到终点的移动动画，它走过的路线如图 5.11(见彩图)所示。请注意，在查询阶段我们给出了起点和终点位置。路线图的下一个节点用黄色表示，而绿点线显示机器人的路径。沿路线图移动就是总向下一个具有最小成本的相邻节点前进，即离终点最近的相邻节点。我们一直重复以上步骤，直至到达离终点最近的节点，然后就直接向终点移动。

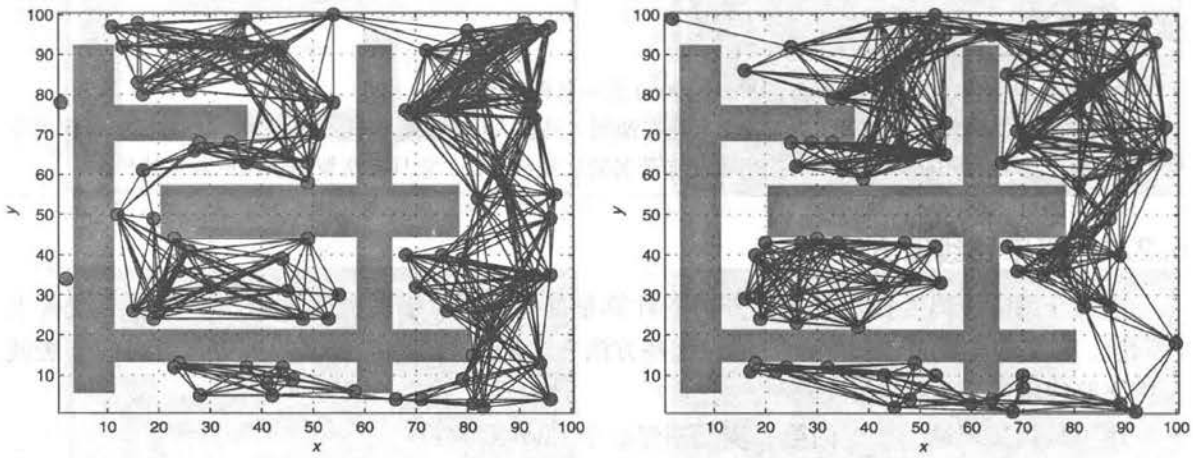


图 5.10 概率路线图(PRM)规划器以及规划阶段产生的随机图。(a)几乎全部连接起来的图，除了左边缘的两个节点；(b)包含一个大的未连接组块的图

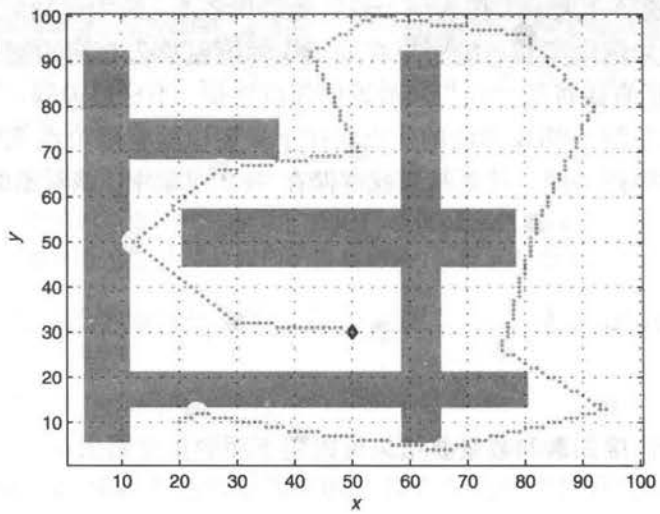


图 5.11 概率路线图(PRM)规划器得到的机器人路径，用绿点线表示。机器人走过的路线图节点用黄色加亮显示

这种规划器的优点就是一旦在规划阶段生成好路线图，以后就可以低成本地改变起点和终点，只需重新进行一遍查询即可。

但这种方法算出的路径并不是最优的，它经过的总距离为

```
>> p = prm.path(start, goal);
>> numcols(p)
ans =
   299
```

可以看出它比距离转换法得出的距离大，但优于 bug2 法。

为了提高运算效率，我们不得不做出一些重要的牺牲。首先，在自由空间中随机取样意味着每次运行该规划器都会产生一个不同的路线图，从而导致生成不同路径及路径长度。例如，我们再运行一次规划器：

```
>> prm.plan();
```

它将产生如图 5.10(b)所示的图，该图具有与图 5.10(a)不同的节点和不同的边。

背景材料 1：随机数。MATLAB 的随机数发生器(用于 rand 和 randn)会产生一个很长的数列，该数列是一个非常好的近似随机序列。发生器会保持一种内部状态，使之处于该数列中一个位置。启动后，MATLAB 总是产生下面的随机数序列：

```
>> rand
ans =
    0.8147
>> rand
ans =
    0.9058
>> rand
ans =
    0.1270
```

在本书中讨论的许多算法都使用了随机数，这意味着产生的随机数可能永远不会重复。在本书所有这样的例子之前都隐含地调用了 randinit 函数，它使随机数发生器复位到一个已知状态：

```
>> randinit
>> rand
ans =
    0.8147
>> rand
ans =
    0.9058
```

可以看到，随机序列已经被重置。

背景材料 2：一个真正的机器人不是一个点。我们前面都假设机器人是一个点，只占用网格中的一个单元。但如果实际机器人比一个单元大，那么规划所得的那些紧贴障碍物侧面的路径就不适用了。要解决这个问题，与其改变功能强大且性能优越的规划算法，不如让障碍物变形。采用闵可夫斯基和(Minkowski sum)使障碍物膨胀，使之能包容以最差姿态靠近的机器人。这样所有障碍将由虚拟障碍取代，该虚拟障碍是由实际障碍物与紧贴障碍物的所有姿态机器人组合而成的。机器人的所有姿态都可以保守地模型化成一个包含机器人的圆圈。

如果把占用网格看作一张图片，那么障碍物的膨胀可以通过使用一种图像处理技术来实现，该技术被称作扩张(见 12.5 节)。用一个半径为 3 单元的圆圈来膨胀障碍物，可以使用如下工具箱指令：

```
>> prm = PRM(map, 'inflate', 3);
```

其次，如果创建的网络中包含未连接的元素，规划任务就算失败。在图 5.10(a)的左侧就有两个未连接的节点，而在图 5.10(b)中则有一个大的未连接组块。如果起点和终点位置没有通过路线图连在一起，即它们分属不同的组块，Path 方法就会报告错误。该问题的唯一解决办法就是重新运行规划器。

再次，障碍物之间狭长的通道很难被采纳进路线，因为随机选中这些狭长通道中点的概率很低。例如在本例中，规划器选择的就是逆时针绕过障碍物的较长的路线，而不像距离变换规划器选择的通过左侧狭长通道的更优路线。

5.2.5 RRT

下面将介绍最后一种规划器，该规划器可以考虑机器人的运动模型，从而不用像以前的规划器那样假设机器人具有全方向运动能力。

图 5.12 显示了在不同前进后退速度以及转向角情况下，方程(4.2)给出的自行车模型在一定时间内行走出的路线族。它清楚地表示了一个非完整约束机器人从一个给定初始位置可以到达的全部可能位形的子集。在这个例子中，我们已经从一个初始姿态计算出一个时期后 22 种机器人可以达到的姿态。下一个时期后，从每一个姿态又可以算出另外 22 种机器人姿态，以此类推。多个时期之后，我们就能得到大量的可能姿态值。

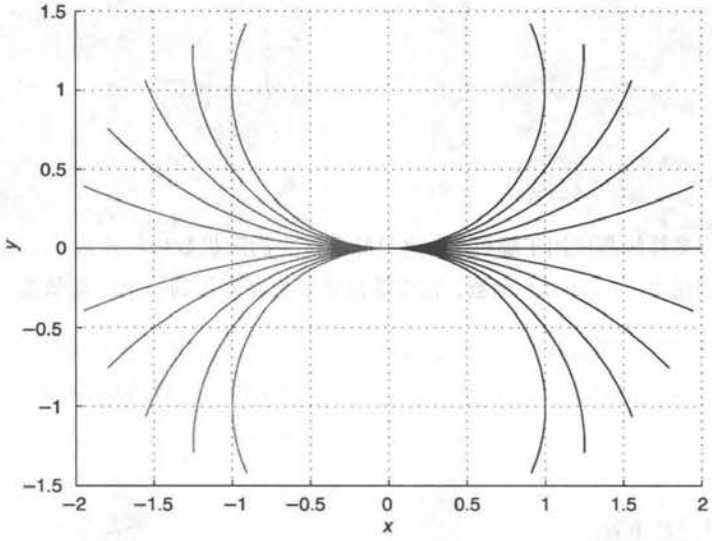


图 5.12 自行车模型的机器人从初始位置(0,0,0)出发可能行走的一组路径。经过时间为2秒，且$v = \pm 1, \alpha \in [-1, 1]$，左侧的浅灰线对应$v < 0$

对于任何期望的目标姿态，我们总能找到一个与之最接近的预先计算姿态。再以此姿态倒推回起始姿态，就可以得出从初始到目标姿态需要经历的一系列转向角和速度值序列。这与之前讨论过的路线图方法有些相像，但其限制因素是姿态数量的组合爆炸。

我们讨论的这种规划器称为快速搜索随机树(Rapidly-exploring Random Tree)，简写为 RRT。它和 PRM 一样，也是一种概率算法，主要步骤如下：保持一份机器人的位姿图，其中每个节点对应一个机器人位姿为$\xi \in SE(2)$，用三维向量$\xi \sim (x, y, \theta)$表示。图中的第一个节点是机器人的某个初始位姿。然后选择一个随机位姿ξ_{rand}，再从图中找出一个与该随机位姿最接近的节点ξ_{near}——接近程度是用包括距离和方向的成本函数来衡量的。接着计算出一个控制指令，使机器人在一个固定时间段内从ξ_{near}移动到ξ_{rand}。它达到的点记作ξ_{new}，并加入到图形中[①]。

我们还是采用熟悉的编程模式来生成一个避开障碍物的 RRT 路线图。我们先创建一个 RRT 对象：

```
>> rrt = RRT()
```

其中包括一个默认的自行车运动模型，以及速度和转向角的限制。

① 距离测量必须考虑位置和方向的差异，并且需要给这些参量适当的权重。但是仅从单位的角度考虑，这样做不是十分恰当，因为我们把距离的单位米和方向的单位弧度直接相加。

我们创建了一个规划，并绘制出结果：

```
>> rrt.plan();
>> rrt.plot();
```

规划图如图 5.13 所示。可以看到在位姿空间中各个路径都有良好的收敛性，不仅在 x 和 y 方向上，还包括姿态，这也是该算法被称为“快速搜索”的原因。

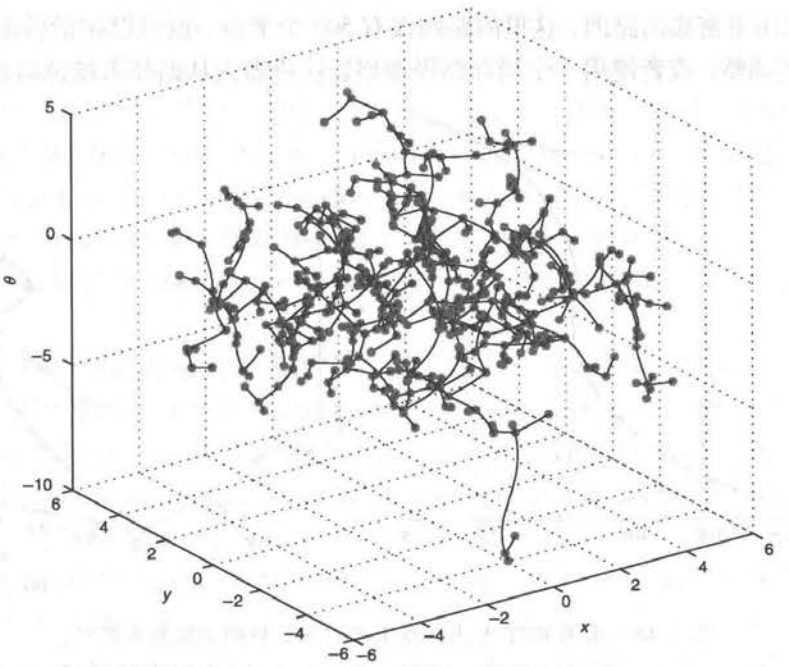


图 5.13　用自行车运动模型计算得到的一个 RRT，其中速度区间为 $\pm 1\ \mathrm{ms}^{-1}$，转向角区间为 $\pm 1.2\ \mathrm{ms}^{-1}$，运行时间为1s，初始位姿为(0,0,0)。每个节点用灰色圆点表示，代表三维空间里机器人的位姿(x,y,θ)

RRT 算法一个很重要的部分就是要计算一个控制输入让机器人从起始点行进到 ξ_{rand}。在 4.2 节中我们已经看到了将一个非完整约束的机器人驱使到某个特定姿态的难度。但这里我们将不使用像 4.2.4 节中那样的非线性控制方程，而是采用一种更简单的、类似于随机抽样策略的方法。控制器先随机选择机器人前进或后退以及转向角，再模拟出一个固定的时间段内自行车模型的运动，最后计算出到 ξ_{rand} 最近的距离。如此重复多次，选择出最佳的控制输入。在其路径上最接近 ξ_{rand} 的一点被选出作为 ξ_{near}，并成为图中一个新的节点①。

用 RRT 处理障碍问题也十分简单。如果点 ξ_{rand} 位于障碍物内部就直接舍去；如果从 ξ_{near} 到 ξ_{rand} 的路径与障碍物相交就不把点 ξ_{near} 加入图中。得到的结果就是这个非完整约束机器人可行驶并不会碰障碍物的一组路径，即路线图。

我们用一个非完整约束机器人的横向移动来说明上述问题。具体来说我们要找到一个路径使机器人横向移动 2 m，并保证机器人头部指向在初始和最后两个位置相同：

```
>> p = rrt.path([0 0 0], [0 2 0]);
```

其结果是一个连续的路径：

```
>> about(p)
p [double] : 30x3 (720 bytes)
```

① 在转向角限值内均匀随机分布。

我们可以绘制出该路径：

```
>> rrt.path([0 0 0], [0 2 0]);
```

结果如图 5.14 所示。这是一条非完整约束机器人可行的光滑路径。该机器人具有一个初始的方向角为 0，这意味着它是朝向正 x 轴方向，所以在这里它已经向后移动到期望的姿态。注意，运动并没有正好在期望姿态结束，而是结束于路网中最接近期望姿态的一个节点。要减小这个误差，我们可以采用更密集的路网，这里的路网仅有 500 个节点，也可以在最后运动段通过施加转向命令进行一些调整，或者使用一个局部路径规划器让机器人从路径末端再运动到目标姿态。

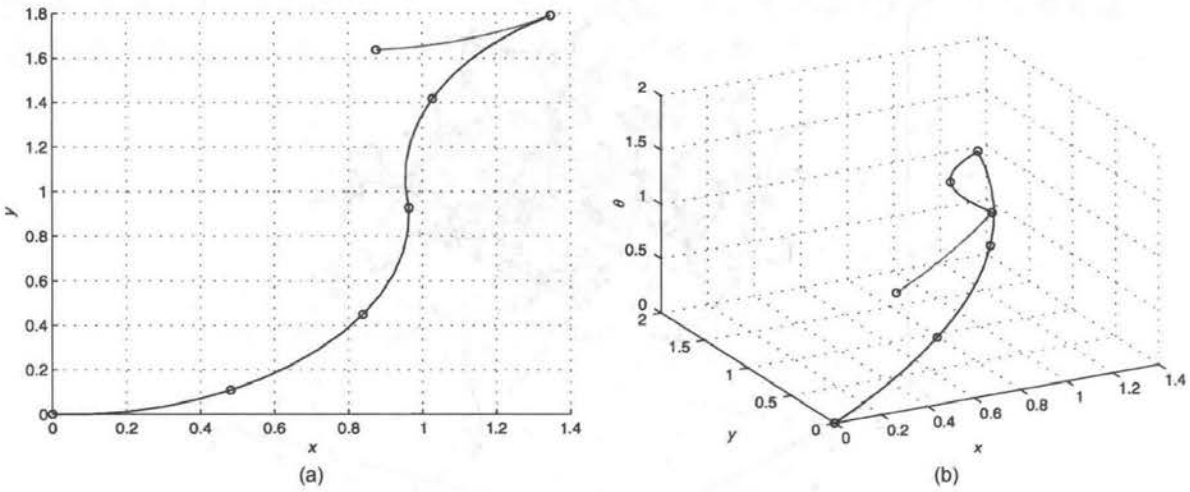


图 5.14 采用 RRT 方法计算出来的非完整约束机器人路径：
(a) xy 平面内路径；(b) (x, y, θ) 空间内的路径

5.3 本章总结

机器人导航问题是如何引导机器人朝着一个目标前进，本章已经介绍了多种方法。最简单的是纯反应式的 Braitenberg 型车。然后我们通过增加有限的记忆创建了基于自动机的状态机器，如 bug2，它可以躲避障碍，但是找出的路径并非最佳。

随后我们介绍了几种不同的以地图为基础的规划算法。距离变换是一种计算密集型的方法，可以找到一个到达目标的最优路径。D* 也可以找到一个最优路径，但它依赖于每个单元的可通过性，而不是把它们看作是自由空间或障碍物。在地图有细小变化时，D* 还支持计算量不大的增量式重新规划。PRM 方法通过随机采样减小了计算的负担，但代价是得到的路径不是最佳，尤其是它可能无法找出自由空间之间的狭窄路线。另一种采样方法是 RRT，它使用机器人的运动模型来生成一组可行驶路径，而且它们还考虑到了机器人的方向和位置。所有以地图为基础的方法都需要一张地图，并要知道机器人的位置，而这两个问题将在下一章探讨。

扩展阅读

最早提出控制论的专著是由维纳在 1948 年写成的，并于 1965 年再版[Wiener, 1965]。格雷·沃尔特则根据他关于机器龟的理论及实验，发表了许多受欢迎的文章[Walter, 1950, 1951]，并且出版了一本书[Walter, 1953]。

关于 Braitenberg 车最权威的参考书是他们自己的专著[Braitenberg, 1986]，它是一系列异想天开且富有诗意的思维实验。作者开发了复杂性不断增加的车(共有 14 种车)，其中有些包括非线性、记忆和逻辑，从而使这些车具有类人的特性，如爱、恐惧、攻击性和自负。书的第二部分描述了这些机器具有的类似动物神经结构的事实基础。bug1 和 bug2 算法最早由鲁梅尔斯基和斯特帕诺夫提出[Lumelsky and Stepanov, 1986]。最近，11 种变种的 bug 算法也已实现，并在许多不同的环境中开展了对比试验[Ng and Bräunl, 2007]。

早期基于行为的机器人包括建于 20 世纪 60 年代的约翰霍普金斯野兽(Johns Hopkins Beast)，和建于 1989 年的成吉思(Genghis)[Brooks, 1989]。阿尔金的书[Arkin 1999]和《机器人技术手册》(*Robotics Handbook*)[Siciliano and Khatib, 2008, §38]里都论述了基于行为的机器人。马塔里奇的《机器人入门》(*Robotics Primer*)[Matarić, 2007]及其相关的网络资源也对反应式控制、基于行为的控制和机器人导航做了很好的介绍。更多关于自动控制机器方面的资料还可以在网站 http://cyberneticzoo.com 中找到，其中包括格雷·沃尔特的机器龟和约翰霍普金斯野兽。

伯格夫斯在 1986 年就对距离转变进行了很好的论述[Borgefors, 1986]，而它在机器人导航上的早期应用则由贾维斯和拜恩在 1988 年开发[Jarvis and Byrne, 1988]。D* 算法是经典的 A* 算法[Nilsson, 1971]的一种扩展，它是 1994 年由斯坦茨提出的[Stentz, 1994]，后来的演变包括 Field D*(Ferguson and Stentz, 2006)和 D* lite(Koenig and Likhachev, 2002)。D* 算法在 DARPA 挑战赛中被很多汽车所使用。

PRM 的思想在 20 世纪 90 年代中期就开始出现，由卡夫拉奇等人首先提出[Kavraki, 1996]。随后在基本 PRM 算法基础上又出现了许多变异算法，吉拉尔茨和奥维马斯对这些算法的功效进行了比较[Geraerts and Overmars, 2004]。集成了机器人动力学的规划方法包括状态空间采样法[Howard et al., 2008]和 RRT。RRT 在拉瓦勒的专著[LaValle, 1998, 2006]及网站 http://msl.cs.uiuc.edu/rrt 中都有详细介绍。

最近有两本书几乎堪称机器人规划方面的百科全书。乔塞特等人所写的关于机器人运动的专著[Choset et al., 2005]涵盖了几乎所有几何和随机规划方法，以及它们在具有动力学和非完整约束的机器人上的应用。拉瓦勒所写的关于机器人运动规划的专著[LaValle, 2006]涵盖了运动规划、非确定性规划、基于传感器的规划、强化学习、非线性系统、轨迹规划和非完整性规划。这些书中讨论的规划技术非常强大，不仅可应用于机器人系统，还可以应用到诸如复杂机构甚至是分子结构等的高维系统。更简明的介绍可参看[Siegwartet al., 2011]、[Spong et al., 2006]、[Siciliano et al., 2008]和《机器人技术手册》[Siciliano and Khatib, 2008, §35]。

习题

1. Braitenberg 车(5.1.1 节)。
   a)采用不同的初始位姿和控制增益进行实验。
   b)修改转向信号的标志，使机器人躲避光亮。
   c)修改 `sensorfield` 函数，使峰值随时间变化。
   d)机器人逐渐接近最大值。通过编程增加一个停止动作，当任何一个传感器检测到大于 0.95 的值时机器人停止。
   e)建立一个有两个峰值的标量场。你能找出一个初始姿态，使机器人在这个标量场里不知所向吗?

2. bug 算法(5.1.2 节):
   a) 使用 makemap 函数来创建一张新的地图来挑战 bug2。试试不同的起点。容易使 bug2 陷入困境吗?
   b) 创建一张含有迷宫的障碍图,bug2 可以走出迷宫吗?
   c) 试验其他的 bug 算法如 bug1 和 tangent bug,它们表现得更好还是更糟?
3. 如果一个单元的大小是 1 米,那么需要多少内存来表示地球表面?又需要多少内存来表示地球上的陆地面积?如果仅有 1GB 的内存可用,贵国的地图需要多大的单元尺寸?
4. 距离变换(5.2.1 节)。一个真实的机器人具有确定的尺寸,如果想用理想化的机器人点的规划方法,通常的做法是将障碍物大小扩大到机器人半径的一半。尝试使用工具箱函数 imorph(见第 317 页)使障碍物膨胀 4 个网格单元。
5. D* 规划器(5.2.2 节)。增加崎岖地形的成本,观察会发生什么。在机器人路径旁边添加一个地形成本非常低的区域(小于 1),观察会发生什么。
6. PRM 规划器(5.2.4 节)。
   a) 运行 PRM 规划器 100 次,收集产生的路径长度的统计数据。
   b) 改变距离阈值参数并观察效果。
   c) 实现一个无网格的 PRM 规划器。该机器人由任意的多边形表示,如障碍物一样。你将需要一些功能函数来确定一个多边形与另一个多边形是否相交还是是否包含关系(参见工具箱 Polygon 类)。请在钢琴搬运工问题上测试此方法。
7. RRT 规划器(5.2.5 节)。
   a) 找到一条路径来实现三点转弯。
   b) 定义一个障碍场并重复规划。
   c) 用不同的 RRT 参数做实验,如点的数目、机器人转向角限值以及路径完成时间。
   d) 前述 RRT 选择的是转向角在限值范围内均匀分布这一模式。但人们往往会更谨慎地驾驶,如果选择转向角高斯分布会发生些什么?
   e) 每幅图节点处的附加信息决定到达该节点的控制输入。绘制出从初始位姿移动到目标位姿这个过程中的转向角和速度的变化。
   f) 设计一种局部规划器,使机器人从初始位姿行驶到最近的节点,以及从最后的节点行驶到目标位姿。
   g) 确定一条穿过图形的路径,以使机器人移动时方向的折返最少。
   h) 假设机器人是一个有限大小的矩形,环境中的障碍是多边形,尝试添加一个更复杂的碰撞检测器。你将需要一些函数来确定是否一个多边形与另一个多边形是相交还是包含关系(参见工具箱 Polygon 类)。

第6章　机器人定位

想要去哪里，先要知道自己当前在哪里

在前面基于地图的导航讨论中，我们均假设机器人有办法知道自身的位置。在本章中，我们就要探讨一些能估计机器人实际位置的常用方法，这个过程称为定位。

今天，用 GPS 实现室外环境定位非常容易，以至于我们常常认为其无所不能。然而 GPS 远非完美无缺的定位传感器，因为它完全依赖远距离在轨卫星发送来的微弱无线电信号。这就意味着在无线电信号无法到达的环境中 GPS 是无法工作的，例如在室内、水下、地下，甚至在城市街道峡谷中，或在深挖的矿坑里。另外，GPS 信号还极其微弱，而且很容易拥塞，这对于某些应用场合也是不可接受的。

GPS 的使用仅仅始于 1995 年，而人类定位导航的历史已经有几千年了。本章将介绍一些经典导航原理，如航迹推算和路标使用，它们是现代机器人导航的基础。

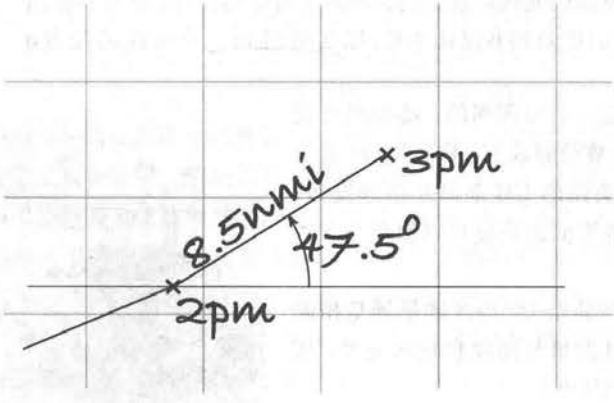


图 6.1　采用航迹推算法进行位置估计。船在下午三点所处的位置可以通过其两点钟所处位置、预测航行距离和平均罗盘航向来确定

航迹推算是一种基于预测的速度、方向、运行时间以及前一个位置信息，对下一个位置进行估计的方法。图 6.1 展示了如何在航图上推算船位置的方法。给定前一小时内船的平均航向以及下一小时要航行的距离，运用初等几何知识就能推算出从下午两点起到下午三点船所在的位置。然而，这种推算测量方法既有系统误差，也存在随机误差。现代测量仪器的精度都很高，但 500 年前的时钟、指南针和测速仪则比较原始，精度较低。在上述这种递归式推算过程中，每一次估计都要基于上一个位置的信息，这样就会产生误差累积，因此这种方法在长达几个月的海上航行中是不可靠的。

背景材料：海上速度测量。船舶计程仪是用来估计船已经行驶距离的一种仪器。最古老的测量船在海上行驶速度的方法是荷兰人制作的计程仪——一个可以漂浮的物体被从船头扔进水里，然后用一个沙漏测量它到达船尾的时间。后来出现了拖板计程仪，它采用一块四分之一圆形状的木板，圆弧底端挂上铅垂，这样既可以使木板直立漂浮，同时又不会被水流带走。使用时将它抛到海里，同时它上面还拖带了一条绳索，绳索上每隔 50 英尺就打了一个结。一个被称作计程仪杯的特殊沙漏会计时 30 秒。在那段时间里，记录被拖下船的绳结数目，每个绳结对应的船速将近 1 海里/小时，或 1 节。1 海里(nmi)现在定义为 1.852 千米。

腓尼基人4000年以前就在海上航行了，但他们当时还没有指南针——指南针是2000年后才由中国人发明的。腓尼基人采用最原始的航迹推算法进行导航，但他们随时会用其他信息来校正其定位误差——认识的岛屿和陆岬、原始地图以及太阳和北极星的位置，等等。

地标是指环境中的一个可见特征，其位置在参考坐标系中已知。示意图6.2展示了一张地图和多个灯塔地标，以及如何用它们来确定所在位置。首先，我们使用一个指南针将地图向北轴线与地球北极方向对准。然后利用一个地标的方向在地图上画出一条定位方向线，再用另一地标志在图上画出第二条定位方向线，两条方向线的交点必然就是我们所在的位置——该过程被称作后方交会法[①]。例如，图中灯塔 **A** 确定的方向线为黑线，灯塔 **C** 确定的方向线为灰线，两线交点 **p** 即是我们的实际位置。

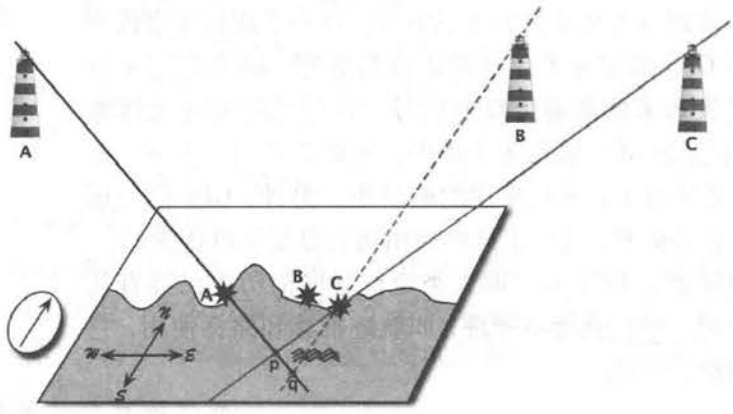
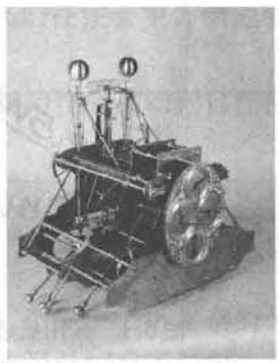


图6.2 用地图估计位置。利用灯塔 **A** 和 **C** 发出的光线以及它们在地图上对应的位置，就可以估测出我们所在的位置**p**。然而，如果我们把灯塔**B**误认为**C**，那么将得到一个错误的位置**q**

背景材料：天文导航。天体在天空中的位置是一个可预测的，随时间及观测者所在经度、纬度变化的函数。该位置信息可以被绘制成表，称作星历（意为“每日”）。这些星历数据自1767年开始，每年由英国的HM航海年历局发布，名为“航海年历（*The Nautical Almanac*）”。天体到地平面的高度可以使用一个六分仪来测量，它是一种手持式光学仪器。



时间和经度是耦合的，某地一小时后的星场与其东15°地区的星场是相同的。然而由于北极星距离地球北极非常近，因此其仰角与经度和时间无关，从而可以仅用一个六分仪就能方便地测出所在位置的纬度。

解决经度测量问题便成了18世纪欧洲各国最艰巨的科学挑战，因为它严重阻碍了全球航运及海上霸权。1714年，英国甚至专门通过了一个“经度法案”（British Longitude Act），为此设立一个20 000英镑的大奖。这极大促进了诸如航海计时器、航海时钟这类船舶用高精度仪器的发展。50多年后，约翰·哈里森（John Harrison）开发出了一个精良的计时器，随后詹姆斯·库克船长就在他的第二次远航（1772—1775）中使用了这个计时器的复制品为船队导航。经过3年的航行，用这种计时器对经度测量的误差仅有13千米。随着计时精度的提高，恒星的仰角就可以用来估计经度和纬度。这一技术的进步促进了全球贸易和新大陆探索。

然而，这个过程完全依赖于对被观察地标与地图上对应特征的正确匹配。例如，如果我们

① 后方交会法是通过测量相对已知地标的方位角来定位。三角测量法则是通过测量相对未知地标点的方位角来定位。

看错了一个灯塔，把看见的 **C** 误认为是地图上的 **B**[①]，那么由灰色虚线就会定位出一个错误位置 **q**，而我们还会误认为自己身在位置 **q**。这种差错会导致我们过估船与海岸之间的距离。如果我们这时选择将船驶向岸边，就会发生触礁，那时也许我们还在惊讶为什么礁石会出现在本不该出现的位置。遗憾的是，这种错误在现实中经常发生，数不清的船只为此葬身大海。这也就是为什么要让灯塔闪烁发光的原因！到 18 世纪，技术的进步已经可以让灯塔发出特有闪烁模式的光线，这样船员就能可靠识别出特殊的灯塔，并与海图上的对应点正确匹配。

当然，最早的水手根本没有地图、灯塔、甚至指南针，他们只能通过一次次的航行来自己创作地图，从已知海域逐渐向外推进，不断标示出那些非人工特征物的位置。这项工作导致很多早期探路者命丧大海，因而很长一段时期海事地图都是国家机密[②]。

今天，机器人在没有 GPS 的环境中作业，面临与古代航海同样的问题，当然也大量借鉴了有几百年历史的航海导航策略。机器人运动时，对自己行驶过的距离很难准确估计，而且它可能也没有地图，或者仅有一张不精确或不完整的地图。虽然通过观测周围特征地标获取附加信息有助于减小机器人的定位误差，但信息数据错误匹配的可能性仍然存在[③]。

下面，我们给机器人定位问题一个更正式的描述：$\boldsymbol{x}$ 是机器人所在真实但未知的位置，而 $\hat{\boldsymbol{x}}$ 是我们用最好方法估计出的机器人位置。我们还希望知道这种估计的不确定度，用统计学术语就是与估计位置 $\hat{\boldsymbol{x}}$ 相关的标准差。

背景材料：无线电定位。最早的无线电定位系统之一是 LORAN，它是基于英国二战时的 GEE 系统建立起来的。世界各地的 LORAN 发射器发射同步的无线电脉冲，然后由一个接收器测出从两台发射器来的无线电脉冲抵达时间差。如果已知两台发射器的位置及其脉冲时间差(TD)，那么接收器的位置就被约束在一条双曲线上(导航图上标示为 TD 线)。如果再使用另外一对发射器(可以包含面前那一对中的一个)，给出另一条约束双曲线，那么接收器必须位于这两条曲线的交点上。

全球定位系统(GPS)最早于 1973 年提出，但直到 1995 年才真正投入使用。目前，该系统包括大约 30 个卫星，它们在距离地球表面 20 200 千米的 6 个平面上运行。某个 GPS 接收器工作时，它会测量从 4 个或者更多卫星发射来的无线电信号的传输时间，而每个卫星的轨道位置已经编码进了 GPS 信号中。已知空间中 4 个卫星位置及其无线电信号的延迟时间，就有可能计算出接收器的位置$(x、y、z)$以及时间信息。如果 GPS 信号是经过了某个表面反射后才被收到，那么它传播的路程就会增加，并且会产生位置估计的误差。这种影响被称为“多路径效应”，在有大规模工业基础设施的环境中很常见。

而在空旷环境中，无线电波穿过大气层时的传播速度变化是产生位置估计误差的主要原因。然而，这种误差随时间变化非常缓慢，在一个大的区域内可近似为常数。这样就可以在 GPS 基准站测量这个误差，然后传送给附近兼容的接收器，用以修正该误差——这种技术被称为差分 GPS(DGPS)。许多国家在沿海都设有无线电广播网来为船舶播报这种修正信息。对飞机而言，这种信息要通过另一套卫星网络来播报，被称作“广域增强系统”(WAAS)。RTK GPS 能获得更高的时间测量精度，它使用了载波信号中的相位信息。最初的 GPS 系统还故意增加了测量误差，美其名曰“选择可用性”，主要是为了降低军事上敌方使用 GPS 的效力，但这种做法在 2000 年 5 月被废止。其他卫星导航系统还有俄罗斯的格洛纳斯、欧洲的伽利略和中国的北斗。

---

① 如果我们把看见的 **B** 误认为是地图上的 **C**，这也同样糟糕。

② 1519 年，麦哲伦探险队(共有 237 名船员和 5 艘船舶)起航。但大多数人都在途中失踪了，包括麦哲伦本人。只有 18 名船员和 1 艘船舶返回。

③ 轮式机器人可以通过测量车轮转动来估算行驶的距离，而空中或水下机器人就无法做到。但是测量车轮转动的方法也并非完美，主要由于轮径大小的变化和不确定性、相对地面的滑动以及转弯带来的影响。计算机视觉也可用来建立一个视觉测距系统，它通过观测经过机器人的环境变化来估算距离。

一种描述机器人位置的有效办法是用概率密度函数(PDF)来表示机器人所有可能的位置。图 6.3 所示是一些概率密度函数的例子，其中函数的幅值对应的是机器人出现在该位置的相对可能性大小。常用的概率密度函数是高斯函数，因其平均值和标准差都非常简洁。机器人最有可能位于函数峰值(平均值)所在位置，而距峰值越远的位置机器人出现的可能性越小。在图 6.3(a)中只有一个峰值，而且标准差非常小，这非常清楚地说明机器人所在位置就在峰值处，而出现在 * 号所示位置的概率几乎为零。图 6.3(b)存在一个较大的标准偏差，这意味着我们不太确定机器人的位置。机器人位于 * 号所示点的可能性也存在。使用概率密度函数也允许对机器人位置有多个设定，如图 6.3(c)所示的 PDF 图，基本上能够确定机器人会在四个峰值点之一。这种情况看似意义不大，但实际上很有用。假设有一个室内机器人，它观察到一台自动售货机。但在地图上标出有 4 台这样的售货机。因此在没有任何其他信息的情况下，机器人在这 4 台自动售货机中的任何一个附近都是等可能的。这个问题将在 6.5 节中再予讨论。

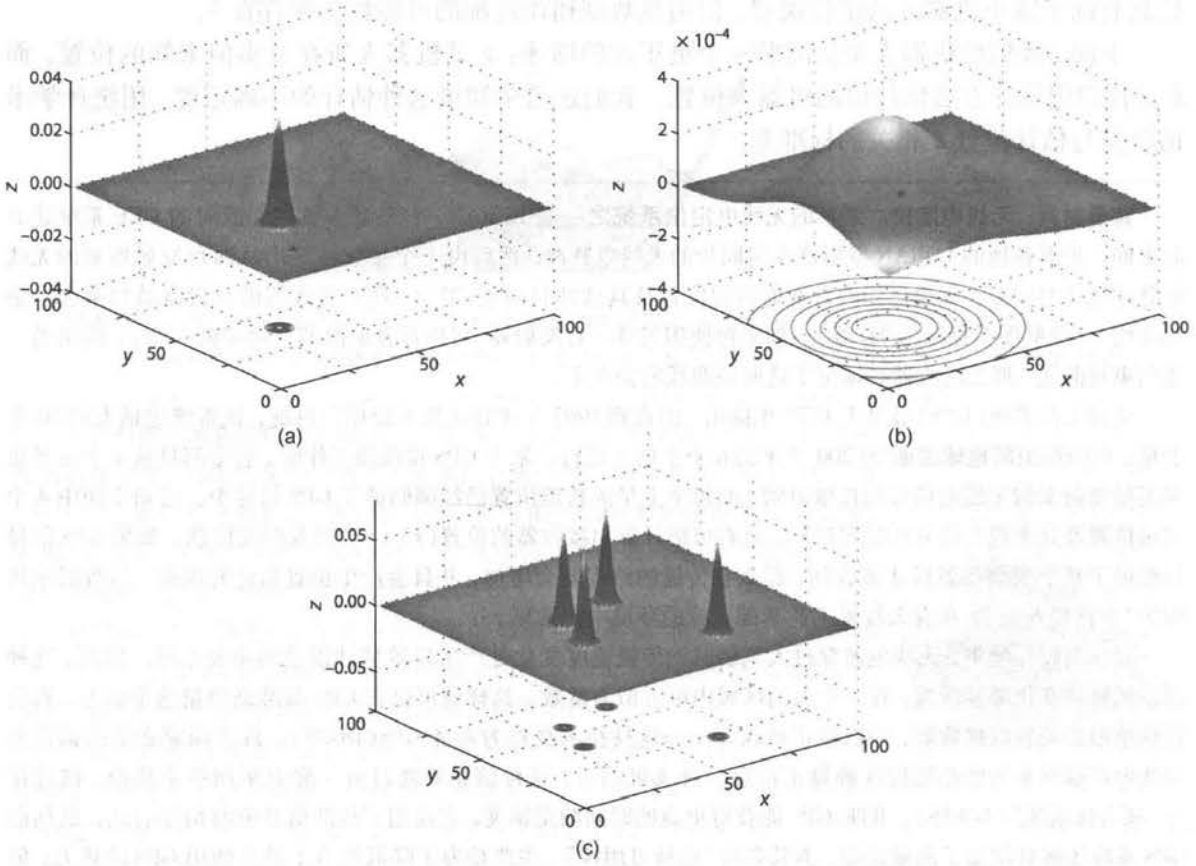


图 6.3 在 xy 平面上机器人位置的表示及其不确定性。垂直轴表示机器人在那个位置上的相对可能性大小，其等高线显示在下面的平面中。(a)机器人所在位置不确定性小，$\sigma = 1$；(b)机器人所在位置不确定性大，$\sigma = 20$；(c)机器人所在位置有多种可能性，每一个均是$\sigma = 1$

依据机器人的运动方式及其对周围环境的观测来确定其位置的 PDF，对这种估值问题可以给出一个普遍定义：

通过处理那些与 x 有某种相关性的数据，来推导出真正想要的 x 值。

例如，一名船舶导航员或土地测量员就是通过测量已知地标或天体的方位角来估计所在位置，而一个 GPS 接收器则是通过观察位置已知卫星的信号时间延迟来估算所在的纬度和经度。

对于我们的机器人定位问题，其真实状态与估计状态都用向量表示，因此估计的不确定性将用协方差矩阵表示(请参阅附录 F)。矩阵对角线上的元素表示相应状态的不确定性，非对角线元素则代表不同状态之间的相互关系。

6.1　航迹推算

航迹推算是在前一位置基础上，根据预计的速度、方向和运行时间来估计机器人下一位置的方法。

6.1.1　机器人建模

机器人位置估计的第一步是要写出一个函数 $\boldsymbol{f}(\cdot)$，用于描述机器人位形如何随时间变化。像方程(4.2)这样的机器人模型就把机器人的位形变化描述成其控制输入的函数，然而对于真正的机器人，我们很少有机会得到这些控制输入。大多数的机器人平台都有专用运动控制系统，它们能接受操作者发出的运动命令(速度和方向)，并告知操作者机器人的行程信息。

里程计是一种传感器，用于测量机器人的行驶距离，通常的办法是测量车轮的转动角度。机器人行进方向的测量可以使用电子罗盘，或者用陀螺仪或差分里程计来测量航向变化[①]。然而上述传感器都很难实现精确测量，原因在于其中存在系统误差——如车轮半径不准或陀螺仪漂移，以及随机误差——如车轮与地面之间打滑或转弯的影响[②]。尽管如此，在下面的讨论中仍然以里程计为例，来同时获取机器人的距离和航向信息。

我们不直接使用方程(4.2)，而是基于里程计信息写出一个关于机器人位形演变的离散时间模型，其中 $\delta\langle k\rangle = (\delta_d, \delta_\theta)$ 表示在上个时间间隔机器人驶过的距离及航向变化量，k 为时间间隔对应的序号。在 $SE(2)$ 中，机器人的初始位姿表示为

$$\xi\langle k\rangle \sim \begin{pmatrix} \cos\theta\langle k\rangle & -\sin\theta\langle k\rangle & x\langle k\rangle \\ \sin\theta\langle k\rangle & \cos\theta\langle k\rangle & y\langle k\rangle \\ 0 & 0 & 1 \end{pmatrix}$$

假设在时间间隔内的运动量都很小，这样施加运动的顺序就无关紧要了。我们选择先沿机器人 x 方向行进 δ_d，然后再旋转 δ_θ，从而得到一个新的位形：

$$\begin{aligned} \xi\langle k+1\rangle &\sim \begin{pmatrix} \cos\theta\langle k\rangle & -\sin\theta\langle k\rangle & x\langle k\rangle \\ \sin\theta\langle k\rangle & \cos\theta\langle k\rangle & y\langle k\rangle \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & \delta_d \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos\delta_\theta & -\sin\delta_\theta & 0 \\ \sin\delta_\theta & \cos\delta_\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \\ &\sim \begin{pmatrix} \cos(\theta\langle k\rangle + \delta_\theta) & -\sin(\theta\langle k\rangle + \delta_\theta) & x\langle k\rangle + \delta_d\cos\theta\langle k\rangle \\ \sin(\theta\langle k\rangle + \delta_\theta) & \cos(\theta\langle k\rangle + \delta_\theta) & y\langle k\rangle + \delta_d\sin\theta\langle k\rangle \\ 0 & 0 & 1 \end{pmatrix} \end{aligned}$$

或者将其表示成一个三元素向量为

① 通过测量左右两侧车轮的角速度差。

② 在转弯时，外侧车轮比内侧车轮运动快，这可以通过测量两个轮子的速度得到。

$$\xi\langle k+1\rangle \sim \begin{pmatrix} x\langle k\rangle + \delta_d\langle k\rangle \cos\theta\langle k\rangle \\ y\langle k\rangle + \delta_d\langle k\rangle \sin\theta\langle k\rangle \\ \theta\langle k\rangle + \delta_\theta \end{pmatrix} \tag{6.1}$$

上式给出了用前一个位形和里程计信息表示的新位形。

然而，上式成立的假设条件是从里程计得到的数据准确无误，但这是不现实的。为了给里程计的误差建模，我们分别给 δ_d 和 δ_θ 中增加一个连续随机变量 v_d 和 v_θ。这样在下一时刻，包含里程计误差的机器人新位形是

$$\xi\langle k+1\rangle = \begin{pmatrix} x\langle k\rangle + (\delta_d\langle k\rangle + v_d)\cos(\theta\langle k\rangle + \delta_\theta + v_\theta) \\ y\langle k\rangle + (\delta_d\langle k\rangle + v_d)\sin(\theta\langle k\rangle + \delta_\theta + v_\theta) \\ \theta\langle k\rangle + \delta_\theta + v_\theta \end{pmatrix} \tag{6.2}$$

这是所需要的函数 $\boldsymbol{f}(\cdot)$

$$\boldsymbol{x}\langle k+1\rangle = \boldsymbol{f}(\boldsymbol{x}\langle k\rangle, \delta\langle k\rangle, \boldsymbol{v}\langle k\rangle) \tag{6.3}$$

其中，k 为时间序号，$\delta\langle k\rangle$ 是里程计的测量数据，$\boldsymbol{v}\langle k\rangle$ 是上一时间间隔内的随机测量噪声①。

通常情况下，我们可以将里程计噪声建模为：$\boldsymbol{v} = (v_d, v_\theta) \sim N(0, \boldsymbol{V})$，一个零均值的高斯过程的方差为

$$\boldsymbol{V} = \begin{pmatrix} \sigma_d^2 & 0 \\ 0 & \sigma_\theta^2 \end{pmatrix}$$

该矩阵也称协方差矩阵，这里它是一个对角阵，表明在距离和航向上的误差是独立的②。要为 $\boldsymbol{V}$ 选择一个恰当的值并不容易，但我们可以通过做实验，或者做一些合理的工程假设来确定。在下面的例子中，我们在每个采样间隔取值 $\sigma_d = 2$ cm 和 $\sigma_\theta = 0.5°$，这样可得协方差矩阵为

```
>> V = diag([0.02, 0.5*pi/180].^2);
```

使用工具箱中的 `Vehicle` 类，可以仿真方程(4.2)描述的自行车模型，以及方程(6.2)描述的基于里程计的位形更新模型。使用时，我们要创建一个 `Vehicle` 对象：

```
>> veh = Vehicle(V)
Vehicle object
  L=1, maxspeed=5, alphalim=0.5, T=0.100000, V=(0.0004,0.00121847), nhist=0
  x=0, y=0, theta=0
```

上面列出了对象所含参数的默认值，如机器人长度、最大速度、极限转向角，以及默认为 0.1 秒的采样时间。该对象还提供了一个方法来实现单步仿真：

```
>> odo = veh.step(1, 0.3)
odo =
    0.1002    0.0322
>> odo = veh.step(1, 0.3)
odo =
    0.0991    0.0311
```

其中已经设定速度为 1 米/秒，转向角为 0.3 弧度。该函数能单步更新机器人的真实位形，并返回一个含有噪声的里程计读数。如果采样时间为 0.1 秒，那么它每次都会报告机器人移动

① 这种情况下，里程计噪声位于过程模型内部，因而被称为“过程噪声”。

② 实际上这种情况是不可能的，因为航向角变化量大时，里程计距离误差往往也会增大。

了约 0.1 米的距离，而且航向角改变了约 0.03 弧度。机器人的真实位形这里没有给出，但可以通过显示这个对象看见：

```
>> veh
veh =
Vehicle object
  L=1, maxspeed=5, alphalim=0.5, T=0.100000, V=(0.0004,0.00121847), nhist=2
  x=0.199955, y=0.00299955, theta=0.06
```

如果我们希望仿真过程持续较长一段时间，但又不想让机器人走出限定的空间区域，那么可以使用 RandomPath 类。这个类是一个驱动器，它能将机器人引导至某个特定区域内随机选择的地点。下面创建一个该驱动器对象的实例，并把它与机器人关联在一起：

```
>> veh.add_driver( RandomPath(10) )
```

其中，传给 RandomPath 构造函数的参数限定了一个工作区域，它是在 x 和 y 方向上各有 ±10 m 跨度的一个正方形。使用下面的函数，可以显示机器人在驱动器作用下的动画：

```
>> veh.run()
```

该函数将重复调用 step 函数，并将仿真过程中机器人真实状态的所有记录保存在 Vehicle 对象里[①]。RandomPath 类和 Vehicle 类还有很多参数和方法，相关资料可查阅其在线文档。

6.1.2　位姿估计

正如船舶导航一样，我们面临的问题是如何在已知上一个位姿和里程计噪声时，最好地估计出新的位姿。我们将用到卡尔曼滤波器这个数学工具，附录 H 中有其方法的完整阐述。假设噪声为零均值的高斯分布时，这个滤波器提供了一个对系统状态（这里为位置）的最优估计。在此应用中，卡尔曼滤波器的状态就是被估计的机器人位形。该滤波器使用一个递归算法，它在每个时间步上都会根据上一次估计值和噪声测量值，给出下一个未知真实位形的最优估计以及有关该估计的不确定性。也可以说，它提供了关于我们在哪里以及有多大的确定性的最好估计。

卡尔曼滤波器针对的是线性系统，但方程(6.3)给出的机器人运动模型却是非线性的，因此我们需要对函数 $\hat{\boldsymbol{x}}\langle k\rangle$ 进行一个局部线性近似或线性化：[②]

$$\hat{\boldsymbol{x}}\langle k+1\rangle = \hat{\boldsymbol{x}}\langle k\rangle + \boldsymbol{F}_x\big(\boldsymbol{x}\langle k\rangle - \hat{\boldsymbol{x}}\langle k\rangle\big) + \boldsymbol{F}_v\boldsymbol{v}\langle k\rangle$$

它是相对于当前状态估计值 $\hat{\boldsymbol{x}}\langle k\rangle$ 进行的线性化。其中，$\boldsymbol{F}_x$ 和 $\boldsymbol{F}_v$ 是雅克比矩阵，它们分别是导数 $\partial \boldsymbol{f}/\partial \boldsymbol{x}$ 和 $\partial \boldsymbol{f}/\partial \boldsymbol{v}$ 的向量形式。这种针对非线性系统的估计方法也被称为扩展卡尔曼滤波器，或者 EKF。有关雅克比矩阵的知识请参见附录 G。

通过对方程(6.2)求微分可以得到雅克比矩阵，在 $\boldsymbol{v}=0$ 时有

$$\boldsymbol{F}_x = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right|_{v=0} = \begin{pmatrix} 1 & 0 & -\delta_d\langle k\rangle \sin\big(\theta\langle k\rangle + \delta_\theta\big) \\ 0 & 1 & \delta_d\langle k\rangle \cos\big(\theta\langle k\rangle + \delta_\theta\big) \\ 0 & 0 & 1 \end{pmatrix} \tag{6.4}$$

$$\boldsymbol{F}_v = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{v}}\right|_{v=0} = \begin{pmatrix} \cos\big(\theta\langle k\rangle + \delta_\theta\big) & -\delta_d\langle k\rangle \sin\big(\theta\langle k\rangle + \delta_\theta\big) \\ \sin\big(\theta\langle k\rangle + \delta_\theta\big) & \delta_d\langle k\rangle \cos\big(\theta\langle k\rangle + \delta_\theta\big) \\ 0 & 1 \end{pmatrix} \tag{6.5}$$

① 在对象的显示值中，nhist = 后的数值表示了历史记录的数量。hist 的属性是一个结构数组，它保存了每个时间步对应的机器人状态。

② 一个截断的泰勒级数。

A New Approach to Linear Filtering and Prediction Problems[1]

R. E. KALMAN

Introduction

背景材料：鲁道夫·卡尔曼(Rudolf Kálmán，1930 年—)出生于布达佩斯，是一名数学系统理论家。他获得了麻省理工学院的电气工程学士和硕士学位，并于 1957 年从哥伦比亚大学博士毕业。从 1958 到 1964 年，他在巴尔的摩的高等研究院(Research Institute for Advanced Study)担任一名研究数学家，并在这期间发展了自己的估计理论。这些理论早期曾遭到同行们的怀疑，于是他选择了一个机械工程(而非电子工程)的期刊来发表他的论文"线性滤波和预测问题的新方法(*A new approach to linear filtering and prediction problems*)"，因为他说"当你害怕带着既得利益踩上神圣大地时，最好的方法是另辟蹊径"。他获得了许多奖项，包括 IEEE 的荣誉勋章(Medal of Honor)、京都奖(Kyoto Prize)和查尔斯·斯塔克·德雷珀奖(Charles Stark Draper Prize)。

斯坦利·斯密特(Stanley F. Schmidt)是一名在 NASA 埃姆斯研究中心(Ames Research Center)工作的研究科学家，也是卡尔曼滤波方法的早期倡导者。他是第一个实现了卡尔曼滤波的人，并且还开发了非线性版的扩展卡尔曼滤波器。这个研究成果还被嵌入到阿波罗飞船的导航计算机中，用于飞船轨迹预测。(左侧是经 ASME 授权，从卡尔曼的著名论文(1960)中截取的一部分)

Vehicle 对象提供了方法 FX 和 FV 来计算雅克比矩阵，例如：

```
>> veh.Fx( [0,0,0], [0.5, 0.1] )
ans =
    1.0000         0   -0.0499
         0    1.0000    0.4975
         0         0    1.0000
```

其中传入的第一个参数是计算雅克比矩阵时的位形状态值，第二个是里程计变化量。

现在我们可以写出 EKF 预测方程：①

$$\hat{\boldsymbol{x}}\langle k+1|k\rangle = \boldsymbol{f}(\hat{\boldsymbol{x}}\langle k\rangle, \delta\langle k\rangle, \boldsymbol{0}) \tag{6.6}$$

$$\hat{\boldsymbol{P}}\langle k+1|k\rangle = \boldsymbol{F}_x\langle k\rangle \hat{\boldsymbol{P}}\langle k|k\rangle \boldsymbol{F}_x\langle k\rangle^{\mathrm{T}} + \boldsymbol{F}_v\langle k\rangle \hat{\boldsymbol{V}} \boldsymbol{F}_v\langle k\rangle^{\mathrm{T}} \tag{6.7}$$

该方程描述了状态和协方差随时间变化的过程。其中 $\hat{\boldsymbol{x}}\langle k+1|k\rangle$ 表示在时间 $k+1$ 时 $\boldsymbol{x}=(\hat{x}, \hat{y}, \hat{\theta})$ 的估计值，它是基于时间 k 及之前的信息得到的。$\hat{\boldsymbol{P}} \in \mathbb{R}^{3\times 3}$ 是一个协方差矩阵，代表机器人估计位形的不确定性。方程(6.7)中的第二项是正定的，表示位形不确定性 $\hat{\boldsymbol{P}}$ 不会减小。$\hat{\boldsymbol{V}}$ 是我们对里程计噪声协方差的估计值，实际上我们是不知道的。

为了使用工具箱对机器人和 EKF 方法进行仿真，我们将初始协方差定义得非常小，因为可以假设我们知道从哪里开始：

```
>> P0 = diag([0.005, 0.005, 0.001].^2);
```

然后将该初始值传递给一个 EKF 对象的构造函数：

```
>> ekf = EKF(veh, V, P0);
```

最后执行该过滤器 1000 个时间步：

```
>> ekf.run(1000);
```

像之前一样，它将驱动机器人在一条随机路径上运动。在每一个时间步，过滤器将采用 Vehicle 对象提供的多种方法更新状态估计。

① 附录 H 中的卡尔曼滤波分两个步骤：基于模型的预测和基于传感器数据的更新。在这个航迹推算的例子中，我们只用了预测方程。

背景材料：误差椭圆。如果机器人的位置(不考虑姿态)是如图 6.3 所示的 PDF，那么它的水平横截面将是一个椭圆。二维高斯概率密度函数是

$$p(x,y)=\frac{1}{(2\pi)\det(\boldsymbol{P})^{1/2}}\exp\left\{-\frac{1}{2}(\boldsymbol{x}-\mu_x)^{\mathrm{T}}\boldsymbol{P}^{-1}(\boldsymbol{x}-\mu_x)\right\}$$

其中，$\mu_x\in\mathbb{R}^2$ 是 $\boldsymbol{x}$ 的均值，$\boldsymbol{P}\in\mathbb{R}^{2\times2}$ 是协方差矩阵。1σ 边界被定义为所有满足下式的点 $\boldsymbol{x}$：

$$(\boldsymbol{x}-\mu_x)^{\mathrm{T}}\boldsymbol{P}^{-1}(\boldsymbol{x}-\mu_x)=1$$

画出一个如图 6.4 所示的椭圆是非常有用的，它表示了位置的不确定性。一个较大的椭圆对应一个更宽的 PDF 峰值区和更小的位置确定性。

一个简便的衡量总不确定度的方法是用椭圆的面积 $\pi r_1 r_2$，其中 $r_i=\sqrt{\lambda_i}$，λ_i 是 $\boldsymbol{P}$ 矩阵的特征值。因为 $\det(\boldsymbol{P})=\Pi\lambda_i$，故椭圆面积(总不确定度)正比于 $\sqrt{\det(\boldsymbol{P})}$。详细论述情参见附录 E 和 F。

我们可以绘制出机器人走过的真实路径：[①]

```
>> veh.plot_xy()
```

以及过滤器估计的路径：[②]

```
>> hold on
>> ekf.plot_xy('r')
```

这些路径都显示在图 6.4 中。我们可以看到真实路径与估计路径之间存在一些差异。

在第 700 个时间步，协方差矩阵是

```
>> P700 = ekf.history(700).P
P700 =
    0.4674    0.0120   -0.0295
    0.0120    0.7394    0.0501
   -0.0295    0.0501    0.0267
```

对角元素是与状态相关的估计方差，即分别是 σ_x^2，σ_y^2 和 σ_θ^2。x 轴方向的 PDF 标准差是

```
>> sqrt(P700(1,1))
ans =
   0.6837
```

在这种情况下，机器人的 x 坐标有 95% 的可能性是位于 $\pm2\sigma$ 或 ±1.37 m 之间。用类似方法也可以得到关于 y 和 θ 的不确定性。

非对角元素是相关性系数，表示对应两个变量的不确定性是相关的。例如，$P_{2,3}=P_{3,2}=0.0501$ 就表示 x 和 θ 这两个变量的不确定性是相关的，说明航向角 θ 的变化会影响 x 方向的位置。反过来，有关 θ 的新信息可以用来修正 θ 和 x。有关机器人位置的不确定性是通过矩阵 $\hat{\boldsymbol{P}}$ 左上方的 2×2 协方差子距阵来表示的，它可以被理解为一个定义了位置置信界限的椭圆。我们可以绘制出对应机器人不同位置的这些椭圆：

```
>> ekf.plot_ellipse([], 'g')
```

如图 6.4 所示。这些椭圆对应的是 1σ 的置信界限。机器人从坐标原点出发，随着它向前移动，我们发现椭圆逐渐变大，因为估计的不确定性也增加了。椭圆仅仅表示了 x 和 y 两个变量，实际上 θ 的不确定性也在增加。

① 保存在 Vehicle 对象中。

② 保存在 EKF 对象中。

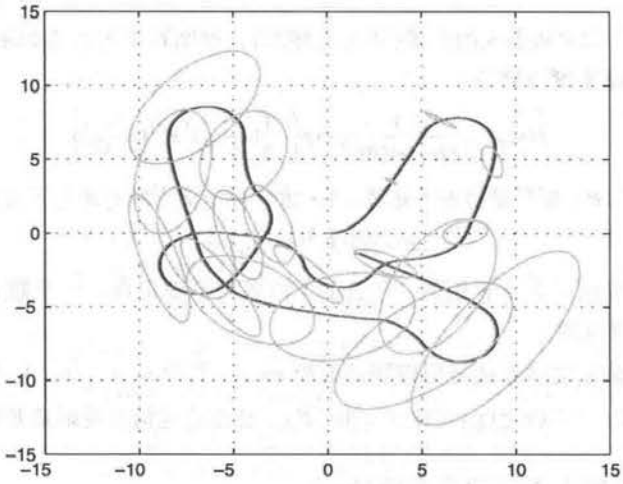


图 6.4 采用 EKF 的航迹推算。黑色曲线表示机器人的真实路径，灰色曲线表示估计路径。机器人从原点出发，不确定性椭圆用浅灰色表示

包括位置和航向的总不确定性可以通过$\sqrt{\det(\hat{\boldsymbol{P}})}$给出①，并且可以绘制成随时间变化的曲线：

```
>> ekf.plot_P();
```

如图 6.5 所示，我们注意到它是只增不减的。

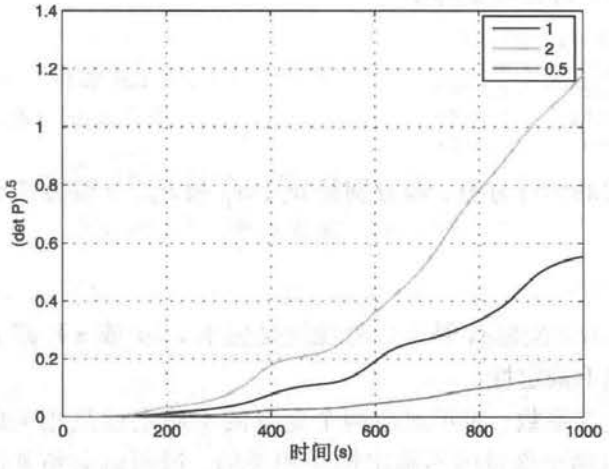


图 6.5 总的不确定性可以通过$\sqrt{\det(\hat{\boldsymbol{P}})}$得到，它是单调递增的(黑色)。改变 $\boldsymbol{V}$ 值的大小就可以改变不确定性增加的速率。图中3条曲线代表的是$\boldsymbol{V}=\alpha\boldsymbol{V}^*$，其中$\alpha=1/2,1,2$

注意，我们已经两次使用了里程计协方差矩阵 V。第一次是在 Vehicle 构造器中，用作高斯噪声的协方差 $\boldsymbol{V}$，它被加到真实里程计读数后来仿真里程计误差，如方程(6.3)所示。在实际应用中，这个噪声处理是被隐藏在机器人中的，我们并不知道其中的参数。第二次是在 EKF 构造器中，作为我们对里程计协方差矩阵的最佳估计 $\hat{\boldsymbol{V}}$，用在滤波器状态协方差更新方程

① $\boldsymbol{P}$ 矩阵的元素有不同的单位：m^2 和 rad。因此，总不确定性应是空间位置与角度不确定性的混合，且带有隐含权重。通常 $x, y \gg \pi$，所以位置不确定性占主导地位。

中，如方程(6.7)所示。$\boldsymbol{V}$ 和 $\hat{\boldsymbol{V}}$ 的相对关系决定了不确定性增长的快慢，如图 6.5 所示。当 $\hat{\boldsymbol{V}} > \boldsymbol{V}$ 时，$\boldsymbol{P}$ 就会大于其实际值，说明滤波器对它的估计比较悲观；当 $\hat{\boldsymbol{V}} < \boldsymbol{V}$ 时，$\boldsymbol{P}$ 就会小于其实际值，说明滤波器对它的估计过于自信，即实际不确定性要大于估计的不确定性。在实际中，需要通过一些实验来确定这个协方差矩阵的近似值。

6.2 使用地图

在上一节中我们已经看到，如果仅使用航迹推算法，那么机器人的位置不确定性就会无限制地增长。解决这个问题可以借鉴4000年前腓尼基人的办法，即通过观察周围环境中的已知特征物来引入新信息。在下面的例子中，我们将使用一张包含 N 个固定路标的地图，这些路标是随意放置的，但位置已知。

工具箱支持一个 Map 对象：

```
>> map = Map(20, 10)
```

这个地图中包括了20个特征物，它们在 x 和 y 方向各 $N=20$ 范围内随机散布。用下面的方法可以绘制出该地图：

```
>> map.plot()
```

机器人上配备了一个传感器用于观测特征物。这些特征物相对于机器人的观测信息可以表示为

$$\boldsymbol{z} = \boldsymbol{h}(\boldsymbol{x}_v, \boldsymbol{x}_f, \boldsymbol{w}) \tag{6.8}$$

其中，$\boldsymbol{x}_v$ 是机器人的状态，$\boldsymbol{x}_f$ 是被观察特征物在世界坐标系中的已知位置，$\boldsymbol{w}$ 是表征传感器误差的一个随机变量。

为了将以上描述具体化，我们采用一些常见的传感器来测量环境中地标的位置和方位角，例如一个雷达或者一个扫描激光测距仪，如图 6.6 所示。将传感器安置在机器人上，它对第 i 个特征物的观测值是 $\boldsymbol{x}_{fi} = (x_i, y_i)$

$$\boldsymbol{z} = \begin{pmatrix} \sqrt{(y_i - y_v)^2 + (x_i - x_v)^2} \\ \arctan(y_i - y_v)/(x_i - x_v) - \theta_v \end{pmatrix} + \begin{pmatrix} w_r \\ w_\beta \end{pmatrix} \tag{6.9}$$

其中，$z = (r, \beta)^{\mathrm{T}}$，$r$ 是距离，β 是方位角，测量噪声是

$$\begin{pmatrix} w_r \\ w_\beta \end{pmatrix} \sim N(0, \boldsymbol{W}),\ \boldsymbol{W} = \begin{pmatrix} \sigma_r^2 & 0 \\ 0 & \sigma_\beta^2 \end{pmatrix}$$

协方差矩阵仅有对角线上的元素，表明距离和方位角的误差是互相独立的①。

图 6.6 一个扫描激光测距仪。这个传感器中有一个旋转的装置，它能发射红外线激光脉冲，然后测量激光反射回来所需的时间。传感器的最大测量距离是30 m，扫描角度范围是270°，角度分辨率是0.25°。它每秒可以扫描40次（照片由Hokuyo Automatic公司提供）

① 它还表示协方差与距离无关。但实际上协方差是随距离的增加而增大的，因为无论是激光还是雷达，其返回信号的强度都会随目标距离 d 的增加而迅速下降($1/d^4$)。

在这个例子中，我们设定传感器的不确定度为 $\sigma_r = 0.1$ m，$\sigma_\beta = 1°$，从而给出传感器的协方差矩阵如下：

```
>> W = diag([0.1, 1*pi/180].^2);
```

在工具箱里，我们使用一个 RangeBearingSensor 对象来为这类传感器建模：①

```
>> sensor = RangeBearingSensor(veh, map, W)
```

它把传感器与机器人和地图关联在一起，传感器的协方差矩阵 W 也被确定。对于地图上某个随机选取的特征物，可以使用 reading 方法获得它的距离和方位角，以及该特征物的身份号：②

```
>> [z,i] = sensor.reading()
z =
   31.9681
    1.6189
i =
    18
```

身份号是一个整数 $i \in [1,20]$，因为地图中设置了 20 个特征物。我们只需要知道被测特征物的身份号，这样就可以避免数据关联问题。地图上第 18 号特征物的位置可以用以下方式查出：

```
>> map.feature(18)
   -8.0574
    6.4692
```

机器人如果从地图上知道自己的估计位置以及特征物的已知位置，它也能用方程(6.9)估算出它到特征物的距离和方位角。观测值与估计值之间的任何差异说明机器人位置估计存在误差，它并不在它设想的位置上。这个差异值为

$$\boldsymbol{\nu}\langle k+1\rangle = \boldsymbol{z}\langle k+1\rangle - \boldsymbol{h}\left(\hat{\boldsymbol{x}}(k+1|k), \boldsymbol{x}_f, 0\right)$$

它是实现卡尔曼滤波器的关键③。该差值也被称为“新息”，因为它代表了新的信息。卡尔曼滤波器利用这个新息来修正状态估计，并更新不确定度估计 $\boldsymbol{P}<k>$。

我们可以像前面(6.1.2 节，第 109 页)那样，将观测方程(6.8)线性化，得到

$$\boldsymbol{z}\langle k\rangle = \hat{\boldsymbol{h}} + \boldsymbol{H}_x\left(\boldsymbol{x}\langle k\rangle - \hat{\boldsymbol{x}}\langle k\rangle\right) + \boldsymbol{H}_w\boldsymbol{w}\langle k\rangle \tag{6.10}$$

其中雅克比矩阵通过对方程(6.9)求微分得到：

$$\boldsymbol{H}_{x_i} = \left.\frac{\partial \boldsymbol{h}}{\partial \boldsymbol{x}_v}\right|_{w=0} = \begin{pmatrix} -\dfrac{x_i - x_v\langle k\rangle}{r} & -\dfrac{y_i - y_v\langle k\rangle}{r} & 0 \\ \dfrac{y_i - y_v\langle k\rangle}{r^2} & -\dfrac{x_i - x_v\langle k\rangle}{r^2} & -1 \end{pmatrix} \tag{6.11}$$

$$\boldsymbol{H}_w = \left.\frac{\partial \boldsymbol{h}}{\partial \boldsymbol{w}}\right|_{w=0} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \tag{6.12}$$

上面的 RangeBearingSensor 对象中，还包括了方法 h 来实现方程(6.9)，以及方法 H_x 和 H_y 来分别计算雅克比矩阵。

现在我们用新息来更新之前用方程(6.6)和方程(6.7)计算得到的预计状态：

$$\hat{\boldsymbol{x}}\langle k+1|k+1\rangle = \hat{\boldsymbol{x}}\langle k+1|k\rangle + \boldsymbol{K}\langle k+1\rangle\boldsymbol{\nu}\langle k+1\rangle \tag{6.13}$$

$$\hat{\boldsymbol{P}}\langle k+1|k+1\rangle = \hat{\boldsymbol{P}}\langle k+1|k\rangle - \boldsymbol{K}\langle k+1\rangle\boldsymbol{H}_x\langle k+1\rangle\hat{\boldsymbol{P}}\langle k+1|k\rangle \tag{6.14}$$

① Sensor 里的一个子类。

② 如果将 interval 的属性设置为 N，则该方法每隔 N 次调用才返回一个读数。用 i 表示一个非测绘量，其值为 NaN。

③ 见附录 H。

上式就是卡尔曼滤波器更新方程。它们先从方程(6.6)和方程(6.7)得到下一时间步的预测值，表示为 $k+1|k$，然后使用时间步 $k+1$ 上的新息来计算新的预测值，表示为 $k+1|k+1$。新息是在乘以卡尔曼增益矩阵 $\boldsymbol{K}$ 后才被加到估计状态中的。卡尔曼增益矩阵 $\boldsymbol{K}$ 被定义为

$$\boldsymbol{S}\langle k+1\rangle = \boldsymbol{H}_x\langle k+1\rangle \hat{\boldsymbol{P}}\langle k+1|k\rangle \boldsymbol{H}_x\langle k+1\rangle^{\mathrm{T}} + \boldsymbol{H}_w\langle k+1\rangle \hat{\boldsymbol{W}}\langle k+1\rangle \boldsymbol{H}_w\langle k+1\rangle^{\mathrm{T}} \tag{6.15}$$

$$\boldsymbol{K}\langle k+1\rangle = \hat{\boldsymbol{P}}\langle k+1|k\rangle \boldsymbol{H}_x\langle k+1\rangle^{\mathrm{T}} \boldsymbol{S}\langle k+1\rangle^{-1} \tag{6.16}$$

其中，$\hat{\boldsymbol{W}}$ 表示传感器噪声的估计协方差矩阵。我们注意到，方程(6.14)中的第二项是从协方差里被减去的，这意味着协方差有可能减小，而这点在方程(6.7)描述的航迹推算法中是不可能的。

我们现在已具备了所有基础，可以利用里程计和对地图特征的观测来创建一个估计器。工具箱中的实现方式如下：

```
>> map = Map(20);
>> veh = Vehicle(V);
>> veh.add_driver( RandomPath(map.dim) );
>> sensor = RangeBearingSensor(veh, map, W);
>> ekf = EKF(veh, V, P0, sensor, W, map);
```

Map 构造器的默认地图大小是 ±10 m，该尺寸可以通过其 dim 属性修改。

下面运行这个仿真 1000 个时间步：

```
>> ekf.run(1000);
```

运行后将显示一个机器人移动的动画和相对地标的观测值。我们可以将保存的结果绘图显示：

```
>> map.plot()
>> veh.plot_xy();
>> ekf.plot_xy('r');
>> ekf.plot_ellipse([], 'k')
```

如图 6.7(见彩图)所示，我们现在很难看到误差椭圆，因为现在它们太小了。

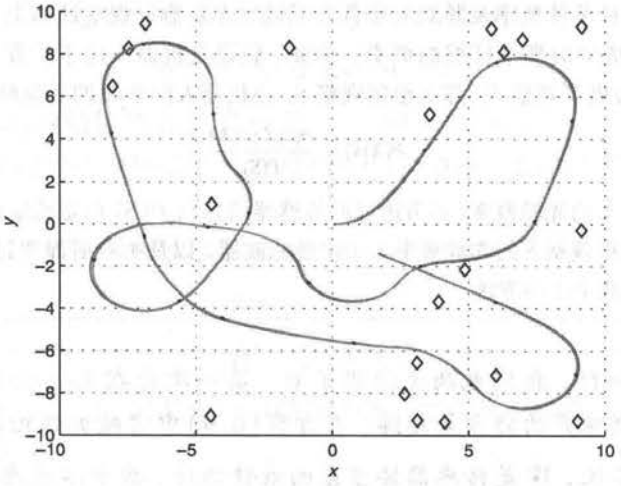


图 6.7　EKF 定位方法，显示了机器人的实际路径(蓝色)，以及用里程计和地标估计出的路径(红色)。机器人从原点出发

图 6.8(a)给出了机器人实际路径与估计路径的一个局部放大图。我们可以看到在估计路径上一个很小的误差椭圆，还可以看到该路径上有一小段突变折线。根据方程(4.2)的自行车模型，机器人的状态将随着时间平滑变化，但在这里由于从传感器获取的新信息可以对状态进

行更新，那么有时就会引起突变，使得状态估计沿各个方向跳跃。图 6.8(b)则表明不确定度不再是单调递增的了，新信息通过方程(6.14)使不确定度减小。

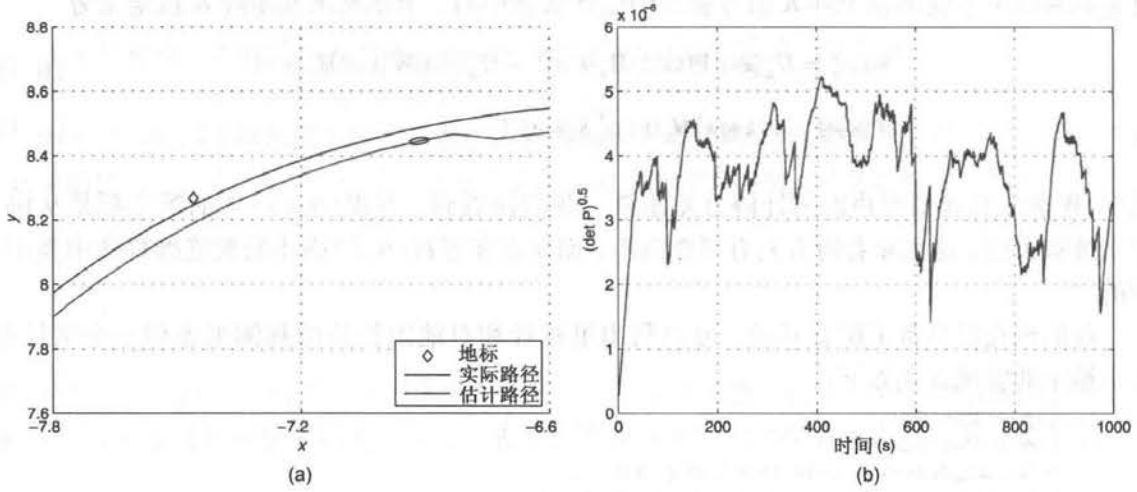


图 6.8 (a)机器人实际与估计路径的一段放大图；(b)协方差的行列式随时间变化曲线。总不确定度由det(**P**)给出，图示它不随时间增加

背景材料： 托马斯·贝叶斯(Thomas Bayes，1702—1761)是一位不循规蹈矩的长老会牧师。他在爱丁堡大学研究过逻辑和神学，并在肯特郡的 Tunbridge-Wells 工作和生活过。在那里，通过与第二世斯坦霍普伯爵(Earl Stanhope)的接触，他开始对数学感兴趣，并于 1742 年选入英国皇家学会。他去世后，他的朋友理查德·普莱斯(Richard Price)于 1763 年整理并出版了他的遗作《机遇理论中一个问题的解》(*An Essay towards Solving a Problem in the Doctrine of Chances*)，其中包含了对贝叶斯定理一种特殊情况的论述。贝叶斯埋葬在伦敦的 Bunhill Fields 公墓。

贝叶斯定理反映了一种条件概率及其对立事件之间的关系：给定观察证据下一个假设的概率，与假设成立时那个证据的概率。例如，假设是机器人位于位置 X，它用传感器对已知地标做了观察 S。那么给定观察 S，该机器人位于 X 的后验概率是

$$P(X|S)=\frac{P(S|X)P(X)}{P(S)}$$

其中，$P(x)$是机器人位于 X 的先验概率(不考虑任何传感器信息)；$P(S|X)$是给定机器人位于 X，传感器得到观察 S 的概率；$P(S)$是获得观察 S 的先验概率。卡尔曼滤波器，以及本章后面要讨论的蒙特卡洛估计器，本质上都是解决上述问题的两个不同方法。

和前面讨论的 V 一样，我们也两次使用了 W。第一次是在 RangeBearingSensor 对象的构造器中，$\boldsymbol{W}$ 是高斯噪声的协方差矩阵，在方程(6.9)中它被加到距离和方位角计算值后来仿真传感器误差。第二次，$\hat{\boldsymbol{W}}$ 是传感器协方差的最佳估计，被卡尔曼滤波器方程(6.15)使用。

这个 EKF 框架还允许采用多个以及多种传感器的数据来更新状态，这就是为什么该估计问题也被称作传感器融合。例如，罗盘得到的航向角、陀螺仪得到的偏航角速度、摄像机得到的目标方位角，以及 GPS 得到的位置，它们都可以用来更新状态。对于每个传感器，我们仅仅需要提供观测方程 $\boldsymbol{h}(\cdot)$、雅克比矩阵 $\boldsymbol{H}_x$ 和 $\boldsymbol{H}_w$ 以及对传感器输出协方差 $\boldsymbol{W}$ 的估计。方程 $\boldsymbol{h}(\cdot)$可以是非线性的，甚至是不可求逆的。其余都交给 EKF 就可以了。

在这个例子中，以及在接下来的两节里，我们都假设传感器可以提供目标相对于机器人的位置和目标身份等信息。然而在实际中，大多数地标都是匿名的[①]，也就是说，我们不知道它们的身份，因而也无法知道它们在环境中的真实位置。因此我们需要解决目标对应或关联问题，即给定地图和观测，我们需要确定哪个特征物是最有可能已被发现了。这一步如果出现差错，如系统看到目标 i 却认为它是目标 j，那么将很快导致整个估计的失败。状态向量被错误地更新，这又使得它更容易错误地关联目标，如此恶性循环。在这个过程中，协方差有可能不会大幅增加，这反而是更危险的事情——估计错误却还信心满满。对于室内机器人，传感器有可能检测到虚假目标，比如环境中移动的人，同时人也可能遮挡地标。

上述问题的一种解决方法是采用多假设估计器，例如将在6.5节中介绍的粒子过滤器，它能对观测到地标A或地标B的可能性建模，并用以后的观测强化某个假设而弱化另外的假设。而前面介绍的扩展卡尔曼滤波器使用了一个高斯分布的模型，只有一个峰值，因此将位置限定了只有一个假设。

6.3 创建地图

到目前为止，我们都认为地图是现成的，唾手可得。考虑到现今互联网上地图这么多样而且可以免费下载，这种想法可以理解。然而，地图终归是需要某个人或某个东西去制作的。下面我们要考虑的例子，就是让机器人在有地标的环境中移动，并绘制出环境地图。

和以前一样，我们在机器人上还是安装了一个距离和方位角传感器，它们能不太精确地测量特征物相对于机器人的位置。另外，我们还假设在环境中一共有 N 个特征物，而且机器人能够确定每个被观察特征物的身份。然而在这里讨论的例子中，我们假定该机器人能非常准确地知道它自己的位置，尽管这个假设不现实，但它是我们下面要讨论内容的一个重要基础[②]。

由于机器人的位姿准确可知，我们不需要去估计它，但我们确实需要估计地标的坐标。对于这个问题，设状态向量中包含了已观察到的 M 个地标的估计坐标：

$$\hat{\boldsymbol{x}} = (x_1, y_1, x_2, y_2, \cdots x_M, y_M)^{\mathrm{T}}$$

它共有 $2M$ 个元素，相应的估计协方差 $\hat{\boldsymbol{P}}$ 将是一个 $2M \times 2M$ 矩阵。不过状态向量的长度是可变的，因为我们不能提前知道在环境中可以观察到多少地标。最初可设 $M=0$，然后每次观察到一个新地标就加1。

由于地标位置固定，所以这种情况下预测方程就很简单：

$$\hat{\boldsymbol{x}}\langle k+1|k\rangle = \hat{\boldsymbol{x}}\langle k|k\rangle \tag{6.17}$$

$$\hat{\boldsymbol{P}}\langle k+1|k\rangle = \hat{\boldsymbol{P}}\langle k|k\rangle \tag{6.18}$$

我们引入一个函数 $\boldsymbol{g}(\cdot)$，它是 $\boldsymbol{h}(\cdot)$ 的反函数，能够基于机器人已知位姿和传感器观测给出被观察地标的坐标：

$$\boldsymbol{g}(\boldsymbol{x}_v, \boldsymbol{z}) = \begin{pmatrix} x_v + r_z \cos(\theta_v + \theta_z) \\ y_v + r_z \sin(\theta_v + \theta_z) \end{pmatrix}$$

由于 $\hat{\boldsymbol{x}}$ 长度可变，每当我们遇到以前没有见过的地标，就需要扩展状态向量和协方差矩阵。状

① 在某些应用中，如室内移动机器人，条形码可以用来提供准确的目标身份识别。

② 一种比较现实的近似，是在一个没有建筑和小山遮挡卫星的环境中使用高端RTK GPS系统。

态向量的扩展使用函数 $\boldsymbol{y}(\cdot)$：

$$\boldsymbol{x}\langle k|k\rangle^{+} = \boldsymbol{y}\left(\boldsymbol{x}\langle k|k\rangle, \boldsymbol{z}\langle k\rangle, \boldsymbol{x}_v\langle k|k\rangle\right) \tag{6.19}$$

$$= \begin{pmatrix} \boldsymbol{x}\langle k|k\rangle \\ \boldsymbol{g}\left(\boldsymbol{x}_v\langle k|k\rangle, \boldsymbol{z}\langle k\rangle\right) \end{pmatrix} \tag{6.20}$$

它将新地标的坐标估计值附加到已有坐标的后面，因此状态向量中地标坐标的顺序取决于它们被观测到的顺序。

观测到新地标后，协方差矩阵同样也要相应扩展。这可以通过下式实现：

$$\hat{\boldsymbol{P}}\langle k|k\rangle^{+} = \boldsymbol{Y}_z \begin{pmatrix} \hat{\boldsymbol{P}}\langle k|k\rangle & 0 \\ 0 & \hat{\boldsymbol{W}} \end{pmatrix} \boldsymbol{Y}_z^{\mathrm{T}}$$

其中 $\boldsymbol{Y}_z$ 是一个雅可比矩阵：

$$\boldsymbol{Y}_z = \frac{\partial \boldsymbol{y}}{\partial \boldsymbol{z}} = \begin{pmatrix} \boldsymbol{I}_{n\times n} & & \boldsymbol{0}_{n\times 2} \\ \boldsymbol{G}_x & \boldsymbol{0}_{2\times n-3} & \boldsymbol{G}_z \end{pmatrix} \tag{6.21}$$

$$\boldsymbol{G}_x = \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{x}_v} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \tag{6.22}$$

$$\boldsymbol{G}_z = \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{z}} = \begin{pmatrix} \cos(\theta_v + \theta_z) & -r_z \sin(\theta_v + \theta_z) \\ \sin(\theta_v + \theta_z) & r_z \cos(\theta_v + \theta_z) \end{pmatrix} \tag{6.23}$$

上式中，n 是 $\hat{\boldsymbol{P}}$ 被扩展之前的维度。

函数 $\boldsymbol{h}(\cdot)$ 的另一个雅可比矩阵是

$$\boldsymbol{H}_{x_i} = \frac{\partial \boldsymbol{h}}{\partial \boldsymbol{x}_i} = \begin{pmatrix} \frac{x_i - x_v}{r} & \frac{y_i - y_v}{r} \\ -\frac{x_i - x_v}{r^2} & \frac{y_i - y_v}{r^2} \end{pmatrix} \tag{6.24}$$

它将地图特征中的变化与观察中的变化相关联，并且由 Sensor 类中的 H_xf 方法来完成。

在方程(6.14)中用到的雅可比矩阵 $\boldsymbol{H}_x$ 描述了特征观测信息随状态向量变化的情况。然而，在本例中观测信息仅依赖于单一被观察特征，所以该雅可比矩阵的大部分元素为零：

$$\boldsymbol{H}_x = (0 \cdots \boldsymbol{H}_{x_i} \cdots 0) \tag{6.25}$$

其中，$\boldsymbol{H}_{xi}$ 位于与状态 $\boldsymbol{x}_i$ 对应的位置。

上述算法的工具箱实现方法如下：

```
>> map = Map(20);
>> veh = Vehicle([]);  % error free vehicle
>> veh.add_driver( RandomPath(map.dim) );
>> W = diag([0.1, 1*pi/180].^2);
>> sensor = RangeBearingSensor(veh, map, W);
>> ekf = EKF(veh, [], [], sensor, W, []);
```

其中，传递给函数 EKF 的 3 个空矩阵分别表示：没有机器人估计里程协方差(估计没有误差)，没有初始机器人状态协方差，以及地图未知。我们运行该仿真 1000 个时间步：

```
>> ekf.run(1000);
```

仿真结果将显示机器人移动的动画，以及随时间变化的地图特征物的协方差椭圆。下面绘制出估计的地标位置：

```
>> map.plot();
>> ekf.plot_map(5, 'g');
>> veh.plot_xy('b');
```

它们在图 6.9(a)中用 5σ 的置信椭圆表示(为了能看见)，图中还显示了真实的地标位置和机

器人运动的路径。协方差矩阵具有块对角结构，可以用图形方式显示出来：

```
>> spy( ekf.P_est );
```

如图 6.9(b)所示。灰点代表非零元素，并且每个 2×2 的块代表一个地标位置的协方差[①]。体现关联关系的非对角元素都是零，这意味着地标估计之间是无关联的或独立的。这一结论显而易见，因为观测特征 i 并没有为观测特征 $j(j\neq i)$ 提供新的信息。

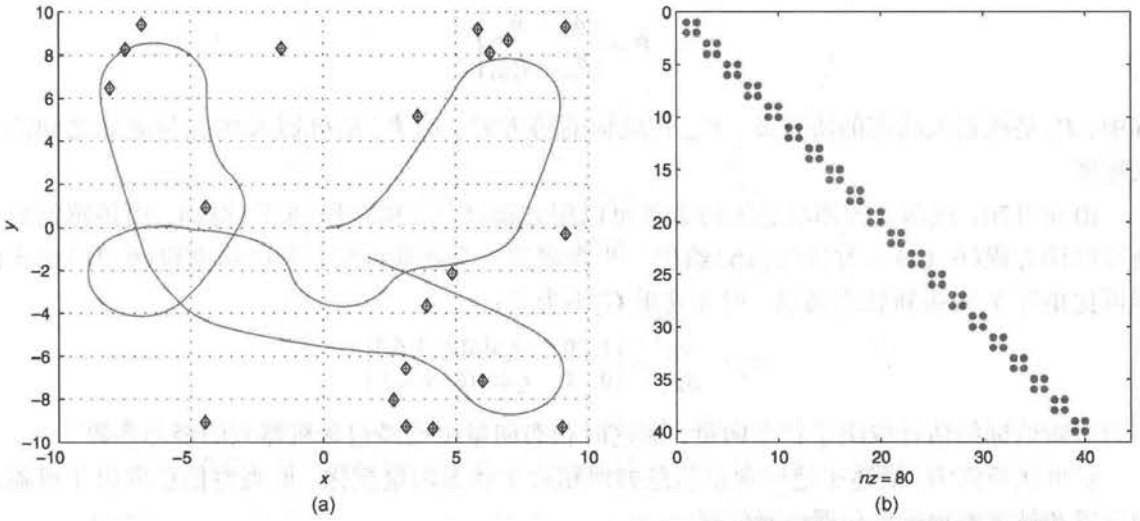


图 6.9　EKF 地图构建结果。(a)估计地标用符号 + 表示，并附带 5σ 的置信椭圆，真实地标位置用黑色符号◇表示，机器人的真实路径用灰线表示；(b)最终的协方差矩阵中的非零元素

EKF 对象的内部保存了一张表，该表将函数 RangeBearingSensor 返回的特征身份号与状态向量中特征坐标的位置关联。例如，对于身份号为 10 的地标：

```
>> ekf.features(:,10)
ans =
    19
    51
```

说明该地标的坐标位于 $\hat{\boldsymbol{x}}$ 的第 19 个和第 20 个元素，而且一共在仿真过程中出现过 51 次。再调用下面的函数：

```
>> ekf.x_est(19:20)'
ans =
    5.8441    9.1898
```

得到该地标的估计位置。它的估计协方差为

```
>> ekf.P_est(19:20,19:20)
ans =
   1.0e-03 *
    0.2363   -0.0854
   -0.0854    0.2807
```

6.4　定位并制图

最后要解决的问题是确定我们的位置，同时创建一张地图。这其实是海洋导航与制图中的一个古老问题——在使用现有地图导航的同时，又不断给地图增添新的东西。在机器人学

① 这就是所谓的稀疏矩阵。对于这种大多数元素为零的矩阵，有很多数值算法能对它们进行高效计算。MATLAB 的函数 spy 可以图形化显示稀疏矩阵的结构。请查看“help sparse”以获取更多细节。

中，这个问题被称为同步定位与测绘(SLAM)或并行测绘与定位(CML)。有时这个问题也被比作“鸡与蛋”的问题，但是根据我们在前几节所学的知识，它其实是不难解决的。

状态向量中包括机器人的位形，以及已被观察到的 M 个地标的坐标：

$$\hat{\boldsymbol{x}} = (x_v, y_v, \theta_v, x_1, y_1, x_2, y_2, \cdots x_M, y_M)$$

它有 $2M+3$ 个元素。它的协方差是一个 $(2M+3)\times(2M+3)$ 矩阵，并具有如下结构：

$$\hat{\boldsymbol{P}} = \begin{pmatrix} \hat{\boldsymbol{P}}_{vv} & \hat{\boldsymbol{P}}_{vm} \\ \hat{\boldsymbol{P}}_{vm}^{\mathrm{T}} & \hat{\boldsymbol{P}}_{mm} \end{pmatrix}$$

其中，$\hat{\boldsymbol{P}}_{vv}$ 是机器人状态的协方差，$\hat{\boldsymbol{P}}_{mm}$ 是地标的协方差，而 $\hat{\boldsymbol{P}}_{vm}$ 是机器人状态与地标之间的关联矩阵。

由前可知，机器人预测状态的协方差可以用方程(6.6)和方程(6.7)得到，而传感器的更新可以用方程(6.13)～方程(6.16)给出。当观测到一个新地标时，可以用方程(6.21)给出的雅可比矩阵 $\boldsymbol{Y}_z$ 来更新状态向量，但在这里 $\boldsymbol{G}_x$ 不为零：

$$\boldsymbol{G}_x = \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{x}_v} = \begin{pmatrix} 1 & 0 & -r_z \sin(\theta_v + \theta_z) \\ 0 & 1 & r_z \cos(\theta_v + \theta_z) \end{pmatrix}$$

因为对新地标的估计取决于状态向量，而这时状态向量中已经包含机器人的姿态参数了。

雅可比矩阵 $\boldsymbol{H}_x$ 描述了地标观察信息如何相对于状态向量变化，而观察信息取决于机器人的位置和被观察地标的位置，因此有

$$\boldsymbol{H}_x = (\boldsymbol{H}_{x_v} \cdots 0 \cdots \boldsymbol{H}_{x_i} \cdots 0) \tag{6.26}$$

其中，$\boldsymbol{H}_{x_i}$ 的位置对应于 $\boldsymbol{x}_i$ 在状态向量中的位置。该式与方程(6.25)有些类似，但在它左边有一个非零块 $\boldsymbol{H}_{xv}$，代表了机器人位置的影响。

卡尔曼增益矩阵 $\boldsymbol{K}$ 被乘以了从地标观测得到的新息(一个二维向量)，这样每次状态向量的元素(包括机器人的位姿和每个地标的位置)变化时，$\boldsymbol{K}$ 都会更新。

上述算法的工具箱实现方式与前面类似：

```
>> P0 = diag([.01, .01, 0.005].^2);
>> map = Map(20);
>> veh = Vehicle(WV);
>> veh.add_driver( RandomPath(map.dim) );
>> sensor = RangeBearingSensor(veh, map, W);
>> ekf = EKF(veh, V, P0, sensor, W, []);
```

其中，传递给 EKF 空矩阵表明地图是未知的，P0 为初始的机器人状态 3×3 协方差矩阵。

我们仍然运行该仿真 1000 个时间步：

```
>> ekf.run(1000);
```

和前面一样，它也会显示一个机器人移动的动画。我们还能看到与地标关联的协方差椭圆随着时间演变的情况。我们可以绘制出结果：

```
>> map.plot();
>> ekf.plot_map(5, 'g');
>> ekf.plot_xy('r');
>> veh.plot_xy('b');
```

如图 6.10 所示(见彩图)。

图 6.11(a)显示出不确定性随时间呈下降趋势。图 6.11(b)显示了一个图像化的最终协方差矩阵，我们看到一个复杂的结构。与前面单独创建地图不同，此时 $\hat{\boldsymbol{P}}_{mm}$ 不是一个块对角矩阵，不为零的非对角元素表明地图上各个地标之间的关联性。地标的不确定性不会增加，预测

模型是：它们不会改变，但不会低于机器人的初始不确定度。一个地标的位置估计是机器人位置的函数，机器人的定位误差就会体现为地标的定位误差——反之亦然。

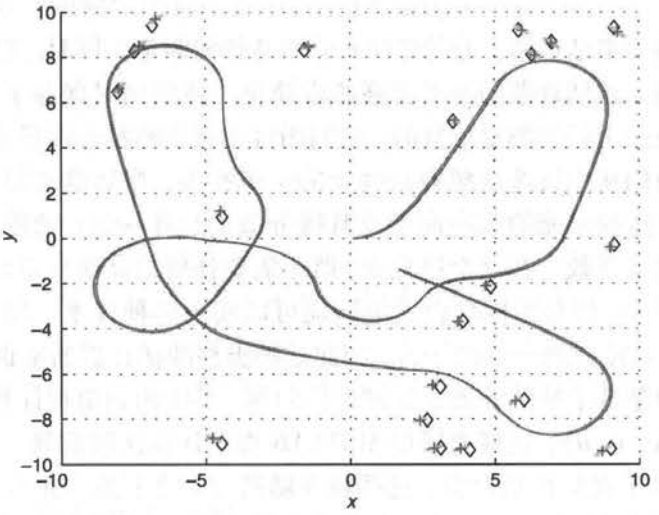


图 6.10　同时定位并制图。图中显示了机器人真实路径（蓝色）和估计路径（红色），它们叠加在真实地图上（黑色符号◇）。估计的地图特征用符号 + 表示，附带一个5σ置信椭圆

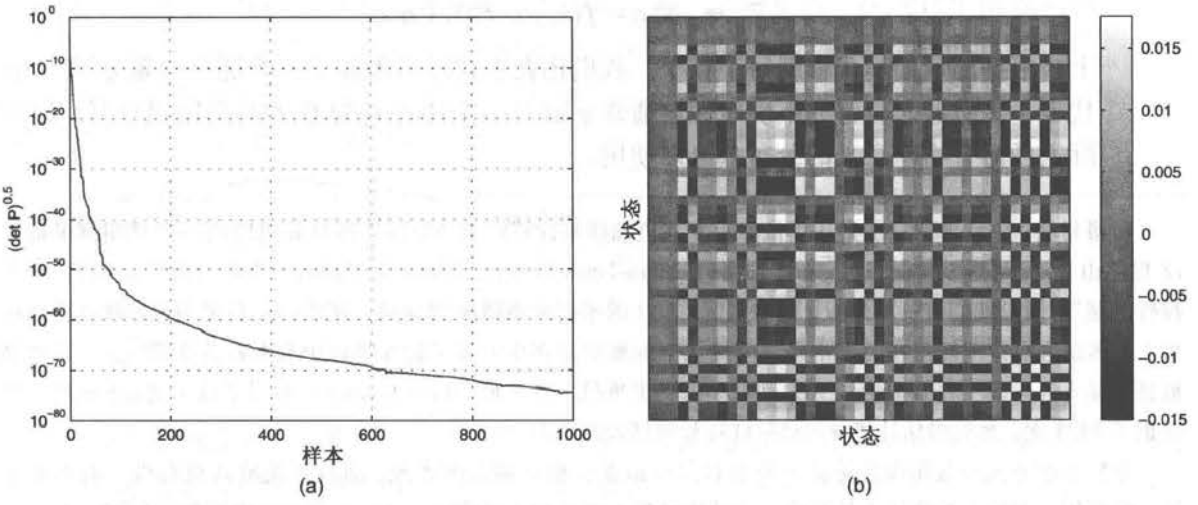


图 6.11　同时定位并制图。（a）协方差随时间变化曲线；（b）最终的协方差矩阵（元素值已按 0 ~ 255 分挡显示）

这种相关性会使得对任何地标观测的信息都会影响对地图上其他每个地标的估计，以及对机器人位姿的估计。这就好比所有状态都用一根弹簧连着，任何一个状态的运动都要影响到其他的状态。

6.5　蒙特卡罗定位

在前面所有关于估计的实例中，我们都假定那些用于测量机器人里程和地标距离及方位的传感器，其误差分布都是一个高斯概率密度函数。但实际中，我们可能会发现某个传感器的误差呈现一种单边分布（如泊松分布），或者呈现一种有多个峰值的多模分布。卡尔曼滤波器中使用的函数，如方程（6.3）和方程（6.8），都呈现出强烈的非线性，这意味着具有高斯分布

的传感器噪声，其函数结果不再呈现高斯误差分布——详见附录 H。与机器人位形相关联的概率密度函数可能具有多个峰值以对应多种假设，从而能更合理地解释如图 6.3 所示的传感器数据。

本节讨论的蒙特卡洛估计器，无需对误差分布做任何假定。同时，它还可以处理对于系统状态的多种假设问题。该估计器的基本思路非常简单。我们同时保存了许多不同版本的机器人状态向量。当出现一个新的测量信息时，我们用每个版本的状态向量去解释这些数据，并给解释的好坏打分。我们从中挑选出解释最吻合的一组状态，然后给它们一个随机扰动以产生一组新的状态。多个这些可能的状态向量及其得分值汇总在一起，就能近似得出一个我们想要估计状态的概率密度函数。在这个过程中，既不需要任何的高斯分布假设，也无需对系统线性化。虽然计算量很大，但是用标准台式电脑就可以实现这种技术。如果把这些状态向量绘成图上一个个的点，就能得到一个粒子云，因此这种类型的估计器通常也被称作粒子滤波器。

下面我们将应用蒙特卡罗估计器来处理定位问题，其中用到里程计和地图。在这里，我们只估计 3 个状态量(x, y, θ)，这样直接用 MATLAB 程序计算比较容易。首先，通过创建 N 个粒子 $\boldsymbol{x}_{v,i}$，$i \in [1, N]$来初始化估计器，这些粒子随机分布在机器人的位形空间中。所有粒子具有相同的初始权重或可能性 $w_i = 1/N$。在主迭代算法中，计算步骤如下：

1. 更新每个粒子状态：

$$\boldsymbol{x}_{v,i}\langle k+1\rangle = \boldsymbol{f}\left(\boldsymbol{x}_{v,i}\langle k\rangle, \delta\langle k\rangle\right) + \boldsymbol{q}\langle k\rangle$$

上式根据测量的里程移动每个粒子。我们还在上式最后附加了一个随机向量 $\boldsymbol{q}\langle k\rangle$，它代表了该机器人位置的不确定性。通常 $\boldsymbol{q}$ 取自一个协方差为 $\boldsymbol{Q}$ 的高斯随机变量，但实际上任何有物理意义的分布都可以使用。

背景材料：蒙特卡罗方法是一类依靠重复随机抽样来计算结果的算法。该计算思路的一个早期例子是在 18 世纪由乔治-路易·勒克莱尔，布丰伯爵(Georges-Louis Leclerc, Comte de Buffon, 1707—1788)提出的"布丰投针问题"：假设我们有一块地板，是由宽度均为 t 的平行木条拼接而成的。现在一根长度为 l 的针被扔到地板上，那么这根针正好落在两根木条之间缝线上的概率是多少？如果扔 n 根针中有 h 根落在缝线上，其概率被证明是 $h/n = 2l/t\pi$。1901 年，意大利数学家马里奥拉·扎里尼(Mario Lazzarini)进行了这个实验，他将一根针扔了 3408 次，并获得估计值 $\pi \approx 355/113(3.14159292)$。

蒙特卡洛方法经常用在要仿真一个具有多自由度且相互耦合的系统，而且系统输入具有很大的不确定性。而当用一种确定性算法去计算出一个精确结果不可行或不可能时，蒙特卡洛方法更能派上用场。由于这类算法主要依靠重复计算以及随机或伪随机数，使它们非常适合于计算机运算。该方法最早是在二战时期作为曼哈顿计划的一部分，在美国洛斯阿拉莫斯国家实验室里进行研究，参与的数学家包括约翰·冯·纽曼(John Von Neuman)、斯塔尼斯拉夫·乌拉姆(Stanislaw Ulam)和尼古拉·梅特罗波利斯(Nicholas Metropolis)。取名"蒙特卡洛"(Monte Carlo)容易让人想到碰运气游戏，也是该秘密项目的代号。

---

2. 对地图上坐标为 $\boldsymbol{x}_f$的地标 i 获取一个观察信息 $\boldsymbol{z}$，然后对每个粒子计算相应的新息：

$$\boldsymbol{v}_i = \boldsymbol{h}(\boldsymbol{x}_{v,i}, \boldsymbol{x}_f) - \boldsymbol{z}$$

它是预测与实际地标观察之间的误差。我们用一个似然函数来产生一个标量，用它来度量某个特定粒子解读这个观察的好坏。在本例中，我们选择了高斯似然函数：

$$w = \exp(-0.5\boldsymbol{v}^{\mathrm{T}}\boldsymbol{L}\boldsymbol{v}) + w_0$$

其中，w 被称为粒子的重要性或权重，$\boldsymbol{L}$ 是协方差矩阵，而 $w_0 > 0$ 可以确保在状态空间

中的任一点都有一个有限但很小的概率。我们这里使用高斯函数仅仅是为了方便，该函数并不需要是平滑的或可逆的，只要能充分描述一个观察的概率即可。

3. 选择能够最好解释观察信息的粒子，这个过程被称为重采样。一种常用策略是按照它们的权重随机选择粒子。若给定 N 个粒子 $\boldsymbol{x}_i$，对应的权重为 w_i，我们首先将权重标准化 $w'_i = w_i / \sum_{i=1}^{N} w_i$，并构建一个累计直方图 $c_j = \sum_{i=1}^{j} w'_i$。然后抽取一个均匀随机数 $r \in [0,1]$，并找出最小的 $|c_j - r|$，其中粒子 j 被选中用于下一轮迭代。上述整个过程重复 N 次。有大权重的粒子将对应于累积直方图垂直跨度的更大部分，因此它们更容易被选择。结果是得到数量相同的粒子，有一些被复制了多次，另一些却一次都没被复制[①]。

工具箱的实现方法和前面的例子基本相似。我们创建一个地图：

```
>> map = Map(20);
```

再创建一个具有噪声里程计和一个初始条件的机器人：

```
>> W = diag([0.1, 1*pi/180].^2);
>> veh = Vehicle(W);
>> veh.add_driver( RandomPath(10) );
```

然后再创建一个具有噪声读数的传感器：

```
>> V = diag([0.005, 0.5*pi/180].^2);
>> sensor = RangeBearingSensor(veh, map, V);
```

对于粒子过滤器，我们需要定义两个协方差矩阵。第一个是在每一次迭代中附加到粒子状态中的随机噪声协方差，它代表位形的不确定性。我们选择的协方差值与 W 中的值可比：

```
>> Q = diag([0.1, 0.1, 1*pi/180]).^2;
```

第二个是用于新息的似然函数协方差：

```
>> L = diag([0.1 0.1]);
```

最后我们构建一个 ParticleFilter 估计量：

```
>> pf = ParticleFilter(veh, sensor, Q, L, 1000);
```

它包含 1000 个粒子，这些粒子最初是均匀分布在三维位形空间中的。

下面运行仿真 1000 个时间步：

```
>> pf.run(1000);
```

我们可以观察到粒子变化的动画，图 6.12 所示为从中截取的两张图。可以看到，随着粒子状态被里程信息和随机扰动不断更新，这些粒子也不断移动。初始随机分布的粒子开始聚集在那些能最好解释传感器观察结果的位形空间区域。用达尔文进化论观点来说，这些粒子的权重变得越来越高，因而在重新采样过程中更容易存活，而那些权重小的粒子则将灭亡。由于绘制的画面是三维的，所以在动画演示中可以通过旋转画面来观察由粒子高度(z 轴)代表的航向角状态。

粒子分布类似于机器人位形的概率密度函数。机器人最可能的位形就是所有粒子的期望值或平均值，而估计的不确定性就用粒子云的分散度或它的标准差来衡量。在 ParticleFilter 对象中，记录了每一时间步上粒子状态的平均值和标准差。和以前一样，我们还是绘制出仿真结果：

```
>> map.plot();
>> veh.plot_xy('b');
```

图上再叠加粒子云的平均值：

---

① 下次迭代的第一步将会通过加 $\boldsymbol{q}\langle k \rangle$ 而把这些复制项传递出去。

```
>> pf.plot_xy('r');
```

如图 6.13(见彩图)所示。估计路径的初始阶段标准差较大，这是因为粒子还没有收敛到真实位形。我们可以绘制出标准差随时间变化的图形：

```
>> plot(pf.std(1:100,:))
```

结果如图 6.13(b)所示。我们注意到曲线在 10 ~ 20 的时间段内急剧下降，这是由于那些远离真实解的粒子被剔除掉了。前面我们曾提到粒子分布是 PDF 的一个采样近似，可以用图形显示为

```
>> pf.plot_pdf()
```

我们刚才解决的这个问题在机器人学中也称为“机器人绑架问题”，即机器人对其初始方位一无所知。为了表示这种大的不确定性，我们将粒子均匀分布在三维位形空间里。然而粒子的稀疏分布将导致粒子滤波器要用很长的时间才能收敛，除非我们采用非常大数量的粒子。上述问题是否有现实意义也一直存在争议。通常，我们还是可以知道机器人大概的初始位姿，因此就能把粒子初始化到位形空间对应的部分。例如，如果我们知道机器人在一条走廊里，我们就可以把粒子放在地图上对应走廊的地方，或者如果我们知道机器人是朝北的，那么就把所有粒子指向那个方向。

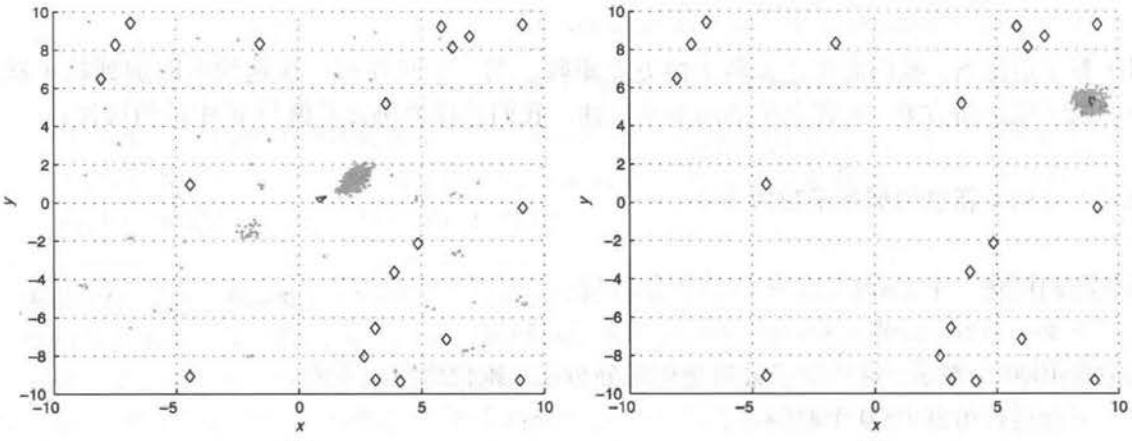


图 6.12 粒子过滤器结果，显示了粒子云随时间的演化过程。机器人用三角形表示

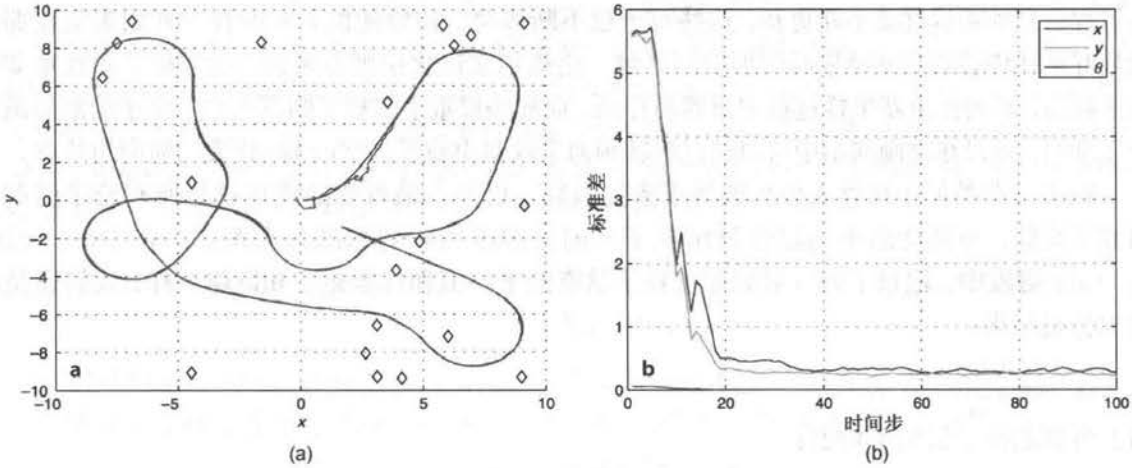


图 6.13 粒子滤波器的结果。(a)蓝色代表机器人的真实路径，红色代表估计路径；(b)粒子标准差随时间变化的曲线

为粒子过滤器设定参数需要一些经验，最好的方法就是通过实验来学习。对于机器人绑架问题，可以将 Q 和粒子数的值设定得大一些，这样粒子就可以遍寻位形空间，而一旦粒子过滤器收敛了，我们就可以将设定数值改小一些。粒子过滤器也有很多变种，主要是在似然函数和重采样策略上有所不同。

6.6　本章总结

在本章中，我们学习了机器人位置估计的两种方法：通过航迹推算，以及通过观察地图上位置已知的特征物。航迹推算是基于行驶里程和航向角变化两种信息的融合。但随着时间的推移，误差会积累，从而导致机器人位姿的不确定性增加。

我们通过在传感器输出中加入噪声来建立里程计误差的模型。噪声值取自某种分布函数，该函数描述了这种特殊传感器的误差。在我们的仿真中使用了具有特定协方差的零均值高斯噪声，但那仅仅是因为我们没有那个特殊传感器的其他信息，这种情况下就应该使用现有最实用的噪声模型。然后，我们介绍了卡尔曼滤波器，它提供了一个基于噪声测量的对机器人真实位形的最优估计。然而卡尔曼滤波器只适用于零均值高斯噪声及模型为线性的情况。描述机器人位形如何随时间变化的模型是非线性的，我们用了一个线性模型来近似它，这就需要计算一些雅可比矩阵，这种方法被称为广义卡尔曼滤波。

卡尔曼滤波器也估计了与位形估计相关的不确定性，而且我们看到不确定性的大小从来不会减少，而总是没有约束地增加。只有通过额外的信息资源才可以减少这种增长，于是我们看到了如何使用那些与机器人相关且位置已知的地标观测信息。我们再次用到了卡尔曼滤波器，但这次我们同时使用了滤波器的预测及更新步骤。我们看到，在这种情况下一次地标观测就可以使不确定性减少，而且在长时间内不确定性也不会增加。

然后我们应用卡尔曼滤波器解决了以下问题，即已知机器人精确位置的情况下估计地标位置。这种情况下，滤波器的状态向量就是地标本身的坐标。

最后我们将所有方法综合来估计机器人的位置、地标的位置以及它们的不确定性。这种情况下，状态向量中包含了机器人的位形和地标的坐标。

使用地标的一个很重要问题是数据关联。我们必须明确传感器知道或观察到了哪个地标，这样就可以在一个标示出所有已知或估计地标的地图中找到对应地标的位置。如果找错了对应地标，那么定位机器人就会出现错误。

最后，我们介绍了蒙特卡罗估计及微粒过滤器。这种技术需要大量的计算，但它不需要预先设定传感器的误差分布，或要求机器人模型为线性，并且支持多种假设。

扩展阅读

在博伦斯坦等人的专著中[Borenstein et al.,1996]，对机器人传感器做了较全面的概述，并对里程计做了特别论述。尽管这本书已经不再出版了，但在网上还是可以获得电子版。埃弗里特的书[Everett, 1995]则介绍了里程计、测距传感器和方位传感器，以及无线电、超声波和光学定位系统。遗憾的是，测距和方位传感器的讨论在现在看来非常过时，因为这些技术在过去 10 年中发展太快。有关上述传感器更新的介绍可查看《机器人技术手册》(*Robotics Handbook*)[Siciliano and Khatib, 2008]的第 20 章和第 22 章。

由于博伦斯坦等人和埃弗里特的书是在 GPS 投入使用前就出版了，所有没有关于 GPS 的介绍。格罗夫斯的书[Groves 2008]则详细介绍了 GPS 全球定位系统和其他基于无线电定位系统的原理。《机器人技术手册》的第 20 章中简要介绍了 GPS，而且该手册网站上还给出了很多关于 GPS 技术数据的链接网址。

关于卡尔曼滤波，至今已有许多出版物和在线资源。卡尔曼最初的论文发表于 1960 年[Kálmán, 1960]，距现在已超过了 50 年。渣坎和穆索夫的专著[Zarchanand Musoff 2005]非常清楚地介绍了卡尔曼滤波方法，值得一读。我个人一直觉得雅温斯基 1970 年出版的书是关于卡尔曼滤波的经典著作[Jazwinski, 1970]，非常有阅读价值，并且最近它已经再版印刷。巴沙罗姆等人在论著中全面描述了估算理论和 GPS 的使用[Bar-Shalom et al., 2001]。关于 EKF 和蒙特卡洛定位，斯鲁恩等人写的书是一本很好的参考资料[Thrun et al. 2005]。数据关联是一个重要的研究题目，该内容在巴沙罗姆和福特曼的书里有非常详细的描述[Bar-Shalom and Fortmann, 1988]。在《机器人技术手册》的第 4 章中也包含了卡尔曼滤波和数据关联。另外，韦尔奇和毕晓普还提供了一个网上资源，网址为 http://www.cs.unc.edu/~welch/kalman，上面有相关论文、课程和软件的指针，以及其他相关网站的链接。

EKF 的一个重要限制是它需要一阶线性化，尤其对于强非线性过程来说。替代方法包括雅温斯基提出的迭代 EKF[Jazwinski, 1970]，以及朱里亚和乌尔曼的无迹卡尔曼滤波(UKF)[Julier and Uhlmann, 2004]，其中 UKF 采用离散采样点来近似 PDF(概率密度函数)。上述议题在《机器人技术手册》第 25 章中也有介绍，作为多传感器融合的一部分。西格沃特等人的书[Siegwart, 2011]也对机器人定位有很好的论述。

SLAM 中的学问非常大，本章只涉及一些皮毛。本章讨论的是一个非常经典的基于 EKF 的方法，但这种方法对于有大规模特征物的情况可扩展性较差。快速 SLAM 方法[Montemerloet al., 2002]是一个针对上述大规模应用的最新算法。网上也有许多与 SLAM 相关的在线资源。很多 SLAM 夏季培训学校还有自己的网站，其中有许多优秀的在线资源，如讲义和上机实践。

很多开放源代码的 SLAM 实现算法(如 gmapping 和 iSam)可以从 OpenSLAM(http://www.openslam.org)中获得。MATLAB 的实现方法包括了针对平面 SLAM 的 CASRobot Navigation Toolbox(http://www.cas.kth.se/toolbox)和一个六自由度 SLAM 系统(http://homepages.laas.fr/jsola/JoanSola/eng/toolbox.html)。

关于经度问题以及约翰·哈里森的航海计时器，*Longitude* 这本书值得一读[Sobel, 1996]。

工具箱实现的补充说明

本章介绍了多个工具箱类来解决定位和地图绘制问题。主要原则是将问题分解成功能明确的子系统，然后用一组协同的类来实现它们，这样就可以只用很少的几行代码来解决相当复杂的问题。

对象之间的关系以及各个对象中的方法和属性如图 6.14 所示，更多的说明可以通过联机帮助系统或代码中的注释获得。

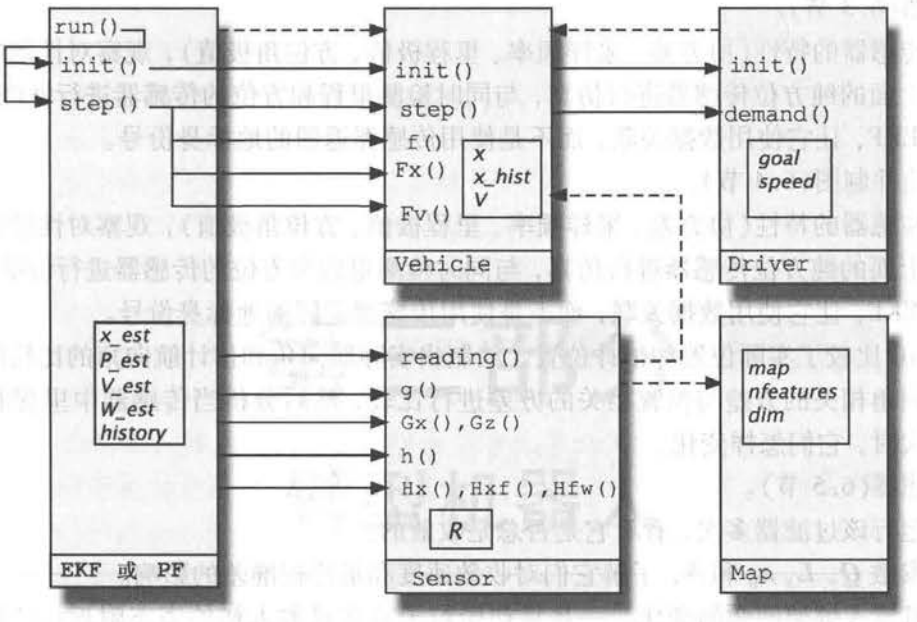


图6.14 定位与制图工具箱类之间的关系。每个类用一个矩形表示，方法调用采用从调用类到被调用类的箭头表示，属性被放在一个方框内，而虚线代表对象引用

习题

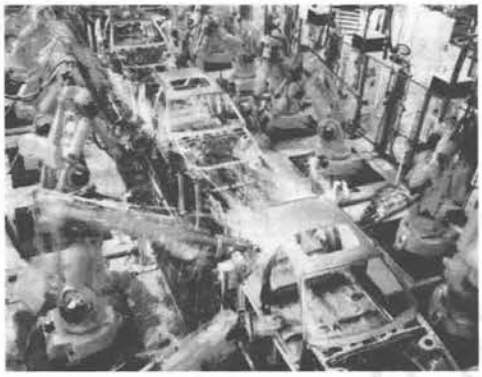
1. 换算成今天的货币价格，“经度奖”值多少钱？
2. 尝试实现一个驱动器对象(6.1.2节)，使机器人在一个指定圆心和半径的圆圈里移动。
3. 基于机器人左轮和右轮的旋转速度，推导出里程计算的方程。
4. 航迹推算(6.1.1节)。
   a) 采用不同的 $\boldsymbol{P}_0$、$\boldsymbol{V}$ 和 $\hat{\boldsymbol{V}}$ 值完成相关实验。
   b) 图6.4比较了实际位置和估计位置，绘制出实际航向角和估计航向角的图形。
   c) 将航向角相关的方差与位置相关的方差进行比较。然后分析当传感器中里程和方位角方差增大时，它们怎样变化？
5. 使用地图(6.2节)。
   a) 改变传感器的特性(协方差、采样频率、里程极值、方位角极值)，观察对性能的影响。
   b) 改变 $\boldsymbol{W}$ 和 $\hat{\boldsymbol{W}}$ 的值，观察估算误差和最终的协方差会发生什么变化。
   c) 修改 `RangeBearingSensor` 对象，创建一个纯方位传感器，即该传感器只传回方位角信息而不传回里程信息。然后完成包括所有雅可比矩阵的仿真。观察性能。
   d) 修改传感器模型，使其传回偶然误差(指定误差频率)，如不正确的里程或航标识别。观察发生了什么。
   e) 修改EKF，让它使用数据关联，而不是使用传感器返回的地标身份号。
   f) 图6.6比较了实际位置和估计位置。绘制出实际航向角和估计航向角的比较图形。
   g) 将航向角相关的方差与位置相关的方差进行比较，然后分析当传感器中里程和方位角方差增大时，它们怎样变化。

6. 创建地图(6.3 节)。
   a)改变传感器的特性(协方差、采样频率、里程极值、方位角极值),观察对性能的影响。
   b)使用上面的纯方位传感器进行仿真,与同时检测里程和方位的传感器进行性能对比。
   c)修改 EKF,让它使用数据关联,而不是使用传感器返回的地标身份号。
7. 同时定位并制图(6.4 节)。
   a)改变传感器的特性(协方差、采样频率、里程极值、方位角极值),观察对性能的影响。
   b)使用上面的纯方位传感器进行仿真,与同时检测里程和方位的传感器进行性能对比。
   c)修改 EKF,让它使用数据关联,而不是使用传感器返回的地标身份号。
   d)图 6.10 比较了实际位置和估计位置。绘制出实际航向角和估计航向角的比较图形。
   e)将航向角相关的方差与位置相关的方差进行比较,然后分析当传感器中里程和方位角方差增大时,它们怎样变化。
8. 粒子过滤器(6.5 节)。
   a)反复运行该过滤器多次,看看它是否总是收敛的?
   b)更改参数 $\boldsymbol{Q}$, $\boldsymbol{L}$, w_0 和 N,了解它们对收敛速度和最终标准差的影响。
   c)研究机器人绑架问题的变化。一是将初始粒子放在机器人初始姿态附近;二是将粒子均匀放在 xy 平面上,但把它们的方向都设为机器人的实际方向。
   d)使用一种不同类型的似然函数,比较其性能。

第三部分

臂型机器人

第7章　机械臂运动学
第8章　速度关系
第9章　动力学与控制



臂型机器人或机械臂是一种我们常见和熟知的机器人类型。我们经常看到它们工作的照片或录像，如在工厂中完成诸如装配、焊接、操作等任务，或在手术室中做手术。世界上第一台机械臂诞生于50多年前，并在工业应用中获得了巨大成功——今天已有几百万台机械臂正在世界各地进行各种作业。我们现在买的很多产品都是由机器人装配、包装或处理过的。

与上一部分讨论的移动机器人不同，机器人机械臂不能在环境中任意移动，它们有一个固定的基座，所以它们的工作空间有限。现在世界上有很多不同类型的机械臂，图 III.1 中展示了其中的一部分。最常见的是六自由度机械臂，它们由一系列刚性连杆和转动关节组成。这种串联式连杆机械臂是这一部分将要讨论的主要对象。SCARA(Selective Compliance Assembly Robot Arm，选择柔性装配机器人手臂)机器人在竖直方向是刚性的，而在水平面上具有柔性，因此它非常适合平面作业任务，如电子电路板的装配。一个直角坐标机器人沿着高架轨道有一个或两个运动自由度，它具有非常大的工作空间。在一个并联机械臂上，所有连杆都是并行地与末端工具相连，这使它具有诸多优势，例如所有电机都安装在基座上，结构刚性很强。

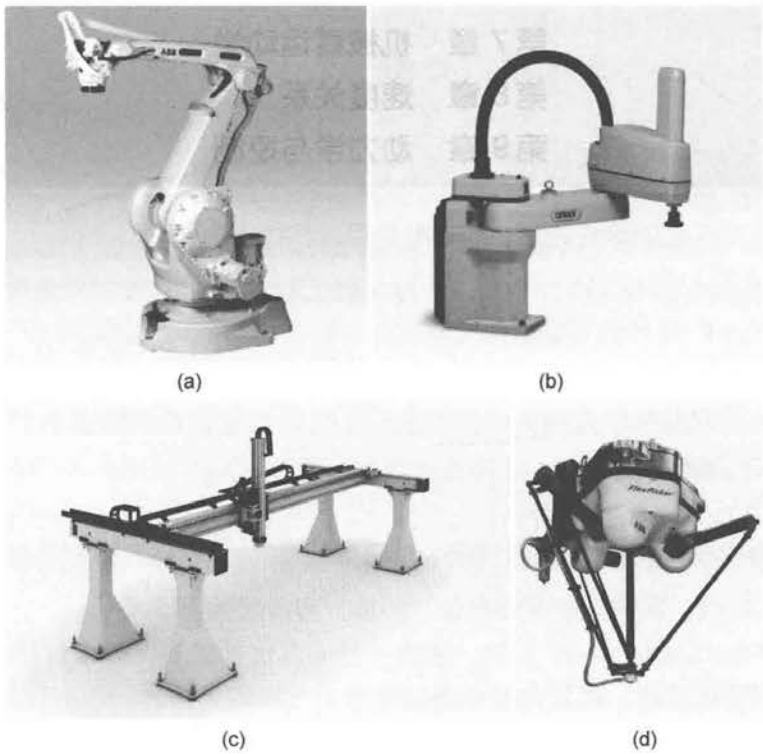
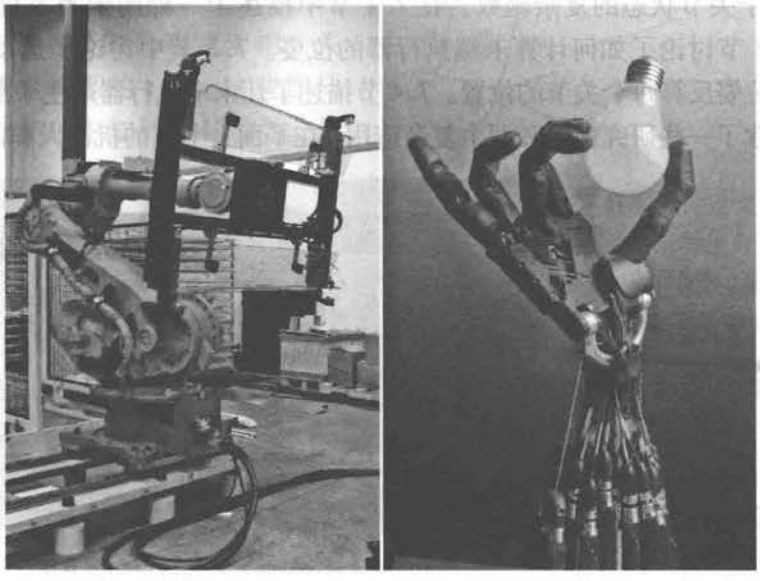


图 III.1 (a)一个 6DOF 串联机械臂。通用工业机械臂(来自 ABB 公司)；(b)有 4 个自由度的SCARA机器人，通常用于电路板装配(Adept Cobra s600 SCARA机器人的照片，由AdeptTechnology公司提供)；(c)一台直角坐标机器人，机械臂在一个高架导轨上移动(图片由瑞典Güdel AG公司的Mario Rothenbühler提供，www. gudel. com)；(d)一个并联机械臂，末端执行器由6根并联的连杆驱动(来自ABB公司)

这些非移动的机器人大大简化了诸如感知力和安全性等问题。对于一个工业机器人，其作业环境可以被安排得井然有序，这样机器人不仅能高速和高精度运动，而且其作业对象的位置也能提前获知。安全问题被简化是因为机器人的作业空间有限——只需直接用安全护栏把人员挡在机器人的工作空间之外即可。

一个机械臂用它的末端执行器或工具来操作其作业对象，如图 III.2 所示。末端执行器可以是一个简单的两指或平行卡爪夹持器，它也可以是一个复杂的仿人机械手，具有多个可弯曲的手指关节和一个对生拇指。

这一部分中的各章涵盖了串联机械臂的基础知识。第 7 章涉及串联机械臂的运动学，它给出了机器人各个关节的角度与末端执行器位姿之间的几何关系。我们还讨论了如何为机器人创建平滑运动的路径，并给出了机器人在一个平面上写一个字母的示例。此外还给出了一个四腿步行机器人的示例。第 8 章介绍了关节坐标变化速率与末端执行器速度之间的关系，这种关系由机械臂的雅克比矩阵来描述。该章还介绍了另一种在笛卡儿坐标系中产生路径的方法，并给出了作用在末端执行器上的力与关节上力矩之间的关系。第 9 章讨论了串联机械臂的动力学及其影响因素，如惯性、陀螺力、摩擦和重力等。进一步，我们讨论了机器人关节控制的策略，包括独立关节控制以及完全基于模型的非线性控制。



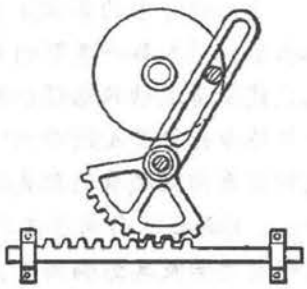
(a)　(b)

图 III.2　机器人末端执行器。(a)一个真空夹持器拿起一块玻璃；(b)一个仿人机械手(Shadow机器人公司，2008)

第7章　机械臂运动学

迷上运动学，你会获得回报。它比几何学更加丰富多彩，让我们进入四维空间。

——切比雪夫(Chebyshev)



运动学[①]是力学的一个分支，它在不考虑质量和作用力的前提下研究一个刚体，或者由多个刚体组成系统的运动。一个串联机械手是由多个连杆和运动副组成的运动链。通过运动副的连接，一个连杆能相对于另一个连杆做相对运动。运动链的一端，即机械手的基座，通常上是固定不动的，而运动链另一端上一般有夹持工具或者末端执行器，可以在空间中自由运动。

图7.1展示了两个经典的机器人，如今所有的臂式机器人均由它们演变而来。每个机器人有6个关节，显而易见末端执行器的位姿是关于每个关节状态的复杂函数。在7.1节中描述了一种用于表示机器人连杆和关节结构的符号。7.2节讨论了如何计算末端执行器的位姿。7.3节中讨论了运动学逆问题，即已知末端执行器的位姿反算每个关节的位置。7.4节描述了让末端执行器产生平滑路径的方法。本章其余部分则包含了一些研究前沿和两个复杂应用：在平面上书写的机器人和四足步行机器人。

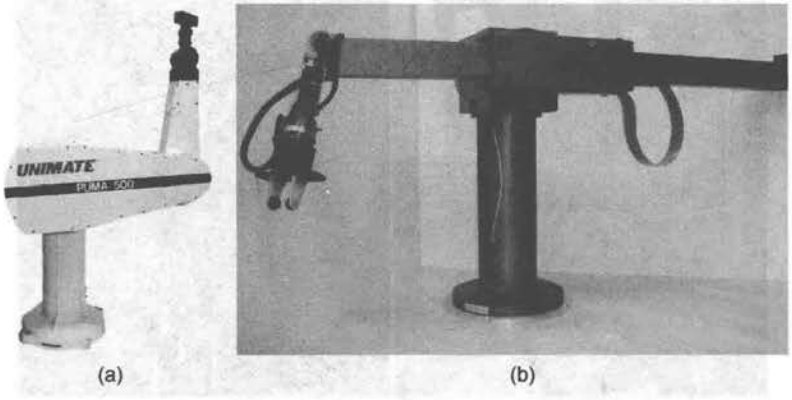


图7.1　(a)Puma 560机器人是世界上第一款现代工业机器人(图片由Oussama Khatib提供)；(b)斯坦福臂是一款早期研究用机械臂，它与众不同的地方在于它有一个移动关节(斯坦福大学人工智能实验室1972研制；图片由Oussama Khatib提供)。两款机械臂都是由机器人的先驱——维克多沙因曼(Victor Scheinman)设计的，如今两款机器人都收藏在华盛顿的史密森历史博物馆(Smithsonian Museum of American History)

7.1　描述一台机械臂

一个串联机械臂的整个链路是由一组被称为连杆的刚体组成的，这些连杆由关节(运动副)连接起来。每个关节有一个平移(滑动或移动副)或者转动(转动副)的自由度。关节的运动

① 来源于希腊文，代表运动。

改变了其邻接连杆的相对角度和位置。通常大部分机器人的关节都是回转的，然而图 7.1(b)所示的斯坦福臂有一个移动关节。

一个机器人的关节结构可以用一串字符来描述，例如 Puma 560 为“RRRRRR”，斯坦福臂为“RRPRRR”，其中每个字符代表相应关节的类型，R 是转动副，P 为移动副。迪拉维特(Denavit)和哈滕贝格(Hartenberg)于 1955 年提出了一种用于描述这种串联式链路上连杆和关节几何关系的系统方法，这就是如今熟知的 D-H 参数法。

有 N 个关节(编号从 1 到 N)的机械臂具有 $N+1$ 个连杆(编号从 0 到 N)。连杆 0 是机械臂的基座，连杆 N 则固接末端执行器或者工具。关节 j 将连杆 $j-1$ 连接到连杆 j 上，因此关节 j 带动连杆 j。连杆被看作一个刚体，它确定了其相邻两个关节轴线的空间关系。一个连杆由长度 a_j 和扭转角 α_j 两个参数来描述。同样，一个关节也由两个参数来描述，其中连杆偏移 d_j 是指沿着关节轴线从一个连杆坐标系到另一个连杆坐标系的距离，而关节角 θ_j 则是指一个连杆相对于另一个连杆关于关节轴线的转动角度。

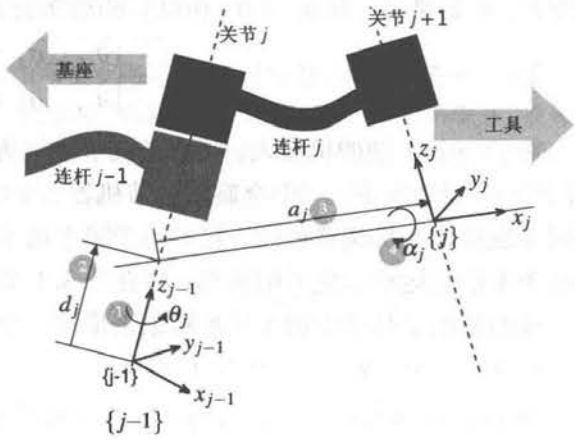


图 7.2　标准 D-H 参数的定义。深灰色和黑色分别表示与连杆$j+1$和连杆j关联的所有参数。而圆圈中的数字表示进行基本变换的顺序

图 7.2 对这种参数法进行了图示说明。$\{j\}$ 坐标系固接于连杆 j 的远端或末梢。关节 j 的轴线即为该坐标系的 z 轴。这些连杆和关节的参数即我们所熟知的 D-H 参数，总结于表 7.1 中。

按照约定，第一个关节，即关节 1，连接连杆 0 和连杆 1。连杆 0 为机器人的基座。通常来说第一个连杆有 $d_1 = \alpha_1 = 0$，然而我们可以设定 $d_1 > 0$ 用来表示肩关节的高度超出基座。在一个制造系统中，基座通常是固定在环境中的，然而它有可能安装在一个移动的基座上，例如一架航天飞机、一个水下机器人或者一辆卡车。

表 7.1　D-H 参数：包括它们的物理含义、符号和定义

关节角	θ_j	x_{j-1} 轴和 x_j 轴之间关于 z_{j-1} 轴的角度	转动关节变量
连杆偏移	d_j	沿着 z_{j-1} 轴，$j-1$ 坐标系原点到 x_j 轴的距离	移动关节变量
连杆长度	a_j	沿着 x_j 轴，z_{j-1} 轴和 z_j 轴之间的距离；对于相交轴，它与 $\hat{z}_{j-1} \times \hat{z}_j$ 平行	常量
连杆扭转角	α_j	z_{j-1} 轴和 z_j 轴之间关于 x_j 轴的角度	常量
关节类型	σ_j	$\sigma = 0$ 为转动副，$\sigma = 1$ 为移动副	常量

最后一个关节，即关节 N，连接连杆 $N-1$ 和连杆 N。连杆 N 是机器人的工具，它的相关参数 d_N 和 a_N 分别代表了工具的长度及其 x 轴的偏移量。如图 2.14 所示，这个工具通常指向 z 轴。

从连杆坐标系 $\{j-1\}$ 到坐标系 $\{j\}$ 的变换被定义为基本旋转[①]和平移，其形式如下：

$$^{j-1}\mathbf{A}_j(\theta_j, d_j, a_j, \alpha_j) = \mathbf{T}_{Rz}(\theta_j)\mathbf{T}_z(d_j)\mathbf{T}_x(a_j)\mathbf{T}_{Rx}(\alpha_j) \tag{7.1}$$

它可以展开为

① 这个 3×3 阶的标准正交矩阵通过增加为 0 的平移元素后，扩展形成一个 4×4 阶齐次变换矩阵。

$$ {}^{j-1}\boldsymbol{A}_j = \begin{pmatrix} \cos\theta_j & -\sin\theta_j\cos\alpha_j & \sin\theta_j\sin\alpha_j & a_j\cos\theta_j \\ \sin\theta_j & \cos\theta_j\cos\alpha_j & -\cos\theta_j\sin\alpha_j & a_j\sin\theta_j \\ 0 & \sin\alpha_j & \cos\alpha_j & d_j \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{7.2} $$

背景材料： 雅克·迪拉维特(Jacques Denavit)和理查德·哈滕贝格(Richard Hartenberg)在他们1955年的一篇文章[Denavit and Hartenberg 1955]和随后那本经典教材《连杆机构的运动综合》(*Kinematic Synthesis of Linkages*)[Hartenberg and Denavit 1964]里，描述了很多关于串联机械臂运动学的重要概念。

参数 a_j 和 α_j 始终是常量。对于转动关节 θ_j 是关节变量且 d_j 是常量，然而对于移动关节 d_j 是变量，θ_j 是常量，且 $\alpha_j = 0$。在以后的很多公式中，我们将使用如下的广义关节坐标：

$$ q_j = \begin{cases} \theta_j, & \sigma_j = 0 \text{ 转动关节} \\ d_j, & \sigma_j = 1 \text{ 移动关节} \end{cases} $$

对于一个 N 轴的机器人，其广义关节坐标为 $q \in \mathcal{C}$，其中 $\mathcal{C} \subset \mathbb{R}^N$ 被称为关节空间或者位形空间①。一般情况下，一个全旋转关节机器人 $\mathcal{C} \subset \mathbb{S}^N$ 的关节坐标被称为关节角度。这种关节坐标同时也被称为机械臂姿态，它不同于属于笛卡儿姿态 $\xi \in SE(3)$ 的末端执行器的姿态。位形这个术语是运动学位形的简称，将在7.3.1节中进行讨论。

我们使用工具箱中的 `Link` 对象来描述一个机器人连杆，其创建如下：

```
>> L = Link([0, 0.1, 0.2, pi/2, 0])
L =
 theta=q, d=0.1, a=0.2, alpha=1.571 (R,stdDH)
```

其中，输入向量的元素按顺序分别是 θ_j，d_j，a_j，α_j。而可选的第5个元素 σ_j 则表明该关节是转动的($\sigma_j = 0$)还是移动的($\sigma_j = 1$)，如果未指定该值则默认为转动关节。

该 `Link` 对象的显示值不仅给出了其运动学参数，还表示了其他的一些状态，例如它是一个转动关节(标签"`R`")，以及使用了标准的D-H参数表达(标签"`stdDH`")。②

尽管赋给了 θ 一个值，但并没有显示出来——这个值在运动学参数系列中仅仅作为一个占位者。θ 将被关节变量 q 所取代，因此它的值在 `Link` 对象中被忽略掉了。这个值将由 `Link` 对象进行管理。

`Link` 对象有许多的参数和方法，详细内容可参见在线文件。然而最常用的参数和方法将通过下面例子进行说明。当 $q = 0.5$ 时，方程(7.2)表示的连杆变换如下：

```
>> L.A(0.5)
ans =
    0.8776   -0.0000    0.4794    0.1755
    0.4794    0.0000   -0.8776    0.0959
         0    1.0000    0.0000    0.1000
         0         0         0    1.0000
```

这是一个4×4阶的齐次变换矩阵。我们可以对不同连杆参数的值进行读取或更改。例如：

```
>> L.RP
ans =
     R
```

① 这和4.1节介绍的移动机器人具有相同的概念。

② 一种有细微差异，经过修正的D-H参数法将在7.5.3节中进行讨论。

它表明这个连杆是一个转动型的。而

```
>> L.a
ans =
    0.2000
```

它返回运动参数 a 的值。最后，一个连杆还能包含一个偏移量：

```
>> L.offset = 0.5;
>> L.A(0)
ans =
    0.8776   -0.0000    0.4794    0.1755
    0.4794    0.0000   -0.8776    0.0959
         0    1.0000    0.0000    0.1000
         0         0         0    1.0000
```

该值将在计算连杆变换之前加到关节变量中，详见 7.5.1 节。

背景材料：雅克·迪拉维特(1930—)出生于巴黎，在那里获得了他的学士学位，随后在美国的西北大学取得了他在机械工程专业的硕士和博士学位。1958 年，他加入了西北大学的机械工程和宇航科学系，在那里他和哈滕贝格的合作正式开始。除了在动力学和运动学方面的兴趣，迪拉维特同样对等离子体物理学和运动学感兴趣。在他的专著出版后，他加入了加利福尼亚州的劳伦斯利弗莫尔国家实验室(Lawrence Livermore National Lab)，在那里他开展了等离子体物理学问题的计算机分析研究。他于 1982 年退休。

7.2　正运动学

机械臂的正运动学通常表述为如下的函数形式：

$$\xi_E = \mathcal{K}(\boldsymbol{q}) \tag{7.3}$$

它表明末端执行器的位姿是基于关节坐标的一个函数。若使用齐次变换，其表达式将是由方程(7.2)所给的单个连杆变换矩阵的简单乘积。对于一个 N 轴机械臂，有

$$\xi_E \sim {}^0\boldsymbol{T}_E = {}^0\boldsymbol{A}_1\,{}^1\boldsymbol{A}_2 \cdots {}^{N-1}\boldsymbol{A}_N \tag{7.4}$$

对于任何一个串联机械臂，无论其关节的数量和类型如何，都可以计算出其正向运动学的解。如何确定机器人上每个连杆的 D-H 参数将在 7.5.2 节中详细讨论。

末端执行器的位姿 $\xi_E \sim \boldsymbol{T}_E \in SE(3)$ 共有 6 个自由度——3 个移动自由度和 3 个转动自由度。因此为了使末端执行器具有任意的位姿，机械臂通常具有 6 个关节或者自由度。对于一个 6 轴机器人，其总的变换矩阵通常被写作 $\boldsymbol{T}_6$。

背景材料：理查德·哈滕贝格(1907—1997)出生于芝加哥，他在美国威斯康星大学学习并获得了学位。他曾服务于商船队，还与太空飞行先驱西奥多·冯·卡门一起在哥廷根大学开展了两年的航空学研究。后来他一直在美国西北大学担任机械工程学的教授，为期 56 年。他在运动学方面的研究引领了该领域在 20 世纪 60 年代的复兴，他的努力还使运动学成为用计算机分析和设计复杂机械的科学基础。他还写了大量关于机械工程历史的文章。

7.2.1　两连杆机器人

我们将要讨论的第一个机器人是如图 7.3 所示的两连杆平面机械手。它的 D-H 参数如下所示，然后我们用这些参数创建一个 Link 对象的向量：

连杆	θ_i	d_i	a_i	α_i	σ_i
1	q_1	0	1	0	0
2	q_2	0	1	0	0

```
>> L(1) = Link([0 0 1 0]);
>> L(2) = Link([0 0 1 0]);
>> L
L =
 theta=q1, d=0, a=1, alpha=0 (R,stdDH)
 theta=q2, d=0, a=1, alpha=0 (R,stdDH)
```

这个向量将被传递给构造函数 SerialLink：

```
>> two_link = SerialLink(L, 'name', 'two link');
```

它将会返回一个 SerialLink 对象，显示如下：

```
>> two_link
two_link =
two link (2 axis, RR, stdDH)
+---+-----------+-----------+-----------+-----------+
| j |     theta |         d |         a |     alpha |
+---+-----------+-----------+-----------+-----------+
|  1|         q1|          0|          1|          0|
|  2|         q2|          0|          1|          0|
+---+-----------+-----------+-----------+-----------+

grav =    0  base = 1  0  0  0   tool =  1  0  0  0
          0           0  1  0  0          0  1  0  0
       9.81           0  0  1  0          0  0  1  0
                      0  0  0  1          0  0  0  1
```

它为这个机器人提供了一种简明的描述。可以看出，该机器人的结构字符串为“RR”，表明它有两个转动关节，而且它是使用标准的 D-H 参数定义的，另外重力默认作用于 z 轴方向[①]。同时对象中还列出了连杆的运动学参数，而关节变量用“q1”和“q2”表示。我们还为机器人取了一个名字，它在任何图形显示机器人时都会出现。输入指令：

```
>> mdl_planar2
```

它将直接执行上述步骤，将该机器人定义在 MATLAB 的工作空间，并创建名为 twolink 的 SerialLink 对象。

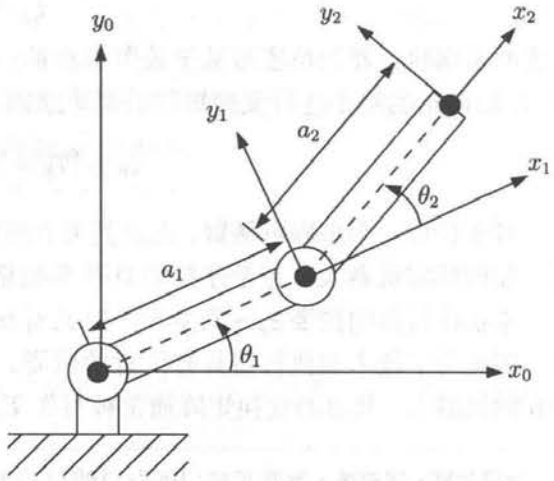


图 7.3 两连杆机器人([Spong et al. 2006],图3.6,获授权复制)

该 SerialLink 对象是使用机器人工具箱的关键。它含有大量的方法和特性，我们将在后续部分中进行介绍，更详细的说明参见在线文档。一些简单的实例如下：

```
>> twolink.n
ans =
     2
```

该指令返回关节数目。

① 与机器人运动平面垂直。

```
>> links = twolink.links
L =
theta=q1, d=0, a=1, alpha=0 (R,stdDH)
theta=q2, d=0, a=1, alpha=0 (R,stdDH)
```

该指令返回一个 Link 对象的向量，Link 对象中包含机器人①。我们还可以复制一个 SerialLink 对象，并命名为 bob②：

```
>> clone = SerialLink(twolink, 'name', 'bob')
clone =
bob (2 axis, RR, stdDH)
+---+-----------+-----------+-----------+-----------+
| j |     theta |         d |         a |     alpha |
+---+-----------+-----------+-----------+-----------+
|  1|         q1|          0|          1|          0|
|  2|         q2|          0|          1|          0|
+---+-----------+-----------+-----------+-----------+

grav =    0  base = 1  0  0  0   tool =  1  0  0  0
          0         0  1  0  0           0  1  0  0
       9.81         0  0  1  0           0  0  1  0
                    0  0  0  1           0  0  0  1
```

现在可以让机械臂工作了。正向运动学的计算使用了 fkine 方法。这个例子中，$q_1 = q_2 = 0$。

```
>> twolink.fkine([0 0])
ans =
     1     0     0     2
     0     1     0     0
     0     0     1     0
     0     0     0     1
```

该方法将返回代表机器人第二个连杆坐标系位姿的齐次变换 $\boldsymbol{T}_2$。对于一个不同的位形，工具位姿为

```
>> twolink.fkine([pi/4 -pi/4])
ans =
    1.0000         0         0    1.7071
         0    1.0000         0    0.7071
         0         0    1.0000         0
         0         0         0    1.0000
```

按照惯例，工具箱使用的关节坐标都是行向量。

用以下指令可以画出机器人的图形：

```
>> twolink.plot([0 0])
>> twolink.plot([pi/4 -pi/4])
```

如图 7.4 所示。图中包含了机器人的名称，末端连杆的坐标系（此例的 $\boldsymbol{T}_2$），所有的关节和它们的轴线，以及机器人投影在地面上的影子。有关 plot 方法的更多特性，如多视图和多机器人绘制，将在 7.8 节中介绍，更详细的说明参见在线文档。

上面介绍的简单两连杆机器人所能达到的姿态有限，因为它的运动完全在 xy 平面内，其任务空间是 $\mathcal{T} \subset \mathbb{R}^2$。

---

① Link 对象来源于 MATLAB 的 handle 类，并能适时进行设置。因此我们可以写入如"twolink.links(1).a = 2"的代码来改变运动学参数 a_2。

② 当要图形显示多个机器人时，每个机器人都需要一个唯一的名称，详见 7.8 节。

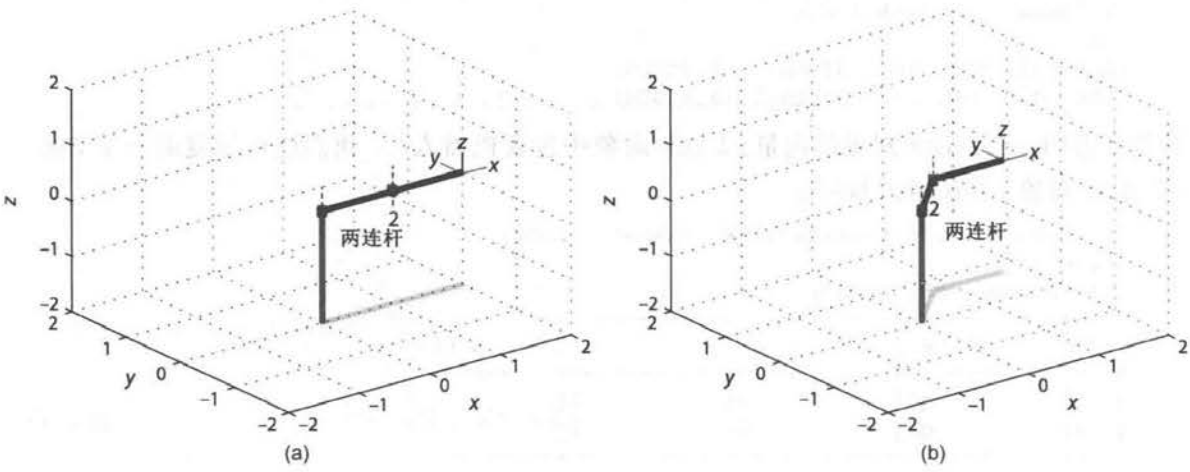


图 7.4 两种不同姿态的两连杆机器人。(a)姿态为(0,0)；(b)姿态为$\left(\frac{\pi}{4}, -\frac{\pi}{4}\right)$。注意图形的细节。旋转关节用圆柱体表示，虚线段表示关节轴线。末端连杆的坐标系以及机械臂在地面上的影子也都显示了出来

7.2.2 六轴机器人

真正实用的机器人的任务空间是 $\mathcal{T} \subset SE(3)$，这就使得其末端执行器能到达任意的位置和姿态。这个任务空间有 6 个空间自由度：3 个平移和 3 个转动。这需要一个具有位形空间 $\mathcal{C} \subset \mathbb{R}^6$ 的机器人，即一个有 6 关节的机器人。本节中，我们将以 Puma 560 机器人作为所有全旋转六轴机械臂的一个例子来介绍。我们使用如下指令来定义一个 Puma 560 机器人的实例[①]：

```
>> mdl_puma560
```

它将在工作区中创建一个名为 p560 的 SerialLink 对象：

```
>> p560
p560 =
Puma 560 (6 axis, RRRRRR, stdDH)
 Unimation; viscous friction; params of 8/95;
+---+-----------+-----------+-----------+-----------+
| j |     theta |         d |         a |     alpha |
+---+-----------+-----------+-----------+-----------+
|  1|         q1|          0|          0|      1.571|
|  2|         q2|          0|     0.4318|          0|
|  3|         q3|       0.15|     0.0203|     -1.571|
|  4|         q4|     0.4318|          0|      1.571|
|  5|         q5|          0|          0|     -1.571|
|  6|         q6|          0|          0|          0|
+---+-----------+-----------+-----------+-----------+

grav =    0  base = 1  0  0  0   tool =  1  0  0  0
          0           0  1  0  0          0  1  0  0
       9.81           0  0  1  0          0  0  1  0
                      0  0  0  1          0  0  0  1
```

请注意，a_j 和 d_j 都采用国际标准单位，这意味着正向运动学的平移部分也将使用国际标准单位。

① 工具箱中自带模板指令来定义很多种常见的工业机器人，包括 Motoman HP6、Fanuc 10L、ABB S4 2.8 以及 斯坦福臂。

背景材料：如图 7.1 所示的 Puma 560 机器人(PUMA 代表 Programmable Universal Manipulator for Assembly，即用于装配的可编程通用机械臂)于 1978 年正式推出，并从此闻名于世。它的特色包括拟人化的设计、电动马达和一个球形的手腕，这使得它成为第一个现代化的工业机器人，也是后续其他机器人的原型。

Puma 560 促进了 20 世纪 80 年代的机器人研究，当时是一种很常见的实验室机器人。虽然现在它已经过时且不多见，但为了纪念其在机器人研究中的重要作用，这里我们仍然选用它来作例子。对于我们来说，这个机器人的优势是对它研究已经非常成熟，而且它的参数也是众所周知的——它有机器人研究的“白鼠”之称。

大多数现代的六轴工业机器人在结构上都非常相似，运动学上只需简单地改变 D-H 参数就能通用。工具箱中包含有大量常见工业机器人的运动学模型，包括 Motoman HP6、Fanuc 10L 及 ABB S4 2.8。

指令“mdl_puma560”还在工作区中创建了大量关节坐标向量，它们代表了一些典型的机器人位形：

qz	$(0,0,0,0,0,0)$	零角度
qr	$\left(0,\frac{\pi}{2},-\frac{\pi}{2},0,0,0\right)$	就绪状态，机械臂伸直且垂直
qs	$\left(0,0,-\frac{\pi}{2},0,0,0\right)$	伸展状态，机械臂伸直且水平
qn	$\left(0,\frac{\pi}{4},-\pi,0,\frac{\pi}{4},0\right)$	标准状态，机械臂处于一个灵巧工作姿态①

上述 4 种位形如图 7.5 所示。这些图是用 plot 方法绘制而成的，例如：

```
>> p560.plot(qz)
```

机器人的正向运动学计算如下：

```
>> p560.fkine(qz)
ans =
    1.0000         0         0    0.4521
         0    1.0000         0   -0.1500
         0         0    1.0000    0.4318
         0         0         0    1.0000
```

它返回一个对应末端执行器位姿的齐次变换 $\boldsymbol{T}_6$。机器人上连杆 6 坐标系 $\{6\}$ 的原点被定义②为最后三个关节轴的交点——这个点在机器人的腕关节内部。通过如下的命令：

```
>> p560.tool = transl(0, 0, 0.2);
```

我们可以定义一个工具变换，从 $\boldsymbol{T}_6$ 坐标系变换到实际的工具顶端。在本例中，工具顶端是沿着 $\boldsymbol{T}_6$ 的 z 轴延伸了 200 mm③。此时，工具顶端(也称工具中心点或者 TCP)的位姿变成

```
>> p560.fkine(qz)
ans =
    1.0000         0         0    0.4521
         0    1.0000         0   -0.1500
         0         0    1.0000    0.6318
         0         0         0    1.0000
```

我们在定义上述运动学变换时，认为机器人的基座点是位于机器人结构内部的腰关节和肩关

① 远离奇异点，将在 7.4.3 节中讨论。

② 通过 mdl_puma560 指令。

③ 另外，我们可以改变运动参数 d_6。但工具变换的方法更普遍，因为最后一个连杆的运动学参数只允许设置 d_6、a_6 和 α_6 的值，它们分别对应沿 z 轴平移、沿 x 轴平移和绕 x 轴旋转。

节轴线的交点上。Puma 560 机器人有一个 30 英寸高的底座结构。我们可以通过一个基座变换将机器人的原点从其内部转移到底座的下端部：

```
>> p560.base = transl(0, 0, 30*0.0254);
```

其中，为了一致，我们将底座高度转换为了国际标准单位。现在，经过基座变换和工具变换之后，正向运动学变为

```
>> p560.fkine(qz)
ans =
    1.0000         0         0    0.4521
         0    1.0000         0   -0.1500
         0         0    1.0000    1.3938
         0         0         0    1.0000
```

可以看到，现在工具的 z 坐标值比以前更大。

我们还可以做更多有趣的事情，例如：

```
>> p560.base = transl(0,0,3) * trotx(pi);
>> p560.fkine(qz)
ans =
    1.0000         0         0    0.4521
         0   -1.0000   -0.0000    0.1500
         0    0.0000   -1.0000    2.3682
         0         0         0    1.0000
```

它将机器人的坐标原点比世界坐标原点升高了 3 m，并让机器人坐标系绕着 x 轴旋转 180°。现在机器人就像挂在天花板上一样！

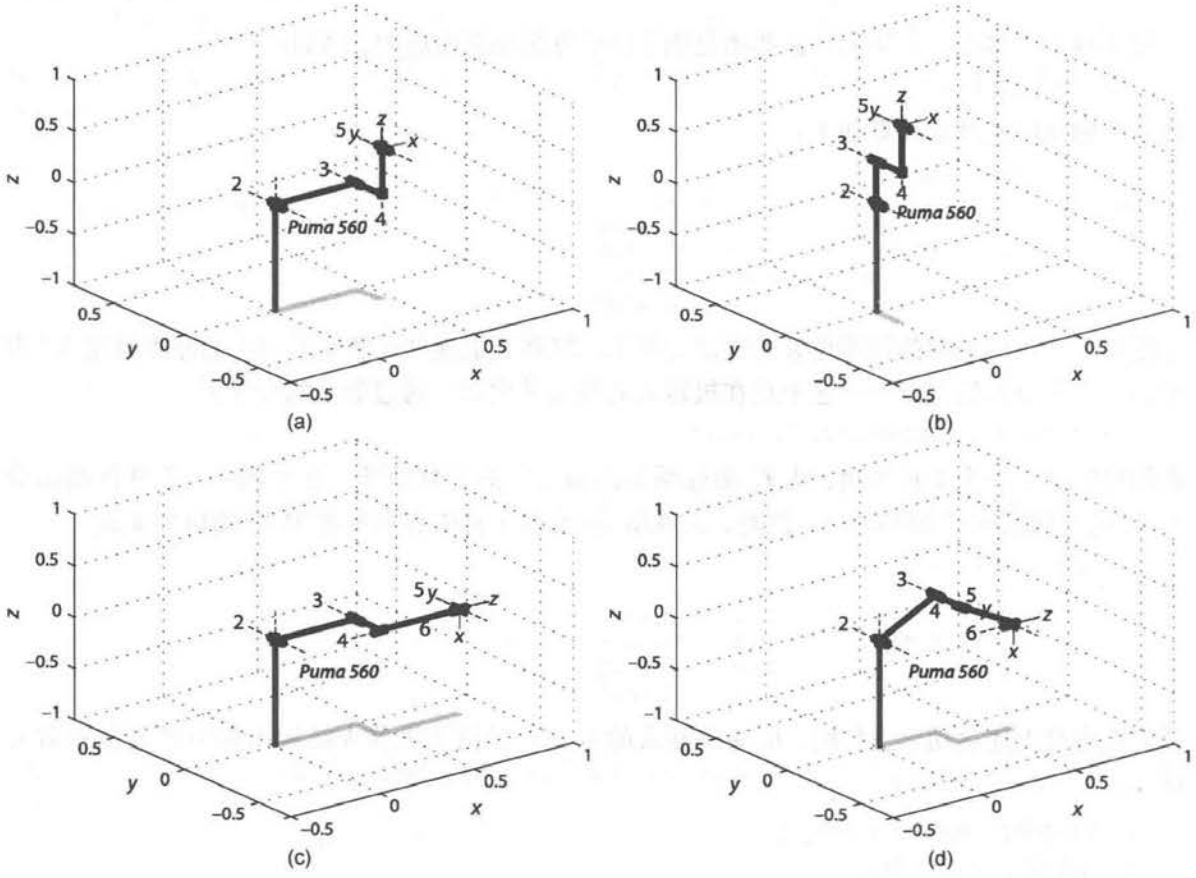


图 7.5 Puma 560 机器人的 4 种不同位形。(a)零角度；(b)就绪；(c)伸展；(d)标准

背景材料：拟人意味着有类似人类的特点。Puma 560 机器人被设计为具有和人类近似的尺寸和伸展范围。它也有一个球形关节，就像人类的手腕。

机器人专家也倾向于用拟人化的术语描述机器人。我们使用像腰、肩、肘和腕这样的名称来描述一个串联机械臂。对于 Puma 560 机器人，这些名称分别对应于其关节 1、2、3 及 4～6。

工具箱支持关节角按时间序列或按轨迹线的显示方式，例如：

```
>> q
q =
         0         0         0         0         0         0
         0    0.0365   -0.0365         0         0         0
         0    0.2273   -0.2273         0         0         0
         0    0.5779   -0.5779         0         0         0
         0    0.9929   -0.9929         0         0         0
         0    1.3435   -1.3435         0         0         0
         0    1.5343   -1.5343         0         0         0
         0    1.5708   -1.5708         0         0         0
```

其中每一行表示不同时间步下的关节坐标值，而每列的数字则对应每个关节[①]。此时再使用方法 fkine：

```
>> T = p560.fkine(q);
```

它将返回一个三维矩阵：

```
>> about(T)
T [double] : 4x4x8 (1024 bytes)
```

其中，前两维是一个 4×4 阶的齐次变换矩阵，第三维是时间步。对应于 q 中第 4 行关节坐标值的齐次变换矩阵是

```
>> T(:,:,4)
ans =
    1.0000    0.0000         0    0.3820
    0.0000    1.0000   -0.0000   -0.1500
         0   -0.0000    1.0000    1.6297
         0         0         0    1.0000
```

如何创建一个轨迹将在 7.4 节讨论。

7.3　逆运动学

上一节已经展示了机械臂的正运动学求解方法，即在给定关节坐标以及可选工具和基座变换的条件下，如何确定末端执行器的位姿。

实际应用中的另一个问题是运动学的逆问题：给定期望的末端执行器位姿 ξ_E，如何求取所需的关节坐标？例如，如果我们知道一个物体的笛卡儿坐标位姿，那么机器人需要什么样的关节坐标才能接近这个物体？这就是逆运动学问题，写成函数形式如下：

$$\boldsymbol{q} = \mathcal{K}^{-1}(\xi) \tag{7.5}$$

一般而言这个函数的解不是唯一的，而且对于某些类型的机械臂甚至没有封闭形式的解存在，只能求出一个数值解。

① 由函数 jtraj 产生，将在 7.4.1 节中讨论。

7.3.1 封闭形式解

对于一个六轴机器人，其逆运动学具有封闭形式解的一个必要条件是：三个腕关节的轴相交于一个点。这意味着腕关节的运动只改变末端执行器的姿态，而不改变其位置。这种机构被称为球腕，而且几乎所有的工业机器人都具有这样的腕关节。

我们将继续使用 Puma 机器人模型来探讨逆运动学问题。

```
>> mdl_puma560
```

机器人标准状态的关节坐标为

```
>> qn
qn =
         0    0.7854    3.1416         0    0.7854         0
```

对应的末端执行器位姿是

```
>> T = p560.fkine(qn)
T =
   -0.0000    0.0000    1.0000    0.5963
   -0.0000    1.0000   -0.0000   -0.1501
   -1.0000   -0.0000   -0.0000   -0.0144
         0         0         0    1.0000
```

Puma 560 是一种包含一个球腕的六轴机器人，因此我们使用 `ikine6s` 方法来计算其逆运动学的封闭解[①]。要实现位姿 `T`，所需的关节坐标为

```
>> qi = p560.ikine6s(T)
qi =
    2.6486   -3.9270    0.0940    2.5326    0.9743    0.3734
```

但令人惊讶的是，这些解与我们开始采用的标准状态关节坐标完全不同。但是，如果我们进一步探究一下：

```
>> p560.fkine(qi)
ans =
   -0.0000    0.0000    1.0000    0.5963
    0.0000    1.0000   -0.0000   -0.1500
   -1.0000    0.0000   -0.0000   -0.0144
         0         0         0    1.0000
```

我们发现，两组不同的关节坐标值得到了相同的末端执行器位姿，而这两组坐标值对应的机器人位形如图 7.6 所示。该 Puma 机器人的肩关节相对于腰部有个水平偏移，所以一组坐标解对应的机械臂在腰部的左侧，而另一组则对应在右侧，它们分别被称为左手运动学位形和右手运动学位形。一般来说，有 8 组不同的关节坐标值可以得到相同的末端执行器位姿——正如前面所提到的，逆解不是唯一的。

我们可以人为指定获得右手位形解：

```
>> qi = p560.ikine6s(T, 'ru')
qi =
   -0.0000    0.7854    3.1416    0.0000    0.7854   -0.0000
```

它就是我们最初的那组关节角度，并且是肘关节在上的右手位形。

除了左手位形和右手位形的解，还有一些解对应肘关节在上或在下[②]，以及手腕翻转或不翻转。对于 Puma 560 机器人的腕关节 θ_4，具有一个较大的旋转范围，因此其可以采用两个相

① `ikine6s` 方法会检查机器人的 D-H 参数，以确定它是否满足这些标准。

② 更准确地讲，肘关节是高于或低于肩关节的。

差 π 的关节角度。又由于大多数机器人夹持器只需两个手指，如图 2.14 所示，因此相差 π 并不会改变它抓取物体的效果。

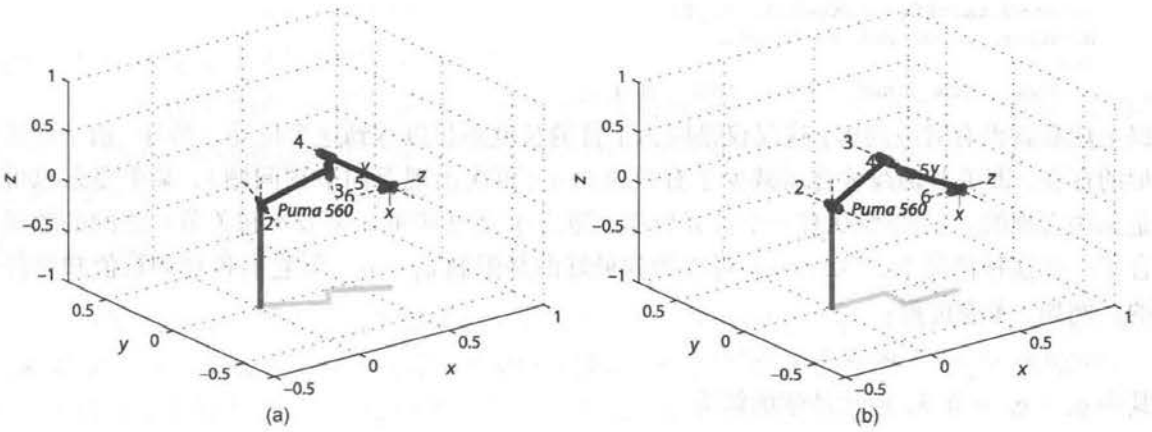


图 7.6　逆运动学问题的两组解——左手位形解和右手位形解。机械臂的阴影清楚地表明了两种不同的位形

各种不同的运动学位形如图 7.7 所示。由 ikine6s 返回的运动学位形由一个或多个字符标志来控制：

左手或右手	'l','r'
肘部在上或在下	'u','d'
手腕翻转或不翻转	'f','n'

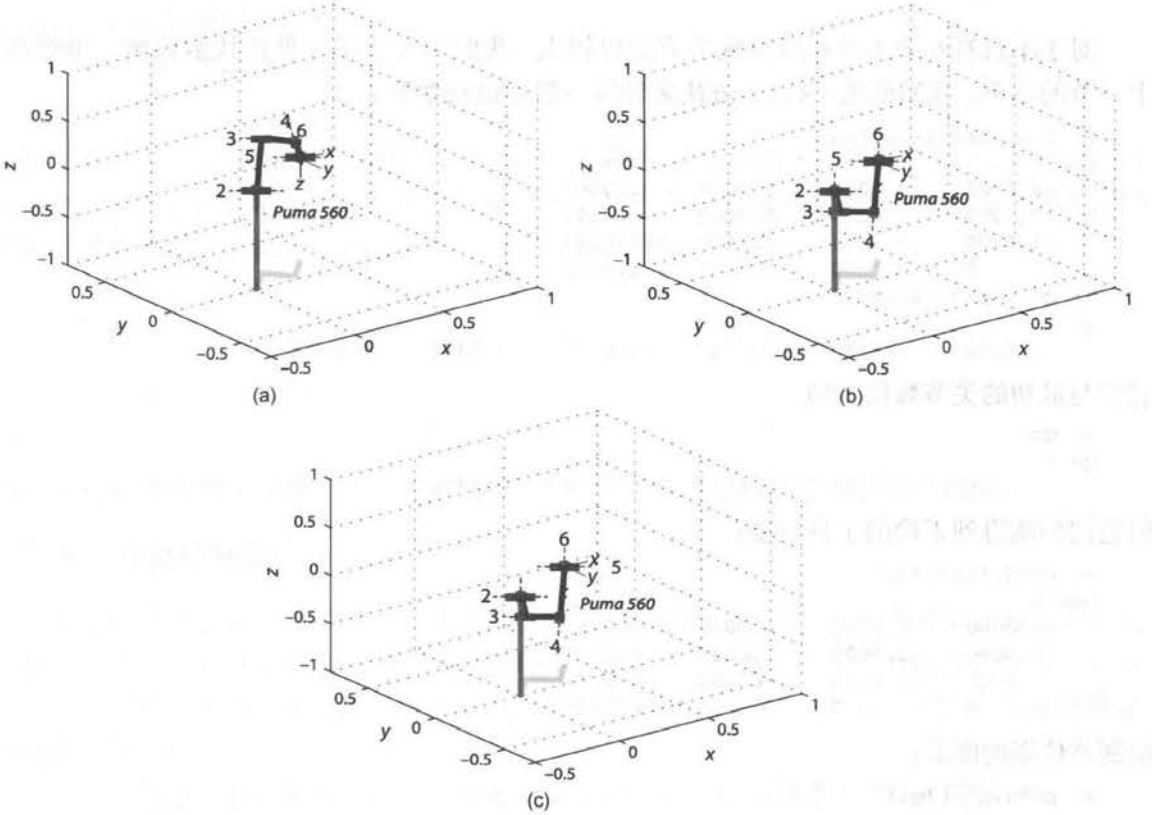


图 7.7　Puma 560 机器人的不同位形。(a)右手-肘上-腕不翻转；(b)右手-肘下-腕不翻转；(c)右手-肘下-腕翻转

由于关节转角上的机械限制以及连杆之间可能发生的碰撞，并不是所有 8 种位形解在物理上都可以实现，有时甚至可能不存在可以实现的解。例如：

```
>> p560.ikine6s( transl(3, 0, 0) )
Warning: point not reachable
ans =
   NaN   NaN   NaN   NaN   NaN   NaN
```

以上位姿就没有对应的解，这仅仅是因为手臂的长度不足以达到这个位姿。另外，由于奇异位形的存在，即关节轴线重合而减少了有效自由度(再次出现万向节锁问题)，某个姿态也可能是无法实现的。Puma 560 有一个奇异位形，即当 q_5 等于零时，关节 4 和关节 6 之间的轴线重合了。在这种情况下，ikine6s 可以做到最好的是限制 $q_4 + q_6$，但它们各自的数值却是任意的。例如，考虑位形：

```
>> q = [0 pi/4 pi 0.1 0 0.2];
```

其中 $q_4 + q_6 = 0.3$，逆运动学的解为

```
>> p560.ikine6s(p560.fkine(q), 'ru')
ans =
  -0.0000    0.7854    3.1416    2.9956    0.0000   -2.6956
```

q_4 和 q_6 的值完全不一样，但是它们的和始终等于 0.3。

```
>> q(4)+q(6)
ans =
    0.3000
```

7.3.2　数值解

对于不具有 6 个关节和球形腕关节的机器人，我们需要使用一种迭代数值解。继续使用上一节的示例，我们使用 ikine 方法来计算一般逆运动学解：

```
>> T = p560.fkine(qn)
ans =
   -0.0000    0.0000    1.0000    0.5963
   -0.0000    1.0000   -0.0000   -0.1501
   -1.0000   -0.0000   -0.0000   -0.0144
         0         0         0    1.0000
>> qi = p560.ikine(T)
qi =
    0.0000   -0.8335    0.0940   -0.0000   -0.8312    0.0000
```

它们与最初的关节数值不同：

```
>> qn
qn =
         0    0.7854    3.1416         0    0.7854         0
```

但它们仍能得到正确的工具位姿：

```
>> p560.fkine(qi)
ans =
   -0.0000    0.0000    1.0000    0.5963
   -0.0000    1.0000   -0.0000   -0.1501
   -1.0000   -0.0000   -0.0000   -0.0144
         0         0         0    1.0000
```

绘制该位姿的图形：

```
>> p560.plot(qi)
```

结果清楚地显示 ikine 方法找到了肘关节在下的位形。

这个通用数值解法的局限性是：它不像解析法能提供对手臂运动位形的明显控制，只有通

过设定关节坐标初始值(默认为零)的隐含控制。如果指定初始关节坐标：

```
>> qi = p560.ikine(T, [0 0 3 0 0 0])
qi =
    0.0000    0.7854    3.1416    0.0000    0.7854   -0.0000
```

这时其数值解收敛于肘关节在上的位形①。

正如所预料的那样，通用数值方法 ikine 要比解析方法 ikine6s 慢很多。然而，它也具有很大的优势，它能够求解处于奇异位形以及非 6 关节型的机械臂。有关 Ikine 方法的详细原理见 8.4 节。

7.3.3　欠驱动机械臂

一个欠驱动机械臂是指关节数少于 6 个的机械臂，因此其末端执行器可以达到的位姿是受限制的。第三部分开始时展示的 SCARA 机器人就是一种典型的例子，它们通常有一个 x-y-z-θ 的任务空间 $\mathcal{T} \subset \mathbb{R}^3 \times \mathbb{S}$，以及一个位形空间 $\mathcal{C} \subset \mathbb{S}^3 \times \mathbb{R}$。

我们再次回到 7.2.1 节中的两连杆机械臂例子，首先定义该机器人：

```
>> mdl_twolink
```

然后定义所期望的末端执行器的位姿：

```
>> T = transl(0.4, 0.5, 0.6);
```

然而此位姿对于该两连杆机器人是过约束的——工具既不能满足在 $\hat{\boldsymbol{x}}_2$、$\hat{\boldsymbol{y}}_2$ 方向上的姿态约束，也不能实现在 z 轴上除零之外的任何平移量。因此，我们要求 ikine 方法只考虑在 x 轴和 y 轴平移方向上的误差，而忽略其他所有笛卡儿坐标的自由度。为了达到这个目的，我们指定一个遮盖向量作为第四个参数：

```
>> q = twolink.ikine(T, [0 0], [1 1 0 0 0 0])
q =
   -0.3488    2.4898
```

该遮盖向量的元素分别对应末端执行器坐标系中的 3 个移动和 3 个转动：t_x, t_y, t_z, r_x, r_y, r_z。在这个示例中，我们规定只考虑在 x 轴和 y 轴平移方向上的误差(向量中的非零元素)。产生的关节角度值对应于某个末端位姿。求解其正运动学解：

```
>> twolink.fkine(q)
ans =
ans =
   -0.5398   -0.8418         0    0.4000
    0.8418   -0.5398         0    0.5000
         0         0    1.0000         0
         0         0         0    1.0000
```

得到我们所期望的 x 轴和 y 轴上的平移量，但姿态方向和 z 轴的平移是不正确的。

7.3.4　冗余机械臂

冗余机械臂是指具有多于 6 个关节的机器人。正如前面提到的，6 个关节在理论上就足以达到任何笛卡儿任务空间 $\mathcal{T} \subset SE(3)$ 中所期望的位姿。然而，由于关节限制和奇异位形等实际问题的存在，并非所有机器人可达空间内的位姿都可以实现。增加额外的关节是克服这个问题的一种方法。

为了说明这一点，我们将创建一个冗余机械臂。我们使用熟悉的 Puma 机器人：

---

① 当求解如第 141 页所示的一个轨迹时，其上一点的逆运动解可被用作下一个点求解的初始值。

```
>> mdl_puma560
```

把它放在一个能在 xy 平面移动的平台上，用以模仿一个安装在车辆上的机器人。这个新的机器人具有和 Puma 机器人相同的任务空间 $\mathcal{T} \subset SE(3)$，但其位形空间变为 $\mathcal{C} \subset \mathbb{R}^2 \times \mathbb{S}^6$。位形空间的维数超过任务空间的维数。

此机器人基座的 D-H 参数如下：

连杆	θ_i	d_i	a_i	α_i	σ_i
1	0	q_1	0	$-\frac{\pi}{2}$	1
2	$-\frac{\pi}{2}$	q_2	0	$\frac{\pi}{2}$	1

然后我们直接使用一种速记语法从 D-H 参数中创建 SerialLink 对象

```
>> platform = SerialLink( [0 0 0 -pi/2 1; -pi/2 0 0 pi/2 1], ...
     'base', troty(pi/2), 'name', 'platform' );
```

D-H 符号系统要求移动关节产生沿局部坐标 z 轴方向的平移，基座变换则将该 z 轴旋转为世界坐标的 x 轴方向。这种复杂性是 D-H 符号系统常见的，而另一种更好的表示方法将在 7.5.2 节中介绍。

这个机器人的关节坐标即是其 x 和 y 坐标位置。我们可以通过测试来证明我们使用的 D-H 参数是正确的。

```
>> platform.fkine([1, 2])
ans =
    1.0000    0.0000    0.0000    1.0000
   -0.0000    1.0000    0.0000    2.0000
         0   -0.0000    1.0000    0.0000
         0         0         0    1.0000
```

我们看到，旋转子矩阵为单位矩阵，这意味着该平台的坐标系与世界坐标系相平行，且 x 和 y 方向上的位移正是我们所要求的。

我们通过将两个机器人串联起来从而将 Puma 机器人安装在此平台上。在工具箱中，我们可以用两种不同的方式来表达这种串联。一种是通过乘法：

```
>> p8 = platform * p560;
```

另一种是通过关联：

```
p8 = SerialLink( [platform, p560]);
```

关联方式还允许指定所创建 SerialLink 对象的其他属性。

然而，机器人的串联也存在一个小的问题。我们想要把 Puma 手臂安装在平台顶部的高大底座上。在此之前，我们通过基座变换增加了底座，但基座变换只能存在于一个运动链的开始，而现在这个变换是在八轴机器人的关节 2 和关节 3 之间。因此这里就直接通过设置 Puma 的 d_1 为底座高度来实现该底座添加：

```
>> p560.links(1).d = 30 * 0.0254;
```

现在可以将这两个机器人结合起来了：

```
>> p8 = SerialLink( [platform, p560], 'name', 'P8')
p8 =

P8 (8 axis, PPRRRRRR, stdDH)
```

```
+---+-----------+-----------+-----------+-----------+
| j |     theta |         d |         a |     alpha |
+---+-----------+-----------+-----------+-----------+
|  1|         0|        q1|         0|    -1.571|
|  2|    -1.571|        q2|         0|     1.571|
|  3|        q3|     0.762|         0|     1.571|
|  4|        q4|         0|    0.4318|         0|
|  5|        q5|      0.15|    0.0203|    -1.571|
|  6|        q6|    0.4318|         0|     1.571|
|  7|        q7|         0|         0|    -1.571|
|  8|        q8|         0|         0|         0|
+---+-----------+-----------+-----------+-----------+

grav =     0  base = 0  0  0  0   tool =  1  0  0  0
           0         0  1  0  0           0  1  0  0
        9.81         1  0  0  0           0  0  1  0
                     0  0  0  1           0  0  0  1
```

其结果即是命名为 P8 的一个新的八轴机器人。我们注意到，新的机器人继承了该平台的基座变换，且机器人底座位移以 d_3 参数出现。

我们使用通用的数值解方法可以找到这个新冗余机械臂的逆运动学解。例如，将末端执行器移动到 (0.5,1.0,0.7)，并且使工具在 xz 平面中指向下方，其笛卡儿位姿为

```
>> T = transl(0.5, 1.0, 0.7) * rpy2tr(0, 3*pi/4, 0);
```

所需的关节坐标为①

```
>> qi = p8.ikine(T)
qi =
    -0.2854    1.0528    0.1246   -0.5377   -0.6091   -0.0939
         -1.2147    0.1212
```

其中前两个元素是移动关节在基平面内的位移，后面六个是机器人手臂的关节角度。我们看到，这个解很好地利用了机器人的第二个关节，即平台沿 y 轴的移动，使机械臂的底座靠近所期望的点。我们可以绘制出机器人的该位形图：

```
>> p8.plot(qi)
```

如图 7.8 所示。另外，我们也可以显示普通的 Puma 机器人不能达到我们指定的点：

```
>> p560.ikine6s(T)
Warning: point not reachable
```

这个机器人有 8 个自由度。直观上看它应该比相应的六轴机器人更加灵巧，因为对每个笛卡儿自由度都有不止一个关节可以使之产生运动。该问题更正式的描述是可操作性，这个概念将在 8.1.4 节详细讨论。这里，我们将它作为一个标量尺度，用来反映机械臂上的工具在各个笛卡儿方向上所能移动的难易程度。六轴机器人的可操作性为

```
>> p560.maniplty(qn)
ans =
    0.0786
```

而八轴机器人的可操作性为

```
>> p8.maniplty([0 0 qn])
ans =
    1.1151
```

它远大于六轴机器人，说明八轴机器人的灵巧性增加了。

① 此解不是唯一的，还存在有限数量的关节坐标向量能得到相同的末端执行器位姿。这个问题涉及 8.2.3 节中将要讨论的零空间运动。

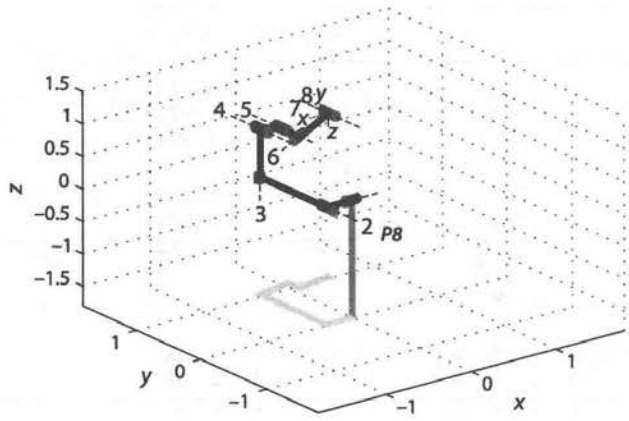


图 7.8 冗余机械臂 P8

7.4 轨迹

机器人学中最常见的要求之一，是把机器人末端执行器平滑地从位姿 A 移动至位姿 B。基于 3.1 节所学的知识，我们将讨论两种方法来产生这种轨迹：在关节空间中的直线和在笛卡儿空间中的直线。这两种直线分别被称为关节空间运动和笛卡儿空间运动。

7.4.1 关节空间运动

我们考虑末端执行器在两个笛卡儿位姿之间移动[①]：

```
>> T1 = transl(0.4, 0.2, 0) * trotx(pi);
>> T2 = transl(0.4, -0.2, 0) * trotx(pi/2);
```

它们位于 xy 平面内，且末端执行器指向朝下。与这两个位姿对应的初始和最终关节坐标向量为

```
>> q1 = p560.ikine6s(T1);
>> q2 = p560.ikine6s(T2);
```

而我们要求整个运动发生在 2 秒时间内，采样时间间隔为 50 微秒。

```
>> t = [0:0.05:2]';
```

通过在 q1 和 q2 两个位形之间的平滑插值就能得到一个关节空间轨迹。我们可以用 3.1 节中的标量插补函数 tpoly 或 lspb，再配合多轴驱动函数 mtraj 一起实现：

```
>> q = mtraj(@tpoly, q1, q2, t);
```

或者

```
>> q = mtraj(@lspb, q1, q2, t);
```

其结果是得到一个 50 × 6 阶的矩阵 q，其中每行对应一个采样时间步，每列对应一个关节值。但在这里及以后，我们将使用 jtraj 这个更方便的函数：

```
>> q = jtraj(q1, q2, t);
```

其作用相当于具有 tpoly 插值的 mtraj，但它对多轴情况进行了优化，而且还允许使用额外

① 我们选择在这些位姿上机器人的工具 z 轴朝下，这是因为机器人要下探工作在某一水平表面时只能是这种姿态。这种情况下，Puma 560 机器人工具的 z 轴要朝上在物理上是不可能的。

的参数来设置初始和最终速度。对于 mtraj 和 tpoly，最后一个参数可以是一个时间向量，如示例，或者是一个指定采样时间步数的整数。

我们可以通过可选的输出参数，获得随时间变化的关节速度和加速度向量：

```
>> [q,qd,qdd] = jtraj(q1, q2, t);
```

SerialLink 类中的 jtraj 方法为上述步骤提供了一个更简洁的表达方式：

```
>> q = p560.jtraj(T1, T2, t)
```

所得轨迹可以绘制出动画进行观察：

```
>> p560.plot(q)
```

我们也可以绘制出关节角的图形，例如随时间变化的参数 q_2：

```
>> plot(t, q(:,2))
```

或者所有随时间变化的关节角：

```
>> qplot(t, q);
```

如图 7.9(a)所示。关节坐标轨迹虽然是平滑的，但我们并不知道机器人末端执行器在笛卡儿空间中将如何移动。然而通过将正运动学作用于关节坐标轨迹，可以很容易地确定其效果

```
>> T = p560.fkine(q);
```

其结果得到一个三维的笛卡儿轨迹。该轨迹的位移部分为

```
>> p = transl(T);
```

其矩阵形式为

```
>> about(p)
p [double] : 41x3 (984 bytes)
```

其中，每一行代表一个采样时间上的末端执行器位置向量。绘制出的各坐标分量随时间变化的图形如图 7.9(b)所示。我们也可以绘制出末端执行器在 xy 平面内的轨迹：

```
>> plot(p(:,1), p(:,2))
```

如图 7.9(c)所示。很显然，这条轨迹不是一条直线。这是我们预料之中的，因为我们仅仅指定了两个端点的笛卡儿坐标。当机器人在运动过程中旋转其腰部关节时，末端执行器的运动轨迹会自然地划出一个圆弧。在实际工作中，这可能会导致机器人和附近物体之间发生碰撞，即使它们不在位姿 A 和 B 之间的路径上。我们还可以绘制出末端执行器的姿态随时间变化的图形(横滚-俯仰-偏航角形式)：

```
>> plot(t, tr2rpy(T))
```

如图 7.9(d)所示。请注意，横滚角[①]的变化范围是我们指定的 π 和 $\frac{\pi}{2}$之间。然而当俯仰角已满足其边界条件时，它就沿着路径而变化。还要注意初始横滚角显示为 $-\pi$，但在下一个采样时间步它变为 π。这是一个有限精度算法的产物，且这两个角度在圆周上是等价的。

我们也可以在 Simulink 模块中实现上述示例：

```
>> sl_jspace
```

其框图模型如图 7.10 所示。jtraj 模块的参数是关节坐标的初始值和最终值，以及运动的持续时间[②]。平滑变化的关节角被分成两路传送，一路到 plot 模块中，该模块能在一个单独的

① 相对于 x 轴旋转，见 2.2.1.2 节。

② Simulink 的积分时间需要设置为等于运动段时间，设置可以通过 Simulation 菜单或通过 MATLAB 命令 >>sim('sl_jspace', '10'); 。

窗口中显示机器人运动的动画；另一路到 fkine 模块用于计算机器人的正运动学。plot 和 fkine 两个模块有一个共同的 SerialLink 对象参数，本示例中该参数为 p560。使用 T2xyz 模块能够将末端执行器位置中的笛卡儿位置提取出来，它类似于工具箱函数 transl。XY Graph 模块可以绘制随 x 变化的 y。

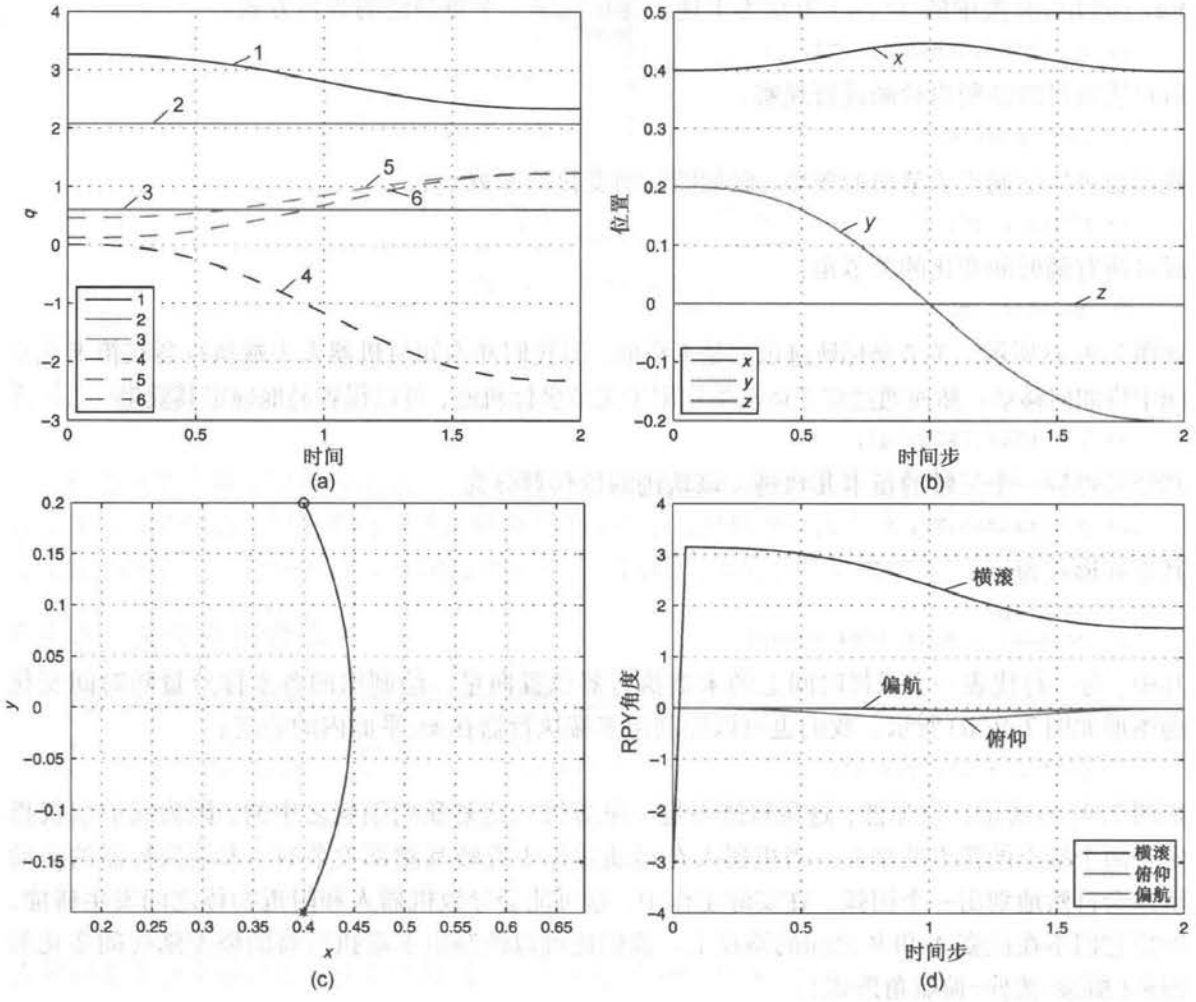


图 7.9 关节空间运动。(a)随时间变化的关节坐标；(b)随时间变化的笛卡儿位置；(c)位于xy平面的笛卡儿位置；(d)随时间变化的横滚-俯仰-偏航角

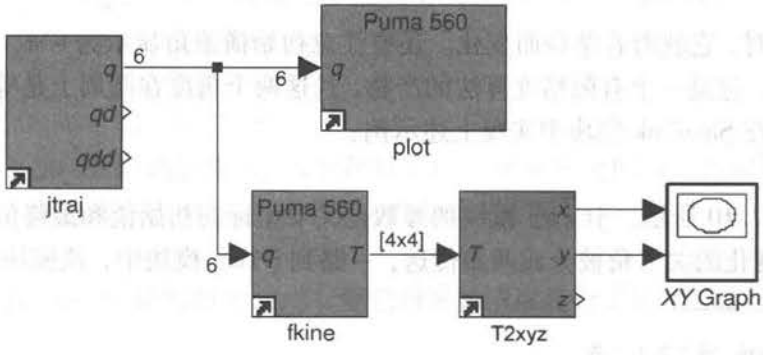


图 7.10 关节空间运动的 Simulink 模型 sl_jspace

7.4.2　笛卡儿运动

对于许多应用，我们都需要在笛卡儿空间中的直线运动，这称为笛卡儿运动。该运动可以通过使用工具箱函数 ctraj 来实现，这个函数在第 3 章中已经介绍过。它的用法与 jtraj 很相似：

```
>> Ts = ctraj(T1, T2, length(t));
```

其输入参数是初始和最后的位姿，以及时间步数，它返回的是一个三维矩阵形式的轨迹。

对于前面的关节空间运动的例子，我们将提取并绘制其位置分量：

```
>> plot(t, transl(Ts));
```

以及姿态分量：

```
>> plot(t, tr2rpy(Ts));
```

各分量的运动以及 xy 平面内末端执行器的路径如图 7.11 所示。相比图 7.9(b)和(c)，我们注意到一些重要的差异。第一，图 7.11(b)和(d)中的位置和姿态随时间线性变化。姿态的俯仰角是为零的常数且不随路径变化。其次，如图 7.11(c)所示，末端执行器在 xy 平面内沿着一条直线运动。

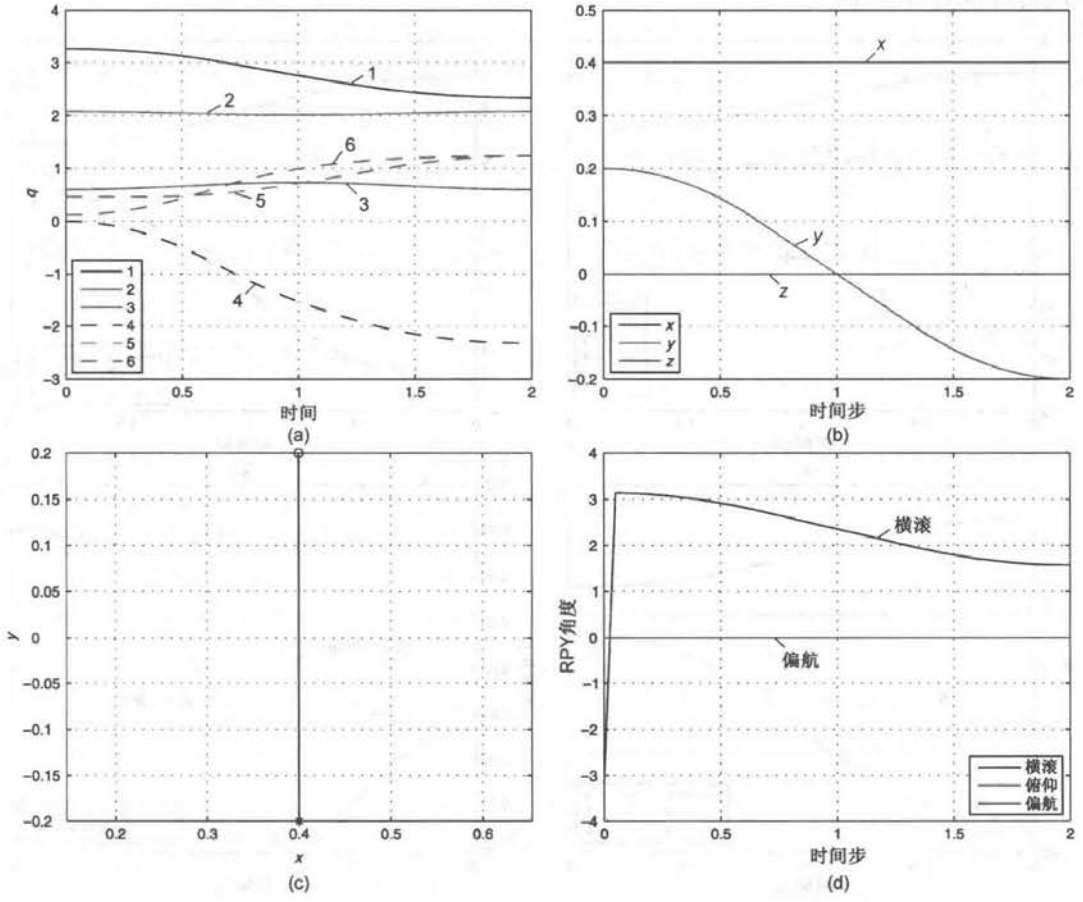


图 7.11　笛卡儿运动。(a)随时间变化的关节坐标；(b)随时间变化的笛卡儿位置；(c) xy 平面内的笛卡儿位置；(d)随时间变化的横滚-俯仰-偏航角

通过逆运动学求解，我们可以得到相应的关节空间轨迹：

```
>> qc = p560.ikine6s(Ts);
```

该轨迹如图 7.11(a)所示，虽然其与图 7.9(a)大致相同，但也存在细微差别，正是这些差别才导致直线笛卡儿运动。

7.4.3 通过奇异位形的运动

前面已经简要提到过奇异位形的概念，在下一章我们还会再次重温它。在接下来的示例中，我们特意选择一个通过机器人奇异位形的运动轨迹。我们将先前示例中的笛卡儿端点变为

```
>> T1 = transl(0.5, 0.3, 0.44) * troty(pi/2);
>> T2 = transl(0.5, -0.3, 0.44) * troty(pi/2);
```

其结果是得到一个沿着 y 方向的运动，且末端执行器的 z 轴指向 x 方向。其笛卡儿路径是

```
>> Ts = ctraj(T1, T2, length(t));
```

我们将其转换到关节坐标：

```
>> qc = p560.ikine6s(Ts)
```

各关节轨迹如图 7.12(a)所示。在 $t \approx 0.7$ s 时，我们观察到手腕关节角度 q_4 和 q_6 的变化率非常高①。其原因是 q_5 几乎已经为零，这意味着 q_4 和 q_6 的旋转轴几乎对齐重合了——另一种万向节锁情况或奇异位形。

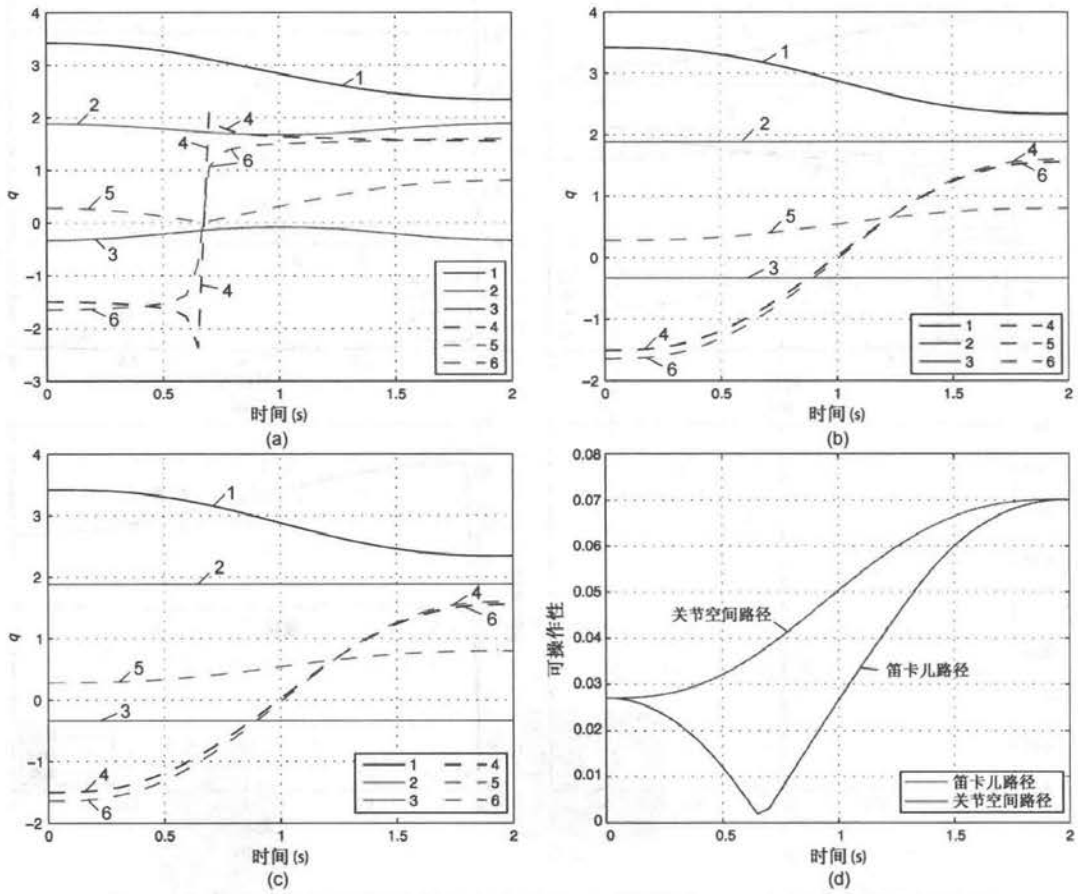


图 7.12 通过一个腕关节奇异位形的笛卡儿运动。(a)由逆运动学(ikine6s)计算的关节坐标；(b)由广义逆运动学(ikine)计算的关节坐标；(c)关节空间运动的关节坐标；(d)可操作性②

① q_6 上升得非常快，同时 q_4 下降得非常快，然后突然从 $-\pi$ 跳转到 π。

② (b)和(c)错。作者勘误：应该用新的 ikunc()方法代替原代码中的 ikine()。代码为：>>qik2 =p560.ikunc (Ts)；>>qplot(qik2);——译者注

关节轴重合意味着该机器人已经失去了一个自由度，实际上现在是一个五轴机器人。从运动学上我们只能求解出 $q_4 + q_6$ 的和值，而针对同一个和值可以有无数的 q_4 和 q_6 解。但如果是采用两个位姿之间的关节空间运动，如图 7.12(b)所示，则可以避免上述问题，因为它不需要求解逆运动学。然而，关节空间运动不会在整个路径中始终维持工具在 x 方向上的姿态，除了在两个端点。从图 7.12(c)中我们观察到，广义逆运动学的方法 ikine 能很容易地处理奇异性问题。

该路径的可操作性为

```
>> m = p560.maniplty(qc);
```

其绘制如图 7.12(d)。它显示在腕关节快速运动处周围的可操作性几乎是零。可操作性和广义逆运动学函数都是基于机械臂的雅可比矩阵，将在下一章中探讨。

7.4.4　位形转换

前面我们讨论了机械臂的运动位形，并给出了其左手或右手方式，以及肘部在上或在下的方式。考虑如下的机器人作业问题，它先用左手方式在一个工作站作业一段时间后，然后又以右手方式在另一工作站作业。机器人从一个位形转变到另一个位形，而末端执行器的位姿毫无变化，因为这两个位形具有相同的运动学解。因此，我们不能在笛卡儿空间创建一个轨迹来实现这一转变，我们只能使用关节空间运动。

要移动机器人手臂从左手位形转变到右手位形，我们首先定义某个末端执行器的位姿：

```
>> T = transl(0.4, 0.2, 0) * trotx(pi);
```

然后分别确定右手-肘在上位形和左手-肘在上位形对应的关节坐标：

```
>> qr = p560.ikine6s(T, 'ru');
>> ql = p560.ikine6s(T, 'lu');
```

最后创建这两个关节坐标之间的一个路径：

```
>> q = jtraj(qr, ql, t);
```

虽然初始的和最终的末端执行器位姿是相同的，机器人仍然执行了一种十分显著的关节空间运动，如图 7.13 所示。但在实际应用中，你要当心机器人在运动中不会碰到其他东西。我们可以用动画来呈现机器人的运动：

```
>> p560.plot(q)
```

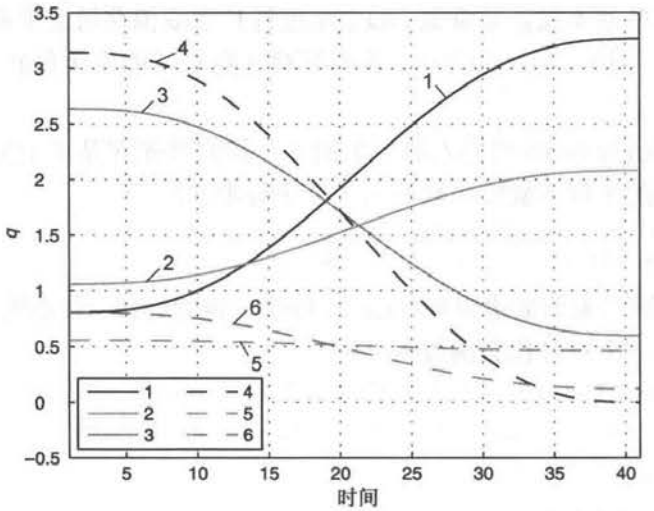


图 7.13　右手位形转变到左手位形的关节空间运动

7.5 高级问题

7.5.1 关节角偏移

机器人的零关节角位姿往往是某个很不寻常的(甚至是机械无法实现的)位姿。对于 Puma 机器人，其零角度位姿就呈现出一种不是很明显的 L 形状，机器人的上臂水平，下臂垂直向上，如图 7.5(a)所示。这一结果是由机器人的 D-H 参数形式所决定的。机器人的控制设计人员可能会选择零关节角位姿为一些更为明显的位形，如图 7.5(b)或(c)所示。

关节坐标偏移提供了一种机制，它使我们可以设置任意位形作为零关节坐标。偏移向量 q_0 要在任何运动学或动力学函数被调用之前被加到用户指定的关节角度上[①]。例如：

$$\xi = \mathcal{K}(q + q_0) \tag{7.6}$$

类似地，它要在执行一次运算操作后被减去，如逆运动学：

$$q = \mathcal{K}^{-1}(\xi) - q_0 \tag{7.7}$$

偏移量设置是通过设定 Link 对象的 offset 属性来完成的。例如：

```
>> L = Link([0 0 1 0]);
>> L.offset = pi/4
```

或者

```
>> p560.links(2).offset = pi/2;
```

7.5.2 确定 D-H 参数

确定 D-H 参数的经典方法是系统地为每个连杆分配一个坐标系。Puma 机器人的连杆坐标系就是采用标准 D-H 形式进行设定的，如图 7.14(a)所示。然而，设定每一个坐标系上都有很强的约束，因为关节旋转必须围绕 z 轴且连杆位移必须沿 x 方向。这也为基座和末端执行器坐标系的设定施加了约束，进而最终决定了前面讨论过的零角度位姿。因此为一个全新的机械臂确定 D-H 参数和连杆坐标系要比想象的更加困难——即便是经验丰富的机器人专家。

工具箱还支持另一种方法，它简单地将机械臂描述为一系列从末端执行器基座到顶端的基本平移和旋转。有些基本操作是常数，如表示连杆长度或偏移量的平移，而一些操作是广义关节坐标 q_i 的函数。不同于传统的方法，我们不对这些有可能发生旋转和平移的轴施加任何约束。

如图 7.14(b)所示为 Puma 机器人的一个例子。我们首先在基座上随便定义一个坐标系，然后把从基座到顶端的平移和旋转序列写入一个字符串：

```
>> s = 'Tz(L1) Rz(q1) Ry(q2) Ty(L2) Tz(L3) Ry(q3) Tx(L6) Ty(L4)
        Tz(L5) Rz(q4) Ry(q5) Rz(q6)'
```

请注意，我们已经将第二关节描述为 Ry(q2)，即绕 y 轴的旋转，它不能使用 D-H 形式。

该字符串被输入一个符号代数函数[②]：

```
>> dh = DHFactor(s);
```

① 实际上它在 Link 对象内实现。

② 该函数是用 Java 语言和部分机器人工具箱编写的，没有用到 MATLAB 的符号工具箱。

它返回一个 DHFactor 对象①，其中保存了已被分解为 D-H 参数的机器人运动学结构。我们可以将该结构以一种可读形式显示出来：

```
>> dh.display
DH(q1, L1, 0, -90).DH(q2+90, 0, -L3, 0).DH(q3-90, L2+L4, L6, 90).
DH(q4, L5, 0, -90).DH(q5, 0, 0, 90).DH(q6, 0, 0, 0)
```

其显示的各关节 D-H 参数顺序分别为 θ, d, a 和 α。关节角偏移量(一个加到或减去的常量，相对于从关节角变量，如 q_2 和 q_3)是自动生成的，基座和工具变换也是自动生成的。这个对象还可以生成一个字符串，它是一个工具箱命令，用于创建一个名为“puma”的机器人：

```
>> cmd = dh.command('puma')
cmd =
SerialLink([-pi/2, 0, 0, L1; 0, -L3, 0, 0; pi/2, L6, 0, L2+L4;
      -pi/2, 0, 0, L5; pi/2, 0, 0, 0; 0, 0, 0, 0; ], ...
 'name', 'puma', ...
 'base', eye(4,4), 'tool', eye(4,4), ...
 'offset', [0 pi/2 -pi/2 0 0 0 ])
```

这个命令可以被执行：

```
>> robot = eval(cmd)
```

它创建一个名为 robot 的工作空间变量，该变量是一个 SerialLink 对象②。

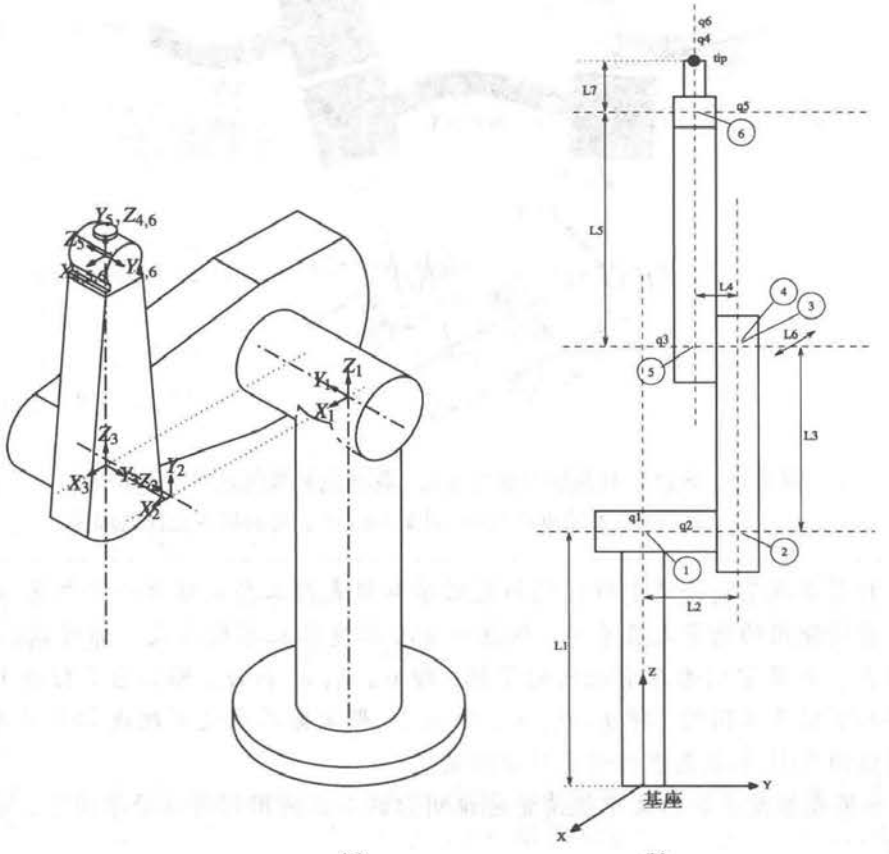


图 7.14　Puma 560 机器人坐标系。(a)Puma 机器人在零角度位姿下的标准 D-H 连杆坐标系[Corke 1996b]；(b)另一种坐标方法，展示了从基座到顶端的基本变换顺序。旋转绕着虚线所示的轴进行[Corke 2007]

① 实际上是一个 Java 对象。

② 长度参数 L1 至 L6 必须首先被定义在工作空间中。

7.5.3　改进 D-H 参数

本节介绍的 D-H 参数法是许多机器人教材中的经典内容。1986 年，克雷格[Craig 1986]首先提出了一种改进的 D-H 参数，如图 7.15 所示，其中每个连杆坐标系被固接到该连杆的近端(靠近前一个连杆)，而不是其远端。经过这种修改，使得参数符号在某些方面显得更加清晰和简洁，因而目前这种 D-H 参数法也更为常用。然而它的引入也增加了更多的混乱，特别是对那些机器人运动学的初学者。该问题的根源在于运动学、雅可比矩阵和动力学的算法都是依赖于其使用的运动学约定。根据克雷格的约定，连杆变换矩阵是

$$^{j-1}\boldsymbol{A}_j = \boldsymbol{R}_x(\alpha_{j-1})\boldsymbol{T}_x(a_{j-1})\boldsymbol{R}_z(\theta_j)\boldsymbol{T}_z(d_j) \tag{7.8}$$

克雷格将其表示为 $^{j-1}_{j}\boldsymbol{A}$。它与方程(7.1)具有相同的因子项，但是顺序不同——记住旋转顺序是不可交换的——而这正是问题的核心。a_j 始终表示连杆 j 的长度，但在一种约定中它是坐标系 $\{j\}$ 和 $\{j+1\}$ 之间原点的位移，而在另一种约定里它又变成坐标系 $\{j-1\}$ 和 $\{j\}$ 之间的原点位移。

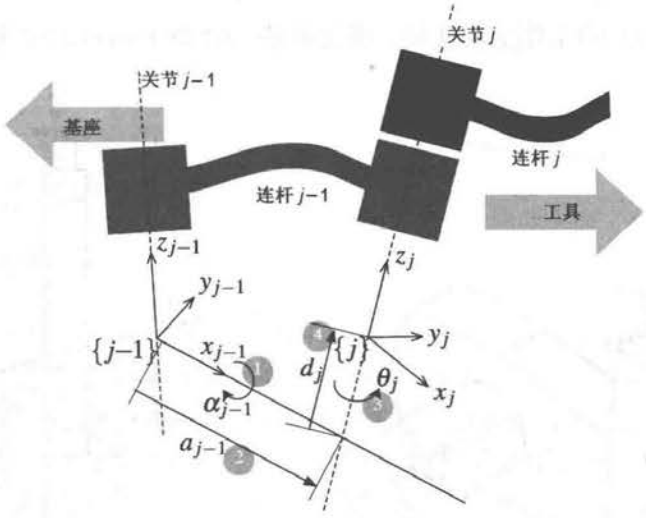


图 7.15　改进 D-H 连杆参数的定义。深灰色和黑色分别表示与连杆 j-1和j相关的所有参数。圆圈中的数字表示基本变换的顺序

如果你打算根据某篇论文中所提供的运动学参数表用工具箱建立一个机器人模型，弄清楚该表格作者所使用的约定尤其重要，但这一重要信息往往不被提及。你可以从表格的栏标题上寻找线索。如果它们都具有相同的下标，即 θ_j，d_j，a_j 和 α_j，那么它是标准 D-H 参数法。如果有一半的下标是不同的，即 θ_j，d_j，a_{j-1} 和 α_{j-1}，那么你正在使用改进 D-H 参数法。总之，你必须知道你的 D-H 参数属于何种运动学约定。

当然，如果在你发表的论文中能清楚地说明你的参数使用何种运动学约定，那对读者肯定会有所帮助。

工具箱对于两种参数形式都可以处理。对于改进 D-H 参数法，只需要在创建连杆对象时增加一个可选参数来指定：

```
>> L1 = link([0 1 0 0 0], 'modified')
L1 =
         q1            0            1            0 (modDH)
```

而标准形式则不需要：

```
>> L1 = link([0 1 0 0 0])
L1 =
         q1            0            1            0 (stdDH)
```

从这以后，其他所有操作，包括创建机器人对象，以及运动学和动力学函数等，都与以前描述的做法相同了。

如方程(7.1)和方程(7.8)所示，两种形式中连杆变换都是由一连串的基本旋转和平移变换组成的，因此它们是可以互换的。先给出一个标准 D-H 参数法的变换链：

$$\underbrace{\boldsymbol{T}_{Rz}(\theta_1)\boldsymbol{T}_z(d_1)\boldsymbol{T}_x(a_1)\boldsymbol{T}_{Rx}(\alpha_1)}_{\mathrm{DH}_1}\cdot\underbrace{\boldsymbol{T}_{Rz}(\theta_2)\boldsymbol{T}_z(d_2)\boldsymbol{T}_x(a_2)\boldsymbol{T}_{Rx}(\alpha_2)}_{\mathrm{DH}_2}\cdots$$

我们可以将其重新组合为

$$\underbrace{\boldsymbol{T}_{Rz}(\theta_1)\boldsymbol{T}_z(d_1)}_{\mathrm{base}}\cdot\underbrace{\boldsymbol{T}_x(a_1)\boldsymbol{T}_{Rx}(\alpha_1)\boldsymbol{T}_{Rz}(\theta_2)\boldsymbol{T}_z(d_2)}_{\mathrm{MDH}_1}\cdot\underbrace{\boldsymbol{T}_x(a_2)\boldsymbol{T}_{Rx}(\alpha_2)\cdots}_{\mathrm{MDH}_2}$$

其中标记为 MDH_j 的一项具有与方程(7.8)相同的形式，因为关于同一轴的平移和旋转是可以交换顺序的，即 $\boldsymbol{T}_{Rk}(\theta)\boldsymbol{T}_k(d) = \boldsymbol{T}_k(d)\boldsymbol{T}_{Rk}(\theta)$，其中 $k \in \{x,y,z\}$。

7.6　应用：绘图

我们的目标是创建一个让机器人绘制字母“E”的轨迹。首先，我们定义一些能体现该字母笔画的途经点：

```
>> path = [ 1 0 1; 1 0 0; 0 0 0; 0 2 0; 1 2 0;
     1 2 1; 0 1 1; 0 1 0; 1 1 0; 1 1 1];
```

它们被定义在 xy 平面上，无特定单位。绘图笔的放下和抬起分别对应 $z = 0$ 和 $z > 0$。分段路径可以绘制如下：

```
>> plot3(path(:,1), path(:,2), path(:,3), 'color', 'k', 'LineWidth', 2)
```

如图 7.16 所示。我们将其转换为一条连续路径：

```
>> p = mstraj(path, [0.5 0.5 0.3], [], [2 2 2], 0.02, 0.2);
```

其中第二个参数是分别在 x、y 和 z 方向上的最大速度，第四个参数中分别是初始坐标、采样间隔和加速时间。插值路径的步数为

```
>> about(p)
p [double] : 1563x3 (37512 bytes)
```

在 20 ms 的采样间隔下，共花费 31.26 s 的时间。

```
>> numrows(p) * 0.02
ans =
   31.2600
```

p 是一个 x-y-z 坐标的序列，我们必须把它转换成笛卡儿位姿。我们已定义路径绘制出的字母高为两个单位，宽为一个单位，因此需要将坐标值都缩小为原来的 1/10(使得实际字母高 200 毫米，宽 100 毫米)：

```
>> Tp = transl(0.1 * p);
```

Tp 是一系列的齐次变换矩阵，代表了路径上每个点的位姿。字母的起点被设置在工作空间中的 (0.4,0,0)：

```
>> Tp = homtrans( transl(0.4, 0, 0), Tp);
```

以上指令将 Tp 中的每一个位姿都前乘第一个参数。

最后，我们需要考虑姿态。假定在 Tp 中定义的每个坐标系其 z 轴都是垂直向上的。然而 Puma 机器人工作在水平表面上，且其肘部在上，就像一个人在桌子上书写一样。因此其工具的方向，即其 z 轴，必须垂直向下。要解决这种不匹配，一种方法是通过设置机器人的工具转换使工具轴指向上方：

```
>> p560.tool = trotx(pi);
```

下面我们就可以应用逆运动学来确定关节坐标了：

```
>> q = p560.ikine6s(Tp);
```

然后绘制出它的动画：

```
>> p560.plot(q)
```

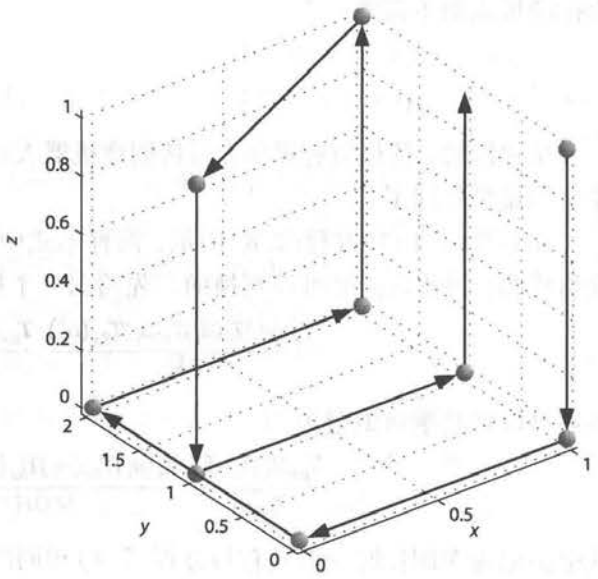


图 7.16 以 10 点路径绘制的字母“E”。标记点表示途经点而实线表示运动路段

我们可以看到 Puma 机器人正在书写字母“E”，并在笔画中间不断地抬起笔！这种方法具有通用性，我们可以很容易地改变字母的大小，把它写在任意一个平面上，或者使用具有不同运动学的机器人。

7.7 应用：一个简单的步行机器人

四条腿好，两条腿差！

雪球猪，《动物庄园》(*Animal Farm*)——乔治·奥威尔

我们的目标是创建一个四足步行机器人。首先，我们创建一个三轴机器臂，将其作为步行机器人的一条腿，并为其规划一个适合步行的轨迹，然后将四条腿组合以完成最后的步行机器人。

7.7.1 运动学

从运动学上看机器人的腿更像是一个机械臂。而对于这里的应用，三关节的串联机械臂已经足够用了。然而即使对这样一个简单的机器人，要确定其 D-H 参数也是一个复杂的过程，而零角度偏移量需要另一个单独的步骤来确定。因此，我们将使用 7.5.2 节中介绍的步骤程序。

首先要确定我们的坐标系。如图 7.17 所示是机器人腿在零角度位姿时的坐标系设置。我们选择使用航空坐标体系，其 x 轴指向前，z 轴指向下，因而 y 轴就指向右侧。腿的第一个关节将实现髋关节的前后摆动，它绕 z 轴旋转，或 $\boldsymbol{R}_z(q_1)$。第二关节是实现髋关节的上下摆动，它绕 x 轴旋转，或 $\boldsymbol{R}_x(q_2)$。因为上述两个旋转轴相交，它们就形成了一个球髋关节。膝盖位于沿着 y 轴方向平移 L_1 的位置，或 $\boldsymbol{T}_y(L_1)$。第三个关节实现靠近和远离机体的膝关节运动，即 $\boldsymbol{R}_x(q_3)$。脚尖位于沿着 z 轴方向平移 L_2 的位置，或 $\boldsymbol{T}_z(L_2)$。因此该机器人从髋部到脚尖的变换顺序为 $\boldsymbol{R}_z(q_1)\boldsymbol{R}_x(q_2)\boldsymbol{T}_y(L_1)\boldsymbol{R}_x(q_3)\boldsymbol{T}_z(L_2)$。

我们使用 7.5.2 节中的方法将该顺序写为下面的字符串：

```
>> s = 'Rz(q1).Rx(q2).Ty(L1).Rx(q3).Tz(L2)';
```

注意，长度常数必须以字母 L 开始。这个字符串被自动转化为 D-H 因子：

```
>> dh = DHFactor(s)
DH(q1+90, 0, 0, +90).DH(q2, L1, 0, 0).
DH(q3-90, L2, 0, 0).Rz(+90).Rx(-90).Rz(-90)
```

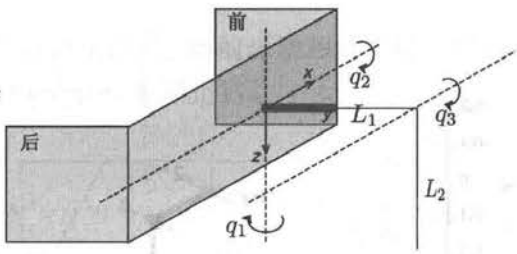


图 7.17　简单腿的坐标系及其轴旋转。该腿处于零角度位姿

这个因子序列中，最后三项是一个工具变换：

```
 >> dh.tool
ans =
trotz(pi/2)*trotx(-pi/2)*trotz(-pi/2)
```

它改变了脚部坐标系的方向。然而，在这个问题中脚与地面只是一个简单的点接触，所以我们并不关注它的方向。方法 dh.command 生成一个工具箱命令字符串，用以创建一个 SerialLink 对象：

```
>> dh.command('leg')
ans =
SerialLink([0, 0, 0, pi/2; 0, 0, L1, 0; 0, 0, -L1, 0; ],
  'name', 'leg', 'base', eye(4,4),
  'tool', trotz(pi/2)*trotx(-pi/2)*trotz(-pi/2),
  'offset', [pi/2 0 -pi/2 ])
```

在 MATLAB 工作区里将腿的各个部分长度都设置为 100 毫米之后，将 SerialLink 对象输入 MATLAB 的 eval 命令中：

```
>> L1 = 0.1; L2 = 0.1;
>> leg = eval( dh.command('leg') )
>> leg
leg =
leg (3 axis, RRR, stdDH)
+---+-----------+-----------+-----------+-----------+
| j |     theta |         d |         a |     alpha |
+---+-----------+-----------+-----------+-----------+
|  1|        q1|          0|          0|      1.571|
|  2|        q2|          0|        0.1|          0|
|  3|        q3|          0|       -0.1|          0|
+---+-----------+-----------+-----------+-----------+

grav =     0  base = 1  0  0  0   tool =  0  0 -1  0
           0         0  1  0  0           0  1  0  0
        9.81         0  0  1  0           1  0  0  0
                     0  0  0  1           0  0  0  1
```

通常，我们会对机器人进行了一个快速的完整性检查。在零关节角度时，脚尖处于我们所设计的位置上：

```
>> transl( leg.fkine([0,0,0]) )'
ans =
         0    0.1000    0.1000
```

我们可以绘制出机器人的零角度位姿：

```
>> leg.plot([0,0,0], 'nobase', 'noshadow')
>> set(gca, 'Zdir', 'reverse'); view(137,48);
```

如图 7.18 所示。请注意，我们告诉了 MATLAB 机器人的 z 轴要指向下方。现在，我们应该测试其他关节是否能产生期望的运动。增加 q_1：

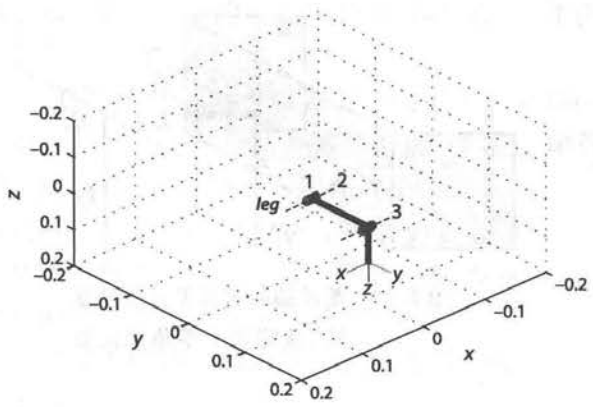


图 7.18　处于零角度位姿的机器人腿

```
>> transl( leg.fkine([0.2,0,0]) )'
ans =
   -0.0199    0.0980    0.1000
```

它会产生在 xy 平面内的运动。再增加 q_2：

```
>> transl( leg.fkine([0,0.2,0]) )'
ans =
   -0.0000    0.0781    0.1179
```

它产生在 yz 平面内的运动。增加 q_3 的结果与 q_2 相同：

```
>> transl( leg.fkine([0,0,0.2]) )'
ans =
   -0.0000    0.0801    0.0980
```

至此，我们已经创建并验证了一个简单的机器人腿。

7.7.2　单腿运动

下一步是定义的腿的末端执行器(即它的脚)将运行的路径。首先考虑的是：所有的脚都在地面上以相同的速度向后移动，其作用是在脚实际上没有相对地面滑动的情况下推动机器人身体向前运动。由于每条腿的运动范围有限，因此它不能后退很长的距离。在某个点我们必须复位机器人腿——抬起脚，向前移动，再放在地面上。第二个考虑是静态稳定性——任何时候机器人必须至少有三只脚同时站在地面上，所以每条腿必须按顺序依次复位。这就要求在一个周期内每条腿有 3/4 的时间与地面接触，而另外 1/4 的时间在复位。其结果是，腿在复位期间必须加快移动，因为它要用相对少的时间来完成一段更长的路径。

所需的轨迹是由所有途经点所决定的：

```
>> xf = 50; xb = -xf;  y = 50; zu = 20; zd = 50;
>> path = [xf y zd; xb y zd; xb y zu; xf y zu; xf y zd] * 1e-3;
```

其中，xf 和 xb 分别是 x 方向上腿部前进和后退运动的极限(以毫米为单位)，y 是 y 方向上脚到机体的距离，而 zu 和 zd 分别是在 z 方向上脚抬起和放下时脚的高度。在这个示例中，脚是从髋部前面 50 毫米运动至后面 50 毫米。当脚部放下时，它位于髋部下方 50 毫米，而在复位期间脚提升到髋部下方 20 毫米处。在 path 中的点包含一个完整的周期，对应于站立的开始阶段、站立的结束阶段、脚尖抬起阶段、脚尖返回阶段、最后回到站立的开始，如图 7.19(a)所示。

下面以 100 Hz 的频率对这个多段路径进行采样：

```
>> p = mstraj(path, [], [0, 3, 0.25, 0.5, 0.25]', path(1,:), 0.01, 0);
```

此例中，我们已经指定了一个包含各段运动时间的向量，而不是最大的关节速度[①]。最后 3 个参数分别是初始位置、采样间隔和加速时间。这个轨迹总的运行时间为 4 秒，因此它包含了 400 个点。

我们用逆运动学来确定脚尖路径所需的关节角度轨迹。这个机器人是欠驱动的，所以我们使用广义逆运动学 ikine，并通过设置遮盖向量以只求解末端执行器的平移：

```
>> qcycle = leg.ikine( transl(p), [], [1 1 1 0 0 0] );
```

我们可以通过动画来观察腿部的运动：

① 这样我们就可以确保复位占了一个周期的 1/4。

```
>> leg.plot(qcycle, 'loop')
```

以验证它是否如我们所愿：先沿着地面缓慢运动，然后快速提升、向前运动和放下脚。'loop'选项将使轨迹的显示一直循环不断，而你需要键入 Control-C 来停止它。

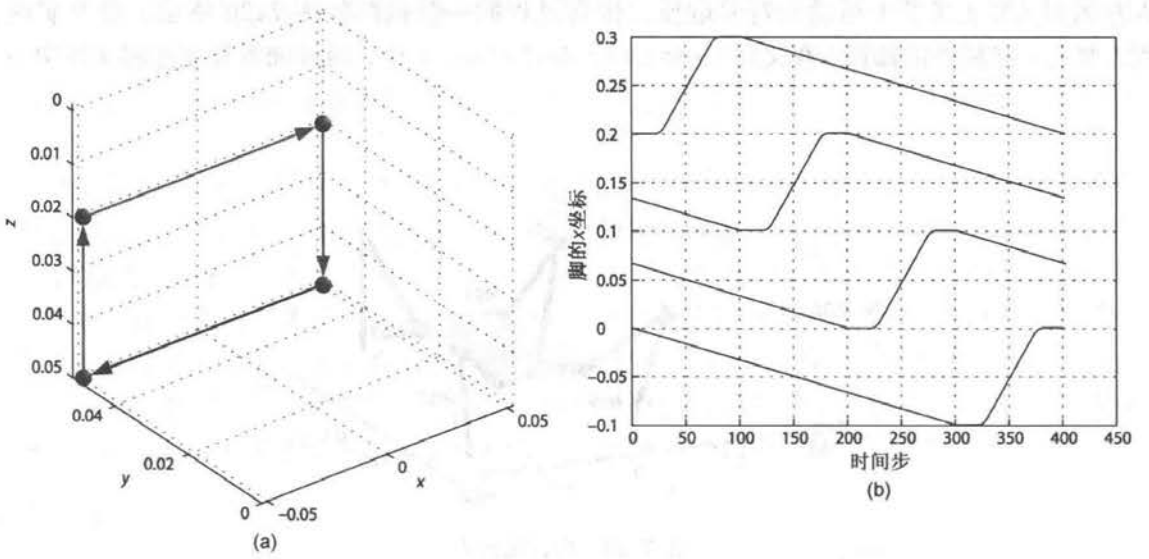


图 7.19　(a)单腿采取的轨迹。回想图 7.17 中其 z 轴是向下的。灰色线段表示腿在复位；(b)每条腿 x 方向运动所显示的步态。x 方向速度更快的时间段就是腿复位时间

7.7.3　四腿运动

我们的机器人宽度和长度为

```
>> W = 0.1; L = 0.2;
```

通过复制先前生成的 leg 对象，就可以创建多条腿的机器人。但其中我们要提供不同的基座转换，以将腿连接到机体上不同的点：

```
>> legs(1) = SerialLink(leg, 'name', 'leg1');
>> legs(2) = SerialLink(leg, 'name', 'leg2', 'base', transl(-L, 0, 0));
>> legs(3) = SerialLink(leg, 'name', 'leg3', 'base', transl(-L, -W, 0)*trotz(pi));
>> legs(4) = SerialLink(leg, 'name', 'leg4', 'base', transl(0, -W, 0)*trotz(pi));
```

得到的结果是一个包含多个 SerialLink 对象的向量。请注意，机体左侧的第 3 条和第 4 条腿已经绕着 z 轴进行了旋转，使得它们都朝向身体外侧。

如前所述，每条腿必须按照各自的顺序进行复位。由于腿的轨迹是一个周期运动，我们可以通过使每个腿的运动轨迹有一个相位偏移来实现按顺序复位，该相移时间为总循环周期的 1/4。由于总周期有 400 点，因此每条腿的相移为 100 个点。然后我们使用模运算来模拟每条腿的循环步态，其结果如图 7.19(b)所示。

行走程序的核心是

```
k = 1;
while 1
    q = qleg(p,:);
    legs(1).plot( gait(qcycle, k, 0,   0) );
    legs(2).plot( gait(qcycle, k, 100, 0) );
    legs(3).plot( gait(qcycle, k, 200, 1) );
    legs(4).plot( gait(qcycle, k, 300, 1) );
    drawnow
    k = k+1;
end
```

其中的函数

```
gait(q, k, ph, flip)
```

将通过模运算返回 q 的第 k + ph 个元素，而模运算将 q 看作一个周期。参数 flip 是将机器人左侧两条腿上关节 1 运动的符号取反。仿真过程的一张截图如图 7.20 所示。整个实现过程，加上一些额外的修饰，在文件 examples/walking.m 中，而详细解释在注解文件中。

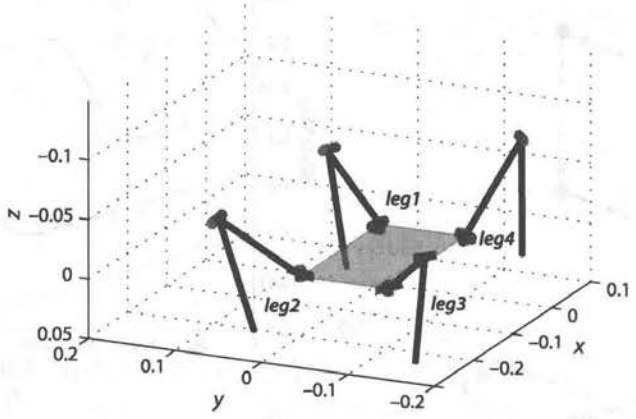


图 7.20 步行机器人

7.8 本章总结

在本章中，我们学习了如何根据每个连杆的一组 D-H 参数来描述一个串联机械臂。我们可以计算出每个连杆随其关节变量变化的相对位姿，然后将这些位姿组合成为机器人末端执行器相对于基座的位姿。对于具有 6 个关节且手腕为球关节的机器人，我们可以计算其逆运动学的解析解，它是一组对应于某个特定末端执行器位姿的关节角度。但这个逆解不是唯一的，机器人可能有多个关节位形对应于同一个末端执行器位姿。

对于不具有 6 个关节和球腕的机器人，可以使用一种迭代数值方法来求解它的逆运动学。我们展示了该方法如何用于一个欠驱动的两连杆机器人，以及一个八连杆的冗余机器人。我们还简要探讨了由于关节轴线重合引发的奇异问题。

我们还学习了如何创建一条路径，从而使末端执行器在两个位姿之间平滑移动。关节空间路径比较容易计算，但却不能得到笛卡儿空间中的直线路径，因而不适合于某些应用场合。笛卡儿空间中的直线路径也可以生成，但工作空间中的奇异位形可能会导致非常高的关节速度。

扩展阅读

串行机械臂的运动学涵盖于所有标准的机器人书籍中，如[Paul, 1981]、[Spong et al., 2006]、[Siciliano et al., 2008]等书。克雷格的书[Craig, 2004]也是一本很好的介绍机器人运动学的书籍，它使用改进的 D-H 符号，其第三版中的示例还使用了老版本的机器人工具箱。大多数这些书籍都推导了一个两连杆机械臂和一个拥有球腕的六连杆机械臂的逆运动学。第一本完整描述六连杆机械臂逆运动学的书是[Paul and Zhang, 1986]。

在[Paul, 1981]一书中描述到，通过写下一些运动学关系式并求出关节角度用符号推导出封闭形式的逆运动学解。另外，市面上也有很多商业软件包可以自动生成一个给定机器人的正、逆运动学，其中包括 Robotica[Nethery and Nethery, 1994]和 SYMORO + [Khalil and Creusot, 1997]。

迪拉维特和哈滕贝格最早的成果是他们于 1955 发表的论文[Denavit and Hartenberg, 1955]和他们出版的教材[Hartenberg and Denavit, 1964]。教材中介绍了运动学及其发展历史，但目前已绝版，只有在网上可以找到一个版本。在[Siciliano et al., 2008]一书中，非常清楚地描述了为任意一个机器人设置 D-H 参数的过程。另一种基于符号因子化的方法则在[Corke, 2007]一书中进行了详细介绍。这种方法与帕克[Park, 1994]所提出的指数积(POE)形式有一些相似之处。Puma 560 机器人的参数的最终值发表于论文[Corke and Armstrong-Hélouvry, 1995]中。

机器人步行本身就是一个巨大的研究领域，而这里给出的示例是非常简单的。很多具有复杂步态的机器已经被研制出来，包括跑步和飞奔，但这些步态更多是依赖于动力学而不是静态平衡。在《机器人技术手册》[Siciliano and Khatib, 2008, §16]中很好地介绍了腿式步行机器人。

在第三部分开始时简要提到了并联机械臂，这种机械臂具有驱动力强、刚度大等优势(由于驱动器形成了一个类桁架结构)。对于这一类的机构，逆运动学通常是封闭形式的，而正运动学往往需要数值解。针对这一类机器人，初次接触的读者可以参考以下书籍：[Siciliano, 2008]、《机器人技术手册》[Siciliano and Khatib, 2008, §12]和[Merlet, 2006]。

plot 方法

7.2.1 节中介绍的 plot 方法显示出机器人的位姿。所绘图形的许多方面，如缩放、阴影、关节轴和标签、工具坐标系标签等，均可由用户自定义。plot 方法也支持同一个机器人的多视图显示，且图形可以包含多个机器人。本节中提供了上述一些功能的示例，当然，更详细的介绍还要参见在线文档。

hold 方法以一种类比的方式进行正常数据绘制，并允许将多个机器人绘制在同一个参考坐标系中。首先创建一个标准 Puma 机器人的复本，其基座处在不同的位置：

```
>> p560_2 = SerialLink(p560, 'name', 'puma #2', ...
 'base', transl(0.5,-0.5,0));
```

先绘制第一个机器人：

```
>> p560.plot(qr)
```

然后添加第二个机器人：

```
>> hold on
>> p560_2.plot(qr)
```

得到的结果如图 7.21(a)所示。我们可以分别让图中的机器人产生一个运动。例如：

```
>> p560.plot(qz)
```

该指令仅仅改变了第一个机器人的位姿，如图 7.21(b)所示。

我们也可以创建相同机器人或机器人群的多个视图。继续上面的示例，我们创建了一个新的图形：

```
>> figure
>> p560.plot(qz)
```

然后绘制了一个相同的机器人。在这个新图形中，可以通过使用 MATLAB 的图形工具栏选项来改变图形的视角。如：

```
>> p560.plot(qr)
```

该指令使机器人在两个图形中都发生改变。问题的关键在于机器人的名称，不论出现在哪个图形内，名称被用于识别机器人。每个机器人的名称必须是唯一的，且以这个特征工作。

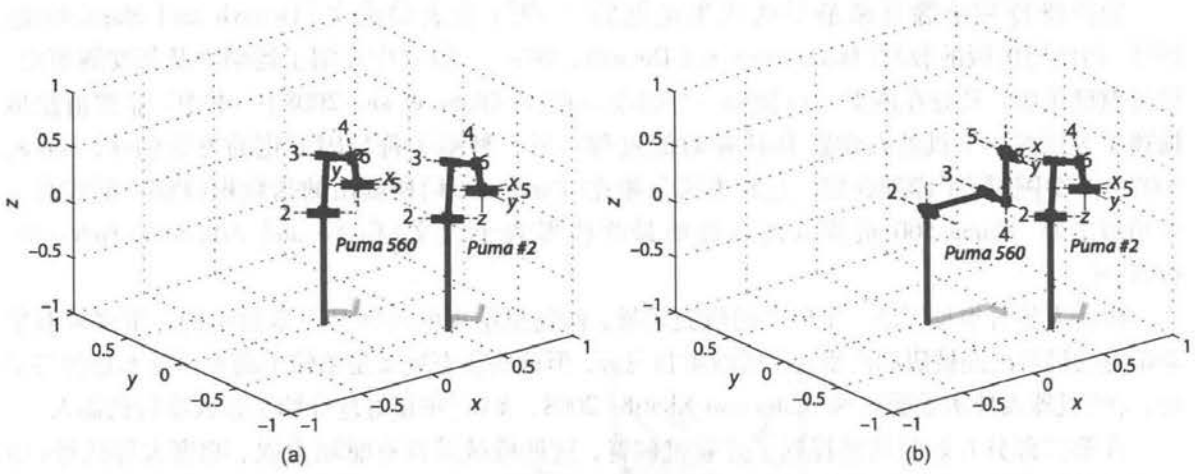


图 7.21 plot 显示每轴的多机器人。(a)两个机器人具有相同的关节坐标;(b)两个机器人具有不同的关节坐标

对于已有的图形，plot 方法所使用的算法包括：

1. 如果具有指定名称的机器人存在于当前图形中，则以指定的关节角度重绘。这时在其他图形中同一名称的机器人也会更新。
2. 如果当前图形中没有这个名称的机器人存在，这时如果 hold 方法已启动，机器人将被添加到当前图形中，否则将在当前图形窗口中创建一个指定名称的新机器人。
3. 如果没有图形，则将创建一个新的图形。

我们可以使用一个图形化教学界面来手动地驱动机器人：

```
>> p560.teach()
```

它显示了一个图形用户界面(GUI)，其中机器人的每个关节都对应一个滑动条。当用户移动某个滑动条时，机器人对应的关节也移动，而且是在所有图形中同名的机器人都动。

习题

1. 使用 teach 方法进行试验。
2. 推导两连杆机器人的正向运动学。给定 q_1 和 q_2，末端执行器的位置 (x,y) 是什么?
3. 推导两连杆机器人(7.2.1 节)的逆向运动学。给定末端执行器的位置 (x,y)，关节角度 (q_1, q_2) 是什么?
4. 当 Puma 560 机器人在不同位姿时，比较 ikine6s 和 ikine 得到的解。它们在精度上有何差异? ikine 方法慢多少?
5. 当 Puma 560 处于 qn 位形时，演示出机器人从肘部在上到肘部在下的位形变化。
6. 绘制字母“E”(7.6 节)。
   a)更改字母的大小。
   b)构造字母“C”的路径。
   c)在垂直平面上书写一个字母。
   d)在斜面上书写一个字母。
   e)将机器人从 Puma 560 变为 Fanuc 10 L。

f)在一个球体上书写字母。

g)这个书写任务其实并不需要六自由度(6DOF)，因为笔关于其自身轴的旋转并不重要。请去掉 Puma 560 机器人模型中的最后一个关节，并重复以上练习。

7. 步行机器人(7.7 节)。

a)通过减少复位期间的抬腿高度来缩短复位的轨迹。

b)增加腿的步幅。

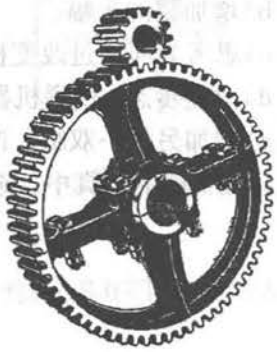
c)思考如何通过改变机器人身体一侧的步长来控制机器人转弯。

d)改变步态，使得机器人像螃蟹一样侧身移动。

e)增加另外一双腿。改变步态以同时复位两条腿或三条腿。

f)在目前的仿真中，腿移动但身体并没有向前移动。修改仿真效果使得身体移动。

第8章 速度关系



在这一章中，我们将考虑关节坐标的变化率(即关节速度)与末端执行器速度之间的关系。这个三维的末端执行器位姿 $\xi \in SE(3)$ 有一个由六维向量所表示的速度，被称为空间速度。关节速度和末端执行器速度之间的相关性将通过机械臂的雅可比矩阵体现，该矩阵为机械臂位姿的函数。

8.1 节将使用一种数值方法来引入机械臂的雅可比矩阵。接下来，我们将介绍其他的雅可比矩阵，包括可以用来变换各坐标系之间速度的雅可比矩阵，以及用来变换不同角度表示法之间角速度的雅可比矩阵。雅可比矩阵的数值属性可以帮助我们理解机械臂的灵巧性(它可以和不可以轻易移动的那些方向)，并且更深刻地了解奇异位形。在 8.2 节中将使用逆雅可比矩阵来生成笛卡儿路径，它无需求解机械臂的逆运动学，并且该方法可以运用到过驱动和欠驱动的机器人。8.3 节展示了如何使用雅可比转置矩阵来转换坐标系之间的力，以及从末端执行器到关节的力。最后，8.4 节更全面深入地介绍了上一章使用过的数值逆运动学解。

8.1 机械手的雅可比矩阵

我们首先研究关节坐标的微小变化如何影响末端执行器的位姿。使用齐次坐标变换表示位姿，我们可以通过一个一阶微分近似得到位姿相对于关节坐标的导数：

$$\frac{\mathrm{d}\boldsymbol{T}}{\mathrm{d}\boldsymbol{q}} \approx \frac{\boldsymbol{T}(\boldsymbol{q}+\delta_q)-\boldsymbol{T}(\boldsymbol{q})}{\delta_q}$$

而回顾方程(2.19)中 $\boldsymbol{T}$ 的定义，可以得到

$$\frac{\mathrm{d}\boldsymbol{T}}{\mathrm{d}\boldsymbol{q}} \approx \frac{1}{\delta_q}\left(\begin{array}{c|c} \boldsymbol{R}(\boldsymbol{q}+\delta_q)-\boldsymbol{R}(\boldsymbol{q}) & \begin{matrix}\delta_x\\ \delta_y\\ \delta_z\end{matrix} \\ \hline 000 & 0 \end{array}\right) \tag{8.1}$$

其中，$(\delta_x,\delta_y,\delta_z)$ 表示末端执行器的平移位移。

这里再次使用 Puma 560 机器人作为我们的示例：

```
>> mdl_puma560
```

在如图 8.1 所示的标准位形 qn 下，末端执行器的位姿是

```
>> T0 = p560.fkine(qn);
```

我们给关节 1 一个微小扰动量 dq：

```
>> dq = 1e-6;
```

计算得到新的末端执行器位姿：

```
>> Tp = p560.fkine(qn + [dq 0 0 0 0 0]);
```

再利用方程(8.1)得到其导数：

```
>> dTdq1 = (Tp - T0) / dq
dTdq1 =
         0   -1.0000   -0.0000    0.1500
   -0.0000   -0.0000    1.0000    0.5963
         0         0         0         0
         0         0         0         0
```

这显然不再是一个齐次变换矩阵，其左上方的3×3的矩阵不是一个正交矩阵，且右下角的元素不是1。这结果是什么意思呢?

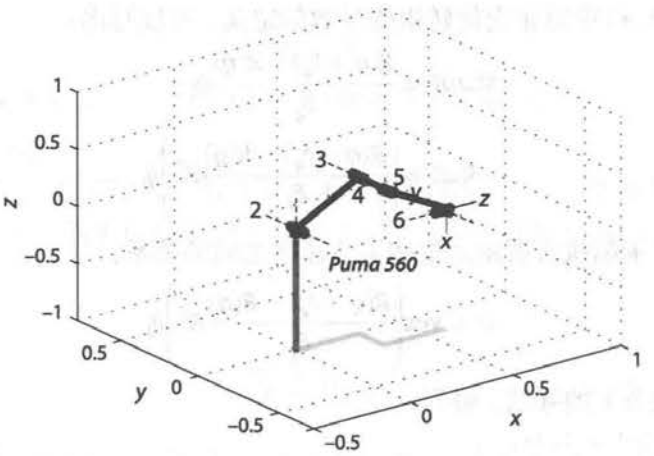


图8.1 Puma机器人处于标准位形 qn。末端执行器的 z 轴指向世界坐标系的 x 方向，而 x 轴指向下

将 dTdq1 中的第四列元素与方程(8.1)的矩阵中相应元素取等，得到

$$\begin{pmatrix}\delta_x\\ \delta_y\\ \delta_z\end{pmatrix}=\begin{pmatrix}0.1500\\ 0.5963\\ 0\end{pmatrix}\delta_{q_1}$$

它代表了 q_1 的变化引起的末端执行器位置的变化。从图8.1我们能更容易地理解——腰关节(q_1)的一个微小的旋转将使得末端执行器在水平的 xy 平面内移动，但却不产生垂直方向上的移动。

我们再对下一个关节重复上述步骤，得到

```
>> Tp = p560.fkine(qn + [0 dq 0 0 0 0]);
>> dTdq2 = (Tp - T0) / dq
dTdq =
    1.0000   -0.0000   -0.0000    0.0144
    0.0000         0    0.0000         0
    0.0000    0.0000    1.0000    0.5963
         0         0         0         0
```

同样，可以写出

$$\begin{pmatrix}\delta_x\\ \delta_y\\ \delta_z\end{pmatrix}=\begin{pmatrix}0.0144\\ 0\\ 0.5963\end{pmatrix}\delta_{q_2}$$

正如我们所预料的，肩关节 q_2 的一个微小转动会引起末端执行器在垂直的 xz 平面内的运动，而没有 y 方向运动。等式两边同除以一个无限小的时间长度 δ_t，则得到关节角速度和末端执行器速度之间的一个关系：

$$\begin{pmatrix}\dot{x}\\ \dot{y}\\ \dot{z}\end{pmatrix}=\begin{pmatrix}0.0144\\ 0\\ 0.5963\end{pmatrix}\dot{q}_1$$

现在考虑方程(8.1)的矩阵中左上方的 3×3 子矩阵。我们将其乘以 δ_q/δ_t，从而得到 $\boldsymbol{R}$ 的一阶近似导数：

$$\dot{\boldsymbol{R}}\approx\left(\frac{\boldsymbol{R}(\boldsymbol{q}+\delta_q)-\boldsymbol{R}(\boldsymbol{q})}{\delta_q}\right)\frac{\delta_q}{\delta_t}$$

回顾前面方程(3.4)中对正交旋转矩阵导数的定义，可以写出：

$$\boldsymbol{S}(\omega)\boldsymbol{R}\approx\frac{\boldsymbol{R}(\boldsymbol{q}+\delta_q)-\boldsymbol{R}(\boldsymbol{q})}{\delta_q}\dot{q}_1$$

$$\boldsymbol{S}(\omega)\approx\left(\frac{\boldsymbol{R}(\boldsymbol{q}+\delta_q)-\boldsymbol{R}(\boldsymbol{q})}{\delta_q}\boldsymbol{R}^T\right)\dot{q}_1$$

从上式可以得出一个末端执行器角速度和关节速度之间的关系：

$$\omega\approx\mathrm{vex}\left(\frac{\boldsymbol{R}(\boldsymbol{q}+\delta_q)-\boldsymbol{R}(\boldsymbol{q})}{\delta_q}\boldsymbol{R}^T\right)\dot{q}_1$$

继续利用前面算的关节 1 的导数，得到：

```
>> dRdq1 = dTdq1(1:3,1:3);
>> R = T0(1:3, 1:3);
>> S = dRdq1 * R'
S =
   -0.0000   -1.0000    0.0000
    1.0000   -0.0000    0.0000
         0         0         0
```

这是一个反对称矩阵，从中可以提取出角速度向量：

```
>> vex(S)
ans =
     0     0     1
```

最后写出：

$$\begin{pmatrix}\omega_x\\ \omega_y\\ \omega_z\end{pmatrix}=\begin{pmatrix}0\\ 0\\ 1\end{pmatrix}\dot{q}_1$$

从图 8.1 可以很容易地理解上式，末端执行器的角速度就是关节 1 绕着世界坐标系 z 轴旋转的角速度。

对第二关节的微小运动重复上述过程，得到

```
>> dRdq2 = dTdq2(1:3,1:3);
>> S = dRdq2 * inv( R )
S =
   -0.0000   -0.0000   -1.0000
    0.0000   -0.0000   -0.0000
    1.0000   -0.0000   -0.0000
>> vex(S)
    0.0000   -1.0000    0.0000
```

一个雅可比矩阵就是一个向量关于另一个向量的向量值函数导数。如果 $\boldsymbol{y}=F(\boldsymbol{x})$，且 $\boldsymbol{x}\in\mathbb{R}_n$ 和 $\boldsymbol{y}\in\mathbb{R}_m$，则雅可比矩阵是一个 $m\times n$ 阶的矩阵：

$$J=\frac{\partial \boldsymbol{F}}{\partial \boldsymbol{x}}\begin{pmatrix}\frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n}\\ \vdots & \ddots & \vdots\\ \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n}\end{pmatrix}$$

雅可比矩阵是以卡尔·雅可比(Carl Jacobi)的名字命名的，详情参见附录G。

从中可以写出：

$$\begin{pmatrix}\omega_x\\ \omega_y\\ \omega_z\end{pmatrix}=\begin{pmatrix}0\\ -1\\ 0\end{pmatrix}\dot{q}_2$$

再次参见图8.1来理解上式，关节2的转动轴线在负y轴方向上，其旋转将使机械臂得到一个负y轴方向上的角速度。

我们已经建立了各个关节的速度和机器人末端执行器的移动速度和角速度之间的数字关系。上一章方程(7.3)中，我们将机械臂的正运动学写为以下函数形式：

$$\xi=\mathcal{K}(\boldsymbol{q})$$

对其求导数得到：

$$\boldsymbol{\nu}=\boldsymbol{J}(\boldsymbol{q})\dot{\boldsymbol{q}} \tag{8.2}$$

这是一个瞬时正运动学，其中$\boldsymbol{\nu}=(v_x,v_y,v_z,\omega_x,\omega_y,\omega_z)\in\mathbb{R}^6$是一个空间速度，包含平移速度和旋转速度分量。矩阵$\boldsymbol{J}(\boldsymbol{q})\in\mathbb{R}^{6\times N}$是机械臂的雅可比矩阵或几何雅可比矩阵。

雅可比矩阵可以直接由SerialLink对象的jacob0方法计算：①

```
>> J = p560.jacob0(qn)
J =
    0.1501    0.0144    0.3197         0         0         0
    0.5963    0.0000    0.0000         0         0         0
         0    0.5963    0.2910         0         0         0
         0   -0.0000   -0.0000    0.7071   -0.0000   -0.0000
         0   -1.0000   -1.0000   -0.0000   -1.0000   -0.0000
    1.0000    0.0000    0.0000   -0.7071    0.0000   -1.0000
```

矩阵的行对应于笛卡儿自由度，而列对应于各个关节——它们是对应于各相应关节单位速度的末端执行器空间速度。前两列即为前面使用近似导数所计算出的结果。其右上方3×3阶的"0"表明手腕关节的运动对末端执行器的平移运动没有任何影响——这是机械臂拥有球腕和一个零长度工具的必然结果。

背景材料：卡尔·古斯塔夫·雅各·雅可比(Carl Gustav Jacob Jacobi，1804—1851)是普鲁士数学家。他于1825年从柏林大学获得了哲学博士学位。1827年，他被任命为斯堡大学的数学教授，并一直任教到1842年，他由于工作过度导致身体崩溃。

雅可比于1829年写了一篇关于椭圆函数的经典论文，还介绍了具有n个变量的m个函数的导数，并以他的名字命名。1836年，他当选为瑞典皇家科学院的外籍院士。他去世后被安葬在柏林三一教堂的墓地。

① 函数jacob0不使用有限差分。它有一个基于机器臂D-H参数的直接形式[Paul and Shimano 1978]。

8.1.1　坐标系之间的速度转换

假设有两个坐标系 $\{A\}$ 和 $\{B\}$，它们之间有如下的关系：

$$ {}^{A}T_{B} = \begin{pmatrix} R & t \\ 0 & 1 \end{pmatrix} $$

而坐标系 $\{A\}$ 的空间速度可以由坐标系 $\{B\}$ 的空间速度表示为

$$ {}^{B}\nu = {}^{B}J_{A}\,{}^{A}\nu \tag{8.3} $$

其中雅可比矩阵①

$$ {}^{B}J_{A} = J_{v}({}^{A}T_{B}) = \begin{pmatrix} {}^{B}R_{A} & 0_{3\times3} \\ 0_{3\times3} & {}^{B}R_{A} \end{pmatrix} \tag{8.4} $$

是一个 6×6 阶的矩阵，是关于坐标系之间的相对姿态的函数②。

若我们知道固接于一个刚体的坐标系 $\{A\}$ 的原点速度，我们要确定固接于同一刚体的坐标系 $\{B\}$ 的原点速度，雅可比矩阵变为

$$ {}^{B}J_{A} = \overline{J}_{v}({}^{A}T_{B}) = \begin{pmatrix} {}^{B}R_{A} & -{}^{B}R_{A}S({}^{A}t_{B}) \\ 0_{3\times3} & {}^{B}R_{A} \end{pmatrix} \tag{8.5} $$

这些雅可比矩阵都有一个有趣的结构。旋转矩阵都出现了两次，分别将平移速度和角速度从坐标系 $\{A\}$ 旋转到坐标系 $\{B\}$。左下角的 3×3 阶零矩阵表示坐标系 $\{B\}$ 的旋转速度只依赖于坐标系 $\{A\}$ 的旋转速度。如果两个坐标系被固接于同一刚体但相隔一定的距离，那么右上角的 3×3 阶矩阵就是非零的，它意味着坐标系 $\{B\}$ 的平移速度与坐标系 A 的平移速度和旋转速度均相关——即“杆臂效应”。

例如，如果两个坐标系均固接于同一个刚体，第二个坐标系和第一个坐标系之间具有如下变换关系：

```
>> T = transl(1, 0, 0)*troty(pi/2);
```

则雅可比矩阵为

```
>> J = tr2jac(T)
J =
    0.0000         0   -1.0000         0    1.0000         0
         0    1.0000         0         0         0    1.0000
    1.0000         0    0.0000         0   -0.0000         0
         0         0         0    0.0000         0   -1.0000
         0         0         0         0    1.0000         0
         0         0         0    1.0000         0    0.0000
```

则雅可比矩阵可以由以下步骤得出：

坐标系 $\{A\}$ 在 x 方向上的单位速度被变换为坐标系 $\{B\}$ 中的速度：

```
>> vB = J*[1 0 0 0 0 0]';
>> vB'
ans =
    0.0000         0    1.0000         0         0         0
```

---

① 正如第 15 页中讨论的那样，方程(8.3)中间的两个“A”字母，可以删除不要。

② 记作 ${}^{B}R_{A} = {}^{A}R_{B}^{T}$。

8.1.2 末端执行器坐标系的雅可比矩阵

通过 jacob0 方法计算的雅可比矩阵将关节速度映射到世界坐标系中的末端执行器空间速度——因此 jacob0 方法用"0"作为后缀。为了获得末端执行器自身坐标系中的空间速度，我们使用 jacobn 方法来替代：

```
>> p560.jacobn(qn)
ans =
   -0.0000   -0.5963   -0.2910         0         0         0
    0.5963    0.0000    0.0000         0         0         0
    0.1500    0.0144    0.3197         0         0         0
   -1.0000         0         0    0.7071         0         0
   -0.0000   -1.0000   -1.0000   -0.0000   -1.0000         0
   -0.0000    0.0000    0.0000    0.7071    0.0000    1.0000
```

从两种雅可比矩阵方法的代码可以看出，前面讨论的 jacob0 实际上是在 jacobn 的基础上附加了一个从末端执行器坐标系到世界坐标系的速度变换，该速度变换是基于 $\boldsymbol{T}_6$ 的逆矩阵。回到方程(8.3)，我们得到：

$$\begin{aligned}{}^0\boldsymbol{\nu} &= {}^0\boldsymbol{J}_N\,{}^N\boldsymbol{\nu} \\ &= \begin{pmatrix} {}^0\boldsymbol{R}_N & \boldsymbol{0}_{3\times3} \\ \boldsymbol{0}_{3\times3} & {}^0\boldsymbol{R}_N \end{pmatrix} {}^N\boldsymbol{J}(\boldsymbol{q})\dot{\boldsymbol{q}} \\ &= {}^0\boldsymbol{J}(\boldsymbol{q})\dot{\boldsymbol{q}}\end{aligned}$$

8.1.3 解析雅可比矩阵

在方程(8.2)中，空间速度是以平移和角速度向量的形式表示的。我们可以用横滚-俯仰-偏航角或欧拉角变化率的形式来更直观地表示旋转速度。考虑横滚-俯仰-偏航角的这种情况，其旋转矩阵为

$$\begin{aligned}\boldsymbol{R} &= \boldsymbol{R}_x(\theta_r)\boldsymbol{R}_y(\theta_p)\boldsymbol{R}_z(\theta_y) \\ &= \begin{pmatrix} c\theta_p c\theta_y & -c\theta_p s\theta_y & s\theta_p \\ c\theta_r s\theta_y + c\theta_y s\theta_p s\theta_r & -s\theta_p s\theta_r s\theta_y + c\theta_r c\theta_y & -c\theta_p s\theta_r \\ s\theta_r s\theta_y - c\theta_r c\theta_y s\theta_p & c\theta_r s\theta_p s\theta_y + c\theta_y s\theta_r & c\theta_p c\theta_r \end{pmatrix}\end{aligned}$$

在这里使用缩写 $c\theta$ 和 $s\theta$ 分别表示 $\cos\theta$ 和 $\sin\theta$。通过一些烦琐的推导，可以写出上述旋转矩阵的导数 $\dot{\boldsymbol{R}}$。回顾方程(3.4)，得到：

$$\dot{\boldsymbol{R}} = \boldsymbol{S}(\omega)\boldsymbol{R}$$

我们可以求解出以横滚-俯仰-偏航角及其变化率表示的 ω：

$$\begin{pmatrix} \omega_x \\ \omega_y \\ \omega_z \end{pmatrix} = \begin{pmatrix} s\theta_p\dot{\theta}_y + \dot{\theta}_r \\ -c\theta_p s\theta_r\dot{\theta}_y + c\theta_r\dot{\theta}_p \\ c\theta_p c\theta_r\dot{\theta}_y + s\theta_r\dot{\theta}_p \end{pmatrix}$$

再将其分解为

$$\omega = \begin{pmatrix} 1 & 0 & s\theta_p \\ 0 & c\theta_r & -c\theta_p s\theta_r \\ 0 & s\theta_r & c\theta_p c\theta_r \end{pmatrix} \begin{pmatrix} \dot{\theta}_r \\ \dot{\theta}_p \\ \dot{\theta}_y \end{pmatrix}$$

上式可以简化为

$$\omega = \boldsymbol{B}(\boldsymbol{\Gamma})\dot{\boldsymbol{\Gamma}}$$

矩阵 $\boldsymbol{B}$ 本身是一个雅可比矩阵，它将横滚-俯仰-偏航角速度映射到空间角速度。该矩阵可以用工具箱函数计算：

```
>> rpy2jac(0.1, 0.2, 0.3)
ans =
    1.0000         0    0.1987
         0    0.9950   -0.0978
         0    0.0998    0.9752
```

如果矩阵 $\boldsymbol{B}$ 是非奇异的，得到解析的雅可比矩阵为

$$\boldsymbol{J}_a(\boldsymbol{q}) = \begin{pmatrix} \boldsymbol{I}_{3\times3} & \boldsymbol{0}_{3\times3} \\ \boldsymbol{0}_{3\times3} & \boldsymbol{B}^{-1}(\boldsymbol{\Gamma}) \end{pmatrix} \boldsymbol{J}(\boldsymbol{q})$$

当 $\cos\phi = 0$ 或俯仰角 $\phi = \pm\frac{\pi}{2}$ 时，矩阵 $\boldsymbol{B}$ 是奇异的，它被称为一个代表性奇异点。对于欧拉角的情况也可以采用类似方法获得相应的雅可比矩阵，这时使用对应的函数 `eul2jac`。

通过传递一个额外的参数给雅可比函数 `jacob0`，也可以计算解析的雅可比矩阵。例如：

```
>> p560.jacob0(qn, 'eul');
```

它将计算欧拉角形式的解析矩阵。

8.1.4　雅可比条件及可操纵性

我们已经讨论了雅可比矩阵如何将关节速率映射到末端执行器的笛卡儿速度，但其逆问题也具有很强的实际应用价值——达到所要求的末端执行器笛卡儿速度需要什么样的关节速度？如果 $\boldsymbol{J}$ 是方形矩阵且不为奇异，则可以将方程(8.2)转置，得到：

$$\dot{\boldsymbol{q}} = \boldsymbol{J}(\boldsymbol{q})^{-1}\boldsymbol{\nu} \tag{8.6}$$

上式是一种称为“分解速率运动控制”算法的基础，该算法将在下一节讨论。

对于一个 N 关节机器人，其雅可比矩阵是一个 $6\times N$ 阶的矩阵，所以一个方形雅可比矩阵需要一个六关节的机器人。当 $\det(\boldsymbol{J}(\boldsymbol{q})) = 0$ 时，机器人位形 $\boldsymbol{q}$ 即为奇异的或退化的。奇异发生时机械臂上有一个或多个关节轴重合，并引发自由度的损失——这又是万向节锁的问题。

例如，在 Puma 机器人的就绪位姿时，两个腕关节(关节 4 和 6)重合造成了一个自由度的损失。这种情况下雅可比矩阵为

```
>> J = p560.jacob0(qr)
J =
    0.1500   -0.8636   -0.4318         0         0         0
    0.0203    0.0000    0.0000         0         0         0
         0    0.0203    0.0203         0         0         0
         0         0         0         0         0         0
         0   -1.0000   -1.0000         0   -1.0000         0
    1.0000    0.0000    0.0000    1.0000    0.0000    1.0000
```

该矩阵是非满秩的，其秩仅为

```
>> rank(J)
ans =
     5
```

而对于 6×6 雅可比矩阵，其秩的最大值为 6。观察该雅可比矩阵可发现，它的第 4 列和第 6 列是完全相同的，这意味着这两个关节的运动将导致相同的笛卡儿速度[①]。函数 `jsingu` 可以自

① 对于 Puma 560 机器人，其关节 4 和 6 是唯一可能会重合而导致奇异的关节。机器人连杆之间的偏移距离 d_j 和 a_j 可以防止其他轴线发生重合。

动执行这种分析。例如：

```
>> jsingu(J)
1 linearly dependent joints:
  q6 depends on: q4
```

它表明 q_6 的速度可以完全用 q_4 的速度来表达。

但是，如果机器人只是接近，但实际上并没有处在一个奇异位形时，我们就会遇到问题了，这时其中一些末端执行器的笛卡儿速度将要求非常高的关节速率——在奇异位形时，这些速率将达到无穷大。我们可以选择一个离奇异位形 qr 稍远的位姿来举例说明。我们将 q_5 设置为一个很小但非零的值——5 度：

```
>> q = qr
>> q(5) = 5 * pi/180
q =
         0    1.5708   -1.5708         0    0.0873         0
```

此时雅可比矩阵为

```
>> J=p560.jacob0(q);
```

为了实现末端执行器在 z 方向上相对缓慢的运动——0.1 ms^{-1}，要求各关节的速度为

```
>> qd = inv(J)*[0 0 0.1 0 0 0]' ;
>> qd'
ans =   -0.0000   -4.9261    9.8522    0.0000   -4.9261         0
```

我们看到肩关节和肘关节的运动速度都非常高——肘关节的速度为 9.85 rad/s，或将近 600 deg/s。其原因是，虽然机器人没有处于奇异位形，但其雅可比矩阵的行列式值仍然非常小：

```
>> det(J)
ans =
  -1.5509e-05
```

或者可以说，它的条件数非常大，即雅可比矩阵是条件病态的：

```
>> cond(J)
ans =
  235.2498
```

然而，对于某些动作，比如这种情况下的旋转，雅可比矩阵的条件病态并不是什么问题。如果我们希望工具绕 y 轴旋转，则有

```
>> qd = inv(J)*[0 0 0 0 0.2 0]';
>> qd'
ans =    0.0000   -0.0000         0    0.0000   -0.2000         0
```

它需要的关节速率是非常合适的。

上述问题便引出了可操作性的概念。考虑一个关节速度单位向量：

$$\dot{\boldsymbol{q}}^{\mathrm{T}}\dot{\boldsymbol{q}}=1$$

它位于 N 维关节速度空间的超球面表面。将其代入方程(8.6)，得

$$\boldsymbol{\nu}^{\mathrm{T}}\left(\boldsymbol{J}(\boldsymbol{q})\boldsymbol{J}(\boldsymbol{q})^{\mathrm{T}}\right)^{-1}\boldsymbol{\nu}=1$$

该式描述了一些点，这些点位于末端执行器速度空间中的一个六维椭球的表面上。如果该椭球接近一个圆球形，即它所有的半径都在同一数量级，那么代表一切都好——末端执行器可以实现任意的笛卡儿速度。然而，如果其中一个或多个半径要小很多，这表明末端执行器不能达到那些小半径所对应的速度。

为了说明这一点，我们让机器人返回到标准位形，然后计算在世界坐标系中对应的平移速

度[①]的椭球：

```
>> J = p560.jacob0(qn);
>> J = J(1:3, :);
```

然后绘制出相应的速度椭球：

```
>> plot_ellipse(J*J')
```

如图 8.2(a)所示。我们看到，末端执行器可以在 y 和 z 方向上实现比 x 方向上更高的速度。在附录 E 中有关于椭圆和椭球更详细的讨论。

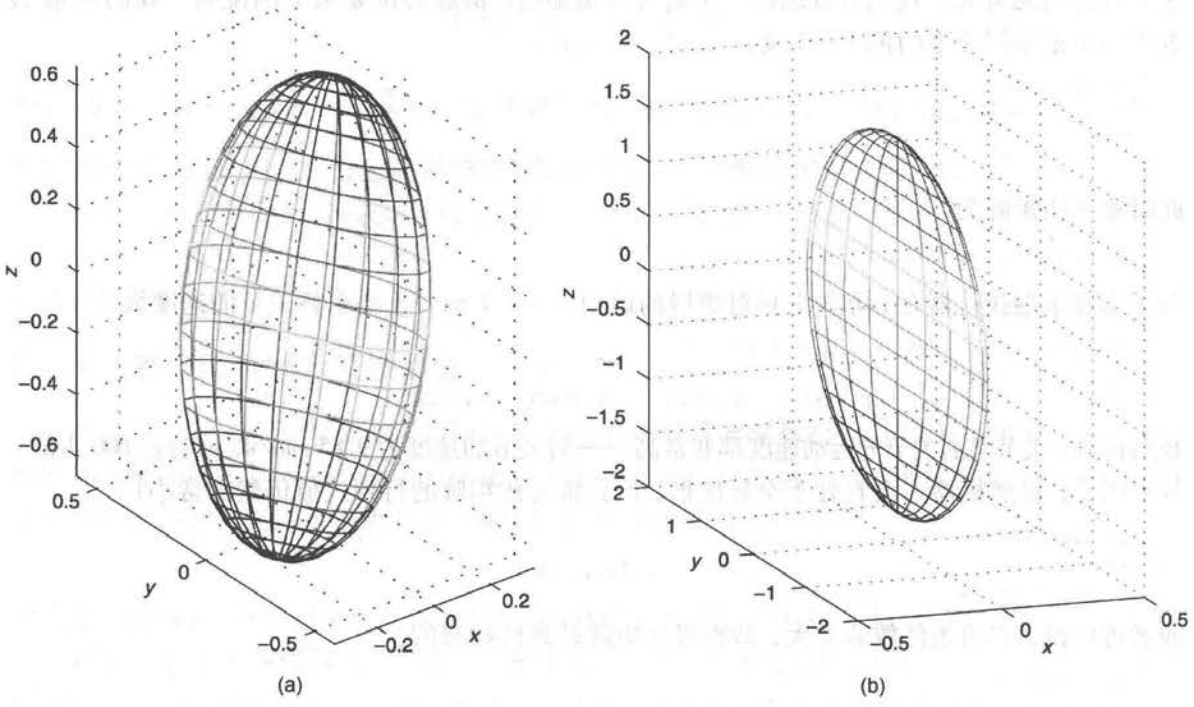


图 8.2 末端执行器的速度椭球。(a)标准位姿下的平移速度椭球；(b)奇异位姿附近的旋转速度椭球，该椭球演变成一个椭圆形板

另一方面，在奇异位形附近的旋转速度椭球为

```
>> J = p560.jacob0(qr);
>> J = J(4:6, :);
>> plot_ellipse(J*J')
```

如图 8.2(b)所示，它是一个厚度几乎为零的椭圆形板[②]。这表明机器人不能绕小半径对应的方向旋转，在本示例中是绕 x 轴的旋转。这就是那个丢失掉的自由度——关节 4 和 6 都提供了关于世界坐标系 z 轴的旋转，关节 5 提供关于世界坐标系 y 轴的旋转，但缺少关于世界坐标系 x 轴的旋转。椭球的形状揭示了机械臂执行某些运动的条件状态。而可操作性则是可以衡量这个椭球的圆球度的一个简单标量，例如最小半径与最大半径的比率[③]。工具箱的方法 maniplty 可以计算机器人的 Yoshikawa 可操作度：

$$m = \sqrt{\det(\boldsymbol{J}\boldsymbol{J}^{\mathrm{T}})}$$

① 因为我们只能绘制三维图形。

② 如果改变视角，可以更容易观察到。

③ 正如在附录 E 中讨论的，这些半径是 $\boldsymbol{J}(\boldsymbol{q})\boldsymbol{J}(\boldsymbol{q})^{\mathrm{T}}$ 特征值的平方根。

它与椭球的体积成正比。例如：

```
>> p560.maniplty(qr, 'yoshikawa')
ans =
     0
```

它表示在这个位姿下完全不具有可操作性——机器人处于一个奇异状态。在标准位姿下机器人的可操作度大一点[①]：

```
>> p560.maniplty(qn, 'yoshikawa')
ans =
    0.0786
```

但仍然不是特别高。实际应用中我们发现，有的机器人看似工作空间较大，但由于关节限制、自碰撞、奇异性以及可操作度小的区域，已经被大大减少了。这个量度仅仅是基于机构的运动学，而没有考虑质量和惯性——移动一个小腕关节肯定比移动一个大关节更容易。其他基于加速度和惯性的可操作度则考虑到了以上因素，将于9.1.6节中讨论。

8.2 分解速率运动控制

分解速率运动控制要利用方程(8.5)：

$$\dot{\boldsymbol{q}} = \boldsymbol{J}(\boldsymbol{q})^{-1}\boldsymbol{\nu}$$

将所需要的笛卡儿速度映射到关节速度，而不需要我们先前使用的逆运动学。现在，我们先假定雅可比矩阵是6×6的方阵且是非奇异的，但后面我们会放宽这些限制。

运动控制方案通常以离散时间形式实现：

$$\begin{aligned}\dot{\boldsymbol{q}}^*\langle k\rangle &= \boldsymbol{J}\big(\boldsymbol{q}\langle k\rangle\big)^{-1}\boldsymbol{\nu}^*\\ \boldsymbol{q}^*\langle k+1\rangle &= \boldsymbol{q}\langle k\rangle + \delta t\,\dot{\boldsymbol{q}}^*\langle k\rangle\end{aligned} \tag{8.7}$$

其中，δt 是采样时间间隔。这个前向矩形积分给出了用当前关节角度和末端执行器期望速度 $\boldsymbol{\nu}^*$ 表示的下一个时刻期望的关节角度 $\boldsymbol{q}^*\langle k+1\rangle$。

该算法可以用以下 Simulink® 模型实现：

```
>> sl_rrmc
```

如图8.3所示。笛卡儿速度是沿 y 方向上的常数0.1 m/s。Jacobian 模块的输入是当前的机械臂关节角度，输出是一个6×6的雅可比矩阵。然后将这个雅可比矩阵求逆，再乘以期望的笛卡儿速度就得到了期望的关节速度。机器人被模型化为一个积分器，即它充当了一个速度伺服器[②]。

运行该算法的仿真：

```
>> r = sim('sl_rrmc');
```

我们可以看到机械臂的末端执行器以恒定的速度在笛卡儿空间中运动的一个动画。仿真结果返回到仿真对象 r 中，而从 r 中可以提取时间和关节坐标：

```
>> t = r.find('tout');
>> q = r.find('yout');
```

---

① 这个可操作度包含了平移速度和旋转速度信息，但有不同的单位。可以通过在函数中加入选项"T"或"R"来分别计算平移和旋转速度的可操作度。

② 在这个模型中，我们假定它是一个理想的机器人，即实际关节角度始终等于期望关节角度 $\boldsymbol{q}^*$。跟踪误差的问题将在9.4节中讨论。

我们运用正向运动学来确定末端执行器的位置：

```
>> T = p560.fkine(q);
>> xyz = transl(T);
```

随后绘制出位置随时间变化的曲线[①]：

```
>> mplot(t, xyz(:,1:3))
```

如图 8.4(a)所示。笛卡儿运动在 y 方向上是我们期望的 0.1 mm/s 速度，但我们也发现在 x 方向和 z 方向上存在一些微小的扰动。我们再绘制出机械臂前三个关节的运动：

```
>> mplot(t, q(:,1:3))
```

如图 8.4(b)所示，它们没有随时间呈线性变化——反映了机械臂不断变化的运动学位形。

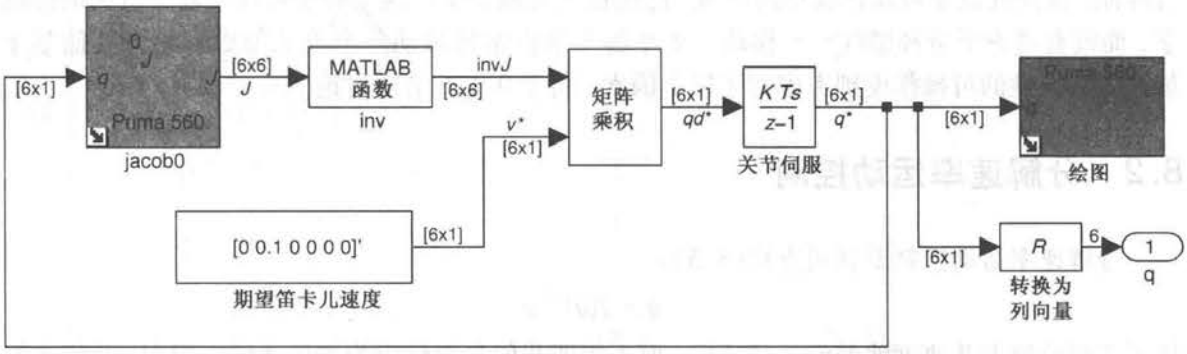


图 8.3 分解速率运动控制的 Simulink 模型 sl_rrmc，让末端执行器以恒定速度运动

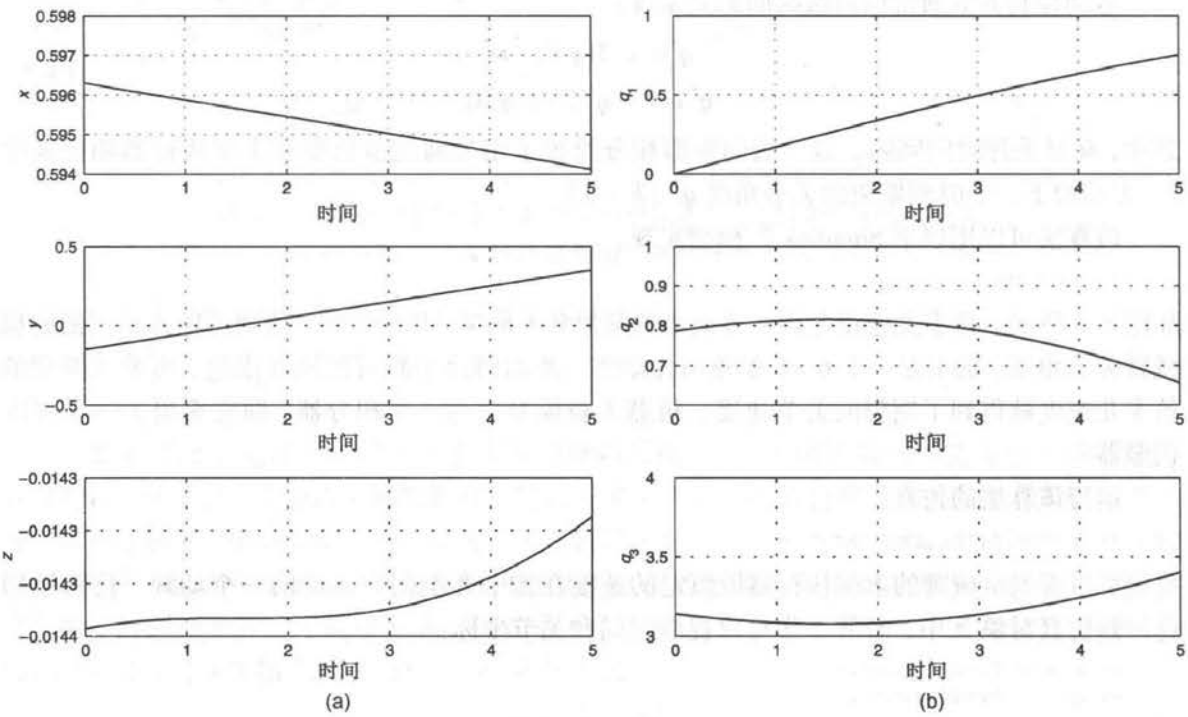


图 8.4 分解速率运动控制，随时间变化的笛卡儿和关节坐标。(a)末端执行器的笛卡儿位置；(b)关节坐标

① 函数 mplot 是工具箱里的一个工具，它能分别绘制出矩阵中各列数据的子图。

上述方法完全基于积分运算，因此也会有误差的积累，如图 8.4(a)所示在 x 和 z 方向上不需要的运动。要消除这种误差，可以通过将该算法变成一种闭环形式，使其基于期望和实际位姿之间的差：

$$\begin{aligned} \dot{\boldsymbol{q}}^*\langle k\rangle &= \boldsymbol{J}\left(\boldsymbol{q}\langle k\rangle\right)^{-1}\left(\xi^*\langle k\rangle \ominus \mathcal{K}\left(\boldsymbol{q}\langle k\rangle\right)\right) \\ \boldsymbol{q}^*\langle k+1\rangle &= \boldsymbol{q}\langle k\rangle + K_p \delta t \dot{\boldsymbol{q}}^*\langle k\rangle \end{aligned} \tag{8.8}$$

其中，K_p 是比例增益。现在的输入是随时间变化的期望位姿 $\xi^*\langle k\rangle$，而不是 $\boldsymbol{\nu}^*$。当前实际位姿是由基于实测关节坐标的正向运动学确定的。两个位姿之间的差是一个六维向量，它用方程(3.10)给出的函数 $\Delta(\cdot)$ 计算，由工具箱函数 `tr2delta` 来实现。

用以上算法画一个圆周路径的 Simulink ® 模型为

```
>> sl_rrmc2
```

如图 8.5 所示。Puma 560 机器人的工具将绘制出一个半径为 50 毫米的圆轨迹。轨迹随时间变化的 x，y 和 z 坐标由图中左下侧的模块计算，并转换成一个齐次变换。期望笛卡儿位姿与当前笛卡儿位姿之间的差通过 `tr2delta` 模块计算，并产生一个笛卡儿微分或比例空间速度，表示为一个六维向量。Jacobian 模块的输入是机械臂当前的关节角度，输出为雅可比矩阵。经过一个比例增益的作用后，结果是纠正笛卡儿误差所要求的关节空间运动。

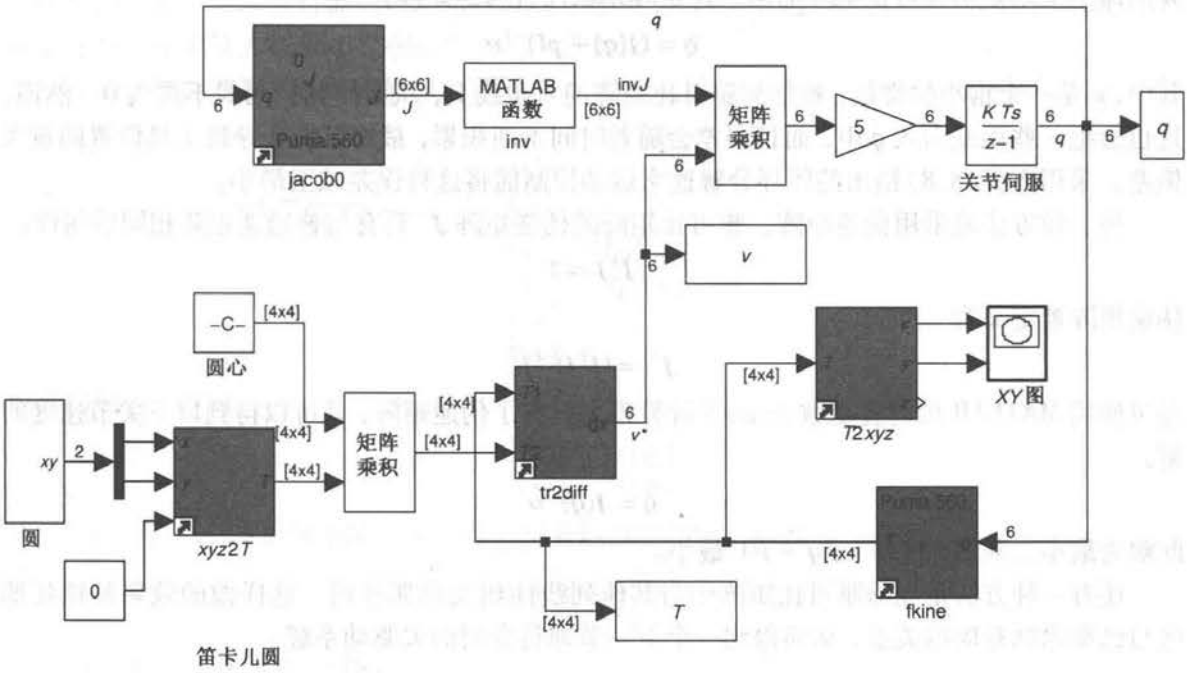


图 8.5 闭环分解速率运动控制的 Simulink ® 模型 `sl_rrmc2`，使末端执行器作圆周运动

到目前为止，我们都假定雅可比矩阵是方阵。对于非方阵的情况，用图 8.6 所示的框图可以帮助我们理解速度关系：

$$\boldsymbol{\nu} = \boldsymbol{J}(\boldsymbol{q})\dot{\boldsymbol{q}}$$

这里的雅可比矩阵是一个 $6 \times N$ 阶的矩阵，关节速度是一个 N 维向量，$\boldsymbol{\nu}$ 是一个 6 维向量。

在 $N<6$ 的情况下，被称为欠驱动机器人，而 $N>6$ 则是过驱动或冗余机器人。对于欠驱动的情况是无解的，因为方程组是欠约束的(超定方程组)，但也可以通过删除 $\boldsymbol{\nu}$ 和 $\boldsymbol{J}$ 中的一些

行使得方程组变为方形的(恰定方程组)——即接受在关节数不足的情况下一些笛卡儿自由度是不可控的。对于过驱动的情况，方程组是过约束的(不定方程组)，我们能找到它的一个最小二乘解。另外，也可以通过删除雅可比矩阵的一些列使之变为可逆方阵——即有效地锁死相应的关节轴。

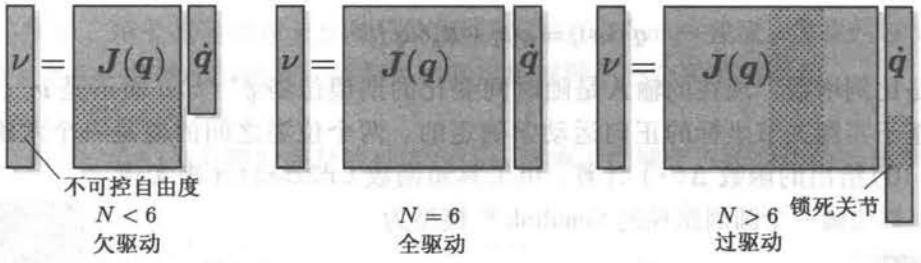


图 8.6 不同 N 情况下雅可比矩阵、$\boldsymbol{\nu}$ 和 $\dot{\boldsymbol{q}}$ 的关系示意。小点区域表示矩阵和向量中可以删除的部分，从而构建出一个有解的方形方程组

8.2.1 雅可比矩阵的奇异性

如果一个方形雅可比矩阵的行列式等于 0，我们就不能直接求解方程(8.6)。一种处理奇异的简单策略是用雅可比矩阵的阻尼逆矩阵来取代普通逆矩阵：

$$\dot{\boldsymbol{q}} = (\boldsymbol{J}(\boldsymbol{q}) + p\boldsymbol{I})^{-1}\boldsymbol{\nu}$$

其中，p 是一个很小的常数，被加到雅可比矩阵的对角线上，从而使其行列式不再为 0。然而，这也会把一些误差引入 $\dot{\boldsymbol{q}}$ 中，而且误差会随着时间不断积累，最终可能会导致工具位置的重大偏差。采用方程(8.8)给出的闭环分解速率运动控制能将这种误差减至最小。

另一种方法是采用伪逆矩阵。雅可比矩阵的伪逆矩阵 $\boldsymbol{J}^+$ 具有与普通逆矩阵相同的属性：

$$\boldsymbol{J}^+\boldsymbol{J} = \boldsymbol{I}$$

伪逆矩阵被定义为

$$\boldsymbol{J}^+ = (\boldsymbol{J}^{\mathrm{T}}\boldsymbol{J})^{-1}\boldsymbol{J}^{\mathrm{T}}$$

并可使用 MATLAB 的内置函数 `pinv`[①] 计算得出。有了伪逆矩阵，就可以得到以下关节速度的解：

$$\dot{\boldsymbol{q}} = \boldsymbol{J}(\boldsymbol{q})^+\boldsymbol{\nu}$$

此解为最小二乘解，使得 $|\boldsymbol{J}\dot{\boldsymbol{q}} - \boldsymbol{\nu}|$ 最小。

还有一种方法是删除雅可比矩阵中与其他列线性相关的那些列。这样做的效果是锁死那些与已删除列对应的关节，从而得到一个下一节即将介绍的欠驱动系统。

8.2.2 欠驱动机器人的雅可比矩阵

一个欠驱动机器人，其 $N<6$，且雅可比矩阵的高度大于宽度。以 7.3.3 节中处于标准位姿的两连杆机械臂为例：

```
>> mdl_twolink
>> qn = [1 1];
```

其雅可比矩阵为

① 这是左广义逆或伪逆，更多信息见附录 D。

```
>> J = jacob0(twolink, qn)
J =
   -1.7508   -0.9093
    0.1242   -0.4161
         0         0
         0         0
         0         0
    1.0000    1.0000
```

我们无法使用伪逆的方法求解方程(8.6)，因为它会尝试满足机械臂不能实现的运动约束。例如，我们期望末端执行器在 x 方向上有一个 0.1 ms^{-1} 的运动，用伪逆法计算需要如下的关节速度：

```
>> qd = pinv(J) * [0.1 0 0 0 0 0]'
qd =
   -0.0698
    0.0431
```

上述关节速度对应的末端执行器的速度为

```
>> xd = J*qd;
>> xd'
ans =
    0.0829   -0.0266         0         0         0   -0.0266
```

它虽然能基本满足在 x 方向上的期望运动，但同时也引发了 y 方向上不期望的平移和 z 方向上不期望的转动。末端执行器的转动是不能独立控制的(因为它是一个关于 q_1 和 q_2 的函数)，然而上述解法在求解方程最小二乘解时没有考虑到这一点。

我们必须面对现实，即我们只有两个自由度用来控制 v_x 和 v_y。我们将方程(8.2)进行分割，改写为

$$\left(\begin{array}{c} v_x \\ v_y \\ \hline v_z \\ \omega_x \\ \omega_y \\ \omega_z \end{array}\right) = \left(\begin{array}{c} J_{xy} \\ \hline J_0 \end{array}\right)\begin{pmatrix} \dot{q}_1 \\ \dot{q}_2 \end{pmatrix}$$

取其前两行，得到

$$\begin{pmatrix} v_x \\ v_y \end{pmatrix} = J_{xy}\begin{pmatrix} \dot{q}_1 \\ \dot{q}_2 \end{pmatrix}$$

其中，$\boldsymbol{J}_{xy}$ 是一个 2×2 阶矩阵。两边再分别乘以其逆矩阵，得到

$$\begin{pmatrix} \dot{q}_1 \\ \dot{q}_2 \end{pmatrix} = J_{xy}^{-1}\begin{pmatrix} v_x \\ v_y \end{pmatrix}$$

如果 $\det(\boldsymbol{J}_{xy}) \neq 0$，可以求解：

```
>> Jxy = J(1:2,:);
>> qd = inv(Jxy)* [0.1 0]'
qd =
   -0.0495
   -0.0148
```

由以上关节速度得到末端执行器的速度为

```
>> xd = J*qd;
>> xd'
ans =
    0.1000    0.0000         0         0         0   -0.0642
```

我们实现了 x 方向上期望的运动，而且除了 z 轴的旋转外没有其他多余的运动。z 轴的多余旋

转是不可避免的——我们已经使用了两个自由度来控制 x 和 y 方向上的平移，而没有控制 z 上的旋转。

8.2.3　过驱动机器人的雅可比矩阵

过驱动或冗余机器人具有 $N>6$，其雅可比矩阵的宽度大于高度。在这种情况下，我们使用矩阵的左伪逆重写方程(8.6)：

$$\dot{\boldsymbol{q}} = \boldsymbol{J}(\boldsymbol{q})^{+}\boldsymbol{\nu} \tag{8.9}$$

上式是从原有无数的解中挑选了一个使 $|\dot{\boldsymbol{q}}|$ 最小的解——最小范数解。

以 7.3.4 节中处于标准位姿的 8 轴 P8 机器人为例：

```
>> qn8 = [0 0 qn];
```

其雅可比矩阵是一个 6×8 阶矩阵：

```
>> J = jacob0(p8, qn8);
>> about(J)
J [double] : 6x8 (384 bytes)
```

现在我们要使其末端执行器在 x,y,z 方向上均以 0.2 ms^{-1} 的速度平移。使用方程(8.9)计算关节速度：

```
>> xd = [0.2 0.2 0.2 0 0 0]';
>> q = pinv(J) * xd;
>> q'
ans =
    0.1801    0.1800    0.0336    0.3197    0.0322    0.0475   -0.3519   -0.0336
```

我们看到，8 个关节均有非零的速度，各自为末端执行器的期望运动做出贡献。如果机器人不断重复一条运动路径，那么关节角度可能会随着时间产生漂移，即它们可能不会始终跟随那条路径，而有可能走向关节极限。对此，可以使用零空间控制以提供额外的约束。

该雅可比矩阵有 8 列，而其秩为 6：

```
>> rank(J)
ans =
     6
```

其零空间有两列：

```
>> N = null(J)
N =
    0.2543   -0.0320
    0.1086    0.2635
   -0.1821   -0.4419
    0.3543   -0.1534
   -0.7260    0.3144
   -0.2576   -0.6250
    0.3718   -0.1610
    0.1821    0.4419
```

这些列是跨越零空间的正交向量。也就是说，任何以这两个列向量线性组合而成的关节速度，都不会产生末端执行器的运动。证明如下：

```
>> norm( J * (N(:,1) + N(:,2)) )
ans =
   5.7168e-16
```

这一性质非常有用，可以用它改写方程(8.9)：

$$\dot{\boldsymbol{q}} = \underbrace{\boldsymbol{J}(\boldsymbol{q})^{+}\boldsymbol{\nu}}_{\text{末端执行器的运动}} + \underbrace{\boldsymbol{N}\boldsymbol{N}^{+}\dot{\boldsymbol{q}}_{ns}}_{\text{零空间运动}} \tag{8.10}$$

其中，$N \times N$ 阶的矩阵 NN^{+} 会将期望的关节运动映射到零空间中，因此它们就不会影响末端执行器的运动，实现了两种运动的叠加。

零空间运动可用于超冗余的机器人，用于连杆之间以及连杆和障碍之间的避碰，或使得关节坐标远离其机械极限位置。考虑下述情况，除了期望的笛卡儿速度 xd，我们还希望同时使关节 5(Puma 的肘关节)的角度接近零(它当前为 π)，所以我们设置了一个期望的关节速度：

```
>> qd_ns = [0 0 0 0 -0.1 0 0 0]';
```

并将其映射到零空间内：

```
>> qp = N * pinv(N) * qd_ns;
>> qp'
    0.0195   -0.0004    0.0007    0.0305   -0.0626    0.0009    0.0321   -0.0007
```

该映射引入了一个比例变换，然而关节 5 的速度不是我们想要的 -0.1，但我们可以用一个比例因子来校正：

```
>> qp = qp / qp(5) * qd_ns(5)
qp =
    0.0312   -0.0006    0.0011    0.0487   -0.1000    0.0014    0.0513   -0.0011
```

其他的关节速度提供所需的补偿运动，以确保末端执行器的位姿不被扰乱，如下所示：

```
>> norm( J * qp)
ans =
   3.1107e-17
```

8.3 力的关系

力和力矩应该是下一章有关动力学内容的主题之一，但现在讨论它们是因为它们也是雅可比矩阵的另一个非常有用的属性。

在前面的讨论中，我们介绍了空间速度的概念 $\boldsymbol{\nu} = (v_x, v_y, v_z, \omega_x, \omega_y, \omega_z)$。对于力来说，也有一个空间当量概念，称为力旋量 $\boldsymbol{g} = (f_x, f_y, f_z, m_x, m_y, m_z) \in \mathbb{R}^6$，它是包含力和力矩的一个向量。

8.3.1 坐标系间的力旋量转换

通过方程(8.3)雅可比矩阵将空间速度从一个坐标系映射到另一个坐标系。同理，雅可比矩阵也可以用来映射两个坐标系之间的力旋量。对于两个坐标系都固接到同一个刚体的情况，有

$$ {}^{A}\boldsymbol{g} = ({}^{B}\boldsymbol{J}_{A})^{\mathrm{T}}\, {}^{B}\boldsymbol{g} \tag{8.11} $$

其中，${}^{B}\boldsymbol{J}_{A}$ 由方程(8.4)或方程(8.5)给出，并且是从坐标系 $\{A\}$ 到 $\{B\}$ 的相对位姿 ${}^{B}\boldsymbol{T}_{A}$ 的函数。注意，力变换与速度变换的不同之处在于，力变换使用了雅可比转置矩阵，且其映射方向是相反的——从坐标系 $\{B\}$ 到 $\{A\}$。

例如，考虑两个固接到相同刚体的坐标系，它们在 x 方向上相距 2 米。我们创建如下雅可比矩阵：

```
>> J = tr2jac( transl(2, 0, 0) );
```

然后将一个作用在第二个坐标系 y 方向上的 3 N 的力转换到第一个坐标系：

```
>> F = J'*[0 3 0 0 0 0]';
>> F'
ans =
     0     3     0     0     0     6
```

其结果是在第一个坐标系的 y 方向上仍然有一个 3 N 的力。另外，由于杆臂效应在 z 轴上还增加了一个 6 Nm 的力矩。

8.3.2 力旋量转换至关节空间

通过方程(8.2)，机械臂的雅可比矩阵可将关节速度转换为末端执行器的空间速度。同理，雅可比转置矩阵也能将末端执行器的力旋量转换为各关节的力矩和力[①]：

$$\boldsymbol{Q} = {}^{0}\boldsymbol{J}(\boldsymbol{q})^{\mathrm{T}}\,{}^{0}\boldsymbol{g} \tag{8.12}$$

其中 $\boldsymbol{g}$ 是世界坐标系中的力旋量，$\boldsymbol{Q}$ 是广义的关节力向量。$\boldsymbol{Q}$ 中的元素可以是对应转动关节的力矩，或是对应移动关节的力。

如果力旋量是在末端执行器的坐标系中定义的，那么上式变为

$$\boldsymbol{Q} = {}^{N}\boldsymbol{J}(\boldsymbol{q})^{\mathrm{T}}\,{}^{N}\boldsymbol{g} \tag{8.13}$$

有趣的是，这种从外部变量(力旋量)到关节变量(广义力)的映射永远不存在奇异，对于速度也是一样。我们在下一节将利用这个特性来求逆运动学问题的数值解。

对于图 8.1 所示处于标准位姿的 Puma 560 机器人，如果在世界坐标系的 y 轴方向上作用一个 20 N 的力，那么作用到各关节的力矩为

```
>> tau = p560.jacob0(qn)' * [0 20 0 0 0 0]';
>> tau'
ans =
   11.9261    0.0000    0.0000         0         0         0
```

由于这个 y 向力是从一侧来推机械臂，因此只有腰部关节会旋转——由于杆臂效应而产生的 11.93 Nm 力矩。而如果在世界坐标系 x 方向上作用一个 20 N 的力，则各关节力矩为

```
>> tau = p560.jacob0(qn)' * [20 0 0 0 0 0]';
>> tau'
ans =
    3.0010    0.2871    6.3937         0         0         0
```

这个 20 N 的力是要把末端执行器拉离基座，结果将使前 3 个关节都产生力矩。

8.4 逆运动学：一个通用数值方法

在 7.3 节中，我们使用了一个解析方法求解了机械臂逆运动学问题，但此方法的前提是机器人必须为 6 个关节且有一个球腕关节。对于那些不满足此要求的机器人，例如关节多于或少于 6 个的机器人，我们就需要考虑用数值解。在这里，我们将开发一种基于正向运动学和雅可比转置矩阵的通用方法，对于任何机械臂位形都可以计算其正向运动学和雅可比转置矩阵——这些函数不存在奇异性。

该方法的原理如图 8.7 所示。这个虚拟机器人的当前位姿用清晰的实线画出，而期望的位姿则用模糊部分表示。根据这个重叠位姿图，可以写出

$$\xi_E^* = \xi_E \oplus \xi_\Delta$$

将上式重新排列为

$$\xi_\Delta = \ominus\xi_E \oplus \xi_E^*$$

① 通过虚功原理推导得出，参见书籍[Spong et al. 2006, 4.10 节]。

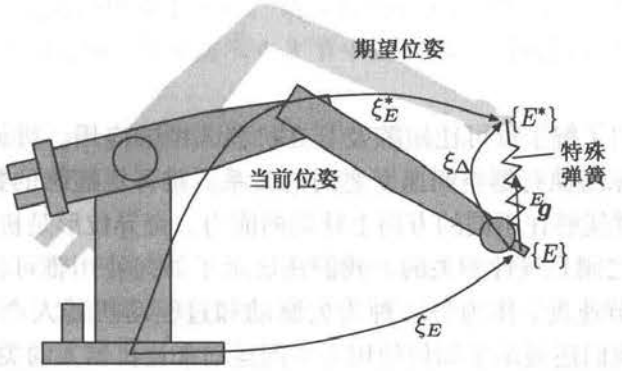


图 8.7 逆运动学数值方法的示意图，显示了当前位姿 ξ_E 和期望姿态 ξ_E^*

假设在末端执行器的两个位姿之间有一个特殊弹簧，其作用是将末端执行器拉向(加扭转)期望位姿①，且施加的力旋量与两个位姿之间的差异成正比：

$$ {}^E\boldsymbol{g} \propto \Delta(\xi_E, \xi_E^*) $$

其中，函数 $\Delta(\cdot)$ 由方程(3.11)给出，它估计出两个位姿之间的差异，而这个差异值是一个包含平移和旋转分量的六维向量②。力旋量也是一个六维向量，包括力和力矩。我们将上式具体化为

$$ {}^E\boldsymbol{g} = \gamma\Delta(\xi_E, \xi_E^*) \tag{8.14} $$

其中，γ 是一个常数，当前位姿用正向运动学计算：

$$ \xi_E\langle k\rangle = \mathcal{K}(\boldsymbol{q}\langle k\rangle) \tag{8.15} $$

其中，$\boldsymbol{q}\langle k\rangle$ 是逆运动学解的当前估值。

使用雅可比转置矩阵方程(8.13)，由方程(8.14)得到的末端执行器力旋量可以被转换为关节力：

$$ \boldsymbol{Q}\langle k\rangle = {}^N\boldsymbol{J}(\boldsymbol{q}\langle k\rangle)^{\mathrm{T}}\,{}^E\boldsymbol{g}\langle k\rangle \tag{8.16} $$

假设这个虚拟机器人没有关节电机而只有黏滞阻尼器，这样施加的关节力与它所产生的关节速度之间将成比例关系：

$$ \dot{\boldsymbol{q}}\langle k\rangle = \boldsymbol{Q}\langle k\rangle / B $$

其中，B 是关节阻尼系数(假设所有的阻尼器是相同的)。现在就可以写出下一个离散时间的新关节坐标：

$$ \boldsymbol{q}\langle k+1\rangle = \alpha\dot{\boldsymbol{q}}\langle k\rangle + \boldsymbol{q}\langle k\rangle \tag{8.17} $$

其中，α 是一个精心挑选的增益。我们可以一直重复方程(8.14)至方程(8.17)的计算，直到力旋量 ${}^E\boldsymbol{g}$ 的值变得足够小。在上述计算中使用了雅可比转置矩阵，这样就可以不必面对使用雅可比逆矩阵可能出现的问题，即非方形阵或奇异性问题。

在 7.3.3 节中，我们在计算一个 $N<6$ 的机器人的逆运动学时使用了一个遮盖向量。将该遮盖向量 $\boldsymbol{m}$ 放到方程(8.16)中，使其变为

$$ \boldsymbol{Q}\langle k\rangle = {}^N\boldsymbol{J}(\boldsymbol{q}\langle k\rangle)^{\mathrm{T}}\,\mathrm{diag}(\boldsymbol{m})\,{}^E\boldsymbol{g}\langle k\rangle \tag{8.18} $$

① 为了改变末端执行器的姿态及位置，它必须是一个特殊弹簧。

② 这里假设两个位姿之间的姿态差异“很小”，这样估值会变得更好，因为解是收敛的。

8.5 本章总结

在这一章中，我们了解了雅可比矩阵及其在机器人中的应用。机械臂的雅可比矩阵表征了关节坐标变化率与末端执行器空间速度之间的关系。雅可比矩阵的数值特性还揭示了可操作性的概念，即机械臂能够在不同的方向上移动的能力。奇异位形是机械臂的极端状况，表现在雅可比矩阵的各列之间是线性相关的。我们还展示了如何使用雅可比的逆矩阵来求解期望笛卡儿速度对应的关节速度，作为另一种为欠驱动和过驱动机器人产生笛卡儿路径的方法。对于过驱动机器人，我们还展示了如何使用零空间运动来让机器人的关节运动，但同时不会影响末端执行器的位姿。

我们还介绍了其他的雅可比矩阵。一个雅可比矩阵可以用来映射两个坐标系之间的空间速度。解析雅可比矩阵还可以将笛卡儿角速度转换为横滚-俯仰-偏航角速率或欧拉角速率。

雅可比的转置矩阵可用于将末端执行器上的力旋量转换为关节力矩，同样它也能转换坐标系之间的力旋量。最后，我们展示了如何使用雅可比转置矩阵和正向运动学来计算逆运动学数值解，该方法可用于任意机器人，且包括位于奇异位形。

扩展阅读

机械臂的雅可比矩阵几乎出现在所有标准的机器人教材中，如[Spong et al. 2006]，[Craig 2004]，[Siciliano et al. 2008]，[Paul 1981]，以及《机器人技术手册》第 1 章[Siciliano and Khatib, 2008]。有关可操作性和速度椭球的精彩论述可参见[Siciliano et al. 2008]，而在[Yoshikawa 1984]中则提出了一种最常用的可操作性衡量方法。在[Paul and Shimano 1978]中首次描述了基于 D-H 参数的计算机械臂雅可比矩阵的方法。

惠特尼[Whitney 1969]提出了分解速率运动控制的方案，其扩展方法包括基于雅可比伪逆矩阵的控制[Klein and Huang 1983]，以及阻尼最小二乘方法[Deo and Walker 1995]。工具箱中使用的逆运动学数值方法是基于奇异点运动控制的研究结果[Chiaverini et al. 1991]。

习题

1. 对于 Puma 560 机器人，设计出一种位形，使其中 3 个关节轴相互平行。
2. 推导出用于欧拉角转换的解析雅可比矩阵。
3. 可操作性(8.1.4 节)。
   a) 对于一个末端执行器位置处于其工作空间中的两连杆机械臂，画出其速度椭圆(x 和 y 方向上的速度)。每个椭圆的中心应在末端执行器的位置上。
   b) 对于 Puma 560 机器人，找到一个位形，使其可操作性大于在 `qn` 位形。
   c) 把用 `plot` 方法绘制的机械臂平移或旋转速度椭球进行叠加，并创建一个动画，显示当机器人沿轨迹运动时它是如何改变形状的。
4. 分解速率运动控制(8.2 节)。
   a) 在不同的笛卡儿平移和旋转速度及其组合下进行实验。
   b) 扩展图 8.4 中 Simulink® 系统的功能，也可以将雅可比矩阵的行列式记录到工作空间中。

c)在图8.4中，机器人运动仿真了5秒。请延长仿真时间到10秒，并解释发生的现象。

d)设置初始位姿和运动方向，并模仿7.4.3节中的实验。当机器人达到奇异点时，会发生什么事?

e)使用MATLAB® 函数pinv替换图8.3中的雅可比矩阵求逆模块。

f)使用一个阻尼最小二乘函数替换图8.3中的雅可比矩阵求逆模块，并研究不同阻尼系数的影响。

g)使用一个基于MATLAB® 函数`lscov`的模块替换图8.3中的雅可比矩阵求逆模块。

h)使用7.3.4节中的8轴机器人修改图8.5所示的仿真。观察关节坐标轨迹，其是否和末端执行器轨迹一样是重复不变的?

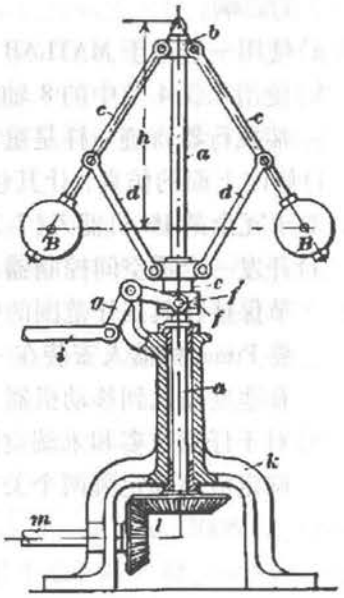
i)修改上面的仿真，让其包含零空间运动，使得机器人关节远离各自的极限位置。

5. 对于冗余的P8机器人(7.3.4节)。

a)开发一个零空间控制器，通过用前两个关节来定位Puma机器人的底座，使得后六个关节保持在其工作范围的中间。这样修改的P8机器人具有最大的可操作度。现在再考虑将Puma机器人安装在一个非完整的机器人上，设计一个控制器，使之产生适当的转向和速度输入到移动机器人(有挑战性)。

b)对于任意位姿和末端空间速度，我们要移动六个关节并锁定两个关节。写一个算法，以确定应该锁定哪两个关节。

第9章　动力学与控制



在这一章中，我们将讨论一个串联机械臂的动力学与控制。机械臂上每一个连杆由其前一个连杆的反作用力和力矩支撑，另外它还受其自身的重力，以及来自它所支撑的后一个连杆的反作用力和力矩。

9.1节中将介绍的运动方程是一组耦合动力学方程，它们描述了要达到某个特定的机械臂状态所必需的关节力矩。该方程组中包含惯性力、重力和陀螺耦合项。通过对该运动方程组的研究，我们还探索了一些重要问题。例如，一个关节的运动如何对其他关节施加干扰力，惯性力和重力与机械臂位形的关节，以及负载的影响。9.2节介绍了真实的传动系统问题，比如齿轮传动和摩擦。9.3节中介绍了正向动力学，即描述了驱动器在关节上施加的力和扭矩如何使机械臂运动，即机械臂的位形如何随时间的演变。在9.4节中我们介绍了机器人控制系统，不管机器人动态特性或关节灵活性如何变化，这些控制系统通过计算关节力能使末端执行器沿着期望的轨迹运动。

9.1　运动方程

考虑驱动串联机械臂第j个旋转关节的一个电机。从图7.2中我们还记得，关节j连接了连杆$j-1$和连杆j。该电机对外侧连杆j施加了一个加速转动的力矩，同时它也对内侧连杆$j-1$施加了一个反作用力矩。作用于连杆j到N上的重力还对连杆施加了一个重力，旋转中的连杆还相互施加了陀螺力。作用于电机上的惯性力是关于所有外侧连杆位形的一个函数。

单个连杆的受力情况是相当复杂的，但对于一系列连杆，其结果则可以用一组优雅和简洁的矩阵形式的耦合微分方程表示：

$$\boldsymbol{Q}=\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}}+\boldsymbol{C}(\boldsymbol{q},\dot{\boldsymbol{q}})\dot{\boldsymbol{q}}+\boldsymbol{F}(\dot{\boldsymbol{q}})+\boldsymbol{G}(\boldsymbol{q})+\boldsymbol{J}(\boldsymbol{q})^{\mathrm{T}}\boldsymbol{f} \tag{9.1}$$

其中，$\boldsymbol{q}$，$\dot{\boldsymbol{q}}$，$\ddot{\boldsymbol{q}}$分别为广义的关节位置、速度和加速度的向量，$\boldsymbol{M}$是关节空间惯量矩阵，$\boldsymbol{C}$是科氏力和向心力耦合矩阵，$\boldsymbol{F}$为摩擦力，$\boldsymbol{G}$是重力负荷，$\boldsymbol{Q}$是与广义坐标$\boldsymbol{q}$对应的广义驱动力向量。最后一项给出了施加在末端执行器上的力旋量$\boldsymbol{g}$所产生的关节力，其中$\boldsymbol{J}$是机械臂的雅可比矩阵。该方程描述了机械臂刚体动力学，也被称为逆动力学——给定位姿、速度和加速度，计算所需的关节力或扭矩。这些方程可以用任何经典动力学方法推导出来，如牛顿第二定律和欧拉运动方程，或拉格朗日能量法。计算方程(9.1)的一个非常有效的方法是用递归的牛顿-欧拉算法。它先从基座开始，在向外递推中通过增加各关节的速度和加速度，以确定每一个连杆的速度和加速度。然后再从工具回归到基座，计算作用于每一个连杆的力和扭矩，从而得

到关节扭矩[①]。递归牛顿-欧拉算法有 $O(N)$ 的复杂性，可以写成函数形式：

$$\boldsymbol{Q} = \mathcal{D}(\boldsymbol{q}, \dot{\boldsymbol{q}}, \ddot{\boldsymbol{q}}) \tag{9.2}$$

在工具箱中，该算法是由 SerialLink 对象中的 rne 方法实现的。考虑 Puma 560 机器人：

```
>> mdl_puma560
```

机器人处于标准位姿，且关节速度和加速度均为 0。此时广义关节力(这里为广义力矩)为

```
>> Q = p560.rne(qn, qz, qz)
Q =
   -0.0000   31.6399    6.0351    0.0000    0.0283         0
```

由于机器人是不动的(假设 $\dot{\boldsymbol{q}} = \ddot{\boldsymbol{q}} = 0$)，这些力矩的作用必然是对抗重力保持机器人静止平衡。为了证实这一点，我们在没有重力的情况下重新计算力矩：

```
>> Q = p560.rne(qn, qz, qz, [0 0 0]')
ans =
     0     0     0     0     0     0
```

其中的最后一个参数将取代计算对象默认的重力向量。

如同大多数工具箱方法那样，rne 方法也可以对一个轨迹进行操作：

```
>> q = jtraj(qz, qr, 10)
>> Q = p560.rne(q, 0*q, 0*q)
```

其返回一个 10×6 阶的矩阵：

```
>> about(Q)
Q [double] : 10x6 (480 bytes)
```

其中每行代表对应于某一时间步的广义力 q。如第 5 时间步的关节力矩为

```
>> Q(5,:)
ans =
    0.0000   29.8883    0.2489         0         0         0
```

现在考虑机器人运动的情况。在机器人处于标准位姿的瞬间，只有关节 1 以 1rad/s 的速度转动，且所有关节的加速度为零。此时在忽略重力作用下，关节力矩不再为零：

```
>> p560.rne(qn, [1 0 0 0 0 0], qz, [0 0 0]')
  -24.8240    0.6280   -0.3607   -0.0003   -0.0000         0
```

关节 1 上的力矩由于摩擦，因而与运动方向相反。但有趣的是，力矩已经被施加到关节 2、3 和 4 上。这些是由于陀螺效应(向心力与科氏力)，也被称为速度耦合力矩，因为一个关节的旋转速度引起了其他一些关节上的力矩。

背景材料：三维空间动力学。在三维空间中，运动物体的动力学由两个重要的方程描述。第一个方程是牛顿第二定律，它描述了三维空间中的平移运动：

$$\boldsymbol{f} = m\dot{\boldsymbol{v}}$$

其中，为 m 质量，$\boldsymbol{f}$ 为作用力，$\boldsymbol{v}$ 为速度。第二个方程是欧拉运动方程，描述了三维空间中的旋转运动：

$$\tau = \boldsymbol{J}\dot{\omega} + \omega \times \boldsymbol{J}\omega$$

其中，τ 是扭矩，ω 是角速度，$\boldsymbol{J}$ 是转动惯量矩阵(见第 76 页)。

陀螺运动。一个高速旋转的光盘具有角动量 $\boldsymbol{h} = \boldsymbol{J}\omega$。如果一个力矩 τ 被施加到此旋转物上，那么它会产生一个称为陀螺进动的转动，转动轴与 τ 和 $\boldsymbol{h}$ 均垂直，角速度为 ω_p。这些参数的关系为

$$\tau = \omega_P \times \boldsymbol{h}$$

---

① 逆动力学的递归形式并不显式地计算方程(9.1)中的矩阵 $\boldsymbol{M}$，$\boldsymbol{C}$ 和 $\boldsymbol{G}$。然而，我们可以用递归牛顿-欧拉算法来计算这些矩阵，工具箱函数 inertia 和 coriolis 就采用了沃尔克和奥林的“方法 1”[Walker and Orin 1982]。虽然逆动力学的递归形式计算效率高，但计算方程(9.1)中的各个动力学参数($\boldsymbol{M}$，$\boldsymbol{C}$ 和 $\boldsymbol{G}$)是相当费时的——一个 N 轴的机械臂具有的复杂性为 $O(N^3)$。

如果你曾经尝试去转动一个高速旋转的自行车轮的轴，你会观察到这种效果——绕某个轴施加的“扭转”会让它在你手上产生一个正交方向上转动。

一个捷联式陀螺传感器中包含一个高速飞轮，其 $\boldsymbol{h}$ 值很大。当该陀螺被外力矩转动时，就会产生一个与 ω_P 成正比的回转力，用一个力传感器可以将其测出。

矩阵 $\boldsymbol{M}$, $\boldsymbol{C}$, $\boldsymbol{F}$ 和 $\boldsymbol{G}$ 中的元素是关于连杆的运动学参数（θ_j, d_j, a_j, α_j）和惯性参数的复杂函数。每个连杆都有 10 个独立的惯性参数：连杆质量 m_j；相对于连杆坐标系的质心 $\boldsymbol{r}_j$；以及 6 个二阶矩，它们表示了在以质心为原点且与关节坐标系 $\{j\}$ 平行的坐标系中连杆的惯性，见第 76 页。我们可以通过以下方式查看一个机器人连杆的动力学参数：

```
>> p560.links(1).dyn
l =
 theta=q, d=0, a=0, alpha=1.571 (R,stdDH)
  m     = 0.000000
  r     = 0.000000 0.000000 0.000000
  I     = | 0.000000 0.000000 0.000000 |
          | 0.000000 0.350000 0.000000 |
          | 0.000000 0.000000 0.000000 |
  Jm    = 0.000200
  Bm    = 0.001480
  Tc    = 0.395000(+) -0.435000(-)
  G     = -62.611100
```

其中按顺序分别为是：运动学参数，连杆质量，质心位置，惯量矩阵，电机惯量，电机摩擦，库仑摩擦力和齿轮传动比。

本节其余部分将探讨方程(9.1)中的各种矩阵分量。

9.1.1 重力

$$\boldsymbol{Q} = \boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{F}(\dot{\boldsymbol{q}}) + \boldsymbol{G}(\boldsymbol{q}) + \boldsymbol{J}(\boldsymbol{q})^{\mathrm{T}}\boldsymbol{f}$$

我们以重力项开始讨论，因为它通常在方程(9.1)中占主导地位，而且即使机器人静止或缓慢移动它也一直存在。有些机器人使用平衡重物[①]或者弹簧来减小需要电机提供的重力矩——这样可以使电机更小，从而降低成本。

在前面的讨论中，我们使用了 `rne` 方法并通过设定关节速度和加速度为零来计算重力负荷。一个更方便的做法是使用 `gravload` 方法：

```
>> gravload = p560.gravload(qn)
gravload =
   -0.0000   31.6399    6.0351    0.0000    0.0283         0
```

背景材料：艾萨克·牛顿爵士（1642—1727）是一名英国数学家和炼金术士。他曾任剑桥大学卢卡斯数学教授，皇家造币厂主管，以及第十三任英国皇家学会会长。他的成就包括三大运动定律：万有引力定律，天体运动学，光与色的理论（见 10.1 节），并搭建出第一个反射望远镜。

这些成就多数于 1687 年发表在他的 3 卷伟大的作品《自然哲学的数学原理》（*The Philosophiae Naturalis Principia Mathematica*）之中。他于 1704 年出版了《光学》（*Opticks*）一书，其中研究了光线和色彩的本质以及衍射现象。力的公制单位也以纪念他而命名。他去世后被葬在伦敦的威斯敏斯特教堂。

① 但平衡重物会增加相应关节的惯性，因为它在一个杆臂末端增加了额外的质量，从而增加了机器人的总质量。

SerialLink 对象中包含一个默认的重力加速度向量，且初始化为地球标准值[①]：

```
>> p560.gravity'
ans =
         0         0    9.8100
```

我们可以将其改为月球的重力值：

```
>> p560.gravity = p560.gravity/6;
```

从而减小关节力矩：

```
>> p560.gravload(qn)
ans =
    0.0000    5.2733    1.0059    0.0000    0.0047         0
```

或者可以把我们的月球机器人翻转倒置：

```
>> p560.base = trotx(pi);
>> p560.gravload(qn)
ans =
    0.0000   -5.2733   -1.0059   -0.0000   -0.0047         0
```

我们看到力矩的符号改变了。现在又把我们的机器人带回到地球，且使其右侧朝上：

```
>> mdl_puma560
```

由于机器人重力而施加在关节上的力矩大小完全取决于机器人的位姿。直观地看，手臂水平伸出时的肩关节上力矩

```
>> Q = p560.gravload(qs)
Q =
   -0.0000   46.0069    8.7722    0.0000    0.0283         0
```

要比手臂竖直向上时大得多。

```
>> Q = p560.gravload(qr)
Q =
         0   -0.7752    0.2489         0         0         0
```

在第一种位姿下肘关节上的重力转矩也是非常高的，因为它必须支撑小臂和手腕的重量。我们还可以用下面的方法研究关节 2 和 3 的重力载荷是如何随着关节位形变化的：

```
1  [Q2,Q3] = meshgrid(-pi:0.1:pi, -pi:0.1:pi);
2  for i=1:numcols(Q2),
3      for j=1:numcols(Q3);
4      g = p560.gravload([0 Q2(i,j) Q3(i,j) 0 0 0]);
5      g2(i,j) = g(2);
6      g3(i,j) = g(3);
7      end
8  end
9  surfl(Q2, Q3, g2); surfl(Q2, Q3, g3);
```

结果如图 9.1 所示。关节 2 上的重力矩在 ±40 Nm 之间变化，而关节 3 在 ±10 Nm 之间变化。这种类型的分析在机器人设计中是非常重要的，可以用来确定电机所需的扭矩。

9.1.2　惯量矩阵

$$\boldsymbol{Q} = \boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q},\dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{F}(\dot{\boldsymbol{q}}) + \boldsymbol{G}(\boldsymbol{q}) + \boldsymbol{J}(\boldsymbol{q})^{\mathrm{T}}\boldsymbol{f}$$

惯量矩阵[②]是关于机械臂位姿的函数：

---

① SerialLink 构造函数中的“gravity”选项可以改变这个值。

② 这个惯量矩阵包括电机的转动惯量，它们被加到矩阵对角线元素上。电机和刚体的惯性将在 9.4.2 节中进一步讨论。

```
>> M = p560.inertia(qn)
M =
    3.6594   -0.4044    0.1006   -0.0025    0.0000   -0.0000
   -0.4044    4.4137    0.3509    0.0000    0.0024    0.0000
    0.1006    0.3509    0.9378    0.0000    0.0015    0.0000
   -0.0025    0.0000    0.0000    0.1925    0.0000    0.0000
    0.0000    0.0024    0.0015    0.0000    0.1713    0.0000
   -0.0000    0.0000    0.0000    0.0000    0.0000    0.1941
```

此矩阵是对称的。对角元素 $\boldsymbol{M}_{jj}$ 描述了关节 j 的惯量，即，$Q_j = \boldsymbol{M}_{jj}\ddot{\boldsymbol{q}}_j$。请注意，前两个对角元素分别对应于机器人的腰关节和肩关节，数值很大，因为这些关节的运动涉及沉重的大臂和小臂连杆的转动。非对角线项 $\boldsymbol{M}_{ij} = \boldsymbol{M}_{ji}$，$i \neq j$，它们代表了从关节 j 的加速度到关节 i 上广义力的耦合。

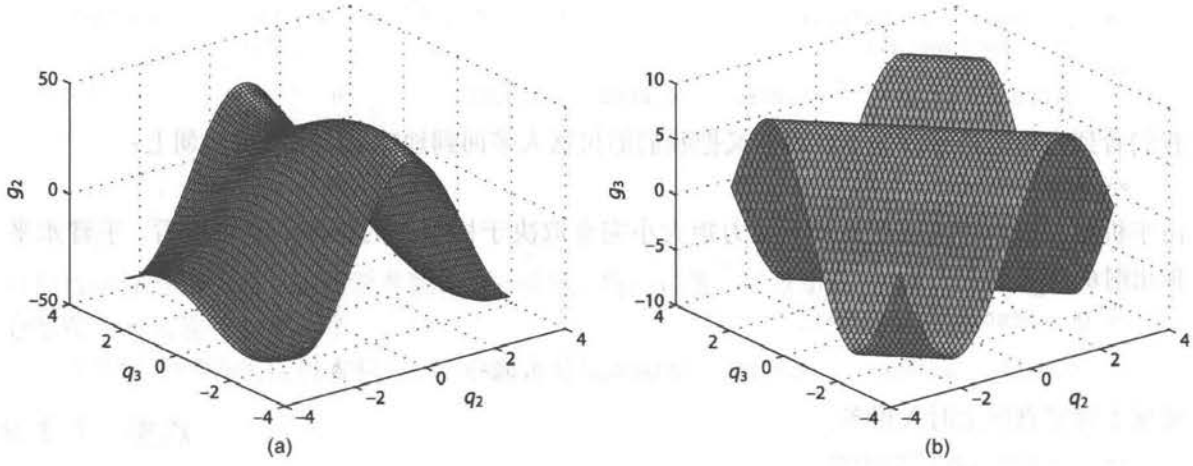


图 9.1 重力载荷随机械臂位姿的变化。(a)肩关节的重力载荷，$g_2(q_2, q_3)$；(b)肘关节的重力载荷 $g_3(q_2, q_3)$

我们可以使用以下简单(但慢而费时)的命令来研究一些惯量矩阵的元素，以及它们如何随机器人的位形而变化：

```
[Q2,Q3] = meshgrid(-pi:0.1:pi, -pi:0.1:pi);
for i=1:numcols(Q2),
    for j=1:numcols(Q3);
        M = p560.inertia([0 Q2(i,j) Q3(i,j) 0 0 0]);
        M11(i,j) = M(1,1);
        M12(i,j) = M(1,2);
    end
end
surfl(Q2, Q3, M11); surfl(Q2, Q3, M12);
```

背景材料：约瑟夫-路易斯·拉格朗日(Joseph-Louis Lagrange，1736—1813)是意大利出生的法国数学家和天文学家。他在分析学、数论、古典力学和天体力学领域都做出了重大贡献。1766 年，他继承欧拉(Euler)成为柏林普鲁士科学院数学院院长，他在那里住了 20 多年，做了大量的工作，并赢得多项法国科学院大奖。他的专著《分析力学》(*Mécanique Analytique*)在 1788 年首次出版，提供了自牛顿以来对于经典力学最全面的论述，为数学物理学在 19 世纪的发展奠定了基础。在 1787 年他成为法国科学院院士，1808 年成为巴黎综合理工学院第一位分析学教授，一名荣誉军团成员，以及帝国伯爵。他去世后被葬在巴黎的先贤祠。

其结果如图 9.2 所示。我们看到 $\boldsymbol{M}_{11}$ 的值变化显著，变化系数为

```
>> max(M11(:)) / min(M11(:))
ans =
    2.1558
```

这对机器人的设计很重要，因为对于一个固定不变的最大电机转矩，转动惯量设置了加速度上限，它反过来会影响路径跟踪精度。

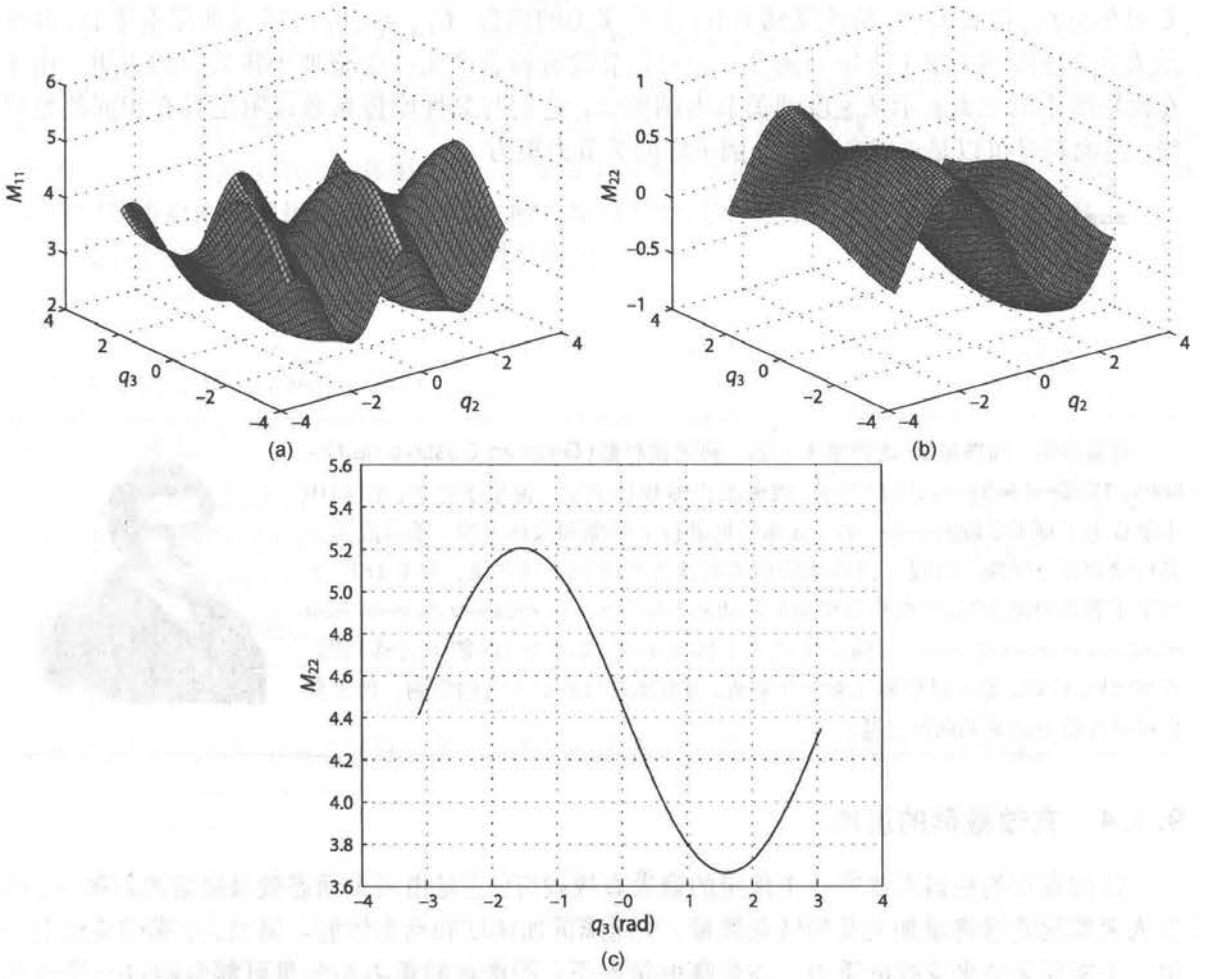


图 9.2　惯量矩阵元素随机械臂位姿的变化。(a)关节 1 的惯量随关节 2 和 3 角度的变化 $\boldsymbol{M}_{11}(q_2, q_3)$；(b)惯性积 $\boldsymbol{M}_{12}(q_2, q_3)$；(c)关节2的惯量随关节2和3角度的变化 $\boldsymbol{M}_{22}(q_2, q_3)$。惯量的单位均为 kgm^2

非对角项 $\boldsymbol{M}_{12}$ 代表了关节 2 的角加速度与关节 1 上力矩之间的耦合。也就是说，如果关节 2 加速，那么将有一个力矩被施加在关节 1 上，反之亦然。

9.1.3　科氏矩阵

$$\boldsymbol{Q} = \boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{F}(\dot{\boldsymbol{q}}) + \boldsymbol{G}(\boldsymbol{q}) + \boldsymbol{J}(\boldsymbol{q})^{\mathrm{T}}\boldsymbol{f}$$

科氏矩阵 $\boldsymbol{C}$ 是关节坐标和关节速度的函数。向心力矩和 $\dot{q}_i^2$ 成正比，而科氏力矩和 $\dot{q}_i\dot{q}_j$ 成正比。例如，在标准位姿下，机器人所有关节以 0.5 rad/s 的速度转动：

```
>> qd = 0.5*[1 1 1 1 1 1];
```

则科氏矩阵为

```
>> C = p560.coriolis(qn, qd)①
C =
    0.0000   -0.9115    0.2173    0.0013   -0.0026    0.0001
    0.3140   -0.0000    0.5786   -0.0011   -0.0001   -0.0000
   -0.1804   -0.1929   -0.0000   -0.0005   -0.0023   -0.0000
   -0.0002    0.0006   -0.0000   -0.0000    0.0003   -0.0000
   -0.0000    0.0000    0.0014   -0.0002   -0.0000   -0.0000
         0    0.0000    0.0000    0.0000    0.0000         0
```

非对角项 $C_{i,j}$ 代表关节 j 的速度到关节 i 上广义力的耦合。$C_{1,2} = -0.9115$ 是非常重要的，并表示关节 2 速度到关节 1 力矩的耦合——肩关节的旋转会产生一个施加于腰关节的力矩。由于在此矩阵中的元素表示从速度到关节力的耦合，它们与黏性摩擦系数或阻尼具有相同的数量级，然而符号可以是正或负。这个例子中的关节力矩为

```
>> C*qd'
ans =
   -0.3478
    0.4457
   -0.1880
    0.0003
    0.0006
    0.0000
```

背景材料：加斯帕德-古斯塔夫·德·科里奥利斯(Gaspard-Gustave de Coriolis，1792—1843)是法国数学家，机械工程师和科学家。他生于巴黎，在 1816 年他成为了理工学院的一名导师，在那里他进行了摩擦和液压实验，随后成为一名桥梁和道路学院的教授。他将动能和功的理念扩展到转动系统，并于 1835 年写下了著名的论文"关于机构系统相对运动的方程"(*Sur les équations du mouvement relatif des systëmes de corps*)。该论文论述了转动系统(如水车)的能量转移问题。在 19 世纪后期，他的思想被气象学界采纳，研究地球自转对天气的影响。他去世后被葬在巴黎的蒙帕纳斯公墓。

9.1.4 有效载荷的影响

任何真正的机器人都有一个规定的最大有效载荷，它是由两个动态效果决定的。第一，机器人末端的质量将增加关节的转动惯量，从而降低加速度和动态性能。第二，末端的质量会产生一个需要关节来支撑的重力。在最坏的情况下，所增加的重力矩分量可能会超过一个或多个电机的转矩额定值。即使没有超过额定值，它用于关节加速的力矩也将减小，而这又降低了动态性能。

作为一个示例，我们将对 Puma 560 机器人末端添加一个 2.5 公斤的重量，这是它额定的最大有效载荷。有效载荷的质心不能是手腕坐标系的中心，因为它在手腕内部，所以我们将有效载荷的质心沿手腕坐标系的 z 轴方向偏移 100 毫米。为此，我们通过修改机器人最后一个连杆的惯性参数来实现这个效果②：

```
>> p560.payload(2.5, [0, 0, 0.1]);
```

现在，在标准位姿下的惯量为

```
>> M_loaded = p560.inertia(qn);
```

① 这里假设最后一个连杆本身没有质量，这是一个合理的近似。

② 作者勘误：科氏矩阵错误，应通过 RTB 方法修正，因此之后的结果也是错误的，但 $C*q'$的结果是对的。——译者注

与先前计算的无有效载荷的惯量之比为

```
>> M_loaded ./ M;
ans =
    1.3363    0.9872    2.1490   49.3960   80.1821    1.0000
    0.9872    1.2667    2.9191    5.9299   74.0092    1.0000
    2.1490    2.9191    1.6601   -2.1092   66.4071    1.0000
   49.3960    5.9299   -2.1092    1.0647   18.0253    1.0000
   83.4369   74.0092   66.4071   18.0253    1.1454    1.0000
    1.0000    1.0000    1.0000    1.0000    1.0000    1.0000
```

我们看到矩阵对角元素值显著增加。例如，肘关节的转动惯量增加了66%，从而使最大加速度降低近2/3。加速度的下降必将损害机器人准确跟踪一条高速路径的能力。然而关节6的惯量没有受影响，因为这个增加的质量正好位于该关节的旋转轴线上。矩阵非对角项的值也显著增加，特别是在第4行和第5行，以及第4列和第5列。这表明，关节4和5，即腕关节托载着附加重物的偏摆运动，会产生很大的且能被其他机器人关节所感受到的反作用力。

关节上的重力载荷也会增加，比例系数为

```
>> p560.gravload(qn) ./ gravload
ans =
    0.3737    1.5222    2.5416   18.7826   86.8056       NaN
```

在肘关节和腕关节的增加尤为明显①。

9.1.5 基座力

一个运动着的机器人对基座也会施加一个力旋量，其中包括一个支撑机器人的垂直力以及手臂运动时产生的其他力和力矩。这个力旋量是 rne 方法的一个可选输出参数：

```
>> [Q,g] = p560.rne(qn, qz, qz);
```

这里 g 就是那个力旋量：

```
>> g'
ans =
         0   -0.0000  230.0445  -48.4024  -31.6399   -0.0000
```

它被作用到基座上并使之保持平衡。230 N 的垂直力是机器人的总重量，其质量为

```
>> sum([p560.links.m])
ans =
   23.4500
```

由于机器人的质心不在基坐标系的原点，因而还存在一个绕 x 轴和 y 轴的力矩。

在机器人不具有固定基座的情况下，该基座力是很重要。如基座是在太空中的卫星上、在一艘船上，在水下航行器上或者在拥有柔软减震的车辆上。

9.1.6 动态可操作性

在8.1.4节我们讨论了机器人的运动学可操作度，即为了实现特定方向上的运动，机器人所在位形的好坏程度。将这一概念延伸，我们要考虑机械臂在不同笛卡儿方向上实现加速的能力好坏。采用以前的一个类似方法，我们给出一个广义关节力的单位向量：

$$\boldsymbol{Q}^{\mathrm{T}}\boldsymbol{Q}=1$$

我们忽略方程(9.1)中的重力，且假设 $\dot{\boldsymbol{q}}=0$，得到

$$\boldsymbol{Q}=\boldsymbol{M}\ddot{\boldsymbol{q}}$$

对方程(8.2)求微分，且同样假设 $\dot{\boldsymbol{q}}=0$，则得到

① 可忽略其中的元素1，4和6，其结果是数值误差造成的。这些元素的原本结果应为0/0，因为无论有效载荷是否存在，关节重力载荷 $g_1=g_6=0$。

$$\dot{\boldsymbol{\nu}} = \boldsymbol{J}(\boldsymbol{q})\ddot{\boldsymbol{q}}$$

将上面 3 个式子组合，得到

$$\dot{\boldsymbol{\nu}}^{\mathrm{T}}(\boldsymbol{J}\boldsymbol{M}^{-1}\boldsymbol{M}^{-\mathrm{T}}\boldsymbol{J}^{\mathrm{T}})^{-1}\dot{\boldsymbol{\nu}} = 1$$

简化为

$$\dot{\boldsymbol{\nu}}^{\mathrm{T}}\boldsymbol{M}_x^{-1}\dot{\boldsymbol{\nu}} = 1$$

这是一个超椭球在笛卡儿加速度空间中的方程。例如，在标准位姿下：

```
>> J = p560.jacob0(qn);
>> M = p560.inertia(qn);
>> Mx = (J * inv(M) * inv(M)' * J');
```

如果只考虑平移加速度，也就是左上角 3×3 阶子矩阵 $\boldsymbol{M}_x$：

```
>> Mx = Mx(1:3, 1:3);
```

这是一个三维的椭球，将其绘制出来：

```
>> plot_ellipse( Mx )
```

如图 9.3 所示。该椭球的长轴方向与该位形机械臂具有最大加速度的方向一致。椭圆的半径是矩阵特征值的平方根：

```
>> sqrt(eig(Mx))
ans =
    0.4412
    0.1039
    0.1677
```

最大加速度的方向由第一个特征向量给出。最小和最大半径之比为

```
>> min(ans)/max(ans)
ans =
    0.2355
```

该比值表征了末端执行器加速度的非一致性。当各向加速能力相同时，该值为单位 1。在图示的情况下，机械臂在 x 和 z 方向的加速能力较好，但在 y 方向上的加速能力比较差。

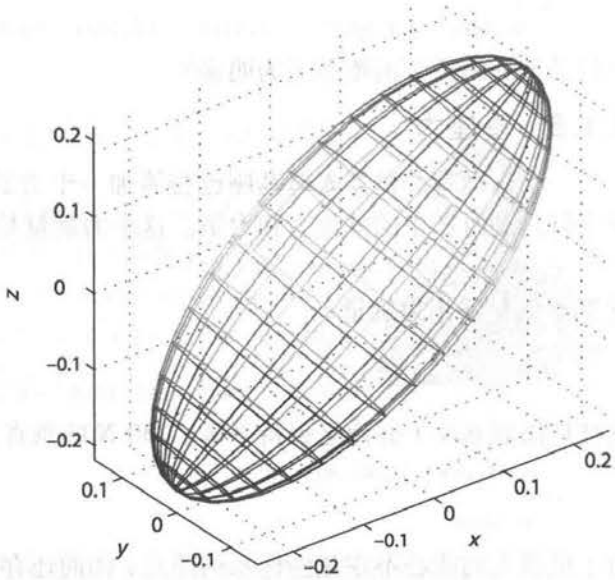


图 9.3　标准位姿下 Puma 560 机器人的空间加速度椭球

浅田(Asada)提出了一种相似的可操作度概念，但其中考虑了特征值的比率：

$$\ddot{\boldsymbol{x}}^{\mathrm{T}}\boldsymbol{J}^{-\mathrm{T}}\boldsymbol{M}\boldsymbol{J}^{-1}\ddot{\boldsymbol{x}} = 1$$

并获得一个一致性度量 $m \in [0,1]$，其中 1 表示在各个方向上的加速度是一致的。在这个示例中：

```
>> p560.maniplty(qn, 'asada')
ans =
    0.2094
```

9.2　传动系统

如今绝大多数机器人都是由电动机驱动的。无刷伺服电机通常用于大型工业机器人，而小型实验室或机器人爱好者更多使用直流有刷电机或步进电机。有的机器人具有非常大的有效载荷，可达数百公斤，它们一般会使用电动液压驱动器。

电动机具有紧凑和高效的特点，但产生不了非常大的转矩。然而，电机的输出转速可以非常高，因此一般要使用齿轮减速装置来降低转速并增加转矩。齿轮变速箱的缺点是增加了成本、重量、摩擦和机械噪声。许多高性能的机器人，如用于高速电子组装的机器人，会使用昂贵的大扭矩电机进行直接驱动，或使用皮带或薄金属带而不是齿轮来获得一个非常低的传动比。

图 9.4 是一个典型机器人关节传动系统的示意图。对于一个 $G{:}1$ 的减速传动，连杆处的扭矩应是电机扭矩的 G 倍。对于旋转关节，连杆处测量值(参照系 l)与电机处参考值(参考系 m)的关系如图 9.4 中的表格所示。例如，如果用手转动电机轴，就会感觉到通过变速箱传递来的负载惯性，但它与摩擦力一样减少到原来的 $1/G^2$。同理，如果用手转动载荷端的轴，会感觉到通过齿轮箱传来的电机惯性，但它与摩擦力一样增加 G^2 倍。

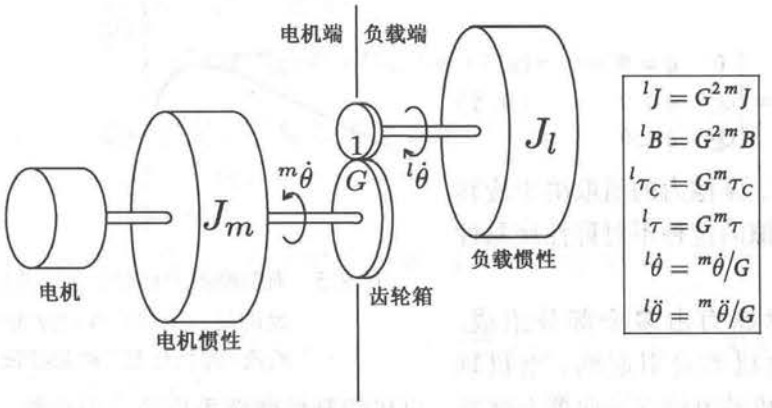


图 9.4　一个典型的机器人关节传动链示意图。显示了电机和负载惯量、齿轮箱和干扰力矩。电机端或负载端的参考系用左侧上标表示。右表显示负载和电机参考值之间的关系，齿轮传动比为 G

从电机看，其惯性由两部分组成。第一是由于电机本身的旋转部分，即转子引起的。它被表示为 J_m，是电机的固有特性，电机制造商在技术数据手册中提供了该值。第二是负载惯性 J_l，它是从动连杆以及连接到它的所有其他连杆的惯性之和。对于关节 j，它就是前面所讨论的机械臂惯性矩阵元素 M_{jj}，是关于机器人位形的一个函数。

因此，从电机上看，关节 j 的总惯量是

$$ {}^mJ_j = J_{m_j} + \frac{1}{G_j^2}M_{jj} \tag{9.3} $$

减速传动系统减小了其中第二项的重要性。由于该项是随机械臂位形变化的，这样就能减少关节总惯性的变化，这对于设计关节的控制系统是非常有益的。

但传动系统的消极方面是增加了黏性摩擦力和非线性效应，如在下一节将讨论的齿轮间隙和库仑摩擦力。另外，电机和关节之间的弹性联轴器和长传动轴，其作用相当于一个扭转弹簧，会引入复杂的动力学，这将在 9.4.4 节中讨论。

9.2.1　摩擦

$$ \boldsymbol{Q} = \boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q},\dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{F}(\dot{\boldsymbol{q}}) + \boldsymbol{G}(\boldsymbol{q}) + \boldsymbol{J}(\boldsymbol{q})^{\mathrm{T}}\boldsymbol{f} $$

对于大多数电机驱动机器人，摩擦是继重力后另一个占主导作用的关节力[①]。对于任何转动机械装置，如电机或齿轮箱，其摩擦力矩相对于速度的特性具有类似图 9.5 的曲线形式。速

① 对于 Puma 机器人，前 3 个关节的摩擦力矩范围为电机最大转矩的 10% ~47%[Corke 1996b]。

度为零时，我们观察到静摩擦力，随后施加的力矩必须超过静摩擦力才能引发转动——这个过程称为克服静摩擦力。

一旦机器运动起来静摩擦力会迅速减小，黏性摩擦力便会占主导地位。黏性摩擦力，如图 9.5 中的虚线所示，常采用下面的数学模型：

$$Q_f = B\dot{q} + Q_C \tag{9.4}$$

其中，斜率 B 代表黏性摩擦系数，偏移量 Q_C 是库仑摩擦力。后者通常被建模为如下非线性函数：

$$Q_C = \begin{cases} 0 & \dot{q} = 0 \\ Q_C^+ & \dot{q} > 0 \\ Q_C^- & \dot{q} < 0 \end{cases} \tag{9.5}$$

一般情况下，摩擦力的值取决于旋转方向，但库仑摩擦的这种不对称性比黏性摩擦更加明显。

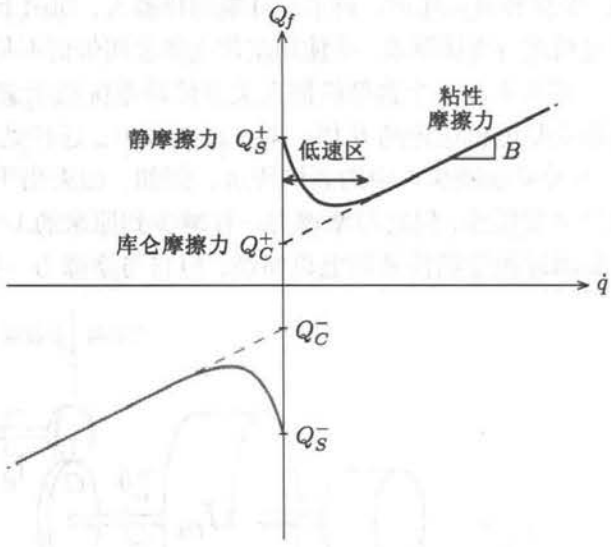


图 9.5 典型摩擦力相对于速度的特性。虚线描绘的是一个简单线性摩擦模型，分为斜线段（黏性摩擦）和截线段（库仑摩擦）

电机上的摩擦力由多个部分组成。第一个是由于电机本身引起的：电机轴承，以及有刷电机的电刷在换向器上摩擦。电机的黏性摩擦系数是一个常数，一般会在厂商的数据手册中提供。但库仑摩擦力的信息一般不提供。其他部分的摩擦则是由齿轮箱和支持连杆的轴承引起的。

背景材料：查尔斯-奥古斯丁·库仑（Charles-Augustin de Coulomb，1736—1806）是法国物理学家。他出生在昂古莱姆一个富裕的家庭，并在四国联合大学的皮埃尔·查尔斯·莫尼尔手下学习数学，后来又在梅济耶尔军事学校学习。他在马提尼克待了 8 年，参与了波旁要塞的建设，但在那里他感染了热带病。

后来，他在罗什福尔的船厂工作，在那里开展了滑动面的静态和动态摩擦实验。他的论文“机器的简单理论”荣获 1781 年科学学院的大奖。他后来研究了电磁学和静电学，最著名的是以他的名字命名的静电力公式，并作为电荷的国际标准单位。法国大革命后，他参与了制定度量衡的新系统。

工具箱在 Link 对象内为摩擦建模。各种摩擦都被建模为集中参数，并以电机为参考，即作用于齿轮箱的电机一端。黏性摩擦是一个标量，且对正负转速都一样①。库仑摩擦是一个二维向量，其包含（Q_C^+，Q_C^-）。例如，Puma 机器人第二个连杆的动态参数：

```
>> p560.links(2).dyn
l =
 theta=q, d=0, a=0.4318, alpha=0 (R,stdDH)
  m     = 17.400000
  r     = -0.363800 0.006000 0.227500
  I     = | 0.130000 0.000000 0.000000 |
          | 0.000000 0.524000 0.000000 |
          | 0.000000 0.000000 0.539000 |
```

① 在实际应用中，一些机构具有随速度变化的摩擦特性。

```
Jm    = 0.000200
Bm    = 0.000817
Tc    = 0.126000(+) -0.071000(-)
G     = 107.815000
```

最后三行分别显示了黏性摩擦系数、库仑摩擦系数以及齿轮传动比。有关 Link 的在线文档详细介绍了如何设置这些参数。

9.3 正向动力学

要根据施加到关节的力和力矩确定机械臂的运动，我们就需要正向动力学或积分动力学。重新排列方程(9.1)，可以得到关节加速度：

$$\ddot{q} = \boldsymbol{M}^{-1}(q)\left(\boldsymbol{Q} - \boldsymbol{C}(q,\dot{q})\dot{q} - \boldsymbol{F}(\dot{q}) - \boldsymbol{G}(q)\right) \tag{9.6}$$

在给定关节坐标、关节速度和所施加关节力矩的条件下，关节加速度可以用 SerialLink 对象的 accel 方法计算得到：

```
>> qdd = p560.accel(q, qd, Q)
```

该功能也封装在 Simulink® 的 Robot 模块中，其应用的一个示例为

```
>> sl_ztorque
```

如图 9.6 所示。施加给机器人的力矩为零，初始关节角被设定为 Robot 模块的一个参数，这里为“零角度位姿”。运行该仿真：

```
>> r = sim('sl_ztorque');
```

关节角作为随时间变化的函数被返回到 r 对象中：

```
>> t = r.find('tout');
>> q = r.find('yout');
```

我们用动画演示机器人的运动：

```
>> p560.plot(q)
```

可以看到它在重力作用下崩溃，因为没有力矩来对抗重力以保持机器人直立。肩关节和肘关节都会垂下并来回摆动，同时腰关节因为科氏耦合力会产生转动[①]。由于黏性摩擦会消耗能量，该运动将慢慢衰退。

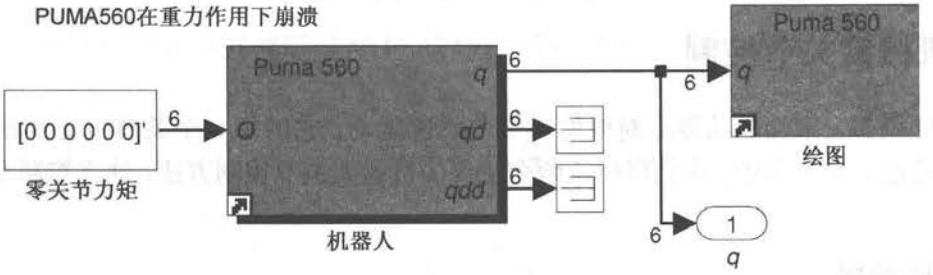


图 9.6 Puma 560 机器人在零关节力矩状态的 Simulink® 模型 sl_ztorque

我们还可以绘制出随时间变化的关节角度：

```
>> plot(t, q(:,1:3))
```

① 这些运动在真正的机器人上是不可能的。

如图 9.7 所示。fdyn 方法可以被当作 Simulink 的一个无图形替代方法，它在在线文档中有描述。

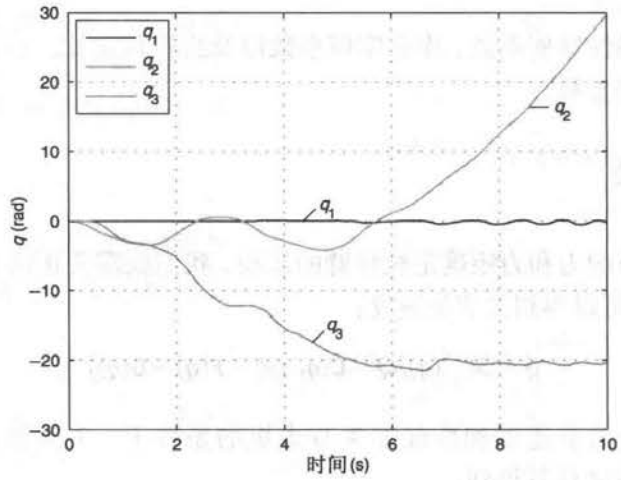


图 9.7 Puma 560 机器人在重力作用下崩溃的关节角度轨迹(从 qz 开始)

但这个示例相当不现实。在实际应用中，关节力矩会作为机器人真实和期望关节角的某个函数被计算。下一节将讨论这个话题。

库仑摩擦力具有很强的非线性，因此当使用数值积分程序解决正向动力学时可能会带来麻烦，通常表现为积分时间很长。固定时间步长的求解器往往适应能力更强，而它们可以通过在 Simulink® 的 Simulation + Configuration Parameters 菜单项进行选择。

使用 mdl_puma560 定义的默认 Puma 560 模型中，每个关节都具有非零的黏性和库仑摩擦力参数。但有时让机器人所有的摩擦参数为零也是很有用的，这可以通过以下指令实现：

```
>> p560_nf = p560.nofriction();
```

它返回机器人对象的一个副本，该副本除了库仑摩擦力为零外，在其他所有方面都与原对象相同。另外，还可以设置库仑和黏性摩擦系数都为零：

```
>> p560_nf = p560.nofriction('all');
```

9.4 机械臂关节控制

为了让机器人末端驱动器沿期望的笛卡儿轨迹运动，它的每一个关节也必须跟随特定的关节空间轨迹。在本节中，我们将讨论两个主要的机器人关节控制方法：独立控制和基于模型的控制。

9.4.1 驱动器

大多数实验室和工业机器人是用电机驱动的。电机可以通过电流或电压控制，这里考虑电流控制的情况①。假设电机驱动器或放大器提供的电流为

---

① 电流控制是通过电子恒流源，或一个带有实际电机电流反馈的可变电压源来实现的。对于后者，由于有电阻和电感的存在，因此必须考虑电机的电动力学。可变电压源通常由一个脉冲宽度调制(PWM)的开关电路实现。

$$i = K_a u$$

它与所施加的控制电压 u 是线性相关的，其中 K_a 是放大器的跨导，单位为 A/V。由电机产生的转矩与电流成正比：

$$\tau = K_m i$$

其中，K_m 是电机的扭矩常数，单位为 Nm/A。电机动力模型由下式描述：

$$J_m\dot{\omega} + B\omega + \tau_c(\omega) = K_m K_a u \tag{9.7}$$

其中，J_m 是方程(3.9)中电机的总惯量，B 是黏性摩擦系数，τ_c 是库仑摩擦力矩。

9.4.2　独立关节控制

机器人关节控制的一种常见方法是将每个关节看作一个独立的控制系统，让每个关节精确地跟随各自的关节角度轨迹。然而，我们应当看到在关节上存在各种干扰力矩，如重力、速度和加速度耦合以及摩擦，它们使控制变得很复杂。

嵌套的控制回路是一种很常见的控制结构。外环是负责保持关节位置，并确定使位置误差最小的关节速度；内环是负责保持外环所要求的关节速度。

速度环。我们将首先学习内部速度环，我们将用 Puma 560 机器人的肩关节作为示例，因为它的参数是众所周知的，见表 9.1。

表 9.1　Puma 560 机器人肩关节的电机和驱动器参数[Corke 1996b]

参数	符号	参数值	单位
电机力矩常数	K_m	0.228	N mA^{-1}
电机惯性	J_m	200×10^{-6}	kg m^2
黏性摩擦系数	B_m	817×10^{-6}	Nms rad^{-1}
库仑摩擦力矩	τ_C^+	0.126	Nm
	τ_C^-	−0.709	Nm
齿轮传动比	G	107.815	
最大力矩	τ_{max}	0.900	Nm
最大速度	$\dot{q}_{max}$	165	rad s^{-1}

忽略库仑摩擦力，我们写出方程(9.7)的拉普拉斯变换：

$$sJ\Omega(s) + B\Omega(s) = K_m K_a U(s)$$

其中，$\Omega(s)$ 和 $U(s)$ 分别是时域信号 $\omega(t)$ 和 $u(t)$ 的拉普拉斯变换。将上式改写为一个线性传递函数：

$$\frac{\Omega(s)}{U(s)} = \frac{K_m K_a}{Js + B}$$

由方程(9.3)得到的有效惯量为

$$J = J_m + \frac{1}{G^2} M_{22} \tag{9.8}$$

如图 9.2 所示，M_{22} 随着机械臂位姿而显著变化，所以这里将采用它的平均值 2 kg/m^2，得到总惯量为

$$J = 200 \times 10^{-6} + \frac{2}{(107.815)^2} = 200 \times 10^{-6} + 172 \times 10^{-6} = 372 \times 10^{-6} \text{ kg m}^2$$

需要注意的是连杆惯性与电机本身的惯性相当。

控制系统的 Simulink 模型如图 9.8 所示。1 毫秒的延迟用来模拟速度控制算法的计算时间，一个饱和器用来模拟电机所能提供的最大扭矩。我们使用了一个基于要求速度与实际速度误差的比例控制器，用于计算给电机驱动器的控制命令：

$$u^* = K_v(\dot{q}^* - \dot{q}) \tag{9.9}$$

电机速度通常是通过在每个采样时间比较电机的位置差异来计算的，而位置是由编码器来测量的。

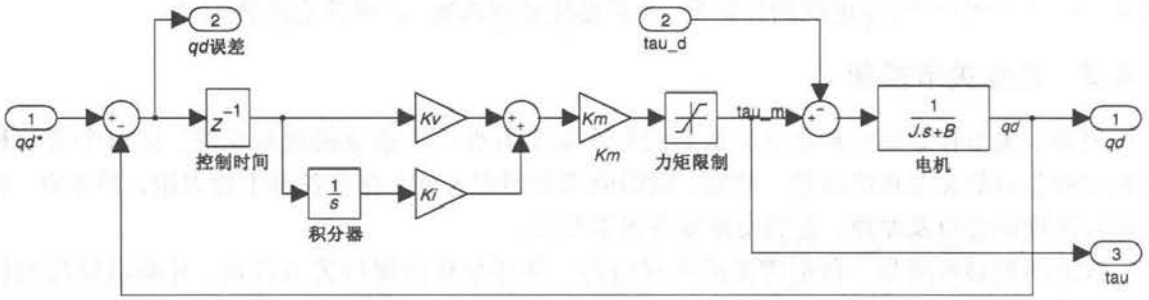


图 9.8 速度控制环，Simulink ® 模型 vloop

为了测试这个速度控制器，我们创建了一个测试器：

```
>> vloop_test
```

其要求速度为一锯齿波形，如图 9.9 所示。运行这个仿真器：

```
>> sim('vloop_test');
```

经过一些尝试，我们发现增益 $K_v = 1$ 可以得到令人满意的性能，如图 9.10 所示。在阶跃处[①]有一些超调，但是较小的增益会导致速度误差的增加，较大的增益又会导致振荡——在控制工程中，这是一个需要权衡的问题。

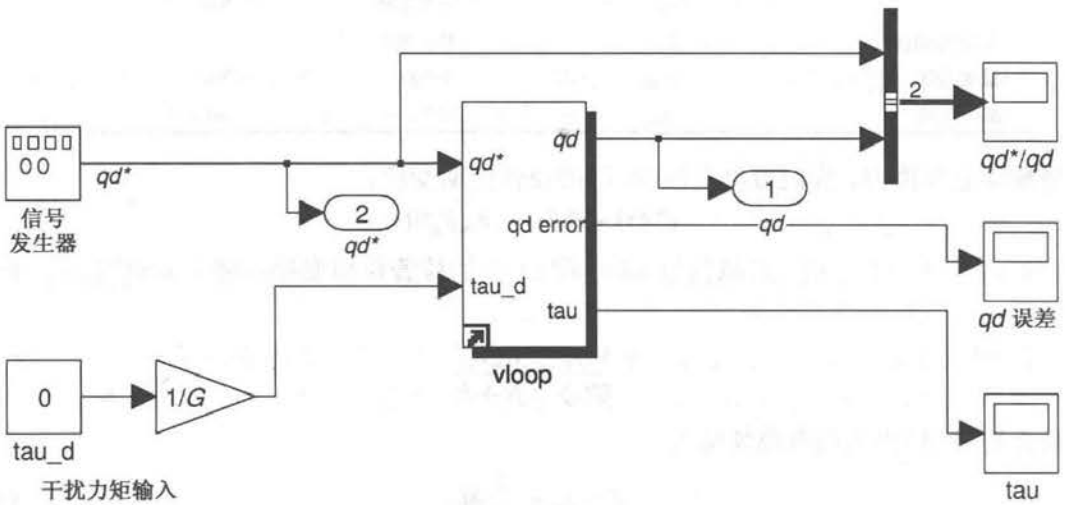


图 9.9 用于速度控制环的测试器，Simulink ® 模型 vloop_test。输入 tau_d 被用来模拟作用在关节上的干扰力矩

到目前为止，我们一直忽略了机器人手臂一个很重要的动力作用——重力。图 9.1(b) 显

① 虽然阶跃响应是一个常用的衡量控制性能的方法，然而实际应用中速度环肯定不会收到一个阶跃要求信号。

示重力矩变化范围从约 -40 Nm 至 40 Nm。现在我们在负载处增加一个干扰力矩，其值为变化范围的一半，只有 20 Nm。我们编辑这个测试器，并重新运行仿真器。结果如图 9.11 所示，可以看出控制性能已经严重退化——跟踪误差增加至超过 1 rad/s。

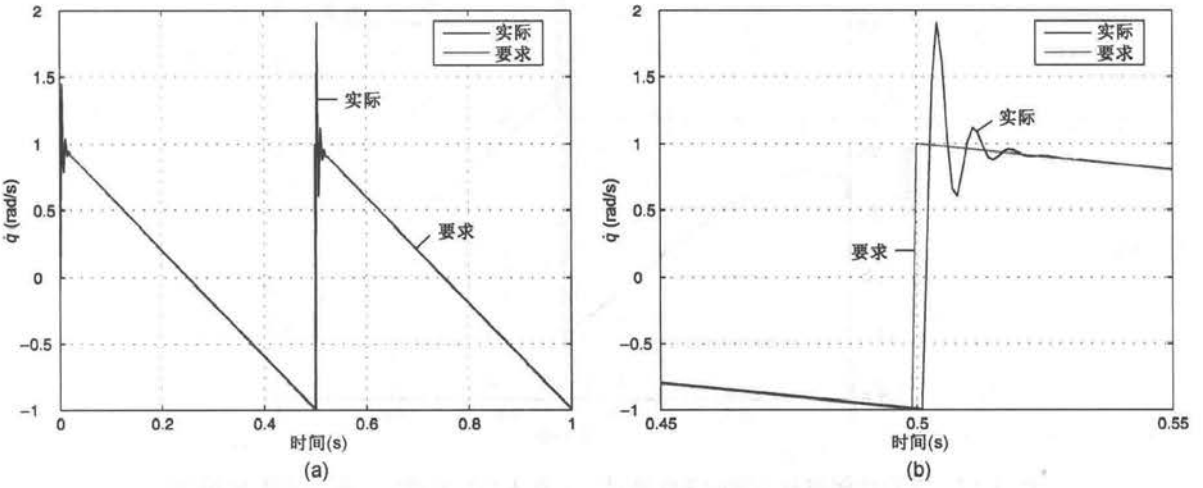


图 9.10　带有锯齿要求信号的速度环。(a) 响应；(b)响应的特写

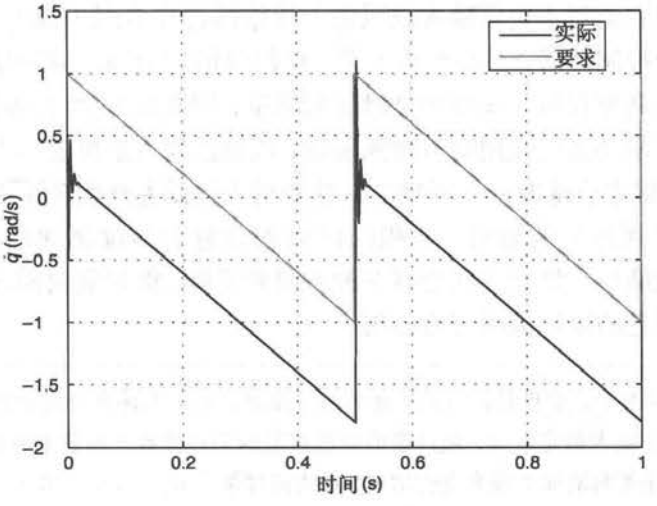


图 9.11　速度环对一个锯齿信号的响应，重力干扰为 20 Nm

有 3 种常见的策略来应对这种误差。最简单的是提高增益，它能减少跟踪误差，但会使系统不稳定。

第二个策略是增加积分作用，它常用于工业电机驱动。我们将方程(9.9)改变为一个比例-积分控制器：

$$u^* = \left(K_v + \frac{K_i}{s}\right)(\dot{q}^* - \dot{q}),\ K_i > 0$$

在图 9.8 所示的 Simulink® 模型中，可以通过设置 `Ki` 为一个非零值来实现。经过一些尝试，我们发现增益 $K_v = 1$ 和 $K_i = 10$ 时系统运行效果较好，其性能如图 9.12 所示。积分器状态随时间不断进化，并逐渐消除了干扰项，我们可以看到误差逐渐衰减到零。在实际应用中，干扰也随着时间变化，积分器跟踪干扰的能力取决于积分增益 K_i 的值。在现实中，其他干扰

也会影响关节控制，例如库仑摩擦力和由于速度和加速度耦合产生的力矩。该控制器需要经过非常精细的调试，才能使这些干扰对跟踪性能的影响最小。

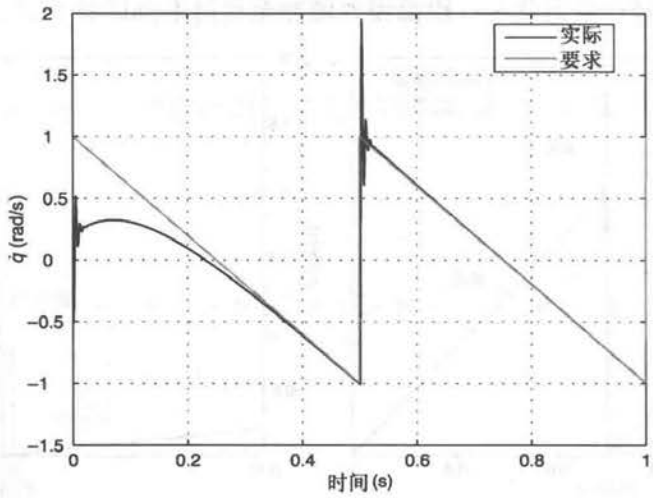


图 9.12 带有比例积分控制的速度环对一个锯齿信号的响应，重力干扰为 20 Nm

从传统控制系统的角度来看，最初的速度环不包含积分块，意味着它属于0类型系统。这类系统的一个特点是它们对于恒定输入或恒定干扰始终存在有限的误差，就像我们从图 9.11 观察到的。添加积分控制器后，系统变为 1 类，它们对恒定的输入或恒定干扰表现为零误差。工程中总是存在一些利弊权衡。积分项会增加超调量，因此 K_i 的增加通常需要减少 K_p 来进行一些补偿。如果关节驱动器已被用到其性能极限，例如达到力矩极限，此时由于电机的加速能力低于所需，则跟踪误差会随时间不断增加。这个增大的误差随积分又不断积累，从而引起积分饱和。当关节最后到达目的地时，累积的巨大积分量会继续驱动电机运动，直至积分衰减——将导致大的超调量。针对此问题有多种不同的策略，例如限制积分器的最大值，或者只允许积分在电机接近它的设定值时开始作用。

材料背景：电机限制因素。电机有转矩和转速的双重限制。其最大转矩是由驱动电路所能提供的最大电流确定的。电机也有一个最大额定电流，超过该值电机可能因为过热或永磁铁退磁而被损坏，这将不可逆地降低其转矩常数。随着速度的增加摩擦力也会增加，最大速度值为 $\omega_{max} = \tau_{max}/B$。

电机的转矩和转速的乘积是其机械输出功率，并且也有一个上限值。电机可以短时间地承受一些过载、峰值功率和峰值扭矩，但持续的性能明显低于高峰。

策略 1 和 2 统称为干扰抑制，主要考虑如何减少一个未知干扰的影响。但是，如果考虑实际的机器人，重力干扰并非是未知的。在 9.1.1 节中展示了如何计算作用于每个关节的重力力矩。如果我们知道该重力矩，还有电机的转矩常数，则可以把它添加到 PI 控制器的输出①。因此，第三个减少干扰的影响的策略就是预测干扰并消除它—— 一种被称为力矩前馈控制的策略。我们在如图 9.8 所示的框图中增加了一个前馈项，即图 9.13 中的虚线框所围部分。

控制设计中最后要考虑的是惯性变化如何影响闭环响应。使用方程(9.8)和图 9.2(c)中的数据，我们发现关节惯量的最小值和最大值分别是 320×10^{-6} kg/m^{-2}和 450×10^{-6} kg/m^{-2}。

① 即使重力负载的数值不准确，前馈控制也可以减少干扰的幅度。

如果仍然使用上面选择的控制增益，图 9.14 中显示了最小和最大的惯量下的速度跟踪误差。我们可以看到在大惯量的情况下跟踪误差衰减变慢，同时也显示了最小惯量下的不稳定性。在实际应用中，我们需要选择增益以优化两个极端情况下的闭环性能。

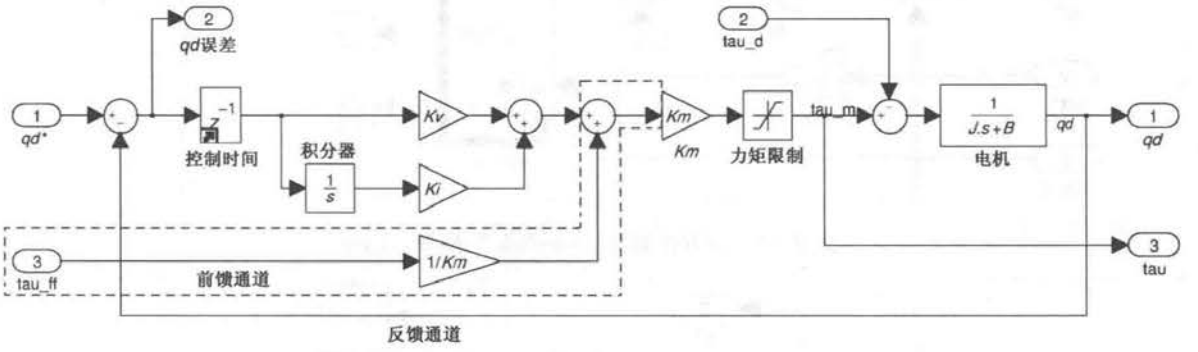


图 9.13　带有前馈(虚线框所围部分)的速度控制环，Simulink ® 模型 vloop_test2

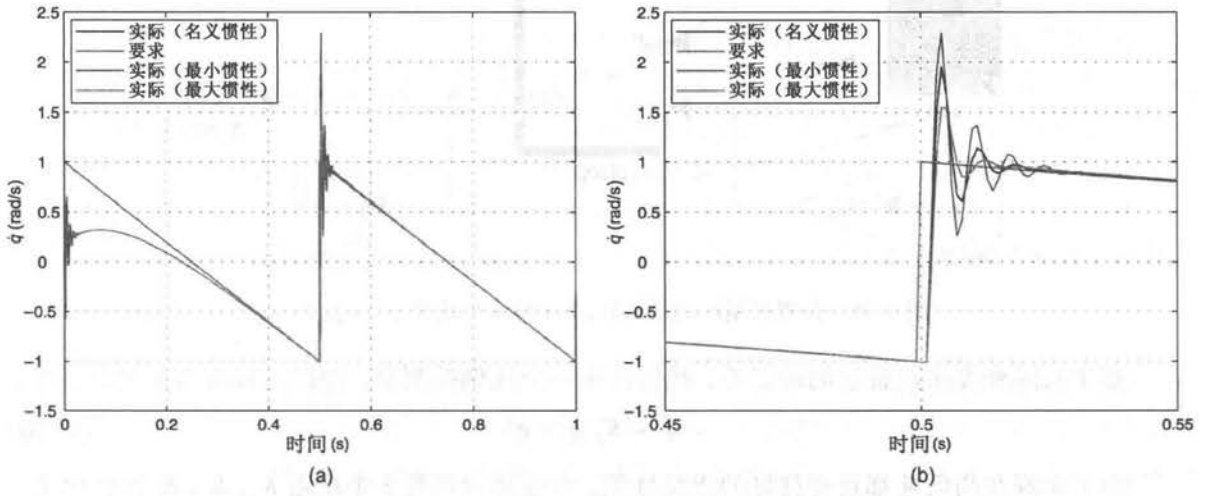


图 9.14　具有锯齿信号和积分作用的速度环，干扰力矩为 20 Nm，惯量 M_{22} 不定。(a) 响应曲线；(b) 响应曲线特写

不仅是机械臂，电机速度控制对于所有类型的机器人都是很重要的。例如在第 4 章中，它被用于控制轮式机器人的车轮速度和一个四旋翼飞行器的转子速度。

补充材料：反电动势。旋转中的电动机就像一台发电机，会产生一种称为反电动势(EMF)的电压 V_b 以抵抗流入电机的电流。反电动势与电机速度成正比 $V_b = K_m\omega$，其中 K_m 为电机的转矩常数，单位为 Vs/rad。当该电压等于驱动器电路可以提供的最大电压时，就没有更多的电流可以流入电机，电机转矩便下降到零。这实际上提供了一个电机转速上限。

位置环。外环负责保持关节位置，它的 Simulink ® 模型如图 9.15 所示。位置误差为内环提供速度要求。

为了测试位置控制环，我们创建了另一个测试器：

```
>> ploop_test
```

位置需求来自一个 LSPB 轨迹发生器，该发生器在 1 s 时间内从 0 运动到 0.5 rad，采样速度为 1000 Hz。图 9.16 所示的测试器也可以在速度环内注入干扰力矩。

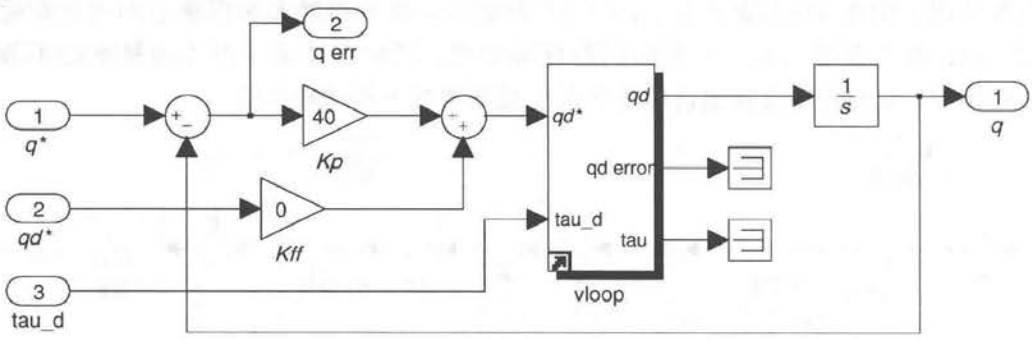


图 9.15 位置控制环，Simulink ® 模型 ploop

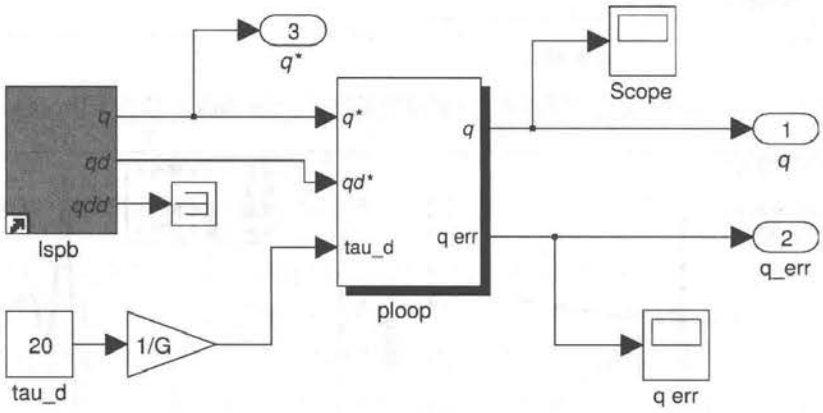


图 9.16 位置控制环的测试器，Simulink ® 模型 ploop_test

基于实际和要求位置之间的误差，我们使用一个比例控制器，用以计算所需的电机转速：

$$\dot{q}^* = K_p(q^* - q) \tag{9.10}$$

为了实现在高低速都具有良好的跟踪性能，需要通过调整 3 个增益 K_p，K_v，K_i 来优化关节控制器。在 K_p = 40 时控制器跟踪和误差性能如图 9.17(a)所示。要求曲线和实际曲线之间的误差是由于速度环的误差积分(具有角度单位)引起的。

位置环采用了基于误差反馈的这种经典方法。但比例控制有一个经常被忽视的特点，当误差为零时给控制系统的要求信号也变为零。在本示例中，零误差就意味着给速度内环的速度要求为零。然而为了实现方程(9.10)中非零关节速度的要求，就需要误差非零，而这当然与理想的精确跟踪相矛盾。幸运的是，LSPB 轨迹函数不仅计算位置，同时也计算随时间变化的速度。如果知道了速度，我们可以将其添加到比例控制回路的输出，即速度环的输入处——该策略称为速度前馈控制。带前馈的时间响应曲线如图 9.17(b)所示，我们看到跟踪误差被大大降低了。

让我们回顾一下所学到的独立关节控制。一种常见的结构是嵌套控制回路。内环使用比例或比例积分控制产生一个力矩，使实际速度紧密跟随要求的速度。外环采用比例控制生成要求的速度，这样实际位置就能紧密跟踪要求的位置。由于重力和其他动态耦合作用而产生的干扰力矩会影响速度环的性能，而这反过来会引起位置跟踪的误差。被控系统中其他参数的变化也会产生这种影响。齿轮箱(传动装置)减少干扰力矩的量值为 $1/G$，减少惯性和摩擦力变化的量为 $1/G^2$，但带来的不足是增加了成本、重量、摩擦和机械噪声。

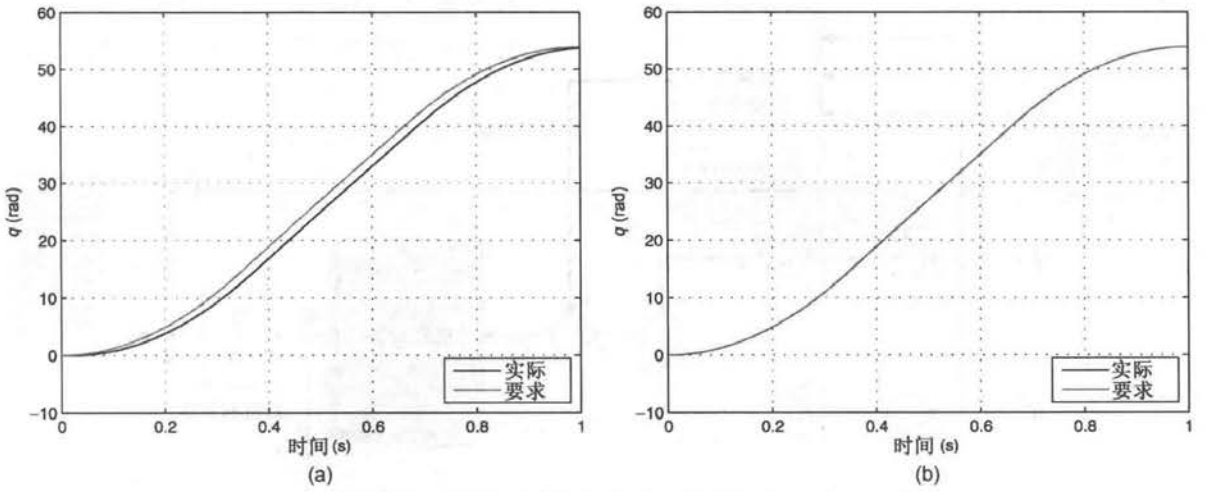


图 9.17　跟踪一个 LSPB 轨迹的位置环。(a) 只有比例控制；(b) 比例控制加上期望速度的前馈

通过添加一个积分控制项，或通过可预测干扰力矩的前馈控制，可以提高速度环的性能。位置环的性能也可以通过所需关节速度的前馈控制来提高。在实际应用中，控制系统一般同时采用前馈和反馈控制。前馈用于注入可以计算出的信号，在本示例中即为关节速度，在上一个示例中为重力力矩。反馈控制补偿所有其余的误差源，包括机械臂位姿和有效载荷引起的惯性变化，随时间和温度变化的摩擦力，以及由于速度和加速度耦合引起的所有干扰力矩。一般情况下，前馈的使用可以让反馈增益减小，因为现在有很大一部分的要求信号来自于前馈。

9.4.3　刚体动力学补偿

前一节我们看到了独立关节控制的局限性，并介绍了重力干扰力矩的前馈补偿概念。惯性变化和动态耦合力并没有明确的处理方法，而是交给了反馈控制器来处理。然而，给定关节角度、关节速度和加速度，转动惯量和耦合力矩就可以根据方程(9.1)计算出。可以将这些力矩包含进我们的控制算法中，采用两个基于模型的方法：前馈控制和计算力矩控制。两种方法在结构上的差异如图 9.18 所示。

9.4.3.1　前馈控制

图 9.18(a)所示的力矩前馈控制器由下式给出：

$$\boldsymbol{Q}^* = \underbrace{\boldsymbol{M}(\boldsymbol{q}^*)\ddot{\boldsymbol{q}}^* + \boldsymbol{C}(\boldsymbol{q}^*, \dot{\boldsymbol{q}}^*)\dot{\boldsymbol{q}}^* + \boldsymbol{F}(\dot{\boldsymbol{q}}^*) + \boldsymbol{G}(\boldsymbol{q}^*)}_{\text{前馈}} + \underbrace{\left\{\boldsymbol{K}_v(\dot{\boldsymbol{q}}^* - \dot{\boldsymbol{q}}) + \boldsymbol{K}_p(\boldsymbol{q}^* - \boldsymbol{q})\right\}}_{\text{反馈}}$$
$$= \mathcal{D}(\boldsymbol{q}^*, \dot{\boldsymbol{q}}^*, \ddot{\boldsymbol{q}}^*) + \left\{\boldsymbol{K}_v(\dot{\boldsymbol{q}}^* - \dot{\boldsymbol{q}}) + \boldsymbol{K}_p(\boldsymbol{q}^* - \boldsymbol{q})\right\}$$

其中，$\boldsymbol{K}_p$ 和 $\boldsymbol{K}_v$ 分别为位置和速度增益(或阻尼)矩阵，$\mathcal{D}(\cdot)$ 是逆动力学函数。增益矩阵通常是对角矩阵。前馈项提供了期望机械臂状态 $(\boldsymbol{q}^*, \dot{\boldsymbol{q}}^*, \ddot{\boldsymbol{q}}^*)$ 所需的关节力，反馈项补偿由于惯性参数的不确定性、未建模的力或外部干扰引起的任何误差。

我们也可以认为，前馈项对作业点 $(\boldsymbol{q}^*, \dot{\boldsymbol{q}}^*, \ddot{\boldsymbol{q}}^*)$ 的非线性动力学进行了线性化。如果线性化是理想的，则误差 $\boldsymbol{e} = \boldsymbol{q}^* - \boldsymbol{q}$ 的动力学方程由下式给出：

$$\boldsymbol{M}(\boldsymbol{q}^*)\ddot{\boldsymbol{e}} + \boldsymbol{K}_v\dot{\boldsymbol{e}} + \boldsymbol{K}_p\boldsymbol{e} = 0 \tag{9.11}$$

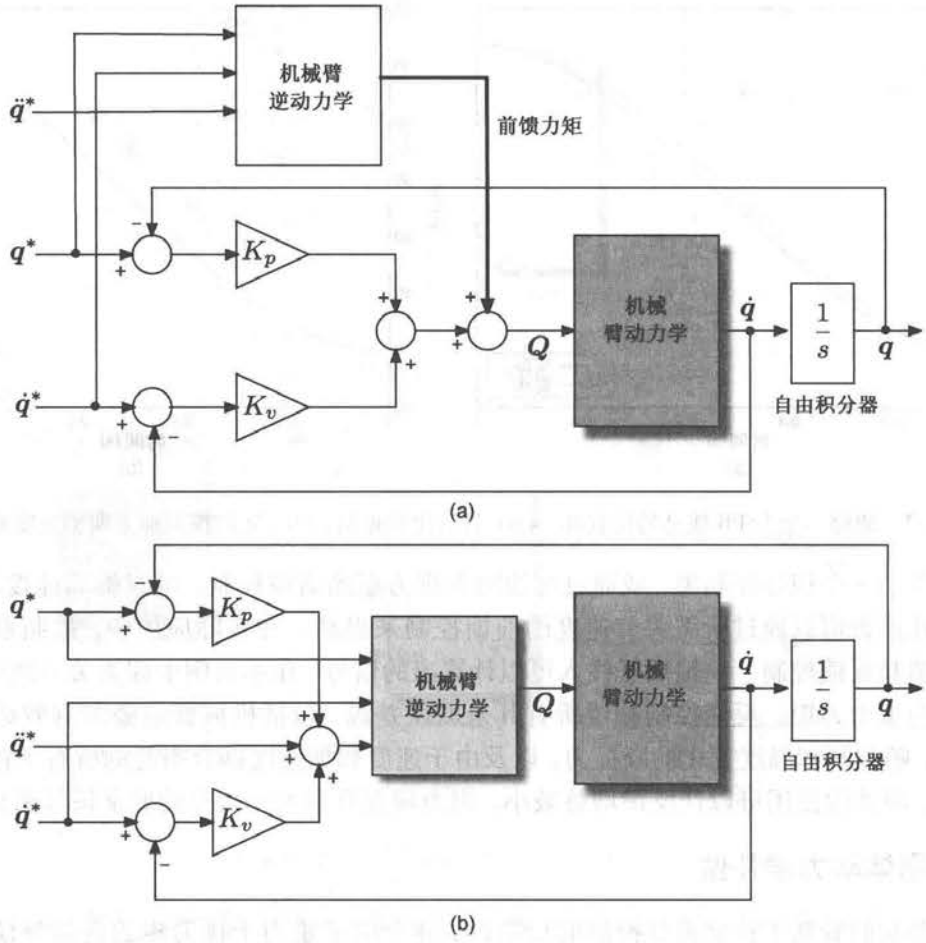


图 9.18 机械臂的控制结构。(a)前馈控制；(b)计算力矩控制

对于精心选择的 $\boldsymbol{K}_p$ 和 $\boldsymbol{K}_v$，误差将衰减到零，但关节误差是耦合的[①]，且它们的动力学依赖于机械臂的位形。

为了使用 Simulink® 测试这个控制器，我们首先创建一个 SerialLink 对象：

```
>> mdl_puma560
```

然后载入如图 9.19 所示的力矩前馈控制器模型：

```
>> sl_fforward
```

前馈力矩使用 REN 模块计算，然后将其加入用位置和速度误差计算出的反馈力矩中。期望的关节角度和速度使用 jtraj 模块生成。

由于机器人位形的变化相对缓慢，因此前馈力矩的采样计算频率 T_{ff} 相对于误差反馈回路力矩的采样计算频率 T_{fb} 可以更慢。图 9.19 中的零阶保持模块就采用了相对较低的 20 Hz 采样计算频率。

9.4.3.2 计算力矩控制

计算力矩控制器如图 9.18(b)所示。它属于一类称为逆动力控制的控制器。其原理是：

① 由于非对角矩阵 $\boldsymbol{M}$。

非线性系统通过它的逆被级联，从而使整个系统具有恒定的单位增益。在实际应用中，求逆是不完美的，所以需要一个反馈回路来处理误差。

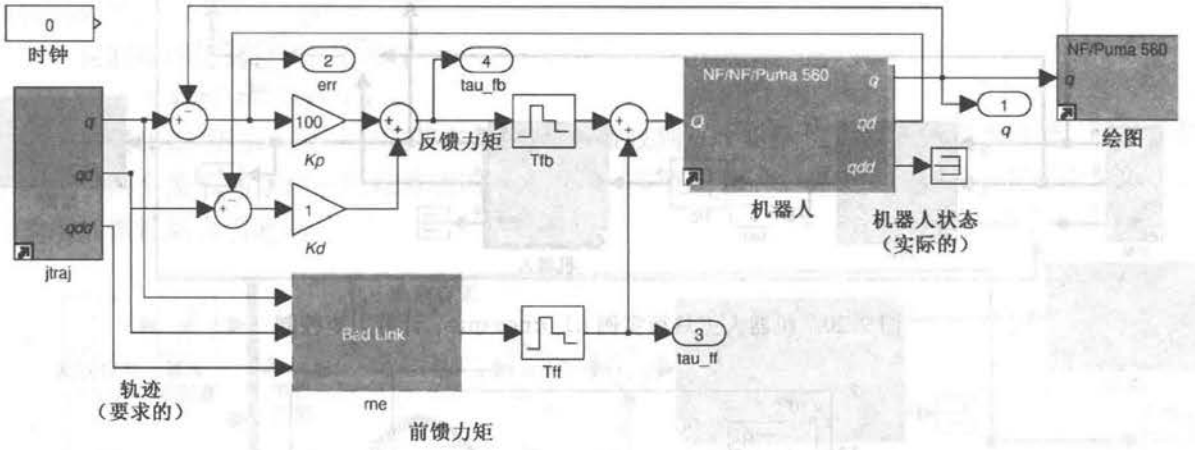


图 9.19　带力矩前馈控制的 Puma 560 机器人，Simulink ® 模型 sl_fforward。具有阶梯形符号的模块是零阶保持模块

计算力矩控制由下式给出：

$$\begin{aligned}\boldsymbol{Q} &= \boldsymbol{M}(\boldsymbol{q})\left\{\ddot{\boldsymbol{q}}^* + \boldsymbol{K}_v(\dot{\boldsymbol{q}}^* - \dot{\boldsymbol{q}}) + \boldsymbol{K}_p(\boldsymbol{q}^* - \boldsymbol{q})\right\} + \boldsymbol{C}(\boldsymbol{q},\dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{F}(\dot{\boldsymbol{q}}) + \boldsymbol{G}(\boldsymbol{q}) \\ &= \mathcal{D}\left(\boldsymbol{q},\dot{\boldsymbol{q}},\left(\ddot{\boldsymbol{q}}^* + \boldsymbol{K}_v(\dot{\boldsymbol{q}}^* - \dot{\boldsymbol{q}}) + \boldsymbol{K}_p(\boldsymbol{q}^* - \boldsymbol{q})\right)\right)\end{aligned}$$

其中，$\boldsymbol{K}_p$ 和 $\boldsymbol{K}_v$ 分别为位置和速度增益（或阻尼）矩阵，$\mathcal{D}(\cdot)$ 是逆动力学函数。

在本示例中，逆动力学必须在每个伺服间隔期进行采样计算，而系数矩阵 $\boldsymbol{M}$，$\boldsymbol{C}$ 和 $\boldsymbol{G}$ 的采样计算频率可以较慢，因为机器人位形的变化相对缓慢。在理想的建模和参数化假设条件下，系统的动力学误差为

$$\ddot{\boldsymbol{e}} + \boldsymbol{K}_v\dot{\boldsymbol{e}} + \boldsymbol{K}_p\boldsymbol{e} = 0 \tag{9.12}$$

其中，$\boldsymbol{e} = \boldsymbol{q}^* - \boldsymbol{q}$。与力矩前馈控制器不同，这里的关节误差都是非耦合的，因此其动力特性相对于机械臂位形是独立的。如果有模型误差存在，那么会有一些轴之间的耦合，方程（9.12）的右侧会变成一个非零的力函数。

使用 Simulink ®，我们首先创建一个 SerialLink 对象，再去除库仑摩擦力，然后加载计算力矩控制器：

```
>> mdl_puma560
>> p560 = p560.nofriction();
>> sl_ctorque
```

如图 9.20 所示。力矩由工具箱的 rne 模块计算，然后加到由位置和速度误差计算出的反馈力矩中。期望的关节角度和速度是由 jtraj 模块生成的，其参数是初始和最终的关节角度。我们运行仿真：

```
>> r = sim('sl_ctorque');
>> t = r.find('tout');
>> q = r.find('yout');
```

跟踪误差如图 9.21 所示。

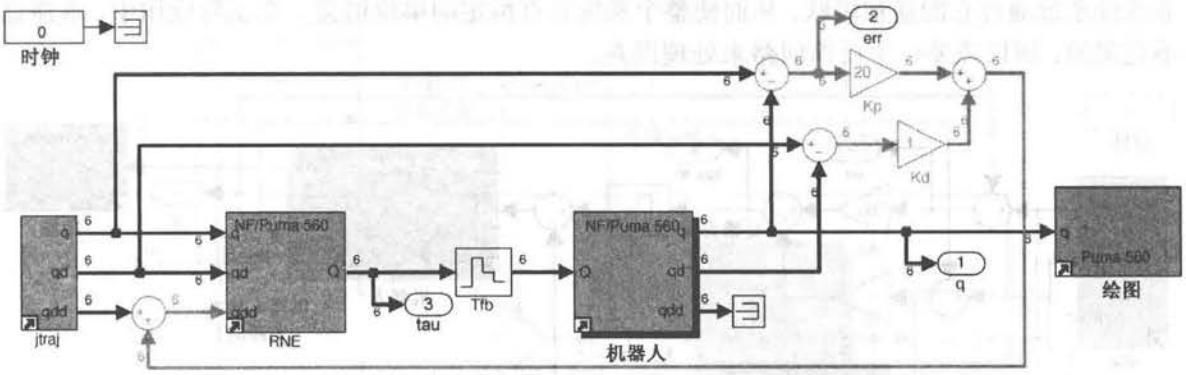


图 9.20 机器人工具箱实例 sl_ctorque，计算力矩控制

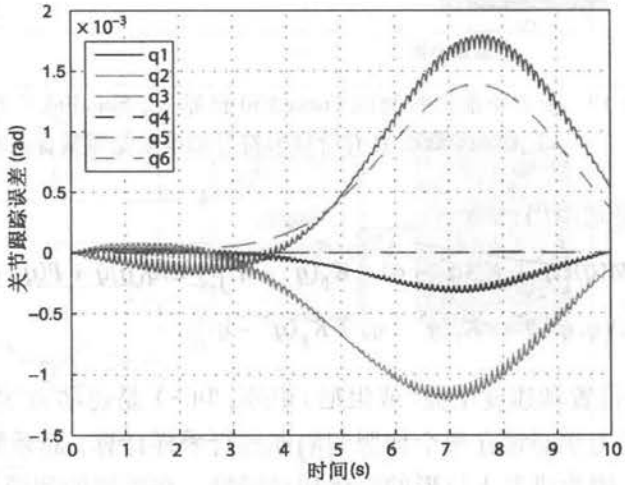


图 9.21 计算力矩控制：前三个关节的跟踪误差

9.4.4 柔性传动

对于许多高性能机器人，电机和连杆之间的传动柔性是一个重要的动力学参数。导致这种柔性的因素很复杂，可能是电机与轴之间的弹性耦合器产生的，也有驱动轴本身的扭转刚度，或者是绳驱动机器人上缆绳的纵向刚度引起的。这种柔性的动力学效果相当于在电机和负载之间增加一个小阻尼弹簧。

补充材料：串联弹性驱动器(SEA)。到目前为止，我们都把电机和负载之间的柔性看作是一种不好的干扰，但在许多情况下它却有一个益处：有一类串联弹性驱动器就刻意在电机和负载之间引入一个柔性弹簧。这对于机器人与人之间的亲密互动是相当有利的，因为它降低了机器人与人发生碰撞时的危险性。对于那些在作业任务中必须对外施加一个力的机器人，通过弹性件的变形还能估计出施加力的大小，进而用于机器人控制。

我们将使用 7.2.1 节中介绍的两连杆机器人来进行演示。其 Simulink® 模型为

```
>> mdl_twolink
>> sl_flex
```

如图 9.22 所示。这个柔性驱动将通过电机角 q_m 和连杆角 q_l 进行建模。作用于连杆的力矩与弹簧常数和阻尼系数的关系为

$$\tau_l = K(q_m - q_l) + B(\dot{q}_m - \dot{q}_l)$$

为简单起见，假设电机和它的控制环都具有无穷大的刚性，即弹簧的反作用力矩不影响电机的角度。

我们运行仿真：

```
>> r = sim('sl_flex')
```

结果如图 9.23 所示。第一个关节在 1s 的时刻接收到一个阶跃位置改变的要求信号，而且我们看到了两个关节都存在一些显著超调。请注意，关节 2 有一个初始的负误差，它是由关节 1 加速度的惯性耦合引起的。

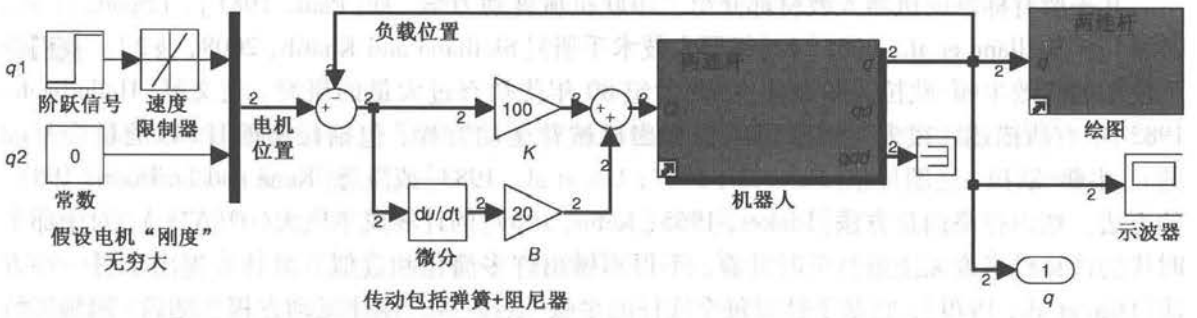


图 9.22　机器人工具箱实例 sl_flex，简单柔性两连杆机械手

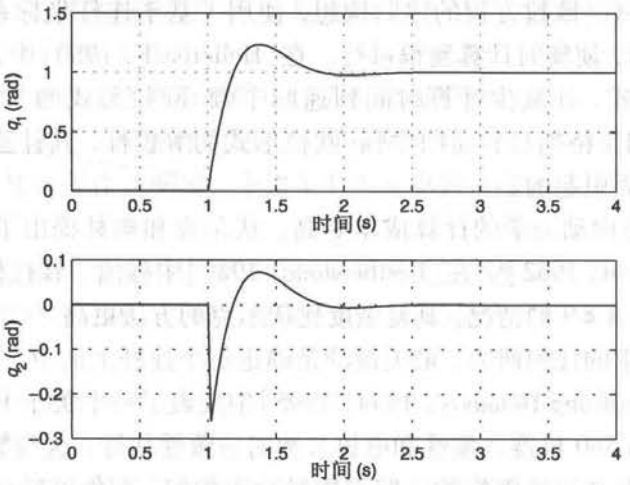


图 9.23　柔性连杆控制：关节角对阶跃输入的响应

要有效地控制该系统，需要测量电机和连杆角度，或电机角和每个关节的弹簧长度。高性能机器人一个更复杂的现象是连杆的柔性，而它不能使用工具箱的刚性连杆假设来建模。

9.5　本章总结

在本章中，我们学习了如何对作用于串联机械臂各个连杆上的力和力矩建模。运动方程组或逆动力学方程组用于计算要实现特定关节角度、速度和加速度所需要的关节力。方程组的各项分别对应惯性、重力、速度耦合、摩擦力和外作用力。我们探讨了上述各项的意义，以及它们是如何随机械臂位形变化的。通过研究运动方程组，我们对一些重要问题有了深刻了解，如一个关节的速度或加速度如何对其他关节施加一个干扰力，这对控制设计是很重要的。

然后，我们讨论了正向动力学，其描述了机器人位形是如何随着驱动器和外力（如重力）施加在关节上的力和力矩而变化的。

然后，我们讨论了控制方法，以最简单的独立关节控制开始，并探讨了干扰力矩和惯性变化的影响。我们展示了如何通过干扰（如重力）的前馈来显著提升控制性能。我们将前馈概念扩展到完全基于模型的控制（使用力矩前馈）和计算力矩控制器。最后，我们讨论了机器人电机和连杆之间柔性的影响。

扩展阅读

几乎所有标准的机器人教材都介绍了串联机械臂动力学，如[Paul, 1981]，[Spong et al., 2006]，[Siciliano et al., 2008]和《机器人技术手册》[Siciliano and Khatib, 2008, §2]。我们今天使用的高效牛顿-欧拉递归方法在 20 世纪 80 年代初有过大量的研究，在文献[Hollerbach, 1982]中有所描述。很多方法都可以推导出机械臂运动方程，包括拉格朗日（以能量为基础的）、牛顿-欧拉、达朗贝尔[Fu et al., 1987; Lee et al., 1983]或凯恩[Kane and Levinson, 1983]的方法。然而拉格朗日方法[Uicker, 1965; Kahn, 1969]的计算成本巨大（$O(N^4)$），对于那个时代的计算机来说无法进行实时计算，不得不做出许多简化和近似。奥林等提出了另一种方法[Orin et al., 1979]，它基于针对每个连杆的牛顿-欧拉（NE）刚体运动方程。随后，阿姆斯特朗展示了递归的牛顿-欧拉法[Armstrong, 1979]，其计算复杂性为 $O(N)$。卢等[Luh et al., 1980]提供了另一种牛顿-欧拉方程的递归构想，使用了基于连杆坐标系的线性角速度，让计算时间提高了 1000 倍，使实时计算变得可行。在[Hollerbach, 1980]中，展示了递归方法如何应用到拉格朗日的形式，并减少计算时间到递归牛顿-欧拉形式的 3 倍以内。而在[Silver, 1982]中则展示了递归拉格朗日和递归牛顿-欧拉形式的等价性，其计算效率的差异是由于角速度的表示方法不同而引起的。

9.3 节中介绍的正向动力学的计算成本更高。沃尔克和奥林提出了一个复杂度为 $O(N^3)$ 的方法[Walker and Orin, 1982]。在[Featherstone, 1987]中提出了铰接体方法，其复杂度只有 $O(N)$，但对于关节数 $N<9$ 的情况，其复杂度比沃尔克的方法更高。

对于机器人动力学的任何研究，最关键的是确定每个连杆上的 10 个惯性参数以及电机参数。在[Corke and Armstrong-Hélouvry, 1994, 1995]中发表了一个关于 Puma 参数的基础研究，提供了一个对于 Puma 560 机器人惯性和电机参数的一致性估计，这些数据中有些是通过分解机器人及测量部件质量和尺寸确定的。假设质量分布均匀，部件的转动惯量可以通过质量及其尺寸估计出来，或者也可以用一个双线摆来测量。

另一种参数估计方法是通过测量机器人移动时的关节力矩或基座反作用力和力矩。在这方面，早期的成果包括[Mayeda et al., 1990]，[Izaguirre and Paul, 1985]，[Khalil and Dombre, 2002]，较近的则是[Siciliano and Khatib, 2008, §14]。参数成功识别的关键是该机器人运动的方式，一定要有足够的力度[Gautier and Khalil, 1992; Armstrong, 1989]。摩擦是一个重要的动力学特性，在[Armstrong, 1988]论文中有很好的描述。而[Armstrong-Hélouvry et al., 1994]则是关于摩擦力建模和控制的一篇易读且研究透彻的文献。在[Corke, 1996a]中描述了一种用于测量电机电气参数（如力矩常数和电枢电阻）的技术，无需从机器人上拆下电机。

本书关于控制的讨论相当简短，并极力强调前馈控制的优势。有关机器人关节控制技术的研究可参看[Spong et al., 2006]，[Craig, 2004]和[Siciliano et al., 2008]，并在[Siciliano and Khatib, 2008, §6]中进行了总结。[Siciliano et al., 2008]及[Klafter et al., 1989]中对驱动器

和传感器进行了很好的论述。[Spong et al., 2006]中讨论了关节机器人的柔性控制。自适应控制可以用来处理随时间变化的惯性参数，这一研究课题的相关文献很多，一些早期的好文献包括克雷格的书[Craig, 1987]，以及论文[Craig et al., 1987]，[Spong, 1989]，[Middleton and Goodwin, 1988]和[Ortega and Spong, 1989]。

在[Spong et al., 2006]和[Siciliano et al., 2008]中讨论了动态可操作性。有关工具箱中使用的 Asada 可操作度可参看文献[Asada, 1983]。牛顿定律是用拉丁文写的，其英文翻译版可从该网站获得：http://www.archive.org/details/newtonspmathema00newtrich。他的其他著作，包括研究笔记，可以在网上找到：http://www.newtonproject.sussex.ac.uk。

习题

1. 运行第189页上的代码，计算关节2和3上随机器人位形变化的重力力矩。在末端添加一个有效载荷后重复运行。
2. 运行第190页上的代码，展示关节1和2的惯性如何随有效载荷变化。
3. 生成图9.2(c)所示的曲线。添加一个有效载荷，然后比较结果。
4. 在关节角度范围内，惯性的变化因子是多少？
5. 为什么机械臂的惯量矩阵是对称的？
6. 当机器人运动时，会对其基座施加一个力旋量(9.1.5节)。假设机器人在一个无摩擦的水平表面上(比如在一个大的冰球上)。创建一个包括机器人手臂动力学和在桌上滑动动力学的仿真模型，然后展示出手臂运动使机器人在桌面平移和旋转。你能设计出一个手臂运动，使得机器人基座从一个位置移动到另一个指定位置，然后停下吗？
7. 在机器人显示上叠加可操作性椭球，并和8.1.4节中的速度椭球进行比较。
8. 独立关节控制(9.4.2节)。
   a) 研究不同 K_p 和 K_i 值以及要求信号的形状和幅度对控制性能的影响。
   b) 执行 `vloop` 的根轨迹分析，以确定比例控制器下的最大允许增益。在PI控制器的情况下重复此分析。
   c) 假设电机由一个电压源而不是电流源来控制，并且电机的阻抗是1 mH和1.6 Ω。相应地修改 `vloop`。扩展该模型使其包含反电动势的影响。
   d) 增加要求的运动速度，使电机的转矩变成饱和。随着积分作用，你会观察到一个积分饱和的现象——检查运动过程中积分器的状态发生了什么。采用不同策略解决这个问题，例如限制积分器的最大值，或者当电机接近它的设定值才允许积分器工作。试验一下这些策略。
   e) 创建一个Puma机器人的Simulink®模型，其每个关节由 `vloop` 和 `ploop` 控制。机器人上不同电机的参数参见[Corke and Armstrong-Hélouvry 1995]。
9. 电机转矩系数的单位为Nm/A，它等于电机反电动势常数，单位为Vs/rad。证明这两个单位是等价的。
10. 基于模型的控制(9.4.3.1节)。
    a) 计算并显示力矩前馈控制和计算力矩控制的关节跟踪误差。采用不同的运动、不同的控制参数和不同的采样频率 T_b 进行实验。
    b) 减小前馈力矩采用计算的频率，并观察其对跟踪误差的影响。

c)在实际应用中，我们不能知道机器人完全准确的动力学模型，我们只能对刚体动力学的最佳估计求逆。在仿真中，可以使用 perturb 方法来对它建模(请参阅在线文档)，该方法返回一个机器人对象，具有按指定百分比加减变化的惯性参数。修改 Simulink® 模型，让 RNE 模块使用一个具有 10% 参数扰动的机器人模型。这意味着逆动力学计算所采用的动力学模型与受控机器人的模型稍有不同，并能展示出模型误差对控制性能的影响。分别研究力矩前馈控制和计算力矩控制情况下误差的影响。

11. 柔性驱动机器人(9.4.4 节)。

a)用不同关节刚度和控制参数开展实验。

b)修改仿真，去除掉电机无限大刚性的假定。也就是说，用一个嵌套的控制环代替电机位置，该控制环受到来自于弹性元件的一个干扰力矩的影响。调节这个控制器以获得良好的性能。

c)引入一个刚性对象到环境中，然后修改仿真，让机器人手臂运动去触摸该刚体对象，并使接触力不超过 5 N。

第四部分

计算机视觉

第10章　光与色彩
第11章　图像形成
第12章　图像处理
第13章　图像特征提取
第14章　使用多幅图像

视觉是从图像中发现周围环境里有什么以及在哪里的一个过程。

——戴维·马尔

几乎所有的动物都有眼睛——事实上通过多次进化，眼睛的形态和功能已是多种多样。图 IV.1 中是自然界中各种不同的眼睛：虻的复眼，蜘蛛的主眼和副眼，扇贝基于反射器的研究，以及人类基于透镜的眼睛。脊椎动物一般有两只眼睛，而蜘蛛和扇贝有很多只眼睛。

即使很简单的动物，如蜜蜂，其大脑中只有 10^6 个神经元(人类有 10^{11} 个)，它们也能用眼睛完成一些复杂且性命攸关的任务，如寻找食物并带其返回蜂巢[Srinivasan and Venkatesh 1997]。当然，拥有一只眼睛也需要很高的生物成本：复杂的眼睛自身，驱动眼睛的肌肉，保护眼睛的眼睑和泪管，以及一个很大的处理相关数据的视觉皮层。

根据我们自己的经验，眼睛是用于识别、导航、避障及操作的最有效传感器。摄像机模仿了眼睛的功能，我们希望用摄像机为机器人创造出一些基于视觉的能力——用数字图像去识别环境中的物体并实现导航。图 IV.2 中展示了一台带有不同摄像机的机器人。

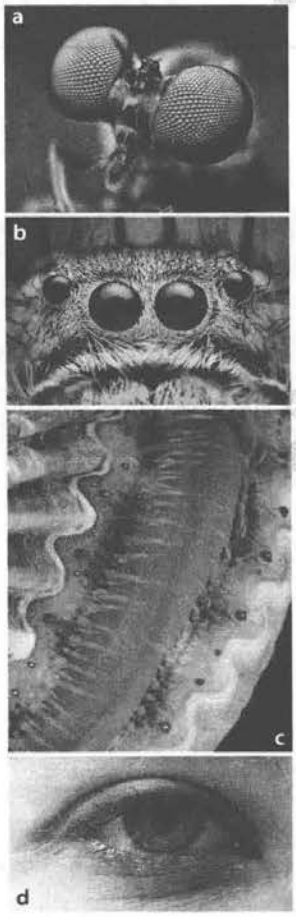


图 IV.1 (a)一只食虫虻，Holocephala Fusca；(b)跳蛛，Phidippus Putnami((a)和(b)的照片由Thomas Shahan提供，thomasshanan.com)；(c)扇贝(照片由Sonke Johnsen提供)，每个深色小点就是一只眼睛；(d)人眼

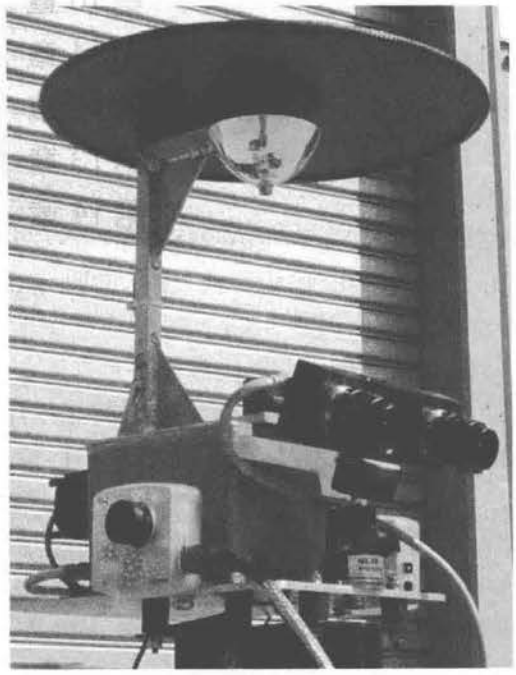


图 IV.2 一台室外移动机器人上的一组摄像机：向前看的一对立体摄像机，侧面的广角摄像机，以及头顶的全景摄像机反光镜(CSIRO移动机器人)

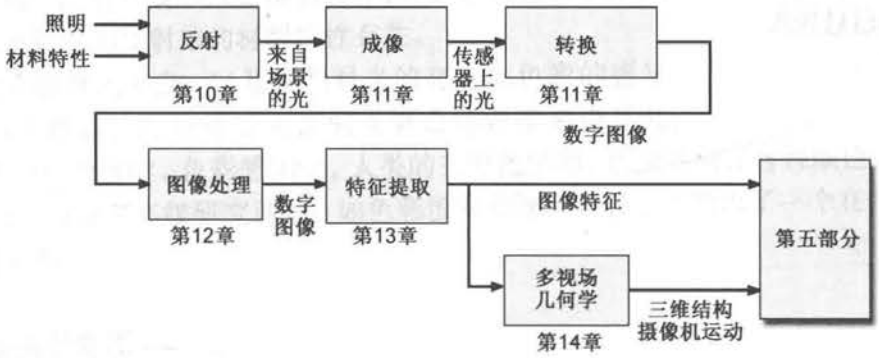
技术进步已经使机器人用摄像机来充当眼睛变得可行。计算机视觉的历史可以追溯到20世纪60年代，但在很长一段时间里电子摄像机既娇气又昂贵，而且计算机的处理能力也不足。今天，我们已经开发出了最早用于手机的CMOS摄像机，其价格仅有几美元，而且个人计算机也拥有了大规模并行处理能力。新的算法、便宜的感应器以及强大的计算能力已经使摄像机成为当今一种实用的传感器。

在第一章里，我们将机器人定义为：

一种能感知、规划和行动的目标导向机器。

本书的这一部分将涉及采用视觉的感知，即视觉感知。无论机器人是在工厂里还是在野外作业，它都需要感知其周围的环境，进而规划出相应的行为。

在本书的这一部分，我们将讨论视觉处理的全部技术：从投射到一个场景上的光线，到光线被反射，然后被一个透镜捕捉，再到被转变为数字图像，以及被各种算法处理，进而提取出支持前述机器人各种能力所需的信息。视觉处理技术的步骤如图IV.3所示。



图IV.3　图像处理过程的步骤

背景材料：眼睛的进化。人们普遍相信，所有动物的眼睛都来自于5亿4千万年前进化成的一种原型眼。然而眼睛的主要进化过程则是发生在近几百万年间。最早的眼睛被称为眼点，它们是单细胞动物体内一些简单的感光蛋白点。多细胞的动物进化出了多细胞眼点，它们能感受光强，但不能分辨其方向。逐渐地，眼点进化为一种浅杯形状，这样根据不同眼点细胞的感光情况，眼睛就具有了一定的分辨光线方向的能力。随着浅杯凹底变深，杯口变窄，以及感光细胞数量的增加，形成了一种能分辨形状的针孔摄像机。然后出现了保护眼点的透明细胞的过度生长，并形成了一个充满的眼腔，进而最终形成我们今天熟知的眼睛。透镜型眼睛在物种间经历了7次不同的独立演变。自然界则进化出了10种完全不同的眼睛，包括图IV.1所示的几种。

在第10章中我们从探讨光线开始，然后重点讨论色彩，因为它是我们观测的世界中一个非常重要的特征。虽然我们在幼儿园就知道了各种色彩，但其复杂性使我们很难对其有完全正确的认知。然后在第11章中我们讨论一幅环境图像如何在传感器上形成，以及怎样被转换为能够被计算机处理的数字图像。基本的图像处理算法将在第12章中介绍，这些算法是在第13章中要讨论的图像特征提取算法的基础。特征提取主要是数据压缩的问题，要从大量像素数据中提取出图像场景的本质特征。例如，我们怎样确定场景中一个圆形红色物体的坐标？要描述这个坐标可能只需要4字节，而场景的图像数据则有几百万字节。为解决这个问题，需要先解决多个重要的子问题，如“什么是红色”、“如何区分红色像素与

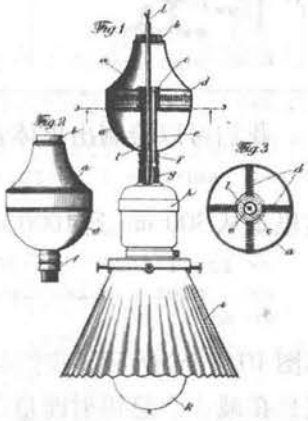
非红色像素”、“我们如何描述这些红色像素的形状”、“如果红色物体不只一个怎么办”，等等。

随着各章内容的逐渐深入，我们会发现只用一个摄像机来观察周围世界有较大的局限性。生物学再次给我们以启示——用多只眼更普遍，而且功能更强大。这引导我们去研究使用多视图来描述周围环境，用一台移动摄像机或多台摄像机从不同角度去观看一个场景。这部分内容将在第 14 章中讨论，它对于理解周围环境的三维结构特别重要。另外，这些内容还为本书最后一部分研究如何将视觉用于闭环控制机械臂或移动机器人打下了基础。

第10章 光与色彩

我无法假装对各种色彩都一视同仁。那些明亮的颜色更能使我愉悦，我也真心地对那些可怜的灰暗色彩感到抱歉。

温斯顿·丘吉尔



在古代，人们认为眼睛会辐射出一个混合着世界上可见物体的视流锥体，它会使观察者产生一种感觉，就像触觉那样，这就是所谓的外射理论。今天我们认识到，从一个发光体发出的光投射到场景上，然后一部分光反射到观察者的眼中，从而产生对那个场景的感知。反射回眼中或者摄像机中的光线与投射到场景上的光照度以及被称为反射率的材料特性有关。

本章内容涉及光本身，以及我们对光的亮度和色彩的感知。10.1 节分别从电磁辐射和连续光谱的光混合体角度来描述光。10.2 节简要介绍色度学、色彩感知学、人类的三原色感知，以及如何在各种颜色空间中表示色彩。10.3 节涉及很多高级研究课题，如色彩恒常和伽玛校正，并给出了一个在图像中区分不同颜色物体的例子。

10.1 光的谱表示

在1670年左右，艾萨克·牛顿先生发现白光是由几种颜色的光混合而成的。现在我们知道其中的每一种颜色都是拥有单一频率或波长的电磁波。我们知道的波长在400～700 nm的颜色如图10.1所示。

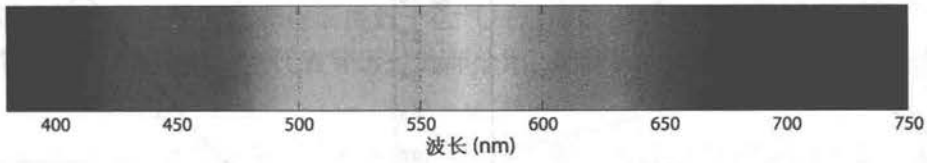


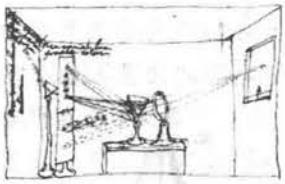
图10.1 纳米级波长的可见光光谱。可见光波长范围取决于观察条件和观察者个体，但通常认为是400～700 nm。波长大于700 nm的为红外线，波长在400 nm以下的为紫外线

通常，我们见到的光都是含有多个波长的混合光，可以用函数 $E(\lambda)$ 来表示，它描述了光强度与波长 λ 的关系。激光器发射出来的光只有单一波长，即单色光，在这种情况下 E 是一个脉冲。

最通常的光源是白炽光，它是由炽热物体（如太阳或灯丝等）发出来的光。在物理学中它被模型化为黑体辐射或普朗克（Planckian）辐射。发射出来的能量是波长 λ 的函数，可以由普朗克辐射公式表示：

$$E(\lambda)=\frac{2hc^2}{\lambda^5(e^{hc/k\lambda T}-1)}\ \mathrm{W\,sr^{-1}\,m^{-2}\,m^{-1}} \tag{10.1}$$

其中，T 是黑体的热力学温度(K)，h 是普朗克常数，k 是玻尔兹曼常数，c 是光速①。该式描述的是单位面积单位波长所散发出来的能量。



背景材料：光谱。在 1665～1666 年的大瘟疫年间，艾萨克·牛顿发明了他的光和色彩理论。他证明了用棱镜可以将白色光分解成色彩光谱，还证明透镜之后再加一块棱镜可以使多色彩光谱再合成为白色光。另外，他还向人们展示了光经过不同物体反射其颜色不会发生变化，从而得出一个重要结论，即色彩是光的本质特性而不是物体的。(左图是牛顿画的草图)

我们可以绘制出黑体在不同温度下发出的光谱。首先定义波长范围：

```
>> lambda = [300:10:1000]*1e-9;
```

这里是从 300 nm 到 1000 nm。然后计算黑体光谱：

```
>> for T=1000:1000:6000
>>    plot( lambda*1e9, blackbody(lambda, T)); hold all
>> end
```

如图 10.2(a)所示。我们可以看到，随着温度的升高，能量的最大值在增加，但能量峰值处的波长在减小。总辐射能量(单位面积)等于黑体曲线下方的面积，由斯蒂芬-玻尔兹曼定律给出其公式：

$$\frac{2\pi^5 k^4}{15c^2h^3}T^4 \ \mathrm{W\,m^{-2}}$$

黑体曲线峰值所对应的波长由维恩位移律给出：

$$\lambda_{\max} = \frac{2.8978\times10^{-3}}{T}\ \mathrm{m}$$

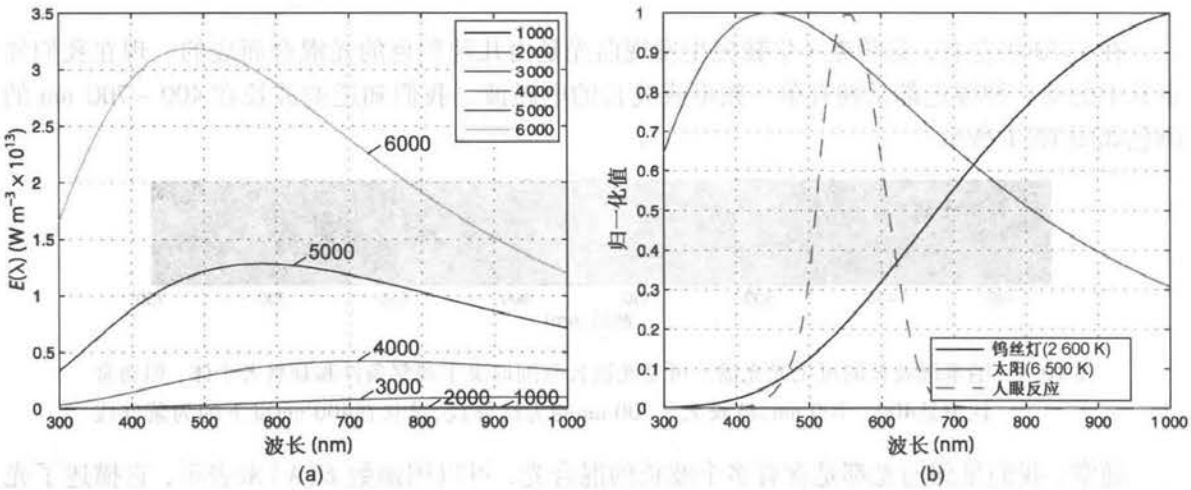


图 10.2 黑体光谱。(a)对应温度 1000 K～6000 K 的黑体辐射光谱；(b)太阳(6500 K)、钨丝灯(2600 K)、人眼反应的黑体光谱——为了易读，所有值进行了归一化

① $c = 2.998\times10^{8}\ \mathrm{m\,s^{-1}}$
$h = 6.626\times10^{-34}\ \mathrm{J\,s}$
$k = 1.381\times10^{-23}\ \mathrm{J\,K^{-1}}$

随着温度的提高，峰值波长在减小，这和我们加热物体时观察到的现象是一致的。当加热到800 K左右时物体开始变得微红，然后随着温度的升高，逐渐变为橙黄、到黄色、再到白色①。

钨灯丝在2600 K时变为白热。太阳表面的温度为6500 K。这些光源的光谱为

```
>> lamp = blackbody(lambda, 2600);
>> sun = blackbody(lambda, 6500);
>> plot(lambda*1e9, [lamp/max(lamp) sun/max(sun)])
```

并在图10.2(b)中进行了比较。钨灯曲线在数量级上要小得多，但为了研读方便对其进行了放大。太阳辐射峰值波长在450 nm左右，它在可见光部分发出相当大的能量。钨灯的峰值波长更长，但却只有很少的能量落在人眼可见范围内，其大部分能量集中在我们感觉热但却不亮的红外区。

背景材料： 汉弗莱·戴维(Humphry Davy)爵士在1802年用铂做灯丝发明了第一个电用白炽灯。约瑟夫·斯万(Joseph Swan)爵士在1850年用碳化纸做灯丝制作了他的第一个灯泡。然而，直到1865年真空泵的应用，这些灯才能达到可适用的寿命。斯万在1878年申请了碳化棉灯丝专利，并在1881年申请了碳化纤维灯丝的专利。在1880年之后，他发明的灯泡开始大范围应用，英国的萨沃伊剧院在1881年实现了完全用电照明。在美国，直到1878年托马斯·爱迪生才开始对白炽灯进行研究，但他在1879年就申请了碳化竹灯丝的长期专利，并且进行大量生产。1883年，斯万与爱迪生的公司合并。

随后，灯泡成了地球上光的主要来源。但如今由于它的能源利用率低，正在被逐渐淘汰。(图片来自 Douglas Brackett, inv., Edisonian.com)

10.1.1 吸收

太阳光在地球表面上的光谱已经测量出来并被表格化：

```
>> sun_ground = loadspectrum(lambda, 'solar.dat');
>> plot(lambda*1e9, sun_ground)
```

如图10.3(a)所示。它明显地不同于黑体光谱，因为阳光中有更多的波长已经被大气层吸收了。我们眼睛的峰值敏感度已经逐渐接近于大气层过滤的阳光光谱峰值。

透光率 T 与吸收率相反，它代表穿透过的部分光，是波长的函数。透光率可由比尔定律描述

$$T = 10^{-Ad} \tag{10.2}$$

其中，A 是吸收系数，单位是 m^{-1}，d 是穿透路径长度。水的吸收谱 $A(\lambda)$ 可从表格数据中获取：

```
>> [A, lambda] = loadspectrum([400:10:700]*1e-9, 'water.dat');
```

穿过5 m水的透光率是

```
>> d = 5;
>> T = 10.^(-A*d);
>> plot(lambda*1e9, T);
```

绘制的曲线如图10.3(b)所示。当我们在水下拍摄图像时，考虑不同波长光的吸收率差异是至关重要的。我们会在10.3.1节重温这个课题。

① 开始红热770 K~820K，黑红热920 K~1020 K，亮红热1120 K~1220 K，黄红热1320 K~1420 K，开始白热1520 K~1620 K，白热1720 K~1820 K。

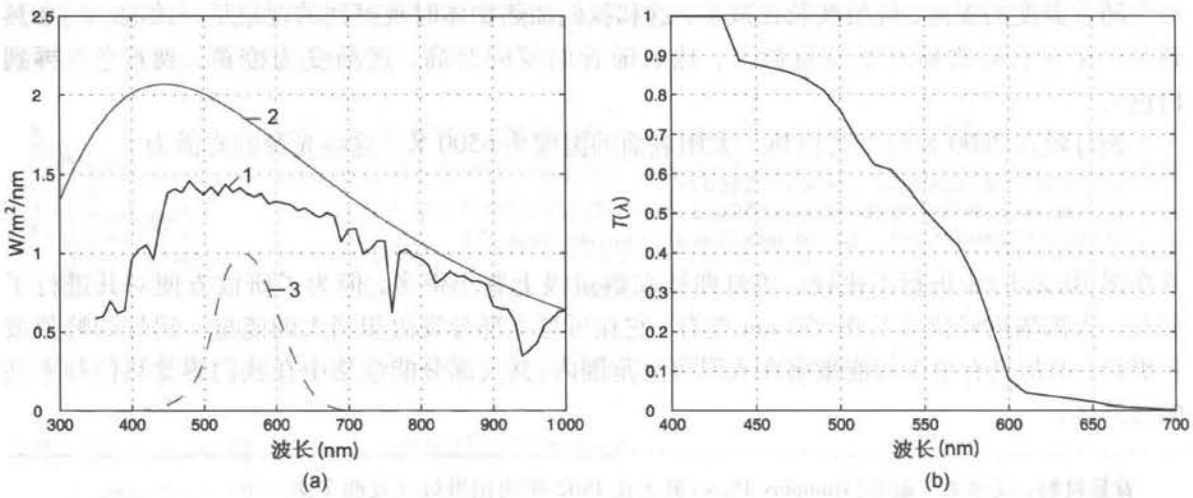


图 10.3 (a)改进的地平线日光光谱(曲线1)。日光光谱中的凹陷地方对应着不同物质吸收带。CO_2 吸收在红外线区域的辐射,臭氧 O_3 在紫外线区域的吸收很强烈。图中曲线2代表太阳的黑体光谱(归一化),虚线3代表人类眼睛的反应;(b)穿过5 m水的透光率。波长较长的光,如红色光,已经强烈衰减了

10.1.2 反射

从物体表面反射回来的光,其亮度的光谱如下:

$$L(\lambda) = E(\lambda)R(\lambda)\ \mathrm{W\,m^{-2}} \tag{10.3}$$

其中,E 是入射光的照度;$R \in [0,1]$ 是反射率或者表面反射比,它是波长的函数。例如,白纸的反射率是70%左右。许多材料的反射光谱已经被测量出来并制成表格①。以红色墙砖的反射率为例:

```
>> [R, lambda] = loadspectrum([100:10:10000]*1e-9, 'redbrick.dat');
>> plot(lambda*1e6, R);
```

绘制的曲线如图 10.4 所示。我们可以看到它反射的红色光比蓝色光多。

太阳在地面的可见光区域中:

```
>> lambda = [400:10:700]*1e-9;          % visible spectrum
```

其照明度为

```
>> E = loadspectrum(lambda, 'solar.dat');
```

砖的反射率是

```
>> R = loadspectrum(lambda, 'redbrick.dat');
```

砖反射回来的光是

```
>> L = E .* R;
>> plot(lambda*1e9, L);
```

如图 10.5 所示。正是这种光谱被我们的眼睛解读为红色。

① 来自于 http://speclib.jpl.nasa.gov/weathered red brick(0412UUUBRK)。

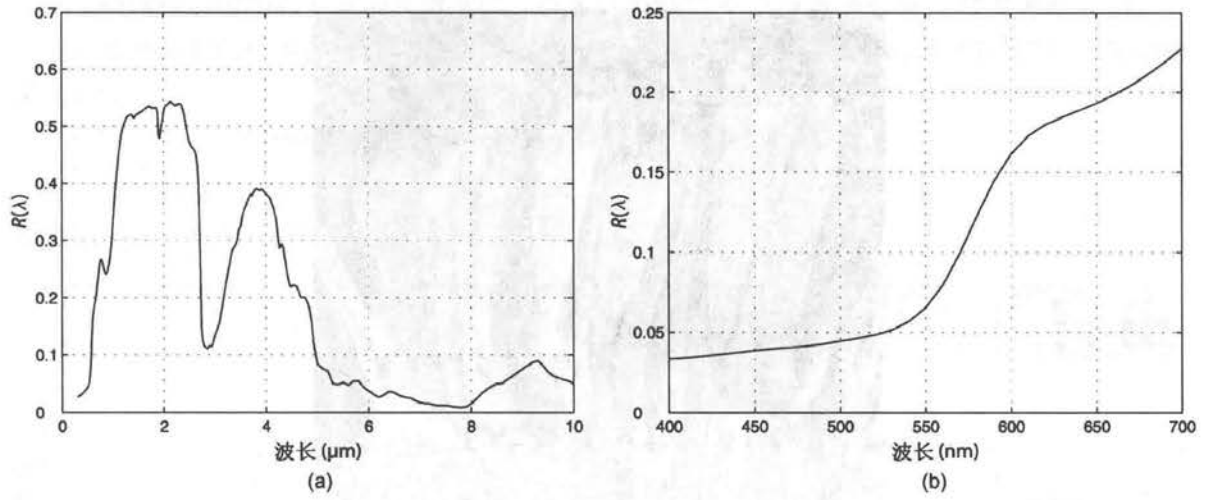


图 10.4 旧红砖的反射率(数据来自 ASTER, [Baldridge et al. 2009])。(a)在300～1000 nm范围内测量(红外线);(b)在可见光范围内测量

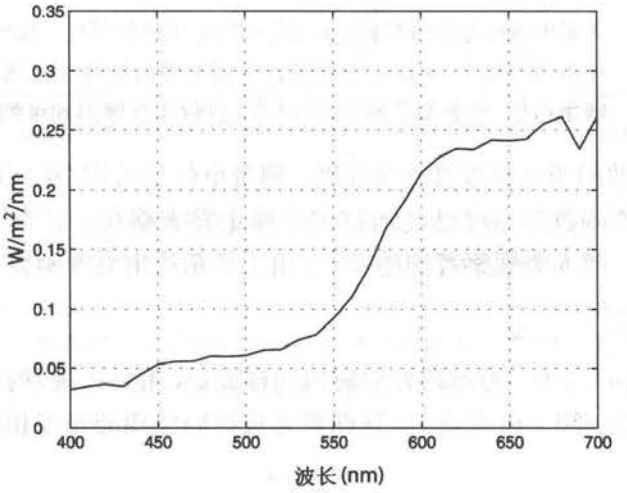


图 10.5 在地面太阳光照下旧红墙砖的亮度,基于来自图 10.3(a)和图 10.4(b)的数据

10.2 色彩

色彩是对由眼睛视网膜及其附属神经系统的活动所产生的所有感觉的一般称谓。眼睛的这种活动几乎存在于正常人的每时每刻,它是对具有一定波长和强度的辐射能量的一种特定反应。

——托兰(T. L. Troland),美国光学学会色度学委员会报道,1920—1921

我们已经从能量与波长的关系来描述了光谱,但是我们自己对光的感知则是基于一些主观参数,如明亮度和色彩。正如图 10.1 所示,人类的可见光波长范围从400 nm(紫色)到700 nm(红色),中间还有蓝色、绿色、黄色和橙黄色。

如图 10.6 所示(见彩图),我们的眼睛有两种敏感细胞。视锥形细胞对特别的颜色有反应,给我们提供正常的白天的视觉。视杆细胞比视锥细胞要灵敏得多,但是只对光强起反应,并且在夜间发挥作用①。

① 因此在夜晚你没有颜色视觉。

图 10.6 人眼中视杆细胞和锥细胞的彩色扫描电子显微照片。这些细胞的直径范围是0.5～4 μm。它们包含不同类型的被称为光视蛋白的光敏蛋白质。感光细胞的细胞体(红色)位于杆细胞和锥细胞的上层

一个特定波长光波的明亮程度被称为亮度，测量单位是流明/瓦。对于我们日光视锥细胞产生的视觉，作为波长函数的亮度已经通过实验确定并表格化，它为 1931CIE 标准奠定了基础，这个标准代表了一般人类观察者的感受①。由工具箱给出亮度函数：

```
>> human = luminos(lambda);
>> plot(lambda*1e9,  human)
```

绘制的曲线如图 10.7(a)所示。考虑两种辐射出同样能量(用瓦计量)的光，一种波长为 550 nm(绿光)，另一种波长为 450 nm(蓝光)。这两种光被我们感知的亮度相差很大，事实上蓝光的亮度仅为

```
>> luminos(450e-9) / luminos(550e-9)
ans =
    0.0382
```

或者说只有绿光亮度的 3.8%。

背景材料：辐射度量和光度量。当讨论光时会用到两组完全不同的单位：辐射度的和光度的。辐射计量单位在 10.1 节中用到过，它是基于像能量这类的量，且与 SI 单位制有类似的表达，例如瓦特。

光度计量单位与辐射计量单位类似，但是它考虑了观察者的视觉感受。发光的能量或光通量是一个光源被感知的能量，测量单位用流明(缩写是 lm)而不是瓦特。波长为 555 nm 的一个 1 W 光源可以发射出 683 lm 的光通量，能产生峰值反应。相反，波长为 800 nm 的一个 1 W 光源发射出的光通量为 0 lm，即无法产生视觉感受。

一只 1 W 的照明灯泡能产生小于 15 lm 的感知视觉感受，或 15 lm/W 的发光效率。荧光灯能够达到的效率是 100 lm/W，而白光 LED 灯则高达 150 lm/W。

① 这是使用锥感光细胞的眼睛适光反应，或称明适应反应。使用眼睛中单色杆感光细胞的适黑反应或暗适应反应则有所不同，其峰值波长为 510 nm。

不同物种的眼睛有不同的光谱反应。许多昆虫能够很好地看到紫外线范围内的光谱。用于数码摄像机的硅传感器对红色和红外线部分的光谱有很高的灵敏性，我们可以绘制出它的感光曲线[①]：

```
>> camera = ccdresponse(lambda);
>> hold on
>> plot(lambda*1e9,  camera*max(human), '--')
```

其叠加显示于图 10.7 中。

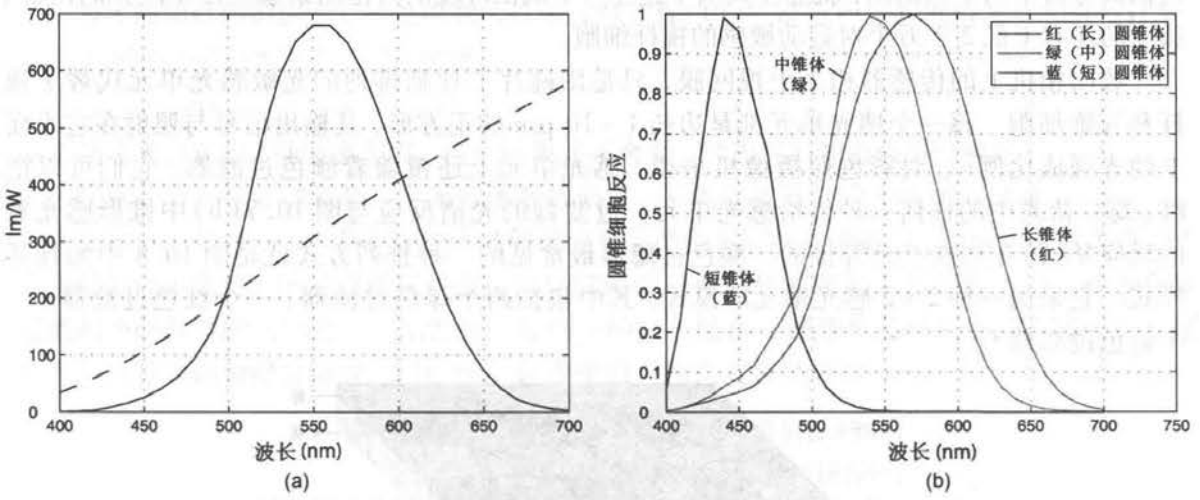


图 10.7　(a)标准人类观察者的感光度曲线。峰值反应是位于波长 555nm 处的 683lm/W。硅 CCD摄像机的反应用虚线画出作为比较；(b)人类视锥细胞的光谱反应(已归一化)

背景材料：光度计、照度和亮度。一个摄影用光度计可以测量光通量，其单位是 1 m/m^2 或勒克斯(lx)。一个点光源的照明强度是指每单位立体角的光通量，单位是 lm/sr 或坎德拉(cd)。对于照明强度为 I 的点光源，垂直照在一个物体表面上的照度 E 是

$$E = \frac{I}{d^2}\ \text{lx}$$

其中，d 是光源与表面的距离。大晴天的照度接近于 10 000 lx，而办公室的照度水平通常在 1000 lx 左右。

物体表面的亮度或明亮程度是

$$L_s = E_i \cos\theta\ \text{nt}$$

其单位是 cd/m^2 或尼特(nt)。其中，E_i 是与表面法线方向呈 θ 角的入射光的照度。

在正常日光情况下，我们的锥形感光器处于活跃状态，它们对色彩很灵敏。人类是三色视者，有三种类型的锥形感光器来对光谱的不同部分做出反应。根据其峰值反应的波长，分别被称为长(L)、中(M)、短(S)型，或更通常的红、绿、蓝型。我们可以加载并绘制这些锥形感光器的光谱反应：

```
>> cones = loadspectrum(lambda, 'cones.dat');
>> plot(lambda*1e9, cones)
```

其中，矩阵 cones 有三列，分别对应 L、M、S 三种锥形感光器的反应，而矩阵每一行对应

① 红外遥控器上的 LED 能被大多数数码相机看见其亮光——用你的手机相机和电视遥控器试试。有些安防摄像机也提供红外拍摄以用于夜晚的监控。注意，有些摄像机安装了红外过滤器以防止传感器因环境和大气辐射而饱和。

lambda 中的波长。$L(\lambda)$、$M(\lambda)$、$S(\lambda)$三种感光器的光谱反应如图 10.7(b)所示①。

其他物种的锥形感光器类型和数量有所不同。鸟类、鱼类和两栖类是四色视者，这就是说它们有四种类型的锥形感光器。许多其他哺乳动物，如狗，是二色视者，它们只有两种锥形感光器。有推测说一些人类女性也是四色视者②。

人类眼睛的视网膜有一个中心区或者中央凹区，它的直径只有 0.6 mm，包含 6 百万个锥细胞的绝大部分：65%感觉红色，33%感觉绿色，只有 2%感觉蓝色。我们通过无意识地扫描我们高分辨率的中央凹区，就能在大脑中建立一个周围环境的大范围景象。另外，分布在视网膜上的还有 1 亿 2 千万个对运动敏感的视杆细胞。

数码相机上的传感器相当于视网膜，只是用硅片上规则排列的光敏感光单元代替了视杆和视锥细胞。每一个感光单元都是边长 1 ~ 10 μm 的正方形，其输出信号与照射在它表面上的光强成比例③。对彩色照摄像机来说，感光单元上还覆盖着颜色过滤器，它们可以把红、绿、蓝光中的任何一种传给感光单元。过滤器的光谱反应与图 10.7(b)中锥形感光器的反应$M(\lambda)$在功能上是等价的。颜色过滤器最常见的一种排列方式就是图 10.8 中的拜耳模式。它采用一种 2×2 感光单元的模式，其中包括两个绿色过滤器、一个红色过滤器、一个蓝色过滤器④。

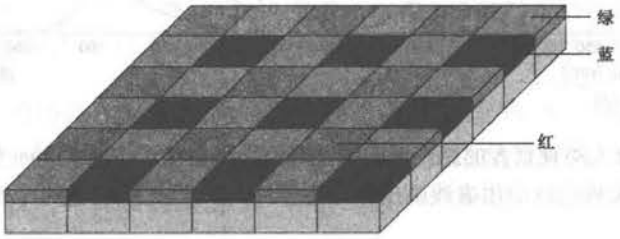


图 10.8 拜耳过滤。浅灰色块代表光敏硅感光单元，在它上面是红、绿、蓝三色的过滤器阵列。发明者是柯达公司的布莱斯·拜耳(Bryce E. Bayer)，美国专利号3,971,065

从方程(10.3)给出的物体亮度$L(\lambda)$，可以得到三种锥形体各自的特定响应：

$$\begin{aligned}\rho &= \int_\lambda L(\lambda)M_r(\lambda)\mathrm{d}\lambda \\ \gamma &= \int_\lambda L(\lambda)M_g(\lambda)\mathrm{d}\lambda \\ \beta &= \int_\lambda L(\lambda)M_b(\lambda)\mathrm{d}\lambda\end{aligned} \tag{10.4}$$

其中，$M_r(\lambda)$、$M_g(\lambda)$、$M_b(\lambda)$分别是图 10.7(b)所示的红、绿、蓝三种锥体的光谱反应。反应是一个三维向量(ρ, γ, β)，分别表示三种颜色刺激。

① 这种光谱特性是由于在锥形细胞中光视蛋白的差异造成的。乔治·沃德(George Wald)因在 20 世纪 50 年代通过实验展示的视网膜光视蛋白的吸收性而获得了 1967 年的诺贝尔医学奖。

② 她们多出一种长波(L)圆锥体的变种，因此具有更强的颜色分辨能力。

③ 更准确的说法是，输出是与感光单元上次被读取以后所捕获的光子总数成正比。

④ 因此每个像素不能提供对红、绿、蓝三色的独立测量，但可以进行估计。例如，在一个蓝光敏感像素上的红光量可以通过对其相邻的红光像素进行插值后获得。但更昂贵的"3CCD"摄像机则可对每个像素提供独立的三色测量，因为入射光会经一组棱镜分成三种原色光，然后再过滤并传给每种原色光对应的 CCD 阵列。数码相机的原始图像文件中包含了拜耳过滤感光单元的实际输出值。

背景材料：色盲或者色弱者失去了正常人具有的辨别不同颜色的感知能力。红、绿、蓝色盲分别是缺少了L、M、S锥形感光器中的一种。更常见的情况是红、绿、蓝色弱，它们是由于锥体的色素突变，并使峰值反应频率发生改变。导致色弱的最通常原因是遗传，因为红、绿光视蛋白被编码在X染色体上。最常见的是绿色弱(包括作者在内的6%的男性会发生)，M锥体的反应会朝着光谱的红色端转移，导致对绿色的敏感度降低，对光谱中红、橙黄、黄、绿区域的色相辨别力较差。

对于前面红砖的例子，三色刺激可以通过对方程(10.4)进行积分并近似为总和进行计算：

```
>> sum( (L*ones(1,3)) .* cones )
ans =
    16.3578   10.0702    2.8219
```

主要反应来自长(L)锥形感光器，这一结果并不奇怪，因为我们知道砖是红色的。

任意一段连续光谱都是一个无穷维数的向量，不能仅仅被三个参数唯一表示。而只用三个参数表示的结果是，许多不同的光谱能量分布会产生相同的视觉刺激，这称为条件等色。更重要的是以下推论：任意的视觉刺激可以仅仅通过三种单色的刺激混合产生。这三种单色就是我们小时候学的三原色。三原色并不是唯一的——任意三种颜色都可以，只要其中一种颜色不是由其他两种组合而成。CIE 1976标准中的三原色波长见表10.1[①]。

表10.1 CIE 1976的三原色(国际照明委员会1987)
(对应于一个汞蒸气灯中发射线的光谱颜色)

	红	绿	蓝
λ(nm)	700.0	546.1	435.8

10.2.1 颜色再造

电脑或电视的显示屏能在每个像素上产生可变数量的三原色。对阴极射线管(CRT)来说，三原色是通过激发屏幕背后的磷光体产生的。对于液晶显像器(LED)，三原色是通过过滤背光源发出来的白光获得的。关键问题是决定每种原色需要多少才能配出给定的三色刺激。

我们从考虑波长为λ_S的单色光刺激开始，定义为

$$L(\lambda)=\begin{cases}L_\lambda & 若\lambda=\lambda_S\\ 0 & 其他\end{cases}$$

对这种刺激的锥体反应由方程(10.4)给出，但由于$L(\cdot)$是一个脉冲，因此可以去掉积分得到三色刺激：

$$\begin{aligned}\rho&=L_\lambda M_r(\lambda_S)\\ \gamma&=L_\lambda M_g(\lambda_S)\\ \beta&=L_\lambda M_b(\lambda_S)\end{aligned}\tag{10.5}$$

接下来考虑三原色光源，波长分别为λ_r、λ_g、λ_b，强度分别为R、G、B。来自这些光源的三色刺激是

① 原色并不是光的基本特性，而属于观察者的基本特性。有三种原色存在，仅仅是因为我们人类的眼睛里有三种类型的圆锥体，是三色视者。鸟类可能有四种原色，而狗可能只有两种。

$$\begin{aligned}\rho &= RM_r(\lambda_r) + GM_r(\lambda_g) + BM_r(\lambda_b)\\ \gamma &= RM_g(\lambda_r) + GM_g(\lambda_g) + BM_g(\lambda_b)\\ \beta &= RM_b(\lambda_r) + GM_b(\lambda_g) + BM_b(\lambda_b)\end{aligned} \tag{10.6}$$

为了使三原色光源合成的颜色与单色光产生的颜色感受一致，两种刺激就必须是相等的。我们联立方程(10.5)和方程(10.6)，并用矩阵形式表示如下：

$$L_\lambda \begin{pmatrix} M_r(\lambda_S)\\ M_g(\lambda_S)\\ M_b(\lambda_S)\end{pmatrix} = \begin{pmatrix} M_r(\lambda_r) & M_r(\lambda_g) & M_r(\lambda_b)\\ M_g(\lambda_r) & M_g(\lambda_g) & M_g(\lambda_b)\\ M_b(\lambda_r) & M_b(\lambda_g) & M_b(\lambda_b)\end{pmatrix}\begin{pmatrix} R\\ G\\ B\end{pmatrix}$$

可以通过对上式求逆，给出三原色的需求量：

$$\begin{pmatrix} R\\ G\\ B\end{pmatrix} = L_\lambda \begin{pmatrix} M_r(\lambda_r) & M_r(\lambda_g) & M_r(\lambda_b)\\ M_g(\lambda_r) & M_g(\lambda_g) & M_g(\lambda_b)\\ M_b(\lambda_r) & M_b(\lambda_g) & M_b(\lambda_b)\end{pmatrix}^{-1}\begin{pmatrix} M_r(\lambda_S)\\ M_g(\lambda_S)\\ M_b(\lambda_S)\end{pmatrix}$$

所需的三色刺激的值只是简单地将锥体对单色光刺激的反应进行线性变换。变换矩阵是常量，但是依赖于锥体对所选三原色(λ_r, λ_g, λ_b)的光谱响应。这就是三色匹配的基础。

背景材料：颜色匹配实验可以用三盏三原色的灯作为光源来演示，而且这三盏灯的光强可以人工调节。三种灯光将被混合后散射出来，并与某种测试光进行比较。在颜色匹配中，三原色的灯分别用 **R**、**G**、**B** 字母表示，它们的强度用 R、G、B 来表示。三盏灯的光强由人主观调节，直到显示与测试光的颜色相同。表示为

$$\mathbf{C} \equiv R\mathbf{R} + G\mathbf{G} + B\mathbf{B}$$

上式可理解为视觉刺激 **C**(测试光颜色)与亮度为 R、G、B 的三原色混合光相匹配，或者看上去相同。符号 $R\mathbf{R}$ 可理解为光强为 R 的灯 **R**。

实验显示，颜色匹配遵循可加性和线性的代数法则，也称为格拉斯曼定律。例如，两种光刺激 $\mathbf{C}_1$ 和 $\mathbf{C}_2$：

$$\mathbf{C}_1 \equiv R_1\mathbf{R} + G_1\mathbf{G} + B_1\mathbf{B}$$
$$\mathbf{C}_2 \equiv R_2\mathbf{R} + G_2\mathbf{G} + B_2\mathbf{B}$$

将它们混合，有

$$\mathbf{C}_1 + \mathbf{C}_2 \equiv (R_1 + R_2)\mathbf{R} + (G_1 + G_2)\mathbf{G} + (B_1 + B_2)\mathbf{B}$$

我们可以用更简洁的方式写出上式：

$$\begin{pmatrix} R\\ G\\ B\end{pmatrix} = \begin{pmatrix} \overline{r}(\lambda_S)\\ \overline{g}(\lambda_S)\\ \overline{b}(\lambda_S)\end{pmatrix} \tag{10.7}$$

其中，$\overline{r}(\lambda)$，$\overline{g}(\lambda)$，$\overline{b}(\lambda)$是颜色匹配函数。对于表 10.1 中列出的标准 CIE 三原色，这些函数已经被表格化，并由函数 cmfrgb 返回：

```
>> lambda = [400:10:700]*1e-9;
>> cmf = cmfrgb(lambda);
>> plot(lambda*1e9, cmf);
```

绘制的曲线如图 10.9 所示。每条曲线都表示需要多少对应的原色才能匹配波长为 λ 的单色光。

例如，为了创造出波长为 600 nm 光(橙色光)的视觉感受，我们需要

```
>> orange = cmfrgb(600e-9)
orange =
    2.8717    0.3007   -0.0043
```

令人吃惊的是，它需要的蓝原色是很小的负值。要产生波长为 500 nm 的绿色光，我们需要

```
>> green = cmfrgb(500e-9)
green =
   -0.2950    0.4906    0.1075
```

它需要的红原色是较大的负值。由于光源没有负亮度，所以这是一个问题。

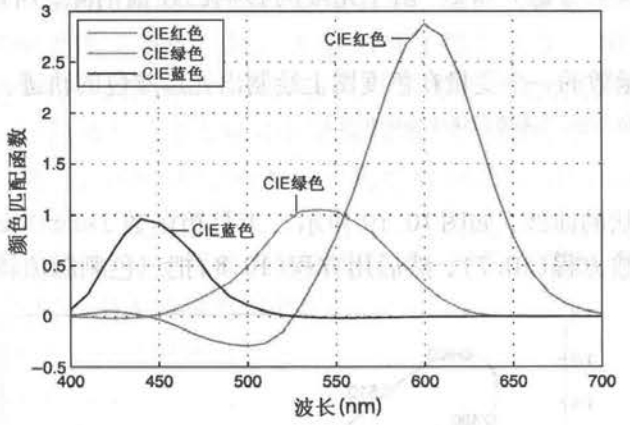


图 10.9　对于标准观察者的 1931 颜色匹配函数，基于 CIE 的三原色

我们通过增加一些白色光($R = G = B = w$，见 10.2.6 节)来解决上述问题，这样使三色值都是正值。例如：

```
>> w = -green(1);
>> white = [w w w ];
>> feasible_green = green + white
feasible_green =
         0    0.7856    0.4025
```

如果把这个颜色与期望的 500 nm 绿色放在一起观察，我们会说产生的这个颜色在色彩上是正确的，但是不够饱和。

饱和是指颜色的纯度。光谱颜色是完全饱和的，但是随着白色光数量的添加，颜色就会欠饱和(更柔和)。这里，我们已经混合了一种刺激：

```
>> white
white =
    0.2950    0.2950    0.2950
```

它是灰色光。

从上述实验可以得到关于颜色再造的重要结论：仅用三原色不可能再造出所有可能的颜色。这个结论很好理解，因为一种颜色理论上是一个无限维的光谱函数，用一个三维向量只能近似模拟它。为了更充分地了解这一点，我们还需要考虑色度空间。

工具箱函数 cmfrgb 也可以计算任意光谱的 CIE 三原色。前面已经计算过在地面被日光照射的红砖光谱，它的三色刺激是

```
>> RGB_brick = cmfrgb(lambda, L)
RGB_brick =
    0.6137    0.1416    0.0374
```

这些是 CIE 三原色各自的数量，它们合成后的视觉感受与红砖颜色相同。

10.2.2　色度空间

三色刺激的值不仅描述了颜色，也包含了亮度。我们可以通过归一化三色刺激值，得到一

组相应的三色刺激值：

$$r=\frac{R}{R+G+B},\ g=\frac{G}{R+G+B},\ b=\frac{B}{R+G+B} \tag{10.8}$$

上式得到的就是色度坐标 r、g、b，它们不随亮度值而改变。由于定义 $r+g+b=1$，所以有一个坐标是多余的，通常只考虑 r 和 g。由于光强的影响已经被消除，所以二维向量(r, g)只代表颜色。

可以用颜色匹配函数的一个变量在色度图上绘制出光谱颜色的轨迹，如彩虹中的各种颜色：

```
>> [r,g] = lambda2rg( [400:700]*1e-9 );
>> plot(r, g)
>> rg_addticks
```

可以得到一条马蹄形状的曲线，如图 10.10 所示。工具箱函数 `lambda2rg` 可以为特定波长的颜色计算颜色匹配函数方程(10.7)，然后用方程(10.8)把三色刺激值转换成色度坐标。

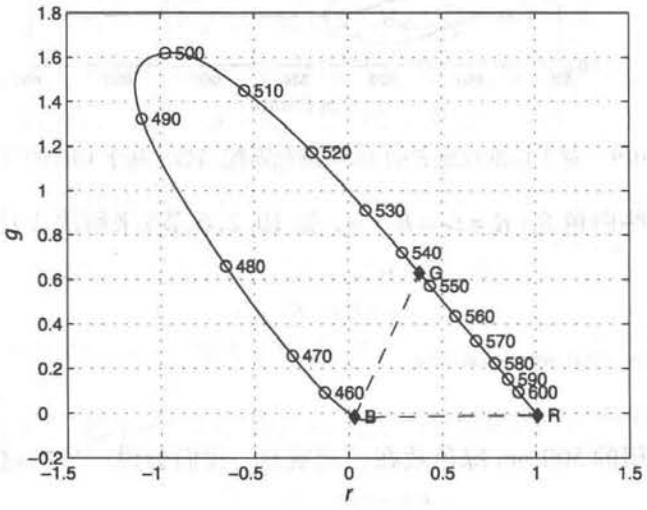


图 10.10 在 $r-g$ 色度平面上的光谱轨迹。CIE 标准三原色用黑色菱形标注。每种光谱颜色的波长(nm)也被标在图上。连接各极限点的曲线是边界，是饱和颜色的轨迹

表 10.1 列出的 CIE 三原色也可以绘制成图：

```
>> primaries = cmfrgb( [700, 546.1, 435.8]*1e-9 );
>> plot(primaries(:,1), primaries(:,2), 'd')
```

即如图 10.10 所示的菱形。

背景材料：色度标准。色度是一个复杂的话题，其标准很重要。CIE 和 ITU 两个组织在这个领域发挥了领导作用。

国际照明委员会(Commission internationale de l' Eclairage，CIE)成立于 1913 年，是一个独立的非营利组织，致力于在有关光学、照明、色彩与视觉及图像技术等方面的世界合作与信息共享。CIE 第八次会议于 1913 年在英国剑桥举行，会议达成了色度规范国际共识，并建立了 *XYZ* 颜色空间。CIE 被 ISO 认可为国际标准组织。查阅 http://www.cie.co.at 可获取更多信息和 CIE 数据库。

国际电信联盟(ITU)是联合国的一个机构，它的成立是为了对国际上无线电和远程通信技术进行标准化和规范化。它最早于 1865 年 5 月 17 号在巴黎作为国际电报联盟诞生。国际无线电咨询委员会或 CCIR (Comité Consultatif International des Radiocommunications)在 1992 年成为 ITU 或 ITU-R 的无线电通信部。它发布了在广播服务(电视)或 BT 系列中关于色度的标准和建议。查看 http://www.itu.int 可获得详细内容。

重心定理描述到：两种颜色的混合色在色度平面上会落在这两种颜色之间的一条直线上。N 种颜色的混合色会落入这些颜色所围成的区域中。我们用图 10.10 来理解这其中的含义。第一，因为所有颜色刺激都是各个光谱刺激的组合，因此所有真实颜色刺激一定位于光谱轨迹上或轨迹内部。第二，我们通过混合三原色创造出的任何颜色只能落在由三原色围成的三角形颜色区域中——色域。从图 10.10 可以清楚地看到，CIE 三原色只定义了所有可能颜色中的一个小子集，即图中虚线三角形区域。大量的真实颜色无法只用三原色创造出来，尤其是那些位于光谱轨迹 460 ~ 545 nm 的彩虹颜色。事实上，无论选取什么来作为三原色，都不可能制造出所有的颜色来①。用几何术语来说就是：区域中任何三个点形成的三角形范围一定小于整个区域。第三，我们观察到大部分轨迹对应的红原色是负值，因此是不能由三原色表示的。

前文(10.2.1 节)展示的 500 nm 绿光问题由此解开，因为它落在了 CIE 三原色区域的外面，如图 10.11 所示。我们画出光谱绿色的色度：

```
>> green_cc = lambda2rg(500e-9);
green_cc =
   -0.9733    1.6187
>> plot2(green_cc, 's')
```

图中用一个正方形标示。白光被定义为 $R=G=B=1$，它的色度为

```
>> white_cc = tristim2cc([1 1 1])
white_cc =
    0.3333    0.3333
>> plot2(white_cc, '*')
```

它被绘制成星号。根据重心定理，我们期望的绿色和白色的混合色肯定位于所示的那条直线上。饱和度最小的可显示绿光的色度位于这条直线与三角色域边界的交叉点处，用圆圈表示。

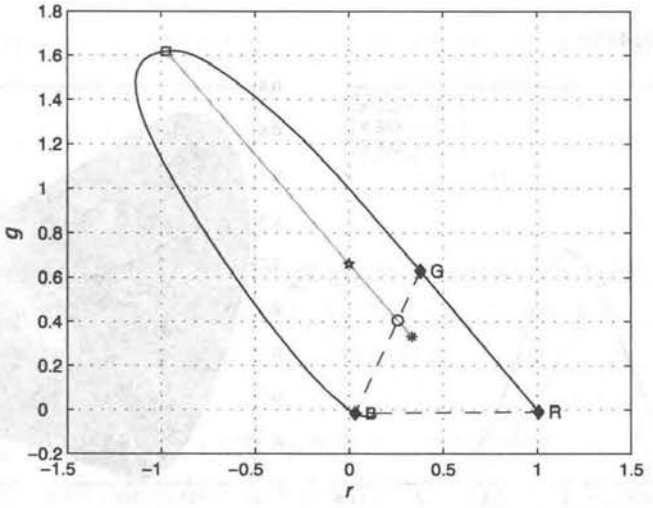


图 10.11 色度图显示了 500 nm 的绿光(正方形)，等能量的白光(星号)，可用的绿光(五角星)和可显示的绿光(圆圈)。不同饱和度的绿光轨迹由线1表示

前面说过，区域中任何三个点形成的三角形范围一定小于整个区域。因此，CIE 在 1931 年提出一种虚构的非物理三原色系统，称其为 **X**、**Y**、**Z**，它们围成的色域完全包围了

① 可以通过选择不同的原色来增加色域范围，或许只需选用一个不同的绿原色就能增大色域。但在实际应用中，要找到一个能产生那种原色的光源(LED 或磷光体)很难。

图 10.10 中的光谱轨迹。**X**、**Z** 具有 0 亮度，亮度完全由 **Y** 提供。这样，所有真实的颜色都能用这三种原色的正值匹配出来①。相应的三色刺激值表示为(X, Y, Z)。

CIE 定义的 XYZ 颜色配制函数为

```
>> cmf = cmfxyz(lambda);
>> plot(lambda*1e-9, cmf);
```

如图 10.12(a)所示。它显示了为匹配一个光谱颜色所需的每种 CIE XYZ 原色的数量，我们注意到这些曲线都没有负值。相应的色度坐标是

$$x=\frac{X}{X+Y+Z}, y=\frac{Y}{X+Y+Z}, z=\frac{Z}{X+Y+Z} \tag{10.9}$$

仍然有 $x+y+z=1$，所以只需要两个参数——在色度图上一般画 y 与 x 的关系。光谱轨迹可以用与前面相同的方式绘制：

```
>> [x,y] = lambda2xy(lambda);
>> plot(x, y);
```

为了显示光谱轨迹内的颜色，可以绘制一个更复杂的图形：

```
>> xycolorspace
```

如图 10.12(b)所示②。对于制图、打印及其他用途，这些坐标是表示颜色的标准方式。例如，嫩绿色(550 nm)的色度坐标是

```
>> lambda2xy(550e-9)
ans =
    0.3016    0.6924
```

标准钨丝灯 2600 K 时的色度坐标是

```
>> lamp = blackbody(lambda, 2600);
>> lambda2xy(lambda, lamp)
ans =
    0.4679    0.4126
```

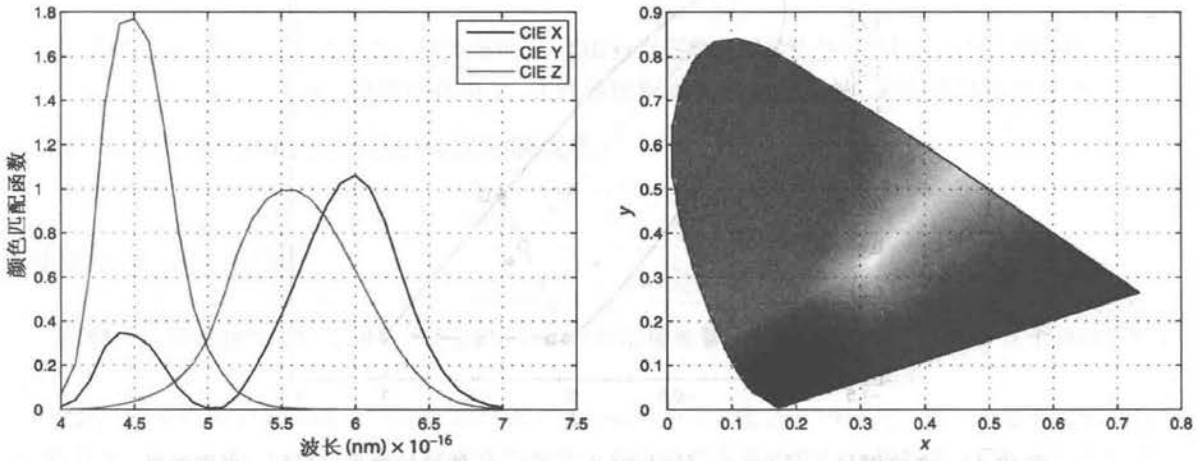


图 10.12　(a)基于虚构三原色的对标准观察者的颜色匹配函数，**X**、**Y**(光强)、**Z**已被CIE(1987)表格化；(b)xy色度平面上的颜色

① 其单位是经过特意选择的：需要等量的三原色来匹配相同能量的白光刺激。

② 注意：绘出的颜色只是实际颜色的近似，这是因为被打印颜色的色域限制。没有一种显示设备有足够大的色域去精确描绘每一点的色度。

10.2.3 色彩名称

色度坐标提供了一种定量的方式来描述和比较颜色。然而，人类是用名称来描述颜色的。许多计算机操作系统包含一个数据库或文件①，它把人们使用的颜色名称映射为相应的(R, G, B)三色刺激值。一个典型的数据库包含 800 多种唯一命名的颜色，从另一方面也诠释了颜色对于人类的重要性。工具箱提供了这样一种文件的副本，以及一个接口函数 colorname。例如，我们查询一个包含特殊子字符串的颜色名称：

```
>> colorname('?burnt')
ans =
    'burntsienna'    'burntumber'
```

这个深褐色的 RGB 三色刺激值是

```
>> colorname('burntsienna')
ans =
    0.5412    0.2118    0.0588
```

以上值已被归一化到区间[0, 1]。我们还可以求出其 xy 色度坐标：

```
>> bs = colorname('burntsienna', 'xy')
bs =
    0.5258    0.3840
```

参见图 10.12(见彩图)，我们发现这个点位于颜色空间的红色-棕色之间的部分，接近巧克力色：

```
>> colorname('chocolate', 'xy')
ans =
    0.5092    0.4026
```

我们也可以求解其逆问题。例如，考虑一个与深褐色接近但又不完全相同的三色刺激值：

```
>> colorname([0.54 0.20 0.06])
ans =
burntsienna
```

它返回一个最接近颜色的名称。我们可以对 xy 色度坐标指定的颜色重复这个过程：

```
>> colorname(bs, 'xy')
ans =
    'burntsienna'
```

10.2.4 其他颜色空间

颜色空间是一个包含所有可能的三色刺激值(所有颜色和所有亮度)的三维空间。如果采用在 2.2 节中讨论的坐标系来考虑颜色空间，那么会有无限多种笛卡儿坐标系可用来定义颜色。前面已经讨论了两种不同的笛卡儿颜色空间：RGB 和 XYZ。然而，我们也可以用极坐标系、球面坐标系或混合坐标系。

色度空间 r-g 或是 x-y 都不考虑光的明亮程度——我们已经在方程(10.8)和方程(10.9)中通过归一化将其去除了。明亮程度的更准确名称是亮度，通常由 Y 表示。来自 ITU 709 号建议书的定义为

$$Y^{709} = 0.2126R + 0.7152G + 0.0722B \tag{10.10}$$

它是 RGB 三色刺激值的加权和，反映了眼睛对绿色的高敏感性和对蓝色的低敏感性。色度加上亮度就构成三维颜色空间，例如 r-g-Y 或 x-y-Y。

① 在大多数 UNIX 系统中，该文件被命名为/etc/rgb.txt。

人们似乎更趋向于用两种特征来考虑色度：色相与饱和度。色相是指主要颜色，即最近的光谱颜色；饱和度是指颜色的纯洁性，即缺少混合的白色。光谱轨迹上的刺激都是完全饱和的，而越接近于其中心的那些颜色的饱和度越低。我们已经讨论过的颜色空间用色相和饱和度都难以解释清楚，因此有人提出了其他的颜色空间。最为人所熟知的两种是 HSV 和 CIE L^*C^*h。在颜色空间中，符号 H 是色相，S 是饱和度(也称为 C 或浓度)。强度维被命名为 V(值)或 L(明度)，但是它们的计算方法大相径庭①。色相和饱和度的概念在图 10.13 中用几何术语做了阐述。

函数 colorspace 可用于实现不同颜色空间之间的转换。例如，对于纯红、纯绿、纯蓝的 RGB 三色刺激值，它们的色相、饱和度和强度是

```
>> colorspace('RGB->HSV', [1, 0, 0])
ans =
     0     1     1
>> colorspace('RGB->HSV', [0, 1, 0])
ans =
   120     1     1
>> colorspace('RGB->HSV', [0, 0, 1])
ans =
   240     1     1
```

对每种颜色，饱和度都是 1，说明颜色都是纯色，而且强度也都是 1。如图 10.13 所示，色相由范围在[0°, 360°)的一个角来表示，红色是 0°，随着光谱颜色波长减小(橙色、黄色、绿色、蓝色、紫色)，角度值逐渐增加。如果降低绿原色的量：

```
>> colorspace('RGB->HSV', [0, 0.5, 0])
ans =
  120.0000    1.0000    0.5000
```

我们就会看到其强度在下降，但是色相和饱和度没变②。对于中灰色：

```
>> colorspace('RGB->HSV', [0.4, 0.4, 0.4])
ans =
  240.0000         0    0.4000
```

其饱和度是 0，它只是白色的混合色，而且由于没有颜色，色相也没有意义。如果在灰色中增加一些绿色：

```
>> colorspace('RGB->HSV', [0, 0.5, 0] + [0.4, 0.4, 0.4])
ans =
  120.0000    0.5556    0.9000
```

我们就会得到绿色相和中等饱和度的值。

函数 colorspace 可以在 13 种不同的颜色空间中进行转换，包括 YUV，YCbCr，L^*a^*b 和 L^*u^*v。许多颜色空间的局限在于在两个相近空间点之间的颜色感受差异依赖于这些点在空间中的位置。这导致了视觉上均匀的颜色空间的发展，例如 CIE L^*u^*v(CIELUV)和 L^*a^*b 空间。

函数 colorspace 也可以应用于图 10.14 所示的彩色图像。在色相图像上，黑色代表红色，亮白代表紫色。红花既表现出有一个很小的色相角(黑色)，也有一个接近于 360°的大色相角。黄花和绿色背景可以看出具有完全不同的色相值。饱和度图像中显示红色和黄色花朵是高饱和的，而绿色的叶和茎是欠饱和的。白花的饱和度很低，因为根据定义白色包含很多白光。这个例子在 10.3.5 节中进行了详细论述和扩展讨论。

---

① L^* 是相对亮度的非线性函数，它近似模拟了人眼的非线性反应。而强度值的计算式为 $V=1/2(\min R,G,B+\max R,G,B)$。

② 对于非常黑的颜色，数值问题会导致色相和饱和度坐标的不准确。

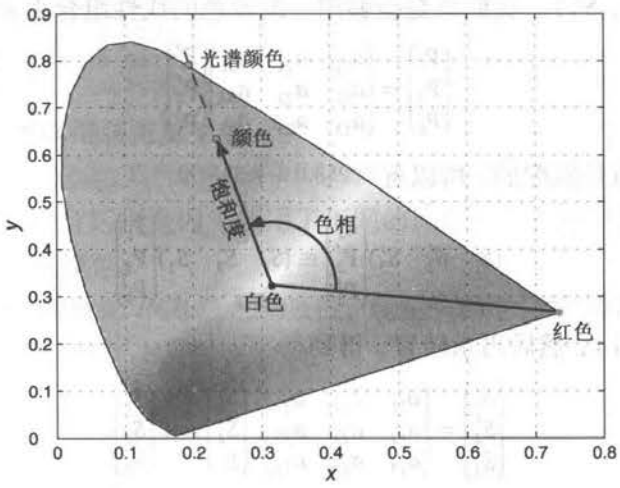


图 10.13 色相与饱和度。一条直线从"白色点"开始，穿过讨论的色度延伸到光谱轨迹。这条直线的角度就是色相，饱和度是相对于到轨迹的距离归一化后的向量长度

图 10.14 花景。(a)原始图像；(b)色相图像；(c)饱和度图像。注意白色花朵的饱和度很低(它们发黑)

10.2.5 两种原色之间的转换

CIE 标准是在 1931 年被确定的(在 20 世纪 50 年代彩色电视机诞生之前)。CIE 的原色是基于汞灯的射线，它对于实验室应用具有很高的可重复性和适用性。早期的电视机使用 CRT 显示器，这种显示器的原色是由磷光体产生的，磷光体被电子轰击时会发射出光线。随后，为追求更明亮的显示，所用磷光体和它们的颜色都一直在发生改变。一项国际协定(ITU 709 号建议书)定义了高分辨率电视(HDTV)的原色，它们在表 10.2 中列出。

表 10.2 标准原色和白色的 *xyz* 色度。包括表 10.1 中的 CIE 原色，以及定义 HDTV 的 ITU709 原色。D_{65} 是在6500 K时黑体辐射发出的白色，E是等能量的白色

	R_{CIE}	G_{CIE}	B_{CIE}	R_{709}	G_{709}	B_{709}	D_{65}	E
x	0.7347	0.2738	0.1666	0.640	0.300	0.150	0.3127	0.3333
y	0.2653	0.7174	0.0089	0.330	0.600	0.060	0.3290	0.3333
z	0.0000	0.0088	0.8245	0.030	0.100	0.790	0.3582	0.3333

这带来了从一组原色到另一组原色之间三色刺激值的转换问题。例如，我们想要显示一个图像，图像的三色刺激值是基于 CIE 原色，而显示屏幕采用的却是 ITU 709 的原色。我们用前面使用的符号来定义两组原色：$\mathbf{P}_1$、$\mathbf{P}_2$、$\mathbf{P}_3$对应三色刺激值(S_1, S_2, S_3)，而 $\mathbf{P}'_1$、$\mathbf{P}'_2$、$\mathbf{P}'_3$对

应三色刺激值(S'_1, S'_2, S'_3)。我们总是能够用一组原色的线性组合来表达另一组原色[①]:

$$\begin{pmatrix}\mathbf{P}_1\\ \mathbf{P}_2\\ \mathbf{P}_3\end{pmatrix}=\begin{pmatrix}a_{11} & a_{21} & a_{31}\\ a_{12} & a_{22} & a_{32}\\ a_{13} & a_{23} & a_{33}\end{pmatrix}\begin{pmatrix}\mathbf{P}'_1\\ \mathbf{P}'_2\\ \mathbf{P}'_3\end{pmatrix} \tag{10.11}$$

由于这两组三色刺激值是匹配的，所以有

$$(S'_1 \quad S'_2 \quad S'_3)\begin{pmatrix}\mathbf{P}'_1\\ \mathbf{P}'_2\\ \mathbf{P}'_3\end{pmatrix}\equiv(S_1 \quad S_2 \quad S_3)\begin{pmatrix}\mathbf{P}_1\\ \mathbf{P}_2\\ \mathbf{P}_3\end{pmatrix} \tag{10.12}$$

将上式代入方程(10.11)，然后再求转置，得到

$$\begin{pmatrix}S'_1\\ S'_2\\ S'_3\end{pmatrix}=\begin{pmatrix}a_{11} & a_{21} & a_{31}\\ a_{12} & a_{22} & a_{32}\\ a_{13} & a_{23} & a_{33}\end{pmatrix}^{\mathrm{T}}\begin{pmatrix}S_1\\ S_2\\ S_3\end{pmatrix}=\boldsymbol{C}\begin{pmatrix}S_1\\ S_2\\ S_3\end{pmatrix} \tag{10.13}$$

它只是三色刺激值的简单线性变换。

考虑从 CIE 原色到 *XYZ* 三色刺激值变换的具体问题。从表 10.2 可得到相对于 *XYZ* 原色的 CIE 原色：

```
>> C = [ 0.7347, 0.2653, 0; 0.2738, 0.7174, 0.0088; 0.1666,
0.0089, 0.8245]'
C =
    0.7347    0.2738    0.1666
    0.2653    0.7174    0.0089
         0    0.0088    0.8245
```

它们正是表 10.2 中的前三列。因此其变换为

$$\begin{pmatrix}X\\ Y\\ Z\end{pmatrix}=\boldsymbol{C}\begin{pmatrix}R\\ G\\ B\end{pmatrix}$$

在前文中我们说过，亮度完全是由 **Y** 原色贡献的。通常采用如下约束条件：单位 *R*、*G*、*B* 值产生单位亮度 *Y* 和具有特定色度的白色。我们选择 D_{65}这种白色，其色度值在表 10.2 中给出，并表示成(x_w, y_w, z_w)。现在可以写出

$$\frac{1}{y_w}\begin{pmatrix}x_w\\ y_w\\ z_w\end{pmatrix}=\boldsymbol{C}\begin{pmatrix}J_R & 0 & 0\\ 0 & J_G & 0\\ 0 & 0 & J_B\end{pmatrix}\begin{pmatrix}1\\ 1\\ 1\end{pmatrix}$$

其中，左侧有 $Y=1$。我们已经介绍过对角矩阵 **J**，它的作用是放大原色的亮度。对于 **J** 中的元素，可以这样求解：

$$\begin{pmatrix}J_R\\ J_G\\ J_B\end{pmatrix}=\boldsymbol{C}^{-1}\begin{pmatrix}x_w\\ y_w\\ z_w\end{pmatrix}\frac{1}{y_w}$$

代入真实值，可以得到

```
>> J = inv(C) * [0.3127 0.3290 0.3582]' * (1/0.3290)
J =
    0.5609
    1.1703
    1.3080
>> C * diag(J)
```

① 系数可以为负值，这样新的原色就不必落在以前原色的色域中了。

```
ans =
    0.4121    0.3204    0.2179
    0.1488    0.8395    0.0116
         0    0.0103    1.0785
```

从这个矩阵的中间一行可以得到亮度关系：

$$Y = 0.1488R + 0.8395G + 0.0116B$$

它与方程(10.10)相似。稍有区别是因为使用了不同的原色——在这里使用了 CIE 原色，而方程(10.10)使用了 Rec.709 原色。

先前我们已经计算过红砖的 *RGB* 三色刺激值，现在来确定它的 *XYZ* 三色刺激值：

```
>> XYZ_brick = C * diag(J) * RGB_brick';
ans =
    0.0351
    0.0224
    0.0039
```

再通过方程(10.9)把它转换成色度坐标：

```
>> tristim2cc(XYZ_brick')
ans =
    0.5729    0.3645
```

参见图 10.12，可以看出这个 *xy* 色度所落区域的名称是

```
>> colorname([0.5729    0.3645], 'xy')
ans =
    'englishred'
```

正是我们期待的“旧红砖色”。

10.2.6 什么是白色

在上一节中我们接触到了白色这个概念。白色既可以是没有任何颜色，也可以是所有颜色的混合。对白色的一种定义是标准日光，在西欧和北欧认定是中午的太阳光，并且已被 CIE 表格化为 D_{65}。它可以用 6500 K 的黑体辐射来近似：

```
>> d65 = blackbody(lambda, 6500);
>> lambda2xy(lambda, d65)
ans =
    0.3136    0.3241
```

可以看出该值接近于表 10.2 中给出的 D_{65} 色度。

另一种定义基于把所有光谱颜色等量混合而成的白色光。它可以用一个均匀光谱来表示：

```
>> ee = ones(size(lambda));
```

又称为等能量的刺激，它的色度是

```
>> lambda2xy(lambda, ee)
ans =
    0.3333    0.3338
```

接近于定义值(1/3,1/3)。

10.3 高级议题

本节将会讨论一些高级议题。首先是照明对一个物体所呈现颜色的影响，这对于在自然光照环境中使用颜色信息的机器人确实是一个严重问题。它将引发对白平衡问题的讨论。第二个议题是介绍伽玛编码，它是一种非常普遍的三色刺激值和实际亮度之间的非线性关系。

最后我们在真实图像中来看颜色的区别，并把图像分割成相似颜色区域——这里仅是一个技术前瞻，我们将在第 12 章和第 13 章中详细讨论。

10.3.1 颜色的不变性

研究显示，人类对白色的感知具有适应性，人们具有很强的能力来适应场景照明的影响，以至于使白色物体看上去就是白色[①]。例如，晚上在淡黄色的钨灯下我们看书的页面总是白色的，但如果事后在不同照明情况下再看上述场景的照片，它看起来就会是黄色。

这个现象对于利用颜色来识别场景的机器人确实是一个严重的问题，因为机器人观察到的色度随着照明不同发生了改变。在户外，早晨或傍晚太阳(3500 K)的颜色与正午太阳(6500 K)的颜色是不同的，阴天和晴天的光线也不同，再加上建筑物或树木的反射光，所有这些因素共同作用将改变照明光谱，进而改变物体的亮度与颜色。为了阐明这个问题，我们再回到红砖的例子：

```
>> lambda = [400:10:700]'*1e-9;
>> R = loadspectrum(lambda, 'redbrick.dat');
```

有两种不同的照明情况。地面上的太阳是

```
>> sun = loadspectrum(lambda, 'solar.dat');
```

钨灯是

```
>> lamp = blackbody(lambda, 2600);
```

然后计算每种照明情况下的 *xy* 色度：

```
>> xy_sun = lambda2xy(lambda, sun .* R)
xy_sun =
    0.4760    0.3784
>> xy_lamp = lambda2xy(lambda, lamp .* R)
xy_lamp =
    0.5724    0.3877
```

我们可以看出色度(呈现的颜色)已经发生了很大的变化。这些值在图 10.15 所示的色度图中描绘出来。

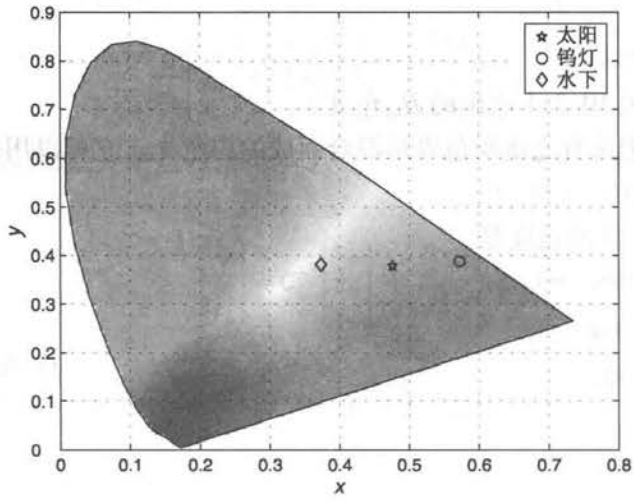


图 10.15 在不同照明情况下红砖的色度

① 人类能调整自己的颜色感知，从而使得对整个场景的总体感受为灰色。这对于颜色温度 5000 K ~ 6500 K 的情况都适用。

10.3.2 白平衡

摄影师们都知道照度的重要性。照度是指光源的色温——来自方程(10.1)的等价的黑体温度。与日光相比，白炽灯显得更黄一些，摄影师会用一个蓝色过滤器(在相机上)来减弱光谱中的红色部分进行补偿。我们可以通过选择矩阵 **J** 来获得一个相似的功能：

$$\begin{pmatrix} R' \\ G' \\ B' \end{pmatrix} = \begin{pmatrix} J_R & 0 & 0 \\ 0 & J_G & 0 \\ 0 & 0 & J_B \end{pmatrix} \begin{pmatrix} R \\ G \\ B \end{pmatrix}$$

它能调节颜色通道的增益。例如，在钨灯照明下，增大 J_B 将会对缺少蓝色进行补偿。这就是白平衡处理——确保我们知道是白色(或灰色)的物体具有恰当的色度。

一些相机允许使用者通过菜单来设定照明的色温，通常的选项有钨灯光、荧光、日光和闪光，它们其实是选择矩阵 **J** 不同的预设值。在手动白平衡中，相机先对准一个灰色或白色物体，然后按下快门。相机将调整它的通道增益 **J** 以便产生相同的三色刺激值 $R' = G' = B'$，这样就能得到我们需要的白光色度[①]。对于不是白色的颜色，这种修正会产生一些颜色误差，但是我们的眼睛仍然会感觉很满意。自动白平衡更经常用到，它融入了启发算法来消除光源色温，但也会被某种特殊颜色占优的场景所蒙蔽。

最实际的解决方案是用场景中已知色度的三种物体的三色刺激值。这可以使方程(10.13)中的矩阵 $\boldsymbol{C}$ 被直接估计，这样就能把传感器的三色刺激值映射为 XYZ 坐标，而这些坐标代表的表面反射率是完全与照明无关的。照明的色度也可以从这里被估计出来。这种方法可用于火星探测器上的全景相机，其标定靶如图 10.16 所示(见彩图)。在不断变化的火星照明条件下，通过定期获取该标定靶的图像就能修正白平衡。

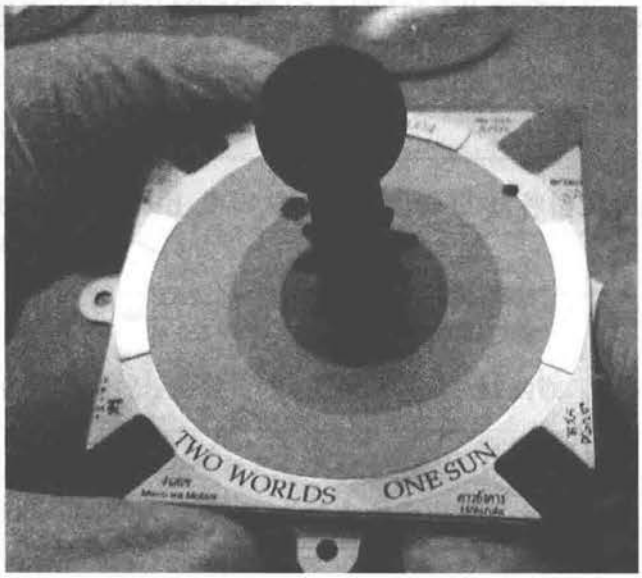


图 10.16 用于火星探测器上全景相机的标定靶。已知反射率和色度(红、黄、绿、蓝和灰色阴影)的区域和用于设定相机的白平衡。中心杆有非常低的反射率，也可充当日晷。它上面刻了一条格言，是关于日晷的最好传说。(照片来自NASA/JPL/Cornell/Jim Bell)

① 通常，$J_G = 1$，而 J_R 和 J_B 是可调的。

10.3.3 由于吸收的颜色变化

颜色问题的一个最极端例子发生在水下。例如，我们考虑一台机器人尝试去寻找一个带有彩色标识的停泊站。像 10.1.1 节中讨论的那样，水充当了过滤器的角色，吸收的红光比蓝光多。对于水下物体，这种过滤不仅影响着照射在物体上的照度，也影响到从物体反射回到摄像机的反射光，即亮度。再考虑红砖的例子：

```
>> [R,lambda] = loadspectrum([400:5:700]*1e-9, 'redbrick.dat');
```

现在是在 1 米深的水下，摄像机距离红砖 1 米多远。水面的照明就是地面的阳光：

```
>> sun = loadspectrum(lambda, 'solar.dat');
```

水的吸收光谱是

```
>> A = loadspectrum(lambda, 'water.dat');
```

穿过水的总路线长是

```
>> d = 2
```

透光率 T 由方程(10.2)的比尔定律给出：

```
>> T = 10 .^ (-d*A);
```

砖的最后亮度是

```
>> L = sun  .* R .* T;
```

如图 10.17 所示。我们看到那些波长较长的光已经被大大削减了。砖的显示颜色是

```
>> xy_water = lambda2xy(lambda, L)
xy_water =
    0.3722    0.3813    0.2465
```

该颜色也在图 10.15 的色度图中被绘制出来。水下的真实情况比这更加复杂。由于存在细小悬浮颗粒对光的散射，这些颗粒会进一步增加对光线的颜色过滤，也会把未经过目标反射的环境光反射到摄像机中。

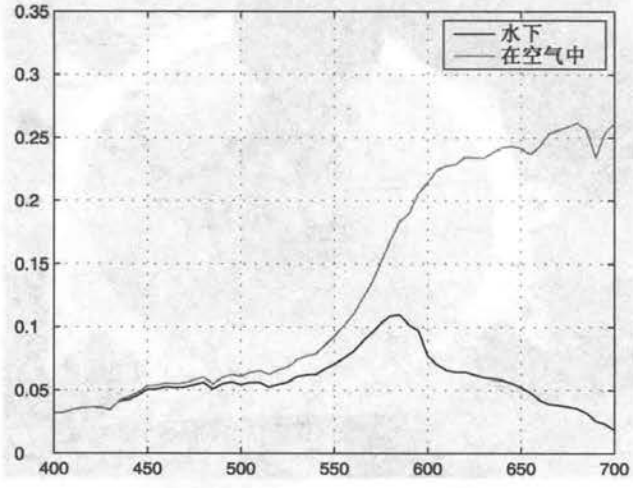


图 10.17 水下观察时的红砖光谱。没有水吸收的光谱用红线表示

10.3.4 伽玛

对于老旧的 CRT 显示器，其显示表面产生的亮度与控制电压 V 呈非线性关系：

$$L = V^{\gamma} \tag{10.14}$$

其中 $\gamma \approx 2.2$。为修正这种非线性，通常采用逆非线性 $V = L^{1/\gamma}$ 给它们的输出信号，这样就形成了一个从摄像机结束到显示器结束的线性系统[①]。

两种操作通常都称为伽玛修正，更准确地说，以摄像机结束的操作是伽玛编码，以显示器结束的操作是伽玛解码[②]。LCD 显示器比 CRT 显示器的非线性更强，但是在显示器中采用了修正表来使它具有标准 $\gamma \approx 2.2$ 的特性[③]。

为说明显示器中伽玛特性的影响，可创建一个简单的测试模式：

```
>> wedge = [0:0.1:1];
>> idisp(wedge)
```

如图 10.18 所示。它类似于摄影师的灰度阶梯条。如果在电脑屏幕上显示这个灰度条，它将与书中印刷出的效果不同。很可能观察到其中第二块和第三块之间的亮度有很大变化，这就是方程(10.14)描述的伽玛解码非线性对你的电脑显示器的影响。

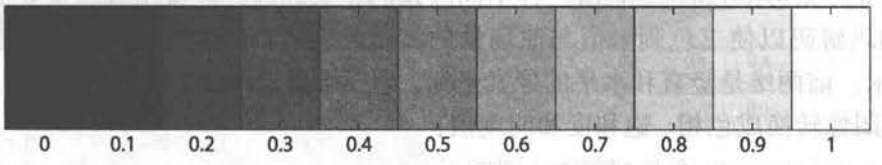


图 10.18　线性强度条

如果应用伽玛编码：

```
>> idisp( wedge .^ (1/2.2) )
```

就会观察到强度变化更加线性化[④]，且接近于书中印刷版的效果。

方程(10.8)和方程(10.9)中的色度坐标是作为与场景中亮度呈线性关系的三色刺激值的比率来计算的。因此在任何色度操作前，摄像机输出信号中的非线性都必须进行修正，即伽玛解码。工具箱函数 igamma 执行了这一操作。若函数 iread 在加载图像时使用了'gamma'选项，伽玛解码也会被自动执行。

如今，大多数的数码摄像机采用 sRGB 格式编码图像[⑤]，这种格式采用 ITU 的 Rec. 709 原色和如下的伽玛编码函数：

$$E' = \begin{cases} 12.92L, & L \leqslant 0.0031308 \\ 1.055L^{1/2.4} - 0.055, & L > 0.0031308 \end{cases}$$

它包含了两段，对小数值来说是线性函数，对大数值来说是指数函数。总的伽玛值接近于 2.2。

颜色空间(如 *HSV* 或 *xyY*)的重要特征是色度坐标相对于强度的变化是固定不变的。许多数码摄像机提供 *YUV* 或 YC_BC_R 格式的输出，其中包含一个亮度分量 *Y* 以及其他两个经常被误

---

① 很多相机都有一个选项来选择伽玛值为 1 或 0.45(=1/2.2)。

② 伽玛编码和伽玛解码经常也被称为伽玛压缩和伽玛解压，因为编码操作压缩了信号的范围，而解码操作则将信号解压缩回来。

③ Macintosh 操作系统的计算机是例外，MacOS 10.6 之前版本的计算机使用 $\gamma = 1.8$，它使屏幕颜色显得更亮、更生动。

④ 对于 MacOS 10.6 之前的 Macintosh 操作系统，使用 $\gamma = 1.8$，而不是 2.2。

⑤ JPEG 文件的头(JFIF 文件格式)有一个标签 Color Space，如果伽玛值或颜色模型未知，该标签将被设置为 sRGB 或 Uncalibrated。

认为色度坐标的分量成分。然而它们并不是色度坐标，实际上它们是颜色差异信号，类似于 U, $C_B \propto B' - Y'$和 V, $C_R \propto R' - Y'$，其中 R'、B'是伽玛编码的三色刺激值，Y'是伽玛编码的强度。伽玛的非线性意味着 UV或 C_BC_R在总体照明水平改变时将不再是一个常量。

来自摄像机的三色刺激值必须首先通过适当的伽玛解码转换成线性三色刺激值，然后再计算色度值。没有捷径。

10.3.5 应用：彩色图像

在这一节中，我们将会把本章介绍的许多概念和工具联系在一起使用，也会预习将在下章介绍的很多函数。我们考虑一个花园场景：

```
>> flowers = iread('flowers4.png', 'double', 'gamma', 'sRGB');
```

如图 10.19(a)(见彩图)所示，它由三种不同颜色的花朵和绿色背景组成。重要的是，我们已经应用了伽玛解码以使三色刺激值与原场景的亮度成比例。图像 flowers 有三个维度，如图 12.2 所示。前两维是竖直和水平的像素坐标，第三维是选择红、绿、蓝像素的颜色平面。

我们把图像转换成色相、饱和度和明度值：

```
>> hsv = colorspace('RGB->HSV', flowers);
```

结果是另一个三维矩阵，但是现在颜色平面代表了色相、饱和度和明度值。我们可以显示色相

```
>> idisp( hsv(:,:,1) )
```

以及饱和度

```
>> idisp( hsv(:,:,2) )
```

图像分别如图 10.14(b)和如图 10.14(c)所示。

如果将每个像素的色度绘制成色度平面上的点，就会观察到对应于该场景不同部分的点族，如红花、黄花、绿叶和阴影。我们把彩色 RGB 图像转换成 XYZ 图像：

```
>> XYZ = colorspace('RGB->XYZ', flowers);
```

然后再转换到 xy 色度坐标

```
>>  [x,y] = tristim2cc(XYZ);
```

其中，x 和 y 是大小与 flowers 相同的每个图像。接着我们计算出(x, y)值的二维直方图，每一维分 100 个格：

```
>> xbins = [0 0.01 100];  ybins = [0 0.01 100];
>> [h,vx,vy] = hist2d(x, y, xbins, ybins);
```

其中，h 是每一格中的点数，vx 和 vy 是相应格的 x、y 坐标。我们把直方图显示为一个等高线图，并叠加到 xy 色度图上：

```
>> xycolorspace
>> hold on
>> contour(vx, vy, h)
```

如图 10.19(b)所示。可以看到图中有 5 到 6 个顶点：在红色区域的宽阔顶点，狭长的白色顶点，以及在绿色区域的几个顶点。

接着我们将会用 k 均值算法进行无监督的聚类：

```
>> [cls, cxy] = colorkmeans(flowers, 7);
```

其中第二个参数指定了像素色度类型或群簇的数量，这里已经设定为 7[①]。k 均值聚类算法是

① k 均值算法的一个限制是必须提前知晓群族的数量，通常由领域知识操控。

迭代的，它会调节每个群簇中心的估计值以及给群簇的像素分配值，直到实现平衡。初始的群簇中心是随机选择的，这就意味着函数在每次运行时都会得出一个不同的结果。

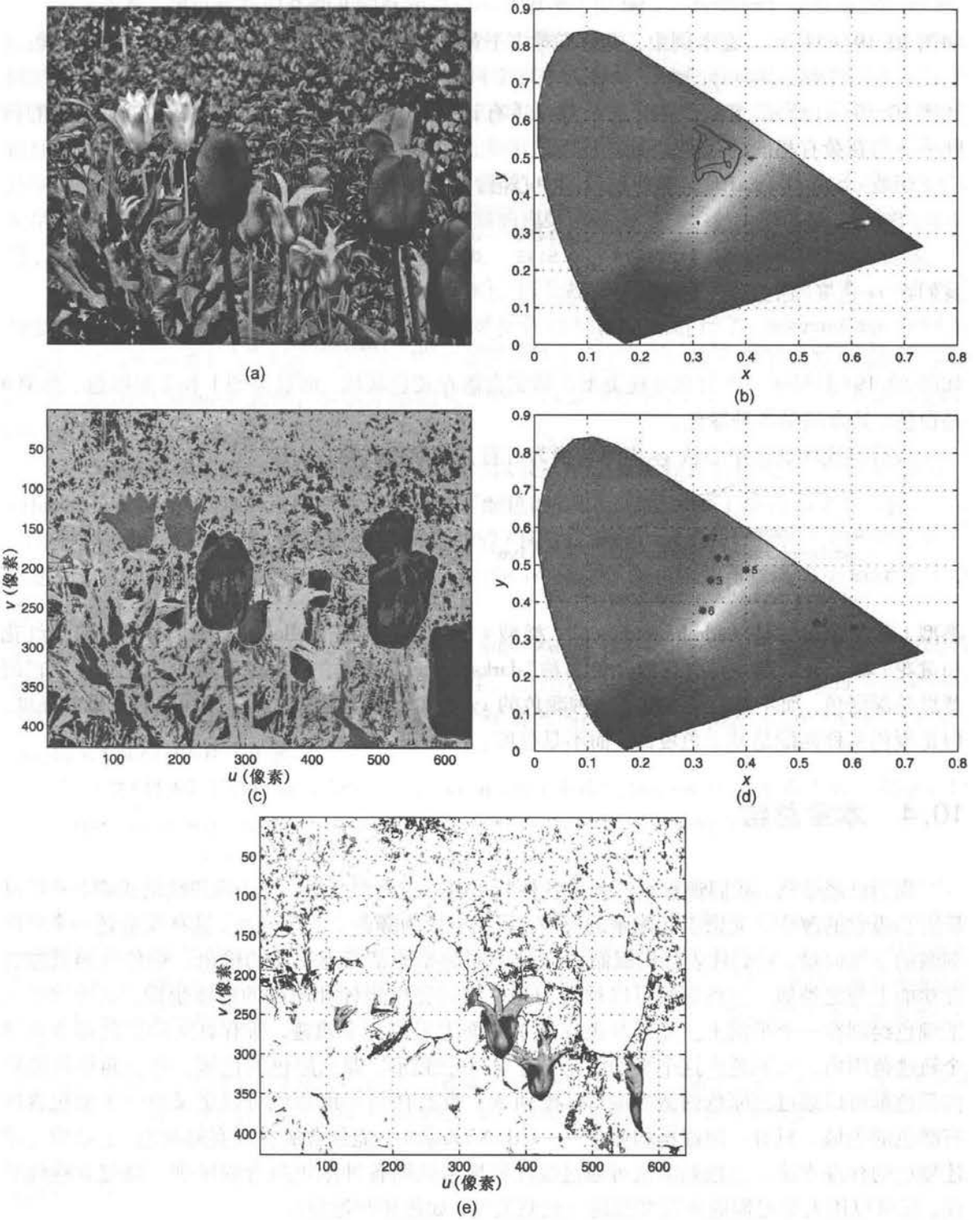


图 10.19　彩色图像分割。(a)原始彩色图像；(b)叠加了频率等高线的 xy 色度平面；(c)颜色错乱的标签图像；(d)色度图上的群族中心；(e)只有类型5(黄色)像素

结果是另一个图像 cls，其中每个像素值表示 flowers 中对应像素被分配到的群族，即它的颜色分类或类型，是 1 ~ 7 之间的一个整数。我们可以显示该图像：

```
>> idisp(cls, 'colormap', 'jet', 'nogui')
```

如图 10.19(c)所示。在本例中，类型 5 对应于黄色花朵，我们可以仅仅显示该类型的像素：

```
>> idisplabel(flowers, cls, 5)
```

如图 10.19(e)所示。除了黄色花朵外，还有许多小的黑色像素群，它们对应于部分的树叶——与花朵有相同的色度，只是亮度低。

函数 colorkmeans 也返回群簇的中心值，在 *xy* 空间是

```
cxy =
    0.6082    0.5378    0.3328    0.3466    0.4017    0.3176    0.3238
    0.3327    0.3436    0.4599    0.5195    0.4874    0.3788    0.5744
```

我们在 *xy* 色度图上把它们绘制出来：

```
>> xycolorspace
>> plot_point(cxy, '*', 'sequence', 'textsize', 10, 'textcolor', 'b')
```

如图 10.19(d)所示，我们会发现类型 5 确实会落在黄色区域，而且类型 1 和 2 是红色，类型 6 是白色，其余的是各种绿色。

最后将这些群簇中心从 *xy* 坐标转换为有意义的颜色名称：

```
>> colorname(cxy, 'xy')
ans =
  Columns 1 through 5
    'cadmiumreddeep'    'brown3'    'olive'    'terreverte'    'yellow4'
  Columns 6 through 7
    'darkseagreen4'    'yellowgreen'
```

类型 1 的颜色名称是“cadmiumreddeep”，类型 4 的颜色名称是“yellow4”，它们分别对应于红花和黄花，都十分恰当。对白花的颜色名称“darkseagreen”来说，就多少有些令人吃惊了，它的意思是深绿色，而不是白色，但是这种颜色的 *xy* 色度十分接近白色点。深绿色有很低的亮度，但是颜色名称匹配是基于色度的，而不是亮度。

10.4 本章总结

我们已经学到，我们观察到的光是各种频率电磁波的混合体，是一段连续的光谱，并经过反射和吸收的改变。光谱引起眼睛的反应，我们解读为颜色。对于人类，这种反应是一个三色刺激的三维向量，它们代表我们眼睛中三种不同类型圆锥形感受器的输出。彩色数码摄像机在功能上与之类似。三色刺激可以被认为是一维的亮度坐标和两维的色度坐标，这样就可以把颜色绘制在一个平面上。光谱颜色在这个平面上形成一个轨迹，所有真实的颜色都落在这个轨迹范围内。三原色在这个平面上形成了一个三角形，即三原色的色域。在三角形内的任何颜色都可以通过三原色的适当混合匹配而成。没有任何一组原色可以定义出一个能包含所有颜色的色域。只有一组虚构的原色——CIE XYZ 系统，能包含所有真实的颜色，它已成为描述颜色的标准方法。三色刺激值可通过线性转换来说明各种原色组合的区别。通过非线性转换，还可以用人类习惯的语言来描述三色刺激值，如色相与饱和度。

我们也讨论了白色的定义、白平衡问题、显示器的非线性反应以及它怎样影响图像和影像的显示。最后，我们用色度信息对真实世界彩色图像上的像素进行了分类。

扩展阅读

表面上看，颜色是我们在幼儿园就知道的一个概念，但实际上它是一个非常复杂的议题。在本章中，我们仅仅是开始触及有关光度学和色度学的表皮。光度学是涉及可见光测量的辐射测量科学的一部分，它对于工程师和计算机科学家来说都是一种挑战，因为其中用到了很多不常见的单位，如流明、球面度、尼特、坎德拉和勒克斯。复杂的另一个原因是像强度和亮度这样的词汇在日常用语中是同义词，但是在光度学中却有着各自特定的含义。色度学是颜色感知的一门科学，它也是一个很大、很复杂的领域，因为人类对颜色的感知取决于很多因素，包括观察者个人、周围环境的照明，甚至是观察的视野。然而色度学对于摄像机、计算机显示器、影像设备和打印机的设计都是至关重要的。

冈萨雷斯和伍兹[Gonzalez and Woods 2008]，以及福赛斯和庞塞[Forsyth and Ponce 2002]分别编著了有关计算机视觉的两本教材，都对颜色和颜色空间进行了讨论，后者还就阴影和互反射的影响进行了讨论。关于计算机视觉的详细在线信息可从 http://homepages.inf.ed.ac.uk/rbf/CVonline 获得，本章介绍的材料在“Image Physics and Sensors and their Properties”这部分中。

关于颜色科学的综合书籍包括[Hunt 1987]以及从电视或工程视角讲述的[Benson 1986]。[Hunter and Harold 1987]采用对话形式讲述了颜色的知识，它还涉及了颜色感受的其他方面，如光彩和光泽。CIE 标准(国际照明委员会 1987)具有权威性，但是很难阅读。CIE 的工作还在进行，它的标准会定期更新，并发布在 www.cie.co.at 上。颜色匹配函数最初在 1931 年表格化，在 1964 年进行了修正。

查尔斯·博伊敦(Charles Poynton)已经在 http://www.poynton.com 上开展关于颜色空间和伽玛的在线教学很长时间了，而且效果极佳。他还出版了介绍其课程内容的书[Poynton 2003]，其中更深入地讨论了数码影像系统。有关伽玛的内容也可参见 http://www.w3.org/Graphics/Color/sRGB 中的描述。

其他 MATLAB 工具包括在 http://cs.joensuu.fi/colorlab_toolbox/中的 ColorLab 工具箱，以及在 http://www.math.ucla.edu/~getreuer/colorspace.html 中的 colorspace 函数。

背景材料：红外线相机。一般相机与人眼在功能上是等价的，对可见光谱很敏感。对红外线很敏感的相机现在也有，而且 CIE 定义了多种红外线波段：IR-A(700 ~ 1400 nm)、IR-B(1400 ~ 3000 nm)、IR-C(3000 nm ~ 1000 μm)。在通常的用法中，IR-A 和 IR-B 被分别称为近红外光谱(NIR)和短波红外线(SWIR)，而 IR-C 则是中波段红外(MWIR，3000 ~ 8000 nm)和长波段红外(LWIR，8000 - 15000 nm)。LWIR 相机也被称为热成像相机。通常紫外线相机工作在靠近紫外线区域(NUV，22 ~ 380 nm)，主要在工业中应用，如在高压电系统中探测电晕放电。

超光谱相机中的感光器种类超过三个，它们通常从红外到紫外的很多点采样入射的光谱，有几十甚至几百个光谱波段。超光谱相机的应用包括对土地资源的航空探测以及对岩石中矿物成分的鉴别。

数据来源

工具箱包括许多描述各种光谱的数据文件，总结在表 10.3 中。每个文件都以米级的波长作为它的首列。文件有不同的波长范围和间隔区间，但是帮助函数 `laodspectrum` 能将数据插入到用户指定的范围和采样区间。

表 10.3 工具箱提供的各种光谱。相对亮度值在区间[0,1]中，相对功率谱分布(SPD)被标准化为在波长550 nm的值等于1.0

文件名	单位	描述
cones.dat	相对亮度	人眼圆锥体的光谱反应
photopic.dat	相对亮度	CIE 1924 明视反应
scotopic.dat	相对亮度	CIE 1951 暗视反应
redbrick.dat	反射率	老旧红砖的反射光谱
solar.dat	$\mathrm{Wm^{-2}nm^{-1}}$	地面上的太阳光谱
water.dat	$\mathrm{lm^{-1}}$	水的光吸收光谱
D65.dat	相对功率谱分布	CIE 标准 D_{65} 光源

本书配套网站上的多个网址中都提供表格形式的光谱数据，包括许多材料的反射率数据，由 NASA 的在线 ASTER 光谱图书馆和来自东芬兰大学色彩研究实验室的数据库提供。圆锥体反应和 CIE 颜色匹配函数来自英国的伦敦大学学院色彩与视觉研究实验室。CIE 的数据也可从网上获得。

习题

1. 你是一个黑色辐射体吗？画出你自己的黑体辐射光谱。你的峰值辐射频率是多少？它属于 EM 光谱的哪部分？你是用什么传感器来测量的？
2. 有一个传感器，它测出对应波长为 λ_1 和 λ_2 的辐射能力分别为 P_1 和 P_2。请基于这四个参数写出黑体温度 T 的表达式。
3. 请用斯蒂芬-玻尔兹曼定律计算太阳表面每平方米发射出的能量。再计算太阳发出的总能量。
4. 请用数值积分方法计算太阳表面每平方米可见光波段(400 ~ 700 nm)发射出的能量。
5. 为何峰值亮度被定义为 683 $\mathrm{lmW^{-1}}$？
6. 根据第 223 页给出的典型室外照明条件，确定太阳光的照明强度。
7. 关于地面上的阳光。请用百分比的形式，确定辐射能量中红外线、可见光和紫外线各占的比例。
8. 请用数值积分方法计算钨灯在 2600 K 时每平方米可见光波段(400 ~ 700 nm)发射出的能量。它占总发射能量的比例是多少？
9. 绘制出人眼的明视光谱反应和暗视光谱反应图，并进行比较。
10. 你能为红砖创造出一个条件等色体(谱成分不同但看上去颜色完全相同)吗？
11. 证明重心定理。
12. 在 *xy* 色度平面上绘制出温度在 1000 K ~ 10 000 K 之间的黑色辐射体的轨迹。
13. 在 *rg* 平面上绘制出 XYZ 的原色。
14. 格灵达-麦克贝斯色卡是一个有 24 种颜色方块的阵列，如图 10.20 所示。该色卡的光谱数据可以从 http://www.rmimaging.com/information/colorchecker.html 获得。计算并绘制出每种颜色的 *xy* 色度。
15. 对于图 10.11，确定其中可用绿色的色度。假设所有原色的值都在[0,1]的范围内，确定最亮的可能三色刺激值。
16. 修改函数 `xycolorspaces`，来产生一个 *rg* 色度平面。

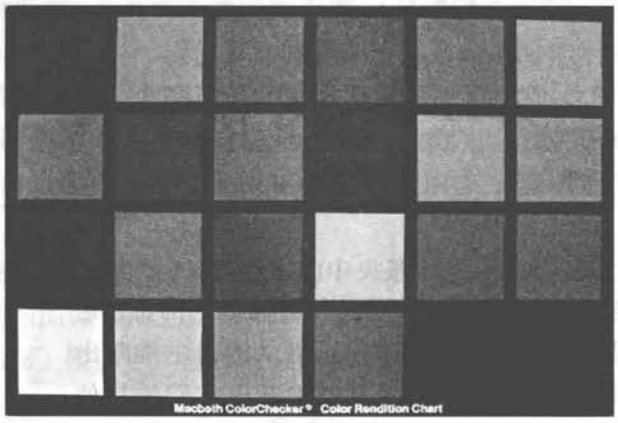


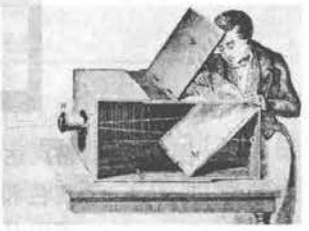
图 10.20 格灵达-麦克贝斯色卡是一个有 24 种颜色方块的阵列，其中包括了不同的灰色和彩色，以及皮肤、天空、树叶等的光谱刺激。这些颜色的光谱数据可以从http://www.cis.rit.edu/research/mcsl/online/cie.php在线获得

17. 使用 Rec. 709 原色，确定红砖的三色刺激值。
18. 在白炽灯照明条件下拍摄一张白色物体的照片。确定平均的 RGB 三色刺激值，并计算其 xy 色度。该色度与白色的色度相差多大？确定颜色平衡矩阵 $\boldsymbol{J}$ 来修正色度值。照明的色度是什么？
19. 在水下观看时，红砖的颜色名称是什么？
20. 画一个如图 10.16 所示的标定靶，它有三种已知色度的颜色块。根据这些颜色块被观测到的色度值，确定从观测到的三色刺激值到 Rec. 709 原色的变换。照明的色度是什么？
21. 考虑一个水下应用的情形，其中有一个在水下 d 米的目标，它被从 m 米远距离的水中观察，水面有阳光照明。你能根据被观测到的色度值确定该目标的真实色度吗？这种估计对于 m 和 d 的误差有多敏感？如果你知道目标的真实色度，能确定它的距离吗？
22. 是否有这种可能，两种不同颜色在某种特殊光照条件下看上去相同？创造出一个能呈现这种效果的颜色和光照的例子。
23. 用你自己的一张照片重复 10.3.5 节中的示例练习。你能区分照片中的不同物体吗？
24. 通过分析或数据证明：比例缩放三色刺激值不会对色度产生影响。如果色度是基于伽马编码的三色刺激值计算的，结果会怎样？

第11章 图像形成

一个小洞察，就像一个小幽默，影响深远。

艾伦·克莱因



在这一章中我们会讨论图像是怎样形成和被捕捉到的，它是机器人和人类感知世界的第一步。我们不仅可以从图像中推断出周围环境中物体的大小、形状和位置，也可以推断出它们的其他特征，如颜色和质地。

很久以前我们就知道，一个简单的小孔能够在暗室的墙上产生一幅完好的倒像。一些海洋软体动物有着类似针孔相机的眼睛，例如鹦鹉螺。所有脊椎动物都是通过一个透镜在视网膜上形成倒像的，而在视网膜上分布着在图 10.6 中所展现的杆状和锥状的光敏细胞。数码相机在原理上与其相似：通过一组玻璃或塑料的透镜在半导体芯片表面形成图像，而芯片上的光感器件阵列则把光转换成数字图像。

眼睛和照相机的成像过程都包含把三维世界投影在二维平面上，从而失去了深度信息，以致我们无法从图像中辨别它是一个远处的大物体，还是一个近处的小物体。这种从三维到二维的转变称为透视投影，这部分内容将在 11.1 节介绍。11.2 节中讨论了照相机校准和透视变换的参数评估这一主题。而在 11.2.3 节中我们会讨论其逆问题，即给定二维图像，如何重建三维世界中的点。在 11.3 节中介绍其他类型的相机，包括具有广角和全景成像功能的相机。

11.1 透视变换

由于针孔相机的辐射能量等于小孔尺寸乘以环境照度（单位：Wm^{-2}），所以它产生的图像非常暗淡。要使图像明亮的关键在于采用透镜或曲面镜进行大面积采光。凸透镜就像针孔一样会产生图像，但是透镜的直径更大，可以允许更多的光通过，使得产生的图像更明亮。

通过一个薄透镜[①]形成图像的基本情形如图 11.1 所示。正方向的 z 轴是相机的光轴，物体的 z 坐标及其图像满足透镜定律：

$$\frac{1}{z_o}+\frac{1}{z_i}=\frac{1}{f} \qquad (11.1)$$

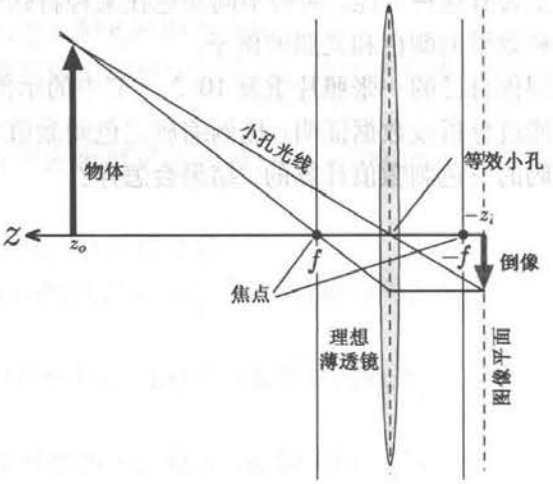


图 11.1 薄凸透镜成像几何的二维横截面：透镜在其两侧各有一个焦点且等距离。按照惯例，照相机的光轴是 z 轴

① 实际相机的透镜是由多个透镜单元组成的，但是在复合透镜组合体的两侧仍然存在焦点。——译者注

其中，z_o 是物体与透镜的距离，z_i 是像与透镜的距离，f 是透镜的焦距。当 $z_o > f$ 时，在 $z_o < -f$ 图像平面上会形成倒像。在相机中，图像平面固定在传感芯片表面，所以通过相机的调焦环让透镜沿着光轴移动，就能使图像平面到透镜的距离为 z_i（对于无穷远处的物体，$z_i = f$）。针孔照相机不需要调焦，因为调焦是为了抵消透镜增加的聚光能力。我们的眼睛里有单一的一个凸透镜，由透明晶体蛋白组成，而高质量照相机则采用由多个玻璃或塑料透镜制成的复合透镜。

在计算机视觉中，通常应用如图 11.2 所示的中央透射成像模型。光线在相机坐标系 $\{C\}$ 原点聚合，并在 $z = f$ 处的图像平面上投射出一个正像。应用相似三角形可以证明，世界坐标系中的一点 $P = (X, Y, Z)$ 根据下式被投射到图像平面上 $p = (x, y)$：

$$x = f\frac{X}{Z}, y = f\frac{Y}{Z} \tag{11.2}$$

背景材料： 在公元前五世纪，中国古代哲学家墨子提到了小孔成倒像的作用。成像器是一个暗房，光线通过小孔进入，并将一个昏暗的倒像投射到暗房的墙上。增大小孔尺寸可以增加图像的亮度，但是聚焦性减弱。

暗房成像器在维多利亚时代是著名的旅游景点，尤其是在英国，现在仍有很多在工作。（右侧图画来自光学仪器收藏绘画 http://vision.mpiwg-berlin.mpg.de/elib）

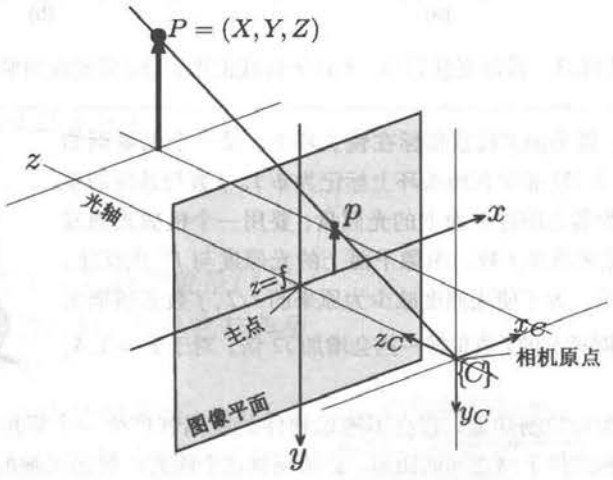


图 11.2　中央透射模型。图像平面是在距离相机原点前 f 的位置，在该平面上形成的是一个正像。相机坐标系采用右手系，其 z 轴定义了视场中心

这就是从空间到图像平面的投影变换，更确切地说是透视投影，它具有以下特征：

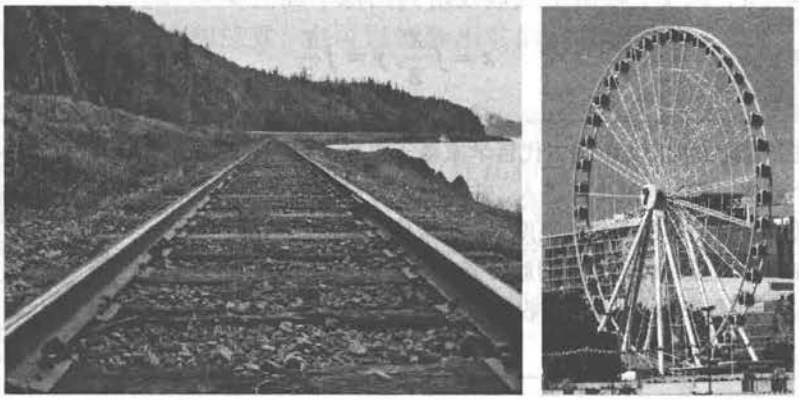
1. 执行了从三维空间到二维像平面的映射：$\mathbb{R}^3 \mapsto \mathbb{R}^2$。
2. 空间中的直线在像平面上的投影仍为直线。
3. 空间中的平行直线投影为如图 11.3(a) 所示的相交于远处消失点的直线，这个效应被称为前缩透视法，但平行于像平面的平面中的直线例外，这些直线不会聚合。
4. 空间中的圆锥截面投影到像平面仍为圆锥截面。例如，一个圆投影后还是一个圆，或者是如图 11.3(b) 所示的椭圆。
5. 这种映射不是一对一的，其逆映射并不唯一。也就是说，给定像平面的点 (x, y) 不能

唯一确定其在空间中的点 (X,Y,Z)。我们只能说空间中的点一定在如图 11.2 所示的射线 OP 上。这个重要命题会在第 14 章中讲解。

6. 这种变换不是保角的——它不能维持原形，因为内角无法维持原来的大小。平移、旋转和比例缩放都是保角变换的样例。一般仿射变换包含对每个轴的平移、旋转和不同比例的缩放，不是保角的。

我们可以写出图像平面上一个点的齐次形式 $\tilde{\boldsymbol{p}} = (x',y',z')$，其中：

$$x' = fX, y' = fY, z' = Z$$



(a)　　(b)

图 11.3　投影变换效应。(a)平行线汇聚；(b)圆变成为椭圆

背景材料：镜头光圈。镜头的 f 数通常标在镜头环上，是一个无量纲的量 $F = f/d$，其中 d 是镜头直径(通常在镜头环上标记为 ϕ)。f 数与透镜的聚光能力成逆相关。为了减少落在图像平面上的光照量，要用一个机械光圈或虹膜来减小有效直径，以此来增加 f 数。图像平面上的光照度与 F^2 成反比，因为光照度取决于聚光面积。为了使光照度减少为原来的 1/2，f 数必须增大 $\sqrt{2}$ 倍或“一挡”。在透镜光圈环上的 f 数每转一挡会增加 $\sqrt{2}$ 倍。对于 $F = 1.4$，f 数按惯例写成 $f/1.4$。

景深。针孔相机没有焦距控制功能，它会不考虑物体的距离就产生一个聚焦图像。而透镜没有这种性质——调焦环改变了透镜和图像平面之间的距离，必须调整这个距离以使感兴趣的目标能够清晰聚焦。摄影师照相时必须参照景深，即物距范围，才能形成满意的聚焦图像。光圈设置较小时(透镜更像是一个针孔)，景深要求大，但这意味着入射光线少，图像杂乱，或者曝光时间长，动感模糊。这就是摄像师的烦恼！

或者，采用更简洁的矩阵形式：

$$\tilde{\boldsymbol{p}} = \begin{pmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \tag{11.3}$$

其中，非齐次的图像平面坐标是

$$x = \frac{x'}{z'}, y = \frac{y'}{z'}$$

它们通常称为视网膜像平面坐标。对于 $f = 1$ 的情况，这些坐标称为归一化或标准的像平面坐标。

如果把空间点的坐标也写成齐次形式 ${}^{C}\tilde{\boldsymbol{P}} = (X,Y,Z,1)^{\mathrm{T}}$，则透视投影可表示为如下线性形式：

$$\tilde{\boldsymbol{p}} = \begin{pmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} {}^{C}\tilde{\boldsymbol{P}} \tag{11.4}$$

或者

$$\tilde{\boldsymbol{p}} = \boldsymbol{C}\, {}^{C}\tilde{\boldsymbol{P}} \tag{11.5}$$

其中，$\boldsymbol{C}$ 是一个 3 × 4 矩阵，也称为相机矩阵。要注意的是，我们用 ${}^{C}\tilde{\boldsymbol{P}}$ 来表示空间点的坐标，这是为了强调该点坐标是关于相机坐标系 $\{C\}$ 的。波浪号表示它是齐次量，附录 I 提供了齐次坐标的补充知识。$\boldsymbol{C}$ 的第三列是在世界坐标系中平行于光轴的一个向量。相机矩阵可以因式分解为

$$\tilde{\boldsymbol{p}} = \begin{pmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} {}^{C}\tilde{\boldsymbol{P}}$$

其中第二个矩阵是投影矩阵。

我们可以利用工具箱创建一个中央投影照相机的模型，例如

```
>> cam = CentralCamera('focal', 0.015);
```

它会返回一个 CentralCamera 对象的实例，该对象有一个 15 mm 的透镜。默认情况下，相机在世界坐标系的原点，其光轴指向世界坐标系的 z 方向，如图 11.2 所示。我们定义一个空间点：

```
>> P = [0.3, 0.4, 3.0]';
```

单位是米，它对应的像平面坐标是

```
>> cam.project(P)
ans =
    0.0015
    0.0020
```

这个像平面上的点相对于主点的位置是(1.5, 2.0)，这是一个非常小的位移，但是它与典型的图像传感器大小相匹配。

通常，在世界坐标系中相机具有一个任意位姿 ξ_C，如图 11.4 所示。空间点相对于照相机坐标系的位置是

$${}^{C}\boldsymbol{P} = (\ominus \xi_C) \bullet {}^{0}\boldsymbol{P}$$

或者齐次坐标形式为

$${}^{C}\boldsymbol{P} = \boldsymbol{T}_C^{-1}\, {}^{0}\boldsymbol{P} \tag{11.6}$$

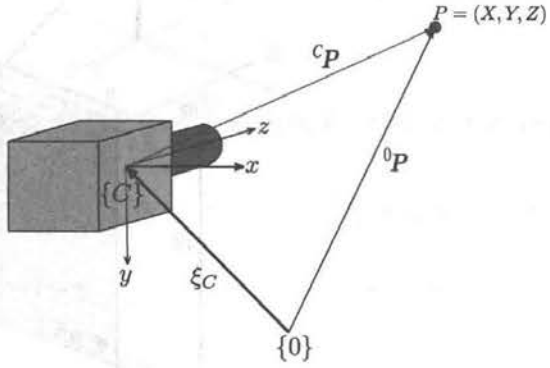


图 11.4　相机的坐标系

我们可以通过将照相机向左移动 0.5 m 来进行证明上述关系：

```
>> cam.project(P, 'Tcam', transl(-0.5, 0, 0) )
ans =
    0.0040
    0.0020
```

其中，第三个参数是作为齐次变换的相机位姿 ξ_C。我们可以看出 x 坐标从 1.5 mm 增加到 4.0 mm，这就是说图像点向右移动了。

背景材料：图像传感器。照相机芯片里的光敏单元即图素，通常是边长为 1 ~ 10 μm 的正方形。专业照相机的像素很大，用来增加光敏感度，而手机中照相机的传感器较小，因而光敏性较低。水平和竖直像素的数量比即纵横比，通常为4∶3 或 16∶9。传感器的尺寸大小是按对角线的长度来测量的，通常单位为英寸，例如1/3、1/4、1/2 英寸。

在数码相机中，图像平面是一个 $W \times H$ 的光敏单元网格，光敏单元也称为图素，它直接与图像的像素相对应，如图 11.5 所示。像素坐标是一个非负整数的二维向量(u,v)，按惯例，坐标系原点在图像平面的左上角。在 MATLAB 中，左上角的像素坐标是(1,1)。每个像素的大小一致，并位于各自网格的中心，所以像素坐标与图像平面坐标直接相关，即

$$u = \frac{x}{\rho_w} + u_0, v = \frac{y}{\rho_h} + v_0$$

其中，ρ_w、ρ_h 是每个像素点各自的宽和高，(u_0,v_0)是主点坐标——主点是图像平面与光轴的交点。通过预设相机参数矩阵为 $\boldsymbol{K}$，可以写出像素坐标的方程(11.4)：

$$\tilde{\boldsymbol{p}} = \underbrace{\begin{pmatrix} 1/\rho_w & 0 & u_0 \\ 0 & 1/\rho_h & v_0 \\ 0 & 0 & 1 \end{pmatrix}}_{\boldsymbol{K}} \begin{pmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} {}^C\tilde{\boldsymbol{P}} \tag{11.7}$$

其中，$\tilde{\boldsymbol{p}} = (u', v', w')$ 是空间点 $\boldsymbol{P}$ 在像素坐标系中的齐次坐标①。非齐次的图像平面像素坐标是

$$\ddot{\boldsymbol{x}}_{\text{av}} = \frac{\dot{\boldsymbol{x}}_{k+1} - \dot{\boldsymbol{x}}_k}{t_{\text{acc}}} \tag{11.8}$$

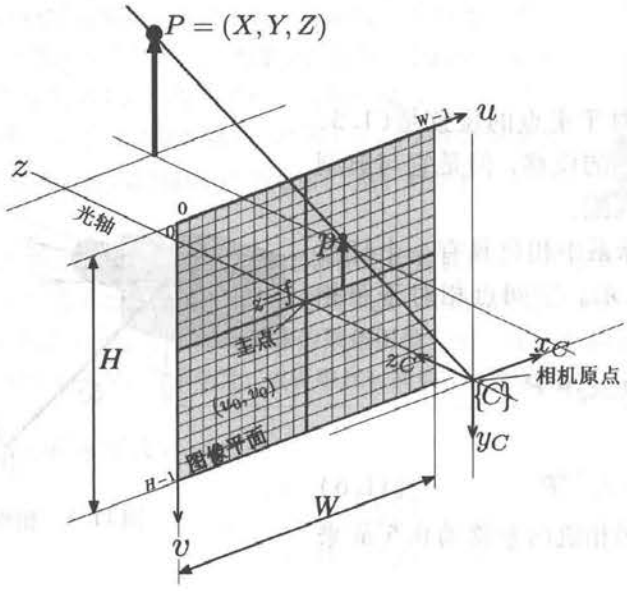


图 11.5　中央投影模型：展示了图像平面和离散的像素

① 矩阵 $\boldsymbol{K}$ 通常写为 $\boldsymbol{K}[1,2]$ 的有限值来代表其斜对称性，这是考虑到了 u 轴和 v 轴非正交的事实。但随着半导体制造工艺精度的提升，这种不正交的可能性很小。

例如，如果每个像素大小是10 μm的方块，像素阵列为1280×1024，并且主点在图像平面的坐标为(640,512)，则有

```
>> cam = CentralCamera('focal', 0.015, 'pixel', 10e-6, ...
    'resolution', [1280 1024], 'centre', [640 512], 'name', 'mycamera')
name: mycamera [central-perspective]
  focal length:   0.015
  pixel size:     (1e-05, 1e-05)
  principal pt:   (640, 512)
  number pixels:  1280 x 1024
  Tcam:
     1  0  0  0
     0  1  0  0
     0  0  1  0
     0  0  0  1
```

以上显示了图像模型的参数。前面定义的空间点对应的非齐次图像平面坐标是

```
>> cam.project(P)
ans =
   790
   712
```

结合方程(11.6)和方程(11.7)，我们可以写出一般形式的相机投影为

$$\begin{aligned}\tilde{\boldsymbol{p}} &= \underbrace{\begin{pmatrix} f/\rho_w & 0 & u_0 \\ 0 & f/\rho_h & v_0 \\ 0 & 0 & 1 \end{pmatrix}}_{\text{内在}} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \underbrace{({}^0\mathbf{T}_C)^{-1}}_{\text{外在}} \tilde{\boldsymbol{P}} \\ &= \boldsymbol{K}\boldsymbol{P}_0\, {}^0\boldsymbol{T}_C^{-1}\tilde{\boldsymbol{P}} \\ &= \boldsymbol{C}\tilde{\boldsymbol{P}}\end{aligned} \tag{11.9}$$

其中，所有项都被包进相机矩阵 $\boldsymbol{C}$ 中[①]。这是一个3×4的齐次变换，执行了缩放、平移和透视投影。该矩阵通常也称为投影矩阵或相机标定矩阵。

我们已经提到过透射投影的本质不明确性，即无法辨别目标是大而远，还是小而近。我们可以重写方程(11.9)：

$$\tilde{\boldsymbol{p}} = (\boldsymbol{C}\boldsymbol{H}^{-1})(\boldsymbol{H}\tilde{\boldsymbol{P}}) = \boldsymbol{C}'\tilde{\boldsymbol{P}}'$$

其中，$\boldsymbol{H}$ 是一个任意非奇异的3×3矩阵。这就意味着有无限多种的相机 $\boldsymbol{C}'$ 和空间点 $\tilde{\boldsymbol{P}}'$ 组合都会在图像平面得到相同的投影 $\tilde{\boldsymbol{p}}$。

这就阐明了从二维投影坐标来确定三维空间坐标的主要难点。我们只有知道相机或者三维物体的相关信息，才能解决这个问题。

该投影也可以写成如下函数形式：

$$\boldsymbol{p} = \mathcal{P}(\boldsymbol{P}, \boldsymbol{K}, \xi_C) \tag{11.10}$$

其中，$\boldsymbol{P}$ 是空间点，$\boldsymbol{K}$ 是相机参数矩阵，ξ_C 是相机位姿。

内在参数是相机和传感器的固有特征，包含 $f, \rho_w, \rho_h, u_0, v_0$；外在参数描述了相机的位姿，至少包含了六个参数来描述在 $SE(3)$ 中的位置和姿态。因此总共有11个参数。相机矩阵中有12个元素，所以有一个自由度是未被约束的，它就是总体比例因子。在实际应用中，这些相机的参数都

---

① f/ρ_w 和 f/ρ_h 是用像素单位表示的焦距长度。

是未知的，它们必须应用相机标定程序来进行估计，这部分内容将在 11.2 节中进行讲解。

当创建工具箱相机对象时，就自动生成了相机矩阵。在本示例中，相机矩阵 C 为

```
>> cam.C
ans =
   1.0e+03 *
    1.5000         0    0.6400         0
         0    1.5000    0.5120         0
         0         0    0.0010         0
```

相机的参数矩阵 K 为

```
>> cam.K
ans =
   1.0e+05 *
    1.0000         0    0.0064
         0    1.0000    0.0051
         0         0    0.0000
```

背景材料：镜头的视场是一个开口的四棱锥，在水平和竖直平面中的扩张角分别为 θ_h,θ_v。传感器芯片外形尺寸 d 采用对角测量，单位通常是英寸。常用尺寸为 1/3、1/4、1/2 英寸。一个标准镜头有 $f \approx d$，广角镜头通常有 $f > d/3$，能获得 110°左右的最大视场。

对于广角镜头来说，更常见的是用一个立体角来描述视场，度量单位是球面度(或 sr)。立体角是指视场投影到单位球面上的面积。半球面的视场为 2πsr，全球面的视场为 4πsr。如果我们通过顶角为 θ 的圆锥来粗略估计相机的视场，则相应的立体角为 $2\pi(1-\cos\theta/2)$ sr。

如果一个相机的视场大于半球面，则被称为全向或全景相机。

相机视场是一个关于焦距长度 f 的函数，广角镜头的焦距长度小，望远镜的焦距长度很大，而变焦镜头的焦距长度是可调的。视场可以根据图 11.5 所示的几何关系确定。水平方向上，视场角的一半为

$$\frac{\theta_h}{2} = \arctan\frac{W/2\rho_w}{f}$$

其中，W 是水平方向上像素点的数目。然后我们可以写出

$$\theta_h = 2\arctan\frac{W\rho_w}{2f}, \theta_v = 2\arctan\frac{H\rho_h}{2f} \tag{11.11}$$

我们注意到视场也是关于相机芯片尺寸 $W\rho_w \times H\rho_h$ 的函数。视场可以通过相机对象的 fov 方法进行计算：

```
>> cam.fov() * 180/pi
ans =
   46.2127   37.6930
```

结果是用度表示的水平方向和竖直方向各自的视场角。

CentralCamera 类是 Camera 类的子类，继承了投影多个点或多条线的能力。运用工具箱，我们可以在 xy 平面上建立一个 3 × 3 的点网格，其总边长为 0.2 m，中心点在(0,0,1)

```
>> P = mkgrid(3, 0.2, 'T', transl(0, 0, 1.0));
```

它会返回一个 3 × 9 的矩阵，其中每一列对应一个网格点，且每一列中的坐标顺序是 X,Y,Z。其前四列为

```
>> P(:,1:4)
ans =
   -0.1000   -0.1000   -0.1000         0
   -0.1000         0    0.1000   -0.1000
    1.0000    1.0000    1.0000    1.0000
```

默认情况下，函数 mkgrid 会在 xy 平面产生一个以原点为中心的网格。给该函数的最后一个可选参数是一个对默认点的齐次变换，它允许平面可以是任意位置和方向。

所有顶点的像平面坐标是

```
>> cam.project(P)
ans =
   490   490   490   640   640   640   790   790   790
   362   512   662   362   512   662   362   512   662
```

可以把它们绘制出来：

```
>> cam.plot(P)
```

得到如图 11.6(a)所示的虚拟相机视场。相机的位姿是

```
>> Tcam = transl(-1,0,0.5)*troty(0.9);
```

得到平面的斜视图：

```
>> cam.plot(P, 'Tcam', Tcam)
```

如图 11.6(b)所示。我们可以清楚地看到透视投影的效果，它扭曲了正方形的形状——原本平行的顶边和底边，投影后变成两条会聚于消失点的直线。

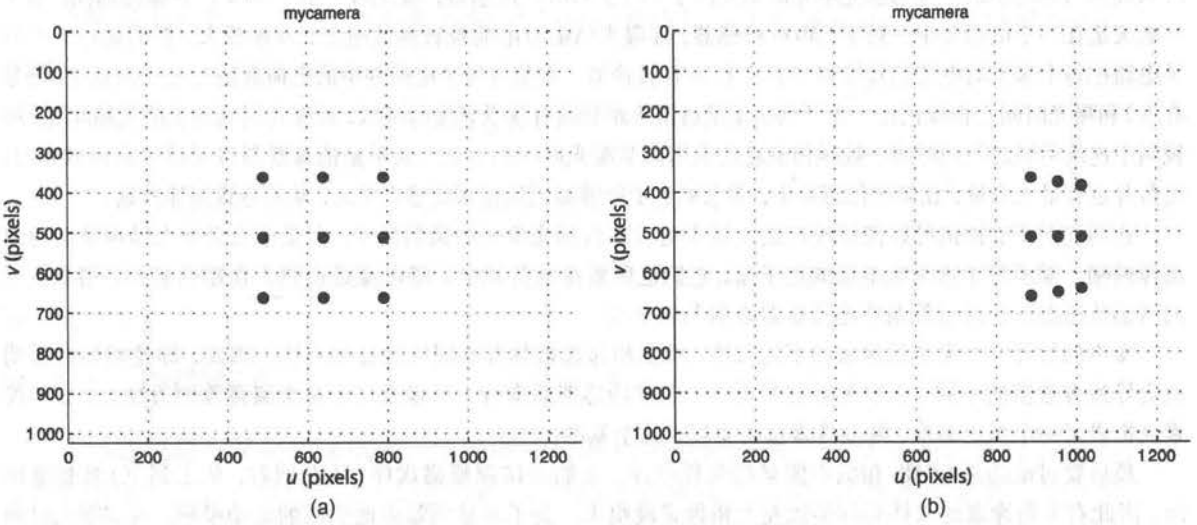


图 11.6　平面点网的两个视图。(a)正视图；(b)斜视图

一条直线的消失点可以根据其理想直线的投影来确定。网格的顶边和底边都平行于世界坐标系中的向量(1,0,0)，其理想直线是(1,0,0,0)。由于最后元素为0，这个齐次坐标直线存在于无穷远处。因此，消失点是

```
>> cam.project([1 0 0 0]', 'Tcam', Tcam)
ans =
   1.0e+03 *
    1.8303
    0.5120
```

它位于(1803,512)，刚好在可见像平面的右边。

方法 plot 可以选择性地返回像平面的坐标：

```
>> p = cam.plot(P, 'Tcam', Tcam)
```

它与方法 project 很相似。对于斜视图情形，像平面坐标为

```
>> p(:,1:4)
ans =
  887.7638  887.7638  887.7638  955.2451
  364.3330  512.0000  659.6670  374.9050
```

其中的值有小数部分，这意味着点没有被投影到图素中心。但在一个图素的表面上，任何位置对光的反应是相同的，所以离散的像素坐标可以通过把以上值圆整获得①。

一个三维的物体，如一个立方体，可以通过类似的方式定义并投影。一个边长为 0.2 m、中心在(0,1,1)的立方体，其顶点可以被定义为

```
>> cube = mkcube(0.2, 'T', transl([0, 0, 1]) );
```

它会返回一个 3×8 的矩阵，其中每一列对应一个顶点。图像平面上的点也可以用之前的方法绘出：

```
>> cam.plot(cube);
```

背景材料： 光子到像素值。在每个感光单元上，光子被转换成电子。转换成电子的光子与全部入射光子之比是传感器的量子效率。由于晶体管和其他设备的存在，感光单元表面上并不是所有区域都是光敏感的。光敏区域面积与总面积之比称为感光单元的填充因子，对于 CMOS 传感器，填充因子小于 50%。在曝光期间，电子都被聚集在一个电荷井中。对于 CMOS 传感器，在曝光结束时电荷被转换成电压，并被放大，然后通过一个开关电路把每个像素的电压信号接到一个片上 A/D 转换器。聚集在每个电荷井中的电荷数量与光子到达率(场景亮度)和曝光时间之积成正比。光子到达率是透镜焦距和所有场景亮度的函数，而曝光时间的上限是帧间隔，即视频中连续两帧之间的时间。数码相机通常采用调节曝光时间的方法，而不是依靠既昂贵又缓慢的机械光圈。电荷井也有最大容量，在某些传感器中，多余的电子能够溢出到相邻的感光单元，从而导致图像外延。

在弱光照下，相机可以使用模拟放大器来增大来自感光单元的微弱信号，但是这也会增大噪声从而导致图像模糊。噪声源于称为暗电流的电子流，它们也聚集在电荷井中。暗电流是由热工作用引起的，用于航天的敏感传感器往往通过低温冷冻传感器来削弱暗电流。

噪声的另外一个来源是像素的不均匀性，即各相邻像素具有不同的增益和补偿。因此，即使均匀的照明也会导致像素值的不同，表现为附加的噪声。图像传感器通常有一些像素位于被金属覆盖的边缘上，这些像素就提供了一个黑色基准，其他像素值需要减去这个基准。

最后要讨论的是 COMS 相机中像素的采样顺序，它们是按照栅格次序(从左到右，从上到下)被相继读出，因此右下角像素的采样时间要比左上角像素晚很多。为了尽量消除由此引起的运动模糊，减小曝光时间是必需的，但它又会导致图像更暗淡，噪声更大。

我们用另一种方法创建一个用边线来表示的立方体：

```
>> [X,Y,Z] = mkcube(0.2, 'T', transl([0, 0, 1.0]), 'edge');
```

然后显示出来：

```
>> cam.mesh(X, Y, Z)
```

如图 11.7 所示，包括正视图和斜视图：

```
>> Tcam = transl(-1,0,0.5)*troty(0.8);
>> cam.mesh(X, Y, Z, 'Tcam', Tcam);
```

这些边线采用了三维网格的格式②，与 MATLAB 内置函数(如 sphere，ellipsoid 和 cylinder)创建的格式相同。

---

① 对于 CMOS 传感器来说这不完全正确。在 CMOS 传感器中，晶体管通过填充因子减少了光敏感区域，即每个感光单元表面对光敏感的部分。

② 网格(i,j)的元素坐标是($\boldsymbol{X}[i,j]$, $\boldsymbol{Y}[i,j]$, $\boldsymbol{Z}[i,j]$)。

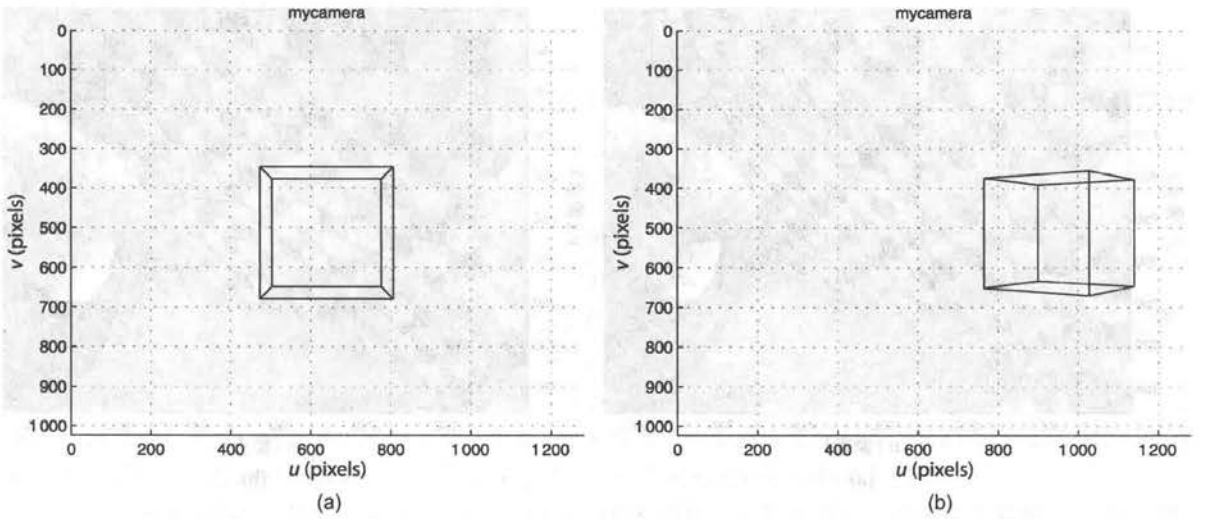


图 11.7　一个立方体的线段表示。(a)正视图；(b)斜视图

连续调用函数 plot 将会不断重绘图中的点和线段，从而展示出一种简单的动画效果。采用以下简短代码：

```
1  theta = [0:500]/100*2*pi;
2  [X,Y,Z] = mkcube(0.2, [], 'edges');
3  for th=theta
4      T_cube = transl(0, 0, 1.5)*trotx(th)*troty(th*1.2)*trotz(th*1.3)
5      cam.mesh( X, Y, Z, 'Tobj', T_cube );
6  end
```

它会展现出一个在空间中翻转的立方体。这个立方体的中心被定义在原点，它的各个顶点在每个时间步变化一次。

11.1.1　透镜畸变

任何透镜都不是完美无瑕的，而许多廉价网络摄像机的透镜质量更差。透镜的瑕疵会导致各种图像畸变，包括色像差(彩色边纹)、球面像差或像散现象(跨场景的焦距变化)和几何畸变(图像平面上的点背离了根据方程(11.3)计算出的理论位置)。

几何畸变通常是我们在机器人应用中遇到的最主要问题，包含两个部分：径向的和切向的。径向畸变造成图像点从主点开始沿着径向线发生位移。径向误差可以通过多项式近似表达：

$$\delta r = k_1 r^3 + k_2 r^5 + k_3 r^7 + \cdots \tag{11.12}$$

其中，r 是图像点与主点之间的距离。桶形畸变发生在放大效果随着远离主点而减弱的时候，它会造成靠近图像边缘的直线向外弯曲。枕形畸变发生在放大效果随着远离主点而增强的时候，它会造成靠近图像边缘的直线向内弯曲。切向畸变或偏心畸变发生在与半径垂直的方向上，但是通常没有径向畸变严重。发生畸变和未畸变的图样如图 11.8 所示。

畸变后，点(u, v)的坐标由下式给出：

$$u^d = u + \delta_u,\ v^d = v + \delta_v \tag{11.13}$$

其中位移是

$$\begin{pmatrix}\delta_u \\ \delta_v\end{pmatrix} = \underbrace{\begin{pmatrix}u(k_1 r^2 + k_2 r^4 + k_3 r^6 + \cdots) \\ v(k_1 r^2 + k_2 r^4 + k_3 r^6 + \cdots)\end{pmatrix}}_{\text{径向}} + \underbrace{\begin{pmatrix}2p_1 uv + p_2(r^2 + 2u^2) \\ p_1(r^2 + 2v^2) + 2p_1 uv\end{pmatrix}}_{\text{切向}} \tag{11.14}$$

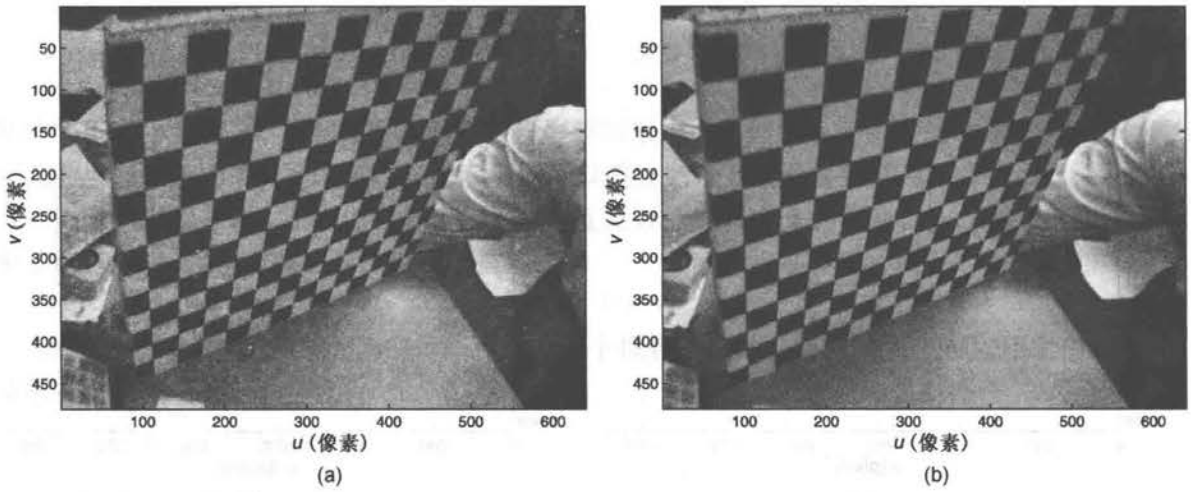


图 11.8 透镜畸变。(a)畸变的图像，最顶上一排正方形的弯曲很明显；(b)未畸变图像。这个图像是来自布盖相机标定工具箱[bouguet 2012]的#19标定图像

对于不同的(u,v)值，位移向量可以绘制出来，如图 11.12(b)所示。这些向量不仅指出了在图像不同点纠正畸变所需要的位移，即($-\delta_u$, $-\delta_v$)，并且还展示出了占主导的径向畸变。

通常三个系数就足以描述径向畸变。这样总畸变模型就可以由(k_1,k_2,k_3,p_1,p_2)参数化，它们被认为是附加的内在参数。畸变模型可以通过 CentralCamera 类使用'distortion'选项建立，例如

```
>> cam = CentralCamera('focal', 0.015, 'pixel', 10e-6, ...
    'resolution', [1280 1024], 'centre', [512 512], ...
    'distortion', [k1 k2 k3 p1 p2] )
```

11.2 相机标定

方程(11.9)所描述的相机投射模型事实上有很多未知参数。通常，主点不是感光单元阵列的中心。透镜的焦距长度只能精确到其标称值的 4%①，只有透镜聚焦到无穷远处时焦距长度才是正确的。一般的经验是，如果镜头被拆下再重新装上，或者调整过焦点或光圈，那么其内在固有参数就会发生改变②。唯一可能获得的不变的内在参数只有像素尺寸 ρ_w、ρ_h，这两个参数可以从传感器制造商的数据列表中获得。外在参数即相机位姿，它的问题是如何确定相机中心点的精确位置。

相机标定是确定相机关于世界坐标系的内在参数和外在参数的一个过程。标定技术主要依靠世界坐标系中的一组点，它们的相对坐标已知，且对应的像平面坐标也已知。最新的技术，如布盖的 MATLAB 标定工具箱[Bouguet 2010]，仅仅需要一些如图 11.11 所示的平面棋盘形靶标图像。正如 11.2.4 节中所讨论的那样，从中就可以估计出内在参数(包括畸变参数)，以及每幅图像中棋盘靶标的相对姿态。传统的标定技术需要相机拍摄一个三维标定靶，但是它不能估计出畸变模型。然而这种传统方法更容易理解，下面我们将从它们开始讨论。

① 根据 ANSI 标准 PH3.13—1958"透镜的焦距长度标记(Focal Length Marking of Lenses)"。

② 改变焦点是把透镜绕光轴进行旋转。如果透镜不是完全对称的，就会使畸变相对于图像平面发生移动。改变光圈就是改变透镜中让光线通过的部件，它也会改变透镜的畸变。

11.2.1 齐次变换法

齐次变换法允许对方程(11.9)中的相机矩阵 $\boldsymbol{C}$ 进行直接估计。这个矩阵中的元素是相机内在参数和外在参数的函数。设 $\tilde{\boldsymbol{p}} = (u, v, 1)$，展开方程(11.9)并代入方程(11.8)中，可以写出

$$\begin{aligned} C_{11}X + C_{12}Y + C_{13}Z + C_{14} - C_{31}uX - C_{32}uY - C_{33}uZ - C_{34}u = 0 \\ C_{21}X + C_{22}Y + C_{23}Z + C_{24} - C_{31}vX - C_{32}vY - C_{33}vZ - C_{34}v = 0 \end{aligned} \tag{11.15}$$

其中，(u, v) 是对应于空间点 (X, Y, Z) 的像素坐标，$C_{ij} = \boldsymbol{C}[i, j]$ 是未知相机矩阵中的元素。

背景材料：相机的中心在哪里？ 一个组合镜头有很多基本点，包括焦点、节点、主点和主面、前透和后透光孔。前透光孔是组合镜头系统光轴上的一个点，也称作透视中心或者无视差点。我们可以把它看作一个虚拟针孔。绕这个点旋转相机和透镜将不会改变透视图片中不同远近目标的相对几何关系。

绕前透光孔旋转对全景摄影很重要，可以避免最后拼接图像时的视差误差。很多网站都在专门讨论如何确定这个点位置的技术。一些网站甚至把常用镜头的前透光孔位置制成表格。尽管网上给出的技术确实可以确定前透光孔，但是很多这种网上文献错误地把这个点称为节点。

依赖于镜头设计，前透光孔可能在镜头系统的后面、里面或前面。

标定需要一个如图 11.9 所示的三维靶标。在靶标坐标系 $\{T\}$ 中，每一个标记点的中心位置 (X_i, Y_i, Z_i)，$i \in [1, N]$ 必须是已知的，但 $\{T\}$ 本身是未知的。拍摄一幅靶标的图像，然后确定标记点对应的像平面坐标 (u_i, v_i)。对每个标记点，可得到方程(11.15)中的两个等式。对于 N 个标记点，将所有等式叠加可以形成矩阵方程：

$$\begin{pmatrix} X_1 & Y_1 & Z_1 & 1 & 0 & 0 & 0 & 0 & -u_1X_1 & -u_1Y_1 & -u_1Z_1 \\ 0 & 0 & 0 & 0 & X_1 & Y_1 & Z_1 & 1 & -v_1X_1 & -v_1Y_1 & -v_1Z_1 \\ & & & & & \vdots & & & & & \\ X_N & Y_N & Z_N & 1 & 0 & 0 & 0 & 0 & -u_NX_N & -u_NY_N & -u_NZ_N \\ 0 & 0 & 0 & 0 & X_N & Y_N & Z_N & 1 & -v_NX_N & -v_NY_N & -v_NZ_N \end{pmatrix} \begin{pmatrix} C_{11} \\ C_{12} \\ \vdots \\ C_{33} \end{pmatrix} = \begin{pmatrix} u_1 \\ v_1 \\ \vdots \\ u_N \\ v_N \end{pmatrix} \tag{11.16}$$

由它可以求解出相机矩阵的元素 $C_{11} \cdots C_{33}$。方程的解还需要乘以一个比例因子，按惯例 C_{34} 被设定为 1。方程(11.16)有 11 个未知量，要想求解，就需要 $N \geqslant 6$。通常使用的标记点会多于 6 个，得到一组可以用最小二乘法求解的过约束方程。

如果所有点是共面的，那么方程(11.16)左侧的矩阵就不是满秩的，方程无确定解，这就是为什么标定靶标必须是三维的，并且标记点阵列要在两个或三个平面上，如图 11.9 所示。

我们用一个示例来阐释该方法。标定靶标是一个立方体，标记点是它的顶点，坐标系 $\{T\}$ 的原点在立方体中心，坐标轴平行于立方体表面。标记点相对于 $\{T\}$ 的坐标是

```
>> P = mkcube(0.2);
```

现在标定靶标是在相机坐标系中的某个“未知位姿” ${}^{C}\xi_T$，我们任意选择为

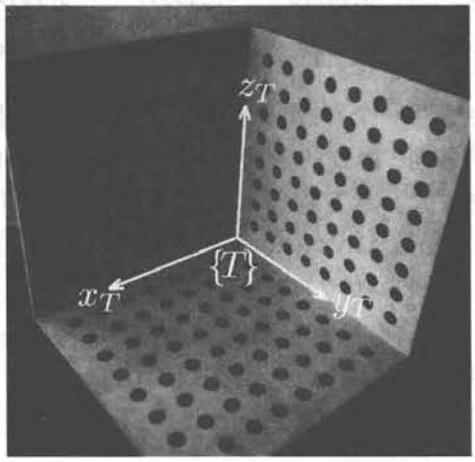


图 11.9 一个三维标定靶标，显示了坐标系 $\{T\}$。这些圆孔的中心看作是标定点。注意，这些标定圆孔位于三个平面上(图片来自法比安·斯宾德勒)

```
>> T_unknown = transl(0.1, 0.2, 1.5) * rpy2tr(0.1, 0.2, 0.3);
```

然后，我们创建一个透视相机，我们将对它的参数进行估计：

```
>> cam = CentralCamera('focal', 0.015, ...
    'pixel', 10e-6, 'resolution', [1280 1024], 'centre', [512 512], ...
    'noise', 0.05);
```

我们特意在(u, v)坐标上加了一个 $\sigma = 0.5$ 的零均值高斯噪声，用来模拟计算机视觉算法中相机的噪声和误差。处于“未知”位姿的标定靶标的像平面坐标是

```
>> p = cam.project(P, 'Tobj', T_unknown);
```

现在我们只使用对象模型 P 和观测到的图像特征 p 来估计相机矩阵：

```
>> C = camcald(P, p)
maxm residual 0.067393 pixels.
C =
  883.1620 -240.0720  531.4419  612.0432
  259.3786  994.1921  234.8182  712.0180
   -0.1043    0.0985    0.6494    1.0000
```

在本例中，最大残余小于0.1 个像素，这表明使用相机矩阵 $\boldsymbol{C}$ 得到的空间点投射与其实际像平面位置之间的最大误差很小。

类似于上述方法的线性标定技术不能估计透镜畸变参数。畸变将会把误差带入相机矩阵元素中，但是对于多数情况，引入的误差较小，可以接受。畸变参数通常在使用非线性方法优化所有参数(一般 16 个或更多)时进行估计，此时把线性方程的解用作初始估计参数。

11.2.2 分解相机标定矩阵

相机矩阵的元素是内在参数和外在参数的函数。然而给定一个相机矩阵，我们也可以通过逆解算得到大部分的参数值。接着上一小节中的例子，我们分解已估计出的相机矩阵：

```
>> est = invcamcal(C)
est =
name: invcamcal [central-perspective]
  focal length:   1504
  pixel size:     (1, 0.9985)
  principal pt:   (518.6, 505)
  Tcam:
       0.93695    -0.29037     0.19446     0.08339
       0.31233     0.94539   -0.093208    -0.39143
      -0.15677     0.14807     0.97647     -1.4671
             0           0           0           1
```

它会返回一个 CentralCamera 对象，用该对象的参数值能获得相同的相机矩阵。但我们马上注意到，得到的焦距长度要比我们镜头的真实焦距长度0.015 m 大很多，并且得到的像素尺寸也很大。从方程(11.9)可以看到，焦距长度和像素尺寸常常作为因子 f/ρ_w 和 f/ρ_h 一起出现[①]。函数 invcamcal 已设定 $\rho_w = 1$，但是被估计参数的比值为

```
>> est.f/est.rho(1)
ans =
   1.5044e+03
```

它非常接近于相机真实参数的比值：

```
>> cam.f/cam.rho(2)
ans =
   1.500e+03
```

被估参数值中的小误差是由于我们在标定中使用了带噪声的像平面坐标值。

① 这些参量是以像素为单位，因为 ρ 的单位是 m/像素。在文献中经常设 $\rho = 1$，这样焦距长度就用像素值给定。如果像素不是正方形，那么在水平和竖直方向上必须分别使用不同的焦距长度 f_u, f_v。

被估相机的位姿是相对于标定靶标的 $\{T\}$，因此是 ${}^{C}\hat{\xi}_{C}$。而靶标相对于相机的真实位置是 ${}^{C}\xi_{T}$。如果我们的估计是精确的，那么 ${}^{C}\xi_{T} \oplus {}^{T}\hat{\xi}_{C}$ 将会为 0。我们先前设定变量 T_unknown 等于 ${}^{C}\xi_{T}$。对于本示例，我们发现：

```
>> T_unknown*est.T
ans =
    0.7557   -0.5163    0.4031   -0.0000
    0.6037    0.7877   -0.1227   -0.0001
   -0.2541    0.3361    0.9069   -0.0041
         0         0         0    1.0000
```

它是位于相机真实和被估位姿之间的相对姿态。相机被估位姿在位置分量上的误差小于 5 mm。

我们可以绘制出用小红球表示的标定标记点，并绘制出位于世界坐标系原点的 $\{T\}$：

```
>> plot_sphere(P, 0.03, 'r')
>> plot_frame(eye(4,4), 'frame', 'T', 'color', 'b', 'length', 0.3)
```

然后再叠加上相机的被估计位姿：

```
>> est.plot_camera()
```

结果如图 11.10 所示。如何确定一个相机相对于标定靶标的位置是摄影测量学中一个很重要的问题，也被称为相机定位问题。

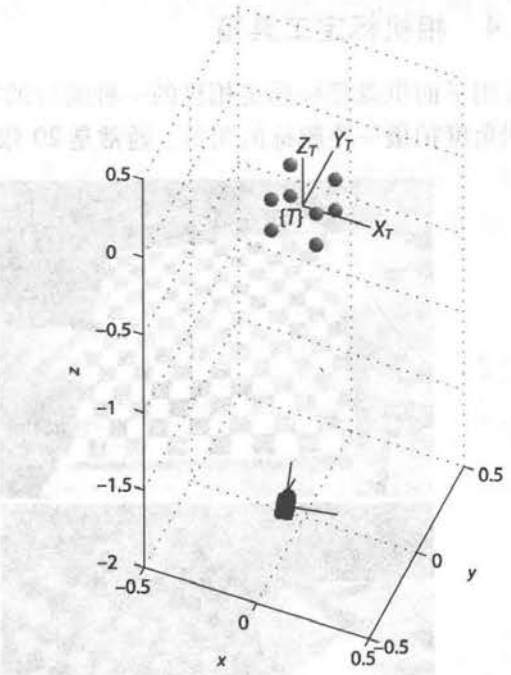


图 11.10　标定靶标的标记点和相对于靶标坐标系 $\{T\}$ 的相机估计位置。坐标系 $\{T\}$ 位于世界坐标系的原点。把相机描绘成一个四棱锥，各条边代表视场的边界

11.2.3　位姿估计

位姿估计问题是要确定靶标坐标系 $\{T\}$ 相对于相机坐标系的位姿 ${}^{C}\xi_{T}$。靶标的几何形体已知，就是说我们知道靶标上某些点在 $\{T\}$ 中的位置 (X_i, Y_i, Z_i)，$i \in [1, N]$。相机的内在参数也是已知的。拍摄一幅靶标的图像，再应用计算机视觉算法，可以确定靶标上那些点对应的图像平面坐标 (u_i, v_i)。

使用 (u_i, v_i)、(X_i, Y_i, Z_i) 和相机内在参数来估计靶标位姿，被称为 n 点透视问题，或简称 PnP 问题。这个问题比相机标定和分解简单，因为被估计的参数相对较少。为了举例说明位姿估计问题，我们要创建一个已标定的相机，且其参数已知：

```
>> cam = CentralCamera('focal', 0.015, 'pixel', 10e-6, ...
    'resolution', [1280 1024], 'centre', [640 512]);
```

我们想要确定位姿的对象是一个立方体，边长 0.2 m，它相对于 $\{T\}$ 的标记点坐标是

```
>> P = mkcube(0.2);
```

我们可以把它认为是对象的一个简单几何模型。该对象处于相机坐标系中某个任意但未知的位姿 ${}^{C}\xi_{T}$：

```
>> T_unknown = transl(0,0,2)*trotx(0.1)*troty(0.2)
T_unknown =
    0.9801         0    0.1987         0
    0.0198    0.9950   -0.0978         0
   -0.1977    0.0998    0.9752    2.0000
         0         0         0    1.0000
```

在这个未知位姿，对象上点的像平面坐标是

```
>> p = cam.project(P, 'Tobj', T_unknown);
```

现在我们只使用对象模型 P、观测到的图像特征 p 和已标定相机 cam 来估计对象的相对位姿 ${}^{C}\xi_{T}$：

```
>> T_est = cam.estpose(P, p)
T_est =
    0.9801    0.0000    0.1987   -0.0000
    0.0198    0.9950   -0.0978   -0.0000
   -0.1977    0.0998    0.9752    2.0000
         0         0         0    1.0000
```

它与对象的未知位姿 T_unknown 相同(精确到小数点后 4 位)。

实际上，图像特征坐标将不能被视觉系统准确地估计。为此，我们通过在图像特征坐标上加进零均值高斯噪声来进行模拟，就像我们在相机标定示例中那样。

11.2.4 相机标定工具箱

应用平面棋盘靶标标定相机的一种流行的实用工具是相机标定工具箱。首先从不同距离和不同角度拍摄一些靶标的图像，通常是 20 张，如图 11.11 所示。

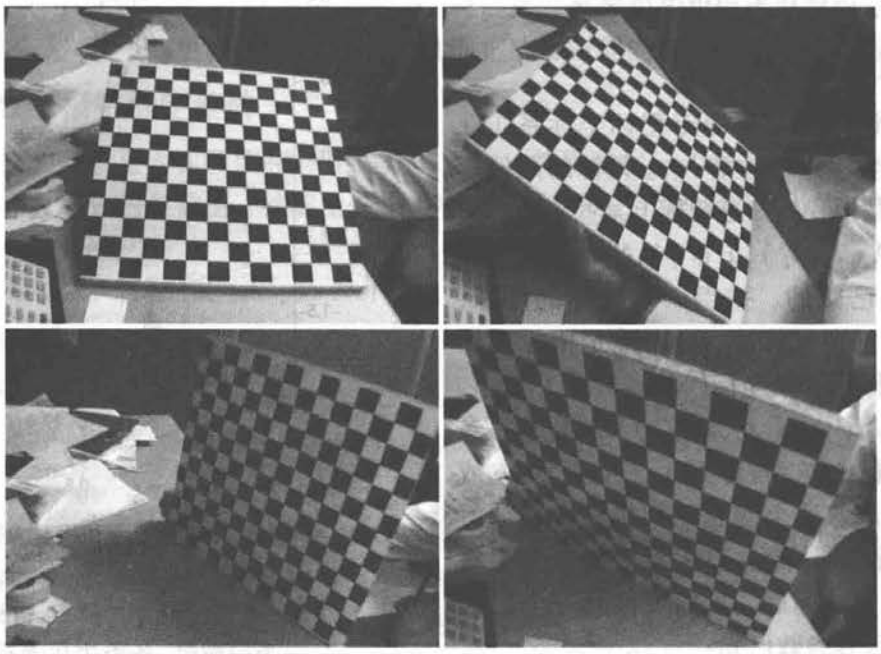


图 11.11 来自布盖标定工具箱的部分示例图样，展示了很多不同方位的标定靶标。这些是标定工具箱示例图样中的第2,5,9,18张

标定工具通过以下命令启动：

```
>> calib_gui
```

然后屏幕上显示一个图形用户界面(GUI)①。第一步是用(Image Names)按钮加载图片；第二步是

① 这个 GUI 是可选择使用的，你可以在自己的程序中直接调用工具箱函数。函数 calib_gui_normal 显示了 GUI 按钮名称与标定工具箱函数名称的对应关系。注意，大部分函数事实上都是脚本程序，且程序状态变量保存在工作空间中。

用(Extract Grid Corners)按钮，它会提示你在每一幅图像上选择标定靶标的角。该项工作比较烦琐，但必须仔细完成。最后一步是用(Calibration)按钮，它将用标定靶信息来估计相机参数值：

```
Focal Length:          fc = [ 661.66988   662.82841 ] ? [ 1.17902   1.26556 ]
Principal point:       cc = [ 306.09593   240.79019 ] ? [ 2.38421   2.17462 ]
Skew:             alpha_c = [ 0.00000 ] ? [ 0.00000  ]
   => angle of pixel axes = 90.00000 ? 0.00000 degrees
Distortion:            kc = [ -0.26424   0.22644   0.00020   0.00023   0.00000 ]
   ? [ 0.00933   0.03826   0.00052   0.00053   0.00000 ]
Pixel error:          err = [ 0.45328   0.38910 ]
```

对每个参数，其不确定性(3σ 范围内)都要被估计，并在问号之后显示出来。

对每一幅标定图像，都可以估计出一个相对于靶标的相机位姿，然后可以使用(Show Extrinsic)按钮进行显示。这个以靶标为中心的视图如图 11.12(a)所示，它展示出了对应于每一个输入图像的被估相机相对位姿。

畸变向量 kc 包含的参数顺序为(k_1, k_2, p_1, p_2, k_3,)——注意 k_3 的顺序错位。畸变图可以用以下命令显示：

```
>> visualize_distortions
```

如图 11.12(b)所示。它展示了从真实到畸变像平面坐标的位移，在本例中占主导的是径向畸变。这种现象与 k_1 和 k_2 的数量级比 p_1 和 p_2 大很多是一致的，大多数透镜也是如此。(Undistort Image)按钮可用于矫正一组畸变图像，图 11.8(b)中展示了有畸变和无畸变图像的对比。使用图像扭曲这种变换的细节将在 12.6.4 节中讨论。

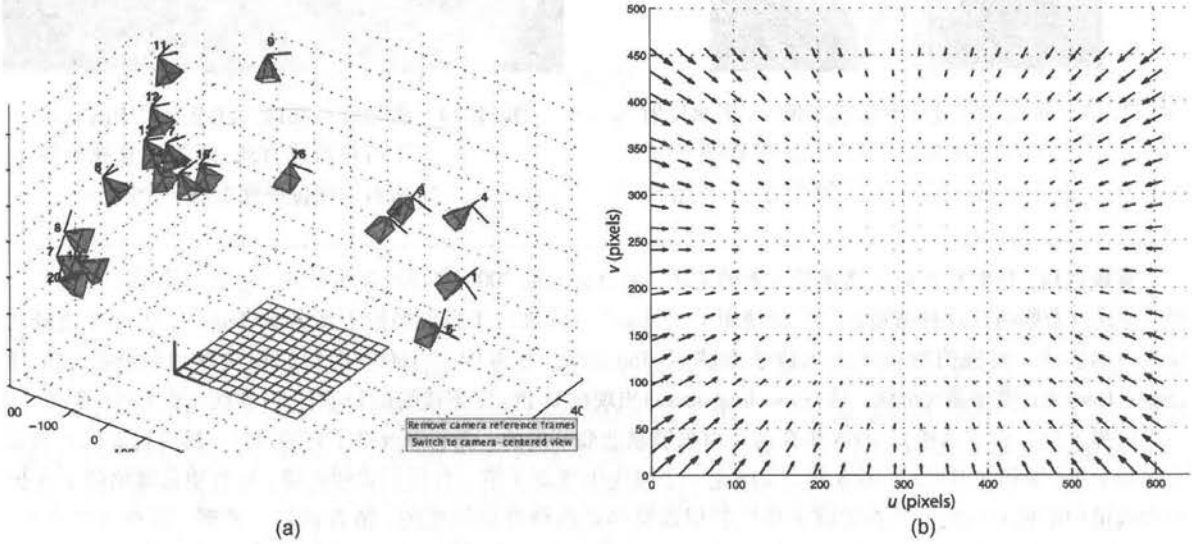


图 11.12　用布盖标定工具箱中图样进行标定的结果。(a)对于每一幅标定图像，相对于靶标的相机估计位姿；(b)畸变图，向量展示了图像中的点将会怎样由于畸变而运动

11.3　非透视成像模型

前面已经非常详细地讨论了透视成像，因为它是人眼和我们遇到的几乎所有相机的成像模型。然而透视成像从根本上把我们约束在一个有限的视场中。透视投影方程(11.3)对于 $Z=0$ 的点是奇异的，从而限制了视场最大是一个半球面——实际透镜的视场更小。而且随着焦距长度的减小，消除径向畸变的难度也会增加，并最终达到一个极限，超过该极限的透镜是不可

能生产出来的。想要更进一步的唯一方法是舍弃透视成像的约束。在 11.3.1 节中，我们将介绍用广角镜头系统成像的几何学。

替代折射光学的另一种方法是用反射面来形成图像，如图 11.13 所示。牛顿望远镜就是基于凹面的反射，而不是透镜的折射。反射镜不存在彩色边纹现象，而且镜面尺寸比透镜更容易做大。自界然中也进化出很多反射光学的实例——长吻银鲛和一些扇贝的眼睛就是基于鸟嘌呤晶体构成的反射镜。

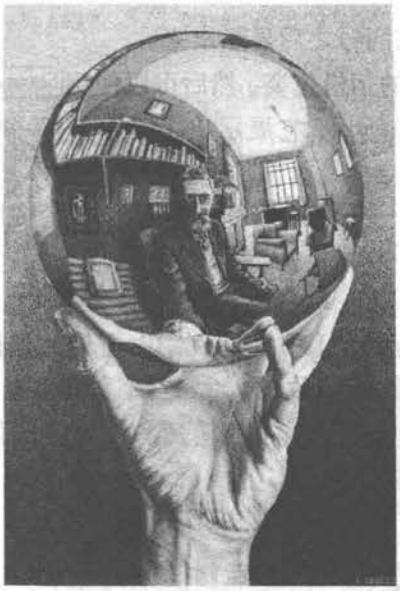


图 11.13 曲面上通过反射形成图像。(手拿反射球的埃舍尔，1935)。注意直线变成了曲线，屋顶的灯倾斜了一个角度

图 11.14 鱼眼镜头图像。注意空间中的直线不再投影成直线，还要注意视场被映射为图像平面上的一个圆形区域

背景材料：折射和反射。透镜最古老的实例，来自公元前 2000 左右的亚述帝国，它是一种用于点火的火镜，等到古希腊时代这种透镜已经广泛使用了。“lens”一词来自于扁豆的拉丁文名字“lentil”，因为凸透镜就像扁豆的形状。阿拉伯物理学家和数学家萨尔(Ibn Sahl，大约 940—1000)用我们现在熟知的斯奈尔定律(Snell'Law)来计算透镜的形状。读石(reading stones)出现在 11 世纪，眼镜则是 13 世纪 80 年代在意大利发明的。

世界上第一台望远镜是 1608 年在荷兰由眼镜制造商开发的，伽利略改进了它的设计，然后就发现了木星的四颗卫星(伽利略卫星)。1668 年，艾萨克·牛顿先生建造了第一台反射式望远镜，这台望远镜消除了在折射望远镜中的色差问题。早期的镜子和反射望远镜都是由镜青铜制成的，镜青铜是一种铜、锡和砷的合金。“speculum”是镜子的拉丁语，意为“金属镜”，并由它产生了一些术语，如“specular reflection”，意为“像镜面一样反射”。

折射光学和反射光学这些术语分别来自古希腊文字“dioptrics”和“catoptrics”。一台反射折射式相机中既包含反射镜也包含折射透镜。

11.3.1 鱼眼镜头相机

我们从图 11.15 所示的模型开始讨论鱼眼镜头相机，图中相机位于原点 O，其光轴是 z 轴。空间点 P 用球面坐标(R,θ,ϕ)表示，其中 θ 是与光轴所夹的角度，ϕ 是绕光轴旋转的角度。我们可以写出

$$R=\sqrt{X^2+Y^2+Z^2},\ \theta=\arccos\frac{Z}{R},\ \phi=\arctan\frac{Y}{X}$$

在相机像平面上，我们用关于主点的极坐标(r,ϕ)来表示投影点 p，其中 $r = r(\theta)$。笛卡儿像平面坐标是

$$u=r\cos\phi,\ v=r\sin\phi$$

像平面偏离主点的位移 $r(\theta)$ 是角度 θ 的函数，它决定了鱼眼镜头的类型。一些常见的投影模型列在表 11.1 中。

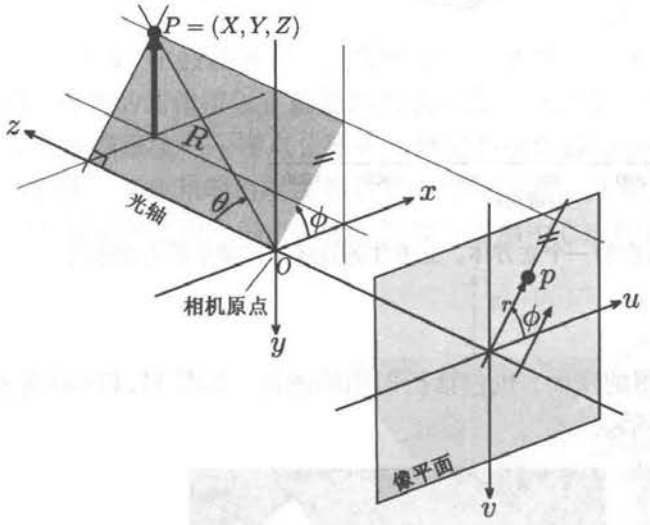


图 11.15　鱼眼镜头相机的成像。空间点 P 用相对于相机原点的球坐标(R,θ,ϕ)表示

表 11.1　鱼眼镜头投影模型

映射	方程
等角	$r=k\theta$
体视	$r=k\tan(\theta/2)$
等立体角	$r=k\sin(\theta/2)$
多项式	$r=k_1\theta+k_2\theta^2+\cdots$

使用工具箱可以创建一个鱼眼镜头相机：

```
>> cam = FishEyeCamera('name', 'fisheye', ...
        'projection', 'equiangular', ...
        'pixel', 10e-6, ...
        'resolution', [1280 1024])
```

它会返回 FishEyeCamera 对象的一个实例，FishEyeCamera 对象是工具箱 Camera 对象的一个子类，并与先前讨论过的 CentralCamera 类相似。如果像本例中那样不指定 k，那么通过计算就会让一个半球面视场被投影到像平面上最大的圆。和透视相机一样，鱼眼相机的主点和像素尺寸之类的参数通常是未知的，必须运用标定过程进行估计。

我们创建一个边长为 0.2 m 的有边立方体模型：

```
>> [X,Y,Z] = mkcube(0.2, 'centre', [0.2, 0, 0.3], 'edge');
```

把它投影到鱼眼相机像平面上：

```
>> cam.mesh(X, Y, Z)
```

结果如图 11.16 所示。

广角透镜可以有 180°甚至是 190°的视场，然而它们存在一些实用性上的不足。首先，由于相机像素分布于更宽广的视场，所以空间分辨率较低。我们从图 11.14 中也可以注意到视场是一个圆形区域，这就意味着几乎 25% 的矩形像平面浪费了。其次，户外图像很有可能包括明亮的天空，因此相机将会自动较少曝光，这会导致场景中一些非天空部分曝光不足。

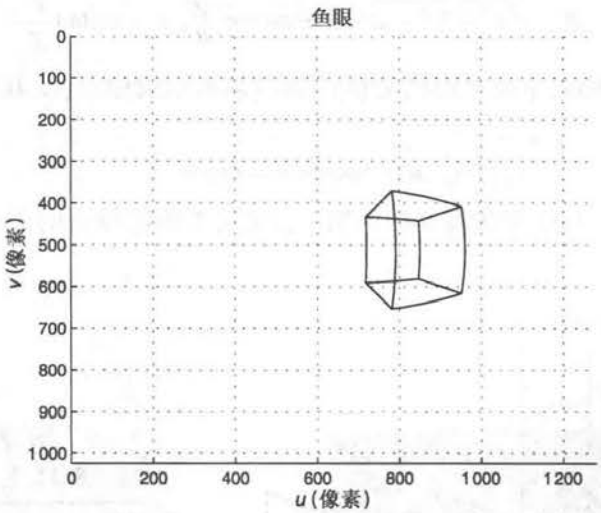


图 11.16 使用 FishEyeCamera 类投影的一个立方体。立方体的直线边在像平面上是曲线

11.3.2 反射折射相机

一个反射折射成像系统既包含反射用的镜面，也包括折射用的透镜，如图 11.17(a)所示。反射折射成像的示例在图 11.17(b)中展示。

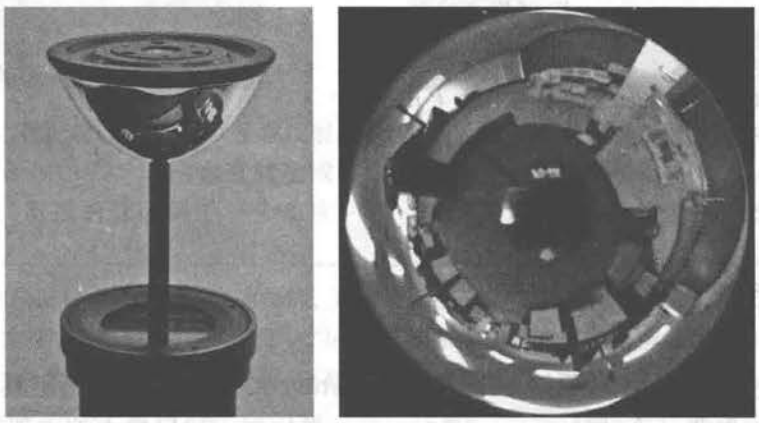


图 11.17 反射折射成像。(a)一个反射折射成像系统包含一个传统透视相机，它向上看反射镜；(b)反射折射图像。注意中心的暗点，它是透镜上方反射镜的支撑杆。地面在图像的中心区域，屋顶在图像的边缘

反射折射成像的几何关系相当复杂，如图 11.18 所示。构建一条从点 P 到反射镜焦距 O 的光线，O 也是相机系统的原点。这条光线与光轴所夹的仰角是

$$\theta = \arctan\frac{Z}{X^2 + Y^2} + \frac{\pi}{2}$$

并且与反射镜相交于点 M。反射回来的光线相对于光轴的角度是 ψ，它是入射光角度的函数，即 $\psi(\theta)$。θ 和 ψ 的关系由反射镜在点 M 处的切线决定，是一个反射镜形状的函数。反射折射成像采用了很多不同的反射镜形状，包括球形的、抛物线形的、椭圆形的和双曲线形的。通常函数 $\psi(\theta)$ 是非线性的，但是一种有趣的反射镜是等角度镜，它有

$$\theta = \alpha\psi$$

反射光线进入到相机中，与光轴的夹角为 ψ。再由透镜的几何关系可以写出

$$r = f\tan\psi$$

这是与主点的距离。像平面上点的极坐标是 $\boldsymbol{p} = (r, \phi)$，相应的笛卡儿坐标是

$$u = r\cos\phi,\ v = r\sin\phi$$

其中 ϕ 是方位角：

$$\phi = \arctan\frac{Y}{X}$$

图 11.18 中假设所有光线都经过了一个单焦点或视点，本示例中是 O 点。这被称为中心成像，得到的图像可以正确地转换成一个透视图像。等角度镜则不会有这种约束，所以是一个非中心成像系统——视点会随着入射光角度的改变而变化，且在反射镜中一条短的焦散轨迹上。圆锥、球面和等角度镜都是非中心成像的。但事实上，相较于整个空间范围，这些反射镜的视点变化是很微小的，因此它们都可以用中心成像模型来近似。

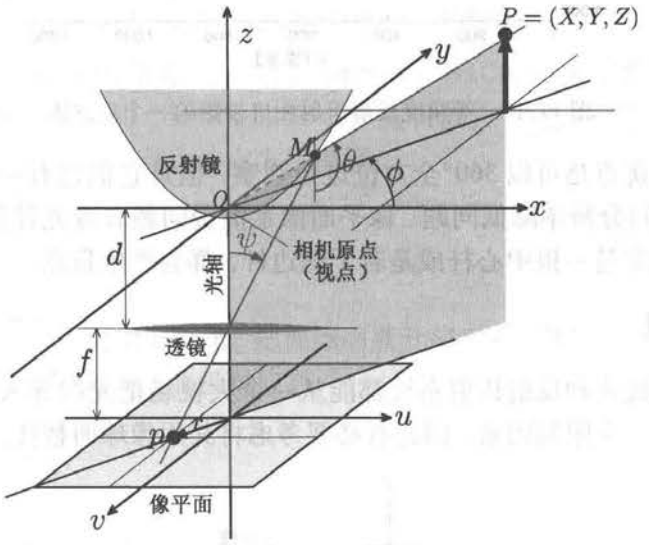


图 11.18　反射折射成像。光线从仰角为 θ、方位角为 ϕ 的点 P 朝 O 点发射，在反射镜表面的 M 点处发生反射，再通过透镜投影到像平面上的点 P 处

工具箱提供了反射折射相机的模型。例如，可以创建一个等角度的反射折射相机：

```
>> cam = CatadioptricCamera('name', 'panocam', ...
          'projection', 'equiangular', ...
          'maxangle', pi/4, ...
          'pixel', 10e-6, ...
          'resolution', [1280 1024])
```

它返回一个 CatadioptricCamera 对象的实例；catadioptricCamera 对象是工具箱 Camera 对象的一个子类，与先前讨论过的 CentralCamera 类相似。选项 maxangle 规定了最大仰角 θ，并由此确定参数 α 和 f，以使最大仰角对应于能最大限度覆盖像平面的一个圆。参数也可以使用选项'alpha'和'focal'单独确定。其他支持的投影模型包括抛物线的和球形的，每一种相机类型有不同的参数选项，详情参见网上文档描述。

这里创建一个有边的立方体模型：

```
>> [X,Y,Z] = mkcube(1, 'centre', [1, 1, 0.8], 'edge');
```

并把它投影到像平面上：

```
>> cam.mesh(X, Y, Z)
```

结果如图 11.19 所示。

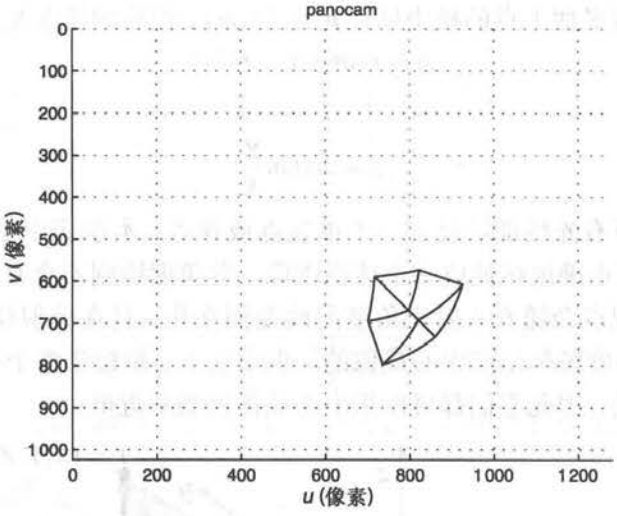


图 11.19　等角度反射折射相机投影的一个立方体

反射折射相机的优点是可以 360°全方位进行观察，但是它们也有一些实用上的缺点。由于装有鱼眼镜头，空间分辨率降低问题、像平面像素浪费问题和曝光控制问题也都不可避免。另外支撑反射镜的通常是一根中心杆或是若干侧边杆，都会产生盲点。

11.3.3　球形相机

刚刚讨论的鱼眼镜头和反射折射系统都能从一个大视场把光线导入像平面上。但最终这个二维像平面成为了一个限制因素，因此有必要考虑将其用像球面替代，如图 11.20 所示。

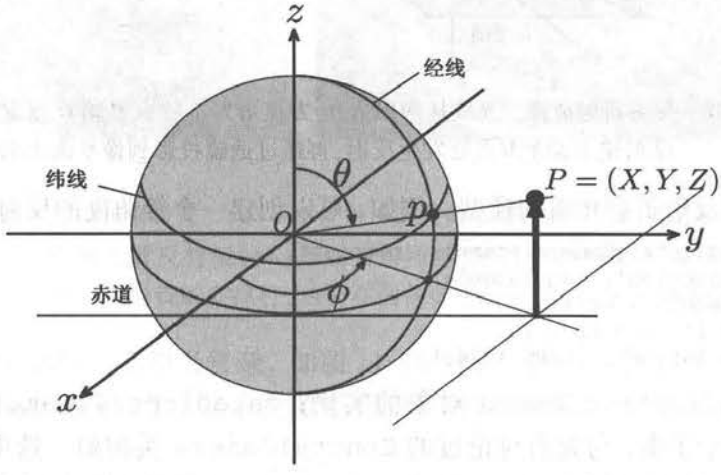


图 11.20　球面图像的形成。空间点 P 被映射到单位球体表面的 p 点，用纬度角 θ 和经度角 ϕ 来表示

一条光线从空间点 P 射向球体原点，投影点 p 是光线与球面的交点。如果我们写出 $p = (x,y,z)$，则有

$$x = \frac{X}{R},\ y = \frac{Y}{R},\ z = \frac{Z}{R} \tag{11.17}$$

其中，$R = \sqrt{X^2 + Y^2 + Z^2}$ 是空间点距离原点的径向长度。球面定义为 $x^2 + y^2 + z^2 = 1$，所以笛卡儿坐标中的一个是冗余的。球面上的一个点可以最少用两个参数（ϕ, θ）表示，它包含纬度角：

$$\theta = \arcsin r,\ \theta \in [0, \pi] \tag{11.18}$$

其中，$r = \sqrt{x^2 + y^2}$，以及方位角（或经度角）：

$$\phi = \arctan \frac{y}{x},\ \phi \in [-\pi, \pi) \tag{11.19}$$

反过来，与点（ϕ, θ）相对应的笛卡儿坐标如下：

$$x = r\cos\phi,\ y = r\sin\phi,\ z = \cos\theta \tag{11.20}$$

其中 $r = \sin\theta$。

我们用工具箱创建一个球形相机：

```
>> cam = SphericalCamera('name', 'spherical')
```

它返回 SphericalCamera 对象的一个实例，SphericalCamera 对象是工具箱 Camera 对象的一个子类，与先前讨论过的 CentralCamera 类相似。

像先前那样，我们创建一个有边的立方体模型：

```
>> [X,Y,Z] = mkcube(1, 'centre', [2, 3, 1], 'edge');
```

把它投影到球面上：

```
>> cam.mesh(X, Y, Z)
```

结果如图11.21所示。为了更加形象化，把像球面展开成矩形：纬度和经度线分别由水平轴和竖直轴表示，顶边和底边分别与南北极对应。

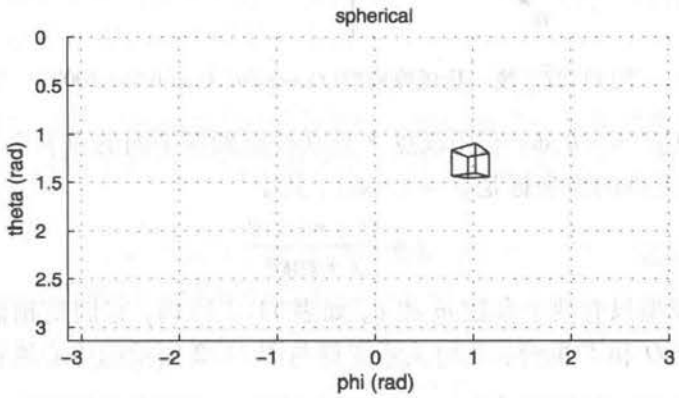


图11.21 一个球形相机投影出的立方体。像球面用笛卡儿坐标表示：水平和竖直轴分别表示纬度和经度角

尽管技术原理已经在几家实验室验证过，但是要想买一架球形相机还是没有可能。球形相机更多的是作为一种概念构想，以简化对广角成像问题的理解。如同下一节将要讨论的那样，我们可以把来自透视、鱼眼或反射折射照相机的图像都转换到一个球面上，这样就可以用统一的方法对其进行处理。

11.4 统一化成像

在本章中，我们已经介绍了很多种不同的成像模型。现在我们将要讨论怎样将一种相机已经拍摄到的图像转变成其他种类相机的图像形式。例如，给出一个鱼眼镜头的投影，我们将生成与之相应的球形相机或透射相机的投影。统一化的成像模型提供了一种强大的工具来研究不同类型的相机，如标准透视的、反射折射的以及多种鱼眼镜头的。

统一成像模型的过程分为两步，如图 11.22 所示。第一步是将空间点 P 球面投影到单位球体表面的 p' 处，这种投影在上一节讨论过，可以用方程(11.17) ~ 方程(11.18)来描述。视点 O 是球心，沿法向 z 轴从球心到像平面的距离是 m。单一视点意味着这是中心成像相机。

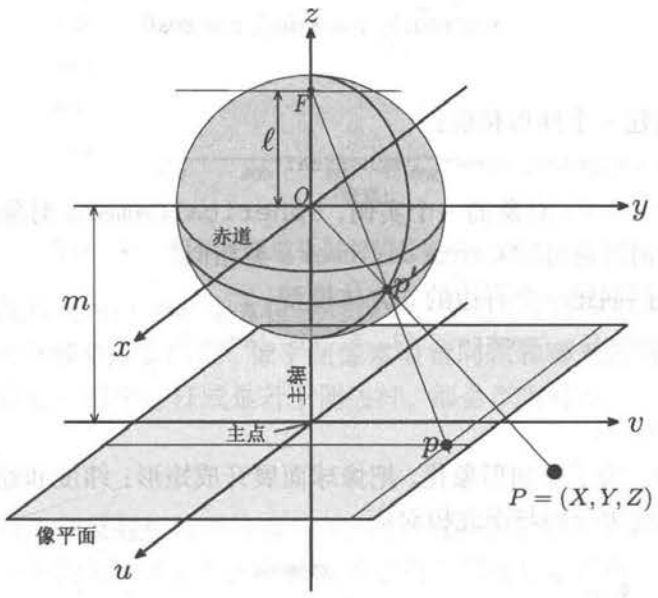


图 11.22 统一化成像模型[Geyer and Daniilidis, 2000]

在第二步中，点 $p' = (\theta,\phi)$ 利用视点 F 重新投射到像平面的点 p 上，F 沿 z 轴高出 O 点的距离为 ℓ。像平面上点的极坐标是 $p = (r,\phi)$，其中

$$r = \frac{(\ell + m)\sin\theta}{\ell + \cos\theta} \tag{11.21}$$

统一化的成像模型只有两个参数 m 和 ℓ，如表 11.2 所列，它们是相机类型的函数。对于透视相机，两个视点 O 和 F 重合，几何关系变得与图 11.2 所示的中心透视模型相同。

表 11.2 统一化成像模型参数 ℓ 和 m，取决于相机类型。ε 是圆锥曲线的离心率，$4p$ 是正焦弦

成像模型	ℓ	m
透视	0	f
体视	1	f
鱼眼	>1	f
反射折射(椭圆 $0<\varepsilon<1$)	$\frac{2\varepsilon}{1+\varepsilon^2}$	$\frac{2\varepsilon(2p-1)}{1+\varepsilon^2}$
反射折射(抛物线 $\varepsilon=1$)	1	$2p-1$
反射折射(双曲线 $\varepsilon>1$)	$\frac{2\varepsilon}{1+\varepsilon^2}$	$\frac{2\varepsilon(2p-1)}{1+\varepsilon^2}$

对于有圆锥反射镜的反射折射相机来说，焦点 F 在球心和北极点之间，即 $0 < \ell < 1$。这个投影模型稍微比图 11.8 所示反射折射相机的几何关系简单一点。成像参数由圆锥曲线参数离心率 ε 和正焦弦 $4p$ 表达[①]。

点 F 在北极点时的投影称为体视投影，并广泛应用于把球体表面投影到一个平面上。当 F 点位于北极点上方时，可以非常近似地模拟很多鱼眼透镜。

11.4.1　映射广角图像到球面上

我们也可以反过来使用统一化成像模型。考虑由广角相机捕捉到的图像，如图 11.23(a)所示的鱼眼图像。如果我们知道 F 点的位置，那么即使我们没有球形相机，也可以把图像上的每一个点投影到球体上来创建一个球面图像。

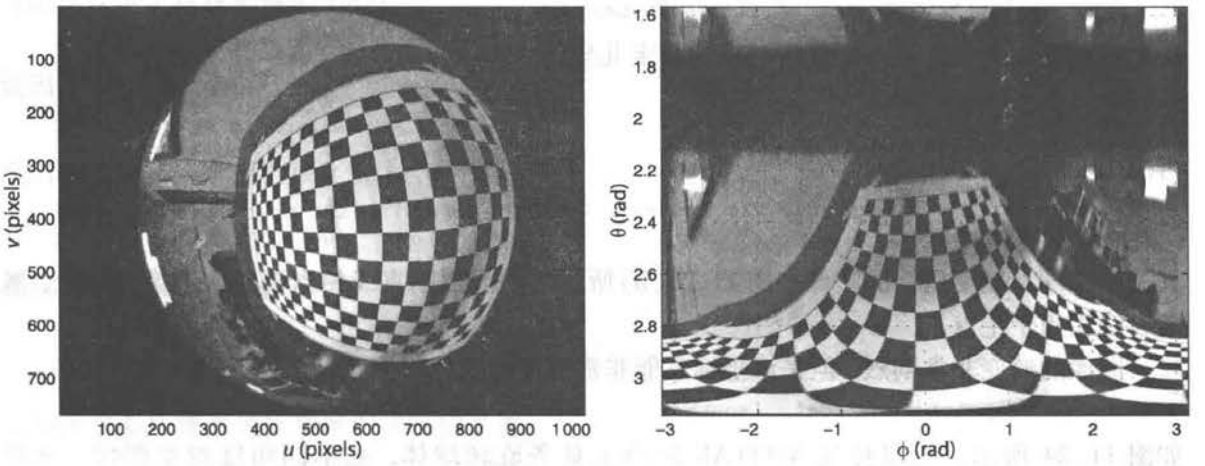


图 11.23　平面标定靶标的鱼眼图像。(a)鱼眼图像(Peter Hansen 提供图片)；(b)变形为(ϕ,θ)坐标的图像

为了完成这种逆映射，首先需要知道图像拍摄相机的一些参数。鱼眼镜头或反射折射相机拍摄图像的共同特征是图像的外边界是一个圆。

这个圆能被找到，而且中心可以精确估计，就是主点。我们采用了一种与 11.2.4 节中相机标定过程略有变化的方法，该方法使用了平面标定靶标在三维空间和像平面上的对应点，如图 11.23(a)所示。这个特殊相机的标定参数被估计为：主点(528.1214,384.0784)，ℓ = 2.7899，m = 996.4617。

考虑鱼眼图像中的一个点，其相对于主点的极坐标为 $p = (r,\phi)$。它在球面上相应点是 $p' = (\theta,\phi)$，其中 θ 可通过反解第二投影方程(11.21)获得：

$$\theta = \arccos\left(\frac{(\ell+m)\sqrt{r^2(1-\ell^2)+(\ell+m)^2}-\ell r^2}{r^2+(\ell+m)^2}\right) \tag{11.22}$$

我们将使用图 11.23 中的图像来展示上述方法：

```
>> fisheye = iread('fisheye_target.png', 'double', 'grey');
```

它通过使用已知的相机标定参数把该图像映射到球体上：

```
>> u0 = 528.1214; v0 = 384.0784;
>> l=2.7899;
>> m=996.4617;
```

① 弦长平行于准线并穿过焦点。

我们将使用图像变形来实现这个映射。在 12.6.4 节中将详细讨论变形问题，而我们在这里预先尝试一下这个方法。首先定义输入图像域：

```
>> [Ui,Vi] = imeshgrid(fisheye);
```

输出域覆盖了下半球：

```
>> n = 500;
>> theta_range = (0:n)/n*pi/2 + pi/2;
>> phi_range = (-n:2:n)/n*pi;
>> [Phi,Theta] = meshgrid(phi_range, theta_range);
```

其中 n 是球体表面上纬度和经度方向网格单元的数量。

为了实现变形，我们需要逆映射，即球体上每一个点的输入图像坐标。这是统一化成像模型方程(11.21)的第二步，我们得到

```
>> r = (1+m)*sin(Theta) ./ (1+cos(Theta));
```

由上式就可以计算出输入图像中对应的笛卡儿坐标：

```
>> U = r.*cos(Phi) + u0;
>> V = r.*sin(Phi) + v0;
```

再应用变形：

```
>> spherical = interp2(Ui, Vi, fisheye, U, V);
>> idisp(spherical)
```

其关于纬度和经度坐标的图像如图 11.23(b)所示。图像顶端直线相当于输入图像的周长，底端直线相当于主点。

图像的畸变非常剧烈，但是该坐标系能非常方便地变回球体。

```
>> sphere_paint(spherical, 'south')
```

如图 11.24 所示。可以使用 MATLAB 图形工具条旋转球体，从不同角度观察图像。选项 'south'表明图像只在下半球或者说南半球，球面图像未定义部分显示的灰色。

任何由中心成像参数表达的广角图像都可以用类似方法投影到球体上。

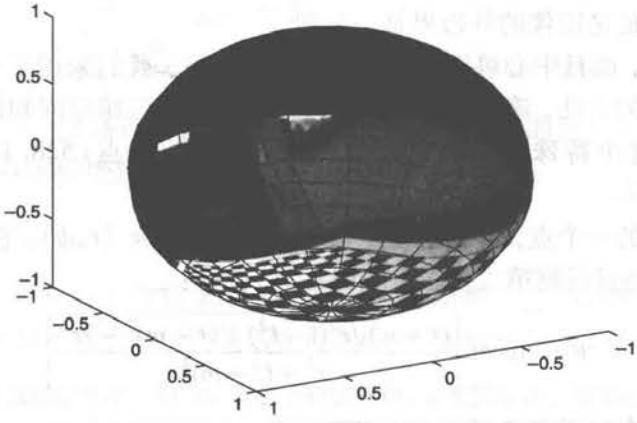


图 11.24 映射到单位球体的鱼眼图像。我们可以清晰地看到桌子上的平面网格，以及顶灯和屋内的其他线性特征

11.4.2 合成透视图像

我们将以把球面图像映射为透视图像的示例来结束本章。这是统一化成像模型的第二步，其中 F 是球体中心，在这种情况下，图 11.22 变得与图 11.2 相似。透视相机的光轴是球体的负 z 方向轴。

对于这个示例，我们将要使用在上一节中创建的球面图像。我们希望创建一个 1000 × 1000 像素、视场为 45°的透视图像。视场可以用图像宽度 W 和统一化成像模型参数 m 来表示，即

$$\theta_{\text{FOV}} = 2\arctan\frac{W}{2m}$$

对于 45°的视场，我们要求：

```
>> W = 1000;
>> m = W / 2 / tan(45/2*pi/180)
m =
   1.2071e+03
```

对于透视投影，我们要求：

```
>> l = 0;
```

我们也要求主点在图像的中心：

```
>> u0 = W/2; v0 = W/2;
```

输出图像的区域为

```
>> [Uo,Vo] = meshgrid(0:W-1, 0:W-1);
```

输出图像中每一个点的极坐标 (r,ϕ) 是

```
>> r = sqrt((Uo-u0).^2 + (Vo-v0).^2);
>> phi = atan2((Vo-v0), (Uo-u0));
```

相应的球面坐标 (ϕ,θ) 是

```
>> Phi_o = phi;
>> Theta_o = pi - acos( ((l+m)*sqrt(r.^2*(1-l^2) + (l+m)^2) -
l*r.^2) ./ (r.^2 + (l+m)^2) );
```

我们现在要从球面坐标变形为透视像平面：

```
>> perspective = interp2(Phi, Theta, spherical, Phi_o, Theta_o);
```

结果是

```
>> idisp(perspective)
```

如图 11.25 所示。这是从位于球中心的透视相机穿过南极向下看的场景。我们看到棋盘标定靶标上的线现在已是直线了，正如我们期望的透视图像那样。重要的是对于小视野，我们只能创建这种透视图像。

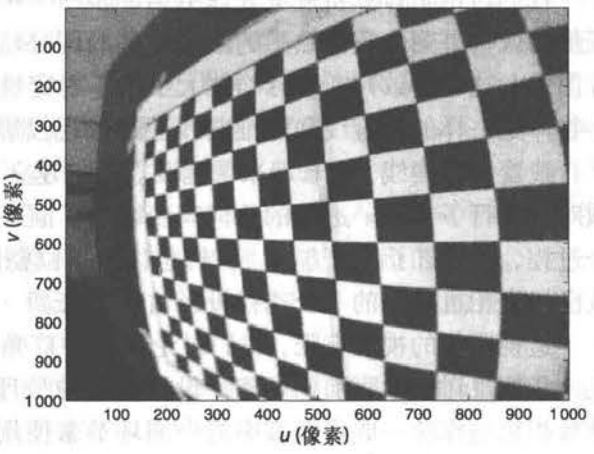


图 11.25　视场为 45°时，图 11.24 所示球面图形的透视投影。注意棋盘上的线现在是直线了

当然，我们并没有被限制仅仅沿着球体的负 z 轴方向进行观察。从图 11.24 中可以看到房间里的其他特征，例如门、白色写字板和一些顶灯。可以通过旋转球面图像，让我们虚拟的透视相机对准这些特征物的方向：

```
>> spherical = sphere_rotate(spherical, troty(1.2)*trotz(-1.5));
```

现在 z 轴负方向就指向远处的墙。重复上述变形过程，结果如图 11.26 所示，从图中可以看到顶灯和白色写字板①。

① 注意，图像的顶部是一条线的反射，这条线对应于最初视场的边缘。

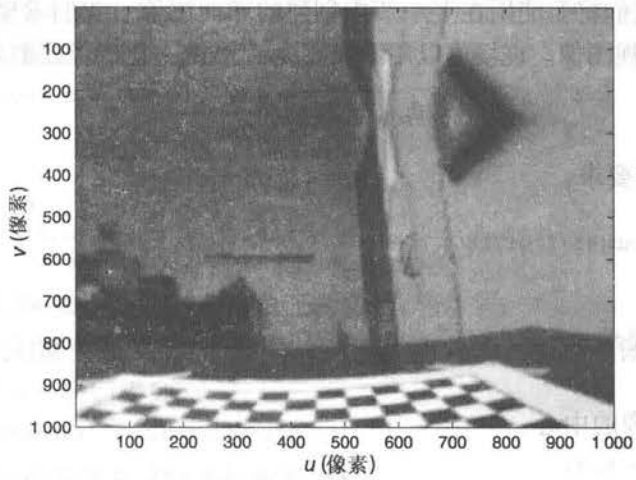


图 11.26 视场是45°时，图 11.24 中球面图像的透视投影。该视角是看向远处的墙和天花板

最初的广角图像包含了许多细节，尽管由于畸变很难看出来。把图像映射到球体之后，我们创建了一个可以沿半球体任一视线观察的虚拟透视相机视野(在这个视野中空间直线仍是直的)。但要想实现它，最初的图像只能采用有单一视点的中心成像相机拍摄。因此理论上我们无法从非中心的广角图像来创建透视图像，但实际上如果焦散很小，那么引入透视图像中的视差误差可以被忽略。

11.5 本章总结

我们讨论了计算机视觉处理中第一步——一幅环境图像的形成，并把它转换为构成数字图像的像素阵列。我们熟悉的图像都是周围环境的透视投影形成的，它把三维实景压缩为二维图像。这也导致判断实际物体大小的不确定性——远处的一个大物体看起来和近处的一个小物体是一样的。直线和圆锥曲线通过透视投影后仍然保持不变，但是发生了形状畸变——平行线变为汇聚线，圆看起来像椭圆。我们建立了透视投影的模型，并用 11 个内在和外在参数对其进行了描述。透镜的几何畸变增加了额外的透镜参数。相机标定是估计这些参数的一个过程，并介绍了两种方法。我们也讨论了位姿估计，其中几何形状已知对象的位姿可以使用从已标定相机获得的一个透视投影估计出来。

透视图像的视场有限，我们讨论了几种广角成像系统，包括鱼眼镜头相机和反射折射相机。我们也讨论了当前仍是理论设想的一种最理想广角相机，即球形相机。然而我们可以把球形相机当作统一成像模型中的中间环节来使用，它提供了一个适用于几乎所有相机几何关系的模型。我们使用统一化成像模型把鱼眼相机图像转变成球面图像，然后再转变为沿指定视线的透视图像。

在本章中，我们把成像当作有少量空间点或线段的纯几何学问题进行研究。在下一章中，我们将要讨论那些源于文件、相机和网络的图像的获取和处理。

扩展阅读

大部分计算机视觉教科书都提供了本章节的相关内容，如[Szeliski, 2011]、[Forsyt and Ponce, 2002]和[Gonzalez and Woods, 2008]。从图像中获得周围环境的几何关系是摄影测量学

的研究内容。这项技术的先驱是法国科学家 Aimé Laussedat，他在 19 世纪 50 年代就用气球进行了地图的航拍。《摄影测量学手册》(*The Manual of Photogrammetry*)[Slama, 1980]对该领域进行了全面而深入的论述，包括航空器和卫星图的历史、理论和应用。迪维特和沃尔夫修订的经典教科书[DeWitt and Wolf, 2000]对摄影测量学的介绍也比较深刻，且可读性强。摄影测量学通常关心的是使用从远距离获得的图像制作地图，但其子领域——近距或地面摄影测量学关心的是相机与目标距离小于 100m 的情况，这直接与机器人学有关。几何计算机视觉中许多课题也被摄影测量学研究，但是使用了不同的语言。例如，相机标定称为相机反切法，位姿估计称为空间反切法。

11.2.1 节中的齐次变换标定[Sutherland, 1974]方法在摄影测量学文献中也称为直接线性变换(DLT)。工具箱执行 `camcald` 命令时，要求标定标记点的图心已被确定，这不是一个小问题[Corke, 1996b, §4.2]。该命令也无法估计透镜畸变。沃尔夫[Wolf, 1974]对线性相机标定进行了扩展，他的模型中包括高达 18 个参数和非线性的优化估计算法。关于非线性标定更详细的论述由[Forsyth and Ponce, 2002]提供。在[Hartley and Zisserman, 2003]中描述了怎样使用诸如场景中直线这类特征来获得线性标定模型。

网上有很多好用的相机标定工具箱。11.2.4 节中讨论的 MATLAB® 工具箱是由布盖(Jean-Yves Bouguet)制作的，可通过 http://www.vision.caltech.edu/bouguetj/calib_doc 购买。该工具箱有广泛的网上文献，包含 11.2 节中使用的标定图像样例。杰尼·海基尔(Janne Heikkil)制作的 MATLAB® 工具箱在 http://www.ee.oulu.fi/~jth/calibr/提供，它采用具有圆点特征的平面或三维靶标，并能估计透镜畸变。

位姿估计在计算机视觉中是一个经典的难题，因此也出现了大量的文献。估计方法可以概括地分为解析式和迭代式。在假设透镜畸变已被矫正的前提下，针对三 3 至 4 个非线性点的解析解由[Fischler and Bolles, 1981]、[DeMenthon and Davis, 1992]和[Horaud et al., 1989]给出。这些方法通常存在多解，但如果 4 个点是共面的，就只有唯一解。6 个或更多点的情况则总能得到唯一解，还能得到内在的相机标定参数。迭代解方法在[Rosenfeld, 1959]和[Lowe, 1991]中进行了描述。基于光束法平差概念的一种新方法则在[Triggs et al., 2000]中进行了讨论。工具箱中的位姿估计是关于一种高效非迭代的透视 n 点(PnP)位姿估计器的一个包装类，[Lepetit et al., 2009]对其进行了描述，在 http://cvlab.epfl.ch/software/EPnP 提供。位姿估计要求有目标对象的几何模型，这种计算机视觉方法被称为基于模型的视觉。关于这种方法的一段有趣的历史回顾是由已故的乔·穆迪(Joe Mundy)在 1987 年提供的录像，可在 http://www.archive.org/details/JosephMu1987 中获得。

近来，人们对广角视觉系统的兴趣正逐渐上升，现在已有质量高且重量轻的鱼眼镜头和反射折射相机系统。纳亚尔[Nayar, 1997]对广角图像技术进行了极好的推介。一些非常有用的网上资源是在 http://www.math.drexel.edu/~ahicks/design 中的反射折射传感器设计页，以及在 http://www.cis.upenn.edu/~kostas/omni.html 中的有关研究小组、公司和专题会议的链接页。等角反射镜系统在[Chahl and Srinivasan, 1997]和[Ollis et al., 1999]中进行了描述。大自然的解决方案，即基于反射的扇贝眼睛，在[Colicchia et al., 2009]中进行了描述。关于全向视觉的许多专题会议从 2000 年开始举办，其论文集是对该领域知识的一种有用介绍。[Daniilidis and Klette, 2006]这本书中收集了很多关于非透视成像的论文，[benosman and Kang, 2001]是另一本更早发布的文件集。一些信息可通过 CVonline 在 http://homepages.inf.ed.ac.uk/rbf/CVonline 的 Sensors and their Properties(传感器及其特征)部分获得。

一款由戴维德·斯卡拉姆扎(Davide Scaramuzza)开发的用于广角相机标定的 MATLAB® 工具箱可以在 http://asl.epfl.ch/~scaramuz/research/Davide_Scaramuzza_files/Research/OcamCalib_Tutorial.htm 中获得。它是受启发于透视相机的布盖工具箱,且用法相似。另一种 MATLAB® 工具箱由胡霍·坎纳拉(Juho Kannala)开发,它针对广角中心成像相机,可在 http://www.ee.oulu.fi/~jkannala/calibration 中获得。

统一化成像模型是在[Geyer and Daniilidis, 2000]中有关反射折射相机的内容里介绍的。后来,[Ying and Hu, 2004]中指出许多鱼眼相机也可以由这个模型描述。11.5.1 节中的鱼眼标定在[Hansen et al., 2010]里进行了描述,估计的是 l,m,而不是斯卡拉姆扎工具箱中的多项式函数 $r(\theta)$。

相机类型

工具箱中的相机类 `CentralCamera`、`FishEyeCamera` 和 `SphericalCamera` 都来自于抽象的超级类 `Camera`。所有类的通用方法如表 11.3 所示。

表 11.3 所有相机类的通用方法

方法	描述
`p = cam.project(p)`	把空间点 `P` 投影到像平面的 `p`
`cam.plot(p)`	绘制出由 `P` 的列定义的空间点
`cam.mesh(X,Y,Z)`	绘制出由 `X`, `Y` 和 `Z` 定义的网格
`cam.showpose(T)`	显示相机在指定位姿
`cam.clf`	清除当前相机视图
`cam.hold`	保持当前相机视图。以后调用 `plot` 绘制的图叠加到此视图上
`cam.rpy(r,p,y)`	设定相机转换到指定的横滚-俯仰-偏航角
`cam.name`	特性:相机名称
`cam.Tcam`	特性:默认相机变换(读和写)

相机视图与 MATLAB® 图形有相似的性质。默认时,`plot` 和 `mesh` 会重新绘制相机的视图。如果没有相机视图存在,那么它就会创建一个。方法 `clf` 和 `hold` 与 MATLAB® 的命令 `clf` 和 `hold` 类似。

所有相机类的构造函数会接受若干的选项参数,如表 11.4 所示。具体的相机子类有唯一的选项,在网上文档中有所描述。没有输入参数时,默认的 `CentralCamera` 参数支持 1024×1024 图像、8 mm 的焦距长度和 10 μm 的方形像素。如果主点没有明确设定,就假定在像平面的中心。

表 11.4 相机类构造函数的通用选项

选项	描述
`'name', name`	显示在窗口标题条中的相机名称
`'resolution', npix`	图像的尺寸。方形图像时 `npix` 是一个标量,否则为 2 维向量
`'centre', pp`	主点坐标
`'pixel', rho`	像素的尺寸。方形像素时 `rho` 是一个标量,否则为 2 维向量
`'noise', sigma`	加到像平面坐标上的高斯噪声的标准差
`'pose', T`	相机位姿的默认值。默认位姿如图 11.2 所示
`'image', im`	根据图像尺寸设置相机的像平面尺寸,并显示图像

习题

1. 创建一个中央成像相机和一个立方体靶标，并且图形显示不同的相机和立方体位姿。
2. 编写一个程序，使得相机绕着立方体周围的一个轨道运动，相机始终对准立方体的中心。
3. 编写一个程序使得相机穿过立方体运动。
4. 创建一个有透镜畸变的中央成像相机，并让相机观察一个 10×10 的点的二维平面网格点。改变畸变参数，观察它对投影网格形状上的影响。创建针垫畸变和圆桶畸变。
5. 重复 11.2.1 节中的齐次相机标定练习，并探讨标定点数量、噪声及相机畸变对于标定残差和被估靶标位姿的影响。
6. 确定扩张角为 θ_u 和 θ_v 的四棱锥视场的立体角。
7. 完成布盖相机标定工具箱中的例 1。
8. 标定你计算机上的相机。
9. 对于相机标定矩阵分解的实例(11.2.2 节)，确定实际相机位姿和估计相机位姿之间的横滚-俯仰-偏航姿态误差。
10. 位姿估计(11.2.3 节)：
    a) 采用不同的对象位姿(更近和更远)重复位姿估计练习。
    b) 采用不同水平的相机噪声重复该练习。
    c) 当点的数量减少时会发生什么现象?
    d) 增加点的数量会抑制噪声增强的影响吗?
    e) 在调用 `estpose` 方法之前改变相机 `cam` 的内在参数。改变焦距长度和主点，如 5%，会有什么影响?
11. 针对鱼眼相机和球形相机重复练习 2 和 3。
12. 参考图 11.18，推导出抛物面反射镜的函数 $\psi(\theta)$。
13. 参考图 11.18，推导出等角反射镜在 xz 平面中的方程 $z(x)$。

第12章 图像处理

图像处理是把一幅或多幅输入图像转变成一幅输出图像的计算过程。图像处理经常用于提高图像质量以便于人们观察和应用理解，例如提高对比度。另一方面，也许对机器人技术更为重要，它是图像特征提取过程的基础(图像特征提取将在下一章详细讨论)。

一幅图像是图像元素(像素)的一个矩形阵列，因此我们将会用一个 MATLAB® 矩阵代表在工作区中的一幅图像，这样就可以应用 MATLAB 强大且高效的矩阵运算和函数库了。

在 12.1 节中，我们先讨论怎样把图像从文件(图片和电影)、相机和网络等输入源加载到 MATLAB® 中。然后讨论不同种类的图像处理算法。这些算法都是针对像素进行操作的，可以是单一图像，或是一对图像，也可能是图像中的局部，我们分别把它们称作一元操作、双值操作和空间操作。一元和双值操作将在 12.2 节和 12.3 节中介绍。空间操作在 12.4 节中描述，其中包括平滑化、边缘检测和模板匹配等操作。与上述操作紧密相关的技术是形态学滤波和数学形态学，这些内容将会在 12.5 节中论述。最后在 12.6 节中，我们会讨论一些改变形状的操作，例如剪裁、收缩或扩展，以及一些更复杂的操作，如旋转和广义的图像变形。

由于噪声、阴影、回声、照度不均等因素，机器人会经常获得一些有瑕疵的环境图像。在这一章中，我们还会讨论一些可以应用于真实图像处理的基础工具和技巧。

12.1 获取图像

如今数字图像无处不在。我们可以使用数码相机、手机或平板电脑方便快捷地获得，因而我们个人收藏的图像至少已有成千上万。除了个人收藏，网上也有大量的数码图像，如 Google 图像、Picasa 或 Flickr，后者的图像中还有元数据来描述图像内容以及地理位置。太空拍摄的地球图像，甚至月球和火星图像也都可以获得。我们也可以从其他人的相机中获得直播的图像数据流——世界上有成千上万的网络相机正在不停地拍摄图像并把它们传到网络上。

12.1.1 来自文件的图像

我们从存储在文件中的图像开始，因为你的电脑中很可能已经存储了有很多这样的图片。本章将会处理一些工具箱提供的图像，但是你可以很容易地换成自己的图像。我们使用工具箱函数 `iread` 把一张图像载入到 MATLAB® 工作区中：

```
>> street = iread('street.png');
```

它会返回一个矩阵：

```
>> about(street)
street [uint8] : 851x1280 (1089280 bytes)
```

矩阵有581行，1280列。通常我们用宽×高来描述一幅图像，因此这是一张1280×851的像素图像。

在第11章中，我们写过一个像素的坐标为(u,v)，分别是水平坐标和竖直坐标。在MATLAB®中，写为矩阵元素(v,u)——注意与坐标的顺序相反。还要注意，在MATLAB®中左上角的像素是(1,1)，而不是(0,0)。

这张图像从文件中读入，文件名为street.png，图像格式为便携式网络图形(PNG)——一种广泛应用于网络的无损耗压缩格式[①]。函数iread先在当前文件夹中搜索图像，然后再顺着MATLAB®路径的文件夹中进行搜索[②]。这幅特殊图像没有颜色，它是一幅灰度或单色图像。

上面还使用了about命令，结果表明矩阵street属于"unit 8"类型——矩阵元素是在[0,255]之间的无符号8位整数。这些元素被称为像素值或者灰度值，与原场景中对应点的明亮度成正比例。对于这个8位图像，像素值从0(最暗)到255(最亮)变化。例如，图像坐标是(300,200)的像素值为

```
>> street(200,300)
ans =
   42
```

它很暗——像素对应的场景点位于最近的门框中。

对于我们稍后将要考虑的一些图像处理操作，把像素值当作浮点数会有很大好处。在这种情况下，每一个像素都是[0,1]范围内的8位MATLAB®双精度数值。可以通过以下方法把一个unit8图像转换成浮点图像：

```
>> street_d = idouble(street);
>> about(street_d)
street_d [double] : 851x1280 (8714240 bytes)
```

或者，在加载图像时，用一个选项来指定图像类型：

```
>> street_d = iread('street.png', 'double');
```

背景材料：传感器的动态范围是它的最大值与最小值之比。对于图像来说，采用该比值的$\log_2$来表达则更好，这使得它等同于摄影中光圈级或曝光量的概念。每个感光单元都包含一个电荷井，在电荷井中产生光子的电子在曝光期间被捕捉。电荷井有一个有限的容量，否则感光单元将饱和，这就定义了它的最大值。电子的最小值不是0，而是热产生的有限数量电子。

一幅8位图像的动态范围大约有8级，高端的10位相机有10级的范围，而负片则可能有10~12级的范围，但不是线性的。

在特殊的适应状态，人的眼睛有10级的范围。但人眼的总适应范围可以达到惊人的20级，它可以通过使用虹膜和杆形光感细胞较慢(几十分钟)的化学适应过程来实现。对低照度的暗适应过程较慢，从明亮的光线下到黑暗的屋内需要很多分钟。从暗处到明处的适应则很快，但有时候很痛苦。

我们以后将会经常看到并频繁使用的一个图像显示工具是idisp：

```
>> idisp(street)
```

① 无损耗意味着压缩图像被解压缩后将会和原图像一模一样。

② 样例图像保存在机器视觉工具箱下面的"images"文件夹中，可以通过iread函数自动搜寻。

它显示了一个图像的矩阵，并允许对像素值交互式查看，如图 12.1 所示。点击某个像素将会在窗口右上方显示该像素坐标和它的灰度值。图像也可以被缩小或放大，我们还能在图像上任意选择两个点，并显示出沿这两点之间直线的直方图或光强分布图。它有很多的选项，而且在网上文档中有详细的描述。

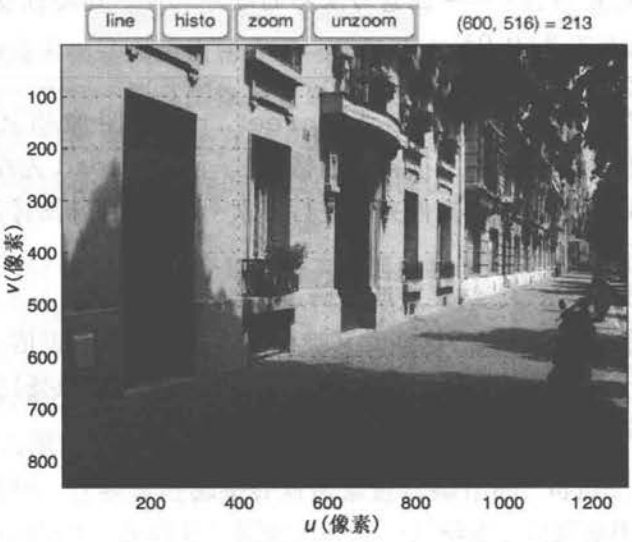


图 12.1 idisp 图像浏览窗口。右上方给出了鼠标点击处的坐标和像素值。左上方几个按钮的功能分别是：绘制出沿一条直线的像素值，显示一个直方图，以及图像缩放

我们可以很容易地加载一幅彩色图片：

```
>> flowers = iread('flowers8.png');
>> about(flowers)
flowers [uint8] : 426x640x3 (817920 bytes)
```

它是一个 unit8 值的三维矩阵，如图 12.2 所示。每一个像素的颜色由 RGB 三色刺激值表示，是一个三维向量。例如像素点(318,276)：

```
>> pix = flowers(276,318,:)
ans(:,:,1) =
   57
ans(:,:,2) =
   91
ans(:,:,3) =
  198
```

其三色刺激值为(57,91,198)。这个像素点对应于一朵蓝色小花，有很大比例的蓝色成分。我们可以使用 idisp 显示图像并交互式地查看它，点击其中的某一个像素点将会显示它的三色刺激值。

变量 pix 已被 MATLAB® 通过一种特殊而烦琐的方式显示出来。这是因为其像素值是一个 1×1×3 矩阵：

```
>> about(pix)
pix [uint8] : 1x1x3 (3 bytes)
```

前两个维称为单元素尺寸，我们可以把它们剔除：

```
>> squeeze(pix)'
ans =
   57   91  198
```

结果是一个我们熟知的三维向量。

图 12.2 所示的图像第三维称为彩色平面索引。例如：

```
>> idisp( flowers(:,:,1) )
```

它将会显示作为灰色等级图像的红色平面，它展示了每个像素点的红色刺激。如果选择索引 2 或 3，将会分别显示绿色平面或蓝色平面。

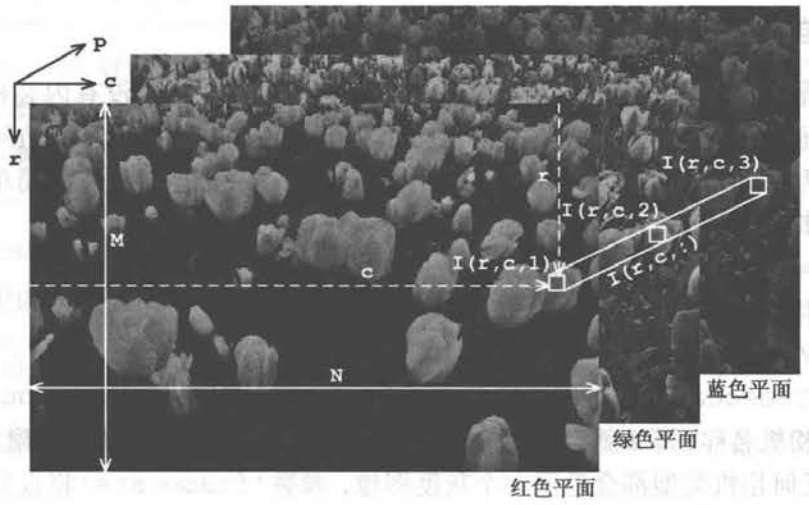


图 12.2　用一个三维结构显示的彩色图像，三维包括：行、列和彩色平面

三色刺激值是范围在[0,255]的 unit8 类型，但是也可以把图像转换成范围在[0,1]的双精度值，方法是在函数 iread 中使用'double'选项，或者直接对整型彩色图像使用函数 idouble，如同对灰度图像那样。使用选项'grey'将肯定会返回一个灰色图像，无论文件中是否包含彩色图像①。

函数 iread 也可以接受通配符文件名，这样它可以加载一系列文件。例如：

```
>> seq = iread('seq/*.png');
>> about(seq)
seq [uint8] : 512x512x9 (2359296 bytes)
```

它从 seq 文件夹中加载了 9 张 PNG 格式的图像。结果是一个三维矩阵，最后的索引数代表这个图形序列中的图像号。也就是说，seq(:,:,k)代表图像序列中的第 k 张图像，它是一张 512×512 的灰色图像。就图 12.2 而言，图像序列是在 p 方向上延伸。如果图像是彩色的，那么结果将会是一个四维矩阵，其中最后一个索引数代表序列中的图像号，而第三个索引数代表彩色平面。

如果不带参数调用函数 iread，则会跳出一个文件浏览窗口，允许通过文件系统导航查找图像。函数也接受 URL，允许它从网上加载图像，但是没有次序。函数可以读取大部分普通的图像文件格式，包括 JPEG、TIFF、GIF、PNG、PGM、PPM、PNM。函数 iread 的其他选项可以在网上文档中查阅。

背景材料：现在已有很多种图像文件格式，在 http://en.wikipedia.org/wiki/Image_file_formats 上对它们进行了全面的分类。最常见的是 JPEG 格式，用于数码相机和网路摄像机。TIFF 格式在许多电脑系统中常见，经常用于扫描器。PNG 和 GIF 格式广泛应用于网络上。这些文件的内部格式都很复杂，但是也有很多各种语

① 默认使用 ITU Rec.709 原色。也可以参见函数 imono。

言的高质量开放资源软件用来读写这些文件。MATLAB 就能够读取很多这样的图像文件格式。

更为简单的格式是 PBM、PGM 和 PPM(同属 PNM),它们大量应用于 UNIX 系统,属于未压缩的图像,有时也作为可读 ASCII 文件。许多开放资源工具,例如 ImageMagick 提供了 UNIX、MacOS X 和 Windows 系统之间的格式转换和图像操作功能。

12.1.2 来自附带相机的图像

现在许多笔记本电脑都内装了相机以方便召开视频会议。对于没有内装相机的电脑,也可以很容易地通过 USB 或者 Firewire 来外接一个相机。各种电脑操作系统访问相机的方法也不同,工具箱为 MacOS、Linux 和 Windows 操作系统提供了一个连接相机的简单接口。电脑所有附带相机及其能力的一个列表可以通过以下命令获得:

```
>> VideoCamera('?')
```

我们打开一个特殊相机:

```
>> cam = VideoCamera('name')
```

它会返回一个 VideoCamera 对象的实例。VideoCamera 对象是 ImageSource 类的子类。如果没有指定相机名称,那么就会使用最先找到的相机。构造函数接受若干附加参数,例如参数'grey'对任何相机类型都会返回一个灰度图像,参数'framerate'将设置每秒捕捉的图像帧数量。

相机返回图像的大小由 size 方法给定:

```
>> cam.size()
```

而使用 grab 方法则抓取一幅图像:

```
>> im = cam.grab();
```

这时它会等待直到获得下一帧图像①。

12.1.3 来自电影文件的图像

在 12.1.1 节中,我们加载了一个图像序列到计算机内存中,其中每一张图像都来自单独的图像文件。更多情况是:图像序列存储在电影文件格式中,如 MPEG4 或 AVI 格式,但在内存中不大可能保留文件所有的图像序列。

工具箱支持读取来自一个电影文件的图像帧,文件可以是任何常用格式,如 AVI、MPEG 和 MPEG4。例如,可以打开一个电影文件:

```
>> cam = Movie('traffic_sequence.mpg');
720 x 576 @ 2.999970e+01 fps
350 frames
```

背景材料:JPEG 格式采用有损失的压缩方式来减小文件大小,这就意味着解压后文件和原图像不是完全一样的。它利用了人眼的局限性,丢弃了那些不引人注意的信息,如微小色变(人眼对这些微小色变的感知比对亮度微变的感知弱)和细微纹理。请记住,JPEG 格式是用于压缩那些要给人眼看的图像。对于分析图像的计算机算法来说,颜色细节和微细纹理的丢失也许就成问题了。

JPEG 格式是为了更好地展现自然风景而设计的,但对于具有高空间频率含量的字符和画线,其显示效果就不太好了。信息丢失度可以通过调整所谓的质量因子进行改变,从而允许在图像质量和文件大小之间进行权衡。JPEG 可以用于灰度或彩色图像。

① 由于图像帧是按照选项'framerate'的要求以每秒 R 帧的速率产生的,所以等待的时间分布在区间 $[0,1/R)$。

那些通常被称为JPEG的文件(扩展名为.jpg或.jpeg的文件),更准确地说应是JPEG JFJF文件。JFJF是保存JPEG压缩图像和元数据文件的一种格式。EXIF文件格式(可交换图像文件格式)是一种有关相机元数据的标准,如相机设置、时间、位置等。这些元数据可以从函数iread的第二输出参数中取回,或者直接使用一个命令行指令取回,如exiftool(http://www.sno.phy.queensu.ca/~phil/exiftool)。更多细节请查看独立的JPEG群网站http://www.ijg.org。

它会返回一个Movie对象,它是ImageSource类的一个子类的实例,因此与刚才描述的VideoCamera类相似。这个电影共有350帧,且每秒钟捕获30帧。

电影中每一帧的大小是

```
>> cam.size()
ans =
   720   576
```

而从电影文件中读取下一帧的方法是

```
>> im = cam.grab();
>> about(im)
im [uint8] : 576x720x3 (1244160 bytes)
```

这是一张720×576大小的彩色图像。基于这些原始图像帧,可以写出一个非常简单的电影播放器:

```
1      while 1
2          im = cam.grab;
3          if isempty(im), break; end
4          image(im)
5      end
```

其中,第三行的条件测试是为了检测文件是否结束,如果是,grab返回一个空矩阵。

方法nframes和framerate分别提供了总的帧数量和每秒的帧数量。方法skiptotime和skiptoframe提供了跳转到电影中任意帧的功能。

12.1.4 来自网络的图像

网络相机这个名称现在可以代表任何通过USB或Firewire连接的相机,但是在这里我们用它来特指连接到互联网上的一部相机,它通过一个网络服务器可以传输所需的图像。现在世界上有成千上万的网络相机正对准各种场景,从平凡生活到惊人事件。通过来自安讯士公司(Axis Communications)网络相机的URL,就可以从世界任何地方的相机中获得图像,并且把它放到自己MATLAB工作区的矩阵中[①]。

例如,我们可以连接在新罕布什尔州达特茅斯学院中的一台相机:

```
>> cam = AxisWebCamera('http://wc2.dartmouth.edu');
```

它会返回一个AxisWebCamera对象,它是ImageSource类的一个子类的实例,因此与先前描述的VideoCamera类和Movie类相似。

本例中图像大小是

```
>> cam.size()
ans =
   480   640
```

① 网络相机支持可以嵌入到URL中的很多种选项,但目前还没有统一标准。这个函数现在只对安讯士公司的网络相机起作用。

纵横比是图像的宽度与高度之比。对于不同的成像和显示技术，纵横比差异很大。对于 35 mm 的胶卷，其纵横比是 3:2(1.5)，与 4×6″(1.5)的打印尺寸相匹配。其他打印尺寸则有不同的纵横比：5×7″(1.4)和 8×10″(1.25)，它们都要求裁剪图像的竖直边缘以适应尺寸大小。

电视机和早期的电脑显示器使用 4:3(1.33)的纵横比，例如最普遍的 640×480 格式。高清电视的纵横比设定为 16:9(1.78)。现代数码单反相机通常使用 1.81 的纵横比，接近高清电视。在电影院人们认为很宽的图像会更好，因而纵横比被设为 1.85，甚至是 2.39。宽银幕电影是在 20 世纪 20 年代由二十世纪福克斯公司(20th Century Fox)基于亨利克莱琴(Henri Chrétien)的研究工作开发而来。电影拍摄时，摄影机上的一个变形镜头把宽幅图像压缩成标准纵横比的图像，电影放映时，放映机中的变形镜头再把图像恢复成宽幅的。

下一幅图像可以通过以下方式获得：

```
>> im = cam.grab();
```

它会返回一张如图 12.3 所示的彩色图像。网络相机由它们的主人进行配置以定期拍摄照片，从每秒一张到每分钟一张都有。在相机拍摄完下一张图片前，重复访问只会获得相同的图像。

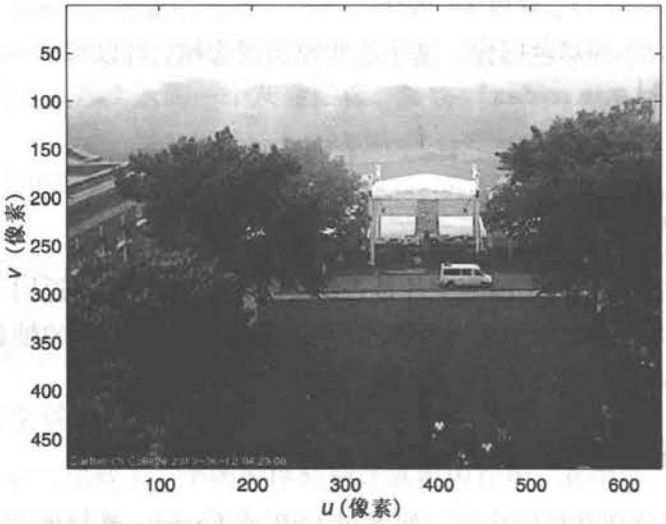


图 12.3 来自达特茅斯学院网络相机的一幅图像，相机对准学院的主草坪拍摄

12.1.5 来自代码的图像

当调试某种图像处理算法时，在使用更具挑战性的真实图像之前，先用那些完美而简单的图像会更有帮助。工具箱函数 testpattern 可以产生各种形式的简单图像，包括直线、网格点或方块、光强斜坡变化和光强正弦变化。例如：

```
>> im = testpattern('rampx', 256, 2);
>> im = testpattern('siny', 256, 2);
>> im = testpattern('squares', 256, 50, 25);
>> im = testpattern('dots', 256, 256, 100);
```

如图 12.4(a)所示。第二个参数代表所建图像的大小，在这里它们都是 256×256 像素，其他参数则要根据图像的类型特殊设定。更多详情见网上文档。

我们也可以用简单图元来构建图像。首先，建立一个全是黑色像素的帆布(像素值是 0)：

```
>> canvas = zeros(200, 200);
```

视频文件格式。正像图像文件那样，视频文件也有许多不同的文件格式。最常用的格式是 MPEG 和 AVI。区别开文件的格式(容器)与文件中图像的压缩类型(编码解码器)很重要，从技术上讲 AVI 是一种文件格式。

使用某些工具，可以将 MPEG 和 AVI 格式的文件转换成包含一系列图像帧的独立文件，如使用 FFmpeg 软件或 ImageMagick 套件中的 `convert` 命令。然后文件中的每一帧可以使用 `iread` 单独加载到 MATLA 中进行处理。工具箱中的 Movie 类提供了一种更简便的方式从普通视频格式文件中直接读取图像帧，而不必先把电影文件转换成一系列单独帧。

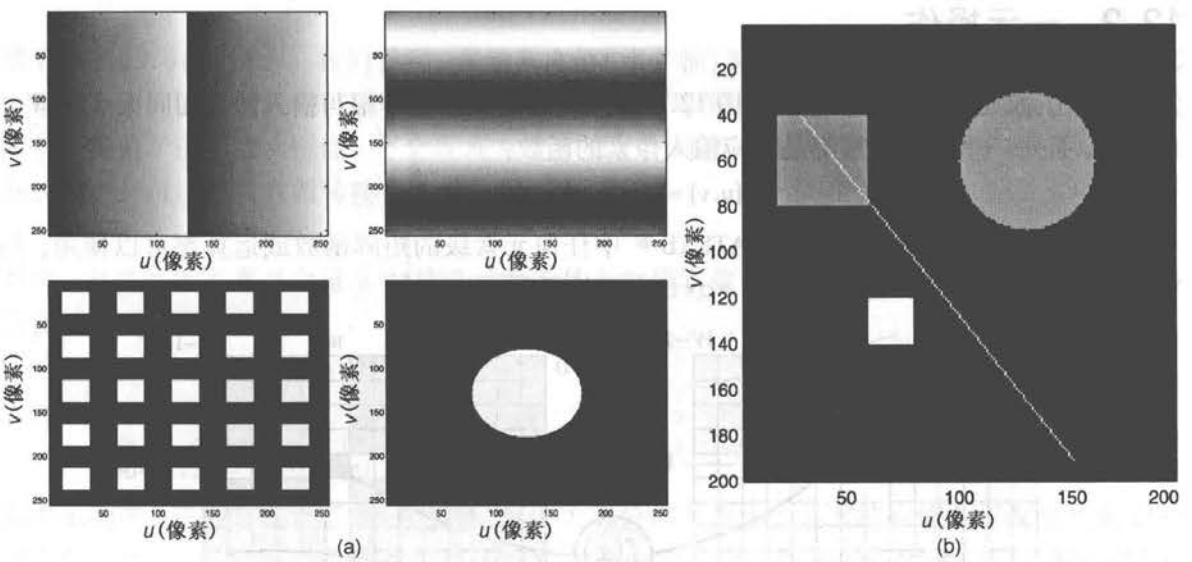


图 12.4 来自代码的图像。(a)工具箱生成的测试模板；(b)由图元创造的简单图像

然后创建两个正方形模板：

```
>> sq1 = 0.5 * ones(40, 40);
>> sq2 = 0.9 * ones(20, 20);
```

第一个正方形的像素值是 0.5(中等灰)，大小为 40×40。第二个正方形只有 20×20，但是像素值是 0.9，更明亮一些。现在我们把它们粘贴到帆布上：

```
>> canvas = ipaste(canvas, sq1, [20, 40]);
>> canvas = ipaste(canvas, sq2, [60, 120]);
```

其中，最后一个参数指定了帆布上的坐标 (u,v)，两个方形模板的左上角将会被粘贴到帆布的该位置。我们再创建一个圆：

```
>> circle = 0.6 * kcircle(30);
```

半径为 30 个像素，灰度值为 0.6。工具箱函数 `kcircle` 返回一个方阵：

```
>> size(circle)
ans =
    61    61
```

位于该矩阵中心最大圆盘上的元素值为 1，其余为 0。我们再把它粘贴到帆布上：

```
>> canvas = ipaste(canvas, circle, [100, 30]);
```

最后，在帆布上画一条线段：

```
>> canvas = iline( canvas, [30, 40], [150, 190], 0.8);
```

线段从点(30,40)连到点(150,190)，其上所有点的像素设定为0.8[①]。最后结果为

```
>> idisp(canvas)
```

如图12.4(b)所示。我们可以清晰地看到这些图形有着不同的亮度，我们还注意到，在线段和圆上显现出一些数字化效果，表现为带有阶梯形或锯齿形[②]。

注意：所有的这些函数均采用 (u,v) 格式来表示坐标，而不是MATLAB先行后列的格式。左上角的像素是(1,1)而不是(0,0)。

12.2 一元操作

一元图像处理操作的示意图如图12.5所示。输出结果是一幅与输入图像相同大小的 $W \times H$ 图像，且每一个输出像素都是对应输入像素的函数：

$$O[u,v] = f(I[u,v]),\ \forall (u,v) \in I$$

由于图像由矩阵表示，因此MATLAB® 中任何元素级的矩阵函数或运算都可以使用，例如标量的乘法或加法，或者 abs 和 sqrt 之类的函数。

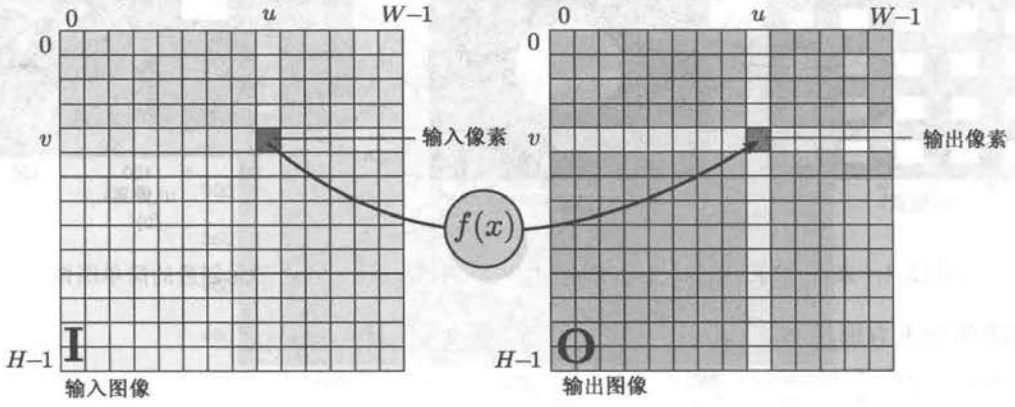


图12.5 一元图像处理操作。每一个输出像素都是对应输入像素的函数

每一个像素的数据类型都可以改变，例如从 unit8(范围在[0,255]的整数像素)到范围在[0,1]的双精度值：

```
>> imd = idouble(im);
```

反之亦然：

```
>> im = iint(imd);
```

彩色图像有三个维数，但我们也可以把它看作是二维图像，其中每一个像素值都是一个三维向量。一元操作可以将彩色图像转变成灰度图像，其中每一个输出像素值是一个标量，代表了相应输入像素的亮度：

```
>> grey = imono(flowers);
```

逆运算是

---

① 一个图像/矩阵可以使用命令 openvar('canvas')来进行编辑，它会产生一个类似电子表格的界面。

② 在计算机图形学中，常用抗锯齿技术。其中，边缘及边缘附近的像素设定为分数的灰度值，这样能给人一种光滑线条的印象。

```
>> color = icolor(grey);
```

它会返回一个三维彩色图像，其中每个颜色平面都等同于 grey——当显示时仍呈现出单色图像。我们可以创建一幅彩色图像，其中红色平面等同于输入图像：

```
>> color = icolor(grey, [1 0 0]);
```

它是原图像浸染红色后的版本。

许多一元操作都涉及修改图像中灰度值的分布规律。分布规律可以通过计算图像的直方图来确定①，它表示了每一个像素值出现的次数。例如，街道场景的直方图可以通过以下方法进行计算和显示：

```
>> ihist( street )
```

结果如图 12.6(a)所示。我们看到，虽然灰度值(水平轴)跨越了从 0 到 255 的整个范围，但其分布极其不均。通过观察可以看出有 3 个值得注意的峰值，但是如果更仔细地观察，也许会看出 9 个峰值。适当放大图像，还会发现有很多微小的峰值。峰值的判断取决于我们所考察数据的大小。可以把直方图转换为一对向量：

```
>> [n,v] = ihist(street);
```

其中，向量 n 的元素是向量 v 中对应元素所具有的像素值出现的次数。工具箱函数 peak 将会自动找到峰值的位置：

```
>> [~,x] = peak(n, v)
x =
     8    40    43    51   197    17    60    26    75   213
    88    92   218   176   153   108   111   147   113   119
   121   126   130   138   230   244
```

在本示例中，它一共找到了 26 个峰值，其中大部分都非常小。显著的峰值不仅仅是比它们紧邻点的值大，而是比附近范围其他点的值都大——现在的问题是如何定义所谓的附近范围。例如，比水平方向上 ± 25 像素范围内的像素值都大的峰值是

```
>> [~,x] = peak(n, v, 'scale', 25)
x =
     8    40   197
```

这就是我们可以用眼睛观察到的 3 个显著的峰值。发现峰值的关键是选择合适的范围。峰值寻找这个问题在后面还会遇到，另外在附录 K 中也进行了讨论。

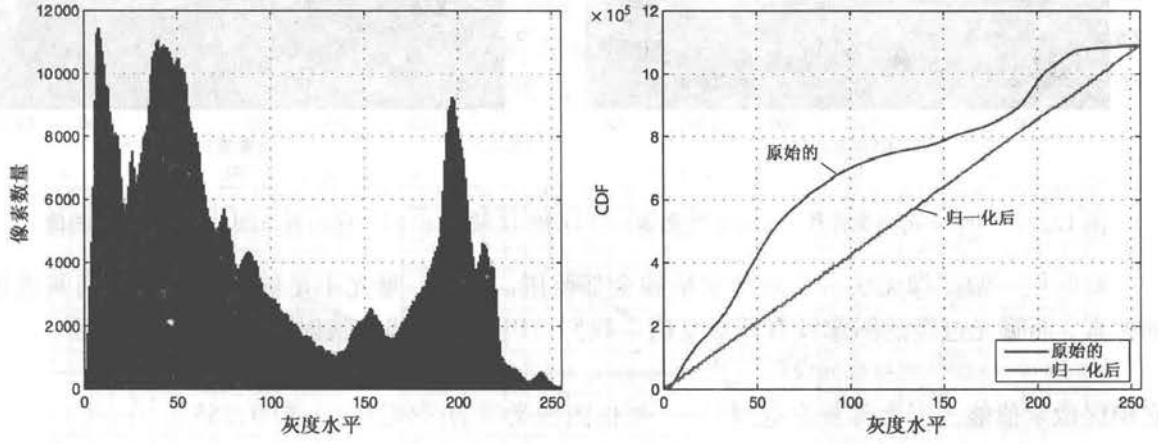


图 12.6　街道场景。(a)直方图；(b)归一化前后的累积直方图

① 这不是一元操作，因为结果不是一幅图像。在技术上，它是一种特征提取形式，我们将在下一章中讨论。

直方图上的峰值对应于图像中特殊的像素群体。例如，灰度值 40 附近的峰值对应于一大群黑色像素，它们主要是由于阴影形成的。我们可以通过一种一元逻辑运算识别这个峰值中的像素：

```
>> shadows = (street >= 30) & (street<= 80);
>> idisp(shadows)
```

所得图像如图 12.7(b)所示。所有位于区间[30,80]的像素显示为白色。这张图像是逻辑(logical)型的，但是当用于算术运算时，MATLAB 会自动把逻辑的真与假转换成相应的 1 和 0 值。这是一个相当简单的点对点阈值转换示例，其中仅仅凭借它们的亮度选出了某种像素。

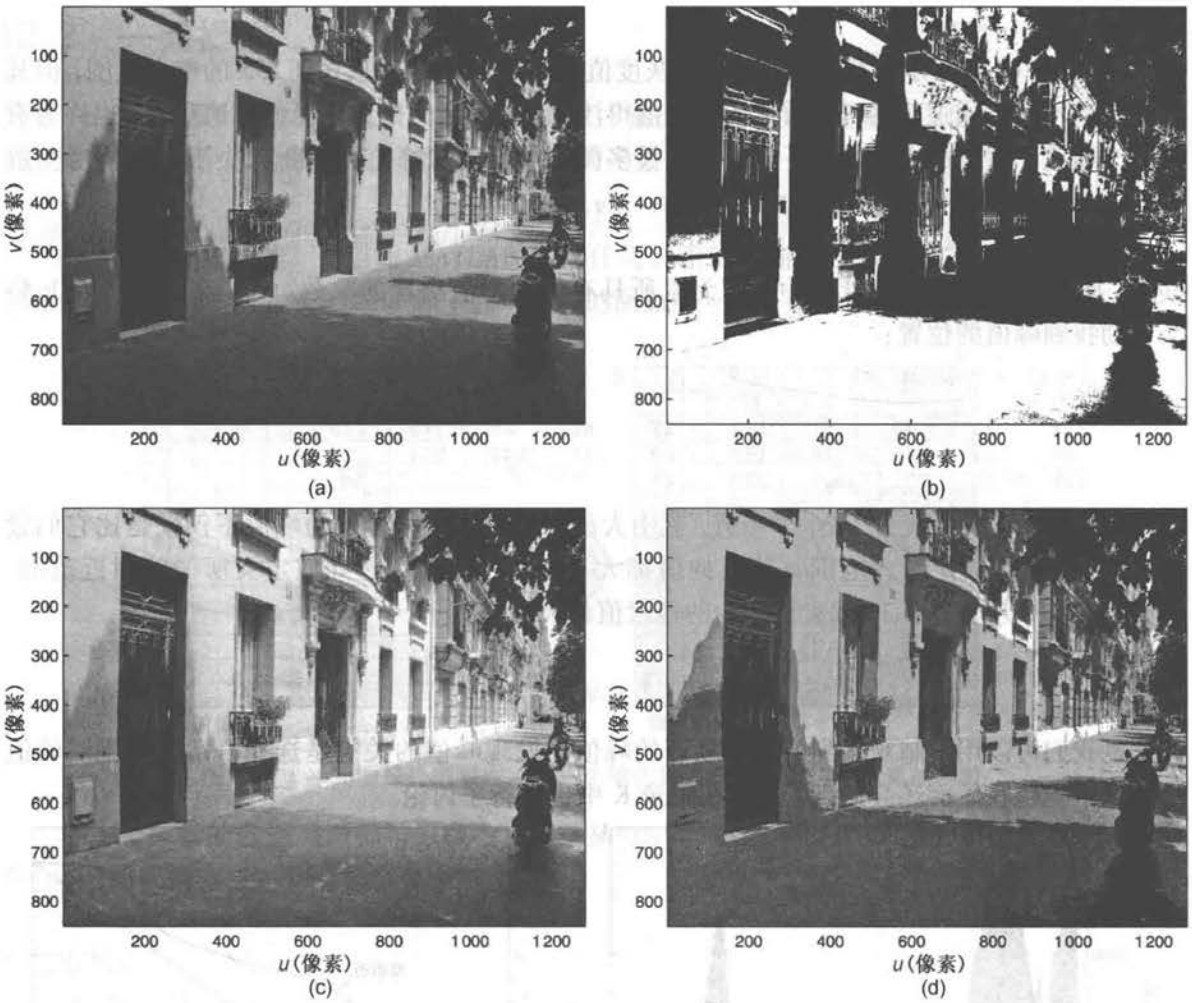


图 12.7 一些一元图像操作。(a)原始图像；(b)阴影区域；(c)归一化的直方图；(d)海报化图像

有时候一幅图像无法占满灰度水平的全部范围。例如，曝光不足的图像将没有高灰度值的像素，而曝光过度的图像没有低灰度值。我们可以用一种线性映射来调整灰度值范围：

```
>> im = istretch(street);
```

它确保像素值能占据全部灰度范围①——根据图像类型为区间[0,1]或[0,255]。

① 这样一幅图像的直方图将会有缺口。如果 M 是最大可能像素值，且 $N < M$ 是图像中的最大值，那么拉伸后的图像将会只有最大为 N 的像素值，这意味着 $M - N$ 之间的像素值不会出现。

一种更复杂的处理方法是直方图归一化或直方图均衡化：

```
>> im = inormhist(street);
```

如图 12.7(c)所示，它确保像素光强度的累积分布是线性的。我们可以看到阴影区域的灰度水平已经得到了提升和延展，使得阴影区域中的细节更加可视化。像素值的累积直方图可以使用以下命令绘制出来：

```
>> ihist(street, 'cdf');
```

归一化前后的累积分布如图 12.6(b)所示。

类似于 istretch 和 istretch 这样的运算操作可以提升观察者对图像的视觉感受，但是要记住这些操作没有加入任何新信息到图像中，并且对随后的图像处理过程也没有任何帮助。

如 10.3.4 节中讨论的那样，一个相机的输出通常是用伽玛编码的，因此像素值是相应感光单元所感受亮度的非线性函数 L^γ。这样的图像可以通过一种非线性一元操作进行伽玛解码：

```
>> im = igamma(street, 1/0.45);
```

它将提升每个像素到指定的功率。或者使用另一种参数：

```
>> im = igamma(street, 'sRGB');
```

它将用 sRGB 标准伽玛编码来解码图像。得到的线性图像有灰度等级或者三色刺激值，它们与原始场景的亮度成正比。

另外一种简单的非线性一元操作是海报化或条带化。这种流行艺术效果通过减少灰度水平的数量来获得：

```
>> idisp( street/64 )
```

如图 12.7(d)所示。由于使用了整数除法，所得图像的像素值范围是[0,3]，因此仅有 4 种不同的灰色阴影。

12.3　二元操作

二元操作的示意图如图 12.8 所示。两个输入矩阵得到一个简单的输出矩阵，并且三个图像的大小相同。每一个输出像素都是两个输入图像中对应像素点的函数：

$$O[u,v] = f(I_1[u,v], I_2[u,v]),\ \forall (u,v) \in I_1$$

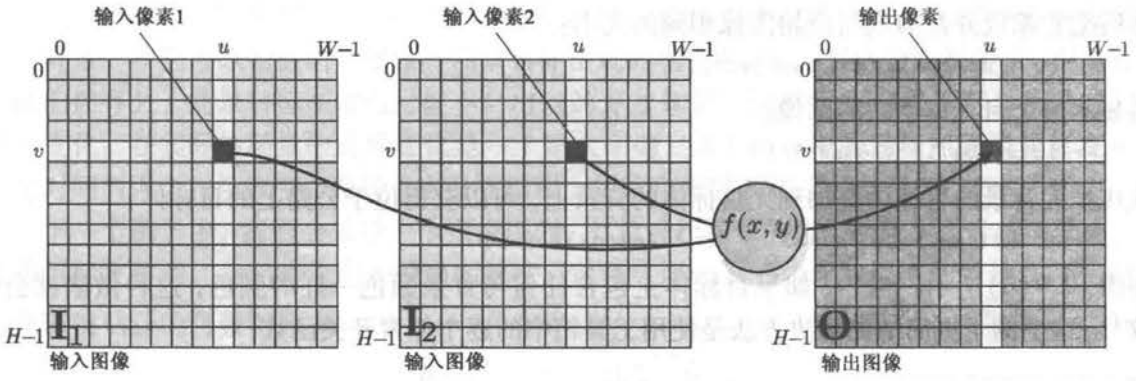


图 12.8　二元图像处理操作。每一个输出像素都是两个对应输入像素的函数

常用的二元操作包括二进制算术运算，如加法、减法、元素操作，或者是 MATLAB 内置的二元矩阵函数，如 max、min、bitand、atan2 等。

用一个 unit8 图像减去另一个 unit8 图像，就会产生另外一个 unit8 图像，尽管结果可能是负的。MATLAB 却可以正确地把值取在[0,255]区间中，所以小数减大数将会得出 0，而不是负值。另外，大于 255 的结果将会被设为 255。为了实现这种纠正，图像应该首先用 MATLAB® 函数 cast 转换成有符号整型，或者使用工具箱函数 idouble 转换成浮点型。

我们将会举两个例子来说明二元操作。第一个例子是电视上用的一门技术，该技术允许人像被叠加到一些背景上，例如天气预报播音员可以被叠加到气象图上。目标主体以蓝色或绿色为背景进行拍摄，这样很容易通过像素值来区分开背景和目标体。我们加载一张在绿色背景下拍摄的目标体图像：

```
>> subject = iread('greenscreen.jpg', 'double');
```

如图 12.9(a)所示。然后在第一次把伽玛解码的彩色图像转换成线性三色刺激值之后，计算色度坐标方程(10.8)：

```
>> linear = igamma(subject, 'sRGB');
>> [r,g] = tristim2cc(linear);
```

在这种情况下，单独使用 g 则足以区分背景像素。其色度值的直方图为

```
>> ihist(g)
```

如图 12.9(b)所示，它指出图像背景是在 0.55 左右的一群像素，那么另一群像素就属于目标体。我们可以保险地说目标体上的任何一个像素，它的 $g < 0.45$。这样，我们定义一个掩膜图像：

```
>> mask = g < 0.45;
>> idisp(mask)
```

如图 12.9(c)所示，如果某个像素是目标体的一部分，那么它的值就为真(等于 1)。我们需要把这个掩膜覆盖到所有三个彩色平面上，所以要复制掩膜：

```
>> mask3 = icolor( idouble(mask) );
```

没有背景的目标体图像是

```
>> idisp(mask3 .* subject);
```

接着加载需要的背景图像：

```
>> bg = iread('road.png', 'double');
```

然后把它缩放并裁剪到与原始图像相同的大小：

```
>> bg = isamesize(bg, subject);
```

再显示这个目标体空缺的图像：

```
>> idisp(bg .* (1-mask3))
```

现在把无背景的目标体叠加到无目标体的背景上，可以获得位于公路上的目标体：

```
>> idisp( subject.*mask3  + bg.*(1-mask3) );
```

如图 12.9(d)所示。当然，如果目标体上包含任何与背景颜色一样的颜色，这种做法就会失败[①]。这个例子更简洁的解决方法是使用工具箱中的逐个像素开关函数：

---

① 在早期的电视节目制作中，多使用蓝色背景屏幕。现在，由于蓝眼睛和蓝色牛仔服饰的出现，绿色背景更加盛行。

```
>> ipixswitch(mask, subject, bg);
```

其中，所有三个参数都是宽和高相同的图像，并且每个输出像素是根据第一幅图像中对应的像素逻辑值从第二或第三幅图像的对应像素中挑选出来的。

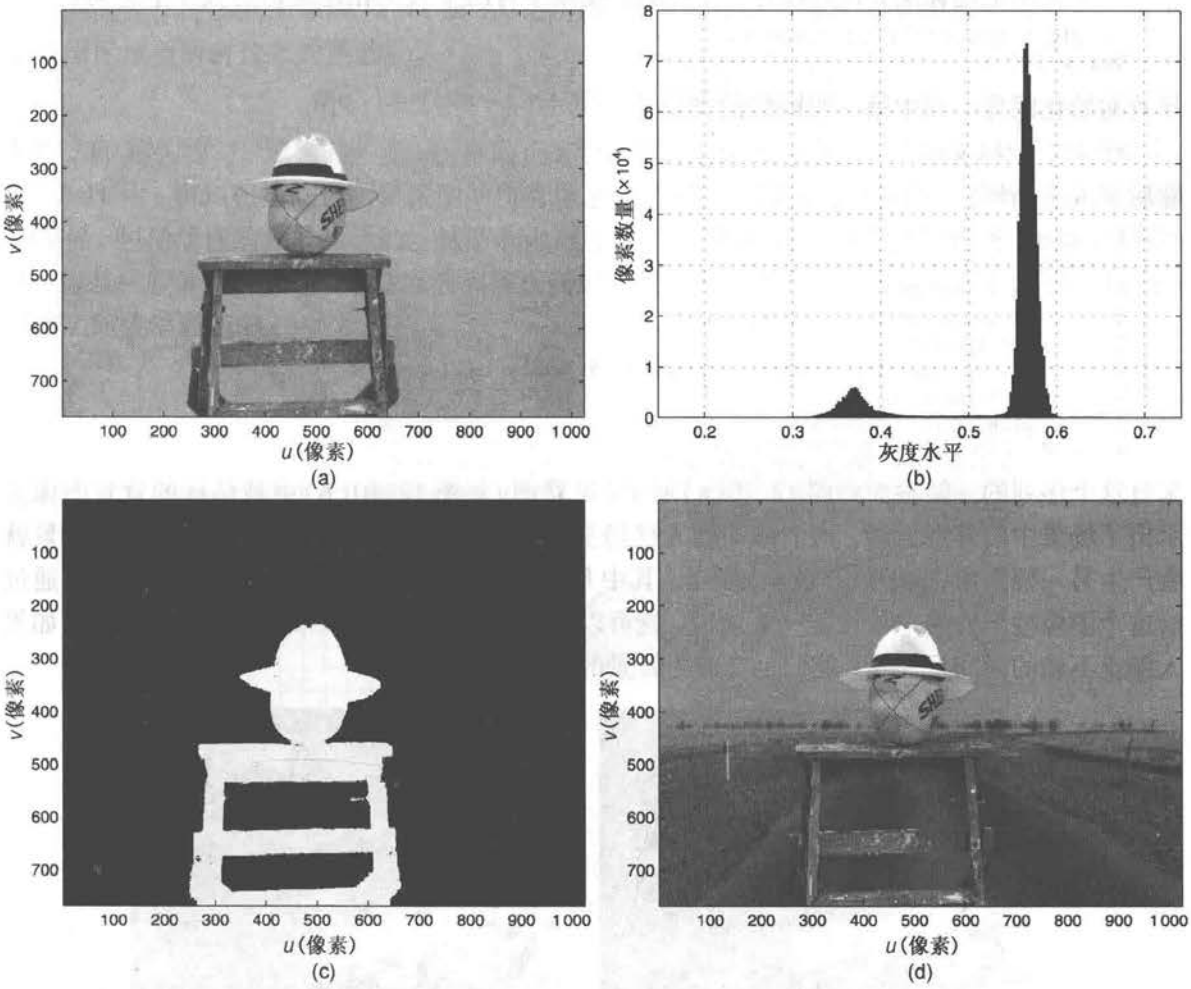


图 12.9　色度调整。(a)以绿色为背景的目标体；(b)绿色的色度值直方图；(c)计算出的掩膜图像，其中真值对应白色；(d)目标体叠加到一个背景场景中(图像由Fiona Corke提供)

从背景中区分开前景对象是机器人视觉中的一个重要问题，但是前景和背景这些术语的界定不够明确，且随着不同应用改变。我们很难保证会拥有之前色度调整示例中那样奢华的特殊背景。我们可以拍摄出一张没有前景目标出现的场景照片，并且认为它是背景，但是这要求我们拥有关于前景目标在什么时候不会出现的专业知识。同时也假定背景不会随着时间而发生变化。在实际的场景中这种变化是一个重大问题，其中场景中的环境照明和阴影在相当短的时间内就会发生改变，而场景本身在结构上的改变则需要相当长的时间。

下一个例子中，我们会处理一个图像序列，并且进行场景的背景估计，尽管在场景中会有很多目标在移动。我们将使用一种递归算法，它将根据先前的估计和现在的图像在每一时间步更新被估计的背景图像 $\hat{\boldsymbol{B}}$：

$$\hat{\boldsymbol{B}}\langle k+1\rangle = \hat{\boldsymbol{B}}\langle k\rangle + c\left(\boldsymbol{I}\langle k\rangle - \hat{\boldsymbol{B}}\langle k\rangle\right)$$

其中，k 是时间步长，$c(\cdot)$ 是一个一元图像饱和函数：

$$c(x)=\begin{cases}\sigma, & x>\sigma\\ x, & -\sigma\leqslant x\leqslant\sigma\\ -\sigma, & x<-\sigma\end{cases}$$

为了演示上述算法，我们打开一个视频，视频中有两个人正在建筑物的大厅中走动：

```
>> vid = Movie('LeftBag.mpg');
384 x 288 @ 24.999750, 1440 frames
```

接着初始化图像序列中第一张图像的背景：

```
>> bg = vid.grab();
```

然后进入主循环：

```
1   sigma = 2;
2   while 1
3      im = vid.grab;
4      if isempty(im), break; end
5      d = im-bg;
6      d = max(min(d, sigma), -sigma);  % apply c(.)
7      bg = bg + d;
8      idisp(bg);  drawnow
9   end
```

来自这个序列的一帧画面如图 12.10(a)所示(见彩图)。图 12.10(b)中被估计的背景图像显示出了场景中的静态元素，两个移动的人已经变得模糊不清。从被估计背景中减去原场景就会产生另一幅图像，如图 12.10(c)所示，其中凡是场景与背景不同的地方像素都变亮。通过给这个图像差异的绝对值设定一个阈值，就可以显示出图像中哪个区域存在运动。当然，如果人停止不动的时间足够长，他们也会成为背景的一部分。

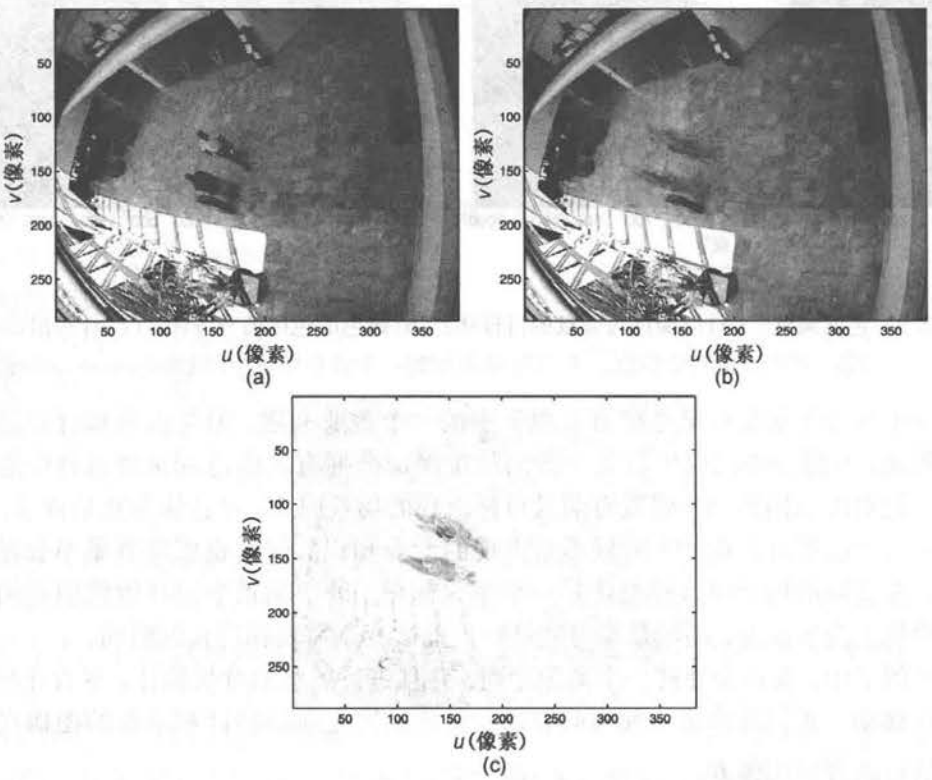


图 12.10 对于 INRIA 的 LeftBag 图像序列在第 250 帧处的图像序列分析示例。(a)当前图像；(b)被估计的背景图像；(c)当前图像与被估计背景图像之间的差值，其中，白色部分为0，红色和蓝色部分分别是负值和正值，值的大小通过颜色强度来表示(视频来自EC投资的CAVIAR工程集/IST 2001 37540)

12.4　空间操作

空间操作的示意图如图 12.11 所示。输出图像中的每一个像素都是输入图像中对应像素点周围区域里所有像素的函数：

$$O[u,v] = f(I[u+i, v+j]),\ \forall (i,j) \in \mathcal{W},\ \forall (u,v) \in I$$

其中，$\mathcal{W}$ 称为窗口，通常是一个 $w \times w$ 的正方形区域，边长为奇数 $w = 2h+1$，$h \in \mathbb{Z}^+$ 是半宽。图 12.11 中，窗口包括红色阴影区中的所有像素。空间操作的功能很强大，因为可用的函数多种多样，包括线性的和非线性的。这节剩余部分将主要讨论线性空间操作，如平滑化和边缘检测，以及一些非线性操作，如秩滤波和模板匹配。下一节中将讨论一种重要的非线性空间操作类型，即数学形态学。

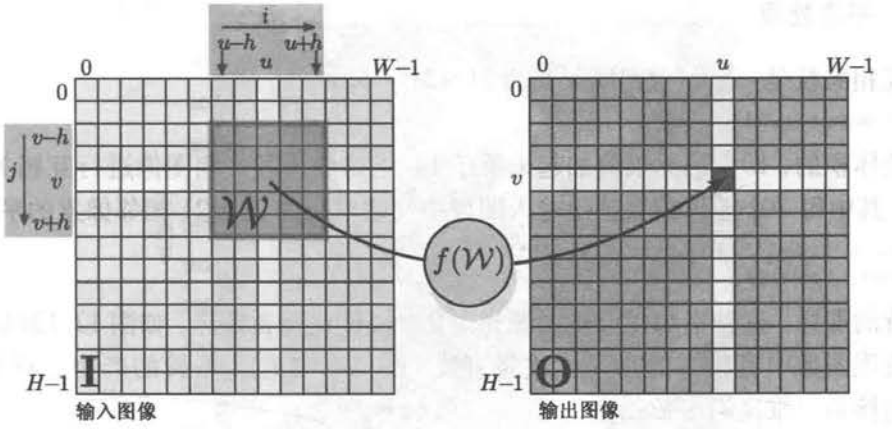


图 12.11　空间图像处理操作。阴影区域窗口 $\mathcal{W}$，是用于计算输出像素（粗箭头所指处）的一组输入像素点

12.4.1　互相关

一种非常重要的线性空间运算操作是互相关：

$$\mathbf{O}[u,v] = \sum_{(i,j)\in\mathcal{W}} I[u+i, v+j]K[i,j],\ \forall (u,v) \in \mathbf{I}$$

其中，$\boldsymbol{K} \in \mathbb{R}^{w\times w}$ 是互相关核。对于每一个输出像素，对应输入图像中的像素窗口 $\mathcal{W}$ 要逐个元素地与互相关核 $\boldsymbol{K}$ 相乘。窗口和核的中心被认为在坐标 $(0,0)$，且 $i,j \in [-h,h]$。这个互相关可以看作是窗口中像素的加权和，其中权值由核 $\boldsymbol{K}$ 定义。正如将要看到的那样，互相关是图像处理的主力军，可以通过选择核 $\boldsymbol{K}$ 来执行不同的函数，如平滑、梯度计算或者边缘检测。通常互相关的运算符写为

$$\mathbf{O} = \boldsymbol{K} \otimes \mathbf{I}$$

互相关的计算量很大，一个具有 $w \times w$ 核的 $N \times N$ 输入图像，需要 w^2N^2 次的乘法和加法运算。在工具箱中，互相关使用函数 `iconv` 来执行：

```
>> O = iconv(K, I);
```

如果 `I` 有多个彩色平面，则输出图像也会有同样多的彩色平面——每一个输出彩色平面是对应输入平面与核 `K` 的互相关。

背景材料：互相关的性质。互相关遵循常见的代数法则，具有可交换性：

$$A \otimes B = B \otimes A$$

可结合性：

$$A \otimes B \otimes C = (A \otimes B) \otimes C = A \otimes (B \otimes C)$$

可分配性(叠加应用)：

$$A \otimes (B + C) = A \otimes B + A \otimes C$$

线性：

$$A \otimes (\alpha B) = \alpha (A \otimes B)$$

以及平移不变性——如果 $S(\cdot)$ 是一个空间平移，则

$$A \otimes S(B) = S(A \otimes B)$$

这就是说，与一个被平移图像的互相关等同于先与未平移图像互相关，再对互相关结果进行平移。

12.4.1.1 平滑处理

考虑互相关核是一个包含相同元素的 21 ×21 方阵：

```
>> K = ones(21,21) / 21^2;
```

并且是单位体积的，即它的元素值加起来等于 1。把这个核和一幅图像进行互相关，得到一幅输出图像，其中每一个输出像素都是输入图像中与之对应的 21 ×21 相邻像素的平均值：

```
>> lena = iread('lena.pgm', 'double');
>> idisp( iconv(K, lena) );
```

正如所期待的那样，这种平均化会使图像光滑化、模糊化或者散焦，如图 12.12(b)所示[①]。仔细观察会发现图像中有模糊的水平线或竖直线——一种被称为振铃的产物。对于平滑处理，更为合适的核是二维高斯函数：

$$G(u,v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}} \tag{12.1}$$

它关于原点对称，且曲线下方的体积为 1。高斯函数的扩展由标准差参数 σ 控制。把这个核应用到图像上：

```
>> K = kgauss(5);
>> idisp( iconv(K, lena) );
```

会产生如图 12.12(c)所示的结果。在这里已经指定了高斯标准差为 5 像素。高斯函数的离散近似为

```
>> about(K)
K [double] : 31x31 (7688 bytes)
```

它是一个 31 ×31 核。为更便捷地实现对图像的平滑处理，可以使用工具箱函数 ismooth：

```
>> idisp( ismooth(lena, 5) )
```

图像模糊化是一种反直觉的图像处理操作，因为我们一般花费精力进行图像处理的目的都是为了获得更清晰的图像。故意破坏图像的效果虽然表面上看起来有些粗鲁，然而就像我们即将看到的那样，高斯平滑处理的结果是非常有用的。

① 散焦现象涉及的核是一个二维艾里图样或者辛格函数。高斯函数在形状上与之相似，但是通常是正的，而艾里图样有小的负值，能形成振荡。

图 12.12 平滑处理。(a)原始图像;(b)用一个 21×21 平均核平滑处理后的图像;(c)用一个31×31高斯$G(\sigma=5)$核平滑处理后的图像

背景材料: 这张 Lena 图像在 20 世纪 70 年代成为图像处理研究的一个标准图样。它来自 1972 年 11 月《花花公子》杂志的中心插图,在 1973 年被南加州大学信号和图像处理研究所(SIPI)的图像压缩研究人员进行了数字化处理。这个模特是居住在瑞典的 Lena Soderberg(nee Sjööblom)。据报道对于这样使用她的照片,她感到很高兴,并且还出席了 1997 年的成像技术会议。也可查阅 http://www.lenna.org。

互相关核本身就是一个矩阵,因此可以用一幅图像把它显示出来:

```
>> idisp( K );
```

如图 12.13(a)所示。我们清楚地看到核中心的灰度值最大,并且在所有方向上都平滑减小。我们也可以用一个面来显示互相关核:

```
>> surfl(-15:15, -15:15, K);
```

如图 12.13(b)所示。对高斯函数的一种粗略近似是一个大礼帽形的互相关核,它具有垂直的

圆柱侧面，其高斯值不再平滑缓慢地减小。函数 kcircle 可以创建一个具有单位高度以及指定半径的圆柱形互相关核：

```
>> K = kcircle(8, 15);
```

如图 12.13(c)所示。参数指定其半径为 8 像素，所在窗口的半宽为 $h = 15$。

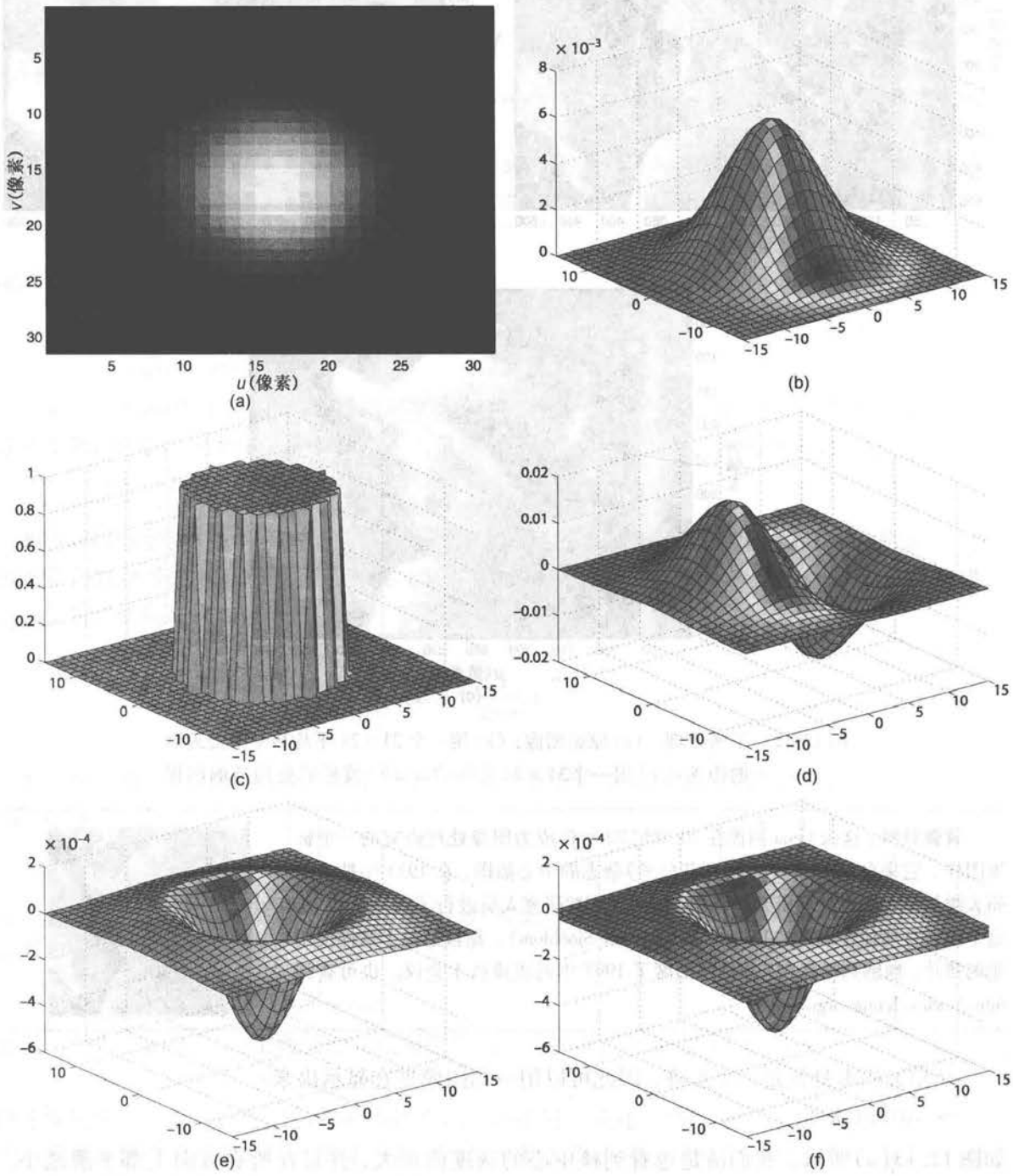


图 12.13 常用的互相关核图集，$h = 15, \sigma = 5$。(a)用强度图像显示的高斯函数；(b)高斯函数；(c)大礼帽形($r = 8$)；(d)高斯导数(DoG)；(e)高斯-拉普拉斯(LoG)；(f)高斯差分(DiffG)

背景材料：高斯函数有多宽？ 当选择高斯互相关核时，我们需要考虑标准差（通常由任务确定）和包含离散高斯函数的互相关核 $\mathcal{W} \in \mathbb{R}^{w \times w}$ 的尺寸。由于计算时间与 w^2 成正比，所以理想上我们希望窗口尽可能小。高斯在所有方向上单调递减，但是不会降到0。因此我们将选择窗口的半宽 h，以使高斯值小于 $w \times w$ 互相关窗口外的某个阈值。

在窗口边缘，离中心的距离为 h 处，高斯值是 $e^{-h^2/2\sigma^2}$。设 $\sigma = 1, h = 2$，则高斯值将会是 $e^2 \approx 0.14$；设 $h = 3$，则为 $e^{-4.5} \approx 0.01$；设 $h = 4$，则为 $e^{-8} \approx 3.4 \times 10^{-4}$。如果 h 没有指定，工具箱会选择 $h = 3\sigma$。设 $\sigma = 1$，则是一个 7×7 的窗口，其中包含了所有大于峰值1%的高斯值。

背景材料：高斯性质。 高斯函数有一些特性。两个高斯函数的互相关也是一个高斯函数：

$$\mathbf{G}(\sigma_1) \otimes \mathbf{G}(\sigma_2) = \mathbf{G}\left(\sqrt{\sigma_1^2 + \sigma_2^2}\right)$$

对于 $\sigma_1 = \sigma_2 = \sigma$ 这种情况，则有

$$\mathbf{G}(\sigma) \otimes \mathbf{G}(\sigma) = \mathbf{G}\left(\sqrt{2}\sigma\right)$$

二维高斯函数是可拆分的，即它可以写成两个一维高斯函数的积：

$$\mathbf{G}(u, v) = \frac{1}{\sqrt{2\pi}\sigma} e^{\frac{u^2}{2\sigma^2}} \times \frac{1}{\sqrt{2\pi}\sigma} e^{\frac{v^2}{2\sigma^2}}$$

这就意味着在计算与一个二维高斯函数的互相关时，可以先用每一行与一个一维高斯函数互相关，然后用每一列与另一个一维高斯函数互相关。这样，运算总数减小到 $2wN^2$，比减小 w 更好。高斯函数也在空间域和频域中有相同的形状。

12.4.1.2 边界效应

互相关的难点出现在窗口接近输入图像边缘的时候，如图12.14所示。在这种情况下，决定输出像素的窗口包含了输入图像边缘外的像素。对于这个问题，有几种补救办法。首先，可以假定输入图像外的像素有一个特殊值，通常选为0，而且这也是工具箱函数 iconv 采用的默认方式。我们可以从图12.12中看到这样做的效果，其中平滑处理后的图像边界由于这些0的影响是黑的。

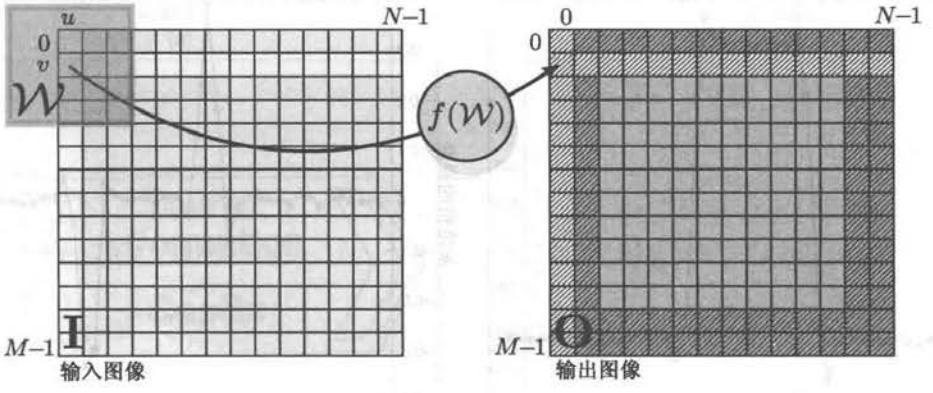


图12.14 当窗口 $\mathcal{W}$ 落在输入图像边缘时，在 (u, v) 处的输出像素无法确定。输出图像中所有画阴影线的像素都是输出值不确定的

另一个方法是当窗口超过输入图像边界时认为其输出结果无效。无效的输出像素如图12.14中画阴影线的部分所示。该方法的结果是得到一个 $(M - 2h) \times (N - 2h)$ 的输出图像。通过在函数 iconv 中输入选项'valid'就能实现这种方法。

12.4.1.3 边缘检测

我们经常会对一个场景中找出目标物体的边缘感兴趣。考虑一幅图像：

```
>> castle = iread('castle_sign.jpg', 'double', 'grey');
```

如图 12.15(a)所示。沿着图像中一个一维轮廓观察其像素值，你会获得很多信息。在 $v = 360$ 处图像中的一条水平线是

```
>> p = castle(360,:);
```

它是一个向量，我们可以绘制出它相对于横坐标 u 的图形：

```
>> plot(p);
```

如图 12.15(b)所示。其中，显而易见的高峰值对应于这个标牌上的白色字母和其他标记。更仔细地观察其中的一个峰值，如图 12.15(c)所示，我们会看到穿过字母 T 竖直笔画时的强度轮廓。背景的强度≈0.3，明亮处的强度≈0.9，但都与照明水平有关。然而，这种仅仅跨越几个像素距离的强度突变是很明显的，比起任何依靠实际灰度水平确定边缘的方式，用这种突变来判别更为可靠。

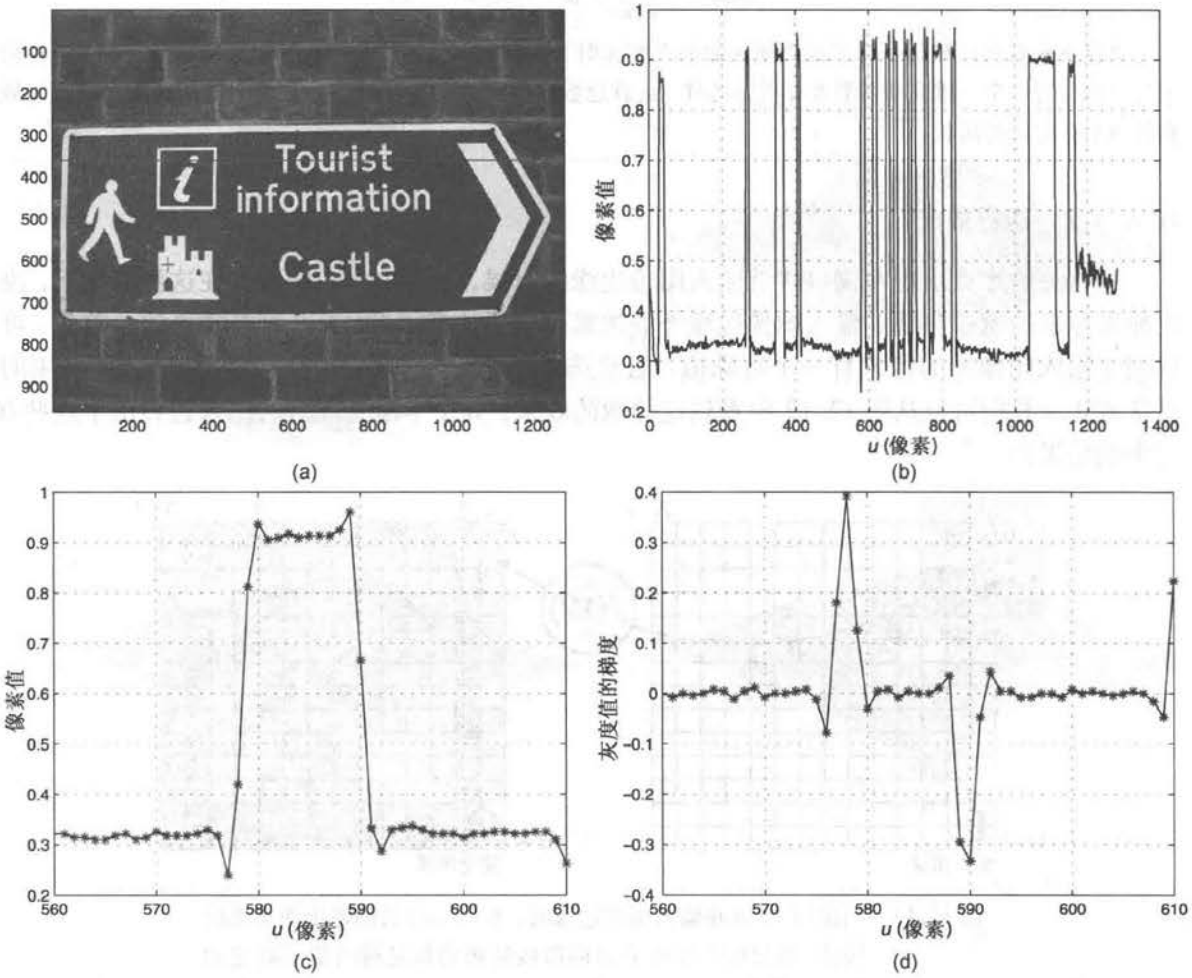


图 12.15 边缘强度轮廓。(a)原始图像；(b) $v = 360$ 沿着水平线的灰度轮廓；(c) $u \approx 580$ 处峰值的特写镜头；(d)图(c)的导数(图片来自ICDAR 2005 OCR资料组；Lucas 2005)

沿着这个横截面的一阶导数是

$$p'[v] = p[v] - p[v-1]$$

它可以使用 MATLAB® 函数 diff 来计算：

```
>> plot(diff(p))
```

如图 12.15(d)所示。信号在一般情况下为 0，只有在目标边缘处出现明显的非零响应，在这里是字母 T 竖直笔画的边缘。

在点 v 处的导数也可以写成对称的一阶差分：

$$p'[v] = \frac{1}{2}\big(p[v+1] - p[v-1]\big)$$

它等同于和下面的一维互相关核求互相关运算：

$$\boldsymbol{K} = \left(-\frac{1}{2}\ \ 0\ \ \frac{1}{2}\right)$$

将图像与这个核求互相关：

```
>> K = [-0.5 0 0.5];
>> idisp( iconv(castle, K), 'invsigned')
```

会产生一个与图 12.16(a)非常相似的结果，图中具有高水平梯度的竖直边缘清晰可见。

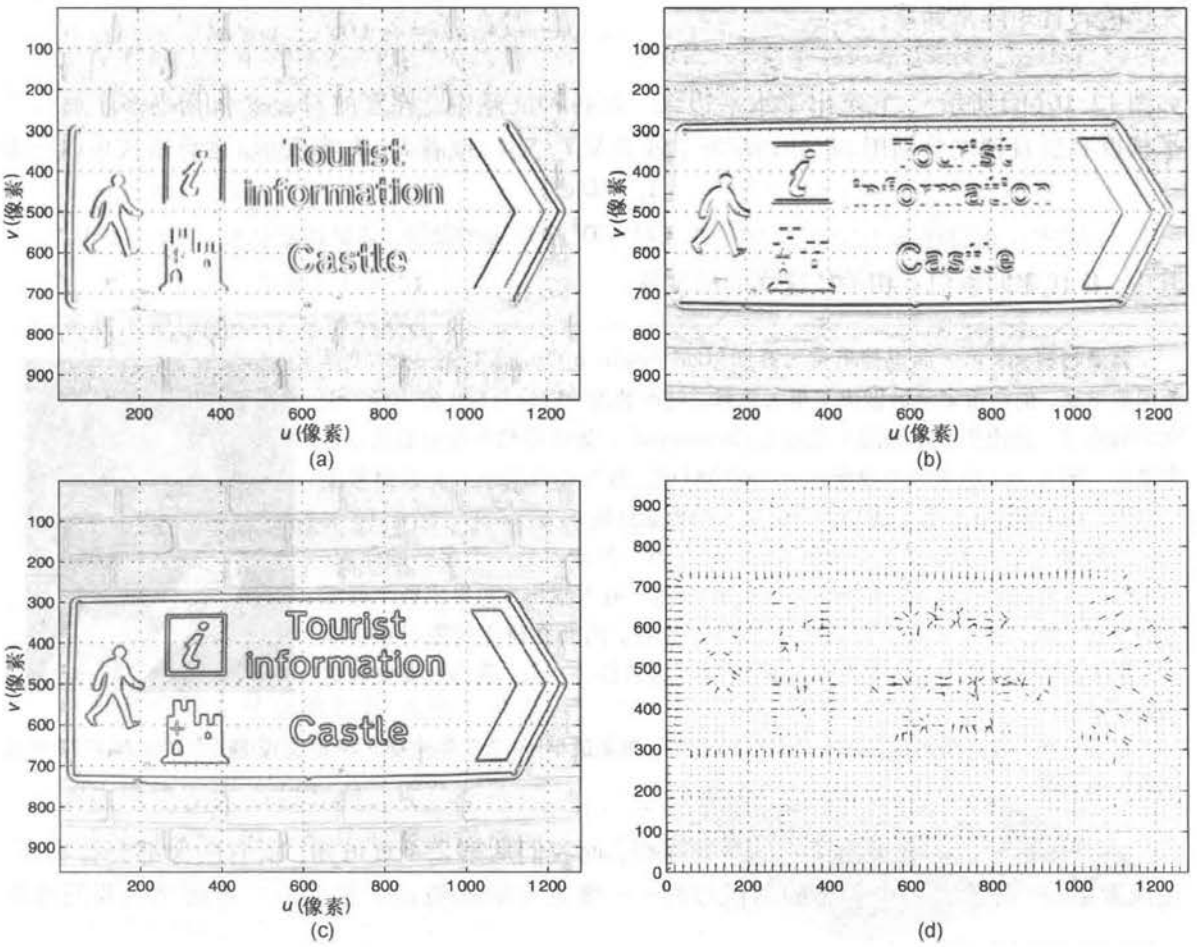


图 12.16　边缘梯度。(a) u 轴梯度；(b) v 轴梯度；(c)梯度幅值；(d)梯度方向

由于这个核包含带符号的值，所以互相关的结果也将是有符号的，即像素点的梯度可为正，也可为负，如图 12.15(d)所示。Idisp 总是把最小值(大部分为负)显示为黑色，把最大值(大部分为正)显示为白色，因而零值显示为中度灰色。如果在 idisp 中采用选项'signed'，那么就能用红色和蓝色阴影来清晰地表示符号：0 是黑色，负像素是红色，正像素是蓝色，并且颜色强度和像素值的大小成正比。如果用选项'invsigned'，那么除了 0 用白色表示外，其他的都相同。

学者们提出了许多互相关核用来计算水平梯度。最常见的是索伯(Sobel)互相关核：

```
>> Du = ksobel
Du =
    -1     0     1
    -2     0     2
    -1     0     1
```

它的每一行与前面定义的一维互相关核 K 相似。总的结果是三行水平梯度的加权和。用我们的图像与这个核做互相关：

```
>> idisp( iconv(castle, Du), 'invsigned')
```

将会产生一个水平梯度图像，如图 12.16(a)所示，它突出了竖直边缘。竖直梯度使用该互相关核的转置矩阵来计算：

```
>> idisp( iconv(castle, Du'), 'invsigned')
```

如图 12.16(b)所示[①]，它突出了水平边缘。在各种文献中，梯度的符号变化相当多。通常水平梯度和竖直梯度分别用 $\partial \boldsymbol{I}/\partial u, \partial \boldsymbol{I}/\partial v$，或者 $\nabla_u \boldsymbol{I}$，$\nabla_v \boldsymbol{I}$，或者 $\boldsymbol{I}_u$，$\boldsymbol{I}_v$ 来表示。运算格式可以写为

$$\mathbf{I}_u = \mathbf{D} \otimes \mathbf{I}$$
$$\mathbf{I}_v = \mathbf{D}^{\mathrm{T}} \otimes \mathbf{I}$$

其中，**D** 代表如索伯互相关核这样的梯度核。

背景材料： 卡尔·弗里德里希·高斯(Carl Friedrich Gauss，1777—1855)是德国数学家，他在很多领域做出了重大贡献，例如数论、微分几何、磁力学、天文学和光学。他出生于德国的不伦瑞克(Brunswick)，父母都没有受过教育，是家里唯一的儿子。他是一个神童，在三岁的时候，他靠心算纠正了父亲财务上的错误，在十多岁的时候，成功取得了第一个数学发现。

高斯是一个完美主义者，并且是一个工作狂，但却不是一个多产的作家。他出版的论著都是他认为完整且无可指责的东西。有人指出，如果出版了他所有的发现，那么数学可能会向前推动 50 年。据传，高斯在研究问题时被打断了，并被告知他的妻子即将死去——他回应说“告诉她等一会，我马上要突破这个问题了”。

正态分布，或高斯函数，并不是高斯的成果，而是最早于 1733 年由德·摩根发现的，之后又在 1778 年被拉普拉斯发现。

求一个信号的导数突出了它的高频噪声，而我们从第 254 页可知，所有图像都会有噪声。在像素层级，噪声是一个静态的随机过程——像素之间的值是不相关的。然而那些我们感兴

① 过滤器可以被设计为在任何一个角度来响应边缘。索伯互相关核本身可以看作是一幅图像，并使用 irotate 进行旋转。为了获得需要的角度精度，通常需要一个较大的核，例如由 kdgauss 产生的核。

趣的特征，如边缘，却在一个较大的空间范围内存在像素值的相关变化，如图 12.15(c)所示。我们可以在求导数之前通过平滑处理图像来减少噪声的影响：

$$\mathbf{I}_u = \mathbf{D} \otimes (\mathbf{G}(\sigma) \otimes \mathbf{I})$$

我们不会先求高斯函数与图像的互相关，然后求导，而是利用互相关的结合性写出：

$$\nabla \mathbf{I} = \mathbf{D} \otimes (\mathbf{G}(\sigma) \otimes I) = \underbrace{(\mathbf{D} \otimes \mathbf{G}(\sigma))}_{\text{DoG}} \otimes \mathbf{I}$$

我们计算了图像与高斯导数(DoG)的互相关，其中高斯导数的数值解可由以下函数获得：

```
>> Gu = iconv( Du, kgauss(sigma) );
```

或者也可以通过求高斯函数方程(12.1)在 u 方向的偏导数来获得解析解：

$$\mathbf{G}_u(u,v) = -\frac{u}{2\pi\sigma^2} \mathrm{e}^{\frac{u^2+v^2}{2\sigma^2}} \tag{12.2}$$

上式通过工具箱函数 kdgauss 来计算，如图 12.13(d)所示。标准差 σ 控制被检测边缘的尺度。较大的 σ 意味着增加平滑性，它将裁剪掉纹理细微的边缘，只留下有大特征的边缘。这种在不同空间尺度寻找边缘的能力非常重要，而且支持了 13.3.2 节中将要讨论的尺度空间的概念。这种运算的另一种阐述是作为一种空间的带通滤波器，因为它是由低通滤波器(平滑)和高通滤波器(微分)级联而成的。

计算每一个像素点的水平梯度分量和竖直梯度分量：

```
>> Iu = iconv( castle, -kdgauss(2) );
>> Iv = iconv( castle, -kdgauss(2)' );
```

就可以求出每一个像素点的梯度幅值大小：

```
>> m = sqrt( Iu.^2 + Iv.^2 );
```

图 12.16(c)所示的边缘强度图像清楚地显示了所有边缘。每一个像素点的梯度方向是

```
>> th = atan2( Iv, Iu);
```

然后绘制出最便于观察的稀疏箭头图：

```
>> quiver(1:20:numcols(th), 1:20:numrows(th), ...
       Iu(1:20:end,1:20:end), Iv(1:20:end,1:20:end))
```

如图 12.16(d)所示。边缘梯度方向图比梯度幅值图的噪声更大。边缘梯度强的地方，如标牌的边界或者字母的边缘，梯度方向是垂直于边缘的，但是对于尺度细小的砖结构，边缘方向则呈现出随机性。梯度图像可以使用工具箱函数简便地计算出来：

```
>> [du,dv] = isobel( castle, kdgauss(2) );
```

其中最后一个参数替代了默认的索伯互相关核。

一种公认的非常有效的边缘检测器是坎尼(Canny)边缘算子。它使用了刚才我们计算的边缘梯度大小和方向，并且还执行了两个附加步骤。第一步是抑制非局部的最大值。考虑图 12.16(c)中的梯度幅值图像，将其转化成图 12.17 所示的三维曲面，其中高度与亮度成正比例。从图中看到了一系列的山峰和山脊，我们期望找到那些

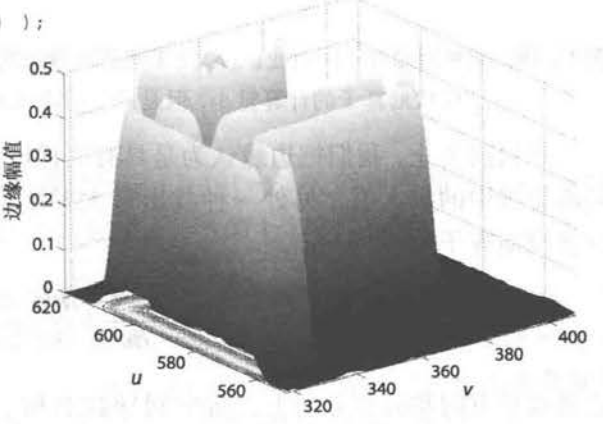


图 12.17　用三维曲面显示的字母 T 周围的梯度幅值

位于山顶或沿着山脊线上的像素。通过检测垂直于边缘方向(即沿边缘梯度方向)一个局部邻域中的像素值，可以找到最大像素值，然后设定其余的像素值为0。结果是得到一组对应于山顶和山脊线的非零像素点。第二步是滞后阈值化。对于每一个超过阈值上限的非零像素点，用其紧邻的且超出阈值下限的像素点创建一个像素链。其他任何像素值都设为0。

背景材料：皮埃尔-西蒙·拉普拉斯(Pierre-Simon Laplace，1749—1827)是法国数学家和天文学家，他的五部《天体力学》(*Mécanique Céleste*)专著强化了数理天文学的理论。在青少年时期，他的数学能力就给帮助他取得教授职位的达朗伯(d'Alembert)留下了深刻印象。据报道，当被拿破仑问到为什么在他关于天文学的书中没有提到上帝时，他回答说"我不需要那种假说"。在1806年他成为皇家伯爵，后来又成为侯爵。

拉普拉斯算子(一个二阶微分算子)，以及拉普拉斯变换都是以他的名字命名的。

可以直接把坎尼算子应用到我们的示例图像中：

```
>> edges = icanny(castle, 2);
```

它返回如图12.18(a)所示的一幅图像，其中边缘是通过对应于那处像素点梯度幅值的非零强度值标出。我们观察到，这个边缘比图12.18(b)所示的高斯导数幅值算子所获得的图像边缘要细，本例中高斯导数运算的 $\sigma = 2$。这种滞后的阈值参数可以通过选项参数进行设定。

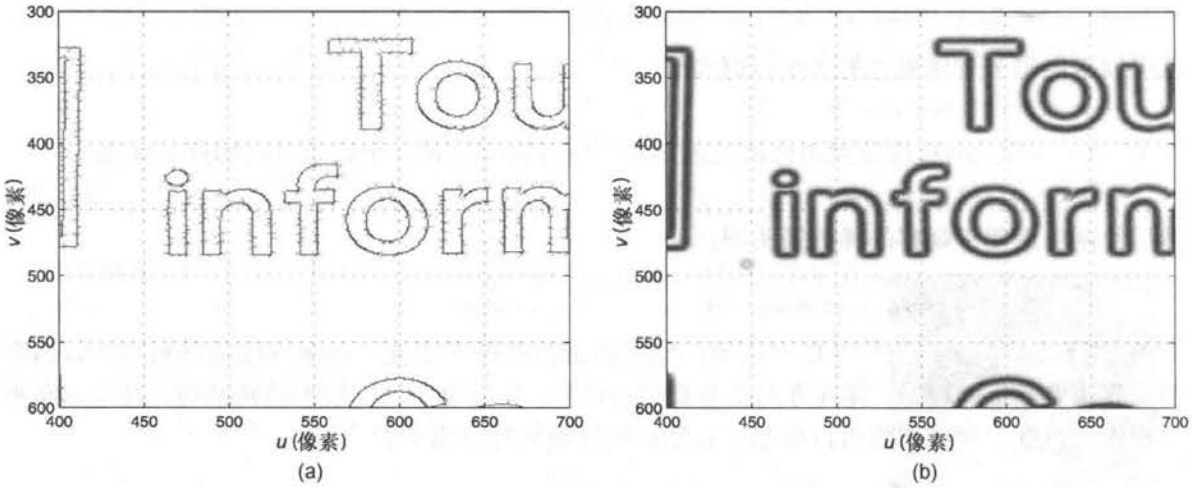


图12.18 两种边缘算子的对比。(a)使用默认参数的坎尼算子；(b)高斯核导数($\sigma = 2$)的幅值。|DoG|算子比坎尼算子的计算量小，但是产生的边缘较厚。这两种情况的结果都采用反色显示，白色值是0

到目前为止，我们把边缘认为是具有高梯度值的一个点，并且采用了非最大值抑制来搜索局部邻域中的最大值。另外一种找出最大梯度值点的方法是计算二阶导数并确定零点位置。拉普拉斯算子为

$$\nabla^2 \mathbf{I} = \frac{\partial^2 \mathbf{I}}{\partial u^2} + \frac{\partial^2 \mathbf{I}}{\partial v^2} = \mathbf{I}_{uu} + \mathbf{I}_{vv} = \mathbf{L} \otimes \mathbf{I} \tag{12.3}$$

它是水平方向和竖直方向上二阶空间导数的和。对于一个离散图像，上式可以通过与拉普拉斯核进行互相关来计算：

```
>> L = klaplace()
L =
     0     1     0
     1    -4     1
     0     1     0
```

该拉普拉斯核是各向同性的，即它在任何方向上对边缘的响应都相同。二阶导数比一阶导数对噪声更加敏感，因此它通常又与经高斯平滑处理的图像一起使用：

$$\mathbf{L} \otimes (\mathbf{G}(\sigma) \otimes \mathbf{I}) = \underbrace{(\mathbf{L} \otimes \mathbf{G}(\sigma))}_{\text{LoG}} \otimes \mathbf{I} \tag{12.4}$$

式中把它们合成为高斯-拉普拉斯核(LoG)，其中 $\mathbf{L}$ 是上文给定的拉普拉斯核。高斯-拉普拉斯核的解析式也可写作为

$$\mathbf{LoG}(u,v) = \frac{\partial^2 \mathbf{G}}{\partial u^2} + \frac{\partial^2 \mathbf{G}}{\partial v^2} \tag{12.5}$$

$$= \frac{1}{\pi\sigma^4}\left(\frac{u^2+v^2}{2\sigma^2} - 1\right) \mathrm{e}^{\frac{u^2+v^2}{2\sigma^2}} \tag{12.6}$$

上式也称为马尔-希尔德累思(Marr-Hildreth)算子或者墨西哥帽互相关核，如图 12.13(e)所示。

把这个核应用到我们的图像中：

```
>> lap = iconv( castle, klog(2) );
```

结果如图 12.19(a)和图 12.19(b)所示(见彩图)。梯度最大值出现在二阶导数为 0 的地方，但是有效边缘是从一个强正值(蓝色)到一个强负值(红色)之间的跨零区。考虑图 12.19(b)中字母 T 的拉普拉斯算子特写图。我们生成一个 $v = 360$ 处字母 T 竖直笔画的水平横截面：

```
>> p = lap(360,570:600);
>> plot(570:600, p, '-*');
```

背景材料：高斯差分。高斯-拉普拉斯算子可以近似为两个高斯函数的差分：

$$\mathbf{DiffG}(u,v;\sigma_1,\sigma_1) = \mathbf{G}(\sigma_1) - \mathbf{G}(\sigma_2) = \frac{1}{2\pi\sigma_1^2\sigma_2^2}\left(\sigma_2^2 \mathrm{e}^{\frac{u^2+v^2}{2\sigma_1^2}} - \sigma_1^2 \mathrm{e}^{\frac{u^2+v^2}{2\sigma_2^2}}\right)$$

其中，$\sigma_1 = 1.6\sigma_2$，且通常 $\sigma_1 > \sigma_2$。图 12.13(e)和图 12.13(f)分别比较了 LoG 核和 DiffG 核。

这种近似对于我们将在 13.3.2 节中讨论的尺度空间序列是有用的。考虑一个图像序列 $\mathbf{I}(k)$，其中 $\mathbf{I}(k+1) = \mathbf{G}(\sigma) \otimes \mathbf{I}(k)$，就是说这个序列中的图像越来越平滑。因此，序列中的任意两幅图像之间的差分等同于把 DiffG($\sqrt{2}\sigma,\sigma$) 应用到原始图像上。

如图 12.19(c)所示。我们看到二阶导数为 0 的地方位于像素之间。一个跨零检测器将选择紧邻跨零区的像素：

```
>> zc = zcross(lap);
```

如图 12.19(d)所示。我们看到边缘是双层的。再对照图 12.19(c)，我们观察到在 $u \in [573, 574]$ 之间有一个较弱的跨零区，更加确定的跨零区位于 $u \in [578, 579]$ 之间。

所有边缘检测方法的一个根本限制是强度值边缘不一定就是目标对象的边界。目标对象可能与背景的反差不明显，从而导致较模糊的边界边缘。相反，目标上可能有一个清晰的条纹，但不是它的边缘。通常阴影有锐利边缘，但却不是真正的目标。目标纹理结构也会导致那些不在边界上的点在边缘检测器中产生强烈输出，如图 12.15(b)所示的砖的例子。

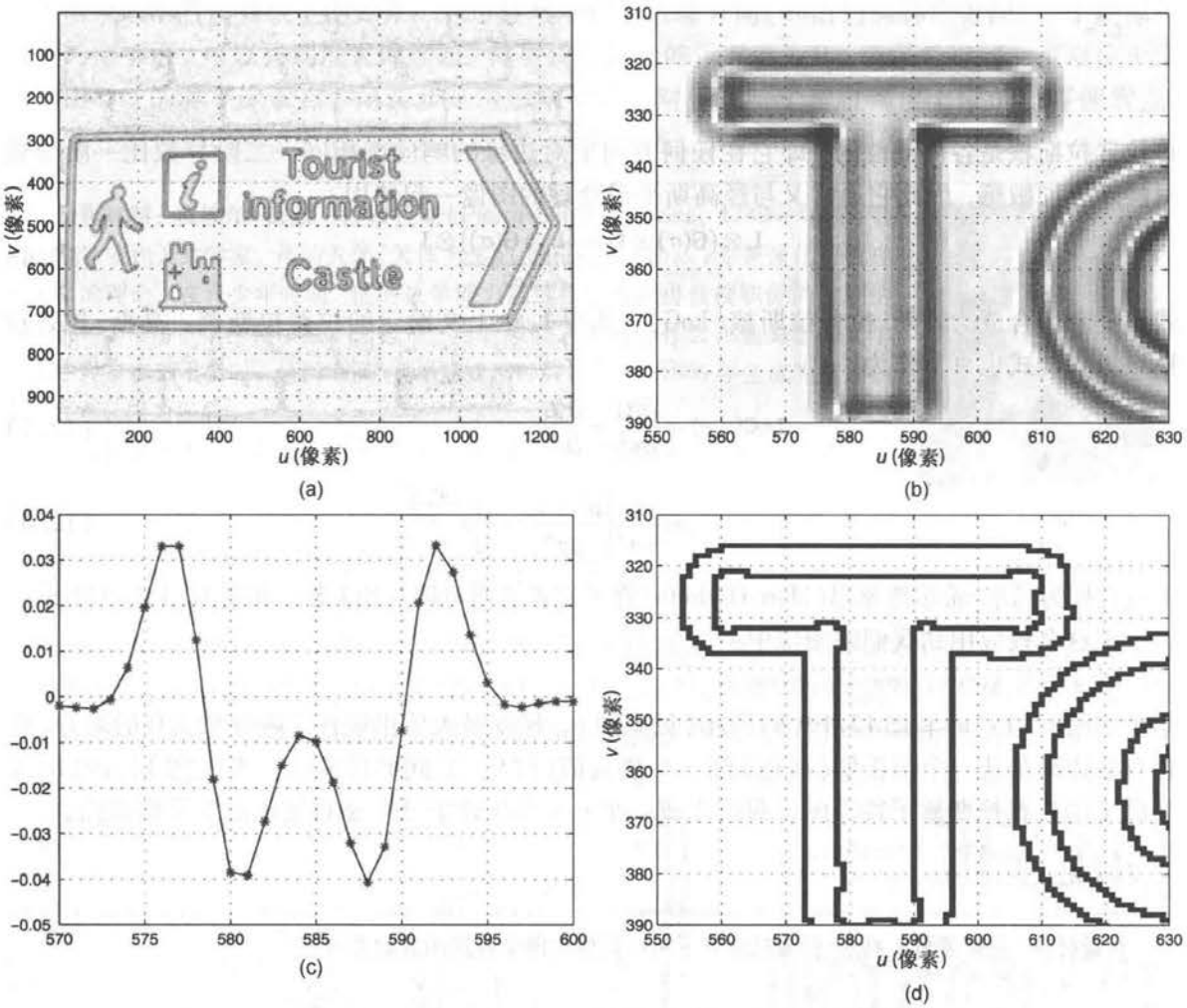


图 12.19 高斯-拉普拉斯算子。(a)高斯-拉普拉斯算子结果;(b)字母 T 周围的特写,其中蓝色和红色分别表示正值和负值;(c)穿过字母T竖直笔画的LoG水平横截面;(d)字母T处跨零区探测器的输出特写

12.4.2 模板匹配

在前面的讨论中使用了内核来代表数学函数,如高斯函数、高斯函数的导数及高斯-拉普拉斯算子。我们也曾经把互相关核看作一个矩阵、一个三维向量的形状,甚至是如图 12.13(a)所示的一个图像。在这一节中,我们将会把内核考虑为一幅图像或者图像的一部分,这样的内核被称为模板。在模板匹配中,我们期望找出输入图像中的哪些部分与模板最为相似。

模板匹配的示意图如图 12.20 所示。输出图像中每一个像素点都由下式给出:

$$\mathbf{O}[u,v] = s(\mathbf{T}, \mathcal{W}), \ \forall (u,v) \in \mathbf{I}$$

其中,$\mathbf{T}$ 是 $w \times w$ 模板,即我们要寻找的像素模式,模板边长为奇数 $w = 2h + 1$;$\mathcal{W}$ 是输入图像里中心位于 (u,v) 的 $w \times w$ 窗口。函数 $s(\mathbf{I}_1, \mathbf{I}_2)$ 是一个标量测度,用来描述两幅相同大小的图像 $\mathbf{I}_1$ 和 $\mathbf{I}_2$ 之间的相似性。

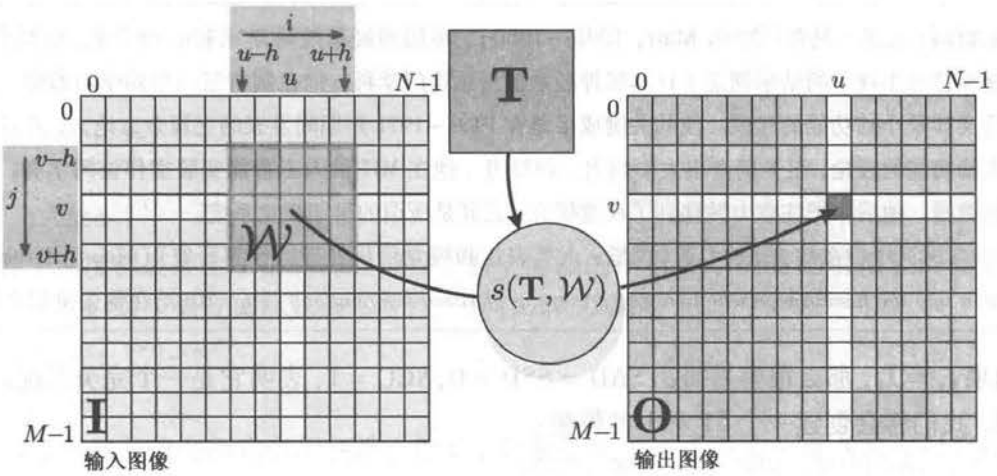


图 12.20　空间图像处理运算。输入图像中的阴影区代表窗口 $\mathcal{W}$，其中的一组像素被用于计算输出像素(右侧箭头所指处)

表 12.1 中给出了很多常见的相似性测度[①]。最直观的算法是计算 $\boldsymbol{T}-\mathcal{W}$ 之间每个对应像素的差值，再求出绝对差值和(SAD)或差值平方和(SSD)。在两个图像完全相同的情况下这些度量值都等于 0，但随着图像差异的增多它们也会增大。很难说什么样的测度值可算作匹配较差，但我们可以使用一个相似性测度的排序来判定最佳匹配。

表 12.1　两个大小相同的图像区域 I_1 和 I_2 之间的相似性测度。有前缀 Z 的表示该测度考虑到了零偏移或两张图像平均值的差异[Banks and Corke,2001]。$\bar{I}_1$，$\bar{I}_{12}$ 分别是图像区域 I_1，I_2 的平均值。相应的工具箱函数列在最后一列

绝对差值和		
SAD	$s=\sum_{(u,v)\in I}\lvert I_1[u,v]-I_2[u,v]\rvert$	sad
ZSAD	$s=\sum_{(u,v)\in I}\lvert I_1[u,v]-\bar{I}_1)-(I_2[u,v]-\bar{I}_2)\rvert$	zsad
差值平方和		
SSD	$s=\sum_{(u,v)\in I}(I_1[u,v]-I_2[u,v])^2$	ssd
ZSSD	$s=\sum_{(u,v)\in I}((I_1[u,v]-\bar{I}_1)-(I_2[u,v]-\bar{I}_2))^2$	zssd
互相关		
NCC	$s=\frac{\sum_{(u,v)\in I}I_1[u,v]\cdot I_2[u,v]}{\sum_{(u,v)\in I}I_1^2[u,v]\cdot\sum_{(u,v)\in I}I_2^2[u,v]}$	ncc
ZNCC	$s=\frac{\sum_{(u,v)\in I}(I_1[u,v]-\bar{I}_1)\cdot(I_2[u,v]-\bar{I}_2)}{\sqrt{\sum_{(u,v)\in I}(I_1[u,v]-\bar{I}_1)^2\cdot\sum_{(u,v)\in I}(I_2^2[u,v]-\bar{I}_2)^2}}$	zncc

更为复杂的测度，如归一化互相关，会在区间[−1，+1]进行打分，+1 表示完全相同区。实际应用中，大于 0.8 的值都被认为是匹配良好的。然而归一化互相关的计算量更大——需要乘法、除法和平方根运算。注意，如果分母为 0，那么互相关的结果就不确定，这种情况会发生在 $\boldsymbol{I}_1$ 或者 $\boldsymbol{I}_2$ 中自身所有元素都相同时。

① 这些测度可以通过一个高斯加权来增强，以淡化出现在两个窗口边缘的不同。

背景材料：大卫·马尔(David Marr, 1945—1980)，英国神经系统科学家和心理学家，他综合心理学、人工智能和神经生理学的结果创立了计算机神经系统科学这门学科。他在剑桥三一学院学习数学，他的心理学博士论文涉及小脑功能的建模。他的关键成果是在1969～1971年期间发表的三篇杂志论文，并且形成了哺乳动物大脑功能的理论，至今仍有很大影响力。1973年，他在MIT的人工智能实验室作访问学者，随后成为心理系的教授。他后来把注意力转移到了视觉研究，尤其是所谓的早期视觉系统。

他35岁时死于白血病，他的专著《视觉：人类表达和视觉信息处理的计算研究》(*Vision: A computational investigation into the human representation and processing of visual information*)[Marr 2010]在他去世后出版。

如果$\boldsymbol{I}_1 \equiv \boldsymbol{I}_2$，那么很容易得出SAD = SSD = 0，NCC = 1，表明它是一个完美匹配。为了展示说明，我们将会创建一个51×51的模板：

```
>> T = iroi(lena, [240 290; 250 300]);
```

然后，分别评估三种常用测度：

```
>> sad(T, T)
ans =
     0
>> ssd(T, T)
ans =
     0
>> ncc(T, T)
ans =
     1
```

现在考虑这种情况：两幅图像具有相同的场景，但是其中一幅比另一幅暗——照明或者相机曝光发生了变化。在这种情况下$I_1 = \alpha I_2$，现在：

```
>> sad(T, T*0.9)
ans =
  132.8090
>> ssd(T, T*0.9)
ans =
    7.3307
```

这些测度都表现出了很高的差异性。然而采用归一化互相关：

```
 >> ncc(T, T*0.9)
ans =
     1
```

对于光强的改变，它保持了不变。

接下来，我们考虑像素值的偏移，这时$I_2 = \alpha I_1 + \beta$。可以发现[①]：

```
>> sad(T, T+0.1)
ans =
  260.1000
>> ssd(T, T+0.1)
ans =
   26.0100
>> ncc(T, T+0.1)
ans =
    0.9990
```

所有测度都表现出了差异性。这个偏移问题的解决方法是先让**T**和$\mathcal{W}$各自减去它们自己的平均值：

① 这可能是由于不正确的暗电平设定所致。相机的暗电平是对应于无光时的像素值，通常大于0。

```
>> zsad(T, T+0.1)
ans =
   1.7300e-11
>> zssd(T, T+0.1)
ans =
   1.1507e-25
>> zncc(T, T+0.1)
ans =
    1.0000
```

这样，各个测度都表现出了很高的相似度。有 z 前缀的函数表示的是之前描述的相似性测度的变形体，它们对于强度偏移具有不变性。只有 ZNCC 测度

```
>> zncc(T, T*0.9+0.1)
ans =
    1.0000
```

对于增益及偏移的改变都具有不变性。但是，如果图像发生了哪怕一点点的相对旋转或者尺寸变化，那么所有的这些方法都将会失败。

现在考虑这个问题，它来自众所周知的儿童读物《沃利在哪儿?》(*Where's Wally*?)或《沃尔多在哪儿?》(*Where's Waldo*?)——其乐趣就在于要在一大群人中找出沃利的脸：

```
>> crowd = iread('wheres-wally.png', 'double');
>> idisp(crowd)
```

幸运的是我们大概知道他长什么样子，模板为

```
>> T = iread('wally.png', 'double');
>> idisp(T)
```

它是从另一幅图像中提取出来的，且进行了放大，以便其头部与人群场景中其他人头部的宽度大致相等(大约 21 个像素宽)。

我们的模板 T 与每一个可能窗口位置的相似性，可以通过使用匹配测度 ZNCC 来计算：

```
>> S = isimilarity(T, crowd, @zncc);
```

结果是

```
>> idisp(S, 'colormap', 'jet', 'bar')
```

如图 12.21 所示，像素的颜色表明了 ZNCC 相似性(通过旁边的颜色条指示)。我们可以看到很多高度相似的点，它们都是沃利可能在的候选位置。在一个局部的 3×3 窗口中，它们的峰值是

```
>> [mx,p] = peak2(S, 1, 'npeaks', 5);
>> mx
mx =
    0.5258    0.5230    0.5222    0.5032    0.5023
```

按降序排列。第二个参数指定了窗口的半宽 $h = 1$，第三个参数指定了返回的特征数量。最大值 0.5258 是找到的最强匹配的相似性。这些匹配发生在坐标 (u,v) 处，由第二个返回值 p 给出，我们可以在场景上使用透明蓝圈和数字标号来突出这些点：

```
>> idisp(crowd);
>> plot_circle(p, 30, 'fillcolor', 'b', 'alpha', 0.3, 'edgecolor', 'none')
>> plot_point(p, 'sequence', 'bold', 'textsize', 24, 'textcolor', 'k')
```

其中，在(261, 377)处的最好匹配事实上就是正确答案——我们找到了沃利！我们再观察其他序号靠前的候选者，会发现在图像底部序号为 2 和 3 的也是戴着棒球帽的人，看起来也非常相似。

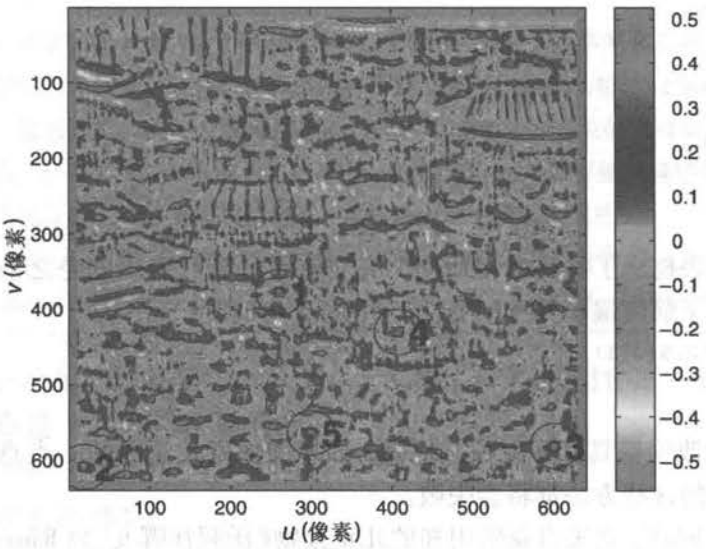


图 12.21 标记了前五个沃利候选者的相似图形 S。旁边的颜色条指示了相似程度。请注意具有不确定值的图像边界，那是由于模板窗口已经出了输入图像的边缘

这个例子中有几个重点需要注意。图像的分辨率很低，并且模板只有 21 × 21——只是与沃利粗略相似。匹配不是很强——相较于最大的可能值 1，它只有 0.5258，其成因有多个方面。首先，匹配测度是随图像的大小比例变化的，即随着相对尺寸(缩放)的变化，相似度下降很快。实际应用中，$\mathbf{T}$ 和 $\mathcal{W}$ 之间尺度变化在 10% ~20% 或许还能忍受，但对于这个例子，模板只是近似进行了缩放。其次，并不是所有的沃利都是相同的。模板中的沃利面朝前方，但是我们在图像中找到的沃利却正看向我们的左边。另一个问题是方形模板中通常既包含我们感兴趣的目标像素，也包含来自背景的像素。随着目标的移动，背景像素可能发生改变，从而导致较低的相似度。这称为混合像素问题。实际应用中，由于透视变形还会产生另一个问题。图像中心的像素为整齐的方形排列，但在图像的边缘处像素则会呈现拱顶形状，这样也不会与方形模板产生很好的匹配。

无参数的相似性测度对于混合像素问题有更好的鲁棒性。这一类测度中常见的两种方法是统计度量和排秩变换。对于统计度量，每一个窗口、模板和候选者都被映射为一个位串，其中每一位都对应于那个窗口的一个像素，如图 12.22 所示。如果某个像素值比中心像素大，则它相应的位就会设为 1，否则设为 0。对于一个 $w \times w$ 的窗口来说，这个串的长度是 $w^2 - 1$ 位。比较两个位串可以使用汉明(Hamming)距离，即两个位串中对应位不同的数量。它可以通过对两个位串进行异或运算，再计算结果串中置 1 位的个数来获得。因此，与传统方法相比，它几乎不需要算术运算——没有平方根或者除法运算，这样的算法对于在特殊目的硬件或者 FPGAs 中实施是更为可靠的。这种方法的另一个优势是考虑了相对于窗口中心像素的强度，使得它对于整体强度变化或渐变的强度梯度具有不变性[①]。

排秩变换把每一个窗口投影为一个标量，即窗口中比中心像素值大的元素的数量。这种测度方法抓住了围绕中心像素区域的本质，而且由于它是基于局部的相对灰度值，所以与统计

① 对于一个 32 位的整数 `uint32`，它把窗口限制在 5 × 5 大小，除非采用一种稀疏映射[Humenberger et al., 2009]。一个 64 位的整数 `uint64` 支持 7 × 7 窗口。

度量一样，对于整体强度变化或渐变的强度梯度也具有不变性。排秩变换通常用作预处理步骤，在使用简单的经典相似性测度(如 SAD)之前把它应用到图像的每一个像素。工具箱函数 isimilarity 使用'census'和'rank'选项来支持这些测度方法。

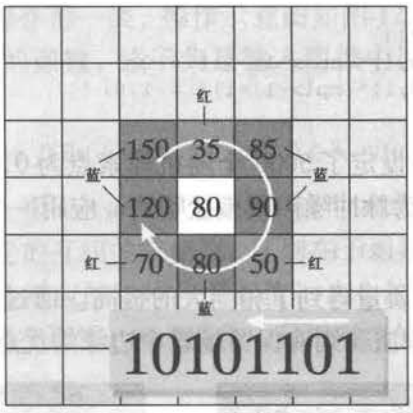


图 12.22 对于一个 3×3 窗口的统计度量和排秩变换示例。如果某个像素值小于中心像素值就标记为红色，若大于或等于就标记为蓝色。然后这些布尔值会被装进二进制的字中，方向如图所示，从最低有效位向上。统计值是10101101或十进制数173。秩变换值是所有置1位的数量，即5

模板匹配的常见问题是出现错误匹配。在前面的示例中，第二个坐标点的相似度只比第一个低 0.5%，而第五个坐标也只低 5%。实际应用中，在匹配被接受之前还要应用很多规则：相似度必须超过某个阈值，并且第一候选者必须超出第二候选者某个量级。另一种办法是引入更多信息来处理这个问题，例如相机或目标已知的运动。举一个例子，如果在一个图像序列中一帧一帧地追踪沃利，那么可能会挑选距离他前一帧图像中的位置最近的候选者作为最佳。另一种方式是我们可能会建立一个运动模型，并假定他在两个连续帧之间移动的方向和距离几乎相同。然后我们会预测他的未来位置，并挑出最接近于预测位置的沃利。当然，我们可能不得不处理一些实际难题，例如沃利停止不动、改变方向或者变得模糊，等等。

12.4.3 非线性操作

另外一种空间操作是基于窗口中像素点的非线性函数。例如：

```
>> out = iwindow(lena, 3, 'var');
```

它计算每一个 7 × 7 窗口中像素的变化。参数指定了窗口半宽 $h = 3$ 以及 MATLAB 内置函数 var。函数被调用是还带有一个 49 × 1 的向量参数，它包含了窗口中已被排成一个列向量的像素值。函数的返回值变成对应的输出像素值。这个操作相当于一个边缘检测器，因为它对均质区域都会得出一个较低的值，而不考虑它们的明度。然而它的计算量很大，函数 var 被调用了 260 000 次。任何 MATLAB 函数，无论是内嵌的还是自己编写的 M 文件，只要接受向量输入，并返回一个标量，都能以这种方式使用。

排序滤波器根据值的大小将窗口中的像素进行了分类，并从分类列表中返回一个指定元素。在 5 × 5 窗口中，所有像素的最大值就是窗口中排列第一的像素：

```
>> mx = irank(lena, 1, 2);
```

其中，参数是排序号和窗口半宽 $h = 2$。一个 5 × 5 窗口中，中间值的排名是第十二：

```
>> med = irank(lena, 12, 2);
```

这个值可用于滤波器去除脉冲型噪声。我们可以通过把脉冲噪声加到一幅图像上来说明：

```
>> spotty = lena;
>> npix = prod(size(lena));
>> spotty(round(rand(5000,1)*(npix-1)+1)) = 0;
>> spotty(round(rand(5000,1)*(npix-1)+1)) = 1.0;
>> idisp(spotty)
```

如图 12.23(a)所示。我们已经设定了 5000 个随机像素点为 0，另外 5000 个随机像素点为最大值。这种类型噪声通常被称为脉冲噪声或椒盐噪声。应用一个 3 × 3 中值滤波器：

```
>> idisp( irank(spotty, 5, 1) )
```

结果如图 12.23(b)所示，图像质量得到了相当大的提高。通过平滑处理也可以达到相同的效果，但是会使图像模糊，中值滤波器则有保留场景中边缘的优点。

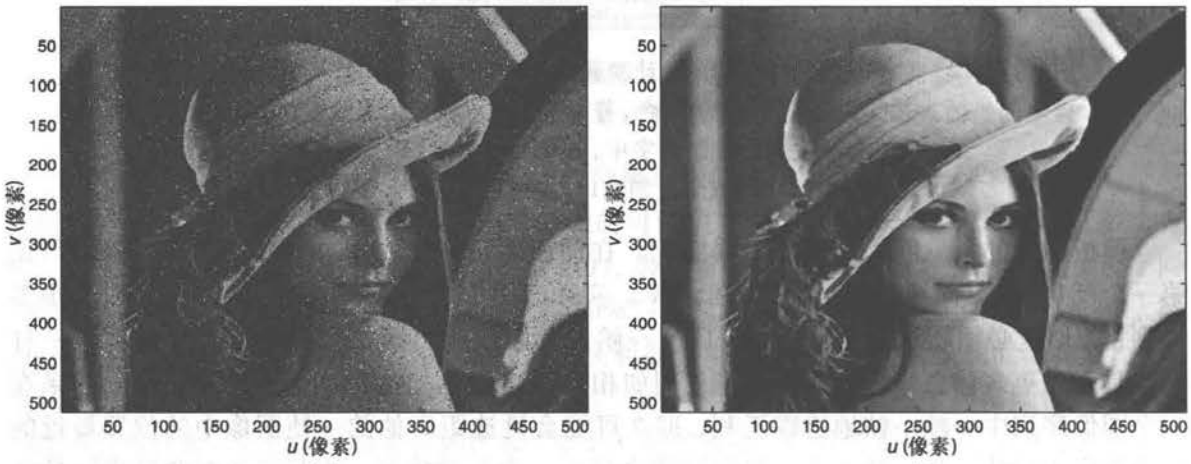


图 12.23 清除脉冲噪声的中值滤波器。(a)损坏的图像；(b)中值滤波后的结果

函数 irank 的第三个参数是一个矩阵而不是标量，这样就能实现一些非常强大的运算。例如：

```
>> M = ones(3,3);
>> M(2,2) = 0
M =
     1     1     1
     1     0     1
     1     1     1
>> mxn = irank(lena, 1, M);
```

它指定了窗口的一个像素子集中排名第一者(最大值)，该子集对应于 M 中的非零元素。在这个例子中，M 指定了除中心像素之外的 8 个邻近像素。结果 mxn 是输入图像中对应的 8 个邻近像素点的最大值。可以使用下面的命令来显示所有这些点，它们的像素值比其邻近点的都大：

```
>> idisp(lena > mxn)
```

然后执行非局部最大值抑制。如果把图像看成是一个曲面，那么这些点对应于局部最大值或峰值。这个掩膜矩阵与下一节将要遇到的一个结构化元素非常相似。

12.5　数学形态学

数学形态学是非线性空间操作的一类，操作示意图如图 12.24 所示(见彩图)。输出矩阵中每一个像素是一个像素子集的函数，该子集是输入图像中围绕对应像素的一个区域中的像素：

$$\mathbf{O}[u,v] = f(\mathbf{I}[u+i, v+j]),\ \forall (i,j) \in \mathcal{S},\ \forall (u,v) \in \mathbf{I} \tag{12.7}$$

其中，$\mathcal{S}$是结构化元素，通常是一个 $w \times w$ 的方形区域，这个区域边长为奇数 $w = 2h+1$，$h \in \mathbb{Z}^+$ 是半宽。结构化元素与先前讨论的互相关核相似，区别在于函数 $f(\cdot)$ 只用于窗口中一个指定的像素子集。选中的像素是那些对应的结构化元素值不为 0 的点，如图 12.24 中的红色像素所示。数学形态学，正如它的名字那样，涉及图像中目标对象的形态或形状。

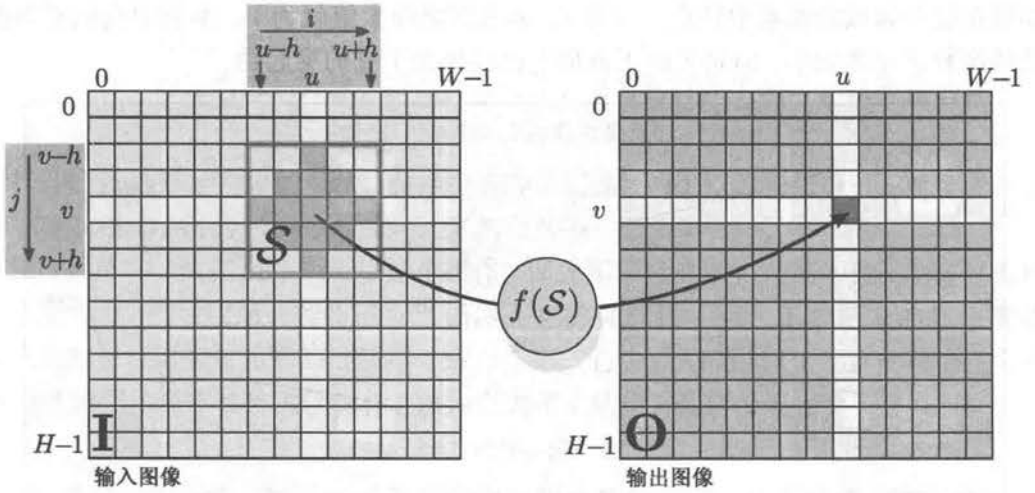


图 12.24　形态学图像处理操作。该操作只针对结构化元素(红色方形轮廓)中的选中像素(红色)

解释这个概念的最好方法是举一个简单的例子，这里采用一个脚本创建的合成二值图像：

```
>> eg_morph1
>> idisp(im)
```

再把它复制并向下排成一列，如图 12.25 中的第一列。结构化元素显示为红色，在每一行的最后。如果考虑最上面一行，其结构化元素是一个方形：

```
>> S = ones(5,5);
```

然后使用最小值运算把它应用到原始图像：

```
>> mn = imorph(im, S, 'min');
```

结果显示在第二列。对于输入图像中的每一个像素，该操作求取 5×5 窗口中所有像素的最小值。如果在这个邻域的像素中有任何一个为 0，那么产生的像素值将会为 0。结果很有戏剧性——两个对象已经完全消失，两个正方形也已经分离并且变得更小。消失的两个对象与结构化元素的形状不一致。这也是形态或形状的连接所在——只有那些包含结构化元素的形状才会在输出图像中呈现。

结构化元素可以定义一个圆、一个环状物、一个五角星、一条长为 20 像素并与水平方向

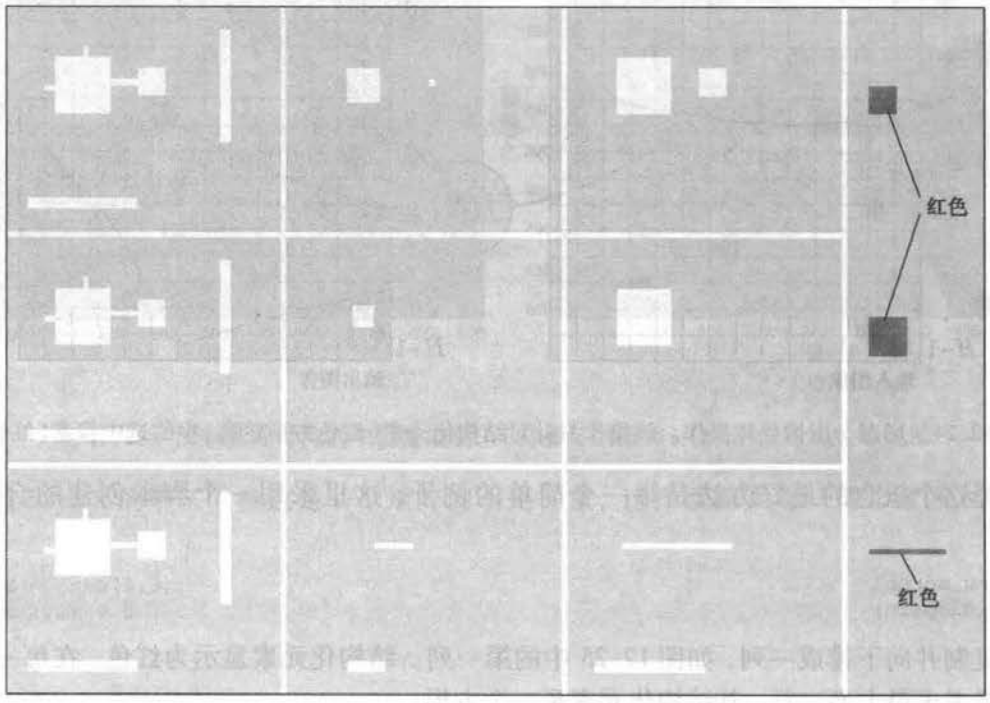
成30°角的线段，或者一个鸭子。这种技术允许建立非常强大的基于形状的过滤器。第二行中显示了针对更大方形结构化元素的结果，更大的结构化元素导致较小的正方形完全消失，较大的正方形也进一步减少。第三行的结构化元素是一条宽为 14 像素的水平直线，其结果中仅保留了一条长的水平线。

我们刚才进行的操作通常被称为腐蚀，因为大的对象被腐蚀掉或者变得更小——本例中 5×5 的结构化元素使得图像中每一个形状的周边都被切掉了两个像素。原来是 5×5 的小方正形现在只有 1×1。如果重复这个操作，则小正方形最终将会消失，并且大的正方形也将会更进一步减小[①]。

上述操作的逆操作则是膨胀，它使对象变大。在图 12.25 中，我们把膨胀应用到第二列中的图像上：

```
>> mx = imorph(mn, S, 'max');
```

结果显示在第三列。对于输入图像的每一个元素，该操作求在 5×5 窗口中所有像素的最大值。如果在这个邻域的像素中任意一点为 1，那么结果像素值就为 1。本例中我们看到两个正方形已经恢复到原来大小，但是大的正方形上已经失去了它的突起物。



12.25 数学形态学示例。像素值为0(灰色)或1(白色)。每一列对应于使用结构化元素的处理结果，最后一列的红色(本书此处显示为深灰色)为结构化元素。第一列是原始图像，第二列是被结构化元素腐蚀的图像，第三列是第二列被结构化元素膨胀的图像

形态学操作经常被写成算子的形式。腐蚀是

$$\boldsymbol{O} = \boldsymbol{I} \ominus S$$

其中，在方程(12.7)中 $f(\cdot) = \min(\cdot)$。膨胀是

$$\boldsymbol{O} = \boldsymbol{I} \oplus S$$

① 结构化元素的半宽。

其中，在方程(12.7)中$f(\cdot) = \max(\cdot)$。

腐蚀和膨胀是依下式相关的：

$$A \oplus B = \overline{\overline{A} \ominus B}$$

其中上横线表示像素值的逻辑补。本质上该式可以被描述为：腐蚀白色像素等同于膨胀黑色像素，反之亦然。对于下面的形态学运算：

$$(A \oplus B) \oplus C = A \oplus (B \oplus C)$$

$$(A \ominus B) \ominus C = A \ominus (B \oplus C)$$

其含义是：连续地使用结构化元素进行腐蚀或膨胀等同于只使用一个较大的结构化元素，但是前者的计算量更小[①]。这些运算也被分别称为闵可夫斯基(Minkowski)减法和加法。它们的速记函数是

```
>> out = ierode(im, S);
>> out = idilate(im, S);
```

可以用来代替低级函数 imorph。

按顺序操作，即先腐蚀后膨胀，被称为开操作，因为它打开了缺口。该操作的算子记为

$$I \circ S = (I \ominus S) \oplus S$$

开操作不仅选择了特定的形状，而且也清理了图像：两个正方形被分开，而且大正方形上的突起物也被清除，因为它们与结构化元素的形状不一致。

在图 12.26 中，我们逆序执行了这个操作，即先膨胀后腐蚀。在第一行，没有丢失任何形状，它们先变大，然后收缩，大的正方形仍然有它的突起物。但大正方形上的那个洞被填满了，这是因为它与结构化元素的形状不一致。在第二行中，大的结构化元素使两个正方形结合在一起。这种数学操作也称为闭操作，因为它关闭了缺口，其算子记为

$$I \bullet S = (I \oplus S) \ominus S$$

注意，在最后一行中，保留了两条与图像边缘相接的线段，这是处理边缘像素时的默认行为。

背景材料：处理边缘像素。输入图像边缘附近的互相关窗口问题已在 12.4 节进行了讨论。对于形态学空间操作，也有相同的问题存在，工具箱函数 imorph、irank 和 iwindow 也支持选项'valid'，就像函数 iconv 那样。其他选项可以使得返回的图像与输入图像大小相同：

- 'replicate'(默认)边界像素被复制，即使用的是最近的边界像素值。
- 'none'超出边界的像素不包含在结构化元素指定的像素集中。
- 'wrap'假定图像要变形，从左到右，从上到下。

开操作和闭操作是分别通过工具箱函数 iopen 和 iclose 执行的。与腐蚀和膨胀不同，重复使用开操作或闭操作是无效的，因为这些操作本身就是幂等的：

$$(I \circ S) \circ S = I \circ S$$

$$(I \bullet S) \bullet S = I \bullet S$$

---

① 例如，连续使用两次的 3×3 方形结构化元素等同于使用一次 5×5 的方形结构化元素。前者包含 $2 \times (3 \times 3 \times N^2) = 18N^2$ 步运算，而后者包含 $5 \times 5 \times N^2 = 25N^2$ 步运算。

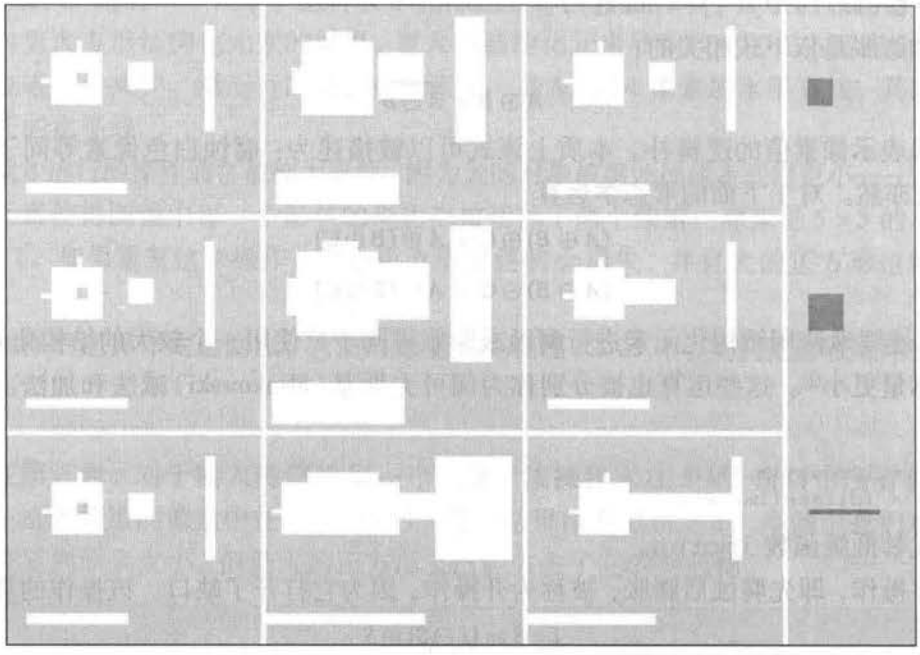


图 12.26 数学形态学示例。像素值为 0(灰色)或 1(白色)。每一列对应于使用结构化元素的处理结果,最后一列的红色(本书图中显示为深灰色)为相应的结构化元素。第一列是原始图像,第二列是被结构化元素腐蚀的图像,第三列是第二列被结构化元素膨胀的图像

12.5.1 去除噪声

形态学开操作的常见应用是去除图像中的噪声。读取一个图像:

```
>> objects = iread('segmentation.png');
```

如图 12.27(a)所示,这个噪声二值图像是一个相当破旧的目标对象经分割操作后的输出结果。我们希望去除不属于对象的黑色像素,并且填满 4 个黑色矩形中的孔。

我们选择一个对称的半径为 3 的圆形结构化元素:

```
>> S = kcircle(3)
S =
     0     0     0     1     0     0     0
     0     1     1     1     1     1     0
     0     1     1     1     1     1     0
     1     1     1     1     1     1     1
     0     1     1     1     1     1     0
     0     1     1     1     1     1     0
     0     0     0     1     0     0     0
```

应用闭操作填充对象中的孔:

```
>> closed = iclose(objects, S);
```

结果如图 12.27(b)所示。孔已经被填满,但是噪声像素也已经成长为小圆圈,有些已经结块。我们再通过开操作将其消除:

```
>> clean = iopen(closed, S);
```

结果如图 12.27(c)所示,是一个相当干净的图像。如果将上述顺序反过来,即先开操作再闭操作:

```
>> opened = iopen(objects, S);
>> closed = iclose(opened, S);
```

结果如图12.27(d)所示，图像更加破败。尽管开操作已经去除了孤立的噪声像素，但是它也清除了目标对象中的大组块，且无法修复。

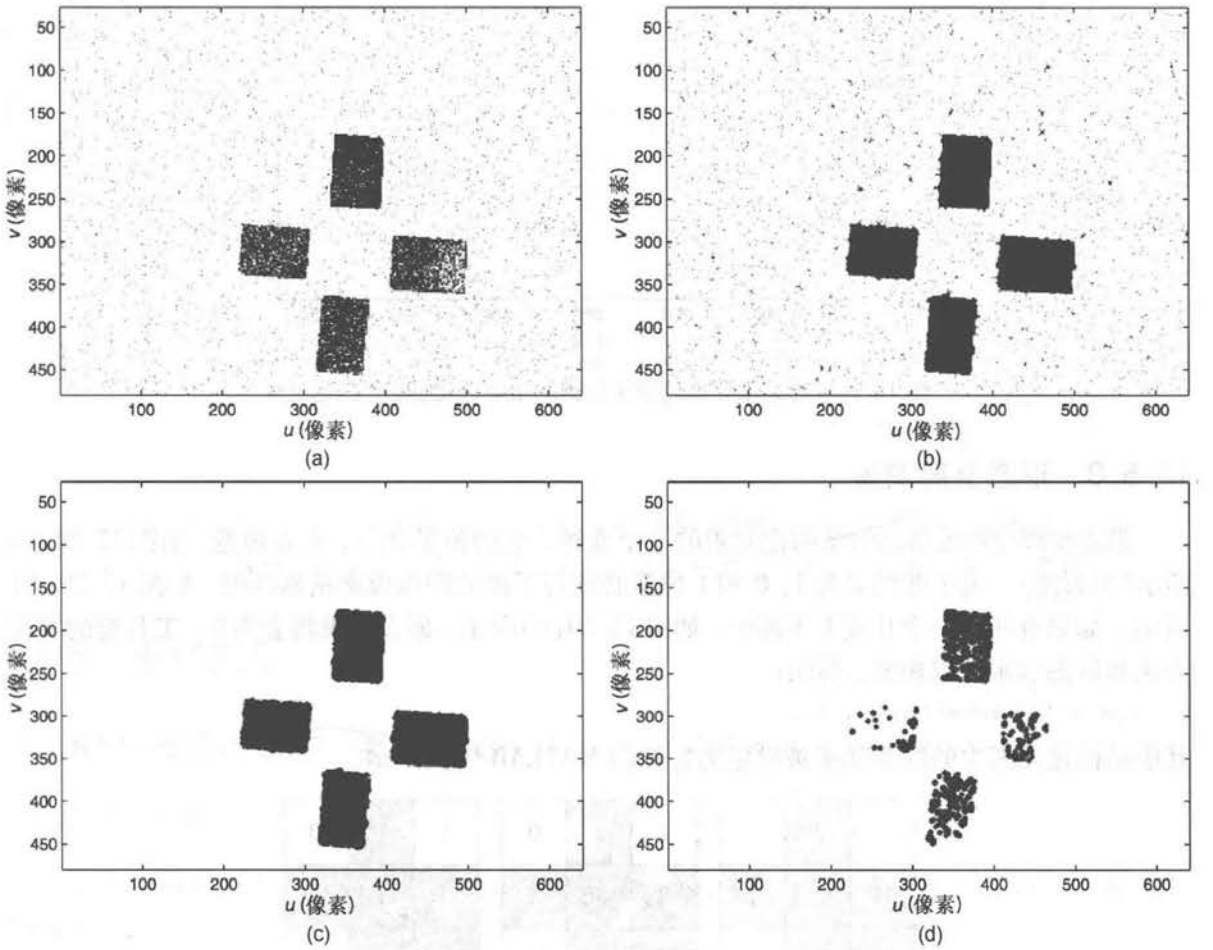


图12.27 形态学善后处理。(a)原始图像；(b)开操作处理后的图像；(c)开操作后再闭操作的结果；(d)闭操作后再开操作的结果。结构化元素是半径为3的圆。颜色图是反色的，置1的像素显示为黑色

12.5.2 边界检测

我们也可以使用形态学操作检测目标的边缘。接着上面的例子，使用图12.27(c)所示的图像clean，我们用一个圆形结构化元素计算这个图像的腐蚀：

```
>> eroded = imorph(clean, kcircle(1), 'min');
```

图像中的目标都变得稍微小了一点儿，这是由于结构化元素使每一个目标的外边被削掉了一个像素。从原始图像中减去腐蚀后的图像：

```
>> idisp(clean-eroded)
```

结果是每个目标对象的周边剩下一层像素点，如图12.28所示。

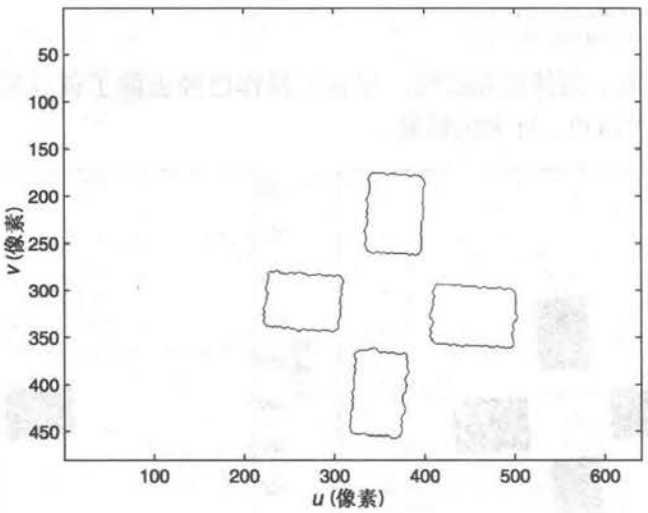


图 12.28 形态学处理的边缘检测。结果是反色的，白色是 0

12.5.3 形态交离变换

形态交离变换是形态学结构化元素的一个变种。它的值是 0，1，或者随意，如图 12.29(a)所示(见彩图)。为了使结果为 1，0 和 1 像素必须与下面的图像像素精确匹配，如图 12.29(b)所示。如果有任何一个 0 或 1 不匹配，如图 12.29(c)所示，那么结果将会为 0。工具箱的实现方法和形态学函数很相似，例如：

```
out = hitormiss(image, S);
```

其中结构化元素中的随意元素被设定为特殊的 MATLAB® 值 NaN。

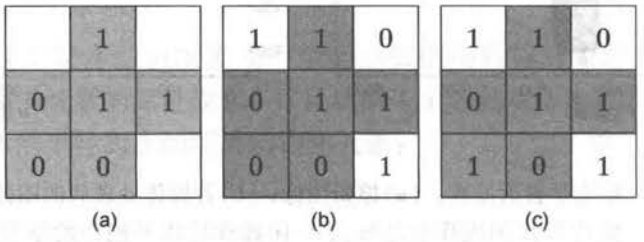


图 12.29 击中与否运算。(a)结构元素值可取 0,1 或者显示为白色；(b)击中样例；(c)未击中样例，红色像素与结构元素不一致

形态交离变换将反复地与不同结构化元素一起使用来执行操作，例如骨架化和线性特征检测。目标的骨架可以通过下面的方法计算：

```
>> skeleton = ithin(clean);
```

如图 12.30(a)所示。这些线条都是单像素宽，且是广义泰森多边形图的边缘——它们根据最接近的形状边界来描绘像素集。然后可以找到骨架的终止点：

```
>> ends = iendpoint(skeleton);
```

也可以找到三叉点，即三条线的交点：

```
>> joins = itriplepoint(skeleton);
```

分别如图 12.30(b)和(c)所示。

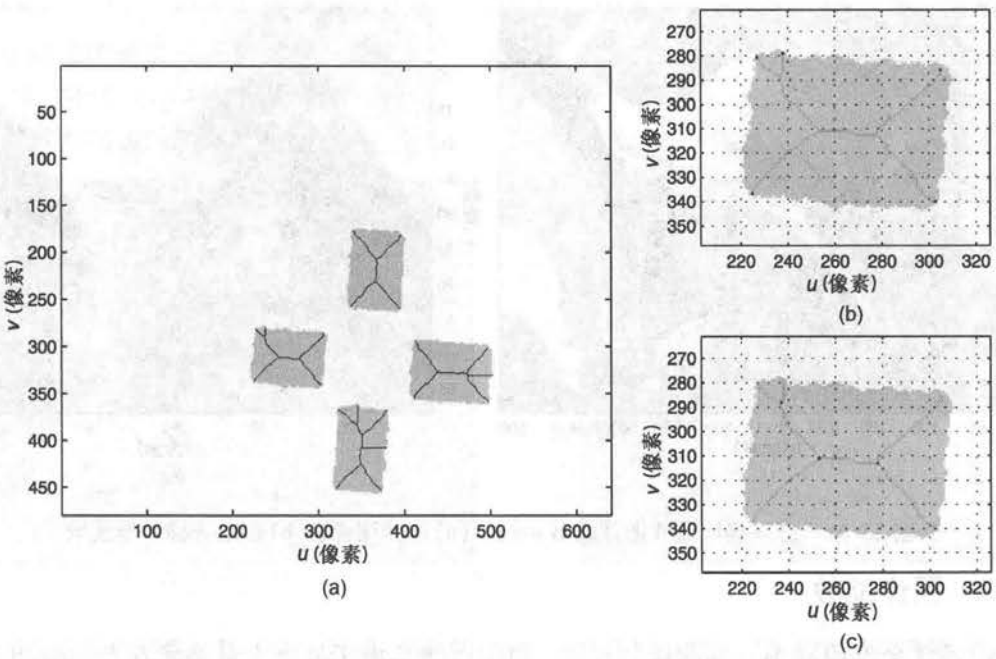


图 12.30 形态交离变换操作。(a)骨架化;(b)终止点检测;(c)三叉点检测。图像都是反色显示,原始二值图像以灰色的叠加上去。终止点和三叉点显示为黑色像素

12.6 形状变化

我们将要讨论的最后一类图像处理方法是改变图像形状和大小的操作。

12.6.1 裁剪

最简单的形状变化是从图像中选出一个矩形区域,这是最常见的裁剪操作。考虑如下图像:

```
>> lena = iread('lena.pgm');
```

如图 12.31(a)所示,我们将通过在这张图像上点击或拖曳一个选择框,交互式地指定一片感兴趣的区域(ROI):

```
>> [eyes,roi] = iroi(lena);
>> idisp(eyes)
```

在本例中,我们选择眼睛部位,如图 12.3(b)所示。被选区域的角点可以被返回,这里是

```
>> roi
roi =
   239   359
   237   294
```

其中,两列分别对应被选区域左上角和右下角的坐标 (u,v),两行分别对应 u 轴和 v 轴的值。也可以使用函数来非交互式地指定一个 ROI:

```
>> mouth = iroi(lena, [253, 330; 332, 370]);
```

这里选择了嘴巴。

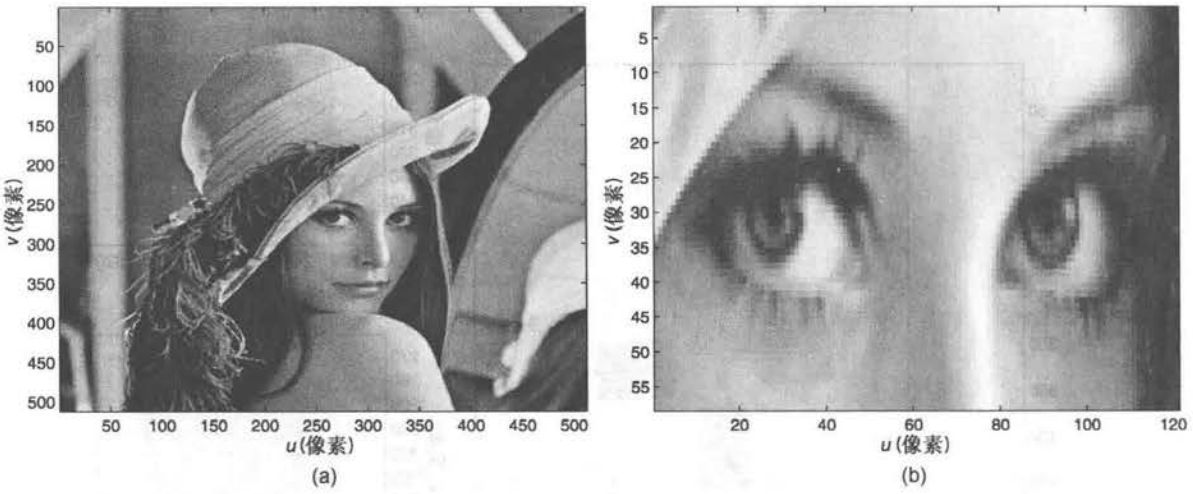


图 12.31 感兴趣区域或图像裁剪示例。(a)原始图像;(b)选定的感兴趣区域

12.6.2 图像缩放

我们经常会希望减小一幅图像的尺寸,原因可能是由于处理大量像素花费的时间长,或者需要更多的内存。我们用一个分辨率高的图像来说明这个问题:

```
>> roof = iread('roof.jpg', 'grey');
>> about(roof)
roof [uint8] : 1668x2009 (3351012 bytes)
```

如图 12.32(a)所示。减小图像尺寸最简单的方法是亚抽样或抽取,即在 u 方向和 v 方向上分别选出每隔 m 个单位的像素,其中 $m \in \mathbb{Z}^+$ 是亚抽样因子。例如,$m = 2$,则 $N \times N$ 图像会变成一幅 $N/2 \times N/2$ 图像,像素数量是原始图像的 1/4。

对于这个示例,我们将要在每个方向上采用因子 7 来减小其尺寸:

```
>> smaller = roof(1:7:end,1:7:end);
```

它使用标准的 MATLAB® 索引语法来选择每隔 7 的行和列。结果如图 12.32(b)所示,我们观察到屋顶上存在原始图像中没有的显著曲线,这些都是抽样处理的产物。亚抽样降低了图像的空间采样率,它将导致纹理或锐利边缘产生的高频成分的空间混叠。为了确保满足香农 - 奈奎斯特采样定理,在对图像亚抽样之前,必须使用一种抗锯齿低通空间滤波器来减小图像空间带宽。这是图像模糊化的另一个用途,而高斯核是实现这一目的的一种合适的低通过滤器。平滑处理和亚抽样的联合操作在工具箱中通过以下函数实现[①]:

```
>> smaller =  idecimate(roof, 7);
```

$m = 7$ 时的结果如图 12.32(c)所示。我们注意到曲线产物不再出现。

亚抽样的逆操作是像素复制,其中每一个输入像素要被当作一个 $m \times m$ 块复制到输出图像中:

```
>> bigger = ireplicate( smaller, 7 );
```

如图 12.32(d)所示,其中沿着屋顶边缘和地平线还是呈现出一点点的斑驳。在抽取阶段去除

① 任何可行的低通过滤器都有一个高于它的截止频率的响应极限。事实上,挑选出来的截止频率要比理论截止频率低很多,以便过滤器对奈奎斯特频率的响应能足够小。作为经验法则,对于 $m = 2$ 的亚抽样,会使用 $s = 1$ 的高斯函数。

了图像 98% 的像素，但把图像恢复到原始大小时却没有添加任何新信息[①]。然而我们可以通过平滑处理使图像看起来更舒服一些：

```
>> smoother = ismooth( bigger, 4);
```

它将会衰减方块边缘的高频信息。

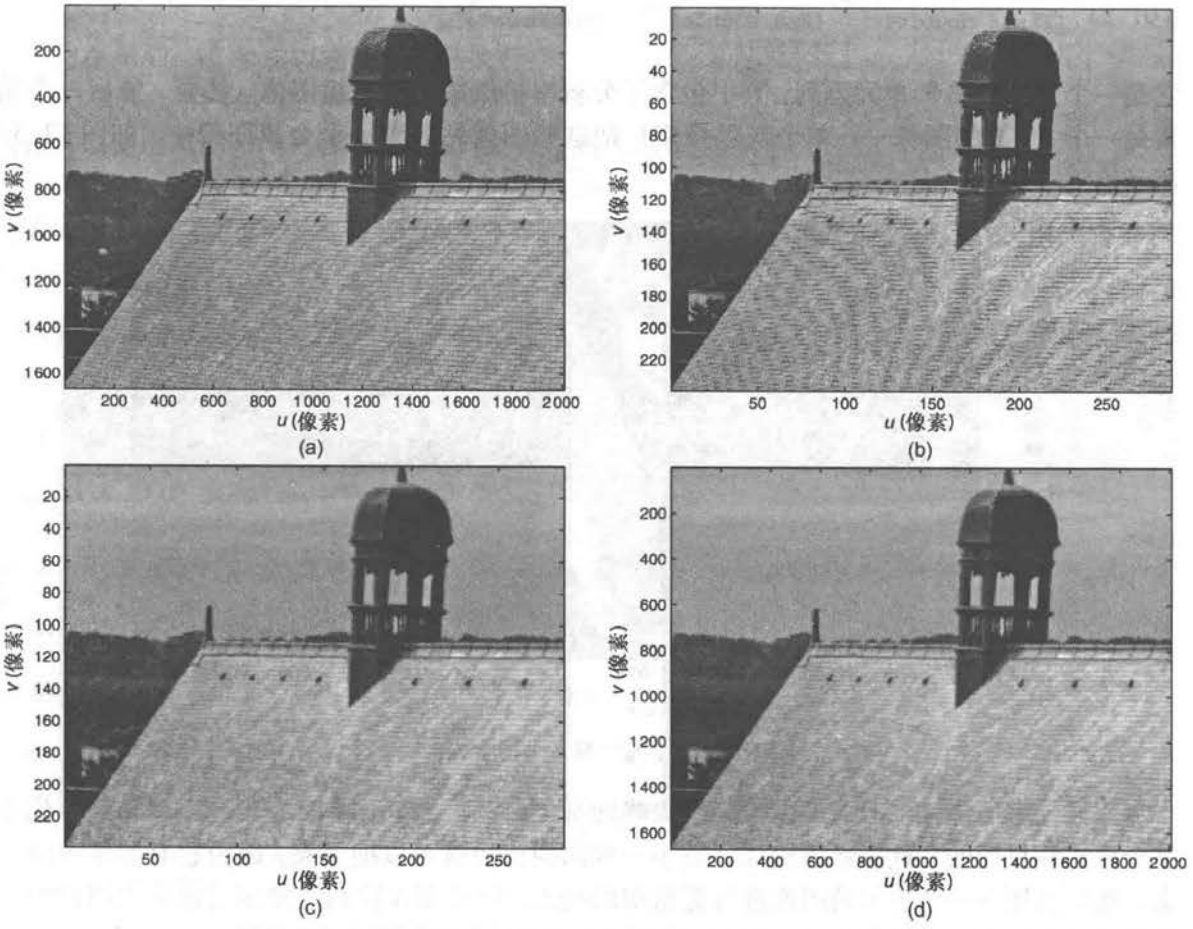


图 12.32　图像放缩示例。(a)原始图像；(b) $m = 7$ 的亚抽样，注意轴标度；(c)平滑处理后的 $m = 7$ 亚抽样；(d)通过像素复制把图像(c)恢复成原始大小

也可以使用工具箱函数 iscale 来执行相同的功能，该函数通过任意因子 $m \in \mathbb{R}^+$ 缩放图像。例如：

```
>> smaller = iscale(lena, 0.1);
>> bigger = iscale(smaller, 10);
```

其中第二个参数是放缩因子，如果 $m < 1$，则图像将会缩小；如果 $m > 1$[②]，则图像会增大。

12.6.3　图像金字塔

计算机视觉中一个重要的概念是尺度空间(我们将在下一章中再对它进行讨论)。工具箱函数 ipyramid 返回输入图像的塔式分解：

① 有点类似于相机的数码缩放功能。

② 图像应该首先使用'smooth'选项设定高斯核的宽度来进行平滑处理。

```
>> p = ipyramid(lena)
p =
  Columns 1 through 5
    [512x512 double]    [256x256 double]    [128x128 double]
    [64x64 double]    [32x32 double]

  Columns 6 through 10
    [16x16 double]    [8x8 double]    [4x4 double]
    [2x2 double]    [20.8959]
```

它是一个 MATLAB® 单元阵列，其中包含了分辨率持续减小的一组图像。注意，最后一个元素是一个 1×1 的图像——单个灰色像素！把这些图像粘贴到一个合成图像中，如图 12.33 所示。

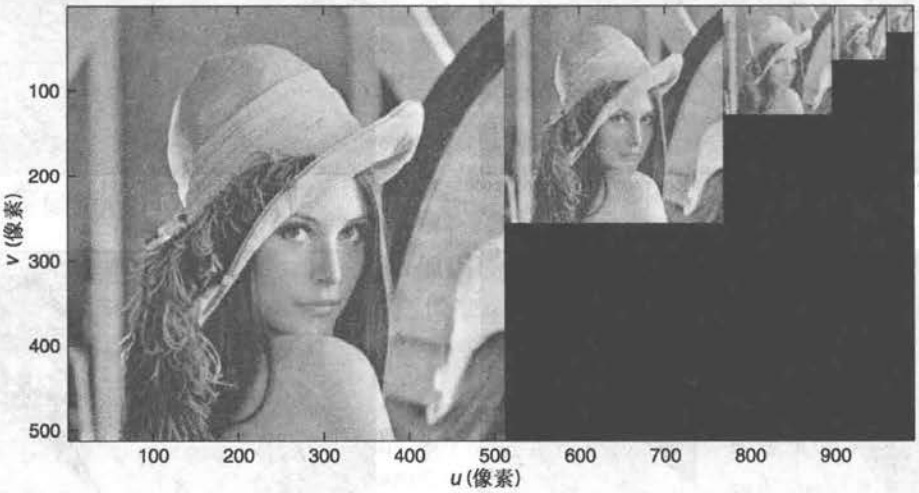


图 12.33 图像金字塔。在一组图像中，每一幅的分辨率(用边长表示)是其左侧图像的一半

图像金字塔是许多所谓的由粗到精策略的基础。我们考虑这样一个问题，即寻找能代表某个感兴趣目标的一种像素值模式。最小一幅图像很快就可以搜寻完，因为它有非常少的像素。然后再用下一个更大的图像进行更精细的搜索，但是现在我们已经知道这个大图像中需要搜寻哪个区域。一直重复这个过程，直到在最高分辨率的图像中找到目标。

12.6.4 图像变形

图像变形是一种像素坐标的变换，而不是像素值。图像变形被用来放大或缩小图像，旋转图像或者使图像产生任意形状的改变。在新的视图中，像素坐标(u',v')用原始视图中像素坐标的函数表示：

$$u' = f_u(u,v),\ v' = f_v(u,v) \tag{12.8}$$

考虑一个简单示例，其中图像尺寸在两个方向上以因子 4 缩小，然后平移以便其原点(即图像左上角)坐标为(100,200)。以上描述可以精确地表达为

$$u' = u/4 + 100,\ v' = v/4 + 200 \tag{12.9}$$

首先，我们建立一对坐标矩阵[①]，它们覆盖了输入图像的全域：

```
>> [Ui,Vi] = imeshgrid(lena);
```

① 坐标矩阵需满足 U(u,v) = u 和 V(u,v) = v，且它们是 MATLAB® 中的常见结构，参见有关 meshgrid 的文献。

再建立另一对覆盖输出图像全域的坐标矩阵，这里我们任意选择为400×400：

```
>> [Uo,Vo] = imeshgrid(400, 400);
```

现在，对于输出图像中的每一个像素，输入图像中对应的坐标通过函数f_u，f_v的逆运算给出。对于这个例子，方程(12.9)的逆运算是

$$u = 4(u' - 100),\ v = 4(v' - 200) \tag{12.10}$$

它在MATLAB®中以矩阵形式被执行：

```
>> U = 4*(Uo-100); V = 4*(Vo-200);
```

现在可以使用MATLAB®函数interp2来使输入图像变形：

```
>> lena_small = interp2(Ui, Vi, idouble(lena), U, V);
```

结果如图12.34(a)所示。注意。函数interp2需要在浮点型图像上进行操作，因此我们使用idouble来转换输入图像。

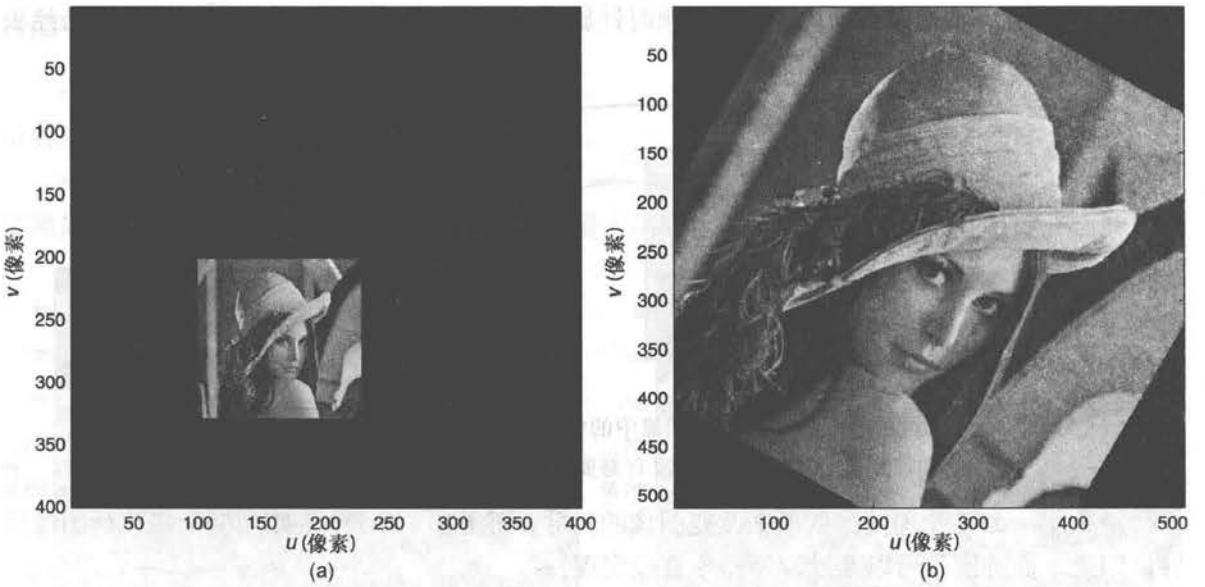


图12.34 变形的图像。(a)缩放和平移；(b)绕中心旋转30°角

图像下还是发生了一些微妙变化。首先，当(u',v')是整数坐标时，输入图像坐标(u,v)可能不再是整数。这时像素值[①]必须从输入图像中的相邻像素点经过插值[②]得到。其次，输出图像中不是所有的像素都在输入图像中有对应的像素，如图12.35所示。对于我们来说，幸运的是函数interp2控制着所有的问题，输入图像中不存在的像素在输出图像中呈现为黑色。在出现极端畸变的映射时，很多相邻的输出像素可能映射到同一个输入像素，这就导致输出图像中的像素化或结块。

现在让我们尝试一下更大胆的事情，把图像旋转30°并放入与输入图像相同大小的输出图像中：

```
>> [Uo,Vo] = imeshgrid(lena);
```

① 不同的插值模式可以通过interp2后面的一个参数进行选择，但是默认选项是双线性插值。在坐标$(u+\delta_u, v+\delta_v)$处的像素是像素(u,v)、$(u+1,v)$、$(u,v+1)$和$(u+1,v+1)$的函数，其中$u,v \in \mathbb{Z}$，δ_u、$\delta_v \in [0,1)$。

② 插值函数充当了一个弱的抗锯齿过滤器，但是对于很大尺度上的缩减，图像应该首先使用高斯核进行平滑处理。

我们希望图像绕它的中心旋转，但是由于输出图像的原点是左上角，所以必须先把原点变换到中心，然后旋转，再把原点变换回左上角。因此这个变形方程是

$$\begin{pmatrix} u' \\ v' \end{pmatrix} = \underbrace{\begin{pmatrix} \cos\pi/6 & -\sin\pi/6 \\ \sin\pi/6 & \cos\pi/6 \end{pmatrix}}_{R(\pi/6)} \begin{pmatrix} u - u_c \\ v - v_c \end{pmatrix} + \begin{pmatrix} u_c \\ v_c \end{pmatrix} \tag{12.11}$$

其中，(u_c, v_c) 是图像中心的坐标，$R\left(\frac{\pi}{6}\right)$ 是 $SE(2)$ 中的一个旋转矩阵。该方程在执行运算时可以重新按倒序排列如下：

```
>> R = se2(0, 0, pi/6); uc = 256; vc = 256;
>> U = R(1,1)*(Uo-uc) + R(2,1)*(Vo-vc) + uc;
>> V = R(1,2)*(Uo-uc) + R(2,2)*(Vo-vc) + vc;
>> lena_rotated = interp2(Ui, Vi, idouble(lena), U, V);
```

结果如图 12.34(b)所示。注意旋转方向：之所以如此定义 x 轴和 y 轴(分别平行于 u 轴和 v 轴)，是为了让 z 轴指向页面内，从而使顺时针旋转为正。还要注意原始图像的角落也已经丢失，它们落到了输出图像的边界之外。

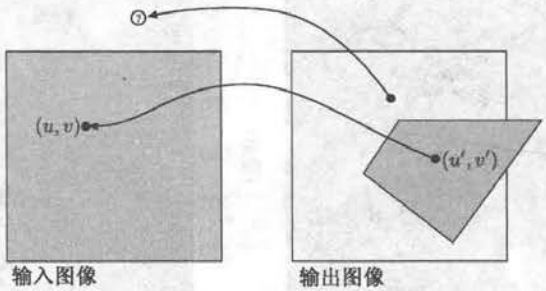


图 12.35 图像变形的坐标表示。输出图像中的像素 (u', v') 源于输入图像中 (u, v) 处的像素，如图中箭头所示。变形后图像没有必要是多边形的，也没有必要完全在第二幅图像中

函数 iscale 使用图像变形来改变图像的尺寸，而函数 irotate 则用变形来执行图像旋转。因此上面的例子可以通过以下命令直接实现：

```
>> lena_rotated = irotate(lena, pi/6);
```

最后，再回头看看 11.1.1 节中的透镜变形示例。来自相机的扭曲图像是输入图像，并将要通过图像变形来去除畸变。我们很幸运，因为畸变模型方程(11.13)已经是逆运算的形式。回想一下：

$$u^d = u + \delta_u$$
$$v^d = v + \delta_v$$

其中，δ_u 和 δ_v 是 (u, v) 的函数。

首先，我们加载畸变图像，并建立畸变和未畸变图像的坐标矩阵：

```
>> distorted = iread('Image18.tif', 'double');
>> [Ui,Vi] = imeshgrid(distorted);
>> Uo = Ui; Vo = Vi;
```

然后加载相机标定的结果：

```
>> load  Bouguet
```

这里为了简便，我们直接从标定工具箱变量 cc、fc 和 kc 中取出需要的参数：

```
>> k = kc([1 2 5]); p = kc([3 4]);
>> u0 = cc(1); v0 = cc(2); fpix_u = fc(1); fpix_v = fc(2);
```

它们分别是径向和切向畸变向量、主点和以像素为单位的焦距长度。接着我们把像素坐标转变成归一化图像坐标[①]：

```
>> u = (Uo-u0) / fpix_u;
>> v = (Vo-v0) / fpix_v;
```

从相机主点到像素的径向距离是

```
>> r = sqrt( u.^2 + v.^2 );
```

由于畸变产生的像素坐标误差是

```
>> delta_u = u .* (k(1)*r.^2 + k(2)*r.^4 + k(3)*r.^6) + ...
   2*p(1)*u.*v + p(2)*(r.^2 + 2*u.^2);
>> delta_v = v .* (k(1)*r.^2 + k(2)*r.^4 + k(3)*r.^6) + ...
   p(1)*(r.^2 + 2*v.^2) + 2*p(2)*u.*v;
```

以米为单位的畸变像素坐标是

```
>> ud = u + delta_u;  vd = v + delta_v;
```

再把畸变像素坐标变回像素坐标：

```
>> U = ud * fpix_u + u0;
>> V = vd * fpix_v + v0;
```

最后应用我们的图像变形函数：

```
>> undistorted = interp2(Ui, Vi, distorted, U, V);
```

结果如图 12.36 所示。变化虽然不大，但在 r 最大的图像边缘和角落处却十分明显。

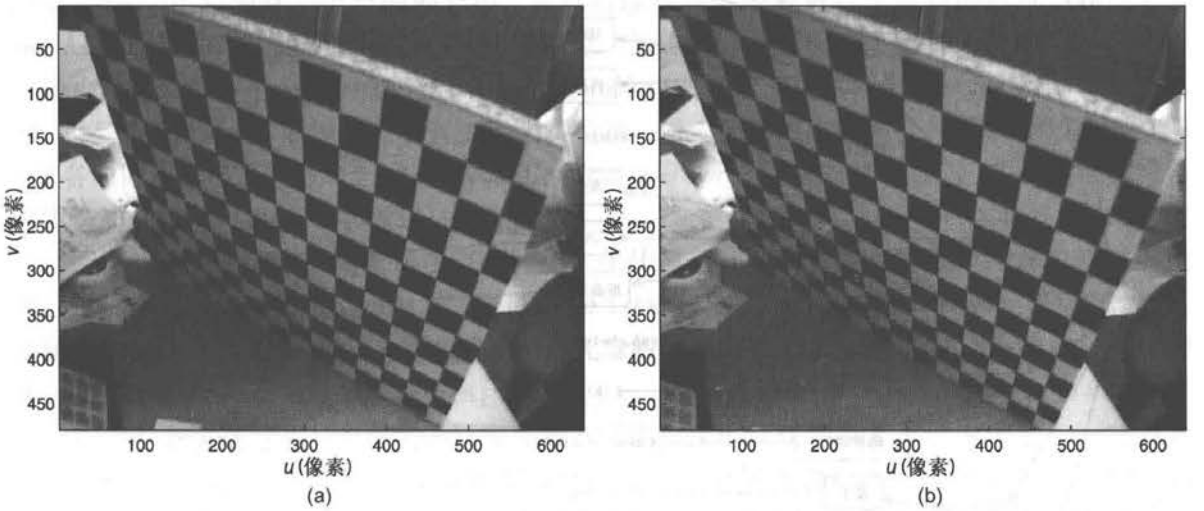


图 12.36 畸变到未畸变的图像。(a)原始畸变图像；(b)矫正后的图像。注意棋盘的上边缘已经成为直线(图例来自布盖相机标定工具箱中的图片18)

12.7 本章总结

在本章中，我们学习了怎样从图像文件、电影文件、视频相机和网络等各种各样的输入源中获取图像，并把它们加载到 MATLAB® 的工作区中。然后讨论了大量的图像处理操作和它们的分类法，如图 12.37 所示。用于单一图像的操作包括：一元算术操作、类型转换、各种颜色变换

① 单位为米，相对于相机主点。

和灰度级延伸；非线性操作有直方图归一化和伽玛编码解码；还有逻辑操作，例如阈值转换法。我们也讨论了用于多幅图像的操作，例如绿色筛分、背景估计和移动物体的检测等。

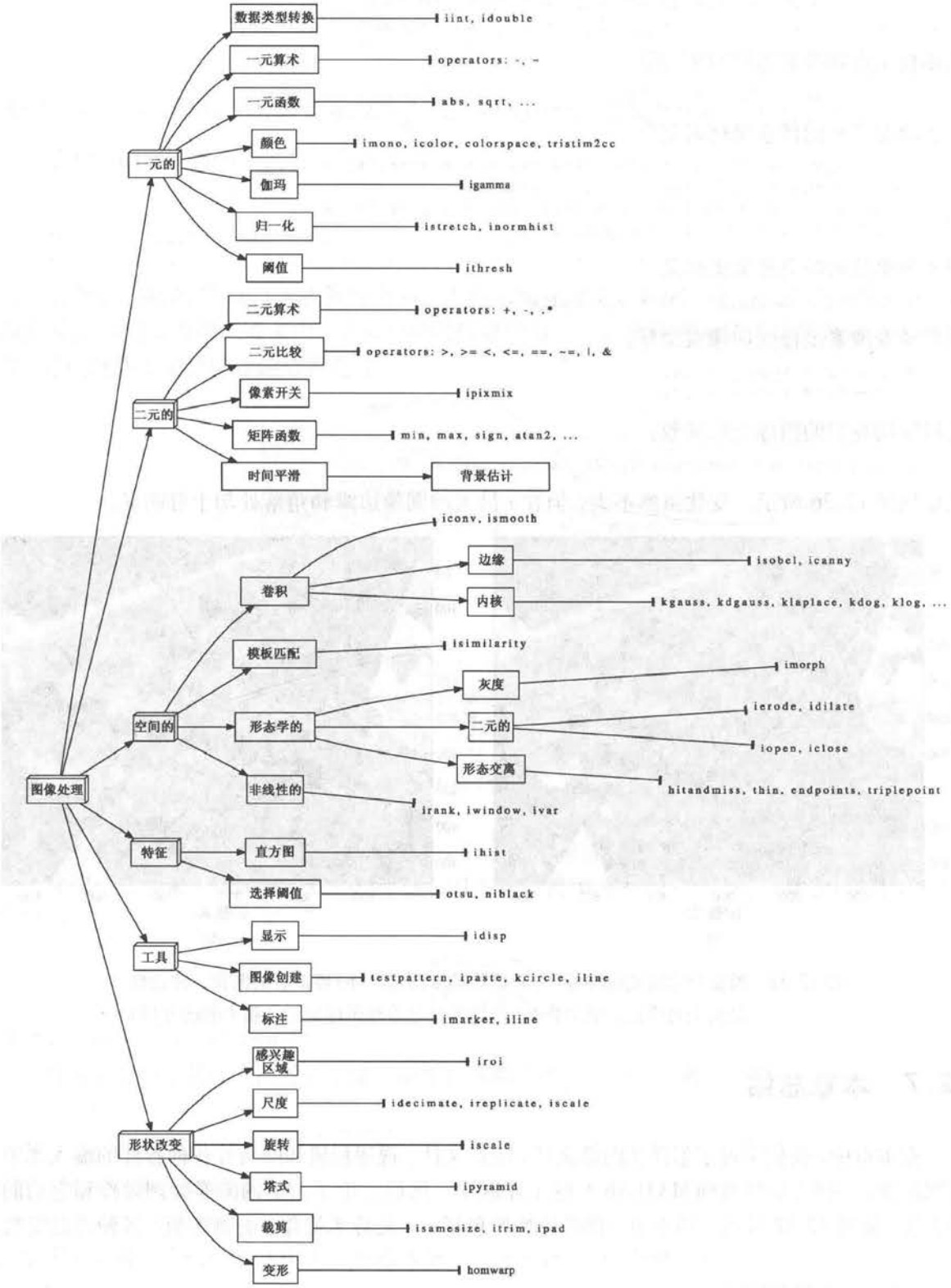


图 12.37 本章讨论的图像处理算法分类

最大也是种类最多的是空间操作。我们讨论了可以用于平滑图像和检测边缘的互相关操作。平滑处理对于减小边缘检测中图像噪声的影响很有帮助，也可用于在亚抽样之前的图像低通滤波器。非线性操作可以用于执行模块匹配并计算整个窗口的排秩统计。排秩统计的一个特殊用途是中值滤波器，可用来减少某些类型的图像噪声。一门相关技术是数学形态学，它可以基于形状来过滤图像、整修二值图像并执行骨架化。

最后讨论了形状改变操作，例如选择兴趣区、尺度变换、混叠现象引起的问题，以及可以用于旋转或图像矫正的广义图像变形。所有这些图像处理技术都是下一章讨论的图像特征提取算法的基础。

扩展阅读

图像处理是一个很大的研究领域，本章介绍了很多从机器人学角度来看最有用的技术。在教科书[Szeliski, 2011]和[Gonzalez and Woods, 2008]中，对于图像处理技术进行了全面的讨论。它们详述了本章中介绍的方法，并进行了扩展，包含了额外的议题，如灰度等级形态学、图像修复、小波频域方法和图像压缩。关于计算机视觉的网上信息可以通过 CVonline 在网站 http://homepages. inf. ed. ac. uk/rbf/CVonline 获取，本章的内容包含在“Image Transformations and Filters”这一部分中。

边缘检测是图像处理的一个子集，但关于它的文献却特别多。在[Forsyth and Ponce, 2002]中对边缘检测有一个全面的介绍，并对边缘检测的限制进行了有益的讨论。在[Nixon and Aguado, 2008]中也研究了边缘检测的相位一致方法，并且比较了多种边缘检测器。用于边缘检测的索伯核是由埃尔文·索伯(Irwin Sobel)和杰里米·费尔德曼(Jerome Feldman)提出的，并呈现在 1968 年斯坦福大学人工智能实验室一份未公开的出版物《一个用于图像处理的 3×3 各向同性梯度算子》(*A* 3×3 *Isotropic Gradient Operator for Image Processing*)中。坎尼边缘检测器最初是在[Canny 1983, 1987]中描述的。

图像相似性的无参数测度是在 20 世纪 90 年代流行起来的，期间出现了几篇关键性的论文，如[Zabih and Woodfill, 1994]、[Banks and Corke, 2001]和[Bhat and Nayar, 2002]。有几个研发小组还使用诸如 FPGAs 的高速逻辑把无参数方法应用到实时图像处理系统中[Corke et al., 1999]；[Woodfill and Von Herzen, 1997]。

数学形态学是另外一个很大的研究领域，而我们这里仅仅触及了其研究表层。一些重要的技术，如灰度形态学和分水岭，我们都没有涉及。这个领域的先锋人物是吉恩·塞拉(Jean Serra)，对于想进一步研究的读者，可以参考他 1983 年的著作[Serra, 1983]，或者在网站 http://cmm. ensmp. fr/index_eng. html 上观看数学形态学中心的网上教程。另一本内容较全面的著作是[Soille, 200]，而[Dougherty and Latufo, 2003]则是一本手把手传授相关知识的书。冈萨雷斯(Gonzalez)和伍兹(Woods)也对灰度形态学做过有益的探讨。

本书中介绍的计算视觉的方法通常被称为自底向上的处理。本章涉及的是针对像素操作的低层级视觉技术。下一章将介绍高层级的视觉技术，其中像素集将被分组，然后被描述以代表场景中的目标对象。

图像数据的来源

现在网上有成千上万的网络相机，同时也有很多网站在收集这些网络相机，并提供了它们按位置分类的列表。在这些列表页上一些内容相当可疑，所以要多加小心。更多的这类网站

不会让你直接与网络相机相连，所以通常相机的 URL 是得不到的。类似 Opentopia 这样的网站 http://www.opentopia.com 确实列举了大量的网络相机，而且相机的 URL 也被列在图像下面标记有“Website:”的直线上。对于 AxisWebCamera 类，URL 的根部(在第一个单一斜线之前)是必需的。

用于 12.3 节中的动作序列视频，可通过在 http://homepages.inf.ed.ac.uk/rbf/CAVIARDAT 中的 CAVIAR 项目获得，在[Fisher, 2004]中也有所描述。

很多图像可在工具箱的文件夹 images 中寻得。

MATLAB® 软件工具

图像处理工具箱(IPT)可以从 MATLAB 的发行商 Mathwork Inc. 公司购买。与[Gonzalez and Woods, 2008]相伴的一本书是 2009 年出版的基于 MATLAB 的[Gonzalez et al., 2009]，该书使用 MATLAB® 提供了有关图像处理的详细内容，还包括很多 IPT 中没有的函数。这些函数有 P 码格式(无源码和帮助)的免费版，也有付费的 M 文件。

本书中描述的机器视觉工具箱(MVTB)是开源的。MVTB 功能上与 IPT 有重合，但是 MVTB 包含 IPT 中没有的函数，而 IPT 也包含 MVTB 中没有的函数。在 MATLAB® CENTRAL http://www.mathworks.com/matlabcentral/fileexchange 上的图像处理目录中列出了 500 多个文件。

通用的软件工具

在 MATLAB® 环境之外，还有很多高质量的图像和视频操作软件工具。在 http://opencv.org 中的 OpenCV 是一个成熟的开源计算机视觉软件工程。在[Bradski and Kaehler, 2008]这本书中很好地介绍了这个软件和一般意义上的计算机视觉。

http://www.imagemagick.org 中的 ImageMagick 是一个跨平台的函数库和命令行工具集合，用于图像格式转换(大约 100 种格式)、操作和合成，并且其 API 绑定了很多种语言；http://netpbm.sourceforge.net 中的 NetPBM 可以建在大多数的平台上，也是一个图像格式转换(大约 100 种格式)和操作的命令行工具集。这个 NetPBM 实用工具对于很多 Linux 分派来说是一个可选安装。以上两种程序包都很适合于对大规模的图像集执行批操作。工具箱函数 pnmfilt 可以通过其中一种外部程序来处理一幅图像，例如：

```
>> rlena = pnmfilt(lena, 'pnmrotate 30');
```

对于视频操作，http://www.ffmpeg.org 的 FFmpeg 是一种很好、很全面的跨平台工具。它支持视频格式之间的转换，也支持视频到静止图像之间的相互转换。

习题

1. 请尽快熟悉用于灰度图像和彩色图像的函数 idisp。探究图像中的像素值，以及缩放、画线和直方图按钮。
2. 从你的计算机相机或电影文件中截取一些图像帧，然后显示出来。
3. 写一个循环程序，它从你的相机中抓取一帧图像，然后显示出来。在显示之前给图像添加一些效果，例如负像、阈值转换、海报化、伪色彩、边缘过滤等。

4. 运动检测：
   a) 写一个循环程序，它使用你相机中的一些图像帧并执行背景估计。当你在场景中移动目标物体或让它们停留一段时间时，分别会有什么现象发生？再探索改变参数 σ 的效果。
   b) 修改第 288 页上的 *LeftBag* 示例，突出移动的人。
   c) 结合运动检测与色度调整，把移动的人从大厅放到沙漠里。
5. 互相关。
   a) 分别使用一个 21 ×21 的平均核和高斯核进行平滑处理，比较它们的结果。在前者中你可以观察到振铃产物吗？
   b) 为什么我们选择一个和等于 1 的平滑处理核呢？
   c) 比较一个简单水平梯度核 $K = (-0.5, 0.5)$ 和索伯核的效果。
   d) 研究高斯核的过滤，采用不同的 σ 值和核大小进行比较。
   e) 创建一个 31 × 31 核来检测 60°处的直线。
   f) 推导高斯函数在 x 方向上的导数，即方程(12.2)。
   g) 推导高斯-拉普拉斯的导数，即方程(12.6)。
   h) 推导第 301 页上的高斯差分。
6. 分析并展示光强度误差对 SSD 和 NCC 两种相似性测度的影响。
7. 模板匹配：
   a) 使用函数 `iroi` 选择 Lena 的一只眼睛作为模板。模板的边长应该为奇数。
   b) 使用 `isimilarity` 计算 Lena 这个相似图像。最佳匹配是什么？它发生在什么地方？与另一只眼睛的相似性是什么？第二佳的匹配发生在哪里？它的相似度分值是多少？
   c) 增大和减小 Lena 图像的光强度，研究它们对峰值相似性的影响。
   d) 在 Lena 图像的强度上加一个偏移，调查它对峰值相似性的影响。
   e) 采用不同的相似性测度，如 SAD、SSD、排秩和统计，重复 c) 和 d)。
   f) 采用不同的因子(用 iscale，在 0.5 ~ 2.0 之间，步长为 0.05)缩放模板的尺寸大小，调查它们对峰值相似性的影响。绘制出峰值相似性相对于尺度的曲线。
   g) 采用 -0.2 ~ 0.2 弧度之间，步长为 0.05 的因子旋转模板，重复 f)。
8. 执行第 316 页上的亚抽样示例，观察屋顶瓦片上锋利边缘和规则纹理周围的混叠现象。对于一个依靠 M 的抽取，什么是合适的平滑处理核宽？
9. 写一个函数，它从 `ipyramid` 的输出中创建图 12.33。
10. 创建一个变形函数来模拟你喜欢的哈哈镜。
11. 把图像变形为相对于图像中心的极坐标 (r,θ)，其中水平轴是 r，竖直轴是 θ。

第 13 章　图像特征提取

在上一章中我们讨论了图像的获取和处理。我们了解到图像仅仅是一个很大的像素点阵列，但是对于机器人应用来说，图像不仅数据量大，而且信息也不充足。我们需要能回答一些简明扼要的问题，如目标的位姿是什么？目标的类型是什么？它移动多快？我移动多快？等等。这些问题的答案都来自从图像中获得的测量值，我们称之为图像特征。图像特征是一个场景中的关键点，也是机器人控制所需的原材料。

上一章讲述的图像处理方法是对一张或多张的输入图像进行操作，并返回另一张图像。与之相反，特征提取是对一张图像进行操作，返回一张或多张输出图像。通常，特征为标量(如面积和纵横比)或者短向量(如一个目标的坐标或一条线的参数)。如果要用图像数据来控制机器人，图像特征提取就是必要的第一步。它是信息凝练的一个步骤，要把数据速率从相机输出时的$10^6 \sim 10^8$ 字节/秒减少到几乎是每帧几十个特征的程度，这样才可以用作机器人控制系统的输入。

在这一章中，我们将讨论特征以及怎样从图像中提取特征。借助于上一章的图像处理技术，我们将要讨论几种特征类型：区域、线和兴趣点。13.1 节讨论了区域特征，它们是相邻的且具有某种共同像素特性的像素组合。例如，在非红色背景中代表一个红色目标的像素集。13.2 节讨论了描述空间直线的线特征。直线在人工环境中比较显眼，也很常见——例如门廊、建筑物和道路的边缘。最后一种特征是兴趣点，将在 13.3 节中进行讲解。它们是在场景中特别明显的一些点，可以从不同视角在同一场景中被可靠地发现。

图像特征是对组成图像的像素点信息的一种总结，而且从真实世界到像素的映射一定存在重要信息的丢失，大家在学习过程中要始终记住这些。通常对付信息丢失的办法是基于我们的环境知识做出一些假设，但是我们的系统也就只能局限于这些假设的有效性了。例如，我们可能使用图像特征来描述对应于某个红色物体的一组红色像素的位置和形状。然而，尺寸特征，通常指像素数量，不会告诉我们任何关于实际空间中红色物体大小的事情——我们需要额外的信息，如相机与物体之间的距离和相机的固有参数。我们也需要假设物体不是被局部遮挡的，它会使得观察到的尺寸小于实际尺寸。更进一步，我们需要假设照明能满足这样的条件，即从目标物体反射光的色度是红色的。我们还可能在一幅图像中找到一些没有对应任何实际物体的特征——装饰性的标记、阴影的粗边或者窗户的反光。

13.1　区域特征

图像分割是把一幅图像划分成具有应用意义的若干区域的过程，如图 13.1 所示(见彩图)。其目的是把代表感兴趣目标的像素从场景的其他像素中分割或分离出来。这是最早的场景理解的方法之一，虽然概念上直截了当，但却是一个很具有挑战性的问题。关键的要求是

具有鲁棒性，即虽然隐含的假设条件被违反了，如场景照明或视点改变，但是方法仍然保持一定的有效性。

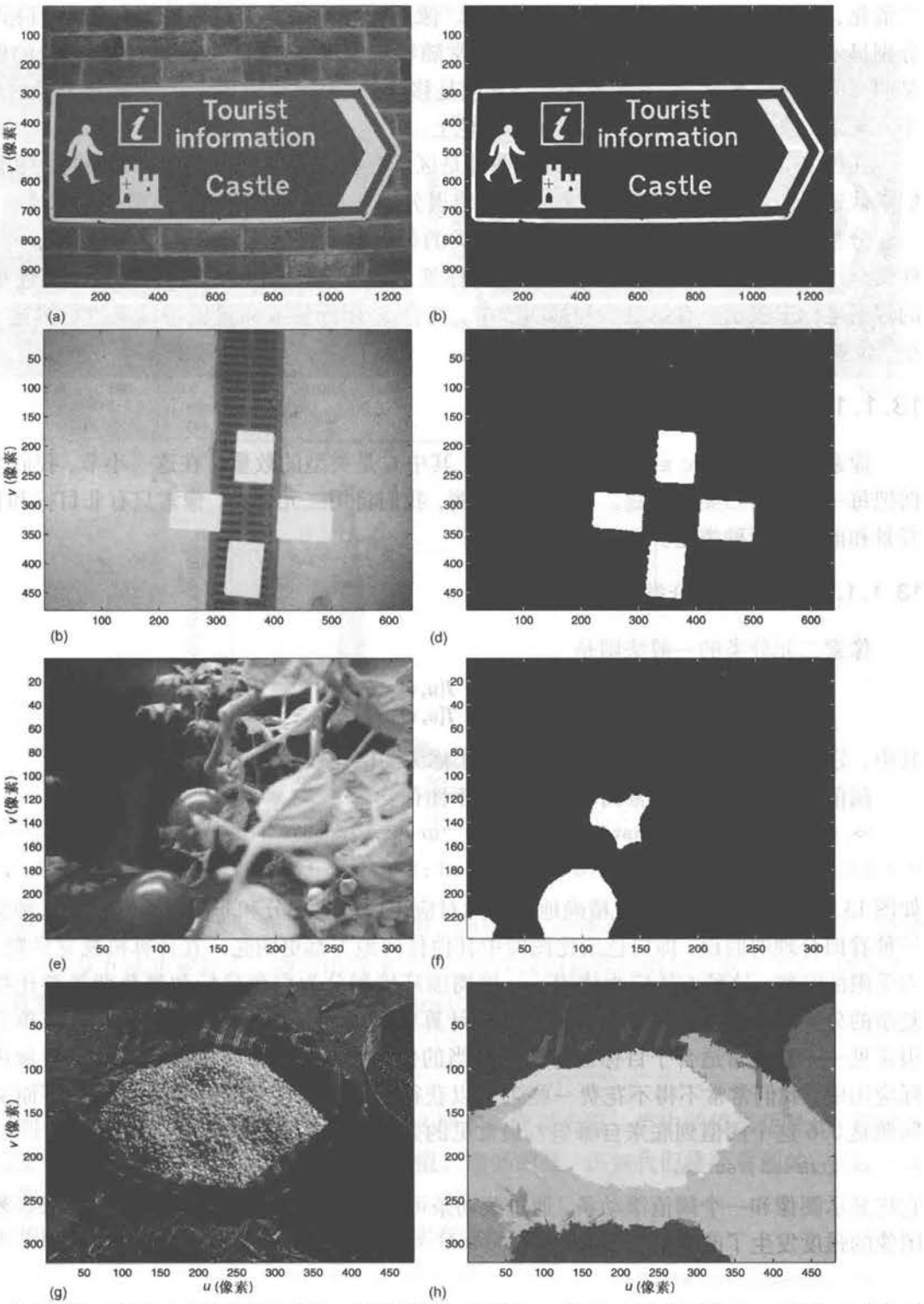


图 13.1　像素分类的实例。左侧是输入图像，右侧是分类后图像。分类是针对不同应用的，像素已经被分类为目标物体（白色）或非目标物体（黑色）。感兴趣的目标是：(a) 标牌上的单独字母；(b) 黄色目标；(c) 红色番茄。(d) 是一个多级分割，其中像素已经被分为28类，它们代表了在场景中局部共性的像素组

图像分割可以看作是三个子问题。第一个是分类，它是作用于每一个像素并把它们归类为 C 类型 $c \in \{0 \cdots C-1\}$ 之一的判定过程。通常我们设定 $C=2$，这种分类被称为二元分类或二值化，一些示例如图 13.1(a)～(f)所示。像素已经被归类为目标($c=1$)或非目标($c=0$)，分别显示为白色或黑色像素。这种分类通常随特定应用而不同——例如目标对应的像素可能是明亮的，或是黄色的，或是红色的，又或是移动的。图 13.1(h)是一个多级分类的示例，其中 $C=28$，像素类型反映在它们显示的颜色上。

在图 13.1 的示例中，隐含的假设条件是区域各自具有某种共同的像素特性。实际上，我们承认这一阶段是有缺陷的，像素可能被错误分类——下一工序就是处理这些问题。

分割处理的第二步是表示，其中同一类的邻近像素被连接起来形成空间集合 $S_1 \cdots S_m$。这些集合可以通过给每个像素设定一个集合标签来表示，也可以由一系列定义了相连集合边界的像素坐标来表示。在第三步和第四步中，集合 S_i 用标量或向量值特征来进行描述，例如大小、位置和形状。

13.1.1 分类

像素类型由整数 $c \in \{0 \cdots C-1\}$ 表示，其中 C 是类型的数量。在这一小节，我们会讨论如何把每一个像素归类的问题。对于很多示例，我们将用二元分类，像素只有非目标和目标，或背景和前景这两种类型。

13.1.1.1 灰度等级分类

像素二元分类的一般法则是

$$c[u,v] = \begin{cases} 0 & I[u,v] < t \\ 1 & I[u,v] \geqslant t \end{cases} \quad \forall (u,v) \in I$$

其中，分类决策仅仅基于像素值。这种方法称为阈值变换法，t 为阈值。

阈值变换法的实施非常简单。考虑如下图像：

```
>> castle = iread('castle_sign.jpg', 'grey', 'double');
>> idisp(castle >= 0.6)
```

如图 13.2 所示。像素已经被精确地分类为对应于白色漆部分和非白色漆部分。这种分类基于一种看似合理的假设，即白色漆比图像中其他任何地方都更明亮。在计算机视觉早期，计算能力受限的时候，这种方法广为使用——把周围环境划分为白色目标和黑色背景要比执行更为复杂的分类简单得多。在基于普通嵌入式计算机的现代工业检测系统中，这种简单方法仍然很常见——它非常适合于目标在对比色恰当的传送带上，或者在检查站的标准轮廓内。在实际应用中，我们常常不得不花费一些工夫以获得有用的灰度等级分类。一个重要而又困难的问题是 0.6 这个阈值到底来自哪里？最常见的方法是反复尝试！使用工具箱函数 ithresh：

```
>> ithresh(castle)
```

它将显示图像和一个阈值滚动条，通过滚动条可以调整阈值，直到获得满意的结果。然而如果图像的强度发生了改变：

```
>> ithresh(castle*0.9)
```

就会需要不同的阈值。

一种比反复尝试更有规律可循的方法是分析图像的直方图：

```
>> ihist(castle, 'b');
```

如图 13.2(c)所示。直方图上有两个清晰的峰值，即双峰分布，分别对应于像素的两大群体。较小的峰值大约出现在0.9 的位置，对应明亮的像素点，值的变化范围非常小。较宽和较高的峰值出现在0.3 左右的位置，对应标牌和墙壁这些较暗的像素，亮度的变化范围更大。

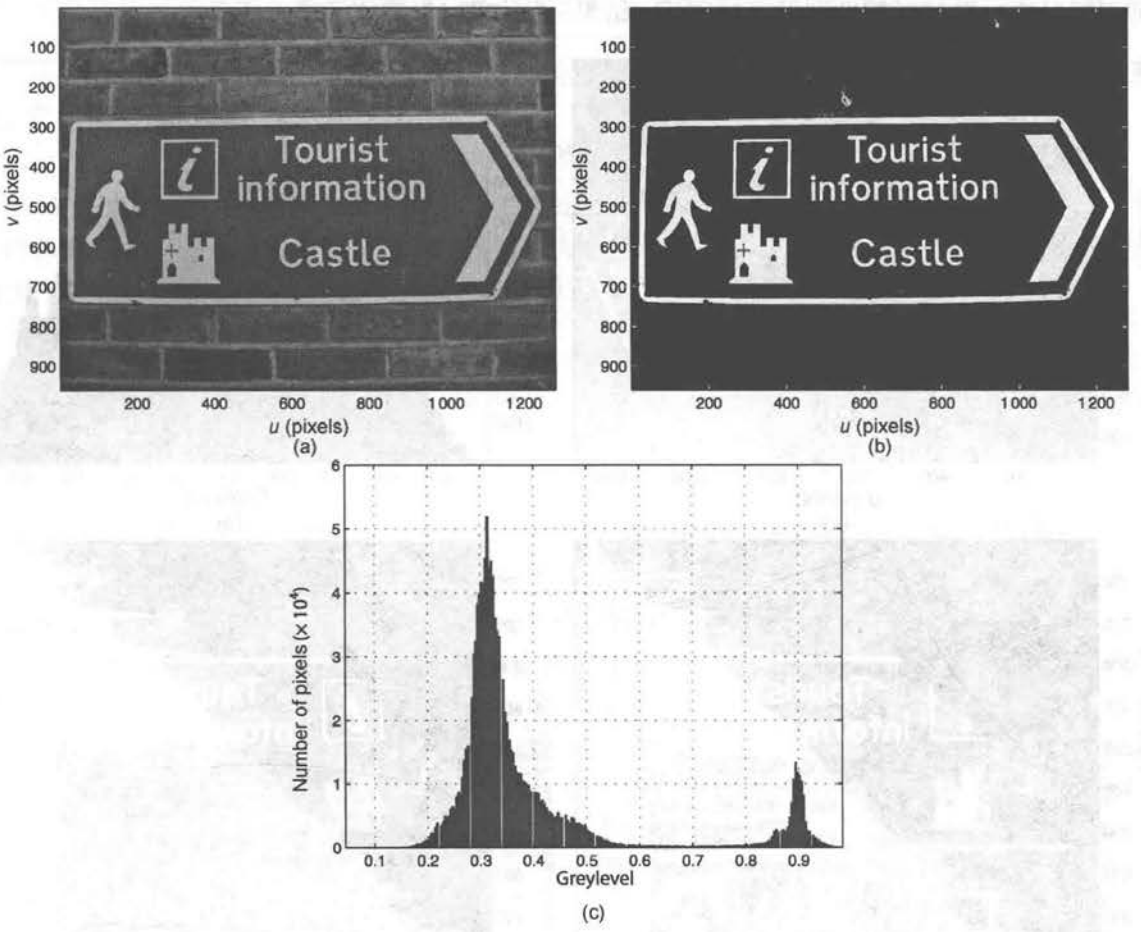


图 13.2　二元分类。(a)原始图像(图像源于 ICDAR 集；Lucas 2005)；(b)二元分类；(c)灰度像素值的直方图

为了分离这两个像素类型，我们把判定边界，即阈值，选择在两个峰值之间的山谷。就此而言，选择 $t = 0.6$ 就非常好了。因为山谷很宽广，所以我们选择阈值的范围相当大，例如 $t = 0.7$ 也是有效的。最优的阈值可以使用大津(Otsu)方法进行计算：

```
>> t = otsu(castle)
t =
    0.6000
```

它以某一种方式把一幅图像分离成两类像素，这种方式使得在每一类中的值变化最小，而两类之间的值变化最大——假设直方图只有两个峰值。遗憾的是，正如我们将要看到的，实际情况很少如此便利。

考虑相同的图像，但图像中增加了一个现实存在的对角线强度梯度：

```
>> castle = iread('castle_sign2.png', 'double', 'grey');
```

如图 13.3(a)所示。图 13.2(b)所示的直方图就与之前的完全不一样了，且更加复杂。对我们来说，遗憾的是再也没有两个明显不同的像素群可以区分。在这种情况下，大津方法会计算出它的阈值：

```
>> t = otsu(castle)
t =
    0.5412
```

应用这个阈值的结果如图 13.2(c)所示。像素分类很差，因为它违反了大津方法的基本假设，即双峰分布。取稍微偏低和偏高的阈值，结果分别如图 13.2(d)和(e)所示。

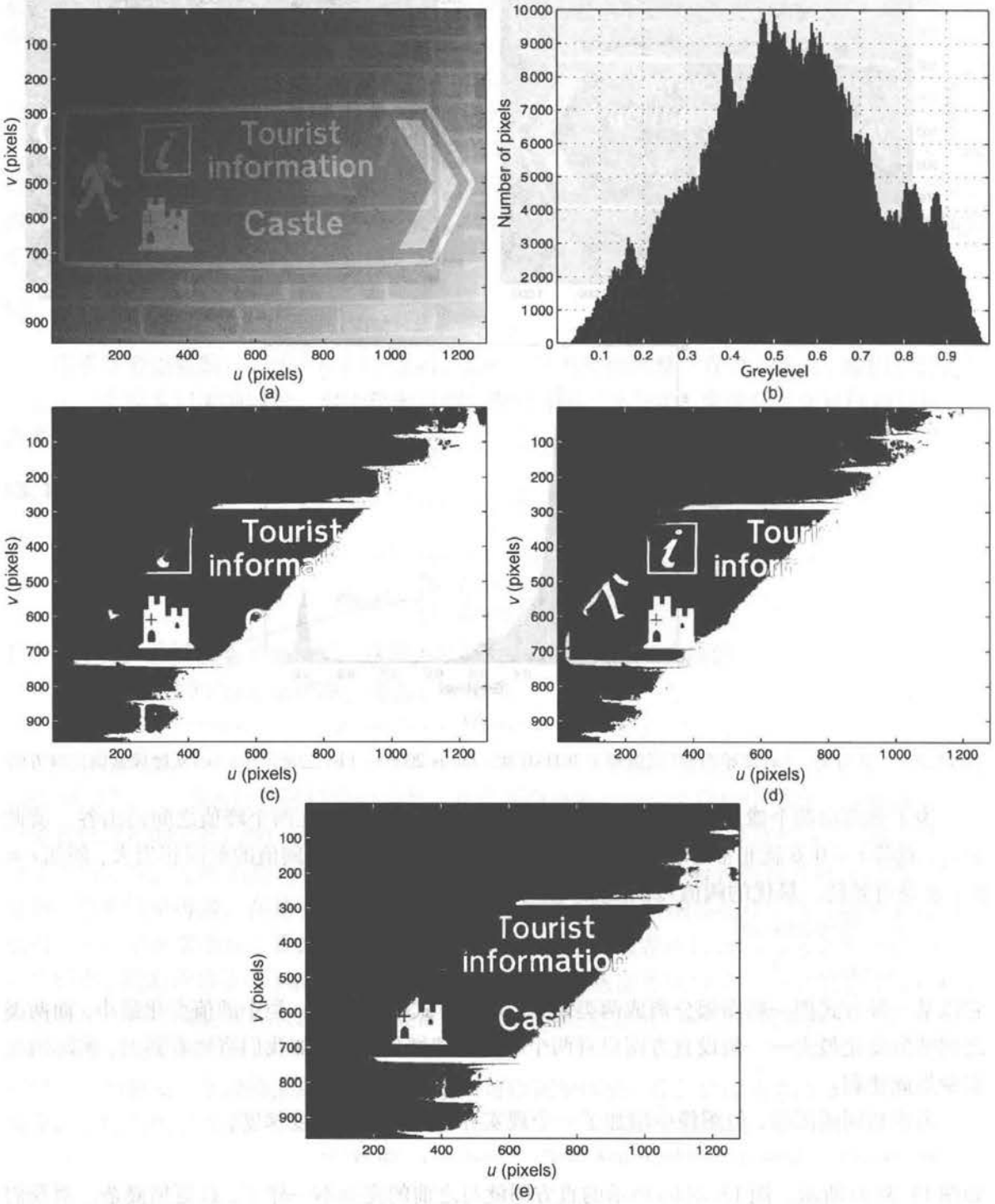


图 13.3 二元分割实例。(a)有对角线强度梯度的灰度图像；(b)直方图；(c)Otsu阈值在0.54；(d)阈值在0.50；(e)阈值在0.60

众所周知，基于阈值的技术很不稳定，场景照明的一个微小变化就意味着我们之前选择的阈值不再适用。在大多数真实场景中，没有从像素值到具体目标的简单映射，例如我们无法选择一个可以区分摩托车或鸭子的阈值。从背景中辨别出目标仍是计算机视觉中的一个难题。

一种替代方法是选择局部阈值而不是全局阈值。Niblack 算法广泛用于光学字符辨识系统，它计算局部阈值的方法为

$$t[u,v] = \mu(\mathcal{W}) + k\sigma(\mathcal{W})$$

其中 $\mathcal{W}$ 是点 (u,v) 周围的区域，$\mu(\cdot)$、$\sigma(\cdot)$ 分别是平均值和标准差。窗口 $\mathcal{W}$ 的大小是一个关键参数，应该和我们正在寻找的目标物体的大小接近。对于本示例，我们假设场景中字符大约 50 ~ 70 像素高，这样我们选择窗口的半宽为 35 像素：

```
>> t = niblack(castle, -0.2, 35);
>> idisp(t)
```

其中，$k = -0.2$。得到的局部阈值 t 如图 13.4(a)所示。我们把该阈值逐一像素地应用到原始图像中：

```
>> idisp(castle >= t)
```

产生的分类结果如图 13.4(b)所示。属于字母的所有像素已经被正确分类，但是与图 13.2(c)相比，仍有很多误报——非目标的像素被归为目标一类。随后，在本节中我们将会讨论消除这种误报的技术。注意，分类过程再也不仅仅是输入像素的函数，现在它是关于像素及其邻近点的一个复杂函数。虽然不再需要选择 t，但是我们现在需要选择参数 k 和窗口尺寸，而且这种选择又是常用的针对特殊的场景类型的反复尝试过程。

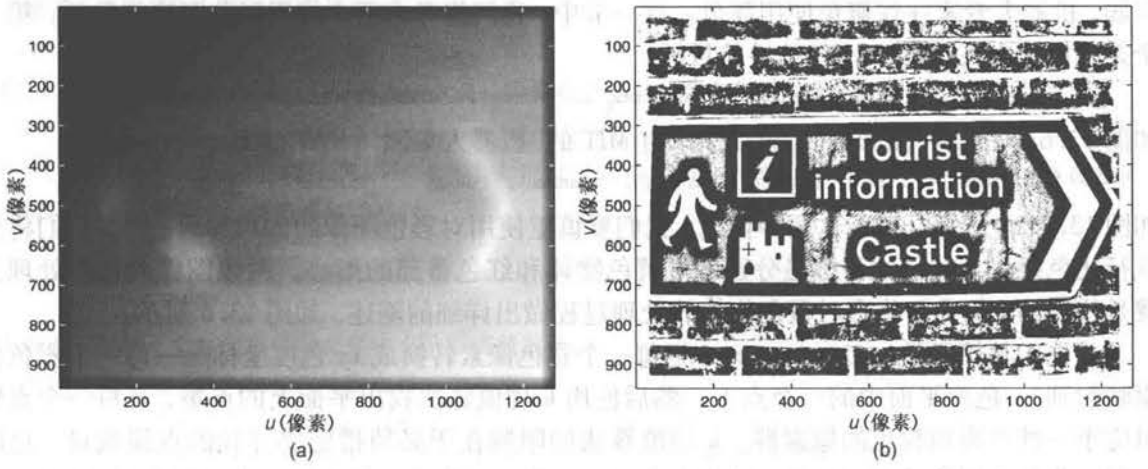


图 13.4　Niblack 阈值转换法。(a)显示为一幅图像的局部阈值；(b)二元分割

图 13.3(c) ~ (e)所示的结果乍一看会让人有些失望，但是我们可以看到每一个目标都通过某个阈值得到了正确的分类。事实上，能正确分割每个目标的是一个阈值范围，我们想要的是用全部的阈值对图像进行分类，然后再组合起来形成一个分割图像。而这恰好就是最大稳定极值区域(MESR)算法所做的。它通过工具箱函数 imser 来执行：

```
>> [mser,nsets] = imser(castle, 'light');
```

对于这幅图像：

```
>> nsets
nsets =
    40
```

这是其所有的稳定集合。另一个返回值是一幅图像：

```
>> idisp(mser, 'colormap', 'jet')
```

如图 13.5 所示，它是一幅伪色彩图像。每一个非零像素对应着一个稳定集合，像素值是分配给那个稳定集合的标签，显示为单一颜色。所有字母都被正确分类。一部分边界被错误地归类为背景，一部分被归类为图像右侧的墙砖纹理。选项'light'表明我们正在寻找与黑暗背景相对的明亮目标①。

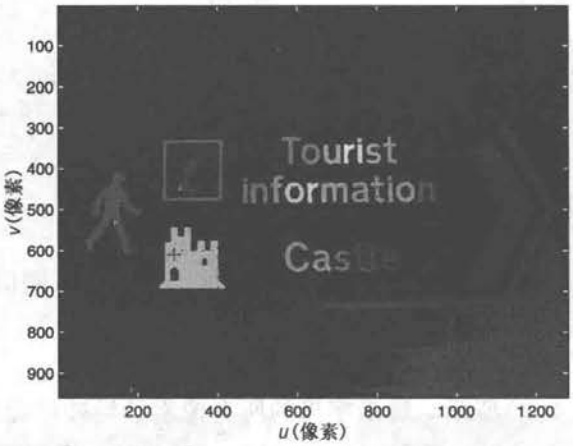


图 13.5 使用最大稳定极值区域(MSER)的分割。认定的区域具有唯一颜色编码

13.1.1.2 颜色分类

颜色对图像分割来说是一个很有用的线索，但是由于存在 10.3.1 节中讨论的颜色恒常性问题，机器人专家往往避免使用颜色。这一节中，我们将考虑两个使用彩色图像的示例。第一个是室内无人飞行器(UAV)的导航目标：

```
>> im_targets = iread('yellowtargets.png', 'gamma', 'sRGB', 'double');
```

如图 13.6(a)所示(见彩图)，第二个来自 MIT 的“机器人菜园”(Robotic Garden)项目：

```
>> im_garden = iread('tomato_124.jpg', 'gamma', 'sRGB', 'double');
```

如图 13.7(a)所示。对于这两种情况，我们要慎重使用对彩色图像的伽玛解码，因为我们将会执行比色操作。我们的目标是分别确定黄色物体和红色番茄的形心。两幅图像的初始处理步骤是相同的，但我们将会对黄色物体的处理过程做出详细的阐述，如图 13.6 所示。

首先工具箱函数 colorkmeans 把每一个彩色像素转换成 xy 色度坐标——每一个彩色像素映射到 xy 色度平面中的一个点上。然后使用 k 均值算法找出平面上的点集，且每一个点集对应于一种可辨识颜色的像素群。k 均值算法的限制在于必须指定要寻找的点集数量。这时就需要我们对图像的预先认知，即这个特别的场景有 4 种不同颜色的元素：黄色目标物体、花朵、排水沟金属盖和黑暗的阴影。通过以下函数，像素被类聚到 4 个色度类型($C=4$)：

```
>> [cls, cxy,resid] = colorkmeans(im_targets, 4);
```

它会返回每一个点集的 xy 色度：

```
>> cxy
cxy =
    0.3331    0.2862    0.4049    0.2947
    0.3544    0.3088    0.4414    0.3215
```

① 尽管没有明确地给定阈值，但是 imser 有很多参数，本示例中它们的默认值已经给出令人满意的结果。请查看网上文献获得关于这些参数的详细信息。

其中每一个点集为一列。我们可以在 xy 平面上绘制这些点集的中心：

```
>> xycolorspace(cxy);
```

如图 13.6(c)所示。我们看到点集 3 距离黄色最近：

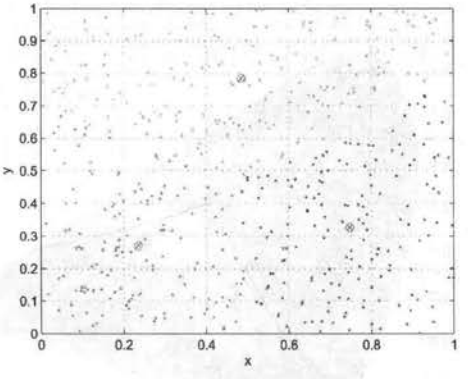
```
>> colorname(cxy(:,3), 'xy')
ans =
    'banana'
```

其残差为

```
>> resid
resid =
    6.2583
```

是每一个点到它所指派的点集中心的距离范数。由于算法使用了随机初始化，我们在每次运行时都会获得不同的点集和分类，因此有不同的残差①。

背景材料： k 均值聚类是一种迭代运算，它把 n 维点分组成 k 个空间点集。每一个点集由其中心点定义，是一个 n 维向量 c_k。在每一次迭代中，所有的点被分派到最近的点集中心，然后每一个点集中心被更新为分到这个点集的所有点的平均值。



这种算法通过工具箱函数 kmeans 来执行。距离的度量标准是欧几里得距离②。k 均值算法需要每一个点集的初始中心估计值，可以通过三种方法获得。初始点集中心可由用户提供，选项'random'将会随机选择所提供点的 k 值，'spread'将会从包含这个点集的超立方体中选择 k 个随机点。这种随机初始化意味着算法将会在每一次调用中返回不同的结果③。

例如，选择 500 个随机二维点：

```
>> a = rand(2,500);
```

其中，a 是一个 2 × 500 矩阵，每一列为一个点。我们将会把这些数据分成 3 个集合：

```
>> [cls,centre,r] = kmeans(a, 3);
```

其中，cls 是一个 500 维的向量，其中每一个元素指定了 a 中相应列的类别。centre 是一个 2 × 3 的矩阵，它的每一列指定每一个二维点集的中心：

```
>> r
r =
    4.3166
```

它是残差，即每一个点到它指定点集中心的距离范数。

我们使用红色、绿色和蓝色绘制出每一个点集：

```
>> plot_point(a(:,cls==1), 'r.')
>> plot_point(a(:,cls==2), 'g.'); plot_point(a(:,cls==3), 'b.')
```

再把每个点集中心叠加上去：

```
>> plot_point(centre, 'o');  plot_point(centre, 'x')
```

然后我们看到这些点已经被有效分区开了。(右上图中显示：上方浅色点为绿色，左下方中深灰色点为红色，右下方较深灰点为蓝色。)

---

① 一种选择是运行 k 均值算法很多次，然后取残差最小的为点集中心。

② 对于彩色空间，欧几里得距离也许不合适，因为人类对色差的感知依赖于在 xy 色度平面中的距离，也依赖于平面内的位置。

③ 函数 randinit 可以把 MATLAB® 随机数发生器设置成一种已知状态，这样就可以确保可重复的结果。

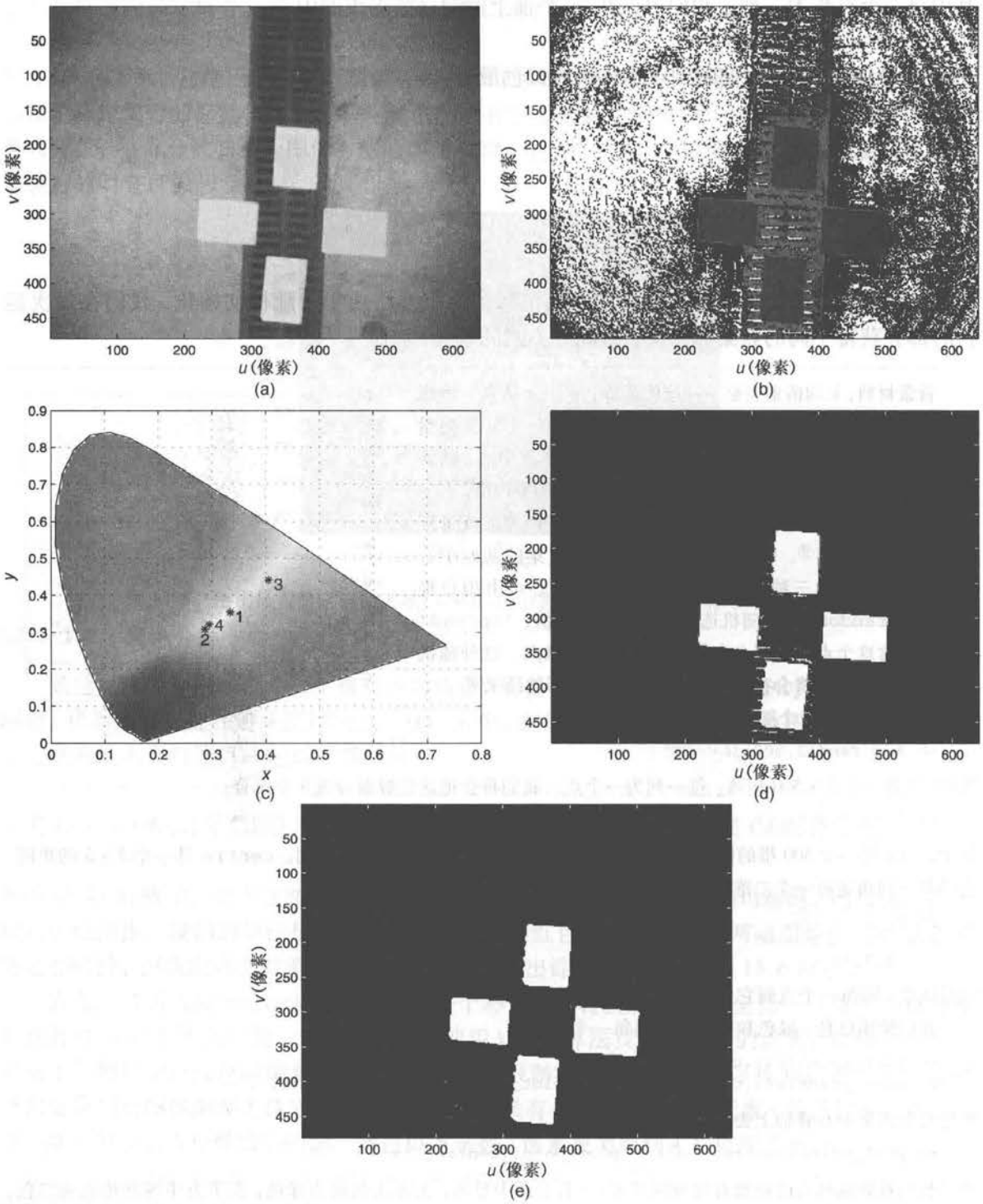


图 13.6 目标图像示例。(a)原始图像;(b)用伪色彩显示的像素分类($C=4$);(c) xy 色度空间的点集中心;(d)类型$c=3$的所有像素;(e)用一个圆形结构化元素进行形态学闭操作后的结果

函数 colorkmeans 也会返回像素分类，我们可以用伪色彩显示其图像：

```
>> idisp(cls, 'colormap', 'flag')
```

如图 13.6(b)所示。图像中的像素具有值 $c = \{0,1,2,3\}$，它表明相应的输入像素被分配到了哪一个类型。如果我们使用 idisp 来交互式地查询像素的值，那么会看到黄色目标物体实际上已经分配到 $c=3$ 那一类，显示为蓝色。

k 均值聚类计算量很大，因此不适合于实时应用。然而，我们可以把这个过程分成训练阶段和分类阶段。在训练阶段，很多示例图像将会被组合起来并传给 colorkmeans，它将确定每一个类型的点集中心。随后在分类阶段，我们就能相对简单地把像素分配到离它们最近的点集：

```
>> cls = colorkmeans(im_targets, cxy);
```

可以选择属于类型 3 的像素：

```
>> cls3 = (cls == 3);
```

它是一个局部图像，可以显示出来：

```
>> idisp(cls3)
```

如图 13.6(d)所示。类型 3 的所有像素被显示为白色[①]，对应于原始图像中的黄色目标。这张二值图像是一个好的分类，但还不是最好的，其中存在一些不属于目标的白色像素——这些像素的色度接近黄色但是强度较低，所以它们看起来不是黄色的[②]。

12.5 节中讨论的形态学开操作将会消除这些缺陷。我们使用一个半径为 2 的对称结构化元素：

```
>> targets_binary = iopen(cls3, kcircle(2));
```

结果如图 13.6(e)所示。它显示了一个干净的二元分割，像素被分成两个类型：目标和非目标。

背景材料：镜面反射是一种类似于镜子的来自目标表面的反射。光不能穿透表面，会发生散射，这就是通常目标对象把色彩传给反射光的方法。镜面反射的色彩是光源的色彩，而不是反射表面的色彩。一个镜面反射体，如镜子，遵循下述反射规律：即出射光线的反射角等于入射光线的入射角。粗糙表面是一个漫反射体，或兰伯特(Lambertian)反射体，它在所有的视角中有相同的视亮度。大部分的实际表面都是镜面反射体与兰伯特伯反射体的混合体。

减少镜面反射问题有几种方法：第一，消除镜面反射的光源。第二，使用相机附近的一个漫射光源，例如可以套在相机镜头上的环形照明器。第三，使用偏振滤波器减弱镜面反射，因为介质表面反射的光是偏振的。

对于菜园图像，我们采用一个非常相似的处理过程。由于我们预先知道场景中包含了番茄、明亮的叶子、黑暗的叶子和黑暗的背景，我们把像素分成 4 个点集($C=4$)：

```
>> [cls, cxy] = colorkmeans(im_garden, 4);
>> cxy
cxy =
    0.3610    0.5388    0.3076    0.3249
    0.4829    0.3335    0.3359    0.4034
```

用伪色彩显示的像素类型如图 13.7(b)所示(见彩图)。对应于番茄的像素被分配到型类 $c=2$，显示为白色。点集中心在图 13.7(c)的 xy 色度平面上标记出来。可以挑选出红色像素：

```
>> cls2 = (cls == 2);
```

结果局部图像如图 13.7(d)所示。这个分割完全称不上完美，两个番茄由于镜像反射都出现了空洞，特别是位于上部的番茄，其洞的大小甚至超过了目标本身。左下方的一部分像素也被

---

① MATLAB® 通过整数值 1 和 0 分别表示逻辑值真和假。它们分别显示为白色和黑色。

② 我们遇到与 10.3.5 节中的花朵图像相似的情况。

错误地归类为番茄。我们可以使用一个带有大圆形核的形态学闭操作来提高结果质量，大圆形核与番茄的大小一致：

```
>> tomatoes_binary = iclose(cls2, kcircle(15));
```

结果如图 13.7(e)所示。闭操作部分恢复了果实的形状，但也产生了不想要的结果——左下角误分类的像素群被放大。不过，这幅图像中确实把像素分成了两类：番茄和非番茄类。

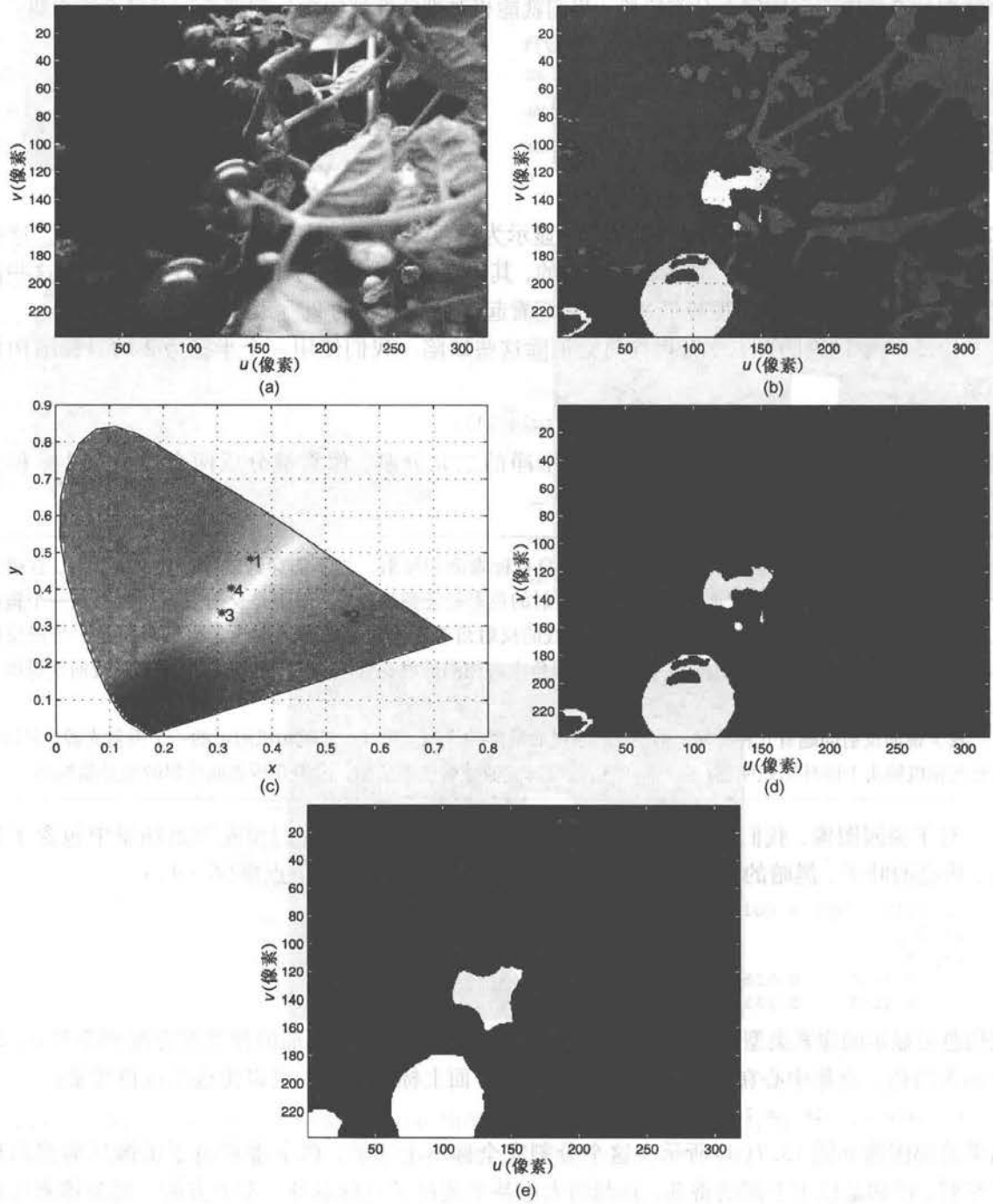


图 13.7 菜园图像示例。(a)原始图像(由 MIT 分布式机器人菜园项目提供)；(b)用伪色彩显示的像素分类($C=4$)；(c)在xy色度二维空间中的点集中心；(d)类型$c=2$的所有像素；(e)用半径为15的圆形结构化元素进行形态学闭操作后的结果

菜园图像阐明了两个真实世界中常见的成像干扰：镜面反射和遮挡。番茄的表面非常光亮，相机都看到了室内照明的反射——这些像素是白色的而不是红色的[①]。另外，上面那个番茄也被叶子和枝干部分地遮盖住了。这种遮挡可能是个问题，也可能不是个问题，要根据具体应用而定。由于遮挡物的存在，从拍摄相片的方向是无法触及到番茄的，所以事实上不把它归类为番茄更为恰当。

这些示例已经展示了把图像像素分为目标和非目标的一个切实可行的分类方法。产生的白色像素集群通常被称为斑块。有趣的是，我们没有指定任何阈值或者任何目标颜色的定义，但是又不得不指定类型的数量并确定哪种类型与感兴趣的目标相对应[②]。我们还不得不选择图像处理步骤的顺序以及每个步骤中的参数，如结构化元素的半径。像素分类是一个很困难的问题，但可以通过利用对问题的预先认知、良好的图像处理技巧以及经验来获得更好的结果。

13.1.2　图像表示

在前面一节中，我们使用了复杂的灰度或彩色的图像，并对它们进行处理以产生二值图像或斑块图像。图像表示是把同类的相邻像素点连接起来形成空间集合 $S_1 \cdots S_m$ 的一个子问题。

考虑这个小型 10 × 8 二值图像：

```
>> im = ilabeltest;
```

把它显示出来：

```
>> idisp(im)
```

如图 13.8(a)所示。我们很快确定了场景中的三个白色斑块。但是一个斑块是如何定义的呢？答案是互相连接的同一类型像素的集合，在本示例中为白色。更为正式的定义是：一个斑块是空间上连续的同一类像素构成的区域。斑块也被称为区域或者连通区域。

工具箱可以在二值图像上执行连通区域或者进行连通性分析：

```
>> [label, m] = ilabel(im);
```

在这幅图像中，集合或者区域的数量是

```
>> m
m =
     5
```

它们是三个白色斑块和两个黑色斑块(背景和洞)。这些斑块被贴上从 1 到 5 的标签。返回的标签矩阵为

```
>> label
label =
    2    2    2    1    1    2    2    2    2    2
    2    2    2    1    1    2    3    3    3    2
    2    3    2    2    2    2    2    3    2    2
    2    3    2    3    2    3    2    3    2    2
    2    3    3    3    3    3    3    3    3    2
    2    2    2    2    2    3    4    4    3    2
    2    5    5    2    3    3    3    3    3    2
    2    2    2    2    2    2    2    2    2    2
```

和原始图像大小相同，每一个元素包含对应的输入像素所属集合的标签 $s \in \{1 \cdots m\}$。这个标

① 观察到它们的色度与黑色背景相同，是类型 3 的像素，位于 xy 平面上靠近白色点的位置。

② 这是一个相对简单的问题。黄色的色度是众所周知的(可以使用 colorname 函数来找到它)，因此可以计算每个点集中心到标准颜色的距离，并且选择最近的点集。

签矩阵可以被显示为一幅伪色彩的图像[①]：

```
>> idisp(label, 'colormap', 'jet')
```

如图 13.8(b)所示。每一个连通区域有唯一的标签，因此有唯一的颜色。观察上文所示的矩阵 label 中的元素，或者通过使用 idisp 交互式地查看显示的标签图像，可以看到背景已经被标记为 2，顶部的小方形被标记为 1，较大的区域被标记为 3，区域中的洞被标记为 4。

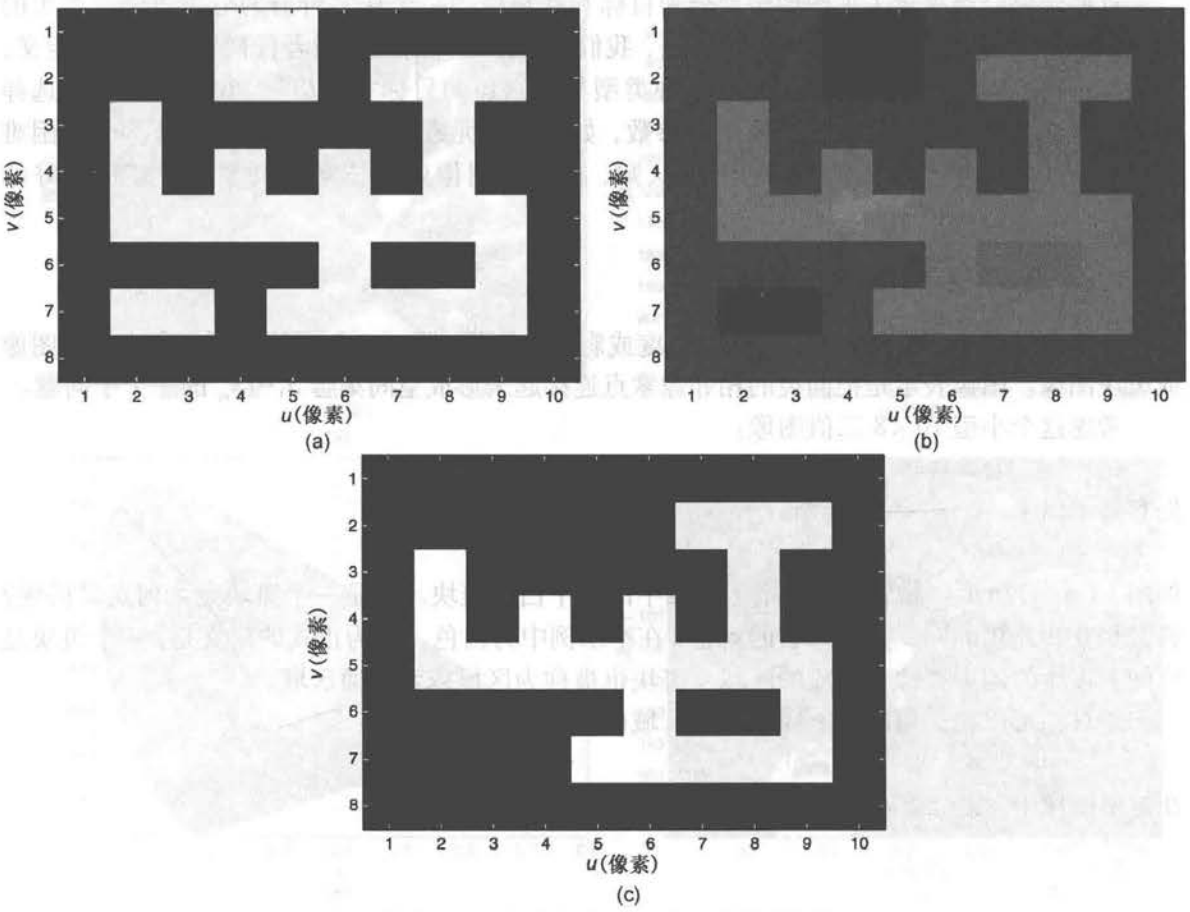


图 13.8 图像标记示例。(a)二值图像；(b)贴上标签的图像；(c)有标签 3 的所有像素

现在很容易获得一幅只包含一个特殊斑块的图像，选择属于区域 3 的所有像素，即较大的不规则区域，我们创建一个局部图像：

```
>> reg3 = (label==3);
>> idisp(reg3)
```

如图 13.8(c)所示。

连通性分析可以返回其他的输出值：

```
>> [label, m, parents, cls] = ilabel(im);
```

其中，向量 parents 描述了区域的拓扑结构或等级关系：

```
>> parents'
ans =
     0     0     2     3     2
```

① 我们以前已经见过标签图像。图 13.5 中 MSER 函数的输出就是一个标签图像。

例如，该向量表明区域 4

```
>> parents(4)
ans =
     3
```

是被区域 3 包围的——区域 3 是区域 4 的父区域。区域 3 和区域 5 被背景区域 2 包围。区域 1 和 2 的父区域为 0，这表明它们已经连接到图像的边缘，没有被任何区域包围。每一个连通区域包含单一类型的像素，对于每一个区域，像素种类通过向量 cls 给定：

```
>> cls'
ans =
     1     0     1     0     1
```

它表明区域 1、3、5 由类型 1(白色)的像素组成，区域 2 和 4 由类型 0(黑色)的像素组成①。

在这个示例中，我们假设了 4 个方向的连通性，即像素仅仅通过其北、南、东、西 4 个方向的同类邻近像素接入一个区域。8 个方向的连通性选项允许通过 8 个方向的任意邻近像素进行接入②。

现在回到上一节中的示例。对于着色的目标

```
>> targets_label = ilabel(targets_binary);
>> idisp(targets_label, 'colormap', 'jet');
```

和菜园图像

```
>> tomatoes_label = ilabel(tomatoes_binary);
>> idisp(tomatoes_label, 'colormap', 'jet');
```

连通区域为如图 13.9 所示的伪色彩图像。

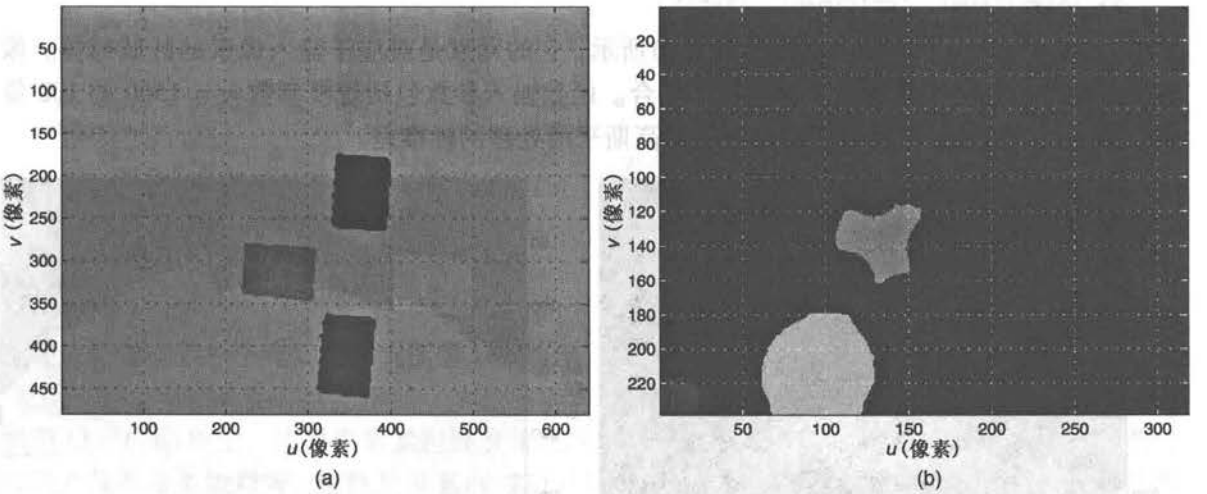


图 13.9　彩色目标和菜园示例的伪色彩标签图像。每一个像素值是相应输入像素所属空间连续集合的标签

13.1.2.1　基于图的分割

到目前为止，我们已经根据目标的一些一致性特征对像素进行了分类，例如强度或者颜色。现在考虑一个复杂的场景：

① 我们使用缩写 cls，因为名称 class 已被 MATLAB® 中的函数使用。

② 8 个方向的连通性会导致令人意外的结果。例如，黑色和白色象棋棋盘本应该就有两个区域，所有白色方形是一个区域，所有黑色方形是另一个区域。

```
>> im = iread('58060.jpg');
```

如图 13.10(a)所示。格式塔(Gestalt)的出现原理指出，我们识别目标是把它看成一个整体而不是许多单体的集合——我们看到的是一碗谷子，而不是通过看见其中每一粒谷子才推断出它是一碗谷子。然而具体到逐个像素的分割时，情况往往就变得带有主观性——不同的人会根据自己的重要性判断而采取不同的分割。例如，一块布料上的彩色条纹应该被分割出来吗①？如果分割物是要代表真实世界的目标对象，那么根据格式塔的观点这块布料就应该只是一个分割物。然而这些彩色条纹也是真实存在的，是设计师花精力创造出来的，所以它们也应该被分割出来。这就是为什么分割是一个难题——人们无法就什么是正确分割达成统一。没有任何一种计算机算法可以或者被期望做出这种类型的判断。

不过一些最近开发出来的算法可以在复杂的真实世界场景中做得较好。图像可以用一个图来表示(见附录 J)，其中每一个像素是一个顶点或节点，通过 8 条边连接着邻近像素点。每条边的权重是两个像素之间差异性的一个非负测度值——颜色差别的绝对值。算法从每一个分配到自身集合的顶点开始。每次迭代时都会重新检测边的权重，如果顶点属于不同的集合，而边的权重又低于某个阈值，那么就合并这两个顶点集合。阈值是集合的大小及全局参数 k 的一个函数，k 用于设置分割的规模——k 值越大会导致连通区域也偏大。

对于我们讨论的图像，基于图的分割通过以下函数执行：

```
>> [label, m] = igraphseg(im, 1500, 100, 0.5);
>> m
m =
    28
>> idisp(label, 'colormap', 'jet')
```

其中，label 是一个矩阵，如图 13.10(b)所示，它的元素是对应于输入像素的区域标签。像素分类步骤已经与图像表示步骤进行了整合。函数输入参数包括规模参数 k = 1500 的 100 像素的最小区域大小，以及对图像进行初始高斯平滑处理的标准差。

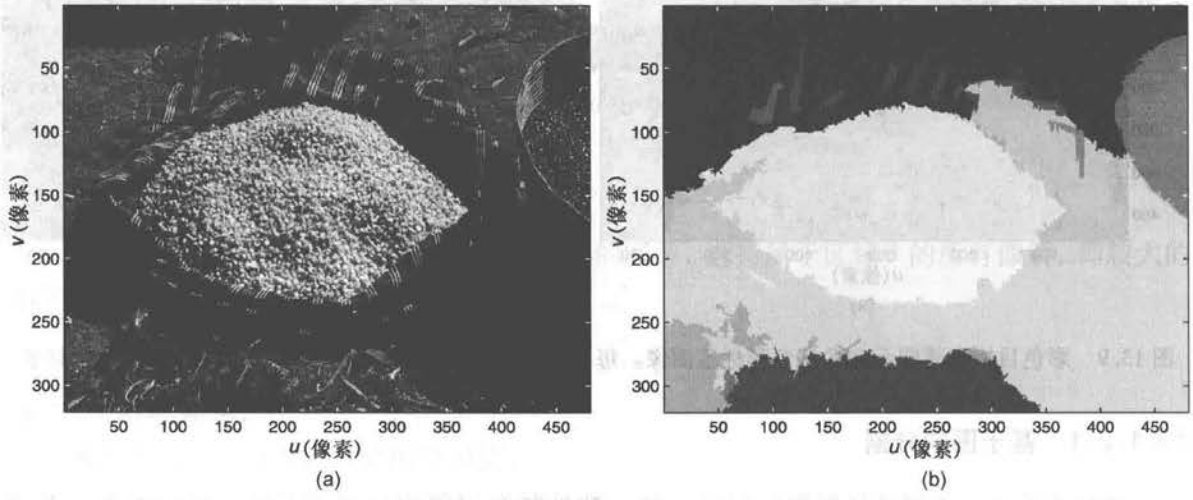


图 13.10 复杂分割示例。(a)原始图像(图像来自伯克利分割资料组；Martin et al.,2001)；(b)基于图的分割

① 伯克利分割网站 http://www.eecs.berkeley.edu/Research/Projects/CS/vision/bsds 中包括了这些图像，以及很多不同的人工分割。

13.1.3　图像描述

在上一节中我们学习了怎样在图像中寻找连通区域，以及怎样如图 13.8(c)所示那样隔离出特殊的区域。然而这种区域表示仍然只是一个局部像素值的图像，而不是区域尺寸、位置和形状的精确描述。

13.1.3.1　边框

表示尺寸和形状的最简单方法是用边框——能包围区域，且各边与 u 轴和 v 轴平行的一个最小矩形。回到彩色目标图像，我们可以在区域 5 中选择所有的像素：

```
>> reg5 = (targets_label == 5);
>> idisp(reg5);
```

产生的逻辑图像如图 13.11(b)所示。这个区域中的像素数量只需简单求一个和值：

```
>> sum(reg5(:))
ans =
     6004
```

所有非零(目标)像素的坐标是

```
>> [v,u] = find(reg5);
```

其中 u 和 v 是各自的坐标向量，它们的大小是

```
>> about(u)
u [double] : 6004x1 (48032 bytes)
```

区域的边界为[①]

```
>> umin = min(u)
umin =
   310
>> umax = max(u)
umax =
   382
>> vmin = min(v)
vmin =
   361
>> vmax = max(v)
vmax =
   460
```

这些边界定义了一个我们附加到图像上的矩形：

```
>> plot_box(umin, vmin, umax, vmax, 'b')
```

如图 13.11(b)所示。边界框紧紧包裹着斑块四周，它的中心可以看作是斑块的中心。然而边界框并没有与斑块对齐，也就是说它的边与斑块的边不平行。这就意味着尽管斑块的尺寸和形状不发生改变，但是随着斑块的旋转，边界框的尺寸和形状将会改变。

13.1.3.2　矩

图像矩是一个内涵丰富且可减少计算量的图像特征类，用它可以描述区域的形状、大小和位置。图像 $\mathbf{I}$ 的矩是一个标量：

$$m_{pq} = \sum_{(u,v)\in \mathbf{I}} u^p v^q \mathbf{I}[u,v] \tag{13.1}$$

① 使用工具箱函数 ibbox 可以更为简单地获得边界。

其中，($p+q$)是矩的阶数。0 阶矩($p=q=0$)是

$$m_{pq}=\sum_{(u,v)\in \mathbf{I}}\mathbf{I}[u,v] \tag{13.2}$$

对于一个背景像素为 0 的二值图像来说，0 阶矩仅仅表示非 0 像素(白色)的数量——区域的面积。

图像矩可以通过工具箱函数 mpq 来计算，对于彩色目标图像的区域 5，0 阶矩是

```
>> m00 = mpq(reg5, 0, 0)
m00 =
        6004
```

它是以像素为度量单位的区域的面积。

如果把图像函数看成是一个质量分布体，则可以给图像矩一个物理解释。假想图像区域是用一片薄金属板制成的，其中每一个像素都具有单位面积和单位质量。区域的总质量是 m_{00}，质量中心或者区域形心是

$$u_c=\frac{m_{10}}{m_{00}},\quad v_c=\frac{m_{01}}{m_{00}} \tag{13.3}$$

其中 m_{10} 和 m_{01} 是一阶矩。对于我们的示例，目标区域的形心是

```
>> uc = mpq(reg5, 1, 0) / m00
uc =
  346.7539
>> vc = mpq(reg5, 0, 1) / m00
vc =
  410.4446
```

可以把它们绘制出来：

```
>> plot(uc, vc, 'gx'); plot(uc, vc, 'go');
```

如图 13.11(b)所示。

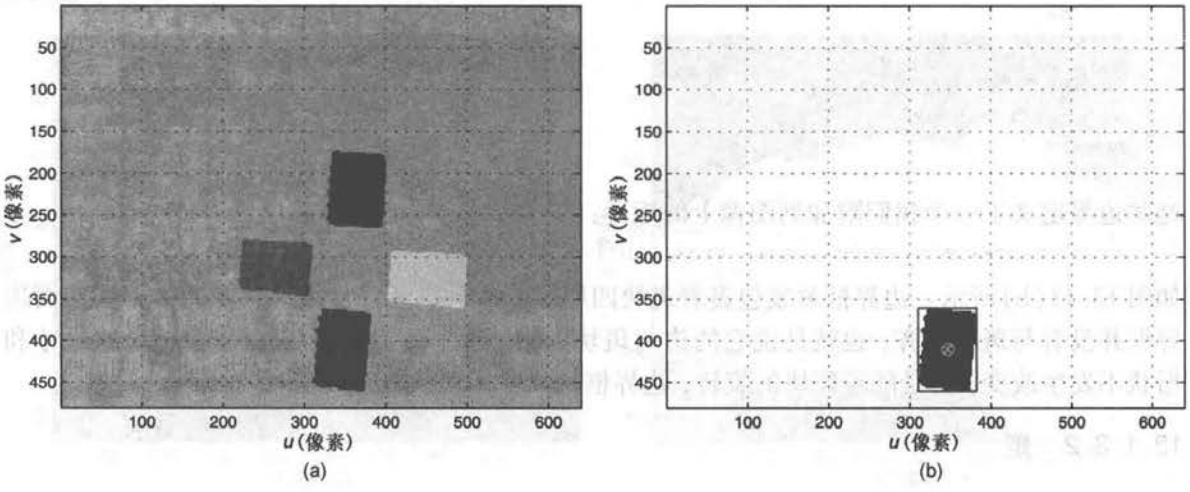


图 13.11 目标图像。(a)伪色彩显示的区域标签；(b)有边框且形心被标记的区域 1(反色，黑色为真)

中心矩 μ_{pq} 是相对于形心进行计算的：

$$\mu_{pq}=\sum_{(u,v)\in \mathbf{I}}(u-u_c)^p(v-v_c)^q\mathbf{I}[u,v] \tag{13.4}$$

它们与区域的位置无关。它们与矩 m_{pq} 的关系可由下式表示：

$$\mu_{10}=0,\quad \mu_{01}=0$$
$$\mu_{20}=m_{20}-\frac{m_{10}^2}{m_{00}},\quad \mu_{02}=m_{02}-\frac{m_{01}^2}{m_{00}},\quad \mu_{11}=m_{11}-\frac{m_{10}m_{01}}{m_{00}} \tag{13.5}$$

并可以通过工具箱函数 upq 来计算。

再一次使用薄金属板进行类比，区域的惯性矩阵是

$$\mathbf{J}=\begin{pmatrix}\mu_{20} & \mu_{11}\\ \mu_{11} & \mu_{02}\end{pmatrix} \tag{13.6}$$

其轴线与 u 轴和 v 轴平行，且相交于区域形心。二阶中心矩 μ_{20} ,μ_{02} 是惯性矩，μ_{11} 是惯性积。如果形状关于区域的轴线不对称，那么惯性积不等于 0。

等价椭圆是指和区域有相同惯性矩阵的椭圆形。对于我们的示例，有

```
>> u20 = upq(reg5, 2, 0); u02 = upq(reg5, 0, 2); u11 = upq(reg5, 1, 1);
>> J = [ u20 u11; u11 u02]
J =
   1.0e+06 *
    2.0436   -0.2329
   -0.2329    4.4181
```

J 的特征值和特征向量与椭圆的半径以及长轴和短轴的方向有关(见附录 E)。对于本示例，特征值为①

```
>> lambda = eig(J)
lambda =
   1.0e+06 *
    2.0210
    4.4408
```

它们是区域的主惯性矩。等价椭圆的最大和最小的半径分别是

$$a=2\sqrt{\frac{\lambda_2}{m_{00}}},\quad b=2\sqrt{\frac{\lambda_1}{m_{00}}} \tag{13.7}$$

其中 $\lambda_2 \geqslant \lambda_1$。在 MATLAB 中它们是②

```
>> a = 2 * sqrt(lambda(2) / m00)
a =
   54.3880
>> b = 2 * sqrt(lambda(1) / m00)
b =
   36.6904
```

度量单位是像素。它们的比值为

```
>> b/a
ans =
    0.6746
```

即区域的纵横比，它对于表征具有缩放和旋转不变性的区域形状来说是一种有用的测度。

J 的特征向量是椭圆的主轴——长轴和短轴的方向。长轴是与最大特征值对应的特征向量 v。对于我们的示例，特征向量是

```
>> [x,lambda] = eig(J);
>> x
x =
   -0.9954   -0.0962
   -0.0962    0.9954
```

① 函数 eig 以上升次序返回特征值。

② MATLAB 以上升次序返回特征值：先 λ_1 后 λ_2。

由于 MATLAB® 按升序返回特征值，所以向量 v 通常在返回的特征向量矩阵的最后一列：

```
>> v = x(:,end);
```

这个向量相对于水平轴的角度是

$$\theta = \arctan \frac{v_y}{v_x}$$

对于我们这个示例，它是

```
>> atan2( v(2), v(1) ) * 180/pi
ans =
  95.5498
```

单位是度。这个值表明等价椭圆的主轴接近于竖直。最后可以把椭圆叠加到整个区域上：

```
>> plot_ellipse(4*J/m00, [uc, vc], 'r');
```

结果如图 13.12 所示。等价椭圆的方向和纵横比有效指示了区域方向和形状。

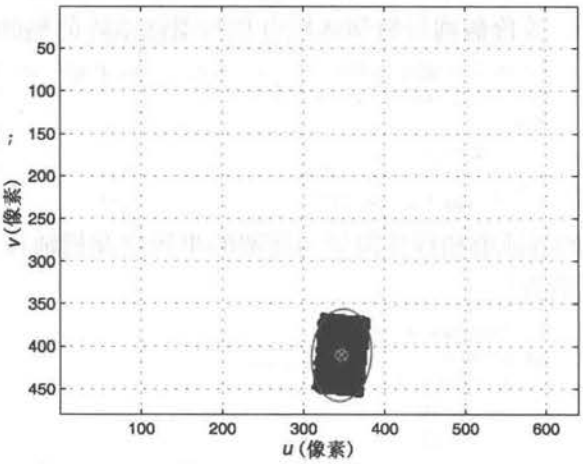


图 13.12 彩色目标图像区域 5 的等价椭圆和形心

简而言之，我们已经创建了一幅包含一个空间连续像素集合的图像，这个空间连续像素集对应于场景中我们从原始彩色图像分割出来的一个目标。我们已经确定了它的面积（即完全包围它的边框）、其形心的位置及其方向。

工具箱提供了一个更为简便的方法来完成上述步骤：

```
>> blob = imoments(reg5)
blob =
area=6005, cent=(346.8,410.4), theta=1.67, b/a=0.675
```

它会返回一个 RegionFeature 对象，该对象包含许多描述这个区域的特征，包括它的面积、它的形心、方向和纵横比（最小半径与最大半径之比）。这些值可以作为对象特征获得，例如：

```
>> blob.uc
ans =
  346.7539
>> blob.theta
ans =
    1.6677
>> blob.shape
ans =
    0.6746
```

以及零阶矩、一阶矩和二阶中心矩：

```
>> blob.moments.m00
ans =
        6005
>> blob.moments.u11
ans =
  -2.3293e+05
```

工具箱提供了一个高级函数，用来计算图像中每一个区域的特征：

```
>> f = iblobs(targets_binary)
f =
(1) area=5927, cent=(365.1,219.2), theta=-1.53, b/a=0.768, class=1,
     label=1, touch=0, parent=4
(2) area=5376, cent=(264.9,311.6), theta=0.10, b/a=0.707, class=1,
     label=2, touch=0, parent=4
(3) area=6315, cent=(452.3,327.1), theta=0.07, b/a=0.687, class=1,
     label=3, touch=0, parent=4
(4) area=283577, cent=(317.1,234.1), theta=-0.02, b/a=0.738, class=0,
     label=4, touch=1, parent=0
(5) area=6005, cent=(346.8,410.4), theta=-1.47, b/a=0.675, class=1,
     label=5, touch=0, parent=4
```

它会返回 RegionFeature 对象的一个向量 f。这种显示方法展示了区域属性的概要信息，第一列圆括号中的数字表示了向量中的区域索引。每一个 RegionFeature 对象包含了边框面积、形心、原始矩和中心矩及等价椭圆参数，这些与函数 imoments 返回的相同。但同时它还返回了一些附加的属性，如区域中像素的类型、区域标签、父区域的标签以及斑块是否接触边缘①。这个类中的一些属性示例如下：

```
>> f(1).class
ans =
     1
>> f(1).parent
ans =
     4
>> f(1).touch
ans =
     0
>> f(1).umin
ans =
   328
>> f(1).shape
ans =
    0.7677
```

RegionFeature 对象也有绘制方法，例如：

```
>> f(1).plot_box('g')
```

它会在当前的绘图窗口中用绿色覆盖特征 f(1)的边界框。其他绘制方法包括 plot_centroid 和 plot_ellipse。所有的方法都能把一个对象或者对象的一个向量添加到当前绘图中，并进行操作，例如：

```
>> f.plot_box('r:')
```

它会对 f 中所有的斑块用红色小点覆盖住它们的边界框。

children 属性是 parent 属性的逆映射。它是 RegionFeature 对象中特征向量的一个索引列表，是这个特征的子属性。例如，背景斑块 f(4)有它的子斑块：

```
>> f(4).children
ans =
     1     2     3     5
```

它们是 f(1)，f(2)，f(3)和 f(5)。

重要的是，函数 iblobs 可以执行过滤功能。对于菜园图像，我们可能预知一个番茄最大或最小尺寸这些信息，所以会在可能的区块上设置边界：

---

① 矩和面积的计算依赖于对每个像素的形状认知。在现代相机中像素是正方形的，且通常水平方向的像素比竖直方向的多。但有一些视觉系统提供正方形图像，例如 512×512，且像素的宽比它们的高要大。在这种情况下，'aspect'选项应该提供给函数 iblobs 来定义一个用像素高度比上像素宽度的不同像素纵横比。

```
>> f = iblobs(tomatoes_binary, 'area', [1000, 5000])
f =
(1) area=1496, cent=(132.5,135.2), theta=0.21, b/a=0.871, class =1,
     label=2, touch=0, parent=1
(2) area=3380, cent=(95.8,210.9), theta=-0.31, b/a=0.886, class =1,
     label=3, touch=1, parent=0
```

它会只返回面积在1000到5000像素之间的斑块。对于番茄图像，我们可能还希望只接受没有接触到边缘的斑块：

```
>> f = iblobs(tomatoes_binary, 'touch', 0)
f =
(1) area=1496, cent=(132.5,135.2), theta=0.21, b/a=0.871, class =1,
     label=2, touch=0, parent=1
```

这些过滤规则可以实行级联，例如：

```
>> f = iblobs(tomatoes_binary, 'touch', 0, 'area', [500 2000], 'class', 1);
```

一个斑块必须通过所有的规则才能被接受。其他过滤参数包括像素类型、形状和纵横比，更多详情请见网上文献。

13.1.3.3 不变性

为了辨识出特殊的目标，我们想要某一种形状测度，这种测度不会随相机与目标之间相对位姿的变化而改变，即具有不变性。在这一小节中，我们将只考虑与相机前额平行，且可以平移、旋转和缩放变化的平面目标对象。

目标的形状可以通过纵横比来非常简单地描述。纵横比是椭圆长轴与短轴的长度之比 a/b，它对于平移、旋转和缩放都是固定不变的。另一个常用的直观形状特征是圆度，可以定义为

$$\rho = \frac{4\pi m_{00}}{p^2} \tag{13.9}$$

其中，p 是区域的周长(在下一节中进行讨论)。圆度有最大值，即圆形时为 $\rho = 1$，对于方形是 $\rho = \frac{\pi}{4}$，对于一条无限长的直线来说等于0。圆度对于平移、旋转和缩放也是固定不变的。

更为复杂的矩之比也可以用于在识别平面目标时产生与位置、方向和缩放无关的不变量。例如图13.15(a)所示的图像：

```
>> im = iread('P2.png');
```

它有两个P形区域：

```
>> [f,L] = iblobs(im, 'class', 1);
>> f
f =
(1) area=1375, cent=(94.9,108.2), theta=1.98, b/a=0.648, color=1,
 label=2, touch=0, parent=1
(2) area=795, cent=(204.6,122.4), theta=2.76, b/a=0.645, color=1,
 label=4, touch=0, parent=1
```

分别是标签2和4。第二个输出参数是标签图像。不变矩是

```
>> humoments(L==2)
ans =
    0.4181    0.0292    0.0230    0.0017    0.0000    0.0003    0.0000
>> humoments(L==4)
ans =
    0.4052    0.0280    0.0210    0.0014    0.0000    0.0002    0.0000
```

我们看到，尽管两个 P 形具有不同的位置、方向和大小，但是它们的矩很接近[①]。这个形状描述符可以看作是七维空间中的一个点，与其他形状的相似性可以按照欧几里得距离进行定义。

背景材料：不变矩。归一化的矩为

$$\eta_{pq}=\frac{\mu_{pq}}{\mu_{00}^{\gamma}},\ \gamma=\frac{1}{2}(p+q)+1,\ p+q=2,3,\cdots \tag{13.8}$$

它对于平移、缩放是不变的，可以通过工具箱函数 npq 来计算。

三阶矩允许在一个平面内创建对平移、缩放和方向都保持不变的量。一组由[Hu，1962]提出的这样的矩为

$$\begin{aligned}
\phi_1&=\eta_{20}+\eta_{02}\\
\phi_2&=(\eta_{20}-\eta_{02})^2+4\eta_{11}^2\\
\phi_3&=(\eta_{30}-3\eta_{12})^2+(3\eta_{21}-\eta_{03})^2\\
\phi_4&=(\eta_{30}+\eta_{12})^2+(\eta_{21}+\eta_{03})^2\\
\phi_5&=(\eta_{30}-3\eta_{12})(\eta_{30}+\eta_{12})\left[(\eta_{30}+\eta_{12})^2-3(\eta_{21}+\eta_{03})^2\right]\\
&\quad+(3\eta_{21}-\eta_{03})(\eta_{21}+\eta_{03})\left[3(\eta_{30}+\eta_{12})^2-(\eta_{21}+\eta_{03})^2\right]\\
\phi_6&=(\eta_{20}-\eta_{02})\left[(\eta_{30}+\eta_{12})^2-(\eta_{21}+\eta_{03})^2\right]\\
&\quad+4\eta_{11}(\eta_{30}+\eta_{12})(\eta_{21}+\eta_{03})\\
\phi_7&=(3\eta_{21}-\eta_{03})(\eta_{30}+\eta_{12})\left[(\eta_{30}+\eta_{12})^2-3(\eta_{21}+\eta_{03})^2\right]\\
&\quad+(3\eta_{12}-\eta_{30})(\eta_{21}+\eta_{03})\left[3(\eta_{30}+\eta_{12})^2-(\eta_{21}+\eta_{03})^2\right]
\end{aligned}$$

它们可以通过工具箱函数 humoments 来计算。

区域特征及其不变属性的摘要信息如表 13.1 所示。

表 13.1　区域特征和它们对于相对运动的不变性。相对运动包括平移、绕目标形心的旋转和比例缩放

	平　移	旋　转	缩　放
面积	√	√	×
形心	×	√	√
纵横比	√	√	√
方向 θ	√	×	√
圆度	√	√	√
Hu 的矩	√	√	√

13.1.3.4　边界表示

一个区域也可以通过它的边界或周界形状来描述。图 13.13 显示有 3 种常用方式来表示区域的周界。一段链码是区域最外面的一系列像素，它们的中心用短线段连接起来。如果一段链码具有 4 邻接，那么连续的像素之间必须是相邻的，且周界线段的方向应为 $k\times 90°$，其中 $k\in\{0\cdots3\}$。对于有 8 邻接的链码，也称弗里曼(Freeman)链码，其周界线段的方向为 $k\times 45°$，其中 $k\in\{0\cdots7\}$。缝隙码的连接线段位于区域边缘像素与区域外界像素之间的缝隙处。这些线段的方向为 $k\times 90°$，其中 $k\in\{0\cdots3\}$。

① 事实上，这种像素数据的差异意味着不变性只能是近似的。

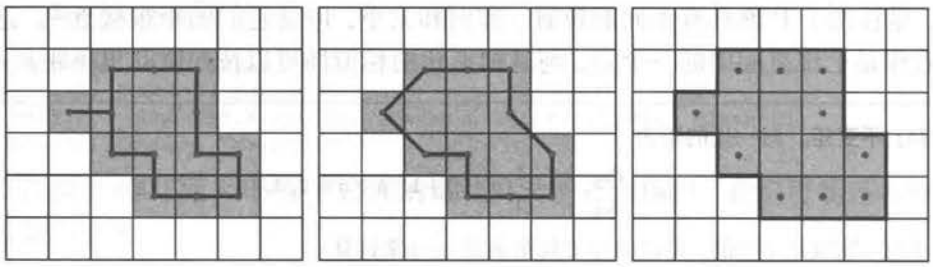


图 13.13 边界表示法，区域像素显示为灰色，周界线段显示为蓝色(本图中为黑色粗线段)，边界像素的中心由红点(本图中为深色圆点)标记。(a)有4方向的链码；(b)有8方向的弗里曼链码；(c)缝隙码。本例中的边界周长分别是14、12.2和18像素

周界可以被编码为一系列像素坐标(u_i, v_i)，或者编码为更紧凑的一个位串，其中仅用2~3位来表示每个线段的k值。这些各种各样的表示方法是等价的，且任何一种表示都可以转换成另一种。

注意，对于链码的边界，它要比区域的真实边界往里约半个像素，因此链码边界的周长变小了。这种误差对于小区域来说举足轻重。

默认情况下，工具箱不计算目标的边界①，但是如果需要可以使用'boundary'选项：

```
>> f = iblobs(tomatoes_binary, 'boundary', 'class', 1)
f =
(1) area=1434, cent=(132.9,135.8), theta=-3.10, b/a=0.852, class=1,
 label=2, touch=0, parent=1, perim=176.5, circ=0.641
(2) area=3380, cent=(95.9,210.9), theta=2.83, b/a=0.888, class=1,
 label=3, touch=1, parent=0, perim=222.1, circ=0.958
(3) area=456, cent=(11.7,228.4), theta=-2.98, b/a=0.864, class=1,
 label=4, touch=1, parent=0, perim=82.1, circ=0.951
```

我们注意到现在显示出了周长和圆度信息②。在这里我们使用了一个斑块过滤器，它只挑选像素类型1(着色番茄)的斑块。边界是一系列的边缘点，用一个矩阵表示，每一个边缘点对应矩阵的一列。在这种情况下，一共有154个边缘点：

```
>> about(f(1).edge)
 [double] : 2x154 (2464 bytes)
```

且边缘前五个点是

```
>> f(1).edge(:,1:5)
ans =
   142   142   141   140   139
   116   117   118   119   120
```

边缘可以使用对象的 plot_boundary 方法来叠加到当前图：

```
>> f(1).plot_boundary('g.')
```

在这里为一系列绿色(本书图中显示为黑色)点。绘制方法可以对一个特征向量调用：

```
>> idisp(tomatoes_binary)
>> f.plot_boundary('r.')
>> f.plot_box('g')
```

如图 13.14 所示。

---

① 因为计算量很大。

② 周长通过图 13.13(b)所示的 8 邻接链码的边界长度计算出来。注意，它比区域的实际周长稍小。

每一个目标对象都有一个外部边界，它可能包括图像的边界，对于图 13.14 中较低的两个斑块就是这样。对于有洞的目标，每一个洞有一个内部边界。但是工具箱返回的只是外部边界，因此只能把每个洞看作是目标的子区域，通过返回该子区域的外部边界得到洞的内部边界。外部边界包含所有关于区域形状的基本信息。事实上，如果一个区域没有洞且边界给定，那么使用函数 mpq_poly，upq_poly 和 npq_poly 就可以计算出它的矩。

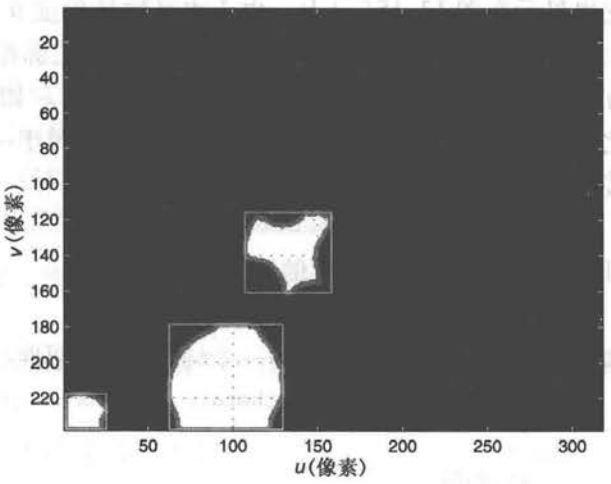


图 13.14　标记出边界框和周界的菜园图像

再一次考虑二值图像：

```
>> im = iread('P2.png');
```

如图 13.15(a)所示，它包含两个相似的形状，但是有着不同的平移、方向和大小。两个白色目标的特征是

```
>> f = iblobs( im, 'boundary', 'class', 1)
f =
(1) area=1375, cent=(94.9,108.2), theta=-1.16, b/a=0.648, class=1,
    label=2, touch=0, parent=1, perim=214.8, circ=0.417
(2) area=795, cent=(204.6,122.4), theta=-0.38, b/a=0.645, class =1,
     label=4, touch=0, parent=1, perim=162.5, circ=0.421
>> f.plot()
```

两个形状有着相似的纵横比。然而，本应该相同的圆度却是大相径庭，原因在于上文中提到的对周长的低估。面积之比是

```
>> f(2).area / f(1).area
ans =
    0.5782
```

这意味着尺寸上的减少，缩放因子是

```
>> sqrt(ans)
ans =
    0.7604
```

两个形状的相对方向是

```
>> (f(2).theta - f(1).theta) * 180/pi
ans =
   45.1267
```

单位是度。

我们可以显示两个区域相对于形心的边界点：

```
>> f(1).boundary();
```

如图 13.15(b)所示。通过可选择的返回参数，函数还提供了极坐标形式的相对于形心的边界点：

```
>> [r1,theta1] = f(1).boundary();
```

它是这个区域边界独一无二的特征：

```
>> plot(theta1, r1)
```

如图 13.15(c)所示。角度是相对于区域中最左上方的点，极坐标向量默认有 400 个元素，跨度区间为 $[-\pi,\pi)$ 弧度。区域 2 的形状轮廓为

```
>> [r2, theta2] = f(2).boundary();
```

它也显示在图 13.15(c)上。由于水平轴是角度 $\theta \in \mathbb{S}$，所以如果把该曲线图包裹在一个圆柱上，其左和右两端将会接上。我们看到一个轮廓相对于另一个发生了水平偏移(P 旋转)，竖直方向上的幅值也存在着差异(字母尺寸不同)。如果把每一个轮廓相对于其最大值进行归一化，那么它们将只会在水平偏移上有差别。对于一个轮廓相对于另一个可能发生的每一种旋转，我们计算其相似性：

```
>> [sim,scale] = boundmatch(r1, r2);
```

可以绘制出它相对于角度的图形：

```
>> plot(theta1, sim)
```

如图 13.15(d)所示。凸显的一个峰值(强相似性)表示相对旋转的程度大小：

```
>> [s,th] = peak(sim, theta1, 'npeaks', 1);
>> th * 180/pi
ans =
   41.4004
```

它与已知的旋转和先前的估计相差无几。估计的尺度比是

```
>> 1/scale
ans =
    0.7651
```

与矩相比，周界是更加丰富、更多维数的形状描述符，本例中是一个 400 维的向量。从周界中，我们可以认出直线段和曲线。

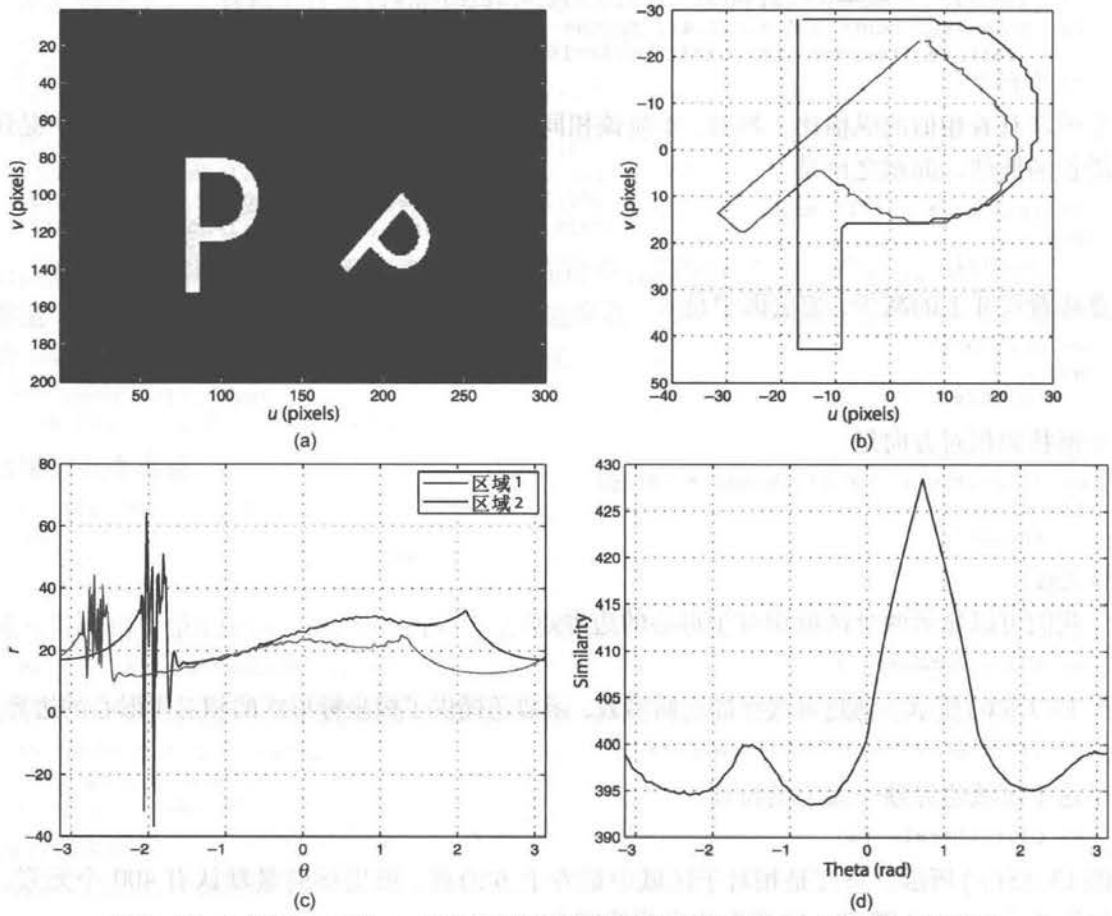


图 13.15 区域边界匹配。(a)标记目标形心的原始二值图像；(b)相对于目标形心的目标边界；(c)边界特征，极坐标 $r-\theta$；(d)两个边界特征的相关性

13.1.4　简要回顾

我们已经讨论了怎样把输入的灰度图像或彩色图像转换成场景中简明的区域描述符。对于什么才构成一个区域，其标准需要根据不同的应用而定。对于番茄采摘机器人来说，它是一个红色圆形区域；而对于一架无人机的降落，它可能会是地面上的一个黄色目标。

提出的处理流程对机器人视觉应用来说是经典的从底向上的方法，其中关键步骤如下：

1. 根据特定应用的标准，将像素进行分类，例如红色、黄色或者运动。每一个像素都被划分为一个类型 c。
2. 把同一类型的相邻像素组成集合，每一个像素被分派一个标签 s，用来表示它被分配到的集合。
3. 使用空间幅度、矩、椭圆和边界等特征来描述各个集合。

这些步骤是按从低级到高级的顺序推进的。低级操作孤立地考虑像素，而高级操作则考虑更多抽象的概念，如大小和形状。MSER 和图割算法都很强大，因为它们结合了步骤 1 和 2，并且为创建一个分割考虑了像素区域和局部不同点。

重要的是，这些步骤中没有一步是完美无缺的。第 1 步或许会出现误报情况，如小的噪声像素，我们可以通过形态学操作来消除它们，或者通过基于尺寸的连通性分析后去掉它们。第 1 步也可能会有漏报情况，例如镜面反射和遮挡可能会造成一些像素分类错误。在这种情况下，我们需要开发一些启发式方法，例如形态学处理，来填补由于反射在水果上产生的空白。另一种选择是对场景进行超分割——增加区域数量，并使用面向特定应用的知识合并相邻区域。例如，镜面反射的彩色区域可能与周围区域合并，从而建立一个对应于整个水果的区域。

另一种有些人认为更好的方法是控制照明，以便从一开始就创造出最好效果的图像。可以关掉或者遮挡住那些产生多余反射的杂散光，亦或使用偏振过滤器减弱反射。理想的照明应该是来自相机后面的漫反射照明，例如来自一个环形照明器。

对场景的预先认知往往是一个强大的工具。假定我们已知场景中包含番茄和其他植物，而实际上我们却观察到一大片不是圆形的红色区域，这时就运用我们的预先认知来推断出果实被遮挡了。因此我们可能命令机器人寻找不被遮挡的果实，然后移动到另一个可能接近被遮挡果实的位置。目标分割仍然是机器人视觉中最难的部分之一，并且没有什么好的解决办法。它需要成像的知识、基本的图像处理算法、洞察力、好的工具箱以及耐心。

13.2　直线特征

直线是一种非常明显的视觉特征，在人工环境中特别常见，例如马路、建筑物和门廊的边缘。在 12.4.1.3 节中，我们讨论了怎样使用图像强度梯度来寻找图像边缘，在本节中我们将考虑用直线段来契合这些边缘。

我们将会使用一个简单场景来展示这个原理：

```
>> im = iread('5points.png', 'double');
```

如图 13.16(a)所示。考虑这些点中的任何一个——经过这个点有无穷多条直线。如果这个点能够对这些线投票，那么每条可能经过这个点的直线都会收到一票。现在考虑另外一个类似的点，它也给所有可能经过它的直线投票。这样就有一条线(两个点都在这条线上)将会收到这两个点的投票，总共有两票，而其他的线只能收到 0 票或者 1 票。

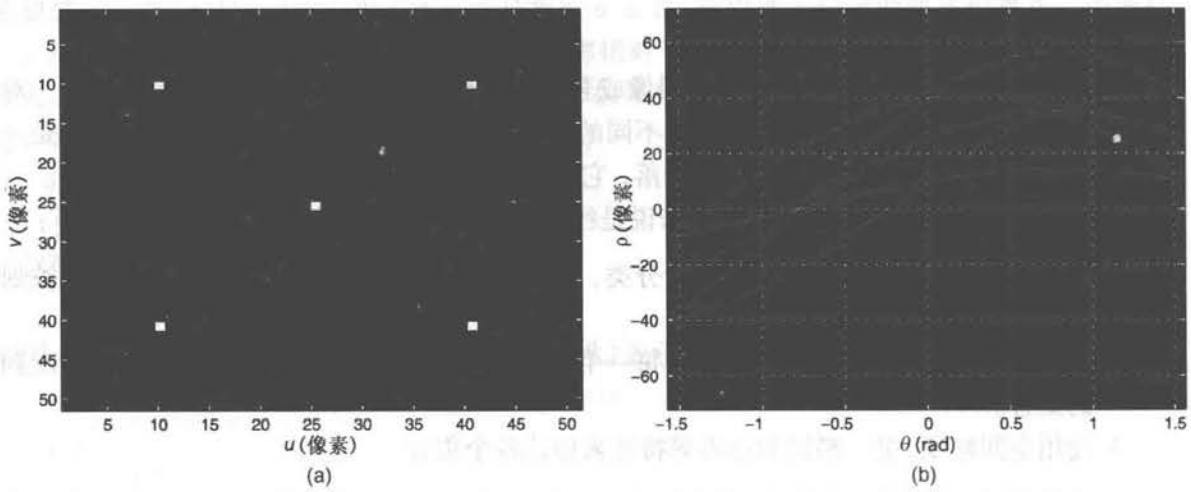


图 13.16 霍夫(Hough)变换基础。(a)定义 6 条直线的 5 个点;(b)霍夫累加数组。水平轴是角度 $\theta \in \mathbb{S}$,所以可以想象把这个图包裹在一个圆柱上,其左侧边缘和右侧边缘将会连接起来。在连接处 ρ 的符号发生了改变,所以左侧和右侧边缘上的曲线交点是等价的

我们需要用最少的参数来描述每一条线,但是标准形式 $v = mu + c$ 对于竖直线(即 $m = \infty$)是有问题的。因此通常使用 (ρ,θ) 参数来代表一条直线:

$$v = -u\tan\theta + \frac{\rho}{\cos\theta} \tag{13.10}$$

其中,$\theta \in \left[-\frac{\pi}{2},\frac{\pi}{2}\right)$ 是水平轴与该直线垂线的夹角,$\rho \in [-\rho_{\min},\rho_{\max}]$ 是原点到该直线的垂直距离,如图 13.17 所示(见彩图)。水平线的 $\theta = 0$,竖直线的 $\theta = -\frac{\pi}{2}$。因此,每一条直线可以看作是二维空间中的一个点 (ρ,θ),该二维空间代表了所有可能直线。

事实上,我们无法考虑经过每一个点的无数条直线,所以只考虑有限集合中的直线。把 $\theta\rho$ 空间量化,并用一个相应的 $N_\theta \times N_\rho$ 数组 A 来记录票数,A 被称为累加数组。对于一幅 $W \times H$ 输入图像,有

$$\rho_{\max} = -\rho_{\min} = \sqrt{W^2 + H^2}$$

A 有 N_ρ 个元素位于区间 $\rho \in [-\rho_{\min},\rho_{\max}]$,还有 N_θ 个元素位于区间 $\theta \in \left[-\frac{\pi}{2},\frac{\pi}{2}\right)$。数组的索引是整数 $(i,j) \subset \mathbf{Z}^2$,因此有

$$i \in [1,N_\theta] \mapsto \theta \in [-\tfrac{\pi}{2},\tfrac{\pi}{2})$$
$$j \in [1,N_\rho] \mapsto \rho \in [-\rho_{\max},\rho_{\max}]$$

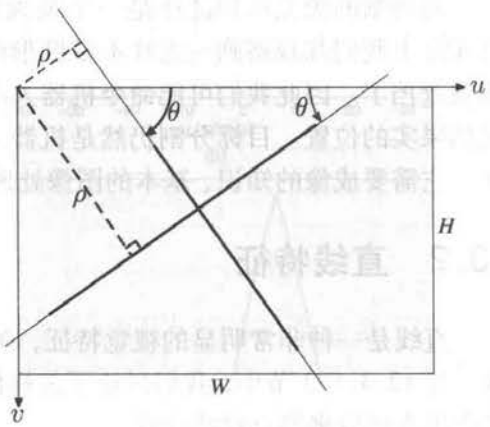


图 13.17 两条线段的 (θ,ρ) 参数化。正值显示为蓝色,负值显示为红色

边缘点 (u,v) 将投票给所有满足方程(13.10)的直线,它包括所有的 (i,j) 对,满足

$$\rho = u\sin\theta + v\cos\theta \tag{13.11}$$

并且元素 $A[i,j]$ 都增大了。对于每一个 $i \in [1, N_\theta]$ 都可以计算出相应的 θ 值,然后根据等式(13.11)计算 ρ,再把 ρ 映射到相应的整数 j。

在最后的过程中，$\boldsymbol{A}$ 中获得最多投票的这些元素对应于场景中的主导线。如图 13.16(a)中的示例，产生的累加数组如图 13.16(b)所示。数组的大多数元素获得 0 票(黑色)，红色曲线(即本书图中反白曲线)是获得 1 票的轨迹，对应于五个输入点中的每一个点。这些曲线交叉，这些点对应于获得多于 1 票的直线。我们看到两两曲线交叉的 4 个位置，说明其单元格有两票，这些位置对应于图 13.16(a)中连接外边 4 个点的直线。水平轴代表角度 $\theta \in \mathbb{S}$，所以左侧和右侧边缘可以连接在一起，d 改变符号——在数组左侧和右侧边缘上的曲线交叉点是等价的。我们也可以看到有三条曲线交叉的两个位置，说明其单元格有三票，这些位置对应于图 13.16(a)中穿过中间点的对角线。这种技术称为霍夫变换。

考虑更为复杂的例子，一个实正方形绕逆时针方向旋转 0.3 rad：

```
>> im = testpattern('squares', 256, 256, 128);
>> im = irotate(im, -0.3);
```

可以计算其边界点：

```
>> edges = icanny(im);
```

如图 13.18(a)所示。霍夫变换可以通过以下函数计算

```
>> h = Hough(edges)
Hough: nd=401, ntheta=400, interp=3x3, distance=1
```

并返回一个 Hough 类的实例。它的属性包括二维选票累加数组 A，其中行方向有 nd 个元素，列方向有 ntheta 个元素。默认情况下，$\theta\rho$ 平面被量化为 401 × 400 个方格[①]。累加数组可以被显示为一幅图像：

```
>> h.show();
```

如图 13.18(b)所示。4 个亮点对应于输入图像的主要边缘。我们也可以看到许多其他的线也收到了一小部分选票。

下一步是找到累加数组的峰值：

```
>> lines = h.lines()
lines =
theta=-1.27224, rho=-19.9157, strength=1
theta=-1.28025, rho=-150.009, strength=0.944099
theta=-1.25704, rho=-146.514, strength=0.801242
theta=0.298531, rho=224.022, strength=0.757764
theta=0.306266, rho=95.7803, strength=0.745342
theta=0.28664, rho=93.9642, strength=0.732919
theta=-1.24916, rho=-16.353, strength=0.68323
theta=0.279094, rho=222.237, strength=0.627329
theta=-1.29124, rho=-22.7355, strength=0.614907
theta=-1.29922, rho=-152.765, strength=0.565217
theta=0.322013, rho=225.884, strength=0.515528
```

它返回 LineFeature 对象的一个向量，其中包括得票数靠前的一些直线，以及这些直线的票数(票数已经相对于最多选票进行了归一化)。如果调用函数时没有输出参数，那么被识别的峰值将被显示在一张累加器选票强度图像上。

注意，尽管目标只有 4 条边，但是在累加数组中的峰值超过了 4 个。我们也注意到第一个和第四个峰值有非常相似的直线参数，累加器这个区域的更多细节显示在图 13.18(c)中。我们看到若干个紧密相连的亮点(高票数)，这是由于量化的效应。在第 285 页上讨论的峰值尺度概念可以应用在这里，我们又一次应用非局部最大值抑制来消除最大值周围的小峰值：

---

① ρ 关于零点对称，所以包括 0 点在内，它有奇数个元素。θ 范围是 $\left[-\frac{\pi}{2}, \frac{\pi}{2}\right)$，它关于零点不对称，有偶数个元素。

```
>> h = Hough(edges, 'suppress', 5)
h =
Hough: nd=401, ntheta=400, interp=3x3, distance=5
```

在本例中，距离是5个累加器单元——最大值抑制了5个单元半径内较小的局部最大值。这会导致只剩下4个峰值：

```
>> lines = h.lines()
lines =
theta=-1.27224, rho=-19.9157, strength=1
theta=-1.28025, rho=-150.009, strength=0.944099
theta=0.298531, rho=224.022, strength=0.757764
theta=0.306266, rho=95.7803, strength=0.745342
```

它们对应于目标的4条边。

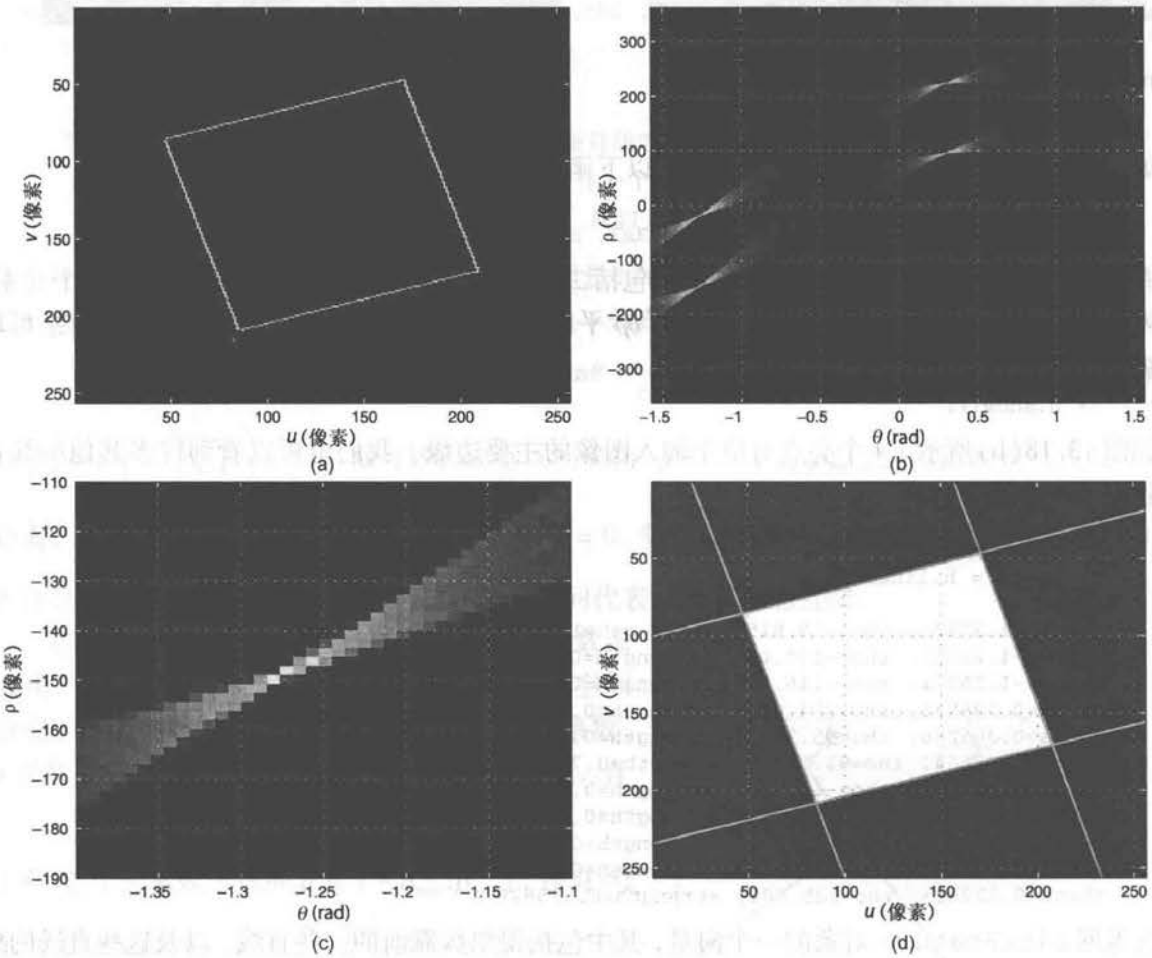


图 13.18 旋转正方形的霍夫变换。(a)边缘图像；(b)霍夫累加器；(c)霍夫累加器特写；(d)叠加在原始图像上的估计直线

由于直线参数被量化，方法 lines 使用插值法来提取局部峰值(见附录K)。默认情况下，插值在以局部选票最大值为中心的3×3窗口中执行。一旦峰值被找到，被抑制范围内的其他所有票数均为0，以便消除任何一个接近的最大值。这个过程可以对选票数组中的所有选定峰值反复使用，其中峰值选择的依据是要超过最大峰值的指定分数①。

① 没有指定参数时，所有大于'houghThresh'的峰值全部会显示出来。默认的'houghThresh'为0.5，但是可以由函数 Hough 的选项'houghThresh'来设定。

我们把检测到的直线投影到原始图像上：

```
>> idisp(im);
>> h.plot('b')
```

结果如图 13.18(d)所示。

一个真实的图像示例为

```
>> im = iread('church.png', 'grey', 'double');
>> edges = icanny(im);
>> h = Hough(edges, 'suppress', 5);
>> lines = h.lines();
```

其中最强的 10 条直线是：

```
>> idisp(im);
>> lines(1:10).plot();
```

如图 13.19 所示(见彩图)。图像中许多很强的直线已被找到，但是对于尖塔上窗户边缘产生的那些竖直线，由于在较短长度上存在角度量化误差，所以不是很垂直。

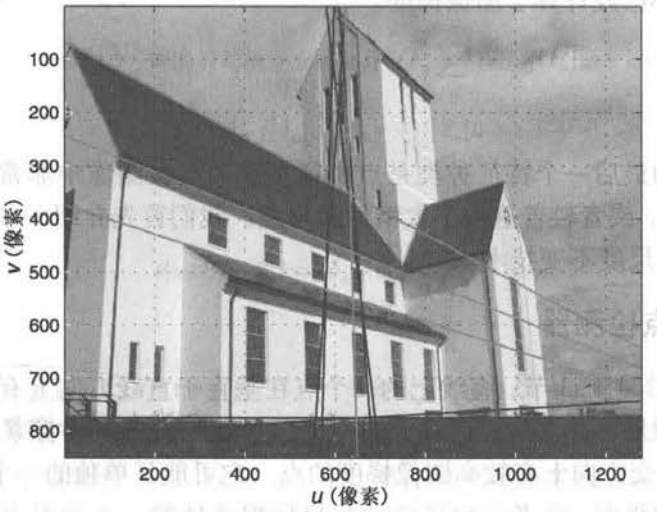


图 13.19　真实图像的霍夫变换。蓝色线对应于 10 条最强的选票峰值。叠加的绿色线是边缘段长度至少为100像素的那些直线

判断一条边缘是否重要的另一种测度可以通过把直线再投影到边缘图像上，然后对直线上连续的边缘像素点计数来获得：

```
>> lines = lines.seglength(edges);
```

它会返回 LineFeature 对象的一个向量，与 lines 方法返回的相似，但是属性 length 被设置为最大边缘段长度：

```
>> lines(1)
ans =
theta=0.000638093, rho=775.415, strength=1, length=47
```

本例中为 47 像素。边缘段被定义为一个几乎完全连续的边缘像素组合，其中像素缺口不超过 5 个(默认值)像素。然后我们选出那些边缘段长度大于 100 像素所对应的所有霍夫峰值：

```
>> k = find( lines.length > 100);
```

之后用绿色突出显示那些直线：

```
>> lines(k).plot('g')
```

如图 13.19 所示。3 条非垂直线聚合在一个远方的消失点。

霍夫变换的原理非常优雅，实际应用时它可能效果很好，也可能相当差。当场景包含很多纹理或者模糊不清的边缘时，它的执行效果会比较差。纹理会引发投票在累加数组上广泛分布，但没有一致性，这样容易掩盖住真正的峰值。因此对于边缘检测器的参数和霍夫峰值检测器中的各种阈值，需要做很多试验来选择。函数 Hough 有很多选项，在网上文献中对它们有详细的描述。每一个边缘点的投票可以是 1，或者用那个点的边缘强度，以强调更强的边缘。那些小于 edgeThresh 与最大边缘强度乘积的边缘强度都被认为是 0。Hough 对象也可以通过一个边缘坐标数组和选票来构建，在这种情况下，第一个参数是一个 (u,v) 坐标的 $2 \times N$ 矩阵，每列对应一个坐标，且有相同的票数。另一种方法是把输入改为一个 $3 \times N$ 矩阵，它的第三行是投票的强度。

霍夫变换通过把一条直线契合到边缘像素上，从而估计出直线的方向。这个方法忽略了我们在第 299 页上讨论过的有关每个像素点边缘方向的大量信息。由于没有使用我们享有的所有信息，导致最终的估计准确性降低。使用每个像素点的方向并不会增加额外负担，因为我们为了估计边缘大小已经计算了图像梯度。

13.3 点特征

我们将要讨论的最后一个特征种类是点特征。这些点在图像中非常明显，通常被称为兴趣点、显点、关键点，或者较普遍但不太精确的角点。我们首先介绍一些寻找兴趣点的传统技术，然后讨论较新的尺度不变技术。

13.3.1 经典角点检测器

我们回顾一下 12.4.1.3 节，直线上的一个点在垂直于直线方向上有最强的梯度。然而沿着直线方向的梯度较低，这意味着直线上的一个像素点与它邻近的像素点看起来相似。与之相反，兴趣点是在正交方向上有较高图像梯度的点。它可能是单独的一个像素，其强度明显区别于周围所有像素的强度，或者它也可能就是目标拐角处的一个像素点。由于角点的特征非常明显，即使在同一场景的不同视角图像中它们被检测到的可靠性也很高，因此它们是多视角技术的关键，例如我们将在下一章讨论的立体视觉和运动估计。

最早的角点检测器是莫拉维克(Moravec)兴趣算子，之所以这样命名是因为它检出的场景点是从追踪角度看非常有趣的一个点。这个兴趣算子基于一种直觉，即如果一个图像区域 W 明确无误地位于另一幅图像中，那么它与所有重叠的邻近区域必须存在显著不同。对于一个以 (u,v) 为中心的区域和一个相对平移了 (δ_u,δ_v) 的邻近区域，莫拉维克定义了它们之间的相似性：

$$s(u,v,\delta_u,\delta_v)=\sum_{(i,j)\in\mathcal{W}}\left(I[u+\delta_u+i,v+\delta_v+j]-I[u+i,v+j]\right)^2 \tag{13.12}$$

其中，$\mathcal{W}$ 是某个局部图像区域，通常是一个 $N \times N$ 的正方形窗口。这就是我们前面讨论过的表 12.1 中的 SSD 相似性测度。分别针对 8 个基本方向上的平移 $(\delta_u,\delta_v) \in \mathcal{D}$ 进行了相似性评估，而其中的最小值就是兴趣测度：[①]

$$C_M(u,v)=\min_{(\delta_u,\delta_v)\in\mathcal{D}} s(u,v,\delta_u,\delta_v) \tag{13.13}$$

① N,NE,E,…,W,NW 或者 $i,j \in \{-1,0,1\}$。

函数 $C_M(\cdot)$ 针对图像中每一个像素进行了评估，兴趣点是那些 C_M 值大的点。莫拉维克检测器主要的局限在于它是各向异性的，因为它仅在有限数量的方向上检测了图像的变化(即梯度)。所以该检测器对于直线上的一个点给出了强的输出，这是我们不想要的。

通过把相似性定义为图像区域和平移后区域之间的方差加权和，我们可以一般化上述方法：

$$s(u,v,\delta_u,\delta_v)=\sum_{(i,j)\in\mathcal{W}} W[i,j]\Big(\underbrace{I[u+\delta_u+i,v+\delta_v+j]}-I[u+i,v+j]\Big)^2$$

其中，$\boldsymbol{W}$ 是加权矩阵，它更强调靠近窗口 $\mathcal{W}$ 中心的点。式中指示项可以由一个截断泰勒级数来近似：

$$I[u+\delta_u+j,v+\delta_v+i]\approx I[u+i,v+j]+I_u[u+i,v+j]\delta_u+I_v[u+i,v+j]\delta_v$$

其中，$\boldsymbol{I}_u$ 和 $\boldsymbol{I}_v$ 分别是水平方向和垂直方向的图像梯度。现在可以写出

$$\begin{aligned}s(u,v,\delta_u,\delta_v)&=\sum_{(i,j)\in\mathcal{W}} W[i,j]\big(I_u[u+i,v+j]\delta_u+I_v[u+i,v+j]\delta_v\big)^2\\&=\delta_u^2\sum_{(i,j)\in\mathcal{W}} W[i,j]I_u^2[u+i,v+j]+\delta_v^2\sum_{(i,j)\in\mathcal{W}} W[i,j]I_v^2[u+i,v+j]\\&\quad+2\delta_u\delta_v\sum_{(i,j)\in\mathcal{W}} W[i,j]I_u[u+i,v+j]I_v[u+i,v+j]\end{aligned}$$

可以简写为

$$s(u,v,\delta_u,\delta_v)=(\delta_u\quad\delta_v)\boldsymbol{A}\begin{pmatrix}\delta_u\\\delta_v\end{pmatrix}$$

其中，

$$\boldsymbol{A}=\begin{pmatrix}\Sigma W[i,j]I_u^2[u+i,v+j] & \Sigma W[i,j]I_u[u+i,v+j]I_v[u+i,v+j]\\\Sigma W[i,j]I_u[u+i,v+j]I_v[u+i,v+j] & \Sigma W[i,j]I_v^2[u+i,v+j]\end{pmatrix}$$

如果加权矩阵是一个高斯核 $\boldsymbol{W}=\boldsymbol{G}(\sigma_I)$，并且用互相关代替求和，那么

$$\boldsymbol{A}=\begin{pmatrix}G(\sigma_I)\otimes I_u^2 & G(\sigma_I)\otimes I_uI_v\\G(\sigma_I)\otimes I_uI_v & G(\sigma_I)\otimes I_v^2\end{pmatrix}\tag{13.14}$$

它是一个对称的 2 × 2 矩阵，有各种不同的名称：结构张量、自相关矩阵或者二阶矩矩阵。这个矩阵获取了局部邻域的强度结构，并且它的特征值提供了一个对邻域的旋转不变性描述。$\boldsymbol{A}$ 矩阵中的元素是先用图像梯度值进行平方或相乘，然后再通过加权矩阵平滑处理后计算得到的。这样减少了噪声，并提高了检测器的稳定性和可靠性。梯度图像 $\boldsymbol{I}_u$ 和 $\boldsymbol{I}_v$ 通常使用带有一个平滑参数 σ_D 的高斯核差分方法(12.4.1.3 节)进行计算。

兴趣点 (u,v) 是对于向量 (δ_u,δ_v) 的所有方向 $s(\cdot)$ 都高的一个点。就是说无论我们朝哪个方向移动这个窗口，它都会迅速变得与原区域不同。如果把原始图像 $\boldsymbol{I}$ 看成是一个表面，那么 $\boldsymbol{A}$ 的特征值就是在那一点表面的主曲率。如果两个特征值都很小，那么表面就是平的，即图像区域有几乎不变的局部强度。如果有一个特征值高，另一个低，那么表面呈山脊形状，表示有一个边缘。如果两个特征值都高，那么表面急剧凸起，我们认为它是一个角点①。

Shi-Tomasi 角点检测器把角点的强度(或角点量)看作最小特征值：

① 回想一下，图像压缩去除了图像中的高频细节，而这正是对一个角点的定义。理想情况下，角点检测器应该被用于还没有被压缩和已经解压缩的图像上。

$$C_{ST}(u,v) = \min(\lambda_1, \lambda_2) \tag{13.15}$$

其中 λ_i 是 $\boldsymbol{A}$ 的特征值。图像中这个测度值高的点被称为“好的跟踪特征”。Harris 检测器也是基于同样的认识，但是把角点量定义为①

$$C_H(u,v) = \det(\boldsymbol{A}) - k\,\mathrm{tr}^2(\boldsymbol{A}) \tag{13.16}$$

如果算得一个大的值，就代表很强很突出的角点。由于 $\det(\boldsymbol{A}) = \lambda_1\lambda_2$ 且 $\mathrm{tr}(\boldsymbol{A}) = \lambda_1 + \lambda_2$，所以当两个特征值都大时，Harris 检测器才会响应。而且该检测器有效避免了 $\boldsymbol{A}$ 矩阵特征值的计算，毕竟其计算量还是比较大的②。通常使用的 k 值为 0.4。Harris 检测器的另一个变体是 Noble 检测器：

$$C_N(u,v) = \frac{\det(\boldsymbol{A})}{\mathrm{tr}(\boldsymbol{A})} \tag{13.17}$$

它在计算方法上更简单，但存在奇异的可能性。

通常针对图像中的每一个像素进行角点强度计算，结果就是一幅角点强度图像。然后使用非局部最大值抑制，只保留那些比它们附近像素都大的值。得到一系列这样的局部最大点，然后按角点强度对它们进行降序排列。用一个阈值来筛选出那些高于特定强度的角点，或那些高于最强角点特定分数值的角点，抑或直接选出最强的前 N 个角点。

工具箱提供了 Harris 角点检测器，我们使用一幅真实图像来展示它的使用：

```
>> b1 = iread('building2-1.png', 'grey', 'double');
>> idisp(b1)
```

Harris 特征通过以下函数来计算：

```
>> C = icorner(b1, 'nfeat', 200);
7712 corners found (0.8%),  200 corner features saved
```

它会返回一个 PointFeature 对象的向量。检测器找到了 7000 多个角点，它们都是角点强度图像中的局部极大值，占图像中总像素量的 0.8%。在这里，我们需要 200 个最强的角点。这个向量中包含的角点是按角点强度降序排列的，并且每一个 PointFeature 对象中都包含角点坐标 (u,v)、角点强度和一个描述符，其中描述符是以向量形式包括了结构张量中的独立元素 $(\boldsymbol{A}_{11}, \boldsymbol{A}_{22}, \boldsymbol{A}_{33})$。这个描述符也可以作为角点的一个简单特征来使用，以帮助匹配不同视角之间的相应角点。

我们可以把角点表示为白色正方形并叠加到图像上：

```
>> idisp(b1, 'dark');
>> C.plot('ws');
```

如图 13.20(a)所示。函数 idisp 的'dark'选项减少了图像的量度，以使叠加的角点标记更加清晰可见。局部特写见图 13.20(b)，可以看到这些特征点的确位于目标的拐角处。

背景材料：另一种确定图像曲率的方法是使用 Hessian 行列式(DoH)。Hessian 是一个点的二阶梯度矩阵：

$$\boldsymbol{H} = \begin{pmatrix} I_{uu} & I_{uv} \\ I_{uv} & I_{vv} \end{pmatrix}$$

其中，$I_{uu} = \partial^2 I/\partial u^2$，$I_{vv} = \partial^2 I/\partial v^2$，$I_{uv} = \partial^2 I/\partial u\partial v$。当在两个方向上有灰度变化时，其行列式 $\det(H)$ 值大。然而二阶导数使得图像噪声增强，甚至比一阶导数还要大，因此图像首先必须平滑处理。

---

① 有时候在文献中也称作 Plessey 角点检测器。

② 求一个 2×2 矩阵的特征值涉及求解一个二次方程，所以需要平方根运算。

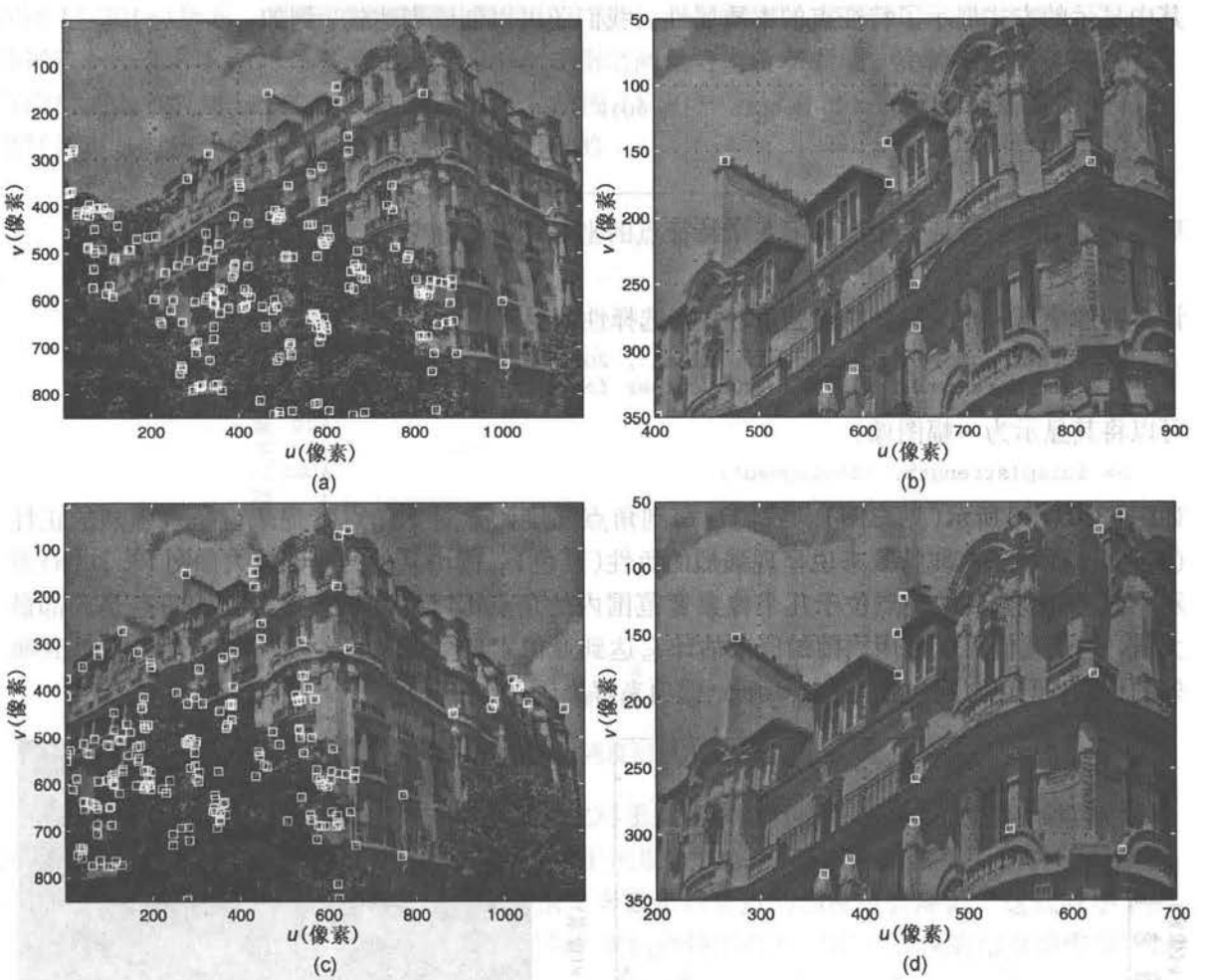


图 13.20　应用于相同建筑物两种视角的 Harris 角点检测器。(a)视图 1；(b)局部放大视图 1；(c)视图 2；(d)局部放大视图2。注意，两个视图中相当多的被检测到的角点都位于建筑物上相同的特征点

我们也看到角点有不均匀聚集的倾向，在高对比度和纹理突出的区域比较密集。这种现象对于某些应用会引发一些问题。为了更加均匀地分散角点，我们可以通过指定角点之间的最小距离来增加用于局部最大值抑制的距离：

```
>> C1 = icorner(b1, 'nfeat', 200, 'suppress', 10);
7422 corners found (0.7%),  200 corner features saved
```

本例中，最小距离为 10 个像素。

对于包含 PointFeature 对象的向量，可以使用标准的 MATLAB® 操作和指令，例如：

```
>> length(C)
ans =
    200
```

然后索引：

```
>> C(1:4)
ans =
  (3,3), strength=0.00760968, descrip=(0.112697  0.112524 0.0551586)
  (600,662), strength=0.0054555, descrip=(0.0910059   0.0716812 -0.00303761)
  (24,277), strength=0.0039721, descrip=(0.0923745 0.0577764 -0.021521)
  (54,407), strength=0.00393328, descrip=(0.0998847  0.0482311 0.00259472)
```

其中显示的方式展示了特征点的本质属性。我们还可以创建表达式，例如：

```
>> C(1:5).strength
ans =
    0.0076    0.0055    0.0040    0.0039    0.0038
>> C(1).u
ans =
     3
```

要绘制前 100 个特征点中间每隔 5 个特征点的坐标，其表达式为

```
>> C(1:5:100).plot()
```

计算图像中每一个像素的角点强度，并有选择性地返回：

```
>> [C,strength] = icorner(b1, 'nfeat', 200);
7422 corners found (0.7%),  200 corner features saved
```

可以将其显示为一幅图像：

```
>> idisp(strength, 'invsigned')
```

如图 13.21(a)所示(见彩图)。我们观察到角点强度函数对于角点特征来说呈现强烈的正性(蓝色)，而对于直线特征来说呈现强烈的负性(红色)。图中某处的局部放大如图 13.21(b)所示，它表明检测到的角点位于几个像素宽范围内的角点山峰的顶端。检测到的角点是局部最大值，但是我们可以使用周围的值来估计它达到亚像素精度的位置(见附录 K)。这牵涉到额外的计算，但可以通过使用'interp'选项来解决。

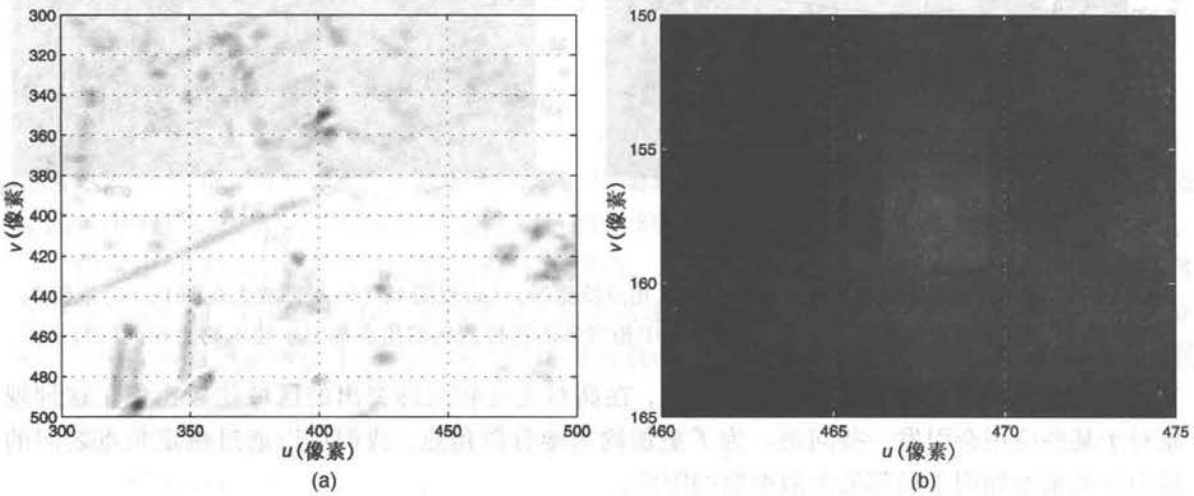


图 13.21 Harris 角点强度。(a)显示为一幅图像的角点强度放大视图(蓝色为正，红色为负)；(b)显示检出特征点的角点强度放大视图(亮度与强度成比例)

检出角点强度的累积直方图为

```
>>  ihist(strength, 'normcdf')
```

如图 13.22 所示。最强的角点有 $C_H \approx 0.007$，95% 以上的角点超过了这个值的一半。

下面考虑另一张从不同位置拍摄的同一建筑物的图像：

```
>> b2 = iread('building2-2.png', 'grey', 'double');
```

检测到的角点是

```
>> C2 = icorner(b2,  'nfeat', 200);
7666 corners found (0.8%),  200 corner features saved
>> idisp(b2,'dark')
>> C2.plot('ws');
```

如图 13.20(c)所示。对于许多机器人视觉中的应用——如跟踪、拼接和下一章将要讨论的立体视觉——非常重要的一项技术是无论两个视图之间是否发生了照明、旋转或放缩这些变化，都能检测出对应同一个空间点的角点特征。从图 13.20 中可以看出，在两个视图中的确有很多特征点是对应于实际建筑物上的同一空间点的。

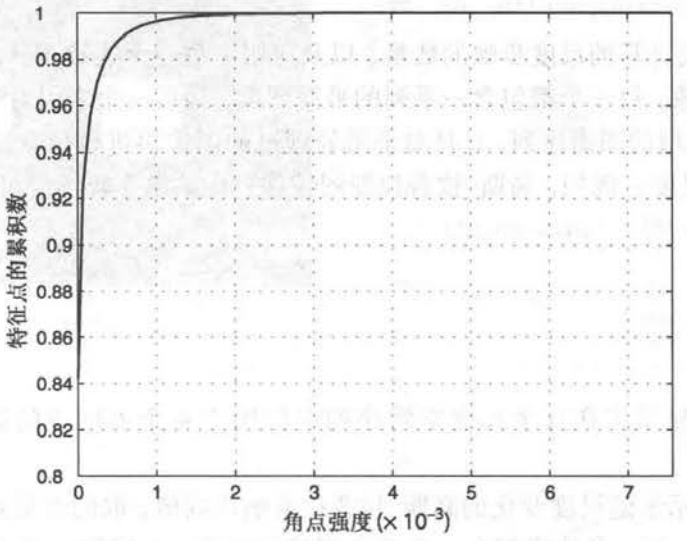


图 13.22　正角点强度的累积直方图

Harris 检测器是基于图像梯度计算的，因此对于照度偏移具有鲁棒性，且结构张量 $\boldsymbol{A}$ 的特征值对于旋转也具有不变性。然而，检测器对于尺度上的变化却不是一成不变的。随着我们放大图像，角点周围的梯度会降低，并且周围更多数量像素点的强度也会降低。这就减小了图像的曲率，也降低了角点强度。针对上述问题，我们要使用具有尺度不变性的角点检测器，该内容将在下一节讨论。

对于彩色图像来说，可以使用每个独立颜色平面的梯度图像来计算结构张量，但它与第一次根据方程(10.10)把图像转换成灰度稍有不同。事实上，颜色的使用出乎我们的意料——除了计算量猛增之外，它对大多数场景几乎没有产生什么不同。函数 icorner 接受很多的选项：k、导数核和平滑化核的大小 σ_D 和 σ_I、绝对和\或相对的角点强度阈值、以及角点间的规定最小距离。选项'st'和'noble'允许分别用方程(13.15)和方程(13.17)来计算角点测度。有关细节可参见网上文献。

13.3.2　尺度空间角点检测器

上一节介绍的 Harris 角点检测器实际应用效果很好，其不足是对于尺度上的变化应对较差。遗憾的是，由于相机与场景之间距离改变或者变焦操作，尺度变化在很多实际应用中很常见。我们也注意到 Harris 检测器对细密纹理结构的响应很好，例如图 13.20 中的树叶，但是我们也希望能够检测出与大尺度场景结构相关联的特征。

图 13.23(见彩图)展示了尺度空间特征检测的基本原理。首先我们加载一幅人工合成图像：

```
>> im = iread('scale-space.png', 'double');
```

如图 13.23(a)所示。图像中包含 4 个不同尺寸的正方形：5×5、9×9、17×17 和 33×33。尺

度空间序列通过使用 σ 渐增的高斯核来计算，σ 渐增会导致区域越来越模糊，小的区域逐渐从视图中消失。在这个序列中的每一步，高斯平滑处理后的图像都与拉普拉斯核方程(12.3)进行互相关运算，这将使明亮的斑块产生很强的负反应①。

在工具箱中我们通过使用以下函数计算尺度空间序列：

```
>> [G,L,s] = iscalespace(im, 60, 2);
```

其中，输入参数是要计算的尺度步骤的数量，以及应用于每一个连续步骤的高斯核的 σ。函数返回两个三维的图像，每一个都包含一系列的平面图像，最后一个索引对应尺度。G 是对图像 im 平滑处理程度递增的图像序列，L 是对平滑处理过的图像再进行拉普拉斯操作得到的图像序列，s 是相应的尺度。例如，高斯-拉普拉斯图像序列中的第 5 幅图像可以显示为

```
>> idisp(L(:,:,5), 'invsigned')
```

它的尺度是

```
>> s(5)
ans =
    4.0311
```

图 13.23(b)～(e)显示了在这个尺度空间序列中作用于 4 个不同点的高斯-拉普拉斯操作结果。

图 13.23(f)显示了随尺度变化的高斯-拉普拉斯响应幅值，取的点是对应于输入图像中每一个正方形的中心。每一条曲线都有一个意义明确的峰值，与峰值相关联的尺度与区域的大小成比例。这就是区域的特征尺度。

如果我们把三维图像 L 看作是一本画册，那么尺度空间特征点就是其中 3D 最大值的一个像素点，也就是比它三个维度上的 26 个邻近点都大的元素——邻近点是在当前尺度、上一尺度和下一尺度上的空间邻近点。这样一些点可以通过函数 iscalemax 检测出来：

```
>> f = iscalemax(L, s)
f =
  (64,64), scale=2.91548, strength=1.96449
  (128,64), scale=4.06202, strength=1.72512
  (128,128), scale=18.1246, strength=1.54391
  (64,128), scale=8.97218, strength=1.54057
  (96,128), scale=15.5081, strength=0.345028
  (97,128), scale=14.7139, strength=0.34459
```

它会返回一个 ScalePointFeature 对象的数组，而 ScalePointFeature 对象是 PointFeature 对象的子类。每一个对象中都包含特征点的坐标、强度和尺度属性。这些特征点按照强度递减的顺序排列，我们可以看到 4 个具有较大强度的特征对应于 4 个白色物体。我们可以在原始图像中叠加这些检出特征：

```
>> idisp(im)
>> f(1:4).plot('g*')
```

结果如图 13.24 所示。

与峰值相关联的尺度可以很容易地使用半径等于特征尺度的圆来直观显示：

```
>> f(1:4).plot_scale('r')
```

结果也如图 13.24 所示。我们看到辨识出的特征点位于每一个目标的中心，特征尺度与目标的大小有关。圆内区域称为特征的支持区域。

---

① 我们实际上计算了高斯近似与高斯-拉普拉斯算子的差别，如图 13.26 所示。

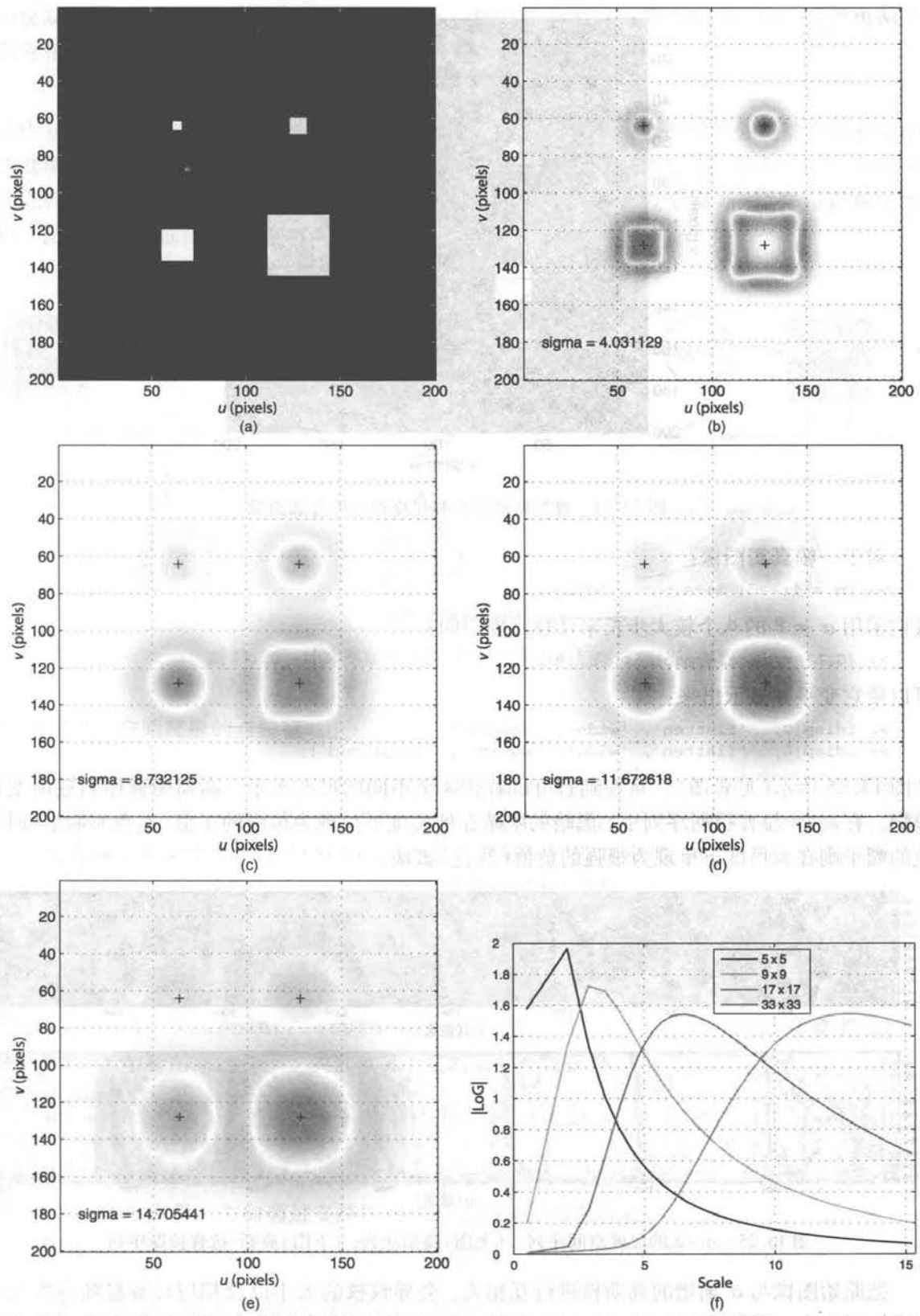


图 13.23　空间尺度示例。(a)具有方块大小分别为 5×5、9×9、17×17 和 33×33 的合成图像 I, ; (b)~(e)对于渐增的尺度值,归一化的高斯-拉普拉斯操作 $\sigma^2 L \otimes G(\sigma) \otimes I$,$\sigma$ 值显示在左下角。使用了伪色彩显示:红色代表负、蓝色代表正;(f)每个正方形中心处(由"+"指示)随 σ 变化的高斯-拉普拉斯操作的幅值

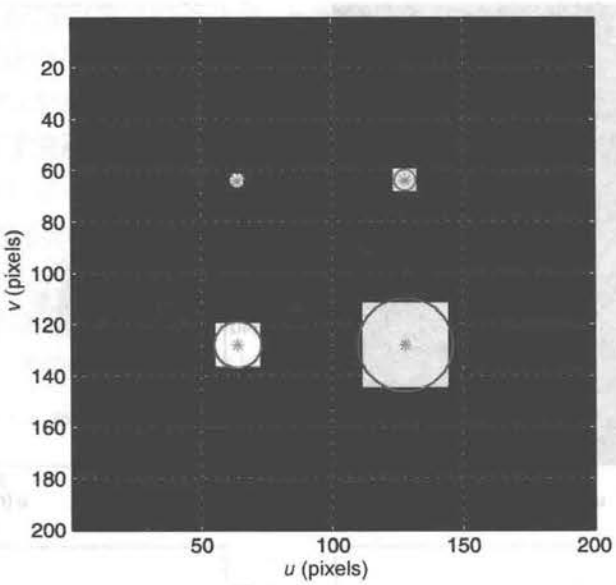


图 13.24 叠加特征中心和尺度指示的合成图像

对于一幅真实图像：

```
>> im = iread('lena.pgm', 'double');
```

我们采用 $\sigma = 8$ 的八个较大步长来计算尺度空间：

```
>> [G,L] = iscalespace(im, 8, 8);
```

可以使它变平并显示出来：

```
>> idisp(G,  'flatten', 'wide', 'square');
>> idisp(L,  'flatten', 'wide', 'square', 'invsigned');
```

如图 13.25 所示(见彩图)。从左到右可以看到 8 个不同的尺度水平。高斯图像序列逐渐变得模糊。在高斯-拉普拉斯序列中，黑暗的眼睛在低尺度下呈现为很强的正值(蓝色)斑块，而浅色的帽子则在大尺度下呈现为很强的负值(红色)斑块。

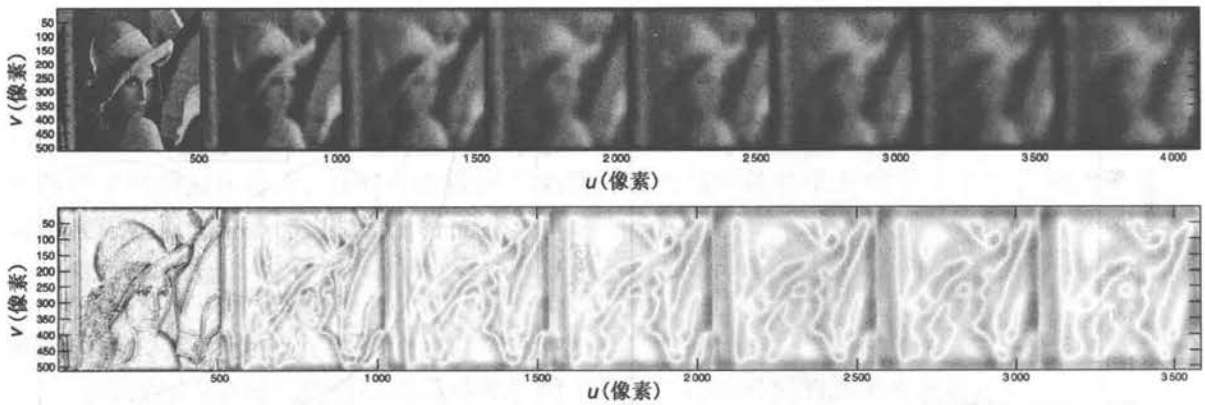


图 13.25 $\sigma = 2$ 的尺度空间序列。(上图)高斯序列；(下图)高斯-拉普拉斯序列

把原始图像与 σ 渐增的高斯核进行互相关，会导致核的大小以及相应计算量随每次尺度的增加而变大。回顾第 295 页上介绍的高斯函数性质，两个高斯分布的互相关会得到另一个更加宽广的高斯分布。因此，我们不再把原始图像与不断变宽的高斯分布进行互相关计算，而

是反复把同一高斯分布作用到前面的结果上。我们再回想一下第 293 页，LoG 核是由两个高斯分布的差分来近似代替的。利用互相关的性质，可以写出

$$(G(\sigma_1) - G(\sigma_2)) \otimes I = G(\sigma_1) \otimes I - G(\sigma_2) \otimes I$$

其中 $\sigma_1 > \sigma_2$。上式表明，将高斯算子的差分作用在图像上，等价于图像分别被两种程度的平滑处理结果的差。如果连续应用一个高斯算子来执行图像的平滑处理，那么会有一系列平滑处理程度渐增的图像。因此，这个序列中两个连续步之间的差就是高斯-拉普拉斯算子的近似。图 13.26 用框图形式展示了上述操作。

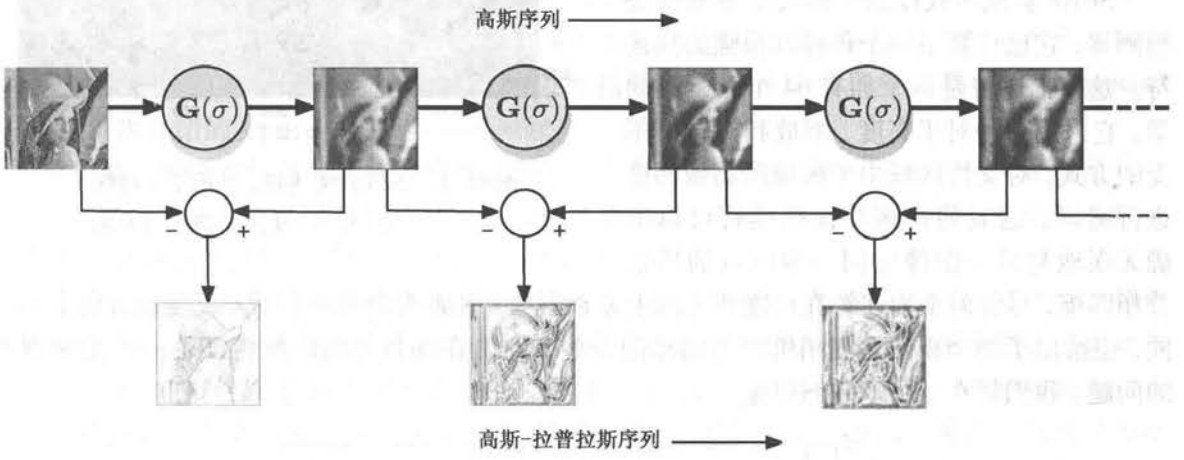


图 13.26　计算高斯和高斯-拉普拉斯空间尺度序列的示意图

13.3.2.1　空间尺度的点特征

我们刚刚讨论的空间尺度概念支撑了很多流行的特征检测器，这些检测器可以找到一幅图像中的重要点，并且确定它们的尺度和方向。尺度不变特征变换(SIFT)是基于高斯序列之差的最大值。快速鲁棒性特征(SURF)则是基于高斯序列近似的 Hessian 行列式的最大值。

举例说明，我们计算先前使用的建筑物图像的 SURF 特征：

```
>> surf1 = isurf(b1)
surf1 =
4034 features (listing suppressed)
  Properties: theta image_id scale u v strength descriptor
```

它会返回一个 SurfPointFeature 对象的数组，SurfPointFeature 对象是 ScalePointFeature 对象的一个子类。例如，它的第一个特征是

```
>> surf1(1)
ans = (117.587,511.978), theta=0.453513, scale=2.16257,
strength=0.0244179, descrip= ..
```

每一个对象包括特征的坐标(估计到亚像素精度)、尺度、方向和描述符，其中描述符为一个拥有 64 个元素的向量。方向通过支持区域内的主边缘方向来定义。

图像一共包含 4000 多个 SURF 特征，但是我们可以显示出前 50 个最强的特征：

```
>> idisp(b1, 'dark');
>> surf1(1:50:end).plot_scale('g', 'clock')
```

结果如图 13.27 所示。plot_scale 方法在特征位置周围画了一个圆，半径表示它的尺度——支持区域的大小。选项'clock'将画出一条指示 SURF 特征方向的径线。

特征尺度的变化范围很广，其直方图为

```
>> hist(surf1.scale, 100);
```

如图 13.28 所示，它表明有很多与图像细节和纹理相关联的微小特征。大部分的特征尺度小于 25 个像素，但是有一些尺度却高于 40 个像素。函数 isurf 可以接受很多选项，网上文档中对此有详细的描述。

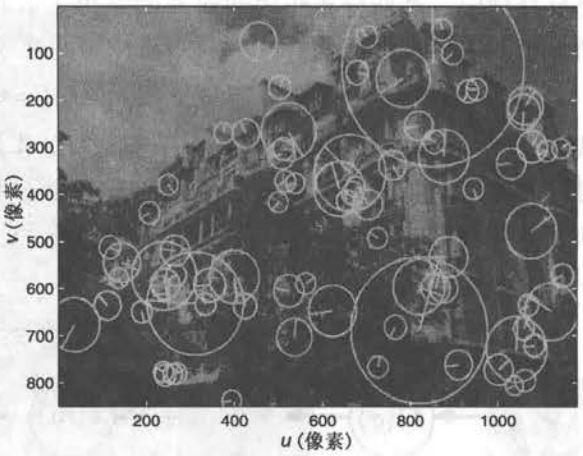


图 13.27　SURF 描述符，显示了支持区域(尺度)及径线指示的方向

SURF 算法不仅仅是一种尺度不变特征检测器，它也计算了一个鲁棒性很强的描述符。这个描述符是一个拥有 64 个元素的向量，它采用一种对于亮度、缩放和旋转都不变的方式，对支持区域中子区域的图像梯度进行编码。这就使得该特征描述符可以准确无误地与另一图像中同一空间点的描述符相匹配，尽管两个描述符在尺度和方向上差别很大。相匹配特征在位置、尺度和方向上的不同，也给出了两个视图之间相机相对运动的一些信息。在场景之间匹配特征是一个至关重要的问题，我们将在下一章重点讨论。

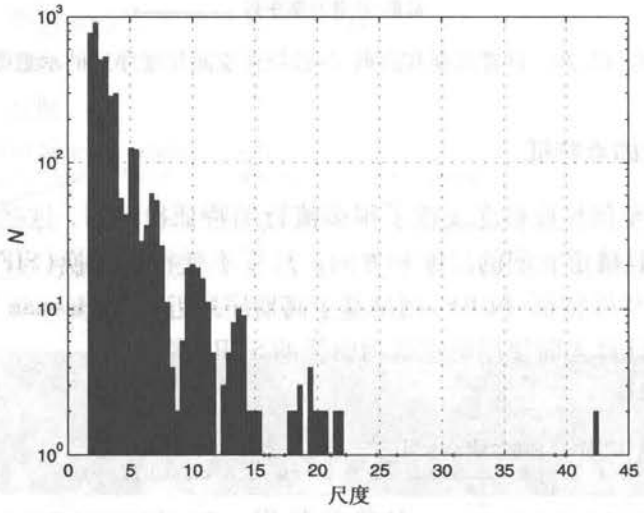


图 13.28　特征尺度的直方图，竖直尺度取对数

13.4　本章总结

本章讨论了图像特征的提取。我们不再把图像看作是成千上万的单独像素值，而是简洁地描述图像中对应于实际环境中不同目标对象的区域。例如，我们可以找出那些具有强度或颜色一致性的区域，然后描述出它们的特征，如边框、形心、等价椭圆、纵横比、圆度或者周界形状。对于目标识别来说，特征对于平移、绕光轴旋转和缩放都具有不变性是非常重要的。直线是人工环境中常见的视觉特征，我们示范了怎样通过霍夫变换去寻找和描述图像中不同的直线。

我们还展示了怎样在图像中寻找与实际环境里的特殊点可靠关联的兴趣点，且与相机的视角无关。它们是相机运动评估、立体视觉、图像检索，以及下一章要讨论的图像追踪和拼接等技术的关键。

扩展阅读

基于区域的图像分割和斑块分析是很多书籍和论文中提到的传统技术。[Gonzalez and Woods, 2008]和[Szeliski, 2011]对本章介绍的方法进行了较详尽的描述，尤其是对于阈值法和边界描述符。确定阈值的大津算法是在[Otsu, 1975]中引入的，而可变阈值的 Niblack 算法是在[Niblack, 1985]中首次介绍的。[Nixon and Aguado, 2008]这本著作对本章中相关的材料进行了扩展，并介绍了诸如可变形模板和边界描述符等技术。弗里曼链码是在[Freeman, 1974]中首次介绍的。

除了基于强度和颜色的区域一致性之外，也要尽可能描述区域的纹理——像素强度的空间分布，它们的统计资料可以参见[Gonzalez and Woods, 2008]。然后区域就可以按照纹理进行分割，例如从有纹理的草坪中分割出一条平坦的路。

数据聚类是机械学习中的一个重要的课题[Bishop, 2006]。本章中我们已经使用了简单的 k 均值方法，它是一种比较老旧的分类方法，需要提前知道聚类的数量。更先进的分类算法是梯级式的，它们使用像 kd 树这样的数据结构来加速邻近点的寻找。群聚中心的初始化对于聚类结果也十分关键。[Forsyth and Ponce, 2002]介绍了更多用于机器人视觉的一般聚类方法和基于图的方法。用于分割的图割算法在[Felzenszwalb and Huttenlocher, 2004]中进行了描述，工具箱中的图割算法实现就是基于 Pedro Felzenszwalb 的程序代码，可从网址 http://people.cs.uchicago.edu/~pff/segment 获得。最大稳定极值区域(MSER)算法在[Matas et al., 2004]中进行了介绍，工具箱的具体实现是基于 Andrea Vedaldi 和 Brian Fulkerson 的工作，可从 http://vlfeat.org 获取。http://www.eecs.berkeley.edu/Research/Projects/CS/vision/bsds 网页上的伯克利分割数据库(Berkeley Segmentation Dataset)包含很多复杂的真实图像，每一张都进行了多种分割。

本章还呈现了传统的自下向上的特征提取方法，从像素开始逐步迈向更高层级的概念，例如区域和直线。[Forsyth and Ponce, 2002]较好地介绍了使用概率技术的高层级视觉，这种技术可用于目标识别(如脸部识别)和图像检索等很多问题。

霍夫变换最早是在保罗·霍夫(Paul Hough)的美国专利 3,069,654“识别复杂图案的方法和手段(*Method and Means for Recognizing Complex Patterns*)”中进行了介绍，它的发展历史在[Hart, 2009]中进行了讨论。该技术最初的应用是气泡室照片的自动化分析，它把有问题的斜率截距参数化法用在直线上。而现在人们熟知的 (ρ,θ) 参数化形式则是最早来自[Duda and Hart, 1972]，被称为“广义霍夫变换”。这篇论文可以从网址 http://www.ai.sri.com/pubs/files/tn036-duda71.pdf 获得。霍夫变换在[Gonzalez and Woods, 2008]和[Forsyth and Ponce, 2002]这类教科书中也有所涉及。其中，后者对一般的形状匹配以及对分散数据点具有很强鲁棒性的评估器都进行了精彩的讨论。基本的霍夫变换已经从许多方面进行了扩展，并有大量的相关的文献。[Leavers, 1993]中回顾了这个变换，并介绍了它的一些变体，值得一读。这个变换可以扩展应用于其他的形状[Ballard, 1981]，例如尺寸不变的圆形，其中选票将投给圆中心的坐标。对于未知尺寸的圆形，需要一个三维投票数组用于圆心和半径。

兴趣点算子方面的早期文献包括[Moravec, 1980]，以及[Föstner and Gülch, 1987]和

[Föstner 1994]。Harris 角点检测器[Harris and Stephens, 1988]在机器人视觉上的应用是从 20 世纪 80 年代末开始的，因为它能够在当时的计算机上实时运行，而且特征在图像与图像之间非常稳定[Tissainayagam and Suter, 2004]。Noble 检测器在[Noble, 1988]中进行了描述。[Shi and Tomasi, 1994]和[Tomasi and Kanade, 1991]中则介绍了 Shi-Tomasi 检测器和 Kanade-Lucas-Tomasi(KLT)追踪器。文献[Deriche and Giraudon, 1993]及[Mikolajczyk and Schmid, 2004]对很多角点检测器的相对性能进行了比较总结。

尺度空间这个概念很早就在计算机视觉中使用了。[Koenderink, 1984]、[Lindeberg, 1993]和[ter Haar Romeny, 1996]对这个议题都做了较好的介绍。空间尺度与经典角点检测器结合就产生了混合检测器，如尺度-哈里斯检测器[Mikolajczyk and Schmid, 2004]。尺度空间特征检测器中一项重要的进展是尺度不变特征变换(SIFT)，它是 21 世纪初期在[Lowe, 2004]中提出，并对追踪、图像辨识这类应用来说具有重大意义。遗憾的是，这项技术被申请了专利，因此没有在这本书中使用。然而有失必有得，一种更为有效的替代技术很快就出现了，它被称为快速鲁棒性特征(SURT)[Bay et al., 2008]。工具箱函数 `isurf` 包装了由 Dirk-Jan Kroon 编写的 MATLAB 实现方法，可从网址 http://www.mathworks.com/matlabcentral/fileexchange/28300-opensurf-including-image-warp 获取，而这个 MATLAB 方法是基于 Chris Evans 用 C++和 C#编写的 OpenSurf 实现方法，它可在 http://www.chrisevansdev.com/computer-vision-opensurf.html 获取。其他 SURT 的实现方法可从 http://www.vision.ee.ethz.ch/~surf 获取，并且基于 GPU 的并行实现也已经开发出来。SIFT 和 SURT 检测器确实给出了不同的结果，在[Bauer et al., 2007]中对它们进行了比较。如果你对测试 SIFT 有兴趣，可以使用工具箱函数 isift，它提供了与 `isurf` 相似的界面，并返回一个 `SiftPointFeature` 类的特征向量。这个函数只是对 MATLAB 实现方法的一个包装，你需要从 http://www.vlfeat.org 下载和编译。

最近学者们还提出了很多其他特征检测器。[Rosten et al., 2010]中提出的 FAST 声称具有很小的计算工作量和很高的可重复性，其 C 语言和 MATLAB® 软件可从 http://mi.eng.cam.ac.uk/~er258/work/fast.html 获取。[Agrawal et al., 2008]和[Rosten et al., 2010]中提出的 CenSurE 则声称能以较低成本获得比 SIFT、SURT 和 FAST 更好的效果。[Calonder et al. 2010]中提出的 BRIEF 不是一个特征检测器，它是一个低成本且简洁的特征描述符，每个特征只有 256 个比特，而不是 64 个浮点数。

习题

1. 灰度分类。
   a)在图像 `castle_sign.png` 和 `castle_sign2.png` 上用函数 `ithresh` 进行试验。
   b)用 Niblack 算法进行试验，并改变 k 值和窗口大小。
   c)把 `iblobs` 应用到 MSER 分割的输出上。开发一种算法，使用边界框的宽度和高度来只提取那些是字母的斑块。
   d)函数 `imser` 有很多参数：`'Delta'`、`'MinDiversity'`、`'MaxVariation'`、`'MinArea'`、`'MaxArea'`。探索调整这些参数对输出的影响。
   e)把函数 `igraphcut` 应用到 `castle_sign2.png` 图像。理解并调整参数来提高性能。
2. 颜色分类。
   a)改变颜色分类示例中的聚类的数量 k，试验其效果。有最佳值吗？

b)k 均值算法有选项'random'或'spread'，它可以执行随机初始化。运行 k 均值算法若干次，然后确定最后的聚类结果有哪些不同。

c)写一个函数来确定哪一个聚类代表目标对象，即黄色群集或红色群集。

d)把函数 igraphcut 应用到彩色目标和菜园图像上。它是怎样执行的？理解并调整参数来提高它的性能。

e)针对用于彩色目标和菜园图像的形态学“清理”，对其参数进行试验。

f)编写一个程序，它能依次读取你计算机相机拍摄到的图像，然后进行分类并显示结果。分类可以用灰度阈值法，或预先告知颜色聚集数的颜色聚类(见 colorkmeans)。

3. 斑块。创建一幅图像，其中一个目标对象上有几个洞。你可以手绘然后拍照获取，也可以从绘图软件中输出这幅图像，或者写程序代码产生。

a)确定目标的外边界、内边界和整体边界。

b)把洞中的小目标放入目标中。编写代码展示场景中斑块的拓扑结构。

c)对于不同尺度的同一形状，绘制出形状的圆度随尺度变化的曲线。解释这个曲线的形状。

d)创建一系列正方形目标，并绘制出估计与真实的周长随正方形边长变化的曲线。当正方形比较小时会出现什么情况？

e)创建一幅包含很多不同形状目标对象的简单场景图像。使用形状不变特征(纵横比、圆度)创建一个简单的形状分类器。它的运行效果如何？使用 Hu 矩特征，再重复试验。

f)针对你创建的一些目标，重复边界匹配的示例。修改代码来创建边界分割角 (k) 相对于 θ 的变化曲线，然后重复边界匹配的示例。

g)另一个常用的特征是对齐的矩形，但工具箱不支持该特征。这是最小的矩形，它的边与等价椭圆的轴对准，并完全包含了斑块。这个矩形的纵横比以及斑块与矩形面积之比都具有对缩放和旋转的不变特性。编写代码来计算这个矩形，并把这个矩形叠加在图像上，然后计算出这两个特征。

4. 霍夫变换。

a)改变霍夫累加器的大小，并进行实验。

b)用索伯边缘操作代替坎尼操作，并进行实验。

c)改变参数'supress'、'interpSize'、'EdgeThresh'、'houghThresh'，并进行实验。

d)把霍夫变换应用到你自己的一张图像中。

e)编写一个程序，它能读取你计算机相机拍摄到的图像，寻找图像中的两条主直线，并把它们叠加在显示图像的窗口上。

5. 角点检测。

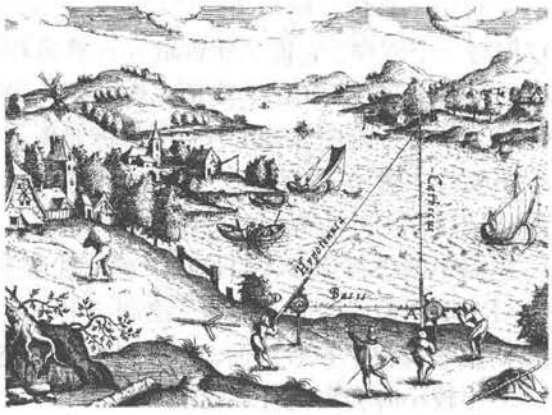
a)通过改变参数 k、σ_D 和 σ_I 对 Harris 检测器进行实验。

b)比较 Harris、Noble 和 Shi-Tomasi 三种角点检测器的性能。

c)实现 Moravec 检测器，并与 Harris 检测器进行对比。

d)创建一个经平滑处理的二阶导数 $\boldsymbol{I}_{uu}$、$\boldsymbol{I}_{vv}$ 和 $\boldsymbol{I}_{uv}$。

第 14 章　使用多幅图像



前一章学习了角点的检测，用来寻找场景中特别显著的点。不管视角或照明状态如何，这些点在同一场景的不同视图中都能被可靠检出。这些点的特征是在正交方向具有较高的图像梯度，通常出现在目标的拐角处。然而对应空间点的三维坐标在透视投影过程中会被丢失（透视投影在第 11 章中讨论过，即把一个三维点映射为二维图像坐标）。我们所知道的仅仅是这个空间点位于空间中与像素坐标对应的某条射线上，如图 11.1 所示。为了找回丢失的第三维，需要附加信息。在 11.2.3 节中，附加的信息是相机标定参数加上几何目标模型，根据这些信息就可以从二维图像数据中估测出目标的三维位姿。

这一章考虑另一种方法。在这种方法中，附加信息来自同一场景的多个视图。正如已经提到的那样，单一视图的像素坐标只能把空间点约束在某一条射线上。如果我们可以把同一空间点定位在另一张图像中，且已知该图像是从不同的位姿拍摄的，那么就可以确定这个空间点所在的另一条射线。最后，两条射线的相点就是空间点所在的位置——这个过程称为三角测量或者三维重构。更强大的功能是，如果我们观察到足够多的点，就可以估计出这些视图之间相机的三维运动，以及整个环境的三维结构[①]。

潜在的挑战是如何在多幅图像中找到同一个空间点。这里存在对应性的问题，这是一个不一般的重要问题，我们将在 14.1 节对其进行讨论。在 14.2 节中，我们将回顾第 11 章中讲过的成像几何基础。如果你还没有读到那一章，或者你已经读过很长一段时间了，这一节将会对你熟悉这些内容很有帮助。我们将把单一相机成像的几何学拓展到有多个图像平面的情况，并展示这些图像之间的几何关系。立体视觉对于机器人的三维感知是一项重要技术，在 14.3 节中将对它进行详细讨论。我们将把从不同视点拍摄的同一场景的两幅图像信息综合起来，确定空间环境的三维结构。14.4 节介绍了如何从运动中恢复结构，其中一系列图像的视觉信息被用于确定空间环境的三维结构，以及机器人是怎样在环境中运动的，后者也被称为视觉测程。

本章，也是本书这一部分的最后，我们将基于已学的概念给出 4 个应用实例。14.5 节中描述了我们怎样把有明显透视畸变的图像转换成一幅没有畸变的图像，有效合成了在不同位置的一个虚拟相机中的视图。14.6 节描述了图像拼接，即用移动中的相机拍摄出连续图像，然后把它们缝合在一起，形成一幅大的虚构图像。14.7 节描述了图像检索问题，即在已有的一系列图像中找到与某个新图像最相似的那一幅。机器人可以运用这一技术来确定它之前是

---

① 几乎没有误差！我们可以仅仅依靠一个未知的比例系数就确定相机的位移，即位移是 λt，其中 t 的方向是已知的，但 λ 是未知的。

否曾经到过一个特定地点或者看到过相同的目标。14.8 节描述了怎样从移动的相机中处理一系列的图像来定位相同的空间点，并估计相机的运动和三维空间结构。

14.1　特征匹配

匹配问题是在两幅不同的图像中找出对应于实际空间中同一点的像素坐标①。考虑两幅真实图像：

```
>> im1 = iread('eiffel2-1.jpg', 'mono', 'double');
>> im2 = iread('eiffel2-2.jpg', 'mono', 'double');
```

如图 14.1 所示。它们展示了使用不同相机从两个不同位置观察到的同一场景——相机的像素大小、焦距长度和每一幅图像的像素数量都完全不同。场景非常复杂，我们很快发现确定匹配的特征不是一件简单的事情。每一幅场景中，超过半数的像素对应着蓝天，把一幅图像中的蓝色像素与另一幅中对应的蓝色像素匹配起来是不可能的——这些像素不够独特。这种情形是很常见的，它会发生在颜色一致的图像区域，例如黑暗的阴影、平滑的水体、白雪或者光滑的人造目标，如墙壁或汽车的两侧。

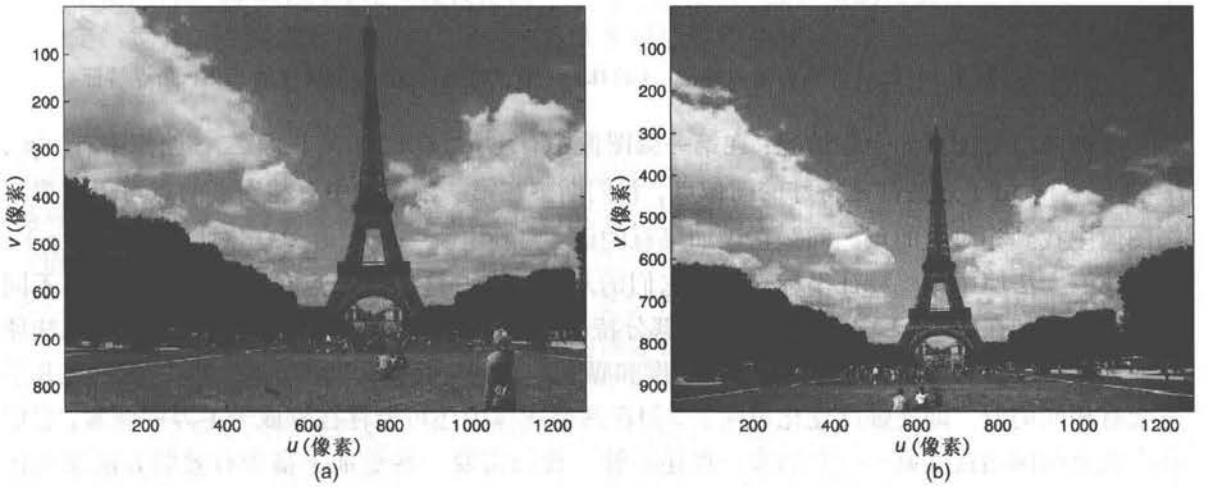


图 14.1　埃菲尔铁塔的两幅图片。这两幅图像几乎是同时拍摄的，但是使用了两种不同手持式数码相机。(a)7兆像素相机，$f=7.4$ mm；(b)10兆像素相机，$f=5.2$ mm(由Lucy Corke拍摄)。两幅图像具有不同的尺度，塔高在(a)和(b)中分别为700像素和600像素。拍摄图像(b)的人位于(a)图的右下角

解决方法是只选择那些与众不同的点。我们可以使用上一章中介绍的兴趣点检测方法来找到 Harris 角点特征：

```
>> harris = icorner(im1, 'nfeat', 200);
>> idisp(im1); harris.plot('gs');
```

或者 SURF 特征②：

① 这是数据关联问题的又一个实例。

② SURF 检测不能处理彩色图像，它会把彩色图像转换成灰度图像。Harris 检测可以分别计算单个彩色平面的平方梯度，然后再把它们结合起来。所有的检测都可以处理以维数大于二的矩阵形式提供的一组图像序列。在一幅彩色图像和一组长度为 3 的图像序列之间存在着不确定性。如果图像的第三维大小是 3，那么它就被视为一幅彩色图像，而不是一个序列。一幅四维图像就能确定是一个彩色图像序列。

```
>> sf = isurf(im1, 'nfeat', 200);
>> idisp(im1); sf.plot_scale('g');
```

这些特征点如图 14.2 所示。我们已经把问题大大简化了——仅仅使用 200 个特征点来代替数百万的像素。

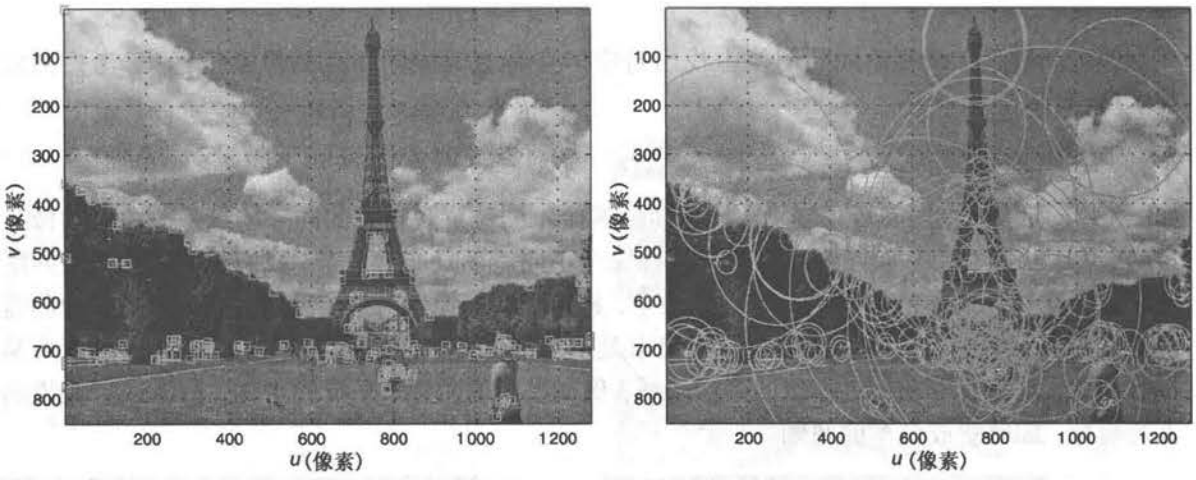


图 14.2 对图 14.1(a)计算的角点特征。(a)Harris 角点特征；(b)显示尺度的 SURF 角点特征

考虑两组特征点的一般情况：在第一幅图像中有 $\{{}^1\boldsymbol{p}_i,\ i = 1\cdots N_1\}$，第二幅图像中有 $\{{}^2\boldsymbol{p}_j,\ j = 1\cdots N_2\}$。由于这些都是特殊的图像点，我们期望在第一幅图像中有相当多的点可以在第二幅图像中找到相对应的点。问题在于如果有的话，要确定哪一个 $({}^2u_j, {}^2v_j)$ 对应着 $({}^1u_i, {}^1v_i)$。

我们无法使用这些特征坐标来确定它们的对应性——这些特征点在每一幅图像中有不同的坐标。例如在图 14.1 中，我们看到大部分特征点比右边图像中的要低一点。我们也无法使用像素的强度或者颜色，因为白平衡、照度和曝光设置的变化使得那些本该对应的像素点几乎不会有相同的值。即使强度变化消除了，但在两幅图像中还可能存在着成千上万的像素，它们具有完全相同的强度值——它的唯一性还不够。我们需要一些更加丰富和有效的方法来描述每一个特征。

实际应用中，我们描述了角点周围的像素区域，它提供了一种对角点及其周围环境独一无二的描述——特征描述符。在工具箱中，对于角点的特征描述符是一个向量——PointFeature 超类的 descriptor 属性。对于 Harris 角点特征，其描述符是

```
>> harris(1).descriptor'
ans =
    0.0805    0.0821    0.0371
```

它是一个三维向量，包含着结构张量方程(13.14)中的独立元素。这个低维的描述符计算量较小，因为它的元素在确定角点强度的时候已经计算出来。这些描述符的元素是梯度，它们的优势是对于图像强度的偏移具有鲁棒性。两个描述符的相似度是基于欧几里得距离计算的，并且最好的匹配值是 0。例如，角点特征 1 和 2 的相似度是

```
>> harris(1).distance( harris(2) )
ans =
    0.0518
```

然而我们却很难知道这个值是否能够代表有较高的相似性，因为其单位是任意的。通常我们

将会把特征 ${}^1\boldsymbol{p}_i$ 与另一幅图像中的所有特征点 $\{{}^2\boldsymbol{p}_j, j = 1\cdots N_2\}$ 进行比较，然后选择最相似的一个①。然而像这样的一个低维描述符向量仍然不够独特，容易发生错误匹配。

我们可以通过把特征点周围的正方形窗口描述成一个向量来创建一个更大的描述符向量。例如，

```
>> harris = icorner(im1, 'nfeat', 200, 'color', 'patch', 5)
```

它为窗口中的每一个角点创建一个拥有 121 个元素的描述符向量，窗口大小是参数指定的半宽——在这里为一个 11 ×11 的窗口。像素值通过其平均值进行了平移，并且归一化为一个单位向量。我们可以重新把表 12.1 中 ZNCC 相似性测度写成一个一维的形式：

$$
\begin{aligned}
s &= \frac{\sum_{i=1}^{N}(I_1[i]-\bar{I}_1)\cdot(I_2[i]-\bar{I}_2)}{\sqrt{\sum_{i=1}^{N}(I_1[i]-\bar{I}_1)^2\cdot\sum_{i=1}^{N}(I_2[i]-\bar{I}_2)^2}} \\
&= \underbrace{\frac{\sum_{i=1}^{N}(I_1[i]-\bar{I}_1)}{\sqrt{\sum_{i=1}^{N}(I_1[i]-\bar{I}_1)^2}}}_{\hat{d}_1}\cdot\underbrace{\frac{\sum_{i=1}^{N}(I_2[i]-\bar{I}_2)}{\sqrt{\sum_{i=1}^{N}(I_2[i]-\bar{I}_2)^2}}}_{\hat{d}_2}
\end{aligned}
\tag{14.1}
$$

我们已经把它分解成与每一个图像碎片关联的单位向量的点积。这些归一化的向量 $\boldsymbol{d}_i$ 可以用作特征描述符。归一化的互相关仅仅是两个描述符的点积，其产生的相似性测度 $s \in [-1,1]$ 有一定的意义。完美匹配是 $s = 1$，通常 $s \geqslant 0.8$ 可以看作是较好的匹配。对于我们的示例：

```
>> harris(1).ncc( harris(2) )
ans =
   -0.0292
```

相关性分数表明它是一个较差的匹配。描述符具有独特性，且不随图像强度的变化而改变，但是对于缩放或者旋转来说却是可变的。我们能够使用的对于周围区域的其他描述符还包括统计值和排秩值，也有强度或者颜色的直方图。直方图的优势在于它相对于旋转是不变的，但是它们无法表达像素之间的空间关系，就是说在完全不同位置的相同像素值也有相同的直方图。

SURF 算法通过计算一个拥有 64 个元素的描述符向量来描述特征点②，它的描述方式相对于缩放和旋转是不变的，并且是以特征点周围区域内的像素为基础的。它是从对应特征尺度的尺度空间序列中的图像创建，并且根据特征方向进行旋转。该向量被归一化为一个单位向量，以提高它相对于图像强度变化的不变性。描述符之间的相似性测度基于欧几里得距离。这个描述符对于图像强度、缩放和旋转来说都相当稳定。SURF 不仅是一种角点检测器，还是一种描述符，然而 Harris 运算仅仅是一种角点检测器，必须与众多不同描述符中的一种共同使用③。

在本章的剩余部分，我们将使用 SURF 特征。虽然它们的计算量比较大，但是就同一场景不同视图之间的匹配质量而言，值得一用。我们计算每一幅图像的 SURF 特征：

```
>> s1 = isurf(im1)
s1 =
1288 features (listing suppressed)
  Properties: theta image_id scale u v strength descriptor
>> s2 = isurf(im2)
s2 =
1426 features (listing suppressed)
  Properties: theta image_id scale u v strength descriptor
```

① 如果空间点在图像 2 中根本不可见，那么选最相似的特征将会发生错误匹配。

② 128 个元素的向量可以通过把选项'extended'传递给函数 isurf 来创建。

③ 可以想象将 SURF 描述符和一个 Harris 角点一起使用。

它会产生包含 SurfPointFeature 对象的两个向量。在每一幅图像中都找到上千的角点。

接着我们基于 SURF 描述符之间的距离来匹配两组 SURF 特征：

```
>> m = s1.match(s2)
m =
644 corresponding points (listing suppressed)
```

它会产生包含 FeatureMatch 对象的一个向量，其中有 644 个候选对应点。前 5 个候选点的对应关系是①

```
>> m(1:5)
ans =
(819.56, 358.557) <-> (708.008, 563.342), dist=0.002137
(1028.3, 231.748) <-> (880.14, 461.094), dist=0.004057
(1027.6, 571.118) <-> (885.147, 742.088), dist=0.004297
(927.724, 509.93) <-> (800.833, 692.564), dist=0.004371
(854.35, 401.633) <-> (737.504, 602.187), dist=0.004417
```

它显示了第一幅图和第二幅图中的特征坐标，以及两个特征向量之间的欧几里得距离。匹配是按照相似度递减的顺序排列的，使用了一个特征相似度的阈值。

我们可以在两幅原始图像上叠加这些匹配的一个子集：

```
>> idisp({im1, im2})
>> m.subset(100).plot('w')
```

结果如图 14.3 所示。白色线把每一幅图像中的匹配特征连接起来，这些线展示出了一致的模式。这些连接的大多数看上去很合理，但是有一些很显然是错误的。注意，我们把图像的一个单元数组传递给了函数 idisp，因此它显示了一个水平方向拼接的单一图像。FeatureMatch 类的 subset 方法返回一个包含指定数量 FeatureMatch 对象的向量，这些对象是从原始向量中均匀取样而得的。如果把所有的匹配都显示出来，我们将仅仅看到一片白色。

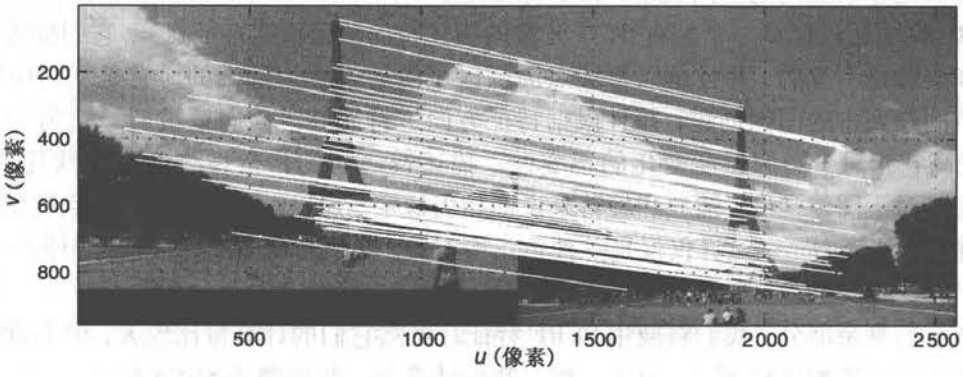


图 14.3 特征匹配。基于 SURF 描述符相似度的匹配子集(1664 个中的100个)。我们注意到一些匹配是明显错误的

对应匹配可以通过一个可选返回值来获得：

```
>> [m,corresp] = s1.match(s2);
>> corresp(:,1:5)
ans =
         215         389         357        1044         853
         246         418         312        1240         765
```

① 我们把它们称作候选点是因为尽管它们很可能是对应点，但是还没有被确认。

它是一个矩阵，每一对匹配对应一列。第一列表示图像 1 中的特征 215 与图像 2 中的特征 246 相对应，等等。用工作空间变量表示，它们是 s1(215)和 s2(246)。

相匹配的特征描述符之间的欧几里得距离可以通过 distance 属性来给出，在没有使用阈值的情况下，它们的分布是

```
>> m2 = s1.match(s2, 'thresh', []);
>> ihist(m2.distance, 'normcdf')
```

如图 14.4 所示。它显示所有匹配中 35% 的描述符距离都低于默认阈值 0.05，而最大的距离能超过其 10 倍——这种匹配不太可能是有效的。不用固定的阈值，我们可以选择距离最小的 N 个匹配：

```
>> m = s1.match(s2, 'top', N);
```

当然，我们也可以选那些都低于某个距离阈值的匹配：

```
>> m = s1.match(s2, 'thresh', 0.04);
```

或者所有那些低于中值的匹配：

```
>> m = s1.match(s2, 'median');
```

特征匹配的计算量大——它是一个 $O(N^2)$ 问题，因为在一幅图像中的每一个特征都必须与另一幅图像中的每一个特征相比较。

尽管如图 14.3 所示的匹配质量看起来很好，但是在这个小子集中仍然存在着一些明显的错误匹配。我们可以从连接对应点的直线中分辨出一种模式，它们从左到右有轻微的收敛和向下倾斜。这种模式是两个相机视角之间相对位姿的函数，并且对它的正确理解是确定候选匹配是否正确的关键。这部分内容将在下一节讨论。

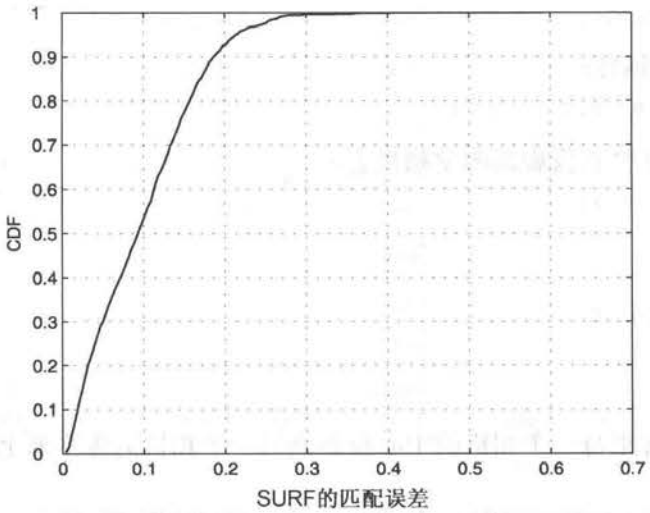


图 14.4　特征距离的累积分布

14.2　多视图几何学

我们从单一空间点 **P** 的不同视图之间的几何关系开始，如图 14.5 所示。这个几何图形可以代表两个相机同时观察同一场景的情形，或者代表一个相机从不同的视点拍摄到的图像。

每一个相机的中心，即坐标系 {1} 和 {2} 的原点，加上空间点 **P** 定义了空间中的一个平面——核面。空间点 **P** 投影到两个相机的像平面上，像素坐标分别为 ${}^1\mathbf{p}$ 和 ${}^2\mathbf{p}$，这些点被称为共轭点①。

考虑图像 1。图像点 ${}^1\mathbf{e}$ 是相机 2 位置的函数。图像点 ${}^1\mathbf{p}$ 是空间点 **P** 的函数。相机中心、${}^1\mathbf{e}$ 和 ${}^1\mathbf{p}$ 就定义了核面，从而也定义了图像 2 中的核线 ${}^2\ell$。根据定义，共轭点 ${}^2\mathbf{p}$ 必然也在那条核线上。反过来，${}^1\mathbf{p}$ 必然落在图像 1 的核线 ${}^1\ell$ 上，${}^1\ell$ 由图像 2 中的 ${}^2\mathbf{p}$ 定义。

这是一个很基础，也很重要的几何关系——考虑一幅图像中的一个点，我们知道其共轭点被约束在另一幅图像中的一条线上。我们举一个简单的例子对其进行说明，这个例子模拟了图 14.5 中的几何关系：

```
>> T1 = transl(-0.1, 0, 0) * troty(0.4);
>> cam1 = CentralCamera('name', 'camera 1', 'default', ...
     'focal', 0.002, 'pose', T1)
```

它会返回先前在 11.1 节中讨论的 CentralCamera 类的一个实例。同样，对于第二个相机有

```
>> T2 = transl(0.1, 0,0)*troty(-0.4);
>> cam2 = CentralCamera('name', 'camera 2', 'default', ...
     'focal', 0.002, 'pose', T2);
```

并且两个相机的位姿可以通过下面的函数绘制出来：

```
>> axis([-0.5 0.5 -0.5 0.5 0 1])
>> cam1.plot_camera('color', 'b', 'label')
>> cam2.plot_camera('color', 'r', 'label')
```

它也显示在图 14.6 中。我们定义一个任意的空间点：

```
>> P=[0.5 0.1 0.8]';
```

让它显示为一个小的球体：

```
>> plot_sphere(P, 0.03, 'b');
```

如图 14.6 所示。把这个点投影到两个相机上：

```
>> p1 = cam1.plot(P)
p1 =
  561.6861
  532.6079
>> p2 = cam2.plot(P)
p2 =
  746.0323
  546.4186
```

如图 14.7 所示。通过把每一个相机的中心投影到另一个相机的像平面上，可以计算出核点：

```
>> cam1.hold
>> e1 = cam1.plot( cam2.centre, 'Marker', 'd', 'MarkerFaceColor', 'k')
e1 =
  985.0445  512.0000
>> cam2.hold
>> e2 = cam2.plot( cam1.centre, 'Marker', 'd', 'MarkerFaceColor', 'k')
e2 =
   38.9555  512.0000
```

如图 14.7 所示的黑色◆标记。

① 假定空间点不移动。

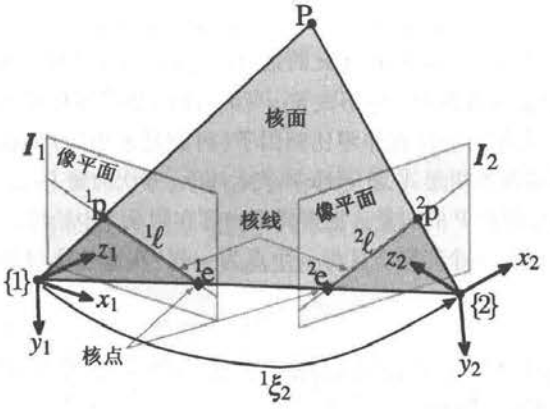


图 14.5　核面几何学，显示了与两个相机关联的坐标系 {1}、{2} 以及像平面。空间点 **P** 和两个相机中心形成一个核面，并且这个平面与图像平面相交形成核线

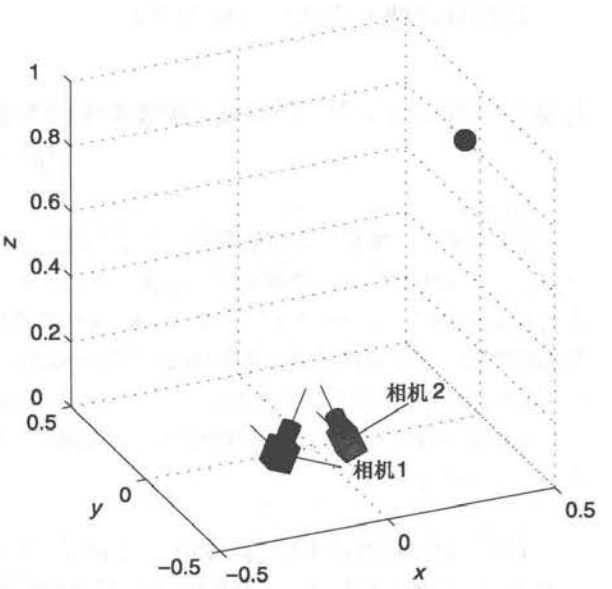


图 14.6　两个相机和一个目标点的模拟。两个相机的原点沿着 x 轴平移，并且两个相机是汇聚的，也就是说它们的光轴相交

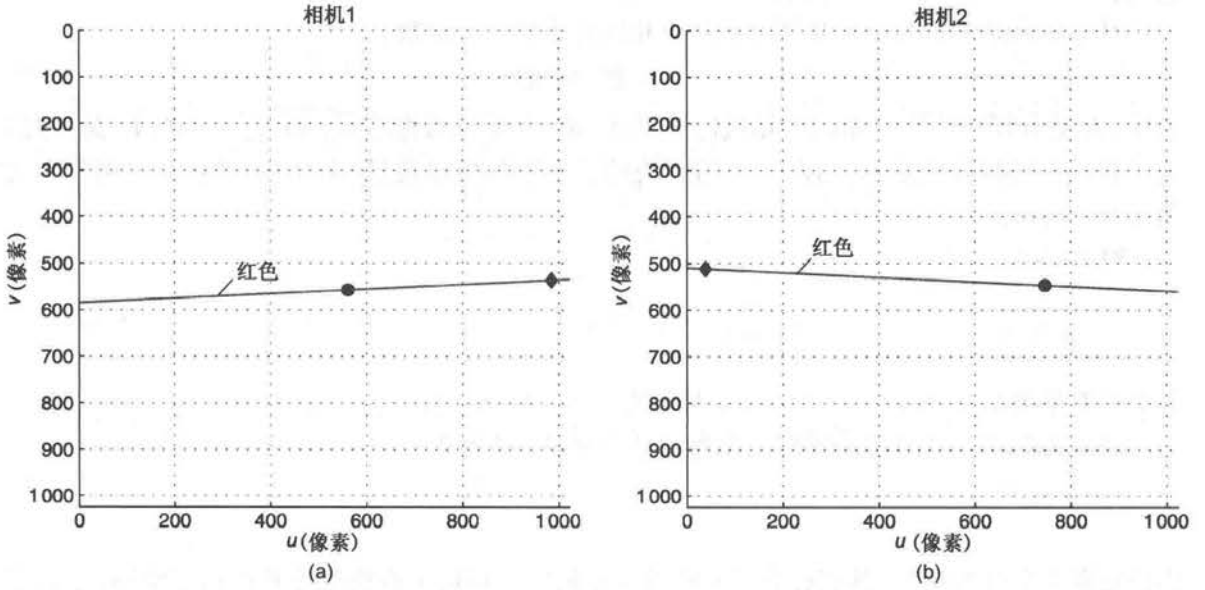


图 14.7　核面几何学仿真，展示了两个工具箱 `CentralCamera` 对象的虚拟像平面。点 **P** 的透视投影是一个黑色圆圈，另一个相机中心的投影是一个黑色◆标记，核线显示为红色

14.2.1　基本矩阵

图 14.5 中表示出来的核面几何关系可以简洁而完美地表达为

$$^2\tilde{\boldsymbol{p}}^T \boldsymbol{F}\, ^1\tilde{\boldsymbol{p}} = 0 \tag{14.2}$$

其中，$^1\tilde{\boldsymbol{p}}$ 和 $^2\tilde{\boldsymbol{p}}$ 是齐次形式的图像点 $^1\mathbf{p}$ 和 $^2\mathbf{p}$，$\boldsymbol{F}$ 是一个 3×3 的矩阵，称为基本矩阵。

我们可以把上式的后两项组合为

$$^2\tilde{\ell} \simeq \boldsymbol{F}\,^1\tilde{\boldsymbol{p}} \tag{14.3}$$

这是一条直线的方程，即核线，图像 2 中的共轭点必须位于该直线上：

$$^2\tilde{\boldsymbol{p}}^T\,^2\tilde{\ell} = 0 \tag{14.4}$$

背景材料：二维投影几何学简介。投影平面 $\mathbb{P}^2$ 是所有点 (x_1, x_2, x_3)，$x_1 \in \mathbb{R}$ 的集合，其中 x_i 不全为 0。通常把这个三元组看作是一个列向量。点 $\boldsymbol{p} = (u, v)$ 在 $\mathbb{P}^2$ 中表示为齐次坐标形式 $\tilde{\boldsymbol{p}} = (u, v, 1)$。比例缩放对于齐次量没有影响，我们把它表达为 $\tilde{\boldsymbol{p}} \simeq \lambda\tilde{\boldsymbol{p}}$，其中运算符 $\simeq$ 表示对应任意非零比例因子（可能是未知的）都相等。$\mathbb{P}2$ 中的一个点可以被表示为 $\mathbb{R}^2$ 中的非齐次形式，或者欧几里得形式，$\boldsymbol{p} = (x_1/x_3, x_2/x_3)$。齐次向量 (u, v, f)（其中 f 是以像素为单位的焦距长度）是从相机原点指向空间点 P 的向量。更多详细内容在附录 I 中给出。

工具箱函数 e2h 和 h2e 可以使点（一个列向量）或者点集（一个矩阵，且每一个点为一列）在欧几里得和齐次坐标之间进行转换。

这条直线是图像 1 中 $^1\tilde{\boldsymbol{p}}$ 点的一个函数（方程(14.3)），它也可以用于检验图像 2 中的某个点是否为可能的共轭点。将方程(14.2)两侧进行转置，得到

$$^1\tilde{\boldsymbol{p}}^T\boldsymbol{F}^{\mathrm{T}}\,^2\tilde{\boldsymbol{p}} = 0 \tag{14.5}$$

从中可以写出相机 1 的核线：

$$^1\tilde{\ell} \simeq \boldsymbol{F}^{\mathrm{T}}\,^2\tilde{\boldsymbol{p}} \tag{14.6}$$

它用相机 2 观察到的一个点表示。

基本矩阵是相机参数和视图之间相对相机位姿的一个函数：

$$\boldsymbol{F} \simeq \boldsymbol{K}^{-1}\boldsymbol{S}(t)\boldsymbol{R}\boldsymbol{K} \tag{14.7}$$

其中，$\boldsymbol{K}$ 是方程(11.7)中给出的相机内在矩阵，$\boldsymbol{S}(\cdot)$ 是反对称矩阵，而 $^2\xi_1 \sim (\boldsymbol{R}, \boldsymbol{t})$ 是相机 1 关于相机 2 的相对位姿①。与两个视图有关的基本矩阵可以通过 CentralCamera 类的 F 方法返回，例如：

```
>> F = cam1.F( cam2 )
F =
         0   -0.0000    0.0010
   -0.0000         0    0.0019
    0.0010    0.0001   -1.0208
```

其中相对位姿是从 CentralCamera 对象的 cam1 到 cam2。

基本矩阵有一些有趣的属性。首先它是奇异的，秩为 2：

```
>> rank(F)
ans =
     2
```

而且它有 7 个自由度②。核点存在于矩阵的零空间中。相机 1 的核点是 $\boldsymbol{F}$ 的右零空间，它的齐次坐标形式为

```
>> null(F)'
ans =
   -0.8873   -0.4612   -0.0009
```

或欧几里得坐标形式为

① 注意，这和你期待的结果是相反的，你期待的是相机 2 相对于相机 1。

② 矩阵 $F \subset \mathbb{R}^{3\times3}$ 有 7 个基本参数，所以它的 9 个元素不是独立的。总体规模没有定义，存在一个约束为 $\det(F) = 0$。

```
>> e1 = h2e(ans)'
e1 =
  985.0445  512.0000
```

它与之前使用 plot 函数确定的结果一致。相机 2 的核点是基本矩阵转置后的左侧零空间：

```
>> null(F');
>> e2 = h2e(ans)'
e2 =
   38.9555  512.0000
```

工具箱可以使用 CentralCamera 类的 plot_epiline 方法来显示核线：

```
>> cam2.plot_epiline(F, p1, 'r')
```

如图 14.7 所示的相机两个图像平面中的红线。正如我们所期待的那样，我们看到 P 的投影落在这条核线上。相机 1 的核线是

```
>> cam1.plot_epiline(F', p2, 'r');
```

14.2.2　本质矩阵

核面几何约束也可以表达成归一化的图像坐标形式：

$$ {}^{2}\tilde{\boldsymbol{x}}^{T}\boldsymbol{E}\,{}^{1}\tilde{\boldsymbol{x}}=0 \tag{14.8} $$

其中，$\boldsymbol{E}$ 是本质矩阵，${}^{1}\tilde{\boldsymbol{x}}$ 和 ${}^{2}\tilde{\boldsymbol{x}}$ 是表示为齐次归一化图像坐标的共轭点[①]。这个矩阵是相对相机位姿的简单函数：

$$ \boldsymbol{E}\simeq\boldsymbol{S}(\boldsymbol{t})\boldsymbol{R} \tag{14.9} $$

其中，${}^{2}\xi_{1}\sim(\boldsymbol{R},\boldsymbol{t})$ 是相机 1 相对于相机 2 的位姿。这个本质矩阵是奇异阵，秩为 2，有两个相等的非零奇异值和一个 0 值[②]。本质矩阵只有 5 个自由度，完全由 3 个旋转参数和 2 个平移参数确定[③]。对于纯旋转，当 $\boldsymbol{t}=0$ 时，本质矩阵是不确定的。

我们从方程(11.7)中可得 $\tilde{\boldsymbol{p}}\simeq\boldsymbol{K}\tilde{\boldsymbol{x}}$，将其代入方程(14.7)，可以写出

$$ {}^{2}\tilde{\boldsymbol{p}}^{T}\underbrace{\boldsymbol{K}_{2}^{-T}\boldsymbol{E}\boldsymbol{K}_{1}^{-1}}_{\boldsymbol{F}}{}^{1}\tilde{\boldsymbol{p}}=0 \tag{14.10} $$

仿照方程(14.2)，可以得出两个矩阵之间的关系为

$$ \boldsymbol{E}\simeq\boldsymbol{K}_{2}^{T}\boldsymbol{F}\boldsymbol{K}_{1} \tag{14.11} $$

它是用两个相关相机的内在参数表示的[④]。这个过程可以由 CentralCamera 类的 E 方法来完成：

```
>> E = cam1.E(F)
E =
         0   -0.0779         0
   -0.0779         0    0.1842
         0   -0.1842    0.0000
```

其中使用的是相机 1 的内在参数(相机 1 与相机 2 相同)。

如同 11.2.2 节中的相机矩阵一样，本质矩阵也可以被分解，得到齐次变换形式的相对位

① 参见第 248 页。
② 参见附录 D。
③ 在未知放缩的情况下，一个三维的平移 (x,y,z) 可以认为是 $(x',y',1)$。
④ 如果两幅图像是由相同的相机拍摄而成，那么 $K_1=K_2$。

姿 ${}^{1}\xi_{2}$。[①]但它的逆矩阵不是唯一的，通常会有两个解：

```
>> sol = cam1.invE(E)
sol(:,:,1) =
    1.0000    0.0000   -0.0000   -0.1842
    0.0000   -1.0000   -0.0000   -0.0000
   -0.0000    0.0000   -1.0000   -0.0779
         0         0         0    1.0000
sol(:,:,2) =
    0.6967    0.0000   -0.7174    0.1842
    0.0000    1.0000    0.0000    0.0000
    0.7174   -0.0000    0.6967    0.0779
         0         0         0    1.0000
```

它会返回一个三维矩阵，其中最后一个数字是解的序号。从视图 2 到视图 1 的真实相对位姿是

```
>> inv(cam1.T) * cam2.T
ans =
    0.6967         0   -0.7174    0.1842
         0    1.0000         0         0
    0.7174         0    0.6967    0.0779
         0         0         0    1.0000
```

它表示在这里解 2 是正确的。

不寻常的是，我们准确地获得了相机的平移参数，但是由于 $\boldsymbol{E} \simeq \lambda \boldsymbol{E}$，齐次变换矩阵的平移部分还存在未知的缩放因子[②]。在本例中它是正确的，因为本质矩阵是直接根据两个相机的相对位姿来确定的。

在这里，我们可以选择正确的解，因为我们知道两个相机的位姿，但是在实际中怎样确定正确的解呢？一种方法是确定空间点是否是可见的。通常我们将会在第一个相机前面的光轴上选择一个点：

```
>> Q = [0 0 10]';
```

把它投影到第一个相机的像平面，得

```
ans =
  429.7889
  512.0000
```

这是一个合理的值。可以使用 `move` 方法取一个任意位移来创建第一个相机的一个实例：

```
>> cam1.move(sol(:,:,1)).project(Q)
ans =
   NaN
   NaN
```

得到的值 `NaN` 表示空间点位于相机后面。对于第二个解，有

```
>> cam1.move(sol(:,:,2)).project(Q)
ans =
  594.2111
  512.0000
```

它是一个有限值，表示这个解是有效的一个。可以通过提供一个检测点来更加简洁地执行上述选择：

```
>> sol = cam1.invE(E, Q)
sol =
    0.6967    0.0000   -0.7174    0.1842
    0.0000    1.0000    0.0000    0.0000
    0.7174   -0.0000    0.6967    0.0779
         0         0         0    1.0000
```

在这种情况下，只返回有效的解。

---

① 尽管方程(14.9)被写成 $\$(R, t) \sim {}^{2}\xi_{1}$，但是工具箱函数返回 ${}^{1}\xi_{2}$。

② 据哈特利和西塞曼的观察[Hartley and Zisserman, 2003]，甚至连 t 的符号也没有办法确定。

总之，这些 3×3 矩阵——基本矩阵和本质矩阵，包含了两个相机的几何关系。基本矩阵和一幅图像中的一个点定义了另一幅图像中的核线，并且它的共轭点必须在这条线上。本质矩阵则体现了两个相机中心的相对位姿，并且该位姿可以被提取出来，它有两个可能解，并且其位移具有未知的缩放因子。在这个例子中，基本矩阵是基于已知的相机运动和内在参数计算出来的。但实际情况并不是这样的——相机运动很难测量，并且相机可能没有被标定。相反，我们可以直接根据相应的图像点，直接估算出基本矩阵。

14.2.3　估计基本矩阵

假定在同一场景的两个不同视图中，我们有 N 对的对应点 $({}^1\boldsymbol{p}_i, {}^2\boldsymbol{p}_i)$，$i = 1\cdots N$。为了展示上述方法，我们创建一个拥有 20 个随机特征点的集合(在一个 2×2×2 的立方体中)，其中心位于相机前方 3 m 处：

```
>> P = homtrans( transl(-1, -1, 2), 2*rand(3,20) );
```

把这些点投影到两个相机的像平面上：

```
>> p1 = cam1.project(P);
>> p2 = cam2.project(P);
```

如果 $N \geqslant 8$，基本矩阵可以从两组对应点中估计出来：

```
>> F = fmatrix(p1, p2)
maximum residual 1.932e-29
F =
    0.0000   -0.0000    0.0239
   -0.0000   -0.0000    0.0460
    0.0239    0.0018  -24.4896
```

其中残差是方程(14.2)左侧的最大值，理想情况下应为 0。这里的值不是 0，但却很小，它来自于有限精度运算造成的累积误差。估计出的矩阵具有所要求的秩特性：

```
>> rank(F)
ans =
    2
```

对于相机 2，可以绘制出投影在它上面的点：

```
>> cam2.plot(P);
```

然后叠加上图像 1 中每个点产生的核线：

```
>> cam2.plot_epiline(F, p1, 'r')
```

如图 14.8 所示。我们看到一组或者一束核线，并且图像 2 中的每一个点都在其中的一条核线上。注意所有的核线是怎样汇聚到核点的，在本例中会发生这种情况，这是因为两个相机是交汇的，如图 14.6 所示①。

为了展示正确点对应的重要性，我们将会重复上例，但是会通过调换 p2 中的两个数据来引入两个坏的数据关联：

```
>> p2(:,[8 7]) = p2(:,[7 8]);
```

基本矩阵估计为

---

① 这个示例是人为故意的，为的是让核点位于图像范围内，也就是说，每一个相机都可以看到另一相机的中心。常见的成像几何是让光轴平行，如图 14.19 所示那样，在这种情况下核点在无穷远处(即齐次坐标的第三个元素为零)，并且所有的核线也是平行的。

```
>> fmatrix(p1, p2)
maximum residual 236.2
ans =
    0.0000   -0.0000    0.0022
    0.0000    0.0000   -0.0023
   -0.0030    0.0014    1.0000
```

现在它出现了一个大的残差——几百像素。这意味着点的匹配不能通过方程(14.2)的关系来解释。

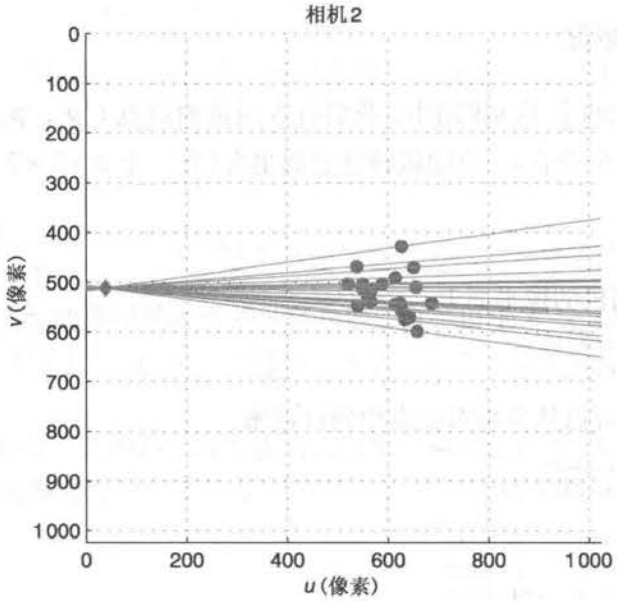


图 14.8 在相机 2 像平面上的一组核线。注意所有的核线是怎样穿过核点的，核点是相机 1 中心的投影

如果知道基本矩阵，就可以通过测量一个点到由另一个点定义的核线之间的距离，来检测这对候选对应点是否是真正的共轭点：

```
>> epidist(F, p1(:,1), p2(:,1))
ans =
   2.3035e-13
>> epidist(F, p1(:,7), p2(:,7))
ans =
   18.8228
```

结果显示点 1 是好的匹配，而点 7(我们把它与点 8 进行了交换)不是一个好的匹配。然而我们测试匹配的前提是必须先估计出基本矩阵，而这又需要点匹配是已知的。我们用一种独特的算法来解开这个死结，这种算法称为随机取样一致性(RANSAC)。

该算法的内在原理非常简单。估计一个基本矩阵需要 8 个点，所以我们先随机选择 8 个候选匹配点(样本)，然后估计 $\boldsymbol{F}$ 创建出一个模型。再用其他所有的候选点对这个模型进行测试，匹配成功[①]的那些候选点都会投票给这个模型。重复上述过程若干次，最后返回拥有最多支持者(一致的)的模型。由于样本小，它包含所有有效候选对的概率比较高。支持模型的点对称为内点，不支持的称为外点。

RANSAC 方法在寻找内点集方面相当有效和高效，即使在有大量外点(超过 50%)出现的场合，它适合于许多各种不同的问题。在工具箱中我们把 RANSAC 作为 fmatrix 函数的驱动

① 在定义的阈值 t 范围内。工具箱函数 epidist 返回点到核线的距离。

程序来调用：

```
>> [F,in,r] = ransac(@fmatrix, [p1; p2], 1e-6, 'verbose')
15 trials
2 outliers
3.48564e-29 final residual
```

我们获得了一个相当不错的最终残差。内点集也被返回：

```
>> in
in =
  Columns 1 through 14
     1     2     3     4     5     6     9    10    11    12    13    14    15    16
  Columns 15 through 18
    17    18    19    20
```

并且两个错误的关联——点 7 和点 8 也被从这个列表中删除。ransac 的第三个输入参数是阈值 t，被用于确定一个点对是否支持模型。如果选择的 t 太小，RANSAC 就需要比其默认最大值更多的试验，这就要求调节其他的参数。也要记住，由于是随机的二次抽样，RANSAC 的结果每一次试验都将会有变化。使用 RANSAC 会涉及用试错的方法并基于最终残差和外点数量来选择正确的阈值。网上文献中也描述了很多其他的选项。

现在回到图 14.3 所示的埃菲尔铁塔的一对图像中。当我们读完那里的内容(第 374)的时候，我们已经基于描述符相似性找出了候选匹配，但是却明显有很多错误的匹配。RANSAC 可以通过 ransac 方法来作用于一个 FeatureMatch 对象的向量：

```
>> F = m.ransac(@fmatrix, 1e-4, 'verbose')
1527 trials
312 outliers
0.000140437 final residual
F =
    0.0000   -0.0000    0.0098
    0.0000    0.0000   -0.0148
   -0.0121    0.0129    3.6393
```

其中试错的公差被设定为 10^{-4}。如果把该值变小，那么就需要更多的 RANSAC 试验，并且要求提高最大允许试验次数的上限，但是结果却不会有多大改变。期待有更小的残差也是不现实的，因为真实图像数据既有像素噪声这类的随机误差，也存在透镜畸变这类系统误差①。

RANSAC 识别出了 312 个外点，或 SURF 特征匹配阶段的错误数据关联，这个数量接近 50% 的候选匹配——初步匹配比它看起来更糟糕。运行 RANSAC 也会更新 FeatureMatch 向量中的元素：

```
>> m.show
ans =
1667 corresponding points
644 corresponding points
332 inliers (51.6%)
312 outliers (48.4%)
```

它现在显示了内点和外点的总体数量。向量中的元素为

```
>> m(1:5)
ans =
(819.56, 358.557) <-> (708.008, 563.342), dist=0.002137 +
(1028.3, 231.748) <-> (880.14, 461.094), dist=0.004057 -
(1027.6, 571.118) <-> (885.147, 742.088), dist=0.004297 +
(927.724, 509.93) <-> (800.833, 692.564), dist=0.004371 +
(854.35, 401.633) <-> (737.504, 602.187), dist=0.004417 +
```

① 透镜畸变造成点在像平面上发生位移，这背离了核面几何学。图像可以用 12.6.4 节中讨论的方法进行矫正，但是计算量很大。另一种更简便的方法是通过逆畸变模型方程(11.13)映射特征点来纠正它们的坐标。

与第 374 页对比，现在每行的最后多了一个正负号，它表示相应的匹配是内点还是外点[①]。我们可以绘制出一些内点：

```
>> idisp({im1, im2});
>> m.inlier.subset(100).plot('g')
```

或者一些外点：

```
>> idisp({im1, im2});
>> m.outlier.subset(100).plot('r')
```

它们如图 14.9 所示。

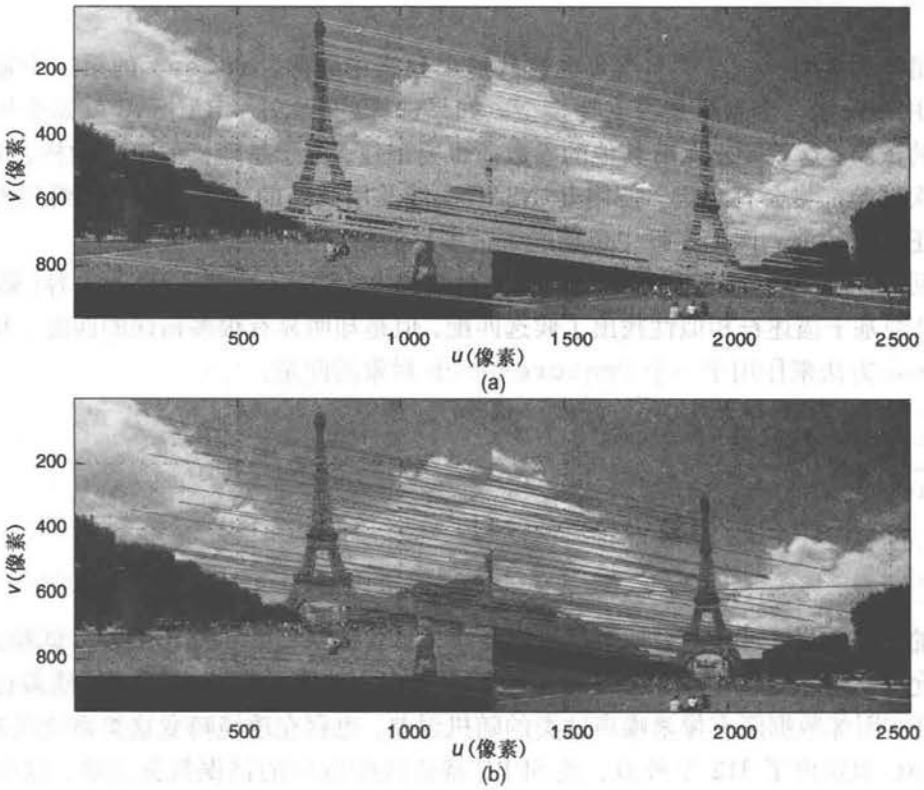


图 14.9 RANSAC 后进行 SURF 特征匹配的结果。(a)所有内点匹配的子集；(b)所有外点匹配的子集，有一些的错误很明显，而其他的不十分明显

另一种创建 CentralCamera 对象的方法是基于一个图像：

```
>> cam = CentralCamera('image', im1);
```

像素数组的大小是从这个图像推断出来的，内在参数被设为默认值。像以前那样，我们可以叠加由第二幅图像中的对应点计算出来的核线：

```
>> cam.plot_epiline(F', m.inlier.subset(20).p2, 'g');
```

结果如图 14.10 所示。所有核线在核点处相交，我们可以清楚地看到这个核点是第二个相机在第一幅图像中的投影[②]。核点位于：

① 第二个匹配已被确认为一个外点，尽管它是基于描述符相似度的第二强候选对。仅仅相似是不够的，两幅图像中的匹配点必须与基本矩阵代表的核面几何学相一致。

② 我们只绘制出一小部分的核线，因为它们数量太大，会使得图像变得模糊不清。

```
>> h2e( null(F))
ans =
   1.0e+03 *
    1.0359
    0.6709
>> cam.plot(ans, 'bo')
```

它也被叠加在图中。对于两个手持相机从同一视角拍摄的情况，我们已经能够在第一幅图中准确定位第二个相机。结果不是很完美——存在着一个大约 20 像素的水平偏移，原因可能是由于相机都是手持的，且只进行了近似同步，所有在一个或两个相机中存在着微小的指向误差①。

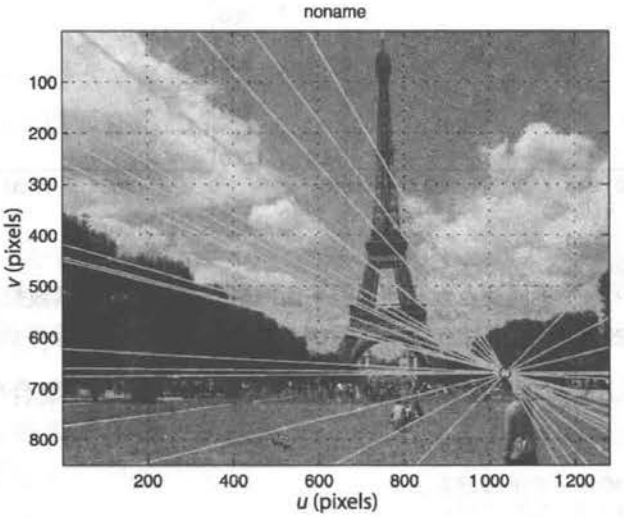


图 14.10　来自图 14.1(a)的图像，显示核线都聚合在第二个相机中心的投影点。本例中，第二个相机在图像的右下方清晰可见

14.2.4　平面单应性

在这一节中，我们将会考虑用相机观察平面上的一组空间点 $\mathbf{P}_i$。用两个不同的相机观察它们，在相机中的投影分别为 ${}^1\boldsymbol{p}_i$ 和 ${}^2\boldsymbol{p}_i$，其关系为

$$ {}^2\tilde{\boldsymbol{p}}_i \simeq \boldsymbol{H}\,{}^1\tilde{\boldsymbol{p}}_i \tag{14.12} $$

其中，$\boldsymbol{H}$ 是一个非奇异的 3×3 矩阵，称为单应性矩阵、平面的单应性矩阵或者由平面引入的单应性矩阵。

例如，再次考虑第 377 页上的一对相机，用它们观测一个 3×3 的网格点：

```
>> Tgrid = transl(0,0,1)*trotx(0.1)*troty(0.2);
>> P = mkgrid(3, 1.0, 'T', Tgrid);
```

其中，Tgrid 是网格坐标系 $\{G\}$ 的位姿，而网格点位于该坐标系 xy 平面的中心。把这些点投影到两个相机上：

```
>> p1 = cam1.plot(P, 'o');
>> p2 = cam2.plot(P, 'o');
```

图像分别如图 14.11(a)和(b)所示。

① 在像平面上焦距长度为一个 20 像素的位移相当于小于 0.5 度的指向误差。

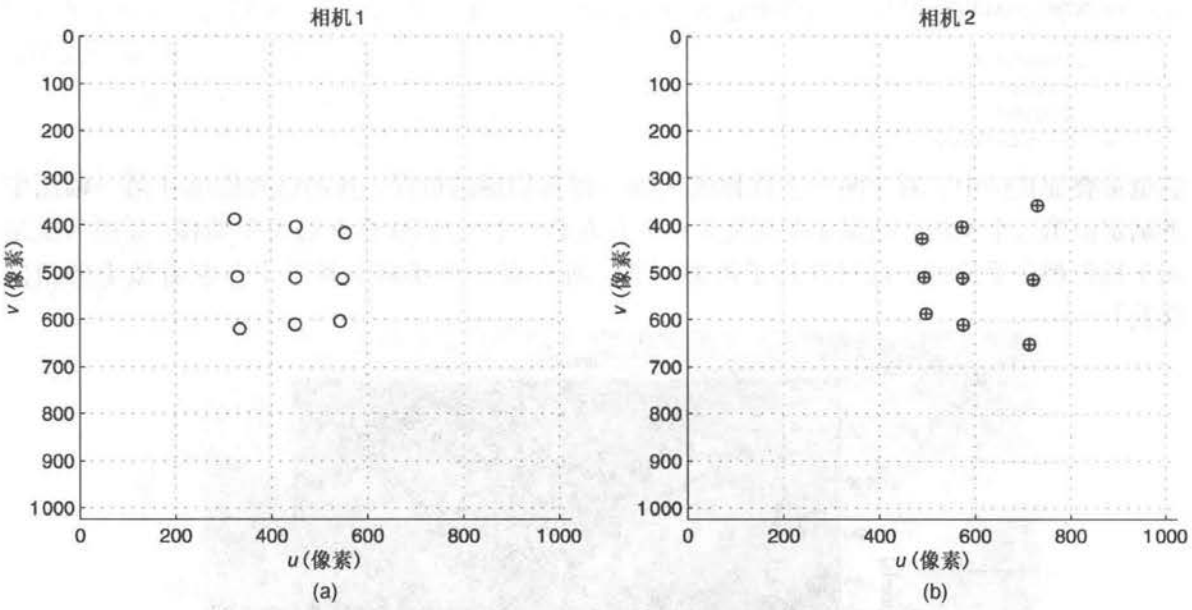


图 14.11 从两个不同视点观察的倾斜平面网格点视图。网格点被投影为空心圆。图(b)中的加号点表示了根据单应性矩阵从相机1像平面变换来的点

就像对基本矩阵那样，如果 $N \geqslant 8$，我们就可以从两个对应点集合中估计出矩阵 H：

```
>> H = homography(p1, p2)
H =
   -0.4282   -0.0006  408.0894
   -0.7030    0.3674  320.1340
   -0.0014   -0.0000    1.0000
```

根据方程(14.12)，可以从图像 1 中对应的坐标预测出图像 2 中的网格点位置：

```
>> p2b = homtrans(H, p1);
```

可以把这些预测点的位置用‘ + ’号叠加到图像 2 中：

```
>> cam2.hold()
>> cam2.plot(p2b, '+')
```

如图 14.11(b)所示，我们看到预测点刚好与空间点的实际投影吻合在一起。单应性矩阵的逆矩阵为

$$ {}^1\tilde{\boldsymbol{p}}_i \simeq \boldsymbol{H}^{-1}\, {}^2\tilde{\boldsymbol{p}}_i \tag{14.13} $$

它执行从图像 2 坐标到图像 1 的逆映射：

```
>> p1b = homtrans(inv(H), p2);
```

基本矩阵包含了位于一条直线上的共轭点，但是单应性矩阵告诉我们这些共轭点在另一幅图像中的具体位置——如果这些点在一个平面上。

我们可以用这个附带条件来检测几个点是否在同一个平面上。我们会把一些额外的空间点添加到示例中[①]：

```
>> Q = [
   -0.2302   -0.0545    0.2537
    0.3287    0.4523    0.6024
    0.4000    0.5000    0.6000  ];
```

① 这些点位于从相机 1 中心到网格平面中另外一行点的射线上。然而它们的 z 坐标已经分别选为 0.4 m、0.5 m、0.6 m。

我们绘制出它们的三维空间图形：

```
>> axis([-1 1 -1 1 0 2])
>> plot_sphere(P, 0.05, 'b')
>> plot_sphere(Q, 0.05, 'r')
>> cam1.plot_camera('color', 'b', 'label')
>> cam2.plot_camera('color', 'r', 'label')
```

如图 14.12 所示。这些新点显示为红色（浅灰色），很显然它们没有和最初的蓝色（黑色）点在同一个平面上。从相机 1 观察它们：

```
>> p1 = cam1.plot([P Q], 'o');
```

如图 14.13(a)所示，这些新点就像是上面所用网格点的另外一行。然而在第二个视图中它们是

```
>> p2 = cam2.plot([P Q], 'o');
```

如图 14.13(b)所示，这些平面外的新点不再形成有规律的网格。如果把单应性矩阵作用到相机 1 的图像点上：

```
>> p2h = homtrans(H, p1);
```

我们发现，如果这些点也属于相机 2 的像平面，那么它们在该平面上应当处于的位置都隐含在单应性矩阵中：

```
>> cam2.plot(p2h, '+')
```

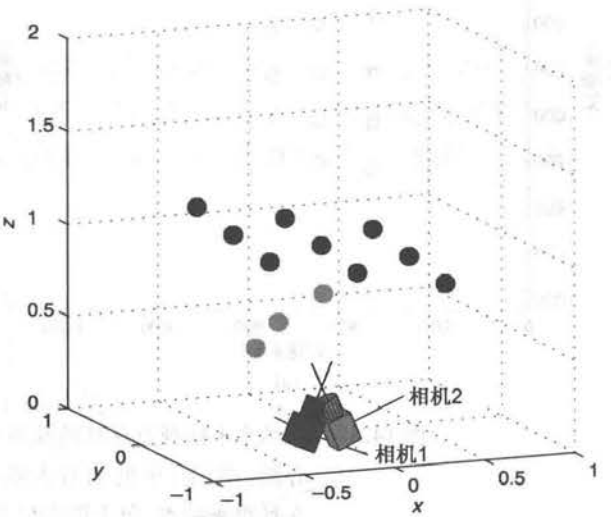


图 14.12　目标点和两个相机位姿的空间视图。蓝色（黑色）点位于平面网格中，而红色（或灰色）点从相机1的视点看似乎位于网格中

我们看到原来的 9 个点叠加上了加号，但是其他 3 个新点没有与加号叠加。我们可以基于预测误差来做一次自动检测：

```
>> colnorm( homtrans(H, p1)-p2 )
ans =
  Columns 1 through 9
    0.0000    0.0000    0.0000    0.0000    0.0000    0.0000
    0.0000    0.0000    0.0000
  Columns 10 through 12
   50.5969   46.4423   45.3836
```

我们看到最后 3 个点的值是比较大的——它们不属于引发单应性矩阵的平面。

在这个示例中，我们基于两组对应点估计了单应性矩阵，这两组对应点是已知平面点的两个投影。但在实际中，我们事先并不知道哪些点属于这个平面，所以要再次使用 RANSAC：

```
>> [H,in] = ransac(@homography, [p1; p2], 0.1)
resid =
   4.0990e-13
H =
   -0.4282   -0.0006  408.0894
   -0.7030    0.3674  320.1340
   -0.0014   -0.0000    1.0000
in =
    1    2    3    4    5    6    7    8    9
```

它找到了能最好解释两个图像点集合之间关系的单应性矩阵。它也识别出了那些支持单应性的点，而那 3 个平面外的点则不在内点集中。

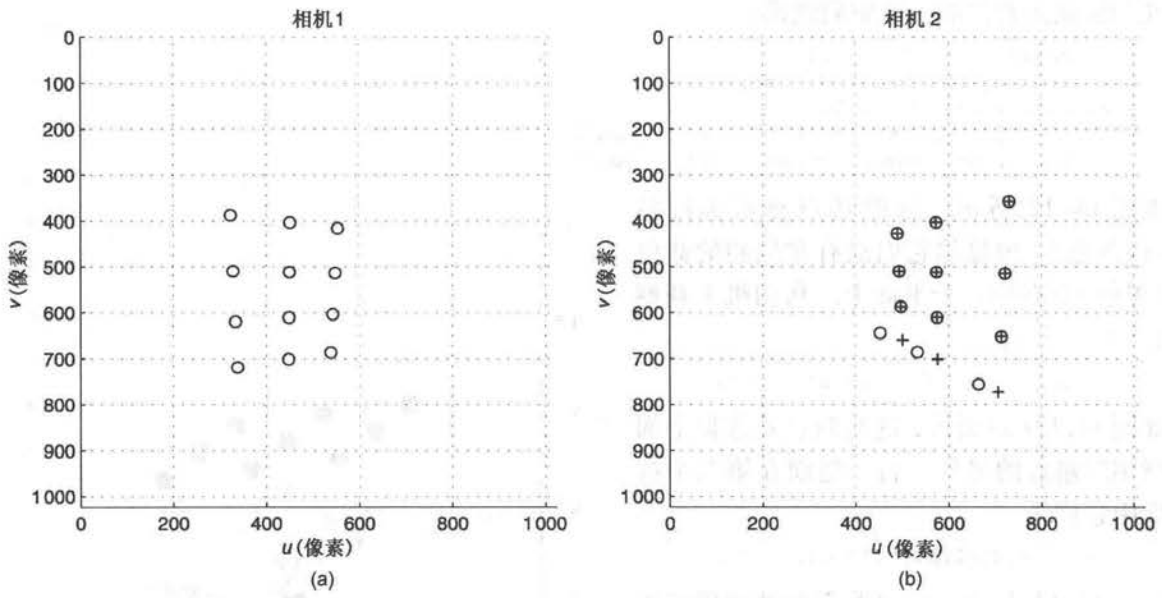


图 14.13 从两个不同视点观察的倾斜平面网格点视图。网格点投影为空心圆。图(b)中的加号表示根据单应性从相机1的像平面上变换出来的点。每个图中最下面一行的点与其他点不是共面的

与单应性相关的几何关系如图 14.14 所示。我们可以用归一化的图像坐标来表示单应性①:

$$^2\tilde{\boldsymbol{x}} \simeq \boldsymbol{H}_E\, ^1\tilde{\boldsymbol{x}}$$

其中，$\boldsymbol{H}_E$ 是欧几里得单应性矩阵，表达式为

$$\boldsymbol{H}_E \simeq \boldsymbol{R} + \frac{\boldsymbol{t}}{d}\boldsymbol{n}^{\mathrm{T}} \qquad (14.14)$$

其中包含了运动 $(\boldsymbol{R}, \boldsymbol{t}) \sim {}^2\xi_1$ 和相对于坐标系 $\{1\}$ 的平面 $\boldsymbol{n}^{\mathrm{T}}\boldsymbol{P} + d = 0$。欧几里得单应性矩阵与投影的单应性矩阵之间的关系是

$$\boldsymbol{H}_E \simeq \boldsymbol{K}^{-1}\boldsymbol{H}\boldsymbol{K}$$

其中 $\boldsymbol{K}$ 是相机的参数矩阵。

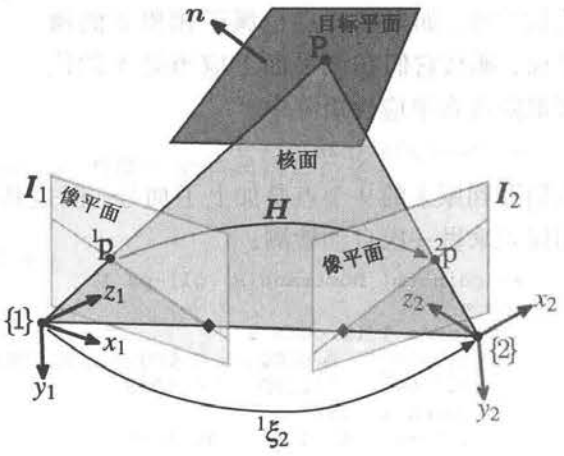


图 14.14 单应性的几何关系，显示了与两个相机相关联的坐标系 $\{1\}$、$\{2\}$ 及图像平面。空间点 $\mathbf{P}$ 属于表面法线为 n 的平面。$\boldsymbol{H}$ 是单应性矩阵，是一个将 ${}^1\mathbf{p}$ 映射为 ${}^2\mathbf{p}$ 的 3×3 矩阵

就像本质矩阵一样，投影单应性矩阵也可以被分解得到齐次变换形式的相对位姿 ${}^1\xi_2$，以及平面的法线。我们使用 `CentralCamera` 类的 `invH` 方法②:

```
>> cam1.invH(H)
solution 1
     T =0.82478     -0.01907     -0.56513     -0.01966
```

① 见第248页。

② 尽管方程(14.14)中是用 $(R, t) \sim {}^2\xi_1$，但是工具箱函数返回 ${}^1\xi_2$。

```
        0.01907      0.99980     -0.00591     -0.01917
        0.56513     -0.00591      0.82498      0.19911
        0.00000      0.00000      0.00000      1.00000
    n = 0.95519      0.00998      0.29582
solution 2
     T =0.69671      0.00000     -0.71736      0.18513
        0.00000      1.00000      0.00000     -0.00000
        0.71736     -0.00000      0.69671      0.07827
        0.00000      0.00000      0.00000      1.00000
    n = -0.19676     -0.09784      0.97556
```

它返回一个短的结构数组。它也有多个解，我们需要使用额外的信息来确定正确的一个。照例，变换矩阵的平移部分有未知的缩放因子。我们从图 14.12 中知道，相机运动主要是在 x 方向，并且平面法线几乎平行于相机的光轴——或者 z 轴，这些知识帮助我们选择第二个解。从相机 1 到相机 2 的真实变换是

```
>> inv(T1)*T2
ans =
    0.6967         0   -0.7174    0.1842
         0    1.0000         0         0
    0.7174         0    0.6967    0.0779
         0         0         0    1.0000
```

它也支持了我们的选择[①]。网格相对于相机 1 的位姿是

```
>> inv(T1)*Tgrid
ans =
    0.9797   -0.0389   -0.1968   -0.2973
    0.0198    0.9950   -0.0978         0
    0.1996    0.0920    0.9756    0.9600
         0         0         0    1.0000
```

其中第三列是网格的法线[②]，它与第二个解关联的估计法线相匹配。

我们可以把这个技术应用到一对真实图像中：

```
>> im1=iread('garden-l.jpg',  'double');
>> im2=iread('garden-r.jpg',  'double');
```

如图 14.15 所示。首先寻找 SURF 特征：

```
>> s1 = isurf(im1);
>> s2 = isurf(im2);
```

以及候选对应点：

```
>> m = s1.match(s2)
m =
323 corresponding points (listing suppressed)
```

然后使用 RANSAC 来找到最可能位于空间中同一个平面上的对应点集合：

```
>> [H,r] = m.ransac(@homography, 2)
H =
    0.9966    0.0061  -15.6385
   -0.0105    1.0081  -29.7619
   -0.0000    0.0000    1.0000
r =
    1.2228
```

内点和外点的数量是

```
>> m.show
ans =
323 corresponding points
83 inliers (25.7%)
240 outliers (74.3%)
```

① 平移缩放因子在这个示例中接近于 1，但通常情况下必须认为是未知的。

② 由于这些点在网格坐标系 $\{G\}$ 的 xy 平面中，所以法线是 z 轴。

本例中，大部分点对都与模型不相符，也就是说它们都不属于引发单应性矩阵 $\boldsymbol{H}$ 的平面。然而有 83 个点确实属于这个平面，我们可以在图像上叠加它们：

```
>> idisp(im1)
>> plot_point(m.inlier.p1, 'w*')
```

如图 14.15(a)所示。RANSAC 已经找到了一致性，那就是包含墙壁的平面。允许误差被设定为 4 个像素，这是因为这个场景中的主要平面(墙壁和草坪)都只是近似于平面。草坪很大，包含了很多 SURF 特征点，但是由 match 找到的草坪上的对应特征点数量很少。如果把这些内点从 FeatureMatch 向量中移除，也就是说只保留外点：

```
>> m = m.outlier
```

然后重复 RANSAC 单应性估计步骤，我们将会发现场景中下一个最主要的平面，结果是前面灌木的顶部。平面在人工环境中很是常见，我们将会在 14.5 节中重温单应性矩阵及其分解。

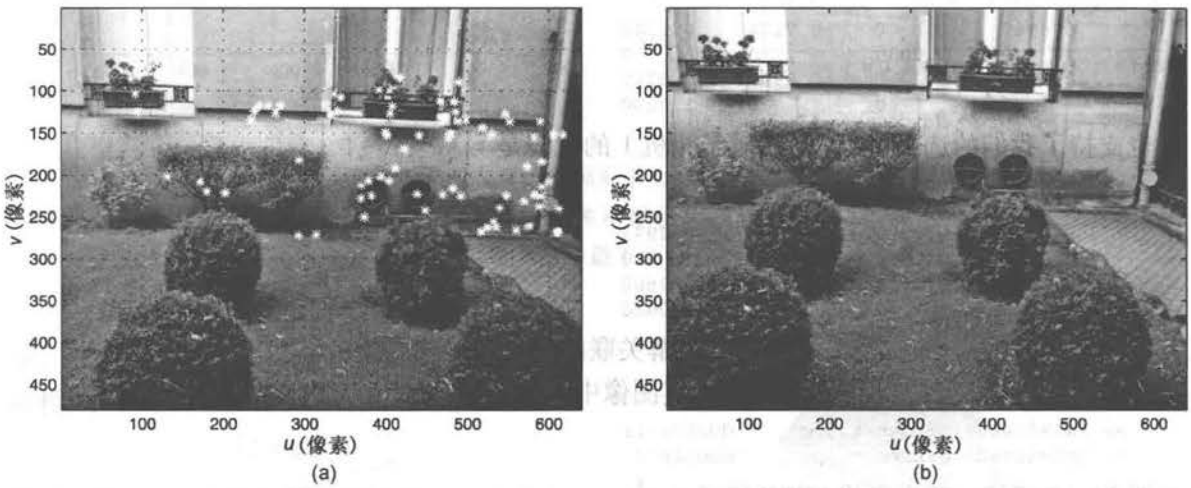


图 14.15 从不同视点拍摄的两幅庭院场景图像。图像(b)相对于图像(a)右移了近 30 cm。图像(a)上叠加了位于同一平面上的特征点。相机手持拍摄，$f = 5.2$ mm

14.3 立体视觉

立体视觉是一门根据不同视角拍摄的两幅图像估计三维空间结构的技术。我们将会讨论两种方法，分别称为稀疏立体匹配和密集立体匹配。稀疏立体匹配是我们学过的特征匹配的自然延伸，它针对每一组对应点恢复其空间坐标。密集立体匹配则是针对于图像中每一个像素点恢复其空间坐标。

14.3.1 稀疏立体匹配

为了说明稀疏立体匹配，我们回到图 14.15 所示的一对庭院图像。我们已经找到了 SURF 特征，所以将要确定基于描述符相似性的候选匹配：

```
>> m = s1.match(s2)
m =
323 corresponding points (listing suppressed)
```

然后估计基本矩阵：

```
>> [F,r] = m.ransac(@fmatrix,1e-4, 'verbose');
77 trials
82 outliers
0.000131656 final residual
```

它获得了两个视图的相对几何关系。我们可以把一部分点的核线叠加显示在左侧图像上：

```
>> cam = CentralCamera('image', im1);
>> cam.plot_epiline(F', m.inlier.subset(30).p2, 'r');
```

如图 14.16 所示。在这里，核线几乎是水平的，且互相平行，这与相机单纯在 x 方向上的平移运动是一致的。图 14.17 显示了立体视觉的核面几何学。很显然，随着空间点远离相机，从 $\mathbf{P}$ 到 $\mathbf{P}'$，右侧图像中的共轭点也沿着核线向右移动。

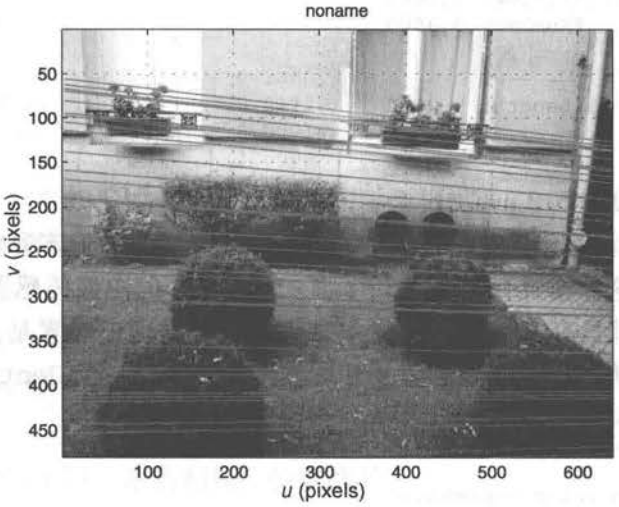


图 14.16　叠加了部分正确匹配点核线的图 14.15(a)所示的图像

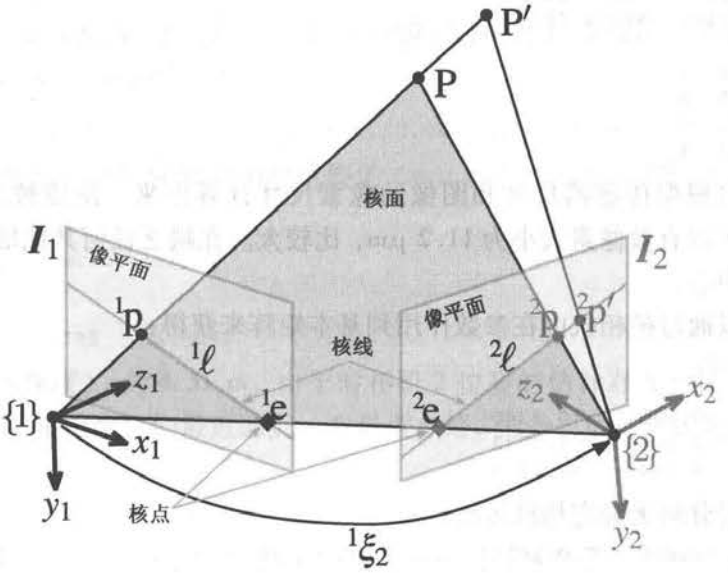


图 14.17　立体视觉的核面几何。可以清楚地看到，随着空间点深度从 $\mathbf{P}$ 到 $\mathbf{P}'$ 的增加，它在第二个图像平面上的投影会沿着核线移动

点 ${}^1\boldsymbol{p}$ 和 ${}^2\boldsymbol{p}$ 各自定义了空间中的一条射线，并在空间点处相交。然而为了确定这些射线，需要知道相机的两种位姿和它的内在参数。我们可以认为相机 1 的坐标系 {1} 还是最初那个，

没有变化，但是相机 2 的位姿仍然未知。然而我们可以通过分解从两个视图计算得到的本质矩阵来估计相机 2 的位姿。我们已经有了基本矩阵，但是为了根据方程(14.11)确定本质矩阵，我们需要相机的内在参数。经过小小的侦查，就可以找到它们！

拍摄图像时使用的焦距长度被存储在保存图像的 EXIF 格式文件头中，可以对其进行查询

```
>> [im1,tags] = iread('garden-1.jpg', 'double', 'mono');
```

其中，tags 是 MATLAB® 的一种结构，它把各种各样的图像特性包含在一个文本字符串结构中。元素 DigitalCamera 描述了相机的参数：

```
>> tags.DigitalCamera
ans =
              ExposureTime: 0.0667
                   FNumber: 3.3000
                         .
                         .
                FocalLength: 5.2000
                         .
                         .
```

从中可以确定焦距长度为 5.2 mm。

像素尺寸 $\rho_w \times \rho_h$ 没有包含在文件头中，但是一些网络上对于该型号相机的研究可以给出答案。研究报告显示这种相机有一个 7.18 mm × 5.32 mm 的图像传感器，并且像素为 3264 × 2448——这两组数字都具有不变的纵横比 4∶3。从中可以确定像素是正方形的，边长为 2.2 μm。我们基于已知的焦距长度、像素大小和图像尺寸来创建一个 CentralCamera 对象：

```
>> cam = CentralCamera('image', im1, 'focal', 5.2e-3, ...
    'sensor', [7.18e-3,5.32e-3])
cam =
name: noname [central-perspective]
  focal length:    0.0052
  pixel size:      (1.122e-05, 1.108e-05)
  principal pt:    (320, 240)
  number pixels:   640 x 480
  Tcam:
      1  0  0  0
      0  1  0  0
      0  0  1  0
      0  0  0  1
```

有效像素尺寸可以根据传感器尺寸和图像的像素尺寸计算出来。图像被二次采样的尺寸为 640 × 480 像素，所以有效像素大小为 11.2 μm，比较大。在缺乏任何其他信息的情况下，主点被假定为图像的中心。

本质矩阵可以通过把相机内在参数作用到基本矩阵来获得：

```
>> E = cam.E(F)
E =
   -0.0064   -0.5401    0.2240
    0.6914   -0.2065   -4.0732
   -0.1902    4.0445   -0.2493
```

然后可以对它进行分解来确定相机运动：

```
>> sol = cam.invE(E, [0,0,10]')
sol =
    0.9995   -0.0011   -0.0331   -4.0549
    0.0029    0.9985    0.0554   -0.2324
    0.0329   -0.0555    0.9979   -0.6766
         0         0         0    1.0000
>> [R,t] = tr2rt(sol);
```

我们选择一个测试点 ${}^1\mathbf{P} = (0,0,10)$，即一个沿光轴的远点，来确定相对相机运动的正确解。

由于相机的姿态保持恒定不变，所以变换矩阵中的旋转部分应该是我们看到的近似单位矩阵，实际上的旋转为

```
>> tr2rpy(R, 'deg')
ans =
   -3.1781   -1.8940    0.0602
```

仅仅是绕 x 轴和 y 轴的很小几度。

从 {1} 到 {2} 的估计平移 t 有一个未知的缩放因子。我们引入一条额外的信息——当我们拍摄图像时，相机位置沿 x 轴正向移动约 0.3 m。估计出的平移具有正确的方向，主要为沿 x 轴的运动，但是符号和大小都是错的。所以我们要对平移部分乘上一个缩放因子：

```
>> t = 0.3 * t/t(1);
>> T = rt2tr(R, t)
T =
    0.9995   -0.0011   -0.0331    0.3000
    0.0029    0.9985    0.0554    0.0172
    0.0329   -0.0555    0.9979    0.0501
         0         0         0    1.0000
```

我们得到了 ${}^1\xi_2$ ——齐次变换形式表示的相机 2 相对于相机 1 的相对位姿。

图像中每一个点 **p** 对应于空间中的一条射线[①]：

$$\mathbf{P} = \alpha \boldsymbol{d} + \mathbf{P}_0, \ \forall \alpha > 0$$

其中，$\mathbf{P}_0$ 是相机的中心，$\boldsymbol{d} \in \mathbb{R}^3$，$|\boldsymbol{d}| = 1$ 是一个射线方向上的单位向量。如果相机矩阵是 $\boldsymbol{C} = (\boldsymbol{\Pi}, \boldsymbol{c}_4)$，其中 $\boldsymbol{\Pi} \in \mathbb{R}^{3\times3}$，$\boldsymbol{c}_4 \in \mathbb{R}^3$，那么射线的参数是

$$\boldsymbol{d} \simeq \boldsymbol{\Pi}\tilde{\boldsymbol{p}}, \ \mathbf{P}_0 = \boldsymbol{\Pi}\boldsymbol{c}_4$$

现在考虑第一对匹配点 m(1)。来自相机 1 的射线是

```
>> r1 = cam.ray(m(1).p1)
r1 =
d=(0.429152, -0.210292, 0.878411), P0=(0, 0, 0)
```

这是一个 Ray3D 对象的实例，其属性 P0 和 d 分别表示 $\mathbf{P}_0$ 和 $\boldsymbol{d}$。来自第二个相机的对应射线是

```
>> r2 = cam.move(T).ray(m(1).p2)
r2 =
d=(0.38276, -0.21969, 0.897347), P0=(0.3, 0.017194, 0.0500546)
```

其中，move 方法返回一个带有指定相对位姿的 CentralCamera 对象的实例副本 cam。这两条射线相交于

```
>> [x,e] = r1.intersect(r2);
>> x'
ans =
    2.2031   -1.0796    4.5094
```

相交点的 z 坐标(或深度)，为 4.51 m。由于对相机 2 位姿的估计存在误差，这两条射线实际上不能相交，返回的是距离它们最近的点。在最近点处，两条射线的间距是

```
>> e
e =
    0.0049
```

大概为 5 cm。考虑到这个练习中存在的误差因素——采用两个手持相机拍摄，且只知道近似的相机镜头位移，我们恢复的深度数据已经很不错了[②]。

---

① 有时候也称为像素射线。它有很多表示方法，包括普吕克(Plücker)坐标。

② 即使相机位姿之间估计旋转部分的小误差也会导致若干米距离的闭合误差。这里观察到的闭合误差是由小于 1 度的旋转误差造成的。

我们从内点集合中选择一个有 20 个对应点的子集，并把它们绘制出来：

```
>> m2 = m.inlier.subset(20);
```

然后计算出针对每一个相机的空间射线：

```
>> r1 = cam.ray( m2.p1 );
>> r2 = cam.move(T).ray( m2.p2 );
```

每个都是 Ray3D 对象的一个向量。它们的相交点是

```
>> [P,e] = r1.intersect(r2);
```

其中，P 是代表最近点的一个矩阵，每一列是一个点，且最后一行是深度坐标：

```
>>  z = P(3,:);
```

向量 e 中的元素则是相应最近点处两条射线之间的距离。我们可以在庭院的图像上叠加到每一个点的距离：

```
>> idisp(im1)
>> plot_point(m.inlier.subset(20).p1, 'w*', 'textcolor', 'w', ...
    'printf', {'%.1f', z});
```

如图 14.18 所示，特征点处标注着估计的深度。

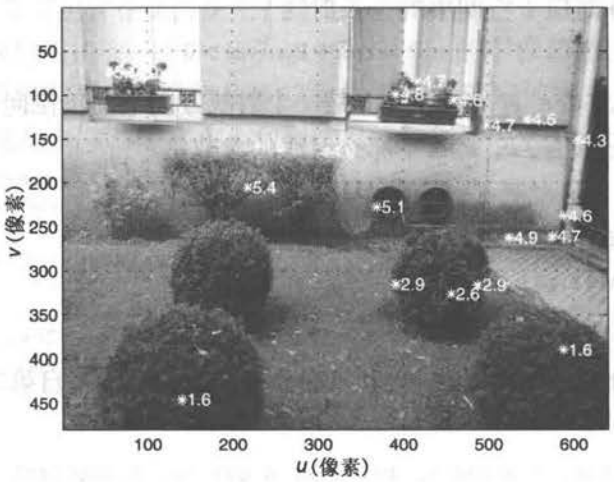


图 14.18　标有选择点深度的图 14.15(a)的图像(单位为米)

这是一个立体视觉的实例，在这里使用了从两幅部分重叠图像获得的信息来推断点在空间中的三维位置。由于显而易见的原因，这里使用的方法称为稀疏立体匹配，因为我们只计算了图像中一小部分像素的距离。更常见的情况是，两个相机之间的相对位姿以及相机的内在参数都已知。

14.3.2　密集立体匹配

一个立体像对是由两个相机拍摄的，两个相机的光轴通常是平行，而且被一个称为相机基线的已知距离分隔开。图 14.9 显示了一个典型的立体相机系统，它可以同时启用两个相机拍摄图像，并且把它们传到图像处理的主机上。

为了解释其原理，我们加载一个立体像对的实例：

```
>> L = iread('rocks2-l.png', 'reduce', 2);
>> R = iread('rocks2-r.png', 'reduce', 2);
```

我们可以交互式地同时查询这两幅图像：

```
>> stdisp(L, R)
```

如图 14.20 所示。点击左侧图像中的一个点可以同时更新右侧图像中标记相同坐标的一对十字准线。点击右侧图像则会设定另一条垂直的十字准线，并显示两个十字准线之间水平坐标的差异。图中显示的十字准线被定位在前方最左侧岩石的小白点，并且我们还观察到一些情况。首先，这个点在两幅图像中有相同的纵坐标，这表示核线是水平的。其次，右侧图像中的小白点向左移动了 82.4 像素。如果检测更多的点，就会看到越靠近相机的点，差异就越大。

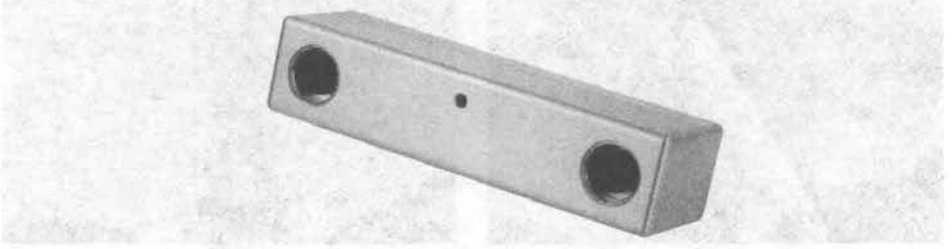


图 14.19　立体视觉传感器，它借助 PC 软件可以以每秒 25 帧的速度计算深度图。两边的相机和透镜是完全相同的，相对位姿恒定不变（图片由Point Grey Research公司提供）

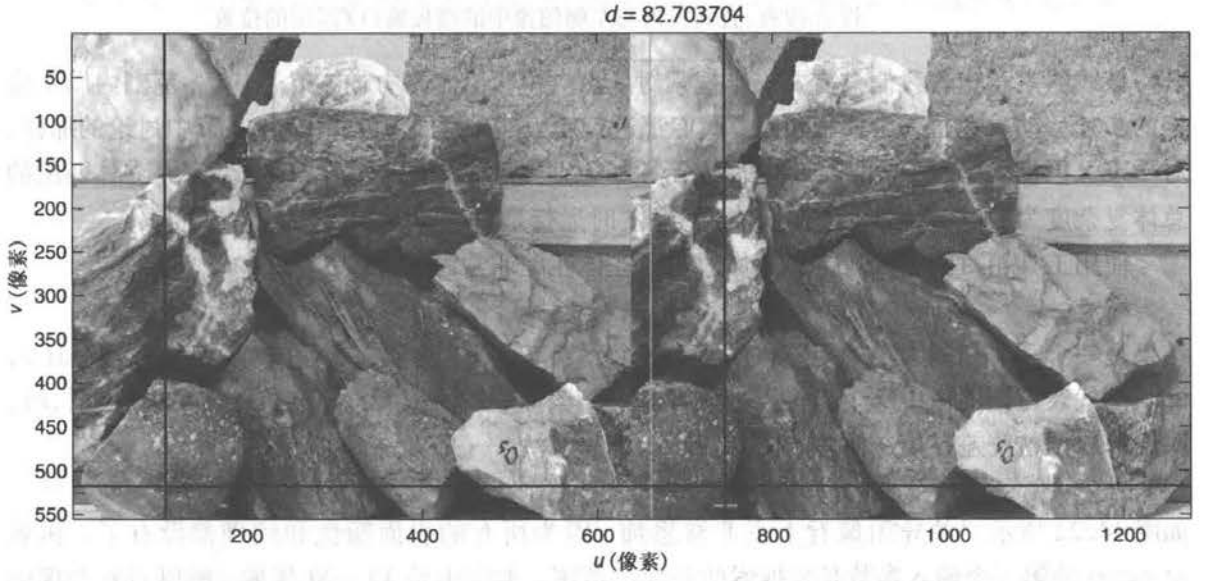


图 14.20　stdisp 图像浏览窗口。左侧图像中的黑色十字准线定位到左下方岩石上的一个小白点。另一个黑色十字准线被自动定位到右侧图像中相同的坐标处。点击右侧图像中的相应点设定一个绿色十字准线，顶部的文字表示在水平方向上向左移动了82.7像素。这个立体图像对来自Middlebury立体数据库[Scharstein and Pal，2007]。焦距长度f/p是3740像素，基线是160 mm。图像经过了剪裁，以使实际的差异为274个像素的偏移

如图 14.17 所示，右侧图像中的共轭点随着深度的增加会沿着核线向右移动。对于光轴平行的相机，它们的核线是平行的，并且是水平的，所以共轭点有相同的 v 坐标。如果两个对应点的坐标是 $({}^{L}u, {}^{L}v)$ 和 $({}^{R}u, {}^{R}v)$，那么 ${}^{R}v = {}^{L}v$。沿水平核线的位移 $d = {}^{L}u - {}^{R}u, d \geqslant 0$，其中 d 称为差异。

密集立体匹配过程可以通过图 14.21 说明。对于左侧图像中 $({}^{L}u, {}^{L}v)$ 处的像素，我们知道它在右侧图像中的对应像素在某一坐标 $({}^{L}u - d, {}^{L}v)$ 处，其中 $d \in [d_{\min}, d_{\max}]$。为了可靠地找到左侧图像中一个像素的对应点，我们创建一个关于那个像素的 $W \times W$ 像素模板区域 $\boldsymbol{T}$。如

图 14.21 所示，我们水平移动这个模板窗口穿过右侧图像。与该模板最相似的位置被认为就是对应点，从而可以计算出差异。与其他遇到的匹配问题相比，这个方法更为简单，因为两幅图像之间的相对尺度和姿态没有改变。

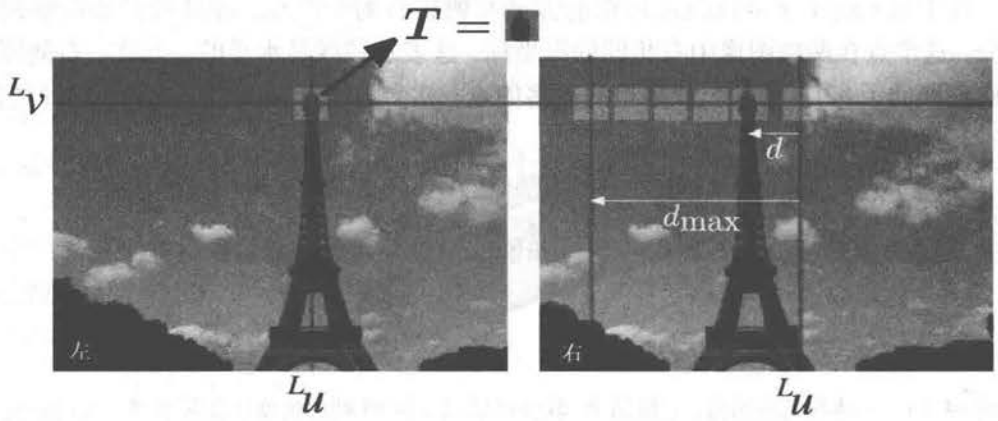


图 14.21 立体匹配。右侧图像中以 (u_L, v_L) 为中心进行沿水平方向的逐点搜查，直到找出与左侧图像中的模板窗口 $\boldsymbol{T}$ 匹配的位置

核线约束意味着我们只需要一维搜索对应点。模板沿水平方向在 $d_{min} \cdots d_{max}$ 范围内以 1 像素的步长移动。在每一个模板位置，我们都进行模板匹配运算，就像 12.4.2 节中讨论的那样。对于 $W \times W$ 像素的窗口，匹配计算的复杂度为 $O(W^2)$。如果模板移动了 D 步，密集立体匹配的总体复杂度为 $O(DW^2N^2)$，计算量较大，但实时运行是可行的。

使用工具箱对图 14.20 中的图像对进行立体匹配非常简单直接：

```
>> d = istereo(L, R, [40, 90], 3);
```

结果是一个和 L 相同大小的矩阵，并且每一个元素 $\boldsymbol{d}[u,v]$ 的值，或者 MATLAB® 中的 d(v, u)，都代表左侧图像中那个坐标点的差异。右侧图像中的对应元素将会位于 $(u - \boldsymbol{d}[u,v], v)$。我们可以把这些差异用一幅差异图像显示出来：

```
>> idisp(d, 'bar')
```

如图 14.22 所示。差异图像看上去非常恐怖，因为所有的表面颜色和纹理都没有了。函数 stereo 的第三个输入参数是要搜索的差异的范围，本例中是 40 ~ 90 像素，所以差异图像中的像素值在[40, 90]范围内。差异范围通过使用函数 stdisp 来检测一些远点和近点来确定①。istereo 的第四个参数是模板的半宽，本例中它表示我们正在使用的是 7 × 7 的窗口。默认情况下，stereo 使用 ZNCC 相似测度。

在差异图像中可以清楚地看到岩石堆底部的岩石有较大的差异，因为它们比那些顶部的岩石更靠近相机。其中也存在一些误差，例如岩石边缘附近的不规则明亮值。这表明这些像素点比实际距离离得更近。图像边缘的相似度得分被设定为 NaN，由于此处的相似度匹配模板已滑出图像的边缘；而针对 ZNCC 相似性测度(表 12.1)分母为 0 的情况②，相似度得分被设为 Inf。NaN 和 Inf 值都显示为黑色。

① 我们也可以选择[0, 90]这个范围，但是它会增加搜索时间：有 91 个差异需要进行评价，而不是 51 个。它也增加了匹配误差的可能性。

② 如果任一模板中的所有像素都具有相同的值，那么这种情况就会发生。

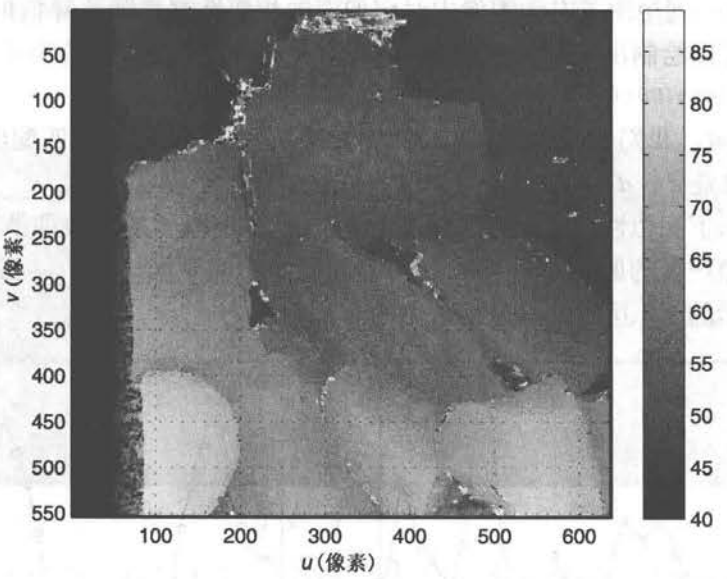


图 14.22　岩石堆立体图像对的差异图像，其中明亮的地方表示有较高的差异值或者较小的范围

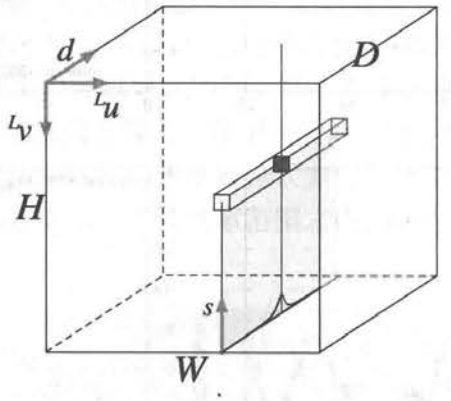


图 14.23　差异空间图像是一个三维图像，其中的元素 $D(u,v,d)$ 代表两个支持区域之间的相似度，左侧图像中的支持区域以 $({}^{L}u,{}^{L}v)$ 为中心，右侧图像中的支持区域以 $({}^{L}u-d,{}^{L}v)$ 为中心

14.3.2.1　立体失效模型

立体函数也可以返回差异空间图像(DSI)：

```
>> [d,sim,DSI] = istereo(L, R, [40 90], 3);
```

它是一个 $H \times W \times D$ 矩阵：

```
>> about(DSI)
DSI [double] : 555x638x51 (144468720 bytes)
```

元素 (u,v,d) 是模板之间的相似性测度，其中一个模板在左侧图像中并以 (u,v) 为中心，另一个模板在右侧图像中并以 $(u-d,v)$ 为中心[①]。我们先前看到的差异图像只是简单给出在每一个像素处沿 d 方向最大值的位置，而矩阵 sim 中则给出这些最大值[②]。

① 这是一个很大的矩阵(144 兆字节)，所以在加载图像时需要缩小图像尺寸。

② 这是一个简化方法。更好的方法是采用正则化，然后估计出一个符合最大相似度点的函数 $g(u,v)$，同时保持平滑性和连续性。

d 方向上的每一列给出了左侧图像中对应像素的相似性测度随差异值的变化。对于(138, 439)处的像素，可以绘制出该变化曲线：

```
>> plot( squeeze(DSI(439,138,:)), 'o-');
```

如图 14.24(a)所示。我们使用的是 ZNCC 测度方法，我们看到完美的匹配出现在差异为 80 像素处，因为水平轴是 $d - d_{\min}$，而 $d_{\min} = 40$。

图 14.24 展示了相似性测度随差异值变化的一些非常典型的函数曲线，它使我们可以洞察立体匹配中误差产生的原因。图 14.24(a)显示了一个非常明确的单一强峰值，它接近于完美匹配的值。幸运的是，这种情况是很常见的。

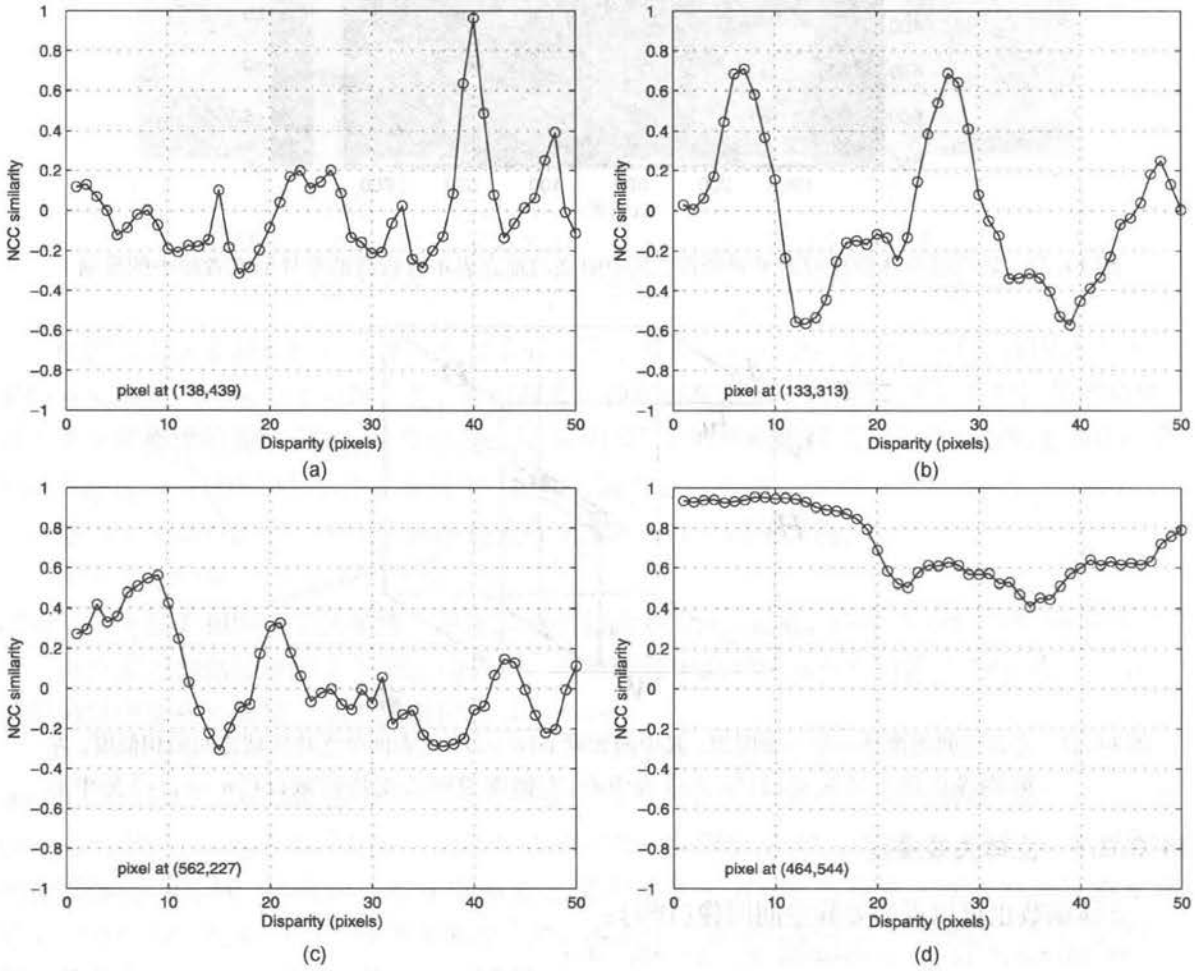


图 14.24 以差异为横坐标的一些典型 ZNCC 测度曲线

图 14.24(b)中显示了两个幅值几乎相同的峰值。这意味着模板的图案在搜索区域中被找到了两次。这种情况通常发生在有较规则垂直特征的场景中，如在人工场景中经常出现的砖墙、成排的窗户、建筑特征或者尖桩木栅栏。如图 14.25 所示，这个问题也通常被称为尖桩栅栏效应，但更准确的称呼应是空间假频。对于这个问题目前还没有真正的解决方案①，但是我们可以检测出它的存在。模糊比是第二个峰值与第一个峰值的高度之比——该比值大表示结

① 采用超过两个相机的多相机立体匹配，是解决这种模糊性的强有力的方法。

果是不确定的，不应该被采用。如果能让函数 istereo 中使用的差异范围尽可能小，就可以减少检到错误峰值的机会，但这要求预先了解目标对象的可能范围。

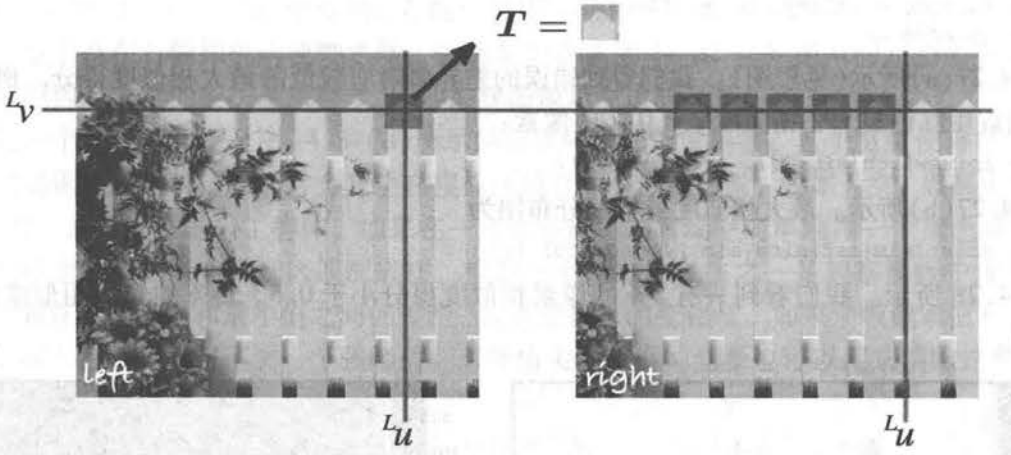


图 14.25　尖桩栅栏效应。模板将会与很多不同的差异位置产生良好匹配。这个问题会出现在有图案重复的任何场景中

图 14.24(c)显示了弱峰值。当右侧视图中对应场景点由于遮挡或部分丢失而不可见时，这种情况通常会发生。遮挡问题在图 14.26 中进行了说明。很明显，点 3 只对左侧相机可见。密集立体匹配将试图在右侧图像中找到最佳匹配，但是它找不到点 3 的对应点。尽管这个图形是一种夸张的描述，但是实际图像在深度变化很快的地方也有这个问题。例如，它会发生在图 14.22 中岩石的边缘，它们恰好是我们观察到错误差异的地方。这个问题会随着基线的增加而变得更为普遍。当对应点没有落在差异搜索范围之内，即差异搜索范围太小的时候，这个问题也会发生。

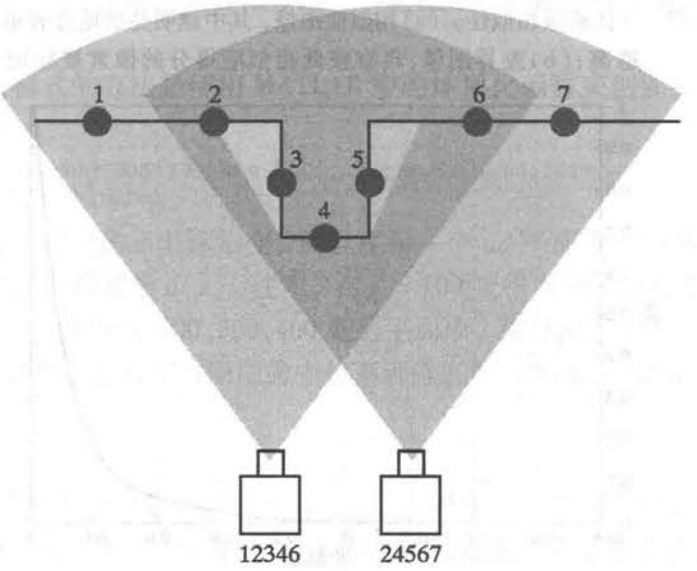


图 14.26　立体视觉中的遮挡。两个相机的视野显示为有色扇形区域。点 1 和点 7 落在重叠视图区域的外边，只能够被其中一个相机看到。点5由于遮挡不能被左侧相机看到，点3由于遮挡不能被右侧相机看到。能够被相机看到的点的序号在相机下面给出

尽管这个问题还没有解决，但是也可以被检测到。最简单的方法是对相似度得分限定一个阈值，并且忽略那些相似度低的差异结果。函数 istereo 可以返回顶峰的相似度得分值：

```
>> [d,sim] = istereo(L, R, [40 90], 3);
>> idisp(sim)
```

如图 14.27(a)所示(见彩图)。我们看到错误的差异值对应较低的最大相似度得分。例如，下面的函数用红色显示了相似度 $s < 0.7$ 的像素：

```
>> ipixswitch(sim<0.7, 'red', d/90);
```

如图 14.27(b)所示。最大相似度得分的分布图为

```
>> ihist(sim(isfinite(sim)), 'normcdf');
```

如图 14.28 所示。我们看到只有 5% 的像素相似度得分小于 0.5，55% 的像素相似度得分高于 0.9。

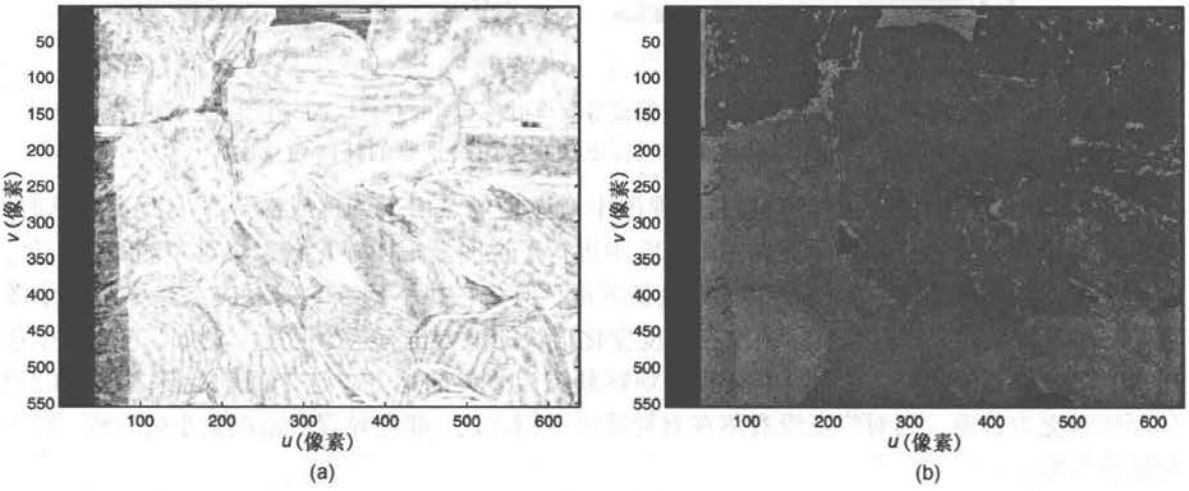


图 14.27 立体模板相似性。(a)相似性图像，其中越明亮的地方表示相似度越高；(b)差异图像，具有较低相似度得分的像素被标记为红色

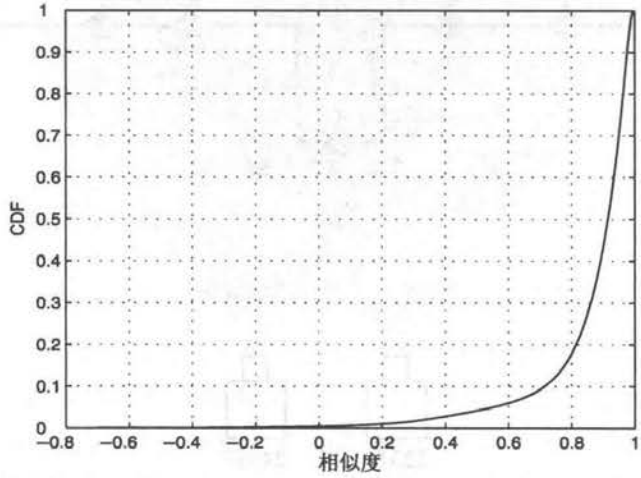


图 14.28 ZNCC 得分的累积概率。得分小于 0.9 的概率为 45%

测试遮挡的更为强大的方法是在两个方向上进行匹配，这称为左右一致性检测。从左侧图像中的一个像素开始，找到右侧图像中的最强匹配。然后再在左侧图像中找到与最强匹配

对应的像素。如果这个像素就是左侧图像中的那个原始像素，那么匹配可以考虑为有效匹配，否则放弃。

从图 14.26 中可以很清晰地看到，左侧图像中左边的那些像素可能完全和右侧图像没有交集——点 1 就在右侧相机的视野之外。这就是为什么图 14.22 差异图像的左边会有大量错误匹配的原因。通常的做法是丢弃 $d_{\max}$，即差异图像最左边的几列。

最后一个出现的问题是图 14.24(d)所示的有宽峰的相似度函数。这个宽度使得精确估计最大值变得困难。这种情况通常会发生在模板区域有很少的纹理时，例如天空，黑暗阴影，水面、雪、冰、或者平滑的人工物体。如果在一片全是灰色的区域中简单地用一个灰色模板，那么它将与任意数量的灰色候选区匹配成功。检测这个问题的方法之一是观察模板中像素值的变化性，如查看最大值和最小值之间的差距，或者像素值的变化量。如果模板像素值有很小的变化量，那么它很可能得不到一个强峰值。对峰值尖锐程度的测量也可以用来消除这种情况，这将在下一节中讨论。

对于上面讨论的各种各样的问题，虽然差异无法断定，但是问题可以被发现。这一点是很重要的，因为它允许那些像素被标记为无已知范围，这样可以让机器人路径规划器对于那些三维结构无法可靠确定的区域更加小心谨慎。

一个立体视觉系统的设计有三个可选参数。第一个是相机之间的基线距离。随着基线的增加，差异会增大，有可能对深度进行更加精确的估计，但是遮挡问题会更加严重。第二个是差异搜索范围。它的设定需要谨慎，如果最大值太大，空间假频的概率就会增加，但是如果太小，那么靠近相机的点将会产生错误匹配和弱匹配。大的差异范围还会增加计算时间。第三是模板大小。它需要在计算时间和差异图像质量之间进行权衡。小模板尺寸可以获得好的深度结构，但是往往给出的结果噪音很大，因为小模板对模糊匹配更加敏感。大模板会给出一幅更加平滑的差异图像，但是会导致计算时间更长。另外，很有可能模板中将会包含那些在不同深度目标对象上的像素，这被称为混合像素问题。它往往在目标边缘处造成较差的匹配，并使差异图模糊不清。

另一种观察失效模式的方法是使用 MATLAB 中的体视化函数来创建穿过差异空间图像的水平切片：

```
>> slice(DSI, [], [100 200 300 400 500], []); view(-52,18)
>> shading interp; colorbar
```

如图 14.29 所示。各个切片的 v 坐标不变，而且在每一个 ud 平面中我们都看到有一条明亮路径(高相似度值)，它代表着差异 $d(u)$。注意，在 v = 100 处平面中的路径是断开的，它们对应深度方向上的突然变化。在 v = 200,300,400 处的平面中，路径在某些地方也消失了。在这些区域中，最大相似度较低，并且在右侧图像中找不到强匹配，而这最可能的原因是遮挡。

14.3.3　峰值细化

每一个像素点的差异是一个整数值 $d \in [d_{\min}, d_{\max}]$，从中可以找到最优的相似性。图 14.24(a)显示了一个非常明确的单一强峰，我们可以使用该峰值邻近的差异值来改善对峰值位置的估计。用一条抛物线来拟合最优差异及其邻近部分[①]：

$$s = Ad^2 + Bd + C \tag{14.15}$$

① 这个二维峰值细化处理在附录 K 中进行了详细讨论。

最优的 ZNCC 相似性测度是一个最大值，这就意味着抛物线应是倒转朝下的，即 $A < 0$。当方程(14.15)的导数等于 0 时，就会出现插值曲线最大值，从中可以估计出拟合抛物线峰值时的水平位置：

$$\hat{d} = \frac{-B}{2A}$$

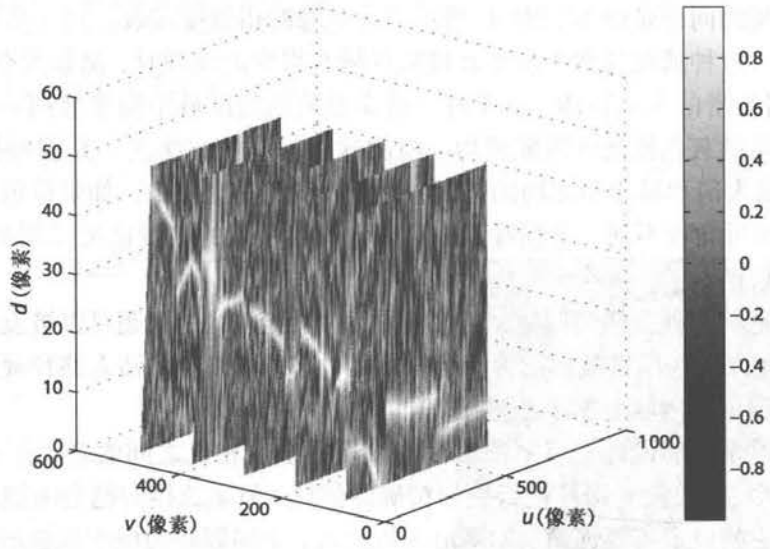


图 14.29 差异空间图像是一个三维图像，其中的元素 $D(u,v,d)$ 代表两个支持区域之间的相似度，左侧图像中的支持区域以(${}^{L}u$,${}^{L}v$)为中心，右侧图像中的支持区域以(${}^{L}u-d$,${}^{L}v$)为中心

对于尖锐的峰值，系数 A 比较大；而对于宽峰的情况，可用一个简单的阈值来处理，这一点将在下一节讨论。

差异峰值细化处理可以通过'interp'选项来执行：

```
>> [di,sim,peak] = istereo(L, R, [40 90], 3, 'interp');
>> idisp(di)
```

产生的差异图像如图 14.30(a)所示。我们看到它比先前图 14.22 中显示的那一幅图像更加平滑。

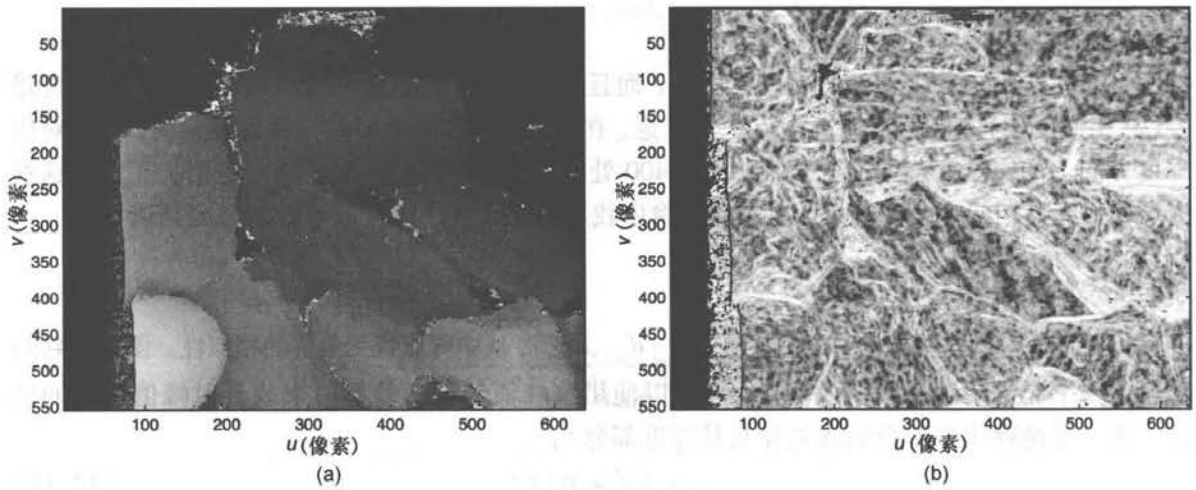


图 14.30 (a)有峰值细化的差异值图像；(b)每一个峰值处 d^2 项的系数。大值(明亮处)对应着尖锐的山峰，它出现在图像纹理高的地方。宽峰(黑暗处)出现在图像纹理低的地方

增加的可选输出参数 peak 是一个结构：

```
>> peak
peak =
     A: [555x638 double]
     B: [555x638 double]
```

它包含了每个像素抛物线系数的值。系数 A 显示为图 14.30(b)所示的图像。

14.3.4　清理和重构

鉴于之前阐述过的原因，立体匹配的结果还存在很多不完美之处，如图 14.22 所示。对于机器人的应用，例如路径规划和越障，了解环境空间三维结构是很重要的，但知道那些我们不了解的东西也极其关键。在那些未能从立体视觉中获得可靠深度信息的地方，机器人应当很谨慎，要与自由环境空间区别对待。我们可以采用很多简单措施来标记差异图像中的元素，以表示无效或者不可靠。

首先创建一个矩阵 status，大小与 d 相同，并初始化为 0：

```
>> status = zeros(size(d));
```

如果元素对应着特定的失效条件，那么把它们设定为非零值：

```
>> [U,V] = imeshgrid(L);
>> status(isnan(d)) = 4;
>> status(U<=90) = 1;
>> status(sim<0.8) = 2;
>> status(peak.A>=-0.1) = 3;
```

可以把这个矩阵显示为一幅图像：

```
>> idisp(status)
>> colormap( colorname({'lightgreen', 'cyan', 'blue', 'orange', 'red'}) )
```

如图 14.31 所示(见彩图)。选择颜色地图来显示状态值，如果是好的立体匹配，显示为浅绿色；如果差异搜索范围超过右侧图像的左边缘，显示为蓝绿色；如果峰值相似度太小，显示为蓝色；峰值太宽，显示为橙色；红色则对应 NaN 值，表示搜索模板已超出图像边缘。令人高兴的是，图上大多数为浅绿色像素！实际上，

```
>> sum(status(:)) / prod(size(status)) * 100
ans =
   86.0674
```

有接近 90% 的差异值通过了我们的质量检测。表示弱峰的蓝色像素出现在岩石的边缘，它们是由于遮挡造成的。表示宽峰的橙色像素出现在相当平滑的地方，要么是岩石之间的深色阴影，要么是非岩石的背景。

先前我们创建过一幅插值处理的差异图像，现在我们要让那些已确定为不可靠的差异值失效：

```
>> di(status>0) = NaN;
```

把它们的值设定为 NaN[①]。可以把这些不可靠像素标记为红色显示出来：

```
>> ipixswitch(isnan(di), 'red', di/90);
```

如图 14.32 所示[②]。现在这种形式对于机器人来说就有用了——它包含了经插值处理的更好差异值，并且所有的不可靠值都被清晰地标记出来。

---

① 特殊的浮点值 NaN(因为不是一个数值)有个很有用的属性，就是任何涉及 NaN 的算术运算结果都是 NaN。很多 MATLAB® 函数，例如 max 或者 min，在输出矩阵中都忽略 NaN 值，而且绘图和制图函数也不会显示这个值，而是在图表上留一个洞。

② 除以 90 可以把[40,90]范围内的浮点差异值转换成范围在[0,1]内的有效灰度值。

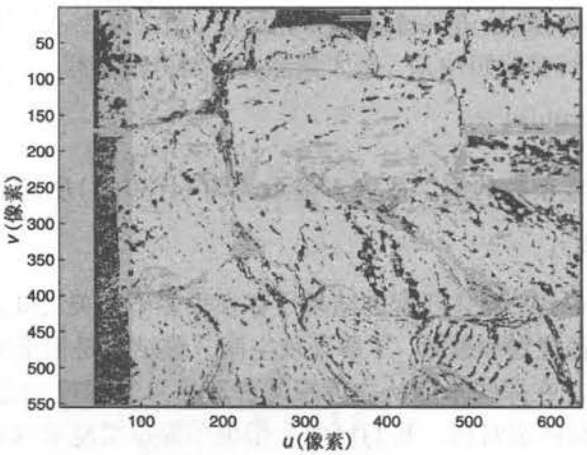


图 14.31 每个像素基础上的立体匹配状态。好的立体匹配(绿色)；差异搜索范围超过右侧图像左边缘(蓝绿色)；最大相似度较低(蓝色)；非尖锐峰值(橙色)；搜索模板超出图像边缘(红色)

最后一步是把以像素计的差异值转换成以米计的空间坐标——这个过程称为三维重构。在前面基于稀疏立体匹配的讨论中，我们根据三维空间中两条射线的相交点确定了空间点坐标。对于图 14.19 所示的平行轴立体相机设备，其几何关系采用图 14.33 来描述会更加简单。对于红色和蓝色三角形，可以写出

$$X = Z\tan\theta_1,\ X - b = Z\tan\theta_2$$

其中，b 是基线，射线的角度对应水平图像坐标 ${}^{i}u$，$i = \{L,R\}$：

$$\tan\theta_i = \frac{\rho_u({}^{i}u - u_0)}{f}$$

代入并消除 X，得到

$$Z = \frac{fb}{\rho_u({}^{L}u - {}^{R}u)} = \frac{fb}{\rho_u d}$$

它显示出深度与差异 $d = {}^{L}u - {}^{R}u$，$d > 0$ 成反比。

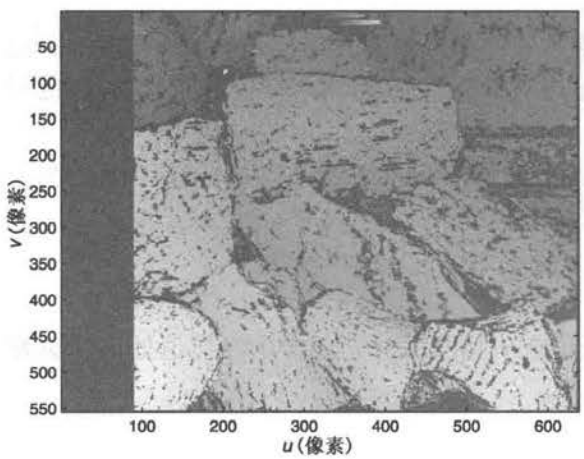


图 14.32 插值处理的差异图像，不可靠的估计显示为红色(本图中显示为各深色区域)

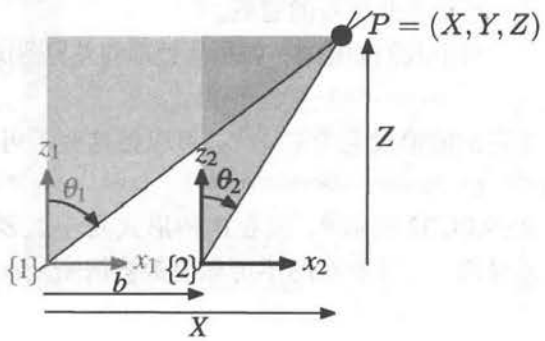


图 14.33 相机轴平行的立体几何学。X 和 Z 是相对于相机1被测量的，b是基线

我们也可以恢复 X,Y 坐标：

$$X = \frac{b({}^{L}u - u_0)}{d},\quad Y = \frac{b({}^{L}v - v_0)}{d}$$

图 14.20 所示的图像(来自 Middlebury 数据库)拍摄时的相机基线很宽。左侧图像的左边缘和右侧图像的右边缘没有重叠，并且是经过剪裁的。从左侧图像的左边剪掉 N 个像素就把差异减小了 N。对于这个立体图像对，实际的差异必须增加 274，以补偿剪裁。

实际差异为

```
>> di = di + 274;
```

我们把每一个像素的 X,Y,Z 坐标作为分离矩阵来计算，以充分利用 MATLAB 的高效矩阵运算①：

```
>> [U,V] = imeshgrid(L);
>> u0 = size(L,2)/2; v0 = size(L,1)/2;
>> b = 0.160;
>> X = b*(U-u0) ./ di; Y = b*(V-v0) ./ di; Z = 3740 * b ./ di;
```

可以把它显示为一个曲面：

```
>> surf(Z)
>> shading interp; view(-150, 75)
>> set(gca,'ZDir', 'reverse'); set(gca,'XDir', 'reverse')
>> colormap(flipud(hot))
```

如图 14.34 所示。这个图印刷出来好像不怎么样，但是如果通过 MATLAB ® 图形工具条三维旋转选项用鼠标旋转图形，那么它的三维形态就表现得非常清楚了。为了从我们的视角让 z 增加，以及维持右手坐标系，需要将轴反转。在这个曲面上有很多代表 NaN 值的洞，它们表示不可靠的差异值。

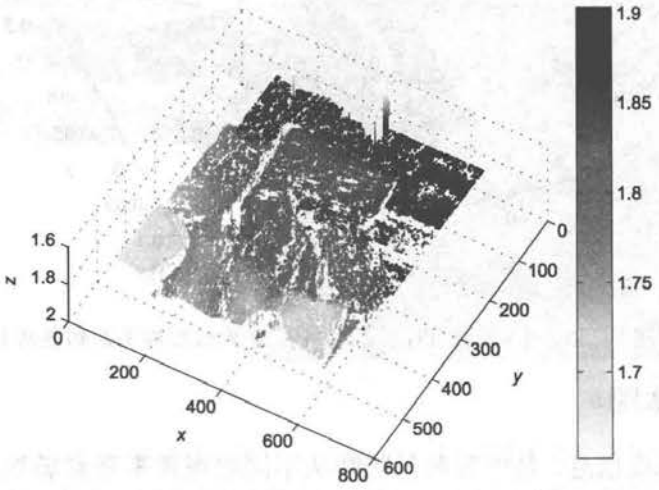


图 14.34　平行轴立体相机的三维重构。颜色越暖，表示它是距离相机越近的表面部分

14.3.5　三维纹理映射显示

对于人类观察者而不是机器人来说，改善曲面的表现形式是有必要的，这样可以使它看起来更加整洁。我们创建一个中值滤波图像：

① 这个过程称为向量化。用矩阵和向量运算代替 for 循环可以增加 MATLAB ® 代码执行的速度。更多详情见 http://www.mathworks.com/support/technotes/1100/1109.html。

```
>> dimf = irank(di, 41, ones(9,9));
```

其中每个输出像素是一个 9 ×9 窗口上的中值。这样就修补了很多较小的洞，但是也产生了不希望的副作用——它使差异图像变得模糊。因此，我们将会保留原始的插值差异图像，只把中值过滤值使用到 NaN 存在的地方：

```
>> di = ipixswitch(isnan(di), dimf, di);
```

再次进行重构：

```
>> X = b*(U-u0) ./ di;  Y = b*(V-v0) ./ di; Z = 3740 * b ./ di;
```

并把它绘制成一个曲面，这个曲面看起来明显更加整洁了。

然而我们还可以做得更好。我们可以使用一种称为纹理映射的处理方法，把左侧图像覆盖到三维表面上。再次加载左侧图像，这一次为彩色：

```
>> Lcolor = iread('rocks2-l.png');
```

然后将图像纹理映射到曲面上：

```
>> surface(X, Y, Z, Lcolor, 'FaceColor', 'texturemap', ...
    'EdgeColor', 'none', 'CDataMapping', 'direct')
>> xyzlabel
>> set(gca,'ZDir', 'reverse'); set(gca,'XDir', 'reverse')
>> view(-160, 75)
```

它产生一幅图像，如图 14.35 所示。可以再次通过 MATLAB ® 图形工具条三维旋转选项来用鼠标旋转图形，从而很容易地得到三维的图像。

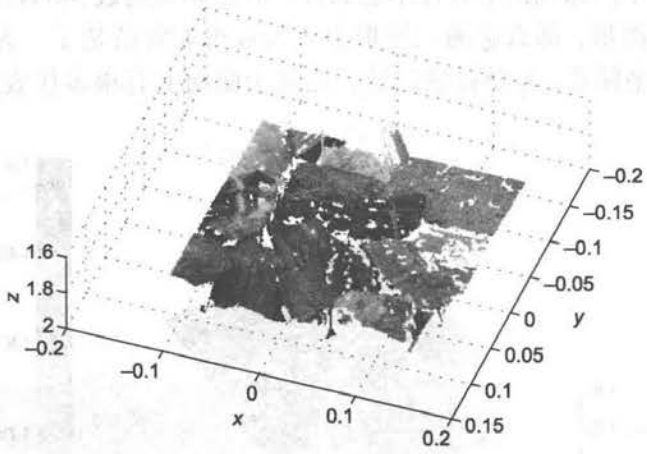


图 14.35 平行轴相机的三维重构，图像纹理被映射到表面上

14.3.6 补色立体图

人类能够感知深度信息，是因为两只眼睛从不同的视角来观察场景。如果我们看一幅三维场景的照片，尽管会减弱，但仍然能得到对深度的感知，这是因为我们的大脑除了立体感知外还使用了很多视觉线索来推断深度。自从 19 世纪摄影发明以来，人们就对三维摄影和三维电影一直抱有浓厚的兴趣，近期 3D 电视的发明就是进一步的佐证。

所有 3D 显示技术的关键是用两个相机拍摄图像，然后再把这些图像呈现到人眼里，两个相机与人的双眼有相同的基线(接近 8cm)。老式立体图需要一套双目观察设备，或者很困难地眯着眼斜视立体图像对。现在，人们观察立体图像对时使用更为现代和方便的液晶快门眼镜或偏光眼镜，还可以用它们观看全彩立体电影。

背景材料：补色立体图。该技术最早出现在法国。1858 年约瑟夫·阿尔梅达(Joseph D'Almeida)把 3D 幻灯片投影显示为红蓝立体照片，观众佩戴红蓝护目镜观看。大约在同一时间，路易斯·奥龙(Louis Du Hauron)打印出了第一张立体照片。之后，大约在 1890 年，威廉·弗莱斯·格林(William Friese-Green)使用双镜头的相机创作了第一部 3D 补色立体动画。在 20 世纪 20 年代，一部名为 *plasticons* 或 *plastigrams* 的补色立体电影曾引发了一阵狂热。

一个世纪之后，人们从网络上已能获得火星表面的补色立体图像 http://marsprogram.jpl.nasa.gov/MPF/mpf/anaglyph-arc.html。

观看和分配立体信息的一种古老但廉价的方法是补色立体图像技术，即在左右两侧图像中分别叠加上不同的颜色。通常红色用于左眼，蓝绿色(发绿的蓝色)用于右眼，但是也可以用很多其他颜色的组合。红色透镜只允许立体图像中的红色部分透过达到左眼，而蓝绿色透镜只允许立体图像的蓝绿色部分透过达到右眼。它的缺点是只能描绘出场景强度，无法描绘颜色。补色立体法的很大一个优点是可以在纸上打印出立体图或者在普通电影中成像，并且可以使用类似图 14.36(a)中那种简单便宜的眼镜进行观看。

岩石堆立体图像对可以被描绘成一个补色立体图：

```
>> anaglyph(L, R, 'rc')
```

如图 14.36(b)所示。参数'rc'表示左侧图像和右侧图像分别用红色和蓝绿色进行了编码。其他颜色选项包括：蓝色、绿色、洋红色和橙色。

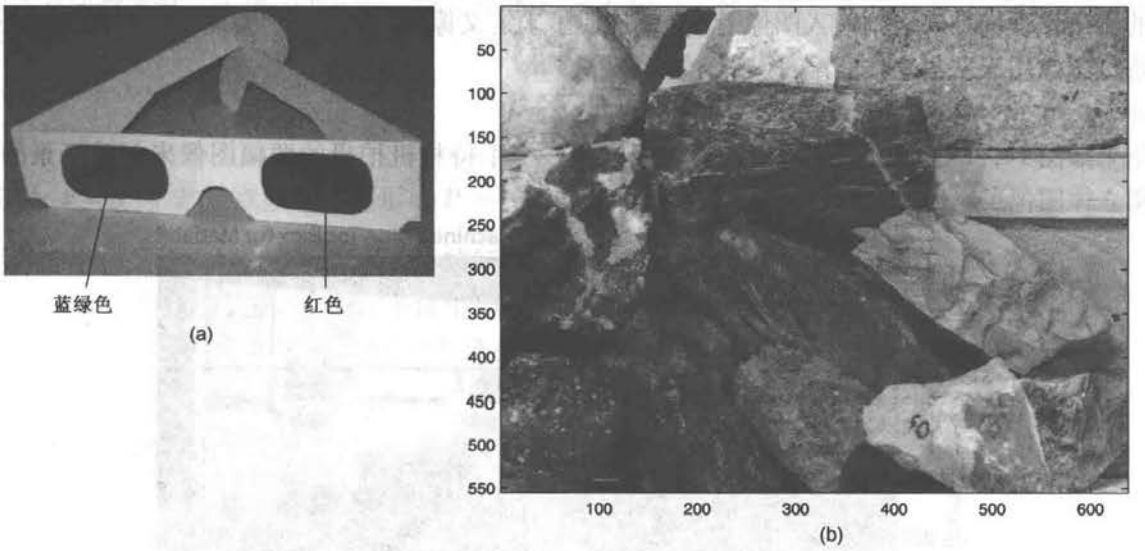


图 14.36　具有立体视感的补色立体图。(a)拥有红蓝镜片的立体眼镜；(b)图14.20岩石场景的补色渲染，左侧为红色，右侧为蓝绿色

14.3.7　图像矫正

图 14.20 的岩石堆立体图像对在左侧和右侧图像的同一行上有对应的点，因此该图像对也被称为一个核线对齐图像对。在本书第 395 页上的立体相机需要很高的制造精度，以确保相机的光轴是平行的，两个传感器芯片的 u 轴和 v 轴也是平行的。然而机械调校的精度是有限制的，透镜畸变也会产生误差。通常，其中的一幅或两幅图像需要通过变形来纠正这些误差——这个过程称为矫正。

我们将使用图 14.15 所示的庭院立体图像对来阐释矫正问题：

```
>> L = iread('garden-l.jpg', 'mono', 'double');
>> R = iread('garden-r.jpg', 'mono', 'double');
```

我们知道它们的核线是没有对齐的。首先找出 SURF 特征：

```
>> sL = isurf(L);
>> sR = isurf(R);
```

再确定坐标匹配：

```
>> m = sL.match(sR);
```

然后确定核线关系：

```
>> F = m.ransac(@fmatrix,1e-4, 'verbose');
46 trials
81 outliers
9.60297e-05 final residual
```

矫正这个步骤需要用到基本矩阵和对应点集合，对应点集合包含在 FeatureMatch 对象的 m 中：

```
>> [Lr,Rr] = irectify(F, m, L, R);
```

以上函数返回两幅输入图像的矫正版本。我们使用 stdisp 把它们显示出来：

```
>> stdisp(Lr, Rr)
```

如图 14.37 所示。我们看到现在场景中的对应点有相同的纵坐标。函数 irectify 通过计算唯一单应性矩阵来变形左侧和右侧图像。正如先前观察到的那样，当变形图像时，并不是所有的输出像素都被映射到输入图像上，这就造成了未定义像素，图中显示为黑色。现在将这一对矫正图像用于密集立体匹配：

```
>> d = istereo(Lr, Rr, [5 120], 4);
```

结果如图 14.38 所示。我们已经能够使用只从一个手持相机拍摄的两幅图像来创建场景的密集立体图像了。

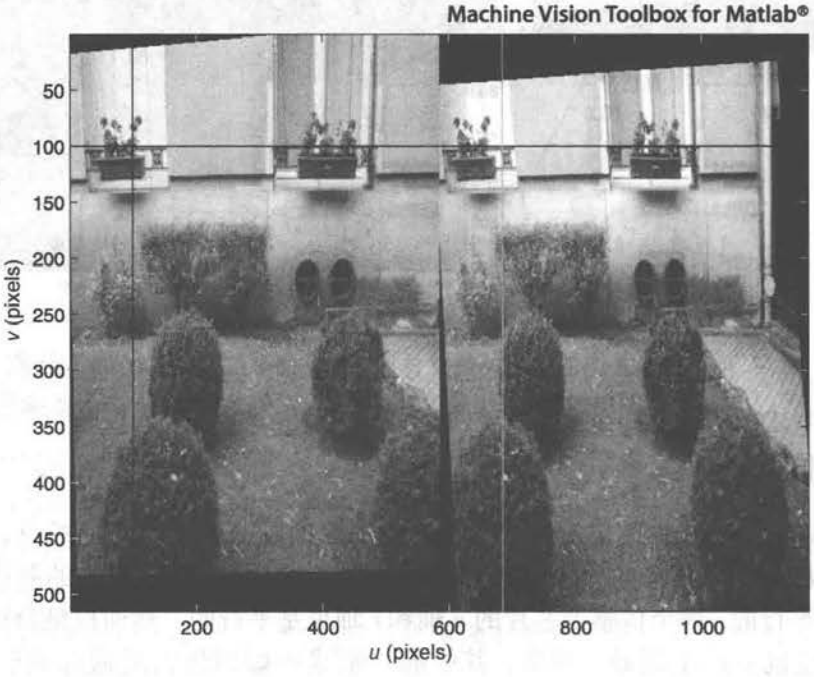


图 14.37 庭院的矫正图像

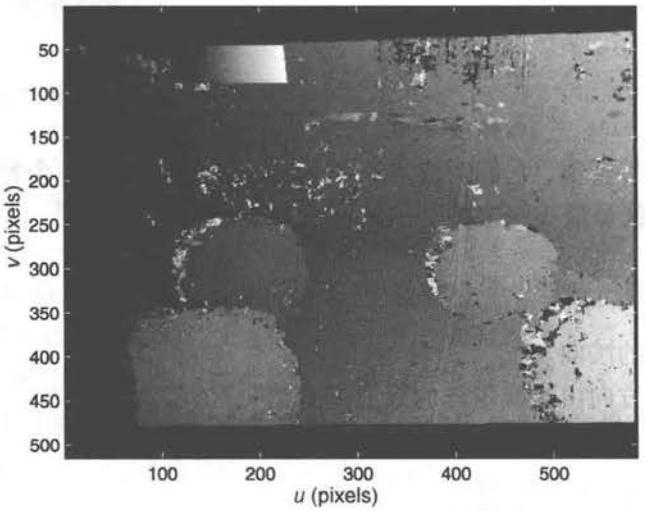


图 14.38　庭院的密集立体差异图像，它显示距离近的目标要比距离远的目标更加明亮

14.3.8　平面拟合

立体视觉引入了一个三维空间点集 $\mathbf{P}_i$，称为点云。通常的问题是如何将这样一个点集拟合为一个平面。一种简单有效的方法是把这些数据拟合成一个椭圆体，这个椭圆体在平面的法线方向将会有一个很小的半径，也就是说它几乎是一个椭圆面。点集的惯性矩阵可以通过下式计算：

$$\boldsymbol{J} = \sum_{i=1}^{N} \boldsymbol{x}_i \boldsymbol{x}_i^{\mathrm{T}} \tag{14.16}$$

其中，$\boldsymbol{x} = \boldsymbol{P}_i - \overline{\boldsymbol{P}}$ 是点相对于其形心 $\overline{\boldsymbol{P}} = \frac{1}{N}\sum_{i=1}^{N} \boldsymbol{P}_i$ 的坐标。椭圆体的中心位于点云的形心。椭圆体的半径是 $\boldsymbol{J}$ 的特征值，对应最小特征值的特征向量是垂直于平面的最小半径方向。

为了说明上述方法，我们在平面中创建一个 10×10 的网格点：

```
>> T = transl(1,2,3) * rpy2tr(0.3, 0.4, 0.5);
>> P = mkgrid(10, 1, T);
>> P = P + 0.02*randn(size(P));
```

齐次变换 T 代表一个任意方向，一些高斯噪声也加入其中。

点云的平均值为

```
>> x0 = mean(P')
ans =
    0.9980    1.9979    2.9973
```

从所有数据点中减去这个平均值：

```
>> P = bsxfun(@minus, P, x0');
```

然后计算出惯性矩：

```
>> J = 0;
>> for x = P,  J = J + x*x';  end
>> J
J =
    8.4143    1.0722   -3.3781
    1.0722    9.6187    2.5577
   -3.3781    2.5577    2.3441
```

其特征值是

```
>> [x,lambda] = eig(J);
>> diag(lambda)'
ans =
    0.0357    9.9135   10.4279
```

我们看到有两个大的特征值，它们对应点在平面中的分布，还有一个特征值代表了平面的厚度。对应第一个，也就是最小一个特征值的特征向量是

```
>> n = x(:,1)'
n =
    0.3896   -0.2779    0.8781
```

它就是估计出的平面的法向。

平面法线的真实方向由变换矩阵的第三列给出[①]：

```
>> T
T =
    0.8083   -0.4416    0.3894    1.0000
    0.5590    0.7832   -0.2722    2.0000
   -0.1848    0.4377    0.8799    3.0000
         0         0         0    1.0000
```

我们看到它与估计的法向很接近。

平面方程表示为点集 $\boldsymbol{x}$ 是

$$\boldsymbol{n}^{\mathrm{T}}(\boldsymbol{x}-\boldsymbol{x}_0)=0 \tag{14.17}$$

其中 $\boldsymbol{n}$ 是法向，$\boldsymbol{x}_0$ 是形心。

外点数据对于这种类型的估计是有问题的，因为它们会使解产生明显的偏移。有很多方法可以用来解决这个问题，其中一种简单方法是用一个权重来修正方程(14.16)：

$$\boldsymbol{J}=\sum_{i=1}^{N} w_i \boldsymbol{x}_i \boldsymbol{x}_i^{\mathrm{T}}$$

该权重与点 $\boldsymbol{x}_i$ 到平面的距离成反比关系，求解时需要迭代。一开始所有的权重 $w_i=1$，在随后的迭代中[②]，权重根据前一步骤估计的点 $\mathbf{P}_i$ 到平面的距离来设定。另一种方法是使用 RANSAC，通过对三点采样来求解方程(14.17)。附录 E 中有更多关于椭圆的细节。

14.3.9 3D 点集匹配

有一个目标对象模型，它由相对于空间坐标系的一个二维或三维点集表示。现在考虑位于空间中某处那个目标的一个实例，我们观察目标上的二维或三维点集。任务是确定相对位姿 ξ，它将通过匹配这两个点集来把模型点变换为观察到的数据点[③]。

更为正式的描述是，考虑到空间坐标系中的两个点集 $\boldsymbol{D}_i$，$\boldsymbol{M}_j \in \mathbb{R}^3$，$i,j \in 1\cdots N$，要确定相对位姿 ξ，以满足

$$\boldsymbol{D}_i=\xi\cdot\boldsymbol{M}_i$$

点 $\boldsymbol{M}_i$ 是三维目标的模型，我们想要把它拟合为观察到的数据 $\boldsymbol{D}_j$。

为了说明上述方法，我们创建一个模型，它是在一个 1 m×1 m×1 m 立方体中的 100 个随

① 由于这些点位于坐标系 xy 平面中，所以法向是坐标系的 z 轴方向。

② 通常使用洛伦茨函数 $w=r^2/(x^2+r^2)$，它在点距为 $0\to\infty$ 范围内是光滑的，并且当 $x=r$ 时，值为 1/2。

③ 对偶问题是相机移动，而目标不动。可以用相同的技术来确定相机运动。

机点集，立方体以原点为中心[①]：

```
>> M = rand(3,100) - 0.5;
```

数据则是模型经过旋转和平移后的一个拷贝：

```
>> T_unknown = transl(1, 2, 3) * rpy2tr(0.2, 0.3, 0.4);
>> D = homtrans(T_unknown, M);
```

但它们之间的相对位姿 ξ 未知。

乍一看，这好像又是一个需要建立两个集合点之间对应关系的问题，但是我们将会介绍另外一种方法，称为“迭代最近点”或 ICP。对于每一个数据点 $\boldsymbol{D}_i$，对应的模型点 $\boldsymbol{M}_j$ 假定为最近点，即 $\boldsymbol{M}_j$ 使 $|\boldsymbol{D}_i - \boldsymbol{M}_j|$ 最小化。对应关系不是唯一的，并且通常是若干数据点与一个模型点相关联，其结果是一些模型点 $\boldsymbol{M}_j$ 必须被拆对。这种对应方法远远达不到完美，但是它对于后面提升点云的匹配已经足够，在下一次迭代中计算出的对应关系将会更加精确。

第一步是计算一个变换，它使两个点云的形心一致[②]：

$$\bar{\boldsymbol{M}} = \frac{1}{N_M}\sum_{i=1}^{N_M}\boldsymbol{M}_i$$

$$\bar{\boldsymbol{D}} = \frac{1}{N_D}\sum_{i=1}^{N_D}\boldsymbol{D}_i$$

从中可以计算出一个位移：

$$\boldsymbol{t} = \bar{\boldsymbol{D}} - \bar{\boldsymbol{M}}$$

下一步计算对应关系。对于每一个数据点 $\boldsymbol{D}_i$，我们要找到最近的模型点 $\boldsymbol{M}_j$，对此我们使用工具箱函数 `closest`：

```
>> corresp = closest(D, M);
```

其中 `corresp(i)` 是 `M` 中的一列，它对应于 `D` 中第 `i` 列。接下来是计算 3×3 的矩量矩阵：

$$\boldsymbol{W} = \sum(\boldsymbol{M}_i - \bar{\boldsymbol{M}})(\boldsymbol{D}_i - \bar{\boldsymbol{D}})^{\mathrm{T}}$$

它包含了两个点集之间的旋转[③]。这个矩阵的奇异值分解是

$$\boldsymbol{W} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\mathrm{T}}$$

从中可以确定旋转矩阵是[④]

$$\boldsymbol{R} = \boldsymbol{V}\boldsymbol{U}^{\mathrm{T}}$$

两个点云之间估计的相对位姿是 $\xi \sim (\boldsymbol{R}, \boldsymbol{t})$。通过该相对位姿对模型点进行变换，使得它们现在更接近于数据点：

$$\boldsymbol{M}_i\langle k+1\rangle = \xi\langle k\rangle \cdot \boldsymbol{M}_i\langle k\rangle, i = 1\cdots N_M$$

重复上述过程，直到收敛为止。所使用的对应关系不可能都是正确的，因此对点集之间的相对姿态估计仅仅是一个近似值。

工具箱提供了一种 ICP 的实现方法：

```
>> [T,d] = icp(M, D, 'plot');
```

它会返回一个位姿 ξ：

---

① 这种技术可以用于二维或三维数据，但是在这里我们将用三维数据来阐释。

② 我们考虑一般情况，即两个点云有不同数量的点，也就是 $N_D \neq N_M$。

③ 这是很多秩为 1 矩阵的和。

④ 见附录 D。

```
>> trprint(T, 'rpy', 'radian')
t=(1 2 3),R=(0.2 0.3 0.4) rad
```

这个位姿正是上面选择的第二个点云的未知相对位姿。残差为

```
>> d
d =
   6.8437e-08
```

它是变换的模型点和数据之间的误差均方根。选项'plot'显示了每一步的模型点和数据点，也显示了最近点的对应关系。ICP 是一种较普遍使用算法，因为它运算快，而且鲁棒性好。

我们可以通过模拟一些实际的传感器误差来证明 ICP 的鲁棒性。首先，我们将随机地从数据中去掉 20 个点：

```
>> D(:,randi(100, 20,1)) = [];
```

这些点是模型中没有被传感器观察到的点。然后加入 10 个不在模型中的伪造点：

```
>> D = [D rand(3,10)];
```

最后，把 $\sigma = 0.05$ 的高斯噪声添加到数据中：

```
>> D = D + randn(size(D)) * 0.05;
```

现在将这个受损数据与模型拟合：

```
>> [T,d] = icp(M, D, 'plot', 'distthresh', 1.5);
```

其中使用了一个附加选项来消除错误的最近点对应。正如上文所描述的那样，对应关系被建立起来，并且计算出了对应点之间的距离中值。在这里，如果点之间的距离大于距离中值的 1.5 倍，那么这个对应关系就不能成立。现在估计的姿态 ξ 是

```
>> trprint(T, 'rpy', 'radian')
t = (1.02161,1.98456,2.98566), R = (0.164436,0.299455,0.418697) rad
```

它仍然接近于理想情况下计算出的值，但是残差比较大：

```
>> d
d =
    0.6780
```

因为模型和噪声损坏的数据点之间不可能再有精确配合①。ICP 对于中等大小的点云来说不仅计算快而且鲁棒性好，但是对应关系确定是一个 $O(N^2)$ 问题，对于有巨大数量的数据点集，它会引发计算瓶颈②。

14.4 结构和运动

在 14.3.1 节的稀疏立体匹配示例中，我们根据对应的图像点估计出了相机的运动，即它的位姿变化。以此为基础，我们就能估计一些点在空间中的三维位置，即场景的结构。

对于机器人应用，我们是希望非常快地执行这些操作，也就是说在每次进行新的观察时，都能更新对机器人位姿和空间结构的估计。我们将会用一个示例来对其进行阐述，这个示例使用了来自单一移动相机的一系列图像。

我们创建一个中央透视相机：

```
>> camera = CentralCamera('name', 'sfm', 'default');
```

---

① 我们会期望残差近似等于 $\sqrt{N}\sigma$，其中 N 是对应点的数量，σ 是附加噪声的标准偏差。

② 对于大规模点集的问题，数据会被保存到 *kd* 树中，这样会减少寻找最近点所需要的时间。

并创建一些位于一个立方体顶点和各个面中心的点：

```
>> cube = mkcube(0.6, 'facepoint')
```

总共是 14 个点[①]。相机在 *xy* 平面中沿着一个半径为 3 米的围绕立方体的圆周运动，而且始终面对着原点：

```
>> nsteps = 50;
>> theta = tpoly(0, 2*pi, nsteps);
>> T = [];
>> for th=theta'
>>     T = cat(3, T, trotz(-th) * transl(0, -3, 0) * trotx(-pi/2) );
>> end
```

其中 T 是路径，即一系列相机位姿，由一个 4 ×4 ×50 的矩阵来表示，其中前两个指数代表一个齐次变换，最后一个指数对应相机的视点。

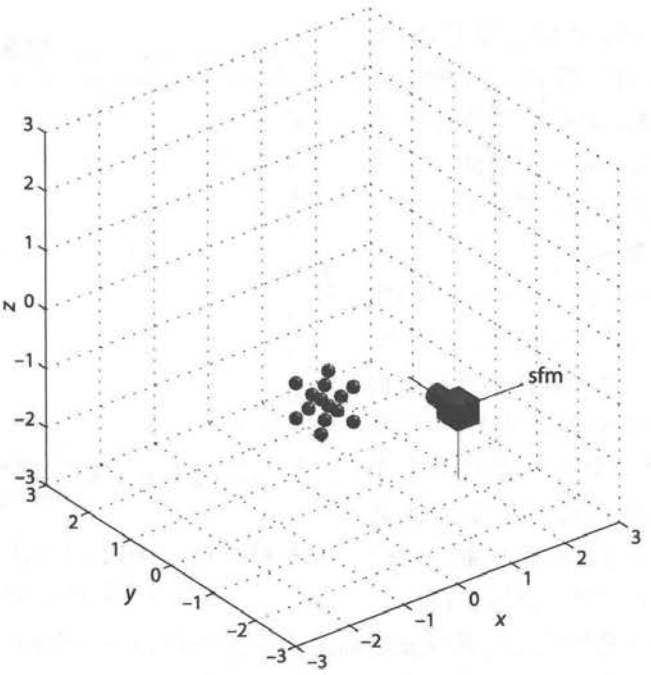


图 14.39　显示了相机和空间点的移动示例的结构

图像平面投影是

```
>> p = camera.plot(cube, 'Tcam', T);
```

它返回一个 2 ×14 ×50 的矩阵，其中第二个指数对应空间点，最后一个指数对应相机视点。

在沿着路径的每一步中执行下面的运算，在这里用第 20 步进行说明：

```
>> k = 20;
```

用当前图像和上一幅图像之间的对应点来估计基本矩阵：

```
>> F = fmatrix( p(:,:,k-1), p(:,:,k) );
```

然后把它升级为一个本质矩阵：

```
>> E = camera.E(F);
```

之后把这个本质矩阵分解成从上一相机位姿到当前位姿的运动估计：

---

① 后续步骤估计基本矩阵至少需要 8 个点。但只用立方体的 8 个顶点会产生不良的基本矩阵估计，因此额外增加了 6 个位于每一个面中心的点。

```
>> relpose = camera.invE(E, [0 0 10]');
solution 2 is good
relpose =
    0.9775   -0.0000    0.2108  -11.2645
    0.0000    1.0000    0.0000   -0.0000
   -0.2108   -0.0000    0.9775    1.2008
         0         0         0    1.0000
```

其中使用了一个沿光轴的空间点来确定两个解之中哪一个是有效的。

从已知相机轨迹中确定的真实相对位姿是

```
>> inv(T(:,:,k-1))*T(:,:,k)
ans =
    0.9775    0.0000    0.2108   -0.6324
   -0.0000    1.0000   -0.0000    0.0000
   -0.2108   -0.0000    0.9775    0.0674
         0         0         0    1.0000
```

可以看出，估计位姿的旋转部分是精确的，但是平移部分却存在着之前讨论过的未知缩放因子问题。图 14.40 显示了估计平移和真实平移的大小随时间变化的曲线。我们看到平移通常被过高估计，并且它和真实平移的大小没有明显的关系①。

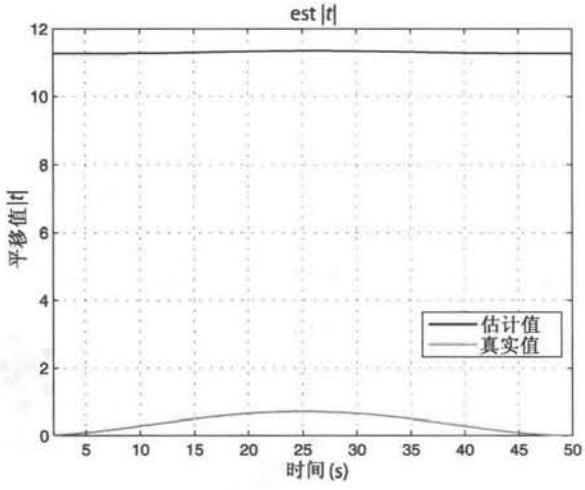


图 14.40 在每个时间步相机平移运动的大小，包括从本质矩阵得到的估计值以及底部的真实值

不知道缩放因子，就会有一定的困惑——我们无法估计视图之间的增量运动。就像之前看到的那样，要求解就必须有其他的信息源。例如，类似里程计或 GPS 之类的其他传感器会提供一个对平移运动大小的估计。我们只要求里程计提供走过距离的信息，因为运动的旋转部分是确定无误的。另外，可以使用立体相机直接提供深度信息，那么在每一个时间步的问题就变成要确定能够使当前和上一步的 3D 点云对齐的相对位姿，而这可使用 ICP 来解决。另一个选项，也是接下来要讨论的，是认为我们知道一个被观察空间点的离地高度。在机器人场景中，我们可能知道机器人是在一个平面上运动，并且一个特殊的特征点位于地面上或者门框的顶部。

对于这个示例，假定我们知道点 j 的高度，这里随意选择：

```
>> j = 1;
```

点的空间坐标是

```
>> P(:,j)'
ans =
   -0.3000   -0.3000   -0.3000
```

由于相机位于空间 xy 平面中，y 轴指向下方，所以 ${}^{C}Y_j = 0.3$：

```
>> Yj = 0.3;
```

这是我们需要的一条额外信息，但我们需要在所有特征图像中找到这个特殊点。

再一次用路径中的第 20 步来演示我们的算法。我们考虑 {1} 是前一步的相机坐标系，

① 真实平移大小显示了由函数 tpoly 创建的平滑变化速度。

{2} 是当前步的坐标系。对应于该已知点在 {1} 中投影的一条射线是

```
>> r1 = camera.ray( p(:,j,k-1) )
r1 =
d=(0.117002, 0.108684, 0.987167), P0=(0, 0, 0)
```

它与平面 ${}^{C}Y = {}^{C}Y_j$ 相交于点：①

```
>> P1 = r1.intersect([0 1 0 -Yj])
P1 =
    0.3230
    0.3000
    2.7249
```

我们用相机观察到的这个空间点在 {2} 中的投影可由方程(11.5)给出：

$$ {}^{2}\tilde{\boldsymbol{p}} \simeq \boldsymbol{C}\,{}^{2}\boldsymbol{T}_1\,{}^{1}\tilde{\boldsymbol{P}} $$

可以用一个齐次形式的缩放因子 λ 对上式进行改写：

$$ \lambda^2\tilde{\boldsymbol{p}} = (\boldsymbol{\Pi} \quad \boldsymbol{c}_4)\begin{pmatrix}\boldsymbol{R} & \sigma\boldsymbol{t}\\ \boldsymbol{0}_{1\times3} & 1\end{pmatrix}\begin{pmatrix}\boldsymbol{P}\\ 1\end{pmatrix} = \sigma\boldsymbol{\Pi}\boldsymbol{t} + \boldsymbol{\Pi}\boldsymbol{R}\boldsymbol{P} + \boldsymbol{c}_4 = \sigma\boldsymbol{a} + \boldsymbol{b} $$

其中，σ 是未知的平移缩放因子，$\boldsymbol{a}$ 和 $\boldsymbol{b}$ 是向量，它们分别是已知相机矩阵 $\boldsymbol{C}$ 和估计相机旋转 $\boldsymbol{R}$ 的函数。观察到的像平面坐标与真实像平面坐标之差被认为是二次投影误差，它的大小是

$$ e = (\sigma\boldsymbol{a} + \boldsymbol{b} - \lambda^2\tilde{\boldsymbol{p}})^{\mathrm{T}}(\sigma\boldsymbol{a} + \boldsymbol{b} - \lambda^2\tilde{\boldsymbol{p}}) $$

为了确定平移放缩因子 σ 和齐次放缩因子 λ 的值，通过对上式求偏导数并把它们设为 0 来最小化这个误差：

$$ \frac{\partial e}{\partial \sigma} = 2(\sigma\boldsymbol{a} + \boldsymbol{b} - \lambda^2\tilde{\boldsymbol{p}})\cdot\boldsymbol{a} = 0 $$

$$ \frac{\partial e}{\partial \alpha} = -2(\sigma\boldsymbol{a} + \boldsymbol{b} - \lambda^2\tilde{\boldsymbol{p}})\cdot {}^{2}\tilde{\boldsymbol{p}} = 0 $$

这两个等式意味着括号中的矢量与 $\boldsymbol{a}$ 和 ${}^{2}\tilde{\boldsymbol{p}}$ 均正交，也等同于与它们的叉积成正比：

$$ \sigma\boldsymbol{a} + \boldsymbol{b} - \lambda^2\tilde{\boldsymbol{p}} \propto \boldsymbol{a} \times {}^{2}\tilde{\boldsymbol{p}} $$

通过引入另一个放缩因子 α，我们把上式写成等式形式：

$$ \sigma\boldsymbol{a} + \boldsymbol{b} - \lambda^2\tilde{\boldsymbol{p}} = \alpha(\boldsymbol{a} \times {}^{2}\tilde{\boldsymbol{p}}) $$

整理后重写为

$$ (-\boldsymbol{a} \quad {}^{2}\tilde{\boldsymbol{p}} \quad \boldsymbol{a} \times {}^{2}\tilde{\boldsymbol{p}})\begin{pmatrix}\sigma\\ \lambda\\ \alpha\end{pmatrix} = \boldsymbol{b} $$

它是一个线性方程，可用于求解未知参数 σ，λ 和 α。

对于这个示例：

```
>> C = cam.C; PI = C(:,1:3); c4 = C(:,4);
>> [R,t] = tr2rt( inv(relpose) )
>> a = PI*t; b = PI*R*P1+c4;
>> p2 = e2h( p(:,j,k) );
```

估计出的参数为

```
>> phi = [-a p2 cross(a, p2)] \ b
phi =
    0.0561
    2.7991
         0
```

① 这个平面的向量形式是 $(0,1,0)\,(X,Y,Z)^{\mathrm{T}} = {}^{C}Y_j$，齐次形式为 $(0,1,0,-{}^{C}Y_j)\,(X,Y,Z,1)^{\mathrm{T}} = 0$。

从中可以计算出位移 ${}^{1}\xi_2$：

```
>> dT = inv( [R phi(1)*t; 0 0 0 1] )
dT =
    0.9775    0.0000    0.2108   -0.6324
   -0.0000    1.0000   -0.0000    0.0000
   -0.2108    0.0000    0.9775    0.0674
         0         0         0    1.0000
```

对于 4 个重要数字而言，这个结果与之前显示的已知相机运动是相同的。丢失的缩放因子已经通过使用已知单一被观察点的离地高度、已知机器人的运动和一个已标定相机得以恢复。空间坐标系中的相机位姿可以通过把坐标系之间的位姿变化进行累加来估算：

$$\hat{\boldsymbol{T}}\langle k\rangle = \hat{\boldsymbol{T}}\langle k-1\rangle {}^{1}\hat{\boldsymbol{T}}_2\langle k-1\rangle$$

这样就可以获得在每一时间步相机相对于初始相机位姿的一个位姿估计。图 14.41 显示了相机沿路径运动时其中心真实和估计的位置，也显示了缩放因子的真实和估计值曲线。这就是用于位置估计的航迹推算法，正如在 6.1 节中讨论的那样，每一步增量运动的估计误差都将会随着时间累积。对于把噪声添加到特征坐标中的情况，结果如图 14.42 所示，我们看到缩放因子的估计精确度较低，并且估计的相机位置与真实位置发生了分离。这个示例的所有代码可以从示例目录中的 `sfm1` 来获得。

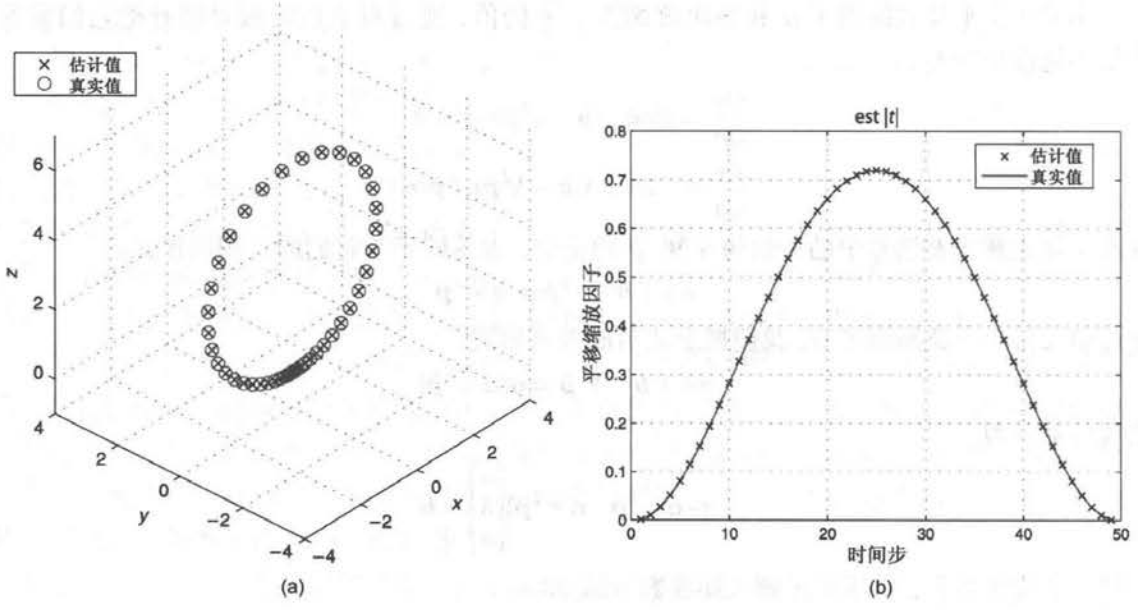


图 14.41 用单一空间点估计缩放因子的运动结构。(a)估计和真实的相机中心；(b)估计和真实的平移放缩因子

接下来我们将会应用之前在 14.3.1 节中使用的稀疏立体匹配方法来估计场景结构。三维空间中对应于前一位置投影点的射线是

```
>> r1 = camera.ray(p(:,:,k-1))
```

在当前位置，它们是

```
>> r2 = camera.move(dT).ray(p(:,:,k))
```

射线相交于

```
>> x = r1.intersect(r2)
```

它是一个矩阵，每一列是交点相对于前一相机位姿的三维位置。使用我们对相机位姿关于空

间坐标系的估计，将这些点变换到空间坐标系：

```
>> xc = homtrans(inv(T_est(:,:,k-1), x))
```

这种方法虽然简单，但是有很多的缺点。最大的问题是在每一个时间步都要估计每一个空间点的三维坐标，而这些估计的好坏很大程度上依赖于相机和空间点之间的相对运动。三角测量涉及三个点，即两个相机的中心点以及图 14.17 所示的空间点。两个相机中心之间的三角边是基线，如果基线小，那么估计很容易受噪声影响。这里就存在这个基线小的问题，因为图像之间的相机运动很小。然而如果第二个相机中心与第一个相机中心和空间点共线——相机运动是沿着朝向或远离这个点的直线——那么就有一个零基线，我们就不能获取那个点的深度信息。

图 14.42 显示了图像噪声对相机位置估计的影响，图 14.43 显示了这个噪声对单一空间点估计位置的影响。我们看到有的估计位置很分散，形成一些误差超过了 1 m 的外点。一种解决方案是采用相隔 N 个坐标系之间而不是两个连续坐标系之间的三角测量来增加有效的基线。另一种选项是将一些低通滤波器应用到估计中①，图 14.43(见彩图)显示了估计位置的中值接近于真实位置②。一种更好的解决方案是求它的逆问题，就像处理 6.1.2 节中的定位问题那样使用一个扩展卡尔曼滤波器③。滤波器的状态包含了相机位姿和地标的空间坐标。观察结果是特征的像素坐标。观察到的像素坐标和预测的像素坐标之差是更新量，它通过卡尔曼增益矩阵更新状态向量。这个框架允许对空间点位置的改进估计，因为它综合了很多时间步的测量值，并且还允许对与空间点相关的不确定性建模。卡尔曼过滤框架还允许融合许多其他传感器，例如里程计、GPS 和激光测距测量，这有助于解决缩放问题。当特征进入或离开相机视场时，还需要做一些记录来跟踪它们。

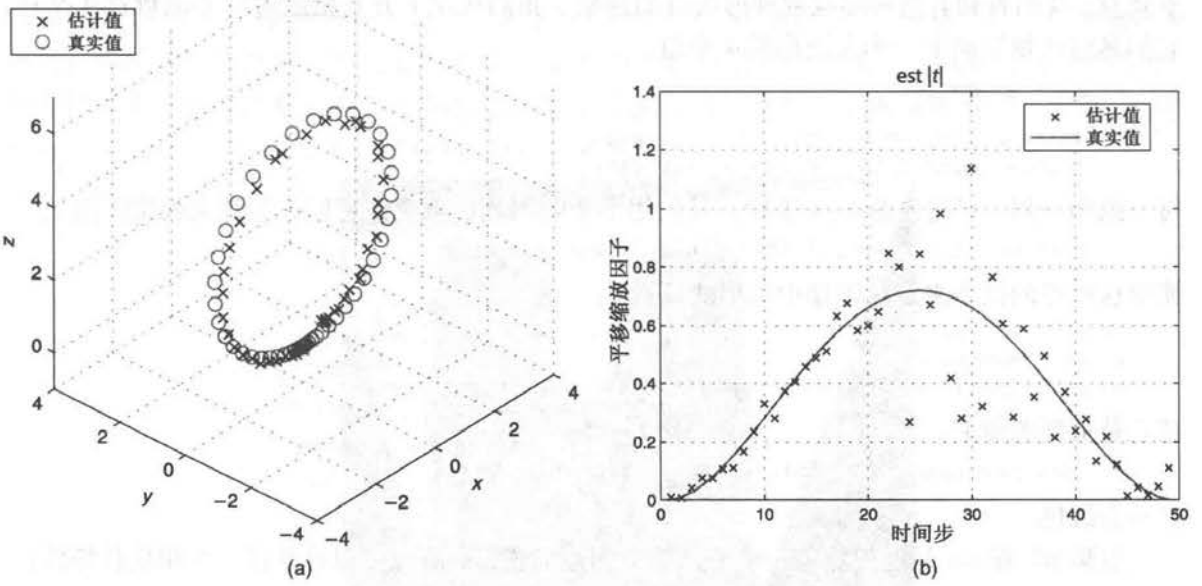


图 14.42　估测比例来自单一空间点的运动结构，像素噪声是 $\sigma = 0.5$。
(a)估测的和真实的相机中心；(b)估测的和真实的平移放缩因子

① 例如滑动平均滤波器，卡尔曼滤波器，或者 $\alpha-\beta$ 追踪滤波器。
② 误差并不呈现为零均值，所以中值估计是有偏差的。
③ 卡尔曼滤波器假定噪声是零均值且呈高斯分布，但这里不像是这种情况。

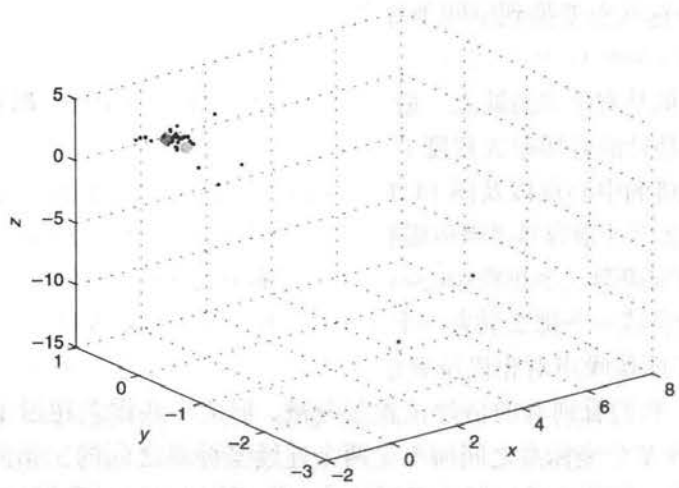


图 14.43 估计位置的扩散，像素噪声是 $\sigma = 0.5$。蓝点是成对估计的点位置，红色菱形标志是空间点（在相机坐标系中）的真实位置，绿色圆圈是估计位置的中值

14.5 应用：透视矫正

考虑以下图像：

```
>> im = iread('notre-dame.jpg', 'double');
>> idisp(im)
```

如图 14.44 所示（见彩图）。建筑物形状产生了严重的畸变，因为相机的光轴与建筑物的表面不垂直，我们看到有透视缩减或梯形失真的迹象。我们从左下方开始顺时针手动挑选 4 个点，它们是建筑物平面上一个大矩形的 4 个角：

```
>> p1 = ginput(4)'
ans =
   44.1364   94.0065  537.8506  611.8247
  377.0654  152.7850  163.4019  366.4486
```

每一点为一列，它包含着 u、v 坐标。我们用半透明的蓝色填充把它们叠加到大教堂图像上

```
>> plot_poly(p1, 'wo', 'fill', 'b', 'alpha', 0.2);
```

使用这些点的极值来定义图像中矩形的顶点①：

```
>> mn = min(p1');
>> mx = max(p1');
>> p2 = [mn(1) mx(2); mn(1) mn(2); mx(1) mn(2); mx(1) mx(2)]';
```

把它叠加到图像上：

```
>> plot_poly(p2, 'k', 'fill', 'r', 'alpha', 0.2)
```

显示为红色。

点集 p1 和 p2 是近似位于一个平面的空间点的投影，所以可以计算出一个单应性矩阵：

```
>> H = homography(p1, p2)
H =
    1.4003    0.3827 -136.5900
   -0.0785    1.8049  -83.1054
   -0.0003    0.0016    1.0000
```

① 在计算机图形学中，术语 alpha 是指表面的透明度，从 0 的完全透明变化到 1 的不透明。

它将把蓝色不规则四边形的顶点转换成红色矩形的顶点：[①]

$$\tilde{p}_2 \simeq H\tilde{p}_1$$

这就是说，单应性矩阵把图像坐标从畸变的梯形形状映射为一个未畸变的矩形形状。

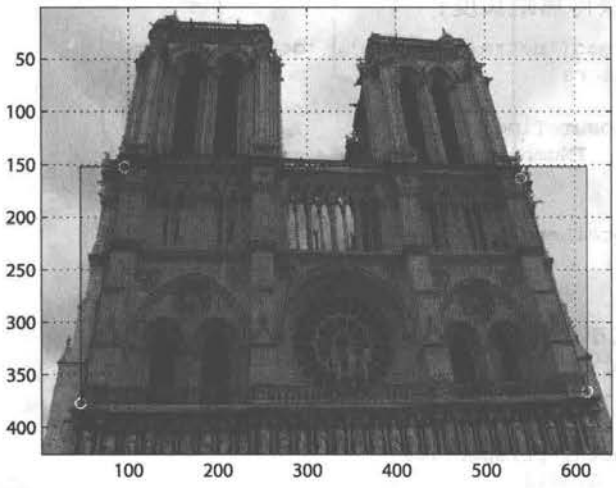


图 14.44　从地面上拍摄的照片显示了透视缩减的影响，它使建筑物呈现为梯形状(也称为梯形失真)。手动挑选了建筑物近似前平面上的4个点，用白色○标记点指示(巴黎圣母院))

我们可以把这个单应性矩阵应用到输出图像中每一个像素的坐标上，为的是使输入图像变形。使用工具箱中通用图像变形函数：

```
>> homwarp(H, im, 'full')
```

结果如图 14.45 所示，它是一个合成的额平行视图，这就相当于在高空中用光轴与教堂表面垂直的相机拍摄到的照片。然而那些不在这个平面上的点就产生了畸变，例如右钟楼的左侧。输出图像中的黑色像素是由于对应的像素坐标没有在输入图像中呈现而造成的。注意，函数没有指定的输出参数，变形图像使用 idisp 来显示。

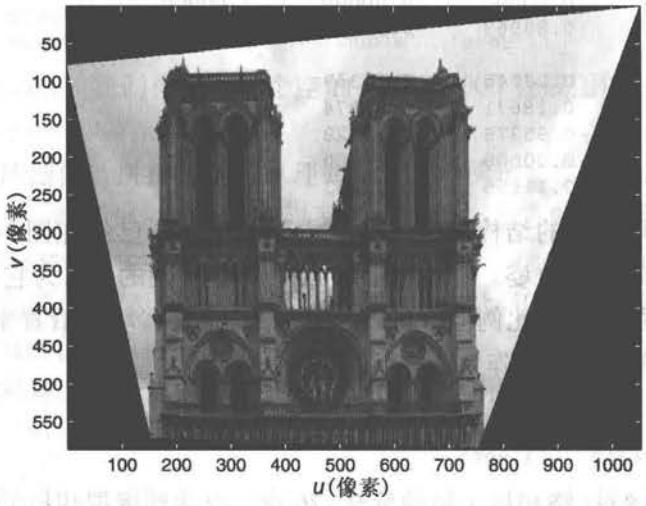


图 14.45　从图 14.44 合成得到的额平行视图。图像经过了变换，使标记点成为图像中矩形的 4 个角点

① 单应性矩阵也可以根据平面中的 4 条直线计算出来，但是工具箱不支持该功能。

除了创建这个合成视图，我们还可以分解单应性矩阵来恢复从实际视点到虚拟视点的相机运动，也可以恢复垂直于教堂的表面。正如在 14.2.4 节中看到的那样，我们需要确定相机标定矩阵，这样就可以把投影单应性矩阵转变成欧几里得单应性矩阵。我们从保存图像的 EXIF 格式的文件头中获得焦距长度：

```
>> [im,tags] = iread('notre-dame.jpg', 'double');
>> tags.DigitalCamera
ans =
              ExposureTime: 0.0031
                   FNumber: 5.6000
                         .
                         .
                         .
               FocalLength: 7.4000
                         .
                         .
                         .
```

像素尺寸与第 392 页上示例中的一致。创建一个已标定的相机：

```
>> cam = CentralCamera('image', im, 'focal', 7.4e-3, ...
    'sensor', [7.18e-3,5.32e-3])
cam =
name: noname [central-perspective]
  focal length:    0.0074
  pixel size:      (5.609e-06, 6.251e-06)
  principal pt:    (640, 425.5)
  number pixels:   1280 x 851
  Tcam:
     1  0  0  0
     0  1  0  0
     0  0  1  0
     0  0  0  1
```

现在使用相机模型来计算并分解欧几里得单应性矩阵：

```
>> sol = cam.invH(H, 'verbose');
solution 1
     T = 0.98415    -0.15470     0.08667     0.23438
         0.15902     0.98623    -0.04534     0.92578
        -0.07846     0.05841     0.99520    -1.29375
         0.00000     0.00000     0.00000     1.00000
    n = -0.23041     0.88969     0.39416
solution 2
     T = 0.95800     0.23545     0.16369     0.07289
        -0.20987     0.18671     0.95974    -1.53146
         0.19541    -0.95378     0.22828     0.48488
         0.00000     0.00000     0.00000     1.00000
    n = -0.28109     0.11855     0.95233
```

它会返回一个有两种可能解的结构数组。这个示例的坐标系已经在图 14.46 中描绘出来，它显示了真实的和虚拟的相机位姿。在这里，第二个解是正确的，因为它代表了绕 x 轴的大旋转。相机的平移向量虽然不成比例但有正确的符号①，主要移动是沿着坐标系 {1} 的负 y 轴和正 z 轴。*XYZ* 角度形式的旋转矩阵为

```
>> tr2rpy(sol(2).T, 'deg')
ans =
  -76.6202    9.4210  -13.8081
```

它表示相机需要向下倾斜(绕相机 x 轴的旋转)76 度，以达到虚拟相机的姿态。教堂额平面的法线 $\boldsymbol{n}$ 被定义在坐标系 {1} 中，期望的应该是相机的 z 方向。

① 见[Malis and Vargas, 2007]。

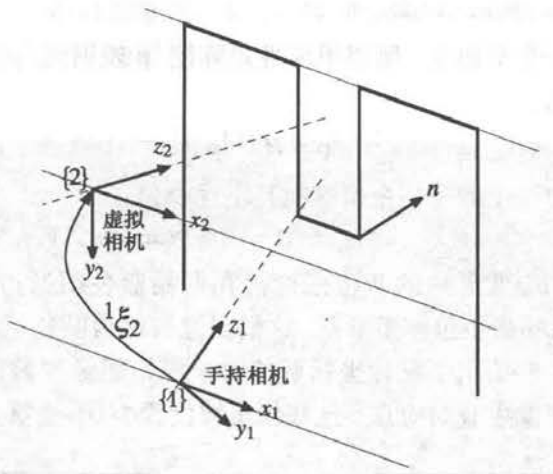


图 14.46　显示了两个相机坐标系的圣母院示例。坐标系{1}是拍摄图像的相机的坐标系，坐标系{2}是合成的额平行视图的视点坐标系

14.6　应用：拼接

拼接或者图像缝补是从众多重叠图像中创建一幅大规模合成图像的过程。通常应用于航拍或卫星图像中来创建地球表面的看似连续的单个图像，也可以用于由水下机器人携带的视角向下相机捕捉到的海底图像。数码相机提供的全景生成软件是拼接技术的另一个实例。

图像拼接处理的输入是一系列重叠的图像①。我们没有必要知道拍摄图像时相机的标定参数或者相机位姿——相机可以在图像之间任意旋转，其缩放比例也可以改变。然而对于我们将要使用的方法，需要假设场景是平的，这个假设对于垂直落差较小的高空摄影来说是合理的②。

将要使用一个示例来说明我们的方法，这个示例采用了一对图像：

```
>> im1 = iread('mosaic/aerial2-1.png', 'double', 'grey');
>> im2 = iread('mosaic/aerial2-2.png', 'double', 'grey');
```

每一幅图像是 1280 × 1024。我们再创建一个空的 2000 × 2000 合成图像：

```
>> composite = zeros(2000,2000);
```

它将用来承装拼接好的图像。拼接过程的原理如图 14.47 所示。

第一幅图像很简单，我们只需要把它粘贴到合成图像右上方的角落即可：

```
>> composite = ipaste(composite, im1, [1 1]);
```

如图 14.47 所示的红色部分。显示为蓝色的下一幅图像则比较复杂，它需要旋转、放缩和平移，以便使它正确地与红色图像拼接。

第一步是识别出被称为接合点的图像特征点。我们使用熟悉的工具：

```
>> surf1 = isurf(im1)
>> surf2 = isurf(im2)
>> m = surf1.match(surf2);
```

然后使用 RANSAC 来估计单应性矩阵：

① 一般说来，图像在正前方有 60% 的区域重叠，而在侧面有 30%。

② 平面上方点的高度与从平面到相机的距离之比。

```
>> [H,in] = m.ransac(@homography, 0.2)
```

由于假定特征点都位于一个平面上，所以单应性矩阵把 ${}^1\mathbf{p}$ 映射到 ${}^2\mathbf{p}$。现在我们要把 ${}^2\mathbf{p}$ 映射为第一幅图像中的对应坐标：

$$ {}^1\mathbf{p} \simeq \boldsymbol{H}^{-1}\,{}^2\mathbf{p} $$

我们通过变形对新图像中的每一个像素执行上述映射：

```
>> [tile,t] = homwarp(inv(H), im2, 'full', 'extrapval', 0);
```

如图 14.47 所示(见彩图)，变形后的蓝色图像落在原始蓝色图像边界之外，选项'full'规定：返回图像是变形图像的最小包容矩形①。这幅图像被称为贴片，并框在黑色虚线内。向量 $\boldsymbol{t}$ 由函数 homwarp 返回，它给出了贴片坐标系相对于原始图像的偏移量。通常并不是贴片中的每一个像素都在输入图像中有对应点，这些像素被设置为 0，由第 5 个输入参数指定②。

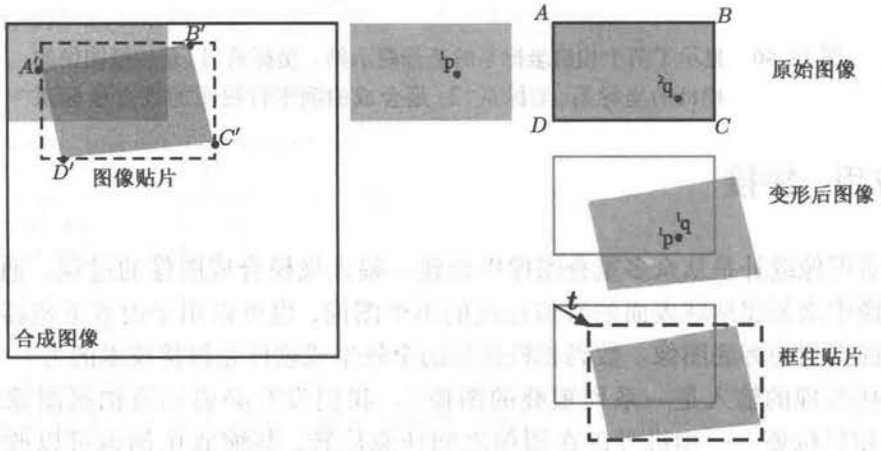


图 14.47　图像序列中的第一幅显示为红色，第二幅显示为蓝色。第二幅图像被变形成图像贴片，然后被拼接进合成图像

现在把这个贴片拼装进合成图像中：

```
>> canvas = ipaste(canvas, tile, t, 'add');
```

结果如图 14.48 所示。我们可以清晰地看到一些图像叠加在一起，并对齐得很好。变形图像中未被映射的像素被设定为 0，因此加上它们不会使合成图像中已有的像素值发生变化。

简单地把贴片加到合成图像中，意味着重叠部分的像素必定要变得明亮，我们可以使用很多不同的技巧来对其进行补救。只要合成图像像素还没有被设定，那么就可以根据贴片设定合成图像中的像素。反过来，我们可以总是根据贴片中的非零点像素来设定合成图像中的像素。另一种方法是可以把合成图像像素设定为贴片和合成图像的平均值。这就要求我们标记出没有映射的贴片像素：

```
>> [tile,t] = homwarp(inv(H), im2, 'full', 'extrapval', NaN);
```

然后使用'mean'选项进行混合③：

```
>> canvas = ipaste(canvas, tile, t, 'mean');
```

---

① 贴片边框的计算方法是通过对图像角落 $A=(1,1), B=(W,1), C=(W,H), D=(1,H)$ 应用单应性矩阵(其中 W 和 H 分别是宽度和高度)，并找出在 u,v 方向上的边界。

② 默认为 NaN。

③ 它忽略了值为 NaN 的那些像素。

如果拍摄这些图像时使用了相同的曝光，那么贴片的边缘将不可见。如果曝光不同，那么不得不分析两组重叠的像素来确定平均强度偏差和缩放因子，并在图像混合前用于矫正贴片——这个过程称为色调匹配。

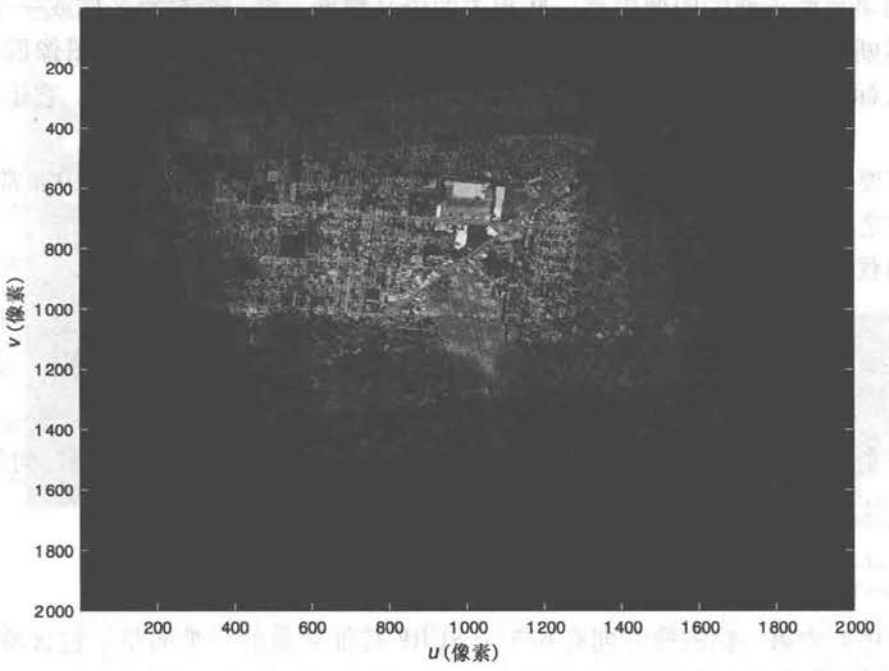


图 14.48　图像拼接示例。在图像的底部，我们可以清晰地看到由三张图叠加起来的那个机场跑道，它显示出图像之间对接得非常好

最后，我们需要考虑图像中不在地平面上的点的影响，如那些在高大建筑物上的点。直接从头顶拍摄的图像将只会显示建筑物的屋顶，但是如果从再远一点的位置拍摄，图像将会是一幅显示了建筑物侧面的斜视图。在一幅拼接好的图像中，我们想要造成这样一种错觉，即我们是正对着图像中的每一个点，所以我们无法看到任何建筑物的侧面[①]。这种类型的图像称为正射影像。与透视图像中射线相交于相机焦点不同，正射影像中的射线都是平行的，这表示其视点在无穷远。在合成图像的每一个像素处，我们可以从任意一张叠加贴片中选择一个像素。为了最佳近似于一幅正射影像，我们应该选择最接近头顶的像素，也就是说在变形之前最接近于主点的像素。

在摄影测量法中，这种类型的拼接被称为不可控的数字拼接，因为它没有使用明确的控制点——手动确定图像中的对应特征。结果是一幅有着无穷远视点的正射影像。相关的所有程序代码可参见示例目录中的 `mosaic1`。这里阐述的原理也可以应用于图像稳定化问题。单应性矩阵可用于把新图像中的特征点映射到它们在之前图像中的位置。

14.7　应用：图像匹配和检索

给出一组图像 $\{\mathbf{I}_j, j = 1 \cdots N\}$ 和一幅新图像 $\mathbf{I}'$，图像匹配问题是确定使得 $\mathbf{I}'$ 和 $\mathbf{I}_j$ 最为相似的 j。当我们考虑视点和曝光变化的影响时，这是一个很难的问题。我们以前使用过的像素级

① 谷歌地图提供了一种并不算完美的正射影像。当我们观察城市时，经常会看到建筑物的斜视图。

的相似性测度，例如 SSD 或者 ZNCC，都不再适合这个问题，因为视点相当小的变化都会导致接近于 0 的相似度。

机器人可以使用图像匹配来确定它之前是否到访过某个特定地点，或看到过相同的目标对象。如果那些先前的图像中有一些相关的语义数据，如一个对象名称或一个地点名称，那么经过推断，那个语义数据就可以应用于新图像。例如，如果一幅新图像匹配了一幅已有的带语义标签“lobby”的图像，那么它就表示机器人正在看相同的场景，它正在或正接近大厅。

我们将要介绍的特殊技术通常称为“词袋”，它最近几年在机器人技术中非常流行。这项技术建立在之前介绍过的技术之上，例如 SURF 点特征和 k 均值聚类。

首先加载一组 20 张的图像：

```
>> images = iread('campus/*.jpg', 'mono');
```

它是一个 426 × 640 × 20 的数组，对于其中每个图像我们都计算其 SURF 特征：

```
>> sf = isurf(images);
```

它会返回一个 MATLAB® 单元数组，它的元素是对应于输入图像的 SURF 特征向量。例如：

```
>> sf{1}
ans =
663 features (listing suppressed)
  Properties: theta image_id scale u v strength descriptor
```

它是对应于序列中第一幅图像并拥有 633 个 SURF 特征对象的一个向量。包含所有图像的所有 SURF 特征的集合是

```
>> sf = [sf{:}]
sf =
17945 features (listing suppressed)
  Properties: theta image_id scale u v strength descriptor
```

它是一个拥有 17 945 个 SURF 特征对象的向量。

考虑一个特定的 SURF 特征：

```
>> sf(380)
ans =
  (48.1448,219.771), theta=4.4736, image_id=1, scale=3.48386,
    strength=0.000615617, descrip= ..
```

我们看到之前讨论过的 SurfPointFeature 属性，例如形心、缩放比例和方向。属性 image_id 表示这个特征点是从原始图像序列中的第一幅图像中提取的。可以显示这幅图像并叠加上特征：

```
>> idisp(images(:,:,1))
>> sf(380).plot('g+')
>> sf(380).plot_scale('g', 'clock')
```

如图 14.49(b)所示。支持区域中显示了砖和窗户的边缘。support 方法使用 image_id 属性来确定哪一幅传递的图像包含了该特征。

词袋技术背后的核心在于，有很多这些特征描述了在视觉上相似的场景元素，例如叶子、窗角、砖、烟囱等。如果把每一个 SURF 特征描述符认为是 64 维空间中的一个点，那么相似的描述符将会形成群聚，产生一个 k 均值聚类问题。找出 2000 个特征集群：

```
>> sf(380).support(images)
```

它会返回一个 BagOfWords 对象，其中包含着原始特征、每一个集群的中心和各种其他信息[①]。每一个集群被称为一个视觉单词，并由一个 64 元素的 SURF 描述符来描述。正如可以依照某个词汇从一个文件中挑选出一组相同单词那样，我们也可以依照视觉词汇在每一幅图像中挑选出一组(袋)视觉单词。

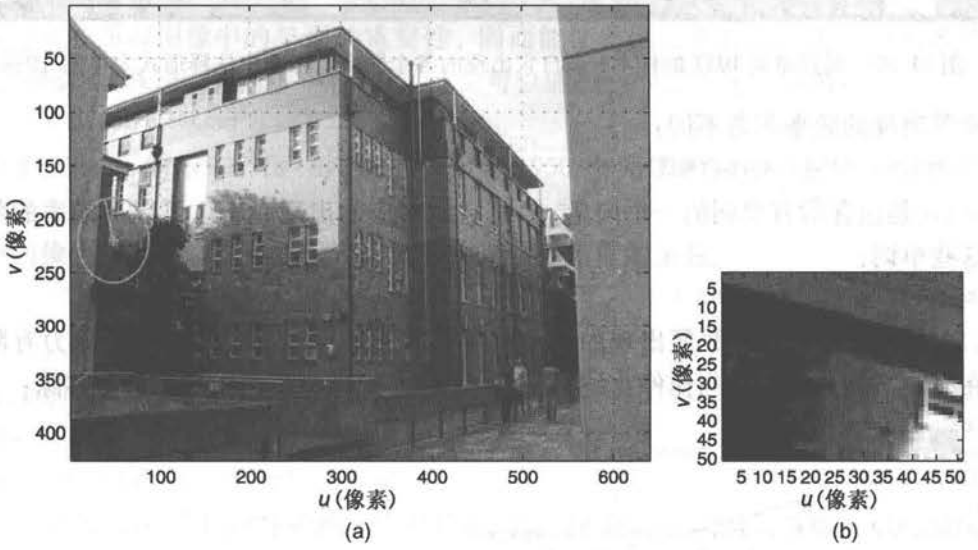


图 14.49　(a)有视觉单词 SURF 特征 380 的图像 1。该特征用绿色圆圈表示大小，用半径线表示方向；(b)正方形支持区域与圆圈具有相同的面积，其水平轴平行于圆圈中的径线方向

聚类操作步骤会给每一个 SURF 特征分配一个视觉单词索引。对于上面显示的特殊特征：

```
>> bag.words(380)
ans =
        1057
```

我们发现 k 均值聚类已经把这个图像特征分配给词汇中的单词 1057——它是视觉单词 1057 的一个实例。这个特殊视觉单词在图像序列中出现的次数为

```
>> bag.occurrence(1057)
ans =
    6
```

它在以下每一幅图像中至少出现了一次：

```
>> bag.contains(1057)
ans =
     1     2     3     4    14    16
```

我们可以用以下函数显示出一些单词 1057 的不同实例：

```
>> bag.exemplars(1057, images)
```

如图 14.50 所示。这些样本实际上看起来很不相同，但需要记住，我们是在把它们当作像素图案进行观察，而相似性是用描述符表达的[②]。然而这些样标确实在一些主要的水平和垂直结构上是相似的。

① BagOfWords 类使用来自 http://www.vlfeat.org/的 MEX 文件 k 均值实现方法。它使用了自己的随机数发生器，并使用 vl_twister('STATE', 0.0)把这个发生器初始化为一个已知状态。

② 描述符包含了哈尔小波探测器的响应，这些响应是在支持区域的多个窗口中计算的。

图 14.50 视觉单词 1057 的样本，来自它出现的各个图像。样本中注释形式为单词/图像

视觉单出现的频率非常不同：

```
>> [word,f] = bag.wordfreq()
```

其中，word 是包含所有单词的一个向量，f 是它们对应的出现频率。可以用频率的降序排列来显示这些单词：

```
>> bar( sort(f, 'descend') )
```

如图 14.51 所示。那些非常频繁出现的单词对于区别两幅图像意义不大，且能力有限。它们类似于在文本文档检索中被当作停止词的英语单词①。把视觉停止词从词袋中移除：

```
>> bag.remove_stop(50)
Removing 1850 features associated with 50 most frequent words
>> bag
bag =
BagOfWords: 16095 features from 20 images
            1950 words, 50 stop words
```

它会留下 16000 个 SURF 特征。这个方法执行了重贴标签，所以单词标签现在的范围是 1 ~ 1950。

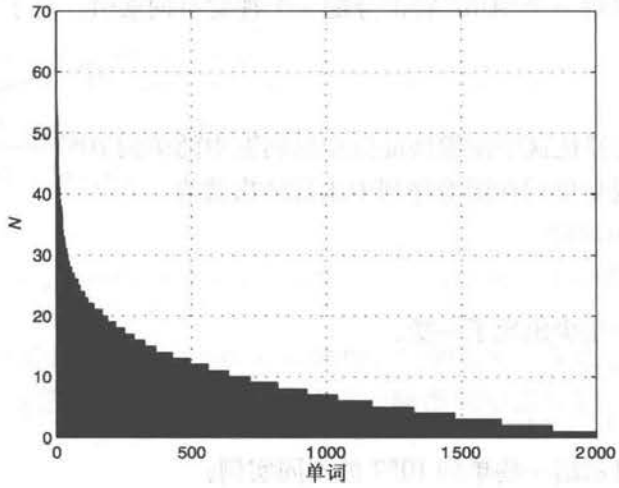


图 14.51 每一个单词(分类的)出现次数的直方图。注意，编号越小的单词出现的频率越高

我们的视觉词库中包含了 K 个视觉单词，在这种情况下，K = 1950。我们采用一种来自文本文档检索中的技术，把每一幅图像用一个包含 K 个元素的向量来描述：

$$\boldsymbol{v}_i = (t_1, \cdots t_j \cdots t_K)$$

它的元素描述了图像中对应视觉单词出现的频率。

① 搜索引擎会忽略类似“a”、“and”、“the”等单词。

$$t_j = \frac{n_{ij}}{n_i} \underbrace{\log \frac{N}{N_j}}_{\text{idf}} \tag{14.18}$$

其中，j 是视觉单词标签，N 是数据库中的图像总数，N_j 是包含单词 j 的图像数量，n_i 是图像 i 中的单词数量，n_{ij} 是单词 j 出现在图像 i 中的次数。逆文档频率(idf)是一个权重项，它可以降低那些经常出现在所有图像中的单词的重要性，因而能使单词之间更加平等。带权重的单词频率向量是 BagOfWords 对象的一种属性，但是可以通过一些方法进行访问：

```
>> M = bag.wordvector;
```

它是一个 1950 × 20 的矩阵，每一列是一个包含 1950 个元素的向量，它精确描述了对应的图像。①

两幅图像之间的相似性是它们对应词频向量之间角度的余弦：

$$s(\boldsymbol{v}_1, \boldsymbol{v}_2) = \frac{\boldsymbol{v}_1 \boldsymbol{v}_2^{\mathrm{T}}}{|\boldsymbol{v}_1||\boldsymbol{v}_2|}$$

它通过 similarity 方法来执行运算。结果为 1 表示最大的相似度。计算两个词袋的互相似性很简单：

```
>> S = bag.similarity(bag)
```

它会返回一个 20 × 20 的相似性矩阵，其中元素 S(i,j)表示 M 的第 i 列和第 j 列之间或图像 i 和图像 j 之间的相似性。这样一个相似性矩阵最好通过一个图来解读：

```
>> idisp(S, 'bar')
```

如图 14.52 所示。作为一个有用的交叉校验，明亮的对角线表示图像 i 和图像 i 是相同的。

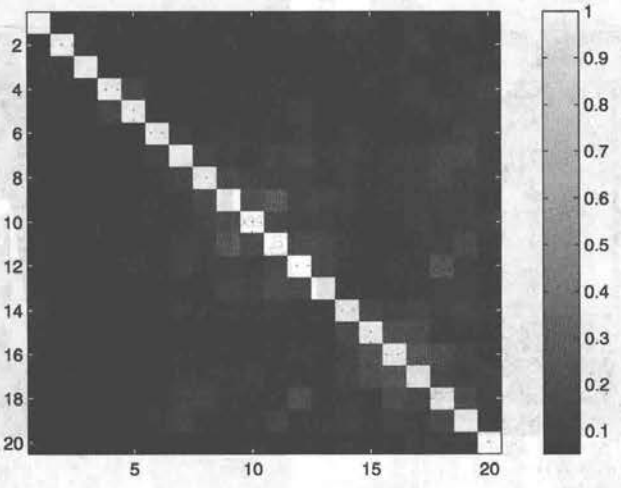


图 14.52　20 幅图像的相似矩阵，其中浅色表示强相似性。元素 (i,j) 表示了图像 i 和图像 j 之间的相似性

考虑图 14.53(a)所示的图像 11。它和其他图像的相似性由相似性矩阵的第 11 行或第 11 列给出：

```
>> s = S(:,11);
```

我们把它们按照相似性大小进行降序排列：

① 这个向量看起来似乎很大，但是它包含的原始图像元素的数量小于 1%。

```
>> [z,k] = sort(s, 'descend');
>> [z k]
ans =
    1.0000   11.0000
    0.3448    9.0000
    0.2377   13.0000
    0.1948   10.0000
    0.1719    7.0000
      .
      .
```

其中每一行包含相似性测度值和对应的图像。正如预料的那样，图像 11 与它自己当然是完全相同的，在这个相似性降序排列中，我们看到有图像 9、13、10 等。这些图像可以在图 14.53 中进行观察，我们看到这种算法召回了同一建筑物差别很大的视图。

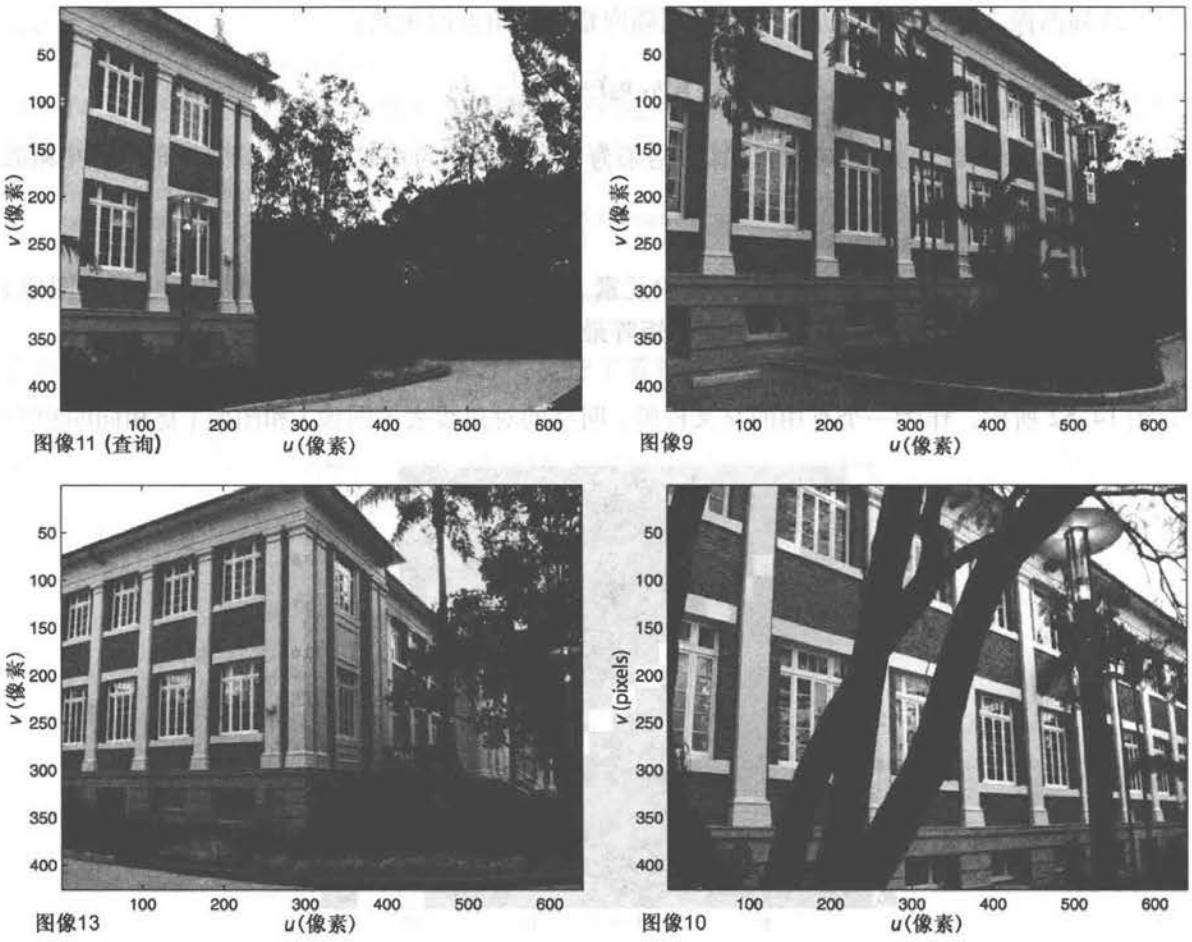


图 14.53 图像召回。图像 11 是查询对象，我们召回了图像 9、16 和 10，并按照匹配质量降序排列

现在考虑有一些新图像，并且想要确定先前图像中哪一幅图像与它们最为相似。实际应用中，机器人可能会拍摄一幅图像，然后把它和现存的图像数据库进行对比。这个问题的处理步骤与前面的示例大体相似：

```
>> images2 = iread('campus/holdout/*.jpg', 'mono');
>> sf2 = isurf(images2)
```

但是我们并不进行聚类，而是给已有的视觉单词集合分配特征，也就是为每一个新特征描述符确定最接近的视觉单词：

```
>> bag2 = BagOfWords(sf2, bag)
bag2 =
BagOfWords: 4708 features from 5 images
            1950 words, 50 stop words
```

这个操作也去掉了先前被确定为停止词的任何特征单词，并根据方程(14.18)计算了词频向量[①]。

最后，在两个词袋中图像之间的相似性是

```
>> S2 = bag.similarity(bag2);
```

它会返回一个 20×5 的矩阵，其中元素 S(i,j)表示已有图像 i 和新图像 j 之间的相似性。每一列中的最大值对应在以前观察的图像集合中最为相似的一幅：

```
>> [z,i] = max(S2)
z =
    0.2751    0.690    0.5121    0.5017    0.3177
i =
     7    11    16    18    20
```

新图像 1 的最佳匹配是原始序列中的图像 7，新图像 2 最佳匹配图像 11，等等。新图像和它们最接近的现存图像如图 14.54 所示。第一个的召回图像有较低的相似性得分，是不正确的——它们是完全不相同的建筑物。然而这两个场景确实有一些相似特征，例如窗户、屋檐线和沟槽。

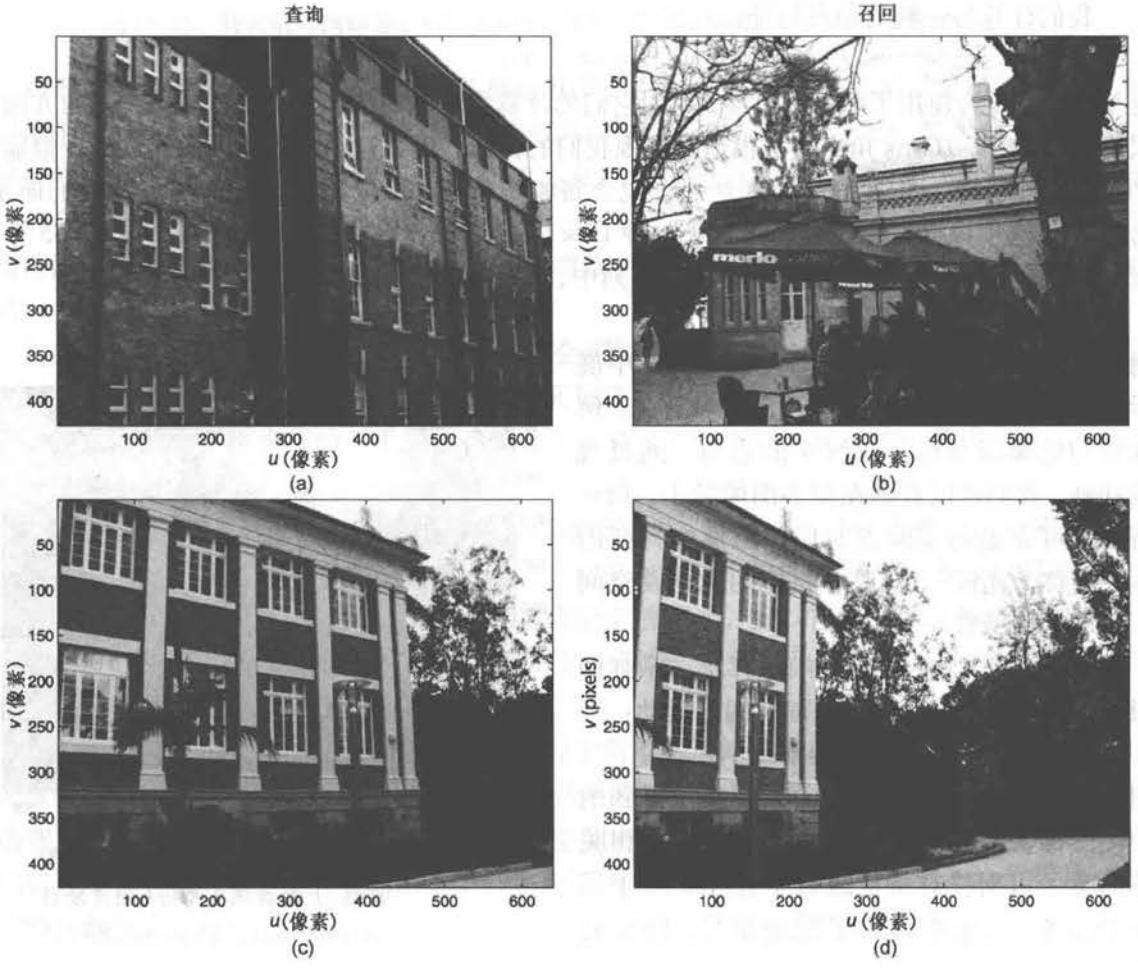


图 14.54　对新图像的图像召回。新的查询图像(a)和(c)分别召回了数据库图像(b)和(d)

① 它需要从现存的词袋中进行单词－图像统计，以计算逆文档频率权重项。

14.8 应用：图像序列处理

在本章的最后一节中，我们要讨论怎样使用工具箱来处理图像序列。我们加载一系列图像，这些图像是开着车在公路上拍摄的：

```
>> im = iread('bridge-l/*.png', 'roi', [20 750; 20 480]);
```

选项'roi'从每一幅图像中选择了一片感兴趣的区域来消除不规则的黑色边界[①]。这些图像具有很少见的 16 位像素：

```
>> about(im)
im [uint16] : 461x731x251 (169169482 bytes)
```

图像 im 属于'uint16'类型。由于这个图像的大小已经接近 200 兆字节，所以无法把它转换成双精度，因为这可能要占用 4 倍的所需内存数量。

图像序列可以用动画形式展示出来：

```
>> ianimate(im, 'fps', 10);
```

每秒 10 帧。

我们计算每一帧的角点特征：

```
>> c = icorner(im, 'nfeat', 200, 'patch', 7);
```

这里，我们改为使用了 Harris 角点，因为它们的计算量小。对于该应用，帧与帧之间的方向和尺度变化较小，Harris 角点特征很适合实现我们的目标。函数返回一个单元数组，每一幅输入图像为一个元素，并且每个元素是一个包含每幅图像中 200 个最强 Harris 角点特征的向量。'patch'选项根据方程(14.1)指定了一种 15×15 的局部邻域描述符，它是一个包含 255 个元素的单位向量。可以把特征叠加在图像序列中，并用动画显示出来：

```
>> ianimate(im, c, 'fps', 10);
```

每秒 10 帧，其中单独的一帧在图 14.55 中展示出来。特征点与具有高梯度的区域有关，例如树的轮廓以及标牌或汽车的边角。通过观看动画，我们可以看到在很多图像帧中，角点特征都可靠地与空间点对应。图像中特征的运动被称为光流[②]，它是相机空间运动及空间三维结构的函数。

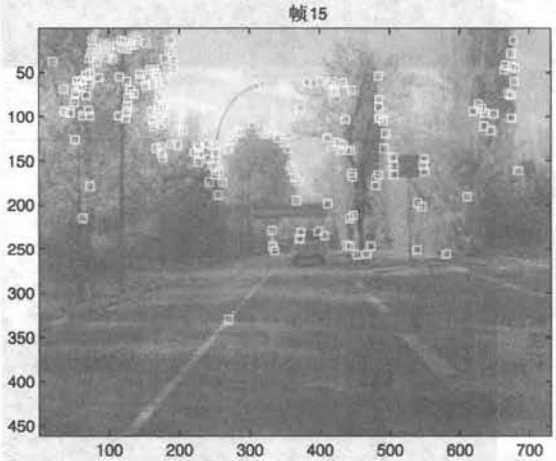


图 14.55 bridge-l 图像序列中的第 15 图像帧，上面叠加了特征(图片来自 .enpeda. 项目，[Kletteetal,2011])

工具箱的 Tracker 类中保存了一系列的跟踪数据，它们是单个特征随时间的运动：

```
>> t = Tracker(im, c);
```

图 14.56(a)显示了从 Tracker 操作得到的一帧图像，其中采用了与某种特定踪迹相关联的唯一识别符对特征进行了标注。对于每一个踪迹，对象中包含了踪迹编号、特征的

① 黑色边界是图像矫正的结果。

② 在下一章中将会再次提到光流。

当前位置、特征描述符，以及一些统计量，如被观察到的次数和从上一次被看到经过的图像帧数。

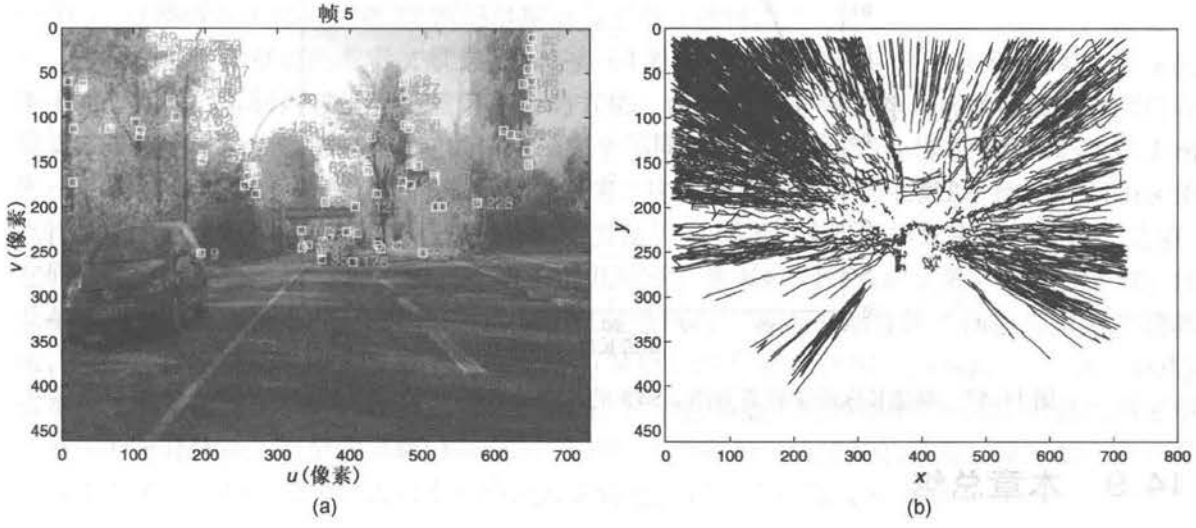


图 14.56　特征的时间踪迹。(a)显示唯一踪迹识别符的特征；(b)所有的特征踪迹

对于每一个新的图像帧，每一个踪迹都会在其前一帧所在位置的指定半径范围内寻找相关特征子集。这些特征的描述符将与前一帧该踪迹的特征描述符进行比较，如果找到匹配，那么踪迹就会被更新。如果任何特征都没有被已有踪迹声明过，那就认为是一个新踪迹的开始——会在列表中建立一个新的入口，并把唯一的踪迹识别符分配给它。如果某个踪迹不能在当前图像帧中找到匹配特征，那么计数就会增加，当计数值超过一定阈值时，该踪迹会被撤销。

与踪迹相关的识别符在每一个时间步中都会更新。采用这种方式，即使连续图像帧的支持区域在外表上发生了变化，我们也能拿新特征与最近时间步的踪迹特征进行比较。

关于踪迹的数据作为类的一个属性存储起来。特征轨迹可以被显示为一条线：

```
>> t.plot()
```

如图 14.56(b)所示。特征背离图像中心的运动清晰可见，特征向外开始延伸的起点称为扩张中心。根据两个相机的核面几何学知识，第二个相机在第一个相机的前面，所以核点靠近图像的中心，且所有核线都穿过它。由于对应点必须位于核线上，所以它们沿着这些辐射线运动，并且运动量与空间点的深度成反比。在本书的下一部分，我们将会更加深入地探讨特征运动和相机运动之间的关系。

我们看到一些没有沿径向的踪迹，这是由过于简单的追踪器造成的，它没有强制执行帧对帧的核线约束，因而 RANSAC 可以将其消除。其中的一个水平踪迹是因为汽车正在桥上行驶，这违背了我们的环境都是刚性的假设，就是说空间点之间不能有相互运动。特征被追踪到的图像帧数量差异很大。方法 `tracklengths` 会返回一个向量，其中记录了每个特征被追踪到的图像帧数量：

```
>> ihist( t.tracklengths(), 'normcdf' )
```

图 14.57 显示了特征轨迹长度的分布。大部分是比较短的，平均值为 14 帧，但是最长的为 140 帧。

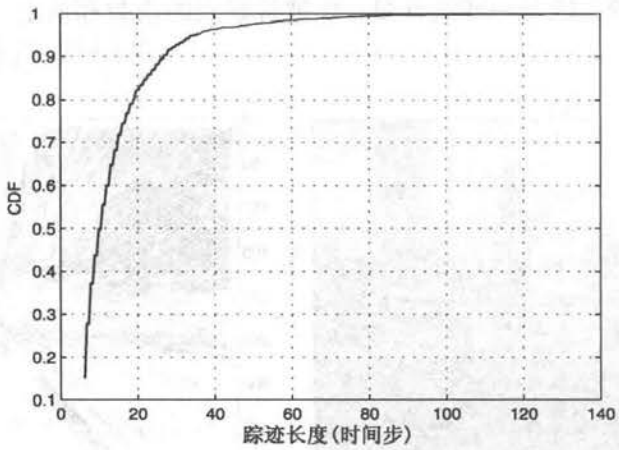


图 14.57 踪迹长度的累积直方图。50% 的踪迹长超过 10 帧，5% 的踪迹长超过 40 帧

14.9 本章总结

这一章涵盖了很多议题，目的是要演示真实机器人视觉系统中需要使用的众多概念。有两条常用的线索贯穿整章。第一条线索是使用角点特征寻找图像中的独特点，然后把它们与另一幅图像中相同的空间点进行匹配。第二条线索是使用附加信息源恢复在透视变换过程中丢失的点的深度或平移缩放因子。

我们把单一相机成像几何学扩展到两个相机的情况，并展示了两幅图像中的对应点是怎样被基本矩阵所约束的。我们指出了从图像数据中估计基本矩阵的方法，错误数据关联的影响，以及怎样使用 RANSAC 算法克服这个影响。使用相机内在参数可以计算出本质矩阵，然后分解本质矩阵，给出两个视图之间的相机运动，但是平移运动存在一个未知的缩放因子。借助平移量大小等一些额外信息，相机运动才可以被完全估计出来。给定相机运动，那么空间点的三维坐标就可以估计出来。

位于同一个平面上的空间点引出了单应性矩阵的概念，即图像之间图像点的线性映射。单应性矩阵可以用来检测不在同一个平面上的点，并且可以进行分解从而给出两个视图之间的相机运动(平移运动又一次存在着未知的缩放因子)和平面的法线。

如果基本矩阵是已知的，那么可以修正一对有重叠图像来创建一个核线平齐的立体图像对，并且密集立体匹配可以用于恢复每一个点的空间坐标。我们还讨论了由于遮挡和纹理缺失等影响造成的误差，并探讨了检测这些误差的技术。

最后，把这些多视图技术应用于很多实例上，如透射矫正、图像拼接、图像检索和图像序列处理等。

扩展阅读

19 世纪中期，从事摄影测量的团体开始了对三维重构和相机位姿估计的研究。20 世纪 60 年代，计算机视觉和人工智能的团体开始对计算机视觉或机器人视觉进行研究。本书采用的语言表达和术语虽然遵循了计算机视觉方面的文献，但要用于理解摄影测量的文献也是没有问题的，仅有很小一点困难。

早期关于多视图几何学的重要研究都是在实验室中开展的，例如在 20 世纪 80 年代和 90

年代的斯坦福大学、斯坦福研究所(SRI)、麻省理工学院人工智能实验室、卡内基梅隆大学(CMU)、喷气推进实验室(JPL)、自动化研究所(INRIA)、牛津大学，以及日本电子竞技联盟(ETL)，这些研究工作导致在21世纪早期出版了很多教材。

多视图几何学重要的参考文献为[Hartley and Zisserman, 2003]和[Ma et al., 2003]。这两本书对于相同的资料内容呈现了截然不同的方法。前者采用了更偏向几何学的方法，而后者则更多是数学方法。遗憾的是，它们使用了完全不同的符号体系，并且与本书使用的也都不相同——表14.1给出了一些重要符号元素的总结。这些书都涵盖了特征提取(使用了Harris角点特征，因为在SIFT这类尺度不变特征检测方法以及SURF角点检测方法被开发出来之前，它们已经出版了)；一个、两个和N个视图的几何学；基本矩阵和本质矩阵；单应性矩阵；以及基于离线批处理技术的三维场景重构和相机运动估计。两本书都提供了伪码形式的关键算法，并在它们的相关网站上公布了一些支持的MATLAB®程序代码。[Faugeras et al., 2001]这本较早的书籍使用了一些相当数学化的方法，其中也涵盖了很多相似的内容，但是仍然采用了不同的符合体系。更早的书籍[Faugeras, 1993]主要聚焦于线特征的稀疏立体匹配。而近期的这本专著[Szeliski, 2010]则对本章中的众多议题进行了非常深入的讨论。

表14.1　罗赛达石碑。两本其他著名的教材与本书之间的符号差异总结

对象	Hartley and Zisserman, 2003	Ma et al., 2003	本书
空间点	$\mathbf{X}$	P	$\mathbf{P}$
图像点	$\mathbf{x},\mathbf{x}'$	x_1, x_2	${}^1\boldsymbol{p}, {}^2\boldsymbol{p}$
第i图像点	$\mathbf{x}_i, \mathbf{x}'_i$	x_1^i, x_2^i	${}^1p_i, {}^2p_i$
相机运动	$\mathrm{R},\mathbf{t}$	R,T	$\boldsymbol{R},\boldsymbol{t}$
归一化坐标	$\mathbf{x},\mathbf{x}'$	x_1, x_2	$(\bar{u},\bar{v})$
相机矩阵	P	Π	$\boldsymbol{C}$
齐次量	$\mathbf{x},\mathbf{X}$	x,P	$\tilde{\boldsymbol{p}},\tilde{\boldsymbol{P}}$
齐次等价	$\mathbf{x}=\mathrm{P}\mathbf{X}$	$\lambda x=\Pi P \quad x\sim\Pi P$	$\tilde{\boldsymbol{p}}\simeq\tilde{\boldsymbol{P}}$

关于SURF和其他特征检测器的参考文献之前在第367页上进行过讨论。特征检测器的性能以及它们的匹配性能均在[Mikolajczyk and Schmid, 2005]中进行了讲解，该书中回顾了很多不同的特征描述符，包括旋转图像和局部喷射①。

[Fischler and Bolles, 1981]中描述的RANSAC算法是本章中讨论的所有基于特征的方法的精华。PROSAC是RANSAC算法近期的一个扩展[Chum and Matas, 2005]，它利用了对应点的排序。[Pilu, 1997]中讨论了怎样把SVD应用到由特征之间的距离形成的一个矩阵，以此来确定对应关系。[Dellaert et al., 2000]描述了一种概率的方法，这种方法可以从一组无序图像中确定结构。

基本矩阵这个术语是在陈德良的论文中定义的[Luong, 1992]。[Xu and Zhang, 1996]这本书则对核面几何学进行了深入浅出的介绍。核面几何学也可适用于非透视相机，在这种情况下核线变成了核曲线[Mičušík and Pajdla, 2003]，[Svoboda and Pajdla, 2002]。对于三个视图的情况，其几何学由三角张量$\mathcal{T}$描述，它是一个具有18个自由度的$3\times3\times3$张量，能把一幅图像中的某个点与其他两幅图像中的核线联系起来[Hartley and Zisserman, 2003]，[Ma et al.,

① 喷射是一个高阶导数的向量，例如$I_{uu'}$, $I_{vv'}$, $I_{uv'}$, $I_{uuu'}$, $I_{uuv'}$, $I_{uvv'}$, $I_{vvv'}$, $I_{uuuu'}$, $I_{uuuv'}$, $I_{uuvv'}$, $I_{uvvv'}$, I_{vvvv}等[Mikolajczyk and Schmid, 2005]。

2003]。对于一个图像序列，在核面几何学方面的一篇重要早期文献是[Bolles et al., 1987]。

本质矩阵的第一次描述是在 10 年前的一封写给 *Nature* 的信中[Longuet-Higgins, 1981]，这封信是有杰出的理论化学家和认知科学家克里斯托弗·郎盖-希金斯(Christopher Longuet-Higgins, 1923—2004)所写。信中描述了根据 8 个对应的点对估计本质矩阵的方法。本质矩阵分解是在[Faugeras 1993, § 7.3.1]中首次描述的，但在教材[Hartley and Zisserman, 2003]和[Ma et al.,2003]中也进行了介绍。在本章中，我们先计算本质矩阵，然后对其进行分解，由此估计出了相机运动。第一步至少需要 8 对的对应点，但是在[Nistér, 2003]和[Li and Hartley, 2006]中介绍的算法可以只用 5 对点就能直接把运动计算出来。单应性矩阵分解在很多文献中都有描述，包括[Faugeras and Lustman, 1988]，[Hartley and Zisserman, 2003]，[Ma et al., 2003]以及[Malis and Vargas, 2007]。这些矩阵之间的关系、相机运动和相关的工具箱函数在图 14.58 中进行了总结。

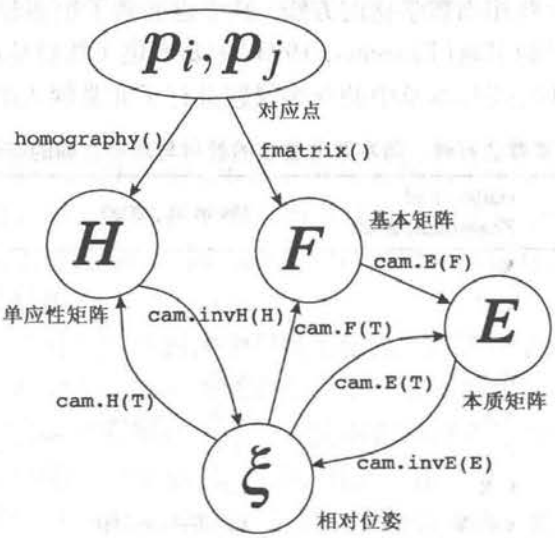


图 14.58 工具箱函数和相机对象方法，以及它们的相互关系

立体相机和立体匹配软件现在可以从几个供销商处获得，它们可以提供高分辨率的深度映射，且速度在标准计算机上超过 10Hz。这在 10 年以前是很难想象的，那时需要包括 FPGAs 在内的定制硬件才能实现实时操作[Corke et al., 1999]，[Woodfill and Von Herzen, 1997]。立体视觉在行星探测器导航上的应用在[Matthies, 1992]中进行了讨论。立体视觉可以使用两个以上的相机，多相机立体视觉在[Okutomi and Kanade, 1993]中进行了介绍，它对于尖桩栅栏效应这类问题具有很强的鲁棒性。

[Brown et al., 2003]对立体视觉技术进行了深入浅出的回顾，主要聚焦于实时性问题。[Shirai, 1987]是一本关于立体视觉原理的老书，但描述非常清晰。在[Scharstein and Szeliski, 2002]中把立体处理分为 4 个步骤：匹配、聚合、差异计算和细化。每一步中不同运算的时间花费和性能是可比的。本章中的示例可以描述为：NCC 匹配，边框过滤器聚合，胜者为王的差异计算和亚像素细化。一些更复杂的方法是考虑了邻近像素的相似性，它们采用了迭代正则化、动态规划或基于图形的方法，这样可以提高不连续处的性能，并可以建立遮挡的显式模型。然而，提高差异数据的平滑性和外观，需要针对图像不能提供有力指示的环境空间做出一些假设，因而在用于机器人控制时需要谨慎。立体相机与我们自己的两只眼睛非常相似，但当

充分依赖立体视觉的同时，也应当意识到它不是我们推断距离的唯一技术[Cutting, 1997]。

ICP 算法的应用很广泛[Besl and McKay, 1992]，从机器人领域到医学成像。ICP 算法很快，但通过最近相邻点确定对应关系的运算量 $O(N^2)$ 很大。ICP 的很多变种也已经成熟，它们使得这种方法对外点数据具有鲁棒性，并提高了针对大量数据的计算速率。[Salvi et al., 2007]中对一些不同算法进行了回顾和比较。确定两个点集之间的相对方向是一个很经典的问题，本章使用的 SVD 方法在[Arun et al., 1987]中进行了描述。[Horn et all., 1988]和[Horn, 1987]中描述了基于四元法和正交旋转矩阵的解决方法。

从运动获取结构(SfM)，即同步恢复空间结构和相机运动，也是计算机视觉中的一个经典问题。相关知识可以参考两篇有用的回顾性论文，分别是提供了分类方法的[Huang and Netravali, 1994]，以及[Jebara et all., 1999]。在[Broida et all., 1990]中，作者使用扩展卡尔曼滤波器(EKF)描述了一种早期的针对单目相机图像序列的递归 SfM 技术，其中每一个空间点由它的 (X,Y,Z) 坐标表示。[McLauchlan, 1999]详细描述了一种用于 SfM 的可变长度状态估计器。[Azarbayejani and Pentland, 1995]中则提出了一种递归方法，其中每个空间点都被一个标量参数化，这个标量是空间点相对于第一幅图像的深度。一种具有有界估计误差的新算法在[Chiuso et all., 2002]中进行了描述，其中还讨论了尺度变化的问题。[Davison et all., 2007]中介绍的 MonoSlam 系统是一个令人印象深刻的单目 SfM 系统，它具有一种局部映射，其中甚至包含了不在当前视野中的特征。SfM 在大规模的城市测绘应用中变得越来越流行，[Pollefeys et all., 2008]中描述了一种对于大规模图像集脱机处理的系统。

脱机 SfM 的问题在本书中并未涉及，它也是通过处理一系列图像来恢复运动和结构的。操作方法通常包括估计每一个视图的相机矩阵，计算一个投影重构，然后把它升级到一个欧几里得重构[Hartley and Zisserman, 2003][Ma et all., 2003]。这些脱机技术的关键部分是光束平差，它是一个调整估计相机位姿和空间点的优化过程，以使估计图像点与真实图像点之间的误差达到最小。[Triggs et all., 2000]对光束平差进行了很好的介绍。

SfM 问题可以通过使用立体的而非单目的图像序列进行简化[Molton and Brady, 2000][Zhang et all., 1992]，也可以通过合并惯性数据进行简化[Strelow and Singh, 2004]。一个相关的技术是视觉测程法(VO)，它只考虑了恢复相机运动。[Nistér et all., 2006]中描述了一种用于单目或立体视觉的基于特征点的系统。[Maimone et all., 2007]中则介绍了在火星探测机器人上使用立体相机 VO 的经验，而[Corke et all., 2004]中描述了在一台行星探测机器人样机上采用的反射折射视觉测程法。

图像拼接是和摄影一样古老的技术。过去它是一项需要精湛技艺和劳动密集的行业，需要用到照片、小刀和砂纸。20 世纪 60 年代月球和邻近行星的表面图像是使用由机器人航天器发回的图像经手工拼接而成的。今天，已有很多高质量的拼接软件工具可以用于创建全景图像，例如在 http://hugin.sourceforge.net 中的 Hugin 开源项目，以及专利产品 AutoStitch。

用于图像检索的“词袋”技术首先在[Sivic and Zisserman, 2003]中提出，随后已经被很多其他的研究者使用。它在机器人应用上的著名扩展技术是 FABMAP[Cummins and Newman, 2008]，它明确指出了特征出现的联合概率，并把一个概率与图像匹配联系起来。

图像序列分析是很多实时机器人视觉系统的核心。实时特征追踪技术在[Hager and Toyama, 1998]和[Lucas and Kanade, 1981]中进行了描述，它通常是基于计算量小的 Harris 角点检测器或者金字塔 KLT 追踪器。SURF 检测器用于这个技术仍然太耗时，尽管一些基于 C 语言和基于 GPU 的实现方法具有一定的实时能力。

资源

计算机视觉领域借助标准数据库的建立有了很大发展。这些数据库使得研究人员能将不同算法作用于相同的数据，并对它们的性能进行定量比较。最早的立体图像对图集之一是是 JISCT 数据库[Bolles et all., 1993]，它是以对该数据库做出贡献的研究组织进行命名的：JPL，INRIA，SRI，CMU 和 Teleos。这个数据库可通过 http://vasc. ri. cmu. edu/idb/html/jisct 进行查阅。在 http://vision. middlebury. edu/stereo 上有更新的 Middlebury 数据库[Scharstein and Szeliski, 2002]，它提供了大量高分辨率的立体图像，这些图像采用了不同的曝光设置，并包含了地面真实数据。来自 NASA 火星探测车“勇气号”和“机遇号”的立体图像可通过 http://marsrover. nasa. gov/gallery/3d 在线获得。这些图像采用了补色立体图像格式，左侧和右侧图像分别被编码成红色和蓝绿色。补色立体图像的红颜色和蓝颜色平面是被矫正后的左侧和右侧图像。运动图像数据库包括在 http://vasc. ri. cmu. edu//idb/html/motion 中的经典室内场景运动图像序列、http://homepages. inf. ed. ac. uk/rbf/CAVIARDATA1 中的人在建筑物内移动图像，http://i21www. ira. uka. de/image_sequences 中的交通场景图像、以及 http://www. mi. aucklland. ac. nz/EISATS 中从一部移动车辆拍摄的图像。

由斯坦·贝奇菲尔德(Stan Birchfield)用 C 语言编写的一种 KTL 特征追踪器程序可通过网址 http://www. ces. clemson. edu/ ~ stb/klt 获得。由克里斯托弗·扎克(Christoper Zach)用 C 语言编写的一种基于 GPU 的 KTL 版本可在 http://www. cs. unc. edu/ ~ cmzach/opensource. html 中获得。SIFT 和 SURF 实现程序的获取路径已在第 367 页上给出。由吉安·卢卡·马里奥蒂尼(Gian Luca Mariottini)和多米尼科·帕拉蒂奇佐(Domenico Prattichizzo)创建的用于 MATLAB 的核面几何学工具箱[Mariottini and Prattichizzo, 2005]可在 http://egt. dii. unisi. it/#tth_sEc7 中获得，它能处理透视和反射折射相机。安德鲁·戴维森(Andrew Davison)用 C 语言编写的用于 MATLAB 的 MonoSLAM 系统可在 http://www. doc. ic. ac. uk/ ~ ajd/software. html 获得。

另外还有一些脱机或批处理 SfM 工具箱。由飞利浦·托尔[Philip Torr, 2002]创建的在 MATLAB® 中的结构和运动工具箱可在 http://www. mathworks. com/matlabcentral/fileexchange/4576-structure-and-motion-toolkit-in-MATLAB® 获得。文森特·吕保德(Vincent Ribaud)创建的“从运动获取结构工具箱(structure from motion toolbox)”集成了很多流行的 SFM 算法[Rabaud, http://vision. ucsd. edu/ ~ vrabaud/toolbox]，可在 http://code. google. com/p/vincents-structure-from-motion-matlab-toolbox 中获得。上述工具箱充分利用了[Lourakis and Argyros, 2009]介绍的稀疏光束平差(SBA)工具。另外一个光束平差软件包是“简单稀疏光束平差(Simple Sparse Bundle Adjustment, SSBA)，它是由克里斯托弗·扎克创建的，可在 http://www. cs. unc. edu/ ~ cmzach/opensource. html 中获得。

一首关于基本矩阵的歌可以在 http://danielwedge. com/fmatrix/找到。

习题

1. 角点特征。针对 Harris 和 SURF 特征，检测角点强度的累积分布。对于特征匹配，选择强角点的适当方式是什么？
2. 特征匹配。在使用 RANSAC 之后，可以用内点百分比来定义基于描述符的特征匹配质量。
   a)选取任何一幅图像。我们将用该图像与它的各种变换图像进行匹配，以检测 SURF 和

Harris 特征的鲁棒性。变换分别是：(a)把光强度缩减 70%；(b)增加标准差为 0.05、0.5 和 2 灰度值的高斯噪声；(c)把图像尺寸按 0.9,0.8,0.7,0.6 和 0.5 缩小；(d)图像旋转 5°, 10°, 15°, 20°, 30°, 40°。

b)对于 Harris 检测器，比较它对基于结构张量的特征的检测性能，斑块描述符大小分别为 3×3,7×7,11×11,15×15。

c)尝试增加 SURF 和 Harris 角点的抑制半径。较低密度的匹配可以提高匹配性能吗?

d)Harris 检测器可以处理彩色图像。与同一图像的灰度版本相比，它会有更好的性能吗?

e)在外点匹配与所涉及的角点特征强度之间有相关性吗?

3. 给定图像 1 中的一个点，写出图像 2 的核线方程。

4. 证明核点是基本矩阵的零空间。

5. 给定相机 1 的基本矩阵和相机矩阵，你能确定相机 2 的相机矩阵 C 吗?

6. 估计基本矩阵(第 381 页)

a)对于合成数据的示例，改变点的数量和添加的高斯噪声，然后观察对残差造成的影响。

b)对于埃菲尔铁塔数据，观察改变 RANSAC 参数之后的影响。使用一个较低强度阈值(默认为 0.002)来计算 SURF 特征，然后重复这个操作。

c)一个随机样本中画出 8 个内点的概率是多少(无替换)?

7. 本质矩阵(第 379 页)。

a)为一台做纯旋转运动的相机创建一组对应点，然后计算基本矩阵和本质矩阵。你能恢复旋转运动吗?

b)对于平移和旋转运动并存的情况，通过分解本质矩阵得出这两种位姿并把它们显示出来。绘制出草图或者使用 `trplot`。

8. 稀疏立体匹配(第 390 页)。

a)射线相交方法可以返回射线之间的最近距离(理想情况下为 0)。绘制出闭合误差的直方图，并计算出平均误差和最大误差。

b)上述试验假定相机平移大小为 30 cm。再针对 25 cm 和 35 cm 的情况重复该试验，闭合误差的统计值改变了吗? 你能确定哪种平移能使这个误差最小吗?

9. 对于相机交汇的情形，推导出用差异表达的深度关系。相机交汇情形是指两个相机的光轴相交，类似于图 14.6 所示的相机。

10. 立体视觉。使用岩石堆示例(第 395 页)。

a)使用 `idisp` 对差异图像进行放大，然后检测图像边界和岩石边缘上的像素值。

b)使用不同的相似性测度和窗口大小进行试验。你能观察到它们对差异图像和计算时间有什么影响吗?

c)改变差异图像进行试验。尝试`[50,90]`, `[30,90]`, `[40,80]`和`[40,100]`。在差异图像上会发生什么? 为什么?

11. 使用岩石堆示例(第 395 页)获得差异空间图像 D。

a)对于选择的像素 (u,v)，绘制出 $D(u,v,d)$ 随 d 变化的曲线。找到具有锐利峰值、宽峰值和弱峰值的像素。对于使用 ZSSD 相似性测度计算出来的立体图像，重复这个过程。对于选择的一行 v，把 $D(u,v,d)$ 显示为一幅图像，它代表了什么?

b)对于一个特定像素，绘制出 s 随 d 变化的曲线，把最大值周围拟合成一条抛物线，再把它叠加到图中。

c)使用来自 DSI 的原始数据，找到每一个像素的第二个峰值，然后计算出模糊比。

d)对于图像 1 中选定的一个点，显示其在图像 2 中对应的核线。

12. 下载一幅补色立体图像，然后把它转换成一对灰度图像，再计算密集立体匹配。

13. 核面几何学。

a)创建两个中央相机，一个在原点，另一个在 x 轴方向存在平移。对于稀疏的额平行空间网格点，显示与图像 1 中投影点对应的图像 2 中的核线群。描述一下这些核线。对于相机 2 沿 y 轴或 z 轴的平移，或者绕 x,y,z 轴旋转的情况，分别重复上述过程。对于复合运动，例如 x 平移加 z 平移或 x 平移加 y 旋转的情况，重复上述过程。

b)图 14.16 中的庭院示例具有稍微向下倾斜的核线。它给出了关于两个相机视点的什么信息？

14. 单应性矩阵(第 385 页)。

a)对于在 x,y,z 方向上的平移和绕 x,y,z 轴的旋转，分别计算它们的欧几里得单应性矩阵。再把它们转换成投影单应性矩阵，并应用到额平行网格点上。所产生的图像运动是你所希望的吗？把这些单应性矩阵应用于对真实图像(如莉娜)的变形处理。

b)分解图 14.15 中庭院图像的单应性矩阵，确定墙相对于相机的平面。你会用到相机的内在参数。

15. 平面拟合(第 409 页)。

a)测试平面拟合算法对附加噪声和外点的鲁棒性。

b)运用一种带权重的迭代方法，使外点的影响最小化。

c)创建一种基于 RANSAC 的平面拟合算法，它需要提取三个点的随机样本来求解方程(14.17)。使用 `fmatrix` 和 `homography` 的程序代码来引导帮助你。你需要创建很多由 `ransac_driver` 调用的函数。

16. ICP(第 410 页)。

a)在你的计算机上运行 ICP 示例，并观察动画。

b)改变点云之间的初始相对位姿。尝试一些很大的旋转。

c)提高加入到数据点中的噪声。

d)在有丢失数据点或假数据点的情况下，使用`'distthresh'`选项的不同值进行试验。

17. 从运动恢复结构(第 412 页)。

a)使用不同等级的相机噪声做试验，观察它们对估计相机位置的影响。

b)噪声对估计相机姿态的影响是什么？

c)基于图形帧之间的已知运动距离，修改建立里程计模型的仿真，然后使用那个信息纠正平移缩放因子。对于里程计中有系统尺度误差的情况，重复上述操作。对于里程计中有添加噪声的情况，重复上述操作。

d)使用一个移动平均滤波器，使帧到帧的空间点估计更平滑。

e)使用一个扩展卡尔曼滤波器，使帧到帧的空间点估计更平滑(具有挑战性)。

18. 透视矫正(第 421 页)。

a)创建一个向下 45°观看教堂正前方的虚拟视图。

b)在原始相机高度创建一个虚拟视图，但相机要向左转 20°。

c)找到另一幅具有透视畸变的真实图像，并尝试对其进行矫正。

19. 拼接(第 421 页)。

a)运行示例文件 mosaic，观察整个拼接过程。

b)更改贴片粘贴到合成图像的方法，使用像素求平均，而不是相加。

c)更改贴片粘贴到合成图像的方法，使用离主点最近的像素。

d)在你自己的一组叠加图像中运行这个软件，并创建一个全景图。

20. 词袋(第 423 页)。

a)使用 exemplars 方法检测不同视觉单词的不同支持区域。

b)研究改变停止词数量的影响。

c)研究改变词库大小的影响。尝试 1000,1500,2500,3000。

d)从你自己的一组图像中建立一个词袋。

21. 图像序列追踪(第 430 页)。

a)在你自己的计算机上运行示例。

b)使用默认的 Harris 特征描述符重复上述过程，即不使用'patch'选项。踪迹的质量发生了什么变化?

c)使用不同的斑块大小重复上述过程。踪迹质量发生了什么变化?

d)使用 SURF 替代 Harris 角点特征重复上述过程。踪迹质量提高了吗? 计算时间是怎样改变的?

e)修改追踪器，以便它能够保存对图像中特征速度的估计(基于前面两帧或更多帧中的特征运动)。改变关联逻辑，使得搜索圆盘以特征的预测位置为中心，而不是它以前的位置。过滤器可以是基于一个简单的匀速模型、一个 $\alpha-\beta$ 追踪器(使用 AlphaBetaFilter 类或子类)、工具箱卡尔曼过滤器(KalmanFilter 抽象超类)。

f)使用 RANSAC 估计每一图像帧的基本矩阵。改变关联逻辑，使用核线约束。

g)估计帧到帧的本质矩阵。需要的相机标定数据在文件 README 中。分解本质矩阵来确定帧与帧之间的位姿变化。对姿态相对于时间的变化进行积分，以提供一种视觉罗盘或陀螺仪功能。你是怎样恢复帧到帧平移中的未知缩放因子的? 你能绘制出从正上方头顶看到的汽车轨迹吗?

h)桥梁图像序列是以立体图像形式记录下来的，但到目前为止我们只使用了左边相机的图像。文件夹 bridge-r 中包含了对应的右边相机图像。使用文件中的相机标定数据对每一个图像对进行稀疏立体匹配重构。使用 ICP 确定连续帧之间汽车位姿的变化。

i)对每一个图像对进行密集立体匹配重构。

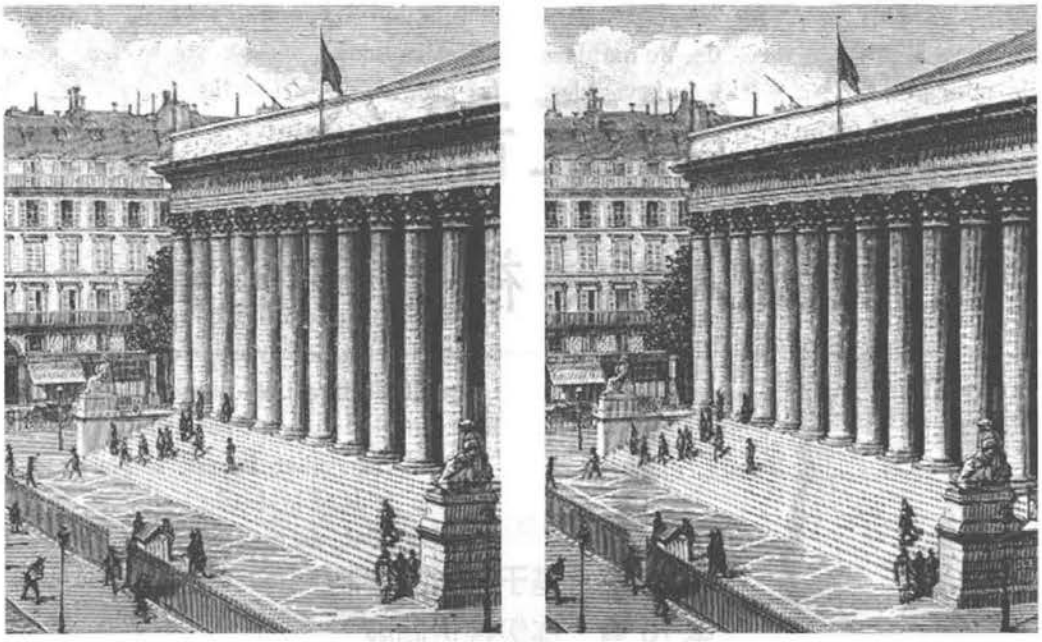
j)对你自己电脑拍摄的直播视频，计算 Harris 特征，并把它们叠加到拍摄的图像帧中。在你的电脑上，它每秒运行多少帧? 使用 Tracker 类的 step 方法来追踪这些直播特征。

第五部分

机器人学、视觉与控制

第 15 章　基于视觉的控制
第 16 章　高级视觉伺服

人们通常说机器人向某个目标物体移动，实际上机器人只是移向它期望物体所在的一个位姿。其中的差别看似微不足道，但确实有很大。如果物体不在期望的位姿，那么机器人就不能抓住物体。另外，如果机器人的机械结构和控制器存在缺陷，并导致其末端执行器不能准确达到指定的位姿，那么它也会抓不住物体。为了使抓取物体的方法成功，我们需要解决两个非常难的问题：确定目标物体的位姿，以及使机器人准确到达那个位姿。



第一个问题——确定一个目标物体的位姿，在机器人工业生产应用中通常可以避免，只要保证物体总能够被准确放置到位。但这需要昂贵的机械夹具和固定装置，而且对于机器人要操作的每一种不同零件都必须定制和安装相应的装置——一定程度上削弱了机器人自动化的灵活性。

第二个问题——确保机器人能到达一个期望的位姿，也不是简单的事情。正如我们在第 7 章讨论的那样，机器人末端执行器要通过计算要求的关节角度才能向一个位姿运动。这就要求机器人的运动学模型是准确的，它反过来就要求机器人的加工制造精度非常高：连杆长度必须准确，关节轴之间必须精确平行或正交。而且连杆还要有很强的刚性，以确保它们在动态载荷或重力作用下不会变形。另外，还要求机器人有精确的关节传感器和高性能的关节控制器，它们能消除由摩擦和重力负载产生的稳态误差。我们在 9.3.4 节中讨论的非线性控制器具有这种高性能，但是它们需要非常准确的动力学模型，这个模型中包含了每根连杆质量、重心和惯性，以及末端有效负载等很多参数。

上述问题中没有一个是不能解决的，但这样做会将我们领上一条非常复杂的道路。结果是一台很重、很大刚性的机器人，它反过来又要求有强有力的驱动器来移动它，以及高质量的传感器和一个复杂的控制器——所有这些都推高了机器人的总成本。然而，除非万不得已，我们应当尽可能避免去解决难题。让我们冷静一会儿，思考一下这个问题，我们可以清楚地意识到这个问题的根源是机器人不能看见它正在做的事情。

考虑一下这种情况，机器人能够看见目标物体和自己的末端执行器，还能使用这些信息引导末端执行器向目标物体运动。这就是人们常说的手眼协同，我们称之为基于视觉的控制或

视觉伺服控制——使用从一个或多个相机获取的信息，引导机器人去完成一项工作任务。

目标的位姿没有必要提前知道，机器人无论在工作空间中的什么地方，它都会朝它观察到的目标运动。这种方法有很多的优点：可以放宽零件的位置公差；有能力处理运动的零件，且基本没有额外代价；可以补偿机器人开环控制产生的任何精度误差。

一个基于视觉的控制系统需要对目标和机器人进行连续测量，使用视觉产生一个反馈信号。然后根据反馈信号驱动机器人机械臂，直到机器人与目标之间的视觉观察误差等于零。基于视觉的控制完全不同于拍一张图像、确定目标位置、再接近目标。视觉连续测量和反馈的优点在于，它对于系统中的任何误差都提供了很强的鲁棒性。当然，在实际应用中也存在一些较复杂的问题。如果相机安装在机器人的末端，它有可能与任务目标产生干涉，或者当机器人离目标很近时相机可能不能聚焦，或者目标可能被末端夹持器遮挡。

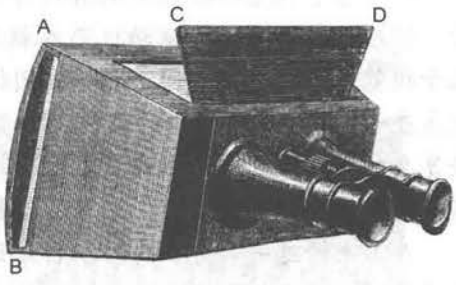
在本书的这一部分，我们将把前面学过的许多内容集成在一起：机械臂和移动机器人的运动学和动力学；成像的几何知识；特征提取。这部分包括两章。第 15 章讨论两种视觉伺服的经典方法，即基于位置的视觉伺服和基于图像的视觉伺服。空间特征的图像坐标被用于驱动机器人向一个与被观察目标相关的期望位姿运动。第一种方法需要根据图像特征明确估计出目标位姿，但由于它是以一种闭环方式执行的，所以能补偿位姿估计中的任何误差。第二种方法无需估计目标位姿，它直接使用图像平面上的信息。两种方法的讨论都是以一个能在 SE(3)中自由运动的透视相机为信息源。我们将比较两种方法各自的优点和缺点。该章还将讨论确定目标深度的问题，以及直线和椭圆图像特征的使用。

第 16 章延伸讨论了混合视觉伺服算法，该算法通过使用基于位置和基于图像两种视觉伺服方法中的最好特征，克服了这两种方法的限制。讨论然后进一步扩展到非透视相机，如鱼眼相机和反射折射相机，以及臂式机器人、完整和非完整约束地面机器人，以及一个飞行机器人。

本书这一部分的定位比前面各部分都要高，它要求对前面各部分的内容都要很熟悉，而且很多更复杂的示例也只是给出概述，没有详细地讨论。文字上只是介绍了每种技术核心的数学和算法原理，而详细信息还要从实现这些控制器的 MATLAB 源程序代码中或 Simulink 仿真图形细节中找出。这些技术的结果也很难在一本书中进行描述，读者最好是自己在计算机上运行相关的 MATLAB 或 Simulink 程序，绘制出结果图形或观看动画仿真。

第 15 章　基于视觉的控制

视觉伺服的任务是使用从图像中提取的视觉特征，控制机器人末端执行器相对于目标的位姿。如图 15.1 所示，相机可以安装在机器人上，随机器人运动，或者固定在周围环境中。图 15.1(a)所示结构是相机安装在机器人的末端执行器上，用以观察目标，这称为端点闭环或手眼。图 15.1(b)所示结构有相机固定在周围环境的一个点上，同时观测目标和机器人的末端执行器，这称为端点开环。在本书的剩余部分中，我们将只讨论相机在机器人上的情况。



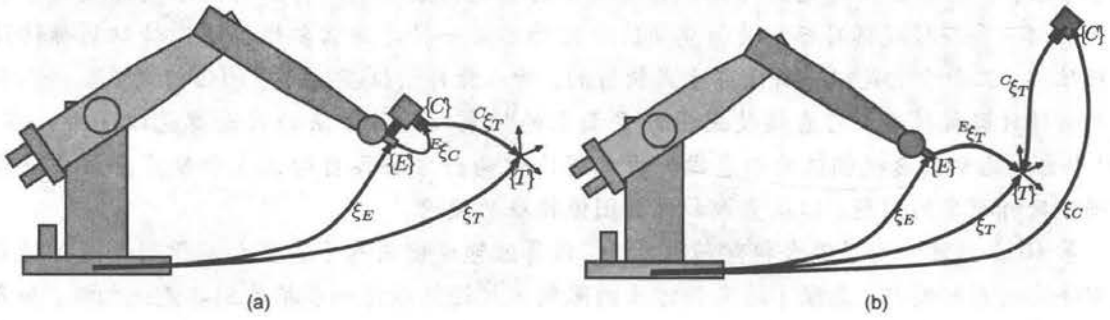


图 15.1　视觉伺服的结构和相关的坐标系：世界，末端执行器{E}、相机{C}和目标{T}。(a)端点闭环结构(相机在机器人上)；(b)端点开环结构

目标的图像是一个相对位姿 ${}^{C}\xi_{T}$ 的函数。诸如点的坐标、直线或椭圆参数这些特征是从图像中提取的，它们也都是相对位姿 ${}^{C}\xi_{T}$ 的函数。有两种完全不同的视觉伺服控制方法：基于位置的视觉伺服(PBVS)和基于图像的视觉伺服(IBVS)。基于位置的视觉伺服，如图所 15.2(a)所示，使用了观察到的视觉特征、一个标定的相机和一个已知的目标几何模型，来确定目标相对于相机的位姿。机器人随后向那个位姿运动，其运动控制在通常为 $SE(3)$ 的任务空间中被执行。对于位姿估计，已有很好的算法存在，但它的计算量很大，而且完全依赖于相机标定和目标几何模型的准确性。PBVS 将在 15.1 节中讨论。

背景材料：伺服机构或伺服器是一种自动装置，它使用一个机构的期望位姿和实际位姿之间的误差反馈来驱动该装置到所需的位置。伺服这个词来自拉丁词根“servus”，意义是奴隶，第一次使用是在 1868 年由法国人法科特(J. J. L Farcot)写的“Le Servomoteur”，被用来描述用于船舶操纵的液压和蒸汽机。

位置误差由一个传感器测量，然后经过放大去驱动一个电机，电机再产生一个力来驱动装置以减少误差。伺服系统在二战时得到快速发展，当时为满足火控的应用需求，需要开发采用电动马达和机电功率放大器的电气伺服系统。后来伺服放大器使用了真空管，而现在多用固态功率放大器(驱动电机)。今天，伺服机构可以说无处不在，它们被用来在光学和磁盘驱动器中定位读/写头，在自动相机中实现对焦，以及用于遥控玩具、卫星跟踪天线、自动机床、机器人关节等。

“伺服”正确地讲应该是名词或形容词，但现在已经变成为一个动词。在基于视觉的控制中，我们使用的是动词形式的“视觉伺服”。

基于图像的视觉伺服，如图所示 15.2(b)，省略了位姿估计的步骤，直接使用图像特征。控制操作是在图像坐标空间 $\mathbb{R}^2$ 中被执行。相对于目标的所需相机位姿是由目的地位姿处的图像特征值隐含定义的。IBVS 是一个具有挑战性的控制难题，因为图像特征是关于相机位姿的一个高度非线性函数。IBVS 将在 15.2 节中讨论。

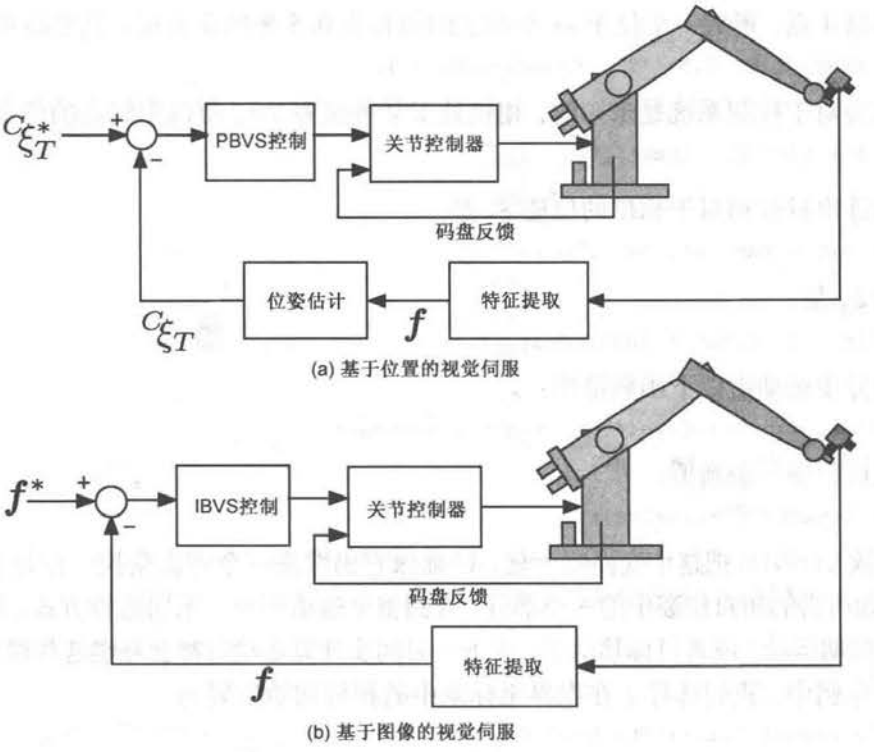


图 15.2　两种不同类型的视觉伺服系统

15.1　基于位置的视觉伺服

PBVS 系统中目标相对于相机的位姿 $^C\xi_T$ 是估计值。位姿估计问题在 11.2.3 节中进行了讨论，它需要知道目标几何形状、相机的内在参数和观察到的图像平面特征。位姿之间的关系如图 15.3 所示。我们指定了相对于目标的期望相对位姿 $^{C^*}\xi_T$，并希望能够确定相机从初始位姿 ξ_C 到期望位姿 ξ_C^* 的运动，其中从 ξ_C 到 ξ_C^* 称为 ξ_Δ。目标的实际位姿 ξ_Δ 是不知道的。根据位姿体系，可以写出

$$\xi_\Delta \oplus {}^{C^*}\xi_T = {}^C\hat{\xi}_T$$

其中，$^C\hat{\xi}_T$ 是目标相对于相机的估计位姿。我们把上式重新整理为

$$\xi_\Delta = {}^C\hat{\xi}_T \ominus {}^{C^*}\xi_T$$

它就是要求去到达期望相对位姿的相机运动。位姿的变化可能是相当大的，所以不要指望这个运动能一步到位，而是先移动到一个接近 $\{C^*\}$ 的点，该点位姿由下式定义：

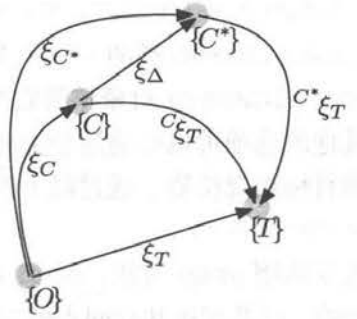


图 15.3　PBVS 示例的相对位姿体系。坐标系 $\{C\}$ 是当前的相机位姿，坐标系 $\{C^*\}$ 是期望的相机位姿

$$\xi_C\langle k+1\rangle = \xi_C\langle k\rangle \oplus \lambda\xi_\Delta\langle k\rangle$$

它是所需平移和旋转总量的一部分，其中 $\lambda \in (0, 1)$。

使用工具箱，我们首先定义一个参数已知的相机：

```
>> cam = CentralCamera('default');
```

目标包括 4 点，形成一个位于 xy 平面上的边长为 0.5 米的正方形，其中心在(0, 0, 3)：

```
>> P = mkgrid( 2, 0.5, 'T', transl(0,0,3) );
```

假设这个位姿对于控制系统是未知的，相机处于某种位姿 Tc，所以空间点的像素坐标是

```
>> p = cam.plot(P, 'Tcam', Tc)
```

从中可以估计出目标相对于相机的位姿 ${}^C\hat{\xi}_T$ 是

```
>> Tc_t_est = cam.estpose(P, p);
```

需要的运动 ξ_Δ 是

```
>> Tdelta = Tc_t_est * inv(TcStar_t);
```

向目的地的分步运动由以下函数给出：

```
>> Tdelta = trinterp(eye(4,4), Tdelta, lambda);
```

然后赋给相机位姿一个新值：

```
>> Tc = trnorm(Tc * Tdelta);
```

我们使用函数 trnorm 把这个变换归一化，以确保它仍然是一个齐次变换。在每个时间步重复该过程，移动所需的相对位姿中的一小部分，直到整个运动完成。采用这种方式，即使机器人有误差而未按预期运动，或者目标移动了，在下一时间步计算运动时都会补偿这些误差。

在这个示例中，我们选择了在世界坐标系中的相机初始位姿为

```
>> Tc0 = transl(1,1,-3)*trotz(0.6);
```

以及期望的相对于相机的目标位姿为

```
>> TcStar_t = transl(0, 0, 1);
```

它表示目标位于相机前方 1 米处，并且是额平行的。我们创建一个 PBVS 类的实例：

```
>> pbvs = PBVS(cam, 'T0', Tc0, 'Tf', TcStar_t)
Visual servo object: camera=noname
  200 iterations, 0 history
  P= -0.25       -0.25        0.25         0.25
     -0.25        0.25        0.25        -0.25
          0           0           0           0
  Tc0:   t = ( 1 1 -3), R = ( 34.3775deg |  0 0 1)
  Tc*_t: t = ( 0 0 1), R = nil
```

这是 VisualServo 类的一个子类，它实现了上面规划的控制器功能。对象构造函数需要一个 CentralCamera 对象作为它的参数，以驱动这个相机达到那个相对于目标的期望位姿。许多其他的选项可以传递这个类构造函数。显示方法能展示空间点的坐标、初始相机位姿及期望的目标相对位姿。通过以下指令运行仿真：

```
>> pbvs.run();
```

它会反复调用 step 方法，step 方法执行单步操作。如图 15.4 所示，仿真动画包含了相机的平面图像，以及相机和空间点的三维可视化图像。当迭代次数达到指定值，或 ξ_Δ 低于某个阈值时，仿真完成。

仿真结果被存储在对象中以供日后分析。我们可以把目标特征的路径绘制在图像中，包括它们随时间变化的笛卡儿速度与位置：

```
>> pbvs.plot_p();
>> pbvs.plot_vel();
>> pbvs.plot_camera();
```

如图 15.5 所示。我们在图像中看到了特征点的弯曲路径，而且相机的位置和姿态都平滑地收敛到期望值。

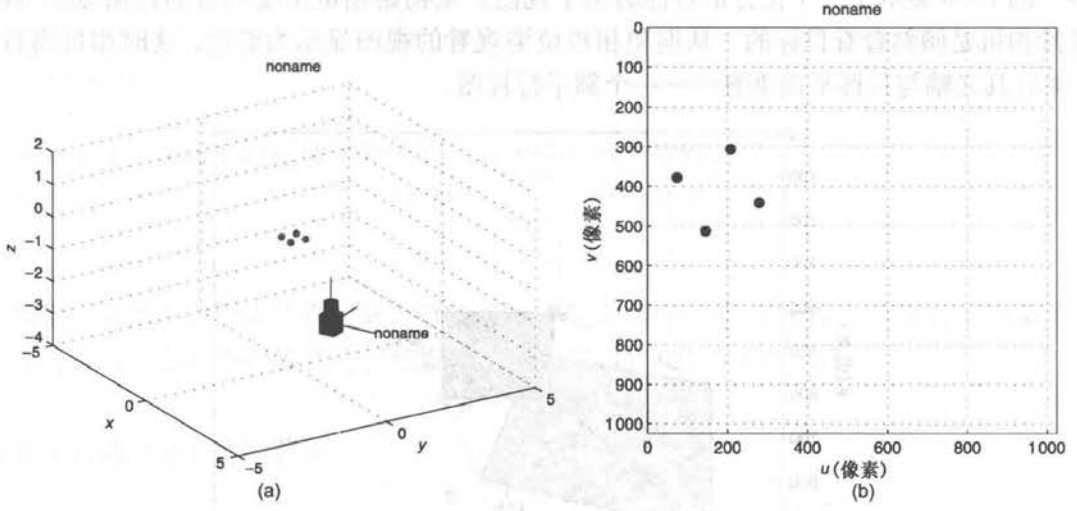


图 15.4　视觉伺服仿真的图像抓拍。(a)显示相机位姿和特征点一个外部视图；(b)显示特征点在图像平面上当前位置的相机视图

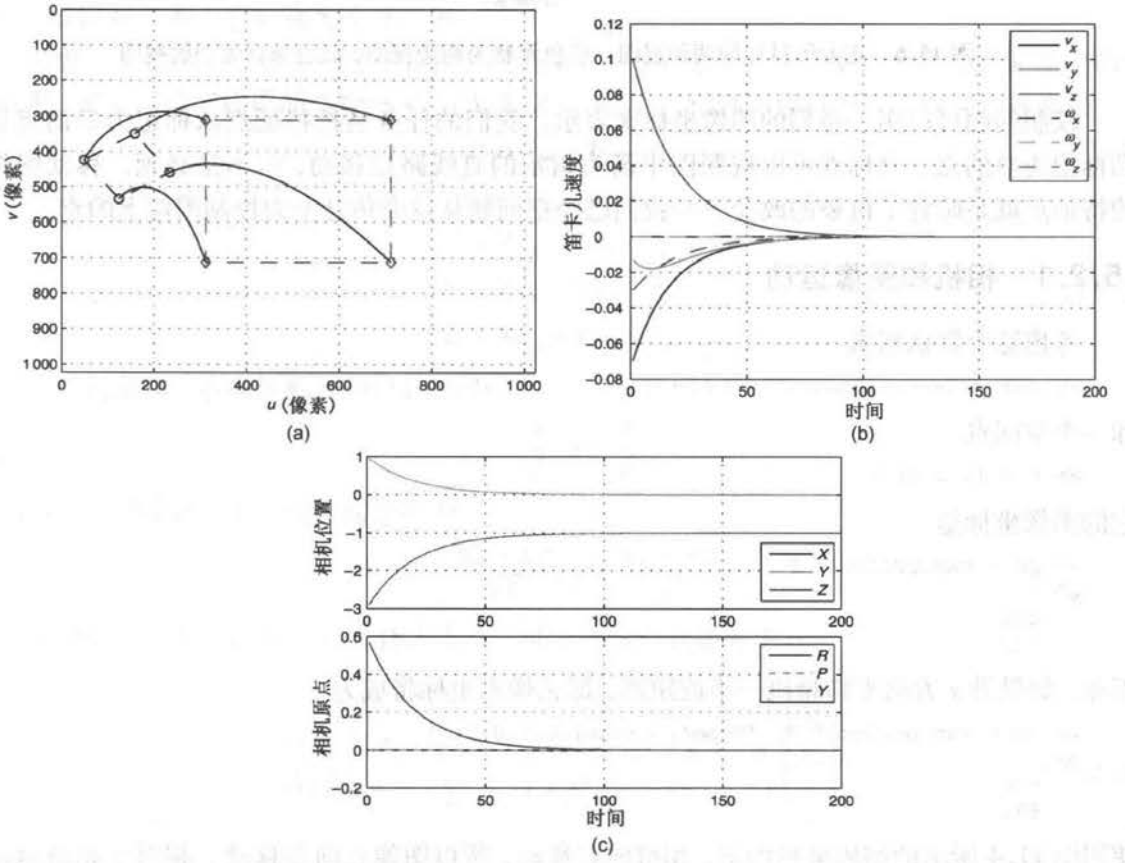


图 15.5　PBVS 仿真结果。(a)图像运动，○为初始特征位置，◇为期望的位置；(b)笛卡儿速度；(c)相机位姿

15.2 基于图像的视觉伺服

IBVS 从根本上与 PBVS 不同，它不估计目标的相对位姿。相对位姿是隐含在图像特征的值中。图 15.6 显示了一个正方形目标的两个视图。从初始相机位姿观看的视图显示为红色，很显然相机是倾斜着看目标的。从期望相机位姿观看的视图显示为蓝色，这时相机离目标更远，而且其光轴与目标平面垂直——一个额平行视图。

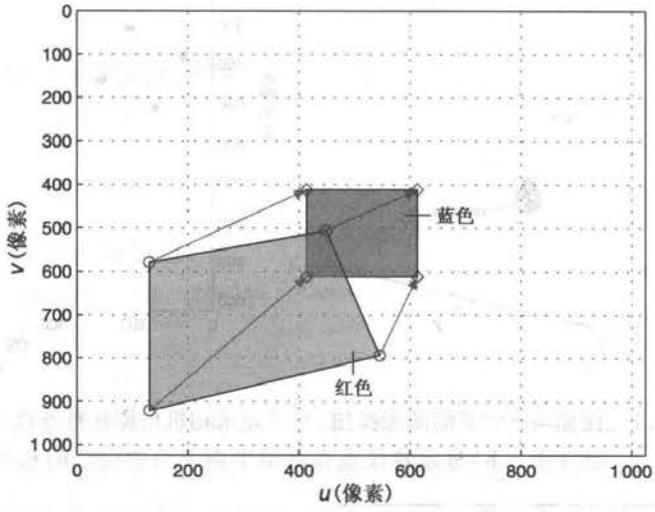


图 15.6 正方形目标的两种视图。蓝色形状为期望视图，红色形状为初始视图

控制问题可以用一系列的图像坐标来表示。我们的任务是把特征点从标记为○的点移动到标记为◇的点。坐标点可以按照图中箭头指示的直线路径移动，但不是必须。移动图像中的特征点就是暗含了位姿的改变——我们已经把问题从位姿估计变为控制图像上的点。

15.2.1 相机和图像运动

考虑这个默认相机

```
>> cam = CentralCamera('default');
```

和一个空间点

```
>> P = [1 1 5]';
```

它的图像坐标是

```
>> p0 = cam.project( P )
p0 =
   672
   672
```

现在，如果沿 x 方向平移相机一小段距离，那么像素坐标将成为

```
>> px = cam.project( P, 'Tcam', transl(0.1,0,0) )
px =
   656
   672
```

使用图 11.4 所示的相机坐标约定，相机向右移动，所以图像点向左移动。相对于相机运动的图像运动灵敏度是

```
>> ( px - p0 ) / 0.1
ans =
  -160
     0
```

这是一个近似的导数 $\partial \boldsymbol{p}/\delta_x$。这表明相机每运动 1 m，会导致在 u 方向产生 −160 像素的特征运动，我们可以用沿 z 轴的平移重复这个操作：

```
>> ( cam.project( P, 'Tcam', transl(0, 0, 0.1) ) - p0 ) / 0.1
ans =
   32.6531
   32.6531
```

它表明沿 u 和 v 方向上的运动是相等的。对于绕 x 轴的旋转：

```
>> ( cam.project( P, 'Tcam', trotx(0.1) ) - p0 ) / 0.1
ans =
   40.9626
  851.8791
```

图像运动主要是在 v 方向。很明显，相机沿着或绕着 $SE(3)$ 中不同轴的运动会导致完全不同的图像点运动。之前我们曾把透视投影表示为方程(11.10)的函数形式：

$$\boldsymbol{p} = \mathcal{P}(\boldsymbol{P}, \boldsymbol{K}, \xi_C)$$

它相对于相机位姿 ξ 的导数是

$$\dot{\boldsymbol{p}} = \boldsymbol{J}_p(\boldsymbol{P}, \boldsymbol{K}, \xi_C)\boldsymbol{\nu}$$

其中，$\boldsymbol{\nu} = (v_x, v_y, v_z, \omega_x, \omega_y, \omega_z) \in \mathbb{R}^6$ 是相机的速度，它是 8.1 节中介绍的空间速度。$\boldsymbol{J}_p$ 是一个类似雅克比矩阵的对象，但因为我们求的是相对于位姿 $\xi \in SE(3)$ 的导数，而不是相对于一个向量，所以在专业上它被称为交互矩阵。

考虑一个相机在世界坐标系中以刚体速度 $\boldsymbol{\nu} = (\boldsymbol{v}, \boldsymbol{\omega})$ 移动，并观察一个世界坐标系中的空间点 $\mathbf{P}$，该点相对于相机的坐标是 $\boldsymbol{P} = (X, Y, Z)$。这个点相对于相机坐标系的运动速度是

$$\dot{\boldsymbol{P}} = -\boldsymbol{\omega} \times \boldsymbol{P} - \boldsymbol{v} \tag{15.1}$$

可以写成标量形式：

$$\begin{aligned} \dot{X} &= Y\omega_z - Z\omega_y - v_x \\ \dot{Y} &= Z\omega_x - X\omega_z - v_y \\ \dot{Z} &= X\omega_y - Y\omega_x - v_z \end{aligned} \tag{15.2}$$

对于归一化坐标，透视投影方程(11.2)为

$$x = \frac{X}{Z}, \; y = \frac{Y}{Z}$$

使用商的求导法则，上式的时间导数为

$$\dot{x} = \frac{\dot{X}Z - X\dot{Z}}{Z^2}, \; \dot{y} = \frac{\dot{Y}Z - Y\dot{Z}}{Z^2}$$

把方程(15.2)，$X = xZ$ 和 $Y = yZ$ 代入上式，可以把它写成矩阵形式：

$$\begin{pmatrix} \dot{x} \\ \dot{y} \end{pmatrix} = \begin{pmatrix} -\frac{1}{Z} & 0 & \frac{x}{Z} & xy & -(1+x^2) & y \\ 0 & -\frac{1}{Z} & \frac{y}{Z} & 1+y^2 & -xy & -x \end{pmatrix} \begin{pmatrix} v_x \\ v_y \\ v_z \\ \omega_x \\ \omega_y \\ \omega_z \end{pmatrix} \tag{15.3}$$

上式把相机速度与归一化图像坐标形式的特征速度联系在一起。

归一化图像平面坐标与像素坐标的关系可以用方程(11.7)表示为

$$u = \frac{f}{\rho_u}x + u_0,\ v = \frac{f}{\rho_v}y + v_0$$

重新整理后得

$$x = \frac{\rho_u}{f}\bar{u},\ y = \frac{\rho_v}{f}\bar{v} \tag{15.4}$$

其中，$\bar{u} = u - u_p$ 和 $\bar{v} = (v - v_0)$ 是相对于主点的像素坐标。上式的时间导数是

$$\dot{x} = \frac{\rho_u}{f}\dot{\bar{u}},\ \dot{y} = \frac{\rho_v}{f}\dot{\bar{v}} \tag{15.5}$$

然后把方程(15.4)和方程(15.5)代入方程(15.3)，得出

$$\begin{pmatrix}\dot{\bar{u}}\\ \dot{\bar{v}}\end{pmatrix} = \underbrace{\begin{pmatrix}-\frac{f}{\rho_u Z} & 0 & \frac{\bar{u}}{Z} & \frac{\rho_u \bar{u}\bar{v}}{f} & -\frac{f^2+\rho_u^2\bar{u}^2}{\rho_u f}\sigma_v\ \bar{v} \\ 0 & -\frac{f}{\rho_v Z} & \frac{\bar{v}}{Z} & \frac{f^2+\rho_v^2\bar{v}^2}{\rho_v f} & -\frac{\rho_v \bar{u}\bar{v}}{f}\ -\bar{u}\end{pmatrix}}_{J_p}\begin{pmatrix}v_x\\ v_y\\ v_z\\ \omega_x\\ \omega_y\\ \omega_z\end{pmatrix} \tag{15.6}$$

上式是用相对于主点的像素坐标表示的。可以把它写成更简洁的矩阵形式：

$$\dot{p} = J_p \nu \tag{15.7}$$

其中，J_p 是针对一个特征点的 2×6 图像雅克比矩阵①。

工具箱 CentralCamera 类提供了方法 visjac_p 来计算图像雅可比矩阵，对于上面的例子它是

```
>> J = cam.visjac_p([672; 672], 5)
J =
  -160     0    32    32  -832   160
     0  -160    32   832   -32  -160
```

其中，第一个参数是兴趣点相对于图像的坐标，第二个参数是该点的深度。在前面计算出的近似值分别被显示在第一列、第三列和第四列。也可以使用直线和圆特征来推导图像雅可比矩阵，该内容将在 15.3 节讨论。

对于一个给定的相机速度，点的速度是该点坐标、该点深度和相机参数的一个函数。雅可比矩阵的每一列代表了相对于速度向量中对应部分的一个特征点的速度。CentralCamera 类的 flowfield 方法能够显示对于某个特定相机速度，在图像平面上一组网格点的特征速度。对于相机沿 x 方向上平移速度的流场为

```
>> cam.flowfield( [1 0 0 0 0 0] );
```

如图 15.7(a)所示，正如预期的那样向右移动相机，将导致所有的特征点向左边移动。点在像平面上的运动称为光流，可以根据图像序列进行计算，如 14.8 节所述。对于沿 z 方向的平移，流场为

```
>> cam.flowfield( [0 0 1 0 0 0] );
```

特征点从主点向外辐射——《星际旅行》中的变形效果，如图 15.7(e)所示。绕 z 轴旋转的流场为

① 它通常是用 u 和 v 来表达，而不是用 $-u$ 和 $-v$ 表达，但这里使用上画线符号是为了强调坐标是相对于主点的，而不是相对于通常在左上角的图像原点。

```
>> cam.flowfield( [0 0 0 0 0 1] )
```

它使得特征点绕主点旋转，如图 15.7(f)所示。

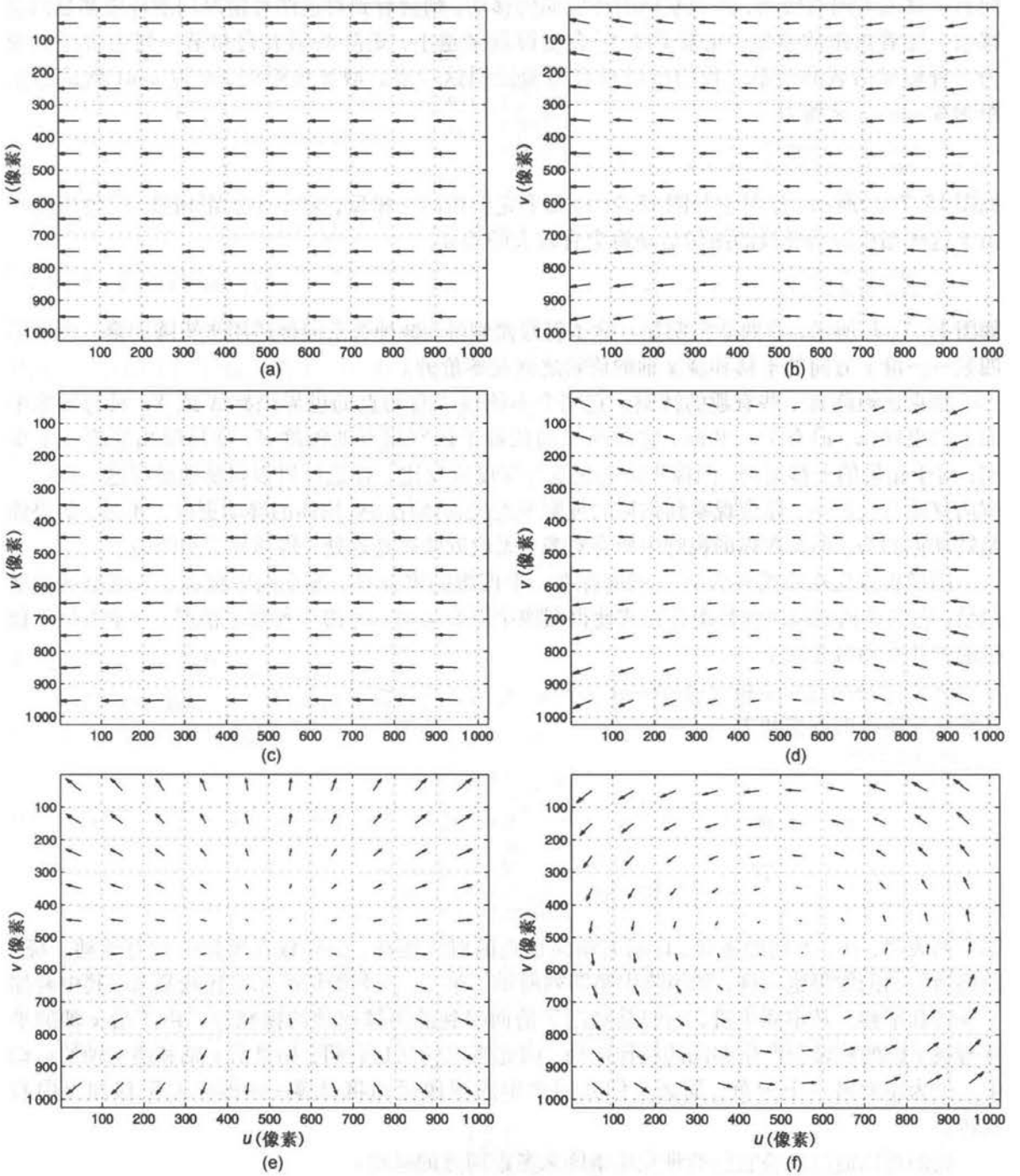


图 15.7　对于典型相机速度的图像平面速度向量，其中所有对应的空间点都在额平行平面上。(a)x 轴平移；(b)y轴旋转，$f=8$毫米；(c)y轴旋转，$f=20$毫米；(d)y轴旋转，$f=4$毫米；(e)z轴平移；(f)z轴旋转

绕 y 轴旋转运动的流场为

```
>> cam.flowfield( [0 0 0 0 1 0] );
```

如图 15.7(b)所示，它与 x 轴平移的情况非常相似，只是对于那些远离主点的特征点有少量弯

曲。导致该现象的原因是，上述图像雅克比矩阵的第一列和第五列大致相等，这意味着沿 x 方向的平移与绕 y 轴的旋转所产生的图像运动几乎相同。这种等价关系很容易证实，把你的头向右平移或者向右转动，并观察周围环境如何移动，则会看到对这两种情况周围环境都是向左移动。随着焦距的增加，元素 $\boldsymbol{J}[2,5]$ 会变得越来越小，矩阵的第五列和第一列也将近似变为一种标量倍数的关系。我们可以很容易地证明这一点，增加焦距到 $f=20$ mm（默认的焦距为 8 mm），流场为

```
>> cam.f = 20e-3;
>> cam.flowfield( [0 0 0 0 1 0] );
```

如图 15.7(c)所示，这时它与图 15.7(a)几乎完全相同。相反，对于小焦距相机（广角相机），由于这些相机运动导致的图像运动将会有较大的差异：

```
>> cam.f = 4e-3;
>> cam.flowfield( [0 0 0 0 1 0] );
```

如图 15.7(d)所示，弯曲非常明显。除了符号差别外，这种关系同样适用于矩阵的第二列和第四列——沿 y 方向的平移和绕 x 轴的旋转之间是等价的。

雅克比矩阵有一些有趣的性质。它完全不依赖于空间点的世界坐标 X 或 Y，只与图像平面上的坐标(u, v)有关。然而，它的前三列依赖于该空间点的深度 Z，并反映出这样一个事实：对于相机的平移运动，图像平面的速度与深度成反比。你也可以很容易地证明这一点——侧向移动自己的头，你会观察到在你的视野中近处物体比远处物体的移动更多。不过，如果你是转动你的头，那么在你的视野中所有对象，无论近处还是远处，移动都是相同的。

图像雅克比矩阵的秩为 2[①]，因此存在一个四维的零空间。零空间中包含了一组空间速度向量，它们单独地或以线性组合方式使得图像中没有运动。考虑一种简单情况，一个点位于相机前方并在相机光轴上：

```
>> J = cam.visjac_p([512; 512], 1)
```

其雅克比矩阵的零空间为

```
>> null(J)
ans =
         0         0   -0.7071         0
         0    0.7071         0         0
    1.0000         0         0         0
         0    0.7071         0         0
         0         0    0.7071         0
         0         0         0    1.0000
```

第一列表明，在 z 方向的运动，即沿着朝向该点的射线运动，会导致在图像中没有运动。绕 z 轴旋转运动的效果也一样，如矩阵中第四列所示。第二、三列的情况相对比较复杂，其中叠加了旋转和平移。从本质上讲，它们是利用了前面提到的图像运动的模糊性。由于沿 x 轴的平移与绕 y 轴的旋转会产生相同的图像运动，因此第三列就是表明，如果沿 x 轴和绕 y 轴的运动中一个为正而另一个为负，那么它们共同产生的图像运动将是零——即向左平移加上向右旋转。

我们可以通过堆叠它们的雅克比矩阵来考虑两点的运动：

$$\begin{pmatrix}\dot{u}_1\\ \dot{v}_1\\ \dot{u}_2\\ \dot{v}_2\end{pmatrix}=\begin{pmatrix}\boldsymbol{J}_{p_1}\\ \boldsymbol{J}_{p_2}\end{pmatrix}\boldsymbol{\nu}$$

① 矩阵的秩不能小于 2，即使 $Z\to\infty$。

它是一个4×6的矩阵，其中有两列代表零空间。其中的一种相机运动对应于绕着两个点连线的旋转。

对于三个点的情况，有

$$\begin{pmatrix}\dot{u}_1\\ \dot{v}_1\\ \dot{u}_2\\ \dot{v}_2\\ \dot{u}_3\\ \dot{v}_3\end{pmatrix}=\begin{pmatrix}\boldsymbol{J}_{p_1}\\ \boldsymbol{J}_{p_2}\\ \boldsymbol{J}_{p_3}\end{pmatrix}\boldsymbol{\nu} \tag{15.8}$$

只要三个点不重合或共线，矩阵就为非奇异矩阵。

15.2.2 控制特征运动

到目前为止，我们已经展示了当相机运动时点是如何在图像平面中移动的。经常遇到的情况是，它的逆问题更为有用——为了使图像点以期望的速度运动，需要什么样的相机运动？

对于三个点$\{(u_i, v_i), i=1\cdots3\}$的情况，相应的速度为$\{(\dot{u}_i, \dot{v}_i)\}$，可以对方程(15.8)求逆：

$$\boldsymbol{\nu}=\begin{pmatrix}\boldsymbol{J}_{p_1}\\ \boldsymbol{J}_{p_2}\\ \boldsymbol{J}_{p_3}\end{pmatrix}^{-1}\begin{pmatrix}\dot{u}_1\\ \dot{v}_1\\ \dot{u}_2\\ \dot{v}_2\\ \dot{u}_3\\ \dot{v}_3\end{pmatrix} \tag{15.9}$$

得出所需的相机速度。

给定特征速度，可以计算出所需的相机运动，但怎么确定特征速度呢？最简单的策略是使用一个简单的线性控制器：

$$\dot{\boldsymbol{p}}^*=\lambda(\boldsymbol{p}^*-\boldsymbol{p}) \tag{15.10}$$

它将驱动特征点朝它们在图像平面上的期望值$\boldsymbol{p}^*$运动。联立方程(15.9)，可得

$$\boldsymbol{\nu}=\lambda\begin{pmatrix}\boldsymbol{J}_{p_1}\\ \boldsymbol{J}_{p_2}\\ \boldsymbol{J}_{p_3}\end{pmatrix}^{-1}(\boldsymbol{p}^*-\boldsymbol{p})$$

就是这个控制器将驱动摄像头，使得特征点移向图像中期望的位置。需要特别注意的是，我们没有要求任何相机位姿或对象位姿[①]，一切都是根据图像平面上测量数据计算的。

对于一般的$N>3$的情况，可以将所有特征的雅克比堆叠，并使用广义逆求出相机的运动：

$$\boldsymbol{\nu}=\lambda\begin{pmatrix}\boldsymbol{J}_1\\ \vdots\\ \boldsymbol{J}_N\end{pmatrix}^{+}(\boldsymbol{p}^*-\boldsymbol{p}) \tag{15.11}$$

注意，有可能指定的一组特征点速度是不一致的，因此能够产生所需图像运动的相机运动则不存在。在这种情况下，广义逆可以找到一个解，它使得特征速度误差的范数最小。

① 我们需要点的深度Z。随后将对它进行讨论。

对于 $N \geqslant 3$ 的情况，如果这些点接近重合或共线，那么矩阵就呈现病态条件数。在实际应用中，这意味着某些相机运动只会引起非常小的图像运动，运动具有低的可感知性。可感知性和 8.1.4 节中讨论的可操作性的概念非常相似，因此我们将采取类似方法对它进行标准化。考虑一个具有单位大小的相机空间速度：

$$\boldsymbol{\nu}^{\mathrm{T}}\boldsymbol{\nu} = 1$$

从方程(15.7)可以写出用广义逆表达的相机速度：

$$\boldsymbol{\nu} = \boldsymbol{J}_p^{+}\dot{\boldsymbol{p}}$$

将上式代入前面的单位速度表达式，得

$$\dot{\boldsymbol{p}}^{\mathrm{T}}\boldsymbol{J}_p^{+\mathrm{T}}\boldsymbol{J}_p^{+}\dot{\boldsymbol{p}} = 1$$

$$\dot{\boldsymbol{p}}^{\mathrm{T}}(\boldsymbol{J}_p\boldsymbol{J}_p^{\mathrm{T}})^{-1}\dot{\boldsymbol{p}} = 1$$

它是在点速度空间中的一个椭球方程。$\boldsymbol{J}_p\boldsymbol{J}_p^{\mathrm{T}}$ 的特征向量定义了椭球的主轴，$\boldsymbol{J}_p$ 的奇异值为半径。最大与最小半径之比由 $\boldsymbol{J}_p$ 的条件数给出，代表了特征运动的各向异性。如果该比值很高，就表明一些点在响应某些相机运动时的运动速度较低。如果不想堆叠所有特征点的雅可比矩阵，另一种方法是选择其中三个进行堆叠，得到一个具有最佳条件数的正方形矩阵，然后对它求逆。

使用工具箱时，首先定义一个相机：

```
>> cam = CentralCamera('default');
```

目标包括 4 个点，形成了一个位于 xy 平面的边长为 0.5 m 的正方形，其中心在(0, 0, 3)：

```
>> P = mkgrid( 2, 0.5, 'T', transl(0,0,3) );
```

假设这个位姿对于控制系统是未知的。目标特征在像平面上的期望位置是一个中心位于主点的 400 ×400 正方形：

```
>> pStar = bsxfun(@plus, 200*[-1 -1 1 1; -1 1 1 -1], cam.pp');
```

它隐含表示相机与正方形目标额平行。

相机处于某种位姿 Tc，所以空间点的像素坐标为

```
>> p = cam.plot(P, 'Tcam', Tc)
```

从中可以计算图像平面误差①：

```
>> e = pStar - p;
```

以及堆叠的图像雅克比矩阵：

```
>> J = visjac_p( ci, p, depth );
```

这里，它是一个 8 ×6 的矩阵，因为 p 包含 4 个点。雅克比矩阵需要点的深度，而它是我们不知道的，所以现在我们只是选择一个常数来表示。这是一个重要的议题，我们将在 15.2.3 节中解决。

控制算法决定了相机所需的平动速度和旋转角速度：

```
>> v = lambda * pinv(J) * e;
```

① 请注意，很多基于任务函数方法的论文，如[Espiau et al., 1992]，都把该项写成负控制需求，并把方程(15.11)中的 λ 写成 $-\lambda$ 以确保负反馈。

其中，lambda 是增益，为一个正数，并且我们采用非方形雅可比矩阵的广义逆来实现方程(15.11)。所产生的速度是在相机坐标系中表示的，再把它在一个单位时间步内进行积分，就能得到对应参量的位移。相机的位姿由下式更新：

$$\xi_C\langle k+1\rangle = \xi_C\langle k\rangle \oplus \Delta^{-1}(\nu\langle k\rangle)$$

其中，$\Delta^{-1}(\cdot)$由方程(3.12)定义。使用工具箱，上式的实现方式为

```
>> Tc = trnorm( Tc * delta2tr(v) );
```

其中，我们通过使用 trnorm 将变换归一化，使得这个变换仍然是一个齐次变换。

在这个示例中，我们在世界坐标系中选择相机的初始位姿为

```
>> Tc0 = transl(1,1,-3)*trotz(0.6);
```

与 PBVS 的示例类似，我们创建一个 IBVS 类的实例：

```
>> ibvs = IBVS(cam, 'T0', Tc0, 'pstar', pStar)
```

这是一个 VisualServo 类的子类，它能实现上文所述的控制器功能。'T0'选项指定了相机的初始位姿，'pstar'指定了特征的期望图像坐标。对象构造函数把一个 CentralCamera 对象作为它的参数，驱动这台相机来达到相对于目标的期望位姿。许多其他选项也可以传递这个类的构造函数。显示方法能展示空间点的坐标、初始绝对位姿，以及期望的图像平面特征坐标。通过以下指令运行仿真：

```
>> ibvs.run();
```

它会反复调用 step 方法，step 方法执行单步操作。仿真动画包含了相机的平面图像，以及相机和空间点的三维可视化图像。

仿真结果被存储在对象中以供日后分析。我们可以把目标特征的路径绘制在图像中，包括它们随时间变化的笛卡儿速度与位置：

```
>> ibvs.plot_p();
>> ibvs.plot_vel();
>> ibvs.plot_camera();
>> ibvs.plot_jcond();
```

如图 15.8 所示。我们在图像中看到特征点的路径几乎是直线，笛卡儿位置也是平滑地向终值变化。在整个运动过程中，图像雅克比矩阵的条件数在减小，表明雅克比的条件变得越来越好，这是由于特征点扩散运动的结果。

如何确定 $\boldsymbol{p}^*$ 呢？图像点可以通过示教找到。示教方法是移动相机到期望的位姿，然后记录下所观察到的图像坐标。另一种方法，如果相机标定参数以及目标的几何形状是已知的，那么对于任何指定的终点位姿，图像坐标都可以计算出来。请注意，这个计算(空间点的投影)的工作量不大，而且只需在视觉伺服开始之前执行一次。

IBVS 系统也可以用一个 Simulink 模型来表示：

```
>> sl_ibvs
```

如图 15.9 所示。它的仿真由以下指令运行：

```
>> r = sim('sl_ibvs')
```

相机位姿、图像平面的特征误差以及相机速度都可以动画显示出来。示波器模块可以绘制出相机速度和特征误差随时间变化的曲线。相机的初始位姿由 pose 模块的一个参数设定，空间点由 camera 模块的参数设定。CentralCamera 对象是传给 camera 模块和视觉 Jacobian 模块的一个参数。

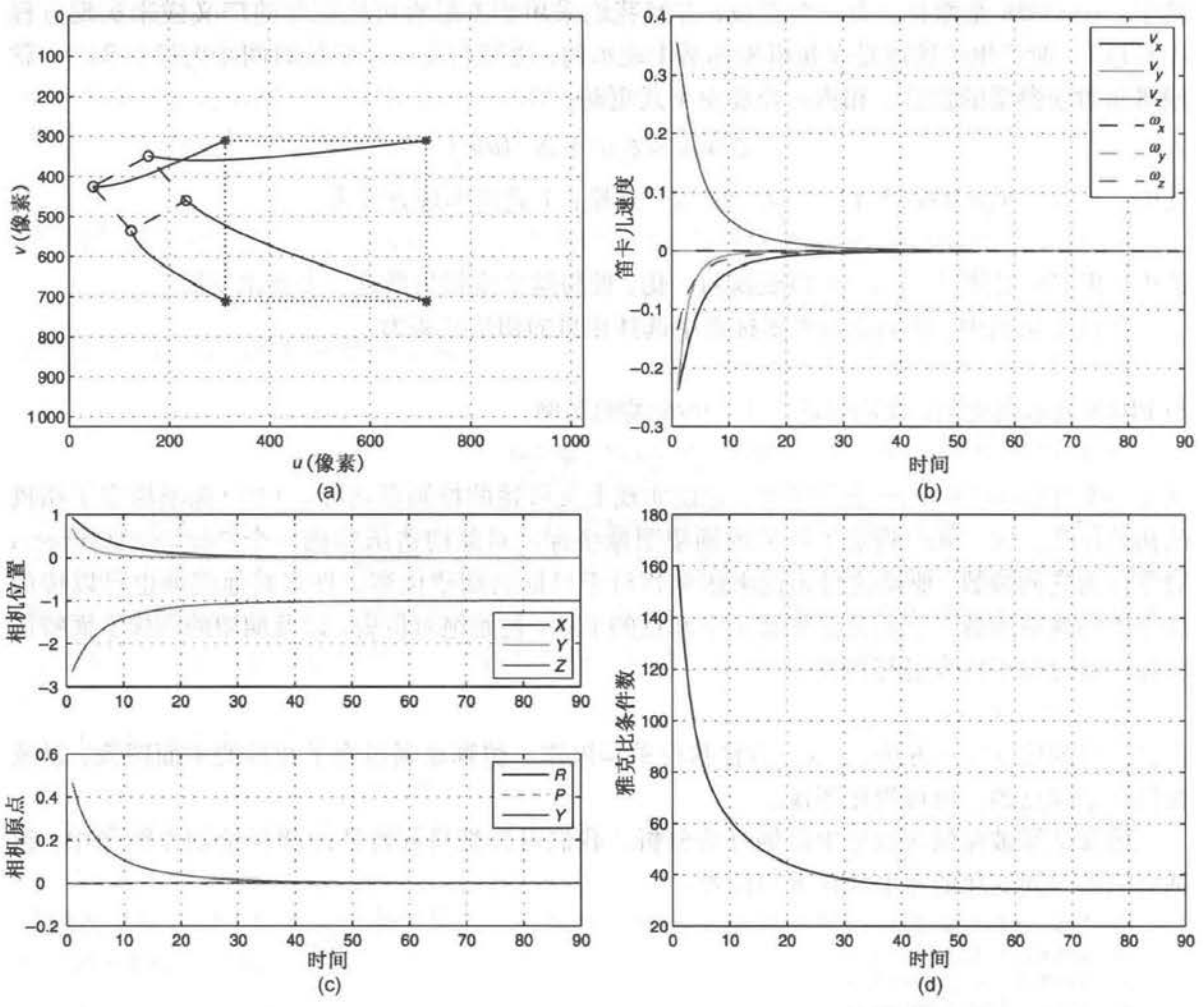


图 15.8 IBVS 的仿真结果，用函数 IBVS 创建。(a)图像平面运动，* 是期望的，○是初始的；(b)空间速度分量；(c)相机位姿；(d)图像雅克比的条件数

仿真结果存储在仿真输出对象 r 中。例如，相机速度是第二条记录的信号[①]：

```
>> t = r.find('tout');
>> v = r.find('yout').signals(2).values;
>> about(v)
v [double] : 501x6 (24048 bytes)
```

其中每个仿真步对应一行，而列是相机的空间速度分量。我们可以绘制出相机速度随时间变化的曲线：

```
>> plot(t, v)
```

图像平面坐标也被记录为第一条信号：

```
>> p = r.find('yout').signals(1).values;
>> about(p)
p [double] : 2x4x1001 (64064 bytes)
```

可以通过以下指令绘制：

```
>> plot2(p)
```

① 它连接到 2 号输出模块。

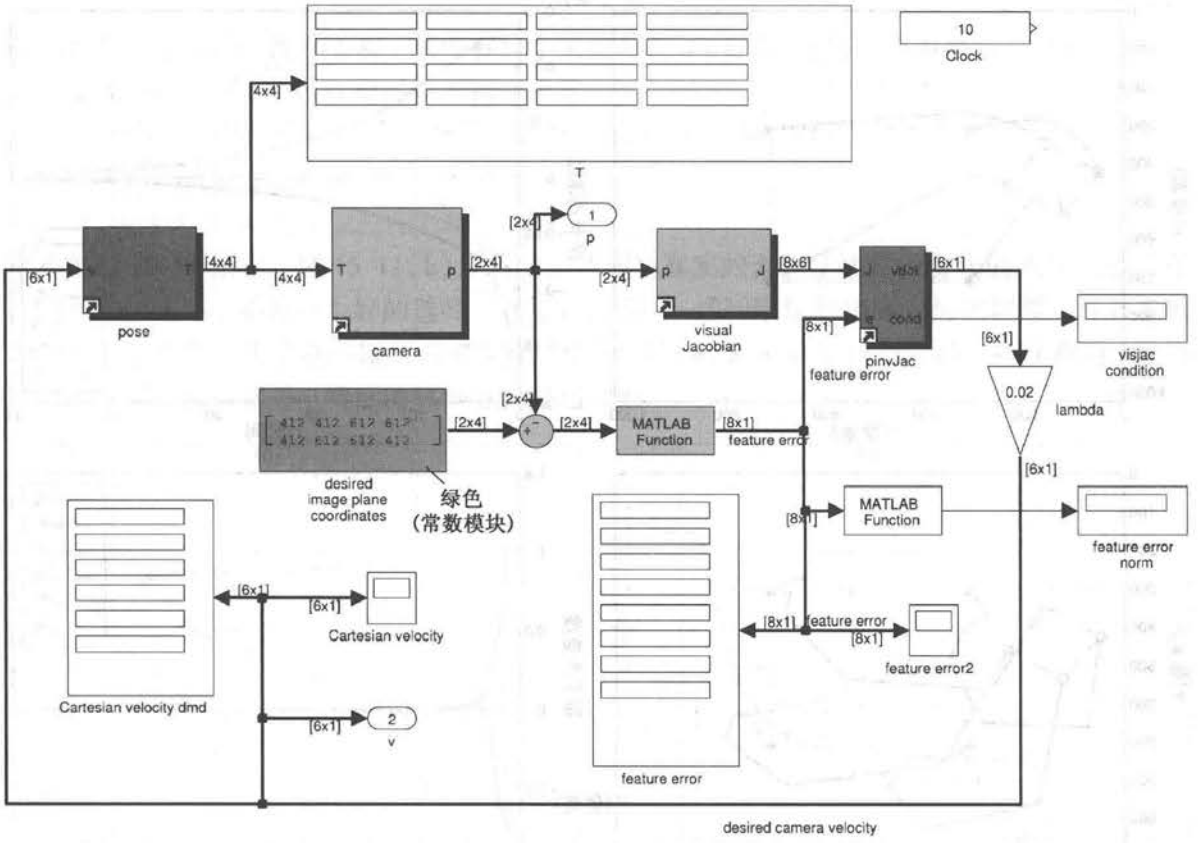


图 15.9　Simulink 模型 sl_ibvs 驱动图像平面上的特征点到期望的位置。最初的相机位姿在pose模块中设置，而期望的图像平面位置$\boldsymbol{p}^*$在绿色的常数模块中设置

15.2.3　深度

计算图像雅克比需要知道相机内在参数、主点位置和焦距，但在实际应用中，对这些参数存在的误差相当宽容。雅克比也需要知道 Z_i，即每个点的深度。在刚刚讨论过的仿真中，我们假设该深度是已知的——这在仿真中很容易，但现实中并非如此。幸运的是，在实际应用中我们发现 IBVS 对于 Z 的误差也很宽容。

学者们已经提出了许多方法来处理深度未知这个问题。最简单的方法就是假定一个恒定不变的深度值，如果所需的相机运动近似地在一个平面中且平行于目标点所在平面，那么这样假设相当合理。为了评估点的深度取不同常数值时的性能，可以对上面的示例分别选择 $z=1$ 和 $z=10$ 来进行比较：

```
>> ibvs = IBVS(cam, 'T0', Tc0, 'pstar', pStar, 'depth', 1)
>> ibvs.run(50)
>> ibvs = IBVS(cam, 'T0', Tc0, 'pstar', pStar, 'depth', 10)
>> ibvs.run(50)
```

结果绘制在图 15.10 中。可以看到，在图像平面上路径不再是直线，因为现在的雅克比矩阵未能很好地近似相机运动和图像特征运动之间的关系。我们也看到，$Z=1$ 时的收敛比 $Z=10$ 时的收敛速度要慢得多。$Z=1$ 时的雅克比高估了光流，所以其逆矩阵就低估了所需的相机速度。然而，对于相当大的误差，真实深度为 $Z=3$，IBVS 收敛了。在 $Z=10$ 的情况下，由于每个时间步的位移都较大，从而导致了一个锯齿状的路径。

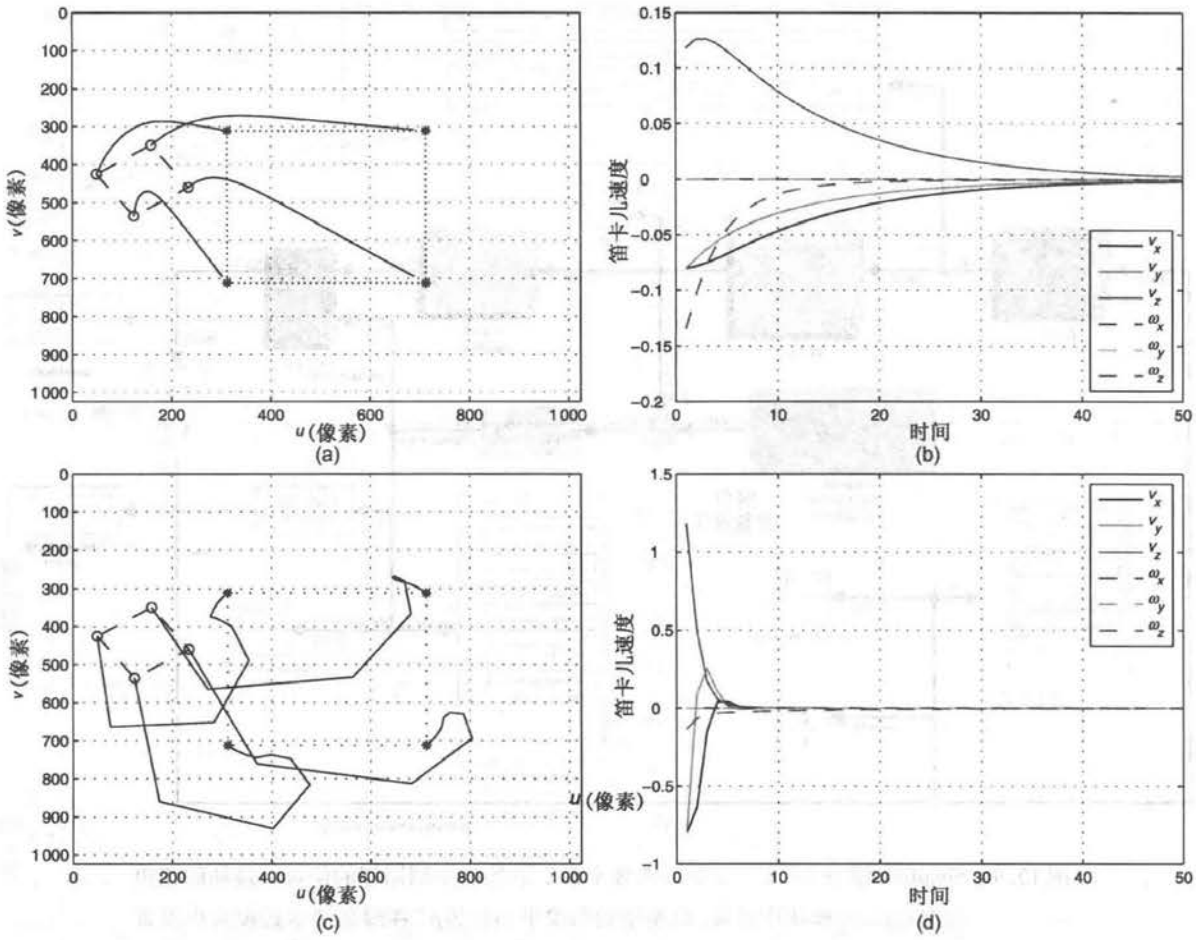


图 15.10 点深度为不同常数时的 IBVS 结果。(a),(b)$Z=1$ 时的图像和相机运动;(c),(d)$Z=10$时的图像和相机运动

第二种方法是使用标准的计算机视觉技术来估算 Z 值。如果相机内在参数已知，则可以使用稀疏立体匹配技术根据连续的相机位置来估计每个特征点的深度。

第三种方法是使用对机器人和图像运动的测量数据来在线估计 Z 值。可以通过对方程(15.6)重新整理来创建一个简单的深度估计器：

$$\begin{pmatrix}\dot{u}\\ \dot{v}\end{pmatrix}=\left(\begin{array}{ccc|ccc}-\frac{f}{\rho_u Z} & 0 & \frac{\bar{u}}{Z} & \frac{\rho_u\bar{u}\bar{v}}{f} & -\frac{f^2+\rho_u^2\bar{u}^2}{\rho_u f} & \bar{v}\\ 0 & -\frac{f}{\rho_v Z} & \frac{\bar{v}}{Z} & \frac{f^2+\rho_v^2\bar{v}^2}{\rho_v f} & -\frac{\rho_v\bar{u}\bar{v}}{f} & -\bar{u}\end{array}\right)\begin{pmatrix}\boldsymbol{v}\\ \omega\end{pmatrix}$$

$$=\left(\frac{1}{Z}J_t \mid J_\omega\right)\begin{pmatrix}\boldsymbol{v}\\ \omega\end{pmatrix}$$

$$=\frac{1}{Z}J_t\boldsymbol{v}+J_\omega\omega$$

将上式重新整理为

$$(J_t\boldsymbol{v})\frac{1}{Z}=\begin{pmatrix}\dot{u}\\ \dot{v}\end{pmatrix}-J_\omega\omega \tag{15.12}$$

该方程的右侧是观察到的光流减去由于相机旋转预计产生的光流——这个过程称为反转光流。减法后，剩余的光流就只是相机平移所致。将方程(15.12)写成紧凑的形式：

$$\mathbf{A}\boldsymbol{\theta} = \mathbf{b} \tag{15.13}$$

我们得到一个简单的线性方程，其中有一个未知参数 $\theta = 1/Z$，它可以使用最小二乘法求解。

在我们的示例中，可以通过以下指令求解：

```
>> ibvs = IBVS(cam, 'T0', Tc0, 'pstar', pStar, 'depthest')
>> ibvs.run()
>> ibvs.plot_z()
>> ibvs.plot_p()
```

结果如图 15.11 所示。图 15.11(b)显示的是估计与真实的点深度随时间变化的曲线。估计深度的初始值为零，不是一个好的选择，但它上升迅速，然后跟踪到实际目标的深度，再后来就能在控制器收敛过程中准确跟踪真实深度。图 15.11(a)显示的是特征运动，可以看到，初始阶段由于深度值有误差，特征的移动方向是错误的。

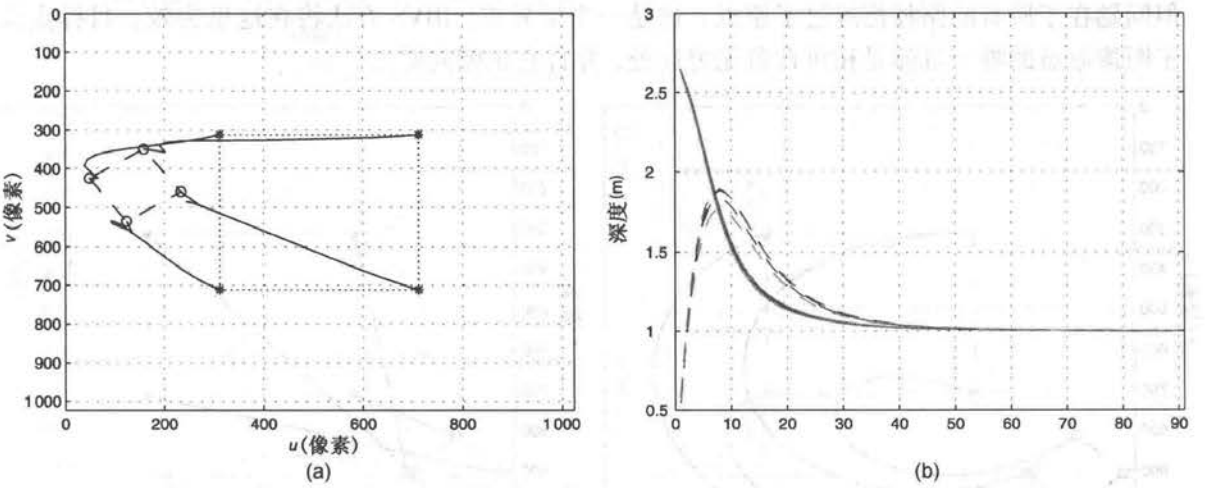


图 15.11　具有在线深度估计器的 IBVS。(a)特征路径；(b)所有 4个点的估计深度(虚线)和真正深度(固体)的比较

15.2.4　控制性能分析

PBVS 的控制律是在三维工作空间中定义的，所以不存在一种能直接调整图像特征运动的机制。对于图 15.5 所示的 PBVS 示例，特征点在图像平面上走过的是一条弯曲路径，因此它们有可能会走出相机视场。我们使用一个不同的初始相机位姿：

```
>> pbvs.T0 = transl(-2.1, 0, -3)*trotz(5*pi/4);
>> pbvs.run()
```

结果如图 15.12(a)所示，我们看到有两个点已经走出了图像边界，这可能会导致 PBVS 控制失败。相比之下，对于相同的初始位姿，IBVS 的控制为

```
>> ibvs = IBVS(cam, 'T0', pbvs.T0, 'pstar', pStar, 'lambda',
0.002, 'niter', Inf, 'eterm', 0.5)
>> ibvs.run()
>> ibvs.plot_p();
```

得出的特征轨迹如图 15.12(b)所示。

相反，基于图像的视觉伺服控制方法没有对相机的笛卡儿运动进行直接控制。有时这会导致令人意外的运动，特别是当目标绕 z 轴旋转时：

```
>> ibvs = IBVS(cam, 'T0', transl(0,0, -1)*trotz(1), 'pstar',
pStar);
>> ibvs.run()
>> ibvs.plot_camera
```

结果如图 15.13 所示(上部)。我们看到，相机进行了不必要的沿 z 轴的移动——先远离目标，然后再返回。这种现象称为相机退化。由此产生的运动不是时间最短的，而且还会要求一些很大的、甚至可能无法实现的相机运动。一个极端的示例是绕光轴纯转动 π 弧度：

```
>> ibvs = IBVS(cam, 'T0', transl(0,0, -1)*trotz(pi), ...
    'pstar', pStar, 'niter', 10);
>> ibvs.run()
>> ibvs.plot_camera
```

结果如图 15.13 所示(下部)。所有特征点与往常一样，朝着它们的期望值在一条直线上移动，但问题在于所有的路径都经过了原点。这是一个奇异点，IBVS 方法将在这里失效。目标点位于图像原点的唯一可能是相机在负无穷远处，并且它正朝向原点！

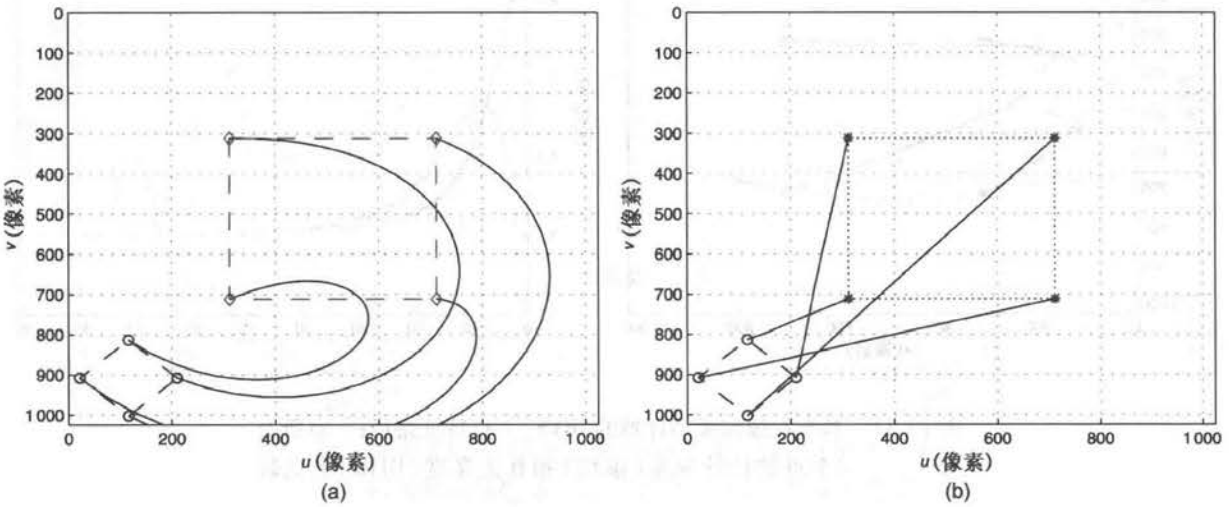


图 15.12 图像平面特征路径，分别采用(a)PBVS 和(b)IBVS

最后需要考虑的是，图像雅克比是对一个高度非线性系统的一种线性化。如果在每个时间步的运动过大，则线性化将失效，特征点也会在图像上走出一条弯曲而不是直线的路径，正如在图 15.10 中所见。如果期望的特征位置离初始位置很远，而且/或者增益 λ 太大，这种情况就可能发生。一种解决方案是限制指令速度的最大范数：

$$\nu = \begin{cases} \nu_{\max}\dfrac{\nu}{|\nu|} & |\nu| > \nu_{\max} \\ \nu & |\nu| \leqslant \nu_{\max} \end{cases}$$

特征路径不一定非要是直线，特征也不一定非要以渐近速度移动——我们在前面这么说只是为了简化问题。使用第 3 章中的轨迹规划方法，特征点可以在图像中沿任意的轨迹运动，而且可以有任意的速度曲线，并具有任意的速度与时间曲线。

综上所述，对于基于视觉的控制 IBVS 是一种鲁棒性非常强的方法。我们已经看到，这种方法对点深度的误差相当宽容。我们还展示了对于绕光轴进行较大旋转的情况，这种方法可以产生稍次于最佳的笛卡儿路径。我们将在下一章讨论对这些问题的补救措施。

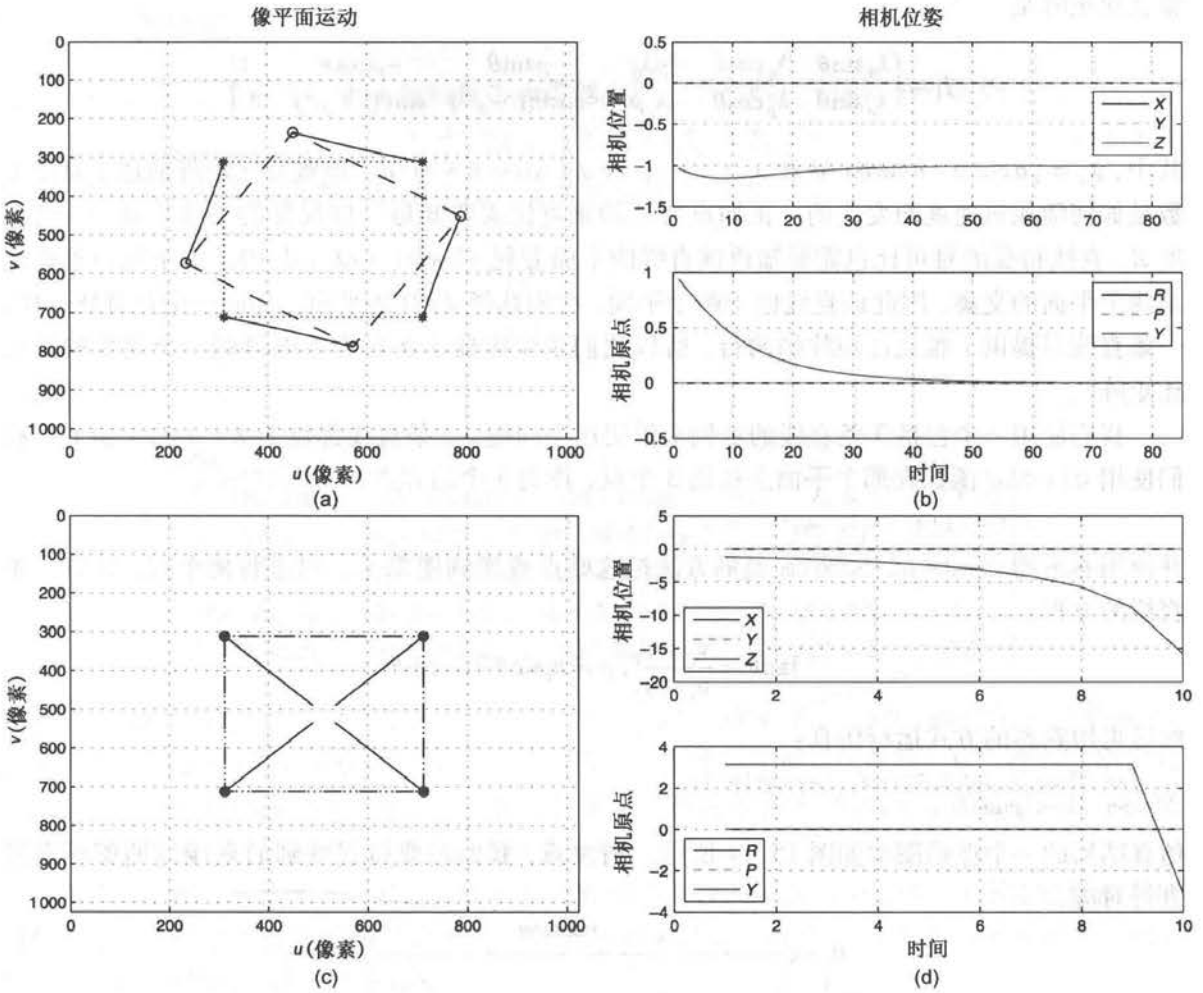


图 15.13　绕光轴做纯旋转的 IBVS。(a)，(b)旋转 1 弧度；(c)，(d)旋转 π 弧度

15.3　使用其他图像特征

到目前为止，我们只考虑了图像中的点特征。在实际的系统中，我们将使用第 13 章中讨论的特征提取技术，而这些点就是特定区域的形心，或是 Harris 或 SURF 角点特征。然后，这些点将在一个 PBVS 控制中用于位姿估计，或直接用在一个 IBVS 控制中。对于 PBVS 或 IBVS，我们需要解决对应问题，也就是说对于每个观察到的特征，必须确定它对应于哪个期望的图像平面坐标。IBVS 也可以与其他图像特征配合工作，如霍夫变换中用的直线，或者一个椭圆形状。

15.3.1　直线特征

对于一条直线，雅克比矩阵可以用(ρ,θ)参数来表示(在 13.2 节的霍夫变换中使用过该参数)：

$$\begin{pmatrix}\dot{\theta}\\ \dot{\rho}\end{pmatrix}=\boldsymbol{J}_l\boldsymbol{\nu}$$

雅克比矩阵是

$$J_L = \begin{pmatrix} \lambda_\theta \sin\theta & \lambda_\theta \cos\theta & -\rho\lambda_\theta & -\rho\sin\theta & -\rho\cos\theta & -1 \\ \lambda_\rho \sin\theta & \lambda_\rho \cos\theta & -\lambda_\rho\rho & -\cos\theta(1+\rho^2) & \sin\theta(1+\rho^2) & 0 \end{pmatrix}$$

其中，$\lambda_\theta = (a\cos\theta - b\sin\theta)/d$ 和 $\lambda_\rho = -(a\rho\sin\theta + b\rho\cos\theta + c)/d$。该雅克比矩阵描述了直线参数是如何随相机速度而变化的。正如点特征的雅克比需要知道一些局部的三维信息——点深度 Z，直线特征的雅可比也需要知道该直线的平面方程 $aX + bY + cZ + d = 0$。由于每一条直线是两个平面的交集，因此该直线位于两个平面，我们选择 $d \neq 0$ 的平面。如同一个点特征一样，一条直线只提供了雅克比矩阵的两行，所以我们至少需要 3 条直线才能得到一个满秩的雅克比矩阵①。

我们使用一个包括 3 条直线的示例来说明这个问题。3 条直线都位于 $Z = 3$ 的平面上。我们使用 `circle` 函数在那个平面上构造 3 个点，作为 3 个边界点：

```
>> P = circle([0 0 3], 0.5, 'n', 3);
```

并使用熟悉的 `CentralCamera` 类的方法把这些点投影到图像上。对于每两个点，计算一条直线的方程：

$$\tan\theta = \frac{v_2 - v_1}{u_1 - u_2},\ \rho = u_1\sin\theta + v_1\cos\theta$$

然后再用熟悉的方式运行仿真：

```
>> ibvs = IBVS_l(cam, 'example');
>> ibvs.run()
```

仿真结果的一个抓拍图像如图 15.14 所示。请注意，我们需要将观察到的直线与期望的直线进行对应。

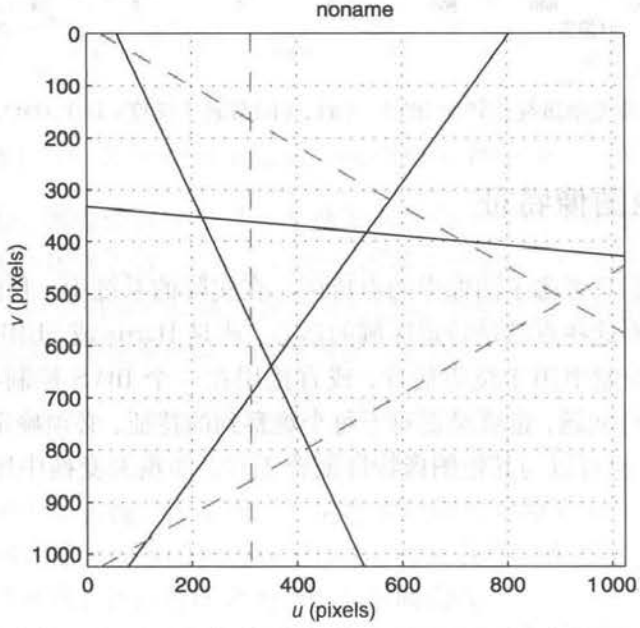


图 15.14 使用直线特征的 IBVS。图像平面中展示了 3 条当前直线（实线）和期望的直线（虚线）

① 有趣的是，一条直线特征只提供了堆叠雅克比矩阵的两行，但定义一条直线的两个点却能提供四行。

15.3.2 圆特征

一般情况下，空间中一个圆会被投影成图像上的一个椭圆，并用以下方程描述[①]：

$$u^2 + E_1 v^2 - 2E_2 uv + 2E_3 u + 2E_4 v + E_5 = 0 \tag{15.14}$$

其中，E_i 是这个椭圆的参数。椭圆参数的变化率与相机速度相关：

$$\begin{pmatrix} \dot{E}_1 \\ \dot{E}_2 \\ \vdots \\ \dot{E}_5 \end{pmatrix} = J_e(\boldsymbol{E}, \rho)\boldsymbol{\nu}$$

雅克比矩阵是

$$J_e(\boldsymbol{E},\rho) = \begin{pmatrix} 2bE_2-2aE_1 & 2E_1(b-aE_2) & 2bE_4-2aE_1E_3 & 2E_4 & 2E_1E_3 & -2E_2(E_1+1) \\ b-aE_2 & bE_2-a(2E_2^2-E_1) & a(E_4-2E_2E_3)+bE_3 & E_3 & 2E_2E_3-E_4 & E_1-2E_2^2-1 \\ c-aE_3 & a(E_4-2E_2E_3)+cE_2 & cE_3-a(2E_3^2-E_5) & -E_2 & 1+2E_3^2-E_5 & E_4-2E_2E_3 \\ E_3b+E_2c-2aE_4 & E_4b+E_1c-2aE_2E_4 & bE_5+cE_4-2aE_3E_4 & E_5-E_1 & 2E_3E_4+E_2 & -2E_2E_4-E_3 \\ 2cE_3-2aE_5 & 2cE_4-2aE_2E_5 & 2cA5-2aE_3E_5 & -2E_4 & 2E_3E_5+2E_3 & -2E_2E_5 \end{pmatrix}$$

其中，$\rho(\alpha,\beta,\gamma)$ 定义了在世界坐标系中的一个平面 $aX + bY + cZ + d = 0$，椭圆位于该平面上，且 $\alpha = -a/d$，$\beta = -b/d$ 和 $\gamma = -c/d$。正如点特征和直线特征雅克比矩阵一样，我们这里也需要提供一些关于目标的深度信息。这个雅克比矩阵的秩通常为5，但如果投影是中心位于图像平面上的一个圆，那么秩就降为3，如果圆变成一个点，则秩是2。

椭圆特征的一个优点是，可以根据所有边界点的集合来计算这个椭圆，而不需要解决对应问题。椭圆特征也可以根据椭圆边界内所有点的矩来计算。我们用一个示例来演示，示例采用了一个圆，圆中包括了10个点：

```
>> P = circle([0 0 3], 0.5, 'n', 10);
```

把这些点用 CentralCamera 类投影到像平面上：

```
>> p = cam.project(P, 'Tcam', Tc);
```

其中，Tc 是当前相机的位姿，我们把它转换为归一化图像坐标：

```
>> p = homtrans( inv(cam.K), p );
```

椭圆的参数使用附录 E 的方法计算：

```
>> a = [y.^2; -2*x.*y; 2*x; 2*y; ones(1,numcols(x))]';
>> b = -(x.^2)';
>> E = a\b;
```

它返回一个椭圆参数的五维向量。一个椭圆特征的图像雅克比可由 CentralCamera 类的一个方法计算：

```
>> J = cam.visjac_e(E, plane);
```

其中，必须指定那个圆所在的平面。对于这个示例，平面是 $Z=3$，所以平面为 `plane = [0 0 1 -3]`。

这个雅克比是一个5×6矩阵，其最大的秩只有5，所以不能从它得到相机速度的唯一解。我

① 这与附录E中给出的椭圆表达式不同，但两种形式只是在系数上相差一个不变的比例因子。

们至少有两个选项来解决该问题。首先，如果最终的视图是一个圆，那么可以不用关心绕圆心的旋转，在这种情况下，可以将雅克比矩阵的第六列删除，把它变成一个方阵，并设定 ω_z 为零。其次，也是本示例所采取的做法，是将椭圆特征和一个单点特征进行组合[①]：

$$\begin{pmatrix}\dot{E}_1\\ \dot{E}_2\\ \vdots\\ \dot{E}_5\\ \dot{u}_1\\ \dot{v}_1\end{pmatrix}=\begin{pmatrix}J_e(\boldsymbol{E})\\ J_p(\boldsymbol{p}_1)\end{pmatrix}\boldsymbol{\nu}$$

堆叠后的雅克比矩阵现在是 7×6，可以用它求解出相机速度。和上一个 IBVS 示例相同，期望的速度与当前特征值和期望特征值之差成正比：

$$\begin{pmatrix}\dot{E}_1\\ \dot{E}_2\\ \vdots\\ \dot{E}_5\\ \dot{u}_1\\ \dot{v}_1\end{pmatrix}=\lambda\begin{pmatrix}E_1^*-E_1\\ E_2^*-E_2\\ \vdots\\ E_5^*-E_5\\ u_1^*-u_1\\ v_1^*-v_1\end{pmatrix}$$

现在以熟悉的方式运行仿真：

```
>> ibvs = IBVS_e(cam, 'example');
>> ibvs.run()
```

仿真结果的一个抓拍图像如图 15.15 所示。

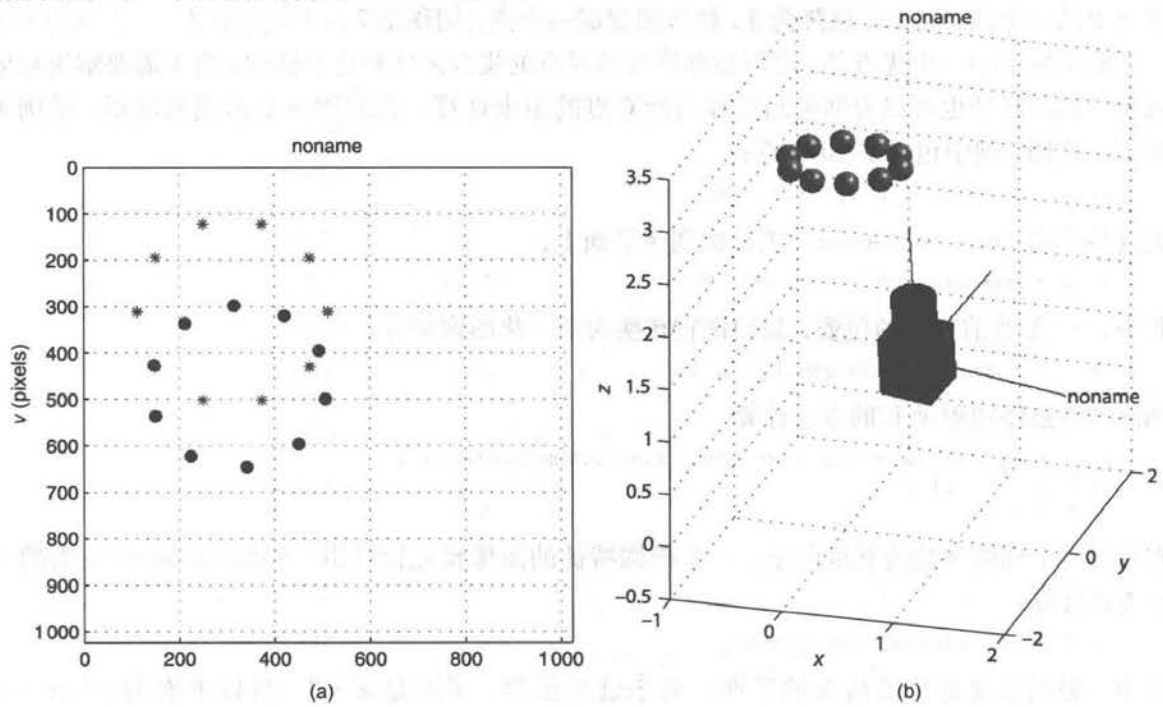


图 15.15　使用椭圆特征的 IBVS。(a)显示了当前点(实点)和要求点(*)的图像平面；(b)显示了点和相机的世界视图

① 在这里，我们选择了第一点。选择任何点都可以。

15.4 本章总结

在这一章中，我们学习了基于视觉的机器人控制基本知识，以及到20世纪90年代中期20多年的时间发展起来的相关基本技术。相机有两种不同的配置方式，可以被安装到机器人上以观察目标，即手眼，或者固定在周围环境中同时观察机器人和目标。另一种区分方式是控制结构：基于位置的视觉伺服(PBVS)和基于图像的视觉伺服(IBVS)。前者涉及基于一个已标定相机和一个目标几何模型的位姿估计，而后者在图像平面上直接进行控制。每种方法都有一定的优点和缺点。PBVS能在世界坐标系中高效执行直线的笛卡儿相机运动，但可能会导致图像特征离开图像平面。IBVS能始终把特征保留在图像平面中，但可能会产生超过机器人运动能力的轨迹，特别是如果它需要一个绕相机光轴的大旋转量。IBVS还需要一部分三维信息，如特征点深度，但它对深度的误差有较强的鲁棒性，而且可以在机器人移动过程中估计深度。IBVS不仅可以与点特征配合工作，同时也可以同直线和椭圆特征一起配合。

到目前为止，我们仅在仿真中确定了需要的相机速度，并使相机相应地移动，而未考虑驱动它的机构。在下一章中，我们将考虑把相机安装到机械臂上、地面移动机器人上以及飞行机器人上。

扩展阅读

[Hutchinson et al., 1996]是第一本对该领域进行全面论述和分类的教程，而更多最近的文献包括[Chaumette and Hutchinson, 2006]和[Siciliano and Khatib, 2008, § 24]，它们对视觉伺服的基础知识进行了精彩的论述。最近出版的教材[Spong et al., 2006, § 12)]和[Siciliano et al., 2008, § 10]中也有视觉伺服的章节。

1993年编辑出版的[Hashimoto, 1993]是最早关于视觉伺服方法和应用的论文合集。1996年出版的[Corke, 1996]现在已经绝版了，但可以从网上免费获得，它涵盖了机器人学和视觉的基础知识，可用于控制基于图像的视觉伺服系统。它还包含了一个广泛但也较陈旧的有关视觉伺服应用的参考文献集，那些应用包括工业应用、用于跟踪的相机控制、高速平面微机械臂、道路车辆引导、飞机加油以及水果采摘。另一个重要的论文集是[Kriegman et al., 1998]，它出自于1998年召开的一个关于控制和视觉之间协同的研讨会：视觉如何被用于控制以及控制如何被用于视觉。[Chesi and Hashimoto, 2010]是最近出版的一本研讨会论文集，它涵盖了视觉算法的开发及应用。

视觉伺服有一个很长的历史——最早的参考文献是[Shirai and Inoue, 1973]，它描述了如何使用视觉反馈回路来矫正机器人的位置以提高任务精度。该文献中展示了一个伺服循环时间为10秒的系统，它突显了当时该领域所面临的一个严酷现实，即特征的实时提取问题。直到20世纪90年代末，要想实现较快的伺服控制仍然需要笨重和昂贵的专用硬件，如图15.16所示。其他早期在工业应用上的重大工作主要由SRI国际公司在20世纪70年代后期开展[Hill and Park, 1979], [Makhlin, 1985]。

在20世纪80年代，[Weiss et al., 1987]中介绍了视觉伺服结构的分类，即基于位置的视觉伺服或基于图像的视觉伺服。该文献中还介绍了视觉伺服与动态看而动之间的区别，前者只使用视觉反馈，而后者则采用了关节反馈和视觉反馈。后者的这个特点现在已被普遍采用，今天大多数的视觉伺服系统都采用了基于关节位置和视觉的反馈。在[Weiss, 1984]中将自适

应控制技术应用于 IBVS 来控制一个没有关节反馈的机器人手臂，但它仅适合于少自由度的机械臂，因为当时视觉处理的采样频率较低。其他学者则研究了把机械臂动力学方程(9.1)在置入控制器中，来直接控制电机力矩[Kelly, 1996][Kelly et al., 2002a,b]，但所有这些控制器仍然需要关节的角度以求解机械臂的雅克比矩阵，还需要关节速度来提供阻尼。闭环视觉控制系统的控制和稳定性也被许多学者探索过[Corke and Good, 1992]，[Espiau et al., 1992][papanikolopoulos et al.,1993]。基于前馈预测，而不是反馈的控制器则在[Corke, 1994]和[Corke and Good, 1996]中提出。

费德马[Feddema and Mitchell, 1989]，[Feddema, 1989]采用了闭环关节控制，以克服由于视觉采样频率低产生的问题，并展示了用于四自由度机械臂的 IBVS。在[Chaumette et al., 1991]和[Rives et al., 1989]中则使用任务函数法[Samson et al., 1990]描述了一种相似的方法，并使用一个四特征的目标展示了机器人定位的实验结果。在[Feddema et al., 1991]中描述了一种算法，它可以选择哪个特征子集可以得到最佳条件数的雅克比矩阵。[Hashimoto et al., 1991]中则展示了使用较大数量的特征以及使用广义逆求解速度的优势。

众所周知，IBVS 对于深度误差非常宽容，在[Marey and Chaumette, 2008]中详细研究深度误差对控制性能的影响。[Feddema and Mitchell, 1989]中进行了部分的三维重建，根据观察到的特征和已知目标几何来确定点深度。[Papanikolopoulo and Khosla, 1993]则描述了自适应控制技术来估计深度，就如同在本章中使用的那样。[Hosoda and Asada, 1994]，[Jägersand et al., 1996]和[Piepmeier et al., 1999]中展示了如何根据对机器人和图像运动的测量来在线估计图像雅克比矩阵。

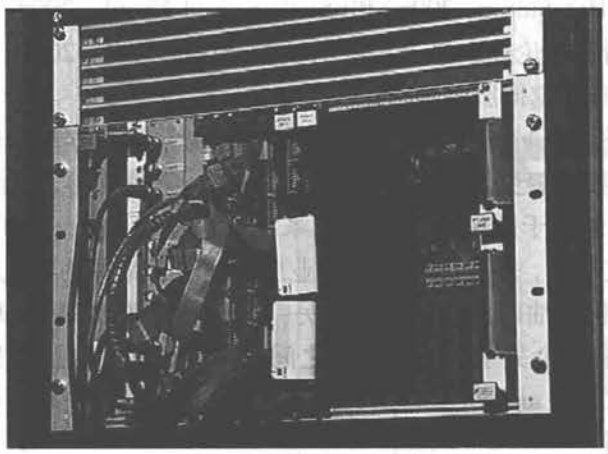


图 15.16 一个 19 英寸的 VME 总线机箱，内装图像处理卡，它捕获 512×512 图像的能力或帧率为每秒10 Mpix。本书作者大约在20世纪90年代初使用它

图像雅克比矩阵通常是基于图像中的运动点，得到但它也可以根据图像平面中直线的参数[Chaumette, 1990]，[Espiau et al.,1992]，以及图像平面中一个椭圆的参数[Espiau et al., 1992]推导得出。最近，已有学者提出把矩的概念用于平面场景的视觉伺服[Chaumette, 2004]，[Tahri and Chaumette, 2005]。

关于 PBVS 的文献要少得多，但[Westmore and Wilson, 1991]是一篇关于 PBVS 的很好论文。作者使用了一个扩展卡尔曼滤波器来隐式地进行位姿估计，目标位姿就是过滤器状态，再把预测值与特征坐标之间的革新用于更新目标位姿状态。[Hashimoto et al., 1991]中呈现了一个仿真，用于比较基于位置和基于图像的方法。

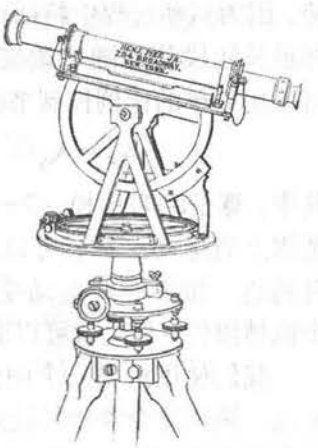
视觉伺服已被应用到各种各样的通常需要人类手眼技能的问题，如打乒乓球[Andersson, 1989]、杂技抛球[Rizzi and Koditschek, 1991]和倒立平衡[Dickmanns and Graefe, 1988a]，[Andersen et al.,1993]、抓取[Sakaguchi et al.,1993]，[Buttazzo et al.,1993]，[Bukowskiet et al.,1991]，[Skofteland and Hirzinger, 1991]，[Skaar et al.,1987]，[Lin et al.,1989]、以及控制一个迷宫游戏[Andersen et al., 1993]。

习题

1. 基于位置的视觉伺服。
   a) 运行 PBVS 的示例。用不同的参数进行实验，如初始相机位姿，路径分割比 λ，并增加像素噪声给相机的输出。
   b) 为 PBVS 创建一个 Simulink 模型。
   c) 使用一个不同的相机模型进行位姿估计(略有不同的焦距或主点)，并观察它们对最终末端执行器位姿的影响。
   d) 基于 PBVS 系统实现一个 EKF，参考[Westmore and Wilson, 1991]中的描述。
2. 光流场。
   a) 针对不同焦距的相机，绘制出光流场图。
   b) 针对某些复合相机运动，如 x 轴加 y 轴的平移、x 轴加 z 轴的平移、x 的平移加绕 z 轴旋转，绘制出光流场图。
3. 对于两个特征点的情况，图像雅克比是 4×6 矩阵，其中零空间有两列。它们所对应的相机运动是什么?
4. 基于图像的视觉伺服。
   a) 运行 IBVS 的示例，通过命令行或 Simulink 版本。采用不同的增益 λ 进行实验。记住 λ 可以是一个标量，也可以是一个允许为每个自由度设置不同增益的对角矩阵。
   b) 实现能够限制指令速度最大范数的功能。
   c) 通过添加像素噪声给相机输出进行实验。
   d) 用不同的初始相机位姿和期望像平面坐标进行实验。
   e) 用不同数量的目标点进行实验，数量从 3 个到 10 个。在 $N>3$ 的情况下，对使用广义逆的性能与只选择三个点(前三个或随机三个)的性能进行比较。请尝试设计一个算法，该算法能够自动挑选三个点，使堆叠的雅克比矩阵具有最佳条件数。
   f) 创建一组期望的图像平面点，它们形成一个矩形，而不是一个正方形。不存在透视点使得一个正方形看起来是一个矩形。IBVS 系统该做什么?
   g) 创建一组不可能达到的期望图像平面点，例如把两个相邻的空间点和图像点调换。IBVS 系统该做什么?
   h) 使用一个不同的相机模型得到图像雅克比矩阵(略有不同的焦距或主点)，并观察它们对最终末端执行器位姿的影响。
   i) 对于 IBVS 我们一般强制点沿直线移动，但这仅仅是为了方便。使用一个轨迹发生器，使这些点从初始到期望位置运动过程中有一些横向运动，如一个半周期或全周期的正弦波。它们对相机笛卡儿运动有什么影响?
5. 推导一个全景相机的图像雅克比矩阵。

6. 当讨论运动可感知性时，我们使用了$(\boldsymbol{J}_p^+)^{\mathrm{T}}\boldsymbol{J}_p^+=(\boldsymbol{J}_p\boldsymbol{J}_p^{\mathrm{T}})^{-1}$。请尝试证明这个等式。提示：使用奇异值分解$\boldsymbol{J}=\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\mathrm{T}}$，并记住$\boldsymbol{U}$和$\boldsymbol{V}$是正交矩阵。
7. 端点开环视觉伺服系统尚未在本书中讨论。考虑在机器人末端执行器上的一组目标点，以及在被操作对象上的一组目标点，它们都被一台相机观察(有挑战性)。
   a)创建一个端点开环的 PBVS 系统。
   b)使用一个不同的相机模型进行位姿估计(略有不同的焦距或主点)，并观察它们对最终末端执行器位姿的影响。
   c)创建一个端点开环的 IBVS 系统。
   d)使用一个不同的相机模型建立图像雅克比矩阵(略有不同的焦距或主点)，并观察它们对最终末端执行器位姿的影响。
8. 运行基于直线的视觉伺服示例。
9. 基于椭圆的视觉伺服。
   a)运行基于椭圆的视觉伺服示例。
   b)修改以上示例，仅使用椭圆的参数(没有点特征)来伺服五个自由度的相机运动。
   c)对于任意的形状，我们可以计算它的用一个惯性矩阵和一个形心表示的等价椭圆。根据惯性矩阵和形心确定方程(15.14)中的椭圆参数。创建一个椭圆特征的视觉伺服，把一个任意形状移动到期望的位姿(具有挑战性)。

第16章　高级视觉伺服



本章以上一章内容为基础，介绍了一些高级的视觉伺服技术和应用。16.1 节中介绍了混合视觉伺服方法，它避免了之前描述的 IBVS 和 PBVS 中的一些限制因素。

广角相机，如鱼眼镜头和反射折射照相机，对于视觉伺服应用有很大的优势。16.2 节中展示了如何使用极坐标代替笛卡儿像平面坐标系来重新构建 IBVS。这种新的 IBVS 与鱼眼镜头直接相关，但同时它也能改进使用透视相机时的旋转控制。11.4 节中统一成像模型允许大多数的相机（透视、鱼眼和反射折射的）用球面映射模型来表示，16.3 节阐述了如何使用球面坐标来重构 IBVS。

剩余各节中给出了很多应用的实例。这些实例展示了视觉伺服如何应用于不同类型的相机（透视和球面）和不同类型的机器人（臂式机器人、地面移动机器人和飞行机器人）。16.4 节考虑了让一个 6 自由度的机器人机械臂来操作相机。16.5 节探讨了让一个地面移动机器人运动到一个特定位姿，该功能可以用于引导机器人穿越门口或入坞。最后，16.6 节介绍了一个四旋翼飞行器的视觉伺服，使它以固定的位姿相对于地面上的目标盘旋。

16.1　*XY/Z* 分割的 IBVS

在上一章中，我们在一个 IBVS 系统中遇到了相机退化的问题。这种现象可以依靠直觉解释为：我们的 IBVS 控制律要让特征点在图像平面上沿直线移动，但是对于一个旋转的相机，这些点将会自然地沿着圆弧移动。线性 IBVS 控制器会动态地改变总体图像尺度，以至于沿着圆弧的运动表现为沿着直线的运动。尺度变化通过沿 z 轴的平移来实现。

分割方法消除相机退化的原理是：使用 IBVS 控制一些自由度，而对于剩下的自由度使用另外的控制器。*XY/Z* 混合方案是把 x、y 轴看作一组，而把 z 轴看作另一组。这种方法基于如下认识。首先，直观来看，相机退化问题是一种 z 轴现象：绕 z 轴的旋转导致了不需要的沿 z 轴的移动。其次，从图 15.7 看，关于 x、y 轴的平移与旋转而产生的图像平面运动是非常相似的，相反，关于 z 轴的旋转和平移产生的光流却完全不同。

我们分割方程（15.7）的特征点光流，得到

$$\dot{\boldsymbol{p}} = \boldsymbol{J}_{xy}\boldsymbol{\nu}_{xy} + \boldsymbol{J}_z\boldsymbol{\nu}_z \tag{16.1}$$

其中，$\boldsymbol{\nu}_{xy} = (v_x, v_y, \omega_x, \omega_y)$，$\boldsymbol{\nu}_z = (v_z, \omega_z)$，且 $\boldsymbol{J}_{xy}$、$\boldsymbol{J}_z$ 分别是 $\boldsymbol{J}_p$ 的列$\{1, 2, 4, 5\}$和列$\{3, 6\}$。由于 $\boldsymbol{\nu}_z$ 要通过不同的控制器进行计算，所以可以把方程（16.1）写成

$$\boldsymbol{\nu}_{xy} = \lambda \boldsymbol{J}_{xy}^{+}\left(\dot{\boldsymbol{p}}^{*} - \boldsymbol{J}_z\boldsymbol{\nu}_z\right) \tag{16.2}$$

其中，$\dot{\boldsymbol{p}}^{*}$ 是期望的特征点速度，与方程（15.10）中传统的 IBVS 方案相同。

z 轴速度 v_z 和 ω_z 是根据两个附加图像特征 A 和 θ 直接计算出来的，如图 16.1 所示。第一个图像特征 $\theta \in [0, 2\pi)$，是 u 轴与连接特征点 i,j 的有向线段之间的夹角。对于数值调节，选择由特征点构造的最长线段具有优势，因为运动过程中特征点配置发生改变，这样最长线段就能随之改变。期望的转动速度可以用简单的比例控制来获得：

$$\omega_z^* = \lambda_{\omega_z}(\theta^* \ominus \theta)$$

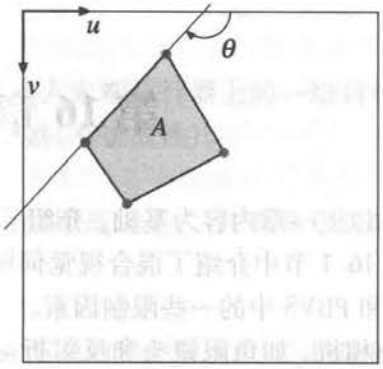


图 16.1 XY/Z 分割的 IBVS 控制图像特征。除了4个点的坐标外，还使用了多边形面积A以及最长线段的角度θ

其中，算子$\ominus$表示模 -2π 的减法。和通常在圆周上的运动一样，可以从两个方向移动到目的地。如果旋转运动受到限制，例如有一个机械限位，那么就可以通过选择 ω_z 的符号来避免遇到这个限位。

我们使用的第二个图像特征是规则多边形面积 A 的一个函数，该多边形的顶点是图像特征点。采用这个参数的优点是：它是一个标量；它具有旋转不变性①，因此可以从 z 轴平移中分离出相机旋转；它的计算量小。多边形面仅仅是一个零次矩 m_{00}，它可以使用工具箱函数 `mpq_poly(p, 0, 0)`来计算。用于控制的特征是面积的二次根：

$$\sigma = \sqrt{m_{00}}$$

它具有长度单位，以像素计量。期望的相机沿 z 轴平移速度可以使用简单的比例控制律来获得：

$$v_z^* = \lambda_{v_z}(\sigma^* - \sigma) \tag{16.3}$$

上文讨论的用于 z 轴平移和旋转控制的特征非常简单，计算量也小，但要想获得最佳控制效果，目标法线与相机光轴之间的夹角还应当在 $\pm 40°$ 范围内。当目标平面与光轴不正交时，它的面积将会由于透视而减少，这会使相机开始靠近目标。透视也会改变感知到的线段角度，这可能会造成较小的，但是不必需的 z 轴旋转运动。

上述控制策略的 Simulink 模型为

```
>> sl_partitioned
```

如图 16.2 所示。相机最初的位姿由 pose 模块的参数来设定。仿真通过以下指令运行：

```
>> sim('sl_partitioned')
```

可以显示出相机位姿、像平面特征误差和相机速度的动画。示波器模块还可以绘制出相对于时间的相机速度和特征误差曲线。

如果特征点朝着视野的边缘移动，那么让它们不出视野的最简单方法是把相机朝远离场景的方向移动。我们定义一个作用在相机上的排斥力，当特征点靠近图像平面边缘时它会把点推开：

$$F_z(\boldsymbol{p}) = \begin{cases} \left(\dfrac{1}{d(\boldsymbol{p})} - \dfrac{1}{d_0}\right)\dfrac{1}{d^2(\boldsymbol{p})}, & d(\boldsymbol{p}) \leqslant d_0 \\ 0, & d(\boldsymbol{p}) > d_0 \end{cases}$$

其中，$d(\boldsymbol{p})$ 是从图像点 $\boldsymbol{p}$ 到图像平面边缘的最短距离，d_0 是有排斥力作用的图像区域的宽度，

① 该旋转不变性是针对绕 z 轴的旋转，而不是绕 x 和 y 轴的旋转。

η 是一个标量增益系数。对于一幅 $W \times H$ 的图像，有

$$d(\boldsymbol{p}) = \min\{u, v, W-u, H-v\} \tag{16.4}$$

把排斥力嵌入到 z 轴平移控制器中：

$$v_z^* = \lambda_{v_z}(\sigma^* - \sigma) - \eta \sum_{i=1}^{N} F_z(\boldsymbol{p}_i)$$

其中，η 是一个阻尼单位的增益常数。排斥力是不连续的，这可能会引发颤振，即特征点反复地进出有排斥力的区域——这可以通过引入平滑过滤器和速度限制来进行补偿。

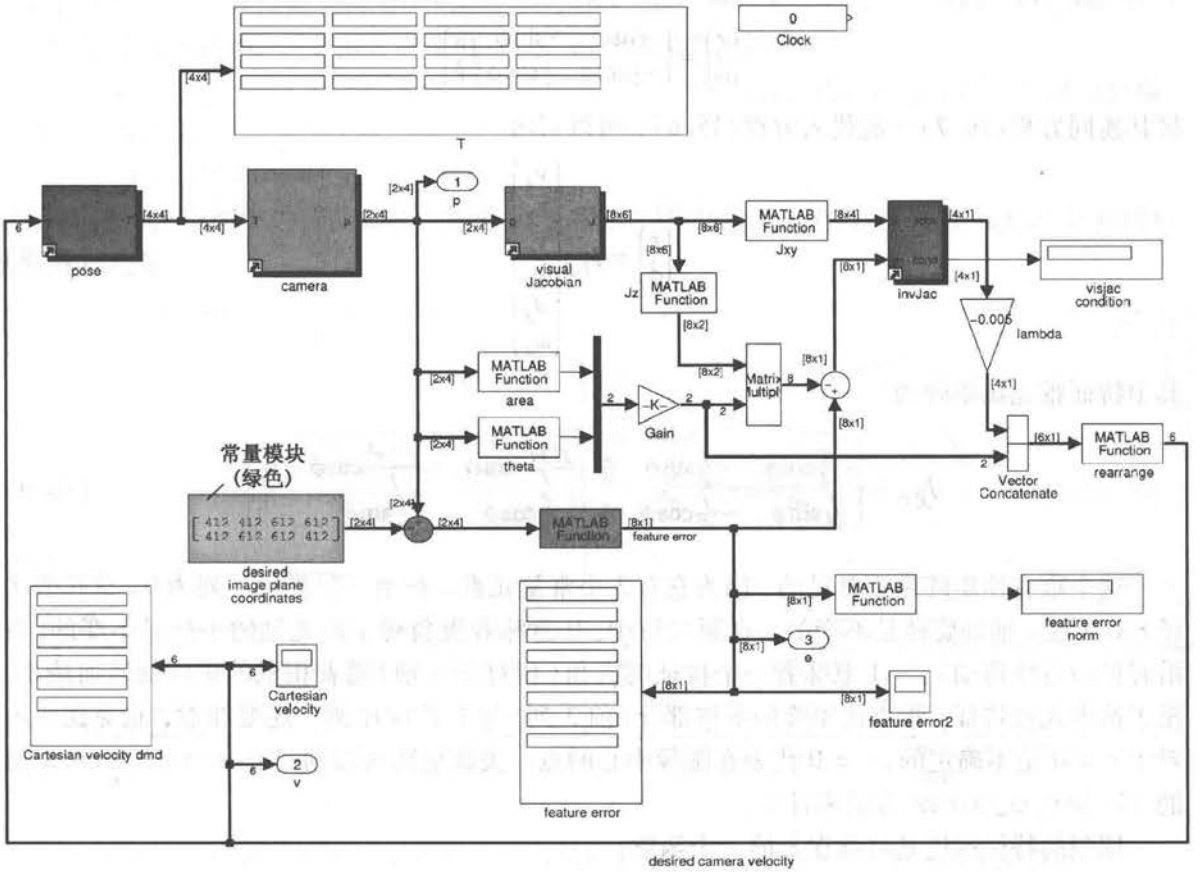


图 16.2 Simulink® 模型 sl_partitioned 是一个 *XY/Z* 分割视觉伺服策略，它是图 15.9 所示的 IBVS 系统的一种扩展。初始相机位姿在pose模块中设定，期望的图像平面点 $\boldsymbol{p}^*$ 在绿色的常量模块中设定

16.2 使用极坐标的 IBVS

在 15.3 节中展示了对于非点特征的图像特征雅克比矩阵，但是在这里我们将要展示在一个不同坐标系中的点特征雅克比矩阵。在极坐标中，图像点表示为 $\boldsymbol{p} = (r, \phi)$，其中 r 是从主点到该点的距离：

$$r = \sqrt{\bar{u}^2 + \bar{v}^2} \tag{16.5}$$

其中，$\bar{u}$ 和 $\bar{v}$ 是相对于主点而不是图像原点的图像坐标。u 轴与连接主点和图像点的直线的夹角是

$$\phi = \arctan\frac{\bar{v}}{\bar{u}} \tag{16.6}$$

这两个坐标表示方法的关联是

$$\bar{u} = r\cos\phi, \bar{v} = r\sin\phi \tag{16.7}$$

将上式对时间求导数，得

$$\begin{pmatrix}\dot{\bar{u}}\\ \dot{\bar{v}}\end{pmatrix} = \begin{pmatrix}\cos\phi & -r\sin\phi\\ \sin\phi & r\cos\phi\end{pmatrix}\begin{pmatrix}\dot{r}\\ \dot{\phi}\end{pmatrix}$$

然后求逆，得

$$\begin{pmatrix}\dot{r}\\ \dot{\phi}\end{pmatrix} = \begin{pmatrix}\cos\phi & \sin\phi\\ -\frac{1}{r}\sin\phi & \frac{1}{r}\cos\phi\end{pmatrix}\begin{pmatrix}\dot{\bar{u}}\\ \dot{\bar{v}}\end{pmatrix}$$

将其连同方程(16.7)一起代入方程(15.6)，可以写出

$$\begin{pmatrix}\dot{r}\\ \dot{\phi}\end{pmatrix} = \boldsymbol{J}_{p,p}\begin{pmatrix}v_x\\ v_y\\ v_z\\ \omega_x\\ \omega_y\\ \omega_z\end{pmatrix} \tag{16.8}$$

其中特征雅克比矩阵为

$$\boldsymbol{J}_{p,p} = \begin{pmatrix}-\frac{f}{Z}\cos\phi & -\frac{f}{Z}\sin\phi & \frac{r}{Z} & \frac{f^2+r^2}{f}\sin\phi & -\frac{f^2+r^2}{f}\cos\phi & 0\\ \frac{f}{rZ}\sin\phi & -\frac{f}{rZ}\cos\phi & 0 & \frac{f}{r}\cos\phi & \frac{f}{r}\sin\phi & -1\end{pmatrix} \tag{16.9}$$

这个雅克比矩阵是不常见的，因为它有 3 个常量元素。在第一行最后一列为 0，意味着半径 r 对于绕 z 轴的旋转是不变的。在第二行中，0 意味着极角对于沿光轴的平移是不变的(点沿着径向直线移动)，-1 意味着一个特征的夹角(相对于 u 轴)随着相机的正向旋转而减小。至于笛卡儿点特征，雅克比矩阵的平移部分(前 3 列)与 $1/Z$ 成比例。还要注意，雅克比矩阵对于 $r = 0$ 是不确定的，$r = 0$ 代表在图像中心的点。关联矩阵可以通过 `CentralCamera` 类的 `visjac_p_polar` 方法来计算。

期望的特征速度是特征误差的一个函数：

$$\dot{\boldsymbol{p}}^* = \lambda\begin{pmatrix}r^* - r\\ \phi^* \ominus \phi\end{pmatrix}$$

其中，$\ominus$是用于角度运算的模 -2π 减法，它是通过工具箱函数 `angdiff` 执行的。注意，$|r| \gg |\phi|$，并进行归一化：

$$r = \frac{\sqrt{\bar{u}^2 + \bar{v}^2}}{\sqrt{W^2 + H^2}}$$

这样 r 和 ϕ 的数量级就近似相同了。

使用极坐标的一个 IBVS 示例是通过 `IBVS_polar` 类执行的。我们首先创建一个典型相机[①]，具有归一化的图像坐标：

① 默认的相机参数导致大的图像坐标值，从而增大了 r 值。另一个特征坐标是一个角度 $\phi \in [-\pi, \pi]$，它的值较小，这会在求解相机速度时导致数值问题。

```
>> cam = CentralCamera('default')
>> Tc0 = transl(0, 0, -2)*trotz(1);
>> vs = IBVS_polar(cam, 'T0', Tc0, 'verbose')
```

然后运行仿真：

```
>> vs.run()
```

动画显示了在图像中的特征运动，以及在一个三维空间视图中相机和空间点的运动。与前一章介绍的笛卡儿 IBVS 策略相比，相机运动已有很大不同。对于以前讨论中存在问题的绕光轴有较大旋转的情况，现在的相机已经能朝着目标移动并旋转了。特征点也能在 $r\phi$ 平面上跟随一条直线路径。可以说极坐标 IBVS 是笛卡儿 IBVS 的一种补充——对于大旋转情况，它会产生比较好的相机运动，但是对于大平移情况，相机运动会比较差。

方法 `plot_error`、`plot_vel` 和 `plot_camera` 可以用于显示在仿真期间记录的数据。另外一种方法是

```
>> vs.plot_features()
```

它显示了在 ϕr 空间中的特征路径，如图 16.3 所示。图中还显示了相机运动，我们没有看到相机退化的迹象。

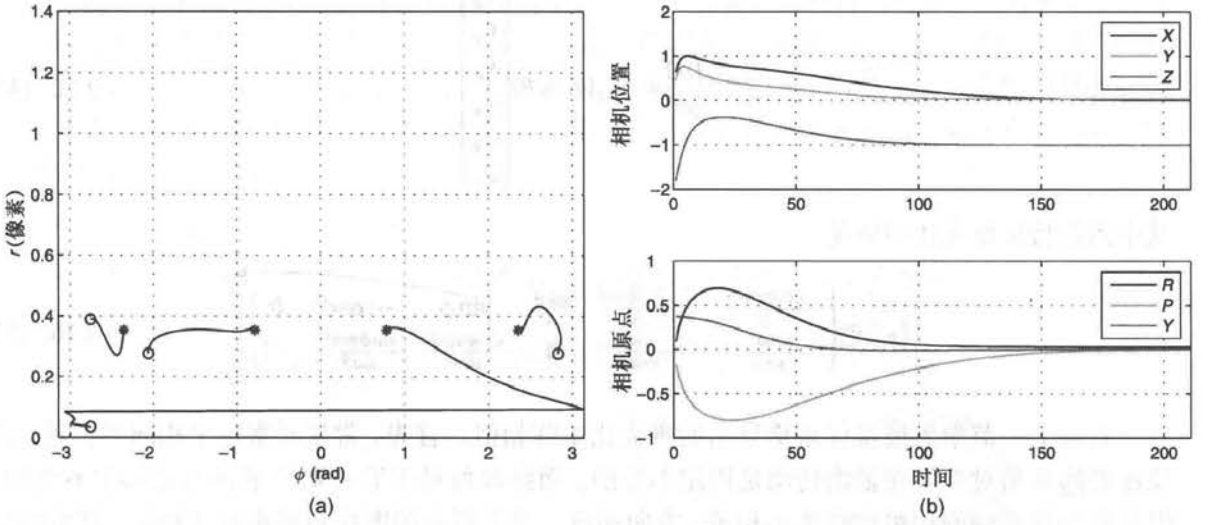


图 16.3　使用极坐标表示的 IBVS。(a) ϕr 空间中的特征运动；(b)笛卡儿空间的相机运动

16.3　对一个球面相机的 IBVS

11.3 节中，我们了解了非透视相机，例如鱼眼镜头相机和反射折射相机。给定它们特殊的投影方程，则可以根据第一原理推导出一个图像特征的雅克比矩阵。然而，有多少不同的透镜和反射镜形状，就会有多少不同的投影模型和图像雅克比矩阵。另一种方法，我们可以把来自任何类型相机的特征投影到球面上(见 11.5.1 节)，然后推导出在球面上用于视觉伺服控制的图像雅克比矩阵。

用于球面的图像雅克比矩阵可以用类似于 15.2 节中透视相机的方法推导出来。参照图 11.20，空间点 $\mathbf{P}$ 由在相机坐标系中的向量 $\boldsymbol{P} = (X,Y,Z)$ 表示出来，并且由一条穿过球面中心的射线把它投影到球体表面的点 $\boldsymbol{p} = (x,y,z)$：

$$x = \frac{X}{R},\ y = \frac{Y}{R},\ z = \frac{Z}{R} \tag{16.10}$$

其中，$R = \sqrt{(X^2 + Y^2 + Z^2)}$ 是从相机原点到空间点的距离。

球面约束 $x^2 + y^2 + z^2 = 1$ 意味着笛卡儿坐标中有一个是多余的，所以我们将会使用一个最小球面坐标系，它包含了纬度角：

$$\theta = \arcsin r,\ \theta \in [0, \pi] \tag{16.11}$$

其中，$r = \sqrt{x^2 + y^2}$。也包含了方位角(或经度角)：

$$\phi = \arctan\frac{y}{x},\ \phi \in [-\pi, \pi) \tag{16.12}$$

它们共同确定点的特征向量 $\boldsymbol{p} = (\theta, \phi)$。

求方程(16.11)和方程(16.12)关于时间的导数，然后把方程(15.2)和下式代入其中：

$$X = R\sin\theta\cos\phi,\ Y = R\sin\theta\sin\phi,\ Z = R\cos\theta \tag{16.13}$$

我们获得矩阵形式的球面光流方程：

$$\begin{pmatrix}\dot{\theta}\\ \dot{\phi}\end{pmatrix} = \boldsymbol{J}_{p,s}(\theta, \phi, R)\begin{pmatrix}v_x\\ v_y\\ v_z\\ \omega_x\\ \omega_y\\ \omega_z\end{pmatrix} \tag{16.14}$$

其中图像特征雅克比矩阵是

$$\boldsymbol{J}_{p,s} = \begin{pmatrix}-\frac{\cos\phi\cos\theta}{R} & -\frac{\sin\phi\cos\theta}{R} & \frac{\sin\theta}{R} & \sin\phi & -\cos\phi & 0\\ \frac{\sin\phi}{R\sin\theta} & -\frac{\cos\phi}{R\sin\theta} & 0 & \frac{\cos\phi\cos\theta}{\sin\theta} & \frac{\sin\phi\cos\theta}{\sin\theta} & -1\end{pmatrix} \tag{16.15}$$

它与上一节中为极坐标系推导出的雅克比矩阵相似。首先，常量元素位于相同的位置，这意味着纬度角对于绕光轴的转动是固定不变的，而经度角对于沿光轴的平移也是固定不变的，但是它与绕光轴的相机旋转大小相等，方向相反。至于所有的图像特征雅克比矩阵，其中的平移子矩阵(前三列)是点深度 $1/R$ 的一个函数。

雅克比矩阵在 $\sin\theta = 0$ 的南极点和北极点处是不确定的，在这些点处的方位也没有意义。这是一个奇异点，正如在 2.2.1.3 节中讨论的那样，在方位用欧拉角表示的情况下，这是使用最小表示法的一个结果。然而对于这种应用，通常收益大于支出。

出于控制目的，我们遵循常规程序先为 N 个特征点中的每个点计算出一个 2×8 的雅克比矩阵，如方程(8.2)所示，然后把它们堆叠起来形成一个 $2N \times 6$ 的矩阵：

$$\begin{pmatrix}\dot{\theta}_1\\ \dot{\phi}_1\\ \vdots\\ \dot{\theta}_N\\ \dot{\phi}_N\end{pmatrix} = \begin{pmatrix}\boldsymbol{J}_1\\ \vdots\\ \boldsymbol{J}_N\end{pmatrix}\boldsymbol{\nu} \tag{16.16}$$

控制律是

$$\boldsymbol{\nu} = \boldsymbol{J}^{+}\dot{\boldsymbol{p}}^{*} \tag{16.17}$$

其中，$\dot{\boldsymbol{p}}^*$ 是在 $\phi\theta$ 空间中特征的期望速度。通常，我们选择它与特征误差成正比：

$$\dot{\boldsymbol{p}}^* = \lambda(\boldsymbol{p}^* \ominus \boldsymbol{p}) \tag{16.18}$$

其中，λ 是正增益，$\boldsymbol{p}$ 是在 $\phi\theta$ 坐标系中表示的当前点，$\dot{\boldsymbol{p}}^*$ 是期望值。该控制律会导致在特征空间内特征的局部线性运动①。$\ominus$表示模减法，在 $\theta \in [0,\pi]$，$\phi \in [-\pi,\pi)$ 条件下它会返回一个最小角距离。

使用球面坐标的 IBVS 示例(见图 16.4)可以由 `IBVS_sph` 类来实现。首先创建一个球面相机：

```
>> cam = SphericalCamera()
```

然后是一个球面 IBVS 目标：

```
>> Tc0 = transl(0.3, 0.3, -2)*trotz(0.4);
>> vs = IBVS_sph(cam, 'T0', Tc0, 'verbose')
```

我们对于 IBVS 失败的情况进行一个仿真：

```
>> vs.run()
```

动画显示了 $\phi\theta$ 平面上的特征运动，以及三维空间视图中的相机和空间点运动。球面成像对于视觉伺服有很多优点。首先，球面相机消除了在视场中明确保留特征的需要，这对基于位置的视觉伺服和一些混合策略都是一个大问题。其次，我们在以前观察到对于小视场情况，在 R_x 与 $-T_y$ 运动(以及 R_y 与 $-T_x$ 运动)的光流场之间存在不确定性。对于一个长焦距的 IBVS，这会减慢收敛和/或减弱特征坐标中对噪声的敏感度。而对于一个球面相机，由于有尽可能大的视场，因此可以减小这种不确定性②。

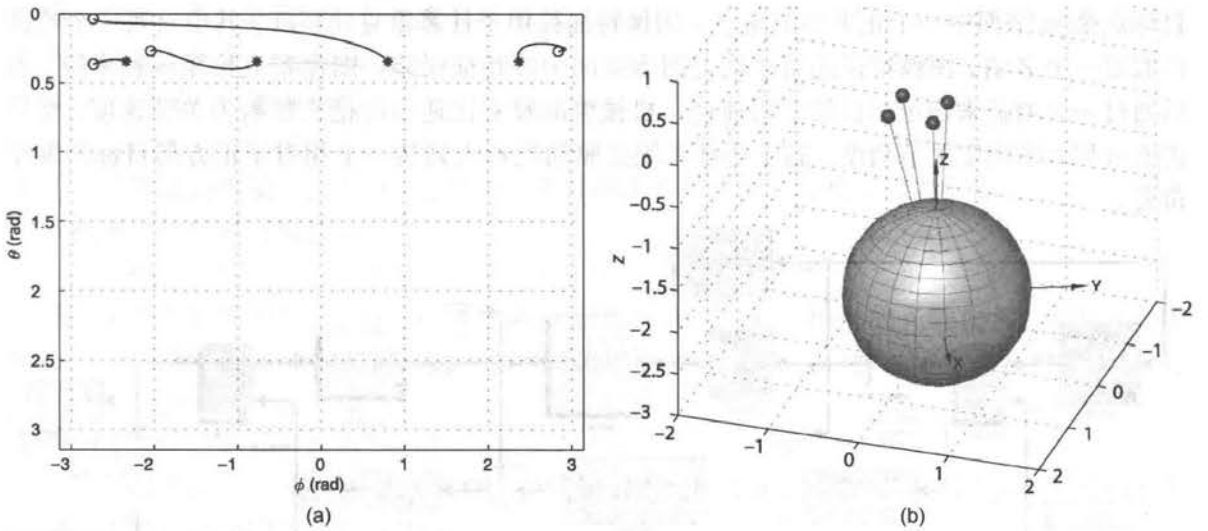


图 16.4　使用球面相机和坐标的 IBVS。(a) $\theta\phi$ 空间中的特征运动；(b)初始位姿时投影到球面上的4个目标点

真正的球面相机至今还不存在，但是我们可以把来自任意类型的一个或多个相机的特征投影到球面特征平面上，然后用球面坐标来计算控制律。

① 注意，这个平面上的运动通常不会是球面上的一个大圆弧——只有沿着纬度线和赤道线的运动才是大圆弧。

② 条件是空间点在球面周围的分布良好。

16.4 应用：机械臂机器人

在这个示例中，相机被安装在一个6自由度的机械臂机器人上，机器人可以控制相机运动的所有6个自由度。我们会假设机器人的关节是理想的速度源，即它们可以按照命令给出的速度进行精确移动。现代的机器人很接近这种理想类型，它们通常具备高性能的关节控制器，使用来自关节处编码器的速度和位置反馈。

对于一个机器人关节的嵌套控制结构在9.4.2节中已经讨论过。内部的速度环使用关节速度反馈来确保关节以要求的速度移动。外部的位置环使用关节位置反馈来确定需要的关节速度以跟踪轨迹。在这个视觉伺服系统中，位置环的功能由视觉系统提供。视觉传感器相对于编码器有一个较低的采样频率，通常为25 Hz或者30 Hz，还经常有一至两个采样时间的高潜伏期。

这个手眼系统的Simulink®模型为

```
>> sl_arm_ibvs
```

如图16.5所示。这是一个复杂的示例，它不仅要仿真相机和IBVS控制，而且还要仿真机器人，这里使用本书第三部分中无处不在的Puma 560机器人。关节角是代表机器人速度环的一个积分器的输出。这些角度又是一个正向运动学模块的输入，该模块的输出是末端执行器位姿。具有默认参数的一个透视相机被安装在机器人的末端执行器上，且相机的轴与机器人末端执行器的坐标系对齐。相机的参数包括`CentralCamera`对象和目标点的空间坐标，这些目标点是yz平面中一个正方形的角点。图像特征被用于计算雅克比矩阵，其中对每一个特征点假定一个Z值，图像特征还用来确定图像空间中的特征误差。图像雅克比矩阵被求逆，然后通过一个增益来确定相机的空间速度。机械臂的雅克比逆矩阵把它映射为关节速度，然后再通过积分来确定关节角度。这个闭环系统会驱动机器人到达一个相对于正方形目标的期望位姿。

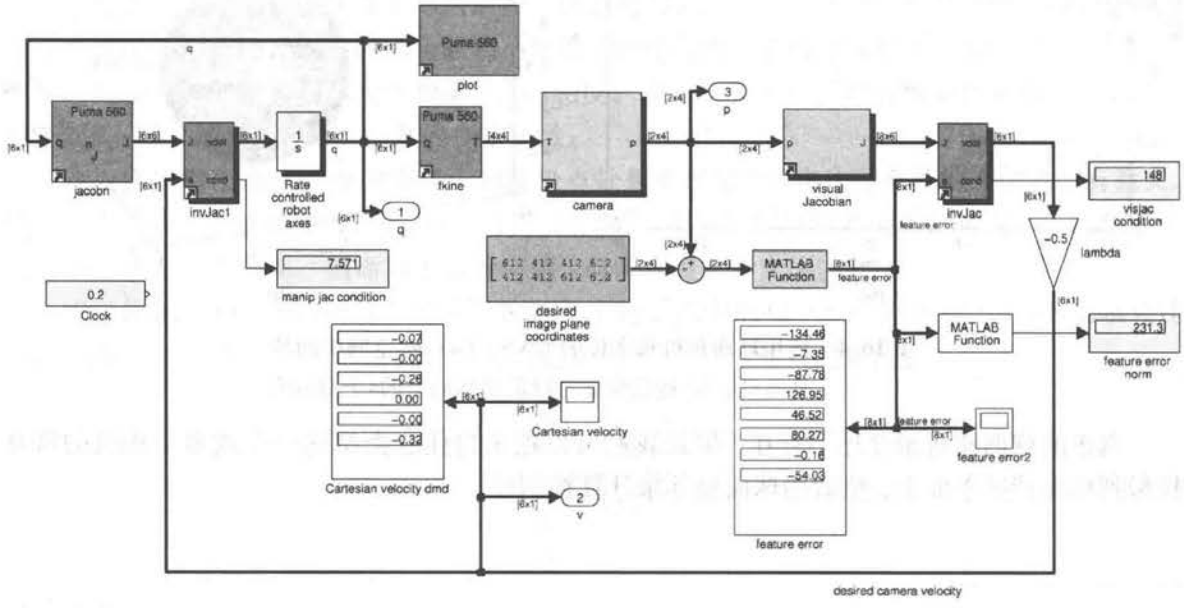


图16.5 Simulink®模型`sl_arm_ibvs`驱动Puma机器人，机器人上安装有使用IBVS的相机

运行这个仿真模型：

```
>> r = sim('sl_arm_ibvs')
```

它会显示机器人的运动以及一个虚拟相机的图像平面。不同输出模块处的信号存储在对象 r 中，而在输出端 1 上的每一个时间步的关节角是

```
>> q = squeeze(out.find('yout').signals(1).values)';
>> about(q)
q [double] : 60x6 (2880 bytes)
```

注意，这个模型中不包括任何机器人或者视觉系统的动力学。关节模型为理想的速度控制装置，视觉系统被建模为没有时间延迟。这个模型可以作为许多更实际系统的建模基础，实际模型中要考虑真实世界的影响。

16.5　应用：移动机器人

在这一节中将考虑把相机安装在移动机器人上，并在平面环境中进行移动。我们首先会考虑一个完整约束的机器人，即该机器人具有全方位底盘并能沿任何方向平移，然后将结果扩展到非完整约束的机器人小车上，它的一些问题在第 4 章中讨论过。相机将观察两个或更多的点路标，这些路标的三维坐标已知，也就是说把它们放置在机器人作业的平面上。视觉伺服控制器将会驱动机器人直到它看见的路标视图与期望的视图匹配。

16.5.1　完整约束移动机器人

对于这个问题，我们假定在机器人上安装了一个中央透视相机，而且在环境中有很多已知位置的路标，在机器人沿路径移动过程中这些路标对相机是连续可见的。机器人坐标系的设置为 x 轴朝向前方，z 轴朝向上方。

我们定义一个透视相机：

```
>> cam = CentralCamera('default', 'focal', 0.002);
```

它有一个较宽的视场，这样无论它移动到哪里都能看见路标。相机安装在机器人小车上，它相对于机器人坐标系的偏移 ${}^{v}\xi_{C}$ 为

```
>> T_vc = transl(0.2, 0.1, 0.3)*trotx(-pi/4);
```

相机位于小车后轴的前方，中心线的左侧，高出地面 30 cm。相机的光轴向前，但向上倾斜 45°，它的 x 轴指向小车的右侧。两个路标高出地面 2 m，位于 $x = 0$，$y = \pm 1$ m 处：

```
>> P = [0 0; 1 -1; 2 2]
```

期望的小车位置是其后轴的中心在 $(-2,0,2)$ 处。

由于机器人只在 xy 平面进行移动，且仅绕 z 轴进行旋转，所以我们可以去除方程(15.6)中对应于禁止运动的列，然后得到：

$$\begin{pmatrix}\dot{u}\\ \dot{v}\end{pmatrix}=\begin{pmatrix}-\frac{f}{\rho_u Z} & 0 & \overline{v}\\ 0 & -\frac{f}{\rho_v Z} & -\overline{u}\end{pmatrix}\begin{pmatrix}v_x\\ v_y\\ \omega_z\end{pmatrix} \tag{16.19}$$

对于标准 IBVS 的情况，我们堆叠这些雅克比矩阵(每一个路标对应一个雅克比矩阵)，然后通过对方程求逆来得到机器人小车的速度。由于只有 3 个未知的速度分量，每一个路标提供两个等式，因此需要两个或更多特征点来求解速度。

Simulink® 模型为

```
>> sl_mobile_vs
```

如图 16.6 所示，它与之前如图 16.5 和图 15.9 所示的模型大体上相似。模型通过以下指令进行仿真：

```
>> r_ = sim('sl_mobile_vs')
```

它展示了在 xy 平面机器人路径和相机视觉的动画。仿真结果存储在对象 `r` 中，也可以像先前的示例那样显示出来。参数和相机是在模型各种各样的模块属性中进行定义的①。

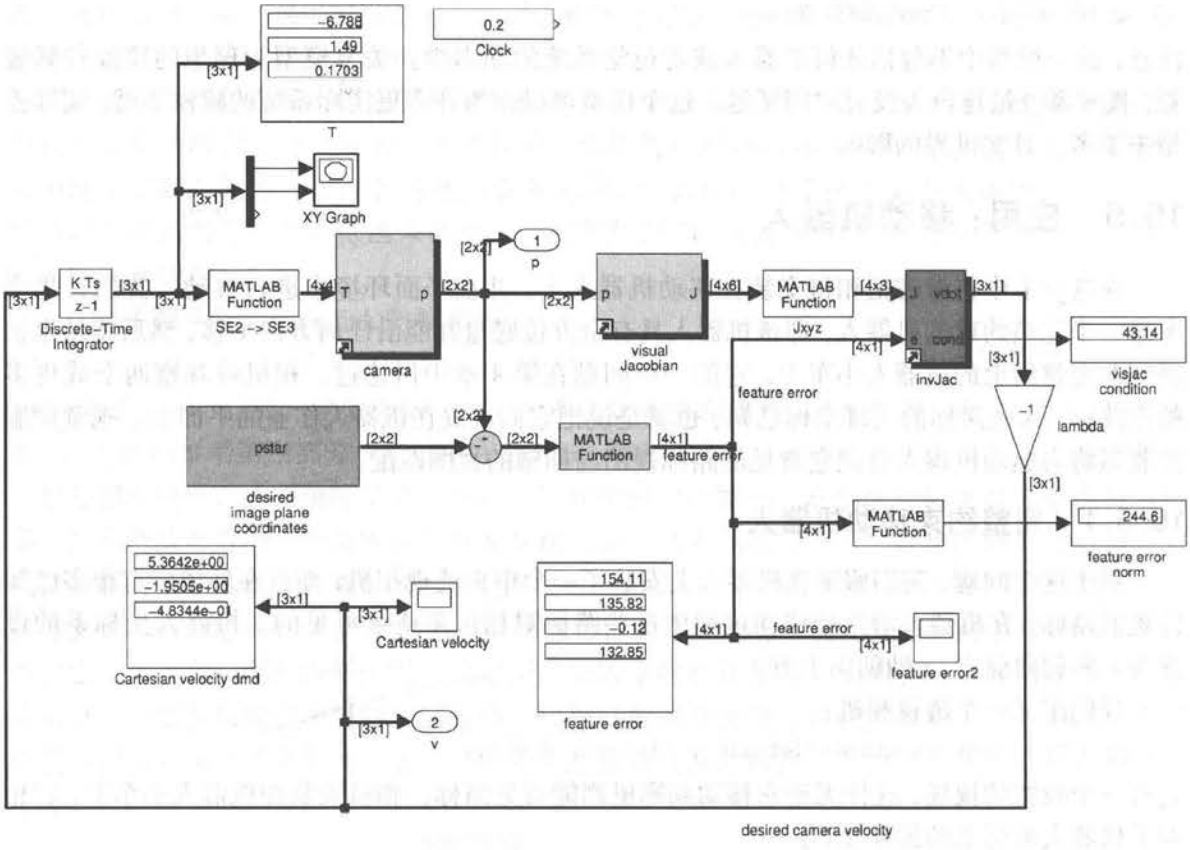


图 16.6 Simulink® 模型 `sl_mobile_vs`，它使用 IBVS 控制策略驱动一个完整约束移动机器人到达一个位姿

16.5.2 非完整约束移动机器人

伺服控制一个非完整约束移动机器人到一个位姿，其难点在 4.2.4 节中已经讨论过，并提出过一个非线性位姿控制器。图 16.7 展示了我们的问题中使用的符号体系，这里再一次使用了一个基于极坐标 ρ，α 和 β 的控制器。对于这个控制示例，我们将会使用 PBVS 技术来估计控制需要的变量。假设有一个以相对位姿 v_{ξ_C} 固定在机器人上的中央透视相机；环境中还有很多已知位置的路标，在机器人沿路径移动过程中这些路标对相机是连续可见的；另外，小车的方位角 θ 也是已知的，可能采用指南针或某种其他传感器获取。

Simulink® 模型为

```
>> sl_drivepose_vs
```

如图 16.8 所示。相机的初始位姿由 `Bicycle` 模块的参数设定。路标的视图通过相机模块进行仿真，相机模块的输出，即投影点，是姿态估计模块的输入，路标的已知位置被设定为参

① Simulink® 菜单 `File + ModelProperties + Callbacks + PreLoadFcn`。当模型被加载时，这些命令被执行一次。

数。正如 11.2.3 节中讨论的那样，至少需要 3 个路标，在这个示例中我们使用了 4 个路标。模型的输出是路标相对于相机的估计位姿，但是由于路标是被定义在全局坐标系{0}中的，所以输出是 ${}^{C}\hat{\xi}_0$。机器人小车在全局坐标系中的位姿可以通过一系列简单的变换操作获得 $\hat{\xi}_V = \ominus {}^{C}\hat{\xi}_0 \ominus {}^{V}\hat{\xi}_C$。这个变换的 x,y 分量与估计的小车方位角结合就能得到一个估计的小车位姿 $(\hat{x},\hat{y},\hat{\theta})$，它是位姿控制器的输入量。系统的剩余部分基本上与图 4.14 中的示例相同①。

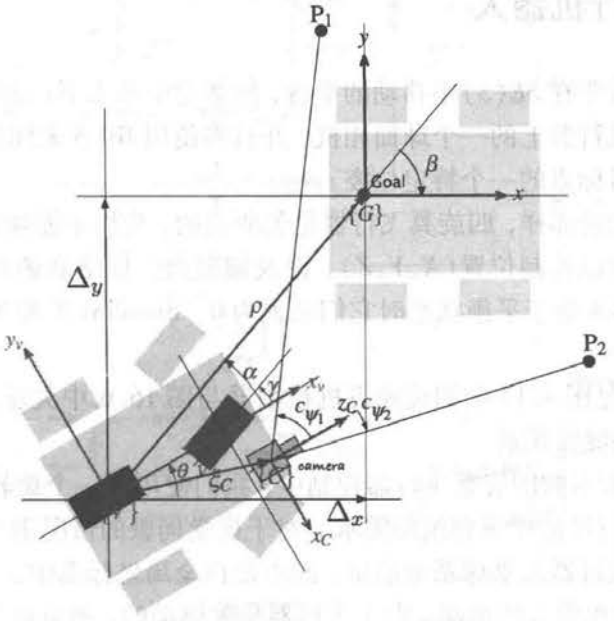


图 16.7　非完整约束机器人小车(自行车模型)的 PBVS，小车朝一个目标位姿移动：ρ 是到目标的距离，β是相对于全局坐标系的目标向量的角度，α是相对于小车坐标系的目标向量的角度。P_1 和 P_2 是路标，它们相对于相机的方位角为ψ_1和ψ_2

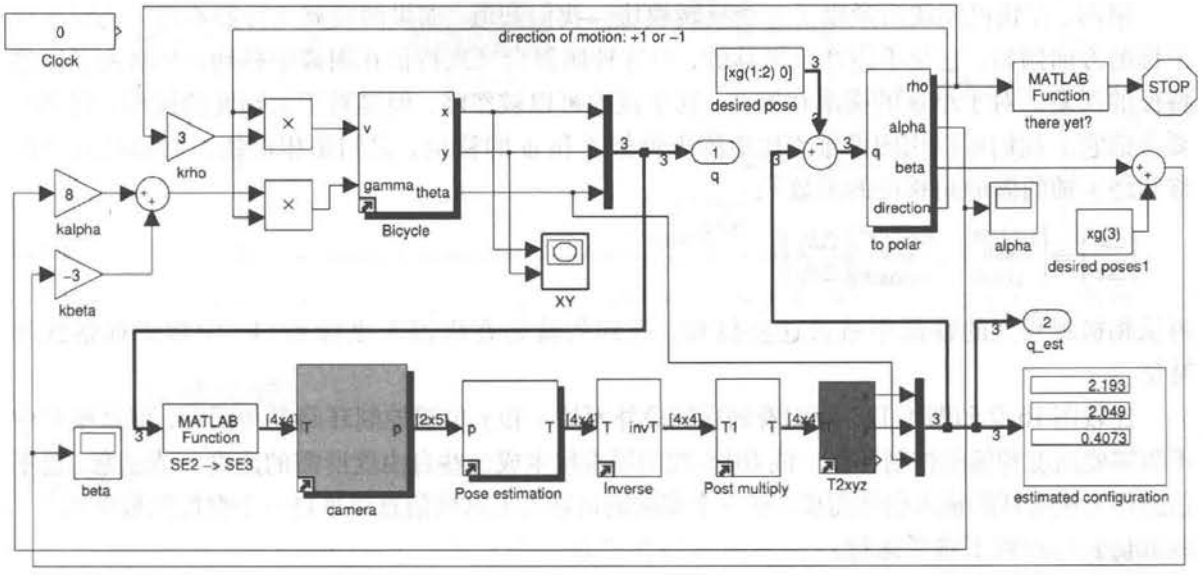


图 16.8　Simulink® 模型 sl_drivepose_vs 驱动非完整约束移动机器人到达一个位姿(修改自图 4.13)

① 在实际系统中，方位角可能会来自一个指南针。在这个仿真中，我们仅使用真实的方位角。

仿真通过以下指令运行：

```
>> r = sim('sl_ibvs')
```

动画显示了相机位姿、图像平面特征误差和相机速度。示波器模块还绘制出相机速度和特征误差相对于时间变化的曲线。仿真结果存储在对象 r 中，也可以像之前的示例那样显示出来。

16.6 应用：飞行机器人

球面相机特别适合于在 $SE(3)$ 中移动的平台，例如空中和水下机器人。在这个示例中，我们考虑安装在四旋翼飞行器上的一个球面相机，并且将使用 IBVS 来伺服控制四旋翼飞行器到达相对于地面上 4 个目标点的一个特定位姿。

正如 4.3 节中讨论的那样，四旋翼飞行器是欠驱动的，我们不能独立控制工作空间中所有的 6 个自由度。我们可以控制位置 (X,Y,Z)，以及偏航角。横滚和俯仰角是被操纵来获得水平面中的平移，当机器人处于平衡状态时它们必须为 0。Simulink® 模型为

```
>> sl_quadrotor_vs
```

如图 16.9 所示。模型是图 4.17 中四旋翼飞机控制器与图 16.6 中欠驱动 IBVS 系统的一个混合体，然而也有很多关键的区别。

第一，在图 4.17 所示的四旋翼飞行器控制中，我们使用了一个旋转矩阵来把全局坐标系中的 xy 误差映射为飞行器的横滚和俯仰需求。对于视觉伺服的情况则不需要这样，因为 xy 误差已经在相机坐标系或机器人坐标系中给出，而不是在全局坐标系中。

第二，类似于移动机器人的情况，由于飞行器是欠驱动的，所有这里的雅克比矩阵仅由对应于 (v_x,v_y,v_z,ω_z) 的列所组成。

第三，我们使用了一个球面相机，所以一个 SphericalCamera 对象被传递到相机模块和视觉雅克比模块。

第四，在相机模块后紧跟了一个反转模块。我们知道，如果四旋翼飞行器不首先向它希望平移的方向倾斜，它是不能产生平移的，但这种倾斜会导致特征在图像中移动，并增大了图像特征的误差。对于小量的横滚和俯仰，这个误差可以被忽略，但是对于大幅度的操作，就必须要考虑它。我们可以用图像雅克比矩阵来近似 θ 和 ϕ 的移位，它们是相机绕 x 轴的横滚角位移和绕 y 轴的俯仰角移位的函数[①]：

$$\begin{pmatrix}\Delta\theta\\ \Delta\phi\end{pmatrix}=\begin{pmatrix}\frac{\cos\phi\cos\theta}{\sin\theta} & \frac{\sin\phi\cos\theta}{\sin\theta}\\ \sin\phi & -\cos\phi\end{pmatrix}\begin{pmatrix}\Delta\theta_R\\ \Delta\theta_P\end{pmatrix}$$

再从相机观察到的特征中减去这些位移，得到的就是在机器人坐标系 $\{V\}$ 中相机观察到的特征。

比较图 16.9 和图 4.17，我们看到它们最外面的 x 和 y 位置控制环路是相同的，但新模型中不再需要高度和偏航控制环路。该 IBVS 控制器直接生成这些自由度所需的速度。请注意，速率信息作为速度环的输入仍然需要，在一个实际的机器人上这些信息将来自一个惯性测量单元。

仿真通过以下指令运行：

```
>> sim('sl_quadrotor_vs')
```

① 这是对特征运动的一阶近似。

动画显示了相机位姿、图像平面的特征误差和相机速度。示波器模块中还绘制出相机速度和特征误差相对于时间变化的曲线。仿真结果可以从仿真输出对象 out 中获得。相机的初始位姿在模型的属性中设置①。

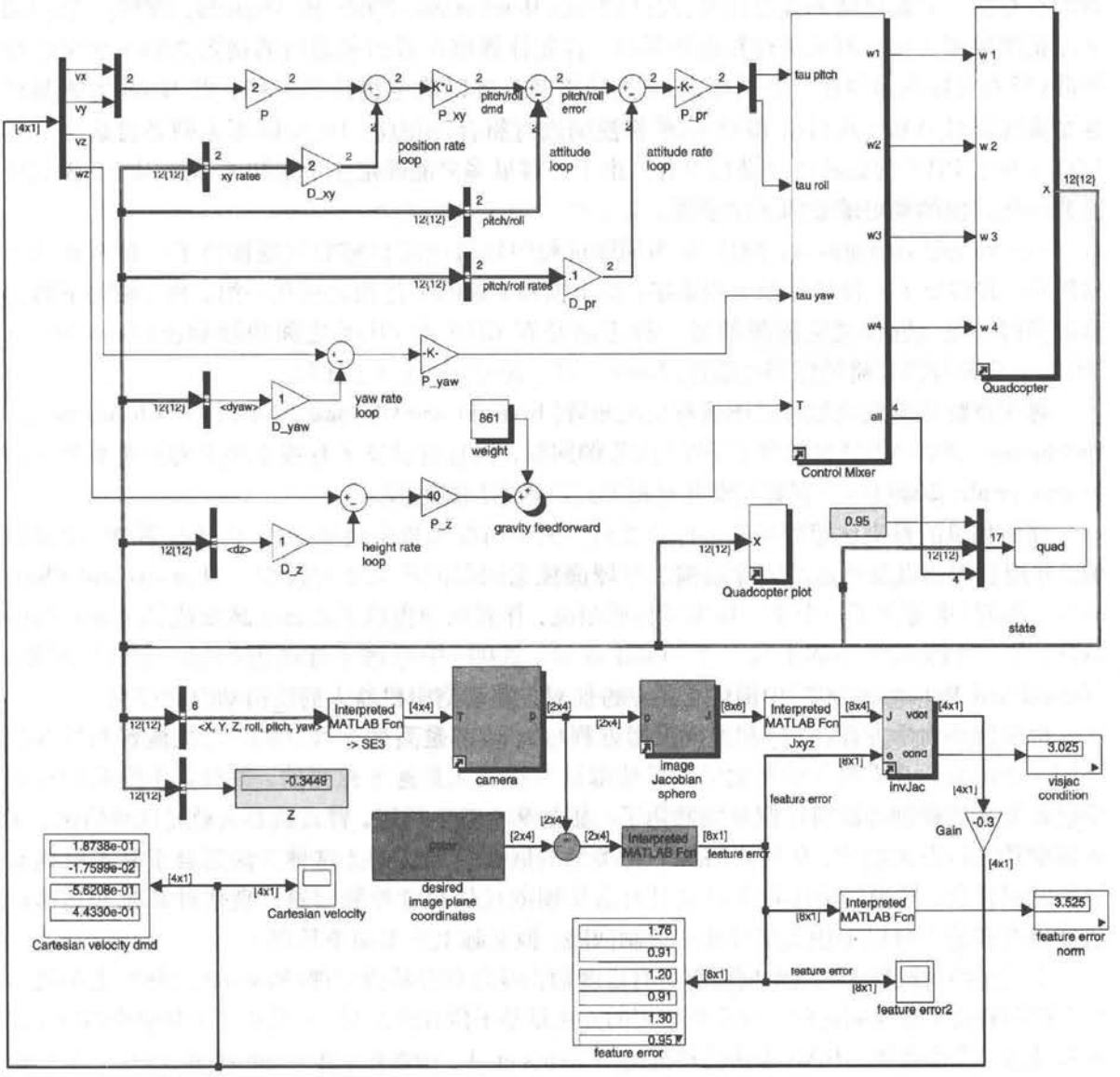


图 16.9　Simulink® 模型 sl_quadrotor_vs。IBVS 用一个球面相机控制机器人在目标上空悬停

16.7　本章总结

扩展阅读

[Chaumette and Hutchinson, 2007]是一篇介绍高级视觉伺服技术的教学文章，[Siciliano and Khatib, 2008, § 24]中也对这些技术做了简要介绍。很多学者对于所谓的混合技术的兴

① Simulink® 菜单 File + ModelProperties + Callbacks + InitFcn。这些命令必须在仿真开始前执行。

趣都是由[Chaumette, 1998]这篇论文引发的，其中介绍了这样一个特别的示例：对于目标绕光轴转动 π 的情况，一个基于点的 IBVS 系统驱动相机到了无限远。解决这个问题最早的方法之一是[Malis, 1999]中提出的一个 2.5D 视觉伺服，它用一个最小的笛卡儿特征增强了基于图像的点特征。其他早期的混合技术方法可参见[Morel et al., 2000]和[Deguchi, 1998]，它们都采用把图像雅克比分割成平移和旋转部分，首先计算出在最初和最后的视图之间一个单应性矩阵(所有目标点必须在一个平面上)，然后将其分解以确定旋转和平移。但 Morel 等人是将这个旋转信息与基于点特征 IBVS 的平移控制进行组合。相反，Deguchi 等人则是将这个平移信息与基于 IBVS 的旋转控制进行组合。由于平移最多只能确定一个未知的缩放因子，因此还需要一些其他的确定缩放因子的手段。

[Corke and Hutchinson, 2001]中为伺服过程中相机远离目标的问题提出了一种直观的几何解释，并提出了一种坐标轴分割策略：沿 x 轴和 y 轴的平移和旋转在一组，沿 z 轴的平移和旋转在另一组。混合视觉伺服的另一种方法是在 IBVS 和 PBVS 之间快速切换[Gans et al., 2003]。几种分割策略的控制性能在[Gans et al., 2003]中进行了比较。

对于点特征的极坐标形式图像雅克比矩阵[Iwatsuki and Okiyama, 2002a]，[Chaumette and Hutchinson, 2007]很好地处理了 IBVS 失效的问题，但它也引发了有些不理想的相机平移运动[Corke et al., 2009]——正好与欧几里得形式出现的问题相反。

球面相机的雅克比矩阵与极坐标的类似。两个角度参数化最早是在[Corke, 2010]中提出的，并用于控制以及从运动估计结构。对球面视觉伺服的研究相对较少。[Fomena and Chaumette, 2007]中考虑了一个单一球形目标的情况，作者从中提取了来自于球面成像平面投影的特征，如一个圆的圆心和半径。在[Tahri et al., 2009]中考虑了球图像特征，如直线和矩。[Hamel and Mahony, 2002]中描述了用点特征对欠驱动空中机器人的反向动力学控制。

机械臂动力学方程(9.1)和透视投影方程(11.2)都是高度非线性的，是机械臂和目标状态的一个函数。几乎所有的视觉伺服系统都认为机器人是速度控制的，而且其中隐藏的动力学已被紧密的控制环路所压制和线性化了。正如 9.4 节中所述，臂式机器人就是这种情况，而在四旋翼飞行器示例中，我们使用了一种类似的嵌套控制结构。这种方法适合于基本结构有短的时间常数、较慢的采样频率以及具有潜伏期的任何视觉控制回路。现代计算机和高速相机使得在理论上可以不用关节级速度控制回路，但实际上它更简单易用。

非完整约束机器人的视觉伺服都是有难度的，因为布洛基的定理[Brockett, 1983]表明线性定常控制器是不能控制它的。本章所使用的方法是基于位置的控制，它是 4.2.4 节中介绍的位姿控制器的一个小改进。IBVS 方法已被提出[Tsakiris et al., 1998]，[Masutani et al.,1994]，但它们要求相机被一个少自由度的机器人安装到基座上。[Mariottini et al., 1994, 2007]中描述了一种两步的伺服方法，其中相机被刚性地连接到基座上，而且由当前和期望的相机视图定义的几何极点也被明确地伺服。[Usher et al., 2003]，[Usher, 2005]中描述了一个开关控制律，它将机器人放在通过期望位姿的一条直线上，然后沿着这条直线到达期望位姿——给出了在户外机器人小车上的实验结果。在[Corke, 2001]中讨论了移动机器人导航与视觉伺服之间的相似性。

习题

1. IBVS 的 *XY/Z* 分割(6.1 节)。

a)对于不同的初始相机平移和旋转组合，研究产生的运动，并与上一章中经典的 IBVS 策略

进行比较。

b)创建一个情形，让特性离开图像。

c)增加一个排斥力区域，以确保特征保留在图像内。

2. 对于不同的初始相机平移和旋转组合，研究极坐标和球面坐标 IBVS 的性能，并与上一章中经典的 IBVS 策略进行比较。

3. 机械臂 IBVS 的示例(16.4 节)。

a)在末端执行器和相机之间的增加一个偏移量(旋转和/或平移)。为此，你的控制器将需要加入一个额外的雅克比矩阵(参见 8.1.1 节)。

b)在相机模块后面添加一个离散时间采样器和延时，来模拟相机的帧频和图像处理时间。研究延时加长的系统反应，以及增益与延时的关系。尝试为这个动态系统绘制一个离散时间的根轨迹图。

c)为一个移动目标建模。提示，使用 `roblocks` 库中的 `camera2` 模块。显示出跟踪误差，即相机和目标之间的距离。

d)研究用前馈技术来提高控制性能[Corke, 1996b]。提示：不要基于目标被相机观察到的位姿来控制，而是要基于未来短时间它将要在的位姿进行控制。多远的未来？什么是对于这个估计的一个好模型？请查看工具箱类 `Alphabeta`，寻找一个简单易用的跟踪滤波器(有挑战性)。

e)对于一个机器人入坞的任务，手眼相机可能会有问题，因为相机会非常接近目标。你将如何配置目标和相机以避免这种情况？

4. 移动机器人视觉伺服(16.5 节)。

a)对于完整约束和非完整约束的情况，使用一个折射反射式相机替换透视相机。

b)对于使用折射反射式相机的完整约束情况，让移动机器人走过一系列的中间点，每个中间点由一组期望的特征坐标定义。

c)对于非完整约束情况，实现第 4 章中的纯追随和直线跟踪控制器，但在这里要使用视觉特征。对于纯追随，考虑被追目标上有一个或两个点特征。对于直线跟踪的情况，考虑使用一个或两个直线特征。

5. 对于极坐标 $r-\phi$ 和球面坐标 $\theta-\phi$ 投影(16.2 节和 16.3 节)，显示出类似图 15.7 的特征流场。对于球面坐标的情况，你可以在一个球体的表面上绘制出流场向量吗？

6. 四旋翼飞机。

a)用一个球面相机替换透视相机。

b)创建一个控制器，使机器人跟踪一系列的点特性，而不是悬停在一个单点上方(有挑战性)。

c)创建一个控制器，使机器人跟踪一系列的点特性，而不是悬停在一个单点上方(有挑战性)。

7. 实现[Malis, 1999]中提出的 2.5D 视觉伺服策略(有挑战性)。

附　录

附录 A　安装工具箱
附录 B　Simulink 软件
附录 C　MATLAB 对象
附录 D　线性代数复习
附录 E　椭圆
附录 F　高斯随机变量
附录 G　雅克比矩阵
附录 H　卡尔曼滤波
附录 I　齐次坐标系
附录 J　图
附录 K　峰值搜索

附录 A　安装工具箱

机器人和机器视觉工具箱都可以从本书网站上免费获取：

http://www.petercorke.com/RVC

该网站上还有很多与本书有关的附加信息，如网站链接(包括所有在书中出现的网站及其他网站)、程序代码、图片、练习题以及勘误。

文件与路径

两个工具箱的文件都存放在一个名为 rvctools 的顶层目录中，在该顶层目录下有以下子目录：

robot	机器人工具箱。
vision	机器视觉工具箱。
common	机器人和机器视觉工具箱共享的功能函数。
simulink	有关机器人和视觉的 Simulink 程序模块，还包括一些例子。
contrib	第三方程序代码。

下载

两个工具箱分别被打包压缩为一个 zip 格式的文件，即 rtb.zip 和 mvtb.zip。在文件下载页面上需要你输入一些信息，如你的国家、单位类型及职业等。但这并没有恶意，只是想收集一些使用者的兴趣好爱以及对该软件的反馈意见。

安装

使用你熟悉的解压缩工具软件将下载的文件解压。

为了把工具箱添加到你的 MATLAB 路径下，请执行以下命令：

```
>> addpath RVCDIR ;
>> startup_rvc
```

其中，RVCDIR 是你解压顶层工具箱目录 rvctools 时所在目录的全路径名。命令 startup_rvc 会将各种子文件夹添加到你给的路径，并显示出工具箱的版本号。

这个命令还能被交互执行，或者把它放到你的 startup.m 文件中，这样你每次打开 MATLAB 软件时它就能自动执行。对于 Linux 和 Mac OS 操作系统的计算机，则可以把路径加到环境变量 MATLABPATH 中，该变量包含了一系列用逗号分隔的路径。

MEX 文件

在工具箱中有些函数被存为 MEX 文件格式，这些函数是为了提高计算效率而采用 C 语言

编写的，但它们能像其他函数一样被 MATLAB 调用。工具箱提供已创建好的针对 Ubuntu Linux、MacOS 10.6(32 bit)和 Windows 系统的 MEX 二进制文件，也提供 C 语言的源代码。

在线讨论组

在本书网站上还有一个在线讨论组，它提供了对各种问题的讨论和答复，以及漏洞修复。

外供代码

工具箱中有很多有用的函数是由第三方提供的，我们给这些函数编写了封装以使它们与工具箱其他函数兼容。所有这些第三方代码都保存在子目录 `contrib` 中。要使用这些函数，你必须先下载文件 `contrib.gz` 或 `contrib.zip`，然后解压缩这个文件到顶层目录 rvctools。

如果你没有下载这些第三方外供代码就去调用其中的函数，则会收到一个出错信息。这些外供代码可以随时下载和安装。

很多的这些外供函数都来自其他大的软件项目，下载文件中包含的只是其中一小部分经过测试的函数。这些函数经过了时间的考验以及改进，其主要功能在本书中也有描述。有关那些软件项目的信息可以参见本书网站提供的链接。

版权许可

所有非第三方提供的代码都是在 LGPL 协议下发布的。这意味着你可以免费使用并随意修改这些代码，前提是要保证这些代码的版权信息和许可协议不被更改。

第三方提供的代码模块则遵循各自不同的开源许可协议。我们为这些代码编写的工具箱兼容封装也遵守相应的协议。

MATLAB 版本信息

本书介绍的工具箱软件是在苹果 Mac OS X(雪豹)操作系统下用 MATLAB 的 R2011a 和 R2012a 版软件进行开发和测试的。由于使用了新版 MATLAB 中的一些新功能，因此这些工具箱软件有可能在旧版的 MATLAB 中不能正常使用。如果你使用的是版本 R2010a 或更低的 MATLAB，就请不用再告知你发现的错误和漏洞了。

Octave 软件

Octave(`www.octave.org`)是一款非常好的 GNU 协议下的免费软件。该软件使用的编程语言与 MATLAB 的不同，但比较接近，主要差异体现在对图形和对象的操作处理上。现在，这两种语言正逐渐趋同，差异也正在缩小。目前本书第 7 ~ 9 章介绍的有关机械臂应用的工具箱函数已经能兼容到 Octave 软件中，这些函数代码放在 RVCDIR/robot/octave 中。但是，想要在不远的将来 Octave 就有与 GUI 或 Simulink 同样的功能却很难。

附录 B　Simulink 软件

Simulink 是 MATLAB 中用于模块框图编辑和仿真的环境。但它的许可证是与 MATLAB 分立的，需要单独购买，只在学生版的 MATLAB 中才包括 Simulink 组件。Simulink 提供了一种非常简便易用的方式去创建并可视化复杂的动态系统，因此特别适合机器人应用。以前没有 Simulink 使用经验的用户建议先阅读相关的 Mathworks 手册，并用其中提供的实例熟悉它的用法。有 Simulink 经验的用户可以直接尝试使用机器人模块。一般来说，用于 Simulink 的每个机器人模块都能在机器人工具箱中找到对应的函数。

使用 Simulink

如果你已安装了工具箱软件包，就可以在 Simulink 中使用机器人模块及模型实例了。要加载和显示工具箱中的模块库，输入命令：

```
>> roblocks
```

这些模块都可以随意拖曳并放入到某个模型程序中。本书中用到的 Simulink 模型实例都放在目录 `rvctools/simulink/examples` 下。这些模型实例的文件名都有 `sl_`的前缀。要在 Simulink 中加载和显示一个模型，只需要在命令提示符后输入该模型的名字，例如：

```
>> sl_lanechange
```

想要显示任意模块下的模型，用鼠标右键单击该模型名并选择菜单中的 Look 选项即可。

信号与显示格式

在一个 Simulink 模型中的导线可以传递一个标量、一个向量或一个矩阵。为了能明确显示出每条导线上所传递信号的类型，需要在 Simulink 的工具条上进行如下选项设置：Format + PortSignal Displays + Signal Dimensions 和 Format + PortSignal Displays + Wide Nonscalar Lines。

工作空间变量与回调函数

大多数工具箱对应的 Simulink 模块都有参数，这些参数可能是包含常数、函数调用或 MATLAB 工作空间变量的任何 MATLAB 表达式。有的 Simulink 模型实例要在工作空间中通过使用一个回调函数来设置参数。每个模型都有很多回调函数以备不同情况下调用，这些回调函数可以通过菜单 File + Model Properties + Callbacks 进行查看。回调函数 PreLoadFcn 当某个模型被加载并且被其他模型用来设置参数时才被调用。作为一个工具箱惯例，该参数初始化代码会在命令窗口中显示一条信息，让用户知道它已经更新了工作空间。

Simulink 的版本

本书使用的 Simulink 模型都是在 Mac OS X 操作系统中使用 R2010a 版本的 MATLAB 进行开发和测试的。开发中应用了很多新版 Simulink 的特性，因此在老版本中很可能不能正常工作。如果你使用的 MATLAB 及其 Simulink 是比版本 R2010a 更低的，就请不用再告知我们你发现的错误了。

编程注释

有些 Simulink 的模块是用 MATLAB 代码编写成的 S 文件。这些模块函数用 MATLAB 的 M 语言以一种特殊格式编写，以便能与 Simulink 的仿真引擎接口。当然，刚开始时对要封装一个已有函数产生畏惧是非常正常的，但只要经过不断尝试和修正，才能编写出一个与 Simulink 环境兼容的函数。想获取有关编写 S 文件的更多信息，请参阅相关的 Mathworks 手册。

Simulink 的模块

与机械臂机器人相关的

Robot	该模块针对的是一个串联式机械臂，以广义关节力作为输入量，输出量为关节坐标、速度和加速度。参数为要仿真的机械臂对象及其初始关节角。该模块计算机械臂的正向动力学。	`SerialLink.fdyn`
RNE	该模块采用递归的牛顿-欧拉算法计算机械臂的逆向动力学。输入量为关节坐标、速度和加速度，输出量是广义关节力。参数为机械臂对象。	`SerialLink.rne`
jacob0	该模块输出一个机械臂在世界坐标系中的雅克比矩阵，输入量为关节坐标向量。参数为机械臂对象。	`SerialLink.jacob0`
jacobn	该模块输出一个机械臂在末端执行器坐标系中的雅克比矩阵，输入量为关节坐标向量。参数为机械臂对象。	`SerialLink.jacobn`
fkine	该模块输出末端执行器位姿的一个齐次变换，末端执行器位姿对应于输入的关节坐标。参数为机械臂对象。	`SerialLink.fkine`
plot	该模块能在一个新窗口中创建机器人的图形动画。参数为机械臂对象。	`SerialLink.plot`

与其他机器人相关的

Bicycle	该模块是用自行车模型建立的一个移动机器人运动学模型。输入为速度和转向角，输出是机器人位置和方向。

Pose integral	该模块执行一个空间速度相对于时间的积分，输出为一个齐次变换。参数是初始位姿。
Quadrotor	该模块是四旋翼直升机的动力学模型。输入为四个旋翼的转速，输出是移动和转动的位置与速度。参数是四旋翼直升机的结构。
ControlMixer	该模块接收推力和力矩指令，输出四旋翼直升机的四个旋转电机速度。
Quadrotor plot	该模块将在一个新窗口中创建四旋翼直升机的一个图形动画。参数为四旋翼直升机的结构。

与路径相关的

jtraj	该模块输出一个点沿一条五次多项式曲线运动时随时间变化的坐标，以及坐标的微分。在起点和终点的速度假设为零。参数包括起点和终点，以及总的运动时间。	`jtraj`
lspb	该模块输出一个点沿一条 LSPB 路径运动时随时间变化的坐标。参数包括起点和终点，以及总的运动时间。	`lspb`
circle	该模块输出一个点沿圆周运动时的 xy 坐标。参数是圆心、半径和角频率。	

与视觉相关的

camera	该模块的输入是摄像机位姿，输出是投影到像平面上的点的坐标。参数为摄像机对象，以及点的位置。	`Camera.project`
camera2	该模块的输入是摄像机的位姿以及点所在坐标系的位姿，输出是投影到像平面上的各点坐标。参数是摄像机对象，以及各点在点坐标系中的相对位置。	`Camera2.project`
image Jacobian	该模块的输入是图像点，输出是点特征雅克比矩阵。参数是摄像机对象。	`CentralCamera.visjac`
image Jacobian sphere	该模块的输入是在球坐标系中的图形点，输出是点特征雅克比矩阵。参数是一个球摄像机对象。	`SphericalCamera.visjac`
Pose estimation	该模块能从图像点计算摄像机位姿。参数是摄像机对象。	`CentralCamera.estpose`

其他模块

Inverse	该模块的输入为一个矩阵，输出为该矩阵的逆矩阵。

Pre multiply	该模块的输入为一个齐次变换，输出为该齐次变换的左侧乘以一个常数参数。	
Post multiply	该模块的输入为一个齐次变换，输出为该齐次变换的右侧乘以一个常数参数。	
inv Jac	该模块的输入是一个正方雅克比矩阵 $\boldsymbol{J}$ 和一个空间速度 $\boldsymbol{\nu}$，输出为 $\boldsymbol{J}^{-1}\boldsymbol{\nu}$ 以及矩阵 $\boldsymbol{J}$ 的条件数。	
pinv Jac	该模块的输入是一个雅克比矩阵 $\boldsymbol{J}$ 和一个空间速度 $\boldsymbol{\nu}$，输出为 $\boldsymbol{J}^{+}\boldsymbol{\nu}$ 以及矩阵 $\boldsymbol{J}$ 的条件数。	
tr2diff	该模块计算 $\Delta(\cdot)$，即两个齐次变换矩阵的差分。它是一个 6 元素向量，包含了齐次变换中平移和转动的差分。	`tr2delta`
xyz2T	该模块将一个平移向量转化为一个齐次变换矩阵。	`transl`
rpy2T	该模块将一个包含横滚-偏摆-俯仰角度的向量转化成一个齐次变换矩阵。	`rpy2tr`
eul2T	该模块将一个欧拉角向量转化成一个齐次变换矩阵。`eul2tr`	
T2xyz	该模块将一个齐次变换矩阵转化成一个平移向量。	`transl`
T2rpy	该模块将一个齐次变换矩阵转化成一个包含横滚-偏摆-俯仰角度的向量。	`tr2rpy`
T2eul	该模块将一个齐次变换矩阵转化成一个欧拉角向量。	`tr2eul`
angdiff	该模块计算两个输入角度的差，差值不超过 2π。	`angdiff`

附录 C　MATLAB 对象

MATLAB 的编程语言，即所谓的“M 语言”，其语法和语义与经典的计算机 Fortran 语言有些相似。例如，数组索引都是从 1 而不是从 0 开始，而且数组下标是用括号内的数来表示，就像函数调用参数。在早期版本的 MATLAB 中，数据类型只有实数或复数的二维矩阵，连标量都要用一个 1 ×1 的矩阵表示。直到 1997 年 5.0 版的 MATLAB 推出后情况才有较大的改变，引入了今天语言中的很多特征：结构，元胞数组，以及类。

早期的计算机语言(Fortran，Pascal，C)都属于命令式语言，程序员是用一系列“动作”来描述计算过程的，这些“动作”将改变程序的状态——数据。用这些语言编写的程序实际上是这样一种逻辑过程：它接收输入数据，再对数据进行处理，然后产生输出数据。随着程序大小和复杂程度增加，命令式语言的局限性变得非常明显，因此需要设计新的语言改正存在的问题。

在 20 世纪 80 年代中期出现了一种非常强大的编程思想，即面向对象的程序设计(OOP)。面向对象的编程模式是围绕“对象”而不是“动作”来组织程序的。每个对象中都包含了数据，以及处理这些数据的函数(也称“方法”)。程序员不需要知道对象内部的细节，只需了解其中的方法提供的接口就能使用这些对象，因而非常容易编写大型程序。

面向对象的编程语言支持“对象类”的概念。例如，我们可以定义一个有关四元数的类，它具有的方法包括返回一个四元数的反四元数，两个四元数乘积，以及用人可读的形式显示一个四元数。在我们的程序中可能会有很多的四元数变量或对象，而每个对象都是上述四元数类的一个实例。每个实例都有各自的值，即对象的数据部分，但这些实例共用了类中定义的方法。

大家熟知的面向对象的语言包括C++，Java，Pyth 和 Ruby，它们在编程风格上虽然还是命令式的，但具有支撑对象的语言特征。MATLAB 的编程语言也具有很多面向对象的特征，详细内容可参见 MATLAB 的相关文件。本书提供的工具箱中也定义了很多类，描述的对象包括机械臂、机械臂关节、四元数、机器人路径规划器以及各种图像特征形式。

对象的应用较好地解决了使用大量 MATLAB 工具箱带来的命名空间污染问题。当程序要调用一个 MATLAB 函数时，它会在一组文件目录(即 MATLAB 搜索路径)中进行搜索。如果搜索路径中包含从各种途径获取的很多工具箱，那么两个函数重名的机会就会大增，问题也就出现了。而如果我们不直接提供函数，而是提供对象中的方法，那么就避免了上述问题，因为这些方法的名字不会占据函数命名空间，而且方法的调用只能通过适当的对象。

使用一个类

下面的例子展示了四元数类的一些功能，该类属于机器人工具箱。首先创建一个四元数对象：

```
>> q = Quaternion( rotx(0.2) );
```

它调用了类中名为 constructor(构造器)的方法。习惯上，类的名字以大写字母开始。以上方法将检查参数类型并计算出其等价四元数。四元数的标量和向量部分都存在于这个特殊的对象里，即四元数类的实例。在 MATLAB 中，一个对象的数据部分称为它的属性(properties)。传

给构造器方法的参数可以是一个旋转矩阵(在本例中使用)，或一个角度加一个向量，也可以是一个包含标量和向量的四维向量，或者另一个四元数。构造器方法的输出结果是在工作空间中的一个新对象：

```
>> about(q)
q [Quaternion] : 1x1 (88 bytes)
```

该对象的类型是 Quaternion。在程序中，可以查询一个对象的类型：

```
>> class(q)
ans =
  Quaternion
```

它返回一个包含对象类名字的字符串。所有 MATLAB 对象都有一个类型，如

```
>> x = 3
>> class(x)
ans =
  double
```

这个名为 double 的类是内部预置的，而 Quaternion 类则是用户定义的。我们可以用以下方式测试一个对象的类型，返回“0”表示不符合，返回“1”表示符合：

```
>> isa(q, 'double')
ans =
     0
>> isa(q, 'Quaternion')
ans =
     1
```

也可以查看这个四元数对象的属性：

```
>> q.s
ans =
    0.9950
```

它返回了这个四元数对象标量部分的值。但工具箱有关 Quaternion 类的机制不允许单独设置它的这个属性：

```
>> q.s = 0.5;
??? Setting the 's' property of the 'Quaternion' class is not allowed.
```

原因是四元数的标量和向量部分必须同时设定，以确保该四元数的协调一致。

计算反四元数的方法如下：

```
>> qi = inv(q);
```

它返回一个新的四元数 qi，它是四元数 q 的反四元数。

MATLAB 将检查第一个参数的类型，因为该参数是一个 Quaternion 类，所以它调用了 Quaternion 类中的 inv 方法。大多数面向对象的语言都使用带“.”的表达式，以上命令可改写为

```
>> qi = q.inv();
```

该表达式对于要调用对象 q 的 inv 方法更为清楚。上述两个表达式在 MATLAB 中都是允许使用的，但在本书中为了表达清晰，我们使用带“.”的方式。MATLAB 也不要求在没有参数时必须使用空括号，因而以上表达式还可以简化为

```
>> qi = q.inv
```

但为了与其他面向对象的语言保持一致，也为了避免与有参数的情况产生混乱，本书中还是要加上空括号。

任何 MATLAB 的表达式如果不带结束分号都将会显示出该表达式的值。例如，

```
>> qi
qi =
0.995 < -0.099833, 0, 0 >
```

它将自动调用这个四元数类的 display 方法。它与下面这个表达式的结果相同：

```
>> qi.display()
qi =
0.995 < -0.099833, 0, 0 >
```

一些命令将调用 char 方法把这个四元数的值转化为一个字符串：

```
>> s = qi.char();
>> about(s)
s [char] : 1x25 (50 bytes)
```

下面将创建另一个四元数对象：

```
>> q2 = Quaternion( roty(0.3) );
```

然后计算两个四元数的乘积，表达式如下：

```
>> q * q2
```

这是一个运算符重载的例子，很多面向对象的语言都具有这一特点。MATLAB 对该表达式的解释为

```
>> q.mtimes(q2)
```

对于更复杂的表达式，运算符重载可使表达更清晰。例如，可以输入以下简单表达式：

```
>> q*q2*q
ans =
0.96906 < 0.19644, 0.14944, 0 >
```

但 MATLAB 的实际计算的过程却相当复杂，首先要计算乘积 q * q2，并保存到一个临时四元数中，然后再将这个临时四元数乘以 q，得到最后结果，并要删除临时四元数。如果不用运算符重载，要完成以上计算的表达式为

```
>>  q.mtimes( q2.mtimes(q) )
ans =
0.96906 < 0.19644, 0.14944, 0 >
```

它既难以阅读，也很难维护。

创建一个类

四元数类是由工具箱文件 Quaternion.m 定义的，该文件共有 500 多行，但其基本结构如下：

```
1  classdef Quaternion
2
3      properties (SetAccess = private)
4          s       % scalar part
5          v       % vector part
6      end
7
8      methods
9
10         function q = Quaternion(a1, a2)
11             % constructor
12         end
13          .
14          % other methods
15          .
16          .
17     end
18 end
```

其中，第3～6行的 properties 程序块定义了与每个四元数实例相关联的数据，内部表示的变量 s 代表标量，v 代表向量。第 8～17 行的方法程序块定义了该类支持的所有方法。而在第一行 classdef 后的名字必须与文件名一致，它也是类的名称。

该类的属性中有一个 SetAccess，其模式为 private，意思是该属性可以被程序直接读取，而不能设置。如果 q 是一个四元数对象，那么 q.s 就是这个四元数标量部分的值。这里涉及到编程风格的问题，一些人更喜欢通过 getter 函数来获取对象的属性，如 q.get_s()。

每个类都必须有一个 constructor 方法，即构造器，它是一个与类同名的函数。构造器负责初始化对象的数据，本例中即为属性 s 和 v。有些面向对象的语言还支持一个 destructor 函数，即析构函数，它是当某个对象不再需要时调用来取消对象的，在 MATLAB 中使用可选的 delete 方法来实现。

在四元数类中包括了 20 种方法，每种方法被编写成一个小的 MATLAB 函数，并以一条 end 语句结束。传给每种方法的第一个参数是四元数对象自身。例如，计算反四元数的方法如下：

```
function qi = inv(q)
    qi = Quaternion( [q.s -q.v] );
end
```

该方法使用了构造器方法 Quaternion 来生成要返回的四元数。

而将一个四元数转化为一个字符串的方法如下：

```
function s = char(q)
    s = [ num2str(q.s), ' < ' num2str(q.v(1)) ...
    ', ' num2str(q.v(2)) ', '   num2str(q.v(3)) ' >' ];
end
```

方法 mtimes 是被调用来实现运算符重载的，但要求星号“*”两边的操作数中至少有一个为四元数对象。

```
function qp = mtimes(q1, q2)
    if ~isa(q1, 'Quaternion')
        error('left-hand side of * must be a Quaternion');
    end

    if isa(q2, 'Quaternion')
       %Multiply unit-quaternion by unit-quaternion
        s1 = q1.s;  v1 = q1.v;
        s2 = q2.s;  v2 = q2.v;
        qp = Quaternion([s1*s2-v1*v2' s1*v2+s2*v1+cross(v1,v2)]);
    elseif  isa(q2, 'double'),
        if length(q2) == 3
            % Multiply vector by unit-quaternion
            qp = q1 * Quaternion([0 q2(:)']) * inv(q1);
            qp = qp.v(:);
        elseif length(q2) == 1
            % Multiply quaternion by scalar
            qp = Quaternion( double(q1)*q2);
        else
            error('quaternion-vector product: must be a 3-vector
or scalar');
        end
    end
end
```

该方法将检测第二个操作数的类型，再根据这个操作数的类型选择执行四元数-四元数、四元数-向量、四元数-标量的乘积。

MATLAB 的类还支持继承，该特性使得一个新创建的类可以继承一个已有类的属性及方

法，并在此基础上增加新的属性和方法。在本书第二部分“移动机器人”中，多种运动规划器，如 Dstar 和 RRT，都继承了 Navigation 类的属性及方法。而在第四部分“计算机视觉”中，不同类型的摄像机，如 CentralCamera 和 FishEyeCamera，也继承了 Camera 这个类。类的继承要在 classdef 一行里体现出来，例如：

```
classdef Dstar < Navigation
```

类的继承，特别是多重继承是一个相当复杂的问题，详细内容可以参见 MATLAB 相关文件。

在 MATLAB 中，函数 methods 和 properties 将返回一个对象包含的方法和属性，而函数 metaclass 将返回一个数据结构，其中包含了所有的方法、属性和父类。

引用传递

类继承的一个特别有用的应用是解决了“值传递”问题。每当一个变量要被传递给某个函数时，MATLAB 会将变量的值，即变量的一个复本传递过去，而不是传递它的一个引用。这种方式虽然显得非常方便，但如果在某个对象中有个方法要改变一个属性时情况就不同了。我们写出下面的命令：

```
>> myobj.set_x(2);
```

MATLAB 会创建对象 myobj 的一个复本，并调用方法 set_x()处理该复本。然而，由于我们并没有将复本对象赋给任何变量，因此对它的处理和更改都会丢失。正确的方式应如下：

```
>> myobj = myobj.set_x(2);
```

但它显得非常烦琐。而如果对象 myobj 属于某个引用类，那么就可以采用如下方式：

```
>> myobj.set_x(2);
```

这时 myobj 的值就可以改变了。为了创建一个引用类，该类必须要继承 handle 类：

```
classdef MyClass < handle
```

事实上，很多在工具箱里的类都属于引用类，但 Quaternion 类除外。使用引用类可能存在的陷阱是对某个引用类对象的赋值：

```
>> myobj2 = myobj;
```

以上赋值操作意味着 myobj2 和 myobj 都指向同一对象。如果 myobj 改变了，那么 myobj2 也会随之改变。解决办法是用对象的构造器接收这个类的类型属性，再返回一个复本：

```
>> myobj2 = MyClass(myobj);
```

现在，对 myobj 的更改就不再会影响 myobj2 了。

对象数组

MATLAB 对于对象数组和对象向量的操作方式类似。以一个由 SIFT 特征对象构成的数组为例：

```
>> s1 = isurf(im1);
```

以上操作将返回一个由 SurfPointFeature 对象构成的向量。我们可以用以下方式查看向量中对象的数量：

```
>> n = length(s1)
n =
    1288
```

也可以执行如下的索引操作：

```
>> x = s1(1:100);
>> y = s1(1:20:end);
```

注意，由于 SurfPointFeature 对象是引用类对象，因此元素 x 和 y 都是由 s1 引用的同种对象。也可以从向量中删除对象，命令如下：

```
>> s1(50:end) = [];
```

可以调用一个方法去处理一个对象数组，例如调用以下假设的方法：

```
>> z = s1.fewer();
```

它将会把整个向量都传递给这个方法去处理：

```
function r = fewer(s)
    r = s(1:20:end);
end
```

因此，方法既可以对单个对象，也可以对对象数组执行操作。

支持向量的一个类必须有一个构造器来专门处理无属性传递的情况。

多文件机制

对于一个很复杂的类，单个文件可能会因为太长而不能正常工作。这种情况下最好采用多个文件来定义方法，每个文件对应一个方法或一组方法。特别是当有的方法是定义在 MEX-文件里，而不是 M 文件中时，就只能采用多文件机制了。

在 MATLAB 中，实现多文件定义只需在 MATLAB 的搜索路径里创建一个目录，目录名前加一个前缀“@”即可。工具箱中代表机器人机械臂的 SerialLink 类就是采用这种方式定义的，它的所有文件都在一个名为@ SerialLink 的目录下。

附录 D　线性代数复习

矩阵的分类如图 D.1 所示(见彩图)。本书中只讨论 $m \times n$ 的实数矩阵：

$$\boldsymbol{A} = \begin{pmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \\ a_{m,1} & a_{n,2} & \cdots & a_{m,n} \end{pmatrix}, \quad \boldsymbol{A} \in \mathbb{R}^{m \times n}$$

其中，m 代表矩阵的行数，n 代表矩阵列数。如果 $m = n$，则表示该矩阵为方阵。

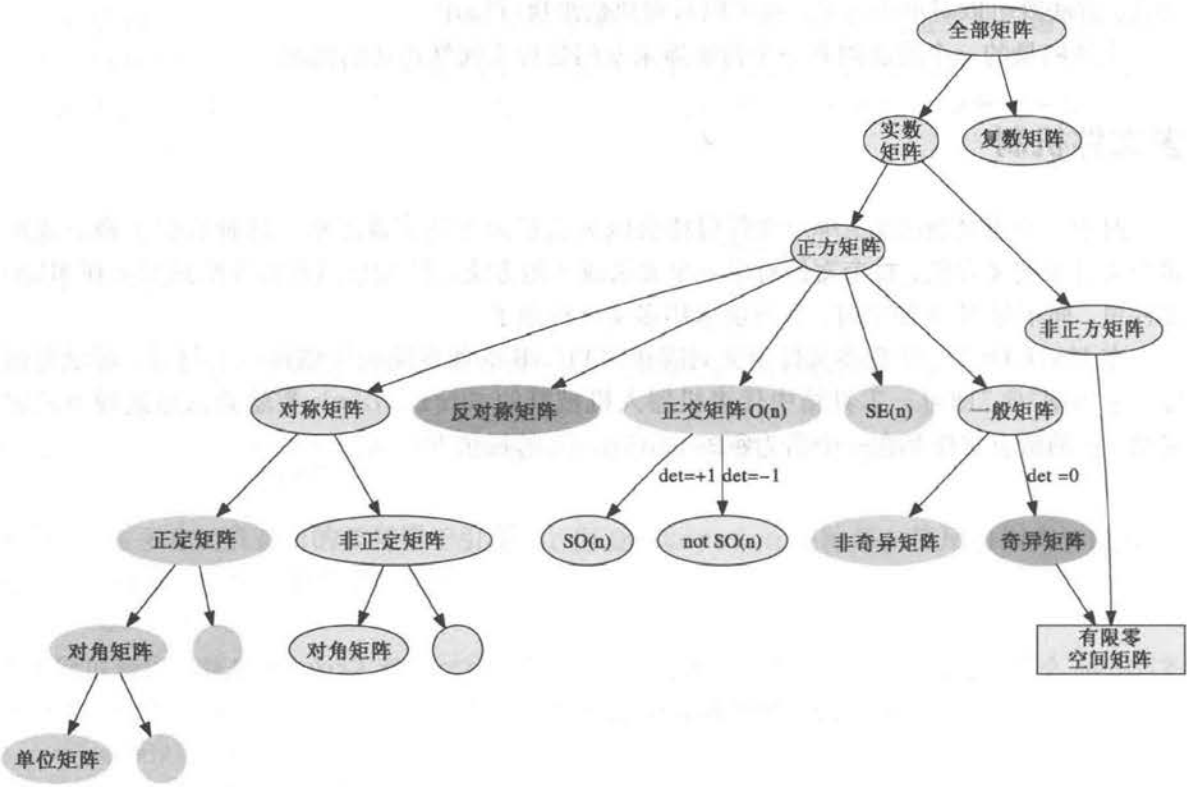


图 D.1　矩阵分类。总是奇异的矩阵用红色表示，总是非奇异的矩阵用蓝色表示

矩阵的转置表达如下：

```
Ai = inv(A)
```

$$\boldsymbol{B} = \boldsymbol{A}^{\mathrm{T}}, b_{i,j} = a_{j,i} \quad \forall i, j$$

并且可以证明下式成立：

$$(\boldsymbol{AB})^{\mathrm{T}} = \boldsymbol{B}^{\mathrm{T}}\boldsymbol{A}^{\mathrm{T}}, (\boldsymbol{ABC})^{\mathrm{T}} = \boldsymbol{C}^{\mathrm{T}}\boldsymbol{B}^{\mathrm{T}}\boldsymbol{A}^{\mathrm{T}}, \text{ 等等}$$

一个正方矩阵如果存在逆矩阵 $\boldsymbol{A}^{-1}$，则有

$$\boldsymbol{A}\boldsymbol{A}^{-1} = \boldsymbol{A}^{-1}\boldsymbol{A} = \boldsymbol{I}_{n \times n}$$

其中，

$$I_{n\times n}=\begin{pmatrix}1 & & 0\\ & 1 & \\ 0 & & 1\end{pmatrix}\in\mathbb{R}^{n\times n}$$

是单位矩阵，即对角线上的值都为 1 的对角矩阵。矩阵存在逆矩阵的条件是该矩阵为非奇异矩阵，即矩阵的行列式 $\det(A)\neq 0$。如果 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 都是正方的非奇异矩阵，那么下列等式成立：

$$(\boldsymbol{AB})^{-1}=\boldsymbol{B}^{-1}\boldsymbol{A}^{-1},(\boldsymbol{ABC})^{-1}=\boldsymbol{C}^{-1}\boldsymbol{B}^{-1}\boldsymbol{A}^{-1},\ \text{等等}$$

并且

$$(\boldsymbol{A}^{\mathrm{T}})^{-1}=(\boldsymbol{A}^{-1})^{\mathrm{T}}$$

对于一个正方矩阵，如果

$\boldsymbol{A}=\boldsymbol{A}^{\mathrm{T}}$　该矩阵为对称矩阵。对称矩阵的逆矩阵也是对称矩阵。我们在机器人学研究中遇到很多矩阵都是对称矩阵，例如协方差矩阵以及机械臂的惯性矩阵。

`S = skew(v)`

$\boldsymbol{A}=-\boldsymbol{A}^{\mathrm{T}}$　该矩阵为反对称矩阵或斜对称矩阵。这种矩阵的对角线上的值都为 0，并具有如下特性 $\boldsymbol{v}^{\mathrm{T}}\boldsymbol{S}(\boldsymbol{v})=0,\ \forall\boldsymbol{v}$。对于 3×3 的矩阵：

$$\boldsymbol{S}(\boldsymbol{v})=\begin{pmatrix}0 & -v_z & v_y\\ v_z & 0 & -v_x\\ -v_y & v_x & 0\end{pmatrix}\qquad(\mathrm{D}.1)$$

上式的逆运算为

`v = vex(s)`

$$\boldsymbol{v}=\mathrm{vex}(\boldsymbol{S})$$

并且有 $\boldsymbol{v}_1\times\boldsymbol{v}_2=\boldsymbol{S}(\boldsymbol{v}_1)\boldsymbol{v}_2$。

$\boldsymbol{A}^{-1}=\boldsymbol{A}^{\mathrm{T}}$　该矩阵为正交矩阵。该矩阵也称为规格化正交矩阵，因为它的所有列向量（或行向量）都是单位向量，并相互正交。两个大小相同的正交矩阵的积还是正交矩阵。$n\times n$ 正交矩阵的集合构成一个群 $O(n)$，称为正交群。一个正交矩阵的行列式只能等于 +1 或 −1。行列式等于 +1 的正交矩阵组成的子群 $SO(n)$ 称为特殊正交群，该群的每个元素都是一个特殊正交矩阵。正交矩阵的所有列（或所有行）都是正交向量，即向量之间的点积等于 0。两个正交矩阵的积仍是正交的。

$\boldsymbol{A}=\boldsymbol{M}^{\mathrm{T}}\boldsymbol{M}$
$\boldsymbol{A}=\boldsymbol{M}\boldsymbol{M}^{\mathrm{T}}$　该矩阵为正规矩阵，它可以被一个正交矩阵 $\boldsymbol{U}$ 变换为对角矩阵，即 $\boldsymbol{U}^{\mathrm{T}}\boldsymbol{A}\boldsymbol{U}$ 为一个对角矩阵。所有对称矩阵、反对称矩阵以及正交矩阵都是正规矩阵。

对于一个非正方矩阵，我们可以求它的广义左逆矩阵，也称伪逆矩阵或 Moore-Penrose 伪逆矩阵。

$$\boldsymbol{A}^{+}\boldsymbol{A}=\boldsymbol{I}_{n\times n}$$

其中，$\boldsymbol{A}^{+}=(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{A})^{-1}\boldsymbol{A}^{\mathrm{T}}$。广义右逆矩阵为

$$\boldsymbol{A}\boldsymbol{A}^{+}=\boldsymbol{I}_{m\times m}$$

其中 $\boldsymbol{A}^{+}=\boldsymbol{A}(\boldsymbol{A}\boldsymbol{A}^{\mathrm{T}})^{-1}$。

正方矩阵 $\mathrm{A}\in\mathbb{R}^{n\times n}$可以用于对一个向量 $\boldsymbol{x}\in\mathbb{R}^{n}$进行线性变换：

$$\boldsymbol{x}'=\boldsymbol{A}\boldsymbol{x}$$

变换结果仍为一个向量，一般是改变了向量的长度和方向。然而有一些特殊情况需要注意。如果 $\boldsymbol{A}\in SO(n)$，则该变换是等距的，即变换前后向量长度不变，$|\boldsymbol{x}'|=|\boldsymbol{x}|$。

在二维平面上，如果 $\boldsymbol{x}$ 是在一个圆周上所有点的集合，则 $\boldsymbol{x}'$就是定义在一个椭圆上点的集合。通过 MATLAB 内置的演示命令：

```
>> eigshow
```

可以非常清楚地看到当拖动 $\boldsymbol{x}$ 向量的末端在一个单位圆周上移动时 `[x,e]=eig(a)` $\boldsymbol{x}'$的变化。

一个正方矩阵的特征向量是向量 $\boldsymbol{x}$，则满足

$$\boldsymbol{A}\boldsymbol{x}=\lambda_i\boldsymbol{x} \tag{D.2}$$

即向量 $\boldsymbol{x}$ 的方向在矩阵变换时不变，它们只是被放大了 λ_i 倍，λ_i 既是向量 $\boldsymbol{x}$ 对应的矩阵特征值。矩阵有 n 个特征值，包括实数和虚数。对于一个正交矩阵，其特征值位于虚平面内的一个单位圆周上，$|\lambda_i|=1$，并且它的特征向量之间都是相互正交的。

如果一个对称矩阵的所有特征值都是正值，则该矩阵为正定矩阵：

$$\lambda_i>0,\ \forall i$$

而如果特征值大于等于 0，则该对称矩阵为半正定矩阵。

$$\lambda_i\geqslant 0,\ \forall i$$

如果 $\boldsymbol{A}$ 为非奇异矩阵，则 $\boldsymbol{A}^{-1}$的特征向量与 $\boldsymbol{A}$ 相同，并且 $\boldsymbol{A}^{-1}$的特征值是 $\boldsymbol{A}$ 的特征值的倒数。$\boldsymbol{A}^{\mathrm{T}}$的特征值与 $\boldsymbol{A}$ 的特征值相同，但特征向量不同。

方程(D.2)的矩阵形式为

$$\boldsymbol{A}\boldsymbol{X}=\boldsymbol{X}\boldsymbol{\Lambda}$$

其中，$\boldsymbol{X}\in\mathbb{R}^{n\times n}$是 $\boldsymbol{A}$ 的特征向量按列构成的矩阵，而 $\boldsymbol{\Lambda}$ 则是由相应的特征值构成的对角矩阵。如果 $\boldsymbol{X}$ 是非奇异矩阵，则上式可重写为

$$\boldsymbol{A}=\boldsymbol{X}\boldsymbol{\Lambda}\boldsymbol{X}^{-1}$$

它表示了矩阵 $\boldsymbol{A}$ 的特征值或谱分解。同时也意味着该矩阵可以通过相似变换变为对角矩阵：

$$\boldsymbol{\Lambda}=\boldsymbol{X}^{-1}\boldsymbol{A}\boldsymbol{X}$$

如果 $\boldsymbol{A}$ 是正规矩阵(如对称矩阵)，那么 $\boldsymbol{X}$ 就是正交矩阵，这时也可以写为

$$\boldsymbol{A}=\boldsymbol{X}\boldsymbol{\Lambda}\boldsymbol{X}^{\mathrm{T}} \tag{D.3}$$

矩阵 $\boldsymbol{A}^{\mathrm{T}}\boldsymbol{A}$ 和 $\boldsymbol{A}\boldsymbol{A}^{\mathrm{T}}$ 总是对称和半正定的，这也意味着任何对称矩阵 $\boldsymbol{A}$ 都可以写成

$$\boldsymbol{A}=\boldsymbol{L}\boldsymbol{L}^{\mathrm{T}}$$

其中 $\boldsymbol{L}$ 代表 $\boldsymbol{A}$ 的 Cholesky 分解。 `L=chol(A)`

如果有矩阵 $\boldsymbol{R}$，且满足

$$\boldsymbol{A}=\boldsymbol{R}\boldsymbol{R}$$

则 $\boldsymbol{R}$ 是 $\boldsymbol{A}$ 的平方根或写成 $\boldsymbol{A}^{1/2}$。

如果 $\boldsymbol{T}$ 是任意非奇异矩阵，且满足

$$\boldsymbol{A}=\boldsymbol{T}\boldsymbol{B}\boldsymbol{T}^{-1}$$

上式称为相似变换，并称 $\boldsymbol{A}$ 与 $\boldsymbol{B}$ 相似，而且可以证明相似变换后矩阵特征值不变。

一个正方矩阵 $\boldsymbol{A}\in\mathbb{R}^{n\times n}$ 的行列式代表了在一个 n 维空间中矩阵变换使体积改变的比例系数。例如，在二维平面上由一系列点 $\boldsymbol{x}_i$ 围成面积为 a 的一个图形，那么由点 $\boldsymbol{Ax}_i$ 所围成图形的面积将是 $a\det(A)$。如果 $\boldsymbol{A}$ 是奇异矩阵，则 $\boldsymbol{Ax}_i$ 的点将位于一个点或一条直线上，围成的面积为零。三维空间中与此类似，矩阵的行列式代表了经过矩阵 $\boldsymbol{A}$ 变换前后一组点所围体积变化的比例系数。一个反对称矩阵的行列式总是等于零，即 $(\boldsymbol{S}(\cdot))=0$。

det(A)

矩阵行列式的值也等于其所有特征值的乘积：

$$\det(\boldsymbol{A})=\prod_{i=1}^{n}\lambda_i$$

因此如果一个矩阵有一个或多个零特征值，则该矩阵奇异。对于一个正定矩阵，由于 $\lambda_i>0$，则有 $\det(A)>0$，因此为非奇异矩阵。矩阵的迹为该矩阵对角元素之和：

trace(A)

$$\mathrm{tr}(\boldsymbol{A})=\sum_{i=1}^{n}\boldsymbol{A}_{ii}$$

它也等于矩阵特征值之和

$$Tr(\boldsymbol{A})=\sum_{i=1}^{n}\lambda_i$$

矩阵的所有列向量 $\boldsymbol{A}=(\boldsymbol{c}_1,\boldsymbol{c}_2,\cdots,\boldsymbol{c}_n)$ 可以被认为是定义“列空间”的向量集。类似地，所有行向量是定义“行空间”的向量集。矩阵的列秩是指 $\boldsymbol{A}$ 中线性独立的列向量的数目，行秩则指 $\boldsymbol{A}$ 中线性独立的行向量的数目。矩阵的行秩和列秩总是相等，因而可以简称为矩阵的秩，而且矩阵秩的上限为 $\min(m,n)$。如果一个正方矩阵的秩 $\mathrm{rank}(\boldsymbol{A})<n$，则称矩阵秩亏或非满秩。

rank(A)

如果矩阵 $\boldsymbol{A}$ 为非满秩阵，那么它就有一个有限零空间或核。一个向量 $\boldsymbol{x}$ 要位于矩阵的零空间内，仅当

$$\boldsymbol{Ax}=\boldsymbol{0}$$

更准确地说，它是位于矩阵的右零空间。一个向量位于左零空间的条件是

$$\boldsymbol{xA}=\boldsymbol{0}$$

矩阵 $\boldsymbol{A}$ 的左零空间等于矩阵 $\boldsymbol{A}^{\mathrm{T}}$ 的右零空间。

矩阵的零空间是由一组正交基向量定义的，这些向量的维数称为矩阵 $\boldsymbol{A}$ 的零度，并等于矩阵的秩 $n-\mathrm{rank}(\boldsymbol{A})$。这些零空间基向量的任意线性组合仍然在零空间内。

null(A)

对于一个非正方矩阵 $\boldsymbol{A}\in\boldsymbol{R}^{m\times n}$，可以得到如下与方程(D.2)相似的式子：

[u,s,Vt]=svd(A)

$$\boldsymbol{Av}_i=\sigma_i\boldsymbol{u}_i$$

其中，$\boldsymbol{u}_i\in\boldsymbol{R}^m$ 和 $\boldsymbol{v}_i\in\boldsymbol{R}^n$ 分别是矩阵 $\boldsymbol{A}$ 的右奇异和左奇异向量，σ_i 是它的奇异值。这些奇异值为非负的实数，等于矩阵 $\boldsymbol{AA}^{\mathrm{T}}$ 特征值的平方根，而 $\boldsymbol{u}_i$ 则是对应的 $\boldsymbol{AA}^{\mathrm{T}}$ 的特征向量。$\boldsymbol{v}_i$ 是矩阵 $\boldsymbol{A}^{\mathrm{T}}\boldsymbol{A}$ 的特征向量。

矩阵 $\boldsymbol{A}$ 的奇异值分解(SVD)表达式为

$$\boldsymbol{A}=\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\mathrm{T}}$$

其中，$\boldsymbol{U}\in\mathbb{R}^{m\times m}$和 $\boldsymbol{V}\in\mathbb{R}^{n\times n}$都是正交矩阵，分别由对应的奇异向量 $\boldsymbol{u}_i$ 和 $\boldsymbol{v}_i$ 按列组合而成。$\boldsymbol{\Sigma}\in\mathbb{R}^{m\times n}$是由奇异值构成的对角矩阵：

$$\Sigma=\begin{pmatrix}\sigma_1 & & & & & \\ & \ddots & & & \mathbf{0} & \\ & & \sigma_r & & & \\ & & & 0 & & \\ & \mathbf{0} & & & \ddots & \\ & & & & & 0\end{pmatrix}$$

其中，$r=\mathrm{rank}(\boldsymbol{A})$是 $\boldsymbol{A}$ 的秩。对于 $r<n$ 的情况，对角线上的其余元素用零补齐，如上式所示。一个矩阵 $\boldsymbol{A}$ 的条件数等于 $\max\sigma_i/\min\sigma_i$，条件数的值越大意味着矩阵越接近于奇异或“病态”。

cond(A)

矩阵的二次型表达式为

$$s=\boldsymbol{x}^{\mathrm{T}}\boldsymbol{A}\boldsymbol{x} \tag{D.4}$$

它是一个标量。对于 $\boldsymbol{A}$ 是对角矩阵的情形，上式可改写为

$$s=\sum_{i=1}^{n}A_{ii}x_i^2$$

它是一个加权平方和。如果 $\boldsymbol{A}$ 是对称矩阵，则有

$$s=\sum_{i=1}^{n}A_{ii}x_i^2+2\sum_{i=1}^{n}\sum_{j=i+1}^{n}A_{ij}x_ix_j$$

该结果也包含了 $\boldsymbol{x}$ 中元素之间的积或相关。

实数矩阵是全部矩阵的一个子集，与其相对的是复数矩阵。对于一般的复数矩阵而言，与实对称矩阵对应的是厄米特(Hermitian)矩阵，与实正交矩阵对应的是酉矩阵。$\boldsymbol{A}^{\mathrm{H}}$ 表示厄米特转置，即对复数矩阵 $\boldsymbol{A}$ 进行复共轭转置。

求解方程组

我们经常需要求解线性方程组：

$$\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}$$

其中，$\boldsymbol{A}\in\mathbb{R}^{n\times m}$和 $\boldsymbol{b}\in\mathbb{R}^{n}$已知，而 $\boldsymbol{x}\in\mathbb{R}^{m}$是未知量。如果 $n=m$，则 $\boldsymbol{A}$ 为正方矩阵，并且如果 $\boldsymbol{A}$ 是非奇异的，那么方程组的解就可以通过矩阵求逆得到：

$$\boldsymbol{x}=\boldsymbol{A}^{-1}\boldsymbol{b}$$

如果 $n>m$，则方程组过约束，我们将使用矩阵伪逆求解：

$$\boldsymbol{x}=\boldsymbol{A}^{+}\boldsymbol{b}$$

上式得到的解 $\boldsymbol{x}$ 将使误差范数 $|\boldsymbol{A}\boldsymbol{x}-\boldsymbol{b}|$ 的值最小。如果对 $\boldsymbol{A}$ 使用 SVD，则 $\boldsymbol{A}=\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\mathrm{T}}$，上式将变为

$$\boldsymbol{x}=\boldsymbol{V}\boldsymbol{\Sigma}^{-1}\boldsymbol{U}^{\mathrm{T}}\boldsymbol{b}$$

其中，求 $\boldsymbol{\Sigma}^{-1}$只是简单地对矩阵 $\boldsymbol{\Sigma}^{\mathrm{T}}$ 对角线上的元素取倒数即可。

如果矩阵 $\boldsymbol{A}$ 奇异，或者方程组欠约束，即 $n<m$，那么方程组有无穷多解。可以再用 SVD

方法：

$$\boldsymbol{x} = \boldsymbol{V}\boldsymbol{\Sigma}^{-1}\boldsymbol{U}^{\mathrm{T}}\boldsymbol{b}$$

其中，$\boldsymbol{\Sigma}^{-1}$这次是对矩阵 $\boldsymbol{\Sigma}$ 的非零对角元素取倒数，而其余零元素位置不变。

在 MATLAB 中，上述方程组求解的所有问题都可以用反斜杠算子来解决：

```
>> x = A\b
```

奇异值分解还可以用于估算一个旋转矩阵，即给定一组向量$\{(\boldsymbol{p}_i, \boldsymbol{q}_i), i = 1\cdots N\}$，求旋转矩阵 $\boldsymbol{R}$ 使得 $\boldsymbol{q}_i = \boldsymbol{R}\boldsymbol{p}_i$。首先计算矩量矩阵：

$$\boldsymbol{M} = \sum_{i=1}^{N} \boldsymbol{q}_i \boldsymbol{p}_i^{\mathrm{T}}$$

然后将 $\boldsymbol{M}$ 进行 SVD，得 $\boldsymbol{M} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\mathrm{T}}$。最后得到旋转矩阵的最小二乘估计为

$$\boldsymbol{R} = \boldsymbol{U}\boldsymbol{V}^{\mathrm{T}}$$

并且可以保证它是一个正交矩阵。

附录E 椭　圆

椭圆属于平面曲线中的圆锥曲线。一个椭圆最简单的隐式表达式为

$$\frac{x^2}{a^2}+\frac{y^2}{b^2}=1$$

其图形如图 E.1(a)所示。这个标准椭圆是以坐标原点为中心，长轴和短轴分别位于 x 轴和 y 轴上。该椭圆在 x 轴方向的曲率半径为 a，在 y 轴方向的曲率半径为 b。a 和 b 中较大者为椭圆长半轴长，较短者为短半轴长。

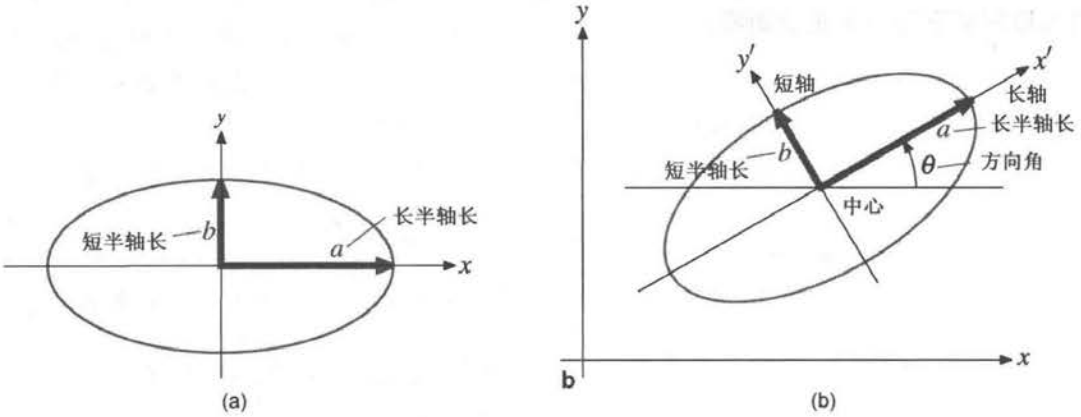


图 E.1　椭圆。(a)标准椭圆是以坐标原点为中心，主轴与 x 轴和 y 轴重合；(b)椭圆的一般形式

也可以用方程(D.4)的矩阵二次型来描述椭圆方程：

$$\begin{pmatrix}x & y\end{pmatrix}\begin{pmatrix}1/a^2 & 0\\ 0 & 1/b^2\end{pmatrix}\begin{pmatrix}x\\ y\end{pmatrix}=1$$

$$\boldsymbol{x}^{\mathrm{T}}\begin{pmatrix}a^2 & 0\\ 0 & b^2\end{pmatrix}^{-1}\boldsymbol{x}=1 \tag{E.1}$$

$$\boldsymbol{x}^{\mathrm{T}}\boldsymbol{E}^{-1}\boldsymbol{x}=1 \tag{E.2}$$

方程(E.2)为圆锥曲线的最一般表达式，其中 $\boldsymbol{E}$ 为对称矩阵：

$$\boldsymbol{E}=\begin{pmatrix}A & C\\ C & B\end{pmatrix} \tag{E.3}$$

它的行列式 $\det(E)=AB-C^2$ 决定了圆锥曲线的类型：

$$\det(\boldsymbol{E})\begin{cases}>0 & \text{椭圆}\\ =0 & \text{抛物线}\\ <0 & \text{双曲线}\end{cases}$$

因此一个椭圆是由一个正定对称矩阵 $\boldsymbol{E}$ 来描述的。反过来，任意正定对称矩阵，如一个惯性矩阵或一个协方差矩阵，也可以用一个椭圆来描述。

非零值 C 决定了椭圆的方向。椭圆的中心也可任意改变，通过下式就能将它的中心定在 $\boldsymbol{x}_c$：

$$(\boldsymbol{x}-\boldsymbol{x}_c)^{\mathrm{T}}\boldsymbol{E}^{-1}(\boldsymbol{x}-\boldsymbol{x}_c)=1$$

上式对应的椭圆一般形式如图 E.1(b)所示。

因为 $\boldsymbol{E}$ 是对称矩阵，可以用方程(D.3)将其对角化：

$$\boldsymbol{E}=\boldsymbol{X}\boldsymbol{\Lambda}\boldsymbol{X}^{\mathrm{T}}$$

其中 $\boldsymbol{X}$ 是一个包含 $\boldsymbol{E}$ 的特征向量的正交矩阵。$\boldsymbol{E}$ 的逆矩阵为

$$\boldsymbol{E}^{-1}=\boldsymbol{X}\boldsymbol{\Lambda}^{-1}\boldsymbol{X}^{\mathrm{T}}$$

因此，二次型方程(E.2)变为

$$\begin{aligned}\boldsymbol{x}^{\mathrm{T}}\boldsymbol{X}\boldsymbol{\Lambda}^{-1}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{x}&=1\\(\boldsymbol{X}^{\mathrm{T}}\boldsymbol{x})^{\mathrm{T}}\boldsymbol{\Lambda}^{-1}(\boldsymbol{X}^{\mathrm{T}}\boldsymbol{x})&=1\\\boldsymbol{x}'^{\mathrm{T}}\boldsymbol{\Lambda}^{-1}\boldsymbol{x}'&=1\end{aligned}$$

上式与方程(E.2)相似，只是这时椭圆已改为由对角矩阵 $\boldsymbol{\Lambda}$ 定义，而且参考坐标系进行了旋转 $\boldsymbol{x}'=\boldsymbol{X}^{\mathrm{T}}\boldsymbol{x}$。椭圆的长轴和短轴与矩阵 $\boldsymbol{E}$ 的特征向量共线，椭圆半轴长的平方等于矩阵 $\boldsymbol{E}$ 的特征值或矩阵 $\boldsymbol{\Lambda}$ 的对角线元素。

对于 $\boldsymbol{E}\in\mathbb{R}^{n\times n}$ 的一般情况，方程(E.2)的结果是在 n 维空间中的椭球体。工具箱函数 `plot_ellipse` 可以画出 $n=2$ 时的椭圆，以及 $n=3$ 时的椭球。

另外，椭圆还可以用多项式描述。我们把椭圆的矩阵形式写为

$$(\boldsymbol{x}-(x_0,y_0))^{\mathrm{T}}\begin{pmatrix}a & c\\ c & b\end{pmatrix}(\boldsymbol{x}-(x_0,y_0))=1$$

将上式展开，得到

$$e_1x^2+e_2y^2+e_3xy+e_4x+e_5y+e_6=0$$

其中，$e_1=a$，$e_2=b$，$e_3=2c$，$e_4=-2(ax_0+cy_0)$，$e_5=-2(by_0+cx_0)$，以及 $e_6=ax_0^2+by_0^2+2cx_0y_0-1$。这个椭圆只有 5 个自由的，包括 2 个中心坐标和 3 个矩阵 $\boldsymbol{E}$ 中的独立元素。对于一个非退化的椭圆，即 $e_1\neq 0$，我们可以将它的多项式正规化：

$$x^2+E_1y^2+E_2xy+E_3x+E_4y+E_5=0 \tag{E.4}$$

可以看出，上式有 5 个独立参数。

特性

椭圆的面积等于 π，偏心率为

$$\varepsilon=\frac{\sqrt{a^2-b^2}}{a}$$

矩阵 $\boldsymbol{E}$ 的特征向量决定了椭圆的主轴方向，而特征值的平方根为相应的半轴长度。

以下式描述的椭圆为例：

$$\boldsymbol{x}\begin{pmatrix}2 & -1\\ -1 & 1\end{pmatrix}^{-1}\boldsymbol{x}=1$$

该椭圆在 MATLAB 中表示为

```
>> E = [2 -1; -1 1];
```

用下面的命令可以画出椭圆图形，如图 E.2 所示。

```
>> plot_ellipse(E)
```

求 $\boldsymbol{E}$ 的特征向量和特征值为

```
>> [x,e] = eig(E)
x =
   -0.5257   -0.8507
   -0.8507    0.5257
e =
    0.3820         0
         0    2.6180
```

求椭圆的半轴长为

```
>> r = sqrt(diag(e))
r =
    0.6180
    1.6180
```

以上两值分别对应 b 和 a。如果其中一个半轴长等于零，则椭圆退化成为一条线。如果两个值都为零，则椭圆变成一个点。

特征向量是在长轴和短轴方向上的单位向量，我们可以用半轴长将这两个特征向量放大，从而得到两个半轴向量，而且可以画出它们的图形：

```
>> arrow([0 0]', x(:,1)*r(1));
>> arrow([0 0]', x(:,2)*r(2));
```

椭圆的方向角是其长轴与水平轴之间的夹角，表达式为

$$\theta = \arctan\frac{x_y}{x_x}$$

对于上面给出的椭圆例子，方向角可以用以下命令求出：

```
>> atan2(x(2,2), x(1,2)) * 180/pi
ans =
  148.2825
```

该结果的单位是度。

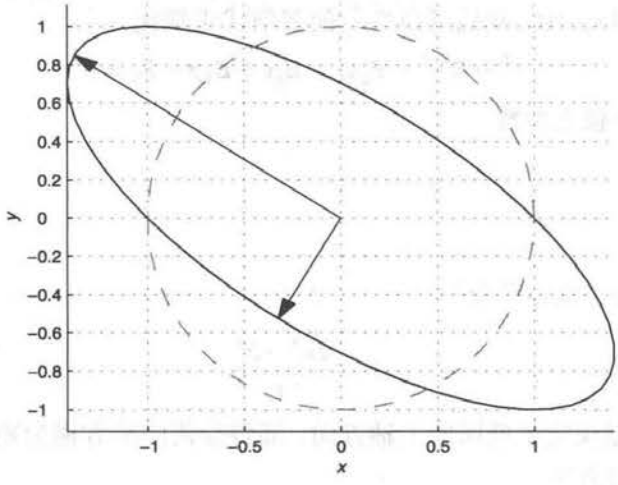


图 E.2 对应于 2×2 对称矩阵的椭圆，虚线代表单位半径的圆周，箭头表示椭圆的长轴和短轴向量

椭圆的面积为 $\pi r_1 r_2$，椭球体的体积为 $\frac{4}{3}\pi r_1 r_2 r_3$，其中半径 $r_i = \sqrt{\lambda_i}$，λ_i 为矩阵 $\boldsymbol{E}$ 的特征值。由于矩阵行列式 $\det(\boldsymbol{E}) = \Pi\lambda_i$，所以面积或体积与 $\sqrt{\det(\boldsymbol{E})}$ 成正比。

画椭圆

为了画出一个椭圆，首先需要在单位圆周上确定一个点 $\boldsymbol{y}=[x, y]^{\mathrm{T}}$，并有

$$\boldsymbol{y}^{\mathrm{T}}\boldsymbol{y}=1$$

然后重写方程(E.3)为

$$\boldsymbol{x}^{\mathrm{T}}\boldsymbol{E}^{-\frac{1}{2}}\boldsymbol{E}^{-\frac{1}{2}}\boldsymbol{x}=1$$

其中，$\boldsymbol{E}^{1/2}$是矩阵平方根(可用 MATLAB 的函数 sqrtm 得到)。将上面两个方程取等，得到

$$\boldsymbol{x}^{\mathrm{T}}\boldsymbol{E}^{-\frac{1}{2}}\boldsymbol{E}^{-\frac{1}{2}}\boldsymbol{x}=\boldsymbol{y}^{\mathrm{T}}\boldsymbol{y}$$

显然，从上式可以得到

$$\boldsymbol{y}=\boldsymbol{E}^{-\frac{1}{2}}\boldsymbol{x}$$

将上式改写为

$$\boldsymbol{x}=\boldsymbol{E}^{\frac{1}{2}}\boldsymbol{y}$$

它代表了将单位圆周上的一个点变换到椭圆上的一点。如果椭圆是以 $\boldsymbol{x}_c$ 为中心，而非坐标原点，我们还可以做一次坐标改变：

$$(\boldsymbol{x}-\boldsymbol{x}_c)^{\mathrm{T}}\boldsymbol{E}^{-\frac{1}{2}}\boldsymbol{E}^{-\frac{1}{2}}(\boldsymbol{x}-\boldsymbol{x}_c)=1$$

从上式可以得到总的变换方程为

$$\boldsymbol{x}=\boldsymbol{E}^{\frac{1}{2}}\boldsymbol{y}+\boldsymbol{x}_c$$

继续以前面给出的椭圆为例，其描述为

```
>> E = [2 -1; -1 1];
```

我们定义一系列在单位圆周上的点：

```
>> th = linspace(0, 2*pi, 50);
>> y = [cos(th);  sin(th)];
```

然后将这些点变换到椭圆的圆周上：

```
>> x = (sqrtm(E) * y)';
>> plot(x(:,1), x(:,2));
```

其中包含了工具箱函数 sqrtm。最后画出椭圆：

```
>> plot_ellipse(E, [0 0])
```

一个椭球体是由一个 3 × 3 的正定对称矩阵定义的。椭球体的画法与椭圆类似，而且命令 plot_ellipse 也可以画出三维椭球体。

椭圆拟合

从一系列内部点拟合

给定椭圆边界内的一系列点数据，我们希望能拟合出一个最佳椭圆方程。一般的方法是找出一个与给定点集有相同质量特性的椭圆。对于 N 个点 $\boldsymbol{x}_i(x_i, y_i)$ 的集合，可以计算它们的几何矩：

$$m_{00} = N$$

$$m_{10} = \sum_{i=1}^{N} x_i$$

$$m_{01} = \sum_{i=1}^{N} y_i$$

椭圆的中心可以认为是在这些点集的质心：

$$(x_c, y_c) = \left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}}\right)$$

在此基础上可以进一步计算二阶中心矩：

$$\mu_{20} = \sum_{i=1}^{N} (x_i - x_c)^2$$

$$\mu_{02} = \sum_{i=1}^{N} (y_i - y_c)^2$$

$$\mu_{11} = \sum_{i=1}^{N} (x_i - x_c)(y_i - y_c)$$

一个一般椭圆的惯性矩阵是一个对称矩阵：

$$\boldsymbol{J} = \begin{pmatrix} \mu_{20} & \mu_{11} \\ \mu_{11} & \mu_{02} \end{pmatrix}$$

其中，对角项是惯性矩，而非对角项为惯性积。惯性可以更直接用下式计算：

$$\boldsymbol{J} = \sum_{i=1}^{N} (\boldsymbol{x} - \boldsymbol{x}_c)(\boldsymbol{x} - \boldsymbol{x}_c)^{\mathrm{T}}$$

惯性矩阵与椭圆对称矩阵之间的关系为

$$\boldsymbol{E} = \frac{4}{m_{00}} \boldsymbol{J}$$

为了演示上述方法，我们可以仍旧用前面给的椭圆例子。一系列的点数据可以在椭圆内任意取。计算程序如下：

```
% generate a set of points within the ellipse
p = [];
while true
    x = (rand(2,1)-0.5)*4;
    if norm(x'*inv(E)*x) <= 1
        p = [p x];
    end
    if numcols(p) >= 500
        break;
    end
end
plot(p(1,:), p(2,:), '.')

% compute the moments
m00 = mpq_point(p, 0,0);
m10 = mpq_point(p, 1,0);
m01 = mpq_point(p, 0,1);
xc = m10/m00; yc = m01/m00;

% compute second moments relative to centroid
pp = bsxfun(@minus, p, [xc; yc]);

```

```
m20 = mpq_point(pp, 2,0);
m02 = mpq_point(pp, 0,2);
m11 = mpq_point(pp, 1,1);

% compute the moments and ellipse matrix
J = [m20 m11; m11 m02];
E_est = 4 * J / m00
```

矩阵 E 的估算结果为

```
>> E_est
E_est =
    0.9776    0.9395
    0.9395    1.8976
```

以上结果与 E 的原始值很接近。点数据分布如图 E.3 所示。我们可以将拟合的椭圆画出并覆盖在点数据上：

```
>> plot_ellipse(E_est, [xc yc],  'r')
```

结果如图 E.3 中的曲线。

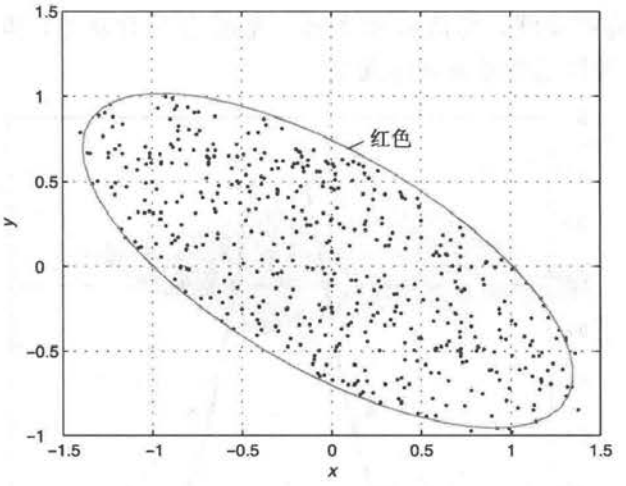


图 E.3 点数据

从一系列边界点拟合

给定椭圆边界圆周上一系列点(x_i, y_i)，我们希望拟合出椭圆方程。这时，对于边界上每个点都可以写出方程(E.4)形式的椭圆多项式，所有多项式用矩阵表达为

$$\begin{pmatrix} y_1^2 & x_1y_1 & x_1 & y_1 & 1 \\ y_2^2 & x_2y_2 & x_2 & y_2 & 1 \\ & & \vdots & & \\ y_N^2 & x_Ny_N & x_N & y_N & 1 \end{pmatrix} \begin{pmatrix} E_1 \\ E_2 \\ E_3 \\ E_4 \\ E_5 \end{pmatrix} = \begin{pmatrix} -x_1^2 \\ -x_2^2 \\ \vdots \\ -x_N^2 \end{pmatrix}$$

当点数 $N \geqslant 5$ 时，就可以求解出椭圆的参数向量了。

附录 F　高斯随机变量

一维高斯函数表达式为

$$g(x)=\frac{1}{\sqrt{\sigma^2 2\pi}}e^{-\frac{1}{2}(x-\mu)^2\frac{1}{\sigma^2}} \tag{F.1}$$

对应的曲线由其峰位置 μ 和宽度 σ 决定。曲线下的总面积为单位 1，而且有 $g(x)>0$，$\forall x$。

该函数曲线可以用工具箱函数 gaussfunc 绘制：

```
>> x = linspace(-6, 6, 500);
>> plot(x, gaussfunc(0, 1, x) )
>> hold on
>> plot(x, gaussfunc(0, 2^2, x), '--' )
```

图 F.1 中画出了两条高斯曲线，均值 μ 都为零，宽度分别为 $\sigma=1$ 和 $\sigma=2$。注意，给函数 gaussfunc 的第二个参数是方差而非标准差。

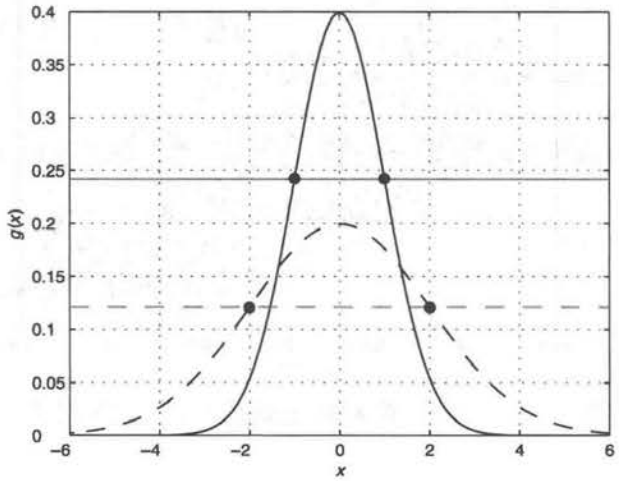


图 F.1　两条高斯函数曲线，均值都为 $\mu=0$，标准差分别为 $\sigma=1$（实线）和 $\sigma=2$（虚线）。4 个小圆点的位置在 $x=\mu\pm1\sigma$。虚线比实线的曲线更宽，但高度更低，这是因为曲线下总的面积都为单位1

如果将高斯函数看作一个概率密度函数（PDF），那么它就是标准的正态分布，其峰位置 μ 代表均值，宽度 σ 代表标准差。一个正态分布的随机变量通常记为 $X\sim N(\mu,\sigma^2)$，因而 $N(0,1)$ 就代表标准正态分布。一个随机值落入区间 $x\in[x_1,x_2]$ 中的概率可以用以下积分式获得：

$$P=\int_{x_1}^{x_2}g(x)\mathrm{d}x=\Phi(x_2)-\Phi(x_1)$$

上式也称累积分布函数 $\Phi(x)$ 评估。图 F.1 中位于 $\mu\pm1\sigma$ 的小圆点界定了 1σ 的置信区间。在该置信区间内的曲线包含的面积为 0.68，因此一个随机数来自于该区间的概率就是 68%。

n 维高斯函数，或多变量正态分布函数的表达式为

$$g(\boldsymbol{x})=\frac{1}{\sqrt{\det(\boldsymbol{C})(2\pi)^{n/2}}}e^{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^T\boldsymbol{C}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})} \tag{F.2}$$

与方程(F.1)的一维函数相比，x 和 μ 从标量变成了 n 维向量，指数上的平方项变成了矩阵二次型，而方差 σ^2 也变成协方差矩阵 $\boldsymbol{C}$。$\boldsymbol{C}$ 中对角线上的元素代表 x_i 的方差，其余元素 $\boldsymbol{C}_{ij}$ 则代表 x_i 和 x_j 之间的相关性。如果变量之间不相关，则矩阵 $\boldsymbol{C}$ 变为对角矩阵。协方差矩阵是对称且正定的。

可以用以下命令绘制一个二维高斯函数曲线：

```
>> [x,y] = meshgrid(-5:0.1:5, -5:0.1:5);
>> C = diag([1 2^2]);
>> g = gaussfunc([0 0], C, x, y) ;
>> axis([-5 5 -5 5 -.05 .12]);  hold on
>> surfc(x, y, g)
```

画出的曲面如图 F.2 所示。在这个高斯函数中，$\mu=(0,0)$，而 $\boldsymbol{C}=\mathrm{diag}(1^2, 2^2)$，说明它的两个变量是标准差分别为 1 和 2 的不相关变量。在图 F.2 中的 xy 平面上还画出很多表示曲面外形的轮廓线，可以看出它们都是椭圆形的，而且沿 y 轴和 x 轴方向的半轴长之比为 2∶1，与标准差之比相同。

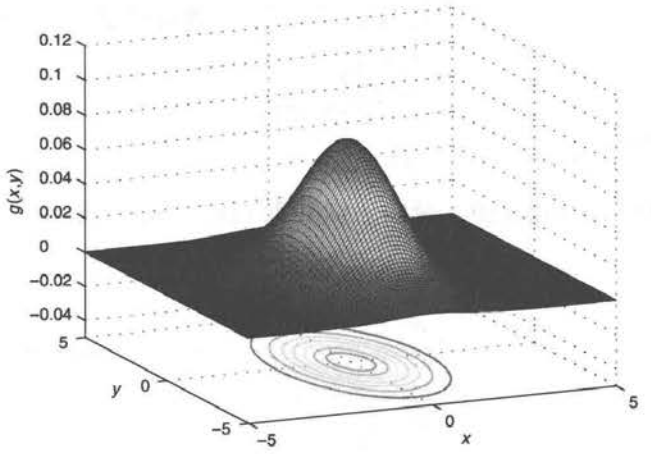


图 F.2　具有协方差矩阵 $\boldsymbol{C}=\mathrm{diag}(1^2,2^2)$ 的二维高斯曲面。在 xy 平面上画出了具有不变概率密度的轮廓线

再观察方程(F.2)中的指数项，发现它正好是一个椭圆方程：

$$(\boldsymbol{x}-\mu)^{\mathrm{T}}\boldsymbol{C}^{-1}(\boldsymbol{x}-\mu)=1$$

满足上式的所有点会得到一个不变的概率密度值，即对应于 1σ 边界的一条椭圆轮廓线。对于 $n>2$ 的高阶高斯函数，对应的置信区间将是 n 维空间中的一个椭球体面。

假设这个二维概率密度函数代表了一个机器人在 xy 平面上的位置，那么这个机器人最有可能在的位置是(0,0)，而且它位于以 1σ 为边界的椭圆内的概率为 68%。

```
>> plot_ellipse(C, [0 0])
```

用以上命令可以画出该椭圆，它的大小体现了机器人位置在空间上的确定性。一个较大的椭圆意味着 68% 的概率存在于一个较大的面积上，而较小的椭圆则代表了相同的概率却在较小的面积上。衡量椭圆的大小可以用附录 E 中讨论过的矩阵行列式 $\det(C)$。另外由于 y 轴方向的半轴长较长，因此也可以说在 y 轴方向的不确定性要大于在 x 轴方向的不确定性。

在本书第 6 章讨论的机器人定位估值滤波时，通常就用一个椭圆来代表机器人不确定区域。如果协方差矩阵是对角的，那么这个椭圆的主轴就像在附录 E 中看到的是与 x 轴和 y 轴重合的。这也表明它的两个变量是独立的，具有零相关性。反过来，主轴不与 x 轴和 y 轴重合的椭圆其协方差矩阵就不是对角的，而且两个变量也是相关的。

如果 $\mathbb{R}^n$ 是多元高斯曲面上的一点，则它到点 μ 的距离为

$$d = \sum_{i=1}^{n} (x_i - \mu_i)^2$$

这个标量值呈现出一种有 n 自由度的卡方分布：

$$d \sim \chi^2(n)$$

其马氏距离为

$$d_M = (\boldsymbol{x} - \mu)^{\mathrm{T}} \boldsymbol{C}^{-1} (\boldsymbol{x} - \mu)$$

该标量值则表示了点 $\boldsymbol{x}$ 相对于 μ 和 $\boldsymbol{C}$ 分布的差异程度。

附录 G　雅克比矩阵

$f:\mathbb{R}^n\to\mathbb{R}$ 是一个自变量为向量的标量值函数，对它求关于向量 $\boldsymbol{x}$ 的导数可得到一个向量：

$$\nabla f(\boldsymbol{x})=\begin{pmatrix}\frac{\partial f}{\partial x_1}\\ \frac{\partial f}{\partial x_2}\\ \vdots\\ \frac{\partial f}{\partial x_n}\end{pmatrix}$$

该向量指向使函数 $f(\boldsymbol{x})$ 具有最大增量的方向。它也经常被记为 $\nabla_x f$，以表明它是相对于 x 的微分。

一个自变量为向量的向量值函数 $\boldsymbol{f}:\mathbb{R}^m\to\mathbb{R}^n$ 可表示为

$$\boldsymbol{f}(\boldsymbol{x})=\begin{pmatrix}f_1(\boldsymbol{x})\\ f_2(\boldsymbol{x})\\ \vdots\\ f_n(\boldsymbol{x})\end{pmatrix}$$

其中，$f_i:\mathbb{R}^m\to\mathbb{R}$ 取 $i\in\{1,2,\cdots,n\}$。对 f 求相对于向量 $\boldsymbol{x}$ 的导数可以得到一个矩阵，称为雅克比矩阵：

$$J=\begin{pmatrix}\frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n}\\ \vdots & \ddots & \vdots\\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n}\end{pmatrix}$$

它也经常被记为

$$\boldsymbol{J}(\boldsymbol{x})=\begin{pmatrix}\nabla f_1^{\mathrm{T}}\\ \nabla f_2^{\mathrm{T}}\\ \vdots\\ \nabla f_n^{\mathrm{T}}\end{pmatrix}$$

这个导数矩阵也称为 f 的切映射，记为 $\boldsymbol{Tf}$，或者叫作 $\boldsymbol{f}$ 的微分，记为 $\boldsymbol{Df}$。为了明确表示该微分是相对于向量 $\boldsymbol{x}$ 的，上述符号也被写为 $\boldsymbol{J}_x$，$\boldsymbol{T}_x\boldsymbol{f}$，$\boldsymbol{D}_x\boldsymbol{f}$，甚至用 $\partial\boldsymbol{f}/\partial\boldsymbol{x}$ 表示。很多优化算法以及扩展卡尔曼滤波都需要用到函数的雅克比矩阵，而且对该矩阵可以进行数值和解析分析。

以方程(6.9)表示的函数为例，在给定小车位姿和路标位置时，它可以求路标的距离和方位角。可以将该函数写成一个简单的 MATLAB 函数：

```
1 function z = zrange(xi, xv, w)
2       z = [ sqrt((xi(1)-xv(1))^2 + (xi(2)-xv(2))^2) + w(1);
3             atan((xi(2)-xv(2))/(xi(1)-xv(1)))-xv(3) + w(2) ];
```

为了估算在 $\boldsymbol{x}_v=\left(1,2,\frac{\pi}{3}\right)$ 和 $\boldsymbol{x}_i=(10,8)$ 时它的雅克比矩阵 $\boldsymbol{H}_{xv}=\partial\boldsymbol{h}/\partial\boldsymbol{x}_v$，我们可以计算它的一阶数值微分：

```
>> xv = [1, 2, pi/3]; xi = [10, 8]; w= [0,0];
>> h0 = zrange(xi, xv, w)
```

```
h0 =
   10.8167
   -0.4592
>> d = 0.001;
>> J = [ zrange(xi, xv+[1,0,0]*d, w)-h0 ...
         zrange(xi, xv+[0,1,0]*d, w)-h0, ...
         zrange(xi, xv+[0,0,1]*d,w)-h0]  /  d
J =
   -0.8320   -0.5547         0
    0.0513   -0.0769   -1.0000
```

该雅克比矩阵与方程(6.4)所给出的雅克比矩阵有相同特征的尾列。注意，在计算该雅克比矩阵时我们已将测量噪声 w 设为了零。这种数值方法的主要难点在于选择 d 的值，它是将求导运算用有限差分近似计算时的步长。这个值如果太大，那么在函数为非线性时计算结果将非常不准；如果太小则会导致数值计算上的问题，也会降低结果的准确性。

另一种选择是用解析法求微分的符号表达式。对于方程(6.9)的函数，由于它相对比较简单，所以用微分学知识能很容易推出其微分解析式。当然，可以用数值求导来快速检验解析表达式的正确性。为了避免推导出错，或者求解更复杂的函数，我们可以选用一款计算机代数软件包来推导函数微分的解析表达式。这里我们用 MATLAB 的符号运算工具箱(Symbolic Math Toolbox)来求方程(6.9)函数的微分。我们可以先定义一些符号变量：

```
>> syms xi yi xv yv thetav wr wb
```

然后给出函数表达式：

```
>> z = zrange([xi yi], [xv yv thetav], [wr wb])
z =
        wr + ((xi - xv)/(yi - yv)^2)^(1/2)
 wb - thetav + atan((yi - yv)/(xi - xv))
```

它就是方程(6.9)的 MATLAB 符号形式。雅克比矩阵可以用一个符号运算工具箱中的函数来计算：

```
>> J = jacobian(z, [xv yv thetav])
J =
[ -(2*xi - 2*xv)/(2*((xi - xv)^2 + (yi - yv)^2)^(1/2)),
    -(2*yi - 2*yv)/(2*((xi - xv)^2 + (yi - yv)^2)^(1/2)),  0]
[ (yi - yv)/((xi - xv)^2*((yi - yv)^2/(xi - xv)^2 + 1)),
    -1/((xi - xv)*((yi - yv)^2/(xi - xv)^2 + 1)), -1]
```

矩阵的维数是

```
>> about(J)
J [sym] : 2x3 (60 bytes)
```

其尾列的特征也很明显。可以将这段代码剪切并粘贴到我们自己的程序中，或者自动创建一个 MATLAB 可调用的函数：

```
>> Jf = matlabFunction(J);
```

其中，Jf 是一个 MATLAB 的函数句柄。现在，可以计算前面给出的具体作业点上的雅克比矩阵：

```
>> xv = [1, 2, pi/3]; xi = [10, 8]; w= [0,0];
>> Jf( xi(1), xv(1), xi(2), xv(2) )
ans =
   -0.8321   -0.5547         0
    0.0513   -0.0769   -1.0000
```

该结果与前面得到的数值近似解接近。函数 matlabFunction 也可以写成 M 文件格式的函数。用函数 ccode 和 fcode 就能分别生成 C 语言和 fortran 语言形式的雅克比矩阵。

另一种有趣的方法是使用 ADOL-C 软件包，它是用于自动求解 C 和C++程序函数微分的开源工具，即给它一个 C 语言的函数，它能返回一个 C 语言的雅克比矩阵函数。该软件工具可以在 http://www.coin-or.org/projects/ADOL-C.xml 获取。

附录 H　卡尔曼滤波

分析下面的离散线性时不变系统：

$$\begin{aligned}\boldsymbol{x}\langle k+1\rangle &= \boldsymbol{F}\boldsymbol{x}\langle k\rangle + \boldsymbol{G}\boldsymbol{u}\langle k\rangle + \boldsymbol{v}\langle k\rangle \\ \boldsymbol{z}\langle k+1\rangle &= \boldsymbol{H}\boldsymbol{x}\langle k\rangle + \boldsymbol{w}\langle k\rangle\end{aligned}$$

其状态变量为向量 $\boldsymbol{x}\in\mathbb{R}^n$。向量 $\boldsymbol{u}\in\mathbb{R}^m$ 是系统在 k 时刻的输入，如一个速度指令，或施加的力和力矩。向量 $\boldsymbol{z}\in\mathbb{R}^p$ 则代表系统的输出，可用传感器测得。矩阵 $\boldsymbol{F}\in\mathbb{R}^{n\times n}$ 表示系统的动态特性，即系统状态如何随时间演变。矩阵 $\boldsymbol{G}\in\mathbb{R}^{n\times m}$ 表示系统输入与系统状态之间的耦合关系。矩阵 $\boldsymbol{H}\in\mathbb{R}^{p\times n}$ 则代表系统状态与被观测输出的映射关系。

为了体现模型（矩阵 $\boldsymbol{F}$ 和 $\boldsymbol{G}$ 代表）中存在的误差以及未被建模的干扰成分，我们引入一个称为过程噪声的高斯随机变量 $\boldsymbol{v}\in\mathbb{R}^n$。该变量满足 $\boldsymbol{v}\langle k\rangle \sim N(0, \boldsymbol{V})$，即它的均值为零，并有协方差矩阵 $\boldsymbol{V}$。传感器的测量模型 $\boldsymbol{H}$ 也存在误差，它可以用另一个称为测量噪声的高斯随机变量 $\boldsymbol{w}\in\mathbb{R}^p$ 来描述，并有 $\boldsymbol{w}\langle k\rangle \sim N(0, \boldsymbol{W})$。协方差矩阵 $\boldsymbol{V}\in\mathbb{R}^{n\times n}$ 和 $\boldsymbol{W}\in\mathbb{R}^{p\times p}$ 都是对称和正定的。

我们遇到的一般问题是：

给定一个系统模型，已知系统输入 $\boldsymbol{u}$ 以及一些噪声的传感器测量值 $\boldsymbol{z}$，要估计出系统的状态 $\boldsymbol{x}$。

在机器人定位时，$\boldsymbol{x}$ 就代表了机器人未知的位置和姿态量，$\boldsymbol{u}$ 代表发送给电机的命令，而 $\boldsymbol{z}$ 就是机器人上各种传感器的输出值。对于一个飞行机器人，$\boldsymbol{x}$ 可能是它的高度，$\boldsymbol{u}$ 是施加在机身上的已知外力，$\boldsymbol{z}$ 是测量出的加速度和角速度值。

卡尔曼滤波实际上是针对过程和测量噪声均为零均值高斯噪声时的最优估计器。滤波过程分为两步，首先是基于先前系统状态和施加的输入预测出系统状态。

$$\hat{\boldsymbol{x}}\langle k+1|k\rangle = \boldsymbol{F}\hat{\boldsymbol{x}}\langle k\rangle + \boldsymbol{G}\boldsymbol{u}\langle k\rangle \tag{H.1}$$

$$\hat{\boldsymbol{P}}\langle k+1|k\rangle = \boldsymbol{F}\hat{\boldsymbol{P}}\langle k|k\rangle\boldsymbol{F}^{\mathrm{T}} + \hat{\boldsymbol{V}} \tag{H.2}$$

其中，$\hat{\boldsymbol{x}}$ 是系统状态估计值，$\hat{\boldsymbol{P}}\in\mathbb{R}^{n\times n}$ 是在 $\hat{\boldsymbol{x}}$ 中估值的协方差矩阵或不确定性。以上这步是开环的，其估值精度完全取决于模型 $\boldsymbol{F}$ 和 $\boldsymbol{G}$ 的质量，以及对输入 $\boldsymbol{u}$ 测量的能力。符号 $k+1|k$ 表明左侧的量是基于 k 时刻的信息对 $k+1$ 时刻信息的估值。

在预测 $\boldsymbol{P}$ 时涉及了两个正定矩阵相加，因此如果在过程中没有新增的信息和不确定量，其不确定性将会增加。为了改进这一缺陷，必须引入新的信息，它们来自于传感器的测量值。这些新增的信息就是所谓的新息：

$$\boldsymbol{\nu}\langle k+1\rangle = \boldsymbol{z}\langle k+1\rangle - \boldsymbol{H}\hat{\boldsymbol{x}}\langle k+1|k\rangle$$

它们是传感器实际测量值与传感器预计测量值的差。有些差异来源于传感器中的噪声，即测量噪声，但其余存在的差异还是表明预测的系统状态是有误差的，不能正确反映传感器观察值。

卡尔曼滤波的第二步就是更新。这一步将使用下式表示的卡尔曼增益来将新息用于对预测状态的修正：

$$\boldsymbol{K}\langle k+1\rangle=\hat{\boldsymbol{P}}\langle k+1|k\rangle\boldsymbol{H}^{\mathrm{T}}\underbrace{\left(\boldsymbol{H}\hat{\boldsymbol{P}}\langle k+1|k\rangle\boldsymbol{H}^{\mathrm{T}}+\hat{\boldsymbol{W}}\right)}_{\boldsymbol{S}}{}^{-1} \tag{H.3}$$

这样就能基于传感器观测值实现对状态估计的优化矫正：

$$\begin{aligned}\hat{\boldsymbol{x}}\langle k+1|k+1\rangle&=\hat{\boldsymbol{x}}\langle k+1|k\rangle+\boldsymbol{K}\langle k+1\rangle\boldsymbol{\nu}\langle k+1\rangle\\ \hat{\boldsymbol{P}}\langle k+1|k+1\rangle&=\hat{\boldsymbol{P}}\langle k+1|k\rangle-\boldsymbol{K}\langle k+1\rangle\boldsymbol{H}\hat{\boldsymbol{P}}\langle k+1|k\rangle\end{aligned}$$

很重要的一点是我们注意到先前的不确定性得以减少或缩小，因为第二项被从估计的协方差矩阵中减去了。字母 s 标注的项是由新息估计的协方差矩阵，包含了系统状态中的不确定量以及测量噪声的协方差矩阵。如果对于某些系统状态而言新息具有高度不确定性，那么这些不确定性将反映在卡尔曼增益中，并使得对那些状态的调整量减小。

协方差矩阵的更新也可以写成约瑟夫形式：

$$\hat{\boldsymbol{P}}\langle k+1|k+1\rangle=\left(\boldsymbol{I}_{n\times n}-\boldsymbol{K}\langle k+1\rangle\boldsymbol{H}\right)\hat{\boldsymbol{P}}\langle k+1|k\rangle\left(\boldsymbol{I}_{n\times n}-\boldsymbol{K}\langle k+1\rangle\boldsymbol{H}\right)^{\mathrm{T}}+\boldsymbol{K}\langle k+1\rangle\hat{\boldsymbol{V}}\boldsymbol{K}\langle k+1\rangle^{\mathrm{T}}$$

该形式改进了其数值性质，并且保证协方差估值矩阵对称，但代价是增加了计算量。

以上方程构成了经典的卡尔曼滤波器，其应用涵盖了从航空航天到经济学的广泛领域。该滤波器有很多重要特性。第一，它是递归的，即前一步迭代的输出是下一步的输入。第二，它是异步的，如果在某个特殊的迭代步没有获得传感器信息，则可以只执行预测，而不进行更新。而如果系统中有不同的传感器，每种传感器都有自己的 $\boldsymbol{H}$ 矩阵和采用频率时，我们就只需要用适当的 $\boldsymbol{z}$ 和 $\boldsymbol{H}$ 进行更新。该滤波器的连续时间版本是卡尔曼-布西(Kalman-Bucy)滤波器。

在迭代计算开始时，必须赋予卡尔曼滤波器合理的初值 $\hat{\boldsymbol{x}}$ 和 $\hat{\boldsymbol{P}}$。该滤波器还要求有最好的过程协方差矩阵估值和测量噪声估值。通常情况下我们不知道真实的 $\boldsymbol{V}$ 和 $\boldsymbol{W}$，但可以估计出它们的值，将 $\hat{\boldsymbol{V}}$ 和 $\hat{\boldsymbol{W}}$ 用于滤波器中。从方程(H.2)可以看出，如果对 $\hat{\boldsymbol{V}}$ 的估值过大，则得到的 $\boldsymbol{P}$ 矩阵估值就会比实际值大，从而导致对状态确定性的悲观估计。相反，如果对 $\hat{\boldsymbol{V}}$ 估计过小，滤波器会对其估值过分自信。

协方差矩阵 $\hat{\boldsymbol{P}}$ 的结构内涵非常丰富，其对角线元素 $\hat{\boldsymbol{P}}_{ii}$ 代表了状态 $\boldsymbol{x}_i$ 的方差或不确定性，而非对角线元素 $\hat{\boldsymbol{P}}_{ij}$ 则表示了状态 $\boldsymbol{x}_i$ 和 $\boldsymbol{x}_j$ 之间的关联。这些状态间的关联性对于允许任何新信息的流入非常关键，而新信息会改变影响某一特定过程输出的多个状态。

在方程(H.2)中这一项 $\boldsymbol{FP}\langle k|k\rangle\boldsymbol{F}^{\mathrm{T}}\langle k\rangle$ 也很有意思。以一维的情况为例，F 是一个标量，状态估值 $\hat{\boldsymbol{x}}\langle k\rangle$ 概率密度函数(PDF)为高斯函数，均值为 $\bar{x}\langle k\rangle$，方差为 $\sigma^2\langle k\rangle$。预测方程将系统状态及其高斯分布函数映射为一个新的高斯分布函数，新函数的均值为 $F\bar{x}\langle k\rangle$，方差为 $F^2\sigma^2\langle k\rangle$。而 $\boldsymbol{FP}\langle k|k\rangle\boldsymbol{F}^{\mathrm{T}}\langle k\rangle$ 就是该映射的矩阵表达，因为有

$$\mathrm{cov}(\boldsymbol{F}\boldsymbol{x})=\boldsymbol{F}\mathrm{cov}(\boldsymbol{x})\boldsymbol{F}^{\mathrm{T}}$$

并适当缩放其协方差。在方程(H.3)中的这一项 $\boldsymbol{HP}\langle k+1|k\rangle\boldsymbol{H}^{\mathrm{T}}$ 则将状态估值的协方差投射为观察值。

现在考虑非线性系统的情形：

$$\begin{aligned}\boldsymbol{x}\langle k+1\rangle &= \boldsymbol{f}\left(\boldsymbol{x}\langle k\rangle, \boldsymbol{u}\langle k\rangle\right) + \boldsymbol{v}\langle k\rangle \\ \boldsymbol{z}\langle k+1\rangle &= \boldsymbol{h}\left(\boldsymbol{x}\langle k\rangle\right) + \boldsymbol{w}\langle k\rangle\end{aligned}$$

其中，$\boldsymbol{f}$ 和 $\boldsymbol{h}$ 现在已变为函数，而非常数矩阵。函数 $\boldsymbol{f}:\mathbb{R}^n, \mathbb{R}^m \to \mathbb{R}^n$ 描述了如何从先前状态量和系统输入得到新的状态量。函数 $\boldsymbol{h}:\mathbb{R}^n \to \mathbb{R}^p$ 则是将状态向量映射成传感器测量值。

为了能将线性卡尔曼滤波用到非线性系统上，首先要做一个局部线性近似：

$$\begin{aligned}\boldsymbol{x}\langle k+1\rangle &= \boldsymbol{f}\left(\hat{\boldsymbol{x}}\langle k\rangle, \boldsymbol{u}\langle k\rangle\right) + \boldsymbol{F}_x\left(\boldsymbol{x}\langle k\rangle - \hat{\boldsymbol{x}}\langle k|k\rangle\right) + \boldsymbol{F}_u\boldsymbol{u}\langle k\rangle + \boldsymbol{F}_v\boldsymbol{v}\langle k\rangle \\ \boldsymbol{z}\langle k+1\rangle &= \boldsymbol{h}\left(\hat{\boldsymbol{x}}\langle k\rangle\right) + \boldsymbol{H}_x\left(\hat{\boldsymbol{x}}\langle k+1|k\rangle - \boldsymbol{x}\langle k\rangle\right) + \boldsymbol{H}_w\boldsymbol{w}\langle k\rangle\end{aligned}$$

其中，$\boldsymbol{F}_x \in \mathbb{R}^{n\times n}$，$\boldsymbol{F}_u \in \mathbb{R}^{n\times m}$，$\boldsymbol{F}_v \in \mathbb{R}^{n\times n}$，$\boldsymbol{H}_x \in \mathbb{R}^{p\times n}$ 和 $\boldsymbol{H}_w \in \mathbb{R}^{p\times p}$ 是函数 $\boldsymbol{f}(\cdot)$ 和 $\boldsymbol{h}(\cdot)$ 的雅克比矩阵，并在每个迭代时间步骤中被重新赋值。

我们定义一个预测误差：

$$\begin{aligned}\tilde{\boldsymbol{x}}\langle k+1|k\rangle &= \boldsymbol{x}\langle k\rangle - \hat{\boldsymbol{x}}\langle k+1|k\rangle \\ &= \boldsymbol{F}_x\tilde{\boldsymbol{x}}\langle k|k\rangle + \boldsymbol{F}_u\boldsymbol{u}\langle k\rangle + \boldsymbol{F}_v\boldsymbol{v}\langle k\rangle\end{aligned}$$

以及一个观测残差：

$$\begin{aligned}\tilde{\boldsymbol{z}}\langle k+1|k\rangle &= \boldsymbol{z}\langle k+1\rangle - \boldsymbol{h}\langle k+1|k\rangle \\ &= \boldsymbol{H}_x\tilde{\boldsymbol{x}} + \boldsymbol{H}_w\boldsymbol{w}\langle k\rangle\end{aligned}$$

它们都是线性的，因此可以采用前面给出的卡尔曼滤波方程。扩展卡尔曼滤波器的预测步方程如下：

$$\begin{aligned}\hat{\boldsymbol{x}}\langle k+1|k\rangle &= \boldsymbol{f}\left(\hat{\boldsymbol{x}}\langle k\rangle, \boldsymbol{u}\langle k\rangle\right) \\ \hat{\boldsymbol{P}}\langle k+1|k\rangle &= \boldsymbol{F}_x\hat{\boldsymbol{P}}\langle k|k\rangle\boldsymbol{F}_x^{\mathrm{T}} + \boldsymbol{F}_v\hat{\boldsymbol{V}}\langle k\rangle\boldsymbol{F}_v^{\mathrm{T}}\end{aligned}$$

更新步方程则为

$$\begin{aligned}\hat{\boldsymbol{x}}\langle k+1|k+1\rangle &= \hat{\boldsymbol{x}}\langle k+1|k\rangle + \boldsymbol{K}\langle k+1\rangle\boldsymbol{\nu}\langle k+1\rangle \\ \hat{\boldsymbol{P}}\langle k+1|k+1\rangle &= \hat{\boldsymbol{P}}\langle k+1|k\rangle - \boldsymbol{K}\langle k+1\rangle\boldsymbol{H}_x\hat{\boldsymbol{P}}\langle k+1|k\rangle\end{aligned}$$

其中，新息方程是

$$\boldsymbol{\nu}\langle k+1\rangle = \boldsymbol{z}\langle k+1\rangle - \boldsymbol{h}\left(\hat{\boldsymbol{x}}\langle k+1|k\rangle\right)$$

卡尔曼增益为

$$\boldsymbol{K}\langle k+1\rangle = \hat{\boldsymbol{P}}\langle k+1|k\rangle\boldsymbol{H}_x^{\mathrm{T}}\left(\boldsymbol{H}_x\hat{\boldsymbol{P}}\langle k+1|k\rangle\boldsymbol{H}_x^{\mathrm{T}} + \boldsymbol{H}_w\hat{\boldsymbol{W}}\boldsymbol{H}_w^{\mathrm{T}}\right)^{-1}$$

使用扩展卡尔曼滤波器的一个根本问题在于：随机变量在经过非线性函数 $\boldsymbol{f}(\cdot)$ 和 $\boldsymbol{h}(\cdot)$ 的作用后，它们的概率密度函数不再呈高斯分布。为了便于说明上述问题，我们以一个标量系统为例，其状态估值的概率密度函数呈 $N(5,2)$ 的高斯分布：

```
>> x = linspace(0, 20, 100);
>> g = gaussfunc(5, 2, x);
>> plot(x, g);
```

下面应用非线性函数 $y = x^2/5$ 后，再来看 y 的概率密度函数：

```
>> y = x.^2 / 5;
>> plot(y, g, 'r');
```

函数曲线如图 H.1 所示。可以看出 y 的概率密度函数曲线与 x 曲线的峰值或众数位置相同，

但不再呈现出高斯分布。由于 y 的曲线不再对称，因此其均值实际上要大于其众数。在扩展卡尔曼滤波(EKF)方程中出现的雅克比矩阵虽然也对协方差进行了适当缩放，但由于结果的非高斯分布使得前面对于卡尔曼滤波是最优估计器的保证不再成立。其他可选的卡尔曼滤波器还包括雅温斯基[Jazwinski, 1970]提出的迭代 EKF，或者朱里亚和乌尔曼[Julier and Uhlmann, 2000]提出的无迹卡尔曼滤波(Unscented Kalman Filter, UKF)，其中 UKF 使用了离散采样点来近似概率密度函数。

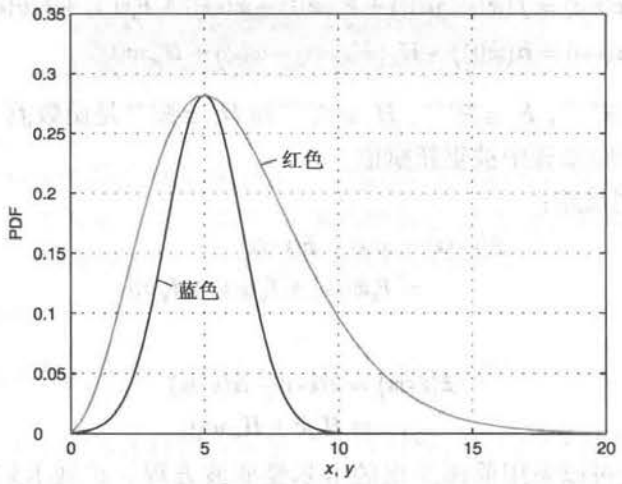


图 H.1 具有高斯分布 $N(5,2)$ 的状态变量 x 的 PDF(蓝色)和非线性函数 $x^2/5$ 的 PDF(红色)

附录 I　齐次坐标系

在 n 维欧拉空间 $\boldsymbol{x} \in \mathbb{R}^n$ 中的一个点可以用一个坐标向量 $(x_1, x_2, \cdots, x_n)$ 来表示。该点在齐次坐标系，或投影空间 $\tilde{\boldsymbol{x}} \in \mathbb{R}^n$ 中可以用一个坐标向量 $\tilde{x}_1, \tilde{x}_2, \cdots, \tilde{x}_{n+1}$ 表示。从投影坐标转换为欧拉坐标的表达式为

$$x_i = \frac{\tilde{x}_i}{\tilde{x}_{n+1}}, \quad i = 1, \cdots, n$$

反过来，从欧拉坐标向量构建齐次坐标向量的方法是

$$\tilde{\boldsymbol{x}} = (x_1, x_2, \cdots, x_n, 1)$$

字母上的波浪符表示它是齐次坐标量。

可以看出，投影坐标系比欧拉坐标系多出一个自由度，其好处是多方面的。首先，投影坐标系中允许点和线在无限远处，即所谓的理想点和线，而且只用实数来表示。使用投影坐标还意味着比例已不重要，也就是说只要 $\alpha \neq 0$，那么 $\tilde{\boldsymbol{x}}$ 和 $\tilde{\boldsymbol{x}}' = \alpha\tilde{\boldsymbol{x}}$ 代表的是欧拉坐标系中的同一点，并可以表示为 $\tilde{\boldsymbol{x}} \simeq \tilde{\boldsymbol{x}}'$。另外，当需要对齐次坐标形式的点相对于某个坐标系旋转或平移时，只要简单地把该点的齐次坐标乘以一个 $(n+1) \times (n+1)$ 维的齐次变换矩阵即可。

齐次坐标向量在计算机视觉中是很重要的概念，特别是当我们要研究在摄像机像平面上的点和线时。我们也可以认为齐次坐标形式代表了在欧拉空间里的一束光线，而点和光线的关系正体现了投影变换的核心。

在投影空间 $\mathbb{R}^2$ 中，一条线可以用一个 3 元组 $\tilde{\boldsymbol{\ell}} = (\ell_1, \ell_2, \ell_3)^{\mathrm{T}}$ 来定义，其中元素为非全零。该直线的方程就是线上所有点的集合：

$$\tilde{\boldsymbol{\ell}}^{\mathrm{T}} \tilde{\boldsymbol{x}} = 0$$

展开后即为 $\ell_1 x + \ell_2 y + \ell_3 = 0$，并可以进一步转化为大家更熟悉的直线方程形式。可以注意到，用这种形式可以表示一条垂直线，即平行于 y 轴的直线，而大家熟知的直线方程形式 $y = mx + c$ 却不能。这种形式是一条直线的点方程。非齐次坐标向量 (ℓ_1, ℓ_2) 是这条直线的垂线，而 $(-\ell_2, \ell_1)$ 是该直线的平行线。

在投影空间中的点和线之间还存在对偶性。一个点被定义为两条直线的交点。如果分别写出两条直线的点方程形式，$\tilde{\boldsymbol{\ell}}_1^{\mathrm{T}} \tilde{\boldsymbol{p}} = 0$ 和 $\tilde{\boldsymbol{\ell}}_2^{\mathrm{T}} \tilde{\boldsymbol{p}} = 0$，那么它们的交点可表示为

$$\tilde{\boldsymbol{p}} = \tilde{\boldsymbol{\ell}}_1 \times \tilde{\boldsymbol{\ell}}_2$$

它就是所谓的一个点的线方程。类似地，经过两个点 $\tilde{\boldsymbol{p}}_1$ 和 $\tilde{\boldsymbol{p}}_2$ 的一条直线可以用这两点的叉积表示：

$$\tilde{\boldsymbol{\ell}}_{12} = \tilde{\boldsymbol{p}}_1 \times \tilde{\boldsymbol{p}}_2$$

以两条平行线为例，且与水平轴成 45°夹角：

```
>> l1 = [1 -1 0]';
>> l2 = [1 -1 -1]';
```

画出其曲线为

```
>> plot_homline(l1, 'b')
>> plot_homline(l2, 'r')
```

两条平行线的交点为

```
>> cross(l1, l2)
ans =
     1     1     0
```

这就是一个理想点，因为其第三个坐标值为零，它对应的欧拉点就在无穷远。投影坐标可以用很简单的形式就表示出无穷远的点和线，而且不需要用特殊的逻辑来处理无穷远这种特殊情况。

一个点 $\tilde{\boldsymbol{p}}$ 到一条线 $\tilde{\boldsymbol{\ell}}$ 的距离是

$$d=\frac{\tilde{\boldsymbol{\ell}}^{\mathrm{T}}\tilde{\boldsymbol{p}}}{p_3\sqrt{\ell_1^2+\ell_2^2}} \tag{I.1}$$

在投影空间 $\mathbb{R}^3$ 中，点和面之间也存在对偶关系：三个点确定一个平面，而三个平面相交处为一点。

附录 J　图

图是对用链环连接起来的一组物体的抽象描述，并以图 J.1 所示的形式画出。数学上，一个图用符号 $G(V,E)$ 表示，其中 V 称为顶点或节点，而连接两顶点之间的链环 E 则称作边或弧。边可以是有向的(带箭头)或无向的(图 J.1 所示即为无向的)。连接顶点之间的边还可以被赋予某个权重值或成本。从一个顶点到另一顶点的一系列边构成一条路径，如果路径的起点和终点为同一顶点则形成一个回路。若一条边的两个顶点为同一顶点，则此边称作环。图可以用来代表交通、通信或社会网络，数学上的这一研究分支称为图论。

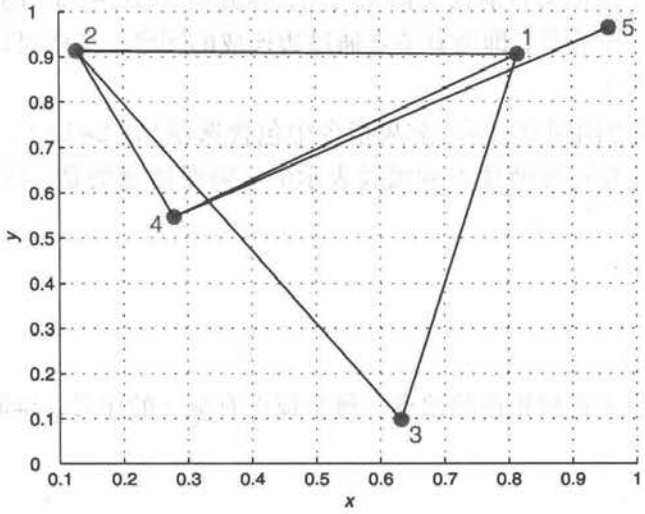


图 J.1.　用 PGraph 类生成的一个示例图形

工具箱中提供了一个名为 PGraph 的 MATLAB 图类，它支持将 n 维空间中的某个点作为顶点的嵌入图。创建一个新图的方法如下：

```
>> g = PGraph()
g =
  2 dimensions
  0 vertices
  0 edges
  0 components
```

默认情况下图的节点属于一个二维空间。我们可以给这个图增添节点：

```
>> g.add_node( rand(2,1) );
>> g.add_node( rand(2,1) );
>> g.add_node( rand(2,1) );
>> g.add_node( rand(2,1) );
>> g.add_node( rand(2,1) );
```

而且每个节点都有一个随机坐标值。使用图的显示指令可以给出关于此图的属性概要：

```
>> g
g =
  2 dimensions
  5 vertices
  0 edges
  5 components
```

如上所示，这个图有 5 个节点，但没有边。节点的编号从 1 到 5，这样就可以在两个节点之间添加边：

```
>> g.add_edge(1, 2);
>> g.add_edge(1, 3);
>> g.add_edge(1, 4);
>> g.add_edge(2, 3);
>> g.add_edge(2, 4);
>> g.add_edge(4, 5);
>> g
g =
  2 dimensions
  5 vertices
  6 edges
  1 components
```

默认情况下节点之间的距离是顶点间的欧拉距离，但该默认设置可以通过给 add_edge 第三个参数被覆盖掉。由于该图类目前仅支持无向图，因此给 add_edge 的两个顶点的先后顺序并不重要。这个图有一个构件，即所有节点通过边连成的网络。该图可以通过以下指令画出：

```
>> g.plot('labels')
```

画出的即为图 J.1。顶点用圆点表示，如果指令中包含选项'labels'，那么顶点的序号还会显示在圆点旁边。边是用连接两顶点的黑线表示的。需要注意的是，只有嵌入二维和三维空间中的图才能画出。

顶点 2 的邻点包括：

```
>> g.neighbours(2)
ans =
     3     4     1
```

它们都是通过边与顶点 2 直接相连的顶点。每个顶点有唯一的序号，与顶点 2 相连的边包括：

```
>> e = g.edges(2)
e =
     4     5     1
```

这些边的成本或长度为

```
>> g.cost(e)
ans =
    0.9597    0.3966    0.6878
```

显而易见，与边 4 和边 1 相比，边 5 的成本最低。边 5 的属性显示如下：

```
>> g.vertices(5)
ans =
     2
     4
```

它连接顶点 2 和顶点 4，显然顶点 4 是顶点 2 最近的邻点。我们经常会需要同时获取一个节点的邻点以及它们之间的距离，这些信息可以用以下指令方便地获得：

```
>> [n,c] = g.neighbours(2)
n =
     3     4     1
c =
    0.9597    0.3966    0.6878
```

为了在这个图上规划一条路径，需要确定目标顶点：

```
>> g.goal(5)
```

以上指令将赋予图中的每个节点到目标顶点的距离为广度优先方式。从一个特定起点寻找一条到目标的路径的指令为

```
>> g.path(3)
ans =
     3     2     4     5
```

在这个例子中，从顶点3到顶点5的最短路径经过顶点2和顶点4。离坐标点(0.5, 0.5)最近的顶点为

```
>> g.closest([0.5, 0.5])
ans =
     4
```

要想计算图中任意两点间最低成本的路径，可以使用一些大家熟知的算法，如 A^* [Nilsson, 1971]：

```
>> g.Astar(3, 5)
ans =
     3     2     4     5
```

或者更早的 Dijstrka 方法[Dijstrka, 1959]。

附录K 峰值搜索

一个经常遇到的问题是要估计出某个离散信号 $y(k)$，$k \in \mathbb{Z}$ 的峰值。以如图 K.1(a)所示(见彩图)的信号曲线为例，可以加载并画出该曲线：

```
>> load peakfit1
>> plot(y,  '-o')
```

为了找出信号中的最大峰值，可以直接用 MATLAB 的 max 函数：

```
>> [ypk,xpk] = max(y)
ypk =
    0.9905
xpk =
     8
```

以上数据表明最大峰值位于第 8 元素，其值为 0.9905。但在这个信号中存在多个峰值，要找出这些峰值，可以用工具箱提供的函数 peak：

```
>> [ypk,xpk] = peak(y)
ypk =
    0.9905    0.6718   -0.5799
xpk =
     8    25    16
```

它将返回从大到小排列的 3 个峰值。衡量某个峰值质量的常用参数是它的幅值大小，以及第二峰值与第三峰值的高度之比：

```
>> ypk(2)/ypk(1)
```

该比值也称为模糊比，其值越小越好。

这个离散信号其实是一个连续基础信号 $y(x)$ 的采样表示，因此信号真正的峰值可能是在两个采样值之间。如果将离散信号图放大，如图 K.1(b)所示，就可以看到虽然第 8 个采样点是最大的，但第 9 个采样点的值仅略微小一点，因此真正峰值应在第 8 个和第 9 个采样点之间。寻找这个峰值的一般方法是用峰值附近的点拟合出一条抛物线：

$$y = a\delta^2 + b\delta + c,\ \delta \in \mathbb{R} \tag{K.1}$$

在离散信号中的峰值点如果在(x_{pk}, y_{pk})，那么 $\delta = 0$ 就对应 x_{pk}，而在其两侧的离散点的 x 坐标分别对应 $\delta = -1$ 和 $\delta = +1$。将 3 个点$(-1, y(-1))$,$(0, y(0))$和$(1, y(1))$代入方程(K.1)，就能得到 3 个等式：

$$\begin{aligned} y(-1) &= a - b + c \\ y(0) &= c \\ y(1) &= a + b + c \end{aligned}$$

将这 3 个等式改写为更紧凑的矩阵形式：

$$\begin{pmatrix} y(-1) \\ y(0) \\ y(1) \end{pmatrix} = \begin{pmatrix} 1 & -1 & 1 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \\ c \end{pmatrix}$$

求解这个矩阵方程就能得到抛物线的系数：

$$\begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} 1 & -1 & 1 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{pmatrix}^{-1} \begin{pmatrix} y(-1) \\ y(0) \\ y(1) \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 1 & -2 & 1 \\ -1 & 0 & 1 \\ 0 & 2 & 0 \end{pmatrix} \begin{pmatrix} y(-1) \\ y(0) \\ y(1) \end{pmatrix} \tag{K.2}$$

抛物线的最大值位于其一阶微分等于零的位置：

$$2a\delta + b = 0$$

将从方程(K.2)中得到的 a 和 b 代入上式，就能得到拟合抛物线峰值相对于离散信号峰值的偏差量：

$$\delta = \frac{1}{2}\frac{y(-1) - y(1)}{y(-1) - 2y(0) + y(1)}$$

因此，精确或插值后的峰值位置位于：

$$\hat{x}_{\text{pk}} = x_{\text{pk}} + \delta$$

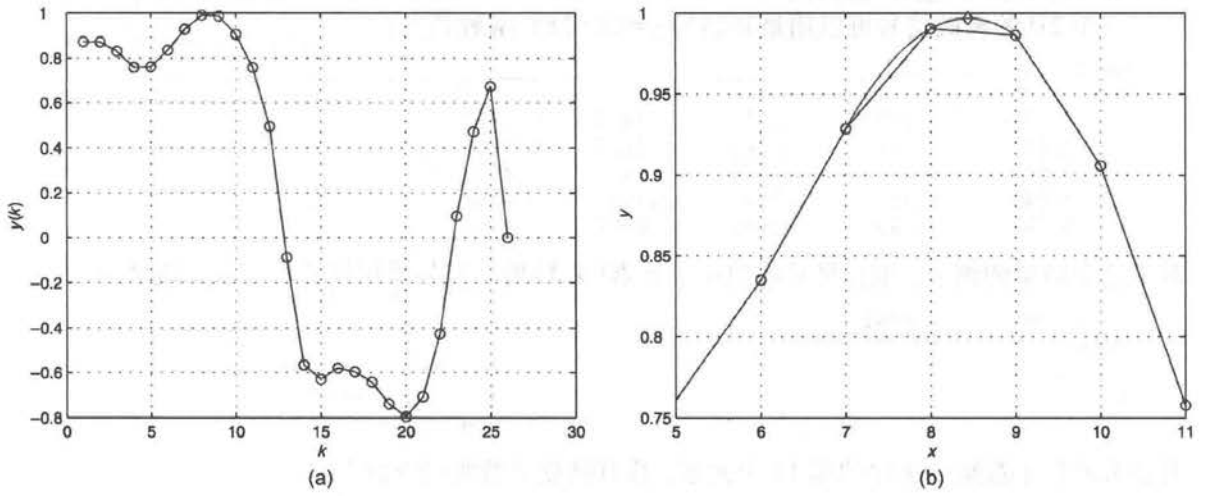


图 K.1 峰值拟合。(a)一个拥有多个局部最大值的离散信号；(b)在第一个最大值附近的局部放大图，含拟合曲线(红色)以及估计的峰值(红色的◇)

在求最大值时，抛物线中的系数 a 为负值，它表示了峰值的尖锐度，用它可以来确定峰值处是否足够尖锐。a 的绝对值大，表明波峰尖锐，容易找到峰值；相反，a 的值小则说明波峰处较平缓，很难从中找出精确的峰值。

回到前面的例子，可以用工具箱的 peak 函数来估计精确的峰值位置：

```
>> [ymax,xmax] = peak(y, 'interp', 2)
ymax =
    0.9905    0.6718   -0.5799
xmax =
    8.4394   24.7299   16.2438
```

其中，在'interp'选项后的参数表明要拟合一个 2 阶多项式，即抛物线。拟合出的抛物线如图 K.1(b)中的红色曲线所示，要画出该拟合曲线只需在 peak 中加入'plot'选项。

如果信号中存在叠加噪声，那么就可能有多个峰值，但其中多数幅度都很小，可以通过设置一个峰值刻度将它们去掉。例如，寻找波峰，其值要大于在水平方向两边 ±5 个点对应的所有值：

```
>> peak(y, 'scale', 5)
ans =
    0.9905    0.8730    0.6718
```

可以看出，这种情况下寻找的峰值与前面没有设刻度时相同，这是因为这个信号非常平滑。

对应 2D 的信号，可以用类似的方法处理，只是需要拟合地变为一个抛物面：

$$z = ax^2 + by^2 + cx + dy + e \tag{K.3}$$

它有5个系数需要确定，可以用中心点值(即离散最大值)及其周围(东、西、南、北)的4个点的值来计算，方法与抛物线拟合类似。拟合完成后，其估计的峰值位置相对于中心点的偏移为

$$\delta_x = \frac{1}{2}\frac{z_e - z_w}{2z_c - z_w - z_e}$$

$$\delta_y = \frac{1}{2}\frac{z_s - z_n}{2z_c - z_n - z_s}$$

这时，抛物面系数 a 和 b 分别代表在 x 和 y 方向上的峰值尖锐度，而且 a 和 b 中的较小者还可用于衡量这个峰值的质量。

一个2D的离散信号可以用最开始的 peakfit1 函数载入：

```
>> z
z =
    0.0800    0.2000    0.3202    0.4400    0.5600
    0.0400    0.1717    0.3662    0.4117    0.5200
    0.0002    0.2062    0.8766    0.4462    0.4802
   -0.0400    0.0917    0.2862    0.3317    0.4400
   -0.0800    0.0400    0.1602    0.2800    0.4000
```

对于这个简单的例子，很显然其峰值位于元素(3,3)处。但如果用计算机寻找，结果为

```
>> [zmax,i] = max(z(:))
zmax =
    0.8766
i =
    13
```

其最大值位于数组行主序的第13个元素，将其转化为数组的下标[①]：

```
>> [ymax,xmax] = ind2sub(size(z), i)
xmax =
     3
ymax =
     3
```

其实，如果采用工具箱的函数 peak2，就可以省去转化步骤：

```
>> [zm,xy]=peak2(z)
zm =
    0.8766
xy =
     3
     3
```

这个函数将返回所有非局部最大值，局部区域的范围由'scale'选项给出。正如一维的情况一样，也可以对它的峰值进行精确化：

```
>> [zm,xy]=peak2(z, 'interp')
zm =
    0.8839
xy =
    2.9637
    3.1090
```

通过插值得到的峰值位于(2.9637, 3.1090)。如果将以上方法应用到图像数据，就是所谓的亚像素插值。

① 行主序是指对数组元素计数时，以数组左上角元素为第1元素，从左到右数完一行，然后转到下一行从最左元素开始计数，重复以上步骤，直到数组的右下角元素。

术　语

关于在机器人学及计算机视觉中使用的数学符号，不同的书可谓千差万别。

下表列出了本书中使用的符号及其可能具有的单位。有些符号可能有多重含义，必须根据上下文才能准确判断。

一个向量中的元素 $\boldsymbol{x}[i]$ 或矩阵中的元素 $\boldsymbol{x}[i, j]$ 是用方括号表示的，而一个时间序列中的元素 $\boldsymbol{x}\langle i\rangle$ 则用尖括号表示。

符号	描述	单位
$\hat{x}$	x 的估值	
$\bar{x}$	x 的均值或相对值	
x^*	x 的期望值	
$\boldsymbol{v}$	一个向量	
$\hat{\boldsymbol{v}}$	一个平行于向量 $\boldsymbol{v}$ 的单位向量	
$\lvert\boldsymbol{v}\rvert$	向量 $\boldsymbol{v}$ 的范数或长度	
$\lvert\tilde{q}\rvert$	四元数 $\tilde{q}$ 的范数	
$\tilde{\boldsymbol{v}}$	向量 $\boldsymbol{v}$ 的齐次表示	
v_x	一个向量中的任一元素	
$\boldsymbol{v}_1 \cdot \boldsymbol{v}_2$	两个向量之间的点积，也写作 $\boldsymbol{v}_1^{\mathrm{T}}\boldsymbol{v}_2$	
$\boldsymbol{v}_1 \times \boldsymbol{v}_2$	两个向量之间的叉积	
$\boldsymbol{A}$	一个矩阵	
$\boldsymbol{A}$	矩阵 $\boldsymbol{A}$ 的逆矩阵	
$\boldsymbol{A}^+$	矩阵 $\boldsymbol{A}$ 的伪逆矩阵	
$\boldsymbol{A}^{\mathrm{T}}$	矩阵 $\boldsymbol{A}$ 的转置矩阵	
$\boldsymbol{A}^{-\mathrm{T}}$	矩阵 $\boldsymbol{A}$ 的逆矩阵的转置矩阵	
$\boldsymbol{A}_{i,j}$	矩阵 $\boldsymbol{A}$ 在 i 行 j 列的元素	
$\boldsymbol{A}[i, j]$	矩阵 $\boldsymbol{A}$ 在 i 行 j 列的元素	
$F(x)$	关于 x 的一个函数	
$F_x(x)$	偏导数 $\partial F/\partial x$	
B	粘性摩擦系数	N m s rad $^{-1}$
$\mathcal{C}$	机器人的构形空间	
$\boldsymbol{C}$	相机矩阵 $\boldsymbol{C} \in \mathbb{R}^{3\times4}$	
$\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})$	机械臂上的离心力和哥氏力项	
$\mathbb{C}$	复数集	
$\mathcal{D}(\cdot)$	机械臂的动力学方程 $\boldsymbol{Q}, \boldsymbol{q}, \dot{\boldsymbol{q}} \mapsto \ddot{\boldsymbol{q}}$	
$\Delta(\xi)$	从增量姿态变化到微分运动的映射 $SE(3) \mapsto \mathbb{R}^6$	
$\Delta^{-1}(\delta)$	从微分运动到增量姿态变化的映射 $\mathbb{R}^6 \mapsto \mathrm{SE}(3)$	
E	照明强度(lux)	lx
f	焦距	m
f	力	N
$\boldsymbol{f}$	图像特征向量	
$F(\ddot{q})$	摩擦力矩	Nm
ϕ	光通量	lm
g	力旋量，一个包含力和力矩的向量$(f_x, f_y, f_z, m_x, m_y, m_z)$	N, Nm
$\boldsymbol{G}(\boldsymbol{q})$	机械臂上的重力负载项	Nm
γ	机器人转向角	rad

续表

符号	描述	单位
$\boldsymbol{\Gamma}$	转动的 3 角度表示 $\boldsymbol{\Gamma}\in\mathbb{R}^3$	rad
$\boldsymbol{\Gamma}$	三维为矩 $\boldsymbol{\Gamma}\in\mathbb{R}^3$	Nm
$\boldsymbol{I}_{n\times n}$	$n\times n$ 单位矩阵	
J	惯量	kg m^2
$\boldsymbol{J}$	惯性张量，3×3 矩阵	kg m^2
$\boldsymbol{J}$	雅克比矩阵	
${}^A\boldsymbol{J}_B$	从坐标系 A 到 B 的速度变换雅克比矩阵	
k, K	常量	
$\boldsymbol{K}$	相机标定矩阵	
K_i	放大增益(跨导)	AV^{-1}
K_m	电机扭矩常量	N m A^{-1}
$\mathcal{K}(\cdot)$	运动学正解	
$\mathcal{K}^{-1}(\cdot)$	运动学逆解	
L	亮度(nit)	nt
λ	波长	m
λ	一个特征值	
m_i	连杆 i 的质量	kg
$\mathbf{M}(\boldsymbol{q})$	机械臂惯性矩阵	kg m^2
$\mathbf{p}$	一个图形平面上的点	
$\mathbf{P}$	一个世界坐标系中的点	
$\mathbb{P}^2$	所有二维点的投影空间，一个 3 元组	
$\mathbb{P}^3$	所有三维点的投影空间，一个 4 元组	
$\mathcal{P}(\cdot)$	投影函数，$\mathbb{P}^3\mapsto\mathbb{P}^2$	
$\dot{q}$	4 元数	
$\dot{q}(\boldsymbol{v})$	速度 $\boldsymbol{v}$ 的纯 4 元数	
$\boldsymbol{q}$	构型，广义坐标	m, rad
$\boldsymbol{Q}$	广义力	N, Nm
ρ_w, ρ_h	像素宽度与高度	m
$\boldsymbol{R}$	一个正交旋转矩阵 $\boldsymbol{R}\in SO(2)$ 或 $SO(3)$	
$\mathbb{R}$	实数集合	
$\mathbb{R}^2$	所有二维点的空间	
$\mathbb{R}^3$	所有三维点的空间	
s	拉普拉斯变换算子	
$\mathbb{S}$	在$[0,2\pi]$中所有角度的集合	
$SE(n)$	特殊欧氏群，在 n 维空间所有位姿的集合	
$SO(n)$	特殊正交群，在 n 维空间所有姿态的集合	
$\boldsymbol{S}(\boldsymbol{v})$	$\boldsymbol{v}$ 的反对称矩阵	
$\boldsymbol{s}_i$	相对于连杆 j 坐标系的连杆 j 的 COM	m
$\boldsymbol{S}_i$	连杆 j 的一阶矩，$\boldsymbol{S}_i=\boldsymbol{m}_i\boldsymbol{s}_i$	kg m
σ	标准差	
σ	机器人关节类型，$\sigma=0$ 代表转动，$\sigma=1$ 代表移动	
t	时间	s
T	采样间隔	s
T	温度	K

续表

符号	描述	单位
T	光传输	
$\boldsymbol{T}$	齐次变换 $T \in SE(2)$ 或 $SE(3)$	
${}^{A}\boldsymbol{T}_{B}$	坐标系 $\lvert B \rvert$ 相对于参考坐标系 $\lvert A \rvert$ 的齐次变换，如果未给出 $\lvert A \rvert$，则参考坐标系默认为世界坐标系 0，注意，$A_B^T = ({}^{B}\boldsymbol{T}_{A})^{-1}$	
θ	角度	rad
$\boldsymbol{\theta}$	角度向量，一般指机器人关节角度	rad
θ_r，θ_p，θ_y	滚动、俯仰、偏转角度	rad
τ	力矩	Nm
τ_C	库仑摩擦力矩	Nm
u，v	相机像平面坐标	Pixels(像素)
$\bar{u}$，$\bar{v}$	归一化像平面坐标，相对于主点	m
u_0，v_0	主点坐标	Pixels(像素)
v	速度	$\mathrm{m\ s^{-1}}$
$\boldsymbol{v}$	速度向量	$\mathrm{m\ s^{-1}}$
$\boldsymbol{\nu}$	革新	
$\boldsymbol{\nu}$	速度旋量 $\boldsymbol{\nu} \in \mathbb{R}^6$，$\boldsymbol{\nu} = (v_x, v_y, v_z, \omega_x, \omega_y, \omega_z)$	
ω	转动速度	$\mathrm{rad\ s^{-1}}$
$\boldsymbol{\omega}$	角速度向量	$\mathrm{rad\ s^{-1}}$
X，Y，Z	笛卡儿坐标	
ξ	三维笛卡儿位姿的抽象表示	
${}^{A}\xi_{B}$	坐标系 $\lvert B \rvert$ 相对于参考坐标系 $\lvert A \rvert$ 三维笛卡儿相对位姿的抽象表示	
$\boldsymbol{\nu}$	笛卡儿速度旋量$(v_x, v_y, v_z, \omega_z, \omega_y, \omega_z)$	
$\bar{x}$，$\bar{y}$	归一化像平面坐标	
$\mathbf{0}_{m\times n}$	一个 $m \times n$ 的全 0 矩阵	
$\mathbf{1}_{m\times n}$	一个 $m \times n$ 的全 1 矩阵	
$\mathbb{Z}$	所有整数的集合	
$\mathbb{Z}^+$	所有正整数的集合	
$\sim$	表示对等	
$\simeq$	齐次坐标等价	
$\oplus$	位姿构成算子	
$\equiv$	比色等价	
$\ominus$	一个位姿的逆解(一元算子)	
$\cdot$	一个点通过一个相对位姿的变换，如 $\xi \cdot \boldsymbol{p}$	
$\ominus$	一个圆周上最小的角度差	rad
$\otimes$	互相关	
$\oplus$	形态学膨胀	
$\ominus$	形态学腐蚀	
$\circ$	形态学开启	
$\bullet$	形态学闭合	
$\lvert F \rvert$	坐标系 F	
$[a, b]$	从 a 到 b 的区间，含 a 和 b	
(a, b)	从 a 到 b 的区间，不含 a 和 b	
$[a, b)$	从 a 到 b 的区间，不含 b，含 a	
$(a, b]$	从 a 到 b 的区间，不含 a，含 b	

MATLAB 工具箱的表达约定

- 一个点的笛卡儿坐标被表示为一个列向量。
- 一组点的坐标表示为一个矩阵，其中每一列代表一个单独点的坐标。
- 一个机器人构型，即一组关节角度值，表示为一个行向量。
- 时序数据表示为一个矩阵，其中相邻行之间为一个时间步距。
- 一个 MATLAB 矩阵具有维数(i,j)，其中i和j分别代表矩阵的行数和列数。图像坐标记为(u,v)，因此用矩阵$\boldsymbol{I}$表示的一幅图像就可以记为$\boldsymbol{I}(v,u)$。
- 维度大于等于 3 个的矩阵比较常用：
  - 一幅彩色图像具有 3 个维度：行、列、颜色平面。
  - 一个灰度图像序列具有 3 个维度：行、列、索引。
  - 一个彩色图像序列具有 4 个维度：行、列、颜色平面、索引。
  - 作为一个齐次变换序列的轨迹具有 3 个维度：行、列、索引。
- 在$SE(3)$中的一个位姿用一个 4×4 矩阵表示。
- 在$SO(3)$中的一个姿态用一个 3×3 矩阵表示。
- 在$SE(2)$中的一个位姿用一个 3×3 矩阵表示。

参考文献

Agrawal M, Konolige K, Blas M (2008) CenSurE: Center surround extremas for realtime feature detection and matching. In: Forsyth D, Torr P, Zisserman A (eds) Lecture notes in computer science. Computer Vision – ECCV 2008, vol 5305. Springer-Verlag, Berlin Heidelberg, pp 102–115

Altmann SL (1989) Hamilton, Rodrigues, and the Quaternion scandal. Math Mag 62(5):291–308

Andersen N, Ravn O, Sørensen A (1993) Real-time vision based control of servomechanical systems. In: Chatila R, Hirzinger G (eds) Lecture Notes in Control and Information Sciences. Experimental Robotics II, vol 190. Springer-Verlag, Berlin Heidelberg, pp 388–402

Andersson RL (1989) Dynamic sensing in a ping-pong playing robot. IEEE T Robotic Autom 5(6): 728–739

Antonelli G (2006) Underwater robots: Motion and force control of vehicle-manipulator systems, 2nd ed. Springer Tracts in Advanced Robotics, vol 2. Springer-Verlag, Berlin Heidelberg

Arkin RC (1999) Behavior-based robotics. The MIT Press

Armstrong WW (1979) Recursive solution to the equations of motion of an N-link manipulator. In: Proc. 5th World Congress on Theory of Machines and Mechanisms, Montreal, Jul, pp 1343–1346

Armstrong BS (1988) Dynamics for robot control: Friction modelling and ensuring excitation during parameter identification. Stanford University

Armstrong B (1989) On finding exciting trajectories for identification experiments involving systems with nonlinear dynamics. Int J Robot Res 8(6):28

Armstrong-Hélouvry B, Dupont P, De Wit CC (1994) A survey of models, analysis tools and compensation methods for the control of machines with friction. Automatica 30(7):1083–1138

Arun KS, Huang TS, Blostein SD (1987) Least-squares fitting of 2 3-D point sets. IEEE T Pattern Anal 9(5):699–700

Asada H (1983) A geometrical representation of manipulator dynamics and its application to arm design. J Dyn Syst-T ASME 105:131

Azarbayejani A, Pentland AP (1995) Recursive estimation of motion, structure, and focal length. IEEE T Pattern Anal 17(6):562–575

Baldridge AM, Hook SJ, Grove CI, Rivera G (2009) The ASTER spectral library version 2.0. Remote Sens Environ 113(4):711–715

Ballard DH (1981) Generalizing the Hough transform to detect arbitrary shapes. Pattern Recogn 13(2):111–122

Banks J, Corke PI (2001) Quantitative evaluation of matching methods and validity measures for stereo vision. Int J Robot Res 20(7):512–532

Bar-Shalom Y, Fortmann T (1988) Tracking and data association. Mathematics in Science and Engineering, vol 182. Academic Press

Bar-Shalom Y, Rong Li X, Thiagalingam Kirubarajan (2001) Estimation with applications to tracking and navigation. Wiley-Interscience

Bauer J, Sünderhauf N, Protzel P (2007) Comparing several implementations of two recently published feature detectors. In: IFAC Symposium on Intelligent Autonomous Vehicles (IAV), Toulouse

Bay H, Ess A, Tuytelaars T, Van Gool L (2008) Speeded-up robust features (SURF). Comput Vis Image Und 110(3):346–359

Benosman R, Kang SB (2001) Panoramic vision: Sensors, theory, and applications. Springer-Verlag

Benson KB (ed) (1986) Television engineering handbook. McGraw-Hill

Besl PJ, McKay HD (1992) A method for registration of 3-D shapes. IEEE T Pattern Anal 14(2):239–256

Bhat DN, Nayar SK (2002) Ordinal measures for image correspondence. IEEE T Pattern Anal 20(4): 415–423

Bishop CM (2006) Pattern recognition and machine learning. Information Science and Statistics. Springer-Verlag, New York

Bolles RC, Baker HH, Marimont DH (1987) Epipolar-plane image analysis: An approach to determining structure from motion. Int J Comput Vision 1(1):7–55, Mar

Bolles RC, Baker HH, Hannah MJ (1993) The JISCT stereo evaluation. In: Image Understanding Workshop: proceedings of a workshop held in Washington, DC apr 18–21, 1993. Morgan Kaufmann, pp 263

Borenstein J, Everett HR, Feng L (1996) Navigating mobile robots: Systems and techniques. AK Peters, Ltd. Natick, MA, USA, Out of print and available at http://www-personal.umich.edu/~johannb/Papers/pos96rep.pdf

Borgefors G (1986) Distance transformations in digital images. Comput Vision Graph 34(3):344–371

Bouguet J-Y (2010) Camera calibration toolbox for MATLAB. http://www.vision.caltech.edu/bouguetj/calib_doc

Bradski G, Kaehler A (2008) Learning OpenCV: Computer vision with the OpenCV library. O'Reilly Media

Brady M, Hollerbach JM, Johnson TL, Lozano-Pérez T, Mason MT (eds) (1982) Robot motion: Planning and control. The MIT Press

Braitenberg V (1986) Vehicles: Experiments in synthetic psychology. The MIT Press

Brockett RW (1983) Asymptotic stability and feedback stabilization. In: Brockett RW, Millmann RS, Sussmann HJ (eds) Progress in mathematics. Differential geometric control theory, vol 27. pp 181–191

Broida TJ, Chandrashekhar S, Chellappa R (1990) Recursive 3-D motion estimation from a monocular image sequence. IEEE T Aero Elec Sys 26(4):639–656

Brooks R (1986) A robust layered control system for a mobile robot. IEEE T Robotic Autom 2(1):14–23

Brooks RA (1989) A robot that walks: Emergent behaviors from a carefully evolved network. MIT AI Lab, Memo 1091

Brown MZ, Burschka D, Hager GD (2003) Advances in computational stereo. IEEE T Pattern Anal 25(8):993–1008

Buehler M, Iagnemma K, Singh S (eds) (2007) The 2005 DARPA grand challenge: The great robot race. Springer Tracts in Advanced Robotics, vol 36. Springer-Verlag

Buehler M, Iagnemma K, Singh S (eds) (2010) The DARPA urban challenge. Tracts in Advanced Robotics, vol 56. Springer-Verlag

Bukowski R, Haynes LS, Geng Z, Coleman N, Santucci A, Lam K, Paz A, May R, DeVito M (1991) Robot hand-eye coordination rapid prototyping environment. In: Proc ISIR, pp 16.15–16.28

Buttazzo GC, Allotta B, Fanizza FP (1993) Mousebuster: A robot system for catching fast moving objects by vision. In: Proc. IEEE Int. Conf. Robotics and Automation, Atlanta. pp 932–937

Calonder M, Lepetit V, Strecha C, Fua P (2010) BRIEF: Binary robust independent elementary features. In: Daniilidis K, Maragos P, Paragios N (eds) Lecture notes in computer science. Computer Vision – ECCV 2010, vol 6311. Springer-Verlag, Berlin Heidelberg, pp 778–792

Canny JF (1983) Finding edges and lines in images. MIT, Artificial Intelligence Laboratory, AI-TR-720. Cambridge, MA

Canny J (1987) A computational approach to edge detection. Readings in computer vision: issues, problems, principles, and paradigms 184

Chahl JS, Srinivasan MV (1997) Reflective surfaces for panoramic imaging. Appl Optics 31(36): 8275–8285

Chaumette F (1990) La relation vision-commande: Théorie et application et des tâches robotiques. Université de Rennes 1

Chaumette F (1998) Potential problems of stability and convergence in image-based and position-based visual servoing. In: Kriegman DJ, Hager GD, Morse AS (eds) Lecture notes in control and information sciences. The confluence of vision and control, vol 237. Springer-Verlag, pp 66–78

Chaumette F (2004) Image moments: A general and useful set of features for visual servoing. IEEE T Robotic Autom 20(4):713–723

Chaumette F, Hutchinson S (2006) Visual servo control 1: Basic approaches. IEEE T Robotic Autom 13(4):82–90

Chaumette F, Hutchinson S (2007) Visual servo control 2: Advanced approaches. IEEE T Robotic Autom 14(1):109–118

Chaumette F, Rives P, Espiau B (1991) Positioning of a robot with respect to an object, tracking it and estimating its velocity by visual servoing. In: Proc. IEEE Int. Conf. Robotics and Automation, Seoul. pp 2248–2253

Chesi G, Hashimoto K (eds) (2010) Visual servoing via advanced numerical methods. Lecture notes in computer science, 401. Springer-Verlag

Chiaverini S, Sciavicco L, Siciliano B (1991) Control of robotic systems through singularities. Lecture Notes in Control and Information Sciences. Advanced Robot Control, Proceedings of the International Workshop on Nonlinear and Adaptive Control: Issues in Robotics, vol 162. Springer-Verlag, pp 285–295

Chiuso A, Favaro P, Jin H, Soatto S (2002) Structure from motion causally integrated over time. IEEE T Pattern Anal 24(4):523–535
Choset HM, Lynch KM, Hutchinson S, Kantor G, Burgard W, Kavraki LE, Thrun S (2005) Principles of robot motion. The MIT Press
Chum O, Matas J (2005) Matching with PROSAC – Progressive sample consensus. In: IEEE Conf. on Computer Vision and Pattern Recognition, vol 2. San Diego, pp 220–226
Colicchia G, Waltner C, Hopf M, Wiesner H (2009) The scallop's eye – A concave mirror in the context of biology. Physics Education 44(2):175–179
Commission Internationale de L'Éclairage (1987) Colorimetry, 2nd ed. Commission Internationale de L'Eclairage, CIE No 15.2
Corke PI (1994) High-performance visual closed-loop robot control. University of Melbourne, Dept. Mechanical and Manufacturing Engineering. http://eprints.unimelb.edu.au/archive/00000547/01/thesis.pdf
Corke PI (1996a) In situ measurement of robot motor electrical constants. Robotica 14(4):433–436
Corke PI (1996b) Visual control of robots: High-performance visual servoing. Mechatronics, vol 2. Research Studies Press (John Wiley). Out of print and available at http://www.petercorke.com/bluebook
Corke PI (2001) Mobile robot navigation as a planar visual servoing problem. In: Jarvis RA, Zelinsky A (eds) Springer tracts in advanced robotics. Robotics Research: The 10th International Symposium, vol 6. IFRR, Lorne, pp 361–372
Corke PI (2007) A simple and systematic approach to assigning Denavit-Hartenberg parameters. IEEE T Robotic Autom 23(3):590–594
Corke PI (2010) Spherical image-based visual servo and structure estimation. In: Proc. IEEE Int. Conf. Robotics and Automation, Anchorage, pp 5550–5555
Corke PI, Armstrong-Hélouvry BS (1994) A search for consensus among model parameters reported for the PUMA 560 robot. In: Proc. IEEE Int. Conf. Robotics and Automation, San Diego. pp 1608–1613
Corke PI, Armstrong-Hélouvry B (1995) A meta-study of PUMA 560 dynamics: A critical appraisal of literature data. Robotica 13(3):253–258
Corke PI, Good MC (1992) Dynamic effects in high-performance visual servoing. In: Proc. IEEE Int. Conf. Robotics and Automation, Nice. pp 1838–1843
Corke PI, Good MC (1996) Dynamic effects in visual closed-loop systems. IEEE T Robotic Autom 12(5):671–683
Corke PI, Hutchinson SA (2001) A new partitioned approach to image-based visual servo control. IEEE T Robotic Autom 17(4):507–515
Corke PI, Dunn PA, Banks JE (1999) Frame-rate stereopsis using non-parametric transforms and programmable logic. In: Proc. IEEE Int. Conf. Robotics and Automation, Detroit. pp 1928–1933
Corke PI, Strelow D, Singh S (2004) Omnidirectional visual odometry for a planetary rover. In: Proc. Int. Conf on Intelligent Robots and Systems (IROS), Sendai. pp 4007–4012
Corke PI, Spindler F, Chaumette F (2009) Combining Cartesian and polar coordinates in IBVS. In: Proc. Int. Conf on Intelligent Robots and Systems (IROS), St. Louis. pp 5962–5967
Craig JJ (1987) Adaptive control of mechanical manipulators. Addison-Wesley
Craig JJ (2004) Introduction to robotics: Mechanics and control. Prentice Hall
Craig JJ, Hsu P, Sastry SS (1987) Adaptive control of mechanical manipulators. Int J Robot Res 6(2):16
Cummins M, Newman P (2008) FAB-MAP: Probabilistic localization and mapping in the space of appearance. Int J Robot Res 27(6):647
Cutting JE (1997) How the eye measures reality and virtual reality. Behav Res Meth Ins C 29(1):27–36
Daniilidis K, Klette R (eds) (2006) Imaging beyond the pinhole camera. Computational Imaging, vol 33. Springer-Verlag
Davison AJ, Reid ID, Molton ND, Stasse O (2007) MonoSLAM: Real-time single camera SLAM. IEEE T Pattern Anal 29(6):1052–1067
Deguchi K (1998) Optimal motion control for image-based visual servoing by decoupling translation and rotation. In: Proc. Int. Conf on Intelligent Robots and Systems (IROS), Victoria, Canada. Oct, pp 705–711
Dellaert F, Seitz SM, Thorpe CE, Thrun S (2000) Structure from motion without correspondence. In: IEEE Conf. on Computer Vision and Pattern Recognition, Hilton Head Island, SC. pp 557–564
DeMenthon D, Davis LS (1992) Exact and approximate solutions of the perspective-three-point problem. IEEE T Pattern Anal 14(11):1100–1105

Denavit J, Hartenberg RS (1955) A kinematic notation for lower-pair mechanisms based on matrices. J Appl Mech-T ASME 22(1):215–221
Deo AS, Walker ID (1995) Overview of damped least-squares methods for inverse kinematics of robot manipulators. J Intell Robot Syst 14(1):43–68
Deriche R, Giraudon G (1993) A computational approach for corner and vertex detection. Int J Comput Vision 10(2):101–124
DeWitt BA, Wolf PR (2000) Elements of photogrammetry (with applications in GIS). McGraw-Hill Higher Education
Dickmanns ED (2007) Dynamic vision for perception and control of motion. Springer-Verlag, London
Dickmanns ED, Graefe V (1988a) Applications of dynamic monocular machine vision. Mach Vision Appl 1:241–261
Dickmanns ED, Graefe V (1988b) Dynamic monocular machine vision. Mach Vision Appl 1(4): 223–240
Dickmanns ED, Zapp A (1987) Autonomous high speed road vehicle guidance by computer vision. In: Tenth Triennial World Congress of the International Federation of Automatic Control, vol 4. Munich, pp 221–226
Dijkstra EW (1959) A note on two problems in connexion with graphs. Numer Math 1(1):269–271
Dougherty ER, Lotufo RA (2003) Hands-on morphological image processing. Society of Photo-Optical Instrumentation Engineers (SPIE)
Duda RO, Hart PE (1972) Use of the Hough transformation to detect lines and curves in pictures. Commun ACM 15(1):11–15
Espiau B, Chaumette F, Rives P (1992) A new approach to visual servoing in robotics. IEEE T Robotic Autom 8(3):313–326
Everett HR (1995) Sensors for mobile robots: Theory and application. AK Peters, Ltd.
Faugeras OD (1993) Three-dimensional computer vision: A geometric viewpoint. The MIT Press
Faugeras OD, Lustman F (1988) Motion and structure from motion in a piecewise planar environment. Int J Pattern Recogn 2(3):485–508
Faugeras O, Luong QT, Papadopoulou T (2001) The geometry of multiple images: The laws that govern the formation of images of a scene and some of their applications. The MIT Press
Featherstone R (1987) Robot dynamics algorithms. Kluwer Academic
Feddema JT (1989) Real time visual feedback control for hand-eye coordinated robotic systems. Purdue University
Feddema JT, Mitchell OR (1989) Vision-guided servoing with feature-based trajectory generation. IEEE T Robotic Autom 5(5):691–700
Feddema JT, Lee CSG, Mitchell OR (1991) Weighted selection of image features for resolved rate visual feedback control. IEEE T Robotic Autom 7(1):31–47
Felzenszwalb PF, Huttenlocher DP (2004) Efficient graph-based image segmentation. Int J Comput Vision 59(2):167–181
Ferguson D, Stentz A (2006) Using interpolation to improve path planning: The Field D* algorithm. J Field Robotics 23(2):79–101
Fischler MA, Bolles RC (1981) Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. Commun ACM 24(6):381–395
Fisher RB (2004) The PETS04 surveillance ground-truth data sets. In: Proc. 6th IEEE International Workshop on Performance Evaluation of Tracking and Surveillance, Prague. pp 1–5
Fomena R, Chaumette F (2007) Visual servoing from spheres using a spherical projection model. In: Proc. IEEE Int. Conf. Robotics and Automation, Rome, pp 2080–2085
Förstner W (1994) A framework for low level feature extraction. In: Ecklundh J-O (ed) Lecture notes in computer science. Computer Vision – ECCV 1994, vol 800. Springer-Verlag, Berlin Heidelberg, pp 383–394
Förstner W, Gülch E (1987) A fast operator for detection and precise location of distinct points, corners and centres of circular features. In: ISPRS Intercommission Workshop, Interlaken, pp 149–155
Forsyth DA, Ponce J (2002) Computer vision: A modern approach. Prentice Hall
Freeman H (1974) Computer processing of line-drawing images. ACM Comput Surv 6(1):57–97
Friedman DP, Felleisen M, Bibby D (1987) The little LISPer. MIT Press
Funda J, Taylor RH, Paul RP (1990) On homogeneous transforms, quaternions, and computational efficiency. IEEE T Robotic Autom 6(3):382–388

Gans NR, Hutchinson SA, Corke PI (2003) Performance tests for visual servo control systems, with application to partitioned approaches to visual servo control. Int J Robot Res 22(10–11):955
Gautier M, Khalil W (1992) Exciting trajectories for the identification of base inertial parameters of robots. Int J Robot Res 11(4):362
Geraerts R, Overmars MH (2004) A comparative study of probabilistic roadmap planners. In: Boissonnat J-D, Burdick J, Goldberg K, Hutchinson S (eds) Springer Tracts in Advanced Robotics. Algorithmic Foundations of Robotics V, vol 7. Springer-Verlag, pp 43–58
Geyer C, Daniilidis K (2000) A unifying theory for central panoramic systems and practical implications. In: Vernon D (ed) Lecture notes in computer science. Computer vision – ECCV 2000, vol 1843. Springer-Verlag, pp 445–461
Goldberg K (ed) (2001) The robot in the garden: Telerobotics and telepistemology in the age of the internet. The MIT Press
Goldberg K, Siegwart R (eds) (2001) Beyond webcams: An introduction to online robots. The MIT Press
Gonzalez R, Woods R (2008) Digital image processing, 3rd ed. Prentice Hall
Gonzalez R, Woods R, Eddins S (2009) Digital image processing using MATLAB, 2nd ed. Gatesmark
Groves PD (2008) Principles of GNSS, inertial, and multisensor integrated navigation systems. Artech House
Hager GD, Toyama K (1998) X Vision: A portable substrate for real-time vision applications. Comput Vis Image Und 69(1):23–37
Hamel T, Mahony R (2002) Visual servoing of an under-actuated dynamic rigid-body system: An image based approach. IEEE T Robotic Autom 18(2):187–198
Hamel T, Mahony R, Lozano R, Ostrowski J (2002) Dynamic modelling and configuration stabilization for an X4-flyer. IFAC World Congress 1(2), p 3. Available at: http://citeseerx.ist.psu.edu/viewdoc/download
Hansen P, Corke PI, Boles W (2010) Wide-angle visual feature matching for outdoor localization. Int J Robot Res 29(1–2):267–297
Harris CG, Stephens MJ (1988) A combined corner and edge detector. In: Proceedings of the Fourth Alvey Vision Conference, Manchester. pp 147–151
Hart PE (2009) How the Hough transform was invented [DSP history]. IEEE Signal Proc Mag 26(6): 18–22
Hartenberg RS, Denavit J (1964) Kinematic synthesis of linkages. McGraw-Hill New York, available online at http://kmoddl.library.cornell.edu/bib.php?m=23
Hartley R, Zisserman A (2003) Multiple view geometry in computer vision. Cambridge University Press, New York
Harvey P (nd) ExifTool. http://www.sno.phy.queensu.ca/~phil/exiftool
Hashimoto K (ed) (1993) Visual servoing. In: Robotics and automated systems, vol 7. World Scientific
Hashimoto K, Kimoto T, Ebine T, Kimura H (1991) Manipulator control with image-based visual servo. In: Proc. IEEE Int. Conf. Robotics and Automation, Seoul. pp 2267–2272
Hellerstein JL, Diao Y, Parekh S, Tilbury DM (2004) Feedback control of computing systems. IEEE Press – Wiley
Hill J, Park WT (1979) Real time control of a robot with a mobile camera. In: Proc. 9th ISIR, SME, Washington, DC. Mar, pp 233–246
Hoag D (1963) Consideration of Apollo IMU gimbal lock. MIT Instrumentation Laboratory, E–1344, http://www.hq.nasa.gov/alsj/e-1344.htm
Hollerbach JM (1980) A recursive Lagrangian formulation of manipulator dynamics and a comparative study of dynamics formulation complexity. IEEE T Syst Man Cyb 10(11):730–736, Nov
Hollerbach JM (1982) Dynamics. In: Brady M, Hollerbach JM, Johnson TL, Lozano-Perez T, Mason MT (eds) Robot motion – Planning and control. The MIT Press, pp 51–71
Horaud R, Canio B, Leboullenx O (1989) An analytic solution for the perspective 4-point problem. Comput Vision Graph 47(1):33–44
Horn BKP (1987) Closed-form solution of absolute orientation using unit quaternions. J Opt Soc Am A 4(4):629–642
Horn BKP, Hilden HM, Negahdaripour S (1988) Closed-form solution of absolute orientation using orthonormal matrices. J Opt Soc Am A 5(7):1127–1135
Hosoda K, Asada M (1994) Versatile visual servoing without knowledge of true Jacobian. In: Proc. Int. Conf on Intelligent Robots and Systems (IROS), Munich. Sep, pp 186–193

Howard TM, Green CJ, Kelly A, Ferguson D (2008) State space sampling of feasible motions for high-perfor-mance mobile robot navigation in complex environments. J Field Robotics 25(6–7):325–345
Hu MK (1962) Visual pattern recognition by moment invariants. IRE T Inform Theor 8:179–187
Huang TS, Netravali AN (1994) Motion and structure from feature correspondences: A review. P IEEE 82(2):252–268
Humenberger M, Zinner C, Kubinger W (2009) Performance evaluation of a census-based stereo matching algorithm on embedded and multi-core hardware. In: Proc. 19th Int. Symp. on Image and Signal Processing and Analysis (ISPA). Sep, pp 388–393
Hunt RWG (1987) The reproduction of colour, 4th ed. Fountain Press
Hunter RS, Harold RW (1987) The measurement of appearance. John Wiley
Hutchinson S, Hager G, Corke PI (1996) A tutorial on visual servo control. IEEE T Robotic Autom 12(5):651–670
Iwatsuki M, Okiyama N (2002a) A new formulation of visual servoing based on cylindrical coordinate system with shiftable origin. In: Proc. Int. Conf on Intelligent Robots and Systems (IROS), Lausanne, pp 354–359
Iwatsuki M, Okiyama N (2002b) Rotation-oriented visual servoing based on cylindrical coordinates. In: Proc. IEEE Int. Conf. Robotics and Automation, Washington, DC. May, pp 4198–4203
Izaguirre A, Paul RP (1985) Computation of the inertial and gravitational coefficients of the dynamics equations for a robot manipulator with a load. In: Proc. IEEE Int. Conf. Robotics and Automation. Mar, pp 1024–1032
Jägersand M, Fuentes O, Nelson R (1996) Experimental evaluation of uncalibrated visual servoing for pre-cision manipulation. In: Proc. IEEE Int. Conf. Robotics and Automation, Albuquerque, NM. pp 2874–2880
Jarvis RA, Byrne JC (1988) An automated guided vehicle with map building and path finding capabilities. In: Robotics Research: The Fourth international symposium. The MIT Press, pp 497–504
Jazwinski AH (1970) Stochastic processes and filtering theory. Academic Press
Jebara T, Azarbayejani A, Pentland A (1999) 3D structure from 2D motion. IEEE Signal Proc Mag 16(3):66–84
Julier SJ, Uhlmann JK (2004) Unscented filtering and nonlinear estimation. P IEEE 92(3):401–422
Kahn ME (1969) The near-minimum time control of open-loop articulated kinematic linkages. Stanford University, AIM-106
Kálmán RE (1960) A new approach to linear filtering and prediction problems. J Basic Eng-T Asme 82(1): 35–45
Kane TR, Levinson DA (1983) The use of Kane's dynamical equations in robotics. Int J Robot Res 2(3):3–21
Kavraki LE, Svestka P, Latombe JC, Overmars MH (1996) Probabilistic roadmaps for path planning in high-dimensional configuration spaces. IEEE T Robotic Autom 12(4):566–580
Kelly R (1996) Robust asymptotically stable visual servoing of planar robots. IEEE T Robotic Autom 12(5):759–766
Kelly R, Carelli R, Nasisi O, Kuchen B, Reyes F (2002a) Stable visual servoing of camera-in-hand robotic systems. IEEE-ASME T Mech 5(1):39–48
Kelly R, Shirkey P, Spong MW (2002b) Fixed-camera visual servo control for planar robots. In: Proc. IEEE Int. Conf. Robotics and Automation. IEEE, Washington, DC, pp 2643–2649
Khalil W, Creusot D (1997) SYMORO+: A system for the symbolic modelling of robots. Robotica 15(2): 153–161
Khalil W, Dombre E (2002) Modeling, identification and control of robots. Kogan Page Science
King-Hele D (2002) Erasmus Darwin's improved design for steering carriages and cars. Notes and Records of the Royal Society of London 56(1):41–62
Klafter RD, Chmielewski TA, Negin M (1989) Robotic engineering – An integrated approach. Prentice-Hall
Klein CA, Huang CH (1983) Review of pseudoinverse control for use with kinematically redundant manipulators. IEEE T Syst Man Cyb 13:245–250
Klette R, Kruger N, Vaudrey T, Pauwels K, van Hulle M, Morales S, Kandil F, Haeusler R, Pugeault N, Rabe C (2011) Performance of correspondence algorithms in vision-based driver assistance using an online image sequence database. IEEE T Veh Technol 60(5):2012–2026
Koenderink JJ (1984) The structure of images. Biol Cybern 50(5):363–370
Koenig S, Likhachev M (2002) D* Lite. In: Proceedings of the National Conference on Artificial Intelligence, Menlo Park, CA; Cambridge, MA; London; AAAI Press; MIT Press; 1999, pp 476–483

Koenig S, Likhachev M (2005) Fast replanning for navigation in unknown terrain. IEEE T Robotic Autom 21(3):354–363
Kriegman DJ, Hager GD, Morse AS (eds) (1998) The confluence of vision and control. Lecture Notes in Control and Information Sciences, vol 237. Springer-Verlag
Kuipers JB (1999) Quaternions and rotation sequences: A primer with applications to orbits, aeroespace and virtual reality. Princeton University Press
Lamport L (1994) LATEX: A document preparation system. User's guide and reference manual. Addison-Wesley Publishing Company, Reading, Ma
LaValle SM (1998) Rapidly-exploring random trees: A new tool for path planning. Computer Science Dept., Iowa State University, TR 98–11
LaValle SM (2006) Planning algorithms. Cambridge Univ Press
LaValle SM, Kuffner JJ (2001) Randomized kinodynamic planning. Int J Robot Res 20(5):378–400
Leavers VF (1993) Which Hough transform? Comput Vis Image Und 58(2):250–264
Lee CSG, Lee BH, Nigham R (1983) Development of the generalized D'Alembert equations of motion for mechanical manipulators. In: Proc. 22nd CDC, San Antonio, Texas. pp 1205–1210
Lepetit V, Moreno-Noguer F, Fua P (2009) EPnP: An accurate O(n) solution to the PnP problem. Int J Comput Vision 81(2):155–166
Li H, Hartley R (2006) Five-point motion estimation made easy. In: 18th Int. Conf. on Pattern Recognition ICPR 2006, Hong Kong, pp 630–633
Lin Z, Zeman V, Patel RV (1989) On-line robot trajectory planning for catching a moving object. In: Proc. IEEE Int. Conf. Robotics and Automation. pp 1726–1731
Lindeberg T (1993) Scale-space theory in computer vision. Springer-Verlag
Lloyd J, Hayward V (1991) Real-time trajectory generation using blend functions. In: Proc. IEEE Int. Conf. Robotics and Automation, Seoul. pp 784–789
Longuet-Higgins H (1981) A computer algorithm for reconstruction of a scene from two projections. Nature 293:133–135
Lourakis MIA, Argyros AA (2009) SBA: A software package for generic sparse bundle adjustment. ACM T Math Software 36(1):1–30
Lovell J, Kluger J (1994) Apollo 13. Coronet Books
Lowe DG (1991) Fitting parametrized three-dimensional models to images. IEEE T Pattern Anal 13(5): 441–450
Lowe DG (2004) Distinctive image features from scale-invariant keypoints. Int J Comput Vision 60(2): 91–110
Lucas SM (2005) ICDAR 2005 text locating competition results. In: Proceedings of the Eighth International Conference on Document Analysis and Recognition, ICDAR05, pp 80–84
Lucas BD, Kanade T (1981) An iterative image registration technique with an application to stereo vision. In: International joint conference on artificial intelligence (IJCAI), Vancouver, vol 2. http://ijcai.org/Past%20Proceedings/IJCAI-81-VOL-2/PDF/017.pdf, pp 674–679
Luh JYS, Walker MW, Paul RPC (1980) On-line computational scheme for mechanical manipulators. J Dyn Syst-T ASME 102(2):69–76
Lumelsky V, Stepanov A (1986) Dynamic path planning for a mobile automaton with limited information on the environment. IEEE T Automat Contr 31(11):1058–1063
Luong QT (1992) matrice fondamentale et autocalibration en vision par ordinateur. Universitéde Paris-Sud, Orsay, France
Ma Y, Kosecka J, Soatto S, Sastry S (2003) An invitation to 3D. Springer-Verlag
Maimone M, Cheng Y, Matthies L (2007) Two years of visual odometry on the Mars exploration rovers. J Field Robotics 24(3):169–186
Makhlin AG (1985) Stability and sensitivity of servo vision systems. In: Proc 5th Int Conf on Robot Vision and Sensory Controls – RoViSeC 5. IFS (Publications), Amsterdam, pp 79–89
Malis E, Vargas M (2007) Deeper understanding of the homography decomposition for vision-based control. INRIA, 6303
Malis E, Chaumette F, Boudet S (1999) 2-1/2D visual servoing. IEEE T Robotic Autom 15(2):238–250
Marey M, Chaumette F (2008) Analysis of classical and new visual servoing control laws. In: Proc. IEEE Int. Conf. Robotics and Automation, Pasadena, pp 3244–3249
Mariottini GL, Prattichizzo D (2005) EGT for multiple view geometry and visual servoing: Robotics vision with pinhole and panoramic cameras. IEEE T Robotic Autom 12(4):26–39

Mariottini GL, Oriolo G, Prattichizzo D (2007) Image-based visual servoing for nonholonomic mobile robots using epipolar geometry. IEEE T Robotic Autom 23(1):87–100
Marr D (2010) Vision: A computational investigation into the human representation and processing of visual information. The MIT Press
Martin D, Fowlkes C, Tal D, Malik J (2001) A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics. Proc. 8th Int'l Conf. Computer Vision, vol 2, pp 416–423
Masutani Y, Mikawa M, Maru N, Miyazaki F (1994) Visual servoing for non-holonomic mobile robots. In: Proc. Int. Conf on Intelligent Robots and Systems (IROS), Munich, pp 1133–1140
Matarić MJ (2007) The robotics primer. MIT Press
Matas J, Chum O, Urban M, Pajdla T (2004) Robust wide-baseline stereo from maximally stable extremal regions. Image Vision Comput 22(10):761–767
Matthews ND, An PE, Harris CJ (1995) Vehicle detection and recognition for autonomous intelligent cruise control. Image, Speech and Intelligent Systems 6
Matthies L (1992) Stereo vision for planetary rovers: Stochastic modeling to near real-time implementation. Int J Comput Vision 8(1):71–91
Mayeda H, Yoshida K, Osuka K (1990) Base parameters of manipulator dynamic models. IEEE T Robotic Autom 6(3):312–321
McLauchlan PF (1999) The variable state dimension filter applied to surface-based structure from motion. University of Surrey, VSSP-TR-4/99
Merlet JP (2006) Parallel robots. Kluwer Academic
Mettler B (2003) Identification modeling and characteristics of miniature rotorcraft. Kluwer Academic
Mičušík B, Pajdla T (2003) Estimation of omnidirectional camera model from epipolar geometry. In: IEEE Conf. on Computer Vision and Pattern Recognition, Madison, vol 1. pp 485–490
Middleton RH, Goodwin GC (1988) Adaptive computed torque control for rigid link manipulations. Syst Control Lett 10(1):9–16
Mikolajczyk K, Schmid C (2004) Scale and affine invariant interest point detectors. Int J Comput Vision 60(1):63–86
Mikolajczyk K, Schmid C (2005) A performance evaluation of local descriptors. IEEE T Pattern Anal 27(10):1615–1630
Mindell DA (2008) Digital Apollo. MIT Press
Molton N, Brady M (2000) Practical structure and motion from stereo when motion is unconstrained. Int J Comput Vision 39(1):5–23
Montemerlo M, Thrun S, Koller D, Wegbreit B (2002) FastSLAM: A factored solution to the simultaneous localization and mapping problem. In: Proceedings of the AAAI National Conference on Artificial Intelligence. AAAI, Edmonton, Canada
Moravec H (1980) Obstacle avoidance and navigation in the real world by a seeing robot rover. Stanford University
Morel G, Liebezeit T, Szewczyk J, Boudet S, Pot J (2000) Explicit incorporation of 2D constraints in vision based control of robot manipulators. In: Corke PI, Trevelyan J (eds) Lecture notes in control and information sciences. Experimental robotics VI, vol 250. Springer-Verlag, pp 99–108
NASA (1970) Apollo 13: Technical air-to-ground voice transcription. Test Division, Apollo Spacecraft Program Office, http://www.hq.nasa.gov/alsj/a13/AS13_TEC.PDF
Nayar SK (1997) Catadioptric omnidirectional camera. In: IEEE Conf. on Computer Vision and Pattern Recognition, Los Alamitos, CA, pp 482–488
Nethery JF, Spong MW (1994) Robotica: A mathematica package for robot analysis. IEEE T Robotic Autom 1(1):13–20
Ng J, Bräunl T (2007) Performance comparison of bug navigation algorithms. J Intell Robot Syst 50(1):73–84
Niblack W (1985) An introduction to digital image processing. Strandberg Publishing Company Birkeroed, Denmark
Nilsson NJ (1971) Problem-solving methods in artificial intelligence. McGraw-Hill
Nistér D (2003) An efficient solution to the five-point relative pose problem. In: IEEE Conf. on Computer Vision and Pattern Recognition, vol. 2. Madison, pp 195–202
Nistér D, Naroditsky O, Bergen J (2006) Visual odometry for ground vehicle applications. J Field Robotics 23(1):3–20
Nixon MS, Aguado AS (2008) Feature extraction and image processing. Academic Press
Noble JA (1988) Finding corners. Image Vision Comput 6(2):121–128

Okutomi M, Kanade T (1993) A multiple-baseline stereo. IEEE T Pattern Anal 15(4):353–363
Ollis M, Herman H, Singh S (1999) Analysis and design of panoramic stereo vision using equi-angular pixel cameras. Robotics Institute, Carnegie Mellon University, CMU-RI-TR-99-04, Citeseer, Pittsburgh, PA
Orin DE, McGhee RB, Vukobratovic M, Hartoch G (1979) Kinematics and kinetic analysis of open-chain linkages utilizing newton-euler methods. Math Biosci 43(1/2):107–130
Ortega R, Spong MW (1989) Adaptive motion control of rigid robots: A tutorial. Automatica 25(6):877–888
Otsu N (1975) A threshold selection method from gray-level histograms. Automatica 11:285–296
Papanikolopoulos NP, Khosla PK (1993) Adaptive robot visual tracking: Theory and experiments. IEEE T Automat Contr 38(3):429–445
Papanikolopoulos NP, Khosla PK, Kanade T (1993) Visual tracking of a moving target by a camera mounted on a robot: A combination of vision and control. IEEE T Robotic Autom 9(1):14–35
Park FC (1994) Computational aspects of the product-of-exponentials formula for robot kinematics. Automatic Control, IEEE Transactions on 39(3):643–647
Paul R (1972) Modelling, trajectory calculation and servoing of a computer controlled arm. Stanford University
Paul R (1979) Manipulator Cartesian path control. IEEE T Syst Man Cyb 9:702–711
Paul RP (1981) Robot manipulators: Mathematics, programming, and control. MIT Press, Cambridge, Massachusetts
Paul RP, Shimano B (1978) Kinematic control equations for simple manipulators. In: IEEE Conference on Decision and Control, vol 17. pp 1398–1406
Paul RP, Zhang H (1986) Computationally efficient kinematics for manipulators with spherical wrists based on the homogeneous transformation representation. Int J Robot Res 5(2):32–44
Piepmeier JA, McMurray G, Lipkin H (1999) A dynamic quasi-Newton method for uncalibrated visual servoing. In: Proc. IEEE Int. Conf. Robotics and Automation, Detroit. pp 1595–1600
Pilu M (1997) A direct method for stereo correspondence based on singular value decomposition. In: Proc. Computer Vision and Pattern Recognition, IEEE Computer Society, San Juan, pp 261–266
Pollefeys M, Nistér D, Frahm JM, Akbarzadeh A, Mordohai P, Clipp B, Engels C, Gallup D, Kim SJ, Merrell P, et al. (2008) Detailed real-time urban 3D reconstruction from video. Int J Comput Vision 78(2):143–167, Jul
Pomerleau D, Jochem T (1995) No hands across America Journal. http://www.cs.cmu.edu/~tjochem/nhaa/Journal.html
Pomerleau D, Jochem T (1996) Rapidly adapting machine vision for automated vehicle steering. IEEE Expert 11(1):19–27
Pounds P (2007) Design, construction and control of a large quadrotor micro air vehicle. Australian National University
Pounds P, Mahony R, Gresham J, Corke PI, Roberts J (2004) Towards dynamically-favourable quad-rotor aerial robots. In: Proc. Australasian Conf. on Robotics and Automation, Canberra
Pounds P, Mahony R, Corke PI (2006) A practical quad-rotor robot. In: Proc. Australasian Conf. on Robotics and Automation, Auckland
Pounds P, Mahony R, Corke PI (2007) System identification and control of an aerobot drive system. In: Information, Decision and Control. pp 154–159
Poynton CA (2003) Digital video and HDTV: Algorithms and interfaces. Morgan Kaufmann
Press WH, Teukolsky SA, Vetterling WT, Flannery BP (2007) Numerical recipes, 3rd ed. Cambridge University Press
Prouty RW (2002) Helicopter performance, stability, and control. Krieger
Pynchon T (2006) Against the day. Jonathan Cape
Rabaud V (nd) Vincent's structure from motion toolbox. http://vision.ucsd.edu/~vrabaud/toolbox
Rives P, Chaumette F, Espiau B (1989) Positioning of a robot with respect to an object, tracking it and estimating its velocity by visual servoing. In: Hayward V, Khatib O (eds) Lecture Notes in Control and Information Sciences. Experimental Robotics I, vol 139. Springer-Verlag, pp 412–428
Rizzi AA, Koditschek DE (1991) Preliminary experiments in spatial robot juggling. In: Chatila R, Hirzinger G (eds) Lecture Notes in Control and Information Sciences. Experimental Robotics II, vol 190. Springer-Verlag, pp 282–298
Roberts LG (1963) Machine perception of three-dimensional solids. MIT Lincoln Laboratory, TR 315, http://www.packet.cc/files/mach-per-3D-solids.html
Rosenfield GH (1959) The problem of exterior orientation in photogrammetry. Photogramm Eng 25(4):536–553

Rosten E, Porter R, Drummond T (2010) FASTER and better: A machine learning approach to corner detection. IEEE T Pattern Anal 32:105–119
Sakaguchi T, Fujita M, Watanabe H, Miyazaki F (1993) Motion planning and control for a robot performer. In: Proc. IEEE Int. Conf. Robotics and Automation, Atlanta. May, pp 925–931
Salvi J, Matabosch C, Fofi D, Forest J (2007) A review of recent range image registration methods with accuracy evaluation. Image Vision Comput 25(5):578–596
Samson C, Espiau B, Le Borgne M (1990) Robot control: The task function approach. Oxford University Press
Sanderson AC, Weiss LE, Neuman CP (1987) Dynamic sensor-based control of robots with visual feedback. IEEE T Robotic Autom RA-3(5):404–417
Scharstein D, Pal C (2007) Learning conditional random fields for stereo. In: IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2007), Minneapolis, MN
Scharstein D, Szeliski R (2002) A taxonomy and evaluation of dense two-frame stereo correspondence algorithms. Int J Comput Vision 47(1):7–42
Serra J (1983) Image analysis and mathematical morphology. Academic Press
Shi J, Tomasi C (1994) Good features to track. In: Proc. Computer Vision and Pattern Recognition. IEEE Computer Society, Seattle, pp 593–593
Shirai Y (1987) Three-dimensional computer vision. Springer-Verlag, New York
Shirai Y, Inoue H (1973) Guiding a robot by visual feedback in assembling tasks. Pattern Recogn 5(2):99–106
Shoemake K (1985) Animating rotation with quaternion curves. In: Proceedings of ACM SIGGRAPH, San Francisco, pp 245–254
Siciliano B, Khatib O (eds) (2008) Springer handbook of robotics. Springer-Verlag, New York
Siciliano B, Sciavicco L, Villani L, Oriolo G (2008) Robotics: Modelling, planning and control. Springer-Verlag
Siegwart R, Nourbakhsh IR, Scaramuzza D (2011) Introduction to autonomous mobile robots. The MIT Press
Silver WM (1982) On the equivalance of Lagrangian and Newton-Euler dynamics for manipulators. Int J Robot Res 1(2):60–70
Sivic J, Zisserman A (2003) Video Google: A text retrieval approach to object matching in videos. In: Proc. Ninth IEEE Int. Conf. on Computer Vision, pp 1470–1477
Skaar SB, Brockman WH, Hanson R (1987) Camera-space manipulation. Int J Robot Res 6(4):20–32
Skofteland G, Hirzinger G (1991) Computing position and orientation of a freeflying polyhedron from 3D data. In: Proc. IEEE Int. Conf. Robotics and Automation, Seoul. pp 150–155
Slama CC (ed) (1980) Manual of photogrammetry, 4th ed. American Society of Photogrammetry
Sobel D (1996) Longitude: The true story of a lone genius who solved the greatest scientific problem of his time. Fourth Estate London
Soille P (2003) Morphological image analysis: Principles and applications. Springer-Verlag
Spong MW (1989) Adaptive control of flexible joint manipulators. Syst Control Lett 13(1):15–21
Spong MW, Hutchinson S, Vidyasagar M (2006) Robot modeling and control. Wiley
Srinivasan VV, Venkatesh S (1997) From living eyes to seeing machines. Oxford University Press
Stentz A (1994) The D* algorithm for real-time planning of optimal traverses. The Robotics Institute, Carnegie-Mellon University, CMU-RI-TR-94-37
Strelow D, Singh S (2004) Motion estimation from image and inertial measurements. Int J Robot Res 23(12):1157–1195
Sussman GJ, Wisdom J, Mayer ME (2001) Structure and interpretation of classical mechanics. The MIT Press
Sutherland IE (1974) Three-dimensional data input by tablet. P IEEE 62(4):453–461
Svoboda T, Pajdla T (2002) Epipolar geometry for central catadioptric cameras. Int J Comput Vision 49(1):23–37
Szeliski R (2011) Computer vision: Algorithms and applications. Springer-Verlag
Tahri O, Chaumette F (2005) Point-based and region-based image moments for visual servoing of planar objects. IEEE T Robotic Autom 21(6):1116–1127
Tahri O, Mezouar Y, Chaumette F, Corke PI (2009) Generic decoupled image-based visual servoing for cameras obeying the unified projection model. In: Proc. IEEE Int. Conf. Robotics and Automation, Kobe, pp 1116–1121
Taylor RA (1979) Planning and execution of straight line manipulator trajectories. IBM J Res Dev 23(4):424–436

ter Haar Romeny BM (1996) Introduction to scale-space theory: Multiscale geometric image analysis. Utrecht Univ.
Thrun S, Burgard W, Fox D (2005) Probabilistic robotics. The MIT Press
Tissainayagam P, Suter D (2004) Assessing the performance of corner detectors for point feature tracking applications. Image Vision Comput 22(8):663–679
Tomasi C, Kanade T (1991) Detection and tracking of point features. Carnegie Mellon University, CMU-CS-91-132
Torr PHS (2002) A structure and motion toolkit in MATLAB – Interactive adventures in S and M. Microsoft Research. MSR-TR-2002-56, Cambridge, UK
Triggs B, McLauchlan P, Hartley R, Fitzgibbon A (2000) Bundle adjustment – A modern synthesis. Lecture notes in computer science. Vision algorithms: theory and practice, vol 1883. Springer-Verlag, pp 153–177
Tsakiris D, Rives P, Samson C (1998) Extending visual servoing techniques to nonholonomic mobile robots. In: Kriegman DJ, Hager GD, Morse AS (eds) Lecture Notes in Control and Information Sciences. The confluence of vision and control, vol 237. Springer-Verlag, pp 106–117
Uicker JJ (1965) On the dynamic analysis of spatial linkages using 4 by 4 matrices. Dept. Mechanical Engineering and Astronautical Sciences, NorthWestern University
Usher K (2005) Visual homing for a car-like vehicle. Queensland University of Technology
Usher K, Ridley P, Corke PI (2003) Visual servoing of a car-like vehicle – An application of omnidirectional vision. In: Proc. IEEE Int. Conf. Robotics and Automation, Taipai. Sep, pp 4288–4293
Vedaldi A, Fulkerson B (2008) VLFeat: An open and portable library of computer vision algorithms. http://www.vlfeat.org
Walker MW, Orin DE (1982) Efficient dynamic computer simulation of robotic mechanisms. J Dyn Syst-T ASME 104(3):205–211
Walter WG (1950) An imitation of life. Sci Am 182(5):42–45
Walter WG (1951) A machine that learns. Sci Am 185(2):60–63
Walter WG (1953) The living brain. Duckworth London
Weiss LE (1984) Dynamic visual servo control of robots: An adaptive image-based approach. Carnegie-Mellon University
Weiss L, Sanderson AC, Neuman CP (1987) Dynamic sensor-based control of robots with visual feedback. IEEE T Robotic Autom 3(1):404–417
Westmore DB, Wilson WJ (1991) Direct dynamic control of a robot using an end-point mounted camera and Kalman filter position estimation. In: Proc. IEEE Int. Conf. Robotics and Automation, Seoul. Apr, pp 2376–2384
Whitney DE (1969) Resolved motion rate control of manipulators and human prostheses. IEEE T Man Machine 10(2):47–53
Wiener N (1965) Cybernetics or control and communication in the animal and the machine. The MIT Press
Wolf PR (1974) Elements of photogrammetry. McGraw-Hill
Woodfill J, Von Herzen B (1997) Real-time stereo vision on the PARTS reconfigurable computer. In: Proc. IEEE Symposium on FPGAs for Custom Computing Machines, Grenoble. pp 201–210
Xu G, Zhang Z (1996) Epipolar geometry in stereo, motion, and object recognition: A unified approach. Springer-Verlag
Ying X, Hu Z (2004) Can we consider central catiodioptric cameras and fisheye cameras within a unified imaging model. In: Pajdla T, Matas J (eds) Lecture notes in computer science. Computer vision – ECCV 2004, vol 3021. Springer-Verlag, pp 442–455
Yoshikawa T (1984) Analysis and control of robot manipulators with redundancy. In: Brady M, Paul R (eds) Robotics Research: The First International Symposium. The MIT Press, pp 735–747
Zabih R, Woodfill J (1994) Non-parametric local transforms for computing visual correspondence. In: Ecklundh J-O (ed) Lecture notes in computer science. Computer Vision – ECCV 1994, vol 800. Springer-Verlag, Berlin Heidelberg, pp 151–158
Zarchan P, Musoff H (2005) Fundamentals of Kalman filtering: A practical approach. Progress in Astronautics and Aeronautics, vol 208. American Institute of Aeronautics and Astronautics
Zhang Z, Faugeras O, Kohonen T, Hunag TS, Schroeder MR (1992) Three D-dynamic scene analysis: A stereo based approach. Springer-Verlag, New York

反侵权盗版声明

电子工业出版社依法对本作品享有专有出版权。任何未经权利人书面许可，复制、销售或通过信息网络传播本作品的行为；歪曲、篡改、剽窃本作品的行为，均违反《中华人民共和国著作权法》，其行为人应承担相应的民事责任和行政责任，构成犯罪的，将被依法追究刑事责任。

为了维护市场秩序，保护权利人的合法权益，我社将依法查处和打击侵权盗版的单位和个人。欢迎社会各界人士积极举报侵权盗版行为，本社将奖励举报有功人员，并保证举报人的信息不被泄露。

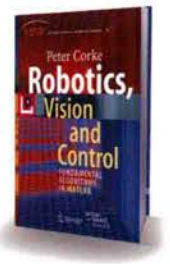
举报电话：（010）88254396；（010）88258888
传　　真：（010）88254397
E-mail：　dbqq@phei.com.cn
通信地址：北京市海淀区万寿路 173 信箱
　　电子工业出版社总编办公室
邮　　编：100036

华信 欢迎登录 免费获取优质教学资源
http://www.hxedu.com.cn

经典译丛

机器人学、机器视觉与控制
——MATLAB算法基础

Robotics, Vision and Control: Fundamental Algorithms in MATLAB



本书是关于机器人学和机器视觉的实用参考书，第一部分“基础知识”（第2章和第3章）介绍机器人及其操作对象的位置和姿态描述，以及机器人路径和运动的表示方法；第二部分“移动机器人”（第4章至第6章）介绍其基本运动控制模式及其导航和定位方法；第三部分“臂型机器人”（第7章至第9章）介绍其运动学、动力学和控制方面的知识；第四部分“计算机视觉”（第10章至第14章）包括光照与色彩，图像形成和处理技术，图像特征提取，以及基于多幅图像的立体视觉技术；第五部分“机器人学、视学与控制”（第15章和第16章）分别讨论基于位置和基于图像的视觉伺服及更先进的混合视觉伺服方法。

本书将机器人学与机器视觉知识有机结合，给出了实例算法和程序。作者有完备的代码可下载，用于验证书中知识点和实例，注重如何利用视觉信息控制机器人的运动。

只要会使用 MATLAB，就能通过本书学习和掌握机器人与机器人视觉的基础知识和控制算法。本书使用的 MATLAB“机器人学工具箱”（Robotics Toolbox）和“机器视觉工具箱”（Machine Vision Toolbox）可从 www.petercorke.com/RVC 下载。

作者简介 Peter Corke

澳大利亚昆士兰理工大学机器人与控制专业教授，CyPhy实验室联合主管（co-director）。研究兴趣包括：基于视觉的机器人控制、野外机器人和无线传感器网络。IEEE会士，*IEEE Robotics & Automation*期刊总编。

译者简介 刘 荣

北京航空航天大学教授，博士生导师。任职于北航机器人研究所，主要从事机器人技术及微机电系统方向的教学和研究。作为项目负责人，承担了国家及省部级项目十余项。开发了玻璃幕墙清洗机器人，光电子器件自动对接机器人，外科手术辅助治疗机器人，以及微小型反恐机器人等。发表论文60余篇，主编教育部高中教材《简易机器人制作》，翻译国外专著4本，编写机器人科普书籍1本。获专利20余项。2001年被评为北京市“科技新星”。

其他相关图书
请扫二维码↓

Springer

ISBN 978-7-121-25990-6

策划编辑：马 岚
责任编辑：周宏敏
封面设计：一克米工作室

定价：99.00元